MOST IMPORTANT FIRST | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB

Google study path (clean full code) - MEDIUM only

197 Medium questions in 43 groups (the same code-template groups as the full book, only the Medium questions). Each group shares ONE code template, shown on the page before its questions. DATED: 156 questions (marked * like this *). ND (no date): 41 questions.

0	197	0
EASY	MEDIUM	HARD

Sections (tap one to jump to it)

1. Frontier search (BFS or DFS), Tier 1: spread, count and measure distance — 6 questions | page 10
2. Sweep over events in time order — 6 questions | page 49
3. Tree recursion, Tier 1: one function that visits a node, asks its children, a... — 4 questions | page 87
4. A heap of what is available: pop the best, put back the changed one — 8 questions | page 115
5. Recurrence on a position or a range (memo or table) — 6 questions | page 165
6. Best-first search with a heap: Dijkstra and its minimax twin — 5 questions | page 203
7. Backtracking: choose, recurse, undo — 8 questions | page 234
8. Runs and groups: extend while it fits, restart or close when it does not — 7 questions | page 282
9. Union-find: merge the pairs, then ask who is together — 7 questions | page 318
10. A dict of per-key state that you update, expire, or rank — 7 questions | page 357
11. Binary search on a yes/no condition — 4 questions | page 399
12. Greedy: scan once, carry the best choice so far, act only when forced — 5 questions | page 425
13. Sets and counters: remember the first collection, check the second aga... — 3 questions | page 452
14. bisect on a sorted list: find a position, count a range, keep it sorted — 5 questions | page 469
15. Prefix sums, Tier 1: precompute once, answer ranges and balance poin... — 5 questions | page 497
16. Sorted intervals: merge while they touch, then read the gaps or search — 5 questions | page 523
17. Trie: nested dicts, one character per level — 7 questions | page 552
18. A dict for lookup plus an ordered container for who is next (lazy deletion) — 6 questions | page 594

Sections, continued (tap one to jump to it)

19. Rolling state: the next state comes only from the previous one	4 questions	page 630
20. Two pointers: move i and j toward each other (or along two lists)	3 questions	page 653
21. Walk two strings at once with two indices	3 questions	page 672
22. Simulate a machine: a class with state and one branch per kind of event	5 questions	page 693
23. Top-K by frequency: Counter plus nsmallest	6 questions	page 723
24. Digits and characters: peel one at a time and carry	3 questions	page 766
25. Look at the input once and keep only a small summary (and count your...	4 questions	page 789
26. Grid DP: each cell from its neighbours above and to the left	3 questions	page 815
27. Nested structures: a stack of contexts, or recursion with a cache	5 questions	page 832
28. Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours...	5 questions	page 865
29. Topological sort (Kahn) and cycle detection	5 questions	page 894
30. Counting pairs by looking up each partner in a Counter or a bucket	4 questions	page 928
31. Tree DP: build a parent order once, then add up from the leaves	3 questions	page 948
32. Counting or reaching an amount: the knapsack loop	5 questions	page 966
33. Prefix sums, Tier 2: running total plus a dict of earlier totals	4 questions	page 998
34. Group by a canonical key	3 questions	page 1022
35. K-way merge: a heap of the next item from each source	2 questions	page 1041
36. Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up	4 questions	page 1054
37. Random pick without repeats: swap the chosen item with the last one...	4 questions	page 1079
38. Build and serialize: write a structure out, read it back with the same w...	3 questions	page 1102
39. Games: win or lose with a cache, and guessing with feedback	1 question	page 1124
40. Sliding window: grow on the right, shrink on the left while it is too bad	4 questions	page 1135
41. Parent pointers: store who is above each node and climb	4 questions	page 1161
42. Build or transform a grid with index arithmetic	4 questions	page 1187
43. Split into pieces, fix each piece, join (and pack words into lines)	2 questions	page 1211

- The master contents lists every main topic and question; each row is a link.
- Inside a topic, the bar at the top of each page moves Prev / Contents / Next; the link at the bottom returns here.
- Questions run Easy, then Medium, then Hard inside each main topic, newest date first.

Go to Contents >

Contents

197 questions | tap a topic, a main topic or a question to jump to it

START HERE: topics in the order of importance, the most important question first inside each

1. Frontier search (BFS or DFS), Tier 1: spread, count and measure distance (6) Go >

- M 1. Number of Islands — 1Point3Acres * 2026-09-30 *
- M 2. Rotten Oranges / Multi-Source BFS (taxis) — 1Point3Acres * 2026-09-08 *
- M 3. Find shortest relationship path using BFS — PracHub * 2026-02-12 *
- M 4. Compute distance to nearest taxi in grid — PracHub * 2026-01-10 *
- M 5. Solve Grid and Counting Problems — PracHub * 2025-01-25 *
- M 6. Farthest City for a New Data Center — ColorStack * 2025-10-10 *

2. Sweep over events in time order (6) Go >

- M 7. Meeting Rooms / Interval Scheduling Variants — 1Point3Acres * 2026-09-29 *
- M 8. Train / Flight Schedule Reachability — 1Point3Acres * 2026-10-08 *
- M 9. Solve Rooms and Top-K Streams — PracHub * 2026-06-05 *
- M 10. Gate Entry / Exit Time Resolution with Tie-Break Rules — 1Point3Acres * 2026-05-04 *
- M 11. Compute servers needed for daily recurring jobs — PracHub * 2025-11-05 *
- M 12. Meeting Rooms III Variant — ND

3. Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upw... Go >

- M 13. Expression Tree Single-Leaf Mutation — 1Point3Acres * 2026-09-27 *
- M 14. Longest Consecutive Path in Binary Tree — 1Point3Acres * 2026-07-02 *
- M 15. Implement zigzag level-order traversal — PracHub * 2025-08-01 *
- M 16. Given the root of a binary tree, return the length of the diameter of the tree. — the tree. ND

4. A heap of what is available: pop the best, put back the changed one (8) Go >

- M 17. Streaming Median (LC 295) with Follow-ups — 1Point3Acres * 2026-08-03 *
- M 18. Compute minimum number of rooms needed — PracHub * 2026-02-12 *
- M 19. Implement a Reusable Locker Allocation System — PracHub * 2026-08-14 *
- M 20. Simulate room assignments for scheduled meetings — PracHub * 2026-01-06 *
- M 21. Minimum Makespan on Identical CPUs — ColorStack * 2025-09-23 *
- M 22. Design server allocation for multi-type nodes — PracHub * 2025-09-06 *
- M 23. Trip Expense Settlement — ND
- M 24. Playlist Shuffle No Adjacent Artists — ND

5. Recurrence on a position or a range (memo or table) (6) Go >

- M 25. Target Expressions from Ordered Digits — 1Point3Acres * 2026-08-13 *
- M 26. Count Word Decompositions into Chemical Element Symbols — 1Point3Acres * 2026-07-14 *
- M 27. Fence Painting Minimum Operations (Gemini Team) — 1Point3Acres * 2026-04-27 *
- M 28. Maximize 1D deliveries within distance and minimize total distance — PracHub * 2025-12-27 *
- M 29. Find the longest palindromic subsequence using dynamic programming. — ND
- M 30. Jump Game with Profit — ND

6. Best-first search with a heap: Dijkstra and its minimax twin (5) Go >

- M 31. Path Minimizing the Maximum Cell (Swim in Rising Water) — 1Point3Acres * 2026-09-29 *
- M 32. Nearest Favorite City by Travel Time — ColorStack * 2025-10-17 *
- M 33. Given a train schedule with departure city, arrival city, start time, end time,... — time, and duration, and given a source and... ND

Contents

197 questions | tap a topic, a main topic or a question to jump to it

M 34. Find the shortest path in a graph. **ND**

M 35. Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat... **the rat to the bread that maximizes the mini... ND**

7. Backtracking: choose, recurse, undo (8) **Go >**

M 36. Mahjong Winning Hand Detection (14 Tiles) **1Point3Acres * 2026-10-09 ***

M 37. Count Prefix-Matching Simple Paths in a Labeled Graph **PracHub * 2026-08-09 ***

M 38. Determine if a 14-tile hand is winning **PracHub * 2026-03-09 ***

M 39. American Football Score Combinations **ColorStack * 2025-08-25 ***

M 40. Count Passcode Patterns on a Keypad **ColorStack * 2025-11-07 ***

M 41. Match people to questions using tags and priorities **PracHub * 2025-12-27 ***

M 42. Generate Parentheses **ND**

M 43. Permutations **ND**

8. Runs and groups: extend while it fits, restart or close when it does not (7) **Go >**

M 44. Longest Non-Decreasing Subarray with One Replacement **1Point3Acres * 2026-10-04 ***

M 45. Piano Hand Movement Grouping **1Point3Acres * 2026-07-14 ***

M 46. Find longest increasing contiguous subarray **PracHub * 2026-01-06 ***

M 47. Sum of Good Subarrays (Adjacent Diff ±1) **1Point3Acres * 2026-02-08 ***

M 48. Sustained-Error Time Window **ColorStack * 2025-09-16 ***

M 49. Longest Non-Decreasing Subarray From Two Arrays **ColorStack * 2025-09-24 ***

M 50. Count Super Streaks **ColorStack * 2025-10-03 ***

9. Union-find: merge the pairs, then ask who is together (7) **Go >**

M 51. Count Islands in a Grid and Track the Count as Land Cells Are Added **PracHub * 2026-10-02 ***

M 52. 2D Points Connected Components by Radius **1Point3Acres * 2026-08-04 ***

M 53. Count Clusters of 2D Points Within a Radius **PracHub * 2026-06-18 ***

M 54. Find A Threshold-Limited Path With Minimum Required Safety **PracHub * 2026-07-06 ***

M 55. Determine Reporting Relationships **PracHub * 2026-02-22 ***

M 56. Determine whether two nodes are related **PracHub * 2026-01-22 ***

M 57. Movie Recommendations via Transitive Similarity **ColorStack * 2025-10-09 ***

10. A dict of per-key state that you update, expire, or rank (7) **Go >**

M 58. Active Communication Top Users **1Point3Acres * 2026-09-22 ***

M 59. Filter Repeated Robot Status Messages **PracHub * 2026-07-05 ***

M 60. Next-Word Predictor **1Point3Acres * 2026-09-02 ***

M 61. Batch and Streaming Log Deduplication **1Point3Acres * 2026-07-23 ***

M 62. Design a logger that returns top-K messages **PracHub * 2026-01-06 ***

M 63. Website Activity Analytics: Unique Users + Session Time **1Point3Acres * 2026-06-26 ***

M 64. Hit Counter **ND**

11. Binary search on a yes/no condition (4) **Go >**

M 65. Cutting Ribbons (LC 1891) **1Point3Acres * 2026-08-16 ***

M 66. First Bad Version with Parallel Search Follow-up **1Point3Acres * 2026-06-15 ***

M 67. Solve Banana Speed and Interval Merge **PracHub * 2025-01-27 ***

M 68. Given a list with response time for each server, calculate how long time it will take... **time it will take to process n requests. ND**

Contents

197 questions | tap a topic, a main topic or a question to jump to it

12. Greedy: scan once, carry the best choice so far, act only when forced (5) Go >

M 69. Visible People in a Line (Last Person, Then Everyone) **1Point3Acres * 2026-10-07 ***
M 70. Bounded Knapsack Variant **1Point3Acres * 2026-10-16 ***
M 71. Find the Largest Monotone Increasing Number **PracHub * 2026-07-04 ***
M 72. Min boards to cover roof holes **PracHub * 2026-02-12 ***
M 73. Jump Game II (Minimum Jumps) **ColorStack * 2025-08-19 ***

13. Sets and counters: remember the first collection, check the second against it (3) Go >

M 74. Reconstruct original array from doubled shuffle **PracHub * 2026-01-04 ***
M 75. Min deletions to avoid overlap in first k **PracHub * 2026-02-12 ***
M 76. Filter Common Elements Per Window (Two Lists) **ColorStack * 2025-10-08 ***

14. bisect on a sorted list: find a position, count a range, keep it sorted (5) Go >

M 77. Sliding-Window Mean After Removing the Largest K Values **PracHub * 2026-09-01 ***
M 78. Streaming Insert + Find K-th Largest **1Point3Acres * 2026-02-04 ***
M 79. Count Sorted Strings With a Given Prefix **ColorStack * 2025-10-08 ***
M 80. Russian Doll Envelopes **ND**
M 81. Given positions of N kids and M ice cream sellers on a 1D line, find the s... **the smallest distance E such that every kid li... ND**

15. Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) (5) Go >

M 82. Range Duplicate Check with Constant-Time Queries **1Point3Acres * 2026-09-29 ***
M 83. Find a Horizontal Cut That Bisects Rectangle Area **PracHub * 2026-06-04 ***
M 84. Check pivot after removing one element **PracHub * 2026-02-12 ***
M 85. Gold Chain Split into Two Equal Halves **1Point3Acres * 2026-04-21 ***
M 86. Count the number of ways to split a string. **ND**

16. Sorted intervals: merge while they touch, then read the gaps or search (5) Go >

M 87. Find Containing Range **PracHub * 2026-06-04 ***
M 88. Union of Sorted Interval Lists **1Point3Acres * 2026-08-16 ***
M 89. Range Module (Insert and Point Query) **ColorStack * 2026-01-20 ***
M 90. Meetings Minus a Do-Not-Schedule Block **ColorStack * 2025-10-03 ***
M 91. Merge Intervals **ND**

17. Trie: nested dicts, one character per level (7) Go >

M 92. URL Request Handler Router **1Point3Acres * 2026-10-16 ***
M 93. Trie Prefix Search / Autocomplete **1Point3Acres * 2026-07-02 ***
M 94. Build Prefix Lookup with a Trie **PracHub * 2026-07-03 ***
M 95. Greedy Longest-Match Dictionary Tokenizer **1Point3Acres * 2026-06-15 ***
M 96. Check if all substrings are dictionary words **PracHub * 2026-03-11 ***
M 97. Return dictionary words matching a prefix **PracHub * 2026-03-09 ***
M 98. Implement Trie Insert and Exact-Word Search **PracHub * 2026-02-20 ***

18. A dict for lookup plus an ordered container for who is next (lazy deletion) (6) Go >

M 99. Design a restaurant waitlist system **PracHub * 2026-03-01 ***
M 100. Implement Python's itertools.tee **1Point3Acres * 2026-08-20 ***
M 101. Restaurant Waitlist: join / delete / find_first_match **1Point3Acres * 2026-02-01 ***

Contents

197 questions | tap a topic, a main topic or a question to jump to it

M 102. Store With Cheapest-Item Removal — **ColorStack * 2025-09-29 ***
M 103. Round-Robin Job Scheduler — **ColorStack * 2025-08-19 ***
M 104. Aggregate FizzBuzz Offer Matches — **ColorStack * 2025-10-13 ***

19. Rolling state: the next state comes only from the previous one (4) — **Go >**

M 105. Maximize sum without choosing adjacent elements — **PracHub * 2026-01-06 ***
M 106. Paint House II — **ColorStack * 2025-10-17 ***
M 107. Given n houses in a line with money in each, find the maximum amount a rob... — **a robber can steal, without stealing fro... ND**
M 108. Write SQL code to publish the Fibonacci series. — **ND**

20. Two pointers: move i and j toward each other (or along two lists) (3) — **Go >**

M 109. Find median of merged RLE arrays — **PracHub * 2026-01-06 ***
M 110. Run-Length Vector Storage + Dot Product — **1Point3Acres * 2026-01-21 ***
M 111. Count Unique Triplets That Sum to Zero — **ColorStack * 2025-09-20 ***

21. Walk two strings at once with two indices (3) — **Go >**

M 112. Can board states be transformed? — **PracHub * 2026-04-01 ***
M 113. Decide string transform with directional tokens and walls — **PracHub * 2025-07-15 ***
M 114. Determine if two sentences are similar. — **ND**

22. Simulate a machine: a class with state and one branch per kind of event (5) — **Go >**

M 115. Tic-Tac-Toe System with K Players and 3-in-a-Row Win — **1Point3Acres * 2026-05-18 ***
M 116. Design Tic-Tac-Toe With K Players — **PracHub * 2026-02-12 ***
M 117. Implement iterator merging lists with filtering — **PracHub * 2026-01-06 ***
M 118. Infinite Deck Card Duel — **ColorStack * 2025-09-26 ***
M 119. Design an elevator control system — **PracHub * 2025-09-06 ***

23. Top-K by frequency: Counter plus nsmallest (6) — **Go >**

M 120. Top-K Frequent in Large Logs — **1Point3Acres * 2026-07-19 ***
M 121. Rearrange Tasks With Cooldown — **PracHub * 2026-06-03 ***
M 122. Implement frequency + distance top-K queries — **PracHub * 2026-02-12 ***
M 123. Find top/bottom-k words in list or stream — **PracHub * 2026-01-11 ***
M 124. Solve chat, grid paths, and car rentals — **PracHub * 2025-12-02 ***
M 125. Top-K Chatters by Word Count — **ColorStack * 2025-09-21 ***

24. Digits and characters: peel one at a time and carry (3) — **Go >**

M 126. Solve several coding interview problems — **PracHub * 2026-03-04 ***
M 127. Check Whether Digits Are Monotone Increasing — **PracHub * 2026-07-04 ***
M 128. Add two big integers from digit lists — **PracHub * 2025-11-24 ***

25. Look at the input once and keep only a small summary (and count your calls) (4) — **Go >**

M 129. Streaming Points: Emit K Within Distance — **1Point3Acres * 2026-06-23 ***
M 130. Two-Day Log Intersection under Memory Limit — **1Point3Acres * 2025-11-27 ***
M 131. Find shared objects across two log files — **PracHub * 2025-11-05 ***
M 132. Minimize calls to find all bad test pairs — **PracHub * 2025-11-05 ***

Contents

197 questions | tap a topic, a main topic or a question to jump to it

26. Grid DP: each cell from its neighbours above and to the left (3) Go >

- M 133. Grid Path Count with Diagonal Moves and Checkpoints — 1Point3Acres * 2026-09-29 *
- M 134. Longest Same-Color Grid Sequence — 1Point3Acres * 2026-09-08 *
- M 135. Count Same-Color Squares in Matrix — ND

27. Nested structures: a stack of contexts, or recursion with a cache (5) Go >

- M 136. Decode String — 1Point3Acres * 2026-07-29 *
- M 137. Template Variable Expansion with Follow-Ups — 1Point3Acres * 2026-10-17 *
- M 138. Expand Recursive Key-Value Placeholders in a Template — PracHub * 2026-08-04 *
- M 139. Nested List Weight Sum (LC 339) with Follow-Ups — 1Point3Acres * 2026-08-16 *
- M 140. Nested Function Expression Evaluator — 1Point3Acres * 2026-06-10 *

28. Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on th... Go >

- M 141. Shortest Path Nodes and Escaping a Pursuer — 1Point3Acres * 2026-09-29 *
- M 142. Radio Broadcast Reachability — ColorStack * 2026-01-30 *
- M 143. Minimum Direction Violations in a Directed Graph — 1Point3Acres * 2026-07-19 *
- M 144. Compute shortest paths with blocked nodes — PracHub * 2025-09-06 *
- M 145. Shortest Directed Cycle Through a Node — ND

29. Topological sort (Kahn) and cycle detection (5) Go >

- M 146. Course Scheduling and Dependency Variants — 1Point3Acres * 2026-10-04 *
- M 147. Validate Race Results (Cycle Detection) — ColorStack * 2025-10-01 *
- M 148. Solve tree, graph, sliding-window problems — PracHub * 2025-08-04 *
- M 149. Validate course catalog dependencies — PracHub * 2025-08-01 *
- M 150. Course Schedule — ND

30. Counting pairs by looking up each partner in a Counter or a bucket (4) Go >

- M 151. Progressive Array Partitioning — 1Point3Acres * 2026-08-11 *
- M 152. K-diff Pairs in an Array — ColorStack * 2025-09-16 *
- M 153. Count Axis-Aligned Squares Among Grid Points — ColorStack * 2025-11-03 *
- M 154. Sum of Two Squares in Two Ways — ND

31. Tree DP: build a parent order once, then add up from the leaves (3) Go >

- M 155. Compute File System Total Size — 1Point3Acres * 2026-09-02 *
- M 156. Minimum-Cost Tree Disconnection — 1Point3Acres * 2026-09-09 *
- M 157. Given a large n-ary tree where the root node can pass information to one... — to one child per iteration, and in subsequent... ND

32. Counting or reaching an amount: the knapsack loop (5) Go >

- M 158. Maximize Shelf Space (Unbounded Knapsack) — ColorStack * 2025-10-17 *
- M 159. Coin Change — ND
- M 160. Partition an array into two sub-arrays with equal sum. — ND
- M 161. Explain how to find a target sum in an array. — ND
- M 162. Reverse Coin Change — ND

Contents

197 questions | tap a topic, a main topic or a question to jump to it

33. Prefix sums, Tier 2: running total plus a dict of earlier totals (4) Go >

- M 163. Solve array-sum and city-community problems — PracHub * 2026-02-20 *
- M 164. Maximum-Sum Subarray With Equal Endpoints — ColorStack * 2025-09-29 *
- M 165. Maximum Subarray Sum With a Minimum Length Constraint — ColorStack * 2025-08-19 *
- M 166. Find the maximum subarray sum. — ND

34. Group by a canonical key (3) Go >

- M 167. Check if all substrings are anagrams of words — PracHub * 2026-03-11 *
- M 168. Group anagrams — ND
- M 169. Given an array of strings, return groups of "buddy strings," where the stri... — the strings have the same length and the dist... ND

35. K-way merge: a heap of the next item from each source (2) Go >

- M 170. K Largest Elements Across Sorted Lists — ColorStack * 2025-10-31 *
- M 171. Sort a nearly sorted (or K sorted) array. — ND

36. Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up (4) Go >

- M 172. Find mode in a trinary search tree — PracHub * 2025-11-16 *
- M 173. Next Upcoming Birthday in a Family Tree — ColorStack * 2025-10-02 *
- M 174. Lowest Common Ancestor — ND
- M 175. Determine if a given binary tree is a binary search tree (BST). — ND

37. Random pick without repeats: swap the chosen item with the last one and pop (4) Go >

- M 176. Design set with O(1) random access — PracHub * 2026-01-01 *
- M 177. Random Generator Without Repeats — ColorStack * 2025-10-10 *
- M 178. Random Song Shuffler With a No-Repeat Window — ColorStack * 2025-10-06 *
- M 179. Unique Random Number Generator — ColorStack * 2025-10-03 *

38. Build and serialize: write a structure out, read it back with the same walk (3) Go >

- M 180. Print a Directory Tree — ColorStack * 2025-09-21 *
- M 181. Construct a binary tree given its preorder and inorder traversals. — ND
- M 182. Write functions to serialize and deserialize a list of strings. — ND

39. Games: win or lose with a cache, and guessing with feedback (1) Go >

- M 183. Choose optimal guesses for green-only Wordle — PracHub * 2025-10-14 *

40. Sliding window: grow on the right, shrink on the left while it is too bad (4) Go >

- M 184. Temperature Monitoring Data Structure — 1Point3Acres * 2026-09-10 *
- M 185. Footpath Beauty After Repainting — ColorStack * 2025-09-22 *
- M 186. Shortest Subarray with K Distinct Integers — ND
- M 187. Find the minimum window substring. — ND

41. Parent pointers: store who is above each node and climb (4) Go >

- M 188. Employee Hierarchy Add / Delete — 1Point3Acres * 2026-06-10 *
- M 189. Validate parent array forms a tree — PracHub * 2026-02-11 *
- M 190. Root a Tree So It Becomes a Binary Tree — ColorStack * 2025-11-07 *

Contents

197 questions | tap a topic, a main topic or a question to jump to it

M 191. Lowest Common Ancestor of a Binary Tree III **ND**

42. Build or transform a grid with index arithmetic (4) **Go >**

M 192. Connected Crop Arrangement on a Grid **1Point3Acres * 2026-10-04 ***

M 193. Solve matrix groups and recipe inventory **PracHub * 2026-01-22 ***

M 194. Range Sum Query 2D (Immutable Matrix) **ColorStack * 2025-08-19 ***

M 195. Rotate Any m×n 2D Matrix **ND**

43. Split into pieces, fix each piece, join (and pack words into lines) (2) **Go >**

M 196. Snake Case to Camel Case with Repository-Wide Refactor **1Point3Acres * 2026-07-14 ***

M 197. Google Doc Line Wrap (Word Wrap with Whitespace and Newlines) **1Point3Acres * 2026-02-01 ***

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 1 OF 43 |

Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

6 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

6
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 6 questions | tap a title to jump to it

READ FIRST: the code template for these 6 questions (tap here) >

M	1. Number of Islands	1Point3Acres	* 2026-09-30 *
M	2. Rotten Oranges / Multi-Source BFS (taxis)	1Point3Acres	* 2026-09-08 *
M	3. Find shortest relationship path using BFS	PracHub	* 2026-02-12 *
M	4. Compute distance to nearest taxi in grid	PracHub	* 2026-01-10 *
M	5. Solve Grid and Counting Problems	PracHub	* 2025-01-25 *
M	6. Farthest City for a New Data Center	ColorStack	* 2025-10-10 *

CODE TEMPLATE | GROUP 1 OF 43 | 6 QUESTIONS, ONE PATTERN

Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

The one idea these questions share

Every one of these keeps a frontier of cells or nodes still to visit, takes one out, looks at its neighbours, and adds the new ones. A dict of distances doubles as the seen set. Change what you start from, which neighbours are allowed, and what you do when a node comes out.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import deque


def frontier_search(starts, neighbors, usable=lambda v: True):
    dist = {s: 0 for s in starts}          # doubles as the seen set
    frontier = deque(starts)               # popleft = BFS, pop = DFS
    while frontier:
        u = frontier.popleft()
        # CHANGE 1: act on u here (repaint it, count it, stop at the goal)
        for v in neighbors(u):             # CHANGE 2: grid steps or adj[u]
            if v not in dist and usable(v):    # CHANGE 3: colour, wall, blocked
                dist[v] = dist[u] + 1
                frontier.append(v)
    return dist
```

2. Helper functions that usually get written next to it

```
def grid_steps(rows, cols):
    def step(cell):
        r, c = cell
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols:
                yield (nr, nc)
    return step


def rebuild_path(parent, target):         # walk parent[] back from the target
    path = [target]
    while parent[path[-1]] is not None:
        path.append(parent[path[-1]])
    return path[::-1]
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
1	Number of Islands	loop over every cell; each unseen land cell starts a new frontier_search (share one seen set); islands += 1 per start
2	Rotten Oranges / Multi-Source BFS (taxis)	starts = every rotten orange; usable = fresh orange; answer = rounds taken, or -1 if a fresh orange stays unreached
3	Find shortest relationship path using BFS	one start; also store parent[v] = u when you first reach v; stop at the target and walk parent back to rebuild the path
4	Compute distance to nearest taxi in grid	starts = every taxi cell; usable = not a wall; the dist table is the answer, cells never reached stay -1
5	Solve Grid and Counting Problems	the island count again (queue version); the second part of the question is a short counting formula, no search

Contents Start the questions >

#	Question	What you change in the template
6	Farthest City for a New Data Center	same on a graph: starts = the data centres, neighbors = adjacency list; answer = the city with the largest distance

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 1 of 197 | topic 1 of 43: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 1 of 6

Template fit: loop over every cell; each unseen land cell starts a new frontier_search (share one seen set); islands += 1 per start | * asked 20 days ago * | frequency Medium

MEDIUM | IN APP LC 200 | 1Point3Acres | onsite-coding, phone-screen | 1 of 197

★ TOP 75: rank 17 in the Medium and Hard priority list

★ Number of Islands

ASKED 2026-09-30 | 20 DAYS AGO | LAST 30 DAYS

STUDY: TYPE **ALL TIME 64.8** **30 DAYS 33** **PRIORITY 23/288**

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

Optimal answer from: your app's interview approach (LeetCode #200)

LeetCode match: LC 200 Number of Islands

Frequency: Medium | Last asked: 2026-09-30 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/number-of-islands

QUESTION

Given a grid of 0s (water) and 1s (land), count the islands. An island is a group of 1s connected up, down, left or right (diagonal cells are not connected).

Function signature

def num_islands(grid: list) -> int

EXAMPLES

Example 1	
Input	`[[1, 1, 0, 0], [1, 1, 0, 0], [0, 0, 1, 1], [0, 0, 1, 1]]`
Output	`2`
Why	`Two 2x2 blocks of land that only touch the water between them: 2 islands.`

Example 2	
Input	`[[1, 0, 1], [0, 1, 0], [1, 0, 1]]`
Output	`5`
Why	`Land that touches only diagonally is not connected, so every 1 is its own island.`

Example 3	
Input	`[[0, 0], [0, 0]]`
Output	`0`
Why	`No land, no islands.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(grid):
    seen, count = set(), 0
    for r, c in all_cells(grid):
        if grid[r][c] == 0 or (r, c) in seen:
            continue
        count += 1
        dfs(r, c)  # marks the whole island
    return count
def dfs(r, c):
    if out_of_grid(r, c) or grid[r][c] == 0:
        return
    if (r, c) in seen:
        return
    seen.add((r, c))
    for dr, dc in DIRECTIONS:
        dfs(r + dr, c + dc)
```

TIME	SPACE
O(m * n)	O(m * n) recursion

WHY IT IS SLOW

Time is already O(m * n); the weakness is the recursion depth, which can reach m * n on a snake-shaped island and overflow the stack. An explicit queue or stack avoids that.

OPTIMAL SOLUTION · Iterative DFS flood fill

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if land can also be removed?

Union-Find cannot split sets. Either rebuild the affected component with a local BFS from each former neighbour, or process the whole timeline offline with a segment tree over time and union-find with rollback (the offline dynamic connectivity technique).

FOLLOW-UP 2 How do you handle a grid too large for memory?

Process it in horizontal strips: label islands within the strip, keep only the boundary row's labels, and merge labels across strips with a union-find over label ids. Memory is O(width + number of labels on the boundary).

FOLLOW-UP 3 Find the area of the largest island.

Count the cells during each search and keep the maximum. Same traversal, one counter.

FOLLOW-UP 4 The grid is huge and does not fit in memory.

Process it in strips of rows, label islands in each strip, and merge labels across strip boundaries with a Union-Find over the labels on the boundary row. **Cost:** O(m * n) time overall and O(width + labels on the boundary row) memory per strip.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Number of Islands
#
# Given a grid of 0s (water) and 1s (land), count the
# islands. An island is a group of 1s connected up, down,
# left or right (diagonal cells are not connected).
#
# Signature:
#   def num_islands(grid: list) -> int
#
# Examples:
# [[1, 1, 0, 0], [1, 1, 0, 0], [0, 0, 1, 1], [0, 0, 1, 1]]
#     ->  2
# [[1, 0, 1], [0, 1, 0], [1, 0, 1]]  ->  5
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if land can also be removed?
# 2. How do you handle a grid too large for memory?
# 3. Find the area of the largest island.
# 4. The grid is huge and does not fit in memory.
#
# INTERVIEW NOTES
# Each island is a connected group of land cells, so count
# how many times you start a fresh search at an unvisited
# land cell. The search itself (DFS or BFS) just marks
# everything reachable. If land is added over time, do not
# rerun the search: Union-Find keeps the count up to date.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def neighbors(r, c, rows, cols):
    # the cells beside (r, c) that are inside the grid
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            result.append((nr, nc))
```

```
    return result


def flood(grid, start, seen):
    # mark all land joined to `start` as seen and return how many cells it has;
    # a stack instead of recursion, so a huge island cannot overflow the call stack
    rows, cols = len(grid), len(grid[0])
    seen.add(start)
    stack = [start]
    size = 0
    while stack:
        r, c = stack.pop()
        size += 1
```

```
        for nr, nc in neighbors(r, c, rows, cols):
            if grid[nr][nc] == 1 and (nr, nc) not in seen:
                seen.add((nr, nc))
                stack.append((nr, nc))
    return size


def island_sizes(grid):
    # one number per island: how many cells it has
    sizes = []
    if not grid:
        return sizes
    seen = set()
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == 1 and (r, c) not in seen:     # land we have not visited starts a new island
                sizes.append(flood(grid, (r, c), seen))
    return sizes


# ---- main question: main function ----
# Idea: scan every cell; each unvisited land cell starts a new
# island, and the flood marks all its connected land as visited.
# Complexity: Time O(m * n), Space O(m * n)
def num_islands(grid):
    return len(island_sizes(grid))


# ---- main question: tests ----
def test_main_question():
    g = [[1, 1, 0, 0], [1, 1, 0, 0], [0, 0, 1, 1], [0, 0, 1, 1]]
    assert num_islands(g) == 2
```

```
    assert num_islands([]) == 0


# ============================================================
# FOLLOW-UP 1: What if land can also be removed?
# ============================================================


# ============================================================
# FOLLOW-UP 2: How do you handle a grid too large for memory?
# ============================================================


# ============================================================
# FOLLOW-UP 4: The grid is huge and does not fit in memory
# ============================================================
# ---- follow-up 1: helper functions ----
# Complexity: Time add_land is O(alpha(m * n)) on average (inverse Ackermann, about O(1)), Space O(number o
f land cells)
class IslandCounter:
    # land cells arrive one at a time; keep the island count up to date
    # with union-find (each island is one group of joined cells)
    def __init__(self, rows, cols):
        self.rows = rows
        self.cols = cols
        self.parent = {}        # land cell -> a cell of its group
        self.count = 0

    def find(self, cell):
        # follow parents to the group's representative (path halving keeps it fast)
        while self.parent[cell] != cell:
            self.parent[cell] = self.parent[self.parent[cell]]
```

```
            cell = self.parent[cell]
        return cell

    def add_land(self, r, c):
        if (r, c) in self.parent:                # adding the same cell twice changes nothing
            return self.count
        self.parent[(r, c)] = (r, c)
        self.count += 1
        for nb in neighbors(r, c, self.rows, self.cols):
            if nb in self.parent:
                a = self.find((r, c))
                b = self.find(nb)
                if a != b:
                    self.parent[a] = b
```

```
                    self.count -= 1              # two islands became one
        return self.count


# ---- follow-up 1: tests ----
def test_follow_up_1():
    ic = IslandCounter(2, 2)
    counts = []
    for r, c in [(0, 0), (0, 1), (1, 1)]:
        counts.append(ic.add_land(r, c))
    assert counts == [1, 1, 1]
    ic = IslandCounter(3, 3)                     # two separate cells make two islands
    assert ic.add_land(0, 0) == 1
    assert ic.add_land(2, 2) == 2
    assert ic.add_land(1, 1) == 3                # the middle cell touches neither corner
# Removing land is harder: process the changes backwards (removal
# becomes adding) or rebuild the affected island.
# Too big for memory: read the grid a row at a time, keep only the
# group labels of the previous row, and merge labels with union-find.


# =============================================================
# FOLLOW-UP 3: Find the area of the largest island
# =============================================================
# ---- follow-up 3: function ----
# Answer: count the cells during each flood and keep the maximum.
# Complexity: Time O(m * n), Space O(m * n)
def largest_island(grid):
    biggest = 0
    for size in island_sizes(grid):
        if size > biggest:
            biggest = size
    return biggest


# ---- follow-up 3: tests ----
def test_follow_up_3():
    g = [[1, 1, 0, 0], [1, 1, 0, 0], [0, 0, 1, 1], [0, 0, 1, 1]]
    assert largest_island(g) == 4
    assert largest_island([[0, 0]]) == 0


# =============================================================
# RUN EVERYTHING
# =============================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write both the static count and the streaming addLand version from memory in under 15 min total.
Be ready to switch between DFS / BFS / Union-Find on demand.

STUDY PATH: question 2 of 197 | topic 1 of 43: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 2 of 6

Template fit: starts = every rotten orange; usable = fresh orange; answer = rounds taken, or -1 if a fresh orange stays unreached | * asked 42 days ago * | frequency Low

MEDIUM | IN APP LC 994 | 1Point3Acres | onsite-coding, phone-screen | 2 of 197

Rotten Oranges / Multi-Source BFS (taxis)

ASKED 2026-09-08 | 42 DAYS AGO | LAST 90 DAYS

STUDY: TYPE **ALL TIME 51.2** **30 DAYS 0** **PRIORITY 77/288**

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Also in your app: LeetCode #994 Rotting Oranges

LeetCode match: LC 994 Rotting Oranges

Frequency: Low | Last asked: 2026-09-08 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/rotten-oranges-multi-source-bfs-taxis

QUESTION

Each cell of a grid is 0 (empty), 1 (a fresh orange) or 2 (a rotten orange). Every minute, each fresh orange that is next to (up, down, left or right) a rotten one becomes rotten. Return the number of minutes until no fresh orange is left, or -1 if some fresh orange can never rot.

Function signature

def rotting_minutes(grid: list) -> int

EXAMPLES

Example 1	
Input	`[[2, 1, 1], [1, 1, 0], [0, 1, 1]]`
Output	`4`
Why	`The rot spreads outward from the corner and reaches the last orange after 4 minutes.`

Example 2	
Input	`[[2, 1, 1], [0, 1, 1], [1, 0, 1]]`
Output	`-1`
Why	`The orange at the bottom left is cut off by empty cells, so it never rots.`

Example 3	
Input	`[[0, 2]]`
Output	`0`
Why	`Nothing fresh to rot: 0 minutes.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    minutes = 0
    while True:
        fresh = fresh_next_to_rotten(grid)
        if not fresh:
            break
        for r, c in fresh:
            grid[r][c] = 2
        minutes += 1
    return -1 if any_fresh(grid) else minutes
```

TIME	SPACE
O(minutes * m * n)	O(m * n)

WHY IT IS SLOW

Each minute scans the entire grid, O(minutes * m * n). A queue of the newly rotten oranges touches each cell only once, O(m * n).

OPTIMAL SOLUTION · Multi-source BFS by minute

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Taxi variant: distance from every empty cell to the nearest taxi

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Sources move or new taxis appear. How do you keep the distances fresh?

Adding a source is cheap: push it at distance 0 and run a BFS that only continues into cells where it beats the stored distance (a decrease-only update, amortized small). Removing or moving a source is hard because distances can increase; either rebuild from scratch with a multi-source BFS, or rebuild periodically and accept bounded staleness in between.

FOLLOW-UP 2 Why is multi-source BFS the same cost as single-source BFS?

A virtual super-source connected to all sources by zero-length edges turns the problem into one BFS. Each cell enters the queue once, so it is O(mn) no matter how many sources exist.

FOLLOW-UP 3 Signal variant: a signal spreads from sources, how long until every target gets it?

Run the BFS and take the maximum distance over the target cells. If any target is unreachable the answer is infeasible (-1); with no targets it is 0. To allow destroying at most one blocker, compute distances from the sources and also from the targets and test each blocker cell as a bridge.

FOLLOW-UP 4 A signal spreads from sources to targets: how long until every target is reached?

One multi-source BFS from the sources, then the answer is the largest distance over the targets (-1 if any target is unreachable, 0 if there are no targets). Per-source BFS would cost O(mn * sources).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Rotten Oranges / Multi-Source BFS (taxis)
#
# Each cell of a grid is 0 (empty), 1 (a fresh orange) or
# 2 (a rotten orange). Every minute, each fresh orange
# that is next to (up, down, left or right) a rotten one
# becomes rotten. Return the number of minutes until no
# fresh orange is left, or -1 if some fresh orange can
# never rot.
#
# Signature:
#   def rotting_minutes(grid: list) -> int
#
# Examples:
# [[2, 1, 1], [1, 1, 0], [0, 1, 1]]  ->  4
# [[2, 1, 1], [0, 1, 1], [1, 0, 1]]  ->  -1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Sources move or new taxis appear. How do you keep the
# distances fresh?
# 2. Why is multi-source BFS the same cost as single-
# source BFS?
# 3. Signal variant: a signal spreads from sources, how
# long until every target gets it?
# 4. A signal spreads from sources to targets: how long
# until every target is reached?
#
# INTERVIEW NOTES
# 'Spread outward from several starting points at the same
# time and count the rounds' is multi-source BFS: put
# every source in the queue before you start, and process
# the queue one round at a time. Rounds are minutes.
# Always count the fresh ones first so that unreachable
# oranges are detected at the end.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
```

```
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def inside(r, c, rows, cols):
    # is (r, c) a real cell of the grid?
    return 0 <= r < rows and 0 <= c < cols
```

```
def find_cells(grid, value):
    # every (row, column) whose cell holds `value`
    cells = []
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == value:
                cells.append((r, c))
    return cells


def bfs_distances(grid, starts, wall):
    # distance from the nearest start to every cell it can reach;
    # a cell holding `wall` can never be entered
    rows, cols = len(grid), len(grid[0])
    dist = {}
    queue = deque()
    for start in starts:
        dist[start] = 0
        queue.append(start)
    while queue:
        r, c = queue.popleft()
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if not inside(nr, nc, rows, cols):
                continue
            if grid[nr][nc] == wall or (nr, nc) in dist:
                continue
            dist[(nr, nc)] = dist[(r, c)] + 1
            queue.append((nr, nc))
    return dist


# ---- main question: main function ----
# Idea: every rotten orange starts at minute 0 in the same queue;
# each BFS level is one minute. A fresh orange that is never
# reached makes the answer -1.
# Complexity: Time O(m * n), Space O(m * n)
```

```
def rotting_minutes(grid):
    rotten = find_cells(grid, 2)
    fresh = find_cells(grid, 1)
    dist = bfs_distances(grid, rotten, 0)      # rot spreads through any non-empty cell
    minutes = 0
    for cell in fresh:
        if cell not in dist:
            return -1
        if dist[cell] > minutes:
            minutes = dist[cell]
    return minutes


# the same search answers "distance to the nearest taxi"
# Complexity: Time O(m * n), Space O(m * n)
def nearest_taxi(grid):
    # 'T' = taxi, '.' = empty, '#' = blocked; None where no taxi can reach
    taxis = find_cells(grid, "T")
    dist = bfs_distances(grid, taxis, "#")
    answer = []
    for r in range(len(grid)):
        row = []
        for c in range(len(grid[0])):
            row.append(dist.get((r, c)))
        answer.append(row)
    return answer
```

```
# ---- main question: tests ----
def test_main_question():
    assert rotting_minutes([[2, 1, 1], [1, 1, 0], [0, 1, 1]]) == 4
    assert rotting_minutes([[2, 0, 1]]) == -1       # a fresh orange walled off by an empty cell
    assert rotting_minutes([[0, 2]]) == 0
    assert nearest_taxi(["T.#", "..."]) == [[0, 1, None], [1, 2, 3]]


# ============================================================
# FOLLOW-UP 1: Sources move or new taxis appear
# ============================================================
# Answer: do not redo everything. A new source only improves the
# cells it beats, so run a BFS from it that stops at cells it does
# not improve (written out in the taxi question of this group).
# Moving sources: recompute from scratch, or keep one BFS per source.
```

```
# ============================================================
# FOLLOW-UP 2: Why is multi-source BFS the same cost as single-source BFS?
# ============================================================
# Answer: every cell is put in the queue once, no matter how many
# sources there are, so the work is O(rows * cols) either way.


# ============================================================
# FOLLOW-UP 3: Signal variant: how long until every target gets it?
# ============================================================


# ============================================================
# FOLLOW-UP 4: A signal spreads from sources to targets
# ============================================================
# ---- follow-up 4: function ----
# Answer: one multi-source BFS from the sources, then the answer is
# the largest distance over the targets (-1 if a target is never
# reached, 0 if there are no targets). One BFS per source would
# cost O(rows * cols * sources).
# Complexity: Time O(m * n + targets), Space O(m * n)
def signal_time(rows, cols, sources, targets, blocked=()):
    grid = []
    for r in range(rows):
        grid.append([0] * cols)
    for r, c in blocked:
        grid[r][c] = 1                              # 1 = wall
    dist = bfs_distances(grid, list(sources), 1)
    slowest = 0
    for target in targets:
        if target not in dist:
            return -1
        if dist[target] > slowest:
            slowest = dist[target]
    return slowest


# ---- follow-up 4: tests ----
def test_follow_up_4():
    assert signal_time(1, 5, [(0, 0), (0, 4)], [(0, 1), (0, 2)]) == 2
    assert signal_time(1, 3, [(0, 0)], [(0, 2)], blocked={(0, 1)}) == -1
    assert signal_time(1, 3, [(0, 0)], []) == 0


# ============================================================
```

```python
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill multi-source BFS from memory in under 10 min.
Prep a 2-minute pitch on incremental-vs-rebuild trade-off for the moving-source follow-up.
Practice the rotting-oranges / walls-and-gates / 01-matrix triplet — all share the multi-source BFS skeleton.

STUDY PATH: question 3 of 197 | topic 1 of 43: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 3 of 6

Template fit: one start; also store parent[v] = u when you first reach v; stop at the target and walk parent back to rebuild the path | * asked 250 days ago *

MEDIUM | IN APP LC 127 | PracHub | Technical Screen | 3 of 197

Find shortest relationship path using BFS

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 41.8 | 30 DAYS 0 | PRIORITY 100/288

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #127 Word Ladder

LeetCode match: LC 127 Word Ladder (BFS shortest path) (inferred from overview)

Posted on PracHub: 2026-02-12 (updated 2026-05-04)

Original: https://prachub.com/coding-questions/find-shortest-relationship-path-using-bfs

QUESTION

People are connected by relationships, given as an adjacency dict that maps each person to the people they are directly related to. shortest_path(adj, s, t) returns the shortest chain of people from s to t (both ends included), or None when t cannot be reached.

Function signature

def shortest_path(adj: dict, s, t) -> list | None

EXAMPLES

Example 1

Input	`adj = {'ann': ['bob', 'cy'], 'bob': ['dee'], 'cy': ['dee', 'eli'], 'dee': ['fay'], 'eli': ['fay'], 'fay': []}`
	`shortest_path(adj, 'ann', 'fay')`
	`shortest_path(adj, 'ann', 'ann')`
	`shortest_path(adj, 'fay', 'ann')`

Output	`shortest_path(adj, 'ann', 'fay') -> ['ann', 'bob', 'dee', 'fay']`
	`shortest_path(adj, 'ann', 'ann') -> ['ann']`
	`shortest_path(adj, 'fay', 'ann') -> null`

Why	Several chains of 4 people exist; BFS finds the first. A person is trivially related to themselves, and fay has no outgoing relationships.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = None
def dfs(node, path):
    global best
    if node == target:
        if not best or len(path) < len(best):
            best = list(path)
        return
    for nxt in graph[node]:
        if nxt not in path:
            dfs(nxt, path + [nxt])
dfs(source, [source])
```

BRUTE FORCE

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

The number of simple paths is exponential. Breadth-first search reaches the target along a shortest path first, O(V + E), and bidirectional BFS explores far fewer nodes on big graphs.

OPTIMAL SOLUTION · Breadth-first search with a parent map

TIME	SPACE
O(V + E)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The graph is huge (a social network).

Bidirectional BFS searches from both ends and stops when the frontiers meet; with branching factor b and distance d it explores about 2 * b^(d/2) nodes instead of b^d.

FOLLOW-UP 2 Return every shortest path, or count them.

Keep, for each node, its distance and a list of all parents at that distance; counting is a DP over the BFS layers, and listing the paths is a DFS over the parent lists.

FOLLOW-UP 3 Relationships have different strengths (weights).

BFS is no longer right: use Dijkstra with a heap for non-negative weights. Keep the same parent map to rebuild the path.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find shortest relationship path using BFS
#
# People are connected by relationships, given as an
# adjacency dict that maps each person to the people they
# are directly related to. shortest_path(adj, s, t)
# returns the shortest chain of people from s to t (both
# ends included), or None when t cannot be reached.
#
# Signature:
#   def shortest_path(adj: dict, s, t) -> list | None
#
# FOLLOW-UPS (answered below, in this order):
# 1. The graph is huge (a social network).
# 2. Return every shortest path, or count them.
# 3. Relationships have different strengths (weights).
#
# INTERVIEW NOTES
# Unweighted means every hop costs the same, so breadth-
# first search finds the fewest hops: it explores all
# people one hop away, then two hops, and the first time
# it reaches the target is the shortest path. Keep a
```

```
# parent map to rebuild the path and a visited set so
# nobody is queued twice.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def walk_back(parent, target):
    # follow the parent links from target back to the start, then reverse
    path = [target]
    while parent[path[-1]] is not None:
        path.append(parent[path[-1]])
    path.reverse()
```

```
    return path


# ---- main question: main function ----
# Idea: queue the source and mark it visited. Pop a person, queue
# each unvisited neighbour with parent = this person, and stop when
# the target is reached; the first time BFS reaches a node is by a
# shortest path.
# Complexity: Time O(V + E), Space O(V)
def shortest_path(adj, s, t):
    if s == t:
        return [s]
    parent = {s: None}
    queue = deque([s])
    while queue:
        u = queue.popleft()
        for v in adj.get(u, []):
            if v not in parent:
                parent[v] = u
                if v == t:
                    return walk_back(parent, t)
                queue.append(v)
    return None


# ---- main question: tests ----
def test_main_question():
    adj = {"a": ["b", "c"], "b": ["d"], "c": ["d"], "d": ["e"], "e": []}
    assert shortest_path(adj, "a", "e") in (["a", "b", "d", "e"], ["a", "c", "d", "e"])
    assert shortest_path(adj, "a", "a") == ["a"]
    assert shortest_path(adj, "e", "a") is None


# ============================================================
# FOLLOW-UP 1: The graph is huge (a social network)
# ============================================================
# ---- follow-up 1: helper functions ----
def expand_one_level(adj, frontier, mine, other):
    # visit everything one step beyond `frontier`; `mine` / `other` hold the
    # distances found from this side / the opposite side.
```

```
    # Returns (new frontier, length of a path found, or None).
    new_frontier = []
    for u in frontier:
        for v in adj.get(u, []):
            if v in other:
```

```
                return new_frontier, mine[u] + 1 + other[v]
            if v not in mine:
                mine[v] = mine[u] + 1
                new_frontier.append(v)
    return new_frontier, None


# ---- follow-up 1: function ----
# Answer: search from both ends and stop when the two searches meet.
# With branching b and distance d that explores about 2 * b^(d/2)
# nodes instead of b^d. Always grow the smaller side.
# Complexity: Time about O(b^(d / 2)) for branching b and distance d, Space the same
def bidirectional(adj, s, t):
    if s == t:
        return 0
    dist_s, dist_t = {s: 0}, {t: 0}
    front_s, front_t = [s], [t]
    while front_s and front_t:
        if len(front_s) <= len(front_t):
            front_s, found = expand_one_level(adj, front_s, dist_s, dist_t)
        else:
            front_t, found = expand_one_level(adj, front_t, dist_t, dist_s)
        if found is not None:
            return found
    return None


# ---- follow-up 1: tests ----
def test_follow_up_1():
    both_ways = {"a": ["b", "c"], "b": ["a", "d"], "c": ["a", "d"],
                 "d": ["b", "c", "e"], "e": ["d"]}
    assert bidirectional(both_ways, "a", "e") == 3
    assert bidirectional(both_ways, "a", "a") == 0


# =========================================================
# FOLLOW-UP 2: Return every shortest path, or count them
# =========================================================
# ---- follow-up 2: function ----
# Answer: remember, for each node, its distance and every parent that
# is exactly one step closer; then list the paths by walking back
# through those parent lists (counting paths is the same walk).
# Complexity: Time O(V + E + number of paths * path length), Space O(V + E + output)
def all_shortest_paths(adj, s, t):
    dist = {s: 0}
```

```
    parents = {}
    queue = deque([s])
    while queue:
        u = queue.popleft()
        for v in adj.get(u, []):
            if v not in dist:
                dist[v] = dist[u] + 1
                queue.append(v)
            if dist[v] == dist[u] + 1:
                if v not in parents:
                    parents[v] = []
                parents[v].append(u)
    if t not in dist:
```

```
        return []
    paths = []

    def build(node, tail):
        if node == s:
            paths.append([s] + tail)
            return
        for p in parents[node]:
            build(p, [node] + tail)

    build(t, [])
    return paths


# ---- follow-up 2: tests ----
def test_follow_up_2():
    adj = {"a": ["b", "c"], "b": ["d"], "c": ["d"], "d": ["e"], "e": []}
    paths = all_shortest_paths(adj, "a", "e")
    assert sorted(paths) == [["a", "b", "d", "e"], ["a", "c", "d", "e"]]


# ============================================================
# FOLLOW-UP 3: Relationships have different strengths (weights)
# ============================================================
# ---- follow-up 3: function ----
# Answer: BFS is no longer right; use Dijkstra with a heap (weights
# must not be negative). Keep the same parent map to rebuild the path.
# Complexity: Time O((V + E) log V), Space O(V)
def weighted_path(graph, s, t):
    dist = {s: 0}
    parent = {s: None}
    heap = [(0, s)]
```

```
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue                        # an older, longer entry
        for v, w in graph.get(u, []):
            if v not in dist or d + w < dist[v]:
                dist[v] = d + w
                parent[v] = u
                heapq.heappush(heap, (d + w, v))
    if t not in dist:
        return None
    return dist[t], walk_back(parent, t)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    graph = {"a": [("b", 1), ("c", 5)], "b": [("c", 1)]}
    assert weighted_path(graph, "a", "c") == (2, ["a", "b", "c"])
    assert weighted_path(graph, "c", "a") is None


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(relationships: list[list[int]], start: int, target: int) -> int:
```

Sample call

```
solution([['A', 'B'], ['B', 'C'], ['C', 'D']], 'A', 'D')
```

Test cases

Input	Expected
`([['A', 'B'], ['B', 'C'], ['C', 'D']], 'A', 'D')`	`3`
`([[1, 2], [2, 5], [1, 3], [3, 4], [4, 5]], 1, 5)`	`2`

ALSO REPORTED · 1 more

Degrees of Friendship | Exponent

https://www.aced.io/questions/3628/degrees-of-friendship

STUDY PATH: question 4 of 197 | topic 1 of 43: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 4 of 6

Template fit: starts = every taxi cell; usable = not a wall; the dist table is the answer, cells never reached stay -1 | * asked 283 days ago *

MEDIUM | IN APP LC 542 | PracHub | Technical Screen | 4 of 197

Compute distance to nearest taxi in grid

POSTED 2026-01-10 | 283 DAYS AGO | OLDER

STUDY: TALK ALL TIME 36.5 30 DAYS 0 PRIORITY 117/288

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #542 01 Matrix

LeetCode match: LC 542 01 Matrix (inferred from overview)

Posted on PracHub: 2026-01-10 (updated 2026-06-20)

Original: https://prachub.com/coding-questions/compute-distance-to-nearest-taxi-in-grid

QUESTION

A grid has 'T' for a taxi, '#' for a blocked cell and '.' for an open cell. For every cell return the number of steps (up, down, left or right, never through '#') to the nearest taxi: 0 for a taxi, and -1 for a blocked cell or a cell no taxi can reach.

Function signature

def taxi_distances(grid: list) -> list

EXAMPLES

Example 1	
Input	`['T.#', '...', '#.T']`
Output	`[[0, 1, -1], [1, 2, 1], [-1, 1, 0]]`
Why	`The two taxis are in opposite corners; the middle cell is 2 steps from both; the blocked cells show -1.`

Example 2	
Input	`['..', '..']`
Output	`[[-1, -1], [-1, -1]]`
Why	`No taxi at all: every open cell is unreachable.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def nearest_taxi(grid, start):
    queue, seen = deque([(start, 0)]), {start}
    while queue:
        cell, d = queue.popleft()
        if is_taxi(grid, cell):
            return d
        for nxt in neighbours(grid, cell):
            if nxt not in seen:
                seen.add(nxt)
                queue.append((nxt, d + 1))
    return -1
answer = {}
for c in cells:
    answer[c] = nearest_taxi(grid, c)
```

TIME	SPACE
O((m * n)^2)	O(m * n)

WHY IT IS SLOW

A separate BFS per cell, O((m * n)^2). One BFS that starts from all taxis at once reaches every cell at its nearest-taxi distance, O(m * n).

OPTIMAL SOLUTION · One multi-source BFS

TIME	SPACE
O(m * n)	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A new taxi appears.

Start a BFS from the new taxi but only continue into cells where you improve the stored distance. The distances can only decrease, so the work is bounded by the cells whose answer actually changes.

FOLLOW-UP 2 Taxis move or disappear.

Distances can increase, which the decrease-only update cannot handle. Rebuild with one multi-source BFS (O(mn)), perhaps lazily every few updates and accepting bounded staleness between rebuilds.

FOLLOW-UP 3 Return which taxi is nearest, not only the distance.

Store a source id with each cell in the BFS: a cell copies the id of the cell that first reached it. Ties go to whichever reached it first, so say how you want ties broken.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Compute distance to nearest taxi in grid
#
# A grid has 'T' for a taxi, '#' for a blocked cell and
# '.' for an open cell. For every cell return the number
# of steps (up, down, left or right, never through '#') to
```

```
# the nearest taxi: 0 for a taxi, and -1 for a blocked
# cell or a cell no taxi can reach.
#
# Signature:
#   def taxi_distances(grid: list) -> list
#
# Examples:
# ['T.#', '...', '#.T']  ->  [[0, 1, -1], [1, 2, 1], [-1,
#     1, 0]]
# ['..', '..']  ->  [[-1, -1], [-1, -1]]
#
# FOLLOW-UPS (answered below, in this order):
# 1. A new taxi appears.
# 2. Taxis move or disappear.
# 3. Return which taxi is nearest, not only the distance.
#
# INTERVIEW NOTES
# Asking for the distance from every cell to its nearest
# source is one BFS started from all sources at once, not
# one BFS per cell. Put all taxis in the queue at distance
# 0; the first time the search reaches a cell is its
# shortest distance, because BFS visits cells in order of
# distance.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from collections import deque


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))
```

```
def spread(grid, dist, queue, allow_improve=False):
    # grow the distances outward from the cells in `queue`;
    # with allow_improve a cell may be lowered when a closer source reaches it
    rows, cols = len(grid), len(grid[0])
    while queue:
        r, c = queue.popleft()
        for dr, dc in STEPS:
            nr, nc = r + dr, c + dc
            if not (0 <= nr < rows and 0 <= nc < cols):
                continue
            if grid[nr][nc] == "#":
                continue
            new = dist[r][c] + 1
            never_seen = dist[nr][nc] == -1
            if never_seen or (allow_improve and new < dist[nr][nc]):
                dist[nr][nc] = new
                queue.append((nr, nc))


def empty_distances(grid):
    # a table the size of the grid, -1 everywhere
    table = []
    for row in grid:
        table.append([-1] * len(row))
```

```
    return table


# ---- main question: main function ----
# Idea: 'T' taxi, '.' road, '#' wall. Start one BFS from every taxi
# at once; a cell gets its distance the first time it is reached,
# and cells no taxi can reach stay at -1.
# Complexity: Time O(m * n), Space O(m * n)
def taxi_distances(grid):
    dist = empty_distances(grid)
    queue = deque()
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == "T":
                dist[r][c] = 0
                queue.append((r, c))
    spread(grid, dist, queue)
    return dist
```

```
# ---- main question: tests ----
def test_main_question():
    d = taxi_distances(["T...", ".#..", "...T"])
    assert d[0] == [0, 1, 2, 2]                    # the taxi at the bottom right is closer to (0, 3)
    assert d[2] == [2, 2, 1, 0]
    assert d[1][1] == -1


# ========================================================
# FOLLOW-UP 1: A new taxi appears
# ========================================================
# ---- follow-up 1: function ----
# Answer: start a BFS from the new taxi but only continue into cells
# where it improves the stored distance. Distances only go down, so
# the work is bounded by the cells whose answer really changes.
# Complexity: Time O(m * n) worst case, usually only the cells it improves; Space O(m * n) for the queue
def add_taxi(grid, dist, r, c):
    dist[r][c] = 0
    spread(grid, dist, deque([(r, c)]), allow_improve=True)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    grid = ["T...", ".#..", "...T"]
    d = taxi_distances(grid)
    add_taxi(grid, d, 0, 2)                        # a new taxi in the middle of the top row
    assert d[0] == [0, 1, 0, 1]
    assert d[2][0] == 2
    assert taxi_distances(["T#.", "##."])[0][2] == -1   # unreachable cells stay -1


# ========================================================
# FOLLOW-UP 2: Taxis move or disappear
# ========================================================
# Answer: a disappearing taxi can make many distances larger, so
# recompute with one BFS from the taxis that are left (or keep, per
# cell, the two nearest taxis so one removal is cheap).


# ========================================================
# FOLLOW-UP 3: Return which taxi is nearest, not only the distance
# ========================================================
# ---- follow-up 3: function ----
# Answer: store a taxi number in each cell; a cell copies the number
# of the cell that first reached it. Ties go to whichever got there
```

```
# first, so say how you want ties broken.
# Complexity: Time O(m * n), Space O(m * n)
def nearest_taxi_ids(grid):
    rows, cols = len(grid), len(grid[0])
    owner = []
    for row in grid:
        owner.append([None] * len(row))
    queue = deque()
    taxi_id = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "T":
                owner[r][c] = taxi_id
                taxi_id += 1
                queue.append((r, c))
    while queue:
        r, c = queue.popleft()
        for dr, dc in STEPS:
            nr, nc = r + dr, c + dc
            if not (0 <= nr < rows and 0 <= nc < cols):
                continue
            if grid[nr][nc] != "#" and owner[nr][nc] is None:
                owner[nr][nc] = owner[r][c]
                queue.append((nr, nc))
    return owner


# ---- follow-up 3: tests ----
def test_follow_up_3():
    ids = nearest_taxi_ids(["T...", ".#..", "...T"])
    assert ids[0][0] == 0 and ids[2][3] == 1
    assert ids[1][1] is None


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> list[list[int]]:
```

Sample call

```
solution([[0,0,0],[0,1,0],[0,0,0]])
```

Test cases

Input	Expected
`([[0,0,0],[0,1,0],[0,0,0]],)`	`[[2,1,2],[1,0,1],[2,1,2]]`

Input	Expected
`([[1,0,0,0],[0,0,0,1],[0,0,0,0]],)`	`[[0,1,2,1],[1,2,1,0],[2,3,2,1]]`

STUDY PATH: question 5 of 197 | topic 1 of 43: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 5 of 6

Template fit: the island count again (queue version); the second part of the question is a short counting formula, no search | * asked 633 days ago *

MEDIUM | PracHub | Technical Screen | 5 of 197

★ TOP 75: rank 27 in the Medium and Hard priority list

★ Solve Grid and Counting Problems

POSTED 2025-01-25 | 633 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 64.8 | 30 DAYS 33 | PRIORITY 22/288

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #200 Number of Islands

LeetCode match: LC 200 Number of Islands (+ a counting task)

Posted on PracHub: 2025-01-25 (updated 2026-04-24)

Original: https://prachub.com/coding-questions/solve-grid-and-counting-problems

QUESTION

A software engineering intern phone screen included two coding tasks: 1.
Count islands in a grid Given an `m x n` grid of characters where `'1'` represents land and `'0'` represents water, return the number of islands. An island is formed by connecting adjacent land cells horizontally or vertically. You may assume all four edges of the grid are surrounded by water. 2.
Count ways to form a target word Given an array of uppercase letters, count how many different ways you can choose letters from the array so that the chosen letters can be rearranged to spell **`GOOGLE`**. Each array element can be used at most once, and letters at different indices are considered different choices even if they contain the same character.
Design efficient algorithms for both tasks, and discuss the time and space complexity of your approach.

Part 1: Count Islands in a GridGiven a 2D grid of characters where each cell is either '1' (land) or '0' (water), return the number of islands. An island is a group of adjacent land cells connected only horizontally or vertically. Diagonal cells do not count as connected. You may assume the grid edges are surrounded by water.
Constraints
0 <= number of rows <= 3000 <= number of columns <= 300grid[r][c] is either '0' or '1'All rows have the same length

EXAMPLES

```
Input: []
```

Expected Output:
Explanation: Empty grid contains no land, so there are no islands.

```
Input: [['1']]
```

Expected Output:
Explanation: A single land cell forms one island.

QUESTION

Hints

Whenever you find an unvisited land cell, start a DFS or BFS to mark the entire island.Only 4-directional neighbors matter: up, down, left, and right.

Part 2: Count Ways to Form the Word GOOGLEGiven an array of uppercase letters, count how many different ways you can choose letters so that the chosen letters can be rearranged to spell GOOGLE. Each array element can be used at most once. Letters at different indices count as different choices even if they contain the same character. The word GOOGLE requires exactly 2 G's, 2 O's, 1 L, and 1 E.

Constraints

0 <= len(letters) <= 200000Each element is an uppercase English letter from 'A' to 'Z'The answer may be large, but fits in Python's integer type

Examples

Input: []

Expected Output:

Explanation: No letters means it is impossible to form GOOGLE.

Input: ['G','O','O','G','L','E']

Expected Output:

Explanation: There is exactly one way to choose all six letters.

Hints

First count how many G, O, L, and E letters appear in the array.You need to choose 2 of the G positions and 2 of the O positions, then 1 L and 1 E.

BRUTE FORCE · Islands: recursive flood fill

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    seen, count = set(), 0
    for r, c in all_cells(grid):
        if grid[r][c] == "0" or (r, c) in seen:
            continue
        count += 1
        dfs(r, c)  # marks the whole island
    return count
```

TIME	SPACE
O(m * n)	**O(m * n)** recursion

WHY IT IS SLOW

Time is O(m * n), but the recursion can be m * n deep on a snake-shaped island and overflow the stack. A queue or an explicit stack avoids that.

BRUTE FORCE · Counting: enumerate every 6-element selection

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    count = 0
    for idx in combinations(range(len(a)), 6):
        letters = sorted(a[i] for i in idx)
        if letters == sorted("GGOOLE"):
            count += 1
    return count
```

BRUTE FORCE · Counting: enumerate every 6-element selection

TIME	SPACE
O(C(n, 6) * 6)	O(1)

WHY IT IS SLOW

C(n, 6) selections. The letters needed are fixed, so the answer is C(#G, 2) * C(#O, 2) * #L * #E from one pass of counts.

OPTIMAL SOLUTION · Islands: iterative BFS

TIME	SPACE
O(m * n)	O(min(m, n)) queue

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Counting: product of binomial coefficients

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Any target word, not only GOOGLE.

Count the needed letters of the word; the answer is the product over distinct letters of C(available, needed).

FOLLOW-UP 2 The letters must appear in order (a subsequence of the array spells the word).

Then it is the 'distinct subsequences' DP: ways[j] counts the ways to spell the first j letters; scan the array and update j from high to low. O(n * len(word)).

FOLLOW-UP 3 Islands arrive one cell at a time.

Use Union-Find: each new land cell makes the count go up by one and each union with a neighbouring different island makes it go down by one (LeetCode 305).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Solve Grid and Counting Problems
#
# A software engineering intern phone screen included two
# coding tasks: 1. **Count islands in a grid** Given an m
# x n grid of characters where '1' represents land and '0'
# represents water, return the number of islands. An
# island is formed by connecting adjacent land cells
# horizontally or vertically. You may assume all four
# edges of the grid are surrounded by water. 2. **Count
# ways to form a target word** Given an array of uppercase
# letters, count how many different ways you can choose
# letters from the array so that the chosen letters can be
# rearranged to spell GOOGLE. Each array element can be
```

```
# used at most once, and letters at different indices are
# considered different choices even if they contain the
# same character. Design efficient algorithms for both
# tasks, and discuss the time and space complexity of your
# approach.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Any target word, not only GOOGLE.
# 2. The letters must appear in order (a subsequence of
# the array spells the word).
# 3. Islands arrive one cell at a time.
#
# INTERVIEW NOTES
# Islands: every unvisited land cell starts a new island;
# mark everything connected to it. Counting: the target
# multiset is fixed (G twice, O twice, L once, E once), so
# the number of ways to choose the indices is a product of
# binomial coefficients, one factor per distinct letter.
# Do not enumerate arrangements.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
```

```
# ============================================================
# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def count_letters(letters):
    # how many times each letter appears
    counts = {}
    for ch in letters:
        counts[ch] = counts.get(ch, 0) + 1
    return counts


def choose(n, k):
    # the number of ways to pick k items out of n (n choose k)
    if k > n:
        return 0
    ways = 1
    for i in range(k):
        ways = ways * (n - i) // (i + 1)
    return ways


# ---- main question: main function ----
# PART 1: number of islands ('1' is land)
# Idea: scan every cell; an unseen land cell starts a new island, and a
# BFS marks all of its land as seen so it is counted once.
# Complexity: Time O(m * n), Space O(m * n)
def num_islands(grid):
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
```

```
    seen = set()
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] != "1" or (r, c) in seen:
                continue
            count += 1
            seen.add((r, c))
            queue = deque([(r, c)])
            while queue:
                x, y = queue.popleft()
                for dx, dy in STEPS:
                    nx, ny = x + dx, y + dy
```

```
                    if not (0 <= nx < rows and 0 <= ny < cols):
                        continue
                    if grid[nx][ny] == "1" and (nx, ny) not in seen:
                        seen.add((nx, ny))
                        queue.append((nx, ny))
    return count


# PART 2: in how many ways can the letters of an array spell GOOGLE?
# Idea: for every letter of the word, pick that many of the copies
# available in the array; multiply the choices (follow-up 1 allows any word).
# Complexity: Time O(n + w) for n letters and a word of w letters, Space O(w)
def ways_to_spell(letters, word="GOOGLE"):
    have = count_letters(letters)
    need = count_letters(word)
    total = 1
    for ch in need:
        total = total * choose(have.get(ch, 0), need[ch])
    return total


# ---- main question: tests ----
def test_main_question():
    assert num_islands([]) == 0
    assert num_islands([["1"]]) == 1
    assert num_islands(["11000", "11000", "00100", "00011"]) == 3
    assert ways_to_spell([]) == 0
    assert ways_to_spell(list("GOOGLE")) == 1
    assert ways_to_spell(list("GOOOGLE")) == 3          # three O's: choose 2 of them
    assert ways_to_spell(list("GGGOOLEE")) == 3 * 2      # an extra G and an extra E
    assert ways_to_spell(list("GOGLE")) == 0
    assert ways_to_spell(list("AABB"), "AB") == 4        # any target word


# ======================================================
# FOLLOW-UP 1: Any target word, not only GOOGLE
# ======================================================
# Answer: already handled by the `word` parameter of ways_to_spell above.


# ======================================================
# FOLLOW-UP 2: The letters must appear in order (a subsequence of the array spells the word)
# ======================================================
# ---- follow-up 2: function ----
# Answer: this is the "distinct subsequences" count. ways[j] is the
```

```
# number of ways to spell the first j letters of the word; go through
# the array and update j from high to low so one letter is not used twice.
# Complexity: Time O(n * w), Space O(w)
def ways_in_order(letters, word="GOOGLE"):
    ways = [0] * (len(word) + 1)
```

```
    ways[0] = 1
    for ch in letters:
        for j in range(len(word), 0, -1):
            if word[j - 1] == ch:
                ways[j] += ways[j - 1]
    return ways[len(word)]


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert ways_in_order(list("GOOGLE")) == 1
    assert ways_in_order(list("GGOOGLE")) == 2
    assert ways_in_order(list("GOGOLE")) == 0
    assert ways_in_order(list("EGOOGL")) == 0


# ========================================================
# FOLLOW-UP 3: Islands arrive one cell at a time
# ========================================================
# Answer: keep the islands in union-find and update the count per
# new cell; the code is IslandCounter in the "Number of Islands" question.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[str]]) -> int:
SAMPLE INPUT
[]
```

Test cases

Input	Expected
`[]`	`0`
`[['1']]`	`1`

STUDY PATH: question 6 of 197 | topic 1 of 43: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 6 of 6

Template fit: same on a graph: starts = the data centres, neighbors = adjacency list; answer = the city with the largest distance | * asked 375 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 6 of 197

Farthest City for a New Data Center

ASKED 2025-10-10 | 375 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 36.5 | 30 DAYS 0 | PRIORITY 119/288

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #542 01 Matrix

LeetCode match: LC 542 01 Matrix / LC 1162 As Far from Land as Possible (related: multi-source BFS)

Frequency: Single report | Last asked: 2025-10-10 | Roles: SWE Intern

QUESTION

Cities are nodes 0..num_cities-1 of an undirected, unweighted graph; edges[i] = [u, v] is a direct connection. You are given a non-empty list dcs of cities that already host a data center. You want to build a new data center in the city that is as far as possible from the nearest existing data center — that is, the city whose shortest-path distance (in number of edges) to its closest data center is maximized. Return that city's index. Break ties by choosing the smallest index. It is guaranteed every city can reach at least one data center.

Function signature

```
def farthest_city(num_cities: int, edges: list[list[int]], dcs: list[int]) -> int:
    ...
```

Follow-up (as reported): Run a single BFS seeded from all data centers at once (distance 0) instead of a BFS per data center.

EXAMPLES

Example 1	
Input	`num_cities = 5, edges = [[0,1],[1,2],[2,3],[3,4]], dcs = [0]`
Output	`4`
Why	`Distances from city 0 are 0,1,2,3,4; city 4 is farthest.`

Example 2	
Input	`num_cities = 5, edges = [[0,1],[1,2],[2,3],[3,4]], dcs = [0, 4]`
Output	`2`
Why	`Nearest-DC distances are 0->0, 1->1, 2->2, 3->1, 4->0; city 2 is farthest at distance 2.`

CONSTRAINTS

- 1 <= num_cities <= 10**5 , 0 <= len(edges) <= 2*10**5 .
- 1 <= len(dcs) <= num_cities , dcs are distinct valid city indices.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
from collections import deque
def brute_farthest(n, edges, dcs):
    adj = [[] for _ in range(n)]
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)
    best = [float("inf")] * n
    for src in dcs:
        dist = {src: 0}
        queue = deque([src])
        while queue:
            u = queue.popleft()
            for v in adj[u]:
                if v not in dist:
                    dist[v] = dist[u] + 1
                    queue.append(v)
        for city, d in dist.items():
            best[city] = min(best[city], d)
    return best.index(max(best))
```

TIME	SPACE
O(D * (V + E))	O(V)

WHY IT IS SLOW

Each data center repeats a full search over the graph.

OPTIMAL SOLUTION · One multi-source BFS

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The roads have lengths. How does it change?

Replace the BFS queue with a min-heap (Dijkstra) seeded with every data center at distance 0. The first time a city is popped its distance is final. O((V + E) log V).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Farthest City for a New Data Center
#
# Cities are nodes 0..num_cities-1 of an undirected,
# unweighted graph; edges[i] = [u, v] is a direct
# connection. You are given a non-empty list dcs of cities
# that already host a data center. You want to build a new
# data center in the city that is as far as possible from
# the nearest existing data center — that is, the city
# whose shortest-path distance (in number of edges) to its
# closest data center is maximized. Return that city's
# index. Break ties by choosing the smallest index. It is
```

```
# guaranteed every city can reach at least one data
# center. Function signature: def
# farthest_city(num_cities: int, edges: list[list[int]],
# dcs: list[int]) -> int: ... Follow-up (as reported): Run
# a single BFS seeded from all data centers at once
# (distance 0) instead of a BFS per data center.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The roads have lengths. How does it change?
#
# INTERVIEW NOTES
# The distance of a city to its nearest data center is the
# shortest path from the closest source, so start one BFS
# with every data center in the queue at distance 0. The
# first time a city is reached is its nearest-DC distance;
# the answer is the city with the largest such distance,
# the smallest index on ties.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import heapq
import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
```

```
def index_of_largest(values):
    # the position of the biggest number (the first one on a tie)
    best = 0
    for i in range(len(values)):
        if values[i] > values[best]:
            best = i
    return best


# ---- main question: main function ----
# Idea: put every data center in the queue at distance 0 and run one
# BFS; the first time a city is reached is by its nearest data center.
# The answer is the city with the biggest distance.
# Complexity: Time O(V + E), Space O(V + E)
def farthest_city(num_cities, edges, dcs):
    adj = []
    for i in range(num_cities):
        adj.append([])
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)
    dist = [-1] * num_cities
    queue = deque()
    for d in dcs:
        dist[d] = 0
        queue.append(d)
    while queue:
        u = queue.popleft()
        for v in adj[u]:
            if dist[v] == -1:
```

```
                dist[v] = dist[u] + 1
                queue.append(v)
    return index_of_largest(dist)


# ---- main question: tests ----
def test_main_question():
    path = [[0, 1], [1, 2], [2, 3], [3, 4]]
    assert farthest_city(5, path, [0]) == 4
    assert farthest_city(5, path, [0, 4]) == 2


# =========================================================
# FOLLOW-UP 1: The roads have lengths. How does it change?
# =========================================================
```

```
# ---- follow-up 1: function ----
# Answer: replace the queue by a min-heap (Dijkstra) that starts with
# every data center at distance 0. The first time a city is taken out
# of the heap its distance is final. O((V + E) log V).
# Complexity: Time O((V + E) log V), Space O(V + E)
def farthest_city_weighted(num_cities, roads, dcs):
    adj = []
    for i in range(num_cities):
        adj.append([])
    for u, v, w in roads:
        adj[u].append((v, w))
        adj[v].append((u, w))
    dist = [float("inf")] * num_cities
    heap = []
    for d in dcs:
        dist[d] = 0
        heap.append((0, d))
    heapq.heapify(heap)
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in adj[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (dist[v], v))
    return index_of_largest(dist)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    # with every road length 1 the weighted version must agree with the plain BFS
    random.seed(58)
    for _ in range(400):
        n = random.randint(2, 8)
        edges = []
        for i in range(1, n):
            edges.append([i, random.randrange(i)])        # keeps the graph connected
        for _ in range(random.randint(0, 3)):
            edges.append([random.randrange(n), random.randrange(n)])
        dcs = random.sample(range(n), random.randint(1, min(3, n)))
        unit = []
        for u, v in edges:
            unit.append([u, v, 1])
        assert farthest_city_weighted(n, unit, dcs) == farthest_city(n, edges, dcs)
```

```


# =========================================================
```

```python
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 2 OF 43 |

Sweep over events in time order

6 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

6
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Sweep over events in time order | 6 questions | tap a title to jump to it

READ FIRST: the code template for these 6 questions (tap here) >

M	7. Meeting Rooms / Interval Scheduling Variants	1Point3Acres * 2026-09-29 *
M	8. Train / Flight Schedule Reachability	1Point3Acres * 2026-10-08 *
M	9. Solve Rooms and Top-K Streams	PracHub * 2026-06-05 *
M	10. Gate Entry / Exit Time Resolution with Tie-Break Rules	1Point3Acres * 2026-05-04 *
M	11. Compute servers needed for daily recurring jobs	PracHub * 2025-11-05 *
M	12. Meeting Rooms III Variant	ND

CODE TEMPLATE | GROUP 2 OF 43 | 6 QUESTIONS, ONE PATTERN

Sweep over events in time order

The one idea these questions share

Turn every item into events (a start adds one, an end removes one, an arrival joins a queue), sort the events by time, and walk them keeping a small state: a counter, a queue, a heap or a running total. What you record at each event is the only thing that changes.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def max_overlap(intervals):
    events = sorted([(s, 1) for s, e in intervals] + [(e, -1) for s, e in intervals])
    cur = best = 0
    for t, delta in events:                  # CHANGE 1: what an event does to your state
        cur += delta
        best = max(best, cur)                # CHANGE 2: what you record
    return best                              # ends sort before starts at the same time (-1 < 1)
```

2. Helper functions that usually get written next to it

```
import heapq
from collections import deque


def two_queue_clock(arrivals):               # who is served when two queues share one server
    order = sorted(range(len(arrivals)), key=lambda i: arrivals[i])
    queue, t, i, served = deque(), 0, 0, []
    while i < len(order) or queue:
        if not queue:
            t = max(t, arrivals[order[i]])
        while i < len(order) and arrivals[order[i]] <= t:
            queue.append(order[i]); i += 1
        served.append((queue.popleft(), t))
        t += 1
    return served
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
7	Meeting Rooms / Interval Scheduling Variants	the template as written: the largest running count is the number of rooms
8	Train / Flight Schedule Reachability	sort legs by departure; a leg is usable if you have already reached its origin before it leaves; keep the earliest arrival per place
9	Solve Rooms and Top-K Streams	sort starts and ends separately and walk two pointers: a start before the earliest end needs a new room; the stream part is a size-K heap
10	Gate Entry / Exit Time Resolution with Tie-Break Rules	the same two-queue simulation with the tie-break rules from the statement
11	Compute servers needed for daily recurring jobs	merge each job's own intervals first so one job counts once, then sweep all of them together
12	Meeting Rooms III Variant	split meetings that cross midnight into two pieces first, then the same sweep

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 7 of 197 | topic 2 of 43: Sweep over events in time order | 1 of 6

Template fit: the template as written: the largest running count is the number of rooms | * asked 21 days ago * | frequency Very high

MEDIUM | IN APP LC 253 | 1Point3Acres | phone-screen, tech-screen, onsite-coding | 7 of 197

Meeting Rooms / Interval Scheduling Variants

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TYPE ALL TIME 58 30 DAYS 0 PRIORITY 51/288

ABOUT THIS QUESTION

Topic: Sweep over events in time order

Optimal answer from: your app's interview approach (LeetCode #253)

LeetCode match: LC 253 Meeting Rooms II

Frequency: Very high | Last asked: 2026-09-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/meeting-rooms-variants

QUESTION

LC 253 (Meeting Rooms II) appears in many forms across phone screen and onsite coding rounds — minimum number of rooms, room assignment, cyclic time, daily server jobs, gate entry/exit. Common follow-ups push toward sweep line, min-heap, or distributed processing.

The base task in every variant: given a list of intervals (start, end) — or pickup/return pairs, meeting time pairs, server job runs — answer one or more of:

Min rooms / cars / servers needed to honor all intervals concurrently (the canonical min-meeting-rooms problem).
Assignment: which interval goes to which room/car, so the caller can use the rental record.
Single-interval overlap test between two intervals.

Variant: parking-lot capacity

Arrival and departure intervals represent cars occupying a parking lot. Compute the peak simultaneous occupancy using the same interval-overlap model.

Variant: cyclic time (24h wrap)

Intervals wrap around midnight; if an interval ends after the day boundary, it carries into the next day.
Follow-up: "if N is huge, how do you optimize?" — expect a sweep-line discussion.

Variant: daily-recurring server jobs

Same set of jobs runs every day. If a job can't finish in one day, truncate at midnight and resume next day.
A server can run multiple identical jobs that overlap on the same machine; different jobs need separate servers.
Follow-up: a single job that runs many days needs its own dedicated server.

Variant: on-call rotation consolidation

Input: on-call rotations {name, start, end} using half-open intervals.
Output maximal contiguous segments during which the active on-call set does not change; omit gaps with nobody on call.
Segments are sorted by start, and names in each segment are the active set for that time range.
Clean approach: sort starts, keep a min-heap by end time, and emit a segment whenever the next start or next end changes the active set.

Variant: gate entry / exit with priority rules

Array of (timestamp, action) where action in {enter, exit}; each index is a different person.
Return for each person what time they actually enter/exit, applying tie-break rules:

QUESTION

If the prior tick had an enter, enter wins on a tie.
If the prior tick had an exit, exit wins on a tie.
If the prior tick had no activity, exit wins.
Within the winning class, lower index wins.

Optimization: instead of stepping time tick-by-tick, jump through the sorted input timestamps to reach O(n).

Variant: Google Calendar free-time intersection

Input: requested attendees plus each person's busy calendar intervals.
Return every time range where all requested attendees are free.
Normalize each person's busy blocks into free gaps, then intersect the candidate ranges across attendees.

Variant: car-rental assignment

Input: rental orders containing an order ID, pickup time, and return time.
Assign every order to a car so that no assigned intervals overlap, minimize the number of cars, and return each car's complete order list. Model each car explicitly with its own rental record.
A current output variant instead returns a dictionary from each reservation ID to its assigned car ID; the follow-up asks for the final fleet size.
A return at time t and another pickup at time t may reuse the same car.
Follow-up: if the fleet is capped at K cars and all requests cannot be served, define how the allocator detects and handles the failure.

Variant: exact-concurrency intervals

Compute the maximum number of simultaneously active meetings.
Given a target concurrency X, also return every maximal time range during which exactly X meetings are active.
Define the event ordering when one meeting starts at the same timestamp another ends; this choice controls whether a zero-width overlap is counted.

Variant: maximum-profit job scheduling

Each job has startTime, endTime, and profit. Choose a set of non-overlapping jobs maximizing total profit. The candidate identified it as LC 1235 (Maximum Profit in Job Scheduling).
Asked as the first question of an L4 onsite round, followed by an unrelated range-query problem in the same slot.

On-call employee metadata variant

Each duty record includes employee ID, employee name, start time, and end time. Return the employee count and identifying information for each time segment.

EXAMPLES

```
Canonical min-meeting-rooms example: [[0,30],[5,10],[15,20]] -> 2.
On-call consolidation: A[10,50], B[20,60], C[30,40], D[30,40] -> [10,20]: A, [20,30]: A,B, [30,40]:
    A,B,C,D, [40,50]: A,B, [50,60]: B.
Single-interval overlap: intervalsOverlap([1,5],[4,7]) -> true.
```

Long-running job: a 36-hour daily job is impossible under the truncate rule; reserve a separate server.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(intervals):
    best = 0
    for start, _ in intervals:
        busy = 0
        for s, e in intervals:
            if s <= start < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Every interval is compared with every other, O(n^2). Sorting start and end events and keeping a running count is O(n log n).

OPTIMAL SOLUTION · Sweep line over start and end events

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Meetings wrap around midnight (cyclic time).

Split every wrapped meeting [s, e) with s >= e into [s, 24h) and [0, e) and run the same sweep: the overlap at any clock time is unchanged. A meeting that is exactly 24 hours long covers the whole day.

FOLLOW-UP 2 The input is huge and spread over many machines.

Each shard sorts its events and produces a compressed timeline of (time, delta) changes; merge the sorted timelines with a k-way merge and take the running maximum, or add the per-shard counts at fixed time buckets. The answer is a max over time of a sum over shards.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Meeting Rooms / Interval Scheduling
# Variants
#
# LC 253 (Meeting Rooms II) appears in many forms across
# phone screen and onsite coding rounds — minimum number
# of rooms, room assignment, cyclic time, daily server
# jobs, gate entry/exit. Common follow-ups push toward
# sweep line, min-heap, or distributed processing. The
# base task in every variant: given a list of intervals
# (start, end) — or pickup/return pairs, meeting time
# pairs, server job runs — answer one or more of: Min
# rooms / cars / servers needed to honor all intervals
# concurrently (the canonical min-meeting-rooms problem).
# Assignment: which interval goes to which room/car, so
# the caller can use the rental record. Single-interval
# overlap test between two intervals. Variant: parking-lot
```

```
# capacity Arrival and departure intervals represent cars
# occupying a parking lot. Compute the peak simultaneous
# occupancy using the same interval-overlap model.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Meetings wrap around midnight (cyclic time).
# 2. The input is huge and spread over many machines.
#
# INTERVIEW NOTES
# The number of rooms needed is the largest number of
# intervals overlapping at any moment. Turn each interval
# into +1 at its start and -1 at its end, sort the events
# (ends before starts at the same time) and keep a running
# count. To say which room each meeting gets, keep a min-
# heap of (end time, room) and reuse the room that frees
# up first.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
```

```
# ============================================================
# ---- main question: main functions ----
# The minimum number of rooms for meetings (start, end).
# Idea (sweep line): a +1 event at every start and a -1 event at every end;
# sort them so an END at time t comes before a START at time t (a room is
# free the moment a meeting ends), and track the largest running count.
# Complexity: Time O(n log n), Space O(n)
def min_rooms(intervals):
    events = []
    for start, end in intervals:
        events.append((start, 1))
        events.append((end, -1))
    events.sort()                                   # (-1) sorts before (+1) at equal times
    running = 0
    best = 0
    for time, delta in events:
        running += delta
        if running > best:
            best = running
    return best


# VARIANT: which room does each meeting get?
# Complexity: Time O(n log n), Space O(n)
def assign_rooms(intervals):
    order = []
    for i in range(len(intervals)):
        order.append((intervals[i][0], intervals[i][1], i))
    order.sort()
    heap = []
    rooms = [0] * len(intervals)
    used = 0
    for start, end, i in order:
        if heap and heap[0][0] <= start:
            freed_at, room = heapq.heappop(heap)    # reuse the room that frees first, if it is free
```

```
        else:
            room = used
            used += 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms


# VARIANT: on-call consolidation. rotations are (name, start, end), half-open.
# Return the segments (start, end, set of people on call), merging neighbouring
```

```
# segments with the same people and leaving out the gaps.
# Complexity: Time O(T log T + T * p) for T change times and p people, Space O(T + p)
def oncall_segments(rotations):
    changes = {}                                    # time -> the (name, +1 or -1) changes at that time
    for name, start, end in rotations:
        if start not in changes:
            changes[start] = []
        changes[start].append((name, 1))
        if end not in changes:
            changes[end] = []
        changes[end].append((name, -1))
    active = {}
    segments = []
    times = sorted(changes)
    for i in range(len(times) - 1):
        t = times[i]
        following = times[i + 1]
        for name, change in changes[t]:
            active[name] = active.get(name, 0) + change
        people = set()
        for name in active:
            if active[name] != 0:
                people.add(name)
        who = frozenset(people)
        if not who:
            continue                                # nobody on call: a gap
        if segments and segments[len(segments) - 1][1] == t and segments[len(segments) - 1][2] == who:
            segments[len(segments) - 1] = (segments[len(segments) - 1][0], following, who)
        else:
            segments.append((t, following, who))
    return segments


# ---- main question: tests ----
def test_main_question():
    base = [(0, 30), (5, 10), (15, 20)]
    assert min_rooms(base) == 2
    assert min_rooms([(5, 8), (8, 9)]) == 1        # touching meetings share a room
    rooms = assign_rooms(base)
    assert rooms[0] != rooms[1] and rooms[1] == rooms[2]
    assert oncall_segments([("A", 1, 5), ("B", 3, 8)]) == [(1, 3, frozenset("A")), (3, 5, frozenset("AB")),
 (5, 8, frozenset("B"))]
    assert len(oncall_segments([("A", 1, 2), ("B", 4, 5)])) == 2     # a gap with nobody on call is left ou
t


# =============================================================
```

```
# FOLLOW-UP 1: Meetings wrap around midnight (cyclic time)
# =============================================================
# ---- follow-up 1: function ----
# Answer: split every meeting that wraps ([s, e) with s >= e) into [s, 24h)
# and [0, e) and run the same sweep: the overlap at any clock time is
# unchanged. (A meeting exactly 24 hours long covers the whole day.)
```

```python
# Complexity: Time O(n log n), Space O(n)
def min_rooms_cyclic(intervals, day=24 * 60):
    parts = []
    for start, end in intervals:
        if start < end:
            parts.append((start, end))
        else:
            parts.append((start, day))            # wraps past midnight: split at the boundary
            parts.append((0, end))
    return min_rooms(parts)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    wrap = [(22 * 60, 2 * 60), (23 * 60, 60), (0, 3 * 60)]
    assert min_rooms_cyclic(wrap) == 3


# ============================================================
# FOLLOW-UP 2: The input is huge and spread over many machines
# ============================================================
# Answer: each machine computes its own list of +1 / -1 events, sorted;
# merge the sorted event lists (a k-way merge, as in the interval-union
# question) and run the same running count: memory O(number of machines).


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill three solutions cold: sweep-line / prefix delta, min-heap on end time, sort + greedy room assignment.
Be ready to convert between the three when interviewers add follow-ups ("give me the assignment", "now handle cyclic days").
Prepare a 1-minute pitch on map-reduce style sharding (per-shard intervals -> merged timeline -> global max).

STUDY PATH: question 8 of 197 | topic 2 of 43: Sweep over events in time order | 2 of 6

Template fit: sort legs by departure; a leg is usable if you have already reached its origin before it leaves; keep the earliest arrival per place | * asked 12 days ago * | frequency Low

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 8 of 197

Train / Flight Schedule Reachability

ASKED 2026-10-08 | 12 DAYS AGO | LAST 30 DAYS

STUDY: TALK NO LEETCODE FREQUENCY PRIORITY 212/288

ABOUT THIS QUESTION

Topic: Sweep over events in time order

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 2092 Find All People With Secret (related)

Frequency: Low | Last asked: 2026-10-08 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/train-schedule-reachability

QUESTION

Given trains (or flights) as (from, to, departure, arrival), decide whether you can travel from a source to a destination, boarding a later leg only if it departs after you arrive. Graph reachability with time state, solved with BFS / DFS.

Input: a schedule of legs, each with a departure station, an arrival station, a departure time, and an arrival time; plus a source and a destination.

Output: whether the destination is reachable from the source, where a leg can only be taken if it departs after you arrive at its departure station.

The same prompt appears as a flight schedule between airports in new-grad R1.

EXAMPLES

```
trains = [
  ("A", "Y", dep=1, arr=20),
  ("A", "Y", dep=2, arr=5),
  ("Y", "B", dep=6, arr=9)
]
source = "A", destination = "B"

Output: true — the first A->Y train arrives too late for the Y->B connection, but the second one makes it
.
```

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def reachable(legs, src, dst):
    stack, seen = [(src, -INF)], set()
    while stack:
        station, now = stack.pop()
        if station == dst:
            return True
        if (station, now) in seen:
            continue
        seen.add((station, now))
        for a, b, dep, arr in legs:
            if a == station and dep > now:
                stack.append((b, arr))
    return False
```

BRUTE FORCE

TIME	SPACE
O(legs^2)	O(legs)

WHY IT IS SLOW

Every state rescans all legs and the same station is reached at many times, O(legs^2). Sorting the legs by departure and keeping only the earliest arrival per station answers in O(L log L).

OPTIMAL SOLUTION · Sort by departure and keep earliest arrivals

TIME	SPACE
O(L log L)	O(S)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the earliest arrival time at the destination.

The table already holds it: earliest_arrivals(legs, source)[dest], or infinity when the station is unreachable. That is the same one pass, so it costs nothing extra.

FOLLOW-UP 2 Print the route, not just yes or no.

Remember the leg that last improved each station's earliest arrival, then walk back from the destination to the source and reverse. Because only a strictly better arrival replaces the stored leg, the stored legs form a valid chain in time.

FOLLOW-UP 3 Many queries on the same schedule: what changes in the graph representation?

Group the legs by origin station and sort each list by departure time. For a query the earliest arrival is relaxed with a heap ordered by time (Dijkstra on time), and the first leg you may board at a station is found with bisect on its departure list, so each query costs O(L log S) instead of re-sorting every time. If the source is fixed and the queries only differ by destination, keep the earliest-arrival table and answer each in O(1).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Train / Flight Schedule Reachability
#
# Given trains (or flights) as (from, to, departure,
# arrival), decide whether you can travel from a source to
# a destination, boarding a later leg only if it departs
# after you arrive. Graph reachability with time state,
# solved with BFS / DFS. Input: a schedule of legs, each
# with a departure station, an arrival station, a
# departure time, and an arrival time; plus a source and a
# destination. Output: whether the destination is
# reachable from the source, where a leg can only be taken
# if it departs after you arrive at its departure station.
# The same prompt appears as a flight schedule between
# airports in new-grad R1. Examples: trains = [ ("A", "Y",
# dep=1, arr=20), ("A", "Y", dep=2, arr=5), ("Y", "B",
# dep=6, arr=9) ] source = "A", destination = "B" Output:
# true — the first A->Y train arrives too late for the Y->B
# connection, but the second one makes it.
```

```
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the earliest arrival time at the destination.
# 2. Print the route, not just yes or no.
# 3. Many queries on the same schedule: what changes in
# the graph representation?
#
# INTERVIEW NOTES
# Plain station-to-station reachability is wrong because a
# leg is only usable if you are already at its departure
# station before it leaves. Time is part of the state, so
# ask what you need to know about each station: the
# earliest time you can be there. Process the legs in
# order of departure time; a leg is usable if the earliest
# arrival at its origin is not later than its departure,
# and then it can improve the earliest arrival at its
# destination. One sort and one pass answer reachability,
# and the same table gives the earliest arrival time.

# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

import random
```

```
INF = float("inf")


# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: helper functions ----
def departure_of(leg):
    # a leg is (from station, to station, departure time, arrival time)
    return leg[2]


# ---- main question: main functions ----
# Which stations can you reach from `source`? Time is part of the state:
# you can only take a leg if you are already at its station BEFORE it leaves.
# Idea: sort the legs by departure time and keep, for every station, the
# EARLIEST time you can be there (the source: minus infinity, you are there
# from the start). Going through the legs in departure order, every leg that
# could feed the current one has already been handled. A leg is usable if we
# are at its station strictly before it departs (strict=False allows a
# connection at the same minute).
# Complexity: Time O(L log L) for L legs, Space O(S) for S stations
def earliest_arrivals(legs, source, strict=True):
    earliest = {source: -INF}
    for from_station, to_station, departs, arrives in sorted(legs, key=departure_of):
        have = earliest.get(from_station, INF)
        usable = have < departs or (not strict and have == departs)
        if usable and arrives < earliest.get(to_station, INF):
            earliest[to_station] = arrives
    return earliest


# Complexity: Time O(L log L), Space O(S)
def reachable(legs, source, dest, strict=True):
    return dest in earliest_arrivals(legs, source, strict)
```

```
# ---- main question: tests ----
LEGS = [("A", "Y", 1, 20), ("A", "Y", 2, 5), ("Y", "B", 6, 9)]


def test_main_question():
    assert reachable(LEGS, "A", "B")
    assert not reachable(LEGS[:1] + LEGS[2:], "A", "B")        # without the early A to Y train the connect
ion is lost
```

```
    assert earliest_arrivals(LEGS, "A")["B"] == 9
    assert reachable(LEGS, "A", "A")
    touching = [("A", "B", 1, 5), ("B", "C", 5, 8)]
    assert not reachable(touching, "A", "C")                   # arriving at 5 and leaving at 5 is too tigh
t ...
    assert reachable(touching, "A", "C", strict=False)         # ... unless same-minute connections are all
owed


# ========================================================
# FOLLOW-UP 1: Return the earliest arrival time at the destination
# ========================================================
# Answer: earliest_arrivals already holds it: earliest_arrivals(legs, source)[dest].


# ========================================================
# FOLLOW-UP 2: Print the route, not just yes or no
# ========================================================
# ---- follow-up 2: function ----
# Answer: remember the leg that last improved each station's earliest
# arrival, then walk back from the destination to the source and reverse.
# Complexity: Time O(L log L), Space O(S)
def itinerary(legs, source, dest, strict=True):
    earliest = {source: -INF}
    came_by = {}                                       # station -> the leg that got us there
    for leg in sorted(legs, key=departure_of):
        from_station, to_station, departs, arrives = leg
        have = earliest.get(from_station, INF)
        usable = have < departs or (not strict and have == departs)
        if usable and arrives < earliest.get(to_station, INF):
            earliest[to_station] = arrives
            came_by[to_station] = leg
    if dest not in earliest:
        return None
    route = []
    while dest != source:
        leg = came_by[dest]
        route.append(leg)
        dest = leg[0]
    route.reverse()
    return route


# ---- follow-up 2: tests ----
def reachable_slowly(legs, source, dest):
    # a different way to check: keep trying every leg until nothing changes
    earliest = {source: -INF}
```

```
    changed = True
    while changed:
        changed = False
        for from_station, to_station, departs, arrives in legs:
            if earliest.get(from_station, INF) < departs and arrives < earliest.get(to_station, INF):
                earliest[to_station] = arrives
                changed = True
```

```
    return dest in earliest


def test_follow_up_2():
    assert itinerary(LEGS, "A", "B") == [LEGS[1], LEGS[2]]
    assert itinerary(LEGS, "B", "A") is None
    random.seed(23)
    for round_number in range(300):
        legs = []
        for i in range(random.randint(0, 8)):
            departs = random.randint(0, 9)
            legs.append((random.choice("ABCD"), random.choice("ABCD"), departs, departs + random.randint(1,
 4)))
        source = random.choice("ABCD")
        dest = random.choice("ABCD")
        assert reachable(legs, source, dest) == reachable_slowly(legs, source, dest)


# =====================================================
# FOLLOW-UP 3: Many queries on the same schedule: what changes in the graph representation?
# =====================================================
# Answer: build the graph once with one NODE per (station, time) event and an
# edge for every leg plus "wait" edges between a station's consecutive times.
# Then every query is a search from the source's first node; answers for the
# same source can be shared.


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement BFS / DFS over (station, arrival time) states that only follows legs departing after the current arrival. Hand-trace the example, including the late A->Y leg that must be rejected.

STUDY PATH: question 9 of 197 | topic 2 of 43: Sweep over events in time order | 3 of 6

Template fit: sort starts and ends separately and walk two pointers: a start before the earliest end needs a new room; the stream part is a size-K heap | * asked 137 days ago *

MEDIUM | PracHub | Technical Screen | 9 of 197

★ TOP 75: rank 69 in the Medium and Hard priority list

★ Solve Rooms and Top-K Streams

POSTED 2026-06-05 | 137 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE **ALL TIME 58** **30 DAYS 0** **PRIORITY 47/288**

ABOUT THIS QUESTION

Topic: Sweep over events in time order

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #253 Meeting Rooms II, LeetCode #703 Kth Largest Element in a Stream

LeetCode match: LC 253 Meeting Rooms II + LC 703 Kth Largest Element in a Stream (inferred from overview)

Posted on PracHub: 2026-06-05 (updated 2026-06-14)

Original: https://prachub.com/coding-questions/solve-rooms-and-top-k-streams

QUESTION

Two small tasks. (1) min_rooms(intervals): the minimum number of rooms needed to hold all meetings; a meeting may start exactly when another ends. (2) TopKStream(k): numbers arrive one at a time with add(x); top() returns the k largest numbers seen so far, largest first.

Function signature

```
def min_rooms(intervals: list) -> int
class TopKStream:
    def __init__(self, k: int)
    def add(self, x: int) -> None
    def top(self) -> list
```

EXAMPLES

Example 1	
Input	`[(0, 30), (5, 10), (15, 20)]`
Output	`2`
Why	`(5, 10) and (15, 20) both overlap the long meeting (0, 30): 2 rooms.`

Example 2	
Input	`[(7, 10), (2, 4)]`
Output	`1`
Why	`The second meeting ends before the first starts: one room is enough.`

QUESTION

Example 3

Input	`s = TopKStream(3)`
	`s.add(5)`
	`s.add(1)`
	`s.add(9)`
	`s.add(7)`
	`s.top()`

Output	`s.top() -> [9, 7, 5]`
Why	`Of 5, 1, 9, 7 the three largest are 9, 7 and 5.`

BRUTE FORCE · Rooms: count the overlap at every time unit

BRUTE FORCE · PSEUDO-CODE

```
def rooms(meetings):
    lo = min(s for s, e in meetings)
    hi = max(e for s, e in meetings)
    best = 0
    for t in range(lo, hi):
        busy = 0
        for s, e in meetings:
            if s <= t < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(range * n)	O(1)

WHY IT IS SLOW

Depends on the length of the time range, not only on n: O(range * n). Sorted starts and ends with two pointers (or a heap of end times) are O(n log n).

BRUTE FORCE · Stream: sort everything on every query

BRUTE FORCE · PSEUDO-CODE

```
values = []
def add(x):
    values.append(x)
    values.sort(reverse=True)
    return values[K - 1]
```

TIME	SPACE
O(n log n) per add	O(n)

WHY IT IS SLOW

Every add re-sorts the whole history, O(n log n). A min-heap of size K keeps the K-th largest at its root in O(log K) per add.

OPTIMAL SOLUTION · Rooms: sorted starts and ends with two pointers

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Stream: min-heap of size K

TIME	SPACE
O(log k) per add	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Rooms: return which room each meeting uses.

Sort by start and keep a min-heap of (end time, room). If the earliest-ending room is free at the next start, reuse it, otherwise open a new room. The heap size never exceeds the answer.

FOLLOW-UP 2 Stream: K changes over time, or you need the median.

A fixed-size heap cannot answer a different K. Use a sorted structure with order statistics (balanced tree or Fenwick tree over values), or two heaps for the median. **Cost:** a balanced tree or Fenwick tree gives O(log n) per insert and per k-th query; two heaps give O(log n) per insert and O(1) per median.

FOLLOW-UP 3 Rooms: meetings on several floors with capacities.

Group meetings by required capacity, schedule each group separately with the sweep, and assign rooms from the smallest sufficient capacity (best fit) when capacities differ.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Solve Rooms and Top-K Streams
#
# Two small tasks. (1) min_rooms(intervals): the minimum
# number of rooms needed to hold all meetings; a meeting
# may start exactly when another ends. (2) TopKStream(k):
# numbers arrive one at a time with add(x); top() returns
# the k largest numbers seen so far, largest first.
#
# Signature:
#   def min_rooms(intervals: list) -> int
#   class TopKStream:
#       def __init__(self, k: int)
#       def add(self, x: int) -> None
#       def top(self) -> list
#
# Examples:
# [(0, 30), (5, 10), (15, 20)]  ->  2
# [(7, 10), (2, 4)]  ->  1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Rooms: return which room each meeting uses.
```

```
# 2. Stream: K changes over time, or you need the median.
# 3. Rooms: meetings on several floors with capacities.
#
# INTERVIEW NOTES
# Rooms: the number of rooms needed is the largest number
# of meetings happening at the same time, so sort the
# starts and the ends separately and sweep: a start that
# comes before the earliest end needs a new room. Stream:
# only the K largest values can matter, so keep exactly
# those in a min-heap; the root is the K-th largest.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq


# ========================================================
# MAIN QUESTION
# ========================================================
```

```
# ---- main question: main functions ----
# PART 1: the minimum number of rooms for meetings (start, end).
# Idea: sort the starts and the ends SEPARATELY. For each start: if it is at
# or after the earliest end that is still unused, that room is reused (move the
# end pointer on); otherwise a new room is needed.
# Complexity: Time O(n log n), Space O(n)
def min_rooms(intervals):
    starts = []
    ends = []
    for start, end in intervals:
        starts.append(start)
        ends.append(end)
    starts.sort()
    ends.sort()
    rooms = 0
    j = 0                                       # the earliest end not yet used
    for start in starts:
        if start >= ends[j]:
            j += 1                              # the earliest-ending meeting is over: reuse its room
        else:
            rooms += 1
    return rooms


# PART 2: the k largest values seen so far in a stream (fixed k).
# Idea: a min-heap of size k: its root is the k-th largest. A new value
# enters only if it beats the root.
# Complexity: Time add O(log k), top O(k log k), Space O(k)
class TopKStream:
    def __init__(self, k):
        self.k = k
        self.heap = []

    def add(self, x):
        if len(self.heap) < self.k:
            heapq.heappush(self.heap, x)
        elif x > self.heap[0]:
            heapq.heapreplace(self.heap, x)

    def top(self):
```

```
        return sorted(self.heap, reverse=True)


# ---- main question: tests ----
def test_main_question():
```

```
    assert min_rooms([(0, 30), (5, 10), (15, 20)]) == 2
    assert min_rooms([(1, 3), (3, 5)]) == 1                    # a meeting may start the moment another e
nds
    assert min_rooms([]) == 0
    stream = TopKStream(3)
    for x in [5, 1, 9, 7, 3, 8]:
        stream.add(x)
    assert stream.top() == [9, 8, 7]


# ========================================================
# FOLLOW-UP 1: Rooms: return which room each meeting uses
# ========================================================
# ---- follow-up 1: function ----
# Answer: sort by start and keep a min-heap of (end time, room). If the
# earliest-ending room is free at the next start, reuse it, otherwise open a
# new room. The heap never holds more rooms than the answer.
# Complexity: Time O(n log n), Space O(n)
def assign_rooms(intervals):
    order = []
    for i in range(len(intervals)):
        order.append((intervals[i][0], intervals[i][1], i))
    order.sort()
    heap = []
    rooms = [0] * len(intervals)
    opened = 0
    for start, end, i in order:
        if heap and heap[0][0] <= start:
            freed_at, room = heapq.heappop(heap)
        else:
            room = opened
            opened += 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms


# ---- follow-up 1: tests ----
def test_follow_up_1():
    rooms = assign_rooms([(0, 30), (5, 10), (15, 20)])
    assert rooms[0] != rooms[1] and rooms[1] == rooms[2]


# ========================================================
# FOLLOW-UP 2: Stream: K changes over time, or you need the median
# ========================================================
```

```
# ---- follow-up 2: function ----
# Answer: a heap of fixed size k cannot grow cheaply when k changes: keep
# the sorted values (or a balanced tree) instead. For the MEDIAN use two
# heaps: the lower half in a max-heap (negated numbers) and the upper half in
# a min-heap, with sizes differing by at most one.
# Complexity: Time add O(log n), median O(1), Space O(n)
class MedianStream:
    def __init__(self):
        self.low = []                               # max-heap (negated numbers)
        self.high = []                              # min-heap

    def add(self, x):
```

```
        heapq.heappush(self.low, -x)
        top = -heapq.heappop(self.low)
        heapq.heappush(self.high, top)
        if len(self.high) > len(self.low):
            back = heapq.heappop(self.high)
            heapq.heappush(self.low, -back)

    def median(self):
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2


# ---- follow-up 2: tests ----
def test_follow_up_2():
    stream = MedianStream()
    for x in [5, 2, 8, 1]:
        stream.add(x)
    assert stream.median() == 3.5


# ============================================================
# FOLLOW-UP 3: Rooms: meetings on several floors with capacities
# ============================================================
# Answer: keep one heap per floor (or per room size); a meeting asks for
# the smallest room that is big enough and free, using the same idea: take the
# room that frees up first among the rooms that fit.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(intervals: list[list[int]]) -> int:
```

Sample call

```
solution([[0, 30], [5, 10], [15, 20]])
```

Test cases

Input	Expected
`([[0, 30], [5, 10], [15, 20]],)`	`2`
`([],)`	`0`

STUDY PATH: question 10 of 197 | topic 2 of 43: Sweep over events in time order | 4 of 6

Template fit: the same two-queue simulation with the tie-break rules from the statement | * asked 169 days ago * | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 10 of 197

Gate Entry / Exit Time Resolution with Tie-Break Rules

ASKED 2026-05-04 | 169 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 27.4** **30 DAYS 0** **PRIORITY 152/288**

ABOUT THIS QUESTION

Topic: Sweep over events in time order

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 2534 Time Taken to Cross the Door

Frequency: Single report | Last asked: 2026-05-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/gate-entry-exit-scheduler

QUESTION

A gate lets one person through per second. events[i] = (time, action) for person i, where action is 'enter' or 'exit'. When several people want the gate in the same second: if the gate was not used in the previous second, people who exit go first; if it was used, the direction used in the previous second goes first again. Within one direction, people go in order of arrival (person id). Return the second at which each person passes.

Function signature

def gate_pass_times(events: list) -> list

EXAMPLES

Example 1

Input	`[(0, 'enter'), (0, 'exit'), (1, 'enter'), (5, 'enter')]`
Output	`[1, 0, 2, 5]`
Why	`At second 0 the gate was idle, so the exit (person 1) goes first. Person 0 follows at 1, person 2 (arrived at 1) at 2, and person 3 arrives alone at 5.`

Example 2

Input	`[(0, 'enter'), (1, 'enter'), (1, 'exit')]`
Output	`[0, 1, 2]`
Why	`The gate was used for entering in the previous second, so the waiting enter keeps priority over the exit.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(arrivals):
    t, prev = 0, None
    enter_q, exit_q = deque(), deque()
    while pending(arrivals, enter_q, exit_q):
        add_arrivals(t, arrivals, enter_q,
                     exit_q)
        queue = choose(prev, enter_q, exit_q)
        if queue:
            person = queue.popleft()
            passed[person] = t
            prev = name_of(queue)
        t += 1
    return passed
```

TIME	SPACE
O(T + n) where T is the last timestamp	**O(n)**

WHY IT IS SLOW

It runs through every second between the first and last timestamp, so a timestamp of 10^9 means a billion iterations. Jumping to the next arrival when the queues are empty makes it O(n log n).

OPTIMAL SOLUTION · Event-driven simulation with two queues

TIME	SPACE
O(n log n) for the sort, O(n) simulation	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why does the optimization need care across an empty gap?

If the door sits idle for even one second the previous state becomes 'none', which favours exits again. Jumping time forward without resetting the state would wrongly keep 'enter' priority from before the gap. Compare t with last_used + 1 to know whether the gap really was idle.

FOLLOW-UP 2 What is the complexity, and what if events come online?

Offline it is O(n log n) for the sort, then O(n) for the simulation, independent of how large the timestamps are. Online, keep the two queues and the previous-state variable and advance the clock on every arrival or tick; the work per person stays O(1).

FOLLOW-UP 3 Timestamps go up to 10^9.

That is why the event-driven loop jumps to the next arrival: its running time is O(n log n) and does not depend on the size of the timestamps. Remember to reset the previous state to idle across a gap.

FOLLOW-UP 4 The tie-break when the gate was idle differs in this version of the question.

Keep the simulation and make the idle winner a parameter: only the line that picks the queue changes. With 'exit first' it equals the standard rule; with 'enter first' an enter and an exit arriving together swap places.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Gate Entry / Exit Time Resolution with
# Tie-Break Rules
#
# A gate lets one person through per second. events[i] =
# (time, action) for person i, where action is 'enter' or
# 'exit'. When several people want the gate in the same
# second: if the gate was not used in the previous second,
# people who exit go first; if it was used, the direction
# used in the previous second goes first again. Within one
# direction, people go in order of arrival (person id).
# Return the second at which each person passes.
#
# Signature:
#   def gate_pass_times(events: list) -> list
#
# Examples:
# [(0, 'enter'), (0, 'exit'), (1, 'enter'), (5, 'enter')]
#     ->  [1, 0, 2, 5]
# [(0, 'enter'), (1, 'enter'), (1, 'exit')]  ->  [0, 1, 2]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why does the optimization need care across an empty
# gap?
# 2. What is the complexity, and what if events come
# online?
# 3. Timestamps go up to 10^9.
# 4. The tie-break when the gate was idle differs in this
# version of the question.
#
# INTERVIEW NOTES
# Write the rule as a small state machine first: the state
# is what the gate did in the previous second (entered,
# exited or idle), and the rule picks who goes next from
# that state. Then replace the second-by-second loop with
# a loop over people: when nobody is waiting jump the
# clock forward, and remember that an idle gap resets the
# state to idle.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================
```

```
from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# People arrive at a gate wanting to enter or to exit (events[i] = (arrival
# time, "enter" or "exit")). The gate lets ONE person through per second.
# Rules when several wait: if the gate was used the second before, the same
# direction keeps going; if it was idle, one direction wins (exit, by default).
# Within a direction, people go in order of arrival, then by number.
# Return the second each person goes through.
# Idea: an event-driven simulation with one queue per direction. Sort the
```

```
# people by (arrival time, number). When nobody waits, jump the clock straight
# to the next arrival (so huge timestamps cost nothing); if that leaves at
# least one idle second, forget the previous direction. Then everyone who has
# arrived joins their queue, and one person goes through.
# (`idle_first` is follow-up 4: who wins when the gate was idle.)
# Complexity: Time O(n log n) (the sort of arrivals; the simulation is O(n)), Space O(n)
def gate_pass_times(events, idle_first="exit"):
    n = len(events)
    arrivals = []
    for i in range(n):
        arrivals.append((events[i][0], i))
    arrivals.sort()
    lines = {"enter": deque(), "exit": deque()}
    result = [0] * n
    t = 0
    next_person = 0
    previous = None                                 # the direction used in the second before
    last_used = -10                                 # the last second the gate was used
    while next_person < n or lines["enter"] or lines["exit"]:
        if not lines["enter"] and not lines["exit"]:
            if arrivals[next_person][0] > t:
                t = arrivals[next_person][0]        # nobody waits: jump to the next arrival
            if t > last_used + 1:
                previous = None                     # an idle second forgets the last direction
        while next_person < n and arrivals[next_person][0] <= t:
            person = arrivals[next_person][1]
            lines[events[person][1]].append(person)   # everyone who has arrived joins their line
            next_person += 1
        if lines["enter"] and lines["exit"]:
            if previous is not None:
                want = previous                     # keep the direction used last second
```

```
            else:
                want = idle_first                   # idle gate: the tie-break rule decides
        elif lines["enter"]:
            want = "enter"
        else:
            want = "exit"
        person = lines[want].popleft()
        result[person] = t
        previous = want
        last_used = t
        t += 1
    return result


# ---- main question: tests ----
def test_main_question():
    both = [(0, "enter"), (0, "exit")]
    assert gate_pass_times(both) == [1, 0]                      # one enter and one exit together: exit fi
rst
    assert gate_pass_times(both, "enter") == [0, 1]             # with "enter" winning the idle tie it swa
ps
    assert gate_pass_times([(0, "enter"), (0, "enter"), (1, "exit")]) == [0, 1, 2]    # the same direction
continues
    assert gate_pass_times([(0, "enter"), (10, "enter"), (10, "exit")]) == [0, 11, 10]  # a long idle gap r
esets the state
    assert gate_pass_times([(10 ** 9, "enter"), (10 ** 9, "exit")]) == [10 ** 9 + 1, 10 ** 9]
    assert gate_pass_times([]) == []


# =======================================================
# FOLLOW-UP 1: Why does the optimization need care across an empty gap?
# =======================================================
# Answer: jumping the clock over an empty gap is only safe if the "previous
```

```
# direction" is forgotten when at least one idle second passes; otherwise a
# person arriving after the gap would be treated as if the gate had just been
# used in the same direction.


# ============================================================
# FOLLOW-UP 2: What is the complexity, and what if events come online?
# ============================================================
# Answer: O(n log n) for the sort and O(n) for the loop. Online, keep the two
# queues and the clock: handle a person when the clock reaches their arrival.


# ============================================================
# FOLLOW-UP 3: Timestamps go up to 10^9
# ============================================================
# Answer: nothing changes: the clock jumps over empty stretches, so the work is
```

```
# one step per person, not per second.


# ============================================================
# FOLLOW-UP 4: The tie-break when the gate was idle differs in this version of the question
# ============================================================
# Answer: make the idle winner a parameter: only the line that picks the
# direction changes. With "exit" it is the standard rule; with "enter" an
# enter and an exit arriving together swap places (see the second test above).


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Translate the four priority rules into an explicit state machine (prev_state in {enter, exit, none}) before writing any code.
Practice scheduling problems that pair a per-tick simulation with an event-driven optimization (CPU scheduler, traffic light, calendar conflict resolver).
Stress-test corner cases: same timestamp containing both an enter and an exit for the same person, all-enters or all-exits batches, and very large gaps between two events.

STUDY PATH: question 11 of 197 | topic 2 of 43: Sweep over events in time order | 5 of 6

Template fit: merge each job's own intervals first so one job counts once, then sweep all of them together | * asked 349 days ago *

MEDIUM | IN APP LC 253 | PracHub | Technical Screen | 11 of 197

★ TOP 75: rank 58 in the Medium and Hard priority list

★ Compute servers needed for daily recurring jobs

POSTED 2025-11-05 | 349 DAYS AGO | OLDER

STUDY: TYPE **ALL TIME 58** **30 DAYS 0** **PRIORITY 48/288**

ABOUT THIS QUESTION

Topic: Sweep over events in time order

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #253 Meeting Rooms II

LeetCode match: LC 253 Meeting Rooms II

Posted on PracHub: 2025-11-05 (updated 2026-06-25)

Original: https://prachub.com/coding-questions/compute-servers-needed-for-daily-recurring-jobs

QUESTION

You operate a cluster of identical servers that run recurring **daily** jobs. For a single day, you are given a list of job execution intervals. Each interval has:

- **`job_id`** (string or integer),
- **`start_time`** (timestamp within that day),
- **`end_time`** (timestamp within that day), with **`start_time < end_time`**. Interpret each interval as a half-open time range **`[start_time, end_time)`** during which that job must be running on some server **on that day**. Rules and constraints:
- Each physical server can execute at most **one job at a time**.
- However, if multiple intervals in the input correspond to the **same `job_id`**, they may overlap in time **on the same server** (they are considered parts or retries of the same job and do not consume extra capacity).
- If a job's true execution would extend past midnight into the next day, it is conceptually **truncated at the end of the current day** for scheduling purposes; for each day you only see the portion that falls within that day's 24-hour window.

Tasks

1. Given all job intervals for a single day (potentially with repeated **`job_id`**s and overlapping intervals), compute the **minimum number of servers** required to schedule all the jobs under the above rules.
2. Describe the time and space complexity of your algorithm in terms of the number of intervals (m).

Follow-up

Some rare jobs are extremely long-running and conceptually span **multiple days** without restart. For such a multi-day job, you decide to reserve a **dedicated server** that is not shared with any other jobs, but which may still handle different intervals of that same **`job_id`**. Explain how you would extend or adapt your solution to:

- Detect which jobs require a dedicated server based on their multi-day duration or configuration.
- Account for these dedicated servers when computing the total number of servers needed for the system.

QUESTION

Part 1: Minimum Servers for One Day with Overlapping Same-Job IntervalsYou are given all job execution intervals for a single day. Each interval is half-open, meaning [start_time, end_time), and represents a time range during which a job must be running. A server can run at most one job at a time. However, if multiple intervals have the same job_id, they are considered parts or retries of the same job, so overlapping intervals with the same job_id consume only one server total during the overlap. Return the minimum number of servers required to run all intervals for the day.
Constraints
0 <= len(intervals) <= 2000000 <= job_id <= 10^90 <= start_time < end_time <= 1440Intervals are half-open: an interval ending at time t does not overlap an interval starting at time t.Multiple intervals may have the same job_id.

EXAMPLES

```
Input: ([],)
Expected Output: 0
```

Explanation: There are no jobs, so no servers are needed.

```
Input: ([[5, 100, 200], [5, 150, 250], [5, 240, 300]],)
Expected Output: 1
```

Explanation: All intervals belong to the same job_id. Even though they overlap, they can share one server.

Hints

At any moment, one server is needed for each distinct active job_id, not for each active interval.Try merging all intervals belonging to the same job_id first, then run a sweep-line algorithm over the merged intervals.

Part 2: Total Servers with Dedicated Multi-Day JobsYou are given true job execution intervals that may span multiple days. Some jobs require dedicated servers. A job is dedicated if its job_id is listed in configured_dedicated, or if any one of its true intervals has duration at least min_dedicated_duration minutes. A dedicated job uses one reserved server for that job_id and does not share that server with other jobs, though multiple intervals of the same dedicated job_id may use the same dedicated server. For a given target_day, compute the total number of

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def servers_needed(intervals):
    best = 0
    for t in range(DAY):
        jobs = set()
        for job_id, s, e in intervals:
            if s <= t < e:
                jobs.add(job_id)
        best = max(best, len(jobs))
    return best
```

TIME	SPACE
O(T * m)	O(m)

WHY IT IS SLOW

Depends on the length of the day, O(T * m). Merge each job's own intervals first, then sort start and end events and keep a running count, O(m log m).

OPTIMAL SOLUTION · Merge each job's intervals, then sweep start and end events

TIME	SPACE
O(m log m)	O(m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Some rare jobs run for several days without restart and get a dedicated server.

Remove those jobs from the daily intervals and add one server per such job: servers = (number of dedicated jobs) + (maximum overlap of the remaining jobs).

FOLLOW-UP 2 Jobs that cross midnight.

Truncate at the end of the day for each day's schedule (as the statement says). For a recurring daily job that wraps, split it into two pieces (the end of the day and the start of the day); the sweep then handles both on the same circular day.

FOLLOW-UP 3 Return which server each job runs on.

Sort merged intervals by start and keep a min-heap of (end time, server id): reuse the server that frees up first if it is free, otherwise open a new one. The number of servers opened equals the maximum overlap.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Compute servers needed for daily
# recurring jobs
#
# You operate a cluster of identical servers that run
# recurring **daily** jobs. For a single day, you are
# given a list of job execution intervals. Each interval
# has: - job_id (string or integer), - start_time
# (timestamp within that day), - end_time (timestamp
# within that day), with start_time < end_time. Interpret
# each interval as a half-open time range [start_time,
# end_time) during which that job must be running on some
# server **on that day**. Rules and constraints: - Each
# physical server can execute at most **one job at a
# time**. - However, if multiple intervals in the input
# correspond to the **same job_id**, they may overlap in
# time **on the same server** (they are considered parts
# or retries of the same job and do not consume extra
# capacity).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Some rare jobs run for several days without restart
# and get a dedicated server.
# 2. Jobs that cross midnight.
# 3. Return which server each job runs on.
#
# INTERVIEW NOTES
# The number of servers is the largest number of jobs
# running at the same time. First remove the twist:
# overlapping intervals of the same job are really one
# longer interval, so merge the intervals of each job_id.
# Then it is the plain maximum overlap: sort start and end
# events and keep a running count.
```

```python


# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

import heapq


# ======================================================
# MAIN QUESTION
```

```python
# ======================================================
# ---- main question: helper functions ----
def merge_by_job(intervals):
    # intervals of the SAME job may overlap but they run on one server, so
    # merge them first: (job, start, end) -> disjoint blocks (start, end, job)
    by_job = {}
    for job, start, end in intervals:
        if job not in by_job:
            by_job[job] = []
        by_job[job].append((start, end))
    blocks = []
    for job in by_job:
        current = None
        for start, end in sorted(by_job[job]):
            if current is not None and start <= current[1]:
                if end > current[1]:
                    current[1] = end
            else:
                if current is not None:
                    blocks.append((current[0], current[1], job))
                current = [start, end]
        blocks.append((current[0], current[1], job))
    return blocks


# ---- main question: main function ----
# Daily jobs (job id, start, end), half-open [start, end). How many servers
# are needed if a job's own overlapping runs share one server?
# Idea: group the intervals by job and merge each job's overlapping or touching
# intervals; then sweep the start and end events of all the merged blocks: the
# answer is the largest number of blocks running at the same moment (at equal
# times an end comes first). `dedicated` jobs (follow-up 1) get a server of
# their own and are left out of the sweep.
# Complexity: Time O(n log n), Space O(n)
def servers_needed(intervals, dedicated=()):
    dedicated = set(dedicated)
    shared = []
    used_dedicated = set()
    for interval in intervals:
        if interval[0] in dedicated:
            used_dedicated.add(interval[0])
        else:
            shared.append(interval)
    events = []
    for start, end, job in merge_by_job(shared):
```

```python
        events.append((start, 1))
        events.append((end, -1))
    running = 0
    best = 0
    for time, delta in sorted(events):
```

```
        running += delta
        if running > best:
            best = running
    return best + len(used_dedicated)


# ---- main question: tests ----
JOBS = [("a", 0, 10), ("a", 5, 12), ("b", 3, 8), ("c", 10, 15)]


def test_main_question():
    assert servers_needed(JOBS) == 2                          # a is one block [0, 12); b overlaps it; c st
arts at 10
    assert servers_needed([("a", 0, 5), ("b", 5, 9)]) == 1    # touching half-open intervals share a server
    assert servers_needed([]) == 0


# =============================================================
# FOLLOW-UP 1: Some rare jobs run for several days without restart and get a dedicated server
# =============================================================
# Answer: take those jobs out of the daily intervals and add one server per
# such job: servers = (number of dedicated jobs) + (the largest overlap of the
# rest). That is the `dedicated` option.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert servers_needed(JOBS, dedicated=["b"]) == 3
    assert servers_needed(JOBS, dedicated=["a", "b"]) == 3


# =============================================================
# FOLLOW-UP 2: Jobs that cross midnight
# =============================================================
# ---- follow-up 2: function ----
# Answer: cut a job that wraps past midnight into two pieces: the part
# before midnight and the part after it (each day's schedule is then
# truncated at the end of the day, as the statement says).
# Complexity: Time O(1), Space O(1)
def split_over_midnight(start, end, day=24 * 60):
    if start < end:
        return [(start, end)]
    return [(start, day), (0, end)]
```

```


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert split_over_midnight(1380, 60) == [(1380, 1440), (0, 60)]
    assert split_over_midnight(60, 120) == [(60, 120)]


# =============================================================
# FOLLOW-UP 3: Return which server each job runs on
# =============================================================
# ---- follow-up 3: function ----
# Answer: sort the merged blocks by start and keep a min-heap of (end time,
# server): reuse the server that frees up first if it is free, otherwise open a
# new one. The number of servers opened equals the largest overlap.
# Complexity: Time O(n log n), Space O(n)
def assign_servers(intervals):
    heap = []
    plan = []
    opened = 0
    for start, end, job in sorted(merge_by_job(intervals)):
        if heap and heap[0][0] <= start:
            freed_at, server = heapq.heappop(heap)
```

```
        else:
            server = opened
            opened += 1
        plan.append((job, server))
        heapq.heappush(heap, (end, server))
    return plan, opened


# ---- follow-up 3: tests ----
def test_follow_up_3():
    plan, count = assign_servers(JOBS)
    assert count == 2 and len(plan) == 3


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(intervals: list[list[int]]) -> int:
```

Sample call

```
solution([])
```

Test cases

Input	Expected
`([],)`	`0`
`([[5, 100, 200], [5, 150, 250], [5, 240, 300]],)`	`1`

ALSO REPORTED · 1 more

Compute minimum servers for cyclic tasks | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-minimum-servers-for-cyclic-tasks

STUDY PATH: question 12 of 197 | topic 2 of 43: Sweep over events in time order | 6 of 6

Template fit: split meetings that cross midnight into two pieces first, then the same sweep | ND

MEDIUM | HackTheRounds | Phone Screen | 12 of 197

★ TOP 75: rank 59 in the Medium and Hard priority list

★ Meeting Rooms III Variant

STUDY: TYPE ALL TIME 58 30 DAYS 0 PRIORITY 46/288

ABOUT THIS QUESTION

Topic: Sweep over events in time order

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #253 Meeting Rooms II

LeetCode match: LC 253 (cyclic variant)

QUESTION

Given an array of meeting time intervals where each interval is [start, end], determine the minimum number of conference rooms required to hold all meetings without overlap. Variant: times are cyclic within a 24-hour period (meetings can wrap around midnight).

EXAMPLES

Example 1	
Input	`intervals = [[0, 30], [5, 10], [15, 20]]`
Output	`2`
Why	`Meeting [0,30] overlaps with [5,10] and [15,20], so we need 2 rooms.`

Example 2	
Input	`intervals = [[7, 10], [2, 4]]`
Output	`1`
Why	`No overlap, 1 room is sufficient.`

Example 3	
Input	`intervals = [[22, 2], [23, 1], [0, 3]]`
Output	`3`
Why	`All three meetings overlap around midnight.`

CONSTRAINTS

- 1 <= intervals.length <= 10^4
- 0 <= start < end <= 24 * 60 (minutes in a day)
- Meetings may wrap around (e.g., [23:00, 01:00])

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def rooms_needed(meetings):
    slots = [minutes_used(m) for m in meetings]
    for k in range(1, len(meetings) + 1):
        rooms = [set() for _ in range(k)]
        if assign(0, rooms, slots):
            return k
def assign(i, rooms, slots):
    if i == len(slots):
        return True
    for room in rooms:
        busy = slots[i]
        if not room & busy:  # no shared minute
            room |= busy
            if assign(i + 1, rooms, slots):
                return True
            room -= busy
    return False
```

TIME	SPACE
exponential	O(n)

WHY IT IS SLOW

Backtracking over room assignments is exponential. Splitting wrapped meetings and sweeping start and end events gives the maximum overlap in O(n log n); that equals the exact answer when meetings do not wrap.

OPTIMAL SOLUTION · Split wrapped meetings and sweep (maximum overlap)

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A wrapped meeting must stay in a single room.

Then it is circular-arc colouring, NP-hard in general. A common practical method: cut the circle at the minute with the fewest active meetings; the meetings crossing the cut use fixed rooms, and the rest are scheduled greedily with a heap. It can use more rooms than the maximum overlap, and the five-arc example shows why.

FOLLOW-UP 2 Return which room each meeting uses.

Sort by start and keep a min-heap of (end time, room id); reuse the earliest-ending room if it is free, otherwise open a new one. For wrapped meetings schedule them first so their rooms are fixed.

FOLLOW-UP 3 Days can repeat (a wrapped meeting recurs every day).

A recurring meeting occupies its slots every day, so the circle model above is exactly right, and the no-wrap shortcut of ordering by start time is not.

FOLLOW-UP 4 Return which room each meeting uses.

Sort by start and keep a heap of (end time, room): reuse the earliest-ending room if it is free, otherwise open a new one. For meetings that do not wrap this uses exactly the minimum number of rooms (verified against the sweep); wrapped meetings need the cut-the-circle method.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Meeting Rooms III Variant
#
# Given an array of meeting time intervals where each
# interval is [start, end], determine the minimum number
# of conference rooms required to hold all meetings
# without overlap. Variant: times are cyclic within a
# 24-hour period (meetings can wrap around midnight).
#
# FOLLOW-UPS (answered below, in this order):
# 1. A wrapped meeting must stay in a single room.
# 2. Return which room each meeting uses.
# 3. Days can repeat (a wrapped meeting recurs every day).
# 4. Return which room each meeting uses.
#
# INTERVIEW NOTES
# Split a wrapped meeting into two pieces (from start to
# the end of the day, and from midnight to its end) and
# run the normal sweep line: the largest number of
# meetings active at once is the number of rooms needed.
# That is exact when pieces may sit in different rooms; if
# a meeting must keep one room, the arcs form a circular-
# arc graph and the true minimum can exceed the maximum
# overlap (five overlapping arcs around a circle need 3
# rooms with overlap 2). Say which reading you assume.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq
import random

DAY = 24


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def pieces(meetings, day=DAY):
    # a meeting whose end is before its start wraps past midnight: split it
    # into the part before midnight and the part after it
```

```
    parts = []
    for start, end in meetings:
        if end < start:
            parts.append((start, day))
            parts.append((0, end))
        else:
```

```
            parts.append((start, end))
    return parts


# ---- main question: main function ----
# How many rooms are needed? Meetings are half-open [start, end), and a
# meeting may wrap past midnight (end < start).
# Idea: turn every meeting into one or two ordinary intervals, make a +1
# event at every start and a -1 event at every end, sort the events (at the
# same time an END comes first: a meeting ending at 5 and one starting at 5
# can share a room) and take the biggest running count.
# Complexity: Time O(n log n), Space O(n)
def min_rooms_sweep(meetings, day=DAY):
    events = []
    for start, end in pieces(meetings, day):
        events.append((start, 1))
        events.append((end, -1))
    events.sort()                                    # (-1) sorts before (+1) at equal times
    running = 0
    best = 0
    for time, step in events:
        running += step
        if running > best:
            best = running
    return best


# ---- main question: tests ----
def test_main_question():
    assert min_rooms_sweep([[0, 30], [5, 10], [15, 20]]) == 2
    assert min_rooms_sweep([[7, 10], [2, 4]]) == 1
    assert min_rooms_sweep([[22, 2], [23, 1], [0, 3]]) == 3
    assert min_rooms_sweep([]) == 0
    assert min_rooms_sweep([[1, 5], [5, 9]]) == 1            # one ends exactly when the next starts: same
room


# ========================================================
# FOLLOW-UP 1: A wrapped meeting must stay in a single room
# ========================================================
```

```
# ---- follow-up 1: helper functions ----
def hours(meeting, day=DAY):
    # the hours a meeting occupies on the 24-hour circle
    start, end = meeting
    length = (end - start) % day
    if length == 0:
        length = day
    occupied = set()
    for k in range(length):
        occupied.add((start + k) % day)
    return occupied


# ---- follow-up 1: function ----
# Answer: this is circular-arc colouring, NP-hard in general, so use the
# practical method: cut the circle at the hour where the FEWEST meetings are
# running; the meetings that cross the cut get their own rooms (blocked
# before their start and after the cut); place the rest greedily, each in the
# room that has been free the longest and still fits.
# Complexity: Time O(day^2 * n + n^2) (a simple scan for the quiet hour and for a room), Space O(n + day)
def assign_single_rooms(meetings, day=DAY):
    busy = []
    for hour in range(day):
        count = 0
```

```
        for meeting in meetings:
            if hour in hours(meeting, day):
                count += 1
        busy.append(count)
    cut = 0
    for hour in range(day):
        if busy[hour] < busy[cut]:
            cut = hour
    line = []                                          # rotate so the cut is time 0: intervals on a line
    for start, end in meetings:
        new_start = (start - cut) % day
        length = (end - start) % day
        if length == 0:
            length = day
        line.append((new_start, new_start + length))
    free_at = []                                       # per room: when it is free
    deadline = []                                      # per room: it must be empty again by then
    room_of = {}
    for i in range(len(line)):
        start, end = line[i]
        if end > day:                                  # crosses the cut: busy until end - day, and again f
rom start on
```

```
            room_of[i] = len(free_at)
            free_at.append(end - day)
            deadline.append(start)
    others = []
    for i in range(len(line)):
        if i not in room_of:
            others.append((line[i], i))
    others.sort()
    for (start, end), i in others:
        best_room = None
        for room in range(len(free_at)):
            if free_at[room] <= start and end <= deadline[room]:
                if best_room is None or free_at[room] > free_at[best_room]:
                    best_room = room                   # the room that has been free the longest
        if best_room is None:
            best_room = len(free_at)
            free_at.append(0)
            deadline.append(day)
        room_of[i] = best_room
        free_at[best_room] = end
    rooms = []
    for i in range(len(meetings)):
        rooms.append(room_of[i])
    return rooms


# ---- follow-up 1: tests ----
def valid_assignment(meetings, rooms, day=DAY):
    # no room may hold two meetings in the same hour
    used = set()
    for i in range(len(meetings)):
        for hour in hours(meetings[i], day):
            if (rooms[i], hour) in used:
                return False
            used.add((rooms[i], hour))
    return True


def test_follow_up_1():
    two = [(22, 2), (1, 5)]
    assert valid_assignment(two, assign_single_rooms(two))
    random.seed(22)
```

```
    for round_number in range(400):
        meetings = []
        for i in range(random.randint(0, 7)):
            start, end = random.sample(range(DAY), 2)
            meetings.append((start, end))
        assert valid_assignment(meetings, assign_single_rooms(meetings))


# =============================================================
# FOLLOW-UP 2: Return which room each meeting uses
# =============================================================


# =============================================================
# FOLLOW-UP 4: Return which room each meeting uses
# =============================================================
# ---- follow-up 2: function ----
# Answer: sort by start and keep a min-heap of (end time, room): reuse the
# room that frees up first if it is free, otherwise open a new room. (For
# wrapped meetings place them first so their rooms are fixed.)
# Complexity: Time O(n log n), Space O(n)
def assign_rooms(meetings):
    order = []
    for i in range(len(meetings)):
        order.append((meetings[i][0], i))
    order.sort()
    heap = []
    room_of = {}
    rooms_opened = 0
    for start, i in order:
        end = meetings[i][1]
        if heap and heap[0][0] <= start:
            freed_at, room = heapq.heappop(heap)
        else:
            room = rooms_opened
            rooms_opened += 1
        room_of[i] = room
        heapq.heappush(heap, (end, room))
    rooms = []
    for i in range(len(meetings)):
        rooms.append(room_of[i])
    return rooms


# ---- follow-up 2: tests ----
def test_follow_up_2():
    flat = [[0, 30], [5, 10], [15, 20]]
    rooms = assign_rooms(flat)
    assert rooms == [0, 1, 1]
    assert len(set(rooms)) == min_rooms_sweep(flat)
    assert valid_assignment(flat, rooms)


# =============================================================
# FOLLOW-UP 3: Days can repeat (a wrapped meeting recurs every day)
# =============================================================
# Answer: the circle repeats every day, so the sweep over one day (with
# wrapped meetings split at midnight) already counts the overlap on every day.


# =============================================================
# RUN EVERYTHING
# =============================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 3 OF 43 |

Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 4 questions | tap a title to jump to it

READ FIRST: the code template for these 4 questions (tap here) >

M 13. Expression Tree Single-Leaf Mutation — 1Point3Acres * 2026-09-27 *

M 14. Longest Consecutive Path in Binary Tree — 1Point3Acres * 2026-07-02 *

M 15. Implement zigzag level-order traversal — PracHub * 2025-08-01 *

M 16. Given the root of a binary tree, return the length of the diameter of the tree. — ND

CODE TEMPLATE | GROUP 3 OF 43 | 4 QUESTIONS, ONE PATTERN

Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

The one idea these questions share

Write dfs(node, depth). An empty node returns a base value, a real node calls dfs on both children, updates the best answer so far, and returns one number or flag to its parent. Only three lines change: the base value, the update, and what you return.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def solve(root):
    best = 0

    def dfs(node, depth=0):
        nonlocal best
        if node is None:
            return 0                          # CHANGE 1: what an empty tree reports
        left = dfs(node.left, depth + 1)
        right = dfs(node.right, depth + 1)
        best = max(best, left + right)        # CHANGE 2: update the answer at this node
        return 1 + max(left, right)           # CHANGE 3: what this node reports upward

    dfs(root)
    return best
```

2. Helper functions that usually get written next to it

```
from collections import deque


class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def levels(root):                             # BFS by level: height, right-side view, zigzag
    out, queue = [], deque([root] if root else [])
    while queue:
        out.append([n.val for n in queue])
        for _ in range(len(queue)):
            n = queue.popleft()
            queue.extend(c for c in (n.left, n.right) if c)
    return out


def preorder(root):                           # iterative: push right first, then left
    stack = [root] if root else []
    while stack:
        n = stack.pop()
        yield n.val
        stack.extend(c for c in (n.right, n.left) if c)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
13	Expression Tree Single-Leaf Mutation	post-order evaluation (children first, then apply the operator); to test one leaf flip, re-evaluate with that leaf changed

#	Question	What you change in the template
14	Longest Consecutive Path in Binary Tree	pass the parent's value and the run length down; extend the run when value == parent + 1, else restart at 1
15	Implement zigzag level-order traversal	collect values by depth; reverse every odd level (or fill each level from the other end)
16	Given the root of a binary tree, return the length of the diameter of the tree.	best = max(best, left + right); return 1 + max(left, right) (the diameter)

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 13 of 197 | topic 3 of 43: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 1 of 4

Template fit: post-order evaluation (children first, then apply the operator); to test one leaf flip, re-evaluate with that leaf changed | * asked 23 days ago * | frequency Low

MEDIUM | 1Point3Acres | onsite-coding | 13 of 197

Expression Tree Single-Leaf Mutation

ASKED 2026-09-27 | 23 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 263/288**

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-27 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/expression-tree-single-leaf-mutation

QUESTION

A boolean expression tree has leaves holding True or False and internal nodes AND, OR, XOR (two children) or NOT (one child). For each leaf in turn, imagine flipping only that leaf and report the value of the whole tree after the flip (every flip is independent of the others). Return the leaves and the list of answers, in leaf order.

Function signature

def flip_answers(root) -> (list, list)

(the leaves, the root value

after flipping each one)

EXAMPLES

Example 1	

Input	`t = Node('AND', Node(val=True), Node('OR', Node(val=False), Node(val=True)))`
	`evaluate(t)`
	`[leaf.val for leaf in flip_answers(t)[0]]`
	`flip_answers(t)[1]`

Output	`evaluate(t) -> true`
	`[leaf.val for leaf in flip_answers(t)[0]] -> [true, false, true]`
	`flip_answers(t)[1] -> [false, true, false]`

Why	`The tree is True AND (False OR True) = True. Flipping the first leaf gives False, flipping the False leaf changes nothing (the OR is still True), flipping the last leaf makes the OR False.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(root, leaves):
    answers = []
    for leaf in leaves:
        leaf.value = not leaf.value
        answers.append(evaluate(root))
        leaf.value = not leaf.value  # flip back
    return answers
```

BRUTE FORCE

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

L leaves times an O(n) evaluation: O(n^2). Caching every node's value and recomputing only the path to the root (stopping when a value does not change) costs O(depth) per leaf.

OPTIMAL SOLUTION · Cache values and parents, recompute only the path with early stop

TIME	SPACE
O(n + L * depth)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The node interface has no parent pointer.

Build the path while collecting leaves: carry the list of ancestors with each stack entry, and use that list for the walk up. It costs O(depth) memory per leaf; the parent dict does the same job with O(n) total.

FOLLOW-UP 2 Apply several flips at once, or flips that stay.

The same cached values work: toggle, walk up with early stop, and keep the changes. For many flips between queries, recompute each affected path in turn.

FOLLOW-UP 3 Which leaves change the answer at all?

Compute top-down 'sensitivity': a leaf matters if every ancestor's other input lets it through (AND with true, OR with false, any XOR or NOT). One pass gives all of them in O(n).

FOLLOW-UP 4 Which leaves can change the answer at all?

Propagate a 'matters' flag downward: through AND the other side must be true, through OR the other side must be false, through XOR and NOT it always passes. A leaf matters exactly when the flag reaches it (checked against flipping every leaf). Subtree values are cached in practice, giving O(n).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Expression Tree Single-Leaf Mutation
#
# A boolean expression tree has leaves holding True or
# False and internal nodes AND, OR, XOR (two children) or
# NOT (one child). For each leaf in turn, imagine flipping
# only that leaf and report the value of the whole tree
# after the flip (every flip is independent of the
# others). Return the leaves and the list of answers, in
# leaf order.
#
# Signature:
#   def flip_answers(root) -> (list, list)    # (the leaves, the root value after flipping each one)
#
```

```
# FOLLOW-UPS (answered below, in this order):
# 1. The node interface has no parent pointer.
# 2. Apply several flips at once, or flips that stay.
# 3. Which leaves change the answer at all?
# 4. Which leaves can change the answer at all?
#
# INTERVIEW NOTES
# Flipping one leaf can only change the nodes on its path
# to the root, so cache every node's value and recompute
# only that path. The path is found through parent links
# you build during the first traversal. Stop early when a
# recomputed node keeps its old value, because nothing
# above it can change.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class Node:
    def __init__(self, op=None, left=None, right=None, val=None):
        # op is "AND", "OR", "XOR", "NOT", or None for a leaf (which holds val)
        self.op = op
        self.val = val
        self.left = left
        self.right = right
```

```
def combine(op, a, b=None):
    # the value of one gate from its input values (NOT has only one input)
    if op == "AND":
        return a and b
    if op == "OR":
        return a or b
    if op == "XOR":
        return a != b
    return not a


def right_value(value, node):
    # the value of the right child, or None when there is none (NOT)
    if node.right is None:
        return None
    return value[node.right]


def evaluate(node):
    # plain recursive evaluation (used as the checker in the tests)
    if node.op is None:
        return node.val
    a = evaluate(node.left)
    b = None
    if node.right is not None:
        b = evaluate(node.right)
    return combine(node.op, a, b)


def build_cache(root):
```

```
    # one pass that stores, for every node, its parent and its value,
    # and a list of the leaves in left-to-right order
    parent = {root: None}
    value = {}
    leaves = []
    order = []
    stack = [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.right, node.left):
            if child is not None:
                parent[child] = node
```

```
                stack.append(child)
    for node in reversed(order):                # children come before their parents
        if node.op is None:
            value[node] = node.val
        else:
            value[node] = combine(node.op, value[node.left], right_value(value, node))
    for node in order:                          # `order` is root, left subtree, right subtree
        if node.op is None:
            leaves.append(node)
    return parent, value, leaves


# ---- main question: main function ----
# Idea: store every node's value and parent once. To flip one leaf, walk
# UP from it recomputing only the nodes whose value really changes, and
# stop as soon as a value stays the same. Read the root's value, then put
# every changed value back for the next leaf.
# Complexity: Time O(n + L * h) for L leaves and height h (each flip walks up only while a value changes),
Space O(n)
def flip_answers(root):
    parent, value, leaves = build_cache(root)
    answers = []
    for leaf in leaves:
        undo = []
        node = leaf
        new = not value[leaf]
        while node is not None and value[node] != new:
            undo.append((node, value[node]))
            value[node] = new
            node = parent[node]
            if node is not None:
                new = combine(node.op, value[node.left], right_value(value, node))
        answers.append(value[root])
        for node, old in reversed(undo):        # restore for the next leaf
            value[node] = old
    return leaves, answers


# ---- main question: tests ----
def example_expression():
    # (True AND True) XOR (NOT False)
    return Node("XOR", Node("AND", Node(val=True), Node(val=True)), Node("NOT", Node(val=False)))


def test_main_question():
    tree = example_expression()
```

```
    leaves, answers = flip_answers(tree)
    assert len(leaves) == 3
    for i in range(len(leaves)):                # every cached answer equals a full re-evaluation
        leaves[i].val = not leaves[i].val
```

```
        assert evaluate(tree) == answers[i]
        leaves[i].val = not leaves[i].val
    assert evaluate(tree) is False                # the tree is unchanged afterwards


# =====================================================
# FOLLOW-UP 1: The node interface has no parent pointer
# =====================================================
# Answer: build the parent map yourself in the first pass (build_cache
# does: parent[node]) instead of reading node.parent.


# =====================================================
# FOLLOW-UP 2: Apply several flips at once, or flips that stay
# =====================================================
# Answer: do not restore the values: flip a leaf, walk up and update, and
# leave the cache as it is for the next flip. For a batch, flip all the
# leaves first and recompute the affected nodes bottom-up once.


# =====================================================
# FOLLOW-UP 3: Which leaves change the answer at all?
# =====================================================


# =====================================================
# FOLLOW-UP 4: Which leaves can change the answer at all?
# =====================================================
# ---- follow-up 3: function ----
# Answer: pass a "matters" flag down from the root. Through AND the
# other side must be True, through OR the other side must be False,
# through XOR and NOT the flag always passes. The leaves that arrive
# with the flag still set are the ones that can change the answer. One
# pass, O(n).
# Complexity: Time O(n), Space O(n)
def sensitive_leaves(root):
    parent, value, leaves = build_cache(root)
    found = []
    stack = [(root, True)]
    while stack:
        node, matters = stack.pop()
```

```
        if node.op is None:
            if matters:
                found.append(node)
            continue
        left, right = node.left, node.right
        left_ok = matters
        right_ok = matters
        if node.op == "AND":
            left_ok = matters and value[right]
            right_ok = matters and value[left]
        elif node.op == "OR":
            left_ok = matters and not value[right]
            right_ok = matters and not value[left]
        stack.append((left, left_ok))
        if right is not None:
            stack.append((right, right_ok))
    return found


# ---- follow-up 3: tests ----
def test_follow_up_3():
    tree = example_expression()
    leaves, answers = flip_answers(tree)
```

```
    base = evaluate(tree)
    matters = sensitive_leaves(tree)
    for leaf in leaves:                          # compare with really flipping each leaf
        leaf.val = not leaf.val
        changes = evaluate(tree) != base
        leaf.val = not leaf.val
        assert changes == (leaf in matters)


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Practice tree evaluation for boolean expression trees.
Add parent pointers or store root paths during leaf collection to support path-only recomputation.
Drill AND / OR / XOR truth tables so updates are mechanical.

STUDY PATH: question 14 of 197 | topic 3 of 43: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 2 of 4

Template fit: pass the parent's value and the run length down; extend the run when value == parent + 1, else restart at 1 | * asked 110 days ago * | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 14 of 197

Longest Consecutive Path in Binary Tree

ASKED 2026-07-02 | 110 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 16.5** **30 DAYS 0** **PRIORITY 178/288**

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #549 Binary Tree Longest Consecutive Sequence II

LeetCode match: LC 549 Binary Tree Longest Consecutive Sequence II (related)

Frequency: Single report | Last asked: 2026-07-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/binary-tree-longest-consecutive-path

QUESTION

Given a binary tree, return the length of the longest path that goes from a parent down to a child where every value is exactly 1 more than the previous one (counted in nodes).

Function signature

def longestConsecutive(root: TreeNode) -> int

EXAMPLES

Example 1	
Input	`[1, null, 3, 2, 4, null, null, null, 5]`
Output	`3`
Why	`3 -> 4 -> 5 is the longest increasing-by-one downward path.`

Example 2	
Input	`[2, null, 3, 2, null, 1]`
Output	`2`
Why	`2 -> 3 is the longest; 3 -> 2 and 2 -> 1 go the wrong way.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def walk(node):
    best = 1
    for child in (node.left, node.right):
        if child and child.val == node.val + 1:
            best = max(best, 1 + walk(child))
    return best
answer = 0
for node in all_nodes(root):
    answer = max(answer, walk(node))
```

BRUTE FORCE

TIME	SPACE
O(n * h)	O(h)

WHY IT IS SLOW

A node deep in a chain is walked again from every ancestor, O(n * h). Passing the current run length down in one traversal makes it O(n).

OPTIMAL SOLUTION · Top-down DFS carrying the current run

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What does the any-direction version return for a single node, or for duplicate values?

A single node is a path of length 1 (0 edges), and an empty tree is 0. Equal values never extend a chain because the step must be exactly +1 or -1, so duplicates reset both counters to 1 at that node.

FOLLOW-UP 2 Can you avoid recursion?

Yes: do an iterative post-order or a BFS that stores (node, run) pairs for the top-down version. The bending version needs child results before the parent, so use an explicit stack with a post-order visit or process nodes in reverse BFS order using a dict from node to (inc, dec). It matters because a chain-shaped tree makes the recursive depth n.

FOLLOW-UP 3 Return the path itself, not only its length.

Carry the current run as a list (or its start node and length) in the traversal and copy it into the best answer when it becomes longer.

FOLLOW-UP 4 The path may go in any direction (LeetCode 549).

Return the pair (longest increasing chain down, longest decreasing chain down) from each node, and update the global best with inc + dec - 1 at the node. Verified against exhaustive path enumeration for small trees.

FOLLOW-UP 5 The values must increase by 1 or decrease by 1 along the path but not mix.

That is exactly the any-direction version: the increasing and decreasing parts meet at the bend; a path cannot zig-zag between increasing and decreasing steps.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Longest Consecutive Path in Binary Tree
#
# Given a binary tree, return the length of the longest
# path that goes from a parent down to a child where every
# value is exactly 1 more than the previous one (counted
# in nodes).
#
```

```
# Signature:
#   def longestConsecutive(root: TreeNode) -> int
#
# Examples:
# [1, null, 3, 2, 4, null, null, null, 5]  ->  3
# [2, null, 3, 2, null, 1]  ->  2
#
# FOLLOW-UPS (answered below, in this order):
# 1. What does the any-direction version return for a
# single node, or for duplicate values?
# 2. Can you avoid recursion?
# 3. Return the path itself, not only its length.
# 4. The path may go in any direction (LeetCode 549).
# 5. The values must increase by 1 or decrease by 1 along
# the path but not mix.
#
# INTERVIEW NOTES
# Each node needs to know the longest run that ends at its
# parent, so pass the current length down and reset it to
# 1 when the value is not parent + 1. For the bending
# version each node must report two lengths upward
# (increasing and decreasing chains going down), and the
# best path through a node combines one of each.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


# ---- main question: main function ----
# Idea: go down the tree carrying the length of the run that ends at
# the current node. A child that is exactly parent + 1 continues the
# run; any other child starts a new run of 1. Keep the best run seen.
# The length is counted in NODES (say that choice out loud).
# This version uses a stack, so it is also the answer to follow-up 2
# (no recursion).
# Complexity: Time O(n), Space O(h) for the height h of the tree
def longestConsecutive(root):
    if root is None:
        return 0
    best = 0
    stack = [(root, 1)]
    while stack:
        node, run = stack.pop()
        if run > best:
            best = run
        for child in (node.left, node.right):
            if child is None:
                continue
            if child.val == node.val + 1:
                stack.append((child, run + 1))
```

```
            else:
                stack.append((child, 1))
    return best


# ---- main question: tests ----
def example_tree():
    #   1
    #    \
    #     3
    #    / \
    #   2   4
    #        \
    #         5
    return TreeNode(1, None, TreeNode(3, TreeNode(2), TreeNode(4, None, TreeNode(5))))


def test_main_question():
    assert longestConsecutive(example_tree()) == 3          # 3 -> 4 -> 5
```

```
    assert longestConsecutive(None) == 0
    assert longestConsecutive(TreeNode(7)) == 1
    assert longestConsecutive(TreeNode(2, TreeNode(2), TreeNode(2))) == 1   # equal values do not extend a
run


# ============================================================
# FOLLOW-UP 1: What does the any-direction version return for a single node, or for duplicate values?
# ============================================================
# Answer: a single node is a path of 1; equal values never extend a
# path (the next value must be exactly +1 or -1). See the tests of follow-up 4.


# ============================================================
# FOLLOW-UP 2: Can you avoid recursion?
# ============================================================
# Answer: yes, longestConsecutive above already uses an explicit stack.


# ============================================================
# FOLLOW-UP 3: Return the path itself, not only its length
# ============================================================
# ---- follow-up 3: function ----
# Answer: carry the run itself (a list of values) instead of its length
# and remember the longest list.
# Complexity: Time O(n * h) (each step copies the run), Space O(h * h)
def longest_consecutive_path(root):
    best_path = []
    if root is None:
        return best_path
    stack = [(root, [root.val])]
    while stack:
        node, run = stack.pop()
        if len(run) > len(best_path):
            best_path = run
        for child in (node.left, node.right):
            if child is None:
                continue
            if child.val == node.val + 1:
                stack.append((child, run + [child.val]))
            else:
                stack.append((child, [child.val]))
    return best_path
```

```
# ---- follow-up 3: tests ----
```

```
def test_follow_up_3():
    assert longest_consecutive_path(example_tree()) == [3, 4, 5]
    assert longest_consecutive_path(None) == []


# =======================================================
# FOLLOW-UP 4: The path may go in any direction (LeetCode 549)
# =======================================================


# =======================================================
# FOLLOW-UP 5: The values must increase by 1 or decrease by 1 along the path but not mix
# =======================================================
# ---- follow-up 4: function ----
# Answer: for every node keep two numbers: the longest chain going down
# that INCREASES by 1 each step (up_chain) and the longest that DECREASES
# by 1 (down_chain). The best path that bends at this node joins the two
# chains: up_chain + down_chain - 1 (the node is counted once). Children
# are handled before their parents.
# Complexity: Time O(n), Space O(n)
def longest_any_direction(root):
    if root is None:
        return 0
    order = []
    stack = [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                stack.append(child)
    up_chain = {}
    down_chain = {}
    best = 1
    for node in reversed(order):                  # children come before their parents
        up = 1
        down = 1
        for child in (node.left, node.right):
            if child is None:
                continue
            if child.val == node.val + 1 and up_chain[child] + 1 > up:
                up = up_chain[child] + 1
            if child.val == node.val - 1 and down_chain[child] + 1 > down:
                down = down_chain[child] + 1
        up_chain[node] = up
```

```
        down_chain[node] = down
        if up + down - 1 > best:
            best = up + down - 1
    return best


# ---- follow-up 4: tests ----
def test_follow_up_4():
    assert longest_any_direction(TreeNode(2, TreeNode(1), TreeNode(3))) == 3     # 1 -> 2 -> 3 bends at 2
    assert longest_any_direction(TreeNode(5)) == 1                               # a single node
    assert longest_any_direction(TreeNode(2, TreeNode(2), TreeNode(2))) == 1      # duplicates do not count
    assert longest_any_direction(example_tree()) == 4                           # 2 -> 3 -> 4 -> 5 bends at
 3


# =======================================================
# RUN EVERYTHING
```

```
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write the parent-to-child traversal in under 10 minutes with a global best.
Practice the bidirectional tree follow-up where each node returns two lengths upward.
Dry-run single-node, duplicate-value, and split-branch cases before discussing complexity.

STUDY PATH: question 15 of 197 | topic 3 of 43: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 3 of 4

Template fit: collect values by depth; reverse every odd level (or fill each level from the other end) | * asked 445 days ago *

MEDIUM | IN APP LC 103 | PracHub | Technical Screen | 15 of 197

Implement zigzag level-order traversal

POSTED 2025-08-01 | 445 DAYS AGO | OLDER

STUDY: TALK ALL TIME 38.7 30 DAYS 0 PRIORITY 110/288

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #103 Binary Tree Zigzag Level Order Traversal

LeetCode match: LC 103 Binary Tree Zigzag Level Order Traversal

Posted on PracHub: 2025-08-01 (updated 2026-06-25)

Original: https://prachub.com/coding-questions/implement-zigzag-level-order-traversal

QUESTION

Given the root of a binary tree, return its values level by level, reading the first level from left to right, the next from right to left, and so on, alternating. The result is a list of lists, one per level. Aim for O(n) time.

Function signature

def zigzag_level_order(root: TreeNode) -> list

EXAMPLES

Example 1	
Input	`[3, 9, 20, null, null, 15, 7]`
Output	`[[3], [20, 9], [15, 7]]`
Why	`Level 0 is [3], level 1 is read right to left [20, 9], level 2 left to right [15, 7].`

Example 2	
Input	`[1]`
Output	`[[1]]`
Why	`A single node.`

Example 3	
Input	`[]`
Output	`[]`
Why	`An empty tree has no levels.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def zigzag(root):
    levels = level_order(root)
    for i in range(1, len(levels), 2):
        levels[i].reverse()
    return levels
```

BRUTE FORCE

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Same O(n) time, plus an extra pass copying the reversed levels. Filling each level from the left or right end of a deque as it is built needs no reversal.

OPTIMAL SOLUTION · BFS filling each level from the left or the right

TIME	SPACE
O(n)	O(width)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Bottom-up level order (leaf level first).

Run the normal traversal and reverse the list of levels at the end.

FOLLOW-UP 2 Do it with DFS.

Pass the depth; append the value to levels[depth], at the end when the depth is even and at the front (insert at 0 or a deque) when odd. Recursion depth is the tree height.

FOLLOW-UP 3 N-ary tree.

Replace the two children by the list of children; the direction flag works the same way.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement zigzag level-order traversal
#
# Given the root of a binary tree, return its values level
# by level, reading the first level from left to right,
# the next from right to left, and so on, alternating. The
# result is a list of lists, one per level. Aim for O(n)
# time.
#
# Signature:
#   def zigzag_level_order(root: TreeNode) -> list
#
# Examples:
# [3, 9, 20, null, null, 15, 7]  ->  [[3], [20, 9], [15,
#     7]]
# [1]  ->  [[1]]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Bottom-up level order (leaf level first).
# 2. Do it with DFS.
# 3. N-ary tree.
#
# INTERVIEW NOTES
# It is a plain level-order traversal where every second
```

```
# level is read in the other direction. Either collect
# each level and reverse the odd ones, or fill each level
# from the correct end of a deque so no reversal is
# needed.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


# ---- main question: main function ----
# Idea: go level by level with a queue. Read the first level left to
# right, the next one right to left, and so on: keep a flag and reverse
# the level list whenever the flag says "right to left".
# Complexity: Time O(n), Space O(w) for the widest level (O(n) with the answer)
def zigzag_level_order(root):
    result = []
    queue = deque()
    if root:
        queue.append(root)
    left_to_right = True
    while queue:
        level = []
        for i in range(len(queue)):              # exactly the nodes of this level
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        if not left_to_right:
            level.reverse()
        result.append(level)
        left_to_right = not left_to_right
    return result


# ---- main question: tests ----
def example_tree():
    return TreeNode(3, TreeNode(9), TreeNode(20, TreeNode(15), TreeNode(7)))


def test_main_question():
    assert zigzag_level_order(example_tree()) == [[3], [20, 9], [15, 7]]
    assert zigzag_level_order(TreeNode(1)) == [[1]]
    assert zigzag_level_order(None) == []
```

```
# ========================================================
# FOLLOW-UP 1: Bottom-up level order (leaf level first)
# ========================================================
```

```
# ---- follow-up 1: function ----
# Answer: do the normal level-order traversal and reverse the list of
# levels at the end.
# Complexity: Time O(n), Space O(n)
def level_order_bottom_up(root):
    levels = []
    queue = deque()
    if root:
        queue.append(root)
    while queue:
        level = []
        for i in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        levels.append(level)
    levels.reverse()
    return levels


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert level_order_bottom_up(example_tree()) == [[15, 7], [9, 20], [3]]


# ========================================================
# FOLLOW-UP 2: Do it with DFS
# ========================================================
# ---- follow-up 2: function ----
# Answer: pass the depth down and add each value to levels[depth]; at the
# end reverse the odd levels. The recursion depth is the tree height.
# Complexity: Time O(n), Space O(n)
def zigzag_dfs(root):
    levels = []

    def visit(node, depth):
        if node is None:
            return
        if depth == len(levels):
            levels.append([])
        levels[depth].append(node.val)
        visit(node.left, depth + 1)
```

```
        visit(node.right, depth + 1)

    visit(root, 0)
    for depth in range(1, len(levels), 2):        # the odd levels are read right to left
        levels[depth].reverse()
    return levels


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert zigzag_dfs(example_tree()) == [[3], [20, 9], [15, 7]]
    assert zigzag_dfs(None) == []


# ========================================================
```

```
# FOLLOW-UP 3: N-ary tree
# ============================================================
# ---- follow-up 3: helper functions ----
class NNode:
    def __init__(self, val, children=()):
        self.val = val
        self.children = list(children)


# ---- follow-up 3: function ----
# Answer: loop over a list of children instead of left and right; the
# direction flag works in the same way.
# Complexity: Time O(n), Space O(n)
def zigzag_nary(root):
    result = []
    queue = deque()
    if root:
        queue.append(root)
    left_to_right = True
    while queue:
        level = []
        for i in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            for child in node.children:
                queue.append(child)
        if not left_to_right:
            level.reverse()
        result.append(level)
        left_to_right = not left_to_right
```

```
    return result


# ---- follow-up 3: tests ----
def test_follow_up_3():
    root = NNode(1, [NNode(2, [NNode(5)]), NNode(3), NNode(4)])
    assert zigzag_nary(root) == [[1], [4, 3, 2], [5]]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def zigzagLevelOrder(values: list[int | None]) -> list[list[int]]:
```

Sample call

```
zigzagLevelOrder([3, 9, 20, None, None, 15, 7])
```

Test cases

Input	Expected
`([3, 9, 20, None, None, 15, 7],)`	`[[3], [20, 9], [15, 7]]`
`([1],)`	`[[1]]`

STUDY PATH: question 16 of 197 | topic 3 of 43: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 4 of 4

Template fit: best = max(best, left + right); return 1 + max(left, right) (the diameter) | ND

MEDIUM | IN APP LC 543 | Exponent | 16 of 197

Given the root of a binary tree, return the length of the diameter of the tree.

ASKED (ANY CO.) ~3 MONTHS AGO | LAST 6 MONTHS

STUDY: TYPE **ALL TIME 44.4** **30 DAYS 0** **PRIORITY 93/288**

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

Optimal answer from: your app's interview approach (LeetCode #543)

LeetCode match: LC 543 Diameter of Binary Tree

Last asked (any company): 3 months ago, about 2026-07 (Exponent shows only a rough age) | +18 interviews

Original: https://www.aced.io/questions/2705/binary-tree-diameter-length

QUESTION

Given the root of a binary tree, create a function diameterOfTree that determines and returns the length of the diameter of the tree, where the diameter of a tree is defined as the number of nodes along the longest path between any two nodes in the tree. For example, given the following binary tree: // tree 1 / \ 2 3 / \ 4 5 The diameter is 4, which is the length of the path [4, 2, 1, 3] or [5, 2, 1, 3].Watch Ali, SDE @ Amazon, answer the question: "Given the root of a binary tree, return the length of the diameter of the tree."No answers contributed by the community yet.Interview experiences6 shared‹›Uber · 3 months ago"interview process is very lengthy took around 5 weeks, and expectations is very high in each round, for LLD working code is required, also dont mess your HM round since i felt like it contained more weightage"Software Engineer · Mid Level / L4Meta · 4 months ago"The interview went smoothly overall. 1 round of phone screen followed by the full loop of 4 rounds packed in a day. 2 coding 1 behavior 1 system design."Software Engineer · Senior / L5Google · 9 months ago"I interviewed for an AI Engineer internship, but the process was mostly DSA-heavy. I had an online coding round first, then a technical round with binary tree and DP questions, then another harder DSA round with DP and linked list questions, and finally an HR round. The online round covered arrays and strings, and the later rounds got progressively more difficult. The only non-DSA part was the last

EXAMPLES

Example 1	
Input	`[1, 2, 3, 4, 5]`
Output	`4`
Why	`The longest path 4 - 2 - 1 - 3 has 4 nodes.`

Example 2	
Input	`[1]`
Output	`1`
Why	`A single node.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def diameter(root):
    best = 0
    for node in all_nodes(root):
        left = height(node.left)  # fresh walks
        right = height(node.right)
        best = max(best, left + right + 1)
    return best
```

TIME	SPACE
O(n^2) worst case	O(h)

WHY IT IS SLOW

Heights of the same subtrees are recomputed for every ancestor, O(n^2) on a skewed tree. One post-order pass returns heights and updates the best on the way, O(n).

OPTIMAL SOLUTION · One post-order pass returning heights and updating the best

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 An n-ary tree.

Take the two largest child heights at each node (not just left and right): best = max(best, top1 + top2). Sorting the children's heights costs O(d log d) per node, or track the top two in O(d).

FOLLOW-UP 2 Edges have weights, or you want the path itself.

Add the edge weight to each child's height before comparing; for the path, also store which child gave each of the two best heights and rebuild from the node that produced the best sum.

FOLLOW-UP 3 Count the diameter in nodes instead of edges.

Add one to the answer (a path of k edges has k + 1 nodes). State which convention the interviewer wants before you code.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given the root of a binary tree, return
# the length of the diameter of the tree.
#
# Given the root of a binary tree, create a function
# diameterOfTree that determines and returns the length of
# the diameter of the tree, where the diameter of a tree
# is defined as the number of nodes along the longest path
# between any two nodes in the tree.
#
# FOLLOW-UPS (answered below, in this order):
```

```
# 1. An n-ary tree.
# 2. Edges have weights, or you want the path itself.
# 3. Count the diameter in nodes instead of edges.
#
# INTERVIEW NOTES
# The longest path in a tree has a highest node, and the
# length of the best path through a node is the height of
# its left subtree plus the height of its right subtree.
# So compute heights bottom-up and, at every node, check
# left height + right height against a running best. One
# traversal does both jobs.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


# ---- main question: main function ----
# Idea: one post-order pass. depth(node) returns the height of the
# node's subtree (in nodes) and, on the way, notes the longest path that
# passes THROUGH this node: left height + right height + 1 nodes. Every
# node is visited once. The diameter is counted in NODES, as the
```

```
# question says (follow-up 3 shows edges).
# Complexity: Time O(n), Space O(h)
def diameter_nodes(root):
    best = 0

    def depth(node):
        nonlocal best
        if node is None:
            return 0
        left = depth(node.left)
        right = depth(node.right)
        if left + right + 1 > best:
            best = left + right + 1
        if left > right:
            return left + 1
        return right + 1

    depth(root)
    return best


# ---- main question: tests ----
def test_main_question():
    root = TreeNode(1, TreeNode(2, TreeNode(4), TreeNode(5)), TreeNode(3))
    assert diameter_nodes(root) == 4              # 4 -> 2 -> 1 -> 3
    assert diameter_nodes(None) == 0
    assert diameter_nodes(TreeNode(1)) == 1
```

```
# ============================================================
# FOLLOW-UP 1: An n-ary tree
# ============================================================
# ---- follow-up 1: helper functions ----
class NNode:
    def __init__(self, val, children=()):
        self.val = val
        self.children = list(children)


def two_largest(numbers):
    # the biggest and the second biggest number (0 when there are fewer)
    first = 0
    second = 0
    for x in numbers:
        if x > first:
            second = first
            first = x
        elif x > second:
            second = x
    return first, second


# ---- follow-up 1: function ----
# Answer: at every node add the TWO tallest child heights (not just left
# and right). Track the top two in one pass over the children: O(d) per
# node (sorting them would cost O(d log d)).
# Complexity: Time O(n), Space O(h)
def diameter_nary(root):
    best = 0

    def depth(node):
        nonlocal best
        heights = []
        for child in node.children:
            heights.append(depth(child))
        first, second = two_largest(heights)
        if first + second + 1 > best:
            best = first + second + 1
        return first + 1

    if root is None:
        return 0
    depth(root)
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    root = NNode(1, [NNode(2, [NNode(5), NNode(6)]), NNode(3), NNode(4)])
    assert diameter_nary(root) == 4              # 5 -> 2 -> 1 -> 3 has 4 nodes
    assert diameter_nary(NNode(1)) == 1
    assert diameter_nary(None) == 0


# ============================================================
# FOLLOW-UP 2: Edges have weights, or you want the path itself
# ============================================================
# ---- follow-up 2: function ----
# Answer: add the edge weight to each child's height before comparing.
# (For the path itself also store which child gave each of the two best
# heights and rebuild it from the node that produced the best sum.)
# Here children are pairs (weight, node) and the answer is the largest
```

```
# total weight of a path.
# Complexity: Time O(n), Space O(h)
def diameter_weighted(root):
    best = 0

    def down(node):
        nonlocal best
        lengths = []
        for weight, child in node.children:
            lengths.append(weight + down(child))
        first, second = two_largest(lengths)
        if first + second > best:
            best = first + second
        return first

    if root is None:
        return 0
    down(root)
    return best


# ---- follow-up 2: tests ----
def test_follow_up_2():
    root = NNode(1, [(5, NNode(2, [(1, NNode(4))])), (2, NNode(3))])
    assert diameter_weighted(root) == 8           # 4 -1- 2 -5- 1 -2- 3  = 1 + 5 + 2


# ============================================================
# FOLLOW-UP 3: Count the diameter in nodes instead of edges
# ============================================================
# ---- follow-up 3: function ----
# Answer: a path of k edges has k + 1 nodes, so the two numbers differ
# by one. State which convention the interviewer wants before you code.
# Complexity: Time O(n), Space O(h)
def diameter_edges(root):
    if root is None:
        return 0
    return diameter_nodes(root) - 1


# ---- follow-up 3: tests ----
def test_follow_up_3():
    root = TreeNode(1, TreeNode(2, TreeNode(4), TreeNode(5)), TreeNode(3))
    assert diameter_edges(root) == 3
    assert diameter_edges(None) == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given the root of a binary tree, create a function diameterOfTree that determines and returns the length of the diameter of the tree, where the diameter of a tree is defined as the number of nodes along the longest path between any two nodes in the tree. For example, given the following binary tree:

```
// tree
       1
      / \
     2   3
    / \
   4   5
```

The diameter is 4, which is the length of the path [4, 2, 1, 3] or [5, 2, 1, 3].Watch Ali, SDE @ Amazon, answer the question: "Given the root of a binary tree, return the length of the diameter of the tree."No answers contributed by the community yet.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 4 OF 43 |

A heap of what is available: pop the best, put back the changed one

8 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

8
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

A heap of what is available: pop the best, put back the changed one | 8 questions | tap a title to jump to it

READ FIRST: the code template for these 8 questions (tap here) >

M	17. Streaming Median (LC 295) with Follow-ups	1Point3Acres * 2026-08-03 *
M	18. Compute minimum number of rooms needed	PracHub * 2026-02-12 *
M	19. Implement a Reusable Locker Allocation System	PracHub * 2026-08-14 *
M	20. Simulate room assignments for scheduled meetings	PracHub * 2026-01-06 *
M	21. Minimum Makespan on Identical CPUs	ColorStack * 2025-09-23 *
M	22. Design server allocation for multi-type nodes	PracHub * 2025-09-06 *
M	23. Trip Expense Settlement	ND
M	24. Playlist Shuffle No Adjacent Artists	ND

CODE TEMPLATE | GROUP 4 OF 43 | 8 QUESTIONS, ONE PATTERN

A heap of what is available: pop the best, put back the changed one

The one idea these questions share

The heap holds the things you can choose from right now: machines and when they are free, rooms, lockers, the biggest remaining count, the two smallest weights. Each step pops the best choice, uses it, and pushes it back changed (or a combined item).

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
import heapq


def run_jobs(jobs, workers):
    free = [0] * workers                        # CHANGE 1: what is available (times, ids, counts)
    heapq.heapify(free)
    finish = 0
    for ready, duration in sorted(jobs):
        start = max(ready, heapq.heappop(free))         # CHANGE 2: take the best available
        heapq.heappush(free, start + duration)          # CHANGE 3: put it back, changed
        finish = max(finish, start + duration)
    return finish
```

2. Helper functions that usually get written next to it

```
def pop_two_push_sum(weights):              # the Huffman step, also the generic "combine two best"
    heap = list(weights)
    heapq.heapify(heap)
    total = 0
    while len(heap) > 1:
        a, b = heapq.heappop(heap), heapq.heappop(heap)
        heapq.heappush(heap, a + b)
        total += a + b
    return total


class MaxHeap:                              # heapq only has a min-heap: negate on the way in and out
    def __init__(self):
        self.h = []

    def push(self, x):
        heapq.heappush(self.h, -x)

    def pop(self):
        return -heapq.heappop(self.h)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
17	Streaming Median (LC 295) with Follow-ups	two heaps: a max-heap for the lower half and a min-heap for the upper half; keep them balanced
18	Compute minimum number of rooms needed	heap of end times; a meeting reuses the room that frees first if it ended, otherwise push a new room
19	Implement a Reusable Locker Allocation System	heap of freed locker numbers; take the smallest freed one before handing out a new number
20	Simulate room assignments for scheduled meetings	a heap of free room numbers and a heap of (end time, room); a delayed meeting takes the earliest-ending room and shifts later

Contents Start the questions >

Master contents (all topics)

#	Question	What you change in the template
21	Minimum Makespan on Identical CPUs	the template as written: heap of the times the CPUs become free
22	Design server allocation for multi-type nodes	a heap per type of (flexibility, server); skip servers already allocated (lazy deletion)
23	Trip Expense Settlement	a heap of creditors and a heap of debtors; the biggest of each settles min(amount) and the remainder goes back
24	Playlist Shuffle No Adjacent Artists	pop the artist with the most songs left, hold it for one turn, then push it back with a smaller count

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 17 of 197 | topic 4 of 43: A heap of what is available: pop the best, put back the changed one | 1 of 8

Template fit: two heaps: a max-heap for the lower half and a min-heap for the upper half; keep them balanced | * asked 78 days ago * | frequency Single report

MEDIUM | IN APP LC 295 | DESIGN-STYLE (skippable) | 1Point3Acres | tech-screen | 17 of 197

Streaming Median (LC 295) with Follow-ups

ASKED 2026-08-03 | 78 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 50.4 | 30 DAYS 0 | PRIORITY 79/288

ABOUT THIS QUESTION

Topic: A heap of what is available: pop the best, put back the changed one

Optimal answer from: your app's interview approach (LeetCode #295)

LeetCode match: LC 295 Find Median from Data Stream

Frequency: Single report | Last asked: 2026-08-03 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/streaming-median-with-follow-ups

QUESTION

Design a structure that receives numbers one at a time and can report the median of everything received so far at any moment. addNum(x) adds a number; findMedian() returns the middle value, or the mean of the two middle values when the count is even.

Function signature

```
class MedianFinder:
    def addNum(self, x: float) -> None
    def findMedian(self) -> float
```

EXAMPLES

Example 1

Input	`m = MedianFinder()`
	`m.addNum(5)`
	`m.addNum(2)`
	`m.findMedian()`
	`m.addNum(8)`
	`m.findMedian()`
	`m.addNum(1)`
	`m.findMedian()`

Output	`m.findMedian() -> 3.5`
	`m.findMedian() -> 5`
	`m.findMedian() -> 3.5`

Why	`Sorted so far: [2, 5] -> 3.5; [2, 5, 8] -> 5; [1, 2, 5, 8] -> 3.5.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
nums = []  # always kept sorted
def add(x):
    insort(nums, x)
def median():
    n = len(nums)
    if n % 2:
        return nums[n // 2]
    return (nums[n // 2 - 1] + nums[n // 2]) / 2
```

TIME	SPACE
O(n) per insert (list shift), O(1) median	**O(n)**

WHY IT IS SLOW

Each insert shifts up to n elements, O(n) per add (fast in practice, but linear). Two heaps keep the median available with O(log n) per add and O(1) per query.

OPTIMAL SOLUTION · Two heaps

TIME	SPACE
O(log n) add, O(1) median	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Almost every value is in a small range (say 0 to 100). Can you do better than two heaps?

Yes: a counting array gives O(1) inserts, and the median needs one pass over at most V counts, so O(V) per query with V fixed. Keep a moving pointer to the median bucket to make queries amortized O(1).

FOLLOW-UP 2 Now support an arbitrary k-th smallest, not only the median.

Two heaps only work for one fixed rank. Use an order-statistics structure: a Fenwick tree over (compressed) values, or a balanced BST augmented with subtree sizes. Both give O(log n) add and O(log n) k-th, and the Fenwick tree is far shorter to write in an interview.

FOLLOW-UP 3 What if the data stream is too large to store?

Exact medians need memory proportional to the data, so use a quantile sketch (t-digest, GK, KLL). They answer any quantile within a chosen error bound in O(1/epsilon) memory.

FOLLOW-UP 4 The median of only the last K values.

Keep the window in a deque (to know what expires) and a sorted list (to read the middle); insert and remove with bisect. O(K) per update because of list shifting; two heaps with lazy deletion give O(log K).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Streaming Median (LC 295) with Follow-ups
#
# Design a structure that receives numbers one at a time
# and can report the median of everything received so far
# at any moment. addNum(x) adds a number; findMedian()
# returns the middle value, or the mean of the two middle
# values when the count is even.
#
# Signature:
#   class MedianFinder:
#       def addNum(self, x: float) -> None
#       def findMedian(self) -> float
#
# FOLLOW-UPS (answered below, in this order):
# 1. Almost every value is in a small range (say 0 to
# 100). Can you do better than two heaps?
# 2. Now support an arbitrary k-th smallest, not only the
# median.
# 3. What if the data stream is too large to store?
# 4. The median of only the last K values.
#
# INTERVIEW NOTES
# Keep the data split at the median: a max-heap for the
# smaller half and a min-heap for the larger half, with
# sizes equal or differing by one. Then the median is read
# from the tops. Insert by pushing through one heap into
# the other to keep every smaller value on the left, and
# rebalance the sizes.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main class ----
# Idea: keep the numbers in two heaps. The LOWER half is in a max-heap
# (store the negatives, since heapq is a min-heap) and the UPPER half in a
```

```
# min-heap. Their sizes differ by at most one (the lower half may have one
# more) and every number in the lower half is at most every number in the
# upper half. The median is the top of the lower heap, or the average of the
# two tops when the sizes are equal. addNum is O(log n), findMedian O(1).
# Complexity: Time addNum O(log n), findMedian O(1), Space O(n)
class MedianFinder:
    def __init__(self):
        self.low = []                               # max-heap (negated numbers)
        self.high = []                              # min-heap

    def addNum(self, x):
        heapq.heappush(self.low, -x)                # go through the lower half so the halves stay ordered
        biggest_of_low = -heapq.heappop(self.low)
        heapq.heappush(self.high, biggest_of_low)
        if len(self.high) > len(self.low):          # keep the lower half the same size or one bigger
            smallest_of_high = heapq.heappop(self.high)
            heapq.heappush(self.low, -smallest_of_high)

    def findMedian(self):
        if len(self.low) > len(self.high):
```

```
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2


# ---- main question: tests ----
def test_main_question():
    m = MedianFinder()
    for x in [5, 2, 8]:
        m.addNum(x)
    assert m.findMedian() == 5
    m.addNum(1)
    assert m.findMedian() == 3.5


# ========================================================
# FOLLOW-UP 1: Almost every value is in a small range (say 0 to 100). Can you do better than two heaps?
# ========================================================
# ---- follow-up 1: function ----
# Answer: yes. A counting array (one counter per possible value) gives O(1)
# inserts, and finding the median is one pass over at most V counters:
# O(V) per query for a fixed V. A moving pointer to the median bucket would
# make queries amortised O(1).
# Complexity: Time addNum O(1), findMedian O(V) for values up to V, Space O(V)
class CountingMedian:
    def __init__(self, max_value):
        self.counts = [0] * (max_value + 1)
        self.n = 0

    def addNum(self, x):
        self.counts[x] += 1
        self.n += 1

    def kth(self, k):
        # the k-th smallest number (1 = the smallest): one pass over the counters
        seen = 0
        for value in range(len(self.counts)):
            seen += self.counts[value]
            if seen >= k:
                return value

    def findMedian(self):
        lower = self.kth((self.n + 1) // 2)
        upper = self.kth(self.n // 2 + 1)
        return (lower + upper) / 2


# ---- follow-up 1: tests ----
def test_follow_up_1():
    counting = CountingMedian(100)
    for x in [5, 2, 8, 1]:
        counting.addNum(x)
    assert counting.findMedian() == 3.5


# ========================================================
# FOLLOW-UP 2: Now support an arbitrary k-th smallest, not only the median
# ========================================================
# ---- follow-up 2: function ----
# Answer: two heaps only work for ONE fixed rank. Use an order-statistics
# structure: a Fenwick tree over the (compressed) values, where tree
# nodes hold counts and kth() walks down the tree, or a balanced search tree
# whose nodes know the size of their subtree. Both give any k-th smallest in
# O(log V).
# Complexity: Time addNum O(log V), kth O(log V), Space O(V)
```

```
class Fenwick:
    def __init__(self, size):
        self.size = size
        self.tree = [0] * (size + 1)

    def addNum(self, value):
```

```
        i = value + 1
        while i <= self.size:
            self.tree[i] += 1
            i += i & -i                             # i & -i is the lowest set bit of i

    def kth(self, k):
        # walk down the tree to the k-th smallest value (k starts at 1)
        position = 0
        step = 1
        while step * 2 <= self.size:
            step *= 2                               # the biggest power of two that fits
        while step > 0:
            following = position + step
            if following <= self.size and self.tree[following] < k:
                position = following
                k -= self.tree[following]
            step = step // 2
        return position


# ---- follow-up 2: tests ----
def test_follow_up_2():
    fenwick = Fenwick(100)
    for x in [5, 2, 8, 1]:
        fenwick.addNum(x)
    ranks = []
    for k in (1, 2, 3, 4):
        ranks.append(fenwick.kth(k))
    assert ranks == [1, 2, 5, 8]


# ========================================================
# FOLLOW-UP 3: What if the data stream is too large to store?
# ========================================================
# Answer: exact medians need all the data (or a value range). If an
# approximation is fine, keep a fixed-size summary (a histogram of buckets, or a
# quantile sketch) and read the median from it.


# ========================================================
# FOLLOW-UP 4: The median of only the last K values
# ========================================================
# ---- follow-up 4: function ----
# Answer: keep the window in order of arrival (to know what expires) and
# in a sorted list (to read the middle). Insert the new value into its sorted
```

```
# place and remove the expired one. O(K) per update because a list shifts;
# two heaps with lazy deletion give O(log K).
# Complexity: Time addNum O(k) (a sorted insert and a removal), findMedian O(1), Space O(k)
class WindowMedian:
    def __init__(self, k):
        self.k = k
        self.order = []                             # the last k values, oldest first
        self.sorted = []                            # the same values in sorted order

    def addNum(self, x):
        self.order.append(x)
        position = 0
```

```
        while position < len(self.sorted) and self.sorted[position] < x:
            position += 1
        self.sorted.insert(position, x)
        if len(self.order) > self.k:
            old = self.order.pop(0)
            self.sorted.remove(old)                # removes one copy of the expired value

    def findMedian(self):
        n = len(self.sorted)
        return (self.sorted[(n - 1) // 2] + self.sorted[n // 2]) / 2


# ---- follow-up 4: tests ----
def test_follow_up_4():
    window = WindowMedian(3)
    for x in [9, 1, 5, 7]:
        window.addNum(x)
    assert window.findMedian() == 5                # the window is [1, 5, 7]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement the two-heap median structure and dry-run odd and even stream lengths.
Sketch a bounded-domain alternative and state its update and query costs.
Compare what must change when the requested order statistic is an arbitrary k rather than a fixed median.

STUDY PATH: question 18 of 197 | topic 4 of 43: A heap of what is available: pop the best, put back the changed one | 2 of 8

Template fit: heap of end times; a meeting reuses the room that frees first if it ended, otherwise push a new room | * asked 250 days ago *

MEDIUM | IN APP LC 253 | PracHub | Technical Screen | 18 of 197

★ TOP 75: rank 50 in the Medium and Hard priority list

★ Compute minimum number of rooms needed

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TYPE **ALL TIME 58** **30 DAYS 0** **PRIORITY 49/288**

ABOUT THIS QUESTION

Topic: A heap of what is available: pop the best, put back the changed one

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #253 Meeting Rooms II

LeetCode match: LC 253 Meeting Rooms II

Posted on PracHub: 2026-02-12 (updated 2026-05-04)

Original: https://prachub.com/coding-questions/compute-minimum-number-of-rooms-needed

QUESTION

Problem

You are given a list of meetings, each with a start time and end time. A single room can host only one meeting at a time. Two meetings **overlap** if one starts before the other ends (treat meetings as half-open intervals **[start, end)** so a meeting ending at time `t` does not conflict with another starting at time `t`).

Task

Return the **minimum number of rooms** required to host all meetings.

Input

- `intervals`: an array of `n` pairs `[start, end]` where `0 <= start <= end`.

Output

- An integer: the minimum number of rooms needed.

Constraints (typical interview bounds)

- `1 <= n <= 200000`
- Times are integers in a reasonable range (e.g., `0 . 10^9`).

Example

- Input: `[[0,30],[5,10],[15,20]]`
- Output: `2` (One room can host `[0,30]`, the other can host `[5,10]` then `[15,20]`.)

You are given a list of meetings, where each meeting is represented as [start, end]. A single room can host only one meeting at a time. Meetings are treated as half-open intervals [start, end), so a meeting ending at time t does not conflict with another meeting starting at time t. Return the minimum number of rooms required to host all meetings. If start == end, the meeting occupies no time and does not require a room.

Constraints

0 <= n <= 2000000 <= start <= end <= 10^9

QUESTION

EXAMPLES

```
Input: ([[0,30],[5,10],[15,20]],)
Expected Output: 2
Explanation: [0,30] overlaps with both [5,10] and [15,20], so at most 2 rooms are needed at the same
    time.
Input: ([[7,10],[2,4],[4,7],[10,12]],)
Expected Output: 1
```

Explanation: These meetings only touch at endpoints, so one room can host all of them in sequence.

Hints

Try turning each meeting into two events: one when it starts and one when it ends.Because intervals are half-open, if a meeting ends at the same time another starts, process the ending event first.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(meetings):
    lo = min(s for s, e in meetings)
    hi = max(e for s, e in meetings)
    best = 0
    for t in range(lo, hi):
        busy = 0
        for s, e in meetings:
            if s <= t < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(range * n)	O(1)

WHY IT IS SLOW

Depends on the length of the time range, O(range * n). A min-heap of room end times, or sorted starts and ends with two pointers, is O(n log n).

OPTIMAL SOLUTION · Sort by start with a min-heap of end times

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Can one person attend all meetings (LeetCode 252)?

Sort by start and check that each meeting ends no later than the next one starts. O(n log n), O(1) extra.

FOLLOW-UP 2 Return which room each meeting uses.

Keep a heap of (end time, room id): reuse the room that frees first, otherwise open the next id, and record the id per meeting.

FOLLOW-UP 3 Meetings arrive online.

Keep the end-time heap between calls: pop every end at or before the new start (those rooms are free), then push the new end. The heap size is the number of rooms in use right now; the peak is the answer so far.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Compute minimum number of rooms needed
#
# ## Problem You are given a list of meetings, each with a
# start time and end time. A single room can host only one
# meeting at a time. Two meetings **overlap** if one
# starts before the other ends (treat meetings as half-
# open intervals **[start, end)** so a meeting ending at
# time t does not conflict with another starting at time
# t). ### Task Return the **minimum number of rooms**
# required to host all meetings. ### Input - intervals: an
# array of n pairs [start, end] where 0 <= start <= end.
# ### Output - An integer: the minimum number of rooms
# needed. ### Constraints (typical interview bounds) - 1
# <= n <= 200000 - Times are integers in a reasonable
# range (e.g., 0 . 10^9).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Can one person attend all meetings (LeetCode 252)?
# 2. Return which room each meeting uses.
# 3. Meetings arrive online.
#
# INTERVIEW NOTES
# The rooms needed equal the largest number of meetings
# that overlap at one moment. Two ways to find that
# without checking every moment: sort by start and keep a
# min-heap of the end times of rooms in use (reuse the
# room that frees up first), or sort starts and ends
# separately and sweep with two pointers.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: handle the meetings in order of start time and keep a min-heap of
# the END times of the rooms in use. If the room that ends earliest is free
```

```
# by this meeting's start, reuse it (replace its end time); otherwise open a
# new room (push). The size of the heap at the end is the number of rooms
# needed. Meetings are half-open [start, end): a meeting ending at t does not
# clash with one starting at t, and an empty meeting needs no room.
# Complexity: Time O(n log n), Space O(n)
def min_meeting_rooms(intervals):
    meetings = []
```

```
    for start, end in intervals:
        if start < end:
            meetings.append((start, end))
    meetings.sort()
    ends = []
    for start, end in meetings:
        if ends and ends[0] <= start:
            heapq.heapreplace(ends, end)           # the earliest room is free: reuse it
        else:
            heapq.heappush(ends, end)
    return len(ends)


# ---- main question: tests ----
def test_main_question():
    assert min_meeting_rooms([(0, 30), (5, 10), (15, 20)]) == 2
    assert min_meeting_rooms([(7, 10), (2, 4)]) == 1
    assert min_meeting_rooms([(1, 3), (3, 5), (4, 4)]) == 1        # touching meetings share a room; empty
ones need none
    assert min_meeting_rooms([]) == 0


# =======================================================
# FOLLOW-UP 1: Can one person attend all meetings (LeetCode 252)?
# =======================================================
# ---- follow-up 1: function ----
# Answer: sort by start and check that every meeting ends no later than the
# next one starts. O(n log n) time, O(1) extra space.
# Complexity: Time O(n log n), Space O(n)
def can_attend_all(intervals):
    ordered = sorted(intervals)
    for i in range(len(ordered) - 1):
        if ordered[i][1] > ordered[i + 1][0]:
            return False
    return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
```

```
    assert not can_attend_all([(0, 30), (5, 10), (15, 20)])
    assert can_attend_all([(7, 10), (2, 4)])


# =======================================================
# FOLLOW-UP 2: Return which room each meeting uses
# =======================================================
# ---- follow-up 2: function ----
# Answer: a heap of (end time, room number): reuse the room that frees up
# first, otherwise open the next room number, and write down the room for
# every meeting.
# Complexity: Time O(n log n), Space O(n)
def assign_rooms(intervals):
    order = []
    for i in range(len(intervals)):
        order.append((intervals[i][0], intervals[i][1], i))
    order.sort()                                       # by start time (the index is kept)
    heap = []
    rooms = [None] * len(intervals)
    rooms_used = 0
    for start, end, i in order:
        if heap and heap[0][0] <= start:
            freed_at, room = heapq.heappop(heap)
        else:
            room = rooms_used
```

```
            rooms_used += 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms


# ---- follow-up 2: tests ----
def test_follow_up_2():
    rooms = assign_rooms([(0, 30), (5, 10), (15, 20)])
    assert rooms[0] != rooms[1] and rooms[1] == rooms[2]


# ========================================================
# FOLLOW-UP 3: Meetings arrive online
# ========================================================
# ---- follow-up 3: function ----
# Answer: keep the heap of end times between calls: take out every end at
# or before the new start (those rooms are free), then push the new end.
# The size of the heap is the number of rooms in use right now; the largest
# size seen is the answer so far. (Meetings must come in order of start time.)
```

```
# Complexity: Time book is O(log n) amortized, Space O(n)
class RoomTracker:
    def __init__(self):
        self.ends = []
        self.peak = 0

    def book(self, start, end):
        while self.ends and self.ends[0] <= start:
            heapq.heappop(self.ends)              # that meeting is over: its room is free
        heapq.heappush(self.ends, end)
        if len(self.ends) > self.peak:
            self.peak = len(self.ends)
        return len(self.ends)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    tracker = RoomTracker()
    in_use = []
    for start, end in [(0, 30), (5, 10), (15, 20)]:
        in_use.append(tracker.book(start, end))
    assert in_use == [1, 2, 2]
    assert tracker.peak == 2


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(intervals: list[list[int]]) -> int:
```

Sample call

```
solution([[0,30],[5,10],[15,20]])
```

Test cases

Input	Expected
`([[0,30],[5,10],[15,20]],)`	`2`
`([[7,10],[2,4],[4,7],[10,12]],)`	`1`

ALSO REPORTED · 4 more

Find Minimum Rooms Needed | PracHub | Technical Screen

https://prachub.com/coding-questions/find-minimum-rooms-needed

Compute minimum rooms for time intervals | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-minimum-rooms-for-time-intervals

Given a list of car rental requests with start and end times, assign cars to rentals such | Exponent

https://www.aced.io/questions/5425/assign-cars-minimize-total-cars-rented

Meeting Rooms / Interval Scheduling Variants | 1Point3Acres | phone-screen, tech-screen, onsite-coding | asked 2026-09-20 | Very high

https://www.1point3acres.com/interview/problems/company/google/meeting-rooms-variants

Asked in Round 1 coding, Round 2 onsite coding, phone screen, and as a warm-up in the FDE loop. Almost every other coding loop contains some flavor of this family.
Interviewer commonly chains two follow-ups: (1) reconstruct assignment, (2) scale out to many shards / regions.
For the assignment follow-up, min-heap keyed on end time is the clean answer: pop the earliest-ending room, push the new interval.
For "distributed across shards", expect open-ended discussion: per-shard sweep, cross-shard merge, reconciliation; the interviewer often acknowledges the question is intentionally vague.
The on-call metadata variant includes detailed complexity follow-ups: account for heap operations, stored employee information, and output size separately, and explain each symbol in the complexity expression.

Preparation: Drill three solutions cold: sweep-line / prefix delta, min-heap on end time, sort + greedy room assignment. Be ready to convert between the three when interviewers add follow-ups ("give me the assignment", "now handle cyclic days").
Prepare a 1-minute pitch on map-reduce style sharding (per-shard intervals -> merged timeline -> global max).

STUDY PATH: question 19 of 197 | topic 4 of 43: A heap of what is available: pop the best, put back the changed one | 3 of 8

Template fit: heap of freed locker numbers; take the smallest freed one before handing out a new number | * asked 67 days ago *

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 19 of 197

Implement a Reusable Locker Allocation System

POSTED 2026-08-14 | 67 DAYS AGO | LAST 90 DAYS

STUDY: TALK **ALL TIME 8.1** **30 DAYS 0** **PRIORITY 191/288**

ABOUT THIS QUESTION

Topic: A heap of what is available: pop the best, put back the changed one

Optimal answer from: the interview site's own solution

LeetCode match: LC 1845 Seat Reservation Manager

Posted on PracHub: 2026-08-14 (updated 2026-08-28)

Original: https://prachub.com/coding-questions/implement-a-reusable-locker-allocation-system

QUESTION

Problem

A locker bank has lockers numbered from 1 through **`n`**. Process operations in order. **`store(customer)`** assigns the smallest available locker and returns its number, or **`-1`** if that customer already has a locker or none is available. **`pickup(customer)`** releases and returns that customer's locker, or **`-1`** if no locker is assigned. **`last()`** returns the locker number from the most recent successful **`store`**, or **`-1`** if there has not been one. Return one result per operation.

Constraints & Assumptions

- **`1 <= n <= 200,000`**.
- There are at most 200,000 operations.
- Customer IDs are non-empty strings.
- A released locker becomes immediately available for reuse.

Clarifications

- Failed store and pickup operations do not change **`last()`**.
- The allocation rule is always the smallest currently free locker number.
- A customer can store again after a successful pickup.

Examples

```
n = 3 operations = [store("a"), store("b"), pickup("a"), store("c"), last()] output = [1, 2, 1, 1, 1]
```

Hints

```
Track free lockers Use an ordered structure that can remove and reinsert the minimum locker efficiently.
```

```
Track ownership A map from customer ID to locker supports validation and pickup.
```

Process a fresh locker bank with lockers numbered 1 through n. Operations are ["store", customer], ["pickup", customer], or ["last"]. Store assigns and returns the smallest available locker, or -1 if the customer already owns one or none is free. Pickup releases and returns that customer's locker, or -1 if none is assigned. Last reports the locker from the most recent successful store, or -1 before any success. Return one integer per operation.

Constraints

1 <= n <= 200000.operations contains at most 200000 encoded store, pickup, or last lists.Customer IDs are non-empty strings.A released locker becomes immediately available for reuse.Failed store and pickup operations do not change last.A customer can store again after a successful pickup.

QUESTION

EXAMPLES

```
Input: (3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])
Expected Output: [1, 2, 1, 1, 1]
```

Explanation: This is the source example; the released smallest locker is reused.

```
Input: (5, [])
Expected Output: []
```

Explanation: An empty batch produces no per-operation results.

Hints

Keep available locker numbers in a min-oriented structure.Use a customer-to-locker map to validate stores and pickups.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
owner = {}  # customer -> locker
def store(c):
    if c in owner:
        return -1
    taken = set(owner.values())
    for locker in range(1, n + 1):
        if locker not in taken:
            owner[c] = locker
            return locker
    return -1
def pickup(c):
    owner.pop(c, None)
```

TIME	SPACE
O(n) per operation	O(n)

WHY IT IS SLOW

Each store scans up to n lockers, O(n). A min-heap of freed numbers plus a counter for the next unused one makes each operation O(log n).

OPTIMAL SOLUTION · Min-heap of released numbers plus a next-unused counter

TIME	SPACE
O(log n) per operation	O(lockers released and not reused)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 There are 10^9 lockers.

The counter and the heap already make the structure independent of n: nothing is allocated up front, and memory depends only on how many lockers have been released. **Cost:** O(log R) per operation and O(R) memory, where R is the number of lockers released and not yet reused; independent of n.

FOLLOW-UP 2 Many threads call store and pickup.

Protect the heap, the counter and the customer map with one lock so a locker number is never handed out twice (or fine-grained: a lock around the (heap, counter) pair).

FOLLOW-UP 3 Lockers of different sizes.

Keep one heap and counter per size class, and let a customer choose the smallest class that fits. The structure per class is the same.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement a Reusable Locker Allocation
# System
#
# ## Problem A locker bank has lockers numbered from 1
# through n. Process operations in order. store(customer)
# assigns the smallest available locker and returns its
# number, or -1 if that customer already has a locker or
# none is available. pickup(customer) releases and returns
# that customer's locker, or -1 if no locker is assigned.
# last() returns the locker number from the most recent
# successful store, or -1 if there has not been one.
# Return one result per operation. ### Constraints &
# Assumptions - 1 <= n <= 200,000. - There are at most
# 200,000 operations. - Customer IDs are non-empty
# strings. - A released locker becomes immediately
# available for reuse. ### Clarifications - Failed store
# and pickup operations do not change last(). - The
# allocation rule is always the smallest currently free
# locker number. - A customer can store again after a
# successful pickup.
#
# FOLLOW-UPS (answered below, in this order):
# 1. There are 10^9 lockers.
# 2. Many threads call store and pickup.
# 3. Lockers of different sizes.
#
# INTERVIEW NOTES
# 'Smallest free number' with numbers coming back means a
# min-heap of returned numbers. Numbers that were never
# handed out do not need to be stored: keep a counter for
# the next unused number and use the heap first. That
# keeps memory proportional to the number of lockers in
# use, not to n.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq
import threading
```

```
# ============================================================
# MAIN QUESTION
```

```
# ========================================================
# ---- main question: main class ----
# n lockers numbered 1 .. n. store(customer) gives the smallest free locker
# number (-1 if the customer already has one or none is free); pickup(customer)
# frees the customer's locker and returns its number (-1 if there is none);
# last() is the locker of the latest SUCCESSFUL store (-1 if none yet).
# Idea: a min-heap of RELEASED locker numbers plus a counter of the next
# never-used number. A released locker is always smaller than any never-used
# one, so: take from the heap if it is not empty, else take the counter if it
# is at most n. No list of 10^9 lockers is ever built (follow-up 1).
# Complexity: Time store and pickup are O(log n) (a heap of free lockers), last is O(1), Space O(n)
class Lockers:
    def __init__(self, n):
        self.n = n
        self.next_new = 1                    # the next locker number never used
        self.free = []                       # released lockers, smallest on top
        self.of = {}                         # customer -> locker number
        self.last_store = -1

    def store(self, customer):
        if customer in self.of:
            return -1
        if self.free:
            locker = heapq.heappop(self.free)
        elif self.next_new <= self.n:
            locker = self.next_new
            self.next_new += 1
        else:
            return -1
        self.of[customer] = locker
        self.last_store = locker
        return locker

    def pickup(self, customer):
        if customer not in self.of:
            return -1
        locker = self.of.pop(customer)
        heapq.heappush(self.free, locker)      # it is available again at once
        return locker

    def last(self):
        return self.last_store                 # failed operations never change this
```

```
# ---- main question: tests ----
def process(n, operations):
    box = Lockers(n)
    answers = []
    for operation in operations:
        if operation[0] == "store":
            answers.append(box.store(operation[1]))
        elif operation[0] == "pickup":
            answers.append(box.pickup(operation[1]))
        else:
            answers.append(box.last())
    return answers


def test_main_question():
    operations = [["store", "a"], ["store", "b"], ["pickup", "a"], ["store", "c"], ["last"]]
    assert process(3, operations) == [1, 2, 1, 1, 1]
    assert process(2, [["last"], ["pickup", "x"]]) == [-1, -1]               # nothing stored yet, a missin
g pickup
    full = [["store", "a"], ["store", "a"], ["store", "b"], ["store", "c"]]
    assert process(2, full) == [1, -1, 2, -1]                                  # a customer who has one, an
```

```
d a full bank
    assert process(2, full + [["last"]])[-1] == 2                              # failed operations leave la
st() alone


# ====================================================
# FOLLOW-UP 1: There are 10^9 lockers
# ====================================================
# ---- follow-up 1: tests ----
# Answer: nothing to change: only the released lockers are stored, and a
# counter stands for all the never-used ones.
def test_follow_up_1():
    assert process(10 ** 9, [["store", "a"], ["last"]]) == [1, 1]


# ====================================================
# FOLLOW-UP 2: Many threads call store and pickup
# ====================================================
# ---- follow-up 2: function ----
# Answer: protect the heap, the counter and the customer map with ONE lock,
# so the same locker number is never given out twice (or use finer locks
# around the pair (heap, counter)).
# Complexity: Time each operation takes one lock and then costs what the plain Lockers class costs, Space O
(n)
class SafeLockers:
    def __init__(self, n):
        self.lockers = Lockers(n)
```

```
        self.lock = threading.Lock()

    def store(self, customer):
        with self.lock:
            return self.lockers.store(customer)

    def pickup(self, customer):
        with self.lock:
            return self.lockers.pickup(customer)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    safe = SafeLockers(2)
    assert safe.store("a") == 1 and safe.pickup("a") == 1


# ====================================================
# FOLLOW-UP 3: Lockers of different sizes
# ====================================================
# Answer: keep one heap of released numbers and one "next new" counter per
# SIZE; a customer asks for a size and the smallest locker of that size (or
# the next larger size when none is free) is given.


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def process_locker_operations(n: int, operations: list[list[str]]) -> list[int]:
```

Sample call

```
process_locker_operations(3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])
```

Test cases

Input	Expected
`(3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])`	`[1, 2, 1, 1, 1]`
`(5, [])`	`[]`

ALSO REPORTED · 1 more

Implement a Reusable Locker Allocation System | PracHub | Technical Screen

https://prachub.com/coding-questions/implement-a-reusable-locker-allocation-system-2

STUDY PATH: question 20 of 197 | topic 4 of 43: A heap of what is available: pop the best, put back the changed one | 4 of 8

Template fit: a heap of free room numbers and a heap of (end time, room); a delayed meeting takes the earliest-ending room and shifts later | * asked 287 days ago *

MEDIUM | IN APP LC 2402 | PracHub | Technical Screen | 20 of 197

Simulate room assignments for scheduled meetings

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TALK ALL TIME 40 30 DAYS 0 PRIORITY 109/288

ABOUT THIS QUESTION

Topic: A heap of what is available: pop the best, put back the changed one

Optimal answer from: your app's interview approach (LeetCode #2402)

LeetCode match: LC 2402 Meeting Rooms III (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-03-29)

Original: https://prachub.com/interview-questions/simulate-room-assignments-for-scheduled-meetings

QUESTION

There are n rooms numbered 0 to n - 1 and a list of meetings (start, end). Meetings are handled in order of start time. A meeting takes the lowest-numbered free room; if every room is busy it waits until the room that frees up first becomes free and then runs for its full original length, so it ends later. Return the room that hosted the most meetings (the lowest room number on a tie).

Function signature

def mostBookedRoom(n: int, meetings: list) -> int

EXAMPLES

Example 1	
Input	`2, [[0, 10], [1, 5], [2, 7], [3, 4]]`
Output	`0`
Why	`Rooms 0 and 1 each host two meetings; the lower number wins the tie.`

Example 2	
Input	`3, [[1, 20], [2, 10], [3, 5], [4, 9], [6, 8]]`
Output	`1`
Why	`With three rooms, room 1 hosts two of the five meetings and no room hosts more.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(n, meetings):
    end = [0] * n
    count = [0] * n
    for start, finish in sorted(meetings):
        free = [r for r in range(n)
                if end[r] <= start]
        if free:
            r = min(free)
            end[r] = finish
        else:
            r = end.index(min(end))  # earliest
            wait = end[r] - start
            end[r] = finish + wait
        count[r] += 1
    return count.index(max(count))
```

TIME	SPACE
O(M * N)	O(N)

WHY IT IS SLOW

Every meeting scans all N rooms, O(M * N). Two heaps (free rooms and busy rooms by end time) make each meeting O(log N).

OPTIMAL SOLUTION · Two heaps: free rooms and busy rooms

TIME	SPACE
O(M log M + M log N)	O(N)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if meetings arrive online and cannot be reordered?

The same two heaps work if you process meetings in arrival order and treat the arrival time as the earliest allowed start, but you can no longer rely on sorted starts, so you must first move every room whose end time is <= the arrival into the free heap. Each call stays O(log N).

FOLLOW-UP 2 How do you report the busiest room by total busy time instead of meeting count?

Add the meeting's actual duration to used[room] instead of 1 when you assign it. Rooms are chosen the same way; only the tally changes, and ties still break toward the lowest index.

FOLLOW-UP 3 Rank rooms by total busy time instead of the number of meetings.

Add each meeting's duration (its original length, not the delay) to the room's tally; the choice of room is unchanged.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Simulate room assignments for scheduled
# meetings
#
# There are n rooms numbered 0 to n - 1 and a list of
```

```
# meetings (start, end). Meetings are handled in order of
# start time. A meeting takes the lowest-numbered free
# room; if every room is busy it waits until the room that
# frees up first becomes free and then runs for its full
# original length, so it ends later. Return the room that
# hosted the most meetings (the lowest room number on a
# tie).
#
# Signature:
#   def mostBookedRoom(n: int, meetings: list) -> int
#
# Examples:
# 2, [[0, 10], [1, 5], [2, 7], [3, 4]]  ->  0
# 3, [[1, 20], [2, 10], [3, 5], [4, 9], [6, 8]]  ->  1
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if meetings arrive online and cannot be
# reordered?
# 2. How do you report the busiest room by total busy time
# instead of meeting count?
# 3. Rank rooms by total busy time instead of the number
# of meetings.
#
# INTERVIEW NOTES
# Simulate in start order with two questions per meeting:
# which rooms are free now (lowest index first), and if
# none, which room frees up first (the meeting is delayed
# but keeps its length). Two heaps answer both quickly:
# one of free room ids, one of (end time, room id) for
# busy rooms.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# n rooms (0 .. n - 1) and meetings (start, end). Meetings are handled in
# order of start time; a meeting takes the lowest-numbered free room. If
# every room is busy it WAITS for the room that frees up first and then
# runs for its full original length (so it ends later). Which room hosts the
# most meetings (the lowest number on a tie)?
# Idea: two heaps. `free` holds the numbers of the free rooms (lowest on
# top); `busy` holds (time the room becomes free, room). For each meeting,
# first move the rooms that finished by its start from `busy` to `free`.
# If a room is free take the lowest one; otherwise take the room that
# frees up first and push the meeting back by the waiting time.
# (`by="time"` is follow-up 3: rank rooms by their total busy time.)
# Complexity: Time O(M log M + M log n) for M meetings and n rooms, Space O(n)
def mostBookedRoom(n, meetings, by="count"):
    free = list(range(n))
    heapq.heapify(free)
    busy = []
    used = [0] * n
    for start, end in sorted(meetings):
        while busy and busy[0][0] <= start:
            finished_at, room = heapq.heappop(busy)
```

```
            heapq.heappush(free, room)                      # this room is free again
        if free:
            room = heapq.heappop(free)
            heapq.heappush(busy, (end, room))
        else:
            finished_at, room = heapq.heappop(busy)         # wait for the room that frees up first
            heapq.heappush(busy, (finished_at + (end - start), room))
        if by == "count":
            used[room] += 1
        else:
            used[room] += end - start                       # the meeting's own length, not the delay
    best = 0
    for room in range(n):
        if used[room] > used[best]:
            best = room                                     # ties keep the lowest room number
    return best


# ---- main question: tests ----
def test_main_question():
    assert mostBookedRoom(2, [[0, 10], [1, 5], [2, 7], [3, 4]]) == 0
```

```
    assert mostBookedRoom(3, [[1, 20], [2, 10], [3, 5], [4, 9], [6, 8]]) == 1
    assert mostBookedRoom(1, [[0, 1], [0, 1]]) == 0           # a single room takes everything


# ============================================================
# FOLLOW-UP 1: What if meetings arrive online and cannot be reordered?
# ============================================================
# Answer: the same two heaps work, because meetings are handled in start
# order. If they can arrive out of order you need to know the earliest
# future start, so keep a buffer of the next few meetings and decide only
# when the order is certain.


# ============================================================
# FOLLOW-UP 2: How do you report the busiest room by total busy time instead of meeting count?
# ============================================================


# ============================================================
# FOLLOW-UP 3: Rank rooms by total busy time instead of the number of meetings
# ============================================================
# Answer: add each meeting's own length (not the delay) to the room's total
# instead of 1: the `by="time"` option above. The choice of room does not change.
# ---- follow-up 3: tests ----
def test_follow_up_3():
    meetings = [[0, 100], [1, 2], [2, 3], [3, 4]]
    assert mostBookedRoom(2, meetings, "count") == 1
    assert mostBookedRoom(2, meetings, "time") == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FULL QUESTION TEXT FROM THE SITE

Problem

You are given n meeting rooms labeled 0..n-1 and a list of meetings meetings, where each meeting is [start, end] with start < end. Meetings are processed in increasing order of start time.

Assign each meeting to a room using these rules:

If one or more rooms are free at start , assign the meeting to the free room with the smallest index .
If no rooms are free at start , the meeting must wait until a room becomes free. It is then scheduled in the room that becomes available earliest (break ties by smallest room index). If it waits, its duration stays the same , so if the original duration is end - start and the room becomes free at time t , the meeting runs on [t, t + (end - start)] .

Each time a meeting is assigned to a room (even after waiting), that room’s meeting count increases by 1.

Task

Return the index of the room that hosted the most meetings. If there is a tie, return the smallest index.

Input/Output
Input: n (int), meetings (list of [start,end])
Output: an integer room index
Constraints (reasonable interview assumptions)
1 <= n <= 10^5
1 <= len(meetings) <= 10^5
Times fit in 32-bit signed integer; waiting can increase end times beyond original but still fits in 64-bit.

ALSO REPORTED · 1 more

Assign meetings to rooms with delays | PracHub | Technical Screen

https://prachub.com/interview-questions/assign-meetings-to-rooms-with-delays

STUDY PATH: question 21 of 197 | topic 4 of 43: A heap of what is available: pop the best, put back the changed one | 5 of 8

Template fit: the template as written: heap of the times the CPUs become free | * asked 392 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 21 of 197

Minimum Makespan on Identical CPUs

ASKED 2025-09-23 | 392 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 221/288

ABOUT THIS QUESTION

Topic: A heap of what is available: pop the best, put back the changed one

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #1834 Single-Threaded CPU

LeetCode match: LC 1834 Single-Threaded CPU (related: identical CPUs)

Frequency: Single report | Last asked: 2025-09-23 | Roles: SWE Intern

QUESTION

You have C identical CPUs and a list release_times , where release_times[i] is the earliest time job i may start. Every job runs non-preemptively for the same fixed duration S . When a job's release time arrives it may start on any free CPU; if all CPUs are busy it waits and starts on the first CPU that becomes free (never before its release time). Each CPU runs one job at a time. Scheduling optimally, compute the minimum possible makespan: the time at which the last job finishes.

Function signature

```
def min_makespan(release_times: list[int], num_cpus: int, duration: int) -> int:
    ...
```

Follow-up (as reported): (Reported) The naive approach tracked an O(n*C) table of CPU availability; reduce it to a min-heap of the C next-free times, popping the earliest-free CPU per job after sorting jobs by release time.

EXAMPLES

Example 1

Input	`release_times = [0, 1, 2, 3], num_cpus = 2, duration = 3`
Output	`7`
Why	`CPU-A: job@0 -> [0,3), job@2 starts at 3 -> [3,6). CPU-B: job@1 -> [1,4), job@3 starts at 4 -> [4,7). Last finish = 7.`

Example 2

Input	`release_times = [0, 0, 0], num_cpus = 1, duration = 2`
Output	`6`
Why	`One CPU runs them back to back: [0,2), [2,4), [4,6).`

Example 3

Input	`release_times = [5], num_cpus = 3, duration = 4`
Output	`9`
Why	`The single job starts at its release time 5, finishes at 9.`

CONSTRAINTS

- 1 <= C <= 10^5
- 0 <= len(release_times) <= 10^5
- 0 <= release_times[i] <= 10^9

QUESTION

- 1 <= S <= 10^9
- If there are no jobs, the makespan is 0 .
- Target complexity: O(n log C) time, O(C) space.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
from itertools import product
def brute_makespan(release, cpus, duration):
    best = float("inf")
    jobs = sorted(release)
    for assign in product(
        range(cpus), repeat=len(jobs)
    ):
        free, last = [0] * cpus, 0
        for r, cpu in zip(jobs, assign):
            free[cpu] = (
                max(free[cpu], r) + duration
            )
            last = max(last, free[cpu])
        best = min(best, last)
    return best if jobs else 0
```

TIME	SPACE
O(C^n)	O(n)

WHY IT IS SLOW

It enumerates all C^n ways to distribute the jobs; each CPU runs its jobs in release order.

OPTIMAL SOLUTION · Sorted jobs and a heap of free times

TIME	SPACE
O(n log n)	O(C)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The first idea tracked every CPU's availability in a list. Why is the heap better?

With a list you scan all C CPUs for the earliest free one on every job: O(n * C). A min-heap returns the earliest free time in O(log C) and updates in O(log C), so the total is O(n log C) after the sort.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Minimum Makespan on Identical CPUs
#
# You have C identical CPUs and a list release_times ,
# where release_times[i] is the earliest time job i may
# start. Every job runs non-preemptively for the same
# fixed duration S . When a job's release time arrives it
# may start on any free CPU; if all CPUs are busy it waits
# and starts on the first CPU that becomes free (never
```

```
# before its release time). Each CPU runs one job at a
# time. Scheduling optimally, compute the minimum possible
# makespan: the time at which the last job finishes.
# Function signature: def min_makespan(release_times:
# list[int], num_cpus: int, duration: int) -> int: ...
# Follow-up (as reported): (Reported) The naive approach
# tracked an O(n*C) table of CPU availability; reduce it
# to a min-heap of the C next-free times, popping the
# earliest-free CPU per job after sorting jobs by release
# time.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The first idea tracked every CPU's availability in a
# list. Why is the heap better?
#
# INTERVIEW NOTES
# Jobs have equal duration, so the greedy order is release
# order: sort by release time and give each job the CPU
# that frees up first. Only the C free times matter, so
# keep them in a min-heap instead of scanning an O(n * C)
# table.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Jobs become available at the times in `release_times`; every job takes
```

```
# `duration`; there are `num_cpus` identical CPUs. When is everything done
# (the makespan) if each job goes to the CPU that frees up first, but never
# starts before its release time?
# Idea: handle the jobs in release order and keep a min-heap of the times
# at which the CPUs become free (all 0 at the start).
# Complexity: Time O(n log n + n log c) for n jobs and c CPUs, Space O(c)
def min_makespan(release_times, num_cpus, duration):
    free = [0] * num_cpus
    last = 0
    for release in sorted(release_times):
        cpu_free = heapq.heappop(free)            # the CPU that frees up first
        start = release
        if cpu_free > start:
            start = cpu_free
        heapq.heappush(free, start + duration)
        if start + duration > last:
            last = start + duration
    return last


# ---- main question: tests ----
def test_main_question():
    assert min_makespan([0, 1, 2, 3], 2, 3) == 7
    assert min_makespan([0, 0, 0], 1, 2) == 6
    assert min_makespan([5], 3, 4) == 9
```

```
# ============================================================
# FOLLOW-UP 1: The first idea tracked every CPU's availability in a list. Why is the heap better?
# ============================================================
# ---- follow-up 1: function ----
# Answer: with a list you have to scan all C CPUs for the earliest free one
# on every job: O(n * C). A min-heap gives the earliest free time in O(1),
# takes it out and puts the new time back in O(log C), so the total is
# O(n log C) after the sort.
# Complexity: Time O(n log n + n * c), Space O(c)
def min_makespan_scan(release_times, num_cpus, duration):
    free = [0] * num_cpus
    for release in sorted(release_times):
        earliest = 0
        for i in range(num_cpus):                  # the scan the heap avoids
            if free[i] < free[earliest]:
                earliest = i
        start = release
        if free[earliest] > start:
            start = free[earliest]
        free[earliest] = start + duration
    if not release_times:
        return 0
    last = 0
    for t in free:
        if t > last:
            last = t
    return last


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert min_makespan_scan([0, 1, 2, 3], 2, 3) == 7
    for jobs in ([0, 0, 0, 5, 9], [3, 1, 2], []):
        for cpus in (1, 2, 3):
            assert min_makespan_scan(jobs, cpus, 4) == min_makespan(jobs, cpus, 4)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 22 of 197 | topic 4 of 43: A heap of what is available: pop the best, put back the changed one | 6 of 8

Template fit: a heap per type of (flexibility, server); skip servers already allocated (lazy deletion) | * asked 409 days ago *

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 22 of 197

Design server allocation for multi-type nodes

POSTED 2025-09-06 | 409 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 280/288

ABOUT THIS QUESTION

Topic: A heap of what is available: pop the best, put back the changed one

Optimal answer from: written and tested here for this exact question

LeetCode match: No direct match; related LC 1912 Design Movie Rental System

Posted on PracHub: 2025-09-06 (updated 2026-08-05)

Original: https://prachub.com/interview-questions/design-server-allocation-for-multi-type-nodes

QUESTION

Design a service that manages a pool of server nodes. Each node supports several instance types. getServer(count, types) hands out up to count servers that together cover all the requested types, preferring the least flexible servers (the ones that support the fewest types) so that versatile servers stay free, and returns None (allocating nothing) if the request cannot be met. releaseServer(ids) returns servers to the pool.

Function signature

class Servers:

def __init__(self, servers: dict)

id -> set of instance types

def getServer(self, count: int, types: list) -> list | None

def releaseServer(self, ids: list) -> None

EXAMPLES

Example 1

Input	`s = Servers({1: {'a'}, 2: {'a', 'b'}, 3: {'b'}, 4: {'a', 'b', 'c'}})`
	`s.getServer(2, ['a', 'b'])`
	`s.getServer(2, ['a', 'b'])`
	`s.getServer(1, ['a'])`
	`s.releaseServer([1])`
	`s.getServer(1, ['a'])`

Output	`s.getServer(2, ['a', 'b']) -> [1, 3]`
	`s.getServer(2, ['a', 'b']) -> [2, 4]`
	`s.getServer(1, ['a']) -> null`
	`s.getServer(1, ['a']) -> [1]`

Why	The first request takes the two least flexible servers (1 for a, 3 for b); the second takes 2 and 4; nothing is left for the third; after server 1 is released it can serve type a again.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def get_server(count, types):
    chosen = []
    for t in types:
        pool = [s for s in free if t in s.types
                and s not in chosen]
        chosen.append(min(pool, key=flexibility))
    rest = [s for s in free if s not in chosen
            and s.types & set(types)]
    chosen += rest[:count - len(chosen)]
    if len(chosen) < count:
        return None  # not enough
    return chosen
```

TIME	SPACE
O(N * T) per request (N servers)	**O(N)**

WHY IT IS SLOW

Every request scans all N servers, O(N * types). Per-type heaps ordered by flexibility with lazy deletion make a request O((count + types) log N).

OPTIMAL SOLUTION · Per-type heaps ordered by flexibility, with lazy deletion

TIME	SPACE
O((count + types) log N) amortized per request	**O(total type memberships)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Many clients call at the same time.

Make allocation atomic: one lock around the allocate step, or shard the pool by type or by id range and route requests to shards. The roll-back-on-failure design means a failed request leaves no trace.

FOLLOW-UP 2 Clients keep asking for rare types and starve.

Use a FIFO queue per request class and hold back versatile servers for waiting rare-type requests, or age requests so an old one outranks new ones. Fragmentation control (least flexible first) already reduces the pressure.

FOLLOW-UP 3 Servers fail while allocated.

Track a lease or heartbeat per allocation; when a server stops responding mark it unavailable, remove it from the heaps (lazy deletion handles stale entries), and tell the client so it can request a replacement.

FOLLOW-UP 4 A client dies without releasing its servers.

Give every allocation a lease (expiry time). A periodic sweep returns servers whose lease has expired to the free pool (the lazy-deletion indexes make a release cheap), and clients renew the lease while they are alive.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Design server allocation for multi-type
# nodes
#
# Design a service that manages a pool of server nodes.
# Each node supports several instance types.
# getServer(count, types) hands out up to count servers
# that together cover all the requested types, preferring
# the least flexible servers (the ones that support the
# fewest types) so that versatile servers stay free, and
# returns None (allocating nothing) if the request cannot
# be met. releaseServer(ids) returns servers to the pool.
#
# Signature:
#   class Servers:
#       def __init__(self, servers: dict)      # id -> set of instance types
#       def getServer(self, count: int, types: list) -> list | None
#       def releaseServer(self, ids: list) -> None
#
# FOLLOW-UPS (answered below, in this order):
# 1. Many clients call at the same time.
# 2. Clients keep asking for rare types and starve.
# 3. Servers fail while allocated.
# 4. A client dies without releasing its servers.
#
# INTERVIEW NOTES
# A request mentions a few types, so index the free
# servers by type and avoid scanning all 100,000. To keep
# versatile servers available for harder requests, always
# prefer the server that supports the fewest types.
# Allocation touches every type index of a server, so use
# lazy deletion: mark it allocated and let the stale heap
# entries be skipped when they surface.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
```

```
# ============================================================
# ---- main question: main class ----
# Servers support several instance types. getServer(count, types) must
# return `count` free servers that TOGETHER cover all the requested types
# (None if it cannot). To keep the versatile servers free for later, use the
# LEAST flexible suitable server first.
# Idea: for every type keep a heap of (number of types the server supports,
# server id) for the servers that support it: the least flexible is on top.
# A server that gets allocated is not removed from the other heaps at once
# ("lazy deletion"): it is skipped and dropped when it reaches the top.
# Pick the servers first and commit only if the whole request works.
# Complexity: Time getServer is O(T log S) for T requested types and S servers (lazy deletion), releaseServ
er O(types of each server * log S), Space O(S * types per server)
```

```
class Servers:
    def __init__(self, servers):
        # servers: id -> set of instance types
        self.types = {}
        for server in servers:
            self.types[server] = set(servers[server])
        self.allocated = set()
        self.heaps = {}                                 # type -> heap of (flexibility, server id)
        self.lease = {}                                 # allocated server -> time its lease runs out (follo
w-up 4)
        for server in self.types:
            for t in self.types[server]:
                if t not in self.heaps:
                    self.heaps[t] = []
                heapq.heappush(self.heaps[t], (len(self.types[server]), server))

    def best_for(self, t, skip):
        # the best free server of type t that is not in `skip` (None if there is none)
        heap = self.heaps.get(t, [])
        looked_past = []
        while heap:
            flexibility, server = heap[0]
            if server in self.allocated:
                heapq.heappop(heap)                     # lazy deletion
            elif server in skip:
                looked_past.append(heapq.heappop(heap))
            else:
                break
        best = None
        if heap:
            best = heap[0]
        for item in looked_past:
            heapq.heappush(heap, item)                  # put back what we only looked past
        return best
```

```
    def covers(self, chosen, t):
        # does one of the chosen servers already support type t?
        for server in chosen:
            if t in self.types[server]:
                return True
        return False

    def getServer(self, count, types):
        chosen = []
        for t in types:
            if self.covers(chosen, t):
                continue
            top = self.best_for(t, set(chosen))
            if top is None:
                return None
            chosen.append(top[1])
        while len(chosen) < count:                      # fill up with the least flexible free servers
            skip = set(chosen)
            options = []
            for t in types:
                found = self.best_for(t, skip)
                if found is not None:
                    options.append(found)
            if not options:
                return None
            chosen.append(min(options)[1])
        if len(chosen) > count:
            return None                                 # more servers were needed to cover the types than a
llowed
```

```
        for server in chosen:
            self.allocated.add(server)
        return chosen

    def releaseServer(self, ids):
        for server in ids:
            if server in self.allocated:
                self.allocated.discard(server)
                self.lease.pop(server, None)
                for t in self.types[server]:
                    heapq.heappush(self.heaps[t], (len(self.types[server]), server))

    # follow-up 4: a client that dies without releasing its servers
    def rent(self, count, types, now, ttl):
        got = self.getServer(count, types)
        if got:
            for server in got:
                self.lease[server] = now + ttl
        return got

    def expire(self, now):
        late = []
        for server in self.lease:
            if self.lease[server] <= now:
                late.append(server)
        self.releaseServer(late)
        return late


# ---- main question: tests ----
def test_main_question():
    pool = Servers({1: {"gpu"}, 2: {"gpu", "big"}, 3: {"big"}, 4: {"cpu"}})
    assert pool.getServer(2, ["gpu", "big"]) == [1, 3]        # the least flexible server of each type goes
 first
    assert pool.getServer(1, ["gpu", "big"]) == [2]           # the versatile server 2 was still there
    assert pool.getServer(1, ["gpu"]) is None                 # nothing is left for a gpu request
    assert pool.getServer(1, ["cpu", "gpu"]) is None          # does not fit in one server: nothing changes
    assert pool.getServer(1, ["cpu"]) == [4]
    pool.releaseServer([2])
    assert pool.getServer(1, ["gpu"]) == [2]


# =======================================================
# FOLLOW-UP 1: Many clients call at the same time
# =======================================================
# Answer: protect the pool with one lock (as in the locker question), or
# shard it by instance type so requests for different types do not wait for each other.


# =======================================================
# FOLLOW-UP 2: Clients keep asking for rare types and starve
# =======================================================
# Answer: keep some servers of each rare type in reserve, or let a request
# wait in a queue per type so older requests are served first.


# =======================================================
# FOLLOW-UP 3: Servers fail while allocated
# =======================================================
# Answer: have the servers send heartbeats; when a server stops, treat it as
# released and tell its client (who can ask for a replacement).
```

```
# =======================================================
```

```
# FOLLOW-UP 4: A client dies without releasing its servers
# ========================================================
# Answer: give every allocation a LEASE (an expiry time). A periodic sweep
# returns the servers whose lease ran out to the free pool, and a live client
# renews its lease. rent() sets the lease and expire() returns the servers
# of dead clients to the pool.
# ---- follow-up 4: tests ----
def test_follow_up_4():
    pool = Servers({1: {"a"}})
    assert pool.rent(1, ["a"], now=0, ttl=10) == [1]
    assert pool.getServer(1, ["a"]) is None
    assert pool.expire(now=5) == []
    assert pool.expire(now=10) == [1]
    assert pool.getServer(1, ["a"]) == [1]


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FULL QUESTION TEXT FROM THE SITE

Design a service to manage a large pool (about 100, 000) of server nodes. Each node can satisfy multiple instance types; clients call getServer(reqNumOfServer, instanceTypes[]) to request servers that collectively meet the requested types, and returnServer(ids[]) to release them. Propose data structures and indexes for fast matching, availability tracking, fragmentation minimization, and scaling. Address concurrency, starvation, and failure handling.

STUDY PATH: question 23 of 197 | topic 4 of 43: A heap of what is available: pop the best, put back the changed one | 7 of 8

Template fit: a heap of creditors and a heap of debtors; the biggest of each settles min(amount) and the remainder goes back | ND

MEDIUM | IN APP LC 465 | HackTheRounds | Onsite | 23 of 197

Trip Expense Settlement

STUDY: TALK ALL TIME 25.5 30 DAYS 0 PRIORITY 161/288

ABOUT THIS QUESTION

Topic: A heap of what is available: pop the best, put back the changed one

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #465 Optimal Account Balancing

LeetCode match: LC 465 (Optimal Account Balancing)

QUESTION

A group of friends go on a trip and each friend may pay for others. Every record gives the payer, the amount and the list of people who share it equally (the payer may be one of them). Compute a small set of payments that settles everyone's debts: each payment is {payer, amount, payee}.

Function signature

def settle_greedy(records: list) -> list

EXAMPLES

Example 1

Input	`trip = [{'payer': 'alice', 'amount': 4000, 'payees': ['bob', 'jess', 'alice', 'sam']}, {'payer': 'jess', 'amount': 2000, 'payees': ['jess', 'alice']}]` `settle_greedy(trip)`
Output	`settle_greedy(trip) -> [{'payer': 'bob', 'amount': 1000, 'payee': 'alice'}, {'payer': 'sam', 'amount': 1000, 'payee': 'alice'}]`
Why	`Alice paid 4000 shared four ways and owes 1000 of Jess's 2000; Bob and Sam each owe 1000, and Jess is even. Two payments settle everything.`

Example 2

Input	`trip = [{'payer': 'alice', 'amount': 3000, 'payees': ['alice', 'bob', 'cy']}]` `settle_greedy(trip)`
Output	`settle_greedy(trip) -> [{'payer': 'bob', 'amount': 1000, 'payee': 'alice'}, {'payer': 'cy', 'amount': 1000, 'payee': 'alice'}]`
Why	`Bob and Cy each owe Alice their 1000 share.`

CONSTRAINTS

- 1 <= payments.length <= 1000
- 1 <= amount <= 10^6
- 1 <= payees.length <= 100

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def settle(i):
    while i < n and net[i] == 0:
        i += 1  # already settled
    if i == n:
        return 0
    best = INF
    for j in range(i + 1, n):
        if net[j] * net[i] < 0:  # opposite sign
            net[j] += net[i]  # move balance i
            best = min(best, 1 + settle(i + 1))
            net[j] -= net[i]  # undo
    return best
```

TIME	SPACE
O(n!) worst case	O(n)

WHY IT IS SLOW

It tries every pairing, up to n! in the worst case. Matching the largest creditor with the largest debtor (heaps) is O(n log n) and uses at most n - 1 payments, though not always the minimum.

OPTIMAL SOLUTION · Heaps: match the largest debtor with the largest creditor

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do you need the true minimum number of payments?

That is NP-hard in general (it is the zero-sum partition problem). The greedy answer is at most n - 1 payments and is usually accepted; for exact answers use the bitmask DP above (about 20 people) or backtracking with pruning.

FOLLOW-UP 2 Amounts that do not divide evenly (cents).

Work in integer cents and decide where the remainder goes (to the payer, or round-robin among payees). Never use floats for money, and make the rule deterministic.

FOLLOW-UP 3 Different shares per person instead of equal split.

Replace the equal share with an explicit amount per payee in each record; the net balance per person is computed the same way.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Trip Expense Settlement
#
# A group of friends go on a trip and each friend may pay
# for others. Every record gives the payer, the amount and
# the list of people who share it equally (the payer may
```

```
# be one of them). Compute a small set of payments that
# settles everyone's debts: each payment is {payer,
# amount, payee}.
#
# Signature:
#   def settle_greedy(records: list) -> list
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do you need the true minimum number of payments?
# 2. Amounts that do not divide evenly (cents).
# 3. Different shares per person instead of equal split.
#
# INTERVIEW NOTES
# First reduce the records to one number per person: net
# balance = paid - owed. Only that matters; who owed whom
# originally is irrelevant. Then settling is moving money
# between people with positive and negative balances.
# Matching largest creditor with largest debtor is quick
# and uses at most n - 1 payments, but only searching
# groups of people who sum to zero gives the true minimum.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq
import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def split_equally(amount, people):
    # share an amount (in whole CENTS, never floats) equally; the cents that
    # do not divide go to the first people in sorted order, so the rule is
    # always the same (follow-up 2)
    people = sorted(people)
    base = amount // len(people)
    extra = amount % len(people)
    shares = {}
    for i in range(len(people)):
        shares[people[i]] = base
        if i < extra:
            shares[people[i]] += 1
    return shares


def shares_of(record):
    # who owes what for one record. `payees` is a list (equal split) or a
    # dictionary {person: amount} for different shares (follow-up 3).
    payees = record["payees"]
    if isinstance(payees, dict):
        total = 0
        for share in payees.values():
            total += share
        if total != record["amount"]:
            raise ValueError("shares do not add up")
        return payees
    return split_equally(record["amount"], payees)
```

```
def balances(records):
    # the net balance of every person: what they paid minus what they owe.
    # Positive: the group owes them money. All balances add up to zero.
    net = {}
    for record in records:
        payer = record["payer"]
        net[payer] = net.get(payer, 0) + record["amount"]
        shares = shares_of(record)
        for person in shares:
            net[person] = net.get(person, 0) - shares[person]
    result = {}
    for person in net:
        if net[person] != 0:
            result[person] = net[person]
    return result


# ---- main question: main function ----
# Idea: the people who are owed money (positive balance) go into one
# max-heap and the people who owe money into another (push negatives to get a
```

```
# max-heap with heapq). Repeatedly let the biggest debtor pay the biggest
# creditor as much as one of them can (the smaller of the two amounts), and
# put back whoever still has something left. At least one person is settled
# per payment, so there are at most n - 1 payments.
# Complexity: Time O(n log n) for n people, Space O(n)
def settle_greedy(records):
    net = balances(records)
    owed = []                                       # creditors: (minus what they are owed, person)
    debt = []                                       # debtors: (minus what they owe, person)
    for person in net:
        if net[person] > 0:
            owed.append((-net[person], person))
        else:
            debt.append((net[person], person))
    heapq.heapify(owed)
    heapq.heapify(debt)
    payments = []
    while owed and debt:
        owed_minus, creditor = heapq.heappop(owed)
        debt_minus, debtor = heapq.heappop(debt)
        pay = min(-owed_minus, -debt_minus)
        payments.append({"payer": debtor, "amount": pay, "payee": creditor})
        if -owed_minus > pay:
            heapq.heappush(owed, (owed_minus + pay, creditor))
        if -debt_minus > pay:
            heapq.heappush(debt, (debt_minus + pay, debtor))
    return payments


# ---- main question: tests ----
def payer_of(payment):
    return payment["payer"]


def test_main_question():
    trip = [{"payer": "alice", "amount": 4000, "payees": ["bob", "jess", "alice", "sam"]},
            {"payer": "jess", "amount": 2000, "payees": ["jess", "alice"]}]
    got = settle_greedy(trip)
    wanted = [{"payer": "bob", "amount": 1000, "payee": "alice"},
              {"payer": "sam", "amount": 1000, "payee": "alice"}]
    assert sorted(got, key=payer_of) == wanted
    assert balances(trip) == {"alice": 2000, "bob": -1000, "sam": -1000}
```

```
# ======================================================
```

```
# FOLLOW-UP 1: Do you need the true minimum number of payments?
# ======================================================
# ---- follow-up 1: function ----
# Answer: that is NP-hard in general (it is a zero-sum partition problem).
# The greedy answer needs at most n - 1 payments and is usually accepted. For
# an exact answer: people split into groups that settle among themselves
# (the group's balances add up to 0), and a group of g people needs g - 1
# payments, so find the MOST groups with a DP over subsets of people
# (fine up to about 20 people).
# Complexity: Time O(n * 2^n), Space O(2^n)
def min_payments_exact(values):
    n = len(values)
    total = [0] * (2 ** n)                          # total[mask] = the balances of the people in the subset
 `mask`
    groups = [0] * (2 ** n)                         # groups[mask] = the most zero-sum groups inside `mask`
    for mask in range(1, 2 ** n):
        best = 0
        for i in range(n):
            if (mask // 2 ** i) % 2 == 1:           # person i is in this subset
                total[mask] += values[i]
                if groups[mask - 2 ** i] > best:
                    best = groups[mask - 2 ** i]
        groups[mask] = best
        if total[mask] == 0:
            groups[mask] += 1
    return n - groups[2 ** n - 1]


# the same answer by backtracking: settle the first open balance against
# every balance of the opposite sign (skipping equal ones, which give the same result)
# Complexity: Time O(n!) in the worst case, Space O(n)
def min_payments_search(values):
    values = [v for v in values if v != 0]

    def settle_from(start):
        while start < len(values) and values[start] == 0:
            start += 1
        if start == len(values):
            return 0
        best = None
        tried = set()
        for j in range(start + 1, len(values)):
            if values[j] * values[start] < 0 and values[j] not in tried:
                tried.add(values[j])
                values[j] += values[start]
                payments = 1 + settle_from(start + 1)
```

```
                values[j] -= values[start]
                if best is None or payments < best:
                    best = payments
        return best

    return settle_from(0)


# ---- follow-up 1: tests ----
def everyone_settled(payments, net):
    # after the payments every balance must be zero
    left = dict(net)
    for pay in payments:
        left[pay["payer"]] = left.get(pay["payer"], 0) + pay["amount"]
        left[pay["payee"]] = left.get(pay["payee"], 0) - pay["amount"]
```

```
    for person in left:
        if left[person] != 0:
            return False
    return True


def test_follow_up_1():
    assert min_payments_exact([5, -5]) == 1
    assert min_payments_exact([3, -3, 2, -2]) == 2
    assert min_payments_search([3, -3, 2, -2]) == 2
    random.seed(20)
    for round_number in range(100):
        records = []
        for i in range(random.randint(1, 5)):
            records.append({"payer": random.choice("abcdef"),
                            "amount": random.randint(1, 500),
                            "payees": random.sample("abcdef", random.randint(1, 4))})
        net = balances(records)
        total = 0
        for value in net.values():
            total += value
        assert total == 0
        payments = settle_greedy(records)
        assert everyone_settled(payments, net)
        assert len(payments) <= max(len(net) - 1, 0)
        values = list(net.values())
        assert min_payments_exact(values) == min_payments_search(values) <= len(payments)


# ============================================================
```

```
# FOLLOW-UP 2: Amounts that do not divide evenly (cents)
# ============================================================
# Answer: work in whole cents and decide where the remainder goes (here to
# the first people in sorted order; or to the payer). Never use floats for
# money, and make the rule deterministic: split_equally above.
# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert split_equally(1000, ["c", "a", "b"]) == {"a": 334, "b": 333, "c": 333}      # 10.00 split three
ways


# ============================================================
# FOLLOW-UP 3: Different shares per person instead of equal split
# ============================================================
# Answer: give each record an explicit amount per person (a dictionary in
# `payees`); the net balance is computed the same way: shares_of.
# ---- follow-up 3: tests ----
def test_follow_up_3():
    custom = [{"payer": "x", "amount": 90, "payees": {"x": 10, "y": 30, "z": 50}}]
    assert balances(custom) == {"x": 80, "y": -30, "z": -50}


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
```

```
run_all_tests()
```

ALSO REPORTED · 1 more

Compute minimal transfers to settle group expenses | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-minimal-transfers-to-settle-group-expenses

STUDY PATH: question 24 of 197 | topic 4 of 43: A heap of what is available: pop the best, put back the changed one | 8 of 8

Template fit: pop the artist with the most songs left, hold it for one turn, then push it back with a smaller count | ND

MEDIUM | HackTheRounds | Phone Screen | 24 of 197

Playlist Shuffle No Adjacent Artists

STUDY: TALK **ALL TIME 34.9** **30 DAYS 0** **PRIORITY 127/288**

ABOUT THIS QUESTION

Topic: A heap of what is available: pop the best, put back the changed one

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #767 Reorganize String

LeetCode match: LC 767 / LC 1054

QUESTION

Given the artist of every song in a playlist, return an order of the artists in which as few neighbours as possible are by the same artist: with no repeats next to each other whenever that is possible.

Function signature

def shuffle_no_adjacent(artists: list) -> list

EXAMPLES

Example 1	
Input	`['a', 'a', 'b', 'b', 'c']`
Output	`['a', 'b', 'a', 'b', 'c']`
Why	`Two a's and two b's alternate; c fits wherever it likes.`

Example 2	
Input	`['a', 'a', 'a', 'b']`
Output	`['a', 'b', 'a', 'a']`
Why	`Three songs by 'a' cannot all be separated by a single 'b': the best is to keep two a's together at the end.`

Example 3	
Input	`['a', 'a']`
Output	`['a', 'a']`
Why	`A single artist: nothing can be done.`

CONSTRAINTS

- 1 <= songs.length <= 10^4

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(songs):
    best, best_pairs = None, INF
    for order in permutations(songs):
        pairs = 0
        for a, b in zip(order, order[1:]):
            if a.artist == b.artist:
                pairs += 1
        if pairs < best_pairs:
            best, best_pairs = order, pairs
    return best
```

TIME	SPACE
O(n! * n)	O(n)

WHY IT IS SLOW

n! orderings. A greedy that always plays the artist with the most songs left (not the one that just played), using a heap, is O(n log A).

OPTIMAL SOLUTION · Greedy with a max-heap, holding back the last artist

TIME	SPACE
O(n log A)	O(A)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The shuffle should look random, not always the same arrangement.

Break ties between equally frequent artists randomly and pick the song within the artist at random, as in the code. The result is valid but not uniform among all valid shuffles; a uniform sampler needs counting arrangements.

FOLLOW-UP 2 Songs by the same artist must be at least K apart.

Same idea with a waiting queue of length K: after playing an artist, hold them for K - 1 more songs before they are eligible again (the LeetCode 358 construction).

FOLLOW-UP 3 What is the minimum number of unavoidable repeats?

With n songs and a dominant artist with m songs, the other n - m songs separate at most n - m + 1 groups, so at least m - (n - m + 1) = 2m - n - 1 neighbouring pairs of that artist are forced (if positive).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Playlist Shuffle No Adjacent Artists
#
# Given the artist of every song in a playlist, return an
# order of the artists in which as few neighbours as
# possible are by the same artist: with no repeats next to
# each other whenever that is possible.
#
# Signature:
#   def shuffle_no_adjacent(artists: list) -> list
```

```
#
# Examples:
# ['a', 'a', 'b', 'b', 'c']  ->  ['a', 'b', 'a', 'b', 'c']
# ['a', 'a', 'a', 'b']  ->  ['a', 'b', 'a', 'a']
#
# FOLLOW-UPS (answered below, in this order):
# 1. The shuffle should look random, not always the same
# arrangement.
# 2. Songs by the same artist must be at least K apart.
# 3. What is the minimum number of unavoidable repeats?
#
# INTERVIEW NOTES
# A perfect arrangement exists exactly when the most
# frequent artist has at most ceil(n / 2) songs. Build it
# greedily: always play from the artist with the most
# songs left, except the artist who just played. A max-
# heap gives the most frequent artist; hold the previous
# artist back for one step. If it is impossible, the
# leftover songs of the dominant artist are the only
# unavoidable adjacencies.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import heapq
import random


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
def count_items(items):
    counts = {}
    for item in items:
        counts[item] = counts.get(item, 0) + 1
    return counts


def adjacent_pairs(artists):
    # how many neighbouring songs are by the same artist?
    pairs = 0
    for i in range(1, len(artists)):
        if artists[i] == artists[i - 1]:
            pairs += 1
    return pairs


# ---- main question: main function ----
# Given the artist of every song, return an order with as few neighbours as
# possible by the same artist (none at all when that is possible).
# Idea (greedy with a max-heap): keep (songs left, artist) in a heap.
# At every step play the artist with the MOST songs left, but hold that
# artist back for one step so they cannot play twice in a row; put them back
# in the heap after the next artist has played. If only the artist just
# played is left, a repeat cannot be avoided.
# Complexity: Time O(N log A) for N songs and A artists, Space O(A)
def shuffle_no_adjacent(artists):
    counts = count_items(artists)
```

```
    heap = []
    for artist in counts:
        heap.append((-counts[artist], artist))        # the most songs left first; ties alphabetical
    heapq.heapify(heap)
    order = []
    held = None                                       # the artist who just played (held back one step)
    while heap or held is not None:
        if heap:
            minus_left, artist = heapq.heappop(heap)
        else:
            minus_left, artist = held                 # only that artist is left: a forced repeat
            held = None
        order.append(artist)
        if held is not None:
            heapq.heappush(heap, held)                # the artist held back may play again now
            held = None
        if minus_left + 1 < 0:
            held = (minus_left + 1, artist)           # one song fewer left; hold this artist back
    return order


# ---- main question: tests ----
def forced_repeats(counts):
    # the fewest unavoidable neighbouring pairs (follow-up 3): the other n - m songs
    # split the biggest artist (m songs) into at most n - m + 1 groups, so at
    # least m - (n - m + 1) = 2m - n - 1 pairs touch
    total = 0
    biggest = 0
    for count in counts:
        total += count
        if count > biggest:
            biggest = count
    forced = 2 * biggest - total - 1
    if forced < 0:
        return 0
    return forced


def test_main_question():
    assert shuffle_no_adjacent(["a", "a", "b", "b", "c"]) == ["a", "b", "a", "b", "c"]
    assert shuffle_no_adjacent(["a", "a", "a", "b"]) == ["a", "b", "a", "a"]     # two a's must touch
    assert shuffle_no_adjacent(["a", "a"]) == ["a", "a"]
    assert shuffle_no_adjacent([]) == []
    random.seed(20)
    for round_number in range(150):                   # random playlists: the number of touching pairs is
the minimum
        artists = []
        for i in range(random.randint(1, 7)):
            artists.append(random.choice("ABC"))
        order = shuffle_no_adjacent(artists)
        assert sorted(order) == sorted(artists)
        counts = list(count_items(artists).values())
        assert adjacent_pairs(order) == forced_repeats(counts)


# =============================================================
# FOLLOW-UP 1: The shuffle should look random, not always the same arrangement
# =============================================================


# =============================================================
# FOLLOW-UP 2: Songs by the same artist must be at least K apart
# =============================================================
```

```
# ---- follow-up 1: helper functions ----
def release(waiting, heap):
    # the artist that has waited longest may play again (if it has songs left)
    entry = waiting.popleft()
    if entry[0] < 0:
        heapq.heappush(heap, entry)


def take_oldest(waiting, heap):
    # break the gap on purpose: the first waiting artist that still has songs plays early
    while waiting:
        entry = waiting.popleft()
        if entry[0] < 0:
            heapq.heappush(heap, entry)
            return


# ---- follow-up 1: function ----
# Answer: for songs (artist, title): break ties between equally frequent
# artists at RANDOM (a random number in the heap entry) and pick the song
# inside an artist at random. The result is valid but not uniform among all
# valid shuffles (that would need counting arrangements). For "at least K
# apart" (follow-up 2) hold an artist back for K - 1 songs in a waiting
# queue (`gap` = K; gap 2 means never twice in a row); strict=True returns
# None when the gap cannot be kept.
# Complexity: Time O(N log A), Space O(N)
def shuffle_playlist(songs, gap=2, strict=False, rng=random):
    by_artist = {}
    for artist, title in songs:
        if artist not in by_artist:
            by_artist[artist] = []
        by_artist[artist].append(title)
    for titles in by_artist.values():
        rng.shuffle(titles)                             # random song order inside an artist
    heap = []
    for artist in by_artist:
        heap.append((-len(by_artist[artist]), rng.random(), artist))
    heapq.heapify(heap)
    waiting = deque()
    order = []
    while len(order) < len(songs):
        if not heap:
            if strict:
                return None                             # everyone left is still waiting
            take_oldest(waiting, heap)
        minus_left, tie, artist = heapq.heappop(heap)
        order.append((artist, by_artist[artist].pop()))
        waiting.append((minus_left + 1, rng.random(), artist))   # waits even with no songs left: it marks
spacing
        if len(waiting) >= gap:
            release(waiting, heap)
    return order


# ---- follow-up 1: tests ----
def test_follow_up_1():
    rng = random.Random(7)
    songs = [("A", "song1"), ("A", "song2"), ("B", "song3"), ("B", "song4"), ("C", "song5")]
    mixed = shuffle_playlist(songs, rng=rng)
    assert sorted(mixed) == sorted(songs)
    artists_in_order = []
    for artist, title in mixed:
        artists_in_order.append(artist)
    assert adjacent_pairs(artists_in_order) == 0
    different = set()
```

```
    for i in range(30):
        different.add(tuple(shuffle_playlist(songs, rng=rng)))
    assert len(different) > 1                          # runs differ from each other but are all valid
    heavy = [("A", str(i)) for i in range(4)] + [("B", "x")]
    assert shuffle_playlist(heavy, strict=True, rng=rng) is None
    assert shuffle_playlist([], rng=rng) == []


# ---- follow-up 2: tests ----
def test_follow_up_2():
    rng = random.Random(8)
    trio = []
    for artist in "ABC":
        for i in range(2):
            trio.append((artist, artist + str(i)))
    far = shuffle_playlist(trio, 3, True, rng)         # at least 3 apart: three artists, two songs each
    assert sorted(far) == sorted(trio)
    for i in range(len(far)):
        for back in range(1, 3):
            if i - back >= 0:
                assert far[i - back][0] != far[i][0]


# ============================================================
# FOLLOW-UP 3: What is the minimum number of unavoidable repeats?
# ============================================================
```

```
# Answer: forced_repeats(counts) above: with n songs and a biggest artist
# with m songs, the other n - m songs split that artist into at most
# n - m + 1 groups, so at least 2m - n - 1 neighbouring pairs of that
# artist are forced (if that is positive).
# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert forced_repeats([4, 1]) == 2
    assert forced_repeats([2, 2, 1]) == 0
    assert forced_repeats([5]) == 4


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 5 OF 43 |

Recurrence on a position or a range (memo or table)

6 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

6
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Recurrence on a position or a range (memo or table) | 6 questions | tap a title to jump to it

READ FIRST: the code template for these 6 questions (tap here) >

M 25. Target Expressions from Ordered Digits **1Point3Acres * 2026-08-13 ***

M 26. Count Word Decompositions into Chemical Element Symbols **1Point3Acres * 2026-07-14 ***

M 27. Fence Painting Minimum Operations (Gemini Team) **1Point3Acres * 2026-04-27 ***

M 28. Maximize 1D deliveries within distance and minimize total distance **PracHub * 2025-12-27 ***

M 29. Find the longest palindromic subsequence using dynamic programming. **ND**

M 30. Jump Game with Profit **ND**

CODE TEMPLATE | GROUP 5 OF 43 | 6 QUESTIONS, ONE PATTERN

Recurrence on a position or a range (memo or table)

The one idea these questions share

Define best(i): the answer for everything from position i onward (or best(i, j) for a range, or a table over two sequences). Write what an empty or finished state returns, then combine the answers of the states you can jump to. Cache it, or fill a table in the right order.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from functools import lru_cache


def solve(seq, valid):
    n = len(seq)

    @lru_cache(maxsize=None)
    def best(i):                          # CHANGE 1: i is a position, or (i, j) for a range
        if i == n:
            return 1                      # CHANGE 2: what a finished state returns
        total = 0
        for j in range(i + 1, n + 1):     # CHANGE 3: which later states can you jump to
            if valid(seq[i:j]):           # CHANGE 4: can this piece be used
                total += best(j)          # CHANGE 5: combine: sum, max, min, or build lists
        return total

    return best(0)
```

2. Helper functions that usually get written next to it

```
def edit_distance(a, b):                    # the two-row table most alignment questions use
    prev = list(range(len(b) + 1))
    for i, ca in enumerate(a, 1):
        cur = [i]
        for j, cb in enumerate(b, 1):
            cur.append(prev[j - 1] if ca == cb else 1 + min(prev[j - 1], prev[j], cur[j - 1]))
        prev = cur
    return prev[-1]
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
25	Target Expressions from Ordered Digits	best(l, r) = every value reachable from digits l..r; split at m and combine the two sides with +, - and *
26	Count Word Decompositions into Chemical Element Symbols	bottom-up ways[i]: add ways[i+1] if the one-letter symbol exists and ways[i+2] if the two-letter symbol exists
27	Fence Painting Minimum Operations (Gemini Team)	range (l, r): paint one stripe at the lowest height, then solve the pieces on both sides; a sparse table finds the lowest point fast
28	Maximize 1D deliveries within distance and minimize total distance	table over (orders used, drivers used): skip an order, skip a driver, or match them when they are within D (count, then minus distance)
29	Find the longest palindromic subsequence using dynamic programming.	table over (i, j): equal ends add 2 to the inside; otherwise the better of dropping either end
30	Jump Game with Profit	best[i] = max over every j before i that can reach i (j + arr[j] >= i) of best[j] + profit[i]

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 25 of 197 | topic 5 of 43: Recurrence on a position or a range (memo or table) | 1 of 6

Template fit: best(l, r) = every value reachable from digits l..r; split at m and combine the two sides with +, - and * | * asked 68 days ago * | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 25 of 197

Target Expressions from Ordered Digits

ASKED 2026-08-13 | 68 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 36.5 | 30 DAYS 0 | PRIORITY 118/288

ABOUT THIS QUESTION

Topic: Recurrence on a position or a range (memo or table)

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 282 Expression Add Operators

Frequency: Single report | Last asked: 2026-08-13 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/target-expressions-ordered-digits

QUESTION

Given a list of digits in order and a target number, put +, - or * between the digits (never reorder them and never glue two digits into one number) and add parentheses wherever you like. Return every distinct expression that evaluates to the target.

Function signature

def expressions_for_target(digits: list, target: int) -> list

EXAMPLES

Example 1	
Input	`[2, 3, 4], 14`
Output	`['2*(3+4)', '2+3*4']`
Why	`2 * (3 + 4) = 14 and 2 + 3 * 4 = 14 are the only two ways.`

Example 2	
Input	`[1, 2, 3], 6`
Output	`['1*2*3', '1+2+3']`
Why	`Three digits can reach 6 several ways.`

Example 3	
Input	`[1, 2], 100`
Output	`[]`
Why	`No way to reach 100 with these digits.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def values(digits):
    if len(digits) == 1:
        return {digits[0]}
    result = set()
    for cut in range(1, len(digits)):
        for a in values(digits[:cut]):
            for b in values(digits[cut:]):
                result |= {a + b, a - b, a * b}
    return result
reachable = target in values(all_digits)
```

TIME	SPACE
exponential (Catalan(n) *...	**O(n)**
exponential (Catalan(n) * 3^(n-1) expressions)	recursion plus the value set

WHY IT IS SLOW

The same sub-interval of digits is recomputed from scratch for every split above it, giving Catalan-number growth. Remembering the values per interval (memoisation) makes it polynomial in the number of distinct values.

OPTIMAL SOLUTION · Interval DP: all expressions for digits[l:r]

TIME	SPACE
exponential in the number...	**O(distinct values per interval)**
exponential in the number of results; polynomial in the number of distinct values	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you handle N digits where listing every expression is too much?

Agree on the contract: if you only need to know whether a target is reachable, store just the set of values per interval (a boolean DP over (l, r, value)), which is far smaller than the set of strings. If you need one witness, store one expression string per value. Values are bounded by the digits, so the table stays practical for moderate N.

FOLLOW-UP 2 What if concatenating digits (23) or division is allowed?

Concatenation adds base cases: for every l and r the number int(digits[l:r]) is also an atom of the interval. Division adds a fourth operator with a zero check and, if exactness matters, Fraction values instead of floats.

FOLLOW-UP 3 Only a yes/no answer is needed for many digits.

Store just the set of reachable values per interval (as target_reachable does) instead of the expression strings; the strings blow up long before the value sets do. Build the expression only for the winning target by remembering one split per value.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Target Expressions from Ordered Digits
#
# Given a list of digits in order and a target number, put
# +, - or * between the digits (never reorder them and
```

```
# never glue two digits into one number) and add
# parentheses wherever you like. Return every distinct
# expression that evaluates to the target.
#
# Signature:
#   def expressions_for_target(digits: list, target: int) -> list
#
# Examples:
# [2, 3, 4], 14  ->  ['2*(3+4)', '2+3*4']
# [1, 2, 3], 6  ->  ['1*2*3', '1+2+3']
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you handle N digits where listing every
# expression is too much?
# 2. What if concatenating digits (23) or division is
# allowed?
# 3. Only a yes/no answer is needed for many digits.
#
# INTERVIEW NOTES
# Any expression over a run of digits is two smaller
# expressions joined by an operator at some split point,
# so the set of values for an interval is built from the
# sets of its two halves. Compute values bottom-up per
# interval and only build text for the target. Be explicit
# about parentheses: they come from the split structure,
# so every split is a candidate.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
PRECEDENCE = {"+": 1, "-": 1, "*": 2}
ATOM = 3                                  # a single digit never needs brackets
```

```
def needs_brackets(op, child_precedence, is_right_child):
    # a child that binds weaker than the operator needs brackets; on the RIGHT
    # of a minus even an equal one does: 5-(3-1) is not 5-3-1
    if child_precedence < PRECEDENCE[op]:
        return True
    if is_right_child and op == "-" and child_precedence == PRECEDENCE[op]:
        return True
    return False


def join(left_text, left_precedence, op, right_text, right_precedence):
    if needs_brackets(op, left_precedence, False):
        left_text = "(" + left_text + ")"
    if needs_brackets(op, right_precedence, True):
        right_text = "(" + right_text + ")"
    return left_text + op + right_text


def expressions_over(digits, left, right, memo):
    # every value that digits[left:right] can make, with every expression that makes it:
    # {value: {(text, precedence), ...}}
    if (left, right) in memo:
        return memo[(left, right)]
```

```
    result = {}
    if right - left == 1:
        result[digits[left]] = {(str(digits[left]), ATOM)}
    for middle in range(left + 1, right):                   # try every split point
        lefts = expressions_over(digits, left, middle, memo)
        rights = expressions_over(digits, middle, right, memo)
        for left_value in lefts:
            for right_value in rights:
                results_of = {"+": left_value + right_value,
                              "-": left_value - right_value,
                              "*": left_value * right_value}
                for op in results_of:
                    value = results_of[op]
                    if value not in result:
                        result[value] = set()
                    for left_text, left_precedence in lefts[left_value]:
                        for right_text, right_precedence in rights[right_value]:
                            text = join(left_text, left_precedence, op, right_text, right_precedence)
                            result[value].add((text, PRECEDENCE[op]))
    memo[(left, right)] = result
    return result
```

```
# ---- main question: main function ----
# Idea (interval DP): an expression over digits[l:r] is two smaller
# expressions over a split point joined by +, - or *. For every range
# remember, per value, the set of expressions that make it. Brackets are
# added only where precedence needs them.
# Complexity: Time exponential in the number of digits (every split and every operator), Space the same
def expressions_for_target(digits, target):
    table = expressions_over(digits, 0, len(digits), {})
    texts = set()
    if target in table:
        for text, precedence in table[target]:
            texts.add(text)
    return sorted(texts)


# ---- main question: tests ----
def test_main_question():
    answer = expressions_for_target([2, 3, 4], 14)
    assert "2*(3+4)" in answer and "2+3*4" in answer
    assert "5-(3-1)" in expressions_for_target([5, 3, 1], 3)        # subtraction needs brackets on the rig
ht
    assert expressions_for_target([2, 3, 4], 100) == []


# =====================================================
# FOLLOW-UP 1: How do you handle N digits where listing every expression is too much?
# =====================================================
# Answer: the number of expressions grows far faster than the number of
# values. Ask whether a yes/no, a count or one example is enough, and keep
# only the SET OF VALUES per range (follow-up 3).


# =====================================================
# FOLLOW-UP 2: What if concatenating digits (23) or division is allowed?
# =====================================================
# Answer: concatenation adds more "leaf" values to each range (the number
# written by digits[l:r] itself); division adds one more operator, but
# skip it when the divisor is 0 and decide how to treat remainders.


# =====================================================
```

```
# FOLLOW-UP 3: Only a yes/no answer is needed for many digits
# ============================================================
# ---- follow-up 3: function ----
```

```
# Answer: store just the set of reachable values per range instead of
# the expression strings; strings blow up long before the value sets do.
# Complexity: Time exponential in the number of digits, Space O(number of distinct values per range)
def target_reachable(digits, target):
    values = {}

    def reachable(left, right):
        if (left, right) in values:
            return values[(left, right)]
        found = set()
        if right - left == 1:
            found.add(digits[left])
        for middle in range(left + 1, right):
            for a in reachable(left, middle):
                for b in reachable(middle, right):
                    found.add(a + b)
                    found.add(a - b)
                    found.add(a * b)
        values[(left, right)] = found
        return found

    return target in reachable(0, len(digits))


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert target_reachable([2, 3, 4], 14)
    assert not target_reachable([2, 3, 4], 100)
    assert target_reachable([1] * 8, 8)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Enumerate operator placements and valid parenthesizations for three or four ordered digits.
Explore memoized dynamic-programming state definitions for the N-digit follow-up and be ready to justify one before coding.
Explain how duplicate expressions or duplicate values would be handled without discarding distinct valid output forms.

STUDY PATH: question 26 of 197 | topic 5 of 43: Recurrence on a position or a range (memo or table) | 2 of 6

Template fit: bottom-up ways[i]: add ways[i+1] if the one-letter symbol exists and ways[i+2] if the two-letter symbol exists | * asked 98 days ago * | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 26 of 197

Count Word Decompositions into Chemical Element Symbols

ASKED 2026-07-14 | 98 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE **ALL TIME 43.5** **30 DAYS 0** **PRIORITY 94/288**

ABOUT THIS QUESTION

Topic: Recurrence on a position or a range (memo or table)

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #139 Word Break

LeetCode match: LC 139 Word Break (related)

Frequency: Single report | Last asked: 2026-07-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/periodic-table-word-decomposition

QUESTION

Chemical elements have symbols of one or two letters (H, He, Si, Cs ...). Given a word, count the ways to write it as a sequence of element symbols (ignore upper and lower case). Different splits count as different ways even if they spell the same word.

Function signature

def count_decompositions(word: str) -> int

EXAMPLES

Example 1	
Input	`'Physics'`
Output	`4`
Why	`P-H-Y-Si-C-S, P-H-Y-Si-Cs, P-H-Y-S-I-C-S and P-H-Y-S-I-Cs: 4 ways.`

Example 2	
Input	`'Ho'`
Output	`2`
Why	`'Ho' is one symbol (holmium) and also H + O: 2 ways.`

Example 3	
Input	`'xqz'`
Output	`0`
Why	`No split works (x, y... cannot all be symbols).`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def ways(i):
    if i == len(word):
        return 1
    total = 0
    if word[i] in SYMBOLS:
        total += ways(i + 1)
    if word[i:i + 2] in SYMBOLS:
        total += ways(i + 2)
    return total
```

TIME	SPACE
exponential (Fibonacci gr... exponential (Fibonacci growth)	**O(n)**

WHY IT IS SLOW

The same suffix is reached by many different splits, so the calls grow like the Fibonacci numbers. Computing ways from the end of the word once per position is O(n).

OPTIMAL SOLUTION · Suffix-count DP

TIME	SPACE
O(n)	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How would you list the decompositions, not just count them?

Keep the same suffix recursion but return the list of splits instead of a number, memoized by start index. The count version is O(n); the list version is output-sensitive because there can be exponentially many splits, so agree on whether the interviewer wants a count, one witness, or all of them.

FOLLOW-UP 2 What if the dictionary is large and symbols can be long?

Put the dictionary in a trie (or a set plus the maximum word length L) and only look at substrings up to length L from each index. That makes the DP O(n * L) instead of O(n^2), which is the standard Word Break optimization.

FOLLOW-UP 3 List the decompositions instead of counting them.

Same recursion, but carry the parts chosen so far and record them at the end. The number of results can be exponential, so ask whether a count or a list is wanted.

FOLLOW-UP 4 A larger dictionary with long entries.

Store the dictionary in a set plus the maximum word length L (only look at substrings up to L), or in a trie, so each position costs O(L) lookups instead of O(n).

FOLLOW-UP 5 Count modulo a prime, or only test whether any decomposition exists.

Replace addition by addition modulo the prime, or by logical or for a yes/no answer. The DP structure is the same.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count Word Decompositions into Chemical
# Element Symbols
#
# Chemical elements have symbols of one or two letters (H,
# He, Si, Cs ...). Given a word, count the ways to write
# it as a sequence of element symbols (ignore upper and
# lower case). Different splits count as different ways
# even if they spell the same word.
#
# Signature:
#   def count_decompositions(word: str) -> int
#
# Examples:
# 'Physics'  ->  4
# 'Ho'  ->  2
#
# FOLLOW-UPS (answered below, in this order):
# 1. How would you list the decompositions, not just count
# them?
# 2. What if the dictionary is large and symbols can be
# long?
# 3. List the decompositions instead of counting them.
# 4. A larger dictionary with long entries.
# 5. Count modulo a prime, or only test whether any
# decomposition exists.
#
# INTERVIEW NOTES
# Counting the ways to cut a string into dictionary pieces
# is Word Break with a count instead of a yes/no. Define
# ways[i] as the number of decompositions of the suffix
# starting at i; it depends only on ways[i + 1] and ways[i
# + 2], because symbols have one or two letters. Compute
# from the end so every value you need already exists.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
```

```
ELEMENTS = set("""h he li be b c n o f ne na mg al si
p s cl ar k ca sc ti v cr mn fe co ni cu zn ga ge as
se br kr rb sr y zr nb mo tc ru rh pd ag cd in sn sb
te i xe cs ba la ce pr nd pm sm eu gd tb dy ho er tm
yb lu hf ta w re os ir pt au hg tl pb bi po at rn fr
ra ac th pa u np pu am cm bk cf es fm md no lr rf db
sg bh hs mt ds rg cn nh fl mc lv ts og""".split())


# ---- main question: main function ----
# Idea: ways[i] = the number of ways to split the END of the word, from
# position i on. A symbol has one or two letters, so
# ways[i] = ways[i + 1] (if word[i] is a symbol)
```

```
#         + ways[i + 2] (if word[i:i + 2] is a symbol).
# An empty ending has exactly one way. `mod` is for follow-up 5.
# Complexity: Time O(n), Space O(n)
def count_decompositions(word, symbols=ELEMENTS, mod=None):
    w = word.lower()
    n = len(w)
    ways = [0] * (n + 2)
    ways[n] = 1
    for i in range(n - 1, -1, -1):
        if w[i] in symbols:
            ways[i] += ways[i + 1]
        if i + 2 <= n and w[i:i + 2] in symbols:
            ways[i] += ways[i + 2]
        if mod:
            ways[i] = ways[i] % mod
    return ways[0]


# ---- main question: tests ----
def test_main_question():
    assert count_decompositions("Physics") == 4
    assert count_decompositions("") == 1


# =========================================================
# FOLLOW-UP 1: How would you list the decompositions, not just count them?
# =========================================================


# =========================================================
# FOLLOW-UP 3: List the decompositions instead of counting them
# =========================================================
```

```
# ---- follow-up 1: function ----
# Answer: the same recursion, but build the list of parts chosen. The
# number of results can be exponential, so ask whether a count or a list
# is wanted.
# Complexity: Time O(n + number of decompositions * n), Space the same
def list_decompositions(word, symbols=ELEMENTS):
    w = word.lower()
    n = len(w)
    memo = {}

    def splits(i):
        # every way to split the ending of the word that starts at i
        if i == n:
            return [[]]
        if i in memo:
            return memo[i]
        result = []
        for size in (1, 2):
            part = w[i:i + size]
            if len(part) == size and part in symbols:
                for rest in splits(i + size):
                    result.append([part] + rest)
        memo[i] = result
        return result

    return splits(0)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    parts = list_decompositions("Physics")
    assert len(parts) == 4
```

```
    assert ["p", "h", "y", "si", "c", "s"] in parts


# ========================================================
# FOLLOW-UP 2: What if the dictionary is large and symbols can be long?
# ========================================================


# ========================================================
# FOLLOW-UP 4: A larger dictionary with long entries
# ========================================================
# ---- follow-up 2: function ----
# Answer: keep the dictionary in a set and remember the length L of
```

```
# the LONGEST entry, so each position looks only at pieces up to L
# letters (or use a trie): O(L) lookups per position instead of O(n).
# Complexity: Time O(n * L) for the longest symbol length L, Space O(n)
def count_with_max_len(word, symbols):
    longest = 0
    for symbol in symbols:
        if len(symbol) > longest:
            longest = len(symbol)
    w = word.lower()
    n = len(w)
    ways = [0] * (n + 1)
    ways[n] = 1
    for i in range(n - 1, -1, -1):
        for size in range(1, longest + 1):
            if i + size <= n and w[i:i + size] in symbols:
                ways[i] += ways[i + size]
    return ways[0]


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert count_with_max_len("Physics", ELEMENTS) == 4


# ========================================================
# FOLLOW-UP 5: Count modulo a prime, or only test whether any decomposition exists
# ========================================================
# ---- follow-up 5: function ----
# Answer: replace the addition by addition modulo the prime (the `mod`
# parameter above), or by "or" for a yes/no answer. The structure is the same.
# Complexity: Time O(n), Space O(n)
def exists_decomposition(word, symbols=ELEMENTS):
    return count_decompositions(word, symbols) > 0


# ---- follow-up 5: tests ----
def test_follow_up_5():
    assert exists_decomposition("Physics")
    assert not exists_decomposition("xq")                    # no element is "x" or "xq"
    assert count_decompositions("Physics", mod=3) == 1       # 4 % 3


# ========================================================
# RUN EVERYTHING
# ========================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_5()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement a suffix-count or prefix-count formulation and state what each state represents.
Test a word with no decomposition, one decomposition, multiple decompositions, and overlapping one- and two-letter symbols.

STUDY PATH: question 27 of 197 | topic 5 of 43: Recurrence on a position or a range (memo or table) | 3 of 6

Template fit: range (l, r): paint one stripe at the lowest height, then solve the pieces on both sides; a sparse table finds the lowest point fast | * asked 176 days ago * | frequency Single report

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 27 of 197

Fence Painting Minimum Operations (Gemini Team)

ASKED 2026-04-27 | 176 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 260/288**

ABOUT THIS QUESTION

Topic: Recurrence on a position or a range (memo or table)

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-04-27 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/fence-painting-min-ops

QUESTION

A fence has n planks, plank i is a[i] units tall. One stroke is either horizontal (paints one row of height h across a run of neighbouring planks that are all at least h tall) or vertical (paints one plank completely, bottom to top). Return the smallest number of strokes needed to paint the whole fence.

Function signature

def paint(a: list) -> int

EXAMPLES

Example 1	
Input	`[2, 2, 1, 2, 1]`
Output	`3`
Why	`Row 1 across all five planks, row 2 across the first two planks, and one vertical stroke on plank 4: 3 strokes.`

Example 2	
Input	`[1, 1, 1]`
Output	`1`
Why	`One horizontal stroke of height 1 across all three planks paints the whole fence.`

Example 3	
Input	`[5]`
Output	`1`
Why	`One tall plank: a single vertical stroke beats five horizontal ones.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def paint(l, r, base):
    if l > r:
        return 0
    m = min(height[l:r + 1])
    vertical = r - l + 1
    horizontal = m - base
    for part in split_at_height(l, r, m):
        horizontal += paint(part.l, part.r, m)
    return min(vertical, horizontal)
```

TIME	SPACE
O(n^2)	**O(n)**
worst case	recursion

WHY IT IS SLOW

Finding the minimum by scanning makes a sorted or constant fence quadratic, O(n^2). A sparse table for range-minimum gives O(n log n).

OPTIMAL SOLUTION · Same recursion with a sparse table for range minimum

TIME	SPACE
O(n log n)	**O(n log n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The horizontal brush is only W planks wide.

A band over a range needs ceil(width / W) strokes per row instead of one, so the band cost becomes (m - base) * ceil(len / W). The vertical option and the splitting are unchanged.

FOLLOW-UP 2 Return the strokes, not just the count.

Record which option won in each range and rebuild the list by walking the recursion again: horizontal bands as (row, l, r), vertical strokes as (plank).

FOLLOW-UP 3 Is it the same as largest rectangle in a histogram?

It shares the idea of splitting at the minimum (a Cartesian tree on the heights), but the objective differs: here you minimise strokes, there you maximise area. **Cost:** the fence recursion is O(n log n) with a sparse table (O(n^2) worst case with a plain scan); the histogram problem has an O(n) stack solution.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Fence Painting Minimum Operations (Gemini
# Team)
#
# A fence has n planks, plank i is a[i] units tall. One
# stroke is either horizontal (paints one row of height h
# across a run of neighbouring planks that are all at
# least h tall) or vertical (paints one plank completely,
```

```
# bottom to top). Return the smallest number of strokes
# needed to paint the whole fence.
#
# Signature:
#   def paint(a: list) -> int
#
# Examples:
# [2, 2, 1, 2, 1]  ->  3
# [1, 1, 1]  ->  1
#
# FOLLOW-UPS (answered below, in this order):
# 1. The horizontal brush is only W planks wide.
# 2. Return the strokes, not just the count.
# 3. Is it the same as largest rectangle in a histogram?
#
# INTERVIEW NOTES
# The shortest plank in a range decides the shape: a
# horizontal band along the bottom, up to that plank's
# height, is free of regret because every plank in the
# range needs it. That splits the range at the shortest
# planks, so recurse on the pieces above the band. The
# other choice for a range is to paint every plank
# vertically. Take whichever is cheaper.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def sparse_table(a):
    # table[j][i] = the index of the smallest plank among a[i : i + 2**j].
    # Built once, it answers "where is the smallest plank in a[l..r]?" in O(1).
```

```
    table = [list(range(len(a)))]
    j = 1
    while (1 << j) <= len(a):
        previous = table[j - 1]
        row = []
        for i in range(len(a) - (1 << j) + 1):
            x = previous[i]
            y = previous[i + (1 << (j - 1))]
            if a[x] <= a[y]:
                row.append(x)
            else:
                row.append(y)
        table.append(row)
        j += 1
    return table


def smallest_index(table, a, left, right):
    # the index of the smallest plank in a[left..right]: two overlapping blocks cover the range
    k = 0
    while (1 << (k + 1)) <= right - left + 1:
        k += 1
    x = table[k][left]
    y = table[k][right - (1 << k) + 1]
    if a[x] <= a[y]:
        return x
```

```
    return y


def paint_strokes(a, width=None):
    # returns (the number of strokes, the list of strokes). `width` is the
    # width of the horizontal brush in planks (None = as wide as you like).
    if len(a) == 0:
        return 0, []
    table = sparse_table(a)

    def solve(left, right, base):
        # paint planks left..right, whose lowest `base` rows are already painted
        if left > right:
            return 0, []
        low = smallest_index(table, a, left, right)
        size = right - left + 1
        per_row = 1
        if width is not None:
            per_row = (size + width - 1) // width          # a narrow brush needs several strokes per row
        band = (a[low] - base) * per_row                   # horizontal strokes across the whole range
        left_count, left_strokes = solve(left, low - 1, a[low])
        right_count, right_strokes = solve(low + 1, right, a[low])
        horizontal = band + left_count + right_count
        if size <= horizontal:                             # painting every plank upright is cheaper
            strokes = []
            for i in range(left, right + 1):
                strokes.append(("vertical", i))
            return size, strokes
        strokes = []
        for row in range(base + 1, a[low] + 1):
            strokes.append(("horizontal", row, left, right))
        return horizontal, strokes + left_strokes + right_strokes

    return solve(0, len(a) - 1, 0)


# ---- main question: main function ----
# Idea: look at the SHORTEST plank in a range. Either paint the rows up
# to its height across the whole range (one stroke per row) and then
# solve the two sides on top of that, or paint every plank of the range
# upright (one stroke per plank): take the cheaper. The sparse table
# finds the shortest plank in O(1) and every plank is the split point at
# most once, so only n ranges are visited.
# Complexity: Time O(n log n), Space O(n log n)
def paint(a, width=None):
    return paint_strokes(a, width)[0]


# ---- main question: tests ----
def test_main_question():
    assert paint([2, 2, 1, 2, 1]) == 3
    assert paint([]) == 0
    assert paint([3]) == 1
    assert paint([4, 4, 4, 4]) == 4                        # a flat fence: one stroke per row
    assert paint([5, 0, 5]) == 2                           # tall thin planks are cheaper painted upright


# ========================================================
# FOLLOW-UP 1: The horizontal brush is only W planks wide
# ========================================================
# Answer: the `width` parameter: a horizontal band over a range of
# `size` planks costs ceil(size / width) strokes per row.
# ---- follow-up 1: tests ----
def test_follow_up_1():
```

```
    assert paint([2, 2, 2, 2], width=2) == 4
    assert paint([2, 2, 2, 2], width=4) == 2


# ======================================================
# FOLLOW-UP 2: Return the strokes, not just the count
# ======================================================
# Answer: paint_strokes records which option won in each range and returns
# the strokes: horizontal bands as ("horizontal", row, left, right) and
# upright strokes as ("vertical", plank).
# ---- follow-up 2: tests ----
def test_follow_up_2():
    count, strokes = paint_strokes([2, 2, 1, 2, 1])
    assert count == len(strokes) == 3


# ======================================================
# FOLLOW-UP 3: Is it the same as largest rectangle in a histogram?
# ======================================================
# Answer: it is related but not the same. Both split at the shortest bar,
# but painting chooses between a horizontal band and upright strokes for each
# range, while the histogram problem maximises one rectangle's area.


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill The canonical largest-rectangle-in-histogram problem — the geometric setup is the same.
Practice the recursive painter pattern on small inputs by hand; mistakes around the band height (min - base) are common.
Be ready for the follow-up: "the brush has width W instead of 1" — extends to bounded horizontal width.

STUDY PATH: question 28 of 197 | topic 5 of 43: Recurrence on a position or a range (memo or table) | 4 of 6

Template fit: table over (orders used, drivers used): skip an order, skip a driver, or match them when they are within D (count, then minus distance) | * asked 297 days ago *

MEDIUM | PracHub | Technical Screen | 28 of 197

Maximize 1D deliveries within distance and minimize total distance

POSTED 2025-12-27 | 297 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 36.9 | 30 DAYS 0 | PRIORITY 116/288

ABOUT THIS QUESTION

Topic: Recurrence on a position or a range (memo or table)

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #455 Assign Cookies, LeetCode #475 Heaters

LeetCode match: No direct match; related LC 455 Assign Cookies / LC 475 Heaters (inferred from overview)

Posted on PracHub: 2025-12-27 (updated 2026-04-23)

Original: https://prachub.com/coding-questions/maximize-1d-deliveries-within-distance-and-minimize-total-distance

QUESTION

Orders and drivers are points on a line. A driver can deliver an order that is at most D away; every driver takes at most one order and every order is delivered by at most one driver. First maximize the number of deliveries; among all plans with that many deliveries, minimize the total distance driven. Return (number of deliveries, total distance).

Function signature

def deliveries(orders: list, drivers: list, D: int) -> tuple

EXAMPLES

Example 1	
Input	`[1, 5, 9], [2, 6, 20], 2`
Output	`(2, 2)`
Why	`1 is served by the driver at 2, and 5 by the driver at 6: 2 deliveries with total distance 1 + 1 = 2.`

Example 2	
Input	`[1, 5, 9], [2, 6, 20], 11`
Output	`(3, 13)`
Why	`With D = 11 the driver at 20 can also take the order at 9 (distance 11): 1 + 1 + 11 = 13 for 3 deliveries.`

Example 3	
Input	`[3, 4], [], 5`
Output	`(0, 0)`
Why	`No drivers, no deliveries.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = (0, 0)  # (matches, -distance)
def go(i, used, matches, dist):
    global best
    if i == len(orders):
        best = max(best, (matches, -dist))
        return
    go(i + 1, used, matches, dist)  # skip
    for d, x in enumerate(drivers):
        gap = abs(orders[i] - x)
        if d not in used and gap <= D:
            go(i + 1, used | {d},
               matches + 1, dist + gap)
```

TIME	SPACE
O(drivers^orders)	O(n)

WHY IT IS SLOW

Each order branches over all drivers: exponential. After sorting, a DP over (orders considered, drivers considered) that compares (matches, minus distance) is O(n * m).

OPTIMAL SOLUTION · DP over sorted orders and drivers, maximising (matches, -distance)

TIME	SPACE
O(n * m)	O(n * m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only the number of deliveries matters.

Two pointers on the sorted lists: match when within D, otherwise advance whichever is further left. O(n log n). It maximises the count (checked against exhaustive search) but not the total distance.

FOLLOW-UP 2 A driver can take several orders (capacity c).

Replicate each driver c times (as separate sorted points) and use the same DP, or treat it as min-cost flow with capacities.

FOLLOW-UP 3 Minimise the largest single distance instead of the total.

Binary search the largest allowed distance D and test whether all orders can be served with the two-pointer matching; the check is monotone in D. **Cost:** O(n log n + (n + m) log(range)): sorting plus a binary search whose check is a two-pointer pass.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Maximize 1D deliveries within distance
# and minimize total distance
#
# Orders and drivers are points on a line. A driver can
```

```
# deliver an order that is at most D away; every driver
# takes at most one order and every order is delivered by
# at most one driver. First maximize the number of
# deliveries; among all plans with that many deliveries,
# minimize the total distance driven. Return (number of
# deliveries, total distance).
#
# Signature:
#   def deliveries(orders: list, drivers: list, D: int) -> tuple
#
# Examples:
# [1, 5, 9], [2, 6, 20], 2  ->  (2, 2)
# [1, 5, 9], [2, 6, 20], 11  ->  (3, 13)
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the number of deliveries matters.
# 2. A driver can take several orders (capacity c).
# 3. Minimise the largest single distance instead of the
# total.
#
# INTERVIEW NOTES
# On a line there is always an optimal matching that does
# not cross itself, so sort both lists and match in order:
# a DP over (orders considered, drivers considered) with
# the pair (matches, total distance) compared
# lexicographically solves both objectives together. A
# greedy that matches each order to the nearest free
# driver gets the count right but not always the smallest
# total distance.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
```

```
def better(x, y):
    # the better of two (deliveries, minus distance) pairs: more deliveries
    # first, then less distance (a bigger "minus distance")
    if x >= y:
        return x
    return y


# ---- main question: main function ----
# Idea: sort the orders and the drivers (positions on a line). Let
# dp[i][j] be the best pair (number of deliveries, MINUS the total
# distance) using only the first i orders and the first j drivers. For
# the next order and driver you can skip the order, skip the driver, or
# match them if they are at most D apart. Pairs compare lexicographically,
# so more deliveries wins and then the smaller total distance.
# Complexity: Time O(m log m + k log k + m * k) for m orders and k drivers, Space O(m * k)
def deliveries(orders, drivers, D):
    o = sorted(orders)
    d = sorted(drivers)
    dp = []
    for i in range(len(o) + 1):
        dp.append([(0, 0)] * (len(d) + 1))
    for i in range(1, len(o) + 1):
```

```
        for j in range(1, len(d) + 1):
            best = better(dp[i - 1][j], dp[i][j - 1])          # skip the order, or skip the driver
            gap = o[i - 1] - d[j - 1]
            if gap < 0:
                gap = -gap
            if gap <= D:                                       # or match them
                count, minus_distance = dp[i - 1][j - 1]
                best = better(best, (count + 1, minus_distance - gap))
            dp[i][j] = best
    count, minus_distance = dp[len(o)][len(d)]
    return count, -minus_distance


# ---- main question: tests ----
def test_main_question():
    assert deliveries([1, 5], [2, 6], 1) == (2, 2)
    assert deliveries([1, 3], [2, 4], 1) == (2, 2)
    assert deliveries([], [1], 5) == (0, 0)
    assert deliveries([1, 2], [2, 3], 0) == (1, 0)            # a limit of 0 needs identical positions


# =======================================================
```

```
# FOLLOW-UP 1: Only the number of deliveries matters
# =======================================================
# ---- follow-up 1: function ----
# Answer: two pointers on the sorted lists: match when the two are
# within D, otherwise move on the one that is further left. O(n log n).
# It maximises the count but not the total distance.
# Complexity: Time O(m log m + k log k), Space O(m + k)
def max_count(orders, drivers, D):
    o = sorted(orders)
    d = sorted(drivers)
    i = 0
    j = 0
    count = 0
    while i < len(o) and j < len(d):
        gap = o[i] - d[j]
        if gap < 0:
            gap = -gap
        if gap <= D:
            count += 1
            i += 1
            j += 1
        elif o[i] < d[j]:
            i += 1                                             # this order is too far left for every drive
r
        else:
            j += 1
    return count


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert max_count([1, 3], [2, 4], 1) == 2
    assert max_count([1, 10], [5], 2) == 0


# =======================================================
# FOLLOW-UP 2: A driver can take several orders (capacity c)
# =======================================================
# ---- follow-up 2: function ----
# Answer: list every driver `cap` times (as separate points) and use the
# same DP; or model it as min-cost flow with capacities.
# Complexity: Time O(m * k * cap), Space O(m * k * cap)
```

```
def with_capacity(orders, drivers, D, cap):
    repeated = []
    for driver in drivers:
        for copy in range(cap):
            repeated.append(driver)
    return deliveries(orders, repeated, D)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert with_capacity([1, 2, 3], [2], 1, 3) == (3, 2)


# ============================================================
# FOLLOW-UP 3: Minimise the largest single distance instead of the total
# ============================================================
# ---- follow-up 3: function ----
# Answer: binary search the largest allowed distance D: "can everyone
# who can be served be served with limit D?" is false for small D and true
# from some D on, and the two-pointer count answers it.
# Complexity: Time O(log(range) * (m log m + k log k)), Space O(m + k)
def smallest_limit(orders, drivers):
    target = min(len(orders), len(drivers))
    low = 0
    high = 10 ** 9
    while low < high:
        mid = (low + high) // 2
        if max_count(orders, drivers, mid) >= target:
            high = mid
        else:
            low = mid + 1
    return low


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert smallest_limit([1, 5], [2, 9]) == 4


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(people: list[int], shops: list[int], D: int) -> tuple[int, int]:
```

Sample call

```
solution([1, 7, 10], [2, 6, 12], 2)
```

Test cases

Input	Expected
`([1, 7, 10], [2, 6, 12], 2)`	`(3, 4)`
`([0, 4], [2, 5], 3)`	`(2, 3)`

STUDY PATH: question 29 of 197 | topic 5 of 43: Recurrence on a position or a range (memo or table) | 5 of 6

Template fit: table over (i, j): equal ends add 2 to the inside; otherwise the better of dropping either end | ND

MEDIUM | IN APP LC 516, 5 | Exponent | 29 of 197

★ TOP 75: rank 45 in the Medium and Hard priority list

★ Find the longest palindromic subsequence using dynamic programming.

ASKED (ANY CO.) ~7 MONTHS AGO | OLDER

STUDY: TYPE | ALL TIME 69.4 | 30 DAYS 0 | PRIORITY 32/288

ABOUT THIS QUESTION

Topic: Recurrence on a position or a range (memo or table)

Optimal answer from: your app's interview approach (LeetCode #516)

LeetCode match: LC 516 Longest Palindromic Subsequence (title) / LC 5 Longest Palindromic Substring (statement)

Last asked (any company): 7 months ago, about 2026-03 (Exponent shows only a rough age) | +37 interviews

Original: https://www.aced.io/questions/1169/longest-palindromic-subsequence-dynamic-programming

QUESTION

Given a string s, return the length of its longest palindromic subsequence: the longest sequence of letters of s, taken in order but not necessarily next to each other, that reads the same forwards and backwards.

Function signature

def lps_length(s: str) -> int

EXAMPLES

Example 1	
Input	`'bbbab'`
Output	`4`
Why	`'bbbb' is a subsequence of 'bbbab' and a palindrome.`

Example 2	
Input	`'cbbd'`
Output	`2`
Why	`'bb' (the two b's) is the longest.`

Example 3	
Input	`'a'`
Output	`1`
Why	`A single letter.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def lps(i, j):
    if i > j:
        return 0
    if i == j:
        return 1
    if s[i] == s[j]:
        return 2 + lps(i + 1, j - 1)
    skip_left = lps(i + 1, j)
    skip_right = lps(i, j - 1)
    return max(skip_left, skip_right)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Two branches per step and the same (i, j) range recurs: O(2^n). A table over all ranges is O(n^2) time, with a rolling row for O(n) memory.

OPTIMAL SOLUTION · Interval DP with a rolling row

TIME	SPACE
O(n^2)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the subsequence itself.

Keep the full table and walk from (0, n - 1): equal ends are taken (both go into the answer), otherwise move toward the side with the larger value. O(n^2) memory is needed for the walk back.

FOLLOW-UP 2 Minimum number of insertions (or deletions) to make the string a palindrome.

It is n - LPS(s): keep the longest palindromic subsequence and add (or remove) the other characters.

FOLLOW-UP 3 Longest palindromic substring instead (contiguous).

Expand around each of the 2n - 1 centres in O(n^2), or use Manacher's algorithm in O(n). A subsequence DP is not needed.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find the longest palindromic subsequence
# using dynamic programming.
#
# Given a string s, return the length of its longest
# palindromic subsequence: the longest sequence of letters
# of s, taken in order but not necessarily next to each
# other, that reads the same forwards and backwards.
#
# Signature:
#   def lps_length(s: str) -> int
#
```

```
# Examples:
# 'bbbab'  ->  4
# 'cbbd'  ->  2
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subsequence itself.
# 2. Minimum number of insertions (or deletions) to make
# the string a palindrome.
# 3. Longest palindromic substring instead (contiguous).
#
# INTERVIEW NOTES
# Look at the two ends of a range. If they are equal they
# both belong to the palindrome and you add 2 to the
# answer for the inside; if not, one of them is dropped,
# so take the better of dropping the left or the right
# end. The state is a range (i, j), giving an O(n^2) table
# filled from short ranges to long ones.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def lps_table(s):
    # dp[i][j] = the length of the longest palindromic SUBSEQUENCE of s[i..j].
```

```
    # Equal end letters are both used (+2); otherwise drop one end.
    n = len(s)
    dp = []
    for i in range(n):
        dp.append([0] * n)
    for i in range(n - 1, -1, -1):
        dp[i][i] = 1
        for j in range(i + 1, n):
            if s[i] == s[j]:
                dp[i][j] = dp[i + 1][j - 1] + 2
            elif dp[i + 1][j] >= dp[i][j - 1]:
                dp[i][j] = dp[i + 1][j]
            else:
                dp[i][j] = dp[i][j - 1]
    return dp


# ---- main question: main function ----
# Idea (interval DP): dp[i][j] is the longest palindromic subsequence of
# s[i..j]. Fill i from the end of the string to the start and j from i + 1
# up. Answer: dp[0][n - 1].
# Complexity: Time O(n^2), Space O(n^2)
def lps_length(s):
    if not s:
        return 0
    return lps_table(s)[0][len(s) - 1]


# ---- main question: tests ----
def test_main_question():
```

```
    assert lps_length("bbbab") == 4
    assert lps_length("cbbd") == 2
    assert lps_length("") == 0


# ====================================================
# FOLLOW-UP 1: Return the subsequence itself
# ====================================================
# ---- follow-up 1: function ----
# Answer: keep the whole table and walk from (0, n - 1): equal end letters
# are both taken (they go into the answer); otherwise move towards the side
# with the larger value. The walk back needs O(n^2) memory.
# Complexity: Time O(n^2), Space O(n^2)
def lps_string(s):
    if not s:
```

```
        return ""
    dp = lps_table(s)
    i = 0
    j = len(s) - 1
    left_half = ""
    middle = ""
    while i <= j:
        if i == j:
            middle = s[i]
            break
        if s[i] == s[j]:
            left_half += s[i]
            i += 1
            j -= 1
        elif dp[i + 1][j] >= dp[i][j - 1]:
            i += 1
        else:
            j -= 1
    return left_half + middle + left_half[::-1]


# ---- follow-up 1: tests ----
def is_subsequence(small, big):
    # do the letters of `small` appear in `big` in the same order?
    position = 0
    for ch in big:
        if position < len(small) and ch == small[position]:
            position += 1
    return position == len(small)


def test_follow_up_1():
    assert lps_string("bbbab") == "bbbb"
    assert lps_string("a") == "a"
    random.seed(4)
    for round_number in range(300):
        s = ""
        for i in range(random.randint(0, 10)):
            s += random.choice("abc")
        found = lps_string(s)
        assert is_subsequence(found, s)                    # the answer is really a subsequence of s
        assert found == found[::-1]                        # and a palindrome
        assert len(found) == lps_length(s)
```

```
# ====================================================
# FOLLOW-UP 2: Minimum number of insertions (or deletions) to make the string a palindrome
# ====================================================
# ---- follow-up 2: function ----
```

```python
# Answer: keep the longest palindromic subsequence and fix everything
# outside it: n - LPS(s) insertions (or deletions).
# Complexity: Time O(n^2), Space O(n^2)
def min_insertions(s):
    return len(s) - lps_length(s)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert min_insertions("abca") == 1
    assert min_insertions("race") == 3
    assert min_insertions("aba") == 0


# ======================================================
# FOLLOW-UP 3: Longest palindromic substring instead (contiguous)
# ======================================================
# ---- follow-up 3: function ----
# Answer: expand around each of the 2n - 1 centres (a letter, or the gap
# between two letters): O(n^2). Manacher's algorithm does it in O(n). No
# subsequence DP is needed.
# Complexity: Time O(n^2), Space O(1) extra
def longest_palindromic_substring(s):
    best = ""
    for centre in range(2 * len(s) - 1):
        i = centre // 2
        j = i + centre % 2
        while i >= 0 and j < len(s) and s[i] == s[j]:
            i -= 1
            j += 1
        if j - i - 1 > len(best):
            best = s[i + 1:j]
    return best


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert longest_palindromic_substring("abacc") == "aba"
    assert longest_palindromic_substring("cbbd") == "bb"
    assert longest_palindromic_substring("") == ""
```

```python
# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given a string s, determine the longest palindromic substring in s. A substring is considered a palindrome if it reads the same forward and backward. A substring is a contiguous sequence of characters within a string. The longest palindromic substring is the substring with the maximum length that satisfies this condition. Examples

OFFICIAL PAGE TEXT

```
Input: "abacc"
Output: "aba"
Explanation: "aba" is longer than "cc"
```

Our solution finds the longest palindromic substring by expanding around potential centers in the input string. For each character in the string, we consider it as the center of a palindrome and expand outwards to determine the maximum length of the palindrome centered at that position. We repeat this for both odd-length and even-length palindromes. For each character, we use a helper function expandAroundCenter that takes two indices (left and right) and expands outwards as long as the characters at these indices are equal and within the bounds of the string. We calculate the lengths of palindromes centered at the current character (odd-length palindromes) and between the current character and the next one (even-length palindromes). We track the starting and ending indices of the longest palindromic substring found so far. By comparing the lengths of the palindromes found in each iteration, we can determine the longest palindromic substring. The use of expansion around centers ensures that we examine all potential palindromes efficiently. Time Complexity: The solution has a time complexity of O(n^2), where n is the length of the input string. This is because, for each character in the string, we potentially expand outwards in both directions, resulting in quadratic time complexity in the worst case. Space Complexity: The solution uses O(1) space beyond the input string. The only extra space used is for a few variables, making it very space-efficient. NoteCheck out our lesson on the two-pointer approach to learn more about what it is and how to use it!No answers contributed by the community yet.

STUDY PATH: question 30 of 197 | topic 5 of 43: Recurrence on a position or a range (memo or table) | 6 of 6

Template fit: best[i] = max over every j before i that can reach i (j + arr[j] >= i) of best[j] + profit[i] | ND | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 30 of 197

Jump Game with Profit

STUDY: TYPE ALL TIME 56 30 DAYS 0 PRIORITY 65/288

ABOUT THIS QUESTION

Topic: Recurrence on a position or a range (memo or table)

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #55 Jump Game

LeetCode match: LC 55 Jump Game (related)

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/jump-game-with-profit

QUESTION

You stand at index 0 of an array. From index i you may jump to any index from i + 1 up to i + arr[i]. Landing on index i earns profit[i] (the start earns profit[0] too). Return the largest total profit of a path that ends exactly at the last index, or -1 if the last index cannot be reached.

Function signature

def max_profit_fast(arr: list, profit: list) -> int

EXAMPLES

Example 1	
Input	`[2, 3, 1, 1, 4], [1, 5, 2, 3, 4]`
Output	`15`
Why	`Jumping one index at a time visits (and collects) everything: 1 + 5 + 2 + 3 + 4 = 15.`

Example 2	
Input	`[1, 0, 1], [1, 1, 1]`
Output	`-1`
Why	`From index 1 you cannot jump at all (arr[1] = 0), so the last index is unreachable.`

BRUTE FORCE · Reading A: recursion over every jump length

BRUTE FORCE · PSEUDO-CODE

```
best = -INF
def go(i, total):
    global best
    if i == n - 1:
        best = max(best, total)
        return
    for step in range(1, arr[i] + 1):
        if i + step < n:
            gain = profit[i + step]
            go(i + step, total + gain)
```

BRUTE FORCE · Reading A: recursion over every jump length

TIME	SPACE
O(max jump ^ n)	O(n)

WHY IT IS SLOW

Every cell branches into up to max-jump choices, so the paths multiply (max jump to the power n). A table of the best profit per cell, filled left to right, is O(n * max jump).

BRUTE FORCE · Reading B: recursion over 'jump or skip' choices

BRUTE FORCE · PSEUDO-CODE

```
def best(i):
    if i >= n:
        return 0
    skip = best(i + 1)
    jump = 0
    if arr[i] > 0:
        jump = profit[i] + best(i + arr[i])
    return max(skip, jump)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Two branches at every cell and the same suffix is solved repeatedly: O(2^n). Computing best(i) once per index from the right is O(n).

OPTIMAL SOLUTION · Reading A: arr[i] is a maximum jump, profit[i] is gained on landing, maximize total profit to the last index

TIME	SPACE
O(n * maxJump)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Reading B: arr[i] is an exact jump; at each index you may jump (gaining profit[i]) or skip one cell for free

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Speed up reading A to near-linear time.

The inner loop asks for the max of best[j] over all j that can still reach i. Maintain a max-heap of (best[j], j + arr[j]) and pop entries whose reach is below i (lazy deletion): O(n log n). If the jump limit is the same for all cells, a monotonic deque over a sliding window makes it O(n).

FOLLOW-UP 2 What if you want to know whether the end is reachable at all (plain Jump Game)?

Greedy: scan left to right keeping the farthest reachable index; if i exceeds it you are stuck. O(n) time, O(1) space. Use the DP when profits matter, and the greedy when only reachability matters.

FOLLOW-UP 3 Only reachability, no profit (LeetCode 55).

Greedy: keep the farthest index reachable so far; if the current index is beyond it you are stuck. O(n), O(1).

FOLLOW-UP 4 Make the profit DP faster.

best[i] needs the maximum of best[j] over j that can reach i. A max-heap with lazy deletion of positions that can no longer reach i makes it O(n log n); with a constant maximum jump a monotonic deque gives O(n).

FOLLOW-UP 5 Fewest jumps (LeetCode 45).

BFS by levels or a greedy that tracks the end of the current jump range and the farthest reach, O(n).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Jump Game with Profit
#
# You stand at index 0 of an array. From index i you may
# jump to any index from i + 1 up to i + arr[i]. Landing
# on index i earns profit[i] (the start earns profit[0]
# too). Return the largest total profit of a path that
# ends exactly at the last index, or -1 if the last index
# cannot be reached.
#
# Signature:
#   def max_profit_fast(arr: list, profit: list) -> int
#
# Examples:
# [2, 3, 1, 1, 4], [1, 5, 2, 3, 4]  ->  15
# [1, 0, 1], [1, 1, 1]  ->  -1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Speed up reading A to near-linear time.
# 2. What if you want to know whether the end is reachable
# at all (plain Jump Game)?
# 3. Only reachability, no profit (LeetCode 55).
# 4. Make the profit DP faster.
# 5. Fewest jumps (LeetCode 45).
#
# INTERVIEW NOTES
# This statement is underspecified, so pin it down first
# (cap or exact jump, when profit is collected, can you
# skip, what is the goal). The reading with a maximum jump
# and profit on landing is a DP over positions: the best
# profit at i is its own profit plus the best of all
# positions that can reach it. The reading with an exact
# jump and a free skip is a one-line DP from the right.

# ====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
```

```
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq

NEG = float("-inf")
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Reading A: arr[i] is the farthest you may jump from cell i, profit[i]
# is gained when you land on i; maximise the total profit on the way to
# the last cell (-1 if it cannot be reached).
# Idea: best[i] = the largest profit possible when standing on i. You can
# land on i from any earlier cell j that reaches it (j + arr[j] >= i):
# best[i] = profit[i] + the largest best[j] among those. Keep the
# candidates in a MAX-heap (push negatives) and drop a candidate for good
# as soon as it can no longer reach the current cell.
# Complexity: Time O(n log n), Space O(n)
def max_profit_fast(arr, profit):
    n = len(arr)
    best = [NEG] * n
    best[0] = profit[0]
    heap = [(-best[0], arr[0])]                 # (minus the profit, how far it reaches)
    for i in range(1, n):
        while heap and heap[0][1] < i:
            heapq.heappop(heap)                 # this cell can no longer reach i or anything after
        if heap:
            best[i] = -heap[0][0] + profit[i]
            heapq.heappush(heap, (-best[i], i + arr[i]))
    if best[n - 1] == NEG:
        return -1
    return best[n - 1]


# ---- main question: tests ----
def test_main_question():
    assert max_profit_fast([2, 3, 1, 1, 4], [1, 5, 2, 3, 4]) == 15
    assert max_profit_fast([1, 0, 1], [1, 1, 1]) == -1


# ============================================================
# FOLLOW-UP 1: Speed up reading A to near-linear time
# ============================================================


# ============================================================
# FOLLOW-UP 4: Make the profit DP faster
# ============================================================
# Answer: the heap above is the speed-up: instead of looking at every
# earlier cell (O(n^2)), the best usable candidate is always at the top of
```

```
# the heap, so the whole thing is O(n log n).


# ============================================================
# FOLLOW-UP 2: What if you want to know whether the end is reachable at all (plain Jump Game)?
# ============================================================


# ============================================================
# FOLLOW-UP 3: Only reachability, no profit (LeetCode 55)
# ============================================================
```

```python

# ---- follow-up 3: function ----
# Answer: greedy. Keep the farthest index reachable so far; if the
# current index is beyond it, you are stuck. O(n) time, O(1) space.
# Complexity: Time O(n), Space O(1)
def can_reach_end(arr):
    farthest = 0
    for i in range(len(arr)):
        if i > farthest:
            return False
        if i + arr[i] > farthest:
            farthest = i + arr[i]
    return True


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert can_reach_end([2, 3, 1, 1, 4])
    assert not can_reach_end([1, 0, 1])


# ============================================================
# FOLLOW-UP 5: Fewest jumps (LeetCode 45)
# ============================================================
# ---- follow-up 5: function ----
# Answer: a BFS by levels, done greedily: track where the current jump
# can end (`end`) and how far any cell seen so far can reach
# (`farthest`). When we get to `end` we must jump: count it and extend
# `end` to `farthest`. O(n).
# Complexity: Time O(n), Space O(1)
def fewest_jumps(arr):
    jumps = 0
    end = 0
    farthest = 0
    for i in range(len(arr) - 1):
        if i + arr[i] > farthest:
            farthest = i + arr[i]
        if i == end:
            jumps += 1
            end = farthest
    return jumps


# ---- follow-up 5: tests ----
def test_follow_up_5():
    assert fewest_jumps([2, 3, 1, 1, 4]) == 2
    assert fewest_jumps([0]) == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    test_follow_up_5()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill the reachability-jump, min-jumps, and coin-change family together so the DP shape is automatic.
Practice clarifying "can I skip without consuming a turn" before coding.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 6 OF 43 |

Best-first search with a heap: Dijkstra and its minimax twin

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

5
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Best-first search with a heap: Dijkstra and its minimax twin | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M 31. Path Minimizing the Maximum Cell (Swim in Rising Water) **1Point3Acres * 2026-09-29 ***

M 32. Nearest Favorite City by Travel Time **ColorStack * 2025-10-17 ***

M 33. Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and... **ND**

M 34. Find the shortest path in a graph. **ND**

M 35. Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the mini... **ND**

CODE TEMPLATE | GROUP 6 OF 43 | 5 QUESTIONS, ONE PATTERN

Best-first search with a heap: Dijkstra and its minimax twin

The one idea these questions share

Keep a heap of (cost so far, node). Pop the cheapest, skip stale entries, and push any neighbour you improved. Dijkstra adds the edge weight to the cost; the minimax questions take the maximum of the cost and the edge instead. Everything else is the same loop.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
import heapq


def best_first(start, neighbors, combine, goal=None):
    best = {start: 0}
    heap = [(0, start)]
    while heap:
        cost, u = heapq.heappop(heap)
        if cost > best[u]:                      # a stale heap entry
            continue
        if u == goal:                           # CHANGE 1: stop at the goal
            return cost
        for v, w in neighbors(u):               # CHANGE 2: edges (or grid steps)
            nxt = combine(cost, w)              # CHANGE 3: cost + w, or max(cost, w)
            if nxt < best.get(v, float("inf")):
                best[v] = nxt
                heapq.heappush(heap, (nxt, v))
    return best
```

2. Helper functions that usually get written next to it

```
def grid_neighbors(grid):                   # (cell, cost of stepping onto it) for minimax grids
    rows, cols = len(grid), len(grid[0])
    def step(cell):
        r, c = cell
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols:
                yield (nr, nc), grid[nr][nc]
    return step


def reverse_graph(adj):                     # for runs from the destination backwards
    rev = {u: [] for u in adj}
    for u, edges in adj.items():
        for v, w in edges:
            rev[v].append((u, w))
    return rev
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
31	Path Minimizing the Maximum Cell (Swim in Rising Water)	the same minimax loop with a given start and end cell (Swim in Rising Water)
32	Nearest Favorite City by Travel Time	combine = cost + w; stop when a favourite is popped (keep popping equal distances to break ties on the smaller index)

#	Question	What you change in the template
33	Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time.	cost = arrival time; neighbors only offers trains that leave at or after the current time; combine = the train's arrival time
34	Find the shortest path in a graph.	combine = cost + w; also store parent[v]; stop at the target and rebuild the path
35	Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat.	maximise the minimum distance to the cat along the path: push negated values, combine = min(value, cat distance of the cell)

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 31 of 197 | topic 6 of 43: Best-first search with a heap: Dijkstra and its minimax twin | 1 of 5

Template fit: the same minimax loop with a given start and end cell (Swim in Rising Water) | * asked 21 days ago * | frequency Low

MEDIUM | IN APP LC 778 | 1Point3Acres | onsite-coding | 31 of 197

Path Minimizing the Maximum Cell (Swim in Rising Water)

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | ALL TIME 40 | 30 DAYS 33 | PRIORITY 62/288

ABOUT THIS QUESTION

Topic: Best-first search with a heap: Dijkstra and its minimax twin

Optimal answer from: your app's interview approach (LeetCode #778)

LeetCode match: LC 778 Swim in Rising Water

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/path-min-max-grid-dijkstra

QUESTION

You get a grid of heights, a start cell and an end cell, and you may move to any of the four neighbouring cells. The cost of a path is the largest height on it (start and end included). Return the smallest cost over all paths from start to end.

Function signature

def min_max_path(g: list, start: tuple, end: tuple) -> int

EXAMPLES

Example 1	
Input	`[[0, 2], [1, 3]], (0, 0), (1, 1)`
Output	`3`
Why	`Both routes pass the cell of height 3 at the end, so the best cost is 3.`

Example 2	
Input	`[[0, 1, 2, 3, 4], [24, 23, 22, 21, 5], [12, 13, 14, 15, 16], [11, 17, 18, 19, 20], [10, 9, 8, 7, 6]], (0, 0), (4, 4)`
Output	`16`
Why	`A 5x5 board: the best route has to climb to 16 somewhere, and no route stays lower.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid, start, end):
    for t in sorted(set(all_values(grid))):
        if bfs_reaches(grid, start, end, t):
            return t  # smallest limit
```

TIME	SPACE
O(V^2)	O(V)

WHY IT IS SLOW

One full BFS per distinct value, O(V^2). Dijkstra on the running maximum finds the answer in one pass, O(V log V).

OPTIMAL SOLUTION · Dijkstra on the running maximum

TIME	SPACE
O(V log V)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Maximise the minimum distance to a cat instead.

Same bottleneck shape with the objective flipped: score each cell by its Manhattan distance to the cat, run Dijkstra with a max-heap on the running minimum, and skip water. The same monotonicity argument holds (extending a path can only lower its minimum).

FOLLOW-UP 2 Why is Dijkstra correct for a max-along-path cost?

The cost max(path) never decreases when a path is extended, so a vertex's cheapest cost cannot be improved by going through a vertex that is popped later. That is the only property Dijkstra needs from its cost function (monotone, non-negative extension).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Path Minimizing the Maximum Cell (Swim in
# Rising Water)
#
# You get a grid of heights, a start cell and an end cell,
# and you may move to any of the four neighbouring cells.
# The cost of a path is the largest height on it (start
# and end included). Return the smallest cost over all
# paths from start to end.
#
# Signature:
#   def min_max_path(g: list, start: tuple, end: tuple) -> int
#
# Examples:
# [[0, 2], [1, 3]], (0, 0), (1, 1)  ->  3
# [[0, 1, 2, 3, 4], [24, 23, 22, 21, 5], [12, 13, 14, 15,
#     16], [11, 17, 18, 19, 20], [10, 9, 8, 7, 6]], (0,
#     0), (4, 4)  ->  16
#
# FOLLOW-UPS (answered below, in this order):
# 1. Maximise the minimum distance to a cat instead.
# 2. Why is Dijkstra correct for a max-along-path cost?
#
# INTERVIEW NOTES
# The cost of a path is its largest cell, not the sum.
# That cost can only grow as the path grows, so Dijkstra
# still works if you replace 'distance + weight' by
# max(distance, weight): the first time a cell leaves the
# heap its cost is final. The cat version flips the
# objective (maximise a minimum), which is the same shape
# with a max-heap and each cell scored by its distance to
# the cat.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
```

```
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq

INF = float("inf")


# ============================================================
```

```
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def neighbors(r, c, rows, cols):
    # the cells above, below, left and right of (r, c) that exist
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            result.append((nr, nc))
    return result


# ---- main question: main function ----
# Idea: Dijkstra on the running maximum. Heap entries are (cost so
# far, cell); taking a cell out fixes its cost; a neighbour gets
# max(cost, its own value). Stop when the end cell comes out; skip
# outdated heap entries.
# Complexity: Time O(m * n log(m * n)), Space O(m * n)
def min_max_path(g, start, end):
    rows, cols = len(g), len(g[0])
    best = {start: g[start[0]][start[1]]}
    heap = [(best[start], start)]
    while heap:
        cost, cell = heapq.heappop(heap)
        if cost > best[cell]:
            continue                            # an outdated entry
        if cell == end:
            return cost
        for nr, nc in neighbors(cell[0], cell[1], rows, cols):
            new_cost = cost
            if g[nr][nc] > new_cost:
                new_cost = g[nr][nc]            # highest cell so far, or this one
            if new_cost < best.get((nr, nc), INF):
                best[(nr, nc)] = new_cost
                heapq.heappush(heap, (new_cost, (nr, nc)))
    return None


# the same idea on a weighted graph: every edge must be below `limit`,
# and the cost of a path is its heaviest edge
# Complexity: Time O(E log V), Space O(V)
def min_bottleneck_edge(graph, s, t, limit):
    # graph[u] = [(v, weight), ...]
    best = {s: 0}
```

```
    heap = [(0, s)]
    while heap:
        cost, u = heapq.heappop(heap)
        if cost > best[u]:
            continue
        if u == t:
            return cost
        for v, w in graph.get(u, []):
            if w >= limit:
```

```
                continue                          # edges at or above the limit are not allowed
            new_cost = cost
            if w > new_cost:
                new_cost = w
            if new_cost < best.get(v, INF):
                best[v] = new_cost
                heapq.heappush(heap, (new_cost, v))
    return -1                                     # no admissible path


# the related plain Dijkstra: when does a signal reach every node?
# Complexity: Time O((V + E) log V), Space O(V + E)
def network_delay(n, edges, src):
    graph = {}
    for u, v, w in edges:
        if u not in graph:
            graph[u] = []
        graph[u].append((v, w))
    dist = {src: 0}
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph.get(u, []):
            if d + w < dist.get(v, INF):
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    if len(dist) < n:
        return -1                                 # some node is never reached
    worst = 0
    for d in dist.values():
        if d > worst:
            worst = d
    return worst
```

```
# ---- main question: tests ----
def test_main_question():
    g = [[0, 1, 2, 3, 4], [24, 23, 22, 21, 5], [12, 13, 14, 15, 16],
         [11, 17, 18, 19, 20], [10, 9, 8, 7, 6]]
    assert min_max_path(g, (0, 0), (4, 0)) == 16
    assert min_max_path(g, (2, 2), (2, 2)) == 14          # start equals end: just that cell
    graph = {0: [(1, 5), (2, 9)], 1: [(2, 3)]}
    assert min_bottleneck_edge(graph, 0, 2, 10) == 5
    assert min_bottleneck_edge(graph, 0, 2, 5) == -1
    chain = [(0, 1, 2), (1, 2, 3)]
    assert network_delay(3, chain, 0) == 5
    assert network_delay(3, [(0, 1, 2)], 0) == -1


# ========================================================
# FOLLOW-UP 1: Maximise the minimum distance to a cat instead
# ========================================================
# ---- follow-up 1: function ----
# Answer: the same bottleneck shape with the goal flipped: score each
# cell by its distance to the cat, always extend the SAFEST frontier
# cell (a max-heap: push negatives), and the path is as safe as its
# closest cell. "#" is water and cannot be entered.
# Complexity: Time O(m * n log(m * n)), Space O(m * n)
def safest_path_score(grid, start, target, cat):
    rows, cols = len(grid), len(grid[0])

    def cat_distance(r, c):
        return abs(r - cat[0]) + abs(c - cat[1])
```

```
    best = {start: cat_distance(start[0], start[1])}
    heap = [(-best[start], start)]
    while heap:
        negative, cell = heapq.heappop(heap)
        safety = -negative
        if safety < best[cell]:
            continue
        if cell == target:
            return safety
        for nr, nc in neighbors(cell[0], cell[1], rows, cols):
            if grid[nr][nc] == "#":
                continue
            new_safety = safety
            if cat_distance(nr, nc) < new_safety:
                new_safety = cat_distance(nr, nc)      # only as safe as the closest cell
```

```
            if new_safety > best.get((nr, nc), -1):
                best[(nr, nc)] = new_safety
                heapq.heappush(heap, (-new_safety, (nr, nc)))
    return None


# ---- follow-up 1: tests ----
def test_follow_up_1():
    grid = ["...", ".#.", "..."]
    assert safest_path_score(grid, (0, 0), (2, 2), (0, 2)) == 2


# ============================================================
# FOLLOW-UP 2: Why is Dijkstra correct for a max-along-path cost?
# ============================================================
# Answer: Dijkstra needs one property: extending a path never makes it
# cheaper. For "max along the path" that is true (the max can only stay
# or grow), so the first time a cell leaves the heap its cost is final.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill the grid and weighted-graph variants back to back, stating the path-cost invariant before implementation. Practice proving correctness for a monotone max-along-path objective and test source-equals-target, disconnected graphs, and values exactly at the threshold.

STUDY PATH: question 32 of 197 | topic 6 of 43: Best-first search with a heap: Dijkstra and its minimax twin | 2 of 5

Template fit: combine = cost + w; stop when a favourite is popped (keep popping equal distances to break ties on the smaller index) | * asked 368 days ago * | frequency Single report

MEDIUM | IN APP LC 743 | ColorStack | technical interview | 32 of 197

Nearest Favorite City by Travel Time

ASKED 2025-10-17 | 368 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 37.9 | 30 DAYS 33 | PRIORITY 68/288

ABOUT THIS QUESTION

Topic: Best-first search with a heap: Dijkstra and its minimax twin

Optimal answer from: your app's interview approach (LeetCode #743)

LeetCode match: LC 743 Network Delay Time (nearest of several targets)

Frequency: Single report | Last asked: 2025-10-17 | Roles: SWE Intern

QUESTION

You are given a network of num_cities cities (0..num_cities-1) connected by weighted, undirected roads roads[i] = [u, v, w] where w > 0 is the travel time. Given a start city and a non-empty list favorites of favorite cities, return the favorite city reachable from start in the shortest total travel time. Break ties by smallest city index. If none of the favorite cities is reachable, return -1 .

Function signature

```
def nearest_favorite(num_cities: int, roads: list[list[int]],
                     start: int, favorites: list[int]) -> int:
    ...
```

EXAMPLES

Example 1	
Input	`num_cities = 5, roads = [[0,1,2],[0,2,5],[1,3,1],[2,4,1]], start = 0, favorites = [3, 4]`
Output	`3`
Why	`Shortest times from 0: to 3 = 2+1 = 3, to 4 = 5+1 = 6. City 3 is nearer.`

Example 2	
Input	`num_cities = 3, roads = [[0,1,4]], start = 0, favorites = [2]`
Output	`-1`
Why	`City 2 is disconnected from start, so no favorite is reachable.`

CONSTRAINTS

- 1 <= num_cities <= 10**5 , 0 <= len(roads) <= 2*10**5 , 1 <= w <= 10**9 .
- The interviewer required the optimal solution (Dijkstra with a priority queue).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute_nearest(n, roads, start, favorites):
    dist = [float("inf")] * n
    dist[start] = 0
    for _ in range(n_-_1):
        for u, v, w in roads:
            dist[u] = min(dist[u], dist[v] + w)
            dist[v] = min(dist[v], dist[u] + w)
    best = min((dist[f], f) for f in favorites)
    return (
        best[1]
        if best[0] < float("inf")
        else:
            -1
    )
```

TIME	SPACE
O(V * E)	O(V)

WHY IT IS SLOW

Bellman-Ford repeats the full relaxation even when most distances are final.

OPTIMAL SOLUTION · Dijkstra from the start

TIME	SPACE
O((V + E) log V)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Answer it for every possible start city in one run.

Reverse the search: run one Dijkstra seeded with every favourite at distance 0, storing for each city the pair (distance, favourite). Because roads are undirected the distance from a city to its nearest favourite is the same, and comparing the pairs lexicographically resolves ties to the smaller favourite. O((V + E) log V) total.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Nearest Favorite City by Travel Time
#
# You are given a network of num_cities cities (
# 0..num_cities-1 ) connected by weighted, undirected
# roads roads[i] = [u, v, w] where w > 0 is the travel
# time. Given a start city and a non-empty list favorites
# of favorite cities, return the favorite city reachable
# from start in the shortest total travel time. Break ties
# by smallest city index. If none of the favorite cities
# is reachable, return -1 . Function signature: def
# nearest_favorite(num_cities: int, roads:
# list[list[int]], start: int, favorites: list[int]) ->
# int: ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Answer it for every possible start city in one run.
#
```

```python

# INTERVIEW NOTES
# Run Dijkstra from the start city: cities leave the heap
# in order of increasing travel time, so the first
# favourite popped is the nearest. For the tie rule keep
# popping while the distance equals the best and take the
# smallest index. Unreachable favourites never get a
# finite distance.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq

INF = float("inf")


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def build_roads(num_cities, roads):
    # roads are two-way: [city, city, travel time]
    adj = []
    for i in range(num_cities):
```

```python
        adj.append([])
    for u, v, w in roads:
        adj[u].append((v, w))
        adj[v].append((u, w))
    return adj


# ---- main question: main function ----
# Idea: Dijkstra from the start finalises cities in order of distance,
# so the first favourite taken out is the nearest. Cities at the same
# distance: the smaller number wins, so keep going while d does not
# exceed the best distance found. Returns -1 if none is reachable.
# Complexity: Time O((V + E) log V), Space O(V + E)
def nearest_favorite(num_cities, roads, start, favorites):
    adj = build_roads(num_cities, roads)
    dist = [INF] * num_cities
    dist[start] = 0
    heap = [(0, start)]
    best_distance = None
    best_city = -1
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        if best_distance is not None and d > best_distance:
            break                                   # everything left is farther
        if u in favorites and (best_distance is None or u < best_city):
            best_distance = d
            best_city = u
        for v, w in adj[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (dist[v], v))
    return best_city
```

```
# ---- main question: tests ----
def test_main_question():
    roads = [[0, 1, 2], [0, 2, 5], [1, 3, 1], [2, 4, 1]]
    assert nearest_favorite(5, roads, 0, [3, 4]) == 3
    assert nearest_favorite(3, [[0, 1, 4]], 0, [2]) == -1


# ========================================================
# FOLLOW-UP 1: Answer it for every possible start city in one run
```

```
# ========================================================
# ---- follow-up 1: function ----
# Answer: turn the search around: ONE Dijkstra that starts from every
# favourite city at distance 0, storing for each city the pair
# (distance, which favourite). Roads are two-way, so this is the same
# distance as searching from the city.
# Complexity: Time O((V + E) log V) for ALL start cities at once, Space O(V + E)
def nearest_favorite_for_all(num_cities, roads, favorites):
    adj = build_roads(num_cities, roads)
    best = []
    for i in range(num_cities):
        best.append((INF, -1))
    heap = []
    for f in favorites:
        best[f] = (0, f)
        heapq.heappush(heap, (0, f, f))
    while heap:
        d, fav, u = heapq.heappop(heap)
        if (d, fav) > best[u]:
            continue
        for v, w in adj[u]:
            candidate = (d + w, fav)
            if candidate < best[v]:
                best[v] = candidate
                heapq.heappush(heap, (candidate[0], fav, v))
    answers = []
    for d, fav in best:
        if d < INF:
            answers.append(fav)
        else:
            answers.append(-1)
    return answers


# ---- follow-up 1: tests ----
def test_follow_up_1():
    roads = [[0, 1, 2], [0, 2, 5], [1, 3, 1], [2, 4, 1]]
    everyone = nearest_favorite_for_all(5, roads, [3, 4])
    for city in range(5):
        assert everyone[city] == nearest_favorite(5, roads, city, [3, 4])


# ========================================================
# RUN EVERYTHING
# ========================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 33 of 197 | topic 6 of 43: Best-first search with a heap: Dijkstra and its minimax twin | 3 of 5

Template fit: cost = arrival time; neighbors only offers trains that leave at or after the current time; combine = the train's arrival time | ND

MEDIUM | Exponent | 33 of 197

Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time.

ASKED (ANY CO.) ~6 MONTHS AGO | OLDER

STUDY: TYPE | ALL TIME 37.9 | 30 DAYS 33 | PRIORITY 70/288

ABOUT THIS QUESTION

Topic: Best-first search with a heap: Dijkstra and its minimax twin

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #743 Network Delay Time, LeetCode #787 Cheapest Flights Within K Stops

LeetCode match: No direct match; related LC 743 Network Delay Time, LC 787 (inferred from short statement)

Last asked (any company): 6 months ago, about 2026-04 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/6331/find-earliest-train-sequence-route

QUESTION

A schedule lists trains as (from, to, departure time, arrival time). Starting in city src at time start, find the sequence of trains that gets you to dst at the earliest possible time. You can only board a train that leaves at or after the time you arrive in its departure city. Return (arrival time, the trains you take), or None if dst cannot be reached.

Function signature

def earliest_arrival(trains: list, src, dst, start=0) -> tuple | None

EXAMPLES

Example 1	
Input	`[('A', 'B', 1, 3), ('B', 'C', 4, 6), ('A', 'C', 2, 9), ('B', 'C', 2, 5)], 'A', 'C'`
Output	`(6, [('A', 'B', 1, 3), ('B', 'C', 4, 6)])`
Why	`A -> B arrives at 3; the train B -> C that leaves at 2 is already gone, so take the one leaving at 4 and arrive at 6 (the direct A -> C train arrives later, at 9).`

Example 2	
Input	`[('A', 'B', 1, 3), ('B', 'C', 4, 6)], 'C', 'A'`
Output	`null`
Why	`No train leaves C, so A cannot be reached from C.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
best = INF
def go(city, time):
    global best
    if city == destination:
        best = min(best, time)
    for tr in trains_from(city):
        if tr.dep >= time:
            go(tr.dst, tr.arr)
```

TIME	SPACE
exponential	O(trains)

WHY IT IS SLOW

The number of train sequences can be exponential. Because waiting is free, an earlier arrival is never worse, so Dijkstra on earliest arrival times is O((T + C) log C).

OPTIMAL SOLUTION · Dijkstra on earliest arrival times, with the train sequence

TIME	SPACE
O((T + C) log C) with trains sorted by departure per city	O(T + C)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A minimum layover is required when changing trains.

Replace the boarding test dep >= t with dep >= t + layover for every city except the origin. The earliest-arrival property still holds (a later arrival never helps because the layover is the same), so Dijkstra remains valid.

FOLLOW-UP 2 Fewest transfers instead of earliest arrival.

Run BFS over (city) counting trains taken, but track the earliest arrival for each count (a state is (city, number of trains)). Or Dijkstra with the pair (transfers, arrival time) compared lexicographically.

FOLLOW-UP 3 The schedule repeats daily.

Add the day to the time (departure + 24h * k) or treat times modulo 24h and carry the day count. The same Dijkstra works on absolute times.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given a train schedule with departure
# city, arrival city, start time, end time, and duration,
# and given a source and destination, find the sequence of
# trains that gets you there at the earliest possible
# time.
#
# A schedule lists trains as (from, to, departure time,
# arrival time). Starting in city src at time start, find
# the sequence of trains that gets you to dst at the
```

```
# earliest possible time. You can only board a train that
# leaves at or after the time you arrive in its departure
# city. Return (arrival time, the trains you take), or
# None if dst cannot be reached.
#
# Signature:
#   def earliest_arrival(trains: list, src, dst, start=0) -> tuple | None
#
# Examples:
# [('A', 'B', 1, 3), ('B', 'C', 4, 6), ('A', 'C', 2, 9),
#     ('B', 'C', 2, 5)], 'A', 'C'  ->  (6, [('A', 'B', 1,
#     3), ('B', 'C', 4, 6)])
# [('A', 'B', 1, 3), ('B', 'C', 4, 6)], 'C', 'A'  ->  null
#
# FOLLOW-UPS (answered below, in this order):
# 1. A minimum layover is required when changing trains.
# 2. Fewest transfers instead of earliest arrival.
# 3. The schedule repeats daily.
#
# INTERVIEW NOTES
# Waiting is free, so arriving earlier at a city is never
# worse than arriving later: that is exactly the property
# Dijkstra needs. Keep the earliest known arrival time per
# city in a heap, and from a city at time t relax only the
# trains that depart at or after t. Remember which train
# gave each improvement to rebuild the sequence.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq
import random
```

```
DAY = 24 * 60          # times are minutes from midnight


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
# A train is a tuple (from_city, to_city, departure_time, arrival_time).
def trains_by_city(trains):
    # city -> the trains that leave from it
    table = {}
    for train in trains:
        if train[0] not in table:
            table[train[0]] = []
        table[train[0]].append(train)
    return table


# ---- main question: main function ----
# Idea: Dijkstra on arrival time. best[city] is the earliest moment you
# can be there. From a city you may board any train that leaves at or
# after your arrival (plus `layover`, follow-up 1; no layover is needed
# in the city you start in). Remember which train improved each city to
# rebuild the sequence. Returns (arrival time, trains taken) or None.
# Complexity: Time O(T log T) for T trains, Space O(T + C) for C cities
def earliest_arrival(trains, src, dst, start=0, layover=0):
    leaving = trains_by_city(trains)
```

```
    best = {src: start}
    took = {}                                   # city -> the train that got us there
    heap = [(start, src)]
    while heap:
        time, city = heapq.heappop(heap)
        if time > best[city]:
            continue                            # an outdated entry
        if city == dst:
            break
        wait = layover
        if city == src:
            wait = 0
        for train in leaving.get(city, []):
            _from, to_city, departs, arrives = train
            if departs >= time + wait and (to_city not in best or arrives < best[to_city]):
                best[to_city] = arrives
                took[to_city] = train
                heapq.heappush(heap, (arrives, to_city))
    if dst not in best:
        return None
    route = []
    city = dst
    while city != src:                          # walk back from the destination
        route.append(took[city])
        city = took[city][0]
    route.reverse()
    return best[dst], route


# ---- main question: tests ----
TRAINS = [("A", "B", 60, 120),
          ("B", "C", 130, 200),
          ("A", "C", 100, 400),
          ("B", "C", 110, 150)]


def test_main_question():
    arrival, route = earliest_arrival(TRAINS, "A", "C")
    assert arrival == 200
    assert route == [("A", "B", 60, 120), ("B", "C", 130, 200)]
    assert earliest_arrival(TRAINS, "C", "A") is None
    assert earliest_arrival(TRAINS, "A", "A") == (0, [])
    example = [("A", "B", 1, 3), ("B", "C", 4, 6), ("A", "C", 2, 9), ("B", "C", 2, 5)]
    assert earliest_arrival(example, "A", "C") == (6, [("A", "B", 1, 3), ("B", "C", 4, 6)])


# ============================================================
# FOLLOW-UP 1: A minimum layover is required when changing trains
# ============================================================
# ---- follow-up 1: tests ----
# Answer: the boarding test becomes  departs >= arrival + layover  for
# every city except where you start (the `layover` parameter above). The
# earliest-arrival idea still holds, so Dijkstra stays correct.
def test_follow_up_1():
    # a 15 minute layover rules out the 130 departure (arrive 120, 120 + 15 = 135):
    # the direct train wins
    arrival, route = earliest_arrival(TRAINS, "A", "C", layover=15)
    assert arrival == 400 and len(route) == 1


# ============================================================
# FOLLOW-UP 2: Fewest transfers instead of earliest arrival
```

```python
# ============================================================
# ---- follow-up 2: function ----
# Answer: do rounds. Round k holds, for every city, the earliest arrival
# using at most k trains (like Bellman-Ford). Stop at the first round
# that reaches the destination. (Or Dijkstra on the pair
# (number of trains, arrival time).)
# Complexity: Time O(T^2) (up to T rounds over all trains), Space O(T * C) for the stored routes
def fewest_trains(trains, src, dst, start=0, layover=0):
    best = {src: (start, [])}                  # city -> (earliest arrival, trains used)
    for round_number in range(len(trains)):
        after = dict(best)
        for train in trains:
            from_city, to_city, departs, arrives = train
            if from_city not in best:
                continue
            time, route = best[from_city]
            wait = layover
            if from_city == src:
                wait = 0
            if departs < time + wait:
                continue
            if to_city not in after or arrives < after[to_city][0]:
                after[to_city] = (arrives, route + [train])
        if dst in after:
            return after[dst]
        best = after
    return None


# ---- follow-up 2: tests ----
def test_follow_up_2():
    arrival, route = fewest_trains(TRAINS, "A", "C")
    assert len(route) == 1 and arrival == 400


# ============================================================
# FOLLOW-UP 3: The schedule repeats daily
# ============================================================
# ---- follow-up 3: function ----
# Answer: unroll the timetable into absolute times (add 24 hours for
# every day); a train that arrives after midnight has an arrival
# number smaller than its departure, so add one day to it. The same
# Dijkstra then works on absolute times.
# Complexity: Time O(days * T), Space O(days * T)
def daily(trains, days):
```

```python
    unrolled = []
    for day in range(days):
        shift = day * DAY
        for from_city, to_city, departs, arrives in trains:
            if arrives < departs:
                arrives = arrives + DAY
            unrolled.append((from_city, to_city, departs + shift, arrives + shift))
    return unrolled


# Complexity: Time O(length of the route), Space O(1)
def route_is_valid(route, src, dst, start, layover, arrival):
    # does the route start at src, end at dst, never board before it may, and end at `arrival`?
    city, time = src, start
    for from_city, to_city, departs, arrives in route:
        wait = layover
        if city == src:
            wait = 0
        if from_city != city or departs < time + wait:
            return False
```

```
        city, time = to_city, arrives
    return city == dst and time == arrival


# ---- follow-up 3: tests ----
def test_follow_up_3():
    week = daily([("A", "B", 1400, 60)], 2)
    assert week[0][3] == 1500 and week[1][2] == 2840      # the 23:20 train arrives at 01:00 next day
    late = daily([("A", "B", 600, 700)], 2)
    assert earliest_arrival(late, "A", "B")[0] == 700
    random.seed(7)
    for round_number in range(300):                         # random timetables: every answer is a legal ro
ute
        trains = []
        for k in range(random.randint(1, 9)):
            a, b = random.sample("ABCD", 2)
            departs = random.randint(0, 100)
            trains.append((a, b, departs, departs + random.randint(1, 50)))
        gap = random.choice([0, 5, 20])
        answer = earliest_arrival(trains, "A", "D", layover=gap)
        if answer is not None:
            assert route_is_valid(answer[1], "A", "D", 0, gap, answer[0])


# =====================================================
# RUN EVERYTHING
```

```
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 34 of 197 | topic 6 of 43: Best-first search with a heap: Dijkstra and its minimax twin | 4 of 5

Template fit: combine = cost + w; also store parent[v]; stop at the target and rebuild the path | ND

MEDIUM | Exponent | 34 of 197

Find the shortest path in a graph.

ASKED (ANY CO.) ~11 DAYS AGO | LAST 30 DAYS

STUDY: TYPE ALL TIME 37.9 30 DAYS 33 PRIORITY 71/288

ABOUT THIS QUESTION

Topic: Best-first search with a heap: Dijkstra and its minimax twin

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #743 Network Delay Time, LeetCode #1091 Shortest Path in Binary Matrix

LeetCode match: No direct match; related LC 743 Network Delay Time, LC 1091 (inferred from short statement)

Last asked (any company): 11 days ago, about 2026-09 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/2094/find-shortest-path-graph

QUESTION

A directed graph is given as a list of weighted edges (u, v, w) with non-negative weights. Return the total length and the nodes of a shortest path from s to t, or None if t cannot be reached.

Function signature

def dijkstra_path(edges: list, s, t) -> tuple | None

(distance, path)

EXAMPLES

Example 1	
Input	`[('A', 'B', 1), ('A', 'C', 4), ('B', 'C', 2), ('C', 'D', 1)], 'A', 'D'`
Output	`(4, ['A', 'B', 'C', 'D'])`
Why	`A -> B -> C -> D costs 1 + 2 + 1 = 4; the direct edge A -> C costs 4, which is longer overall.`

Example 2	
Input	`[('A', 'B', 1), ('B', 'C', 2), ('C', 'D', 1)], 'D', 'A'`
Output	`null`
Why	`There is no edge leading from D back to A.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, source, target):
    dist = [INF] * n
    dist[source] = 0
    for _ in range(n - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    return dist[target]
```

BRUTE FORCE

TIME	SPACE
O(V * E)	O(V)

WHY IT IS SLOW

Each round relaxes every edge: O(V * E). It is the right tool for negative weights; for non-negative weights Dijkstra's heap is O((V + E) log V), and for unweighted graphs BFS is O(V + E).

OPTIMAL SOLUTION · Dijkstra with a heap (non-negative weights), returning the path

TIME	SPACE
O((V + E) log V)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The graph is unweighted.

BFS from the source with a parent map gives a shortest path in O(V + E); Dijkstra would be correct but needlessly slow. Stop as soon as you pop the target.

FOLLOW-UP 2 Weights can be negative.

Dijkstra can give wrong answers. Use Bellman-Ford (O(V * E)); one more pass that still improves a distance proves a negative cycle reachable from the source, where shortest paths are undefined.

FOLLOW-UP 3 Weights are only 0 or 1, or the graph is huge and you know the target.

0-1 BFS uses a deque in O(V + E). With a good distance estimate use A* (Dijkstra ordered by distance plus a heuristic that never overestimates); bidirectional search roughly squares the work saved on large graphs.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find the shortest path in a graph.
#
# A directed graph is given as a list of weighted edges
# (u, v, w) with non-negative weights. Return the total
# length and the nodes of a shortest path from s to t, or
# None if t cannot be reached.
#
# Signature:
#   def dijkstra_path(edges: list, s, t) -> tuple | None      # (distance, path)
#
# Examples:
# [('A', 'B', 1), ('A', 'C', 4), ('B', 'C', 2), ('C', 'D',
#     1)], 'A', 'D'  ->  (4, ['A', 'B', 'C', 'D'])
# [('A', 'B', 1), ('B', 'C', 2), ('C', 'D', 1)], 'D', 'A'
#     ->  null
#
# FOLLOW-UPS (answered below, in this order):
# 1. The graph is unweighted.
# 2. Weights can be negative.
# 3. Weights are only 0 or 1, or the graph is huge and you
# know the target.
```

```
#
# INTERVIEW NOTES
# Ask the graph questions first: directed or undirected,
# weighted or not, can weights be negative, and do you
# want the length or the path. That choice picks the
# algorithm: unweighted means breadth-first search, non-
# negative weights mean Dijkstra with a heap, negative
# weights mean Bellman-Ford. Remember parent pointers if
# the path itself is needed.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import heapq

INF = float("inf")


# ============================================================
```

```
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def make_graph(edges):
    # node -> list of (neighbour, weight); an edge is (u, v) or (u, v, weight)
    graph = {}
    for edge in edges:
        u, v = edge[0], edge[1]
        weight = 1
        if len(edge) == 3:
            weight = edge[2]
        if u not in graph:
            graph[u] = []
        graph[u].append((v, weight))
    return graph


def walk_back(parent, node):
    # follow the parent links back to the start, then reverse
    path = []
    while node is not None:
        path.append(node)
        node = parent[node]
    path.reverse()
    return path


# ---- main question: main function ----
# Idea: keep the best known distance to every node and a min-heap of
# (distance, node). Take the closest unsettled node, relax its edges
# (try to improve each neighbour), and remember who improved whom.
# Works for weights that are not negative. Returns (distance, path).
# Complexity: Time O((V + E) log V), Space O(V + E)
def dijkstra_path(edges, s, t):
    graph = make_graph(edges)
    dist = {s: 0}
    parent = {s: None}
    heap = [(0, s)]
    while heap:
        d, u = heapq.heappop(heap)
```

```
        if d > dist[u]:
            continue                        # an outdated heap entry
        if u == t:
            return d, walk_back(parent, u)  # the first time t comes out of the heap it is final
        for v, w in graph.get(u, []):
```

```
            if d + w < dist.get(v, INF):
                dist[v] = d + w
                parent[v] = u
                heapq.heappush(heap, (d + w, v))
    return None


# ---- main question: tests ----
def test_main_question():
    edges = [(0, 1, 4), (0, 2, 1), (2, 1, 2), (1, 3, 1)]
    assert dijkstra_path(edges, 0, 3) == (4, [0, 2, 1, 3])
    assert dijkstra_path(edges, 3, 0) is None
    assert dijkstra_path(edges, 0, 0) == (0, [0])


# =====================================================
# FOLLOW-UP 1: The graph is unweighted
# =====================================================
# ---- follow-up 1: function ----
# Answer: a BFS with a parent map finds a shortest path in O(V + E);
# Dijkstra would be correct but needlessly slow.
# Complexity: Time O(V + E), Space O(V)
def bfs_path(edges, s, t):
    graph = make_graph(edges)
    parent = {s: None}
    queue = deque([s])
    while queue:
        u = queue.popleft()
        if u == t:
            path = walk_back(parent, u)
            return len(path) - 1, path
        for v, w in graph.get(u, []):
            if v not in parent:
                parent[v] = u
                queue.append(v)
    return None


# ---- follow-up 1: tests ----
def test_follow_up_1():
    edges = [(0, 1, 4), (0, 2, 1), (2, 1, 2), (1, 3, 1)]
    assert bfs_path(edges, 0, 3) == (2, [0, 1, 3])


# =====================================================
```

```
# FOLLOW-UP 2: Weights can be negative
# =====================================================
# ---- follow-up 2: function ----
# Answer: Dijkstra can give wrong answers. Use Bellman-Ford: relax
# every edge n - 1 times (O(V * E)). If one more round still improves a
# distance, a negative cycle is reachable and shortest paths do not exist.
# Complexity: Time O(V * E), Space O(V)
def bellman_ford(n, edges, s):
    dist = [INF] * n
    dist[s] = 0
    for round_number in range(n - 1):
        changed = False
        for u, v, w in edges:
```

```
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                changed = True
        if not changed:
            break
    for u, v, w in edges:
        if dist[u] + w < dist[v]:
            raise ValueError("negative cycle")
    return dist


# ---- follow-up 2: tests ----
def test_follow_up_2():
    negative = [(0, 1, 5), (1, 2, -3)]
    assert bellman_ford(4, negative, 0) == [0, 5, 2, INF]
    try:
        bellman_ford(2, [(0, 1, 1), (1, 0, -3)], 0)
        raise AssertionError("negative cycle not found")
    except ValueError:
        pass


# ============================================================
# FOLLOW-UP 3: Weights are only 0 or 1, or the graph is huge and you know the target
# ============================================================
# ---- follow-up 3: function ----
# Answer: with weights 0 and 1 use a deque (0-1 BFS) in O(V + E): a
# free edge goes to the FRONT, a cost-1 edge to the BACK. For a huge
# graph with a known target use A* (Dijkstra ordered by distance plus
# an estimate that never overestimates) or search from both ends.
# Complexity: Time O(V + E), Space O(V + E)
def zero_one_bfs(n, edges, s):
```

```
    graph = make_graph(edges)
    dist = [INF] * n
    dist[s] = 0
    dq = deque([s])
    while dq:
        u = dq.popleft()
        for v, w in graph.get(u, []):
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    return dist


# ---- follow-up 3: tests ----
def test_follow_up_3():
    cheap = [(0, 1, 1), (1, 2, 0), (0, 2, 1)]
    assert zero_one_bfs(3, cheap, 0) == [0, 1, 1]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 35 of 197 | topic 6 of 43: Best-first search with a heap: Dijkstra and its minimax twin | 5 of 5

Template fit: maximise the minimum distance to the cat along the path: push negated values, combine = min(value, cat distance of the cell) | ND

MEDIUM | Exponent | 35 of 197

Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat.

ASKED (ANY CO.) ~8 MONTHS AGO | OLDER

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 162/288

ABOUT THIS QUESTION

Topic: Best-first search with a heap: Dijkstra and its minimax twin

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #1102 Path With Maximum Minimum Value, LeetCode #1730 Shortest Path to Get Food

LeetCode match: No direct match; related LC 1102 Path With Maximum Minimum Value, LC 1730 (inferred from short statement)

Last asked (any company): 8 months ago, about 2026-02 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/5244/safest-path-maximize-minimum-distance-matrix

QUESTION

A grid contains a rat R, a piece of bread B, a cat C, walls

and open cells '.'. Move

the rat from R to B through open cells (up, down, left, right). The safety of a path is the smallest distance to the cat of any cell on it (distances count steps around the walls). Return the largest safety you can achieve, or -1 if the bread cannot be reached.

Function signature

def safest_path(grid: list) -> int

EXAMPLES

Example 1	
Input	`['R..', '...', 'C.B']`
Output	`2`
Why	`The path along the top and right side keeps at least 2 steps from the cat at the bottom left.`

Example 2	
Input	`['R#B']`
Output	`-1`
Why	`The wall blocks the only route to the bread.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(grid):
    dist = bfs_from_cat(grid)  # per cell
    best = -1
    for path in all_paths(grid, RAT, BREAD):
        cells = [dist[r][c] for r, c in path]
        closest = min(cells)
        best = max(best, closest)
    return best
```

TIME	SPACE
exponential	O(path length)

WHY IT IS SLOW

The number of simple paths grows exponentially with the grid. A max-heap search that extends the path with the best bottleneck so far (or binary search on the threshold) is O(m * n log(m * n)).

OPTIMAL SOLUTION · Multi-source BFS from the cat, then a max-heap bottleneck search

TIME	SPACE
O(m * n log(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Use binary search instead of a heap.

Binary search the safety threshold d and test with BFS whether the rat can reach the bread using only cells at least d from the cat. The test is monotone in d. O(m * n log(m + n)).

FOLLOW-UP 2 Among the safest paths, return the shortest one.

First find the best safety value d*, then run an ordinary BFS restricted to cells at distance at least d*. The result is the shortest path with maximum safety.

FOLLOW-UP 3 The cat moves one step toward the rat each turn.

Then distances depend on time, and the problem becomes a game or a time-expanded graph search; ask for the cat's movement rules before designing anything.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given a matrix with a rat, bread, cat,
# and obstacles, find a path from the rat to the bread
# that maximizes the minimum distance from cat.
#
# A grid contains a rat R, a piece of bread B, a cat C,
# walls # and open cells '.'. Move the rat from R to B
# through open cells (up, down, left, right). The safety
# of a path is the smallest distance to the cat of any
# cell on it (distances count steps around the walls).
# Return the largest safety you can achieve, or -1 if the
# bread cannot be reached.
```

```
#
# Signature:
#   def safest_path(grid: list) -> int
#
# Examples:
# ['R..', '...', 'C.B']  ->  2
# ['R#B']  ->  -1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Use binary search instead of a heap.
# 2. Among the safest paths, return the shortest one.
# 3. The cat moves one step toward the rat each turn.
#
# INTERVIEW NOTES
# Two separate ideas. First, how far every cell is from
# the cat: that is one BFS from the cat (all cats at
# once). Second, choose the path whose worst cell is as
# far as possible: this is a 'bottleneck' (maximin) path.
# A max-heap search that always extends the path with the
# best bottleneck so far (Dijkstra with min instead of
# sum) solves it; binary search on the threshold plus a
# reachability check also works.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import heapq

INF = 10 ** 9
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def open_neighbors(r, c, grid):
    # the cells beside (r, c) that exist and are not an obstacle ("#")
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < len(grid) and 0 <= nc < len(grid[0]):
            if grid[nr][nc] != "#":
                result.append((nr, nc))
    return result


def find(grid, letter):
    # where is the letter ("R" rat, "B" bread, "C" cat)?
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == letter:
                return (r, c)
    return None


def cat_distances(grid):
    # one BFS from the cat gives every cell's distance to the cat
    dist = []
    for row in grid:
```

```
        dist.append([INF] * len(row))
    cat = find(grid, "C")
    if cat is None:
        return dist
    dist[cat[0]][cat[1]] = 0
    queue = deque([cat])
    while queue:
        r, c = queue.popleft()
        for nr, nc in open_neighbors(r, c, grid):
            if dist[nr][nc] == INF:
                dist[nr][nc] = dist[r][c] + 1
                queue.append((nr, nc))
    return dist
```

```
# ---- main question: main function ----
# Idea: a path is only as safe as its cell closest to the cat. Always
# extend the safest frontier cell (like Dijkstra, but with "max of the
# minimum" instead of "min of the sum"; the heap is a max-heap, so we
# push negative values). Returns that safety, or None if no path exists.
# Complexity: Time O(m * n log(m * n)), Space O(m * n)
def safest_path(grid):
    dist = cat_distances(grid)
    start = find(grid, "R")
    goal = find(grid, "B")
    best = {start: dist[start[0]][start[1]]}
    heap = [(-best[start], start)]
    while heap:
        negative, cell = heapq.heappop(heap)
        safety = -negative
        if cell == goal:
            return safety
        if safety < best[cell]:
            continue                                    # an outdated entry
        for nr, nc in open_neighbors(cell[0], cell[1], grid):
            new_safety = safety
            if dist[nr][nc] < new_safety:
                new_safety = dist[nr][nc]               # the path is only as safe as its closest cell
            if new_safety > best.get((nr, nc), -1):
                best[(nr, nc)] = new_safety
                heapq.heappush(heap, (-new_safety, (nr, nc)))
    return None


# ---- main question: tests ----
GRID = ["R.....",
        ".###..",
        "...C.B"]


def test_main_question():
    assert cat_distances(GRID)[0][0] == 5
    assert safest_path(GRID) == 2                       # the bread itself is 2 steps from the cat
    assert safest_path(["R#B"]) is None                 # walled in
    assert safest_path(["R.B"]) == INF                  # no cat at all: every path is perfectly safe


# ========================================================
# FOLLOW-UP 1: Use binary search instead of a heap
# ========================================================
```

```
# ---- follow-up 1: helper functions ----
def shortest_path_at_least(grid, dist, d):
    # BFS using only cells at least d steps from the cat; returns the
    # shortest such path (a list of cells) or None
```

```
    start = find(grid, "R")
    goal = find(grid, "B")
    if dist[start[0]][start[1]] < d:
        return None
    parent = {start: None}
    queue = deque([start])
    while queue:
        cell = queue.popleft()
        if cell == goal:
            path = []
            while cell is not None:
                path.append(cell)
                cell = parent[cell]
            path.reverse()
            return path
        for nxt in open_neighbors(cell[0], cell[1], grid):
            if nxt not in parent and dist[nxt[0]][nxt[1]] >= d:
                parent[nxt] = cell
                queue.append(nxt)
    return None


# ---- follow-up 1: function ----
# Answer: binary search the safety d; for each d ask a BFS whether the
# rat can reach the bread using only cells at least d from the cat.
# The answer "yes" holds for small d and "no" for big d, so binary
# search works: O(m * n * log(m + n)).
# Complexity: Time O(m * n log(m * n)) (a BFS for each of about log(m * n) lengths), Space O(m * n)
def safest_by_search(grid):
    dist = cat_distances(grid)
    if shortest_path_at_least(grid, dist, 0) is None:
        return None
    low, high = 0, INF
    while low < high:
        mid = (low + high + 1) // 2
        if shortest_path_at_least(grid, dist, mid) is not None:
            low = mid
        else:
            high = mid - 1
    return low
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert safest_by_search(GRID) == 2
    assert safest_by_search(["R#B"]) is None


# ========================================================
# FOLLOW-UP 2: Among the safest paths, return the shortest one
# ========================================================
# ---- follow-up 2: function ----
# Answer: first find the best safety, then run an ordinary BFS that is
# only allowed on cells at least that far from the cat. That is the
# shortest of the safest paths.
# Complexity: Time O(m * n log(m * n)), Space O(m * n)
def shortest_safest_path(grid):
    best = safest_path(grid)
    if best is None:
        return None
    return shortest_path_at_least(grid, cat_distances(grid), best)


# ---- follow-up 2: tests ----
def test_follow_up_2():
```

```
    path = shortest_safest_path(GRID)
    assert path[0] == (0, 0) and path[-1] == (2, 5)
    dist = cat_distances(GRID)
    closest = INF
    for r, c in path:
        if dist[r][c] < closest:
            closest = dist[r][c]
    assert closest == 2


# ============================================================
# FOLLOW-UP 3: The cat moves one step toward the rat each turn
# ============================================================
# Answer: now a cell's danger depends on the turn. Search over states
# (cell, turn): the cat can be at most `turn` steps away from where it
# started, so a cell is safe at turn t only if its distance to the cat
# is greater than t.


# ============================================================
# RUN EVERYTHING
```

```
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 7 OF 43 |

Backtracking: choose, recurse, undo

8 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

8
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Backtracking: choose, recurse, undo | 8 questions | tap a title to jump to it

READ FIRST: the code template for these 8 questions (tap here) >

M	36. Mahjong Winning Hand Detection (14 Tiles)	1Point3Acres * 2026-10-09 *
M	37. Count Prefix-Matching Simple Paths in a Labeled Graph	PracHub * 2026-08-09 *
M	38. Determine if a 14-tile hand is winning	PracHub * 2026-03-09 *
M	39. American Football Score Combinations	ColorStack * 2025-08-25 *
M	40. Count Passcode Patterns on a Keypad	ColorStack * 2025-11-07 *
M	41. Match people to questions using tags and priorities	PracHub * 2025-12-27 *
M	42. Generate Parentheses	ND
M	43. Permutations	ND

CODE TEMPLATE | GROUP 7 OF 43 | 8 QUESTIONS, ONE PATTERN

Backtracking: choose, recurse, undo

The one idea these questions share

Build the answer one choice at a time in a list. When the list is complete, record it. Otherwise loop over the allowed choices: append one, recurse, and then remove exactly what you added so the next choice starts from the same state. Pruning is just a check that skips a choice.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def permutations(nums):
    out, cur, used = [], [], [False] * len(nums)

    def go():
        if len(cur) == len(nums):                # CHANGE 1: when a path is complete
            out.append(cur[:])
            return
        for i, x in enumerate(nums):             # CHANGE 2: the choices at each step
            if not used[i]:                      # CHANGE 3: the rule that prunes a choice
                used[i] = True
                cur.append(x)
                go()
                cur.pop()                        # undo exactly what you did
                used[i] = False

    go()
    return out
```

2. Helper functions that usually get written next to it

```
def combinations(nums, k):                       # same shape, with a start index so order does not matter
    out, cur = [], []

    def go(start):
        if len(cur) == k:
            out.append(cur[:])
            return
        for i in range(start, len(nums)):
            cur.append(nums[i])
            go(i + 1)
            cur.pop()

    go(0)
    return out
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
36	Mahjong Winning Hand Detection (14 Tiles)	take the smallest tile; try it as a triple, then as the start of a run; undo the counts after each try
37	Count Prefix-Matching Simple Paths in a Labeled Graph	extend the path along edges whose label matches the next query letter, with a bitmask of visited nodes; count per depth
38	Determine if a 14-tile hand is winning	the same try-triple-then-run recursion over a count array, after choosing the pair
39	American Football Score Combinations	choices are the scoring plays from index i onward (the same play may repeat); complete when the remaining score is 0
40	Count Passcode Patterns on a Keypad	the choice is the next adjacent unused key; complete at k keys; count instead of storing

#	Question	What you change in the template
41	Match people to questions using tags and priorities	try each question of a person; if it is taken, recursively ask its owner to move (augmenting path); undo by not recording on failure
42	Generate Parentheses	choices are '(' and ')'; allowed when opened < n, and ')' only when closed < opened; complete at length 2n
43	Permutations	the template as written

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 36 of 197 | topic 7 of 43: Backtracking: choose, recurse, undo | 1 of 8

Template fit: take the smallest tile; try it as a triple, then as the start of a run; undo the counts after each try | * asked 11 days ago * | frequency Low

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 36 of 197

Mahjong Winning Hand Detection (14 Tiles)

ASKED 2026-10-09 | 11 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 270/288**

ABOUT THIS QUESTION

Topic: Backtracking: choose, recurse, undo

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-09 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/mahjong-winning-hand-check

QUESTION

Mountain View onsite coding round 1: given a hand of 14 tiles, decide whether it is a winning mahjong hand. The candidate must clarify the rule set (typically: 4 melds of three identical tiles or three consecutive ranks + 1 pair).

Input: array of 14 integers representing tile values (clarify suit / range during the interview).
A winning hand consists of:

One pair (two identical tiles), and
Four melds, each being either:

A triplet (3 identical tiles), or
A run (3 consecutive ranks in the same suit).

Minority variant: A newer version instead specifies one pair plus two melds while retaining a 14-tile input. Because that accounts for only eight tiles, clarify the intended meld count and how the remaining six tiles should be handled before coding.

Output: boolean.

EXAMPLES

Example 1	
Input	`[1, 1, 1, 2, 3, 4, 11, 12, 13, 21, 21, 21, 31, 31]`
Output	`true`
Why	`Tiles are numbers: 1-9 one suit, 11-19 a second suit, 21-29 a third, 31 and up honours. Pair 31-31, triplets 1-1-1 and 21-21-21, and sequences 2-3-4 and 11-12-13.`

Example 2	
Input	`[1, 1, 1, 2, 3, 4, 11, 12, 13, 21, 21, 21, 31, 32]`
Output	`false`
Why	`31 and 32 are different honours, so there is no pair to finish the hand.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def is_win(tiles):
    for t in set(tiles):
        if tiles.count(t) >= 2:
            rest = remove(tiles, [t, t])
            if all_melds(rest):
                return True
    return False
def all_melds(tiles):
    if not tiles:
        return True
    first = min(tiles)
    for meld in (triple(first), run(first)):
        if contains(tiles, meld):
            if all_melds(remove(tiles, meld)):
                return True
    return False
```

TIME	SPACE
roughly C(12,3) * C(9,3) ... roughly C(12,3) * C(9,3) * C(6,3) per pair	**O(1)**

WHY IT IS SLOW

Tries every set of three tiles at every step. The smallest remaining tile can only be in a triplet of itself or a run starting at it, so at most two choices remain.

OPTIMAL SOLUTION · Try each pair, then greedy-smallest with two options

TIME	SPACE
O(1) for 14 tiles (about 2^4 branches per pair)	**O(distinct tiles)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A variant needs only two melds plus a pair (8 tiles of 14).

Ask what happens to the other six tiles before coding: if they are free, check whether any 8 tiles form a pair and two melds with the same search over subsets of the 14. The rule set is intentionally vague, so confirm melds, honours and flowers first.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Mahjong Winning Hand Detection (14 Tiles)
#
# Mountain View onsite coding round 1: given a hand of 14
# tiles, decide whether it is a winning mahjong hand. The
# candidate must clarify the rule set (typically: 4 melds
# of three identical tiles or three consecutive ranks + 1
# pair). Input: array of 14 integers representing tile
# values (clarify suit / range during the interview). A
# winning hand consists of: One pair (two identical
# tiles), and Four melds, each being either: A triplet (3
# identical tiles), or A run (3 consecutive ranks in the
# same suit). Minority variant: A newer version instead
# specifies one pair plus two melds while retaining a
```

```
# 14-tile input. Because that accounts for only eight
# tiles, clarify the intended meld count and how the
# remaining six tiles should be handled before coding.
# Output: boolean.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A variant needs only two melds plus a pair (8 tiles
# of 14).
#
# INTERVIEW NOTES
# A winning hand is one pair plus four melds (a triplet of
# identical tiles or a run of three consecutive ranks in
# one suit). Try every tile as the pair, then repeatedly
# take the smallest remaining tile: it can only be covered
# by a triplet of itself or by a run that starts at it, so
# there are at most two choices at each step and the
# search is tiny for 14 tiles.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
# A tile is a number: suit * 10 + rank for the numbered suits (suits 0 to
# 2, ranks 1 to 9), and 30 or more for honour tiles, which never form
# runs. A winning hand is ONE pair plus `melds` melds (4 melds: 14 tiles).
```

```
# A meld is a triplet (three equal tiles) or a run (three consecutive
# ranks of one suit).
def is_run_start(tile, count):
    if tile >= 30 or tile % 10 > 7:
        return False                                # honours never run; a run cannot start at rank 8 or 9
    return count.get(tile + 1, 0) > 0 and count.get(tile + 2, 0) > 0


def smallest_tile(count):
    smallest = None
    for tile in count:
        if count[tile] > 0 and (smallest is None or tile < smallest):
            smallest = tile
    return smallest


def melds_left(count, need):
    # can the tiles in `count` (tile -> how many) form exactly `need` melds?
    tile = smallest_tile(count)
    if tile is None:
        return need == 0
    if need == 0:
        return False
    # the smallest tile is either in a triplet ...
    if count[tile] >= 3:
        count[tile] -= 3
        ok = melds_left(count, need - 1)
        count[tile] += 3                            # undo
        if ok:
            return True
    # ... or the start of a run of three
    if is_run_start(tile, count):
```

```
        for t in (tile, tile + 1, tile + 2):
            count[t] -= 1
        ok = melds_left(count, need - 1)
        for t in (tile, tile + 1, tile + 2):
            count[t] += 1                           # undo
        return ok
    return False


# ---- main question: main function ----
# Idea: count the tiles. Try every tile value that has at least two tiles
# as the PAIR; then the rest must form `melds` melds: always take the
# SMALLEST remaining tile and try the (at most two) ways to use it.
```

```
# melds=2 is the smaller variant with a pair and two melds, 8 tiles
# (follow-up 1).
# Complexity: Time O(1) for a fixed hand size, Space O(1)
def is_winning_hand(hand, melds=4):
    if len(hand) != 2 + 3 * melds:
        return False                                # one pair plus the melds must use every tile
    count = {}
    for tile in hand:
        count[tile] = count.get(tile, 0) + 1
    for tile in list(count):
        if count[tile] >= 2:
            count[tile] -= 2
            ok = melds_left(count, melds)
            count[tile] += 2
            if ok:
                return True
    return False


# ---- main question: tests ----
WIN = [1, 1, 1, 2, 3, 4, 11, 12, 13, 21, 22, 23, 31, 31]


def test_main_question():
    assert is_winning_hand(WIN)
    lose = [1, 1, 1, 2, 3, 5, 11, 12, 13, 21, 22, 23, 31, 31]      # 2, 3, 5 is not a run
    assert not is_winning_hand(lose)
    honours = [30, 30, 31, 31, 31, 32, 32, 32, 33, 33, 33, 34, 34, 34]       # honours only make triplets o
r the pair
    assert is_winning_hand(honours)
    assert not is_winning_hand(WIN[:13])                                       # a wrong tile count is never
 a win


# ========================================================
# FOLLOW-UP 1: A variant needs only two melds plus a pair (8 tiles of 14)
# ========================================================
# ---- follow-up 1: tests ----
# Answer: ask what happens to the other six tiles before coding. If they
# are simply not used, check whether ANY 8 of the 14 tiles form a pair and
# two melds (the same search, tried over the subsets). If the hand is
# only 8 tiles long, the `melds=2` parameter above is all that changes.
def test_follow_up_1():
    assert is_winning_hand([5, 5, 1, 2, 3, 7, 7, 7], 2)
    assert not is_winning_hand([5, 5, 1, 2, 4, 7, 7, 7], 2)
```

```
# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Practice writing the backtracking decomposer for tiles / counters; the canonical "expression add operators" recursion trains the same backtracking skeleton.
Have a tile-frequency-map mental model — sorting is the alternative but harder to reason about for runs across suits.
Pre-plan for the inevitable rules clarification: pull out at least 3 questions before coding.

STUDY PATH: question 37 of 197 | topic 7 of 43: Backtracking: choose, recurse, undo | 2 of 8

Template fit: extend the path along edges whose label matches the next query letter, with a bitmask of visited nodes; count per depth | * asked 72 days ago *

MEDIUM | PracHub | Technical Screen | 37 of 197

Count Prefix-Matching Simple Paths in a Labeled Graph

POSTED 2026-08-09 | 72 DAYS AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 42.5 30 DAYS 0 PRIORITY 96/288

ABOUT THIS QUESTION

Topic: Backtracking: choose, recurse, undo

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #79 Word Search

LeetCode match: No direct match; related LC 79 Word Search (inferred from overview)

Posted on PracHub: 2026-08-09 (updated 2026-08-28)

Original: https://prachub.com/coding-questions/count-prefix-matching-simple-paths-in-a-labeled-graph

QUESTION

A directed graph has labelled vertices (labels[i] is the label of vertex i) and edges (u, v). A simple path never visits a vertex twice. For a query word, count for every prefix length L (1 up to the length of the word) the number of simple paths with L vertices whose labels spell the first L letters of the word. Return the list of counts.

Function signature

def prefix_paths(labels: list, edges: list, query: str) -> list

EXAMPLES

Example 1	
Input	`['a', 'b', 'c', 'a'], [(0, 1), (1, 2), (3, 1)], 'abc'`
Output	`[2, 2, 2]`
Why	`Two paths spell 'a' (vertices 0 and 3), and both continue to b (vertex 1) and then c (vertex 2).`

Example 2	
Input	`['a', 'a'], [(0, 1), (1, 0)], 'aaa'`
Output	`[2, 2, 0]`
Why	`Two paths spell 'a' and two spell 'aa' (0 -> 1 and 1 -> 0); none spells 'aaa' because that would reuse a vertex.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
count = 0
def dfs(path):
    global count
    if len(path) <= len(query):
        if matches_prefix(path):
            count += 1
    for nxt in graph[path[-1]]:
        if nxt not in path:
            dfs(path + [nxt])
for v in vertices:
    dfs([v])
```

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

It explores paths whose first label already disagrees with the query. Extending a path only when the next label matches the next query letter prunes those branches.

OPTIMAL SOLUTION · DFS with label pruning and a visited bitmask

TIME	SPACE
O(number of matching paths * degree)	O(path)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The edges are undirected.

Add every edge in both directions, as above. A path is then counted once per direction unless you divide the counts for length >= 2 by 2 (a path and its reverse are different sequences only if the query is not a palindrome, so say how you count).

FOLLOW-UP 2 Count only the paths that spell the whole query.

Return counts[-1] instead of the list. For long queries on bigger graphs, memoise on (vertex, depth) if repeated vertices are allowed (walks, not simple paths); for simple paths the visited set prevents memoisation.

FOLLOW-UP 3 The graph has up to 100,000 vertices.

Simple paths are exponential, so ask whether walks are allowed. For walks a DP over (depth, vertex) counts them in O(L * E), the standard way to handle large graphs.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count Prefix-Matching Simple Paths in a
# Labeled Graph
#
# A directed graph has labelled vertices (labels[i] is the
# label of vertex i) and edges (u, v). A simple path never
# visits a vertex twice. For a query word, count for every
# prefix length L (1 up to the length of the word) the
# number of simple paths with L vertices whose labels
```

```
# spell the first L letters of the word. Return the list
# of counts.
#
# Signature:
#   def prefix_paths(labels: list, edges: list, query: str) -> list
#
# Examples:
# ['a', 'b', 'c', 'a'], [(0, 1), (1, 2), (3, 1)], 'abc'
#     ->  [2, 2, 2]
# ['a', 'a'], [(0, 1), (1, 0)], 'aaa'  ->  [2, 2, 0]
#
# FOLLOW-UPS (answered below, in this order):
# 1. The edges are undirected.
# 2. Count only the paths that spell the whole query.
# 3. The graph has up to 100,000 vertices.
#
# INTERVIEW NOTES
# A path is only worth extending while its labels still
# match the query, so prune on the label before going
# deeper. Distinctness is tracked with a visited set (a
# bitmask for small graphs). Count at every depth, since
# each prefix length has its own answer.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Every vertex has a label (a letter). A SIMPLE path never visits a vertex
# twice. counts[d] is the number of simple paths whose first d + 1
# labels spell the first d + 1 letters of the query.
```

```
# Idea: DFS with pruning. Start from every vertex whose label is the first
# query letter. From a vertex go only to an UNVISITED neighbour whose
# label is the NEXT query letter; add one to the count for the current
# length at each step, and undo the visit when coming back.
# `undirected=True` lets every edge be used in both directions (follow-up 1).
# Complexity: Time exponential in the query length in the worst case (every simple path is walked), Space O
(L) for the path
def prefix_paths(labels, edges, query, undirected=False):
    adj = {}
    for v in range(len(labels)):
        adj[v] = []
    for u, v in edges:
        adj[u].append(v)
        if undirected:
            adj[v].append(u)
    counts = [0] * len(query)
    visited = set()

    def go(u, depth):
        counts[depth] += 1
        if depth + 1 == len(query):
            return
        for w in adj[u]:
            if w in visited:
                continue                            # a simple path never revisits a vertex
            if labels[w] == query[depth + 1]:
                visited.add(w)
```

```
                go(w, depth + 1)
                visited.remove(w)                    # undo

    for start in range(len(labels)):
        if len(query) > 0 and labels[start] == query[0]:
            visited.add(start)
            go(start, 0)
            visited.remove(start)
    return counts


# ---- main question: tests ----
LABELS = ["a", "b", "a", "b"]
EDGES = [(0, 1), (1, 2), (2, 3), (3, 0)]             # a directed square: a > b > a > b > back to the firs
t a


def test_main_question():
    assert prefix_paths(LABELS, EDGES, "aba") == [2, 2, 2]
    assert prefix_paths(LABELS, EDGES, "ababa") == [2, 2, 2, 2, 0]      # the 5th letter would revisit a ve
rtex
```

```
    assert prefix_paths(LABELS, EDGES, "") == []


# =====================================================
# FOLLOW-UP 1: The edges are undirected
# =====================================================
# ---- follow-up 1: tests ----
# Answer: add each edge in both directions (the `undirected` parameter).
def test_follow_up_1():
    assert prefix_paths(LABELS, EDGES, "ab", True) == [2, 4]


# =====================================================
# FOLLOW-UP 2: Count only the paths that spell the whole query
# =====================================================
# ---- follow-up 2: function ----
# Answer: the last number of the list is exactly that. For long queries on
# bigger graphs: if vertices MAY repeat (walks, not simple paths), memoise on
# (vertex, depth) or do the DP below; for simple paths the visited set
# prevents memoisation.
# Complexity: Time the same as prefix_paths, Space O(L)
def whole_query_paths(labels, edges, query, undirected=False):
    counts = prefix_paths(labels, edges, query, undirected)
    if len(counts) == 0:
        return 0
    return counts[-1]


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert whole_query_paths(LABELS, EDGES, "aba") == 2
    assert whole_query_paths(LABELS, EDGES, "") == 0


# =====================================================
# FOLLOW-UP 3: The graph has up to 100,000 vertices
# =====================================================
# ---- follow-up 3: function ----
# Answer: simple paths cannot be counted fast (the visited set is the
# problem). For WALKS (vertices may repeat) a DP over (depth, vertex) is
# enough: ways[v] is the number of walks that spell the query so far and
# end in v. It costs O(length of the query * edges) and works for very
# large graphs.
```

```
# Complexity: Time O(L * E) for a query of length L, Space O(V)
def prefix_walks(labels, edges, query):
    adj = {}
    for v in range(len(labels)):
        adj[v] = []
    for u, v in edges:
        adj[u].append(v)
    ways = {}
    for v in range(len(labels)):
        if len(query) > 0 and labels[v] == query[0]:
            ways[v] = 1
    counts = []
    if len(query) > 0:
        counts.append(sum(ways.values()))
    for letter in query[1:]:
        following = {}
        for u in ways:
            for w in adj[u]:
                if labels[w] == letter:
                    following[w] = following.get(w, 0) + ways[u]
        ways = following
        counts.append(sum(ways.values()))
    return counts


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert prefix_walks(LABELS, EDGES, "abab") == [2, 2, 2, 2]
    assert prefix_walks(LABELS, EDGES, "ababa") == [2, 2, 2, 2, 2]     # a walk may reuse vertices, a simpl
e path may not
    assert prefix_walks(LABELS, EDGES, "") == []


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def count_prefix_paths(labels: str, edges: list[list[int]], word: str) -> list[int]:
```

Sample call

```
count_prefix_paths('roteoreote', [[0, 1], [0, 2], [0, 3], [1, 4], [1, 5], [3, 6], [4, 7], [5, 8], [6, 9]],
'trees')
```

Test cases

Input	Expected
`('roteoreote', [[0, 1], [0, 2], [0, 3], [1, 4], [1, 5], [3, 6], [4, 7], [5, 8], [6, 9]], 'trees')`	`[2, 2, 1, 1, 0]`
`('a', [], 'aa')`	`[1, 0]`

STUDY PATH: question 38 of 197 | topic 7 of 43: Backtracking: choose, recurse, undo | 3 of 8

Template fit: the same try-triple-then-run recursion over a count array, after choosing the pair | * asked 225 days ago *

MEDIUM | PracHub | Technical Screen | 38 of 197

Determine if a 14-tile hand is winning

POSTED 2026-03-09 | 225 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 170/288

ABOUT THIS QUESTION

Topic: Backtracking: choose, recurse, undo

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #659 Split Array into Consecutive Subsequences

LeetCode match: No direct match; related LC 659 Split Array into Consecutive Subsequences

Posted on PracHub: 2026-03-09 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/determine-if-a-14-tile-hand-is-winning

QUESTION

A hand is 14 tiles with values 1 to 9 of one suit (you may reorder them). It wins if it splits into one pair (two identical tiles) and four melds of three tiles, where a meld is either a triplet (three identical tiles) or a sequence (three consecutive values). Return whether the hand is winning.

Function signature

def is_winning_hand(tiles: list) -> bool

EXAMPLES

Example 1	
Input	`[1, 1, 1, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9, 9]`
Output	`true`
Why	`Pair 9-9, triplets 1-1-1 and 5-5-5, sequences 2-3-4 and 6-7-8.`

Example 2	
Input	`[1, 2, 3, 4, 5, 6, 7, 8, 9, 1, 1, 2, 4, 6]`
Output	`false`
Why	`These tiles cannot be split into a pair and four melds.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def is_win(counts):
    for v in range(1, 10):
        if counts[v] >= 2:
            counts[v] -= 2  # the pair
            ok = remove_melds(counts)
            counts[v] += 2
            if ok:
                return True
    return False
def remove_melds(counts):
    v = first_value_with_tiles(counts)
    if v is None:
        return True
    for meld in (triple(v), run(v)):
        if can_take(counts, meld):
            take(counts, meld)
            ok = remove_melds(counts)
            give_back(counts, meld)
            if ok:
                return True
    return False
```

TIME	SPACE
exponential but small for... exponential but small for 14 tiles	**O(depth)**

WHY IT IS SLOW

It tries the same groupings in many different orders (still tiny for 14 tiles). Always grouping the smallest remaining tile tries each grouping once.

OPTIMAL SOLUTION · Counts, try each pair, then always group the smallest remaining tile

TIME	SPACE
O(9 * small branching) = constant for 14 tiles	**O(1)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Which tile would complete a 13-tile hand?

Try each of the nine values as the 14th tile and run the same check: 9 checks of a tiny search. That is how a player finds the tiles they are waiting for.

FOLLOW-UP 2 More suits or special hands (seven pairs).

Run the single-suit check on each suit separately (melds never cross suits) after splitting the hand; seven pairs is a separate one-line count check (seven values with count 2).

FOLLOW-UP 3 Why is the 'smallest remaining value' rule enough?

The smallest remaining value cannot be the middle or end of a sequence (the smaller values are gone), so it must be in a triplet or the start of a sequence. Trying those two options covers every legal grouping exactly once.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Determine if a 14-tile hand is winning
#
# A hand is 14 tiles with values 1 to 9 of one suit (you
# may reorder them). It wins if it splits into one pair
# (two identical tiles) and four melds of three tiles,
# where a meld is either a triplet (three identical tiles)
# or a sequence (three consecutive values). Return whether
# the hand is winning.
#
# Signature:
#   def is_winning_hand(tiles: list) -> bool
#
# Examples:
# [1, 1, 1, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9, 9]  ->  true
# [1, 2, 3, 4, 5, 6, 7, 8, 9, 1, 1, 2, 4, 6]  ->  false
#
# FOLLOW-UPS (answered below, in this order):
# 1. Which tile would complete a 13-tile hand?
# 2. More suits or special hands (seven pairs).
# 3. Why is the 'smallest remaining value' rule enough?
#
# INTERVIEW NOTES
# Work with counts per value (at most nine numbers), not
# with the tile list. Try every value as the pair, then
# handle the lowest remaining value: it must belong to a
# triplet or to a sequence starting at it (nothing smaller
# is left to pair with it). Always processing the smallest
# value makes the search tiny and avoids trying the same
# grouping in different orders.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
# A hand is made of tiles with values 1 to 9 in one suit. It WINS when its
# 14 tiles split into ONE pair (two equal tiles) and FOUR melds. A meld is
# a triplet (three equal tiles) or a run (three consecutive values).
# Complexity: Time O(n), Space O(1)
```

```
def count_tiles(tiles):
    # counts[v] = how many tiles with value v (index 0 is unused)
    counts = [0] * 12
    for tile in tiles:
        counts[tile] += 1
    return counts


def melds_ok(counts, v=1):
    # can the tiles in counts be split into melds only?
    while v <= 9 and counts[v] == 0:
        v += 1                                      # the smallest value that still has tiles
    if v > 9:
```

```
        return True                                 # no tiles left
    # The smallest tile must be in a triplet or at the START of a run
    # (nothing smaller is left to take it), so only these two choices exist.
    if counts[v] >= 3:
        counts[v] -= 3
        ok = melds_ok(counts, v)
        counts[v] += 3                              # undo
        if ok:
            return True
    if v <= 7 and counts[v + 1] > 0 and counts[v + 2] > 0:
        counts[v] -= 1
        counts[v + 1] -= 1
        counts[v + 2] -= 1
        ok = melds_ok(counts, v)
        counts[v] += 1
        counts[v + 1] += 1
        counts[v + 2] += 1                          # undo
        return ok
    return False


def is_winning_counts(counts):
    # try every value that has two tiles as the PAIR; the rest must be melds
    for pair in range(1, 10):
        if counts[pair] >= 2:
            counts[pair] -= 2
            ok = melds_ok(counts)
            counts[pair] += 2
            if ok:
                return True
    return False
```

```
# ---- main question: main function ----
# Complexity: Time O(1) for a fixed hand size (at most 9 pairs, each followed by a search with 2 choices pe
r step), Space O(1)
def is_winning_hand(tiles):
    return is_winning_counts(count_tiles(tiles))


# ---- main question: tests ----
def test_main_question():
    win = [1, 1, 1, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9, 9]
    assert is_winning_hand(win)
    lose = [1, 2, 4, 5, 7, 8, 1, 2, 4, 5, 7, 8, 9, 9]         # 3 and 6 are missing, so the runs cannot be
formed
    assert not is_winning_hand(lose)


# =========================================================
# FOLLOW-UP 1: Which tile would complete a 13-tile hand?
# =========================================================
# ---- follow-up 1: function ----
# Answer: try each of the nine values as the 14th tile and run the same
# check: 9 checks of a tiny search. That is how a player finds the tiles
# they are waiting for.
# Complexity: Time O(1) (9 hand checks), Space O(1)
def waiting_tiles(tiles13):
    waiting = []
    for tile in range(1, 10):
        if is_winning_hand(tiles13 + [tile]):
            waiting.append(tile)
    return waiting
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    wait = [1, 1, 1, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9]
    assert 9 in waiting_tiles(wait)
    assert waiting_tiles(wait) != []


# ========================================================
# FOLLOW-UP 2: More suits or special hands (seven pairs)
# ========================================================
# ---- follow-up 2: functions ----
# Answer: melds never cross suits, so check each suit on its own: exactly
# one suit holds the pair (its tile count is 2 more than a multiple of
# 3), the others must be melds only. Seven pairs is a separate count
```

```
# check: 14 tiles, every value exactly twice.
# Complexity: Time O(n), Space O(n)
def is_seven_pairs(tiles):
    if len(tiles) != 14:
        return False
    counts = {}
    for tile in tiles:
        counts[tile] = counts.get(tile, 0) + 1
    for tile in counts:
        if counts[tile] != 2:
            return False
    return True


# Complexity: Time O(n), Space O(n)
def is_winning_multi(hand):
    # hand is a list of (suit, value)
    suits = {}
    for suit, value in hand:
        if suit not in suits:
            suits[suit] = []
        suits[suit].append(value)
    pair_suits = []
    for suit in suits:
        if len(suits[suit]) % 3 == 2:
            pair_suits.append(suit)
    if len(pair_suits) != 1:
        return False
    for suit in suits:
        counts = count_tiles(suits[suit])
        if suit == pair_suits[0]:
            if not is_winning_counts(counts):
                return False
        elif len(suits[suit]) % 3 != 0 or not melds_ok(counts):
            return False
    return True


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert is_seven_pairs([1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6, 6, 7, 7])
    assert not is_seven_pairs([1, 1, 1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6, 6])
    hand = [("m", v) for v in (1, 1, 1, 2, 3)] + [("p", v) for v in (4, 5, 6, 7, 7, 7)] + [("s", v) for v i
n (2, 2, 2)]
    assert is_winning_multi(hand)
    assert not is_winning_multi(hand[:-1])
```

```
# ========================================================
```

```
# FOLLOW-UP 3: Why is the "smallest remaining value" rule enough?
# ============================================================
# Answer: the smallest tile can only be in a triplet or at the START of a
# run, because a run containing it as its middle or end would need a
# smaller tile, and none is left. So at every step there are at most two
# choices, and a decision for the smallest tile never has to be reconsidered.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(tiles: list[int]) -> bool:
```

Sample call

```
solution([1,1,1,2,3,4,2,3,4,5,6,7,9,9])
```

Test cases

Input	Expected
`([1,1,1,2,3,4,2,3,4,5,6,7,9,9],)`	`True`
`([2,2,2,3,3,3,4,4,4,5,5,5,9,9],)`	`True`

ALSO REPORTED · 1 more

Mahjong Winning Hand Detection (14 Tiles) | 1Point3Acres | onsite-coding | asked 2026-07-20 | Low

https://www.1point3acres.com/interview/problems/company/google/mahjong-winning-hand-check

Clarify the rules — the prompt is intentionally sparse. Confirm: are honor tiles (winds/dragons) included? Do flowers count? Multi-suit?
Greedy + backtracking is the standard approach:
Try every possible pair (tile, tile); remove the pair.
On the remaining 12 tiles, repeatedly take the smallest remaining tile and try to form a triplet of it or a run starting at it; backtrack on failure.
For 14 tiles this runs in well under 1 ms; the interviewer cares more about the case analysis and clean recursion than micro-optimizations.
The same algorithm extends to special hands (seven pairs / ????) if the interviewer adds them as a follow-up.

Preparation: Practice writing the backtracking decomposer for tiles / counters; the canonical "expression add operators" recursion trains the same backtracking skeleton.
Have a tile-frequency-map mental model — sorting is the alternative but harder to reason about for runs across suits.
Pre-plan for the inevitable rules clarification: pull out at least 3 questions before coding.

STUDY PATH: question 39 of 197 | topic 7 of 43: Backtracking: choose, recurse, undo | 4 of 8

Template fit: choices are the scoring plays from index i onward (the same play may repeat); complete when the remaining score is 0 | * asked 421 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 39 of 197

American Football Score Combinations

ASKED 2025-08-25 | 421 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 37.9 | 30 DAYS 0 | PRIORITY 115/288

ABOUT THIS QUESTION

Topic: Backtracking: choose, recurse, undo

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #518 Coin Change II

LeetCode match: LC 518 Coin Change II (related: list the combinations)

Frequency: Single report | Last asked: 2025-08-25 | Roles: SWE Intern

QUESTION

Bryan's DP question was: enumerate all the ways a football team can accumulate a given total number of points. In American football a single scoring play is worth one of: 2 (safety), 3 (field goal), 6 (touchdown), 7 (touchdown + extra point), or 8 (touchdown + 2-point conversion). Given a target total , return every distinct combination of scoring plays whose values sum to total . Two combinations are the same if they use the same multiset of play values (order does not matter). Each combination should be returned as a list of play values sorted in non-decreasing order. The overall list of combinations may be in any order. A play value may be used any number of times.

Function signature

```
def football_combinations(total: int) -> list[list[int]]:
    """Return all distinct multisets of {2,3,6,7,8} that sum to total, each sorted
ascending."""
    ...
```

Follow-up (as reported): If you only need the count of combinations (not the lists), do it with an O(total) DP table (unbounded coin-change counting).

EXAMPLES

Example 1

Input	`total = 6`
Output	`[[2, 2, 2], [3, 3], [6]]`
Why	`2+2+2 = 3+3 = 6 = 6.`

Example 2

Input	`total = 7`
Output	`[[2, 2, 3], [7]]`
Why	`2+2+3 = 7 and the single play 7.`

Example 3

Input	`total = 0`
Output	`[[]]`
Why	`The empty combination is the only way to score 0.`

CONSTRAINTS

QUESTION

- 0 <= total <= 60
- Allowed play values are exactly [2, 3, 6, 7, 8] .
- If total == 0 , the only combination is the empty one, so return [[]] .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
PLAYS = (2, 3, 6, 7, 8)
def brute_combinations(total):
    found = set()
    def go(left, seq):
        if left == 0:
            found.add(tuple(sorted(seq)))
            return
        for v in PLAYS:
            if v <= left:
                go(left - v, seq + [v])
    go(total, [])
    return sorted(list(c) for c in found)
```

TIME	SPACE
exponential	exponential

WHY IT IS SLOW

It builds every ordering of the plays and only then collapses permutations.

OPTIMAL SOLUTION · Backtracking in increasing order

TIME	SPACE
O(number of combinations * length)	O(total) recursion

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 You only need how many combinations there are.

Use the unbounded coin DP: ways[0] = 1 and for each play value v, ways[t] += ways[t - v] for t from v to total. Looping over the values on the outside counts multisets. O(total * values) time and O(total) space.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: American Football Score Combinations
#
# Bryan's DP question was: enumerate all the ways a
# football team can accumulate a given total number of
# points. In American football a single scoring play is
# worth one of: 2 (safety), 3 (field goal), 6 (touchdown),
# 7 (touchdown + extra point), or 8 (touchdown + 2-point
# conversion). Given a target total , return every
# distinct combination of scoring plays whose values sum
```

```
# to total . Two combinations are the same if they use the
# same multiset of play values (order does not matter).
# Each combination should be returned as a list of play
# values sorted in non-decreasing order. The overall list
# of combinations may be in any order. A play value may be
# used any number of times. Function signature: def
# football_combinations(total: int) -> list[list[int]]:
# """Return all distinct multisets of {2,3,6,7,8} that sum
# to total, each sorted ascending.""" ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. You only need how many combinations there are.
#
# INTERVIEW NOTES
# This is coin change where order does not matter. To
# avoid counting 2+3 and 3+2 twice, fix an order: go
# through the play values one at a time and decide how
# many of each to use (or recurse with a minimum index).
# Listing the combinations is backtracking; counting them
# is the unbounded coin DP.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# A team can score 2, 3, 6, 7 or 8 points at a time. List every
# combination of plays that adds up to `total`.
# Idea (backtracking): choose the plays in NON-DECREASING order, so every
# combination is listed exactly once (2 + 3 and 3 + 2 are the same
# combination). `start` is the index of the smallest play we may still use;
# the same play may be used again, so the index stays the same after a
# choice. Undo the choice (pop) after the recursive call.
# Complexity: Time O(number of combinations * length), Space O(total / 2) for the path plus the answer
def football_combinations(total):
    plays = (2, 3, 6, 7, 8)
    result = []

    def dfs(start, left, path):
        if left == 0:
            result.append(path[:])
            return
        for i in range(start, len(plays)):
            if plays[i] <= left:
                path.append(plays[i])
                dfs(i, left - plays[i], path)
                path.pop()

    dfs(0, total, [])
    return result


# ---- main question: tests ----
def test_main_question():
    assert football_combinations(6) == [[2, 2, 2], [3, 3], [6]]
    assert football_combinations(7) == [[2, 2, 3], [7]]
    assert football_combinations(0) == [[]]
    assert football_combinations(1) == []
```

```python
# ============================================================
# FOLLOW-UP 1: You only need how many combinations there are
# ============================================================
# ---- follow-up 1: function ----
# Answer: use the unbounded coin DP: ways[0] = 1, and for each play value v,
# ways[t] += ways[t - v] for t from v up to the total. Looping over the
# values on the OUTSIDE counts combinations (not orders). O(total * values)
# time and O(total) space.
# Complexity: Time O(total * 5), Space O(total)
def count_combinations(total):
    ways = [1] + [0] * total
    for value in (2, 3, 6, 7, 8):
        for t in range(value, total + 1):
            ways[t] += ways[t - value]
    return ways[total]
```

```python
# ---- follow-up 1: tests ----
def test_follow_up_1():
    for total in range(0, 19):
        assert count_combinations(total) == len(football_combinations(total))


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 40 of 197 | topic 7 of 43: Backtracking: choose, recurse, undo | 5 of 8

Template fit: the choice is the next adjacent unused key; complete at k keys; count instead of storing | * asked 347 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 40 of 197

Count Passcode Patterns on a Keypad

ASKED 2025-11-07 | 347 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 229/288

ABOUT THIS QUESTION

Topic: Backtracking: choose, recurse, undo

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #980 Unique Paths III

LeetCode match: LC 980 / LC 351 Android Unlock Patterns (related: paths on a grid)

Frequency: Single report | Last asked: 2025-11-07 | Roles: SWE Intern

QUESTION

A keypad is a rows x cols grid of keys (the classic phone dial is 3x3):
1 2 3 4 5 6 7 8 9 A passcode pattern is a path across keys where each move goes only to an orthogonally adjacent key (horizontal or vertical — no diagonals), and no key is visited more than once. The order matters: 1 -> 2 -> 3 -> 6 and 6 -> 3 -> 2 -> 1 are different patterns. Given the grid dimensions and an integer k , return the number of distinct passcode patterns that use exactly k keys.

Function signature

```
def count_patterns(rows: int, cols: int, k: int) -> int:
    """Return the number of distinct passcode patterns using exactly k keys,
    moving only to orthogonally adjacent, unvisited keys on a rows x cols grid."""
    ...
```

Follow-up (as reported): Count patterns of length at least k instead of exactly k . Hint: sum the DFS counts over all lengths from k to rows*cols .

EXAMPLES

Example 1

Input	`rows = 3, cols = 3, k = 1`
Output	`9`
Why	`Each of the 9 keys alone is one pattern.`

Example 2

Input	`rows = 3, cols = 3, k = 2`
Output	`24`
Why	`There are 12 adjacent key pairs; each can be traversed in 2 directions, giving 24 patterns.`

Example 3

Input	`rows = 3, cols = 3, k = 3`
Output	`44`
Why	`Summing over each possible middle key b, deg(b)*(deg(b)-1) directed 3-key paths: corners give 4*2=8, edge-mids 4*6=24, center 12; total 44.`

CONSTRAINTS

QUESTION

- 1 <= rows, cols <= 4 .
- 1 <= k <= rows * cols .
- A single key (k = 1) is a valid pattern of length 1.
- Count directed patterns (a path and its reverse are counted separately).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
from itertools import permutations
def brute_patterns(rows, cols, k):
    cells = [
        (r, c)
        for r in range(rows)
        for c in range(cols)
    ]
    count = 0
    for path in permutations(cells,_k):
        if all(
            abs(a[0] - b[0]) + abs(a[1] - b[1])
            == 1
            for a, b in zip(path, path[1:])
        ):
            count += 1
    return count
```

TIME	SPACE
O(P(rc, k) * k)	O(k)

WHY IT IS SLOW

It builds selections that are not even connected before testing them.

OPTIMAL SOLUTION · DFS with backtracking from every key

TIME	SPACE
O(number of simple paths)	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Count patterns that use at least k keys.

Use the same DFS but add one at every depth that is at least k and keep extending instead of stopping.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count Passcode Patterns on a Keypad
#
# A keypad is a rows x cols grid of keys (the classic
# phone dial is 3x3): 1 2 3 4 5 6 7 8 9 A passcode pattern
# is a path across keys where each move goes only to an
# orthogonally adjacent key (horizontal or vertical — no
```

```
# diagonals), and no key is visited more than once. The
# order matters: 1 -> 2 -> 3 -> 6 and 6 -> 3 -> 2 -> 1 are
# different patterns. Given the grid dimensions and an
# integer k , return the number of distinct passcode
# patterns that use exactly k keys. Function signature:
# def count_patterns(rows: int, cols: int, k: int) -> int:
# """Return the number of distinct passcode patterns using
# exactly k keys, moving only to orthogonally adjacent,
# unvisited keys on a rows x cols grid.""" ... Follow-up
# (as reported): Count patterns of length at least k
# instead of exactly k . Hint: sum the DFS counts over all
# lengths from k to rows*cols .
#
# FOLLOW-UPS (answered below, in this order):
# 1. Count patterns that use at least k keys.
#
# INTERVIEW NOTES
# Count simple paths of exactly k keys on a grid: start a
# DFS from every key, mark the key used, step to each
# unused orthogonal neighbour, and count one when the path
# has k keys. Unmark on the way back (backtracking). The
# grid is at most 4x4, so the search stays small.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def neighbours(r, c, rows, cols):
    # the keys right next to key (r, c) on the keypad
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr = r + dr
        nc = c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            result.append((nr, nc))
    return result


# ---- main question: main function ----
# Count the patterns of exactly k keys: start on any key and keep moving to
# an ADJACENT key that is not used yet.
# Idea (DFS with backtracking): from the current key try every unused
# neighbour: mark it used, recurse with one more key, then UNMARK it.
# A pattern is complete when it has k keys.
# Complexity: Time O(rows * cols * 4^k) in the worst case, Space O(k)
def count_patterns(rows, cols, k):
    def dfs(r, c, used, seen):
        if used == k:
            return 1
        total = 0
        for nr, nc in neighbours(r, c, rows, cols):
            if (nr, nc) not in seen:
                seen.add((nr, nc))
                total += dfs(nr, nc, used + 1, seen)
                seen.remove((nr, nc))                # undo
        return total
```

```
    total = 0
    for r in range(rows):
        for c in range(cols):
            total += dfs(r, c, 1, {(r, c)})
    return total


# ---- main question: tests ----
def test_main_question():
    assert count_patterns(3, 3, 1) == 9
    assert count_patterns(3, 3, 2) == 24                # one per direction of every edge
    assert count_patterns(3, 3, 3) == 44


# =========================================================
# FOLLOW-UP 1: Count patterns that use at least k keys
# =========================================================
# ---- follow-up 1: function ----
# Answer: the same DFS, but count one at every depth that is at least k
# and keep extending instead of stopping at k.
```

```
# Complexity: Time O(rows * cols * 4^(rows * cols)) in the worst case, Space O(rows * cols)
def count_patterns_at_least(rows, cols, k):
    def dfs(r, c, used, seen):
        total = 0
        if used >= k:
            total = 1
        for nr, nc in neighbours(r, c, rows, cols):
            if (nr, nc) not in seen:
                seen.add((nr, nc))
                total += dfs(nr, nc, used + 1, seen)
                seen.remove((nr, nc))
        return total

    total = 0
    for r in range(rows):
        for c in range(cols):
            total += dfs(r, c, 1, {(r, c)})
    return total


# ---- follow-up 1: tests ----
def test_follow_up_1():
    for rows, cols in ((1, 4), (2, 2), (2, 3), (3, 3)):
        for k in range(1, 4):
            expected = 0
            for length in range(k, rows * cols + 1):
                expected += count_patterns(rows, cols, length)
            assert count_patterns_at_least(rows, cols, k) == expected


# =========================================================
# RUN EVERYTHING
# =========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 41 of 197 | topic 7 of 43: Backtracking: choose, recurse, undo | 6 of 8

Template fit: try each question of a person; if it is taken, recursively ask its owner to move (augmenting path); undo by not recording on failure | * asked 297 days ago *

MEDIUM | PracHub | Technical Screen | 41 of 197

Match people to questions using tags and priorities

POSTED 2025-12-27 | 297 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 5 | 30 DAYS 0 | PRIORITY 196/288

ABOUT THIS QUESTION

Topic: Backtracking: choose, recurse, undo

Optimal answer from: the interview site's own solution

LeetCode match: LC 1820 Maximum Number of Accepted Invitations (bipartite matching) (inferred from overview)

Posted on PracHub: 2025-12-27 (updated 2026-06-04)

Original: https://prachub.com/coding-questions/match-people-to-questions-using-tags-and-priorities

QUESTION

A set of people and a set of interview questions: adj[p] lists the questions person p is allowed to answer (their tags match). Each person is given at most one question and each question goes to at most one person. Return the largest number of people that can be matched.

Function signature

def max_matching(adj: list) -> int

EXAMPLES

Example 1	
Input	`[[0, 1], [0], [1, 2]]`
Output	`3`
Why	`Person 0 takes question 0, person 1 has only question 0 so person 0 moves to question 1, person 2 takes question 2: all three are matched.`

Example 2	
Input	`[[0], [0], [0]]`
Output	`1`
Why	`Three people can only answer the same single question: just one match.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
best = 0
def go(person, used, matches):
    global best
    if person == len(people):
        best = max(best, matches)
        return
    go(person + 1, used, matches)  # skip
    for q in range(len(questions)):
        if q in used:
            continue
        if not shares_tag(person, q):
            continue
        go(person + 1, used | {q},
           matches + 1)
```

TIME	SPACE
O((Q + 1)^P)	O(P)

WHY IT IS SLOW

Each person branches over all questions: exponential. Augmenting paths (move an earlier person to another question when needed) solve it in O(people * edges).

OPTIMAL SOLUTION · Augmenting paths (Kuhn's algorithm)

TIME	SPACE
O(P * E)	O(P + Q)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Priorities: higher-priority people must be served first.

Process people in decreasing priority and run the same augmenting step. Matchings of this kind form a matroid, so this greedy choice gives a matched set with the maximum total priority (checked against exhaustive search).

FOLLOW-UP 2 Edges have weights (how good a match is).

Then it is maximum-weight bipartite matching: the Hungarian algorithm in O(n^3), or min-cost max-flow. Kuhn's algorithm only maximises the count.

FOLLOW-UP 3 A question may take several people (capacity c).

Replicate the question c times as separate right-hand nodes, or run max flow with capacities on the question-to-sink edges.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Match people to questions using tags and
# priorities
#
# A set of people and a set of interview questions: adj[p]
# lists the questions person p is allowed to answer (their
# tags match). Each person is given at most one question
```

```
# and each question goes to at most one person. Return the
# largest number of people that can be matched.
#
# Signature:
#   def max_matching(adj: list) -> int
#
# Examples:
# [[0, 1], [0], [1, 2]]  ->  3
# [[0], [0], [0]]  ->  1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Priorities: higher-priority people must be served
# first.
# 2. Edges have weights (how good a match is).
# 3. A question may take several people (capacity c).
#
# INTERVIEW NOTES
# 'Assign each person at most one question, each question
# at most one person, allowed pairs given by tags' is
# maximum bipartite matching. The tag index only builds
# the edges faster. For maximum count use augmenting
# paths: try to give a person a question, and if taken ask
# its current owner to move. If priorities decide who is
# served, process people in priority order with the same
# augmenting step.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def build_graph(people, questions):
    # a person can take a question when they share at least one tag.
```

```
    # people: [(name, priority, tags)]   questions: [(id, tags, capacity)]
    adj = {}
    for name, priority, tags in people:
        adj[name] = []
        for question_id, question_tags, capacity in questions:
            if len(set(tags) & set(question_tags)) > 0:
                adj[name].append(question_id)
    return adj


# ---- main question: main function ----
# Idea (Kuhn's augmenting paths): adj[p] is the list of questions person p
# accepts. For each person run a DFS over the questions they accept: if a
# question is free, take it; if it belongs to someone else, try to move
# that owner to ANOTHER question (a recursive try). A `seen` set makes sure
# no question is tried twice in one attempt. The number of people who
# found a question is the size of the maximum matching.
# Complexity: Time O(V * E) (one search per person), Space O(V + E)
def max_matching(adj):
    owner = {}                                      # question -> the person who holds it

    def try_assign(person, seen):
        for question in adj[person]:
            if question in seen:
                continue
```

```
            seen.add(question)
            if question not in owner or try_assign(owner[question], seen):
                owner[question] = person
                return True
        return False

    matched = 0
    for person in range(len(adj)):
        if try_assign(person, set()):
            matched += 1
    return matched


# ---- main question: tests ----
def test_main_question():
    assert max_matching([[0, 1], [0], [1, 2]]) == 3          # person 1 takes 0, so person 0 moves to 1, pe
rson 2 takes 2
    assert max_matching([[0], [0], [0]]) == 1
    assert max_matching([[], []]) == 0
    people = [("ann", 1, {"dp"}), ("bob", 5, {"dp", "graph"}), ("cy", 3, {"graph"})]
    questions = [("q1", {"dp"}, 1), ("q2", {"graph"}, 1)]
```

```
    assert build_graph(people, questions) == {"ann": ["q1"], "bob": ["q1", "q2"], "cy": ["q2"]}


# ========================================================
# FOLLOW-UP 1: Priorities: higher-priority people must be served first
# ========================================================


# ========================================================
# FOLLOW-UP 3: A question may take several people (capacity c)
# ========================================================
# ---- follow-up 1: helper functions ----
def priority_of(person):
    return person[1]                                    # a person is (name, priority, tags)


# ---- follow-up 1: function ----
# Answer: try the people in order of decreasing priority and run the same
# augmenting step. Matchings of this kind form a "matroid", so this greedy
# order ends with the matched set of the highest total priority. A
# question with capacity c holds up to c people: it has a free place when
# fewer than c people hold it, otherwise try to move one of its holders
# to another question.
# Complexity: Time O(V * E) (the people are sorted first, then one search each), Space O(V + E)
def assign(people, questions):
    adj = build_graph(people, questions)
    capacity = {}
    held = {}                                           # question -> the people who hold its places
    for question_id, tags, cap in questions:
        capacity[question_id] = cap
        held[question_id] = []

    def try_assign(person, seen):
        for question in adj[person]:
            if question in seen:
                continue
            seen.add(question)
            if len(held[question]) < capacity[question]:
                held[question].append(person)           # a free place
                return True
            for other in list(held[question]):          # or someone here who can move elsewhere
                if try_assign(other, seen):
                    held[question].remove(other)
```

```
                    held[question].append(person)
                    return True
```

```
        return False

    for name, priority, tags in sorted(people, key=priority_of, reverse=True):
        try_assign(name, set())
    result = {}
    for question in held:
        for person in held[question]:
            result[person] = question
    return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    people = [("ann", 1, {"dp"}), ("bob", 5, {"dp", "graph"}), ("cy", 3, {"graph"})]
    questions = [("q1", {"dp"}, 1), ("q2", {"graph"}, 1)]
    got = assign(people, questions)
    assert len(got) == 2                            # the matching is still the largest possible
    assert got["bob"] in ("q1", "q2")               # bob (priority 5) is served
    both = assign(people[:2], [("q1", {"dp"}, 2)])  # a question with two places takes two people
    assert set(both) == {"ann", "bob"}
    assert assign([("x", 1, {"a"})], [("q", {"b"}, 1)]) == {}      # nobody shares a tag: nobody is matched


# ========================================================
# FOLLOW-UP 2: Edges have weights (how good a match is)
# ========================================================
# Answer: now it is a maximum-WEIGHT matching: use the Hungarian
# algorithm (O(n^3)) or min-cost max-flow. Kuhn's algorithm only maximises
# the NUMBER of matches.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(people: list[list[str]], question_tags: list[list[str]], priorities: list[int]) -> list[int]:
```

Sample call

```
solution([['python', 'sql'], ['java'], ['ml']], [['sql'], ['java', 'python'], ['ml'], ['go']], [5, 7, 4, 10
])
```

Test cases

Input	Expected
`([['python', 'sql'], ['java'], ['ml']], [['sql'], ['java', 'python'], ['ml'], ['go']], [5, 7, 4, 10])`	`(3, 16)`

Input	Expected
`([['a'], ['a']], [['a'], ['a'], ['b']], [1, 10, 100])`	`(2, 11)`

STUDY PATH: question 42 of 197 | topic 7 of 43: Backtracking: choose, recurse, undo | 7 of 8

Template fit: choices are '(' and ')'; allowed when opened < n, and ')' only when closed < opened; complete at length 2n | ND

MEDIUM | IN APP LC 22 | Exponent | 42 of 197

★ TOP 75: rank 19 in the Medium and Hard priority list

★ Generate Parentheses

ASKED (ANY CO.) ~6 MONTHS AGO | OLDER

STUDY: TYPE **ALL TIME 61.6** **30 DAYS 46.9** **PRIORITY 16/288**

ABOUT THIS QUESTION

Topic: Backtracking: choose, recurse, undo

Optimal answer from: your app's interview approach (LeetCode #22)

LeetCode match: LC 22 Generate Parentheses

Last asked (any company): 6 months ago, about 2026-04 (Exponent shows only a rough age) | +36 interviews

Original: https://www.aced.io/questions/1594/generate-parentheses

QUESTION

Generate all combinations of well-formed parentheses. A parenthesis combination is well-formed if each opening parenthesis "(" is closed by a matching closing parenthesis ")" in the correct order. For example, "()", "()()", and "((()))" are all combinations of well-formed parentheses, while ")(", "((" and "(()" are not. Examples n = 1 output: ["()"] n = 2 output: ["(())", "()()"] n = 3 output: ["((()))", "(()())", "(())()", "()(())", "()()()"] n = 4 output should contain: ["(((())))", "((()()))", "((())())", "((()))()", "(()(()))", "(()()())", "(()())()", "(())(())", "(())()()", "()((()))", "()(()())", "()(())()", "()()(())", "()()()()"]The problem is to generate all combinations of well-formed parentheses given n pairs of parentheses. A combination is well-formed if every opening parenthesis has a corresponding closing parenthesis and they are properly nested. Recursive Approach The most common and intuitive way to solve this problem is by using recursion with backtracking. Here's a step-by-step breakdown of the approach: Define the Problem: We need to build strings of length 2n using n opening (and n closing) parentheses. At any point in the string, the number of closing parentheses should not exceed the number of opening parentheses to ensure the sequence is valid. Recursive Function: We use a recursive helper function to build the strings character by character. The function tracks the remaining number of opening and closing parentheses that can still be added. It also

EXAMPLES

Example 1

Input	`3`
Output	`['((()))', '(()())', '(())()', '()(())', '()()()']`
Why	`Five well-formed combinations of three pairs.`

Example 2

Input	`1`
Output	`['()']`
Why	`One pair.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n):
    result = []
    for seq in product("()", repeat=2 * n):
        if is_balanced(seq):
            result.append("".join(seq))
    return result
```

TIME	SPACE
O(2^(2n) * n)	O(2^(2n))

WHY IT IS SLOW

Builds exponentially many invalid strings. Backtracking only adds '(' while fewer than n are used and ')' while fewer than the opens, so it builds only valid prefixes (the Catalan number of results).

OPTIMAL SOLUTION · Backtracking with open and close counters

TIME	SPACE
O(4^n / sqrt(n))	O(n) beyond the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return only the k-th string, or just the number of strings.

The count is the Catalan number C(n) = (2n choose n) / (n + 1). For the k-th string count the completions that start with '(' at each step and skip them if k is larger (unranking), using a small DP over (opens left, balance).

FOLLOW-UP 2 Several bracket types, such as () [] {}.

Multiply the choices: at each position you may open any of the types (until its count is used) and may close only the type on top of a stack. The pruning rule (never close more than you have opened) becomes 'close only the most recent unclosed bracket'.

FOLLOW-UP 3 Avoid recursion.

Use an explicit stack of (string so far, opens, closes) states or build the strings by length with a queue. The pruning conditions stay the same.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Generate Parentheses
#
# Generate all combinations of well-formed parentheses. A
# parenthesis combination is well-formed if each opening
# parenthesis "(" is closed by a matching closing
# parenthesis ")" in the correct order. For example, "()",
# "()()", and "((()))" are all combinations of well-formed
# parentheses, while ")(", "((" and "(()" are not.
```

```
# Examples n = 1 output: ["()"] n = 2 output: ["(())",
# "()()"] n = 3 output: ["((()))", "(()())", "(())()",
# "()(())", "()()()"] n = 4 output should contain:
# ["(((())))", "((()()))", "((())())", "((()))()",
# "(()(()))", "(()()())", "(()())()", "(())(())",
# "(())()()", "()((()))", "()(()())", "()(())()",
# "()()(())", "()()()()"]The problem is to generate all
# combinations of well-formed parentheses given n pairs of
# parentheses. A combination is well-formed if every
# opening parenthesis has a corresponding closing
# parenthesis and they are properly nested.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return only the k-th string, or just the number of
# strings.
# 2. Several bracket types, such as () [] {}.
# 3. Avoid recursion.
#
# INTERVIEW NOTES
# Build the string left to right and only make moves that
# can still lead to a valid result: you may add '(' while
# fewer than n have been opened, and ')' only while there
# are more opens than closes. Because invalid prefixes are
# never extended, every leaf is a valid answer, and the
# count is the Catalan number.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
```

```
# Idea (backtracking): build the string one character at a time. Add "(" as
# long as fewer than n have been placed; add ")" only while the number
# of ")" is smaller than the number of "(" (so it closes something). That
# rule keeps every string valid. At length 2n the string is complete.
# Complexity: Time O(4^n / sqrt(n)) (the Catalan number of strings, each O(n) long), Space O(n) working plu
s the answer
def generate(n):
    result = []

    def build(text, opens, closes):
        if len(text) == 2 * n:
            result.append(text)
            return
        if opens < n:
            build(text + "(", opens + 1, closes)
        if closes < opens:
            build(text + ")", opens, closes + 1)

    build("", 0, 0)
    return result


# ---- main question: tests ----
def test_main_question():
    assert generate(1) == ["()"]
    assert generate(2) == ["(())", "()()"]
    assert sorted(generate(3)) == ["((()))", "(()())", "(())()", "()(())", "()()()"]
```

```
    assert generate(0) == [""]


# =====================================================
# FOLLOW-UP 1: Return only the k-th string, or just the number of strings
# =====================================================
# ---- follow-up 1: helper functions ----
MEMO = {}


def completions(opens_left, balance):
    # in how many ways can a string be finished when opens_left "(" are
    # still to be placed and `balance` brackets are open (unclosed)?
    key = (opens_left, balance)
    if key in MEMO:
        return MEMO[key]
    if opens_left == 0:
        result = 1                                  # only the closing brackets are left: one way
    else:
```

```
        result = completions(opens_left - 1, balance + 1)
        if balance > 0:
            result += completions(opens_left, balance - 1)
    MEMO[key] = result
    return result


# ---- follow-up 1: functions ----
# Answer: the NUMBER of strings is the Catalan number C(n) =
# (2n choose n) / (n + 1), which is completions(n, 0). For the k-th
# string (in sorted order, "(" before ")") count at each step the
# strings that continue with "(" and skip them if k is larger
# (unranking), with a small DP over (opens left, balance).
# Complexity: Time O(n^2), Space O(n^2)
def catalan(n):
    return completions(n, 0)


# Complexity: Time O(n) steps after the O(n^2) table, Space O(n^2)
def kth(n, k):
    if k < 1 or k > catalan(n):
        raise ValueError("k is out of range")
    text = ""
    opens_left = n
    balance = 0
    while len(text) < 2 * n:
        ways = 0
        if opens_left > 0:
            ways = completions(opens_left - 1, balance + 1)       # strings that continue with "("
        if opens_left > 0 and k <= ways:
            text += "("
            opens_left -= 1
            balance += 1
        else:
            k -= ways                                   # skip all the strings that start with "("
            text += ")"
            balance -= 1
    return text


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert catalan(3) == 5 and catalan(10) == 16796
    for n in range(0, 8):
        assert catalan(n) == len(generate(n))
```

```
    for n in range(1, 7):
        everything = sorted(generate(n))
        for k in range(1, len(everything) + 1):
            assert kth(n, k) == everything[k - 1]


# =============================================================
# FOLLOW-UP 2: Several bracket types, such as () [] {}
# =============================================================
# ---- follow-up 2: function ----
# Answer: multiply the choices. Any type may be OPENED (while fewer than n
# pairs were opened) and only the type on TOP of a stack of open brackets
# may be closed.
# Complexity: Time O(Catalan(n) * 3^n * n) for 3 types, Space the same
def generate_multi(n, kinds="()[]{}"):
    pairs = []
    for i in range(0, len(kinds), 2):
        pairs.append(kinds[i:i + 2])
    result = []

    def build(text, stack, opened):
        if len(text) == 2 * n:
            result.append(text)
            return
        if opened < n:
            for open_char, close_char in pairs:
                build(text + open_char, stack + [close_char], opened + 1)
        if len(stack) > 0:
            build(text + stack[-1], stack[:-1], opened)

    build("", [], 0)
    return result


# ---- follow-up 2: tests ----
def test_follow_up_2():
    for n in range(1, 5):                           # Catalan(n) shapes times 3 type choices per pair
        got = generate_multi(n, "()[]{}")
        assert len(got) == catalan(n) * 3 ** n
        assert len(set(got)) == len(got)
    assert sorted(generate_multi(1, "()[]")) == ["()", "[]"]


# =============================================================
# FOLLOW-UP 3: Avoid recursion
```

```
# =============================================================
# ---- follow-up 3: function ----
# Answer: use your own stack of states (text, opens, closes) instead of the
# call stack. The pruning rules stay the same. ")" is pushed first so
# "(" is handled first and the strings come out in sorted order.
# Complexity: Time O(4^n / sqrt(n)), Space O(n) working plus the answer
def generate_iter(n):
    result = []
    stack = [("", 0, 0)]
    while stack:
        text, opens, closes = stack.pop()
        if len(text) == 2 * n:
            result.append(text)
            continue
        if closes < opens:
            stack.append((text + ")", opens, closes + 1))
        if opens < n:
            stack.append((text + "(", opens + 1, closes))
    return result
```

```
# ---- follow-up 3: tests ----
def test_follow_up_3():
    for n in range(0, 8):
        assert generate_iter(n) == sorted(generate(n))


# =======================================================
# RUN EVERYTHING
# =======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Generate all combinations of well-formed parentheses. A parenthesis combination is well-formed if each opening parenthesis "(" is closed by a matching closing parenthesis ")" in the correct order. For example, "()", "()()", and "((()))" are all combinations of well-formed parentheses, while ")(", "((" and "(()" are not. Examples

```
n = 1
output: ["()"]

n = 2
output: ["(())", "()()"]

n = 3
output: ["((()))", "(()())", "(())()", "()(())", "()()()"]

n = 4
output should contain: ["(((())))", "((()()))", "((())())", "((()))()", "(()(()))", "(()()())", "(())(())"
, "(())()()", "()((()))", "()(()())", "()(())()", "()()(())", "()()()()"]
```

OFFICIAL PAGE TEXT

The problem is to generate all combinations of well-formed parentheses given n pairs of parentheses. A combination is well-formed if every opening parenthesis has a corresponding closing parenthesis and they are properly nested. Recursive Approach The most common and intuitive way to solve this problem is by using recursion with backtracking. Here's a step-by-step breakdown of the approach: Define the Problem: We need to build strings of length 2n using n opening (and n closing) parentheses. At any point in the string, the number of closing parentheses should not exceed the number of opening parentheses to ensure the sequence is valid. Recursive Function: We use a recursive helper function to build the strings character by character. The function tracks the remaining number of opening and closing parentheses that can still be added. It also maintains the current combination of the string being formed. Base Case: If no more parentheses are remaining to add (both remaining_open and remaining_close are zero), the current combination is a valid sequence and is added to the result list. Recursive Case: Add an opening parenthesis (if there are any remaining and recursively call the function with one less opening parenthesis. Add a closing parenthesis) if the number of remaining closing parentheses is greater than the remaining opening parentheses and recursively call the function with one less closing parenthesis. Pruning Invalid Paths: If at any point the number of closing parentheses left is less than the number of opening parentheses left, it means the current sequence cannot be completed to a valid sequence, so we stop exploring that path. Implementation Steps Initialize: Create a result list to store all valid sequences. Call the recursive helper function with initial values (n opening and n closing parentheses). Recursive Helper Function: Track the number of remaining opening and closing parentheses. Track the current combination being formed. Append valid combinations to the result list when the base case is reached. Backtrack: After adding an opening or closing parenthesis, remove it (backtrack) before trying the next possibility. This ensures that we explore all possible valid combinations. Example Let's consider n = 3: Start with an empty combination "", 3 opening, and 3 closing parentheses. Add an opening parenthesis: "(", 2 opening, and 3 closing. Continue adding opening parentheses until you can no longer add without violating the rules. Then add closing parentheses and ensure that at no point the

closing parentheses exceed the opening ones. Once you form a valid combination like "((()))", add it to the result list. Backtrack and explore other combinations. Solution Time Complexity: The time complexity of this solution is O(22n)O(2^{2n})O(22n). This is because each position in the string can either be an opening or closing parenthesis, and we need to generate all possible sequences of length 2n2n2n. Space Complexity: The space complexity of the solution is O(2n)O(2n)O(2n) which simplifies to O(n)O(n)O(n). This is because: The recursion depth is 2n2n2n in the worst case (each recursive call adds either an opening or a closing parenthesis). The combination string being formed is of length 2n2n2n. In this mock interview video, Aman (Google SWE) answers the interview question, "Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses."No answers contributed by the community yet.

STUDY PATH: question 43 of 197 | topic 7 of 43: Backtracking: choose, recurse, undo | 8 of 8

Template fit: the template as written | ND

MEDIUM | IN APP LC 46 | Exponent | 43 of 197

Permutations

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 49.8 30 DAYS 0 PRIORITY 80/288

ABOUT THIS QUESTION

Topic: Backtracking: choose, recurse, undo

Optimal answer from: your app's interview approach (LeetCode #46)

LeetCode match: LC 46 Permutations

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +25 interviews

Original: https://www.aced.io/questions/2212/leetcode-medium-permutations-dp

QUESTION

Given an array of distinct integers, return all possible permutations of the array (every rearrangement of its elements), in any order.

Function signature

def permute(nums: list) -> list

EXAMPLES

Example 1	
Input	`[1, 2, 3]`
Output	`[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]`
Why	`Six orders of three numbers.`

Example 2	
Input	`[0, 1]`
Output	`[[0, 1], [1, 0]]`
Why	`Two numbers have two orders.`

Example 3	
Input	`[1]`
Output	`[[1]]`
Why	`One number has a single permutation.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums):
    result = []
    for seq in product(nums, repeat=len(nums)):
        if len(set(seq)) == len(nums):
            result.append(list(seq))
    return result
```

BRUTE FORCE

TIME	SPACE
O(n^n * n)	O(n!)

WHY IT IS SLOW

n^n sequences when only n! are valid, so almost all the work is wasted. Backtracking with a used marker only builds valid prefixes, O(n * n!).

OPTIMAL SOLUTION · Backtracking with a used array

TIME	SPACE
O(n * n!)	O(n) beyond the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The input has duplicates (Permutations II).

Sort first, and skip a value that equals the previous one unless the previous one is already used in the current prefix. That forces equal values to appear in a fixed order and removes duplicates without a set.

FOLLOW-UP 2 Return the next lexicographic permutation in place.

Find the last position i with a[i] < a[i + 1], swap a[i] with the smallest larger value to its right, then reverse the suffix. O(n), O(1) space. Repeat to enumerate in order.

FOLLOW-UP 3 Return only the k-th permutation of 1..n.

Use the factorial number system: the first digit is index (k - 1) // (n - 1)! of the remaining digits, then continue with the remainder. O(n^2), no enumeration.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Permutations
#
# Given an array of distinct integers, return all possible
# permutations of the array (every rearrangement of its
# elements), in any order.
#
# Signature:
#   def permute(nums: list) -> list
#
# Examples:
# [1, 2, 3]  ->  [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3,
#     1], [3, 1, 2], [3, 2, 1]]
# [0, 1]  ->  [[0, 1], [1, 0]]
#
# FOLLOW-UPS (answered below, in this order):
# 1. The input has duplicates (Permutations II).
# 2. Return the next lexicographic permutation in place.
# 3. Return only the k-th permutation of 1..n.
#
# INTERVIEW NOTES
```

```
# The output has n! items, so no algorithm can beat O(n *
# n!); the question is only whether you generate them
# without waste. Build a permutation one position at a
# time, marking which elements are used, and undo the
# choice when you come back (backtracking). Every follow-
# up (duplicates, k-th, next) is a variation on 'what are
# the legal choices at this position'.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea (backtracking): fill the positions from left to right. At each
# position try every value that is not used yet: choose it (mark it used,
# put it in the current list), recurse for the next position, then UNDO
# the choice (remove it, unmark it). Only valid prefixes are ever
# extended, so every complete list is a permutation.
# Complexity: Time O(n * n!), Space O(n * n!) for the answer (O(n) working space)
```

```
def permute(nums):
    result = []
    current = []
    used = [False] * len(nums)

    def go():
        if len(current) == len(nums):
            result.append(current[:])               # a copy, because `current` keeps changing
            return
        for i in range(len(nums)):
            if not used[i]:
                used[i] = True
                current.append(nums[i])
                go()
                current.pop()
                used[i] = False

    go()
    return result


# ---- main question: tests ----
def test_main_question():
    assert permute([1, 2, 3]) == [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
    assert permute([0, 1]) == [[0, 1], [1, 0]]
    assert permute([1]) == [[1]]
    assert permute([]) == [[]]


# ========================================================
# FOLLOW-UP 1: The input has duplicates (Permutations II)
# ========================================================
# ---- follow-up 1: function ----
# Answer: sort first, and skip a value that equals the previous one unless
# the previous one is already used in the current list. That forces equal
# values to appear in a fixed order, which removes the duplicate lists
# without a set.
# Complexity: Time O(n * n!) worst case (fewer with duplicates), Space O(n * n!)
def permute_unique(nums):
```

```
    nums = sorted(nums)
    result = []
    current = []
    used = [False] * len(nums)

    def go():
        if len(current) == len(nums):
            result.append(current[:])
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            if i > 0 and nums[i] == nums[i - 1] and not used[i - 1]:
                continue                            # the same value as before and it is not in use
            used[i] = True
            current.append(nums[i])
            go()
            current.pop()
            used[i] = False

    go()
    return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert permute_unique([1, 1, 2]) == [[1, 1, 2], [1, 2, 1], [2, 1, 1]]
    assert permute_unique([2, 2]) == [[2, 2]]


# ========================================================
# FOLLOW-UP 2: Return the next lexicographic permutation in place
# ========================================================
# ---- follow-up 2: function ----
# Answer: find the LAST position i with a[i] < a[i + 1], swap a[i] with the
# smallest bigger value to its right (the last bigger one, because the
# part after i is in decreasing order), then reverse the part after i so
# it is the smallest. O(n) time, O(1) extra space. Returns False (and
# wraps to the smallest order) when it was the last permutation.
# Complexity: Time O(n), Space O(1)
def next_permutation(a):
    i = len(a) - 2
    while i >= 0 and a[i] >= a[i + 1]:
        i -= 1                                      # find the last position that can still increase
    if i < 0:
        a.reverse()
        return False
    j = len(a) - 1
    while a[j] <= a[i]:
        j -= 1
    a[i], a[j] = a[j], a[i]
    a[i + 1:] = a[i + 1:][::-1]                     # the part after i is decreasing: make it the smallest
    return True


# ---- follow-up 2: tests ----
def test_follow_up_2():
    a = [1, 2, 3]
    seen = [a[:]]
    while next_permutation(a):
        seen.append(a[:])
    assert seen == permute([1, 2, 3])               # repeating it lists all permutations in order
    assert a == [1, 2, 3]                           # and wraps around at the end
```

```
# ============================================================
# FOLLOW-UP 3: Return only the k-th permutation of 1..n
# ============================================================
# ---- follow-up 3: helper functions ----
def factorial(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result


# ---- follow-up 3: function ----
# Answer: the factorial number system. The permutations of 1..n come in
# blocks of (n - 1)!, so the first value is the unused value at index
# (k - 1) // (n - 1)!; continue with the remainder on the remaining
# values. O(n^2), no listing.
# Complexity: Time O(n^2), Space O(n)
def kth_permutation(n, k):
    digits = list(range(1, n + 1))
    result = []
    k -= 1
    for left in range(n, 0, -1):
        block = factorial(left - 1)
        result.append(digits.pop(k // block))
        k %= block
    return result


# ---- follow-up 3: tests ----
def test_follow_up_3():
    for n in range(1, 6):
```

```
        everything = permute(list(range(1, n + 1)))
        for k in range(1, len(everything) + 1):
            assert kth_permutation(n, k) == everything[k - 1]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given an array of distinct integers, return all possible permutations of the array. A permutation of an array is a rearrangement of its elements. Example

```
Input: nums = [1, 2, 3]
Output: [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
```

OFFICIAL PAGE TEXT

Our solution generates all permutations of a given array by utilizing a recursive backtracking approach. The idea is to explore every possible way of arranging the elements by swapping them at each recursive call. Here's a breakdown of how the algorithm works: We define a helper function (backtrack) that recursively builds the permutations. This function takes two parameters: start (the index from where we start permuting) and end (the length of the array). The base case occurs when start == end, meaning that a complete permutation has been formed. We add a copy of the current permutation (stored in nums) to the result. In each recursive call, we iterate over the elements starting from the start index. For each element at position i, we: Swap the elements at start and i to bring a new element into the current position. Recursively call backtrack with start + 1, moving to the next index to fill. After the recursive call, we swap back the elements to restore the original order and continue exploring other possibilities. This process continues until all permutations are generated. The swapping ensures that we explore every possible combination of the elements. Time Complexity: The time complexity of this solution is O(n * n!), where n is the number of elements in the array. The reason for this is that generating each permutation takes O(n) time, and there are n! permutations. Space Complexity: The space complexity is O(n!) due to the storage of all permutations in the result list. Additionally, the recursion stack uses O(n) space for the recursive calls, but this is dominated by the result storage.No answers contributed by the community yet.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 8 OF 43 |

Runs and groups: extend while it fits, restart or close when it does not

7 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

7
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Runs and groups: extend while it fits, restart or close when it does not | 7 questions | tap a title to jump to it

READ FIRST: the code template for these 7 questions (tap here) >

M	44. Longest Non-Decreasing Subarray with One Replacement	1Point3Acres * 2026-10-04 *
M	45. Piano Hand Movement Grouping	1Point3Acres * 2026-07-14 *
M	46. Find longest increasing contiguous subarray	PracHub * 2026-01-06 *
M	47. Sum of Good Subarrays (Adjacent Diff ±1)	1Point3Acres * 2026-02-08 *
M	48. Sustained-Error Time Window	ColorStack * 2025-09-16 *
M	49. Longest Non-Decreasing Subarray From Two Arrays	ColorStack * 2025-09-24 *
M	50. Count Super Streaks	ColorStack * 2025-10-03 *

CODE TEMPLATE | GROUP 8 OF 43 | 7 QUESTIONS, ONE PATTERN

Runs and groups: extend while it fits, restart or close when it does not

The one idea these questions share

Walk the sequence once with a counter. If the next item fits the rule, the run grows; if not, the run restarts (or the group closes). Longest run, first break, number of groups, and sums over runs are all small changes to what you record.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def longest_run(a, fits):
    run = best = 1 if a else 0
    for i in range(1, len(a)):
        # CHANGE 1: the rule for continuing a run
        run = run + 1 if fits(a[i - 1], a[i]) else 1
        best = max(best, run)            # CHANGE 2: record longest, first break, count, sum
    return best
```

2. Helper functions that usually get written next to it

```
def groups_of(items, fits_group):          # close a group when the next item does not fit
    groups, current = [], []
    for x in items:
        if current and not fits_group(current, x):
            groups.append(current)
            current = []
        current.append(x)
    if current:
        groups.append(current)
    return groups


def breaks(a, fits):                       # every index where the rule fails
    return [i for i in range(1, len(a)) if not fits(a[i - 1], a[i])]
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
44	Longest Non-Decreasing Subarray with One Replacement	left[i] and right[i] run lengths; the best answer joins the run before and after one changed element
45	Piano Hand Movement Grouping	start a new group when adding the note would make max - min exceed k
46	Find longest increasing contiguous subarray	fits = a[i] > a[i-1]; also keep a left run and a right run per index so one change can join two runs
47	Sum of Good Subarrays (Adjacent Diff ±1)	keep the running length and running sum of the current up-run and down-run; add both to the total at every index
48	Sustained-Error Time Window	from the given second extend left and right while errors >= baseline; compare the time span with the interval
49	Longest Non-Decreasing Subarray From Two Arrays	two run lengths per index, one for each array you may take the value from
50	Count Super Streaks	sort by time; the same type within T continues the streak; when it breaks, count it if size >= N and span >= X

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 44 of 197 | topic 8 of 43: Runs and groups: extend while it fits, restart or close when it does not | 1 of 7

Template fit: left[i] and right[i] run lengths; the best answer joins the run before and after one changed element | * asked 16 days ago * | frequency Low

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 44 of 197

Longest Non-Decreasing Subarray with One Replacement

ASKED 2026-10-04 | 16 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 269/288**

ABOUT THIS QUESTION

Topic: Runs and groups: extend while it fits, restart or close when it does not

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/longest-non-decreasing-subarray-one-replacement

QUESTION

Find the maximum length of a contiguous non-decreasing subarray if at most one element may be changed to any value. Some rounds start from the plain longest-increasing-subarray question and add the one-replacement follow-up; those versions ask for an increasing run, so clarify strict vs non-strict.

Input: an integer array nums.
You may change at most one element to any value.
Return the maximum length of a contiguous subarray that is non-decreasing (nums[i] <= nums[i + 1] for every adjacent pair inside it) after the change.

Variant: other rounds ask for a strictly increasing run and open with the no-replacement version (longest increasing subarray) before adding the one-replacement follow-up — clarify strict vs non-strict before coding.

Expect to discuss and compare the complexity of several approaches; a brute-force first pass is usually followed by a request to optimize.

EXAMPLES

```
nums = [1, 0, 3, 4, 5, 2, 3]
Output: 5
Change nums[1] from 0 to 1 -> [1, 1, 3, 4, 5, 2, 3]
[1, 1, 3, 4, 5] is a non-decreasing subarray of length 5.
```

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    best = longest_non_decreasing(a)
    for i in range(len(a)):
        for v in candidate_values(a):
            b = a[:i] + [v] + a[i + 1:]
            run = longest_non_decreasing(b)
            best = max(best, run)
    return best
```

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

BRUTE FORCE

O(n^3). The run lengths on both sides of each position, computed once, answer every change in O(1).

OPTIMAL SOLUTION · Left and right run lengths

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The run must be strictly increasing.

Use < in the run lengths, and when the replaced value must fit between the neighbours check a[i+1] - a[i-1] >= 2 (an integer strictly between them exists) instead of a[i-1] <= a[i+1]. Clarify strict versus non-strict before coding.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Longest Non-Decreasing Subarray with One
# Replacement
#
# Find the maximum length of a contiguous non-decreasing
# subarray if at most one element may be changed to any
# value. Some rounds start from the plain longest-
# increasing-subarray question and add the one-replacement
# follow-up; those versions ask for an increasing run, so
# clarify strict vs non-strict. Input: an integer array
# nums. You may change at most one element to any value.
# Return the maximum length of a contiguous subarray that
# is non-decreasing (nums[i] <= nums[i + 1] for every
# adjacent pair inside it) after the change. Variant:
# other rounds ask for a strictly increasing run and open
# with the no-replacement version (longest increasing
# subarray) before adding the one-replacement follow-up —
# clarify strict vs non-strict before coding. Expect to
# discuss and compare the complexity of several
# approaches; a brute-force first pass is usually followed
# by a request to optimize.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The run must be strictly increasing.
#
# INTERVIEW NOTES
# Precompute left[i], the length of the non-decreasing run
# ending at i, and right[i], the length of the run
# starting at i. If you change element i to any value, it
# can join the run on its left and the run on its right
# when a[i-1] <= a[i+1] (a value between them exists);
# otherwise it can join only the longer of the two. Take
# the best over all i.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================
```

```
# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def in_order(x, y, strict):
    # may y follow x? non-decreasing: x <= y; strictly increasing: x < y
    if strict:
        return x < y
    return x <= y


def run_lengths(a, strict):
    # left[i] = length of the run that ENDS at i, right[i] = the run that STARTS at i
    n = len(a)
    left = [1] * n
    right = [1] * n
    for i in range(1, n):
        if in_order(a[i - 1], a[i], strict):
            left[i] = left[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if in_order(a[i], a[i + 1], strict):
            right[i] = right[i + 1] + 1
    return left, right


def bigger(x, y):
    if x > y:
        return x
    return y


# ---- main question: main function ----
# Idea: changing a[i] can join the run on its left and the run on its
# right: left[i-1] + 1 + right[i+1], but only if some value fits between
# the neighbours (a[i-1] <= a[i+1]). Otherwise it extends just the longer
# of the two runs. At the ends there is only one side to join.
# (strict=True is follow-up 1: strictly increasing runs, and then the
# neighbours must differ by at least 2 so that a whole number fits between.)
# Complexity: Time O(n), Space O(n)
def longest_with_one_change(a, strict=False):
    n = len(a)
    if n <= 2:
        return n                                    # nothing to fix
    gap = 0
    if strict:
        gap = 2
    left, right = run_lengths(a, strict)
    best = 1
    for i in range(n):
        left_run = 0
        if i > 0:
            left_run = left[i - 1]
        right_run = 0
        if i + 1 < n:
            right_run = right[i + 1]
        if left_run == 0 or right_run == 0 or a[i + 1] - a[i - 1] >= gap:
            best = bigger(best, left_run + 1 + right_run)      # the change joins both runs
        else:
            best = bigger(best, bigger(left_run, right_run) + 1)   # it joins only the longer one
    return best


# the warm-up: no replacement allowed
```

```
# Complexity: Time O(n), Space O(1)
def longest_without_change(a):
    best = 0
    run = 0
    for i in range(len(a)):
        if i > 0 and a[i - 1] < a[i]:
            run += 1
        else:
            run = 1
        best = bigger(best, run)
    return best


# ---- main question: tests ----
def test_main_question():
    assert longest_with_one_change([1, 0, 3, 4, 5, 2, 3]) == 5      # change 0 to 1: [1, 1, 3, 4, 5]
    assert longest_with_one_change([]) == 0
    assert longest_with_one_change([7]) == 1
    assert longest_without_change([1, 3, 5, 4, 7]) == 3


# ============================================================
# FOLLOW-UP 1: The run must be strictly increasing
# ============================================================
# Answer: use < instead of <= when building the runs, and when the
# replaced value must fit between its neighbours require
# a[i+1] - a[i-1] >= 2 (a whole number strictly between them exists).
# Clear up "strict or not" before you write code.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert longest_with_one_change([1, 2, 2, 3, 4], True) == 4
    assert longest_with_one_change([1, 1, 1, 1], True) == 2
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write the plain longest-run scan, then extend it with left-run / right-run arrays (or a two-state DP: "no change used" / "change used") so you can explain the jump from one to the other.
Test the bridge case explicitly: joining the run ending at i - 1 with the run starting at i + 1 is only valid when nums[i - 1] <= nums[i + 1]; in the strict variant there must be room for a value strictly between the neighbors, i.e. nums[i + 1] - nums[i - 1] >= 2 when the replacement must be an integer (plain < only if any real value is allowed). Otherwise the change only extends one side by one.
Repeat the exercise with the strict-increasing variant to make sure your comparisons flip correctly.

STUDY PATH: question 45 of 197 | topic 8 of 43: Runs and groups: extend while it fits, restart or close when it does not | 2 of 7

Template fit: start a new group when adding the note would make max - min exceed k | * asked 98 days ago * | frequency Low

MEDIUM | 1Point3Acres | onsite-coding | 45 of 197

Piano Hand Movement Grouping

ASKED 2026-07-14 | 98 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 265/288**

ABOUT THIS QUESTION

Topic: Runs and groups: extend while it fits, restart or close when it does not

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-07-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/piano-hand-movement-grouping

QUESTION

A pianist plays a sequence of notes in order (a bigger number is a higher key). The hand can stay in one position while every note it plays lies within a span of k (highest minus lowest at most k). Starting a new hand position costs a move. Return the smallest number of hand positions needed to play all the notes in order.

Function signature

def groups(notes: list, k: int) -> int

EXAMPLES

Example 1	
Input	`[1, 3, 5, 7, 9, 11], 4`
Output	`2`
Why	`[1, 3, 5] fits in a span of 4, then [7, 9, 11]: 2 positions.`

Example 2	
Input	`[1, 2, 3, 4, 5], 4`
Output	`1`
Why	`All five notes fit in a span of 4: one position.`

Example 3	
Input	`[5, 9, 1], 4`
Output	`2`
Why	`5 and 9 fit together, then 1 is too far from them: 2 positions.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = [INF] * (n + 1)
best[0] = 0
for j in range(1, n + 1):
    for i in range(j):
        part = notes[i:j]
        if max(part) - min(part) <= k:
            best[j] = min(best[j], best[i] + 1)
answer = best[n]
```

BRUTE FORCE

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every end tries every start and re-checks the range, O(n^2). A greedy pass that starts a new group only when the next note no longer fits is O(n).

OPTIMAL SOLUTION · Greedy scan, tracking the group's smallest and largest note

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return where each group starts and where to place the hand.

Record the start index when a group opens and emit (start, end, smallest note) when it closes. The state transition is unchanged.

FOLLOW-UP 2 Count lifts, not placements.

Lifts are placements minus one if the first placement is free; the examples differ on this, so ask and state it. In code it is a single subtraction on the group count.

FOLLOW-UP 3 The hand may slide without a lift (costing something).

Then it becomes a DP over positions with a cost per slide distance, solved with the O(n * range) DP or by greedy if the cost is monotone. Ask for the cost model.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Piano Hand Movement Grouping
#
# A pianist plays a sequence of notes in order (a bigger
# number is a higher key). The hand can stay in one
# position while every note it plays lies within a span of
# k (highest minus lowest at most k). Starting a new hand
# position costs a move. Return the smallest number of
# hand positions needed to play all the notes in order.
#
# Signature:
#   def groups(notes: list, k: int) -> int
#
# Examples:
# [1, 3, 5, 7, 9, 11], 4  ->  2
# [1, 2, 3, 4, 5], 4  ->  1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return where each group starts and where to place the
# hand.
# 2. Count lifts, not placements.
```

```
# 3. The hand may slide without a lift (costing
# something).
#
# INTERVIEW NOTES
# Turn the physical rule into an invariant: the current
# group is valid while (largest note - smallest note) <=
# k, because you can place the hand at the smallest. The
# first note that breaks it must start a new group, and
# starting later never helps, so a single greedy pass is
# optimal. Ask whether the first placement counts as a
# move.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def hand_positions(notes, k):
    # split the notes into groups that one hand position (a span of at most
```

```
    # k) can play, as early as possible. Each group is (start index,
    # lowest note, highest note).
    groups_found = []
    for i in range(len(notes)):
        x = notes[i]
        if groups_found:
            start, low, high = groups_found[-1]
            new_low = low
            if x < new_low:
                new_low = x
            new_high = high
            if x > new_high:
                new_high = x
            if new_high - new_low <= k:                # the note still fits the current span
                groups_found[-1] = (start, new_low, new_high)
                continue
        groups_found.append((i, x, x))                 # otherwise the hand must move: a new group starts
    return groups_found


# ---- main question: main function ----
# Idea: a greedy scan. Keep the lowest and highest note of the current
# group; extend the group while high - low stays within k, otherwise
# close it and start a new group at this note. The answer is the number
# of hand placements.
# Complexity: Time O(n), Space O(g) for g hand positions
def groups(notes, k):
    return len(hand_positions(notes, k))


# ---- main question: tests ----
def test_main_question():
    assert groups([1, 3, 5, 7, 9, 11], 4) == 2          # the example: 2 placements for reach 4
    assert groups([], 4) == 0
    assert groups([2, 2, 3], 0) == 2                    # a reach of 0: a new placement for every change


# ========================================================
# FOLLOW-UP 1: Return where each group starts and where to place the hand
```

```
# ============================================================
# ---- follow-up 1: function ----
# Answer: remember the start index when a group opens; a group ends just
# before the next one starts. Put the hand on the lowest note of the group.
# The state changes are the same.
# Complexity: Time O(n), Space O(g)
```

```
def plan(notes, k):
    spots = hand_positions(notes, k)
    result = []
    for n in range(len(spots)):
        start, low, high = spots[n]
        if n + 1 < len(spots):
            end = spots[n + 1][0] - 1
        else:
            end = len(notes) - 1
        result.append((start, end, low))
    return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert plan([1, 3, 5, 7, 9, 11], 4) == [(0, 2, 1), (3, 5, 7)]


# ============================================================
# FOLLOW-UP 2: Count lifts, not placements
# ============================================================
# ---- follow-up 2: function ----
# Answer: the first placement is free and every later one is a lift, so
# lifts = placements - 1 (never below 0). The examples differ on this, so
# ask and say which one you count.
# Complexity: Time O(n), Space O(g)
def lifts(notes, k):
    placements = groups(notes, k)
    if placements == 0:
        return 0
    return placements - 1


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert lifts([1, 2, 3, 4, 5], 4) == 0
    assert lifts([5, 9, 1], 4) == 1


# ============================================================
# FOLLOW-UP 3: The hand may slide without a lift (costing something)
# ============================================================
# Answer: now each step between groups costs min(lift cost, slide cost x
# distance moved). Keep the same groups, and add the cheaper of the two
# options for every move.
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Practice turning the physical reach rule into an explicit interval invariant before writing code.
Implement a one-pass grouping scan, then extend it to emit group boundaries without changing the core state transition.

STUDY PATH: question 46 of 197 | topic 8 of 43: Runs and groups: extend while it fits, restart or close when it does not | 3 of 7

Template fit: fits = a[i] > a[i-1]; also keep a left run and a right run per index so one change can join two runs | * asked 287 days ago *

MEDIUM | PracHub | Technical Screen | 46 of 197

Find longest increasing contiguous subarray

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 167/288

ABOUT THIS QUESTION

Topic: Runs and groups: extend while it fits, restart or close when it does not

Optimal answer from: the interview site's own solution

LeetCode match: LC 674 Longest Continuous Increasing Subsequence (variant: one modification) (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/find-longest-increasing-contiguous-subarray

QUESTION

Given an array you may change at most one element to any value (an integer when integers=True, any real number otherwise). Return the length of the longest strictly increasing contiguous subarray you can obtain.

Function signature

def longest_one_change(a: list, integers: bool = True) -> int

EXAMPLES

Example 1	
Input	`[1, 2, 3, 2, 5, 6]`
Output	`6`
Why	`Change the second 2 to 4 and the whole array 1, 2, 3, 4, 5, 6 increases.`

Example 2	
Input	`[1, 3, 2, 4]`
Output	`3`
Why	`Between 1 and 2 there is no integer, so the array 1, 3, 2, 4 only improves to a run of 3.`

Example 3	
Input	`[1, 3, 2, 4], integers = false`
Output	`4`
Why	`With real numbers, 3 can become 1.5 and the whole array increases.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(a):
    best = 0
    for i in range(len(a)):
        for j in range(i, len(a)):
            part = a[i:j + 1]
            if fixable_with_one_change(part):
                best = max(best, len(part))
    return best
def fixable_with_one_change(part):
    bad = [k for k in range(1, len(part))
           if part[k] <= part[k - 1]]
    if not bad:
        return True
    if len(bad) > 1:
        return False
    return has_room(part, bad[0])
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

O(n^2) subarrays, each tested in O(n): O(n^3). Run lengths ending and starting at each index, then trying each index as the changed element, is O(n).

OPTIMAL SOLUTION · Left and right run lengths, try every position as the changed element

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 No modification allowed (LeetCode 674).

One pass keeping the length of the current strictly increasing run; reset to 1 when a[i] <= a[i - 1]. O(n), O(1).

FOLLOW-UP 2 Up to k modifications.

Use a sliding window over the array that allows at most k positions where the increasing property fails and cannot be repaired; careful with the integer room condition between neighbours. Often handled with DP over (position, changes used).

FOLLOW-UP 3 Values are real numbers, not integers.

Then there is always room between two different neighbours, so the join condition becomes a[i + 1] > a[i - 1] instead of a gap of at least 2.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find longest increasing contiguous
# subarray
#
# Given an array you may change at most one element to any
```

```
# value (an integer when integers=True, any real number
# otherwise). Return the length of the longest strictly
# increasing contiguous subarray you can obtain.
#
# Signature:
#   def longest_one_change(a: list, integers: bool = True) -> int
#
# Examples:
# [1, 2, 3, 2, 5, 6]  ->  6
# [1, 3, 2, 4]  ->  3
#
# FOLLOW-UPS (answered below, in this order):
# 1. No modification allowed (LeetCode 674).
# 2. Up to k modifications.
# 3. Values are real numbers, not integers.
#
# INTERVIEW NOTES
# A run that has a single 'bad' spot can be repaired by
# changing one element, so look at each position i as the
# element to change. The best run through i joins the
# increasing run that ends just before it and the one that
# starts just after it, and the join is legal only if the
# values leave enough room between the two neighbours (for
# integers, a[i + 1] - a[i - 1] >= 2).

# =====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# =====================================================

# =====================================================
# MAIN QUESTION
# =====================================================
# ---- main question: helper functions ----
def bigger(x, y):
    if x > y:
        return x
    return y
```

```
# ---- main question: main function ----
# Idea: left[i] is the length of the strictly increasing run that ENDS
# at i, right[i] the length of the one that STARTS at i. Try every
# position as the one changed value: it can join the run on its left, the
# run on its right, or both if a value fits between its neighbours (for
# whole numbers the neighbours must differ by at least 2; for real
# numbers by more than 0).
# Complexity: Time O(n), Space O(n)
def longest_one_change(a, integers=True):
    n = len(a)
    if n == 0:
        return 0
    left = [1] * n
    right = [1] * n
    for i in range(1, n):
        if a[i] > a[i - 1]:
            left[i] = left[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if a[i] < a[i + 1]:
            right[i] = right[i + 1] + 1
    best = 1
    for i in range(n):
        best = bigger(best, left[i])                    # no change at all
```

```
        if i > 0:
            best = bigger(best, left[i - 1] + 1)          # change a[i] to continue the run on its left
        if i < n - 1:
            best = bigger(best, right[i + 1] + 1)         # or to start the run on its right
        if 0 < i < n - 1:                                 # or to join both, if a value fits between
            room = a[i + 1] - a[i - 1]
            if integers:
                fits = room >= 2
            else:
                fits = room > 0
            if fits:
                best = bigger(best, left[i - 1] + 1 + right[i + 1])
    if best > n:
        return n
    return best


# ---- main question: tests ----
def test_main_question():
    assert longest_one_change([1, 2, 3, 2, 5, 6]) == 6
    assert longest_one_change([1, 3, 2, 4]) == 3
```

```
    assert longest_one_change([1, 3, 5, 4, 7]) == 5
    assert longest_one_change([5, 4, 3, 2, 1]) == 2
    assert longest_one_change([7]) == 1
    assert longest_one_change([]) == 0
    assert longest_one_change([1, 2, 3]) == 3             # never more than the array length


# ========================================================
# FOLLOW-UP 1: No modification allowed (LeetCode 674)
# ========================================================
# ---- follow-up 1: function ----
# Answer: one pass with the length of the current strictly increasing
# run; start again at 1 when a[i] <= a[i - 1]. O(n) time, O(1) space.
# Complexity: Time O(n), Space O(1)
def longest_increasing(a):
    best = 0
    run = 0
    for i in range(len(a)):
        if i > 0 and a[i - 1] < a[i]:
            run += 1
        else:
            run = 1
        best = bigger(best, run)
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert longest_increasing([1, 3, 5, 4, 7]) == 3
    assert longest_increasing([]) == 0


# ========================================================
# FOLLOW-UP 2: Up to k modifications
# ========================================================
# Answer: use a sliding window that is allowed to contain at most k
# "bad" positions (places where the order breaks and has to be repaired);
# shrink from the left when there are more.


# ========================================================
# FOLLOW-UP 3: Values are real numbers, not integers
# ========================================================
```

```
# Answer: the `integers` parameter: with real numbers a value fits
# between two neighbours whenever they differ at all (room > 0), but with
```

```
# whole numbers they must differ by at least 2.
# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert longest_one_change([1, 3, 2, 4], integers=False) == 4
    assert longest_one_change([1, 2, 2, 3]) == 3           # equal neighbours leave no room for a whole num
ber
    assert longest_one_change([1, 2, 2, 3], integers=False) == 4


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def longestIncreasing(nums: list[int]) -> int:
```

Sample call

```
longestIncreasing([1, 2, 2, 3, 4])
```

Test cases

Input	Expected
([1, 2, 2, 3, 4],)	3
([1, 2, 3, 4, 5],)	5

STUDY PATH: question 47 of 197 | topic 8 of 43: Runs and groups: extend while it fits, restart or close when it does not | 4 of 7

Template fit: keep the running length and running sum of the current up-run and down-run; add both to the total at every index | * asked 254 days ago * | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 47 of 197

Sum of Good Subarrays (Adjacent Diff ±1)

ASKED 2026-02-08 | 254 DAYS AGO | OLDER

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 268/288**

ABOUT THIS QUESTION

Topic: Runs and groups: extend while it fits, restart or close when it does not

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-02-08 | Roles: MLE, SWE

Original: https://www.1point3acres.com/interview/problems/company/google/good-subarrays-diff-one

QUESTION

MLE L5 coding round: sum over all contiguous subarrays whose elements form a strictly monotonic ±1 step sequence (every adjacent pair differs by exactly +1, OR every adjacent pair differs by exactly -1). Required time complexity is O(N).

Input: integer array arr of length N.
A subarray is good if its consecutive differences are either all +1 (monotone increasing by 1) or all -1 (monotone decreasing by 1).
Singleton subarrays are always good.
Output: the sum of element-sums across all good subarrays (each good subarray contributes the sum of its elements once).
Required: O(N) time.

EXAMPLES

```
arr = [3, 5, 6, 7, 6]
```

Good subarrays

```
  singletons:           [3], [5], [6], [7], [6]
  monotone +1 runs:     [5, 6], [6, 7], [5, 6, 7]
  monotone -1 runs:     [7, 6]
Not good: [3, 5] (diff 2), [6, 7, 6] (mixes +1 then -1).
```

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    total = 0
    for i in range(len(a)):
        for j in range(i, len(a)):
            part = a[i:j + 1]
            if is_good(part):
                total += sum(part)
    return total
def is_good(part):
    pairs = zip(part, part[1:])
    steps = {y - x for x, y in pairs}
    return steps <= {1} or steps <= {-1}
```

BRUTE FORCE

TIME	SPACE
O(n^2) or O(n^3)	**O(1)**

WHY IT IS SLOW

O(n^2) subarrays (O(n^3) with the sums). Run totals per direction update in O(1) per element, so one pass is O(n).

OPTIMAL SOLUTION · Two running run totals

TIME	SPACE
O(n)	**O(1)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is the singleton subtracted?

A one-element subarray [x] is both an increasing and a decreasing run of length 1, so it appears in both running totals. Subtracting x once at each index keeps it counted exactly once. A subarray of length 2 or more can only be in one total because its step is +1 or -1, not both.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Sum of Good Subarrays (Adjacent Diff ±1)
#
# MLE L5 coding round: sum over all contiguous subarrays
# whose elements form a strictly monotonic ±1 step
# sequence (every adjacent pair differs by exactly +1, OR
# every adjacent pair differs by exactly -1). Required
# time complexity is O(N). Input: integer array arr of
# length N. A subarray is good if its consecutive
# differences are either all +1 (monotone increasing by 1)
# or all -1 (monotone decreasing by 1). Singleton
# subarrays are always good. Output: the sum of element-
# sums across all good subarrays (each good subarray
# contributes the sum of its elements once). Required:
# O(N) time. Examples: arr = [3, 5, 6, 7, 6] Good
# subarrays: singletons: [3], [5], [6], [7], [6] monotone
# +1 runs: [5, 6], [6, 7], [5, 6, 7] monotone -1 runs: [7,
# 6] Not good: [3, 5] (diff 2), [6, 7, 6] (mixes +1 then
# -1).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is the singleton subtracted?
#
# INTERVIEW NOTES
# A good subarray is a stretch where every step is +1 or
# every step is -1, so every good subarray ending at index
# i lies inside the current run. Instead of listing
# subarrays, keep for each direction the total of all
# suffix sums inside the run: extending a run of length L
# by a new element x adds (L + 1) * x to that total. Both
# directions can be updated in O(1), and the singleton is
# counted once.
```

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# A subarray is GOOD when every step between neighbours is +1, or every
# step is -1 (a single element is good). Return the sum of the elements
# of all good subarrays.
# Idea: go left to right and, for each direction, keep the total of the
# sums of all good subarrays that END at the current element, and how
# many there are. When the next element x continues a +1 run, every old
# subarray grows by x and the new one [x] appears: the total grows by
# (count + 1) * x. A break restarts the total at x. The single [x] is in
# both the "up" and the "down" total, so subtract it once.
# Complexity: Time O(n), Space O(1)
def good_subarray_sum(a):
    total = 0
    up_sum = 0
    up_count = 0
    down_sum = 0
    down_count = 0
    for i in range(len(a)):
        x = a[i]
        step = None
        if i > 0:
            step = x - a[i - 1]
        if step == 1:
            up_sum += (up_count + 1) * x
            up_count += 1
        else:
            up_sum = x
            up_count = 1
        if step == -1:
            down_sum += (down_count + 1) * x
            down_count += 1
        else:
            down_sum = x
            down_count = 1
        total += up_sum + down_sum - x
    return total


# ---- main question: tests ----
def test_main_question():
    # singletons 27 + [5,6] 11 + [6,7] 13 + [5,6,7] 18 + [7,6] 13
    assert good_subarray_sum([3, 5, 6, 7, 6]) == 82
    assert good_subarray_sum([]) == 0
    assert good_subarray_sum([4]) == 4


# ========================================================
# FOLLOW-UP 1: Why is the singleton subtracted?
# ========================================================
# Answer: a one-element subarray [x] is good in BOTH directions, so it
# is inside up_sum and inside down_sum; counting it twice would be wrong.
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Practice the "contribution-per-index" pattern: the sum-of-subarray-minimums and sum-of-subsequence-widths problems both use the same technique.
Practice splitting an array into maximal monotone-step runs, then computing per-run contributions in O(1) amortized using prefix sums of element values.
Sanity-check with arr = [1, 2, 3] and arr = [3, 2, 1] by hand — both should sum to 20.

STUDY PATH: question 48 of 197 | topic 8 of 43: Runs and groups: extend while it fits, restart or close when it does not | 5 of 7

Template fit: from the given second extend left and right while errors >= baseline; compare the time span with the interval | * asked 399 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 48 of 197

Sustained-Error Time Window

ASKED 2025-09-16 | 399 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 217/288

ABOUT THIS QUESTION

Topic: Runs and groups: extend while it fits, restart or close when it does not

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #1004 Max Consecutive Ones III

LeetCode match: LC 1004 Max Consecutive Ones III (related: runs of readings)

Frequency: Single report | Last asked: 2025-09-16 | Roles: SWE Intern

QUESTION

Reconstructing the "two arrays - Error and Time" question. You are given two parallel arrays: times (timestamps in seconds, strictly increasing) and errors (the error reading recorded at each timestamp), where errors[i] was recorded at times[i] . You are also given three integers: second , interval , and baseline . Determine whether there exists a contiguous run of readings that:

1. Includes the reading taken at timestamp second (guaranteed to exist exactly once in times), 2. Has every reading in the run satisfy error >= baseline , and 3. Spans a duration of at least interval seconds — that is, times[right] - times[left] >= interval , where left and right are the endpoints of the run. Return True if such a run exists, otherwise False . Note that if the reading at second is itself below baseline , the answer is immediately False .

Function signature

```
def has_sustained_error_window(
    times: list[int],
    errors: list[int],
    second: int,
    interval: int,
    baseline: int,
) -> bool:
    """Return True if a run of readings all >= baseline includes second and spans >=
interval seconds."""
    ...
```

EXAMPLES

Example 1	
Input	`times = [1, 2, 3, 4, 5], errors = [10, 20, 5, 30, 40], second = 1, interval = 1, baseline = 8`
Output	`True`
Why	`Starting at time 1 (err 10 >= 8) the run extends to time 2 (err 20 >= 8) but stops at time 3 (err 5 < 8). The run covers times 1..2, a span of 1 >= interval 1.`

QUESTION

Example 2	
Input	`times = [1, 2, 3, 4, 5], errors = [10, 20, 5, 30, 40], second = 4, interval = 2, baseline = 8`
Output	`False`
Why	`Time 4 (err 30 >= 8) can only extend right to time 5 (err 40 >= 8); time 3 (err 5) blocks going left. The run covers times 4..5, span 1 < interval 2.`

Example 3	
Input	`times = [1, 2, 3], errors = [10, 4, 10], second = 2, interval = 0, baseline = 8`
Output	`False`
Why	`The reading at second=2 is 4 < baseline 8, so no valid run includes it.`

CONSTRAINTS

- 1 <= len(times) == len(errors) <= 10^5 times is strictly increasing; 0 <= times[i] <= 10^9
- -10^9 <= errors[i] <= 10^9 , baseline in the same range second is one of the values in times ; 0 <= interval <= 10^9

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_sustained(
    times, errors, second, interval, baseline
):
    at = times.index(second)
    for left in range(at_+_1):
        for right in range(at,_len(times)):
            run = errors[left : right + 1]
            if (
                min(run) >= baseline
                and times[right] - times[left]
                >= interval
            ):
                return True
    return False
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

Every candidate run is re-checked reading by reading.

OPTIMAL SOLUTION · Expand around the reading

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 You get thousands of (second, interval) queries on the same data.

Compute once, for every index, the start and end of the maximal run of readings at or above the baseline (one left-to-right pass and one right-to-left pass). Each query is then a lookup plus a binary search for the index of second, O(log n).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Sustained-Error Time Window
#
# Reconstructing the "two arrays - Error and Time"
# question. You are given two parallel arrays: times
# (timestamps in seconds, strictly increasing) and errors
# (the error reading recorded at each timestamp), where
# errors[i] was recorded at times[i] . You are also given
# three integers: second , interval , and baseline .
# Determine whether there exists a contiguous run of
# readings that: 1. Includes the reading taken at
# timestamp second (guaranteed to exist exactly once in
# times ), 2. Has every reading in the run satisfy error
# >= baseline , and 3. Spans a duration of at least
# interval seconds — that is, times[right] - times[left]
# >= interval , where left and right are the endpoints of
# the run. Return True if such a run exists, otherwise
# False . Note that if the reading at second is itself
# below baseline , the answer is immediately False .
#
# FOLLOW-UPS (answered below, in this order):
# 1. You get thousands of (second, interval) queries on
# the same data.
#
# INTERVIEW NOTES
# The run that contains the reading at second is the
# maximal block of consecutive readings with error >=

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# The readings are (time, error count) pairs in time order. Is the error
# count AT `second` part of a stretch of readings, all at or above the
# `baseline`, that lasts at least `interval` seconds?
# Idea: the longest stretch around that reading is the best one, so only
# its two ends matter: expand to the left and right while the readings
# stay at or above the baseline.
# Complexity: Time O(n) per query, Space O(1)
```

```
def has_sustained_error_window(times, errors, second, interval, baseline):
    at = times.index(second)
    if errors[at] < baseline:
        return False
    left = at
    right = at
    while left > 0 and errors[left - 1] >= baseline:
```

```
        left -= 1
    while right + 1 < len(times) and errors[right + 1] >= baseline:
        right += 1
    return times[right] - times[left] >= interval


# ---- main question: tests ----
def test_main_question():
    times = [1, 2, 3, 4, 5]
    errors = [10, 20, 5, 30, 40]
    assert has_sustained_error_window(times, errors, 1, 1, 8) is True
    assert has_sustained_error_window(times, errors, 4, 2, 8) is False
    assert has_sustained_error_window(times, errors, 3, 0, 8) is False


# ======================================================
# FOLLOW-UP 1: You get thousands of (second, interval) queries on the same data
# ======================================================
# ---- follow-up 1: helper functions ----
def run_bounds(times, errors, baseline):
    # for every reading: the index where its stretch (readings at or above
    # the baseline) starts and ends; one pass from the left, one from the right
    n = len(times)
    left = [0] * n
    right = [0] * n
    for i in range(n):
        left[i] = i
        if i > 0 and errors[i - 1] >= baseline and errors[i] >= baseline:
            left[i] = left[i - 1]
    for i in range(n - 1, -1, -1):
        right[i] = i
        if i + 1 < n and errors[i + 1] >= baseline and errors[i] >= baseline:
            right[i] = right[i + 1]
    return left, right


def find_index(times, second):
    # binary search in the sorted times: the index of the first time >= second
```

```
    low = 0
    high = len(times)
    while low < high:
        middle = (low + high) // 2
        if times[middle] < second:
            low = middle + 1
        else:
            high = middle
    return low


# ---- follow-up 1: function ----
# Answer: compute once, for every reading, where its stretch starts and
# ends (run_bounds); then each query is a binary search for the reading
# plus two lookups: O(log n) per query.
# Complexity: Time O(log n) per query after an O(n) preparation, Space O(n) for the prepared bounds
def sustained_query(times, errors, bounds, second, interval, baseline):
    at = find_index(times, second)
    if errors[at] < baseline:
        return False
    left, right = bounds
    return times[right[at]] - times[left[at]] >= interval


# ---- follow-up 1: tests ----
def test_follow_up_1():
```

```
    times = [1, 2, 3, 4, 5, 8, 9]
    errors = [10, 20, 5, 30, 40, 50, 9]
    bounds = run_bounds(times, errors, 8)
    for second in times:
        for interval in (0, 1, 2, 3):
            expected = has_sustained_error_window(times, errors, second, interval, 8)
            assert sustained_query(times, errors, bounds, second, interval, 8) == expected


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

STUDY PATH: question 49 of 197 | topic 8 of 43: Runs and groups: extend while it fits, restart or close when it does not | 6 of 7

Template fit: two run lengths per index, one for each array you may take the value from | * asked 391 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 49 of 197

Longest Non-Decreasing Subarray From Two Arrays

ASKED 2025-09-24 | 391 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 222/288

ABOUT THIS QUESTION

Topic: Runs and groups: extend while it fits, restart or close when it does not

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 2771 Longest Non-decreasing Subarray From Two Arrays

Frequency: Single report | Last asked: 2025-09-24 | Roles: SWE Intern

QUESTION

You are given two integer arrays nums1 and nums2 , both of length n . You must construct a third array nums3 of length n where, for each index i , nums3[i] is chosen to be either nums1[i] or nums2[i] (chosen independently per index). Return the length of the longest non-decreasing subarray (a contiguous run) achievable in nums3 over all valid choices.

Function signature

```
def longest_non_decreasing_subarray(nums1: list[int], nums2: list[int]) -> int:
    ...
```

Follow-up (as reported): (Reported) The interviewer first asked for the longest contiguous non-decreasing subarray of a single array, then generalized to picking per index from two arrays; also asked for the time complexity and to write test cases.

EXAMPLES

Example 1	
Input	`nums1 = [2, 3, 1], nums2 = [1, 2, 1]`
Output	`2`
Why	`Choosing nums3 = [2, 3, ...] gives the run [2, 3] of length 2. No choice yields a non-decreasing run of length 3.`

Example 2	
Input	`nums1 = [1, 3, 2, 1], nums2 = [2, 2, 3, 4]`
Output	`4`
Why	`nums3 = [1, 2, 3, 4] (pick nums1[0], nums2[1], nums2[2], nums2[3]) is non-decreasing over all 4 positions.`

Example 3	
Input	`nums1 = [1, 1], nums2 = [2, 2]`
Output	`2`
Why	`Any choice, e.g. [1, 1], is non-decreasing across both.`

CONSTRAINTS

- 1 <= n <= 10^5
- nums1 and nums2 have the same length n .
- 1 <= nums1[i], nums2[i] <= 10^9

QUESTION

- Target complexity: O(n) time, O(1) extra space.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
from itertools import product
def brute_longest(nums1, nums2):
    best = 0
    for picks in product(
        (0, 1), repeat=len(nums1)
    ):
        arr = [
            (nums1, nums2)[p][i]
            for i, p in enumerate(picks)
        ]
        run = 1
        best = max(best, run)
        for a, b in zip(arr, arr[1:]):
            run = run + 1 if a <= b else 1
            best = max(best, run)
    return best
```

TIME	SPACE
O(2^n * n)	O(n)

WHY IT IS SLOW

It builds all 2^n possible nums3 arrays.

OPTIMAL SOLUTION · Two-state DP

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 First solve it for a single array, then generalize to picking per index from two arrays.

One array: run = run + 1 if nums[i - 1] <= nums[i] else 1 and keep the maximum, O(n) time and O(1) space. The two-array version keeps one run length per choice instead of one.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Longest Non-Decreasing Subarray From Two
# Arrays
#
# You are given two integer arrays nums1 and nums2 , both
# of length n . You must construct a third array nums3 of
# length n where, for each index i , nums3[i] is chosen to
# be either nums1[i] or nums2[i] (chosen independently per
# index). Return the length of the longest non-decreasing
# subarray (a contiguous run) achievable in nums3 over all
# valid choices. Function signature: def
```

```
# longest_non_decreasing_subarray(nums1: list[int], nums2:
# list[int]) -> int: ... Follow-up (as reported):
# (Reported) The interviewer first asked for the longest
# contiguous non-decreasing subarray of a single array,
# then generalized to picking per index from two arrays;
# also asked for the time complexity and to write test
# cases.
#
# FOLLOW-UPS (answered below, in this order):
# 1. First solve it for a single array, then generalize to
# picking per index from two arrays.
#
# INTERVIEW NOTES
# Choosing nums3[i] from either array is a two-state DP:
# keep the length of the best run that ends at i if we
# picked nums1[i], and the same if we picked nums2[i].
# Each state extends the previous run when the previous
# choice is not larger than the current value.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def bigger(x, y):
    if x > y:
        return x
    return y
```

```

# ---- main question: main function ----
# At every index you may take the value from nums1 or from nums2. Find
# the longest non-decreasing subarray you can build.
# Idea (two-state DP): dp1 = the best run ending here if we take nums1[i],
# dp2 = the same if we take nums2[i]. Each can extend the run that ended
# at the previous index with either choice, if the order holds.
# Complexity: Time O(n), Space O(1)
def longest_non_decreasing_subarray(nums1, nums2):
    dp1 = 1
    dp2 = 1
    best = 1
    for i in range(1, len(nums1)):
        new1 = 1
        new2 = 1
        if nums1[i - 1] <= nums1[i]:
            new1 = bigger(new1, dp1 + 1)
        if nums2[i - 1] <= nums1[i]:
            new1 = bigger(new1, dp2 + 1)
        if nums1[i - 1] <= nums2[i]:
            new2 = bigger(new2, dp1 + 1)
        if nums2[i - 1] <= nums2[i]:
            new2 = bigger(new2, dp2 + 1)
        dp1 = new1
        dp2 = new2
        best = bigger(best, bigger(dp1, dp2))
    return best


# ---- main question: tests ----
```

```
def test_main_question():
    assert longest_non_decreasing_subarray([2, 3, 1], [1, 2, 1]) == 2
    assert longest_non_decreasing_subarray([1, 3, 2, 1], [2, 2, 3, 4]) == 4
    assert longest_non_decreasing_subarray([1, 1], [2, 2]) == 2


# ========================================================
# FOLLOW-UP 1: First solve it for a single array, then generalize to picking per index from two arrays
# ========================================================
# ---- follow-up 1: function ----
# Answer: for one array: run = run + 1 if nums[i - 1] <= nums[i] else 1,
# and keep the maximum: O(n) time, O(1) space. The two-array version keeps
# one run length per CHOICE instead of one.
# Complexity: Time O(n), Space O(1)
def longest_non_decreasing_one(nums):
```

```
    if not nums:
        return 0
    best = 1
    run = 1
    for i in range(1, len(nums)):
        if nums[i - 1] <= nums[i]:
            run += 1
        else:
            run = 1
        best = bigger(best, run)
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert longest_non_decreasing_one([1, 2, 2, 1, 5]) == 3
    assert longest_non_decreasing_one([]) == 0
    assert longest_non_decreasing_one([3]) == 1


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 50 of 197 | topic 8 of 43: Runs and groups: extend while it fits, restart or close when it does not | 7 of 7

Template fit: sort by time; the same type within T continues the streak; when it breaks, count it if size >= N and span >= X | * asked 382 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 50 of 197

Count Super Streaks

ASKED 2025-10-03 | 382 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 225/288

ABOUT THIS QUESTION

Topic: Runs and groups: extend while it fits, restart or close when it does not

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #128 Longest Consecutive Sequence

LeetCode match: Sorting + grouping (related: LC 128 / session windows)

Frequency: Single report | Last asked: 2025-10-03 | Roles: SWE Intern

QUESTION

A player produces a list of events, each (event_id, event_type, event_time) . Sort events by event_time . A streak is a maximal run of events, consecutive in time order, in which every event has the same event_type AND the gap between each event and the previous one in the run is at most T seconds. A streak ends when the next event has a different event_type or its gap from the previous event exceeds T . A streak is a Super Streak if it contains at least N events AND spans at least X seconds (span = last event time minus first event time within the streak). Return the number of Super Streaks.

Function signature

```
def count_super_streaks(events: list[tuple[int, str, int]],
                        N: int, T: int, X: int) -> int:
    ...
```

Follow-up (as reported): Follow-up 1: events are tagged with user_id ; return a per-user count of Super Streaks. Follow-up 2: after processing, support count_streaks(user_id, time_A, time_B) returning how many of that user's Super Streaks completed entirely within [time_A, time_B] . Store each user's Super-Streak end-times sorted and answer with two binary searches in O(log n) per query.

EXAMPLES

Example 1	
Input	`events = [(1,"A",0),(2,"A",10),(3,"A",20),(4,"A",30),(5,"A",40)], N=4, T=10, X=30`
Output	`1`
Why	`One streak of 5 A-events, gaps all 10 (<=T), span 40 (>=30), size 5 (>=4). Super.`

Example 2	
Input	`events = [(1,"A",0),(2,"A",10),(3,"A",20),(4,"A",30),(5,"A",40), (6,"A",55),(7,"A",56),(8,"A",57)], N=4, T=10, X=30`
Output	`1`
Why	`The jump 40->55 (gap 15 > T) breaks the run. The first streak [0..40] is Super; the second [55,56,57] has only 3 events and span 2, so not Super.`

QUESTION

Example 3	
Input	`events = [(1,"A",0),(2,"B",5),(3,"A",10)], N=2, T=10, X=1`
Output	`0`
Why	`Type changes each step, so every streak has length 1 — none reaches N=2.`

CONSTRAINTS

- 0 <= len(events) <= 10^5 .
- N >= 1 , T >= 0 , X >= 0 ; times are non-negative integers, possibly with duplicate timestamps.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_super(events, N, T, X):
    events = sorted(events, key=lambda e: e[2])
    streaks = []
    for e in events:
        if (
            streaks
            and streaks[-1][-1][1] == e[1]
            and e[2] - streaks[-1][-1][2] <= T
        ):
            streaks[-1].append(e)
        else:
            streaks.append([e])
    return sum(
        1
        for s in streaks
        if len(s) >= N
        and s[-1][2] - s[0][2] >= X
    )
```

TIME	SPACE
O(n log n)	O(n)

WHY IT IS SLOW

It stores every streak as a list although only the size and the first and last time matter.

OPTIMAL SOLUTION · One pass over the sorted events

TIME	SPACE
O(n log n)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Events carry a user_id. Return the number of super streaks for each user.

Group the events by user first and run the same pass on each group, or key the open streak state by user in one pass over the sorted events. The cost stays O(n log n).

FOLLOW-UP 2 Then answer count_streaks(user_id, time_A, time_B): how many of that user's super streaks finished entirely inside [A, B]?

Record each super streak as (start, end) when it closes, per user, sorted by start (they do not overlap). A query counts the streaks with start >= A and end <= B: bisect on the starts and scan, or keep the ends in a second sorted list. O(log s + answer) per query.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count Super Streaks
#
# A player produces a list of events, each (event_id,
# event_type, event_time) . Sort events by event_time . A
# streak is a maximal run of events, consecutive in time
# order, in which every event has the same event_type AND
# the gap between each event and the previous one in the
# run is at most T seconds. A streak ends when the next
# event has a different event_type or its gap from the
# previous event exceeds T . A streak is a Super Streak if
# it contains at least N events AND spans at least X
# seconds (span = last event time minus first event time
# within the streak). Return the number of Super Streaks.
# Function signature: def count_super_streaks(events:
# list[tuple[int, str, int]], N: int, T: int, X: int) ->
# int: ... Follow-up (as reported): Follow-up 1: events
# are tagged with user_id ; return a per-user count of
# Super Streaks.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Events carry a user_id. Return the number of super
# streaks for each user.
# 2. Then answer count_streaks(user_id, time_A, time_B):
# how many of that user's super streaks finished entirely
# inside [A, B]?
#
# INTERVIEW NOTES
# Sort by time, then one pass: extend the current streak
# while the type is the same and the gap from the previous
# event is at most T; otherwise close it. A closed streak
# counts when it has at least N events and spans at least
# X seconds. Only the streak start, the previous time and
# the size are needed.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
```

```
def event_time(event):
    # an event is (id, type, time) or (id, type, time, user)
    return event[2]
```

```python
def find_streaks(events, N, T, X):
    # A STREAK is a run of events of the same type where each event comes at
    # most T after the previous one. It is a SUPER streak if it has at least N
    # events and lasts at least X from its first to its last event.
    # Returns the (start time, end time) of every super streak.
    spans = []
    start = None
    previous = None
    kind = None
    size = 0
    for event in sorted(events, key=event_time):
        event_type = event[1]
        time = event[2]
        if kind == event_type and time - previous <= T:
            size += 1                                   # the open streak continues
        else:
            if size > 0 and size >= N and previous - start >= X:
                spans.append((start, previous))         # the open streak was a super streak
            start = time
            kind = event_type
            size = 1
        previous = time
    if size > 0 and size >= N and previous - start >= X:
        spans.append((start, previous))
    return spans


# ---- main question: main function ----
# Idea: one pass over the events sorted by time. Keep only the open
# streak (when it started, the time of its last event, its size and its
# type); when an event does not continue it, check whether it was a
# super streak and start a new one.
# Complexity: Time O(n log n) for n events (they are sorted by time), Space O(n)
def count_super_streaks(events, N, T, X):
    return len(find_streaks(events, N, T, X))


# ---- main question: tests ----
EVENTS = [(1, "A", 0), (2, "A", 10), (3, "A", 20), (4, "A", 30), (5, "A", 40)]
```

```python
def test_main_question():
    assert count_super_streaks(EVENTS, 4, 10, 30) == 1
    more = EVENTS + [(6, "A", 55), (7, "A", 56), (8, "A", 57)]
    assert count_super_streaks(more, 4, 10, 30) == 1       # the second run is too short
    assert count_super_streaks([], 1, 1, 0) == 0


# ========================================================
# FOLLOW-UP 1: Events carry a user_id. Return the number of super streaks for each user
# ========================================================
# ---- follow-up 1: helper functions ----
def group_by_user(events):
    # events are (id, type, time, user); returns {user: [(id, type, time), ...]}
    by_user = {}
    for event_id, event_type, time, user in events:
        if user not in by_user:
            by_user[user] = []
        by_user[user].append((event_id, event_type, time))
    return by_user


# ---- follow-up 1: function ----
# Answer: group the events by user and run the same pass on each group
# (or keep one open streak per user in a single pass). Cost O(n log n).
```

```
# Complexity: Time O(n log n), Space O(n)
def super_streaks_per_user(events, N, T, X):
    counts = {}
    by_user = group_by_user(events)
    for user in by_user:
        counts[user] = count_super_streaks(by_user[user], N, T, X)
    return counts


# ---- follow-up 1: tests ----
def test_follow_up_1():
    tagged = [(1, "A", 0, "u"), (2, "A", 10, "u"), (3, "A", 20, "u"), (4, "A", 30, "u"),
              (5, "B", 5, "v"), (6, "B", 6, "v")]
    assert super_streaks_per_user(tagged, 4, 10, 30) == {"u": 1, "v": 0}


# ========================================================
# FOLLOW-UP 2: Then answer count_streaks(user_id, time_A, time_B): how many of that user's super streaks fi
nished entirely inside [A, B]?
# ========================================================
# ---- follow-up 2: function ----
```

```
# Answer: record each super streak as (start, end) when it closes, per
# user (they do not overlap, so they are already sorted by start). A
# query counts the streaks with start >= A and end <= B. (For many
# queries, binary-search the first start >= A.)
# Complexity: Time build O(n log n), each query O(streaks of that user), Space O(n)
class StreakIndex:
    def __init__(self, events, N, T, X):
        self.spans = {}
        by_user = group_by_user(events)
        for user in by_user:
            self.spans[user] = find_streaks(by_user[user], N, T, X)

    def count_streaks(self, user, a, b):
        count = 0
        for start, end in self.spans.get(user, []):
            if start >= a and end <= b:
                count += 1
        return count


# ---- follow-up 2: tests ----
def test_follow_up_2():
    tagged = [(1, "A", 0, "u"), (2, "A", 10, "u"), (3, "A", 20, "u"), (4, "A", 30, "u"),
              (5, "A", 100, "u"), (6, "A", 110, "u"), (7, "A", 120, "u"), (8, "A", 130, "u")]
    index = StreakIndex(tagged, 4, 10, 30)
    assert index.count_streaks("u", 0, 200) == 2
    assert index.count_streaks("u", 0, 50) == 1
    assert index.count_streaks("u", 5, 200) == 1           # the first streak started before 5
    assert index.count_streaks("nobody", 0, 200) == 0


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 9 OF 43 |

Union-find: merge the pairs, then ask who is together

7 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

7
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Union-find: merge the pairs, then ask who is together | 7 questions | tap a title to jump to it

READ FIRST: the code template for these 7 questions (tap here) >

M	51. Count Islands in a Grid and Track the Count as Land Cells Are Added	PracHub * 2026-10-02 *
M	52. 2D Points Connected Components by Radius	1Point3Acres * 2026-08-04 *
M	53. Count Clusters of 2D Points Within a Radius	PracHub * 2026-06-18 *
M	54. Find A Threshold-Limited Path With Minimum Required Safety	PracHub * 2026-07-06 *
M	55. Determine Reporting Relationships	PracHub * 2026-02-22 *
M	56. Determine whether two nodes are related	PracHub * 2026-01-22 *
M	57. Movie Recommendations via Transitive Similarity	ColorStack * 2025-10-09 *

CODE TEMPLATE | GROUP 9 OF 43 | 7 QUESTIONS, ONE PATTERN

Union-find: merge the pairs, then ask who is together

The one idea these questions share

Keep a parent array. find climbs to the root (with path compression), union joins two roots. The only thing that changes between questions is where the pairs to merge come from and what you read at the end: same group, group size, number of groups, or the moment everything became one group.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.groups = n

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        self.parent[ra] = rb
        self.groups -= 1
        return True


# for a, b in pairs:        # CHANGE 1: where the pairs come from
#     dsu.union(a, b)
# CHANGE 2: read dsu.find(x) == dsu.find(y), dsu.groups, or the pair that made groups == 1
```

2. Helper functions that usually get written next to it

```
from collections import defaultdict


def cell_buckets(points, r):               # spatial hash: only nearby cells can hold neighbours
    cells = defaultdict(list)
    for i, (x, y) in enumerate(points):
        cells[(x // r, y // r)].append(i)
    return cells


def group_sizes(dsu, n):
    sizes = defaultdict(int)
    for x in range(n):
        sizes[dsu.find(x)] += 1
    return sizes
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
51	Count Islands in a Grid and Track the Count as Land Cells Are Added	each new land cell: groups += 1, then union it with every land neighbour (groups -= 1 per real merge); report groups after each addition
52	2D Points Connected Components by Radius	pairs = points within distance r, found by bucketing points into cells of size r and checking only the 9 nearby cells; answer = groups

Contents Start the questions >

#	Question	What you change in the template
53	Count Clusters of 2D Points Within a Radius	identical to the line above: cell buckets give the candidate pairs, answer = groups
54	Find A Threshold-Limited Path With Minimum Required Safety	pairs = edges sorted from the highest safety down; union until find(s) == find(t); the answer is the safety of that last edge
55	Determine Reporting Relationships	pairs = peer links; keep the boss at the root and carry it over when two groups merge
56	Determine whether two nodes are related	pairs = the given edges; the query is find(a) == find(b)
57	Movie Recommendations via Transitive Similarity	pairs = similar movies (use a dict instead of a list); group = every movie with the same root as the target, sorted by rating

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 51 of 197 | topic 9 of 43: Union-find: merge the pairs, then ask who is together | 1 of 7

Template fit: each new land cell: groups += 1, then union it with every land neighbour (groups -= 1 per real merge); report groups after each addition | * asked 18 days ago *

MEDIUM | IN APP LC 305 | PracHub | Technical Screen | 51 of 197

Count Islands in a Grid and Track the Count as Land Cells Are Added

POSTED 2026-10-02 | 18 DAYS AGO | LAST 30 DAYS

STUDY: TALK | ALL TIME 36 | 30 DAYS 0 | PRIORITY 120/288

ABOUT THIS QUESTION

Topic: Union-find: merge the pairs, then ask who is together

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #305 Number of Islands II

LeetCode match: LC 305 Number of Islands II

Posted on PracHub: 2026-10-02 (updated 2026-10-03)

Original: https://prachub.com/coding-questions/count-islands-in-a-grid-and-track-the-count-as-land-cells-are-added

QUESTION

A map is a grid with `m` rows and `n` columns in which every cell is land (`1`) or water (`0`). An island is a maximal group of land cells connected horizontally or vertically. Cells that touch only at a corner are not connected. You are given the initial map and a sequence of additions, each of which turns one cell into land. Report the number of islands before any addition, and again after each addition.

Function Signature

```
def island_counts(grid: list[list[int]], additions: list[tuple[int, int]]) -> list[int]:
```

Rules

- `grid[r][c]` is `1` for land and `0` for water, where `r` is the row index and `c` is the column index.
- An addition `(r, c)` turns cell `(r, c)` into land. If that cell is already land, either initially or because of an earlier addition, the map does not change.
- Additions are applied in the given order. Land never turns back into water.
- Return a list of length `len(additions) + 1`. Element `0` is the number of islands in the initial grid. For `i >= 1`, element `i` is the number of islands after the first `i` additions have been applied.

Constraints

- `1 <= m <= 1000`, `1 <= n <= 1000` and `m * n <= 200000`
- Every row of `grid` has exactly `n` entries, each `0` or `1`.
- `0 <= len(additions) <= 200000`
- Every addition `(r, c)` satisfies `0 <= r < m` and `0 <= c < n`. The same cell may appear more than once.
- Every count is at most `m * n`, so all values fit in a 32-bit signed integer.

Examples

Example 1

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], additions = []
Output: [3]
```

The islands are {(0, 0), (0, 1)}, {(1, 3), (2, 3)} and {(2, 0)}. With no additions, the only element is the initial count.

Example 2

QUESTION

```
Input: grid = [[0, 0, 0], [0, 0, 0], [0, 0, 0]], additions = [(0, 0), (0, 1), (1, 2), (2, 1), (1, 1), (1, 1)]
Output: [0, 1, 1, 2, 3, 1, 1]
```

The grid starts with no land. (0, 0) creates an island and (0, 1) joins it. (1, 2) and (2, 1) each touch no land, so each creates a new island. (1, 1) touches all three islands and merges them into one. The second (1, 1) is already land, so nothing changes.

Example 3

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], additions = [(0, 2), (0, 3), (1, 0)]
Output: [3, 3, 2, 1]
```

(0, 2) joins the top-left island. It touches (1, 3) only at a corner, so the count stays 3. (0, 3) connects the top-left island to the island at (1, 3) and (2, 3), giving 2. (1, 0) connects the merged island to (2, 0), giving 1.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def add_land(r, c):
    grid[r][c] = 1
    return count_islands(grid)  # full flood fill
answers = [add_land(r, c) for r, c in additions]
```

TIME	SPACE
O(A * m * n)	O(m * n)

WHY IT IS SLOW

Each addition costs a pass over the whole grid, O(A * m * n). Union-Find merges the new cell with its land neighbours and updates the count in near-constant time.

OPTIMAL SOLUTION · Union-Find updated cell by cell

TIME	SPACE
O(m * n + A * alpha(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Land can also be removed (or turned back to water).

Union-Find cannot split sets. Either recompute the affected island with a local flood fill, or process all operations offline with a segment tree over time and a rollback Union-Find.

FOLLOW-UP 2 The grid is 10^9 x 10^9 with a few additions.

Keep the Union-Find on a dictionary keyed by coordinates, as the code does; memory depends on the number of land cells only, never on the grid size. **Cost:** O(a * alpha) for a additions and O(a) memory, independent of the grid size.

FOLLOW-UP 3 Report the size of the largest island after each addition.

Store a size per root in the Union-Find (union by size) and keep a running maximum, which only grows when land is added; each step stays near-constant time. Checked against recomputing the largest island from scratch. **Cost:** O(alpha) per addition with size tracking; the maximum is one running variable.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


class Islands:
    # land cells are added one at a time; the island count is kept up to date
    def __init__(self):
        self.parent = {}        # land cell -> its parent cell (union-find)
        self.size = {}          # size of the island, stored at its leader
        self.count = 0
        self.largest = 0        # used by follow-up 3

    def find(self, cell):
        while self.parent[cell] != cell:
            self.parent[cell] = self.parent[self.parent[cell]]
            cell = self.parent[cell]
        return cell

    def add(self, r, c):
        if (r, c) in self.parent:
            return                                  # already land
        self.parent[(r, c)] = (r, c)
        self.size[(r, c)] = 1
        self.count += 1                             # a new island ...
        if self.largest < 1:
            self.largest = 1
        for dr, dc in STEPS:
            neighbour = (r + dr, c + dc)
            if neighbour in self.parent:
                self.union((r, c), neighbour)       # ... that may join others

    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b:
            return
```

```
        if self.size[a] < self.size[b]:
            a, b = b, a                             # the small island joins the big one
        self.parent[b] = a
        self.size[a] += self.size[b]
        self.count -= 1                             # two islands became one
        if self.size[a] > self.largest:
```

```
            self.largest = self.size[a]


def build_islands(grid):
    # put every land cell (1) of the starting grid into the structure
    islands = Islands()
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == 1:
                islands.add(r, c)
    return islands


# ---- main question: main function ----
# Idea: build the starting islands with unions, then for every added
# cell: ignore it if it is already land; otherwise it is a new island
# (+1) and every time it is joined to a different island the count
# goes down by one. Returns the count before any addition and after each one.
# Complexity: Time O(m * n + K) alpha for the starting grid and K additions, Space O(land cells)
def island_counts(grid, additions):
    islands = build_islands(grid)
    counts = [islands.count]
    for r, c in additions:
        islands.add(r, c)
        counts.append(islands.count)
    return counts


# ---- main question: tests ----
def test_main_question():
    grid = [[1, 0, 0], [0, 0, 0], [0, 0, 1]]
    adds = [(0, 1), (1, 1), (2, 1), (0, 1)]
    assert island_counts(grid, adds) == [2, 2, 2, 1, 1]     # a repeated cell is ignored
    assert island_counts([[1, 0], [0, 1]], []) == [2]      # corners do not connect cells


# ========================================================
# FOLLOW-UP 3: Report the size of the largest island after each addition
# ========================================================
```

```
# ---- follow-up 3: function ----
# Answer: keep a size per leader (union by size) and a running maximum,
# which only grows when land is added; every step stays near-constant time.
# Complexity: Time O(m * n + K) alpha, Space O(land cells)
def largest_after_each(grid, additions):
    islands = build_islands(grid)
    sizes = [islands.largest]
    for r, c in additions:
        islands.add(r, c)
        sizes.append(islands.largest)
    return sizes


# ---- follow-up 3: tests ----
def test_follow_up_3():
    grid = [[1, 0, 0], [0, 0, 0], [0, 0, 1]]
    adds = [(0, 1), (1, 1), (2, 1), (0, 1)]
    assert largest_after_each(grid, adds) == [1, 2, 3, 5, 5]
    big = Islands()                                    # a huge grid is fine: only land cells are stored
    big.add(10 ** 9, 10 ** 9)
    big.add(10 ** 9, 10 ** 9 + 1)
    assert big.count == 1 and big.largest == 2


# ========================================================
```

```
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def island_counts(grid: list[list[int]], additions: list[list[int]]) -> list[int]:
```

Sample call

```
island_counts([[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], [])
```

Test cases

Input	Expected
`([[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], [])`	`[3]`
`([[0, 0, 0], [0, 0, 0], [0, 0, 0]], [[0, 0], [0, 1], [1, 2], [2, 1], [1, 1], [1, 1]])`	`[0, 1, 1, 2, 3, 1, 1]`

STUDY PATH: question 52 of 197 | topic 9 of 43: Union-find: merge the pairs, then ask who is together | 2 of 7

Template fit: pairs = points within distance r, found by bucketing points into cells of size r and checking only the 9 nearby cells; answer = groups | * asked 77 days ago * | frequency Low

MEDIUM | 1Point3Acres | phone-screen | 52 of 197

2D Points Connected Components by Radius

ASKED 2026-08-04 | 77 DAYS AGO | LAST 90 DAYS

STUDY: TYPE **ALL TIME 40.7** **30 DAYS 0** **PRIORITY 104/288**

ABOUT THIS QUESTION

Topic: Union-find: merge the pairs, then ask who is together

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #547 Number of Provinces

LeetCode match: LC 547 Number of Provinces (related)

Frequency: Low | Last asked: 2026-08-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/points-connected-components-by-radius

QUESTION

Given 2D points and radius **r**, connect two points when their distance is at most **r**; connectivity is transitive, and the output is the final number of clusters. The interviewer accepted or preferred a brute-force neighbor scan after discussing optimization ideas.

Input: a list of 2D points and a radius r.
Two points are connected if their Euclidean distance is <= r.
Connectivity is transitive: if A connects to B and B connects to C, all three belong to the same cluster.
Return the number of final clusters.

EXAMPLES

Example 1	
Input	`[(0, 0), (1, 0), (10, 10), (10, 11), (20, 20)], 2`
Output	`3`
Why	`Points within distance 2 of each other form a cluster: two near the origin, two near (10, 10), one alone.`

Example 2	
Input	`[(0, 0), (1, 0), (2, 0), (3, 0)], 1`
Output	`1`
Why	`A chain is one cluster even though its ends are far apart.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(points, r):
    graph = defaultdict(list)
    ids = range(len(points))
    for i, j in combinations(ids, 2):
        if dist2(points[i], points[j]) <= r * r:
            graph[i].append(j)
            graph[j].append(i)
    seen, clusters = set(), 0
    for i in range(len(points)):
        if i not in seen:
            clusters += 1
            dfs(graph, i, seen)
    return clusters
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every pair is compared, O(n^2). Putting points in cells of side r means each point is compared only with the 3 x 3 cells around it.

OPTIMAL SOLUTION · Union-find with a spatial grid

TIME	SPACE
O(n) about when points are spread out (O(n^2) if they all share a cell)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why compare squared distances?

sqrt is slow and inexact; dist <= r is the same as dist^2 <= r^2 for non-negative numbers, and with integer coordinates the comparison is exact. It also avoids floating-point surprises right at the boundary.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: 2D Points Connected Components by Radius
#
# Given 2D points and radius r, connect two points when
# their distance is at most r; connectivity is transitive,
# and the output is the final number of clusters. The
# interviewer accepted or preferred a brute-force neighbor
# scan after discussing optimization ideas. Input: a list
# of 2D points and a radius r. Two points are connected if
# their Euclidean distance is <= r. Connectivity is
# transitive: if A connects to B and B connects to C, all
# three belong to the same cluster. Return the number of
# final clusters.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why compare squared distances?
#
# INTERVIEW NOTES
```

```
# Two points are neighbours when their squared distance is
# at most r squared (no square roots needed), and
# connectivity is transitive, so this is counting
# connected components: union-find or a DFS over the
# neighbour relation. Checking every pair is O(n^2);
# bucketing points into cells of side r means a point only
# needs to be compared with the 3 x 3 cells around it.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x
```

```
    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


def near(p, q, r):
    # compare SQUARED distances: no square root, and exact for integer points
    dx = p[0] - q[0]
    dy = p[1] - q[1]
    return dx * dx + dy * dy <= r * r


# ---- main question: main function ----
# Idea: put each point in the cell (x // r, y // r). Two points within
# distance r are in the same or in neighbouring cells, so only those 9
# cells are compared; union every close pair and count the groups.
# Complexity: Time about O(n) for evenly spread points (9 grid cells per point), O(n^2) worst case; Space O
(n)
def count_clusters(points, r):
    size = r
    if size <= 0:
        size = 1
    cells = {}
    for i in range(len(points)):
        key = (points[i][0] // size, points[i][1] // size)
        if key not in cells:
            cells[key] = []
        cells[key].append(i)
    uf = UnionFind(len(points))
    for i in range(len(points)):
        cx, cy = points[i][0] // size, points[i][1] // size
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                for j in cells.get((cx + dx, cy + dy), []):
                    if j > i and near(points[i], points[j], r):
                        uf.union(i, j)
```

```
    groups = 0
    for i in range(len(points)):
        if uf.find(i) == i:
            groups += 1
    return groups


# ---- main question: tests ----
def count_clusters_slow(points, r):
    # compare every pair: slow but obviously right, used to check the fast one
```

```
    uf = UnionFind(len(points))
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            if near(points[i], points[j], r):
                uf.union(i, j)
    roots = set()
    for i in range(len(points)):
        roots.add(uf.find(i))
    return len(roots)


def test_main_question():
    assert count_clusters([(0, 0), (3, 0), (6, 0)], 3) == 1     # joined only through the middle point
    assert count_clusters([(0, 0), (3, 0), (7, 0)], 3) == 2     # the far point stays alone
    assert count_clusters([(1, 1), (1, 1), (2, 2)], 0) == 2     # radius 0 joins only identical points
    assert count_clusters([], 5) == 0
    points = []
    for i in range(30):
        points.append((i * 7 % 13, i * 5 % 11))
    for r in (1, 2, 3):
        assert count_clusters(points, r) == count_clusters_slow(points, r)


# =============================================================
# FOLLOW-UP 1: Why compare squared distances?
# =============================================================
# Answer: a square root is slow and inexact. dist <= r means exactly the
# same as dist^2 <= r^2 for non-negative numbers, and with integer
# coordinates the comparison is exact, so nothing goes wrong right at
# the boundary (see near() above).


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement connected components once with DFS/BFS and once with Union-Find.
Practice stating the O(n^2) brute-force baseline clearly before proposing spatial indexing.
Dry-run a case where three points form one cluster only through transitive links.

STUDY PATH: question 53 of 197 | topic 9 of 43: Union-find: merge the pairs, then ask who is together | 3 of 7

Template fit: identical to the line above: cell buckets give the candidate pairs, answer = groups | * asked 124 days ago *

MEDIUM | PracHub | Technical Screen | 53 of 197

Count Clusters of 2D Points Within a Radius

POSTED 2026-06-18 | 124 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE **ALL TIME 40.7** **30 DAYS 0** **PRIORITY 103/288**

ABOUT THIS QUESTION

Topic: Union-find: merge the pairs, then ask who is together

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #547 Number of Provinces

LeetCode match: LC 547 Number of Provinces (inferred from overview)

Posted on PracHub: 2026-06-18 (updated 2026-07-01)

Original: https://prachub.com/coding-questions/count-clusters-of-2d-points-within-a-radius

QUESTION

Given 2D points and a radius r, two points are connected when their distance is at most r. A cluster is a group of points connected directly or through chains of other points. Return the number of clusters.

Function signature

def count_clusters(points: list, r: float) -> int

EXAMPLES

Example 1	
Input	`[(0, 0), (1, 0), (10, 10), (10, 11), (20, 20)], 2`
Output	`3`
Why	`Three groups: the two points near the origin, the two near (10, 10), and the lone point at (20, 20).`

Example 2	
Input	`[(0, 0), (1, 0), (2, 0)], 1`
Output	`1`
Why	`A chain: (0,0)-(1,0)-(2,0) are linked neighbour to neighbour, so it is one cluster.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
parent = list(range(n))
def find(x):
    while parent[x] != x:
        x = parent[x]
    return x
for a, b in combinations(range(n), 2):
    if dist2(points[a], points[b]) <= r * r:
        parent[find(a)] = find(b)
clusters = len({find(i) for i in range(n)})
```

TIME	SPACE
O(n^2)	O(n)

BRUTE FORCE

WHY IT IS SLOW

n^2 / 2 distance checks. Hashing points into grid cells of side r means each point is compared only with points in its own and neighbouring cells.

OPTIMAL SOLUTION · Grid hashing with cells of side r, then Union-Find

TIME	SPACE
O(n) expected for evenly spread points	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Report the size of the largest cluster.

Count points per root after the unions (a Counter of find(i)). The unions are the same.

FOLLOW-UP 2 Points arrive one at a time.

Insert the point into its grid cell, union it with every point within r in the 9 surrounding cells, and keep the component count: components += 1 - (number of distinct roots merged).

FOLLOW-UP 3 Millions of points in 3D or higher.

The grid generalises (3^d neighbouring cells), but for high dimensions use a k-d tree or locality-sensitive hashing; or DBSCAN, which is this problem with a minimum neighbour count. **Cost:** grid hashing visits 3^d neighbouring cells per point, O(n * 3^d); a k-d tree builds in O(n log n) with O(n^(1 - 1/d) + k) range queries.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class Clusters:
    # points arrive one by one; the cluster count is kept up to date.
    # Two points are in one cluster when a chain of points, each within
    # distance r of the next, joins them.
    def __init__(self, r):
        self.r = r
        self.cells = {}          # grid cell (side r) -> indices of the points inside it
        self.points = []
        self.parent = []         # union-find over the point indices
        self.components = 0

    def find(self, x):
```

```
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def near(self, p, q):
        dx = p[0] - q[0]
        dy = p[1] - q[1]
        return dx * dx + dy * dy <= self.r * self.r      # squared distances: no square root

    def add(self, x, y):
        i = len(self.points)
        self.points.append((x, y))
        self.parent.append(i)
        self.components += 1
        side = self.r
        if side == 0:
            side = 1
        cx, cy = x // side, y // side
        if (cx, cy) not in self.cells:
            self.cells[(cx, cy)] = []
        self.cells[(cx, cy)].append(i)
```

```
        # a point within r can only sit in this cell or one of the 8 around it
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                for j in self.cells.get((cx + dx, cy + dy), []):
                    if j != i and self.near((x, y), self.points[j]):
                        a, b = self.find(i), self.find(j)
                        if a != b:
                            self.parent[a] = b
                            self.components -= 1
        return self.components

    def largest(self):
        # size of the biggest cluster
        sizes = {}
        for i in range(len(self.points)):
            root = self.find(i)
            sizes[root] = sizes.get(root, 0) + 1
        biggest = 0
        for size in sizes.values():
            if size > biggest:
                biggest = size
        return biggest


# ---- main question: main function ----
# Idea: put each point in the cell (x // r, y // r). A point within
# distance r of another is in the same cell or one of the 8 cells
# around it, so only those are compared; union the close pairs.
# Complexity: Time about O(n) for evenly spread points (each point looks at 9 grid cells), O(n^2) worst cas
e; Space O(n)
def count_clusters(points, r):
    clusters = Clusters(r)
    for x, y in points:
        clusters.add(x, y)
    return clusters.components


# ---- main question: tests ----
def test_main_question():
    line = [(0, 0), (3, 0), (6, 0), (20, 20)]
    assert count_clusters(line, 3) == 2
    assert count_clusters([], 3) == 0
```

```
    assert count_clusters([(1, 1), (1, 1)], 0) == 1


# =====================================================
```

```
# FOLLOW-UP 1: Report the size of the largest cluster
# =====================================================
# ---- follow-up 1: function ----
# Answer: count the points per root after the unions. The unions
# themselves are the same.
# Complexity: Time the same as count_clusters, Space O(n)
def largest_cluster(points, r):
    clusters = Clusters(r)
    for x, y in points:
        clusters.add(x, y)
    return clusters.largest()


# ---- follow-up 1: tests ----
def test_follow_up_1():
    line = [(0, 0), (3, 0), (6, 0), (20, 20)]
    assert largest_cluster(line, 3) == 3


# =====================================================
# FOLLOW-UP 2: Points arrive one at a time
# =====================================================
# Answer: add() above already does it: put the point in its cell,
# union it with every point within r in the 9 cells around, and
# components goes down by one for each union that really joined two groups.
# ---- follow-up 2: tests ----
def test_follow_up_2():
    c = Clusters(2)
    counts = []
    for p in [(0, 0), (10, 10), (1, 1), (5, 5)]:
        counts.append(c.add(p[0], p[1]))
    assert counts == [1, 2, 2, 3]
    assert c.add(3, 3) == 4          # (3, 3) is alone ...
    assert c.add(4, 4) == 3          # ... then (4, 4) bridges it to (5, 5)


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")
```

```

if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def countClusters(points: list[list[int]], r: int) -> int:
```

Sample call

```
countClusters([[0, 0], [1, 0], [10, 10]], 2)
```

Test cases

Input	Expected
`([[0, 0], [1, 0], [10, 10]], 2)`	`2`
`([[0, 0], [3, 0], [6, 0]], 3)`	`1`

ALSO REPORTED · 2 more

Count Connected Clusters of Two-Dimensional Points | PracHub | Technical Screen

https://prachub.com/coding-questions/count-connected-clusters-of-two-dimensional-points

2D Points Connected Components by Radius | 1Point3Acres | phone-screen | asked 2026-08-04 | Low

https://www.1point3acres.com/interview/problems/company/google/points-connected-components-by-radius

DFS, BFS, or Union-Find all fit the core connected-components model.
For many points, discuss how to avoid checking every pair by narrowing neighbor candidates spatially; in this round, the interviewer still preferred the brute-force implementation path.
Clarify squared-distance comparison to avoid unnecessary square roots.

Preparation: Implement connected components once with DFS/BFS and once with Union-Find.
Practice stating the O(n^2) brute-force baseline clearly before proposing spatial indexing.
Dry-run a case where three points form one cluster only through transitive links.

STUDY PATH: question 54 of 197 | topic 9 of 43: Union-find: merge the pairs, then ask who is together | 4 of 7

Template fit: pairs = edges sorted from the highest safety down; union until find(s) == find(t); the answer is the safety of that last edge | * asked 106 days ago *

MEDIUM | IN APP LC 1102 | PracHub | Technical Screen | 54 of 197

Find A Threshold-Limited Path With Minimum Required Safety

POSTED 2026-07-06 | 106 DAYS AGO | LAST 6 MONTHS

STUDY: TALK ALL TIME 25.5 30 DAYS 0 PRIORITY 157/288

ABOUT THIS QUESTION

Topic: Union-find: merge the pairs, then ask who is together

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1102 Path With Maximum Minimum Value

LeetCode match: LC 1102 Path With Maximum Minimum Value (inferred from overview)

Posted on PracHub: 2026-07-06 (updated 2026-07-08)

Original: https://prachub.com/coding-questions/find-a-threshold-limited-path-with-minimum-required-safety

QUESTION

A graph has n nodes and edges (u, v, safety). The safety of a path is the smallest safety among its edges. Using only edges whose safety is at least threshold (no limit when threshold is None), return the best (largest) safety you can reach going from s to t, or -1 if no such path exists. A path from a node to itself has infinite safety.

Function signature

def best_path_safety(n: int, edges: list, s: int, t: int, threshold=None) -> int

EXAMPLES

Example 1	
Input	`3, [(0, 1, 5), (1, 2, 3), (0, 2, 1)], 0, 2`
Output	`3`
Why	`Direct edge 0-2 has safety 1; the route 0-1-2 has safety min(5, 3) = 3, which is better.`

Example 2	
Input	`3, [(0, 1, 5), (1, 2, 3), (0, 2, 1)], 0, 2, threshold = 4`
Output	`-1`
Why	`Only edges with safety at least 4 may be used: 0-1 is allowed but nothing leads to node 2.`

Example 3	
Input	`3, [(0, 1, 5)], 1, 1`
Output	`infinity`
Why	`Source and target are the same node.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(graph, src, dst):
    best = -INF
    for path in all_paths(graph, src, dst):
        weakest = min(edge_safety(path))
        best = max(best, weakest)
    return best
```

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

Exponential. Adding edges from strongest to weakest in a Union-Find until the source and target connect is O(E log E).

OPTIMAL SOLUTION · Union-Find adding edges from strongest to weakest

TIME	SPACE
O(E log E)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Use a max-heap search instead of Union-Find.

Dijkstra where the value of a path is the minimum edge so far and you maximise it: pop the best bottleneck, extend with min(value, edge). O(E log V); Union-Find is simpler when you need only the value.

FOLLOW-UP 2 Answer many (source, target) queries.

Build the maximum spanning tree once (Kruskal) and answer each query as the minimum edge on the tree path, with binary lifting for O(log V) per query.

FOLLOW-UP 3 Edges below a threshold are forbidden (threshold-limited).

Filter them out before the search (or stop adding edges below the threshold in the Union-Find loop); if source and target are still disconnected the answer is -1.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find A Threshold-Limited Path With
# Minimum Required Safety
#
# A graph has n nodes and edges (u, v, safety). The safety
# of a path is the smallest safety among its edges. Using
# only edges whose safety is at least threshold (no limit
# when threshold is None), return the best (largest)
# safety you can reach going from s to t, or -1 if no such
# path exists. A path from a node to itself has infinite
# safety.
#
# Signature:
```

```
#   def best_path_safety(n: int, edges: list, s: int, t: int, threshold=None) -> int
#
# Examples:
# 3, [(0, 1, 5), (1, 2, 3), (0, 2, 1)], 0, 2  ->  3
# 3, [(0, 1, 5), (1, 2, 3), (0, 2, 1)], 0, 2, threshold =
#     4  ->  -1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Use a max-heap search instead of Union-Find.
# 2. Answer many (source, target) queries.
# 3. Edges below a threshold are forbidden (threshold-
# limited).
#
# INTERVIEW NOTES
# When the quality of a path is decided by its weakest
# edge, the best path does not need to be short, only
# strong. Two equivalent tools: add edges from strongest
# to weakest with Union-Find until source and target
# connect (that edge's value is the answer), or run
# Dijkstra with a max-heap where extending a path takes
# the minimum so far. Disconnected graphs must return the
# 'no path' value.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


def safety_of(edge):
    # an edge is (u, v, safety)
    return edge[2]


# ---- main question: main function ----
# Idea: the safety of a path is its weakest edge. Add edges from the
# safest to the least safe; the edge that finally joins s and t is the
# weakest edge of the best path. -1 if s and t never join (or only
# edges below `threshold` are allowed and that is not enough).
# Complexity: Time O(E log E) for the sort, then O(E * alpha(n)); Space O(n)
def best_path_safety(n, edges, s, t, threshold=None):
    if s == t:
        return float("inf")
```

```
    uf = UnionFind(n)
    for u, v, w in sorted(edges, key=safety_of, reverse=True):
        if threshold is not None and w < threshold:
            break                                   # follow-up 3: weaker edges are forbidden
        uf.union(u, v)
        if uf.find(s) == uf.find(t):
            return w
    return -1


# ---- main question: tests ----
def test_main_question():
    edges = [(0, 1, 5), (1, 2, 3), (0, 2, 1), (2, 3, 4)]
```

```
    assert best_path_safety(4, edges, 0, 3) == 3          # 0-1-2-3 has weakest edge 3; 0-2-3 only 1
    assert best_path_safety(3, [(0, 1, 9)], 0, 2) == -1    # not connected
    assert best_path_safety(3, [], 1, 1) == float("inf")    # source equals target


# ========================================================
# FOLLOW-UP 1: Use a max-heap search instead of Union-Find
# ========================================================
# ---- follow-up 1: function ----
# Answer: Dijkstra where the value of a path is its weakest edge, and
# we maximise it: take the best path first (a max-heap, so push the
# negated value) and extend it with min(value, edge): O(E log V).
# Complexity: Time O(E log V), Space O(V + E)
def best_path_safety_heap(n, edges, s, t, threshold=None):
    graph = []
    for i in range(n):
        graph.append([])
    for u, v, w in edges:
        if threshold is None or w >= threshold:
            graph[u].append((v, w))
            graph[v].append((u, w))
    best = [-1] * n
    best[s] = float("inf")
    heap = [(-best[s], s)]
    while heap:
        neg, u = heapq.heappop(heap)
        value = -neg
        if value < best[u]:
            continue                                # an outdated entry
        for v, w in graph[u]:
            candidate = value
            if w < candidate:
                candidate = w                       # extending a path can only lower its weakest edge
            if candidate > best[v]:
                best[v] = candidate
                heapq.heappush(heap, (-candidate, v))
    return best[t]


# ---- follow-up 1: tests ----
def test_follow_up_1():
    edges = [(0, 1, 5), (1, 2, 3), (0, 2, 1), (2, 3, 4)]
    assert best_path_safety_heap(4, edges, 0, 3) == 3
    assert best_path_safety_heap(4, edges, 0, 3, 4) == -1     # edges below the threshold are forbidden
    assert best_path_safety_heap(3, [(0, 1, 9)], 0, 2) == -1
```

```


# ========================================================
# FOLLOW-UP 2: Answer many (source, target) queries
# ========================================================
```

```
# Answer: build the union-find once while adding edges strongest
# first and remember when each pair of groups merged (a Kruskal
# reconstruction tree), or sort the queries too and answer them offline.


# ====================================================
# FOLLOW-UP 3: Edges below a threshold are forbidden
# ====================================================
# Answer: already handled by the `threshold` parameter: stop adding
# edges as soon as one is weaker than the threshold.
# ---- follow-up 3: tests ----
def test_follow_up_3():
    edges = [(0, 1, 5), (1, 2, 3), (0, 2, 1), (2, 3, 4)]
    assert best_path_safety(4, edges, 0, 3, 4) == -1


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Sample call

```
solution(4, [(0,1,5),(1,3,4),(0,2,2),(2,3,8)], 0, 3, 6)
```

Test cases

Input	Expected
(4, [(0,1,5),(1,3,4),(0,2,2),(2,3,8)], 0, 3, 6)	5
(3, [(0,1,7),(1,2,3)], 0, 2, 5)	-1

STUDY PATH: question 55 of 197 | topic 9 of 43: Union-find: merge the pairs, then ask who is together | 5 of 7

Template fit: pairs = peer links; keep the boss at the root and carry it over when two groups merge | * asked 240 days ago *

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 55 of 197

Determine Reporting Relationships

POSTED 2026-02-22 | 240 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 37.4 | 30 DAYS 33 | PRIORITY 73/288

ABOUT THIS QUESTION

Topic: Union-find: merge the pairs, then ask who is together

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #721 Accounts Merge

LeetCode match: No direct match; related LC 721 Accounts Merge (union-find)

Posted on PracHub: 2026-02-22 (updated 2026-04-21)

Original: https://prachub.com/coding-questions/determine-reporting-relationships

QUESTION

Build an in-memory organization model that supports three operations on employee IDs:

1. **`add_manager(a, b)`**: employee **a** is the direct manager of employee **b**.
2. **`add_peer(a, b)`**: employees **a** and **b** share the same direct manager.
3. **`is_manager(a, b)`**: return whether employee **a** is a manager of employee **b**. The answer should account for relationships that can be inferred from prior operations. For example, if **`add_manager(m, x)`** and **`add_peer(x, y)`** were previously recorded, then **`is_manager(m, y)`** should return **`true`**. You may assume the input is logically consistent and does not create invalid cycles in the org chart. Explain how you would design the data structures and implement these operations efficiently.

Implement an in-memory organization model that processes operations on employee IDs in order. Each operation is a tuple **`(op, a, b)`** where:

- **`("add_manager", a, b)`**: employee **a** becomes the direct manager of employee **b**.
- **`("add_peer", a, b)`**: employees **a** and **b** share the same direct manager.
- **`("is_manager", a, b)`**: ask whether employee **a** is a direct or indirect manager of employee **b**. Relationships can be inferred from earlier operations. For example, after **`add_manager(m, x)`** and **`add_peer(x, y)`**, **`is_manager(m, y)`** should be **`True`**. Process the operations sequentially and return a list of answers for all **`is_manager`** queries, in order. Notes:
- The input is logically consistent.
- No operation creates a cycle in the org chart.
- Repeating the same valid relationship is allowed.
- An employee is not considered their own manager.

Constraints

0 <= len(operations) <= 20000Employee IDs are integers in the range [-10^9, 10^9]At most 10000 distinct employee IDs appearThe sequence of operations is logically consistent and does not create cycles

EXAMPLES

```
Input: ([("add_manager", 1, 2), ("add_peer", 2, 3), ("is_manager", 1, 3)])
Expected Output: [True]
```

Explanation: Employee 2 reports to 1. Since 2 and 3 are peers, 3 also reports to 1.

QUESTION

```
Input: ([("add_manager", 2, 4), ("is_manager", 1, 4), ("add_manager", 1, 2), ("is_manager", 1, 4),
    ("is_manager", 2, 4), ("is_manager", 4, 2)])
Expected Output: [False, True, True, False]
```

Explanation: Before 2 gets a manager, 1 is not a manager of 4. After adding 1 as manager of 2, 1 becomes an indirect manager of 4. Hints**`add_peer(a, b)`** means **`a`** and **`b`** share one direct manager, but they are not interchangeable as managers. Group such employees with Union-Find.Once a peer group learns its direct manager, every current member of that group gets the same parent pointer. To answer **`is_manager(a, b)`**, walk upward from **`b`** through parent pointers.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
managers, peers = [], []  # the stored facts
def is_manager(a, b):
    groups = merge_peers(peers)
    manager_of = {}
    for x, y in managers:  # x manages y
        boss = groups.find(x)
        manager_of[groups.find(y)] = boss
    node = groups.find(b)
    while node in manager_of:
        node = manager_of[node]
        if node == groups.find(a):
            return True
    return False
```

TIME	SPACE
O(F * employees) per query	**O(F)**

WHY IT IS SLOW

Each query redoes the whole inference over all facts. Union-Find for peer groups with a manager stored on each group keeps every update near-constant and a query O(depth).

OPTIMAL SOLUTION · Union-Find for peer groups with a manager stored on each group

TIME	SPACE
O(alpha) per update, O(depth) per query	**O(employees)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the closest common manager of two employees.

Collect the chain of managers above one employee into a set, then climb from the other until you meet it. O(depth).

FOLLOW-UP 2 Facts can be wrong or contradictory.

When uniting groups that already have different managers, the input is inconsistent: raise, or record a conflict instead of silently overwriting. Also guard the upward walk with a visited set so a cycle cannot loop forever.

FOLLOW-UP 3 Remove a relationship.

Union-Find cannot split groups; keep the facts and rebuild lazily, or store the peers as an explicit graph and recompute connected components on demand.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Determine Reporting Relationships
#
# Build an in-memory organization model that supports
# three operations on employee IDs: 1. add_manager(a, b):
# employee a is the direct manager of employee b. 2.
# add_peer(a, b): employees a and b share the same direct
# manager. 3. is_manager(a, b): return whether employee a
# is a manager of employee b. The answer should account
# for relationships that can be inferred from prior
# operations. For example, if add_manager(m, x) and
# add_peer(x, y) were previously recorded, then
# is_manager(m, y) should return true. You may assume the
# input is logically consistent and does not create
# invalid cycles in the org chart. Explain how you would
# design the data structures and implement these
# operations efficiently.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the closest common manager of two employees.
# 2. Facts can be wrong or contradictory.
# 3. Remove a relationship.
#
# INTERVIEW NOTES
# Peers form groups (a peer relation is transitive: same
# manager as a person who has the same manager as
# another). A group has at most one direct manager, so
# store the manager on the group. To answer is_manager(a,
# b) go from b's group to its manager, then that manager's
# group to its manager, and so on, looking for a. Union-
# find makes the groups cheap to merge.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: union-find joins peers (people with the same manager); the
# leader of every peer group stores that group's manager.
# add_peer(a, b)    a and b have the same manager
```

```
# add_manager(a, b) a is the direct manager of b (and of b's peers)
# Complexity: Time add_peer, add_manager and manager_of are O(alpha(n)); is_manager is O(depth of the chain
); Space O(n)
class Org:
    def __init__(self):
        self.parent = {}       # union-find over people
        self.boss = {}         # leader of a peer group -> the manager of the group
```

```
    def find(self, x):
        if x not in self.parent:
            self.parent[x] = x
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def add_peer(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return
        if ra in self.boss and rb in self.boss and self.boss[ra] != self.boss[rb]:
            raise ValueError("conflicting managers")          # follow-up 2
        self.parent[rb] = ra
        if rb in self.boss and ra not in self.boss:
            self.boss[ra] = self.boss[rb]                     # the merged group keeps the known manager

    def add_manager(self, a, b):
        self.find(a)
        root = self.find(b)
        if root in self.boss and self.boss[root] != a:
            raise ValueError("conflicting managers")          # follow-up 2
        self.boss[root] = a

    def manager_of(self, x):
        return self.boss.get(self.find(x))

    def is_manager(self, a, b):
        # climb b's chain of managers; `seen` stops a cycle
        seen = set()
        person = b
        while person is not None and person not in seen:
            seen.add(person)
            person = self.manager_of(person)
            if person == a:
                return True
        return False
```

```
# ---- main question: tests ----
def test_main_question():
    o = Org()
    o.add_manager("ceo", "vp")
    o.add_manager("vp", "dev")
    o.add_peer("dev", "qa")
    assert o.is_manager("vp", "qa")                # qa shares dev's manager
    assert o.is_manager("ceo", "qa")
    assert not o.is_manager("qa", "vp")
    assert not o.is_manager("dev", "qa")
    p = Org()                                      # peers added before the manager is known
    p.add_peer("a", "b")
    p.add_manager("boss", "a")
    assert p.is_manager("boss", "b")


# ========================================================
# FOLLOW-UP 1: Return the closest common manager of two employees
# ========================================================
# ---- follow-up 1: function ----
# Answer: collect the chain of managers above one person into a set,
# then climb from the other person until you meet it: O(depth).
# Complexity: Time O(depth), Space O(depth)
```

```
def common_manager(org, a, b):
    chain = set()
    person = org.manager_of(a)
    while person is not None and person not in chain:
        chain.add(person)
        person = org.manager_of(person)
    seen = set()
    person = org.manager_of(b)
    while person is not None and person not in seen:
        if person in chain:
            return person
        seen.add(person)
        person = org.manager_of(person)
    return None


# ---- follow-up 1: tests ----
def test_follow_up_1():
    o = Org()
    o.add_manager("ceo", "vp")
```

```
    o.add_manager("vp", "dev")
    o.add_peer("dev", "qa")
    assert common_manager(o, "dev", "qa") == "vp"
    assert common_manager(o, "vp", "qa") == "ceo"


# ============================================================
# FOLLOW-UP 2: Facts can be wrong or contradictory
# ============================================================
# Answer: two peer groups with different managers, or a second manager
# for the same group, is a contradiction; the Org methods above raise
# ValueError instead of silently picking one.
# ---- follow-up 2: tests ----
def test_follow_up_2():
    o = Org()
    o.add_manager("ceo", "vp")
    o.add_manager("vp", "dev")
    try:
        o.add_manager("ceo", "dev")
        raise AssertionError("should conflict")
    except ValueError:
        pass


# ============================================================
# FOLLOW-UP 3: Remove a relationship
# ============================================================
# Answer: union-find cannot split a group. Keep the list of facts and
# rebuild the structure without the removed fact (or use an offline
# "dynamic connectivity" method if removals are frequent).


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(operations: list[tuple[str, int, int]]) -> list[bool]:
SAMPLE INPUT
([("add_manager", 1, 2), ("add_peer", 2, 3), ("is_manager", 1, 3)])
```

Test cases

Input	Expected
`([("add_manager", 1, 2), ("add_peer", 2, 3), ("is_manager", 1, 3)])`	`[True]`
`([("add_manager", 2, 4), ("is_manager", 1, 4), ("add_manager", 1, 2), ("is_manager", 1, 4), ("is_manager", 2, 4), ("is_manager", 4, 2)])`	`[False, True, True, False]`

STUDY PATH: question 56 of 197 | topic 9 of 43: Union-find: merge the pairs, then ask who is together | 6 of 7

Template fit: pairs = the given edges; the query is find(a) == find(b) | * asked 271 days ago *

MEDIUM | PracHub | Technical Screen | 56 of 197

Determine whether two nodes are related

POSTED 2026-01-22 | 271 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 24.5 | 30 DAYS 0 | PRIORITY 163/288

ABOUT THIS QUESTION

Topic: Union-find: merge the pairs, then ask who is together

Optimal answer from: the interview site's own solution

LeetCode match: LC 1971 Find if Path Exists in Graph (inferred from overview)

Posted on PracHub: 2026-01-22 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/determine-whether-two-nodes-are-related

QUESTION

Nodes 0 to n - 1 are linked by edges, and two nodes are related when a chain of edges joins them (links work in both directions; the graph may be disconnected and may contain cycles). Support union(a, b) to add a link and related(a, b) to answer the query.

Function signature

class Relations:

def __init__(self, n: int, edges=())

def union(self, a: int, b: int) -> None

def related(self, a: int, b: int) -> bool

EXAMPLES

Example 1

Input	`r = Relations(5, [(0, 1), (1, 2)])`
	`r.related(0, 2)`
	`r.related(0, 3)`
	`r.union(3, 4)`
	`r.related(3, 4)`
	`r.related(2, 4)`

Output	`r.related(0, 2) -> true`
	`r.related(0, 3) -> false`
	`r.related(3, 4) -> true`
	`r.related(2, 4) -> false`

Why	`0, 1, 2 form one group and 3, 4 another until they are joined; the two groups are never linked to each other.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def related(a, b):
    graph = build_adjacency(edges)
    seen = set()
    dfs(graph, a, seen)
    return b in seen
```

BRUTE FORCE

TIME	SPACE
O(V + E) per query	O(V)

WHY IT IS SLOW

Every query repeats the same traversal, O(V + E) each. A Union-Find built once answers each query in near-constant time.

OPTIMAL SOLUTION · Union-Find built once, constant-time queries

TIME	SPACE
O(E alpha(V)) to build, O(alpha(V)) per query	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 'Related' means ancestor or descendant, or sharing an ancestor.

Store the data as a parent map and walk up from one node while recording ancestors, then walk up from the other until you hit one. Keep a seen set so a corrupt cycle cannot loop forever.

FOLLOW-UP 2 Relationships arrive over time and you must answer queries between updates.

Union-Find handles incremental unions in near-constant time. If relationships can also be removed, you need offline processing or a dynamic-connectivity structure.

FOLLOW-UP 3 The graph is directed and 'related' means reachability from a to b.

Union-Find no longer works (it ignores direction). Use DFS/BFS per query, or compute strongly connected components and reachability on the condensed DAG for many queries.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Determine whether two nodes are related
#
# Nodes 0 to n - 1 are linked by edges, and two nodes are
# related when a chain of edges joins them (links work in
# both directions; the graph may be disconnected and may
# contain cycles). Support union(a, b) to add a link and
# related(a, b) to answer the query.
#
# Signature:
#   class Relations:
#       def __init__(self, n: int, edges=())
#       def union(self, a: int, b: int) -> None
#       def related(self, a: int, b: int) -> bool
#
# FOLLOW-UPS (answered below, in this order):
# 1. 'Related' means ancestor or descendant, or sharing an
# ancestor.
# 2. Relationships arrive over time and you must answer
# queries between updates.
# 3. The graph is directed and 'related' means
```

```
# reachability from a to b.
#
# INTERVIEW NOTES
# Ask what 'related' means: connected in any direction (a
# connectivity question: union-find or one traversal), or
# ancestor/descendant, or share an ancestor. With many
# queries, preprocess once: components give O(1) answers.
# Always make the visited set part of the design, since
# relationship data can have cycles.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: union the two ends of every relationship; two nodes are
```

```
# related exactly when they have the same leader (root).
# Complexity: Time union and related are O(alpha(n)) (about O(1)) each, building from E edges is O(E * alph
a(n)), Space O(n)
class Relations:
    def __init__(self, n, edges=()):
        self.parent = list(range(n))
        self.size = [1] * n
        for u, v in edges:
            self.union(u, v)

    def find(self, x):
        # walk up to the leader; path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        # relations can arrive over time: near-constant time each
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra                 # the smaller group joins the bigger one
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]

    def related(self, a, b):
        return self.find(a) == self.find(b)


# ---- main question: tests ----
def test_main_question():
    r = Relations(5, [(0, 1), (1, 2)])
    assert r.related(0, 2) and not r.related(0, 3)
    r.union(3, 4)
    assert r.related(3, 4) and not r.related(2, 4)
    r.union(2, 3)
    assert r.related(0, 4)
```

```
    assert r.related(1, 1)


# ========================================================
# FOLLOW-UP 1: "Related" means ancestor or descendant, or sharing an ancestor
# ========================================================
# ---- follow-up 1: helper functions ----
```

```
def ancestors(parent, x):
    # x and everything above it; the check `not in seen` stops a corrupt cycle
    seen = []
    while x is not None and x not in seen:
        seen.append(x)
        x = parent.get(x)
    return seen


# ---- follow-up 1: function ----
# Answer: keep a parent map; list the ancestors of both people and
# they are related when the two lists share a person.
# Complexity: Time O(depth of the two nodes), Space O(depth)
def kin(parent, a, b):
    up_a = ancestors(parent, a)
    up_b = ancestors(parent, b)
    for person in up_a:
        if person in up_b:
            return True
    return False


# ---- follow-up 1: tests ----
def test_follow_up_1():
    parent = {"kid": "mom", "mom": "gran", "uncle": "gran", "stranger": None}
    assert kin(parent, "kid", "gran")
    assert kin(parent, "kid", "uncle")
    assert not kin(parent, "kid", "stranger")
    assert kin({"a": "b", "b": "a"}, "a", "b")         # a corrupt cycle cannot loop forever


# ========================================================
# FOLLOW-UP 2: Relationships arrive over time and you must answer queries between updates
# ========================================================
# Answer: that is exactly what union() above is for: each new
# relationship is one union, and related() can be asked at any moment.


# ========================================================
# FOLLOW-UP 3: The graph is directed and "related" means reachability from a to b
# ========================================================
# ---- follow-up 3: function ----
# Answer: union-find ignores direction, so it cannot answer this. Search
# (BFS) per query, or find strongly connected components once and
# answer reachability on the condensed graph when there are many queries.
```

```
# Complexity: Time O(V + E) per query, Space O(V)
def reachable(graph, a, b):
    seen = {a}
    queue = deque([a])
    while queue:
        u = queue.popleft()
        if u == b:
            return True
        for v in graph.get(u, []):
            if v not in seen:
                seen.add(v)
```

```
                queue.append(v)
    return False


# ---- follow-up 3: tests ----
def test_follow_up_3():
    g = {"a": ["b"], "b": ["c"]}
    assert reachable(g, "a", "c")
    assert not reachable(g, "c", "a")                # direction matters


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(relations: list[list[int]], a: int, b: int) -> bool:
```

Sample call

```
solution([(1, 2), (1, 3), (3, 4), (10, 11)], 2, 4)
```

Test cases

Input	Expected
`([(1, 2), (1, 3), (3, 4), (10, 11)], 2, 4)`	`True`
`([(1, 2), (1, 3), (3, 4), (10, 11)], 2, 11)`	`False`

STUDY PATH: question 57 of 197 | topic 9 of 43: Union-find: merge the pairs, then ask who is together | 7 of 7

Template fit: pairs = similar movies (use a dict instead of a list); group = every movie with the same root as the target, sorted by rating | * asked 376 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 57 of 197

Movie Recommendations via Transitive Similarity

ASKED 2025-10-09 | 376 DAYS AGO | OLDER

STUDY: TYPE | **ALL TIME 37.4** | **30 DAYS 33** | **PRIORITY 72/288**

ABOUT THIS QUESTION

Topic: Union-find: merge the pairs, then ask who is together

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #721 Accounts Merge

LeetCode match: LC 721 Accounts Merge (related: transitive groups)

Frequency: Single report | Last asked: 2025-10-09 | Roles: SWE Intern

QUESTION

You are given a target movie name, a dictionary ratings mapping each movie to its numeric rating, an integer n , and a list similarities of [movieX, movieY] pairs meaning the two movies are directly similar. Similarity is transitive: if A is similar to B and B is similar to C, then A, B, and C all belong to the same similarity group. Return up to n recommended movies from the target's similarity group, excluding the target itself, ordered by rating descending (break ties alphabetically by name). If n == 1 , this returns the single highest- rated similar movie. If the group has fewer than n other movies, return all of them.

Function signature

```
def recommend(target: str, ratings: dict[str, float], n: int,
              similarities: list[list[str]]) -> list[str]:
    ...
```

EXAMPLES

Example 1

Input	`target = "A", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 1, similarities = [["A", "B"], ["B", "C"]]`
Output	`["C"]`
Why	`A, B, C form one group. Excluding A, the highest rated is C (9).`

Example 2

Input	`target = "A", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 2, similarities = [["A", "B"], ["B", "C"]]`
Output	`["C", "B"]`
Why	`Similar to A are {B (5), C (9)}; sorted by rating descending -> [C, B].`

Example 3

Input	`target = "D", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 3, similarities = [["A", "B"], ["B", "C"]]`
Output	`[]`
Why	`D is in no similarity pair, so its group is just {D}; excluding D leaves nothing.`

CONSTRAINTS

- Every movie mentioned appears as a key in ratings .

QUESTION

- 1 <= n ; ratings are numbers, names are distinct strings.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_recommend(target, ratings, n, pairs):
    group, grew = {target}, True
    while grew:
        grew = False
        for x, y in pairs:
            if (x in group) != (y in group):
                group.update((x, y))
                grew = True
    group.discard(target)
    return sorted(
        group, key=lambda m: (-ratings[m], m)
    )[:n]
```

TIME	SPACE
O(P * rounds)	O(P)

WHY IT IS SLOW

Every round rescans all pairs until the group stops growing.

OPTIMAL SOLUTION · Union-find and a sort

TIME	SPACE
O(P alpha + g log g)	O(P)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Thousands of recommendation queries on the same data.

Build the groups once with union-find, sort each group by (-rating, name) once, and store the group id of every movie. A query looks up the group, takes the first n + 1 entries and removes the target: O(n) per query.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Movie Recommendations via Transitive
# Similarity
#
# You are given a target movie name, a dictionary ratings
# mapping each movie to its numeric rating, an integer n ,
# and a list similarities of [movieX, movieY] pairs
# meaning the two movies are directly similar. Similarity
# is transitive: if A is similar to B and B is similar to
# C, then A, B, and C all belong to the same similarity
# group. Return up to n recommended movies from the
# target's similarity group, excluding the target itself,
# ordered by rating descending (break ties alphabetically
# by name). If n == 1 , this returns the single highest-
# rated similar movie. If the group has fewer than n other
```

```
# movies, return all of them. Function signature: def
# recommend(target: str, ratings: dict[str, float], n:
# int, similarities: list[list[str]]) -> list[str]: ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Thousands of recommendation queries on the same data.
#
# INTERVIEW NOTES
# Transitive similarity means connected components: movies
# that can reach each other through the pairs form one
# group. Find the target's component (union-find, or a BFS
# from the target), drop the target, sort the rest by
# rating descending then name, and return the first n.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class UnionFind:
    # union-find over names (any hashable value); unknown names join on first use
    def __init__(self):
        self.parent = {}

    def find(self, x):
```

```
        if x not in self.parent:
            self.parent[x] = x
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


def best_first(movies, ratings):
    # the movies ordered by rating, highest first (ties by name)
    pairs = []
    for movie in movies:
        pairs.append((-ratings[movie], movie))
    pairs.sort()
    ordered = []
    for pair in pairs:
        ordered.append(pair[1])
    return ordered


# ---- main question: main function ----
# Idea: movies joined by "similar" pairs (directly or through other
# movies) form one group. Take the target's group and rank it by rating.
# Complexity: Time O(P alpha + M log M) for P similar pairs and M movies in the group, Space O(M)
def recommend(target, ratings, n, similarities):
    uf = UnionFind()
    for x, y in similarities:
        uf.union(x, y)
    root = uf.find(target)
    group = []
```

```
    for movie in list(uf.parent):
        if movie != target and uf.find(movie) == root:
            group.append(movie)
    return best_first(group, ratings)[:n]


# ---- main question: tests ----
def test_main_question():
    r = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}
    similar = [["A", "B"], ["B", "C"]]
    assert recommend("A", r, 1, similar) == ["C"]
    assert recommend("A", r, 2, similar) == ["C", "B"]
```

```
    assert recommend("D", r, 3, similar) == []


# ============================================================
# FOLLOW-UP 1: Thousands of recommendation queries on the same data
# ============================================================
# ---- follow-up 1: function ----
# Answer: build the groups once with union-find and sort each group by
# rating once, remembering each movie's group. A query then only looks
# up the group and takes the first n movies (skipping the target): O(n).
# Complexity: Time build O(P alpha + M log M), each query O(n), Space O(M)
class Recommender:
    def __init__(self, ratings, similarities):
        uf = UnionFind()
        for x, y in similarities:
            uf.union(x, y)
        members = {}                        # leader -> movies of that group
        self.group_of = {}
        for movie in list(uf.parent):
            root = uf.find(movie)
            self.group_of[movie] = root
            if root not in members:
                members[root] = []
            members[root].append(movie)
        self.groups = {}
        for root in members:
            self.groups[root] = best_first(members[root], ratings)

    def recommend(self, target, n):
        if target not in self.group_of:
            return []
        picked = []
        for movie in self.groups[self.group_of[target]]:
            if movie != target and len(picked) < n:
                picked.append(movie)
        return picked


# ---- follow-up 1: tests ----
def test_follow_up_1():
    r = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}
    rec = Recommender(r, [["A", "B"], ["B", "C"]])
    assert rec.recommend("A", 2) == ["C", "B"]
    assert rec.recommend("D", 3) == []
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 10 OF 43

A dict of per-key state that you update, expire, or rank

7 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

7
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

A dict of per-key state that you update, expire, or rank | 7 questions | tap a title to jump to it

READ FIRST: the code template for these 7 questions (tap here) >

M	58. Active Communication Top Users	1Point3Acres * 2026-09-22 *
M	59. Filter Repeated Robot Status Messages	PracHub * 2026-07-05 *
M	60. Next-Word Predictor	1Point3Acres * 2026-09-02 *
M	61. Batch and Streaming Log Deduplication	1Point3Acres * 2026-07-23 *
M	62. Design a logger that returns top-K messages	PracHub * 2026-01-06 *
M	63. Website Activity Analytics: Unique Users + Session Time	1Point3Acres * 2026-06-26 *
M	64. Hit Counter	ND

CODE TEMPLATE | GROUP 10 OF 43 | 7 QUESTIONS, ONE PATTERN

A dict of per-key state that you update, expire, or rank

The one idea these questions share

Keep a dict from a key (message, user, word, activity) to the little bit of state you need: the last time seen, a Counter, a set, a list of timestamps. Every event updates one key's state; the rule that decides what to accept, expire or report is the only part that changes.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
class KeyedState:
    def __init__(self, window):
        self.window = window
        self.last = {}                          # CHANGE 1: what you keep per key

    def accept(self, t, key):
        if key in self.last and t - self.last[key] < self.window:   # CHANGE 2: the rule
            return False
        self.last[key] = t                      # CHANGE 3: update this key's state
        return True
```

2. Helper functions that usually get written next to it

```
from collections import defaultdict, Counter, deque


def group_by(pairs):                        # key -> list of values, in arrival order
    out = defaultdict(list)
    for k, v in pairs:
        out[k].append(v)
    return out


def hits_in_window(times, now, span=300):
    q = deque(times)
    while q and q[0] <= now - span:
        q.popleft()
    return len(q)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
58	Active Communication Top Users	a set of seen pairs, a Counter of partners per user, and buckets by count to read the top users in O(1)
59	Filter Repeated Robot Status Messages	the same rule applied to a stream of (timestamp, message) pairs; keep what is accepted
60	Next-Word Predictor	defaultdict(Counter): counts[a][b] += 1 for neighbouring words; remember the best next word per word
61	Batch and Streaming Log Deduplication	dict message -> first (or latest) timestamp depending on the policy; sort the result at the end
62	Design a logger that returns top-K messages	dict message -> list of timestamps; top_k counts each list inside the time range with bisect, then takes the k largest
63	Website Activity Analytics: Unique Users + Session Time	dict activity -> set of users for unique counts; dict user -> sorted events to add up login-to-logout session time
64	Hit Counter	one deque of hit times; drop the ones older than 300 seconds; the answer is the length

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 58 of 197 | topic 10 of 43: A dict of per-key state that you update, expire, or rank | 1 of 7

Template fit: a set of seen pairs, a Counter of partners per user, and buckets by count to read the top users in O(1) | * asked 28 days ago * | frequency Low

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | phone-screen | 58 of 197

Active Communication Top Users

ASKED 2026-09-22 | 28 DAYS AGO | LAST 30 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 288/288**

ABOUT THIS QUESTION

Topic: A dict of per-key state that you update, expire, or rank

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-22 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/active-communication-top-users

QUESTION

A stream of messages like 'A-B' means users A and B talked. Each distinct pair counts once (A-B and B-A are the same pair) and a pair of a user with themselves is ignored. A user's count is how many different partners they have. Support add(pair), most_active() (the set of users with the highest count) and top_k(k) (k users by count, ties broken by name).

Function signature

class Active:
    def add(self, pair: str) -> None
    def most_active(self) -> set
    def top_k(self, k: int) -> list

EXAMPLES

Example 1

Input	`a = Active()`
	`a.add('A-B')`
	`a.add('B-C')`
	`a.add('B-A')`
	`a.most_active()`
	`a.top_k(2)`

Output	`a.most_active() -> {'B'}` `a.top_k(2) -> ['B', 'A']`
Why	`B talked to A and C (2 partners). The repeated pair B-A adds nothing. A and C have 1 each, so the top 2 are B, then A (ties by name).`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
pairs = set()
def add(a, b):
    pairs.add((min(a, b), max(a, b)))
def top_users():
    partners = defaultdict(set)
    for a, b in pairs:
        partners[a].add(b)
        partners[b].add(a)
    def rank(u):
        return (-len(partners[u]), u)
    return sorted(partners, key=rank)
```

BRUTE FORCE

TIME	SPACE
O(E + U log U) per query	**O(E)**

WHY IT IS SLOW

Each query rebuilds all counts and sorts, O(E + U log U). Maintaining counts incrementally with count buckets gives O(1) updates and an O(1) most-active lookup.

OPTIMAL SOLUTION · Edge set, counts per user, count buckets for the maximum

TIME	SPACE
O(1) per pair, O(U log k) for top-K, O(1) for the most active user	**O(E)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Self communication (A-A) should now count.

Pass a flag to the class: a self pair is one edge that adds one to that user's count (and only once). The dedup set already handles repeated self pairs.

FOLLOW-UP 2 Top-K with a changing K, or a fixed-size priority queue.

A heap of size K can be wrong after counts change, because an element outside it can overtake one inside. Re-run heapq.nsmallest on the count dict per query (O(U log K)) or maintain a sorted structure keyed by (count, name) updated on every increment.

FOLLOW-UP 3 The space claim.

Distinct pairs can number U^2 / 2 in the worst case, so the edge set is quadratic in the users; the counts alone are linear. If duplicates are not a concern, drop the set and keep O(U).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Active Communication Top Users
#
# A stream of messages like 'A-B' means users A and B
# talked. Each distinct pair counts once (A-B and B-A are
# the same pair) and a pair of a user with themselves is
# ignored. A user's count is how many different partners
# they have. Support add(pair), most_active() (the set of
# users with the highest count) and top_k(k) (k users by
# count, ties broken by name).
#
# Signature:
#   class Active:
#       def add(self, pair: str) -> None
#       def most_active(self) -> set
#       def top_k(self, k: int) -> list
#
```

```
# FOLLOW-UPS (answered below, in this order):
# 1. Self communication (A-A) should now count.
# 2. Top-K with a changing K, or a fixed-size priority
# queue.
# 3. The space claim.
#
# INTERVIEW NOTES
# Normalize the pair first (sorted tuple) so A-B and B-A
# are one edge, then keep an edge set for dedup and a
# count per user. The ranking part is a separate question:
# top-K comes from a heap, and the single most active user
# can be maintained in O(1) because counts only ever go
# up. Mention that the edge set uses quadratic space in
# the worst case.

# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

import heapq


# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: main class ----
```

```
# A stream of messages like "A-B" means users A and B talked. Each DISTINCT
# pair counts once (A-B and B-A are the same pair) and a pair of a user with
# themselves is ignored. A user's count is how many different partners they have.
# add(pair), most_active() = the set of users with the highest count,
# top_k(k) = k users by count, ties broken by name.
# Idea: keep the set of edges seen (to drop repeats), a count per user, and
# BUCKETS: count -> the set of users with exactly that count. A new edge moves
# both users up one bucket, and most_active just reads the top bucket: O(1).
# (count_self=True is follow-up 1: a user talking to themselves counts, once.)
# Complexity: Time add O(1), most_active O(size of the top bucket), top_k O(u log k) for u users, Space O(n
umber of different pairs)
class Active:
    def __init__(self, count_self=False):
        self.edges = set()                    # the distinct pairs seen so far
        self.count = {}                       # user -> how many different partners
        self.bucket = {}                      # count -> the users with exactly that count
        self.top = 0                          # the highest count right now (it never drops)
        self.count_self = count_self

    def bump(self, user):
        old = self.count.get(user, 0)
        if old > 0:
            self.bucket[old].discard(user)    # leave bucket `old` ...
        self.count[user] = old + 1
        if old + 1 not in self.bucket:
            self.bucket[old + 1] = set()
        self.bucket[old + 1].add(user)        # ... and join the next bucket
        if old + 1 > self.top:
            self.top = old + 1

    def add(self, pair):
        a, b = pair.split("-")
        if a == b and not self.count_self:
            return                            # a user talking to themselves is ignored
        if a < b:
```

```
            edge = (a, b)                       # A-B and B-A are the same edge
        else:
            edge = (b, a)
        if edge in self.edges:
            return                              # a repeated pair counts only once
        self.edges.add(edge)
        self.bump(a)
        if a != b:
            self.bump(b)                        # a self pair counts once, not twice

    def most_active(self):
```

```
        if self.top == 0:
            return set()
        return set(self.bucket[self.top])

    def top_k(self, k):
        rows = []
        for user in self.count:
            rows.append((-self.count[user], user))
        best = heapq.nsmallest(k, rows)
        users = []
        for minus_count, user in best:
            users.append(user)
        return users


# ---- main question: tests ----
def test_main_question():
    a = Active()
    for pair in ["A-B", "B-C", "B-A"]:
        a.add(pair)
    assert a.most_active() == {"B"}                    # B talks to A and C; the repeat B-A is ignored
    assert a.count["B"] == 2 and a.count["A"] == 1
    assert a.top_k(2) == ["B", "A"]
    a.add("C-C")
    assert a.count["C"] == 1                           # self pairs are ignored by default
    assert Active().most_active() == set()


# ========================================================
# FOLLOW-UP 1: Self communication (A-A) should now count
# ========================================================
# Answer: pass a flag to the class (count_self=True): a self pair is one
# edge that adds one to that user's count, once. The set of edges already
# drops repeated self pairs.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    s = Active(count_self=True)
    s.add("C-C")
    s.add("C-C")
    assert s.count["C"] == 1


# ========================================================
# FOLLOW-UP 2: Top-K with a changing K, or a fixed-size priority queue
# ========================================================
```

```
# Answer: top_k above builds the ranking on demand with a heap, so k can
# change on every call. For a FIXED k keep a min-heap of size k (updated when a
# count changes), at the price of lazy deletion of old counts.


# ========================================================
# FOLLOW-UP 3: The space claim
```

```
# ============================================================
# Answer: the memory is O(number of distinct pairs) for the edge set plus
# O(users) for the counts and buckets. Without the edge set (no dedup) it would
# be O(users).


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement the pair-normalization layer separately from count maintenance.
Practice maintaining user -> count, edge set, and count -> ordered users views without letting updates drift.
Dry-run duplicate edges and self-edges before discussing top-K optimization.

STUDY PATH: question 59 of 197 | topic 10 of 43: A dict of per-key state that you update, expire, or rank | 2 of 7

Template fit: the same rule applied to a stream of (timestamp, message) pairs; keep what is accepted | * asked 107 days ago *

MEDIUM | PracHub | Technical Screen | 59 of 197

★ TOP 75: rank 51 in the Medium and Hard priority list

★ Filter Repeated Robot Status Messages

POSTED 2026-07-05 | 107 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE **ALL TIME 51.2** **30 DAYS 33** **PRIORITY 39/288**

ABOUT THIS QUESTION

Topic: A dict of per-key state that you update, expire, or rank

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #359 Logger Rate Limiter

LeetCode match: LC 359 Logger Rate Limiter (inferred from overview)

Posted on PracHub: 2026-07-05 (updated 2026-07-15)

Original: https://prachub.com/coding-questions/filter-repeated-robot-status-messages

QUESTION

A robot status stream is a list of (timestamp, message). Show a message when it has never been shown, or when strictly more than gap seconds (default 10) have passed since it was last shown. Messages are compared exactly (case-sensitive), a hidden repeat does not restart the interval, and the shown messages keep their input order. Return the shown (timestamp, message) pairs.

Function signature

def filter_status(stream: list, gap: int = 10) -> list

EXAMPLES

Example 1	
Input	`[(0, 'ok'), (5, 'ok'), (10, 'ok'), (11, 'ok'), (11, 'OK')]`
Output	`[(0, 'ok'), (11, 'ok'), (11, 'OK')]`
Why	`'ok' at 5 and 10 are hidden (not more than 10 seconds after 0); at 11 it is 11 seconds later, so it shows; 'OK' is a different message.`

Example 2	
Input	`[(0, 'low'), (3, 'low'), (7, 'low')], gap = 2`
Output	`[(0, 'low'), (3, 'low'), (7, 'low')]`
Why	`A smaller gap lets repeats through sooner.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
shown = []  # (time, message)
def on_message(t, msg):
    last = None
    for t0, m0 in shown:
        if m0 == msg:
            last = t0  # most recent display
    if last is None or t - last > 10:
        shown.append((t, msg))
        return True
    return False
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each message searches the whole history, O(n^2). A dictionary from message to its last displayed time makes each decision O(1).

OPTIMAL SOLUTION · Dictionary from message to the time it was last shown

TIME	SPACE
O(n)	O(distinct messages)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The stream never ends; memory must be bounded.

A message older than the gap can never be blocked again, so evict it. Keep a min-heap of (time shown, message) and pop entries older than the gap before each decision; delete the dictionary key only if it still points at that time.

FOLLOW-UP 2 Several robots send messages.

Use (robot id, message) as the key, or one filter per robot. The same logic applies per key.

FOLLOW-UP 3 The interval depends on the message severity.

Look up the gap from a severity table and use it in the comparison; the state (last shown time) is unchanged.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Filter Repeated Robot Status Messages
#
# A robot status stream is a list of (timestamp, message).
# Show a message when it has never been shown, or when
# strictly more than gap seconds (default 10) have passed
# since it was last shown. Messages are compared exactly
# (case-sensitive), a hidden repeat does not restart the
# interval, and the shown messages keep their input order.
# Return the shown (timestamp, message) pairs.
#
# Signature:
```

```
#   def filter_status(stream: list, gap: int = 10) -> list
#
# Examples:
# [(0, 'ok'), (5, 'ok'), (10, 'ok'), (11, 'ok'), (11,
#     'OK')]  ->  [(0, 'ok'), (11, 'ok'), (11, 'OK')]
# [(0, 'low'), (3, 'low'), (7, 'low')], gap = 2  ->  [(0,
#     'low'), (3, 'low'), (7, 'low')]
#
# FOLLOW-UPS (answered below, in this order):
# 1. The stream never ends; memory must be bounded.
# 2. Several robots send messages.
# 3. The interval depends on the message severity.
#
# INTERVIEW NOTES
# The decision for a message depends only on when that
# exact message was last displayed, so a dictionary from
# message to last display time is all the state you need.
# The detail that decides correctness: the stored time
# changes only when a message is shown (a hidden repeat
# must not reset the clock), and the comparison is strict
# (> 10).

# =====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# =====================================================

import heapq


# =====================================================
# MAIN QUESTION
```

```
# =====================================================
# ---- main question: main function ----
# The stream is a list of (timestamp, message). Show a message only if MORE
# than `gap` seconds have passed since it was last SHOWN; a hidden repeat
# changes nothing (it does not move the last-shown time).
# Idea: a dictionary from message to the time it was last shown. One lookup
# per message.
# Complexity: Time O(n), Space O(distinct messages)
def filter_status(stream, gap=10):
    last_shown = {}
    shown = []
    for ts, msg in stream:
        if msg not in last_shown or ts - last_shown[msg] > gap:
            last_shown[msg] = ts
            shown.append((ts, msg))
    return shown


# ---- main question: tests ----
STREAM = [(0, "a"), (5, "a"), (10, "a"), (11, "a"), (12, "b"), (22, "a")]


def test_main_question():
    # 10 is hidden (not MORE than 10 after 0); 11 shows
    assert filter_status(STREAM) == [(0, "a"), (11, "a"), (12, "b"), (22, "a")]
    assert filter_status([(0, "A"), (1, "a")]) == [(0, "A"), (1, "a")]       # messages are case sensitive


# =====================================================
# FOLLOW-UP 1: The stream never ends; memory must be bounded
```

```
# ========================================================
# ---- follow-up 1: function ----
# Answer: a message older than the gap can never be blocked again, so
# forget it. Keep a min-heap of (time shown, message) and, before every
# decision, remove the entries older than the gap; delete the dictionary key
# only if it still points at that time (it may have been shown again since).
# Complexity: Time show is O(log n) amortized, Space O(messages inside the window)
class BoundedFilter:
    def __init__(self, gap=10):
        self.gap = gap
        self.last = {}                          # message -> the time it was last shown
        self.heap = []                          # (time shown, message), oldest first

    def show(self, ts, msg):
        while self.heap and ts - self.heap[0][0] > self.gap:
            old_ts, old_msg = heapq.heappop(self.heap)
            if self.last.get(old_msg) == old_ts:
                del self.last[old_msg]          # not shown again since: forget it
        if msg in self.last:
            return False
        self.last[msg] = ts
        heapq.heappush(self.heap, (ts, msg))
        return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    bounded = BoundedFilter()
    answers = []
    for ts, msg in STREAM:
        answers.append(bounded.show(ts, msg))
    assert answers == [True, False, False, True, True, True]
    assert len(bounded.last) <= 2


# ========================================================
# FOLLOW-UP 2: Several robots send messages
# ========================================================


# ========================================================
# FOLLOW-UP 3: The interval depends on the message severity
# ========================================================
# ---- follow-up 2: function ----
# Answer: use (robot, message) as the key (or one filter per robot): the
# same logic applies per key. For severity, look up the gap for that severity
# in a table (with a default).
# Complexity: Time O(1) per message, Space O(distinct robot and message pairs)
class PerRobotFilter:
    def __init__(self, gaps, default=10):
        self.gaps = gaps                        # severity -> its gap
        self.default = default
        self.last = {}

    def show(self, robot, ts, msg, severity=None):
        key = (robot, msg)
        gap = self.gaps.get(severity, self.default)
        last = self.last.get(key)
        if last is not None and ts - last <= gap:
            return False
        self.last[key] = ts
        return True
```

```python
# ---- follow-up 2: tests ----
def test_follow_up_2():
    robots = PerRobotFilter({"high": 2})
    assert robots.show("r1", 0, "x")
    assert not robots.show("r1", 9, "x")
    assert robots.show("r2", 1, "x")                # another robot
    assert robots.show("r1", 1, "y", "high")
    assert robots.show("r1", 4, "y", "high")        # a stricter severity: a gap of 2 is enough


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```python
def filter_messages(events: list[tuple[int, str]]) -> list[tuple[int, str]]:
```

Sample call

```
filter_messages([])
```

Test cases

Input	Expected
`([],)`	`[]`
`([[10, "solar panel activated"], [13, "solar panel activated"], [20, "solar panel activated"], [21, "solar panel activated"]],)`	`[[10, "solar panel activated"], [21, "solar panel activated"]]`

ALSO REPORTED · 1 more

Robot Status Message Deduplication Window | 1Point3Acres | onsite-coding | asked 2026-09-10 | Low

https://www.1point3acres.com/interview/problems/company/google/robot-status-message-deduplication

The state transition occurs only on a displayed event. Updating on every receipt changes the semantics and can suppress a frequently repeated message forever.

Resolve the exact-boundary rule before coding and make the comparison match that contract.

For out-of-order arrival, clarify how the 10-second interval applies to late events and whether an earlier print decision can ever be revisited. These semantics are not specified by the logger variant; do not assume the chronological-stream contract resolves them.

Preparation: Implement the stream processor with per-message state, then trace repeated and interleaved messages by hand.

Write tests for the first occurrence, an event just inside the window, exactly at the boundary, just outside it, and a hidden event followed by an allowed display.

STUDY PATH: question 60 of 197 | topic 10 of 43: A dict of per-key state that you update, expire, or rank | 3 of 7

Template fit: defaultdict(Counter): counts[a][b] += 1 for neighbouring words; remember the best next word per word | * asked 48 days ago * | frequency Single report

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 60 of 197

Next-Word Predictor

ASKED 2026-09-02 | 48 DAYS AGO | LAST 90 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 284/288**

ABOUT THIS QUESTION

Topic: A dict of per-key state that you update, expire, or rank

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/next-word-frequency-predictor

QUESTION

Train on sentences, then predict the word that most often follows a given word. train(sentence) learns every pair of neighbouring words; predict(word) returns the word that followed it most often (ties go to the alphabetically smaller word) or None if nothing ever followed it. predict must run in O(1).

Function signature

```
class Predictor:
    def train(self, sentence: str) -> None
    def predict(self, word: str) -> str | None
```

EXAMPLES

Example 1

Input	`p = Predictor()`
	`p.train('I love apples')`
	`p.train('I love pears')`
	`p.train('you love pears too')`
	`p.predict('love')`
	`p.predict('I')`
	`p.predict('too')`

Output	`p.predict('love') -> 'pears'`
	`p.predict('I') -> 'love'`
	`p.predict('too') -> null`

Why	`After 'love' came 'apples' once and 'pears' twice, so 'pears' wins. Nothing ever followed 'too'.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
sentences = []
def train(sentence):
    sentences.append(sentence.split())
def predict(word):
    counts = Counter()
    for words in sentences:
        for a, b in zip(words, words[1:]):
            if a == word:
                counts[b] += 1
    if not counts:
        return None
    return max(counts, key=counts.get)
```

TIME	SPACE
O(corpus size) per prediction	**O(corpus size)**

WHY IT IS SLOW

Each prediction is a full pass over the corpus. Counting pairs while training and keeping the best successor per word makes prediction O(1).

OPTIMAL SOLUTION · Counts plus a maintained best successor

TIME	SPACE
O(1) per pair when training and per prediction	**O(number of distinct pairs)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the top K predictions.

Keep the Counter per word and use heapq.nsmallest on (-count, word): O(d log K) per query, or keep a sorted structure per word if queries dominate.

FOLLOW-UP 2 Use two words of context (trigrams).

The key becomes the pair (previous two words) and everything else is identical, at the price of more memory. Back off to the bigram table when the pair was never seen.

FOLLOW-UP 3 The corpus keeps changing and old text should fade.

Use an exponentially decayed count (multiply by a factor per training step, or store a timestamp and decay on read), and recheck the best when a count drops below another.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Next-Word Predictor
#
# Train on sentences, then predict the word that most
# often follows a given word. train(sentence) learns every
```

```
# pair of neighbouring words; predict(word) returns the
# word that followed it most often (ties go to the
# alphabetically smaller word) or None if nothing ever
# followed it. predict must run in O(1).
#
# Signature:
#   class Predictor:
#       def train(self, sentence: str) -> None
#       def predict(self, word: str) -> str | None
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the top K predictions.
# 2. Use two words of context (trigrams).
# 3. The corpus keeps changing and old text should fade.
#
# INTERVIEW NOTES
# Prediction in O(1) means the answer must already be
# sitting in a table when the query arrives, so do the
# work while training. Keep counts per (word, next word)
# and, for each word, the current best successor. Each
# count only increases by one, so the best can be updated
# by comparing just the one count that changed.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main class ----
# train(sentence) learns every pair of neighbouring words; predict(word)
# returns the word that followed it most often (ties: the alphabetically
# smaller word), or None. predict must be O(1).
# Idea: counts[word][next] is the number of times `next` followed `word`.
```

```
# After every increment compare that one count with the best successor kept for
# the word (a higher count wins, then the smaller word). predict then only has to
# read the stored best word.
# Complexity: Time train O(W) for W words, predict O(1), Space O(number of different word pairs)
class Predictor:
    def __init__(self):
        self.counts = {}                        # word -> {next word: how often}
        self.best = {}                          # word -> the most likely next word so far

    def train(self, sentence):
        words = sentence.split()
        for i in range(len(words) - 1):
            self.update(words[i], words[i + 1])

    def update(self, word, following):
        if word not in self.counts:
            self.counts[word] = {}
        table = self.counts[word]
        table[following] = table.get(following, 0) + 1
        current = self.best.get(word)
        if current is None:
            self.best[word] = following
        elif table[following] > table[current]:
```

```
            self.best[word] = following
        elif table[following] == table[current] and following < current:
            self.best[word] = following             # a tie goes to the smaller word

    def predict(self, word):
        return self.best.get(word)


# ---- main question: tests ----
def test_main_question():
    p = Predictor()
    p.train("i love dogs")
    p.train("i love cats")
    p.train("i love cats and dogs")
    assert p.predict("love") == "cats"             # "love" is followed by cats twice, dogs once
    assert p.predict("zebra") is None
    q = Predictor()
    q.train("a z")
    q.train("a b")
    assert q.predict("a") == "b"                   # a tie goes to the alphabetically smaller word
```

```
# ======================================================
# FOLLOW-UP 1: Return the top K predictions
# ======================================================
# ---- follow-up 1: function ----
# Answer: the table of counts per word is already there: pick the k best
# (count, word) rows with a heap (heapq.nsmallest on (-count, word)): O(d log k)
# per query for d different followers (or keep a sorted structure per word if
# queries are much more common than training).
# Complexity: Time O(u log k) for u different next words, Space O(u)
def top_k(predictor, word, k):
    rows = []
    for following, count in predictor.counts.get(word, {}).items():
        rows.append((-count, following))
    best = heapq.nsmallest(k, rows)
    words = []
    for minus_count, following in best:
        words.append(following)
    return words


# ---- follow-up 1: tests ----
def test_follow_up_1():
    p = Predictor()
    p.train("i love dogs")
    p.train("i love cats")
    p.train("i love cats and dogs")
    assert top_k(p, "love", 2) == ["cats", "dogs"]
    assert top_k(p, "zebra", 2) == []


# ======================================================
# FOLLOW-UP 2: Use two words of context (trigrams)
# ======================================================
# ---- follow-up 2: function ----
# Answer: the key becomes the PAIR of previous words and everything else is
# the same, at the price of more memory. Try the longest context first and
# back off to a shorter one when that context was never seen. orders=(2, 1)
# means "two words of context, then one".
# Complexity: Time train O(W * orders) for W words, predict O(orders), Space O(W * orders)
class ContextPredictor:
    def __init__(self, orders=(2, 1)):
        self.orders = sorted(orders, reverse=True)        # the longest context first
        self.counts = {}                                  # (order, context words) -> {next word: how often
```

```
}
        self.best = {}                                  # (order, context words) -> the most likely next
word
```

```
    def train(self, sentence):
        words = sentence.split()
        for k in self.orders:
            for i in range(len(words) - k):
                key = (k, tuple(words[i:i + k]))
                self.update(key, words[i + k])

    def update(self, key, following):
        if key not in self.counts:
            self.counts[key] = {}
        table = self.counts[key]
        table[following] = table.get(following, 0) + 1
        current = self.best.get(key)
        if current is None or table[following] > table[current]:
            self.best[key] = following
        elif table[following] == table[current] and following < current:
            self.best[key] = following

    def predict(self, *context):
        for k in self.orders:
            if len(context) >= k:
                key = (k, tuple(context[len(context) - k:]))
                if key in self.best:
                    return self.best[key]
        return None


# ---- follow-up 2: tests ----
def test_follow_up_2():
    p = ContextPredictor()
    p.train("i love dogs")
    p.train("i love cats")
    p.train("i love cats and dogs")
    assert p.predict("love") == "cats"
    assert p.predict("i", "love") == "cats"             # two words of context are tried first
    assert p.predict("we", "love") == "cats"            # an unseen pair backs off to the last word
    assert p.predict("zebra") is None


# ============================================================
# FOLLOW-UP 3: The corpus keeps changing and old text should fade
# ============================================================
# Answer: multiply the old counts by a factor below 1 from time to time
# (exponential decay), or count only the most recent N sentences with a
# queue and subtract the counts of the sentence that leaves.
```

```


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement pair counting and best-successor maintenance during processing, then compare predictions with a full scan of each successor counter.
Trace repeated pairs, a tie, a new leader, an unseen word, and separate training sequences; write down the tie and sequence-boundary conventions before testing.

STUDY PATH: question 61 of 197 | topic 10 of 43: A dict of per-key state that you update, expire, or rank | 4 of 7

Template fit: dict message -> first (or latest) timestamp depending on the policy; sort the result at the end | * asked 89 days ago * | frequency Single report

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 61 of 197

Batch and Streaming Log Deduplication

ASKED 2026-07-23 | 89 DAYS AGO | LAST 90 DAYS

STUDY: READ NO LEETCODE FREQUENCY PRIORITY 286/288

ABOUT THIS QUESTION

Topic: A dict of per-key state that you update, expire, or rank

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Frequency: Single report | Last asked: 2026-07-23 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/batch-streaming-log-deduplication

QUESTION

Given log entries (timestamp, message), keep one entry per message and return the kept entries sorted by timestamp. With policy 'first' keep the entry seen first for each message; with policy 'latest' keep the one with the latest timestamp.

Function signature

def dedupe(logs: list, policy: str = 'first') -> list

EXAMPLES

Example 1	
Input	`[(5, 'a'), (1, 'b'), (9, 'a'), (3, 'c'), (2, 'b')]`
Output	`[(1, 'b'), (3, 'c'), (5, 'a')]`
Why	`Messages 'a' and 'b' each appear twice; the first one seen is kept.`

Example 2	
Input	`[(5, 'a'), (1, 'b'), (9, 'a'), (3, 'c'), (2, 'b')], policy = 'latest'`
Output	`[(2, 'b'), (3, 'c'), (9, 'a')]`
Why	`With 'latest', the later timestamp of each repeated message wins.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(logs, keep_first=True):
    kept = []
    for entry in logs:
        same = [x for x in logs
                if x[1] == entry[1]]
        pick = max(same)
        if keep_first:
            pick = same[0]
        if entry == pick:
            kept.append(entry)
    return sorted(kept)  # by timestamp
```

TIME	SPACE
O(n^2)	O(n)

BRUTE FORCE

WHY IT IS SLOW

Each line scans the others: O(n^2). A dictionary from message to the kept entry is one pass, and a heap with lazy deletion serves the streaming version.

OPTIMAL SOLUTION · Dictionary on the message, then sort

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Streaming: map + min-heap with lazy deletion

TIME	SPACE
O(log n) amortized per add and get_next	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Late data: how do you decide when it is safe to emit?

Use a watermark: only release entries whose timestamp is at most (the largest timestamp seen minus the allowed lateness). get_next then refuses to return anything newer than the watermark, and anything arriving older than it is dropped or sent to a late-data side output.

FOLLOW-UP 2 Memory must stay bounded.

Evict messages older than a retention window (a second heap or the same heap keyed on timestamp) and accept that duplicates arriving after eviction are no longer detected. **Cost:** O(log n) per eviction with a heap; memory O(entries inside the retention window).

FOLLOW-UP 3 Keep the entry with the most occurrences instead.

Store (count, timestamp) per message and update the count on every duplicate; the heap key changes only if you rank by count, so push a new entry when the rank changes and rely on lazy deletion again.

FOLLOW-UP 4 Only release entries once they are safe (late data may still arrive).

Add a watermark: release only entries whose timestamp is at most (the largest timestamp seen minus the allowed lateness), in order; anything older than the watermark that arrives later goes to a late-data path. The lazy-deletion heap pops stale entries as it goes.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Batch and Streaming Log Deduplication
#
# Given log entries (timestamp, message), keep one entry
# per message and return the kept entries sorted by
```

```
# timestamp. With policy 'first' keep the entry seen first
# for each message; with policy 'latest' keep the one with
# the latest timestamp.
#
# Signature:
#   def dedupe(logs: list, policy: str = 'first') -> list
#
# Examples:
# [(5, 'a'), (1, 'b'), (9, 'a'), (3, 'c'), (2, 'b')]  ->
#     [(1, 'b'), (3, 'c'), (5, 'a')]
# [(5, 'a'), (1, 'b'), (9, 'a'), (3, 'c'), (2, 'b')],
#     policy = 'latest'  ->  [(2, 'b'), (3, 'c'), (9,
#     'a')]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Late data: how do you decide when it is safe to emit?
# 2. Memory must stay bounded.
# 3. Keep the entry with the most occurrences instead.
# 4. Only release entries once they are safe (late data
# may still arrive).
#
# INTERVIEW NOTES
# Dedup is a dictionary keyed by the message. Sorting is
# only for the output order. For a stream, a heap gives
# the smallest timestamp quickly, but when an entry is
# replaced you cannot delete from the middle of a heap, so
# leave the stale entry and skip it when it surfaces (lazy
# deletion).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
```

```
# ============================================================
# ---- main question: main function ----
# logs are (timestamp, message) pairs. Keep ONE entry per message: the
# first one seen (policy "first") or the newest (policy "latest").
# Idea: one pass with a dictionary from message to the kept timestamp; then
# sort the kept entries by time for the output.
# Complexity: Time O(n log n) (the final sort), Space O(n)
def dedupe(logs, policy="first"):
    kept = {}
    for ts, msg in logs:
        if msg not in kept:
            kept[msg] = ts
        elif policy == "latest" and ts > kept[msg]:
            kept[msg] = ts
    entries = []
    for msg in kept:
        entries.append((kept[msg], msg))
    entries.sort()
    return entries


# ---- main question: tests ----
def test_main_question():
```

```
    logs = [(5, "b"), (1, "a"), (9, "a"), (3, "b")]
    assert dedupe(logs, "first") == [(1, "a"), (5, "b")]
    assert dedupe(logs, "latest") == [(5, "b"), (9, "a")]


# ====================================================
# FOLLOW-UP 1: Late data: how do you decide when it is safe to emit?
# ====================================================


# ====================================================
# FOLLOW-UP 4: Only release entries once they are safe (late data may still arrive)
# ====================================================
# ---- follow-up 4: function ----
# Answer: use a WATERMARK: release only entries whose timestamp is at most
# (the largest timestamp seen minus the allowed lateness), in time order.
# Anything that arrives older than the watermark goes to a late-data path
# (here: it is dropped). Entries wait in a min-heap; an entry that was replaced
# by a newer one is skipped when it reaches the top ("lazy deletion").
# Complexity: Time add O(log n), get_next O(log n) amortized, Space O(n)
class StreamDedup:
    def __init__(self, policy="first", lateness=0):
        self.policy = policy
        self.lateness = lateness
        self.kept = {}                          # message -> the timestamp currently kept
        self.heap = []                          # (timestamp, message); may hold outdated entries
        self.max_ts = None

    def watermark(self):
        # everything at or below this is safe to release
        if self.max_ts is None:
            return None
        return self.max_ts - self.lateness

    def add(self, ts, msg):
        mark = self.watermark()
        if mark is not None and ts < mark:
            return False                        # too late: older than the watermark
        if self.max_ts is None or ts > self.max_ts:
            self.max_ts = ts
        old = self.kept.get(msg)
        if old is None or (self.policy == "latest" and ts > old):
            self.kept[msg] = ts
            heapq.heappush(self.heap, (ts, msg))
        return True

    def get_next(self):
        mark = self.watermark()
        while self.heap:
            ts, msg = self.heap[0]
            if self.kept.get(msg) != ts:
                heapq.heappop(self.heap)        # this entry was replaced: skip it
                continue
            if ts > mark:
                return None                     # not safe yet: later data may still arrive
            heapq.heappop(self.heap)
            del self.kept[msg]
            return ts, msg
        return None


# ---- follow-up 4: tests ----
def test_follow_up_4():
    s = StreamDedup("first", lateness=2)
```

```
    for ts, msg in [(1, "a"), (3, "c"), (5, "b")]:
        assert s.add(ts, msg)
    assert s.get_next() == (1, "a")                # the watermark is 5 - 2 = 3: release 1, then 3
```

```
    assert s.get_next() == (3, "c")
    assert s.get_next() is None                    # 5 is still above the watermark
    assert not s.add(2, "late")                    # data older than the watermark is dropped
    t = StreamDedup("latest")
    t.add(1, "x")
    t.add(4, "x")
    assert t.get_next() == (4, "x")                # the "latest" policy replaced the kept entry
    assert t.get_next() is None


# ======================================================
# FOLLOW-UP 2: Memory must stay bounded
# ======================================================
# Answer: only messages that can still repeat must be remembered: with a
# watermark, forget a message once its entry has been released and it is
# older than the lateness window (the streaming class does this by deleting
# a message when it is released).


# ======================================================
# FOLLOW-UP 3: Keep the entry with the most occurrences instead
# ======================================================
# Answer: keep a count per message as well and keep the entry with the largest
# count (ties: the earliest); the dictionary then maps a message to (count, ts).


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement both batch retention policies and write deterministic tie tests.
Build the streaming interface with a membership map, min-heap, and lazy deletion, then state the amortized operation costs.
Rehearse a bounded-memory design using expiry and heap rebuilding, plus a partitioning strategy that keeps each message on one shard.

STUDY PATH: question 62 of 197 | topic 10 of 43: A dict of per-key state that you update, expire, or rank | 5 of 7

Template fit: dict message -> list of timestamps; top_k counts each list inside the time range with bisect, then takes the k largest | * asked 287 days ago *

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 62 of 197

★ TOP 75: rank 36 in the Medium and Hard priority list

★ Design a logger that returns top-K messages

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TYPE **ALL TIME 51.2** **30 DAYS 33** **PRIORITY 40/288**

ABOUT THIS QUESTION

Topic: A dict of per-key state that you update, expire, or rank

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #359 Logger Rate Limiter, LeetCode #692 Top K Frequent Words

LeetCode match: LC 359 Logger Rate Limiter + LC 692 Top K Frequent Words (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-05-23)

Original: https://prachub.com/coding-questions/design-a-logger-that-returns-top-k-messages

QUESTION

Design a logger. log(ts, msg) records a message at time ts (timestamps arrive in non-decreasing order). top_k(k, start, end) returns the k most frequent messages whose timestamps fall in [start, end]; leave start and end out to use the whole history. Ties are broken alphabetically.

Function signature

class Logger:

```
def log(self, ts: int, msg: str) -> None
def top_k(self, k: int, start=None, end=None) -> list
```

EXAMPLES

Example 1

Input	`lg = Logger()`
	`lg.log(1, 'disk full')`
	`lg.log(2, 'timeout')`
	`lg.log(3, 'timeout')`
	`lg.log(4, 'disk full')`
	`lg.log(5, 'timeout')`
	`lg.top_k(1)`
	`lg.top_k(2, 4, 5)`
	`lg.top_k(2, 1, 1)`

Output	`lg.top_k(1) -> ['timeout']`
	`lg.top_k(2, 4, 5) -> ['disk full', 'timeout']`
	`lg.top_k(2, 1, 1) -> ['disk full']`

Why	Overall 'timeout' occurred 3 times. In the window [4, 5] 'disk full' and 'timeout' occur once each (tie, alphabetical). The window [1, 1] only holds 'disk full'.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
logs = []  # (timestamp, message)
def top_k(k, start, end):
    counts = Counter()
    for ts, msg in logs:
        if start <= ts <= end:
            counts[msg] += 1
    def rank(m):
        return (-counts[m], m)
    return sorted(counts, key=rank)[:k]
```

TIME	SPACE
O(n + d log d) per query	O(n)

WHY IT IS SLOW

Every query reads the whole log, O(n + d log d). Per-message sorted timestamps with binary search answer a window count in O(log n), and a heap picks the top k.

OPTIMAL SOLUTION · Per-message sorted timestamps plus a heap for the top K

TIME	SPACE
O(1) per log, O(d log n + d log k) per query	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Print a message only if it was not printed in the last 10 seconds (LeetCode 359).

Keep a dict message to the time it was last printed and compare: print if the gap is at least 10 (and update the time only when you print). O(1) per call.

FOLLOW-UP 2 Memory must stay bounded.

Expire timestamps older than the largest window you support (drop from the front of each list, delete messages with no timestamps left), or approximate counts with a count-min sketch plus a heap for the heavy hitters. **Cost:** O(1) amortized to drop expired timestamps; memory O(messages inside the window).

FOLLOW-UP 3 Very high top_k frequency of calls.

Maintain counts per message incrementally and a top-K structure (a heap with lazy updates or a sorted structure) so top_k costs O(k) instead of O(d log k); windowed queries still need per-message timestamps.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Design a logger that returns top-K
# messages
#
# Design a logger. log(ts, msg) records a message at time
```

```
# ts (timestamps arrive in non-decreasing order). top_k(k,
# start, end) returns the k most frequent messages whose
# timestamps fall in [start, end]; leave start and end out
# to use the whole history. Ties are broken
# alphabetically.
#
# Signature:
#   class Logger:
#       def log(self, ts: int, msg: str) -> None
#       def top_k(self, k: int, start=None, end=None) -> list
#
# FOLLOW-UPS (answered below, in this order):
# 1. Print a message only if it was not printed in the
# last 10 seconds (LeetCode 359).
# 2. Memory must stay bounded.
# 3. Very high top_k frequency of calls.
#
# INTERVIEW NOTES
# Store enough to answer both questions without scanning
# everything. A dict from message to its list of
# timestamps (appended in time order) answers 'how many in
# [start, end]' with two binary searches, and a heap of
# size K over (count, message) gives the top K. Say how
# the logger handles repeated messages and what 'top'
# means with ties.

# =====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# =====================================================

import heapq


# =====================================================
# MAIN QUESTION
# =====================================================
# ---- main question: helper functions ----
# Complexity: Time O(log n), Space O(1)
```

```
def first_at_least(items, x):
    # binary search in a sorted list: the first index whose value is >= x
    low = 0
    high = len(items)
    while low < high:
        middle = (low + high) // 2
        if items[middle] < x:
            low = middle + 1
        else:
            high = middle
    return low


# Complexity: Time O(log n), Space O(1)
def first_greater(items, x):
    # binary search in a sorted list: the first index whose value is > x
    low = 0
    high = len(items)
    while low < high:
        middle = (low + high) // 2
        if items[middle] <= x:
            low = middle + 1
        else:
```

```
            high = middle
    return low


# ---- main question: main class ----
# log(ts, msg) records a message at time ts (timestamps never go backwards).
# top_k(k, start, end) returns the k most frequent messages with a timestamp in
# [start, end] (the whole history if they are left out); ties alphabetically.
# Idea: keep, per message, the list of its timestamps. They arrive in
# order, so every list stays sorted. The number of times a message occurs in
# [start, end] is two binary searches: (first index > end) - (first index >= start).
# A heap then picks the k best (count, message) rows.
# Complexity: Time log O(1), top_k O(M log n + M log k) for M messages, expire O(total timestamps), Space O
(n)
class Logger:
    def __init__(self):
        self.times = {}                          # message -> its timestamps, in order of arrival

    def log(self, ts, msg):
        if msg not in self.times:
            self.times[msg] = []
        self.times[msg].append(ts)
```

```
    def top_k(self, k, start=None, end=None):
        rows = []
        for msg in self.times:
            stamps = self.times[msg]
            low = 0
            high = len(stamps)
            if start is not None:
                low = first_at_least(stamps, start)
            if end is not None:
                high = first_greater(stamps, end)
            if high > low:
                rows.append((-(high - low), msg))      # minus the count: the most frequent sorts first
        best = heapq.nsmallest(k, rows)
        messages = []
        for minus_count, msg in best:
            messages.append(msg)
        return messages

    def expire(self, before):
        # bounded memory (follow-up 2): forget timestamps older than `before`
        for msg in list(self.times):
            stamps = self.times[msg]
            self.times[msg] = stamps[first_at_least(stamps, before):]
            if not self.times[msg]:
                del self.times[msg]


# ---- main question: tests ----
def test_main_question():
    logger = Logger()
    for ts, msg in [(1, "a"), (2, "b"), (3, "a"), (4, "c"), (5, "a"), (6, "b")]:
        logger.log(ts, msg)
    assert logger.top_k(2) == ["a", "b"]
    assert logger.top_k(3, 4, 6) == ["a", "b", "c"]          # only the window [4, 6]: a once, b once, c on
ce
    assert logger.top_k(2, 100, 200) == []
    logger.expire(4)
    assert logger.top_k(5) == ["a", "b", "c"]


# ====================================================
# FOLLOW-UP 1: Print a message only if it was not printed in the last 10 seconds (LeetCode 359)
```

```
# ============================================================
# ---- follow-up 1: function ----
# Answer: keep a dictionary message -> the time it was last PRINTED, and
# compare: print if the gap is at least 10 (and update the time only when you
```

```
# print). O(1) per call.
# Complexity: Time O(1) per message, Space O(number of distinct messages)
class RateLimiter:
    def __init__(self, gap=10):
        self.gap = gap
        self.last = {}

    def should_print(self, ts, msg):
        last = self.last.get(msg)
        if last is not None and ts - last < self.gap:
            return False
        self.last[msg] = ts                          # only a printed message starts a new window
        return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    limiter = RateLimiter()
    answers = []
    for t in (1, 2, 11, 12, 21):
        answers.append(limiter.should_print(t, "x"))
    assert answers == [True, False, True, False, True]


# ============================================================
# FOLLOW-UP 2: Memory must stay bounded
# ============================================================
# Answer: forget old data: expire(before) above drops the timestamps older
# than a limit, and removes messages that have none left. Run it from time to
# time (or on every log call for the oldest entries).


# ============================================================
# FOLLOW-UP 3: Very high top_k frequency of calls
# ============================================================
# Answer: do not rescan every message on each call: keep a running count per
# message and a heap or sorted structure that is updated when a count changes,
# and cache the answer for the whole history until the next log call.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(logs: list[tuple[int, str]], queries: list[list[int]]) -> list[list[str]]:
```

Sample call

```
solution([(1, 'A'), (2, 'B'), (3, 'A'), (10, 'C')], [(1, 3, 2), (1, 10, 3)])
```

Test cases

Input	Expected
`([(1, 'A'), (2, 'B'), (3, 'A'), (10, 'C')], [(1, 3, 2), (1, 10, 3)])`	`[['A', 'B'], ['A', 'B', 'C']]`
`([(1, 'dog'), (2, 'cat'), (2, 'dog'), (4, 'cat'), (5, 'ant')], [(1, 4, 2)])`	`[['cat', 'dog']]`

STUDY PATH: question 63 of 197 | topic 10 of 43: A dict of per-key state that you update, expire, or rank | 6 of 7

Template fit: dict activity -> set of users for unique counts; dict user -> sorted events to add up login-to-logout session time | * asked 116 days ago * | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 63 of 197

Website Activity Analytics: Unique Users + Session Time

ASKED 2026-06-26 | 116 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 277/288**

ABOUT THIS QUESTION

Topic: A dict of per-key state that you update, expire, or rank

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-06-26 | Roles: DS

Original: https://www.1point3acres.com/interview/problems/company/google/website-activity-analytics-unique-users-session-time

QUESTION

Each record has a user_id, a timestamp ('YYYY-MM-DD HH:MM:SS') and an activity. Return (1) the number of distinct users per activity and (2) the average number of seconds a user spends on the site. A user's time is the sum of the gaps from each 'login' to the next 'logout'; a login without a logout counts until the user's last event.

Function signature

def activity_stats(records: list) -> (dict, float)

EXAMPLES

Example 1

Input	`rec = lambda u, t, a: {'user_id': u, 'timestamp': '2026-01-01 ' + t, 'activity': a}`
	`records = [rec(1, '10:00:00', 'login'), rec(1, '10:10:00', 'click'), rec(1, '10:30:00', 'logout'), rec(2, '10:05:00', 'login'), rec(2, '10:20:00', 'logout')]`
	`activity_stats(records)`

Output	`activity_stats(records) -> ({'click': 1, 'login': 2, 'logout': 2}, 1350)`
Why	`User 1 spent 30 minutes (1800 s), user 2 spent 15 minutes (900 s): the average is 1350 s. Both users logged in and out; only user 1 clicked.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
for activity in activities:
    users = set()
    for rec in records:
        if rec.activity == activity:
            users.add(rec.user)
    unique[activity] = len(users)
for user in all_users:
    mine = [e for e in records if e.user == user]
    events = sorted(mine)
    spent = 0
    for login, logout in pairs(events):
        spent += logout.time - login.time
    total[user] = spent
average = sum(total.values()) / len(total)
```

BRUTE FORCE

TIME	SPACE
O(U * N + A * N)	O(U + A)

WHY IT IS SLOW

Rescans the whole log for every activity and every user. One pass that groups by activity and by user, then sorts each user's events once, is O(N log N).

OPTIMAL SOLUTION · One pass to group, then sessions per user

TIME	SPACE
O(N log N)	O(N)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Users with several sessions, missing logouts or out-of-order rows.

Sort each user's events by time before pairing; a second login before a logout is ignored (or closes the old session, depending on the contract you agree). A login with no logout ends at the user's last event, or is dropped if you decide it is unreliable. Say the choice out loud: the interviewer accepted a strong discussion over perfect code.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Website Activity Analytics: Unique Users
# + Session Time
#
# Each record has a user_id, a timestamp ('YYYY-MM-DD
# HH:MM:SS') and an activity. Return (1) the number of
# distinct users per activity and (2) the average number
# of seconds a user spends on the site. A user's time is
# the sum of the gaps from each 'login' to the next
# 'logout'; a login without a logout counts until the
# user's last event.
#
# Signature:
#   def activity_stats(records: list) -> (dict, float)
#
# FOLLOW-UPS (answered below, in this order):
# 1. Users with several sessions, missing logouts or out-
# of-order rows.
#
# INTERVIEW NOTES
# Unique users per activity is a map from activity to a
# set of user ids. Time on site needs a definition: pair
# each login with the next logout of the same user, and
# say what you do for a login without a logout (here: the
# session ends at that user's last event). Parse
# timestamps once, sort each user's events by time, and
# agree on out-of-order rows and repeated logins before
# coding.

# ================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
```

```
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from datetime import datetime

FORMAT = "%Y-%m-%d %H:%M:%S"


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def when(record):
```

```
    # the timestamp text of a record as a date and time
    return datetime.strptime(record["timestamp"], FORMAT)


def unique_users_by_activity(records):
    # per activity: how many DIFFERENT users did it (a set counts each user once)
    users = {}
    for record in records:
        if record["activity"] not in users:
            users[record["activity"]] = set()
        users[record["activity"]].add(record["user_id"])
    counts = {}
    for activity in sorted(users):
        counts[activity] = len(users[activity])
    return counts


def session_seconds(events):
    # one user's (time, activity) rows, in any order: the seconds spent logged in.
    # Sort by time first (follow-up 1: rows may arrive out of order). A second
    # login before a logout is ignored; a login with no logout ends at the
    # user's last event.
    total = 0
    start = None
    for t, activity in sorted(events):
        if activity == "login" and start is None:
            start = t
        elif activity == "logout" and start is not None:
            total += (t - start).total_seconds()
            start = None
    if start is not None:
        last = events[0][0]
        for t, activity in events:
            if t > last:
                last = t
        total += (last - start).total_seconds()
    return total


def average_time_per_user(records):
    # the average session time over EVERY user who appears (state the denominator)
    events = {}
    for record in records:
        if record["user_id"] not in events:
            events[record["user_id"]] = []
```

```
        events[record["user_id"]].append((when(record), record["activity"]))
    if not events:
        return 0.0
    total = 0
    for user in events:
```

```
        total += session_seconds(events[user])
    return total / len(events)


# ---- main question: main function ----
# records are {"user_id", "activity", "timestamp"}. Return the number of
# unique users per activity and the average session time per user.
# Idea: one pass groups the records, then each user's rows are paired up
# login -> logout.
# Complexity: Time O(n) for n records, Space O(users + activities)
def activity_stats(records):
    return unique_users_by_activity(records), average_time_per_user(records)


# ---- main question: tests ----
def record(user, minute, activity):
    return {"user_id": user, "activity": activity, "timestamp": "2024-07-26 10:" + str(minute).zfill(2) + "
:00"}


def test_main_question():
    data = [record(1, 0, "login"), record(2, 5, "login"), record(1, 10, "view"), record(1, 15, "purchase"),
            record(2, 20, "view"), record(3, 25, "login"), record(1, 30, "logout")]
    unique, average = activity_stats(data)
    assert unique == {"login": 3, "logout": 1, "purchase": 1, "view": 2}
    assert average == 900                                  # user 1: 1800 s, user 2: 900 s (no logout), us
er 3: 0
    assert activity_stats([]) == ({}, 0.0)


# =========================================================
# FOLLOW-UP 1: Users with several sessions, missing logouts or out-of-order rows
# =========================================================
# Answer: sort each user's events by time before pairing them (session_seconds
# does); two sessions add up; a second login before a logout is ignored (or
# closes the old session, depending on the contract you agree on); a missing
# logout ends the session at the user's last event.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    shuffled = [record(1, 30, "logout"), record(1, 0, "login")]
    assert average_time_per_user(shuffled) == 1800         # out-of-order rows are sorted first
    two = [record(1, 0, "login"), record(1, 5, "logout"), record(1, 10, "login"), record(1, 20, "logout")]
    assert average_time_per_user(two) == 900               # two sessions add up: 300 + 600 = 900


# =========================================================
# RUN EVERYTHING
# =========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Practice writing small Python analytics functions in a plain document without IDE help.
Drill grouping records by user and by activity with dictionaries / sets.
Rehearse explaining incomplete edge-case handling and dry-running a small event stream.

STUDY PATH: question 64 of 197 | topic 10 of 43: A dict of per-key state that you update, expire, or rank | 7 of 7

Template fit: one deque of hit times; drop the ones older than 300 seconds; the answer is the length | ND

MEDIUM | IN APP LC 362 | DESIGN-STYLE (skippable) | HackTheRounds | Phone Screen | 64 of 197

Hit Counter

STUDY: TALK ALL TIME 33.2 30 DAYS 0 PRIORITY 132/288

ABOUT THIS QUESTION

Topic: A dict of per-key state that you update, expire, or rank

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #362 Design Hit Counter

LeetCode match: LC 362

QUESTION

Design a hit counter. hit(timestamp) records a hit at that second, and get_hits(timestamp) returns the number of hits in the past 300 seconds (timestamps never go backwards).

Function signature

class QueueCounter:
    def hit(self, ts: int) -> None
    def get_hits(self, ts: int) -> int

EXAMPLES

Example 1

Input	`h = QueueCounter()`
	`h.hit(1)`
	`h.hit(2)`
	`h.hit(3)`
	`h.get_hits(4)`
	`h.hit(300)`
	`h.get_hits(300)`
	`h.get_hits(301)`

Output	`h.get_hits(4) -> 3`
	`h.get_hits(300) -> 4`
	`h.get_hits(301) -> 3`

Why	`At time 301 the hit at time 1 is exactly 300 seconds old and no longer counts.`

CONSTRAINTS

- 1 <= timestamp <= 2 * 10^9
- Timestamps are monotonically increasing

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
hits = []
def hit(t):
    hits.append(t)
def get_hits(t):
    return sum(1 for x in hits if x > t - 300)
```

BRUTE FORCE

TIME	SPACE
O(n) per getHits	**O(n)**

WHY IT IS SLOW

Memory grows with every hit forever, and each query scans them all, O(n). A queue that drops timestamps older than 300 seconds (or 300 per-second buckets) keeps both bounded.

OPTIMAL SOLUTION · Queue of timestamps with eviction

TIME	SPACE
O(1) amortized	**O(hits in the window)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Millions of hits per second.

Use 300 buckets (one per second) instead of one entry per hit: hit is O(1), getHits is O(300) = O(1), and memory is constant. A bucket whose stored second is older than the window is reset on reuse.

FOLLOW-UP 2 Timestamps can arrive out of order.

A deque no longer works. Keep a sorted structure (a sorted list with bisect or a balanced tree) and count with two binary searches, or use the buckets, which are order-insensitive within the window.

FOLLOW-UP 3 Thread safety.

Guard the queue with a lock (or use per-thread counters merged on read). The bucket version needs the compare-then-reset of a slot to be atomic. **Cost:** one lock per call, O(1) extra; contention rather than complexity is the concern.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Hit Counter
#
# Design a hit counter. hit(timestamp) records a hit at
# that second, and get_hits(timestamp) returns the number
# of hits in the past 300 seconds (timestamps never go
# backwards).
#
# Signature:
#   class QueueCounter:
#       def hit(self, ts: int) -> None
#       def get_hits(self, ts: int) -> int
#
# FOLLOW-UPS (answered below, in this order):
# 1. Millions of hits per second.
# 2. Timestamps can arrive out of order.
# 3. Thread safety.
#
# INTERVIEW NOTES
# Only the last 300 seconds matter, so anything older is
```

```
# dead weight: evict from the front of a queue of
# timestamps. If hits arrive at very high rates, store
# counts per second in a circular array of 300 buckets so
# memory does not depend on the number of hits.

# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

import random
import threading
from collections import deque

WINDOW = 300                                    # seconds


# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: main class ----
# hit(ts) records a hit at second ts; get_hits(ts) returns the number of
# hits in the past 300 seconds (timestamps never go backwards).
# Idea: keep a queue of the hit times in order of arrival. get_hits removes
```

```
# the hits from the front while the oldest is outside the window; what is left
# is the answer. Every hit is added once and removed once: O(1) amortised.
# Complexity: Time hit O(1), get_hits O(1) amortized, Space O(hits in the last 300 seconds)
class QueueCounter:
    def __init__(self):
        self.hits = deque()

    def hit(self, ts):
        self.hits.append(ts)

    def get_hits(self, ts):
        while self.hits and self.hits[0] <= ts - WINDOW:     # keep only the hits in (ts - 300, ts]
            self.hits.popleft()
        return len(self.hits)


# ---- main question: tests ----
def test_main_question():
    counter = QueueCounter()
    counter.hit(1)
    counter.hit(2)
    counter.hit(3)
    assert counter.get_hits(4) == 3
    counter.hit(300)
    assert counter.get_hits(300) == 4
    assert counter.get_hits(301) == 3


# ======================================================
# FOLLOW-UP 1: Millions of hits per second
# ======================================================
# ---- follow-up 1: function ----
# Answer: use 300 BUCKETS (one per second of the window) instead of one
# entry per hit: hit is O(1), get_hits adds up 300 buckets (constant), and
# memory is constant. A bucket that still holds an older second is reset when it
# is reused.
# Complexity: Time hit O(1), get_hits O(window), Space O(window) (fixed)
class BucketCounter:
```

```
    def __init__(self):
        self.stamp = [0] * WINDOW              # which second each bucket holds
        self.count = [0] * WINDOW

    def hit(self, ts):
        slot = ts % WINDOW
        if self.stamp[slot] != ts:
```

```
            self.stamp[slot] = ts              # the bucket still holds an old second: reuse it
            self.count[slot] = 0
        self.count[slot] += 1

    def get_hits(self, ts):
        total = 0
        for slot in range(WINDOW):
            if 0 <= ts - self.stamp[slot] < WINDOW:
                total += self.count[slot]
        return total


# ---- follow-up 1: tests ----
def test_follow_up_1():
    counter = BucketCounter()
    for ts in (1, 2, 3, 300):
        counter.hit(ts)
    assert counter.get_hits(300) == 4
    assert counter.get_hits(301) == 3
    assert counter.get_hits(600) == 0


# ========================================================
# FOLLOW-UP 2: Timestamps can arrive out of order
# ========================================================
# ---- follow-up 2: helper functions ----
def count_not_after(items, x):
    # binary search in a sorted list: how many items are <= x
    low = 0
    high = len(items)
    while low < high:
        middle = (low + high) // 2
        if items[middle] <= x:
            low = middle + 1
        else:
            high = middle
    return low


# ---- follow-up 2: function ----
# Answer: a queue no longer works. Keep the times in a SORTED list and count
# with two binary searches: the hits up to ts minus the hits up to ts - 300.
# (The buckets also work: they do not care about the order inside the window.)
# Complexity: Time hit O(n) (a sorted insert), get_hits O(log n), Space O(n)
class SortedCounter:
```

```
    def __init__(self):
        self.hits = []

    def hit(self, ts):
        position = count_not_after(self.hits, ts)
        self.hits.insert(position, ts)         # insert so the list stays sorted

    def get_hits(self, ts):
        return count_not_after(self.hits, ts) - count_not_after(self.hits, ts - WINDOW)
```

```
# ---- follow-up 2: tests ----
def test_follow_up_2():
    for make in (QueueCounter, BucketCounter, SortedCounter):
        counter = make()
        for ts in (1, 2, 3, 300):
            counter.hit(ts)
        assert counter.get_hits(300) == 4
        assert counter.get_hits(301) == 3               # the hit at second 1 is 300 seconds old: it is ou
t
        assert counter.get_hits(599) == 1
        assert counter.get_hits(600) == 0
        assert counter.get_hits(1000) == 0
    random.seed(24)
    for make in (QueueCounter, BucketCounter, SortedCounter):
        counter = make()
        log = []
        now = 0
        for step in range(500):
            now += random.randint(0, 40)
            if random.random() < 0.6:
                counter.hit(now)
                log.append(now)
            expected = 0
            for t in log:
                if now - t < WINDOW:
                    expected += 1
            assert counter.get_hits(now) == expected
    late = SortedCounter()                              # out-of-order input only works with the sorted ve
rsion
    for ts in (50, 10, 400, 30):
        late.hit(ts)
    assert late.get_hits(400) == 1
    assert late.get_hits(310) == 2


# =====================================================
```

```
# FOLLOW-UP 3: Thread safety
# =====================================================
# ---- follow-up 3: function ----
# Answer: guard the counter with a lock so that check-then-update steps
# (like resetting a bucket) are atomic; or keep a counter per thread and add
# them up when reading. The cost is one lock per call: contention, not
# complexity, is the concern.
# Complexity: Time each call takes one lock and then costs what the counter inside costs, Space O(1) extra
class Locked:
    def __init__(self, counter):
        self.counter = counter
        self.lock = threading.Lock()

    def hit(self, ts):
        with self.lock:
            self.counter.hit(ts)

    def get_hits(self, ts):
        with self.lock:
            return self.counter.get_hits(ts)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    shared = Locked(BucketCounter())

    def spam():
        for i in range(1000):
```

```
            shared.hit(7)

    workers = []
    for i in range(4):
        workers.append(threading.Thread(target=spam))
    for worker in workers:
        worker.start()
    for worker in workers:
        worker.join()
    assert shared.get_hits(7) == 4000


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

ALSO REPORTED · 1 more

Design a rolling hit counter API | PracHub | Technical Screen

https://prachub.com/coding-questions/design-a-rolling-hit-counter-api

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 11 OF 43

Binary search on a yes/no condition

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Binary search on a yes/no condition | 4 questions | tap a title to jump to it

READ FIRST: the code template for these 4 questions (tap here) >

M 65. Cutting Ribbons (LC 1891) **1Point3Acres * 2026-08-16 ***

M 66. First Bad Version with Parallel Search Follow-up **1Point3Acres * 2026-06-15 ***

M 67. Solve Banana Speed and Interval Merge **PracHub * 2025-01-27 ***

M 68. Given a list with response time for each server, calculate how long time it will take to process n requests. **ND**

CODE TEMPLATE | GROUP 11 OF 43 | 4 QUESTIONS, ONE PATTERN

Binary search on a yes/no condition

The one idea these questions share

If answers go no, no, no, yes, yes, yes as the value grows, binary search finds the first yes. Write ok(x), set lo and hi, and halve. The questions only differ in what x is (a length, a speed, a time, an index, a version) and how ok(x) is checked.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def first_true(lo, hi, ok):                    # smallest x in [lo, hi) with ok(x); hi if none
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid):                            # CHANGE: the yes/no test
            hi = mid
        else:
            lo = mid + 1
    return lo
```

2. Helper functions that usually get written next to it

```
def last_true(lo, hi, ok):                     # largest x in [lo, hi] with ok(x); lo - 1 if none
    return first_true(lo, hi + 1, lambda x: not ok(x)) - 1


def ceil_div(a, b):
    return -(-a // b)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
65	Cutting Ribbons (LC 1891)	x = the piece length; ok(x) = the ribbons give at least k pieces; you want the LARGEST ok, so search the first x that fails and subtract 1
66	First Bad Version with Parallel Search Follow-up	x = the version number; ok(x) = is_bad(x)
67	Solve Banana Speed and Interval Merge	x = the eating speed; ok(x) = total hours (ceil of pile / x summed) <= h; first_true over 1..max pile
68	Given a list with response time for each server, calculate how long time it will take to process n requests.	x = the elapsed time; ok(x) = sum(x // time_i) >= n requests

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 65 of 197 | topic 11 of 43: Binary search on a yes/no condition | 1 of 4

Template fit: x = the piece length; ok(x) = the ribbons give at least k pieces; you want the LARGEST ok, so search the first x that fails and subtract 1 | * asked 65 days ago * | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 65 of 197

Cutting Ribbons (LC 1891)

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 19.6 30 DAYS 0 PRIORITY 168/288

ABOUT THIS QUESTION

Topic: Binary search on a yes/no condition

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1891 Cutting Ribbons

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/cutting-ribbons-max-equal-length

QUESTION

You have ribbons of given lengths and an integer k. Each ribbon may be cut into pieces of one common integer length L (leftover bits are thrown away). Return the largest L that gives at least k pieces in total, or 0 if even length 1 cannot give k pieces.

Function signature

def max_length(ribbons: list, k: int) -> int

EXAMPLES

Example 1	
Input	`[9, 7, 5], 3`
Output	`5`
Why	`Length 5 gives 1 + 1 + 1 = 3 pieces, length 6 only 2.`

Example 2	
Input	`[5, 7], 12`
Output	`1`
Why	`Length 1 already gives 12 pieces.`

Example 3	
Input	`[1, 2], 10`
Output	`0`
Why	`Total length is 3 but 10 pieces are needed: impossible, so 0.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(ribbons, k):
    for L in range(max(ribbons), 0, -1):
        pieces = sum(r // L for r in ribbons)
        if pieces >= k:
            return L
    return 0
```

BRUTE FORCE

TIME	SPACE
O(n * max)	O(1)

WHY IT IS SLOW

Tries up to max(ribbon) lengths, each costing O(n); with ribbons up to 10^9 that is a billion probes. The check is monotone, so binary search needs only log2(max) probes.

OPTIMAL SOLUTION · Binary search the answer (integer lengths)

TIME	SPACE
O(n log(max length))	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why does the running time depend on the value range and not just on the number of ribbons?

Each probe costs O(n), and the number of probes is log2(max length) for the integer version. A huge length like 10^9 needs about 30 probes while 10 needs 4, even with the same n. For real numbers the probe count is set by the precision you promise, e.g. log2(max / epsilon).

FOLLOW-UP 2 What if the candidate length is given to you?

Then it is just the feasibility check: sum(r // L for r in ribbons) >= k, in O(n). The binary search exists only to find the best L; the check is the same function you reuse inside it.

FOLLOW-UP 3 What is a good upper bound for the search?

min(max(ribbons), sum(ribbons) // k). If the total length divided by k is smaller than the longest ribbon, no piece longer than that can ever give k pieces, which shrinks the range a little and avoids wasted probes. **Cost:** each probe is O(n); the tighter bound removes only a few of the log2(max) probes, so the total stays O(n log max).

FOLLOW-UP 4 Give the search a tighter upper bound.

No piece can be longer than the longest ribbon, and K pieces of length L need K * L total length, so L <= sum // K. Starting hi at min(max(ribbons), sum // K) saves a few probes and also returns 0 immediately when sum < K.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Cutting Ribbons (LC 1891)
#
# You have ribbons of given lengths and an integer k. Each
# ribbon may be cut into pieces of one common integer
# length L (leftover bits are thrown away). Return the
# largest L that gives at least k pieces in total, or 0 if
# even length 1 cannot give k pieces.
#
# Signature:
#   def max_length(ribbons: list, k: int) -> int
#
```

```
# Examples:
# [9, 7, 5], 3  ->  5
# [5, 7], 12  ->  1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why does the running time depend on the value range
# and not just on the number of ribbons?
# 2. What if the candidate length is given to you?
# 3. What is a good upper bound for the search?
# 4. Give the search a tighter upper bound.
#
# INTERVIEW NOTES
# 'Largest length such that at least K pieces fit' is a
# maximisation with a yes/no check. Larger lengths yield
# fewer pieces, so the check is monotone and you can
# binary search the answer. The check itself is one pass:
# sum of ribbon // length.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def pieces(ribbons, length):
    # how many pieces of this length can we cut? (the check for a GIVEN length)
    total = 0
    for r in ribbons:
        total += r // length
    return total


def upper_bound(ribbons, k):
    # no piece is longer than the longest ribbon, and k pieces of length L
    # need k * L in total, so L <= sum // k
    longest = 0
    total = 0
    for r in ribbons:
        total += r
        if r > longest:
            longest = r
    if total // k < longest:
        return total // k
    return longest


# ---- main question: main function ----
# Idea: if k pieces of length L can be cut, k pieces of any SHORTER
# length can too, so "can we cut k pieces?" goes from yes to no as L
# grows: binary search the largest L that is still yes (whole numbers).
# The answer is 0 when it is impossible.
# Complexity: Time O(n * log(longest ribbon)), Space O(1)
def max_length(ribbons, k):
    low = 1
    high = upper_bound(ribbons, k)
    best = 0
    while low <= high:
        mid = (low + high) // 2
        if pieces(ribbons, mid) >= k:
```

```
            best = mid                    # mid works: remember it and try longer
            low = mid + 1
        else:
            high = mid - 1
    return best


# the same idea when lengths may be fractions: a fixed number of halvings
# Complexity: Time O(n * rounds), Space O(1)
def max_length_float(ribbons, k, rounds=50):
    low = 0.0
    high = float(max(ribbons))
    for i in range(rounds):
        mid = (low + high) / 2
        count = 0
```

```
        for r in ribbons:
            count += int(r // mid)
        if count >= k:
            low = mid                     # enough pieces: the answer is at least mid
        else:
            high = mid
    return low


# ---- main question: tests ----
def test_main_question():
    assert max_length([9, 7, 5], 3) == 5
    assert max_length([7, 5, 9], 4) == 4
    assert max_length([1, 2], 10) == 0                # not enough ribbon in total
    assert abs(max_length_float([1], 3) - 1 / 3) < 1e-9
    assert max_length_float([1], 2) > 0.49


# =======================================================
# FOLLOW-UP 1: Why does the running time depend on the value range and not just on the number of ribbons?
# =======================================================
# Answer: the binary search halves the range of LENGTHS, so it makes about
# log2(max length) checks, and each check costs O(number of ribbons).


# =======================================================
# FOLLOW-UP 2: What if the candidate length is given to you?
# =======================================================
# ---- follow-up 2: tests ----
# Answer: then no search is needed: the check pieces(ribbons, length)
# alone says whether k pieces can be cut.
def test_follow_up_2():
    assert pieces([9, 7, 5], 5) == 3


# =======================================================
# FOLLOW-UP 3: What is a good upper bound for the search?
# =======================================================


# =======================================================
# FOLLOW-UP 4: Give the search a tighter upper bound
# =======================================================
# ---- follow-up 4: tests ----
# Answer: the longest ribbon is a bound, and a better one is
```

```
# min(longest, sum // k): k pieces of length L need k * L in total.
# It saves a few probes and returns 0 at once when the sum is below k.
def test_follow_up_4():
```

```
    assert upper_bound([9, 7, 5], 3) == 7
    assert upper_bound([1, 2], 10) == 0


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement the integer version from memory and prove the monotonic feasibility predicate.
Rework it with floating-point lengths and explain termination under both fixed-iteration and epsilon-based policies.
Practice stating the bound and complexity in terms of the numeric value domain without conflating it with input size.

STUDY PATH: question 66 of 197 | topic 11 of 43: Binary search on a yes/no condition | 2 of 4

Template fit: x = the version number; ok(x) = is_bad(x) | * asked 127 days ago * | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 66 of 197

★ TOP 75: rank 72 in the Medium and Hard priority list

★ First Bad Version with Parallel Search Follow-up

ASKED 2026-06-15 | 127 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE **ALL TIME 41.1** **30 DAYS 33** **PRIORITY 54/288**

ABOUT THIS QUESTION

Topic: Binary search on a yes/no condition

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #278 First Bad Version

LeetCode match: LC 278 First Bad Version

Frequency: Single report | Last asked: 2026-06-15 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/first-bad-version-parallel-search

QUESTION

Versions are numbered 1 to n and from some version on every version is bad. The function is_bad(v) tells whether version v is bad. Find the first bad version with as few calls to is_bad as possible.

Function signature

def first_bad_version(n: int, is_bad) -> int

EXAMPLES

Example 1	
Input	`bad_from = 4` `first_bad_version(10, lambda v: v >= bad_from)`
Output	`first_bad_version(10, lambda v: v >= bad_from) -> 4`
Why	`Versions 4 to 10 are bad, so 4 is the first; binary search needs about log2(10) calls.`

Example 2	
Input	`bad_from = 1` `first_bad_version(1, lambda v: v >= bad_from)`
Output	`first_bad_version(1, lambda v: v >= bad_from) -> 1`
Why	`With a single version that is bad, the answer is 1.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n):
    for v in range(1, n + 1):
        if is_bad_version(v):
            return v
```

BRUTE FORCE

TIME	SPACE
O(n) calls	O(1)

WHY IT IS SLOW

One call per version, O(n) calls; with a billion versions and a slow check that is far too many. The check is monotone, so binary search needs only log2(n) calls, and probing several points in parallel needs fewer rounds.

OPTIMAL SOLUTION · Base problem: binary search

TIME	SPACE
O(log n) calls	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Parallel: probe k points at once, then recurse into the bucket that flips

TIME	SPACE
O(log_{k+1} n) rounds	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What does parallelism buy you, and what does it cost?

It cuts the number of sequential rounds from log2(n) to log_{k+1}(n), so wall-clock time drops when each probe has high latency. It increases total probes (about k per round instead of 1), so it costs more calls and money. Say which metric you are optimizing before choosing k.

FOLLOW-UP 2 What if the oracle is rate limited or can fail?

Cap the pool size to the rate limit, retry transient failures with backoff (probes are idempotent), and never advance the interval from a probe that errored. Cache results by version so a retry or a later round never repeats a call.

FOLLOW-UP 3 Count how many rounds each approach needs.

Single binary search: log2(n) rounds of one probe each. k workers: about log base (k + 1) of n rounds with k probes each. For n = 10^9, one probe per round needs 30 rounds, while 7 probes per round need 10.

FOLLOW-UP 4 Show how many rounds and calls the parallel search uses.

With k workers each round shrinks the interval by about k + 1, so the number of sequential rounds is about log base (k + 1) of n and the total calls at most k times that. For n = 10^6 and 7 workers that is about 7 rounds and 49 calls, against 20 rounds and 20 calls for the sequential search: fewer rounds, more calls.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: First Bad Version with Parallel Search
# Follow-up
#
# Versions are numbered 1 to n and from some version on
# every version is bad. The function is_bad(v) tells
# whether version v is bad. Find the first bad version
# with as few calls to is_bad as possible.
#
# Signature:
#   def first_bad_version(n: int, is_bad) -> int
#
# FOLLOW-UPS (answered below, in this order):
# 1. What does parallelism buy you, and what does it cost?
# 2. What if the oracle is rate limited or can fail?
# 3. Count how many rounds each approach needs.
# 4. Show how many rounds and calls the parallel search
# uses.
#
# INTERVIEW NOTES
# 'First bad version' is a boundary in a sorted yes/no
# sequence, which is binary search. The parallel part is a
# different trade: instead of one probe per round, send
# several probes in the same round and keep the gap that
# contains the boundary. Fewer rounds, more total calls.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from concurrent.futures import ThreadPoolExecutor
from threading import Lock


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: is_bad is monotone (all good versions, then all bad ones), so
# keep the invariant "the first bad version is in [lo, hi]" and halve the
# interval. Use lo + (hi - lo) // 2 in languages with fixed-size integers.
# Complexity: Time O(log n) calls, Space O(1)
def first_bad_version(n, is_bad):
```

```
    lo = 1
    hi = n
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if is_bad(mid):
            hi = mid                        # the first bad version is mid or earlier
        else:
            lo = mid + 1
    return lo


# ---- main question: tests ----
class Oracle:
```

```
    # a pretend is_bad that remembers how often it was called
    def __init__(self, first_bad):
        self.first_bad = first_bad
        self.calls = []
        self.lock = Lock()

    def __call__(self, version):
        with self.lock:
            self.calls.append(version)
        return version >= self.first_bad


CASES = ((1, 1), (10, 1), (10, 10), (1000, 637))


def test_main_question():
    for n, bad in CASES:
        assert first_bad_version(n, Oracle(bad)) == bad
    oracle = Oracle(637)
    first_bad_version(1000, oracle)
    assert len(oracle.calls) <= 10                    # about log2(1000) probes


# ============================================================
# FOLLOW-UP 1: What does parallelism buy you, and what does it cost?
# ============================================================
# Answer: it cuts the number of sequential ROUNDS from log2(n) to about
# log_(k+1)(n) with k workers, so wall-clock time drops when each probe is
# slow. It increases the total number of probes (about k per round instead
# of 1), so it costs more calls and money.
```

```
# ============================================================
# FOLLOW-UP 2: What if the oracle is rate limited or can fail?
# ============================================================
# Answer: cache every answer (never ask twice), retry a failed call a
# few times with a pause, and cap the number of parallel probes at the
# rate limit.


# ============================================================
# FOLLOW-UP 3: Count how many rounds each approach needs
# ============================================================


# ============================================================
# FOLLOW-UP 4: Show how many rounds and calls the parallel search uses
# ============================================================
# ---- follow-up 3: helper functions ----
def probe_points(lo, hi, workers):
    # versions spread evenly over [lo, hi - 1]
    step = (hi - lo + 1) // (workers + 1)
    if step < 1:
        step = 1
    points = set()
    for i in range(workers):
        p = lo + step * (i + 1) - 1
        if p > hi - 1:
            p = hi - 1
        points.add(p)
    return sorted(points)


# ---- follow-up 3: function ----
# Answer: one binary search needs log2(n) rounds of one probe each. With k
```

```
# workers each round shrinks the interval by about k + 1, so about
# log_(k+1)(n) rounds with k probes each. For n = 10^9: 30 rounds with one
# probe per round, but only 10 rounds with 7 probes per round. Total calls
# are at most k times the number of rounds.
# Complexity: Time O(log n / log(k + 1)) rounds with up to k calls each for k workers, Space O(k * rounds)
def first_bad_parallel(n, is_bad, workers=4):
    lo = 1
    hi = n                                  # invariant: the answer is in [lo, hi]
    answers = {}
    rounds = 0
    with ThreadPoolExecutor(workers) as pool:
        while lo < hi:
```

```
            points = probe_points(lo, hi, workers)
            fresh = []
            for p in points:
                if p not in answers:
                    fresh.append(p)
            results = list(pool.map(is_bad, fresh))    # ask about the new points at the same time
            for i in range(len(fresh)):
                answers[fresh[i]] = results[i]
            rounds += 1
            first_bad_point = None
            for p in points:
                if answers[p]:
                    first_bad_point = p
                    break
            if first_bad_point is None:
                lo = points[-1] + 1                    # every probe was good: the answer is after them
            else:
                for p in points:
                    if p < first_bad_point:
                        lo = p + 1                     # the last good probe before the first bad one
                hi = first_bad_point
    return lo, rounds, len(answers)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    for n, bad in CASES:
        answer, rounds, used = first_bad_parallel(n, Oracle(bad), workers=4)
        assert answer == bad
    answer, rounds, used = first_bad_parallel(10 ** 6, Oracle(123456), workers=7)
    assert answer == 123456 and rounds < 20           # fewer rounds than the 20 of a plain binary search
    assert used <= 7 * rounds


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write the single-threaded binary search first and verify it on small n.
Then implement the bucketed parallel version with real concurrency primitives (Python concurrent.futures.ThreadPoolExecutor, or Java ExecutorService / CompletableFuture): split [1, n] into k buckets, probe the k boundaries concurrently, pick the bucket where the predicate flips, and binary-search within it. Practice getting this to compile and run in under 20 minutes.

STUDY PATH: question 67 of 197 | topic 11 of 43: Binary search on a yes/no condition | 3 of 4

Template fit: x = the eating speed; ok(x) = total hours (ceil of pile / x summed) <= h; first_true over 1..max pile | * asked 631 days ago *

MEDIUM | PracHub | Technical Screen | 67 of 197

★ TOP 75: rank 12 in the Medium and Hard priority list

★ Solve Banana Speed and Interval Merge

POSTED 2025-01-27 | 631 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 65.2 | 30 DAYS 41.1 | PRIORITY 11/288

ABOUT THIS QUESTION

Topic: Binary search on a yes/no condition

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #875 Koko Eating Bananas, LeetCode #56 Merge Intervals

LeetCode match: LC 875 Koko Eating Bananas + LC 56 Merge Intervals

Posted on PracHub: 2025-01-27 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/solve-banana-speed-and-interval-merge

QUESTION

During two technical phone screens for a new graduate software engineering role, the candidate was asked to solve two coding problems: minimum eating speed and merging overlapping intervals.

Constraints & Assumptions

- Provide clear algorithms, correctness reasoning, complexity, and edge cases.
- For minimum eating speed, each hour can be spent on only one pile.
- For interval merging, intervals overlap when the next start is less than or equal to the current merged end.
- Use sorted or binary-search approaches where appropriate.

Clarifying Questions

to Ask - Are **`piles`** and **h** always positive?

- Is **h** at least the number of piles?
- Are interval endpoints inclusive?
- Can intervals be empty or unsorted?

Part 1

- Minimum Eating Speed Given **`piles`** and **h**, return the minimum integer speed **k** such that all piles can be eaten within **h** hours.

What This Part Should Cover

- Binary search on answer from 1 to **`max(piles)`**.
- Feasibility function using sum of **`ceil(pile / k)`**.
- Why feasibility is monotonic.
- Complexity and edge cases.

Part 2

- Merge Overlapping Intervals Given intervals **`[start, end]`**, merge all overlapping intervals and return non-overlapping intervals sorted by start.

What This Part Should Cover

QUESTION

- Sort by start.
- Maintain current merged interval.
- Extend on overlap, otherwise append and start a new interval.
- Complexity and edge cases.

What a Strong

Answer Covers - Recognizes monotonic binary search for eating speed.

- Uses integer-safe ceiling division.
- Correctly defines interval overlap.
- Gives clean code and complexity for both problems.

Follow-up Questions

- How would you avoid floating-point division in the banana problem?
- What if **h** is smaller than the number of piles?
- How do you merge touching intervals like **`[1,3]`** and **`[3,5]`**?
- How would you handle streaming intervals?Minimum Eating Speed (Koko Eating Bananas)Koko loves to eat bananas. There are **n** piles of bananas; the i-th pile has **`piles[i]`** bananas. The guards will return in **h** hours. Koko can decide her constant bananas-per-hour eating speed **k**. Each hour she chooses one pile and eats **k** bananas from it. If that pile has fewer than **k** bananas, she eats all of them and does not eat more during that hour. An hour is spent on only one pile. Return the minimum integer speed **k** such that she can finish all the bananas within **h** hours. Example:
- **`piles = [3, 6, 7, 11]`**, **`h = 8`** -> **`4`**
- **`piles = [30, 11, 23, 4, 20]`**, **`h = 5`** -> **`30`**
- **`piles = [30, 11, 23, 4, 20]`**, **`h = 6`** -> **`23`** Constraints: **`1 <= piles.length <= h`**, **`1 <= piles[i] <= 10^9`**.

Constraints

1 <= piles.length <= h1 <= piles[i] <= 10^9h can be as large as 10^9

EXAMPLES

```
Input: ([3, 6, 7, 11], 8)
Expected Output: 4
Explanation: At k=4: ceil(3/4)+ceil(6/4)+ceil(7/4)+ceil(11/4) = 1+2+2+3 = 8 hours. k=3 needs 1+2+3+4 =
    10 > 8, so 4 is minimal.
Input: ([30, 11, 23, 4, 20], 5)
Expected Output: 30
```

Explanation: With exactly 5 piles and 5 hours, each pile must be eaten in one hour, so k must be at least max(piles) = 30.

Hints

If speed k finishes in time, any larger speed also finishes in time. This monotonicity lets you binary search on the answer.The answer lies in [1, max(piles)]: speed 1 is the slowest, and max(piles) finishes every pile in at most one hour.Hours needed at speed k is sum of ceil(pile / k). Use integer-safe ceiling: (pile + k - 1) // k to avoid floating point.Merge Overlapping IntervalsGiven an array of intervals where **`intervals[i] = [start_i, end_i]`**, merge all overlapping intervals and return an array of the non-overlapping intervals that cover all the intervals in the input, sorted by start. Two intervals overlap when the next start is less than or equal to the current merged end

BRUTE FORCE · Bananas: try every speed upward

BRUTE FORCE · PSEUDO-CODE

```
def brute(piles, h):
    for speed in range(1, max(piles) + 1):
        hours = 0
        for p in piles:
            hours += ceil(p / speed)
        if hours <= h:
            return speed
```

TIME	SPACE
O(max * n)	O(1)

WHY IT IS SLOW

Up to max(piles) speeds, each costing O(n); piles can be 10^9. Hours are non-increasing in the speed, so binary search over speeds is O(n log max).

BRUTE FORCE · Intervals: merge any overlapping pair until none is left

BRUTE FORCE · PSEUDO-CODE

```
def merge(intervals):
    intervals = list(intervals)
    merged = True
    while merged:
        merged = False
        for a, b in combinations(intervals, 2):
            if overlaps(a, b):
                intervals.remove(a)
                intervals.remove(b)
                intervals.append(union(a, b))
                merged = True
                break
    return sorted(intervals)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each pass compares all pairs, O(n^2), and several passes may be needed. Sorting by start and sweeping once is O(n log n).

OPTIMAL SOLUTION · Bananas: binary search the speed

TIME	SPACE
O(n log max)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Intervals: sort and sweep

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Bananas: h varies per query, or the piles change.

The check is independent of the search; for many queries keep the piles sorted with prefix sums so the hours needed at speed s can be computed in O(log n) per distinct pile size.

FOLLOW-UP 2 Intervals: insert one new interval into an already merged list (LeetCode 57).

Copy intervals ending before the new one, merge the ones that overlap it, then copy the rest. O(n), no sort needed.

FOLLOW-UP 3 Bananas: the guards come back in fewer hours than piles.

Then no speed works (an hour can finish at most one pile): check h >= len(piles) first and return -1 otherwise.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Solve Banana Speed and Interval Merge
#
# During two technical phone screens for a new graduate
# software engineering role, the candidate was asked to
# solve two coding problems: minimum eating speed and
# merging overlapping intervals. ### Constraints &
# Assumptions - Provide clear algorithms, correctness
# reasoning, complexity, and edge cases. - For minimum
# eating speed, each hour can be spent on only one pile. -
# For interval merging, intervals overlap when the next
# start is less than or equal to the current merged end. -
# Use sorted or binary-search approaches where
# appropriate. ### Clarifying Questions to Ask - Are piles
# and h always positive? - Is h at least the number of
# piles? - Are interval endpoints inclusive? - Can
# intervals be empty or unsorted? ### Part 1 - Minimum
# Eating Speed Given piles and h, return the minimum
# integer speed k such that all piles can be eaten within
# h hours.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Bananas: h varies per query, or the piles change.
# 2. Intervals: insert one new interval into an already
# merged list (LeetCode 57).
# 3. Bananas: the guards come back in fewer hours than
# piles.
#
# INTERVIEW NOTES
# Bananas: the time needed is non-increasing in the speed,
# so the smallest speed that finishes in h hours can be
# found by binary search over speeds, with a simple sum of
# ceil(pile / speed) as the check. Intervals: sort by
# start and sweep with one 'current' interval; the sort
# guarantees nothing earlier can still overlap.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
```

```
# ========================================================
```

```
# ---- main question: helper functions ----
def hours_needed(piles, speed):
    # each hour she works on ONE pile, so every pile takes a rounded-up number of hours
    total = 0
    for p in piles:
        total += (p + speed - 1) // speed
    return total


# ---- main question: main functions ----
# PART 1 (bananas): the smallest eating speed that finishes all piles
# within h hours. Hours needed never go UP when the speed goes up, so
# binary search the speed between 1 and the biggest pile. -1 if h is
# smaller than the number of piles (each pile takes at least one hour).
# Complexity: Time O(n * log(biggest pile)), Space O(1)
def min_eating_speed(piles, h):
    if h < len(piles):
        return -1
    low = 1
    high = max(piles)
    while low < high:
        mid = (low + high) // 2
        if hours_needed(piles, mid) <= h:
            high = mid                    # fast enough: try slower
        else:
            low = mid + 1
    return low


# PART 2 (intervals): merge overlapping intervals. Sort by start; an
# interval that starts before (or when) the last merged one ends is
# absorbed by it. Sharing one point counts as overlapping.
# Complexity: Time O(n log n), Space O(n)
def merge_intervals(intervals):
    merged = []
    for start, end in sorted(intervals):
        if merged and start <= merged[-1][1]:
            if end > merged[-1][1]:
                merged[-1][1] = end
        else:
            merged.append([start, end])
    return merged


# ---- main question: tests ----
```

```
def test_main_question():
    assert min_eating_speed([3, 6, 7, 11], 8) == 4
    assert min_eating_speed([30, 11, 23, 4, 20], 5) == 30
    assert min_eating_speed([30, 11, 23, 4, 20], 6) == 23
    assert min_eating_speed([1, 1, 1], 2) == -1       # fewer hours than piles
    assert hours_needed([3, 6, 7, 11], 4) == 8
    assert merge_intervals([[1, 3], [2, 6], [8, 10], [15, 18]]) == [[1, 6], [8, 10], [15, 18]]
    assert merge_intervals([[1, 4], [4, 5]]) == [[1, 5]]
    assert merge_intervals([]) == []


# ========================================================
# FOLLOW-UP 1: Bananas: h varies per query, or the piles change
# ========================================================
# Answer: for many queries on the same piles, sort the piles and keep
# prefix sums so hours_needed costs O(log n) instead of O(n); when piles
# change, keep the structure updated (a balanced tree or Fenwick tree).
```

```
# ============================================================
# FOLLOW-UP 3: Bananas: the guards come back in fewer hours than piles
# ============================================================
# Answer: then no speed works (every pile needs at least one hour), which
# is why min_eating_speed returns -1 when h < number of piles.


# ============================================================
# FOLLOW-UP 2: Intervals: insert one new interval into an already merged list (LeetCode 57)
# ============================================================
# ---- follow-up 2: function ----
# Answer: copy the intervals that end before the new one starts, absorb
# the ones that overlap it, then copy the rest. O(n), no sort needed.
# Complexity: Time O(n), Space O(n)
def insert_interval(merged, new):
    result = []
    i = 0
    n = len(merged)
    while i < n and merged[i][1] < new[0]:            # 1) intervals completely before the new one
        result.append(merged[i])
        i += 1
    start, end = new[0], new[1]
    while i < n and merged[i][0] <= end:              # 2) intervals that overlap it are absorbed
        if merged[i][0] < start:
            start = merged[i][0]
        if merged[i][1] > end:
```

```
            end = merged[i][1]
        i += 1
    result.append([start, end])
    while i < n:                                      # 3) the rest comes after
        result.append(merged[i])
        i += 1
    return result


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert insert_interval([[1, 3], [6, 9]], [2, 5]) == [[1, 5], [6, 9]]
    assert insert_interval([], [4, 8]) == [[4, 8]]
    assert insert_interval([[1, 2], [5, 6]], [3, 4]) == [[1, 2], [3, 4], [5, 6]]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(piles: list[int], h: int) -> int:
```

Sample call

```
solution([3, 6, 7, 11], 8)
```

Test cases

Input	Expected
([3, 6, 7, 11], 8)	4
([30, 11, 23, 4, 20], 5)	30

STUDY PATH: question 68 of 197 | topic 11 of 43: Binary search on a yes/no condition | 4 of 4

Template fit: x = the elapsed time; ok(x) = sum(x // time_i) >= n requests | ND

MEDIUM | Exponent | 68 of 197

★ TOP 75: rank 24 in the Medium and Hard priority list

★ Given a list with response time for each server, calculate how long time it will take to process n requests.

ASKED (ANY CO.) ~A YEAR AGO | OLDER

STUDY: TYPE | ALL TIME 59.1 | 30 DAYS 46.9 | PRIORITY 21/288

ABOUT THIS QUESTION

Topic: Binary search on a yes/no condition

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #875 Koko Eating Bananas

LeetCode match: No direct match; related LC 875 Koko Eating Bananas (inferred from short statement)

Last asked (any company): a year ago, about 2025-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/5485/calculate-time-to-process-given-server-response-time

QUESTION

Servers work in parallel and server i needs times[i] time units to process one request (optionally server i only starts at starts[i]). Return the earliest time at which n requests have been processed in total.

Function signature

def time_for(times: list, n: int, starts=None) -> int

EXAMPLES

Example 1	
Input	`[1, 2], 3`
Output	`2`
Why	`By time 2 the first server has done 2 requests and the second one: 3 in total.`

Example 2	
Input	`[3], 4`
Output	`12`
Why	`One server needs 3 units per request: 4 requests take 12.`

Example 3	
Input	`[2, 1], 6, starts = [0, 5]`
Output	`8`
Why	`The second server only starts at time 5, so the first one does most of the work.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(times, n):
    heap = [(r, r) for r in times]  # (finish, r)
    heapify(heap)
    last = 0
    for _ in range(n):
        finish, r = heappop(heap)
        last = finish
        heappush(heap, (finish + r, r))
    return last
```

TIME	SPACE
O(n log S)	O(S)

WHY IT IS SLOW

One heap operation per request, O(n log S); with n up to 10^9 requests that is too slow. The finish time is a monotone threshold, so binary search on T with sum(T // response time) >= n is O(S log answer).

OPTIMAL SOLUTION · Binary search on the total time

TIME	SPACE
O(S log(min(times) * n))	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Servers may fail, or there is a startup time per server.

Add the startup time to the check: server i completes max(0, (T - start_i) // r_i) requests. The check stays monotone in T, so binary search still applies.

FOLLOW-UP 2 The inverse question: how many servers are needed to finish within a given time?

Divide: each server finishes time // r requests, so you need ceil(n / that). No search needed.

FOLLOW-UP 3 Requests have different sizes.

Then it is a scheduling problem (minimise the makespan), NP-hard in general; the greedy 'longest request first to the least-loaded server' is within 4/3 of optimal.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given a list with response time for each
# server, calculate how long time it will take to process
# n requests.
#
# Servers work in parallel and server i needs times[i]
# time units to process one request (optionally server i
# only starts at starts[i]). Return the earliest time at
# which n requests have been processed in total.
#
# Signature:
#   def time_for(times: list, n: int, starts=None) -> int
```

```
#
# Examples:
# [1, 2], 3  ->  2
# [3], 4  ->  12
#
# FOLLOW-UPS (answered below, in this order):
# 1. Servers may fail, or there is a startup time per
# server.
# 2. The inverse question: how many servers are needed to
# finish within a given time?
# 3. Requests have different sizes.
#
# INTERVIEW NOTES
# 'Smallest time T so that the servers can finish n
# requests' has a monotone yes/no check: if T is enough,
# any larger time is too. The check is a sum of T //
# response_time over the servers. Whenever the answer is a
# threshold on a monotone condition, binary search the
# threshold instead of simulating.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
```

```
def start_of(starts, i):
    # the startup delay of server i (0 when no delays are given)
    if starts:
        return starts[i]
    return 0


def done_by(times, t, starts=None):
    # how many requests do all servers finish within time t? Server i needs
    # times[i] per request and only begins after its startup delay (follow-up 1).
    total = 0
    for i in range(len(times)):
        working_time = t - start_of(starts, i)
        if working_time > 0:
            total += working_time // times[i]
    return total


# ---- main question: main function ----
# Idea: binary search on the TOTAL TIME. In time t, server i finishes
# t // times[i] requests, and done_by(t) never decreases as t grows, so the
# answer is the smallest t with done_by(t) >= n.
# starts[i] is an optional startup delay (follow-up 1).
# Complexity: Time O(s * log(range)) for s servers, Space O(1)
def time_for(times, n, starts=None):
    if n <= 0:
        return 0
    low = 0
    high = start_of(starts, 0) + times[0] * n      # the first server alone is always enough
    for i in range(len(times)):
```

```
        if start_of(starts, i) + times[i] * n < high:
            high = start_of(starts, i) + times[i] * n
    while low < high:
        mid = (low + high) // 2
        if done_by(times, mid, starts) >= n:
            high = mid
        else:
            low = mid + 1
    return low


# ---- main question: tests ----
def test_main_question():
    assert time_for([1, 2, 3], 6) == 4
    assert time_for([4], 3) == 12
```

```
    assert time_for([5, 5], 0) == 0
    assert time_for([2, 3], 1) == 2


# ========================================================
# FOLLOW-UP 1: Servers may fail, or there is a startup time per server
# ========================================================
# ---- follow-up 1: tests ----
# Answer: add the startup time to the check: server i finishes
# (t - start_i) // r_i requests (nothing before it starts). The check
# is still monotone in t, so the binary search still works. A failed
# server is the same as one that starts "never": drop it.
def test_follow_up_1():
    assert time_for([1, 5], 2, starts=[10, 0]) == 10   # server 0 starts at 10, so server 1 does the work


# ========================================================
# FOLLOW-UP 2: The inverse question: how many servers are needed to finish within a given time?
# ========================================================
# ---- follow-up 2: helper functions ----
def ceil_div(a, b):
    # a / b rounded up
    return (a + b - 1) // b


# ---- follow-up 2: function ----
# Answer: no search is needed: each server (response time r) finishes
# t // r requests, so divide and round up.
# Complexity: Time O(1), Space O(1)
def servers_needed(n, r, t):
    per_server = t // r
    if per_server == 0:
        return None                          # not even one request fits in time t
    return ceil_div(n, per_server)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert servers_needed(10, 4, 8) == 5
    assert servers_needed(10, 4, 3) is None
    assert servers_needed(1, 7, 7) == 1


# ========================================================
# FOLLOW-UP 3: Requests have different sizes
```

```
# ========================================================
# ---- follow-up 3: function ----
# Answer: now it is a scheduling problem (minimise the finish time of the
```

```
# busiest server), which is NP-hard in general. The greedy rule "give the
# biggest request to the least loaded server" (a heap of loads) is
# within 4/3 of the best possible.
# Complexity: Time O(n log n + n log s) for n requests and s servers, Space O(s)
def makespan(sizes, servers):
    loads = [0] * servers
    heapq.heapify(loads)
    for size in sorted(sizes, reverse=True):
        lightest = heapq.heappop(loads)
        heapq.heappush(loads, lightest + size)
    busiest = 0
    for load in loads:
        if load > busiest:
            busiest = load
    return busiest


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert makespan([5, 5, 4, 4, 3, 3, 3], 3) == 11   # the best split is 9, 9, 9; greedy gets 11 (11/9 < 4
/3)
    assert makespan([], 2) == 0


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 12 OF 43

Greedy: scan once, carry the best choice so far, act only when forced

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

5
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Greedy: scan once, carry the best choice so far, act only when forced | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M	69. Visible People in a Line (Last Person, Then Everyone)	1Point3Acres * 2026-10-07 *
M	70. Bounded Knapsack Variant	1Point3Acres * 2026-10-16 *
M	71. Find the Largest Monotone Increasing Number	PracHub * 2026-07-04 *
M	72. Min boards to cover roof holes	PracHub * 2026-02-12 *
M	73. Jump Game II (Minimum Jumps)	ColorStack * 2025-08-19 *

CODE TEMPLATE | GROUP 12 OF 43 | 5 QUESTIONS, ONE PATTERN

Greedy: scan once, carry the best choice so far, act only when forced

The one idea these questions share

Decide locally and never look back: sort or scan, carry one value (how far you can reach, how far you are covered, the largest seen so far), and take an action only at the moment the current choice stops being enough.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def min_cover(points, length):
    count, covered_to = 0, float("-inf")
    for p in sorted(set(points)):               # CHANGE 1: the order you scan in
        if p > covered_to:                      # CHANGE 2: not covered yet, so you must act
            count += 1
            covered_to = p + length - 1         # CHANGE 3: how far the new choice reaches
    return count
```

2. Helper functions that usually get written next to it

```
def circular_gap(day, today, cycle=365):        # how far ahead a date is, wrapping around the year
    return (day - today) % cycle


def farthest_reach(nums):
    reach = 0
    for i, x in enumerate(nums):
        if i > reach:
            return False
        reach = max(reach, i + x)
    return True
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
69	Visible People in a Line (Last Person, Then Everyone)	scan from the end carrying the tallest person seen between; count a person when no taller one blocks the view
70	Bounded Knapsack Variant	sort items by value, highest first; take min(available count, remaining capacity) of each
71	Find the Largest Monotone Increasing Number	scan the digits right to left; when a digit is smaller than the one before it, lower the earlier digit and fill 9s after it
72	Min boards to cover roof holes	the template as written
73	Jump Game II (Minimum Jumps)	scan the indices carrying the farthest reach; when you hit the end of the current jump, you must jump again: jumps += 1, end = farthest

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 69 of 197 | topic 12 of 43: Greedy: scan once, carry the best choice so far, act only when forced | 1 of 5

Template fit: scan from the end carrying the tallest person seen between; count a person when no taller one blocks the view | * asked 13 days ago * | frequency Single report

MEDIUM | 1Point3Acres | tech-screen | 69 of 197

Visible People in a Line (Last Person, Then Everyone)

ASKED 2026-10-07 | 13 DAYS AGO | LAST 30 DAYS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 211/288**

ABOUT THIS QUESTION

Topic: Greedy: scan once, carry the best choice so far, act only when forced

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #1944 Number of Visible People in a Queue

LeetCode match: LC 1944 Number of Visible People in a Queue (related)

Frequency: Single report | Last asked: 2026-10-07 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/visible-people-in-line

QUESTION

Given heights of people in a line, A can see B if nobody between them is taller than both. Part 1: count how many people the last person can see. Part 2: return that count for every person. Written in a plain editor with no compilation.

Input: an array of heights representing a line of people.
Visibility rule: person A can see person B if no one standing between them is taller than both A and B.
Part 1: return how many people the last person in line can see.
Part 2: for the same array, return, for every person in the line, how many people they can see.

EXAMPLES

```
[1, 10, 6, 7, 9, 8, 2, 4, 3, 5] -> 6 (the last person, height 5, can see 3, 4, 2, 8, 9, 10).
```

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def sees(a, i, j):
    lo, hi = min(i, j), max(i, j)
    for k in range(lo + 1, hi):
        if a[k] > max(a[i], a[j]):
            return False
    return True
def count_for(a, i):
    return sum(1 for j in range(len(a))
               if j != i and sees(a, i, j))
```

TIME	SPACE
O(n^3)	**O(1)**
for everyone, O(n^2) for the last person	

WHY IT IS SLOW

Each pair rescans the people between it, O(n^3) for everyone and O(n^2) for the last person. One pass with a running maximum answers the last person in O(n), and a monotonic stack answers everyone in O(n).

OPTIMAL SOLUTION · One pass with a running maximum (last person)

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Part 2: return the number of people each person can see.

Run the right-looking count for every position with a monotonic stack, run it again on the reversed line for the looking-left count, and add the two. The stack makes both passes O(n), instead of the O(n^2) you would get by repeating the last-person scan for every position.

FOLLOW-UP 2 How do ties and the direction of sight change the answer?

Under the stated rule only someone strictly taller than both blocks the view, so equal heights never block: in [5, 5, 5] the first person sees both others. If the interviewer wants 'taller or equal blocks', replace the non-strict comparisons with strict ones. If a person only looks towards the front, drop the reversed pass and use see_right on the reversed line alone.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Visible People in a Line (Last Person,
# Then Everyone)
#
# Given heights of people in a line, A can see B if nobody
# between them is taller than both. Part 1: count how many
# people the last person can see. Part 2: return that
# count for every person. Written in a plain editor with
# no compilation. Input: an array of heights representing
# a line of people. Visibility rule: person A can see
# person B if no one standing between them is taller than
# both A and B. Part 1: return how many people the last
# person in line can see. Part 2: for the same array,
# return, for every person in the line, how many people
# they can see. Examples: [1, 10, 6, 7, 9, 8, 2, 4, 3, 5]
# -> 6 (the last person, height 5, can see 3, 4, 2, 8, 9,
# 10).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Part 2: return the number of people each person can
# see.
# 2. How do ties and the direction of sight change the
# answer?
#
# INTERVIEW NOTES
# Person A sees person B when nobody strictly between them
# is taller than both, which means the tallest person
# between them is at most as tall as the taller of A and
# B. Walking away from the last person and keeping the
# running maximum of everyone passed lets you test each
# person in O(1). For everyone at once the visible people
# to the right form a chain: everything up to the first
# taller person, then only the people who are at least as
# tall as everyone before them. A monotonic stack builds
# those chains for all positions in O(n).

# ======================================================
```

```
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
def bigger(a, b):
    if a > b:
        return a
    return b


# ---- main question: main function ----
# Part 1: how many people can the LAST person see? Person k is seen by
# the last person when nobody standing strictly between them is taller
# than both of them. Walk from the second-last person to the front and
# keep the tallest height seen so far between us.
# Complexity: Time O(n), Space O(1)
def last_sees(a):
    if len(a) == 0:
        return 0
    last = a[-1]
    between = float("-inf")                     # the tallest person strictly between k and the last pers
on
    count = 0
    for k in range(len(a) - 2, -1, -1):
        if between <= bigger(last, a[k]):
            count += 1
        between = bigger(between, a[k])
    return count


# ---- main question: tests ----
LINE = [1, 10, 6, 7, 9, 8, 2, 4, 3, 5]


def test_main_question():
    assert last_sees(LINE) == 6
    assert last_sees([]) == 0 and last_sees([7]) == 0
    assert last_sees([5, 5, 5]) == 2                  # equal heights never block each other


# ============================================================
# FOLLOW-UP 1: Part 2: return the number of people each person can see
# ============================================================
# ---- follow-up 1: functions ----
# Answer: count, for every person, how many people they see looking
# right; do the same on the reversed line for looking left; add the two.
# Looking right from i: everyone up to the first STRICTLY taller person t
# is visible, and after t only the people who are at least as tall as
# everybody from t on (the chain of "next at least as tall" people).
```

```
# Monotonic stacks give "next at least as tall" and "next strictly
# taller" for every position in O(n).
# Complexity: Time O(n), Space O(n)
def see_right(a):
    n = len(a)
    next_ge = [None] * n                        # next person at least as tall
    next_gt = [None] * n                        # next person strictly taller
    stack = []
```

```
    for k in range(n - 1, -1, -1):
        while stack and a[stack[-1]] < a[k]:
            stack.pop()
        if stack:
            next_ge[k] = stack[-1]
        stack.append(k)
    stack = []
    for k in range(n - 1, -1, -1):
        while stack and a[stack[-1]] <= a[k]:
            stack.pop()
        if stack:
            next_gt[k] = stack[-1]
        stack.append(k)
    chain = [0] * n                                # how many "at least as tall" steps follow k
    for k in range(n - 1, -1, -1):
        if next_ge[k] is not None:
            chain[k] = 1 + chain[next_ge[k]]
    result = []
    for i in range(n):
        t = next_gt[i]
        if t is None:
            result.append(n - 1 - i)
        else:
            result.append(t - i + chain[t])
    return result


# Complexity: Time O(n), Space O(n)
def see_all(a):
    # looking left is looking right on the reversed line
    left = see_right(a[::-1])[::-1]
    right = see_right(a)
    total = []
    for i in range(len(a)):
        total.append(left[i] + right[i])
    return total
```

```
# ---- follow-up 1: tests ----
def can_see_slow(a, i, j):
    # i sees j when nobody strictly between them is taller than both
    low, high = i, j
    if i > j:
        low, high = j, i
    for k in range(low + 1, high):
        if a[k] > bigger(a[i], a[j]):
            return False
    return True


def test_follow_up_1():
    assert see_all(LINE)[-1] == 6
    assert see_all([5, 5, 5]) == [2, 2, 2]
    for a in ([3, 1, 2], [2, 2, 1, 3], [1, 2, 3, 4], [4, 3, 2, 1], LINE):     # compare with checking every
 pair
        for i in range(len(a)):
            count = 0
            for j in range(len(a)):
                if i != j and can_see_slow(a, i, j):
                    count += 1
            assert see_all(a)[i] == count


# ====================================================
# FOLLOW-UP 2: How do ties and the direction of sight change the answer?
```

```
# ============================================================
# Answer: with "strictly taller blocks" (used above) equal heights see
# each other; if equal heights also block, the stack pops on <= instead of
# <. Looking left is the same code on the reversed line; "who can see ME"
# is the same count seen from the other side.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write a brute-force Part 1 that checks every earlier person against the rule, then look for a single-pass version before attempting Part 2.
Extend the Part 1 solution to Part 2 and state its complexity before coding.
Hand-trace the example above without running code, including a tie case you define yourself.

STUDY PATH: question 70 of 197 | topic 12 of 43: Greedy: scan once, carry the best choice so far, act only when forced | 2 of 5

Template fit: sort items by value, highest first; take min(available count, remaining capacity) of each | * asked 4 days ago * | frequency Single report

MEDIUM | 1Point3Acres | tech-screen | 70 of 197

Bounded Knapsack Variant

ASKED 2026-10-16 | 4 DAYS AGO | LAST 30 DAYS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 210/288**

ABOUT THIS QUESTION

Topic: Greedy: scan once, carry the best choice so far, act only when forced

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1710 Maximum Units on a Truck

Frequency: Single report | Last asked: 2026-10-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/unit-weight-bounded-knapsack

QUESTION

Each item type has a value and an available count; every item weighs 1. Given capacity W, maximize total value. DP or DFS + memo works at O(n·W), greedy by value after sorting is O(n log n), and the best answer uses quickselect for O(n).

Input: for each item type, its value and the number of copies available. Every item has weight 1.
Input: a maximum total weight W.
Output: the maximum total value achievable without exceeding W.

EXAMPLES

Example 1	
Input	`[(10, 2), (6, 3), (3, 5)], 4`
Output	`32`
Why	`Items are (value, available count), each weighs 1. Take both 10s and two 6s: 20 + 12 = 32.`

Example 2	
Input	`[(10, 2), (6, 3), (3, 5)], 7`
Output	`44`
Why	`A bigger capacity takes more items.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def knapsack(items, W):
    best = [0] * (W + 1)
    for value, count in items:
        for _ in range(count):
            for w in range(W, 0, -1):
                take = best[w - 1] + value
                best[w] = max(best[w], take)
    return best[W]
```

BRUTE FORCE

TIME	SPACE
O(total copies * W)	O(W)

WHY IT IS SLOW

Every copy is its own pass over the table, O(total copies * W). With unit weights the most valuable items are always the right ones, so sorting by value and taking greedily is O(n log n).

OPTIMAL SOLUTION · Sort by value and take greedily

TIME	SPACE
O(n log n)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is greedy correct here but not for a general knapsack?

With unit weights each chosen item uses exactly one slot, so the only thing that matters is value per slot, which is just the value. If an optimal bag held a cheaper item while a dearer one was available, swapping them keeps the weight and raises the value, so the optimal bag can always be rearranged into the greedy one. With different weights a cheap item can leave room for several others, so value per weight and the leftover space interact and greedy fails.

FOLLOW-UP 2 What if the weights are not all 1?

Then it is a real bounded knapsack and greedy no longer works; use DP over the capacity. Splitting each count into 1, 2, 4, ... copies keeps the number of 0/1 items at about log(count) per type, so the cost is O(n * log(max count) * W) instead of O(total copies * W).

FOLLOW-UP 3 W is up to 10^9. Which version still works?

Only the greedy ones: sorting is O(n log n) and quickselect O(n), and neither depends on W, while any table over the capacity needs O(W) memory. The loop simply stops when the remaining capacity reaches zero.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Bounded Knapsack Variant
#
# Each item type has a value and an available count; every
# item weighs 1. Given capacity W, maximize total value.
# DP or DFS + memo works at O(n·W), greedy by value after
# sorting is O(n log n), and the best answer uses
# quickselect for O(n). Input: for each item type, its
# value and the number of copies available. Every item has
# weight 1. Input: a maximum total weight W. Output: the
# maximum total value achievable without exceeding W.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is greedy correct here but not for a general
# knapsack?
# 2. What if the weights are not all 1?
# 3. W is up to 10^9. Which version still works?
```

```
#
# INTERVIEW NOTES
# The word knapsack points at DP, but the statement says
# every item weighs 1. With equal weights every slot in
# the bag is interchangeable, so an item that is worth
# more is never worse than one that is worth less:
# swapping a cheaper item for a dearer one always helps.
# Take item types by value, highest first, until W slots
# are used. You do not even need the full sort:
# quickselect finds where the W-th slot falls in O(n).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: every item weighs 1, so every slot of the bag is worth the same
# and the dearest copies should go in first. items are (value, count)
# pairs and W is the number of slots. Sort by value, highest first; for
# each type take as many copies as still fit and add value * taken.
```

```
# Complexity: Time O(t log t) for t item types, Space O(t)
def max_value_sorted(items, W):
    total = 0
    for value, count in sorted(items, reverse=True):
        take = count
        if W < take:
            take = W                                # only W slots are left
        total += value * take
        W -= take
        if W == 0:
            break
    return total


# the same answer in O(n) expected time: quickselect finds where the W-th
# slot falls without sorting everything
# Complexity: Time O(t) expected (quickselect), Space O(t)
def max_value_select(items, W):
    total = 0
    pool = list(items)
    while pool and W > 0:
        pivot = random.choice(pool)[0]
        high = []
        middle = []
        low = []
        for value, count in pool:
            if value > pivot:
                high.append((value, count))
            elif value == pivot:
                middle.append((value, count))
            else:
                low.append((value, count))
        high_count = 0
        for value, count in high:
            high_count += count
```

```
        if high_count >= W:
            pool = high                         # the bag fills up inside the dearer types
            continue
        for value, count in high:
            total += value * count              # every dearer copy fits
        W -= high_count
        middle_count = 0
        for value, count in middle:
            middle_count += count
        take = middle_count
```

```
        if W < take:
            take = W
        total += pivot * take
        W -= take
        pool = low
    return total


# ---- main question: tests ----
def test_main_question():
    demo = [(3, 2), (5, 1), (4, 4)]
    assert max_value_sorted(demo, 4) == 17          # 5 + 4 + 4 + 4
    assert max_value_select(demo, 4) == 17
    assert max_value_sorted([], 5) == 0
    assert max_value_sorted([(7, 3)], 0) == 0
    assert max_value_sorted([(7, 3)], 10) == 21
    random.seed(9)
    for round_number in range(200):
        items = []
        for i in range(random.randint(0, 6)):
            items.append((random.randint(1, 9), random.randint(1, 4)))
        W = random.randint(0, 12)
        assert max_value_select(items, W) == max_value_sorted(items, W)


# ========================================================
# FOLLOW-UP 1: Why is greedy correct here but not for a general knapsack?
# ========================================================
# Answer: with unit weights all slots are equal, so swapping a cheaper
# copy for a dearer one never hurts (an exchange argument). With
# different weights a dear item can waste space that two smaller items
# would use better.


# ========================================================
# FOLLOW-UP 2: What if the weights are not all 1?
# ========================================================
# ---- follow-up 2: function ----
# Answer: it is a real bounded knapsack and greedy fails; use a DP over
# the capacity. Splitting each count into chunks of 1, 2, 4, ... copies
# keeps the number of items small. items are (value, weight, count).
# Complexity: Time O(n * log(count) * W), Space O(W)
def knapsack_bounded(items, W):
    best = [0] * (W + 1)                        # best[w] = best value with capacity w
    for value, weight, count in items:
```

```
        chunk = 1
        while count > 0:
            take = chunk
            if count < take:
                take = count
            count -= take
            cost = take * weight
            gain = take * value
```

```
            for w in range(W, cost - 1, -1):      # backwards so one chunk is used once
                if best[w - cost] + gain > best[w]:
                    best[w] = best[w - cost] + gain
            chunk *= 2
    return best[W]


# ---- follow-up 2: tests ----
def test_follow_up_2():
    mixed = [(10, 5, 1), (6, 3, 2)]
    assert knapsack_bounded(mixed, 6) == 12
    unit = [(3, 1, 2), (5, 1, 1), (4, 1, 4)]          # with every weight 1 it must agree with the greedy
    assert knapsack_bounded(unit, 4) == 17


# ======================================================
# FOLLOW-UP 3: W is up to 10^9. Which version still works?
# ======================================================
# Answer: the greedy (sort by value, take from the top) only depends on
# the number of item TYPES, not on W, so it still works. The DP needs an
# array of size W and is out of the question.


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write the sort-based greedy and argue its optimality in two sentences (unit weights mean every slot is interchangeable).
Implement quickselect that partitions item types by value and tracks the total count on each side to locate where W runs out.
Compare the O(n × W), O(n log n), and O(n) versions out loud with their complexities.

STUDY PATH: question 71 of 197 | topic 12 of 43: Greedy: scan once, carry the best choice so far, act only when forced | 3 of 5

Template fit: scan the digits right to left; when a digit is smaller than the one before it, lower the earlier digit and fill 9s after it | * asked 108 days ago *

MEDIUM | PracHub | Technical Screen | 71 of 197

Find the Largest Monotone Increasing Number

POSTED 2026-07-04 | 108 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 5** **30 DAYS 0** **PRIORITY 195/288**

ABOUT THIS QUESTION

Topic: Greedy: scan once, carry the best choice so far, act only when forced

Optimal answer from: the interview site's own solution

LeetCode match: LC 738 Monotone Increasing Digits (inferred from overview)

Posted on PracHub: 2026-07-04 (updated 2026-07-15)

Original: https://prachub.com/coding-questions/find-the-largest-monotone-increasing-number

QUESTION

Given an integer n, return the largest number not greater than n whose digits never decrease from left to right (for example 1223 qualifies, 1232 does not).

Function signature

def largest_monotone(n: int) -> int

EXAMPLES

Example 1	
Input	`10`
Output	`9`
Why	`10 itself has a drop (1 then 0); the largest valid number below it is 9.`

Example 2	
Input	`1234`
Output	`1234`
Why	`1234 is already monotone.`

Example 3	
Input	`332`
Output	`299`
Why	`332: lower the first 3 to 2 and fill the rest with 9s: 299.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n):
    while not is_non_decreasing(str(n)):
        n -= 1
    return n
```

BRUTE FORCE

TIME	SPACE
O(n * d)	O(1)

WHY IT IS SLOW

n can be 10^9, so it may take a billion steps. A right-to-left scan that lowers a digit where order breaks and fills the rest with 9s is O(digits).

OPTIMAL SOLUTION · Right-to-left scan, then fill with nines

TIME	SPACE
O(d)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The number is given as a string with 100,000 digits.

The scan works directly on the digit list, so keep it as a list of characters and never convert to an integer until the end (or not at all). The algorithm is O(d).

FOLLOW-UP 2 Return the smallest monotone number that is at least n instead.

Scan left to right for the first digit that is larger than its right neighbour and copy that digit over the whole suffix (332 becomes 333). The prefix before it is already monotone.

FOLLOW-UP 3 Digits must be strictly increasing.

Same idea, but the suffix to fill is the largest strictly increasing run (for example ...6789), and you must check that the prefix leaves enough room. Say the extra care and write the check as a brute-force comparison first.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find the Largest Monotone Increasing
# Number
#
# Given an integer n, return the largest number not
# greater than n whose digits never decrease from left to
# right (for example 1223 qualifies, 1232 does not).
#
# Signature:
#   def largest_monotone(n: int) -> int
#
# Examples:
# 10  ->  9
# 1234  ->  1234
#
# FOLLOW-UPS (answered below, in this order):
# 1. The number is given as a string with 100,000 digits.
# 2. Return the smallest monotone number that is at least
# n instead.
# 3. Digits must be strictly increasing.
#
# INTERVIEW NOTES
```

```
# If a digit is smaller than the one before it, the
# earlier digit has to drop by one and everything after
# that point can become 9 (as large as possible). Dropping
# a digit can break the pair before it, so scan from the
# right: each repair is checked against the digit to its
# left. That is why one right-to-left pass is enough.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: the largest number <= n whose digits never decrease. Scan the
# digits from RIGHT to LEFT. When a digit is smaller than the one before
# it (a drop), the left digit must go down by one, and every digit after
# it can become 9 (the largest choice). Remember where the nines start.
# Complexity: Time O(d) for d digits, Space O(d)
def largest_monotone(n):
    digits = list(str(n))
    nines_from = len(digits)
    for i in range(len(digits) - 1, 0, -1):
        if digits[i - 1] > digits[i]:
            digits[i - 1] = str(int(digits[i - 1]) - 1)
            nines_from = i
    for i in range(nines_from, len(digits)):
        digits[i] = "9"
    return int("".join(digits))


# ---- main question: tests ----
def is_monotone(n):
    text = str(n)
    for i in range(len(text) - 1):
        if text[i] > text[i + 1]:
            return False
    return True


def largest_monotone_slow(n):
    # count down from n until the digits never decrease (obviously right, slow)
    while not is_monotone(n):
        n -= 1
    return n


def test_main_question():
    assert largest_monotone(10) == 9
    assert largest_monotone(1234) == 1234
    assert largest_monotone(332) == 299
    assert largest_monotone(0) == 0
    for n in range(0, 3000):
        assert largest_monotone(n) == largest_monotone_slow(n)


# ============================================================
# FOLLOW-UP 1: The number is given as a string with 100,000 digits
# ============================================================
# ---- follow-up 1: function ----
```

```
# Answer: the scan only needs the digit characters, so keep the digits in
# a list and never convert to an integer: O(d). A leading zero (10 turns
# into 09) is dropped at the end.
# Complexity: Time O(d), Space O(d)
def largest_monotone_string(digits):
    d = list(digits)
    nines_from = len(d)
    for i in range(len(d) - 1, 0, -1):
        if d[i - 1] > d[i]:
            d[i - 1] = str(int(d[i - 1]) - 1)
            nines_from = i
    for i in range(nines_from, len(d)):
        d[i] = "9"
    start = 0
    while start < len(d) - 1 and d[start] == "0":
        start += 1
    return "".join(d[start:])


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert largest_monotone_string("332") == "299"
    assert largest_monotone_string("10") == "9"
    for n in range(0, 3000):
        assert largest_monotone_string(str(n)) == str(largest_monotone(n))
    big = "9" * 50000 + "1" * 50000                    # a 100,000 digit input is fine as a string
    assert largest_monotone_string(big)[:3] == "899"


# ============================================================
# FOLLOW-UP 2: Return the smallest monotone number that is at least n instead
# ============================================================
# ---- follow-up 2: function ----
# Answer: scan LEFT to right for the first digit that is bigger than its
# right neighbour, and copy that digit over the whole rest (332 becomes
# 333). The part before it is already in order.
# Complexity: Time O(d), Space O(d)
def smallest_monotone_at_least(n):
    d = list(str(n))
    for i in range(len(d) - 1):
        if d[i] > d[i + 1]:
            for j in range(i + 1, len(d)):
                d[j] = d[i]
            break
    return int("".join(d))


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert smallest_monotone_at_least(332) == 333
```

```
    assert smallest_monotone_at_least(120) == 122
    for n in range(0, 3000):
        up = n
        while not is_monotone(up):
            up += 1
        assert smallest_monotone_at_least(n) == up


# ============================================================
# FOLLOW-UP 3: Digits must be strictly increasing
# ============================================================
# Answer: the same right-to-left scan, but a drop is also an equal pair
# (digit[i - 1] >= digit[i]), and the filling after the cut is
```

```
# 9, 8, 7... going leftwards (the largest strictly increasing ending).


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Sample call

```
largest_monotone_increasing(0)
```

Test cases

Input	Expected
(0,)	0
(9,)	9

STUDY PATH: question 72 of 197 | topic 12 of 43: Greedy: scan once, carry the best choice so far, act only when forced | 4 of 5

Template fit: the template as written | * asked 250 days ago *

MEDIUM | PracHub | Technical Screen | 72 of 197

Min boards to cover roof holes

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 171/288

ABOUT THIS QUESTION

Topic: Greedy: scan once, carry the best choice so far, act only when forced

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #452 Minimum Number of Arrows to Burst Balloons

LeetCode match: No direct match; related LC 1326 Minimum Number of Taps / LC 452 (inferred from overview)

Posted on PracHub: 2026-02-12 (updated 2026-05-05)

Original: https://prachub.com/coding-questions/min-boards-to-cover-roof-holes

QUESTION

Holes in a roof are at integer positions along a line (the same position may be listed twice). A board of the given length covers that many consecutive positions. Return the fewest boards that cover every hole.

Function signature

def min_boards(holes: list, length: int) -> int

EXAMPLES

Example 1	
Input	`[1, 2, 4, 7, 8], 3`
Output	`3`
Why	`Boards cover [1..3], [4..6] and [7..9].`

Example 2	
Input	`[1, 10], 5`
Output	`2`
Why	`The holes are far apart: one board each.`

Example 3	
Input	`[5, 5, 5], 1`
Output	`1`
Why	`Repeated positions need only one board.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
best = [INF] * (len(holes) + 1)
best[0] = 0
for i in range(1, len(holes) + 1):
    for j in range(i):
        span = holes[i - 1] - holes[j] + 1
        if span <= L:  # one board covers j..i-1
            best[i] = min(best[i], best[j] + 1)
answer = best[len(holes)]
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every i tries every j, O(n^2). Starting a board at the leftmost uncovered hole and skipping everything it covers is optimal and O(n log n) including the sort.

OPTIMAL SOLUTION · Greedy: start a board at the leftmost uncovered hole

TIME	SPACE
O(n log n)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 You have only K boards: what is the smallest board length that covers all holes?

Binary search the length L and test with the greedy (is the number of boards at most K?); feasibility is monotone in L. O(n log range).

FOLLOW-UP 2 Boards of different lengths with a cost each.

The greedy exchange argument breaks. Use DP over the sorted holes: best[i] = min over board types of best[j] + cost, where j is the first hole not covered by that board starting at hole i.

FOLLOW-UP 3 Holes are in a grid and boards are 1 x L horizontally.

Apply the greedy independently on every row. If boards may also be vertical it becomes a covering problem on a bipartite graph (König's theorem).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Min boards to cover roof holes
#
# Holes in a roof are at integer positions along a line
# (the same position may be listed twice). A board of the
# given length covers that many consecutive positions.
# Return the fewest boards that cover every hole.
#
# Signature:
#   def min_boards(holes: list, length: int) -> int
```

```
#
# Examples:
# [1, 2, 4, 7, 8], 3  ->  3
# [1, 10], 5  ->  2
#
# FOLLOW-UPS (answered below, in this order):
# 1. You have only K boards: what is the smallest board
# length that covers all holes?
# 2. Boards of different lengths with a cost each.
# 3. Holes are in a grid and boards are 1 x L
# horizontally.
#
# INTERVIEW NOTES
# Cover the leftmost hole first: any board that covers it
# can be slid right until it starts exactly at that hole
# without uncovering anything to its right, so start a
# board at the leftmost uncovered hole and skip everything
# it covers. Repeating that is optimal because starting
# later never covers more.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from bisect import bisect_left


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea (greedy): sort the holes. The leftmost hole that is not covered
# yet must start a new board, and the best place for it is exactly at
```

```
# that hole (starting earlier covers nothing more). That board covers
# every hole below start + length. Count the boards.
# Complexity: Time O(n log n) (the sort), Space O(n)
def min_boards(holes, length):
    count = 0
    covered_to = float("-inf")               # the last position the boards so far cover
    for position in sorted(set(holes)):
        if position > covered_to:
            count += 1
            covered_to = position + length - 1
    return count


# ---- main question: tests ----
def test_main_question():
    assert min_boards([1, 2, 4, 5, 9], 3) == 3
    assert min_boards([], 3) == 0
    assert min_boards([5, 5, 5], 1) == 1


# ============================================================
# FOLLOW-UP 1: You have only K boards: what is the smallest board length that covers all holes?
# ============================================================
# ---- follow-up 1: function ----
# Answer: binary search the board length L. For a given L the greedy tells
# how many boards are needed; "at most K boards" gets easier as L grows,
# so binary search works: O(n log range).
# Complexity: Time O(n log n * log(range)), Space O(n)
```

```
def smallest_length(holes, k):
    points = sorted(set(holes))
    if not points:
        return 0
    low = 1
    high = points[-1] - points[0] + 1         # one board that covers everything
    while low < high:
        mid = (low + high) // 2
        if min_boards(points, mid) <= k:
            high = mid                        # long enough: try shorter
        else:
            low = mid + 1
    return low


# ---- follow-up 1: tests ----
def test_follow_up_1():
```

```
    assert smallest_length([1, 2, 4, 5, 9], 2) == 5       # boards cover 1..5 and 9..13
    assert smallest_length([1, 2, 4, 5, 9], 1) == 9
    assert smallest_length([], 1) == 0


# ======================================================
# FOLLOW-UP 2: Boards of different lengths with a cost each
# ======================================================
# ---- follow-up 2: function ----
# Answer: the greedy argument breaks. Use a DP over the sorted holes:
# best[i] = the cheapest way to cover the holes from i onward = the
# minimum over the board types of (cost + best[j]), where j is the first
# hole that a board of that type placed at hole i does NOT reach.
# Complexity: Time O(n log n + n * b * log n) for b board types, Space O(n)
def min_cost(holes, boards):
    # boards is a list of (length, cost); you may use any number of each
    points = sorted(set(holes))
    best = [0] * (len(points) + 1)
    for i in range(len(points) - 1, -1, -1):
        best[i] = float("inf")
        for length, cost in boards:
            j = bisect_left(points, points[i] + length)
            if cost + best[j] < best[i]:
                best[i] = cost + best[j]
    return best[0]


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert min_cost([1, 2, 3, 10], [(3, 5), (1, 3)]) == 8     # a long board costs 5, a short one 3
    assert min_cost([1, 2, 3, 4], [(2, 3), (4, 5)]) == 5


# ======================================================
# FOLLOW-UP 3: Holes are in a grid and boards are 1 x L horizontally
# ======================================================
# ---- follow-up 3: function ----
# Answer: boards cannot cross rows, so run the same greedy on every row
# and add the counts. (If boards may also be vertical it becomes a
# covering problem on a bipartite graph: Konig's theorem.)
# Complexity: Time O(cells), Space O(row length)
def boards_per_row(grid, length):
    total = 0
    for row in grid:
        holes = []
```

```
        for c in range(len(row)):
            if row[c] == "#":
                holes.append(c)
        total += min_boards(holes, length)
    return total


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert boards_per_row(["#.#.", "..##", "...."], 2) == 3


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(roof: list[list[int]]) -> int:
SAMPLE INPUT
[[1,0,1],[0,1,1]]
```

Test cases

Input	Expected
[[1,0,1],[0,1,1]]	2
[[1,1],[1,1]]	0

STUDY PATH: question 73 of 197 | topic 12 of 43: Greedy: scan once, carry the best choice so far, act only when forced | 5 of 5

Template fit: scan the indices carrying the farthest reach; when you hit the end of the current jump, you must jump again: jumps += 1, end = farthest | * asked 427 days ago * | frequency Single report

MEDIUM | IN APP LC 45 | ColorStack | technical interview | 73 of 197

Jump Game II (Minimum Jumps)

ASKED 2025-08-19 | 427 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 215/288

ABOUT THIS QUESTION

Topic: Greedy: scan once, carry the best choice so far, act only when forced

Optimal answer from: your app's interview approach (LeetCode #45)

LeetCode match: LC 45 Jump Game II

Frequency: Single report | Last asked: 2025-08-19 | Roles: SWE Intern

QUESTION

You are given a 0-indexed array of non-negative integers nums . You start at index 0. From any index i , you can jump to any index in the range [i + 1, i + nums[i]] (i.e., nums[i] is the maximum forward jump length from i). Return the minimum number of jumps required to reach the last index len(nums)

- 1 . The test cases guarantee that the last index is always reachable.

Function signature

```
def jump(nums: list[int]) -> int:
    """Return the minimum number of jumps to reach the last index."""
    ...
```

Follow-up (as reported): Solve it in O(n) time and O(1) extra space with a greedy "current reach vs. farthest reach" scan rather than O(n) DP.

EXAMPLES

Example 1	
Input	`nums = [2, 3, 1, 1, 4]`
Output	`2`
Why	`Jump 1 step from index 0 to 1, then 3 steps to the last index.`

Example 2	
Input	`nums = [2, 3, 0, 1, 4]`
Output	`2`
Why	`0 -> 1 -> 4.`

Example 3	
Input	`nums = [0]`
Output	`0`
Why	`Already at the last index.`

CONSTRAINTS

- 1 <= len(nums) <= 10^4
- 0 <= nums[i] <= 1000

QUESTION

- If the array has length 1 you are already at the last index, so the answer is 0.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_jump(nums):
    n = len(nums)
    best = [float("inf")] * n
    best[0] = 0
    for i in range(1,_n):
        for j in range(i):
            if j + nums[j] >= i:
                best[i] = min(
                    best[i], best[j] + 1
                )
    return best[-1]
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every index looks back at every earlier index.

OPTIMAL SOLUTION · Greedy by ranges

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the actual indexes you jump through, not just the count.

Remember for each range the index that reached farthest. When the range ends, append that index to the path. The path still costs O(n) time.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Jump Game II (Minimum Jumps)
#
# You are given a 0-indexed array of non-negative integers
# nums . You start at index 0. From any index i , you can
# jump to any index in the range [i + 1, i + nums[i]]
# (i.e., nums[i] is the maximum forward jump length from i
# ). Return the minimum number of jumps required to reach
# the last index len(nums) - 1 . The test cases guarantee
# that the last index is always reachable. Function
# signature: def jump(nums: list[int]) -> int: """Return
# the minimum number of jumps to reach the last index."""
# ... Follow-up (as reported): Solve it in O(n) time and
# O(1) extra space with a greedy "current reach vs.
# farthest reach" scan rather than O(n) DP.
#
```

```
# FOLLOW-UPS (answered below, in this order):
# 1. Return the actual indexes you jump through, not just
# the count.
#
# INTERVIEW NOTES
# Each jump lands in a range of reachable indexes, so this
# is a BFS over ranges: the first jump covers [1,
# nums[0]], the second jump covers everything reachable
# from that range, and so on. Greedy tracks the end of the
# current range and the farthest index seen; reaching the
# range end forces one more jump.

# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

import random


# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: main function ----
# Idea (greedy by ranges): each jump lets you reach a whole range of
# indexes. Walk through the array and remember the farthest index you
# can reach from anything seen so far. When you pass the end of the
# range of the current jump, you must take another jump, and the new
```

```
# range ends at that farthest index.
# Complexity: Time O(n), Space O(1)
def jump(nums):
    jumps = 0
    current_end = 0                      # the last index reachable with `jumps` jumps
    farthest = 0
    for i in range(len(nums) - 1):
        if i + nums[i] > farthest:
            farthest = i + nums[i]
        if i == current_end:             # the range of this jump is used up: take another
            jumps += 1
            current_end = farthest
    return jumps


# ---- main question: tests ----
def test_main_question():
    assert jump([2, 3, 1, 1, 4]) == 2
    assert jump([2, 3, 0, 1, 4]) == 2
    assert jump([0]) == 0
    assert jump([1, 1, 1, 1]) == 3


# ======================================================
# FOLLOW-UP 1: Return the actual indexes you jump through, not just the count
# ======================================================
# ---- follow-up 1: function ----
# Answer: remember, for the current range, the index that reaches
# farthest. When the range ends, that index is where the NEXT jump lands.
# Add it to the path (the first range, which is just index 0, adds
# nothing). The last landing is always the last index. Still O(n).
# Complexity: Time O(n), Space O(number of jumps)
def jump_path(nums):
    last = len(nums) - 1
```

```
    path = [0]
    if last == 0:
        return path
    current_end = 0
    farthest = 0
    best_index = 0                          # the index that reaches farthest so far
    for i in range(last):
        if i + nums[i] > farthest:
            farthest = i + nums[i]
            best_index = i
        if i == current_end:
            if i > 0:
                path.append(best_index)
            current_end = farthest
    path.append(last)
    return path


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert jump_path([2, 3, 1, 1, 4]) == [0, 1, 4]
    assert jump_path([0]) == [0]
    assert jump_path([3, 1, 1, 1]) == [0, 3]
    random.seed(5)
    for round_number in range(300):               # every jump is legal and the length matches jump()
        nums = []
        for i in range(random.randint(1, 10)):
            nums.append(random.randint(1, 4))
        path = jump_path(nums)
        assert path[0] == 0 and path[-1] == len(nums) - 1
        for k in range(len(path) - 1):
            assert 0 < path[k + 1] - path[k] <= nums[path[k]]
        assert len(path) - 1 == jump(nums)


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 13 OF 43

Sets and counters: remember the first collection, check the second against it

3 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Sets and counters: remember the first collection, check the second against it | 3 questions | tap a title to jump to it

READ FIRST: the code template for these 3 questions (tap here) >

M 74. Reconstruct original array from doubled shuffle — **PracHub * 2026-01-04 ***

M 75. Min deletions to avoid overlap in first k — **PracHub * 2026-02-12 ***

M 76. Filter Common Elements Per Window (Two Lists) — **ColorStack * 2025-10-08 ***

CODE TEMPLATE | GROUP 13 OF 43 | 3 QUESTIONS, ONE PATTERN

Sets and counters: remember the first collection, check the second against it

The one idea these questions share

Build a set (or a Counter) from one collection, then walk the other and ask 'is this in it?'. Membership in a set is O(1), which is the whole trick. Duplicates, intersections, filters, and missing values are the same loop with a different question.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import Counter


def keep_not_in(first, second):
    seen = set(first)                          # CHANGE 1: a set, or a Counter if counts matter
    return [x for x in second if x not in seen]     # CHANGE 2: keep, count, or stop at the first hit
```

2. Helper functions that usually get written next to it

```
def common_counts(a, b):                   # how many of each value the two lists share
    ca, cb = Counter(a), Counter(b)
    return {v: min(ca[v], cb[v]) for v in ca if v in cb}


def first_repeat(items):
    seen = set()
    for x in items:
        if x in seen:
            return x
        seen.add(x)
    return None
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
74	Reconstruct original array from doubled shuffle	Counter of the values; go from the smallest, pairing each x with 2x and using both counts up
75	Min deletions to avoid overlap in first k	Counter of the left part and of the right part; for each value in both, delete min(left count, right count)
76	Filter Common Elements Per Window (Two Lists)	build a set per window of the first list; keep the matching window of the second list minus that set

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 74 of 197 | topic 13 of 43: Sets and counters: remember the first collection, check the second against it | 1 of 3

Template fit: Counter of the values; go from the smallest, pairing each x with 2x and using both counts up | * asked 289 days ago *

MEDIUM | PracHub | Technical Screen | 74 of 197

Reconstruct original array from doubled shuffle

POSTED 2026-01-04 | 289 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 29.8 | 30 DAYS 0 | PRIORITY 147/288

ABOUT THIS QUESTION

Topic: Sets and counters: remember the first collection, check the second against it

Optimal answer from: the interview site's own solution

LeetCode match: LC 2007 Find Original Array From Doubled Array (inferred from overview)

Posted on PracHub: 2026-01-04 (updated 2026-05-22)

Original: https://prachub.com/coding-questions/reconstruct-original-array-from-doubled-shuffle

QUESTION

An array of numbers was doubled: for every element x of the original, the value 2x was added, and then all the numbers were shuffled. Given the shuffled array changed, return an original array (in any order), or an empty list if changed cannot be a doubled array. Zero and negative numbers are possible.

Function signature

def find_original(changed: list) -> list

EXAMPLES

Example 1	
Input	`[1, 3, 4, 2, 6, 8]`
Output	`[1, 3, 4]`
Why	`Original [1, 3, 4] gives doubles [2, 6, 8].`

Example 2	
Input	`[1, 3, 2]`
Output	`[]`
Why	`An odd number of values cannot be doubled.`

Example 3	
Input	`[0, 0, 0, 0]`
Output	`[0, 0]`
Why	`Zeros pair with zeros.`

Example 4	
Input	`[-2, -1, -4, -2]`
Output	`[-1, -2]`
Why	`Negatives work too: -2 is the double of -1.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(changed):
    items = sorted(changed, key=abs)
    originals = []
    while items:
        x = items.pop(0)
        if 2 * x not in items:
            return []
        items.remove(2 * x)
        originals.append(x)
    return originals
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

list.remove is a linear search, so each step is O(n) and the loop is O(n^2). A Counter processed in order of absolute value is O(n log n).

OPTIMAL SOLUTION · Counter processed in order of absolute value

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the originals in the order they appear in the input.

Compute the multiset of originals, then scan the input once and keep each element while its remaining original count is positive.

FOLLOW-UP 2 Values are small non-negative integers.

Use a counting array instead of sorting and a Counter: process values from small to large, O(n + range).

FOLLOW-UP 3 Why does processing by absolute value work with negatives?

For negatives the double has a larger absolute value, so the element with the smallest absolute value is always the original of its pair, exactly as with positives.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Reconstruct original array from doubled
# shuffle
#
# An array of numbers was doubled: for every element x of
# the original, the value 2x was added, and then all the
# numbers were shuffled. Given the shuffled array changed,
# return an original array (in any order), or an empty
# list if changed cannot be a doubled array. Zero and
```

```
# negative numbers are possible.
#
# Signature:
#   def find_original(changed: list) -> list
#
# Examples:
# [1, 3, 4, 2, 6, 8]  ->  [1, 3, 4]
# [1, 3, 2]  ->  []
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the originals in the order they appear in the
# input.
# 2. Values are small non-negative integers.
# 3. Why does processing by absolute value work with
# negatives?
#
# INTERVIEW NOTES
# The element with the smallest absolute value cannot be
# the double of anything, so it is an original; its double
# must be present too. Pair them off, take the next
# smallest, and so on. Process values in order of absolute
# value so negatives and zeros follow the same rule, and
# handle zero separately (it is its own double, so it
# needs an even count).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def count_values(values):
    counts = {}
    for v in values:
        counts[v] = counts.get(v, 0) + 1
    return counts


def size_then_value(v):
    # the sort key: smaller absolute value first (a negative number sorts by
    # its size, so -1 comes before 2)
    return (abs(v), v)


# ---- main question: main function ----
# `changed` holds every original x and also its double 2x, shuffled. Find
# the originals (or return [] if `changed` is not such an array).
# Idea: count the values and go through them from the SMALLEST absolute
# value: the smallest remaining value can only be an original, never someone's
# double. Every copy of x needs its own copy of 2x: if there are fewer copies of
# 2x, fail; otherwise use them up and write x down that many times. Zero is its
# own double, so zeros pair with each other. (Going by absolute value also makes
# negatives work: -2 is the double of -1, and |-1| < |-2|: follow-up 3.)
# Complexity: Time O(n log n) (the sort of distinct values), Space O(n)
def find_original(changed):
    if len(changed) % 2 == 1:
        return []
    count = count_values(changed)
    originals = []
```

```
    for x in sorted(count, key=size_then_value):
        if count[x] == 0:
            continue                                    # all copies of x were used as doubles
        if x == 0:
            if count[0] % 2 == 1:
                return []
            for i in range(count[0] // 2):
                originals.append(0)
            count[0] = 0
            continue
        if count.get(2 * x, 0) < count[x]:
            return []                                   # not enough doubles for every x
        for i in range(count[x]):
            originals.append(x)
        count[2 * x] -= count[x]
        count[x] = 0
    return originals
```

```
# ---- main question: tests ----
def test_main_question():
    assert sorted(find_original([1, 3, 4, 2, 6, 8])) == [1, 3, 4]
    assert find_original([6, 3, 0, 1]) == []
    assert find_original([1]) == []
    assert find_original([0, 0, 0, 0]) == [0, 0]
    assert find_original([0, 0, 0]) == []
    assert sorted(find_original([-1, -2, 4, 2])) == [-1, 2]       # -2 is the double of -1


# ========================================================
# FOLLOW-UP 1: Return the originals in the order they appear in the input
# ========================================================
# ---- follow-up 1: function ----
# Answer: first compute the multiset of originals, then scan the input once
# and keep each element while its remaining original count is positive.
# Complexity: Time O(n log n), Space O(n)
def find_original_in_order(changed):
    originals = find_original(changed)
    if not originals and changed:
        return []
    left = count_values(originals)
    result = []
    for x in changed:
        if left.get(x, 0) > 0:
            left[x] -= 1
            result.append(x)
    return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert find_original_in_order([4, 1, 2, 8, 2, 4]) == [4, 1, 2]


# ========================================================
# FOLLOW-UP 2: Values are small non-negative integers
# ========================================================
# ---- follow-up 2: function ----
# Answer: use a counting ARRAY instead of sorting and a dictionary: go
# through the values from small to large, O(n + range).
# Complexity: Time O(n + limit), Space O(limit)
def find_original_small(changed, limit):
    if len(changed) % 2 == 1:
```

```
        return []
    count = [0] * (2 * limit + 2)
    for x in changed:
        count[x] += 1
    if count[0] % 2 == 1:
        return []
    originals = []
    for i in range(count[0] // 2):
        originals.append(0)
    for x in range(1, limit + 1):
        if count[x] > 0:
            if count[2 * x] < count[x]:
                return []
            for i in range(count[x]):
                originals.append(x)
            count[2 * x] -= count[x]
    return originals


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert sorted(find_original_small([1, 2, 4, 8], 8)) == [1, 4]
    assert find_original_small([1, 3], 3) == []


# ========================================================
# FOLLOW-UP 3: Why does processing by absolute value work with negatives?
# ========================================================
# Answer: for a negative x the double 2x is further from zero, so the value
# with the smallest absolute value is always an original; going by absolute
# value handles negatives exactly like positives.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(changed: list[int]) -> list[int]:
```

Sample call

```
solution([2, 1, 4, 8])
```

Test cases

Input	Expected
`([2, 1, 4, 8],)`	`[1, 4]`
`([-4, -2, -2, -1, 1, 2],)`	`[-2, -1, 1]`

STUDY PATH: question 75 of 197 | topic 13 of 43: Sets and counters: remember the first collection, check the second against it | 2 of 3

Template fit: Counter of the left part and of the right part; for each value in both, delete min(left count, right count) | * asked 250 days ago *

MEDIUM | PracHub | Technical Screen | 75 of 197

Min deletions to avoid overlap in first k

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 255/288

ABOUT THIS QUESTION

Topic: Sets and counters: remember the first collection, check the second against it

Optimal answer from: the interview site's own solution

LeetCode match: No direct match (inferred) (inferred from overview)

Posted on PracHub: 2026-02-12 (updated 2026-04-22)

Original: https://prachub.com/coding-questions/min-deletions-to-avoid-overlap-in-first-k

QUESTION

Split an array into its first k elements and the rest. Return the minimum number of elements to delete so that no value appears on both sides (for every shared value delete all copies from the side that has fewer).

Function signature

def min_deletions(a: list, k: int) -> int

EXAMPLES

Example 1	
Input	`[1, 2, 2, 3, 2, 1], 3`
Output	`2`
Why	`Left [1, 2, 2], right [3, 2, 1]: value 1 appears once on each side (delete 1), value 2 appears twice on the left and once on the right (delete 1): 2 deletions.`

Example 2	
Input	`[1, 2, 3, 4], 2`
Output	`0`
Why	`No value is shared between [1, 2] and [3, 4].`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a, k):
    for size in range(len(a) + 1):
        ids = range(len(a))
        for gone in combinations(ids, size):
            keep = [i for i in ids
                    if i not in gone]
            head = {a[i] for i in keep if i < k}
            tail = {a[i] for i in keep if i >= k}
            if not head & tail:
                return size
```

BRUTE FORCE

TIME	SPACE
O(2^n * n)	O(n)

WHY IT IS SLOW

2^n sets to try: exponential. Each shared value must disappear from one side, so the cost is the sum over shared values of the smaller of its two counts, in O(n).

OPTIMAL SOLUTION · Sum of min(count in the first k, count in the rest) over shared values

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only elements of the first part may be deleted.

Then every shared value must be removed from the first part entirely, so the cost is the number of first-part elements whose value occurs in the rest (a set lookup per element).

FOLLOW-UP 2 Elements have weights (deleting some is more costly).

Replace counts by the sum of weights per side for each value; the choice per value is still independent, so take the cheaper side. **Cost:** unchanged O(n); the per-value choice sums weights instead of counts.

FOLLOW-UP 3 The statement may differ from this reading.

Ask whether deletions can come from both parts, whether duplicates count separately, and whether order matters. Everything above relies on those three answers.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Min deletions to avoid overlap in first k
#
# Split an array into its first k elements and the rest.
# Return the minimum number of elements to delete so that
# no value appears on both sides (for every shared value
# delete all copies from the side that has fewer).
#
# Signature:
#   def min_deletions(a: list, k: int) -> int
#
# Examples:
# [1, 2, 2, 3, 2, 1], 3  ->  2
# [1, 2, 3, 4], 2  ->  0
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only elements of the first part may be deleted.
# 2. Elements have weights (deleting some is more costly).
# 3. The statement may differ from this reading.
#
# INTERVIEW NOTES
# Values do not interact with each other, so each value
```

```
# can be decided on its own. A value that appears on both
# sides must disappear from one side entirely, and the
# cheapest side is the one with fewer copies. The answer
# is the sum, over values that appear on both sides, of
# the smaller of the two counts.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def count_values(values):
    counts = {}
    for v in values:
        counts[v] = counts.get(v, 0) + 1
    return counts
```

```
# ---- main question: main function ----
# The values in a[:k] and in a[k:] must not share any value. Delete as few
# elements as possible.
# Idea: count every value in the first part and in the rest. For a value
# that occurs on BOTH sides, one side has to lose ALL of its copies, so delete
# the copies on the side where there are fewer. Add that up over the shared values.
# Complexity: Time O(n), Space O(n)
def min_deletions(a, k):
    left = count_values(a[:k])
    right = count_values(a[k:])
    total = 0
    for v in left:
        if v in right:
            if left[v] < right[v]:
                total += left[v]
            else:
                total += right[v]
    return total


# which indices to delete (either side may lose elements)
# Complexity: Time O(n log n), Space O(n)
def deletions_plan(a, k):
    left = count_values(a[:k])
    right = count_values(a[k:])
    indices = []
    for v in left:
        if v in right:
            if left[v] <= right[v]:
                side = range(k)                         # the first part has fewer copies: delete those
            else:
                side = range(k, len(a))
            for i in side:
                if a[i] == v:
                    indices.append(i)
    indices.sort()
    return indices


# ---- main question: tests ----
def test_main_question():
```

```
    assert min_deletions([1, 2, 2, 3, 2, 1], 3) == 2          # first part [1, 2, 2], rest [3, 2, 1]: 1 an
d 2 are shared
    assert min_deletions([1, 2, 3], 1) == 0
    assert min_deletions([], 0) == 0
    a = [1, 2, 2, 3, 2, 1]
```

```
    plan = deletions_plan(a, 3)
    assert len(plan) == min_deletions(a, 3)
    first = set()
    rest = set()
    for i in range(len(a)):
        if i not in plan:
            if i < 3:
                first.add(a[i])
            else:
                rest.add(a[i])
    assert not (first & rest)                                 # nothing is shared any more


# ========================================================
# FOLLOW-UP 1: Only elements of the first part may be deleted
# ========================================================
# ---- follow-up 1: function ----
# Answer: then every shared value must go from the first part entirely, so
# the cost is the number of first-part elements whose value occurs in the
# rest (one set lookup per element).
# Complexity: Time O(n), Space O(n)
def min_deletions_first_only(a, k):
    rest = set(a[k:])
    count = 0
    for x in a[:k]:
        if x in rest:
            count += 1
    return count


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert min_deletions_first_only([1, 2, 2, 3, 2, 1], 3) == 3


# ========================================================
# FOLLOW-UP 2: Elements have weights (deleting some is more costly)
# ========================================================
# ---- follow-up 2: function ----
# Answer: use the sum of the weights of each value on each side instead of
# counts; the choice for each value is still independent, so take the cheaper
# side. Still O(n).
# Complexity: Time O(n), Space O(n)
def min_deletions_weighted(a, weights, k):
    left = {}
```

```
    right = {}
    for i in range(len(a)):
        if i < k:
            left[a[i]] = left.get(a[i], 0) + weights[i]
        else:
            right[a[i]] = right.get(a[i], 0) + weights[i]
    total = 0
    for v in left:
        if v in right:
            if left[v] < right[v]:
                total += left[v]
            else:
                total += right[v]
```

```
    return total


# ---- follow-up 2: tests ----
def test_follow_up_2():
    a = [1, 2, 2, 3, 2, 1]
    weights = [1, 5, 5, 1, 1, 1]                    # the first two 2's are heavy: delete the other side's
copy of 2
    assert min_deletions_weighted(a, weights, 3) == 1 + 1
    assert min_deletions_weighted(a, [1] * 6, 3) == min_deletions(a, 3)


# ============================================================
# FOLLOW-UP 3: The statement may differ from this reading
# ============================================================
# Answer: ask: must the two parts have no value in common, or only no
# equal values at the same position? Which parts may lose elements? State
# your reading before you code.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(list1: list[int], list2: list[int], k: int) -> int:
```

Sample call

```
solution([1, 2, 3], [2, 1, 4, 5], 2)
```

Test cases

Input	Expected
([1, 2, 3], [2, 1, 4, 5], 2)	2
([1, 2, 3, 4], [2, 5, 3, 6], 2)	1

STUDY PATH: question 76 of 197 | topic 13 of 43: Sets and counters: remember the first collection, check the second against it | 3 of 3

Template fit: build a set per window of the first list; keep the matching window of the second list minus that set | * asked 377 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 76 of 197

Filter Common Elements Per Window (Two Lists)

ASKED 2025-10-08 | 377 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 28.2 | 30 DAYS 0 | PRIORITY 150/288

ABOUT THIS QUESTION

Topic: Sets and counters: remember the first collection, check the second against it

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #350 Intersection of Two Arrays II

LeetCode match: LC 350 Intersection of Two Arrays II (related: per window)

Frequency: Single report | Last asked: 2025-10-08 | Roles: SWE Intern

QUESTION

You are given two lists a and b of equal length n and a window size k that divides n . Split both lists into consecutive, non-overlapping windows of size k (window 0 is indices 0..k-1 , window 1 is k..2k-1 , and so on). For each window, remove from b 's window every value that also appears in the corresponding window of a . Concatenate the surviving elements of b , preserving their original order, and return that list.

Function signature

```
def filter_windows(a: list[int], b: list[int], k: int) -> list[int]:
    ...
```

Follow-up (as reported): The interviewer asked to do the removal in place / O(1) extra space. Naive in-place removal becomes O(n*k); discuss compaction with a write pointer to keep it O(n) time.

EXAMPLES

Example 1	
Input	`a = [1, 2, 3, 4, 5, 6], b = [3, 7, 8, 6, 9, 5], k = 3`
Output	`[7, 8, 9]`
Why	`Window 0: a={1,2,3}, b=[3,7,8] -> remove 3 -> [7,8]. Window 1: a={4,5,6}, b=[6,9,5] -> remove 6 and 5 -> [9]. Concatenate -> [7,8,9].`

Example 2	
Input	`a = [1, 1], b = [1, 2], k = 2`
Output	`[2]`
Why	`Single window: a={1}, b=[1,2] -> remove 1 -> [2].`

CONSTRAINTS

- 0 <= n <= 10**5 , n % k == 0 , 1 <= k <= n (for n > 0).
- Values are integers.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_filter(a, b, k):
    out = []
    for i in range(0,_len(a),_k):
        window = a[i : i + k]
        out += [
            x
            for x in b[i:
                i + k]
            if x not in window
        ]
    return out
```

TIME	SPACE
O(n * k)	O(n)

WHY IT IS SLOW

Every element of b searches a whole window of a linearly.

OPTIMAL SOLUTION · A set per window

TIME	SPACE
O(n)	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it in place: no new output list.

Keep a write pointer w. For each window build the set of a's window, then copy each surviving element of b to b[w] and advance w; at the end truncate b to length w. Time O(n), extra space O(k) for one window's set.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Filter Common Elements Per Window (Two
# Lists)
#
# You are given two lists a and b of equal length n and a
# window size k that divides n . Split both lists into
# consecutive, non-overlapping windows of size k (window 0
# is indices 0..k-1 , window 1 is k..2k-1 , and so on).
# For each window, remove from b 's window every value
# that also appears in the corresponding window of a .
# Concatenate the surviving elements of b , preserving
# their original order, and return that list. Function
# signature: def filter_windows(a: list[int], b:
# list[int], k: int) -> list[int]: ... Follow-up (as
# reported): The interviewer asked to do the removal in
# place / O(1) extra space. Naive in-place removal becomes
# O(n*k); discuss compaction with a write pointer to keep
# it O(n) time.
#
# FOLLOW-UPS (answered below, in this order):
```

```
# 1. Do it in place: no new output list.
#
# INTERVIEW NOTES
# Each window is independent: put the window of a into a
# set and keep the elements of the matching window of b
# that are not in it. The set makes each membership test
# O(1), so the whole job is O(n) instead of O(n * k). In
# place, a write pointer compacts the survivors to the
# front.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Cut both lists into windows of k elements. In every window keep only the
# values of b that do NOT appear in the same window of a.
# Idea: put the window of a into a set (a set lookup is O(1)), then keep the
# elements of the window of b that are not in it.
# Complexity: Time O(n + m), Space O(k) for one window plus the answer
```

```
def filter_windows(a, b, k):
    result = []
    for start in range(0, len(a), k):
        seen = set(a[start:start + k])
        for x in b[start:start + k]:
            if x not in seen:
                result.append(x)
    return result


# ---- main question: tests ----
def test_main_question():
    assert filter_windows([1, 2, 3, 4, 5, 6], [3, 7, 8, 6, 9, 5], 3) == [7, 8, 9]
    assert filter_windows([1, 1], [1, 2], 2) == [2]


# ========================================================
# FOLLOW-UP 1: Do it in place: no new output list
# ========================================================
# ---- follow-up 1: function ----
# Answer: keep a write position `w`. For each window build the set of a's
# window, then copy every surviving element of b to b[w] and move w on; at the
# end cut b down to length w. Time O(n), extra space O(k) for one window's set.
# Complexity: Time O(n + m), Space O(k)
def filter_windows_in_place(a, b, k):
    w = 0
    for start in range(0, len(a), k):
        seen = set(a[start:start + k])
        for j in range(start, min(start + k, len(b))):
            if b[j] not in seen:
                b[w] = b[j]
                w += 1
    del b[w:]
    return b


# ---- follow-up 1: tests ----
def test_follow_up_1():
```

```
    b = [3, 7, 8, 6, 9, 5]
    assert filter_windows_in_place([1, 2, 3, 4, 5, 6], b, 3) == [7, 8, 9]
    assert b == [7, 8, 9]


# =====================================================
# RUN EVERYTHING
```

```
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 14 OF 43

bisect on a sorted list: find a position, count a range, keep it sorted

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

5
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

bisect on a sorted list: find a position, count a range, keep it sorted | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M	77. Sliding-Window Mean After Removing the Largest K Values	PracHub * 2026-09-01 *
M	78. Streaming Insert + Find K-th Largest	1Point3Acres * 2026-02-04 *
M	79. Count Sorted Strings With a Given Prefix	ColorStack * 2025-10-08 *
M	80. Russian Doll Envelopes	ND
M	81. Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid li...	ND

CODE TEMPLATE | GROUP 14 OF 43 | 5 QUESTIONS, ONE PATTERN

bisect on a sorted list: find a position, count a range, keep it sorted

The one idea these questions share

bisect_left and bisect_right give the position of a value in a sorted list in O(log n). Subtract two positions to count a range, slice between them to list a range, insort to keep inserting in sorted order, and replace an element at a position for the 'longest increasing' trick.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from bisect import bisect_left, bisect_right, insort


def count_in_range(sorted_list, low, high):
    hi = bisect_right(sorted_list, high)       # CHANGE: which positions you read
    lo = bisect_left(sorted_list, low)
    return hi - lo


# insort(sorted_list, x)                    keeps the list sorted as items arrive
# sorted_list[i]                            rank queries: i-th smallest, sorted_list[-1 - k] = k-th largest
```

2. Helper functions that usually get written next to it

```
def lis_length(values):                        # the tails list for "longest increasing"
    tails = []
    for v in values:
        i = bisect_left(tails, v)
        if i == len(tails):
            tails.append(v)
        else:
            tails[i] = v
    return len(tails)


def nearest(sorted_list, x):
    i = bisect_left(sorted_list, x)
    near = [sorted_list[j] for j in (i - 1, i) if 0 <= j < len(sorted_list)]
    return min(near, key=lambda v: abs(v - x))
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
77	Sliding-Window Mean After Removing the Largest K Values	keep the window in a sorted list; insort the new value, remove the old one with bisect_left, and track the sum of the top-K region
78	Streaming Insert + Find K-th Largest	insort every new value; the k-th largest is sorted_list[-1 - k]
79	Count Sorted Strings With a Given Prefix	lo = bisect_left(words, prefix); hi = bisect_left(words, prefix + a very large character); the answer is hi - lo
80	Russian Doll Envelopes	sort by width ascending and height descending, then the longest increasing run of heights using a tails list and bisect_left
81	Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller.	for each kid bisect into the sorted sellers and take the nearer of the two neighbours; the answer is the largest of those distances

Contents Start the questions >

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 77 of 197 | topic 14 of 43: bisect on a sorted list: find a position, count a range, keep it sorted | 1 of 5

Template fit: keep the window in a sorted list; insort the new value, remove the old one with bisect_left, and track the sum of the top-K region | * asked 49 days ago *

MEDIUM | PracHub | Technical Screen | 77 of 197

Sliding-Window Mean After Removing the Largest K Values

POSTED 2026-09-01 | 49 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 29.8 | 30 DAYS 0 | PRIORITY 146/288

ABOUT THIS QUESTION

Topic: bisect on a sorted list: find a position, count a range, keep it sorted

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #480 Sliding Window Median

LeetCode match: LC 480 Sliding Window Median (order statistics in a window)

Posted on PracHub: 2026-09-01 (updated 2026-09-23)

Original: https://prachub.com/coding-questions/sliding-window-mean-after-removing-the-largest-k-values

QUESTION

For every window of a fixed size over the array, remove the k largest values of the window (repeated values count separately) and compute the mean of the rest. Return the means in window order as reduced fractions written 'numerator/denominator'. Update the windows incrementally instead of sorting each one again.

Function signature

def trimmed_window_means(nums: list, window: int, k: int) -> list

EXAMPLES

Example 1	
Input	`[1, 2, 3, 4, 5], 3, 1`
Output	`['3/2', '5/2', '7/2']`
Why	`Window [1,2,3] without its largest value 3 averages (1+2)/2 = 3/2; then 5/2 and 7/2.`

Example 2	
Input	`[2, 4, 6, 8], 2, 0`
Output	`['3/1', '5/1', '7/1']`
Why	`With k = 0 nothing is removed: plain window means.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, w, k):
    means = []
    for i in range(len(nums) - w + 1):
        window = sorted(nums[i:i + w])
        kept = window[:w - k]  # drop k largest
        total = sum(kept)
        means.append(Fraction(total, len(kept)))
    return means
```

BRUTE FORCE

TIME	SPACE
O(n * w log w)	O(w)

WHY IT IS SLOW

Each window is sorted from scratch, O(n * w log w). A sorted list updated by one insertion and one removal, with the sum of the top k adjusted at the boundary, avoids it.

OPTIMAL SOLUTION · Sorted window with an incrementally maintained top-K sum

TIME	SPACE
O(n * (log w + w) in memmove, O(n log w) with a balanced tree)	O(w)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Make the update O(log w) in the worst case.

A Python list insert shifts elements (fast in practice, O(w) in theory). A balanced tree or a Fenwick tree over compressed values gives O(log w) insert, delete and 'sum of the top K' via a descent.

FOLLOW-UP 2 Remove the smallest K instead, or both ends (trimmed mean).

Maintain the sum of the K smallest as well (the first K elements of the same sorted list); the mean of the middle values is (total - top - bottom) / (w - 2K).

FOLLOW-UP 3 Duplicates on the boundary.

Removing exactly K occurrences, not K distinct values, is what the positional approach does: elements equal to the boundary value are interchangeable, so which copy sits in the top K does not matter for the sum. **Cost:** unchanged: O(w) per update for list insertion (O(log w) with a balanced tree).

FOLLOW-UP 4 Trim the smallest K as well (a trimmed mean).

Maintain the sum of the K smallest alongside the K largest with the mirror-image boundary updates; the mean of the middle values is (total - top - bottom) / (w - 2K). Checked against sorting every window.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Sliding-Window Mean After Removing the
# Largest K Values
#
# For every window of a fixed size over the array, remove
# the k largest values of the window (repeated values
# count separately) and compute the mean of the rest.
# Return the means in window order as reduced fractions
# written 'numerator/denominator'. Update the windows
# incrementally instead of sorting each one again.
#
# Signature:
#   def trimmed_window_means(nums: list, window: int, k: int) -> list
```

```
#
# Examples:
# [1, 2, 3, 4, 5], 3, 1  ->  ['3/2', '5/2', '7/2']
# [2, 4, 6, 8], 2, 0  ->  ['3/1', '5/1', '7/1']
#
# FOLLOW-UPS (answered below, in this order):
# 1. Make the update O(log w) in the worst case.
# 2. Remove the smallest K instead, or both ends (trimmed
# mean).
# 3. Duplicates on the boundary.
# 4. Trim the smallest K as well (a trimmed mean).
#
# INTERVIEW NOTES
# The mean needs two sums: the whole window and the sum of
# its K largest values. Keep the window in a sorted
# container; the K largest are its last K elements. When
# one value enters or leaves, only one element crosses the
# boundary between 'kept' and 'removed', so the sum of the
# K largest can be updated in O(1) after an O(log) search.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from bisect import bisect_left, bisect_right
from fractions import Fraction
import random


# ============================================================
```

```
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def as_text(fraction):
    # a reduced fraction as text: Fraction(5, 2) -> "5/2"
    return str(fraction.numerator) + "/" + str(fraction.denominator)


# Complexity: Time insert and remove O(w) (a binary search plus a list shift), Space O(w)
class Window:
    # the values of the current window in a SORTED list, with three sums kept
    # up to date while values come and go: the whole sum (total), the sum of
    # the k largest values (top) and the sum of the k smallest values (bottom)
    def __init__(self, k):
        self.k = k
        self.values = []
        self.total = 0
        self.top = 0
        self.bottom = 0

    def insert(self, x):
        k = self.k
        values = self.values
        old_length = len(values)
        place = bisect_right(values, x)
        values.insert(place, x)
        self.total += x
        # the top region is the last k places of the new list
        if old_length < k:
            self.top += x                                  # the list is still short: everything is "top"
        elif place >= old_length + 1 - k:
```

```
            self.top += x - values[old_length - k]         # x entered it, the value just below it left
        # the bottom region is the first k places
        if old_length < k:
            self.bottom += x
        elif place < k:
            self.bottom += x - values[k]                   # x entered it, the value now at index k left

    def remove(self, x):
        k = self.k
        values = self.values
        old_length = len(values)
        place = bisect_left(values, x)
        self.total -= x
        if old_length <= k:
```

```
            self.top -= x
            self.bottom -= x
        else:
            if place >= old_length - k:
                self.top += values[old_length - k - 1] - x     # the value just below the region joins it
            if place < k:
                self.bottom += values[k] - x                   # the value at index k joins the region
            del values[place]


# ---- main question: main function ----
# Idea: for every window of the given size, drop its k largest values
# and take the mean of the rest. Keep the window in a sorted list and keep
# the sum of its k largest values up to date while one value enters and
# one leaves, so no window is sorted again. With both=True the k smallest
# are dropped as well (a trimmed mean, follow-up 4). Each mean is
# returned as an exact reduced fraction in text, like "5/2".
# Complexity: Time O(n * w) for windows of size w (each step is one insert and one remove), Space O(w)
def trimmed_window_means(nums, window, k, both=False):
    w = Window(k)
    means = []
    for i in range(len(nums)):
        w.insert(nums[i])
        if i >= window:
            w.remove(nums[i - window])
        if i >= window - 1:
            kept = window - k
            dropped = w.top
            if both:
                kept = window - 2 * k
                dropped = w.top + w.bottom
            means.append(as_text(Fraction(w.total - dropped, kept)))
    return means


# ---- main question: tests ----
def slow_means(nums, window, k, both=False):
    # sort every window again (obviously right, slow) to check the fast version
    means = []
    for i in range(window - 1, len(nums)):
        piece = sorted(nums[i - window + 1:i + 1])
        if both:
            kept = piece[k:window - k]
        else:
            kept = piece[:window - k]
```

```
        total = 0
        for v in kept:
            total += v
        means.append(as_text(Fraction(total, len(kept))))
```

```
    return means


def test_main_question():
    assert trimmed_window_means([5, 1, 4], 3, 1) == ["5/2"]
    assert trimmed_window_means([3, 3, 3, 3], 3, 1) == ["3/1", "3/1"]       # duplicates at the boundary
    random.seed(33)
    for round_number in range(300):
        nums = []
        for i in range(random.randint(3, 14)):
            nums.append(random.randint(-6, 6))
        window = random.randint(2, len(nums))
        k = random.randint(0, window - 1)
        assert trimmed_window_means(nums, window, k) == slow_means(nums, window, k)


# =========================================================
# FOLLOW-UP 1: Make the update O(log w) in the worst case
# =========================================================
# Answer: inserting into a Python list shifts items (O(w)). For a true
# O(log w) update use a balanced tree or a Fenwick tree over the value
# ranks that also stores sums, so "sum of the k largest" is one query.


# =========================================================
# FOLLOW-UP 3: Duplicates on the boundary
# =========================================================
# Answer: remove exactly k COPIES: it does not matter which equal value is
# dropped because they are equal. The sorted list handles this naturally
# (bisect_right to insert, bisect_left to remove).


# =========================================================
# FOLLOW-UP 2: Remove the smallest K instead, or both ends (trimmed mean)
# =========================================================


# =========================================================
# FOLLOW-UP 4: Trim the smallest K as well (a trimmed mean)
# =========================================================
# ---- follow-up 4: tests ----
```

```
# Answer: the Window class also keeps the sum of the K smallest values (the
# mirror image of the top sum), so the trimmed mean is
# (total - top - bottom) / (window - 2K): trimmed_window_means(..., both=True).
def test_follow_up_4():
    nums = [5, 1, 4, 2, 8, 3, 3, 9]
    assert trimmed_window_means(nums, 5, 1, True) == slow_means(nums, 5, 1, True)
    random.seed(34)
    for round_number in range(300):
        nums = []
        for i in range(random.randint(3, 14)):
            nums.append(random.randint(-6, 6))
        window = random.randint(3, len(nums)) if len(nums) >= 3 else 3
        if window > len(nums):
            continue
        k = random.randint(0, (window - 1) // 2)
        assert trimmed_window_means(nums, window, k, True) == slow_means(nums, window, k, True)


# =========================================================
# RUN EVERYTHING
# =========================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def trimmed_window_means(nums: list[int], window: int, k: int) -> list[str]:
```

Sample call

```
trimmed_window_means([1, 9, 2, 8], 3, 1)
```

Test cases

Input	Expected
`([1, 9, 2, 8], 3, 1)`	`['3/2', '5/1']`
`([-1, 1, 3], 2, 0)`	`['0/1', '2/1']`

ALSO REPORTED · 2 more

Sliding Window Average After Dropping the Largest Values | PracHub | Technical Screen
https://prachub.com/coding-questions/sliding-window-average-after-dropping-the-largest-values

Compute sliding-window average excluding top k | PracHub | Technical Screen
https://prachub.com/coding-questions/compute-sliding-window-average-excluding-top-k

STUDY PATH: question 78 of 197 | topic 14 of 43: bisect on a sorted list: find a position, count a range, keep it sorted | 2 of 5

Template fit: insort every new value; the k-th largest is sorted_list[-1 - k] | * asked 258 days ago * | frequency Single report

MEDIUM | IN APP LC 703 | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 78 of 197

★ TOP 75: rank 68 in the Medium and Hard priority list

★ Streaming Insert + Find K-th Largest

ASKED 2026-02-04 | 258 DAYS AGO | OLDER

STUDY: TALK **ALL TIME 32.6** **30 DAYS 0** **PRIORITY 137/288**

ABOUT THIS QUESTION

Topic: bisect on a sorted list: find a position, count a range, keep it sorted

Optimal answer from: your app's interview approach (LeetCode #703)

LeetCode match: LC 703 Kth Largest Element in a Stream

Frequency: Single report | Last asked: 2026-02-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/insert-find-kth-largest

QUESTION

Design a structure with insert(x), which adds a number, and findLargest(k), which returns the k-th largest number inserted so far (k = 0 is the largest), counting repeated values separately, or None when fewer than k + 1 numbers exist.

Function signature

class StreamKth:
  def insert(self, x: float) -> None
  def findLargest(self, k: int) -> float | None

EXAMPLES

Example 1

Input	`s = StreamKth()`
	`s.insert(3)`
	`s.insert(3)`
	`s.insert(2)`
	`s.findLargest(0)`
	`s.findLargest(1)`
	`s.findLargest(2)`
	`s.findLargest(3)`

Output	`s.findLargest(0) -> 3`
	`s.findLargest(1) -> 3`
	`s.findLargest(2) -> 2`
	`s.findLargest(3) -> null`

Why	`Sorted descending the numbers are 3, 3, 2, so the 0th, 1st and 2nd largest are 3, 3 and 2, and there is no 3rd.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
items = []
def insert(x):
    items.append(x)
def find_largest(k):
    ordered = sorted(items, reverse=True)
    return ordered[k]
```

BRUTE FORCE

TIME	SPACE
O(n log n) per query	**O(n)**

WHY IT IS SLOW

Every query re-sorts the whole history, O(n log n). A sorted list with binary search makes each query O(1) after an O(log n) search plus shift; for a fixed k a size-k heap is O(log k).

OPTIMAL SOLUTION · Sorted list with bisect (k chosen per query)

TIME	SPACE
O(log n) search + O(n) shift per insert, O(1) query	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if values can also be deleted?

The heap can no longer answer because removing an arbitrary element is O(k). Keep a sorted structure that supports delete: a balanced BST or sorted container with order statistics, or a Fenwick tree over compressed values (add -1 to delete). Both give O(log n) insert, delete and k-th.

FOLLOW-UP 2 What is the trade-off between the two designs?

The heap is the smallest and fastest when k is fixed and small (O(log k), O(k) memory) but cannot change k or delete. The sorted list answers any k and is simplest, at O(n) memory and a linear shift per insert. Choose from the requirements, and say which assumption you made.

FOLLOW-UP 3 Values can also be deleted.

A heap cannot delete an arbitrary value efficiently. Keep a sorted structure: a sorted list with bisect (O(n) per delete because of shifting) or a balanced tree / Fenwick tree over compressed values (O(log n) for insert, delete and k-th).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Streaming Insert + Find K-th Largest
#
# Design a structure with insert(x), which adds a number,
# and findLargest(k), which returns the k-th largest
# number inserted so far (k = 0 is the largest), counting
# repeated values separately, or None when fewer than k +
# 1 numbers exist.
#
# Signature:
#   class StreamKth:
#       def insert(self, x: float) -> None
#       def findLargest(self, k: int) -> float | None
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if values can also be deleted?
```

```
# 2. What is the trade-off between the two designs?
# 3. Values can also be deleted.
#
# INTERVIEW NOTES
# If k can change from query to query you need every
# value, in a structure that supports rank queries; if k
# is fixed you only need the k largest, and a min-heap of
# size k gives the answer at its root. Ask which of the
# two it is: it decides everything.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq
from bisect import bisect_left, insort


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: keep every value in an ascending list. insort puts a new value
# in its place with a binary search (O(log n) to find the spot, O(n) to
# shift the others). The k-th largest (k = 0 is the largest, as in the
# examples) is then simply a[-1 - k].
# Complexity: Time insert O(n) (a binary search plus a list shift), delete O(n), findLargest O(1), Space O(
n)
```

```
class StreamKth:
    def __init__(self):
        self.a = []

    def insert(self, x):
        insort(self.a, x)

    def findLargest(self, k):
        if 0 <= k < len(self.a):
            return self.a[-1 - k]
        return None

    # follow-ups 1 and 3: values can also be deleted. A sorted list can
    # delete: find a copy of x with a binary search and remove it.
    def delete(self, x):
        i = bisect_left(self.a, x)
        if i < len(self.a) and self.a[i] == x:
            del self.a[i]
            return True
        return False


# ---- main question: tests ----
def test_main_question():
    s = StreamKth()
    for x in (3, 3, 2):
        s.insert(x)
    assert [s.findLargest(k) for k in range(3)] == [3, 3, 2]
    assert s.findLargest(3) is None
    assert s.delete(3) and not s.delete(9)             # deletes work too
    assert [s.findLargest(k) for k in range(2)] == [3, 2]
```

```
# ======================================================
# FOLLOW-UP 1: What if values can also be deleted?
# ======================================================


# ======================================================
# FOLLOW-UP 3: Values can also be deleted
# ======================================================
# Answer: a heap cannot delete an arbitrary value efficiently (finding it
# costs O(k)), but the sorted list can: StreamKth.delete above.
```

```
# ======================================================
# FOLLOW-UP 2: What is the trade-off between the two designs?
# ======================================================
# ---- follow-up 2: function ----
# Answer: the heap is the smallest and fastest when k is FIXED and small
# (O(log k) per insert, O(k) memory) but it cannot change k or delete.
# The sorted list answers any k and is simplest, at O(n) memory and a
# linear shift on every insert.
# Complexity: Time insert O(log k), kth_largest O(1), Space O(k)
class FixedKth:
    def __init__(self, k):
        self.k = k
        self.heap = []                      # a min-heap holding the k largest values

    def insert(self, x):
        heapq.heappush(self.heap, x)
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)        # drop the smallest: keep only the k largest

    def kth_largest(self):
        if len(self.heap) < self.k:
            return None
        return self.heap[0]                 # the smallest of the k largest


# ---- follow-up 2: tests ----
def test_follow_up_2():
    f = FixedKth(2)
    assert f.kth_largest() is None
    for x in (5, 1, 9, 7):
        f.insert(x)
    assert f.kth_largest() == 7             # the two largest are 9 and 7


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Sketch the size-augmented BST node (left subtree count + duplicate count) on paper before the interview.
Practice the Fenwick-tree order-statistics trick (find k-th by binary search on prefix sums).
Be ready to defend the chosen data structure: balanced BST is clean but verbose; Fenwick is short but assumes bounded values.

STUDY PATH: question 79 of 197 | topic 14 of 43: bisect on a sorted list: find a position, count a range, keep it sorted | 3 of 5

Template fit: lo = bisect_left(words, prefix); hi = bisect_left(words, prefix + a very large character); the answer is hi - lo | * asked 377 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 79 of 197

Count Sorted Strings With a Given Prefix

ASKED 2025-10-08 | 377 DAYS AGO | OLDER

STUDY: TALK **ALL TIME 36** **30 DAYS 0** **PRIORITY 125/288**

ABOUT THIS QUESTION

Topic: bisect on a sorted list: find a position, count a range, keep it sorted

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #208 Implement Trie (Prefix Tree)

LeetCode match: LC 208 Implement Trie (related: counting by prefix on a sorted list)

Frequency: Single report | Last asked: 2025-10-08 | Roles: SWE Intern

QUESTION

Given a lexicographically sorted array of strings words and a query string prefix , return the number of strings in words that start with prefix . Because words is sorted, all matching strings occupy a contiguous range; find that range with binary search in O(log n * L) time (where L is the prefix length), not by scanning.

Function signature

```
def count_with_prefix(words: list[str], prefix: str) -> int:
    ...
```

Follow-up (as reported): Many repeated queries: cache counts in a hash map keyed by prefix. All queries unique: build a trie of all words storing subtree sizes, so each query is O(L) .

EXAMPLES

Example 1

Input	`words = ["app", "apple", "application", "banana", "bat"], prefix = "app"`
Output	`3`
Why	`"app", "apple", "application" all start with "app".`

Example 2

Input	`words = ["app", "apple", "application", "banana", "bat"], prefix = "ba"`
Output	`2`
Why	`"banana" and "bat".`

Example 3

Input	`words = ["app", "apple", "application", "banana", "bat"], prefix = "cat"`
Output	`0`
Why	`No string starts with "cat".`

CONSTRAINTS

- 0 <= len(words) <= 10**6 , sorted in non-decreasing lexicographic order.
- 0 <= len(prefix) <= 100 . An empty prefix matches every string.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_prefix(words, prefix):
    return sum(
        1
        for w in words
        if w.startswith(prefix)
    )
```

TIME	SPACE
O(n * L)	O(1)

WHY IT IS SLOW

It ignores that the list is sorted and tests every word.

OPTIMAL SOLUTION · Two binary searches

TIME	SPACE
O(L log n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Thousands of different queries on the same list. Can each be O(L)?

Build a trie once where every node stores how many words pass through it. A query walks the prefix and returns the count in that node (0 if the path breaks). Building costs O(total characters); a query costs O(L), independent of the number of words.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count Sorted Strings With a Given Prefix
#
# Given a lexicographically sorted array of strings words
# and a query string prefix , return the number of strings
# in words that start with prefix . Because words is
# sorted, all matching strings occupy a contiguous range;
# find that range with binary search in O(log n * L) time
# (where L is the prefix length), not by scanning.
# Function signature: def count_with_prefix(words:
# list[str], prefix: str) -> int: ... Follow-up (as
# reported): Many repeated queries: cache counts in a hash
# map keyed by prefix. All queries unique: build a trie of
# all words storing subtree sizes, so each query is O(L) .
#
# FOLLOW-UPS (answered below, in this order):
# 1. Thousands of different queries on the same list. Can
# each be O(L)?
#
# INTERVIEW NOTES
# In a sorted list all words with a given prefix are
# contiguous. Binary search the first index that is >=
# prefix, and the first index that is greater than every
# word starting with the prefix (the prefix followed by
# the largest possible character); the count is the
```

```
# difference. Each comparison costs O(L).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from bisect import bisect_left


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: in a sorted list the words that start with the prefix form ONE
# block. It starts at the first word >= prefix and ends just before the
# first word >= prefix followed by the biggest possible character. Two
# binary searches find both ends; the answer is the size of the block.
# Complexity: Time O(L log n) for a prefix of length L (each comparison costs up to L), Space O(1)
```

```
def count_with_prefix(words, prefix):
    start = bisect_left(words, prefix)
    end = bisect_left(words, prefix + "\uffff")
    return end - start


# ---- main question: tests ----
def test_main_question():
    words = ["app", "apple", "application", "banana", "bat"]
    assert count_with_prefix(words, "app") == 3
    assert count_with_prefix(words, "ba") == 2
    assert count_with_prefix(words, "cat") == 0
    assert count_with_prefix(words, "") == 5


# ============================================================
# FOLLOW-UP 1: Thousands of different queries on the same list. Can each be O(L)?
# ============================================================
# ---- follow-up 1: function ----
# Answer: build a trie once in which every node stores how many words pass
# through it. A query walks down the prefix (L steps) and returns the
# count stored in the node it reaches (0 if the path breaks).
# Complexity: Time build O(total characters), each query O(L), Space O(total characters)
class PrefixCounter:
    def __init__(self, words):
        self.total = len(words)
        self.root = {"count": 0, "children": {}}
        for word in words:
            node = self.root
            for ch in word:
                if ch not in node["children"]:
                    node["children"][ch] = {"count": 0, "children": {}}
                node = node["children"][ch]
                node["count"] += 1             # one more word passes through this node

    def count(self, prefix):
        node = self.root
        for ch in prefix:
            if ch not in node["children"]:
                return 0
            node = node["children"][ch]
        if prefix == "":
            return self.total
```

```
        return node["count"]
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    words = ["app", "apple", "application", "banana", "bat"]
    counter = PrefixCounter(words)
    for prefix in ("app", "ba", "cat", "", "apple", "applesauce"):
        assert counter.count(prefix) == count_with_prefix(words, prefix)


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 80 of 197 | topic 14 of 43: bisect on a sorted list: find a position, count a range, keep it sorted | 4 of 5

Template fit: sort by width ascending and height descending, then the longest increasing run of heights using a tails list and bisect_left | ND | frequency Single report

MEDIUM | IN APP LC 354 | 1Point3Acres | phone-screen | 80 of 197

Russian Doll Envelopes

STUDY: TYPE | ALL TIME 52.8 | 30 DAYS 33 | PRIORITY 34/288

ABOUT THIS QUESTION

Topic: bisect on a sorted list: find a position, count a range, keep it sorted

Optimal answer from: your app's interview approach (LeetCode #354)

LeetCode match: LC 354 Russian Doll Envelopes

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/russian-doll-envelopes

QUESTION

LeetCode 354. Maximum chain of envelopes that strictly fit inside each other. Sort + LIS reduction.

Input: list of (width, height) envelopes.
Output: longest chain where each envelope strictly fits in the next.
Trick: sort by width ascending, height descending (when widths tie); run LIS (Longest Increasing Subsequence) on heights.
LIS with patience-sort gives O(n log n).

EXAMPLES

```
[[5,4],[6,4],[6,7],[2,3]] -> 3.
```

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
envelopes.sort()  # by width
best = [1] * len(envelopes)
for i in range(len(envelopes)):
    for j in range(i):
        w1, h1 = envelopes[j]
        w2, h2 = envelopes[i]
        if w1 < w2 and h1 < h2:
            best[i] = max(best[i], best[j] + 1)
answer = max(best)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each envelope is compared with all earlier ones, O(n^2). After sorting, a longest increasing subsequence of the heights with binary search is O(n log n).

OPTIMAL SOLUTION · Sort, then LIS with binary search

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why sort heights descending when widths tie?

If two envelopes have the same width they cannot nest, but an ascending height order would let the LIS pick both. Descending order makes a strictly increasing height run use at most one envelope per width.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Russian Doll Envelopes
#
# LeetCode 354. Maximum chain of envelopes that strictly
# fit inside each other. Sort + LIS reduction. Input: list
# of (width, height) envelopes. Output: longest chain
# where each envelope strictly fits in the next. Trick:
# sort by width ascending, height descending (when widths
# tie); run LIS (Longest Increasing Subsequence) on
# heights. LIS with patience-sort gives O(n log n).
# Examples: [[5,4],[6,4],[6,7],[2,3]] -> 3.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why sort heights descending when widths tie?
#
# INTERVIEW NOTES
# Sort by width ascending so that only the heights matter,
# then find the longest strictly increasing subsequence of
# heights. Envelopes with equal width cannot nest, so sort
# those by height descending: then a strictly increasing
# run of heights can never use two envelopes of the same
# width.

# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

from bisect import bisect_left


# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: helper functions ----
def width_then_taller_first(envelope):
    # sort key: width ascending; for EQUAL widths the taller one first
    return (envelope[0], -envelope[1])


# ---- main question: main function ----
# Idea: sort by (width ascending, height descending), then find the
# longest strictly increasing run of heights with a binary search.
# tails[k] is the smallest possible last height of a chain of k + 1
```

```python
# envelopes; each new height either extends the longest chain or lowers
# one of the tails.
# Complexity: Time O(n log n), Space O(n)
def max_envelopes(envelopes):
    ordered = sorted(envelopes, key=width_then_taller_first)
    tails = []
    for width, height in ordered:
        i = bisect_left(tails, height)
        if i == len(tails):
            tails.append(height)
        else:
            tails[i] = height
    return len(tails)


# ---- main question: tests ----
def test_main_question():
    assert max_envelopes([[5, 4], [6, 4], [6, 7], [2, 3]]) == 3
    assert max_envelopes([]) == 0
    assert max_envelopes([[1, 1], [1, 2], [1, 3]]) == 1             # equal widths cannot nest, however the
 heights go
    assert max_envelopes([[4, 5], [4, 6], [6, 7], [2, 3], [1, 1]]) == 4


# ============================================================
# FOLLOW-UP 1: Why sort heights descending when widths tie?
# ============================================================
# Answer: envelopes with the SAME width cannot nest. Putting the taller
# one first means that, among equal widths, heights go DOWN, and a
# strictly increasing run of heights can use at most one of them. Sorted
# ascending, the run could wrongly pick several of the same width.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write LIS O(n log n) from memory in under 8 min.
Justify the descending-height-on-tie trick out loud.
Plain LIS and the mountain-LIS variant make good drill partners.

STUDY PATH: question 81 of 197 | topic 14 of 43: bisect on a sorted list: find a position, count a range, keep it sorted | 5 of 5

Template fit: for each kid bisect into the sorted sellers and take the nearer of the two neighbours; the answer is the largest of those distances | ND

MEDIUM | Exponent | 81 of 197

Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller.

ASKED (ANY CO.) ~A YEAR AGO | OLDER

STUDY: TALK | ALL TIME 26.5 | 30 DAYS 0 | PRIORITY 154/288

ABOUT THIS QUESTION

Topic: bisect on a sorted list: find a position, count a range, keep it sorted

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #475 Heaters

LeetCode match: No direct match; related LC 475 Heaters (inferred from short statement)

Last asked (any company): a year ago, about 2025-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/5138/minimum-distance-kids-ice-cream-sellers

QUESTION

Kids and ice cream sellers stand at positions on a line. Find the smallest distance E such that every kid is within E of at least one seller (the kid at position K is served by a seller at P when P - E <= K <= P + E).

Function signature

def smallest_e(kids: list, sellers: list) -> int

EXAMPLES

Example 1	
Input	`[1, 5, 9], [2, 8]`
Output	`3`
Why	`The kid at 5 is 3 away from both sellers (at 2 and 8); the others are 1 away.`

Example 2	
Input	`[4], [4]`
Output	`0`
Why	`A kid standing on a seller needs distance 0.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(kids, sellers):
    worst = 0
    for kid in kids:
        gaps = [abs(kid - s) for s in sellers]
        nearest = min(gaps)
        worst = max(worst, nearest)
    return worst
```

BRUTE FORCE

TIME	SPACE
O(N * M)	O(1)

WHY IT IS SLOW

N * M distance checks. With the sellers sorted, binary search finds each kid's nearest seller among two neighbours, O((N + M) log M).

OPTIMAL SOLUTION · Sort the sellers, binary search each kid

TIME	SPACE
O((N + M) log M)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Both lists are already sorted, or you want to avoid the binary search.

Walk the kids in order with one pointer on the sellers that only moves forward while the next seller is at least as close. O(N + M) after sorting.

FOLLOW-UP 2 Each seller can serve at most C kids.

Now it is an assignment problem. Binary search the radius E and test feasibility with a greedy left-to-right matching that gives each seller up to C of the leftmost uncovered kids in its range.

FOLLOW-UP 3 Sellers can be placed anywhere (minimise the radius with S sellers).

That is the 1D k-center problem: binary search the radius and greedily place a seller at the farthest point covering the leftmost uncovered kid. O(N log range).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given positions of N kids and M ice cream
# sellers on a 1D line, find the smallest distance E such
# that every kid lies within the range [P - E, P + E] of
# at least one seller.
#
# Kids and ice cream sellers stand at positions on a line.
# Find the smallest distance E such that every kid is
# within E of at least one seller (the kid at position K
# is served by a seller at P when P - E <= K <= P + E).
#
# Signature:
#   def smallest_e(kids: list, sellers: list) -> int
#
# Examples:
# [1, 5, 9], [2, 8]  ->  3
# [4], [4]  ->  0
#
# FOLLOW-UPS (answered below, in this order):
# 1. Both lists are already sorted, or you want to avoid
# the binary search.
```

```
# 2. Each seller can serve at most C kids.
# 3. Sellers can be placed anywhere (minimise the radius
# with S sellers).
#
# INTERVIEW NOTES
# Each kid only cares about its nearest seller, and the
# radius E must cover the worst kid. So E is the maximum
# over kids of the distance to the nearest seller. With
# sellers sorted, the nearest seller of a kid is one of
# the two neighbours around its insertion point, which is
# a binary search.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from bisect import bisect_left


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
def distance_to_nearest(sorted_sellers, kid):
    # a binary search finds where the kid would stand among the sellers; the
    # nearest seller is the one right at or after that spot, or the one before it
    i = bisect_left(sorted_sellers, kid)
    nearest = float("inf")
    if i < len(sorted_sellers):
        nearest = sorted_sellers[i] - kid
    if i > 0 and kid - sorted_sellers[i - 1] < nearest:
        nearest = kid - sorted_sellers[i - 1]
    return nearest


# ---- main question: main function ----
# Idea: every kid should go to the nearest seller, so E is the WORST such
# distance over all kids. Sort the sellers once and binary search each kid.
# Complexity: Time O((n + m) log m), Space O(m)
def smallest_e(kids, sellers):
    sellers = sorted(sellers)
    worst = 0
    for kid in kids:
        d = distance_to_nearest(sellers, kid)
        if d > worst:
            worst = d
    return worst


# ---- main question: tests ----
def test_main_question():
    assert smallest_e([1, 5, 9], [2, 8]) == 3
    assert smallest_e([4], [4]) == 0


# ============================================================
# FOLLOW-UP 1: Both lists are already sorted, or you want to avoid the binary search
# ============================================================
# ---- follow-up 1: function ----
# Answer: walk the kids in order with ONE pointer on the sellers that only
# moves forward while the next seller is at least as close. O(N + M)
```

```
# (after sorting, if the lists were not sorted).
# Complexity: Time O(n log n + m log m) (O(n + m) when already sorted), Space O(n + m)
def smallest_e_sorted(kids, sellers):
    kids = sorted(kids)
    sellers = sorted(sellers)
    worst = 0
    j = 0
    for kid in kids:
        while j + 1 < len(sellers):
            if abs(sellers[j + 1] - kid) > abs(sellers[j] - kid):
                break                       # the next seller is farther: stay
            j += 1
        if abs(sellers[j] - kid) > worst:
            worst = abs(sellers[j] - kid)
    return worst


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert smallest_e_sorted([1, 5, 9], [2, 8]) == 3
    assert smallest_e_sorted([4], [4]) == 0


# =========================================================
# FOLLOW-UP 2: Each seller can serve at most C kids
# =========================================================
# ---- follow-up 2: helper functions ----
def feasible_capacity(kids, sellers, e, cap):
    # can every kid be served when a seller takes at most `cap` kids and
    # only kids within e? Going left to right, each seller takes the
    # leftmost kids that are still unserved and in range.
    i = 0
    for s in sellers:
        if i < len(kids) and kids[i] < s - e:
            return False                    # this kid is too far left for every later seller
        taken = 0
        while i < len(kids) and taken < cap:
            if kids[i] > s + e:
                break
            i += 1
            taken += 1
    return i == len(kids)


# ---- follow-up 2: function ----
# Answer: now it is an assignment problem. Binary search the radius E and
# test it with the greedy left-to-right matching above.
# Complexity: Time O((n + m) log(range)), Space O(n + m)
def smallest_e_capacity(kids, sellers, cap):
    kids = sorted(kids)
    sellers = sorted(sellers)
    if len(kids) > cap * len(sellers):
        return None                         # not enough room at all
    low = 0
    high = max(kids[-1], sellers[-1]) - min(kids[0], sellers[0])
    while low < high:
        mid = (low + high) // 2
        if feasible_capacity(kids, sellers, mid, cap):
            high = mid
        else:
            low = mid + 1
    return low
```

```python


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert smallest_e_capacity([1, 2, 3], [2], 1) is None       # three kids, room for one
    assert smallest_e_capacity([1, 2, 3], [1, 3, 9], 1) == 6
    assert smallest_e_capacity([1, 2], [5, 6], 1) == 4


# ============================================================
# FOLLOW-UP 3: Sellers can be placed anywhere (minimise the radius with S sellers)
# ============================================================
# ---- follow-up 3: helper functions ----
def sellers_needed(kids, e):
    # greedy: the leftmost kid not covered yet, at x, gets a seller at x + e,
    # who covers every kid up to x + 2e
    count = 0
    reach = None
    for kid in sorted(kids):
        if reach is None or kid > reach:
            count += 1
            reach = kid + 2 * e
    return count


# ---- follow-up 3: function ----
# Answer: this is the 1D k-center problem. Binary search the radius and
# test it with the greedy placement above. O(N log range).
# Complexity: Time O(n log(range)), Space O(n)
def best_radius(kids, how_many):
    kids = sorted(kids)
    low = 0
    high = kids[-1] - kids[0]
    while low < high:
        mid = (low + high) // 2
        if sellers_needed(kids, mid) <= how_many:
            high = mid
        else:
            low = mid + 1
    return low


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert sellers_needed([1, 2, 3, 20], 1) == 2
    assert best_radius([1, 2, 3, 20], 2) == 1
    assert best_radius([5], 1) == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 15 OF 43

Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

5
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M	82. Range Duplicate Check with Constant-Time Queries	1Point3Acres * 2026-09-29 *
M	83. Find a Horizontal Cut That Bisects Rectangle Area	PracHub * 2026-06-04 *
M	84. Check pivot after removing one element	PracHub * 2026-02-12 *
M	85. Gold Chain Split into Two Equal Halves	1Point3Acres * 2026-04-21 *
M	86. Count the number of ways to split a string.	ND

CODE TEMPLATE | GROUP 15 OF 43 | 5 QUESTIONS, ONE PATTERN

Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

The one idea these questions share

pre[i] is the total of the first i items. Any range sum is pre[r + 1] - pre[l], and a balance point is an index where the left total equals the right total. Count something other than values (equal pairs, ones, repeats) by accumulating that instead.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def build_prefix(nums):
    pre = [0]
    for x in nums:                          # CHANGE 1: what you accumulate (value, 1 per match, 1 per one)
        pre.append(pre[-1] + x)
    return pre


def range_sum(pre, l, r):                   # inclusive l..r
    return pre[r + 1] - pre[l]


def pivot_index(nums):
    pre = build_prefix(nums)
    for i in range(len(nums)):              # CHANGE 2: the test at each index
        if pre[i] == pre[-1] - pre[i + 1]:
            return i
    return -1
```

2. Helper functions that usually get written next to it

```
def first_last(values):                     # first and last position of every value
    first, last = {}, {}
    for i, v in enumerate(values):
        first.setdefault(v, i)
        last[v] = i
    return first, last
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
82	Range Duplicate Check with Constant-Time Queries	next[i] = next occurrence of the same value; reach[l] = smallest next over l..end; the range l..r is all distinct when reach[l] > r
83	Find a Horizontal Cut That Bisects Rectangle Area	events by height with a running width; accumulate area strip by strip until half the total, then solve inside that strip
84	Check pivot after removing one element	prefix sums plus first/last position maps: look up the value whose removal would balance both sides
85	Gold Chain Split into Two Equal Halves	prefix sums with first/last position maps: find two cut points that leave equal halves around one removed link
86	Count the number of ways to split a string.	positions of the ones; the answer multiplies the gap sizes around the k-th and 2k-th one (zero ones: choose two cut points)

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 82 of 197 | topic 15 of 43: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) | 1 of 5

Template fit: next[i] = next occurrence of the same value; reach[i] = smallest next over l..end; the range l..r is all distinct when reach[l] > r | * asked 21 days ago * | frequency Low

MEDIUM | 1Point3Acres | onsite-coding | 82 of 197

Range Duplicate Check with Constant-Time Queries

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 271/288**

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/range-duplicate-check-constant-time-queries

QUESTION

Preprocess an array once so that each query unique(l, r) answers in O(1) whether all elements from index l to index r (inclusive) are different.

Function signature

class RangeUnique:
  def __init__(self, a: list)
  def unique(self, l: int, r: int) -> bool

EXAMPLES

Example 1

Input	`ru = RangeUnique([1, 2, 3, 1, 4])`
	`ru.unique(0, 2)`
	`ru.unique(0, 3)`
	`ru.unique(1, 4)`

Output	`ru.unique(0, 2) -> true`
	`ru.unique(0, 3) -> false`
	`ru.unique(1, 4) -> true`

Why	`Indexes 0..3 hold 1, 2, 3, 1: the value 1 repeats. Ranges 0..2 and 1..4 have no repeats.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def all_unique(a, l, r):
    seen = set()
    for x in a[l:r + 1]:
        if x in seen:
            return False
        seen.add(x)
    return True
```

TIME	SPACE
O(r - l) per query	**O(r - l)**

BRUTE FORCE

WHY IT IS SLOW

Each query costs the length of the range. A next-occurrence array with suffix minimums answers each query with one comparison.

OPTIMAL SOLUTION · Next-occurrence array with suffix minimum

TIME	SPACE
O(n)	**O(n)**
preprocessing, O(1) per query	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The array changes between queries.

Rebuild next and reach in O(n) after a batch of changes when reads dominate. If single-element updates are frequent, keep per-value sorted index lists and a segment tree of next-occurrence minima so an update and a query cost O(log n) instead.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Range Duplicate Check with Constant-Time
# Queries
#
# Preprocess an array once so that each query unique(l, r)
# answers in O(1) whether all elements from index l to
# index r (inclusive) are different.
#
# Signature:
#   class RangeUnique:
#       def __init__(self, a: list)
#       def unique(self, l: int, r: int) -> bool
#
# FOLLOW-UPS (answered below, in this order):
# 1. The array changes between queries.
#
# INTERVIEW NOTES
# A range [l, r] has a duplicate exactly when some index i
# in it has its next equal element at an index <= r. Let
# next[i] be the next index holding the same value and
# reach[l] the smallest next[i] over all i >= l. Then the
# range is all-unique iff reach[l] > r, a single
# comparison per query after one linear pass.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

INF = float("inf")


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
def unique_with_set(a, l, r):
    # the easy version (say it first): put the range in a set
    seen = set()
    for i in range(l, r + 1):
        seen.add(a[i])
    return len(seen) == r - l + 1
```

```
def longest_unique_window(a):
    # stage 2: a sliding window with a count of every value in it
    count = {}
    left = 0
    best = 0
    for right in range(len(a)):
        count[a[right]] = count.get(a[right], 0) + 1
        while count[a[right]] > 1:              # shrink until the new value occurs only once
            count[a[left]] -= 1
            left += 1
        if right - left + 1 > best:
            best = right - left + 1
    return best


# ---- main question: main function ----
# Idea: for every index i find next[i], the next index holding the same
# value (infinity if none): scan from the right with a dictionary of the
# latest index of every value. Then reach[i] = the earliest "next repeat"
# among indices i, i+1, ... (a minimum from the right). The range [l, r]
# has no duplicate exactly when reach[l] > r. Build O(n), query O(1).
# The array can also change (follow-up 1): the simple answer is to rebuild
# in O(n) when reads dominate.
# Complexity: Time build O(n), each query O(1), update O(n) (it rebuilds), Space O(n)
class RangeUnique:
    def __init__(self, a):
        self.a = list(a)
        self.rebuild()

    def rebuild(self):
        a = self.a
        n = len(a)
        next_same = [INF] * n
        latest = {}
        for i in range(n - 1, -1, -1):
            if a[i] in latest:
                next_same[i] = latest[a[i]]
            latest[a[i]] = i
        self.reach = [INF] * (n + 1)
        for i in range(n - 1, -1, -1):
            if next_same[i] < self.reach[i + 1]:
                self.reach[i] = next_same[i]
            else:
                self.reach[i] = self.reach[i + 1]
```

```
    def unique(self, l, r):
        return self.reach[l] > r

    def update(self, i, value):
        # follow-up 1: reads dominate, so a linear rebuild is fine
        self.a[i] = value
        self.rebuild()


# ---- main question: tests ----
def test_main_question():
```

```
    a = [1, 2, 3, 1, 4]
    q = RangeUnique(a)
    for l in range(len(a)):
        for r in range(l, len(a)):
            assert q.unique(l, r) == unique_with_set(a, l, r)
    assert q.unique(0, 2) and not q.unique(0, 3)
    assert longest_unique_window(a) == 4


# ======================================================
# FOLLOW-UP 1: The array changes between queries
# ======================================================
# Answer: RangeUnique.update changes the value and rebuilds the tables in
# O(n), which is fine when queries are much more common than updates. For
# many updates use a segment tree over "next repeat" with point updates.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    q = RangeUnique([1, 2, 3, 1, 4])
    q.update(3, 9)                                  # an update can remove a duplicate ...
    assert q.unique(0, 4)
    q.update(0, 9)                                  # ... or create one
    assert not q.unique(0, 3)


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

PREPARATION

Precompute, for each index i, the previous index holding the same value (prev[i]) with a single hashmap pass.
Turn that into an O(1) query: a range [l, r] is duplicate-free exactly when every prev[j] < l for j in the range — precompute a running "furthest start that keeps [start, j] unique" array so each query is one comparison.
Explain the cost of an update to one element (which prev entries change) and why it stays linear.

STUDY PATH: question 83 of 197 | topic 15 of 43: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) | 2 of 5

Template fit: events by height with a running width; accumulate area strip by strip until half the total, then solve inside that strip | * asked 138 days ago *

MEDIUM | PracHub | Technical Screen | 83 of 197

Find a Horizontal Cut That Bisects Rectangle Area

POSTED 2026-06-04 | 138 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 8.1** **30 DAYS 0** **PRIORITY 193/288**

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #850 Rectangle Area II

LeetCode match: No direct match; related LC 850 Rectangle Area II (inferred from overview)

Posted on PracHub: 2026-06-04 (updated 2026-08-07)

Original: https://prachub.com/coding-questions/find-a-horizontal-cut-that-bisects-rectangle-area

QUESTION

Given many disjoint axis-aligned rectangles (x1, y1, x2, y2), return the y of the horizontal line that splits their combined area in half, so the area below the line equals the area above it. Return an exact fraction (no floating-point error).

Function signature

def bisecting_cut(rects: list) -> Fraction

EXAMPLES

Example 1	
Input	`[(0, 0, 2, 2)]`
Output	`1`
Why	`One 2x2 square: the middle line is y = 1.`

Example 2	
Input	`[(0, 0, 1, 1), (0, 2, 1, 4)]`
Output	`5/2`
Why	`Areas 1 and 2, total 3, so half is 1.5: all of the first rectangle (area 1) plus 0.5 of the second, which starts at y = 2: y = 5/2.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(rects):
    total = sum(area(r) for r in rects)
    lo = lowest_edge(rects)
    hi = highest_edge(rects)
    for _ in range(100):
        mid = (lo + hi) / 2
        if area_below(rects, mid) >= total / 2:
            hi = mid
        else:
            lo = mid
    return hi
```

BRUTE FORCE

TIME	SPACE
O(R * iterations)	O(1)

WHY IT IS SLOW

Each step sums over all rectangles and the answer is only as precise as floating point allows. A sweep over sorted edges with exact fractions finds the slab and the cut directly in O(R log R).

OPTIMAL SOLUTION · Sweep the y events and interpolate inside one slab

TIME	SPACE
O(R log R)	O(R)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Rectangles may overlap (LeetCode 850).

The slope is then the union width of the rectangles crossing a height, not the sum. Maintain it with a segment tree over compressed x coordinates (cover counts), O(R log R) overall.

FOLLOW-UP 2 Coordinates up to 10^9 and 200,000 rectangles.

Use integers or Fractions (areas reach 10^18, which fits in Python ints but not in 32-bit). The sweep is O(R log R); never use floats for the comparisons that decide the slab.

FOLLOW-UP 3 A vertical cut instead, or a cut at a given ratio.

Swap the axes for a vertical cut. For a ratio p, replace half by p * total in the slab test; the interpolation formula is unchanged.

FOLLOW-UP 4 A cut that leaves a given fraction of the area below it, not exactly half.

Replace 'half of the total' by p * total in the slab test; the interpolation formula inside the slab is the same. The code returns an exact Fraction and is checked by computing the area below the returned cut.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from fractions import Fraction


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
# Rectangles are disjoint and written (x1, y1, x2, y2).
def total_area(rects):
```

```
    total = 0
    for x1, y1, x2, y2 in rects:
        total += (x2 - x1) * (y2 - y1)
    return total


def area_below(rects, y):
    # the exact area under the horizontal line at height y (used to check the answer)
    total = Fraction(0)
    for x1, y1, x2, y2 in rects:
        top = min(Fraction(y), y2)
        if top > y1:
            total += (x2 - x1) * (top - y1)
    return total


# ---- main question: main function ----
# Find the height of a horizontal line with exactly half of the total
# area below it.
# Idea: sweep upward through the heights where a rectangle starts (its
# width is added) or ends (its width is removed). Between two neighbouring
# event heights the covered width is constant, so the area grows linearly
# there. Find the strip where the running area reaches the wanted amount
# and interpolate inside it. `share` is the fraction of the area that
# must be below the cut (follow-up 4); exact Fractions are used, never floats.
# Complexity: Time O(R log R) for R rectangles (the sort of heights), Space O(R)
def cut_height(rects, share=Fraction(1, 2)):
    wanted = share * total_area(rects)
    width_change = {}                                   # height -> width gained when the sweep passes it
```

```
    for x1, y1, x2, y2 in rects:
        width_change[y1] = width_change.get(y1, 0) + (x2 - x1)
        width_change[y2] = width_change.get(y2, 0) - (x2 - x1)
    heights = sorted(width_change)
    width = 0
    so_far = Fraction(0)
    for i in range(len(heights) - 1):
        low = heights[i]
        high = heights[i + 1]
        width += width_change[low]                      # inside this strip the covered width is constant
        gain = width * (high - low)
        if width > 0 and so_far + gain >= wanted:
            return low + (wanted - so_far) / width      # the cut is inside this strip: interpolate
        so_far += gain
    return Fraction(heights[len(heights) - 1])


# ---- main question: tests ----
def test_main_question():
    stacked = [(0, 0, 2, 1), (0, 1, 2, 2)]
    assert cut_height(stacked) == 1
    assert cut_height([(0, 0, 1, 4)]) == 2                          # one tall rectangle: halfway up
    assert cut_height([(0, 0, 1, 1), (0, 5, 1, 6)]) == 1            # a gap between rectangles: the cut sits a
t the first top
    odd = [(0, 0, 3, 1), (0, 1, 1, 2)]                              # the cut falls inside a strip at a fracti
onal height
    assert area_below(odd, cut_height(odd)) * 2 == total_area(odd)
    big = [(0, 0, 10 ** 9, 10 ** 9)]                                # large coordinates stay exact
    assert cut_height(big) == Fraction(10 ** 9, 2)


# ======================================================
# FOLLOW-UP 4: A cut that leaves a given fraction of the area below it, not exactly half
# ======================================================
# Answer: replace "half of the total" by share * total in the strip test
```

```
# (the share parameter); the interpolation inside the strip is the same.
# ---- follow-up 4: tests ----
def test_follow_up_4():
    stacked = [(0, 0, 2, 1), (0, 1, 2, 2)]
    quarter = cut_height(stacked, Fraction(1, 4))
    assert area_below(stacked, quarter) * 4 == total_area(stacked)


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def horizontal_bisector(rectangles: list[list[float]]) -> float:
```

Sample call

```
horizontal_bisector([[0, 0, 4, 2]])
```

Test cases

Input	Expected
`([[0, 0, 4, 2]],)`	`1.0`
`([[0, 0, 2, 4], [10, 1, 2, 2]],)`	`2.0`

ALSO REPORTED · 1 more

Find where to cut cake into equal areas | PracHub | Technical Screen

https://prachub.com/coding-questions/find-where-to-cut-cake-into-equal-areas

STUDY PATH: question 84 of 197 | topic 15 of 43: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) | 3 of 5

Template fit: prefix sums plus first/last position maps: look up the value whose removal would balance both sides | * asked 250 days ago *

MEDIUM | PracHub | Technical Screen | 84 of 197

Check pivot after removing one element

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 33.2 | 30 DAYS 0 | PRIORITY 133/288

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #724 Find Pivot Index

LeetCode match: LC 724 Find Pivot Index (variant: one removal) (inferred from overview)

Posted on PracHub: 2026-02-12 (updated 2026-05-04)

Original: https://prachub.com/coding-questions/check-pivot-after-removing-one-element

QUESTION

A pivot index of an array is an index whose left-side sum equals its right-side sum (the pivot element itself is excluded). Remove exactly one element from the array, then ask whether the remaining array has a pivot index. Return one pair (removed index, pivot index), both as indexes in the original array, or None if no removal creates a pivot. Negative numbers and duplicates are allowed.

Function signature

def pivot_after_removal(a: list) -> tuple | None

EXAMPLES

Example 1	
Input	`[2, 1, -1, 3]`
Output	`(0, 3)`
Why	`Removing the 2 at index 0 leaves [1, -1, 3]; the 3 (original index 3) has left sum 1 + (-1) = 0 and nothing on its right.`

Example 2	
Input	`[1, 2, 3]`
Output	`null`
Why	`Whichever element is removed, no pivot appears.`

Example 3	
Input	`[0, 0]`
Output	`(0, 1)`
Why	`Remove either 0: the other one is a pivot of an array with one element.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    for r in range(len(a)):
        b = a[:r] + a[r + 1:]
        for i in range(len(b)):
            if sum(b[:i]) == sum(b[i + 1:]):
                return True
    return False
```

TIME	SPACE
O(n^3)	**O(n)**
O(n^2) with prefix sums	

WHY IT IS SLOW

n removals times n pivots times a sum: O(n^3) (O(n^2) with prefix sums). For each pivot the value to remove is determined by prefix sums, so a dictionary lookup makes it O(n).

OPTIMAL SOLUTION · Closed-form value to remove, found in a dictionary of positions

TIME	SPACE
O(n)	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Without any removal (LeetCode 724).

One pass: the pivot is the index where left sum equals total minus left minus the element. O(n), O(1).

FOLLOW-UP 2 At most one removal (removal optional).

Answer true if pivot_index finds a pivot, otherwise run the removal test above.

FOLLOW-UP 3 Return which element to remove and the pivot.

Record i together with the index of the matching value (first or last occurrence) when the lookup succeeds, instead of returning True.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Check pivot after removing one element
#
# A pivot index of an array is an index whose left-side
# sum equals its right-side sum (the pivot element itself
# is excluded). Remove exactly one element from the array,
# then ask whether the remaining array has a pivot index.
# Return one pair (removed index, pivot index), both as
# indexes in the original array, or None if no removal
# creates a pivot. Negative numbers and duplicates are
# allowed.
#
```

```
# Signature:
#   def pivot_after_removal(a: list) -> tuple | None
#
# Examples:
# [2, 1, -1, 3]  ->  (0, 3)
# [1, 2, 3]  ->  null
#
# FOLLOW-UPS (answered below, in this order):
# 1. Without any removal (LeetCode 724).
# 2. At most one removal (removal optional).
# 3. Return which element to remove and the pivot.
#
# INTERVIEW NOTES
# Use prefix sums. For a pivot at original index i and a
# removed element on one side, the balance equation fixes
# the value that has to be removed: it is a closed form in
# the prefix sum, the total and a[i]. So for every
# candidate pivot you only have to ask 'does that value
# occur on the correct side?', which a dictionary of first
# and last positions answers in O(1).

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Remove ONE element of the array, then is there a pivot: an element
# whose left sum equals its right sum? Return (removed index, pivot index)
```

```
# or None.
# Idea: let `prefix[j]` be the sum before index j and key[j] =
# 2 * prefix[j] + a[j].
#  - Pivot j AFTER the removed index i: the left sum is prefix[j] - a[i]
#    and the right sum is total - prefix[j] - a[j]; equal means
#    a[i] = key[j] - total. Scan j from the left, remembering the earlier values.
#  - Pivot j BEFORE the removed index i: left = prefix[j], right = total -
#    prefix[j] - a[j] - a[i]; equal means a[i] = total - key[j]. Scan j
#    from the right, remembering the later values.
# Two scans with a dictionary (value -> index): O(n).
# Complexity: Time O(n), Space O(n)
def pivot_after_removal(a):
    n = len(a)
    total = 0
    for x in a:
        total += x
    prefix = [0]
    for x in a:
        prefix.append(prefix[len(prefix) - 1] + x)
    key = []
    for j in range(n):
        key.append(2 * prefix[j] + a[j])
    index_of = {}                                   # a value seen EARLIER -> its index
    for j in range(n):
        i = index_of.get(key[j] - total)
        if i is not None:
            return i, j
        index_of[a[j]] = j
    index_of = {}                                   # a value seen LATER -> its index
    for j in range(n - 1, -1, -1):
```

```
        i = index_of.get(total - key[j])
        if i is not None:
            return i, j
        index_of[a[j]] = j
    return None


# ---- main question: tests ----
def test_main_question():
    # [1,7,3,6,5,6] has a pivot at index 3 without removal, but removing one element never gives another
    assert pivot_after_removal([1, 7, 3, 6, 5, 6]) is None
    assert pivot_after_removal([1, 1, 1, 1]) == (1, 2)     # remove index 1: [1, 1, 1] has the middle pivot


# ======================================================
```

```
# FOLLOW-UP 1: Without any removal (LeetCode 724)
# ======================================================
# ---- follow-up 1: function ----
# Answer: one pass: the pivot is the index where the left sum equals the
# total minus the left sum minus the element itself. O(n) time, O(1) space.
# Complexity: Time O(n), Space O(1)
def pivot_index(a):
    total = 0
    for x in a:
        total += x
    left = 0
    for i in range(len(a)):
        if left == total - left - a[i]:
            return i
        left += a[i]
    return -1


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert pivot_index([1, 1, 1, 1]) == -1
    assert pivot_index([1, 7, 3, 6, 5, 6]) == 3
    assert pivot_index([1, 2, 3]) == -1


# ======================================================
# FOLLOW-UP 2: At most one removal (removal optional)
# ======================================================
# ---- follow-up 2: function ----
# Answer: true if pivot_index finds a pivot; otherwise run the removal test.
# Complexity: Time O(n), Space O(n)
def has_pivot(a, allow_removal=True):
    if pivot_index(a) != -1:
        return True
    if not allow_removal:
        return False
    return pivot_after_removal(a) is not None


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert not has_pivot([1, 2, 3], allow_removal=False)
    assert has_pivot([1, 1, 1, 1])
    assert not has_pivot([1, 1, 1, 1], allow_removal=False)
```

```


# ======================================================
# FOLLOW-UP 3: Return which element to remove and the pivot
# ======================================================
```

```
# Answer: pivot_after_removal already returns (removed index, pivot index).


# ==========================================================
# RUN EVERYTHING
# ==========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(nums: list[int]) -> bool:
```

Sample call

```
solution([1, 2])
```

Test cases

Input	Expected
`([1, 2],)`	`True`
`([2, 1, 1, 2],)`	`True`

STUDY PATH: question 85 of 197 | topic 15 of 43: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) | 4 of 5

Template fit: prefix sums with first/last position maps: find two cut points that leave equal halves around one removed link | * asked 182 days ago * | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 85 of 197

Gold Chain Split into Two Equal Halves

ASKED 2026-04-21 | 182 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 261/288

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-04-21 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/gold-chain-split-equal

QUESTION

A gold chain is an array of segment weights. Remove exactly one segment, reconnect the rest, then cut the chain once so that the two parts have equal total weight. Return whether this is possible for some choice of the removed segment and the cut.

Function signature

def gold_chain(a: list) -> bool

EXAMPLES

Example 1	
Input	`[1, 2, 3, 4]`
Output	`true`
Why	`Remove the 4: [1, 2, 3] splits as [1, 2] and [3], both 3.`

Example 2	
Input	`[1, 1, 5]`
Output	`true`
Why	`Remove the 5: [1, 1] splits into 1 and 1.`

Example 3	
Input	`[1, 2, 4]`
Output	`false`
Why	`Removing 1 leaves 6 (halves of 3 need [2] | [4]: no), removing 2 leaves 5, removing 4 leaves 3: nothing works.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(w):
    for r in range(len(w)):
        chain = w[:r] + w[r + 1:]
        half, run = sum(chain) / 2, 0
        for x in chain:
            run += x
            if run == half:
                return True
    return False
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

n removals, each rebuilt and scanned: O(n^2). Prefix sums with a lookup of the needed prefix value answer each removal in O(1).

OPTIMAL SOLUTION · Prefix sums with first and last occurrence maps

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 List every valid (removed segment, cut position) pair.

The answer itself can be quadratic, so a simple loop is acceptable. With positive weights prefix sums are strictly increasing, so each target has at most one cut position and the listing is O(n) after the hash lookups; with zeros or negatives a value can repeat, so keep a list of indices per prefix value.

FOLLOW-UP 2 The chain is a circle (cut anywhere).

Then a cut is two cut points on the circle and the removed segment just shortens it; use the circular window idea (duplicate the array and slide a window whose sum is half).

FOLLOW-UP 3 Remove k segments instead of one.

The remaining total depends on which k you remove, which becomes a subset-sum style search; ask for the limits before promising a fast solution.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Gold Chain Split into Two Equal Halves
#
# A gold chain is an array of segment weights. Remove
# exactly one segment, reconnect the rest, then cut the
# chain once so that the two parts have equal total
# weight. Return whether this is possible for some choice
# of the removed segment and the cut.
#
```

```
# Signature:
#   def gold_chain(a: list) -> bool
#
# Examples:
# [1, 2, 3, 4]  ->  true
# [1, 1, 5]  ->  true
#
# FOLLOW-UPS (answered below, in this order):
# 1. List every valid (removed segment, cut position)
# pair.
# 2. The chain is a circle (cut anywhere).
# 3. Remove k segments instead of one.
#
# INTERVIEW NOTES
# After removing segment r the target half is (S - a[r]) /
# 2, and a cut means some prefix of the remaining chain
# has exactly that weight. A prefix of the remaining chain
# is either an ordinary prefix of the original array
# (stops before r) or an original prefix minus a[r]
# (passes over r). Each is one value to look up in a hash
# of prefix sums, so every r costs O(1).

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def prefix_sums(a):
    # P[j] = the sum of the first j segments
    P = [0]
    for x in a:
        P.append(P[len(P) - 1] + x)
    return P


# ---- main question: main function ----
# A chain of gold segments with weights a. Remove exactly ONE segment,
# then cut the rest once into two non-empty parts of equal weight. Can it
# be done?
# Idea: with prefix sums P (P[j] = the first j segments) removing segment r
# leaves total - a[r]; each half must weigh half of that. A cut BEFORE the
# removed segment (position j <= r) needs P[j] == half. A cut AFTER it
# (j > r) needs P[j] - a[r] == half. Instead of trying every cut, look up the
# FIRST position with a given prefix sum (for cuts before r) and the LAST
# position (for cuts after r) in two dictionaries: O(n) in total.
# Complexity: Time O(n), Space O(n)
def gold_chain(a):
    n = len(a)
    if n < 3:
        return False
    P = prefix_sums(a)
    total = P[n]
    first = {}                              # prefix sum -> its first cut position
    last = {}                               # prefix sum -> its last cut position
    for j in range(1, n):                   # cuts sit between segments: positions 1 .. n - 1
        if P[j] not in first:
            first[P[j]] = j
        last[P[j]] = j
```

```
    for r in range(n):
        left_over = total - a[r]
        if left_over % 2 == 1:
            continue
        half = left_over // 2
        latest_cut_before = r               # the cut must leave a piece on the right, so j <= n - 2
        if n - 2 < latest_cut_before:
            latest_cut_before = n - 2
        if half in first and first[half] <= latest_cut_before:
            return True                     # cut before the removed segment
        if half + a[r] in last and last[half + a[r]] > r:
            return True                     # cut after it
    return False


# ---- main question: tests ----
def test_main_question():
    assert gold_chain([5, 1, 5])                # remove the middle 1, then cut 5 | 5
```

```
    assert gold_chain([2, 7, 2])
    assert not gold_chain([1, 2, 3])
    assert not gold_chain([1, 1])
    assert not gold_chain([1, 2, 4])


# ========================================================
# FOLLOW-UP 1: List every valid (removed segment, cut position) pair
# ========================================================
# ---- follow-up 1: function ----
# Answer: the answer itself can have quadratic size, so a plain double
# loop is fine: for each removed segment, walk along the rest and
# report every cut where the left part is half the total.
# Complexity: Time O(n^2), Space O(n)
def valid_pairs(a):
    pairs = []
    for r in range(len(a)):
        rest = a[:r] + a[r + 1:]
        total = 0
        for x in rest:
            total += x
        running = 0
        for cut in range(1, len(rest)):         # cut after `cut` segments of the reconnected chain
            running += rest[cut - 1]
            if running * 2 == total:
                pairs.append((r, cut))
    return pairs


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert (1, 1) in valid_pairs([1, 2, 1])
    for code in range(4 ** 5):                  # every chain of 5 segments with weights 1..4
        chain = []
        for i in range(5):
            chain.append(code // 4 ** i % 4 + 1)
        assert gold_chain(chain) == (len(valid_pairs(chain)) > 0)


# ========================================================
# FOLLOW-UP 2: The chain is a circle (cut anywhere)
# ========================================================
# Answer: a circle has no "before" and "after" the removed segment: after
# removing it, the pieces form a line again starting just after it, so
# rotate the array so the removed segment is last and reuse the same test.
```

```
# ============================================================
# FOLLOW-UP 3: Remove k segments instead of one
# ============================================================
# Answer: with prefix sums, removing a block of k neighbouring segments
# is a range sum, so the same lookups work for each block; removing k
# separate segments is a subset problem (much harder).


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill The equal-subset-sum and partition-to-k-equal-sum-subsets problems are the standard drill partners.
Practice enumerating all valid cut positions, not just one — interviewers commonly add the follow-up after the boolean version works.
Watch the boundary: remove + cut should not yield an empty side.

STUDY PATH: question 86 of 197 | topic 15 of 43: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) | 5 of 5

Template fit: positions of the ones; the answer multiplies the gap sizes around the k-th and 2k-th one (zero ones: choose two cut points) | ND

MEDIUM | Exponent | 86 of 197

Count the number of ways to split a string.

ASKED (ANY CO.) ~3 YEARS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 208/288

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1573 Number of Ways to Split a String (likely) (inferred from short statement)

Last asked (any company): 3 years ago, about 2023-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/3256/count-ways-to-split-string

QUESTION

A binary string s is split into three non-empty parts by two cuts. Count the ways in which the three parts contain the same number of 1s, and return the count modulo 1,000,000,007.

Function signature

def numWays(s: str) -> int

EXAMPLES

Example 1	
Input	`'10101'`
Output	`4`
Why	`Four ways to split '10101' so each part has one 1 (for example 1|010|1 and 10|10|1).`

Example 2	
Input	`'1001'`
Output	`0`
Why	`Two 1s cannot be divided into three equal groups.`

Example 3	
Input	`'0000'`
Output	`3`
Why	`No 1s: any two cuts work: choose 2 of the 3 gaps = 3 ways.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(s):
    n, count = len(s), 0
    for a in range(1, n - 1):
        for b in range(a + 1, n):
            parts = (s[:a], s[a:b], s[b:])
            ones = {p.count("1") for p in parts}
            if len(ones) == 1:
                count += 1
    return count
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

O(n^2) pairs. The cuts only depend on where the ones are, so the answer is a product of two gap sizes, computed in O(n).

OPTIMAL SOLUTION · Count the ones, multiply the gap sizes

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Split into m parts with equal ones.

The same argument: each of the m - 1 cut points sits in the gap after a multiple of k ones, so multiply the gap sizes; with no ones at all it is C(n - 1, m - 1).

FOLLOW-UP 2 Count ways to split into palindromes.

That is a different problem: dp[i] counts splits of the prefix, adding dp[j] for every palindromic s[j:i]; precompute the palindromic substrings with expand-around-center or a DP table in O(n^2).

FOLLOW-UP 3 The numbers get huge.

Return the answer modulo 10^9 + 7 and apply the modulus after every multiplication; in languages with fixed-width integers use 64-bit intermediates. **Cost:** O(n) with modular arithmetic; no big integers are needed.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count the number of ways to split a
# string.
#
# A binary string s is split into three non-empty parts by
# two cuts. Count the ways in which the three parts
# contain the same number of 1s, and return the count
# modulo 1,000,000,007.
#
# Signature:
#   def numWays(s: str) -> int
#
```

```
# Examples:
# '10101'  ->  4
# '1001'  ->  0
#
# FOLLOW-UPS (answered below, in this order):
# 1. Split into m parts with equal ones.
# 2. Count ways to split into palindromes.
# 3. The numbers get huge.
#
# INTERVIEW NOTES
# Ask which split question it is before solving. For
# 'equal ones' the answer depends only on where the ones
# are: the cuts have to fall inside the stretches of zeros
# that separate the groups of ones, so the count is a
# product of (zeros in the gap + 1). All-zero strings are
# a special case (choose any two cut points).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

MOD = 10 ** 9 + 7


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def choose(n, k):
    # the number of ways to pick k things out of n
    ways = 1
```

```
    for i in range(k):
        ways = ways * (n - i) // (i + 1)
    return ways


# ---- main question: main function ----
# Cut a binary string into `parts` non-empty pieces (3 in the classic
# question, LeetCode 1573) so that every piece has the same number of 1s.
# Count the ways.
# Idea: let `ones` be the positions of the 1s. If their number is not
# divisible by `parts` the answer is 0. With no 1s at all, any choice of
# parts - 1 cut positions among the n - 1 gaps works: choose(n - 1, parts - 1).
# Otherwise each piece holds `each` ones, and the cut after piece k can go
# anywhere in the run of zeros between the last 1 of that piece and the first
# 1 of the next: (the distance between those two 1s) choices. Multiply the
# choices of all cuts. (parts = m is follow-up 1.)
# For huge answers the result is kept modulo 10^9 + 7 after every
# multiplication (follow-up 3); mod=None gives the exact number.
# Complexity: Time O(n), Space O(n)
def numWays(s, parts=3, mod=MOD):
    n = len(s)
    if n < parts:
        return 0
    ones = []
    for i in range(n):
        if s[i] == "1":
            ones.append(i)
    if len(ones) % parts != 0:
        return 0
    if len(ones) == 0:
```

```
        total = choose(n - 1, parts - 1)
        if mod:
            return total % mod
        return total
    each = len(ones) // parts
    answer = 1
    for cut in range(1, parts):
        gap = ones[cut * each] - ones[cut * each - 1]      # where the cut may go
        answer *= gap
        if mod:
            answer %= mod
    return answer


# ---- main question: tests ----
```

```
def test_main_question():
    assert numWays("10101") == 4
    assert numWays("1001") == 0
    assert numWays("0000") == 3
    assert numWays("100100010100110") == 12
    assert numWays("", parts=3) == 0


# =========================================================
# FOLLOW-UP 1: Split into m parts with equal ones
# =========================================================
# Answer: the same argument with `parts` = m: each of the m - 1 cuts sits in
# the gap after a multiple of k ones; multiply the gap sizes. With no ones it
# is choose(n - 1, m - 1).
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert numWays("1100", parts=2) == 1
    assert numWays("0000", parts=2) == 3


# =========================================================
# FOLLOW-UP 2: Count ways to split into palindromes
# =========================================================
# ---- follow-up 2: function ----
# Answer: a different problem. dp[i] = the number of ways to cut the first
# i letters; add dp[j] for every palindrome s[j:i]. First build a table
# is_pal[i][j] ("s[i..j] is a palindrome") in O(n^2).
# Complexity: Time O(n^2), Space O(n^2)
def count_palindrome_splits(s):
    n = len(s)
    is_pal = []
    for i in range(n):
        is_pal.append([False] * n)
    for i in range(n - 1, -1, -1):
        for j in range(i, n):
            inside_ok = j - i < 2 or is_pal[i + 1][j - 1]
            is_pal[i][j] = s[i] == s[j] and inside_ok
    dp = [1] + [0] * n
    for i in range(1, n + 1):
        for j in range(i):
            if is_pal[j][i - 1]:
                dp[i] += dp[j]
    return dp[n]
```

```
# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert count_palindrome_splits("aab") == 2
    assert count_palindrome_splits("aaa") == 4
```

```
    assert count_palindrome_splits("") == 1


# =====================================================
# FOLLOW-UP 3: The numbers get huge
# =====================================================
# Answer: return the answer modulo 10^9 + 7 and reduce after every
# multiplication (the `mod` parameter); with fixed-width integers use
# 64-bit intermediate values. No big integers are needed: O(n).
# ---- follow-up 3: tests ----
def test_follow_up_3():
    big = "0" * 5000
    assert numWays(big) == choose(4999, 2) % MOD
    assert numWays(big, mod=None) == choose(4999, 2)


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 16 OF 43

Sorted intervals: merge while they touch, then read the gaps or search

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

5
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Sorted intervals: merge while they touch, then read the gaps or search | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M 87. Find Containing Range — PracHub * 2026-06-04 *

M 88. Union of Sorted Interval Lists — 1Point3Acres * 2026-08-16 *

M 89. Range Module (Insert and Point Query) — ColorStack * 2026-01-20 *

M 90. Meetings Minus a Do-Not-Schedule Block — ColorStack * 2025-10-03 *

M 91. Merge Intervals — ND

CODE TEMPLATE | GROUP 16 OF 43 | 5 QUESTIONS, ONE PATTERN

Sorted intervals: merge while they touch, then read the gaps or search

The one idea these questions share

Sort by start. Walk once: if the next interval starts before the last one ends, extend the last; otherwise start a new one. The merged list is sorted and disjoint, so gaps are the spaces between neighbours and any point can be found with bisect.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def merge(intervals):
    out = []
    for s, e in sorted(intervals):
        if out and s <= out[-1][1]:            # CHANGE 1: overlap or touch? (< to keep touching ones apart)
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

2. Helper functions that usually get written next to it

```
from bisect import bisect_left, bisect_right


def gaps(merged, lo=float("-inf"), hi=float("inf")):
    edges = [lo] + [x for iv in merged for x in iv] + [hi]
    return [(edges[i], edges[i + 1]) for i in range(0, len(edges), 2)]


def containing(merged, x):
    starts = [s for s, _ in merged]
    i = bisect_right(starts, x) - 1
    return tuple(merged[i]) if i >= 0 and merged[i][1] >= x else None
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
87	Find Containing Range	merge once, keep the starts in a list, and bisect to find the one interval that could contain x
88	Union of Sorted Interval Lists	walk both sorted lists with two indices, always taking the smaller start next, then the same merge test
89	Range Module (Insert and Point Query)	keep sorted lefts and rights; adding bisects to the first interval that touches and the first that starts after, and replaces that block by one merged interval
90	Meetings Minus a Do-Not-Schedule Block	merge first, then for each merged meeting keep the part before the blocked window and the part after it
91	Merge Intervals	the template as written

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 87 of 197 | topic 16 of 43: Sorted intervals: merge while they touch, then read the gaps or search | 1 of 5

Template fit: merge once, keep the starts in a list, and bisect to find the one interval that could contain x | * asked 138 days ago *

MEDIUM | PracHub | Technical Screen | 87 of 197

★ TOP 75: rank 16 in the Medium and Hard priority list

★ Find Containing Range

POSTED 2026-06-04 | 138 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE **ALL TIME 65.2** **30 DAYS 41.1** **PRIORITY 15/288**

ABOUT THIS QUESTION

Topic: Sorted intervals: merge while they touch, then read the gaps or search

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #56 Merge Intervals

LeetCode match: No direct match; related LC 56 Merge Intervals + binary search (inferred from overview)

Posted on PracHub: 2026-06-04 (updated 2026-06-06)

Original: https://prachub.com/coding-questions/find-containing-range

QUESTION

Given a collection of inclusive ranges [start, end] (overlapping or touching ranges are merged), find(x) returns the merged range that contains the number x, or None when no range contains it.

Function signature

class RangeIndex:
  def __init__(self, ranges: list)
  def find(self, x: int) -> tuple | None

EXAMPLES

Example 1

Input	`r = RangeIndex([(1, 3), (2, 6), (8, 10)])`
	`r.find(5)`
	`r.find(7)`
	`r.find(10)`

Output	`r.find(5) -> (1, 6)`
	`r.find(7) -> null`
	`r.find(10) -> (8, 10)`

Why	`[1, 3] and [2, 6] overlap and merge into [1, 6]; 7 falls in the gap between the merged ranges.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def find_containing(ranges, x):
    for start, end in ranges:
        if start <= x <= end:
            return (start, end)
    return None
```

BRUTE FORCE

TIME	SPACE
O(n) per query	O(n)

WHY IT IS SLOW

Every query walks all n ranges. Merging once (O(n log n)) and binary searching the sorted starts makes each query O(log n).

OPTIMAL SOLUTION · Merge once, then binary search the starts

TIME	SPACE
O(n log n) to build, O(log n) per query	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return every original range that contains x, not the merged one.

Sort by start and consider only ranges starting at or before x (one bisect); among those, keep the ones whose end is at least x. With many overlaps use an interval tree for O(log n + k) per query.

FOLLOW-UP 2 Ranges are inserted and deleted over time.

Keep the disjoint ranges in a sorted container (balanced tree); an insert merges neighbours, and a query is the same predecessor search. O(log n) per operation.

FOLLOW-UP 3 Queries are given in bulk.

Sort the queries and sweep once over the merged ranges with two pointers: O((n + q) log) overall instead of a binary search per query.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find Containing Range
#
# Given a collection of inclusive ranges [start, end]
# (overlapping or touching ranges are merged), find(x)
# returns the merged range that contains the number x, or
# None when no range contains it.
#
# Signature:
#   class RangeIndex:
#       def __init__(self, ranges: list)
#       def find(self, x: int) -> tuple | None
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return every original range that contains x, not the
# merged one.
# 2. Ranges are inserted and deleted over time.
# 3. Queries are given in bulk.
#
# INTERVIEW NOTES
# Merge the ranges once so they are disjoint and sorted;
```

```
# then 'which range contains x' is 'the last range whose
# start is at most x, and does its end reach x'. That is
# one binary search per query. Be clear whether the answer
# is the original range or the merged one, since they
# differ when ranges overlap.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
# Complexity: Time O(log n), Space O(1)
def first_at_least(values, x):
    # binary search in a sorted list: the first index whose value is >= x
    low = 0
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] < x:
            low = middle + 1
        else:
            high = middle
    return low


# Complexity: Time O(log n), Space O(1)
def first_greater(values, x):
    # binary search in a sorted list: the first index whose value is > x
    low = 0
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] <= x:
            low = middle + 1
        else:
            high = middle
    return low


# ---- main question: main class ----
# Given ranges [start, end] (closed), find(x) returns the MERGED range that
# contains x, or None.
# Idea: merge the ranges once (insert them one by one; an insert replaces
# every range it touches by one merged range). After that the merged ranges
# are sorted and disjoint, so the only candidate for x is the range with the
# largest start <= x (one binary search), and it contains x if its end is at
# least x. The merging is paid once.
# Complexity: Time insert O(log n) to search plus O(n) to splice, find O(log n), find_original O(n), find_m
any O(Q log Q + n), Space O(n)
class RangeIndex:
    def __init__(self, ranges=()):
        self.starts = []
        self.ends = []
        self.original = sorted(ranges)
        for start, end in self.original:
            self.insert(start, end)

    def insert(self, start, end):
```

```
        # follow-up 2: ranges can be inserted over time
        i = first_at_least(self.ends, start)               # the first range ending at or after the new sta
rt ...
        j = first_greater(self.starts, end)                # ... up to the last range starting at or before
 the new end
        if i < j:
            if self.starts[i] < start:
                start = self.starts[i]
            if self.ends[j - 1] > end:
```

```
                end = self.ends[j - 1]
        self.starts[i:j] = [start]                         # replace everything the new range touches
        self.ends[i:j] = [end]

    def find(self, x):
        i = first_greater(self.starts, x) - 1
        if i >= 0 and self.ends[i] >= x:
            return (self.starts[i], self.ends[i])
        return None

    def find_original(self, x):
        # follow-up 1: every ORIGINAL range that contains x (only those starting at or before x can)
        found = []
        for start, end in self.original:
            if start > x:
                break
            if end >= x:
                found.append((start, end))
        return found

    def find_many(self, queries):
        # follow-up 3: queries in bulk: sort the queries and sweep once along the ranges
        order = []
        for i in range(len(queries)):
            order.append((queries[i], i))
        order.sort()
        answers = [None] * len(queries)
        j = 0
        for x, i in order:
            while j < len(self.ends) and self.ends[j] < x:
                j += 1                                     # skip ranges that end before x
            if j < len(self.starts) and self.starts[j] <= x:
                answers[i] = (self.starts[j], self.ends[j])
        return answers


# ---- main question: tests ----
def test_main_question():
    index = RangeIndex([(1, 3), (2, 6), (8, 10), (15, 18)])
    assert index.find(4) == (1, 6)
    assert index.find(7) is None
    assert index.find(8) == (8, 10)
    assert index.find(0) is None and index.find(99) is None
```

```
# ========================================================
# FOLLOW-UP 1: Return every original range that contains x, not the merged one
# ========================================================
# Answer: keep the original ranges sorted by start; only ranges starting at
# or before x can contain x, and among them keep those whose end is at least
# x (find_original). With many overlaps use an interval tree: O(log n + k)
# per query for k results.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    index = RangeIndex([(1, 3), (2, 6), (8, 10), (15, 18)])
```

```
    assert index.find_original(2) == [(1, 3), (2, 6)]


# ========================================================
# FOLLOW-UP 2: Ranges are inserted and deleted over time
# ========================================================
# Answer: insert() above splices a new range into the sorted lists
# (merging what it touches). Deleting needs the original ranges too (a
# merged range may have to split again), so keep a count per point of
# coverage, or an interval tree.
# ---- follow-up 2: tests ----
def test_follow_up_2():
    index = RangeIndex([(1, 3), (2, 6), (8, 10), (15, 18)])
    index.insert(6, 8)                                  # (6, 8) joins the first two blocks
    assert index.find(7) == (1, 10)
    assert index.find(12) is None


# ========================================================
# FOLLOW-UP 3: Queries are given in bulk
# ========================================================
# Answer: find_many sorts the queries and sweeps once along the ranges
# instead of one binary search per query: O((n + q) log q).
# ---- follow-up 3: tests ----
def test_follow_up_3():
    index = RangeIndex([(1, 3), (2, 6), (8, 10), (15, 18)])
    queries = [4, 20, 16, 9]
    expected = []
    for q in queries:
        expected.append(index.find(q))
    assert index.find_many(queries) == expected


# ========================================================
# RUN EVERYTHING
```

```
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(ranges: list[list[int]], queries: list[int]) -> list[list[int]]:
```

Sample call

```
solution([[1, 3], [50, 100]], [2, 50, 10])
```

Test cases

Input	Expected
`([[1, 3], [50, 100]], [2, 50, 10])`	`[[1, 3], [50, 100], None]`

Input	Expected
`([[5, 8], [-10, -5], [0, 0]], [-7, 0, 8, 9, -11])`	`[[-10, -5], [0, 0], [5, 8], None, None]`

STUDY PATH: question 88 of 197 | topic 16 of 43: Sorted intervals: merge while they touch, then read the gaps or search | 2 of 5

Template fit: walk both sorted lists with two indices, always taking the smaller start next, then the same merge test | * asked 65 days ago * | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 88 of 197

Union of Sorted Interval Lists

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK **ALL TIME 25.5** **30 DAYS 0** **PRIORITY 160/288**

ABOUT THIS QUESTION

Topic: Sorted intervals: merge while they touch, then read the gaps or search

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #986 Interval List Intersections

LeetCode match: LC 986 Interval List Intersections (related)

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/sorted-interval-list-union

QUESTION

Two lists of intervals are each sorted and non-overlapping on their own. Return the union of both lists as sorted, non-overlapping intervals. Intervals that touch, such as (1, 3) and (3, 5), are merged unless touching=False.

Function signature

def union_two(a: list, b: list, touching=True) -> list

EXAMPLES

Example 1	
Input	`[(1, 3), (6, 9)], [(2, 5), (10, 12)]`
Output	`[(1, 5), (6, 9), (10, 12)]`
Why	`(1, 3) and (2, 5) overlap and become (1, 5); (6, 9) and (10, 12) stay.`

Example 2	
Input	`[(1, 3)], [(3, 5)]`
Output	`[(1, 5)]`
Why	`(1, 3) and (3, 5) touch at 3, so they merge.`

Example 3	
Input	`[(1, 3)], [(3, 5)], touching = false`
Output	`[(1, 3), (3, 5)]`
Why	`With touching=False they only merge when they truly overlap.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(list1, list2):
    merged = []
    for s, e in sorted(list1 + list2):
        if merged and s <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], e)
        else:
            merged.append([s, e])
    return merged
```

TIME	SPACE
O((m + n) log (m + n))	O(m + n)

WHY IT IS SLOW

Sorting ignores that both lists are already sorted: O((m + n) log (m + n)) instead of O(m + n) for a two-pointer merge.

OPTIMAL SOLUTION · Two-pointer merge

TIME	SPACE
O(m + n)	O(m + n) for the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if the lists are too big for memory?

Read each list through an iterator (a file or a paged API). union_k only ever holds one interval per list plus the current merged interval, so memory is O(K) and it can emit results as it goes. Sorted order is what makes streaming possible.

FOLLOW-UP 2 What if one list may overlap itself?

The two-pointer merge already works, because it folds every incoming interval into the last output interval. What breaks is only the assumption that each list is sorted by start; if that also fails, sort each list first (O(n log n)) or push everything through a heap.

FOLLOW-UP 3 Do touching intervals merge?

Decide the endpoint rule before coding. For closed intervals [1, 3] and [3, 5] share a point, so they merge (start <= last end). For half-open intervals [1, 3) and [3, 5) they do not overlap but are adjacent; whether to merge adjacent ones is a product choice, so ask.

FOLLOW-UP 4 Each list may overlap itself or be unsorted.

Normalise each list first (sort and merge it by itself, O(n log n)), after which the two-pointer union applies. Or sort everything together if memory allows.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Union of Sorted Interval Lists
#
```

```
# Two lists of intervals are each sorted and non-
# overlapping on their own. Return the union of both lists
# as sorted, non-overlapping intervals. Intervals that
# touch, such as (1, 3) and (3, 5), are merged unless
# touching=False.
#
# Signature:
#   def union_two(a: list, b: list, touching=True) -> list
#
# Examples:
# [(1, 3), (6, 9)], [(2, 5), (10, 12)]  ->  [(1, 5), (6,
#     9), (10, 12)]
# [(1, 3)], [(3, 5)]  ->  [(1, 5)]
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if the lists are too big for memory?
# 2. What if one list may overlap itself?
# 3. Do touching intervals merge?
# 4. Each list may overlap itself or be unsorted.
#
# INTERVIEW NOTES
# Each list is already sorted, so you can walk both with
# two pointers like the merge step of merge sort: always
# take the interval with the smaller start next. Once the
# intervals come out in start order, merging is the usual
# one-pass fold: extend the last output interval if the
# next one overlaps or touches it, otherwise start a new
# one.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq

END = object()                                  # marks "this list has no more intervals"


# ============================================================
# MAIN QUESTION
```

```
# ============================================================
# ---- main question: helper functions ----
def merge_sorted(lists):
    # merge several sorted lists (or streams) into one sorted stream, one item
    # at a time: a heap holds the next item of every list, so memory is O(K)
    iterators = []
    for items in lists:
        iterators.append(iter(items))
    heap = []
    for i in range(len(iterators)):
        first = next(iterators[i], END)
        if first is not END:
            heap.append((first, i))
    heapq.heapify(heap)
    while heap:
        item, i = heapq.heappop(heap)
        yield item
        following = next(iterators[i], END)
        if following is not END:
            heapq.heappush(heap, (following, i))
```

```python
def union_stream(lists, touching=True):
    # the union of sorted lists of (start, end), as a stream (follow-up 1).
    # Take the interval with the smaller start next; if it touches or overlaps
    # the block being built, extend that block, otherwise finish the block.
    current = None
    for start, end in merge_sorted(lists):
        if current is None:
            current = [start, end]
            continue
        if touching:
            joined = start <= current[1]           # closed intervals: [1, 3] and [3, 5] share the point 3
        else:
            joined = start < current[1]
        if joined:
            if end > current[1]:
                current[1] = end
        else:
            yield tuple(current)                   # finished one block: give it out now
            current = [start, end]
    if current is not None:
        yield tuple(current)
```

```python
# ---- main question: main function ----
# Idea: a two-pointer merge: always take the interval with the smaller
# start next. If it touches or overlaps the last output interval (start <=
# its end), extend that one, otherwise start a new one.
# Complexity: Time O(n + m) for two sorted lists, Space O(n + m) for the answer
def union_two(a, b, touching=True):
    return list(union_stream([a, b], touching))


# the same code with any number of lists: K lists, O(K) memory
# Complexity: Time O(N log k) for N intervals in k lists, Space O(k) plus the answer
def union_k(lists, touching=True):
    return list(union_stream(lists, touching))


# ---- main question: tests ----
def test_main_question():
    a = [(1, 3), (6, 8)]
    b = [(2, 4), (9, 10)]
    assert union_two(a, b) == [(1, 4), (6, 8), (9, 10)]
    assert union_two([(1, 3)], [(3, 5)]) == [(1, 5)]                    # closed intervals that touch mer
ge
    assert union_two([(1, 3)], [(3, 5)], False) == [(1, 3), (3, 5)]     # half-open intervals that only t
ouch stay apart
    assert union_k([a, b, []]) == [(1, 4), (6, 8), (9, 10)]             # three lists, one of them empty


# ========================================================
# FOLLOW-UP 1: What if the lists are too big for memory?
# ========================================================
# Answer: read each list through an iterator (a file or a paged API).
# union_stream only holds one interval per list plus the block being built,
# so memory is O(K) and it can hand out results as it goes.


# ========================================================
# FOLLOW-UP 3: Do touching intervals merge?
# ========================================================
# Answer: the `touching` option: True merges intervals that share an end
# ([1, 3] and [3, 5]); False keeps them apart (half-open intervals).
```

```
# ============================================================
# FOLLOW-UP 2: What if one list may overlap itself?
# ============================================================
```

```
# ============================================================
# FOLLOW-UP 4: Each list may overlap itself or be unsorted
# ============================================================
# ---- follow-up 2: function ----
# Answer: the merge already copes with overlapping intervals, because it
# folds every new interval into the block being built. For an UNSORTED list,
# sort it first (O(n log n)); then union the cleaned lists as before.
# Complexity: Time O(n log n), Space O(n)
def normalise(intervals):
    ordered = sorted(intervals)
    return list(union_stream([ordered]))


# Complexity: Time O(n log n + m log m), Space O(n + m)
def union_unsorted(a, b):
    return union_two(normalise(a), normalise(b))


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert union_unsorted([(5, 9), (1, 6)], [(8, 12)]) == [(1, 12)]     # a list that overlaps itself
    assert union_unsorted([(9, 10), (1, 2)], []) == [(1, 2), (9, 10)]   # an unsorted list


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Code the two-list merge without special-casing more than an empty result and a non-overlapping next interval.
Generalize the implementation to K iterators using a heap, then explain the memory bound.
Rehearse how the invariant changes when individual inputs are not internally disjoint.

STUDY PATH: question 89 of 197 | topic 16 of 43: Sorted intervals: merge while they touch, then read the gaps or search | 3 of 5

Template fit: keep sorted lefts and rights; adding bisects to the first interval that touches and the first that starts after, and replaces that block by one merged interval | * asked 273 days ago * | frequency Single report

MEDIUM | IN APP LC 715, 56 | ColorStack | technical interview | 89 of 197

Range Module (Insert and Point Query)

ASKED 2026-01-20 | 273 DAYS AGO | OLDER

STUDY: TYPE **ALL TIME 65.2** **30 DAYS 41.1** **PRIORITY 14/288**

ABOUT THIS QUESTION

Topic: Sorted intervals: merge while they touch, then read the gaps or search

Optimal answer from: your app's interview approach (LeetCode #715)

LeetCode match: LC 715 Range Module / LC 56 Merge Intervals

Frequency: Single report | Last asked: 2026-01-20 | Roles: SWE Intern

QUESTION

Design a data structure that tracks a set of half-open ranges [left, right) over the number line. add_range(left, right) adds the interval; overlapping or touching intervals are merged so that only disjoint, non-adjacent intervals remain. query_point(x) returns True when x lies inside a tracked interval; because the intervals are half-open, a point equal to a right endpoint is not inside.

Function signature

class RangeModule:
    def add_range(self, left: float, right: float) -> None
    def query_point(self, x: float) -> bool

EXAMPLES

Example 1

Input	`rm = RangeModule()`
	`rm.add_range(10, 20)`
	`rm.query_point(15)`
	`rm.query_point(20)`
	`rm.query_point(10)`

Output	`rm.query_point(15) -> true`
	`rm.query_point(20) -> false`
	`rm.query_point(10) -> true`

Why	`[10, 20) contains 10 and 15 but not 20.`

Example 2

Input	`rm = RangeModule()`
	`rm.add_range(10, 20)`
	`rm.add_range(20, 30)`
	`rm.query_point(20)`
	`rm.query_point(25)`
	`rm.query_point(30)`

Output	`rm.query_point(20) -> true`
	`rm.query_point(25) -> true`
	`rm.query_point(30) -> false`

Why	`The touching ranges merge into [10, 30), so 20 is now covered; 30 still is not.`

QUESTION

Example 3	
Input	`rm = RangeModule()` `rm.query_point(5)`
Output	`rm.query_point(5) -> false`
Why	`Nothing has been added yet.`

CONSTRAINTS

- Endpoints and query points are integers or floats with -10^9 <= value <= 10^9 .
- For every add_range call, left < right .
- Up to 10^5 total operations.
- Touching intervals merge: add_range(1, 5) followed by add_range(5, 10) yields one interval [1,
- 10) .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
class ListRanges:
    def __init__(self):
        self.items = []
    def add_range(self, left, right):
        self.items.append((left, right))
    def query_point(self, x):
        return any(
            a <= x < b for a, b in self.items
        )
```

TIME	SPACE
O(n) per query	**O(n)**

WHY IT IS SLOW

Every query walks every added interval.

OPTIMAL SOLUTION · Merged intervals with bisect

TIME	SPACE
O(log n) query	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Add remove_range(left, right).

Find the block of intervals that overlap [left, right) with bisect, keep the part of the first one left of the removal and the part of the last one right of it, and splice the remainders in.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Range Module (Insert and Point Query)
#
# Design a data structure that tracks a set of half-open
# ranges [left, right) over the number line.
# add_range(left, right) adds the interval; overlapping or
# touching intervals are merged so that only disjoint,
# non-adjacent intervals remain. query_point(x) returns
# True when x lies inside a tracked interval; because the
# intervals are half-open, a point equal to a right
# endpoint is not inside.
#
# Signature:
#   class RangeModule:
#       def add_range(self, left: float, right: float) -> None
#       def query_point(self, x: float) -> bool
#
# FOLLOW-UPS (answered below, in this order):
# 1. Add remove_range(left, right).
#
# INTERVIEW NOTES
# Keep the tracked intervals merged and sorted in two
# parallel lists, lefts and rights. For add_range, bisect
# finds the first interval that touches or overlaps the
# new one and the first one that starts after it;
# everything between them collapses into one interval. For
# query_point, bisect on the lefts finds the only interval
# that could contain x.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
# Complexity: Time O(log n), Space O(1)
def first_at_least(values, x):
    # binary search in a sorted list: the first index whose value is >= x
```

```
    low = 0
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] < x:
            low = middle + 1
        else:
            high = middle
    return low


# Complexity: Time O(log n), Space O(1)
def first_greater(values, x):
```

```
    # binary search in a sorted list: the first index whose value is > x
    low = 0
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] <= x:
            low = middle + 1
        else:
            high = middle
    return low


# ---- main question: main class ----
# Ranges are half-open [left, right): add_range(left, right) adds one,
# query_point(x) says whether x is inside any added range.
# Idea: keep the ranges merged, sorted and disjoint (two lists, lefts and
# rights). Adding replaces every range the new one touches by one merged
# range (found with two binary searches). A query is one binary search for the
# only candidate: the range with the largest left <= x.
# Complexity: Time add_range O(log n) to search plus O(n) to splice, query_point O(log n), Space O(n)
class RangeModule:
    def __init__(self):
        self.lefts = []
        self.rights = []

    def add_range(self, left, right):
        i = first_at_least(self.rights, left)              # the first range ending at or after the new lef
t
        j = first_greater(self.lefts, right)               # the first range starting after the new right
        if i < j:
            if self.lefts[i] < left:
                left = self.lefts[i]
            if self.rights[j - 1] > right:
```

```
                right = self.rights[j - 1]
        self.lefts[i:j] = [left]
        self.rights[i:j] = [right]

    def query_point(self, x):
        i = first_greater(self.lefts, x) - 1
        return i >= 0 and x < self.rights[i]


# ---- main question: tests ----
def test_main_question():
    module = RangeModule()
    module.add_range(10, 20)
    assert module.query_point(15) and module.query_point(10) and not module.query_point(20)
    module.add_range(20, 30)
    assert module.query_point(20) and module.query_point(25) and not module.query_point(30)
    assert not RangeModule().query_point(5)


# ======================================================
# FOLLOW-UP 1: Add remove_range(left, right)
# ======================================================
# ---- follow-up 1: function ----
# Answer: find the block of ranges that overlap [left, right) with two
# binary searches. Keep the part of the first one to the left of the removal
# and the part of the last one to the right of it, and splice those
# remainders in place of the block.
# Complexity: Time remove_range is O(log n) to find the ranges plus O(n) to splice the lists, Space O(n)
class RemovableRangeModule(RangeModule):
    def remove_range(self, left, right):
        i = first_greater(self.rights, left)               # the first range that ends after `left`
```

```
        j = first_at_least(self.lefts, right)              # the first range that starts at or after `right
`
        if i >= j:
            return                                         # nothing overlaps the removal
        new_lefts = []
        new_rights = []
        if self.lefts[i] < left:
            new_lefts.append(self.lefts[i])
            new_rights.append(left)
        if self.rights[j - 1] > right:
            new_lefts.append(right)
            new_rights.append(self.rights[j - 1])
        self.lefts[i:j] = new_lefts
        self.rights[i:j] = new_rights
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    random.seed(83)
    for round_number in range(300):                        # random adds and removes: compare with a set of
 points
        module = RemovableRangeModule()
        points = set()
        for step in range(random.randint(1, 8)):
            a = random.randint(0, 20)
            b = a + random.randint(1, 6)
            if random.random() < 0.6:
                module.add_range(a, b)
                for p in range(a, b):
                    points.add(p)
            else:
                module.remove_range(a, b)
                for p in range(a, b):
                    points.discard(p)
        for p in range(-1, 30):
            assert module.query_point(p) == (p in points)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 90 of 197 | topic 16 of 43: Sorted intervals: merge while they touch, then read the gaps or search | 4 of 5

Template fit: merge first, then for each merged meeting keep the part before the blocked window and the part after it | * asked 382 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 90 of 197

Meetings Minus a Do-Not-Schedule Block

ASKED 2025-10-03 | 382 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 65.2 | 30 DAYS 41.1 | PRIORITY 13/288

ABOUT THIS QUESTION

Topic: Sorted intervals: merge while they touch, then read the gaps or search

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #56 Merge Intervals

LeetCode match: LC 56 Merge Intervals + interval subtraction (related)

Frequency: Single report | Last asked: 2025-10-03 | Roles: SWE Intern

QUESTION

You are given a list of meetings , each a (start, end) interval with start < end , and one "do-not- schedule" (DNS) block (dns_start, dns_end) . Consider the set of all time points covered by at least one meeting. Return the smallest list of non-overlapping, sorted intervals that covers exactly those points except any point lying inside the DNS block (treat the DNS block as removing the open coverage on (dns_start, dns_end) ; equivalently split coverage at those bounds). Meetings may overlap and are given in any order. Return the result sorted by start time.

Function signature

```
def subtract_dns(meetings: list[tuple[int, int]],
                 dns: tuple[int, int]) -> list[tuple[int, int]]:
    ...
```

Follow-up (as reported): Generalize to a list of non-overlapping DNS blocks: merge the meetings first, then walk both sorted lists together subtracting each block.

EXAMPLES

Example 1	
Input	`meetings = [(1,7),(5,10),(12,30),(22,40),(50,65)], dns = (20,35)`
Output	`[(1,10),(12,20),(35,40),(50,65)]`
Why	`Merged coverage is [1,10],[12,40],[50,65]. Removing (20,35) splits [12,40] into [12,20] and [35,40].`

Example 2	
Input	`meetings = [(0,10)], dns = (0, 10)`
Output	`[]`
Why	`The DNS block removes the entire covered range.`

Example 3	
Input	`meetings = [(0,5),(6,9)], dns = (100, 200)`
Output	`[(0,5),(6,9)]`
Why	`The DNS block covers nothing that was scheduled.`

CONSTRAINTS

QUESTION

- 1 <= len(meetings) <= 10^5 .
- Coordinates are integers; 0 <= start < end <= 10^9 .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_subtract(meetings, dns):
    units = set()
    for s, e in meetings:
        units.update(range(s, e))
    units -= set(range(dns[0], dns[1]))
    out = []
    for t in sorted(units):
        if out and out[-1][1] == t:
            out[-1] = (out[-1][0], t + 1)
        else:
            out.append((t, t + 1))
    return out
```

TIME	SPACE
O(total length)	O(total length)

WHY IT IS SLOW

It touches every unit of time covered, which is huge when times are large.

OPTIMAL SOLUTION · Merge, then cut

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 There are several do-not-schedule blocks. How do you subtract all of them?

Merge both the meetings and the blocks. Walk the merged meetings with a pointer into the blocks: keep the part of the meeting before the next block, skip past the block, and continue with what is left. O((n + b) log (n + b)).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Meetings Minus a Do-Not-Schedule Block
#
# You are given a list of meetings , each a (start, end)
# interval with start < end , and one "do-not- schedule"
# (DNS) block (dns_start, dns_end) . Consider the set of
# all time points covered by at least one meeting. Return
# the smallest list of non-overlapping, sorted intervals
# that covers exactly those points except any point lying
# inside the DNS block (treat the DNS block as removing
# the open coverage on (dns_start, dns_end) ; equivalently
# split coverage at those bounds). Meetings may overlap
# and are given in any order. Return the result sorted by
```

```
# start time. Function signature: def
# subtract_dns(meetings: list[tuple[int, int]], dns:
# tuple[int, int]) -> list[tuple[int, int]]: ... Follow-up
# (as reported): Generalize to a list of non-overlapping
# DNS blocks: merge the meetings first, then walk both
# sorted lists together subtracting each block.
#
# FOLLOW-UPS (answered below, in this order):
# 1. There are several do-not-schedule blocks. How do you
# subtract all of them?
#
# INTERVIEW NOTES
# Merge the meetings into disjoint intervals first, then
# cut out the do-not-schedule block: a merged interval
# keeps a left piece [start, min(end, dns_start)] if it
# starts before the block, and a right piece [max(start,
# dns_end), end] if it ends after it. Several blocks
# become a walk over two sorted lists.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def merge_intervals(meetings):
    # sort by start, then join the ones that overlap or touch
    merged = []
    for start, end in sorted(meetings):
        if merged and start <= merged[-1][1]:
            if end > merged[-1][1]:
                merged[-1] = (merged[-1][0], end)
        else:
            merged.append((start, end))
    return merged


# ---- main question: main function ----
# Remove a do-not-schedule block (low, high) from the meetings.
# Idea: merge the meetings first; then every merged meeting keeps at most
# two pieces: the part to the LEFT of the block and the part to the RIGHT of it.
# Complexity: Time O(n log n), Space O(n)
def subtract_dns(meetings, dns):
    low, high = dns
    pieces = []
    for start, end in merge_intervals(meetings):
        if start < low:
            pieces.append((start, min(end, low)))          # the part before the block
        if end > high:
            pieces.append((max(start, high), end))         # the part after the block
    return pieces


# ---- main question: tests ----
def test_main_question():
    assert merge_intervals([(1, 3), (2, 6), (8, 10)]) == [(1, 6), (8, 10)]
    assert subtract_dns([(1, 5), (7, 12)], (4, 9)) == [(1, 4), (9, 12)]
    assert subtract_dns([(5, 6)], (4, 9)) == []                       # a meeting inside the block disappea
rs
```

```python
# ============================================================
# FOLLOW-UP 1: There are several do-not-schedule blocks. How do you subtract all of them?
# ============================================================
# ---- follow-up 1: function ----
# Answer: merge both the meetings and the blocks. For each merged meeting
# keep a moving start `current`; for every block that overlaps the meeting,
# keep the part of the meeting before the block, then jump past the block.
# What is left at the end is the last piece. O((n + b) log (n + b)).
# Complexity: Time O(n log n + b log b + n * b) for b blocks, Space O(n + b)
def subtract_many(meetings, blocks):
    pieces = []
    merged_blocks = merge_intervals(blocks)
    for start, end in merge_intervals(meetings):
        current = start
        for low, high in merged_blocks:
            if high <= current or low >= end:
                continue                                        # this block does not overlap what is le
ft
            if low > current:
                pieces.append((current, low))
            if high > current:
                current = high
        if current < end:
            pieces.append((current, end))
    return pieces


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert subtract_many([(1, 10)], [(2, 3), (5, 6)]) == [(1, 2), (3, 5), (6, 10)]
    assert subtract_many([(1, 5), (7, 12)], [(4, 9)]) == subtract_dns([(1, 5), (7, 12)], (4, 9))


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 91 of 197 | topic 16 of 43: Sorted intervals: merge while they touch, then read the gaps or search | 5 of 5

Template fit: the template as written | ND

MEDIUM | IN APP LC 56 | Exponent | 91 of 197

★ TOP 75: rank 11 in the Medium and Hard priority list

★ Merge Intervals

ASKED (ANY CO.) ~2 MONTHS AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 65.2 30 DAYS 41.1 PRIORITY 12/288

ABOUT THIS QUESTION

Topic: Sorted intervals: merge while they touch, then read the gaps or search

Optimal answer from: your app's interview approach (LeetCode #56)

LeetCode match: LC 56 Merge Intervals

Last asked (any company): 2 months ago, about 2026-08 (Exponent shows only a rough age) | +1219 interviews

Original: https://www.aced.io/questions/1602/merge-intervals

QUESTION

Merge Intervals. An interval is a representation of a range of values along a number line. For example, [1,3] represents all numbers between 1 and 3, inclusive. Given a collection of intervals, the task is to merge all overlapping intervals. For example, consider the intervals [1,3],[2,6],[8,10],[15,18]. The function should return merged intervals where overlapping intervals are combined. In this case, intervals [1,3] and [2,6] overlap, so they should be merged into [1,6]. Examples intervals = [[1,3], [2,6], [8,10], [15,18]] output: [[1,6], [8,10], [15,18]] intervals = [[1,4], [4,5]] output: [[1,5]] intervals = [[1,4], [2,3]] output: [[1,4]] intervals = [[1,2], [3,4], [5,6], [7,8]] output: [[1,2], [3,4], [5,6], [7,8]]The mergeIntervals function takes an array of intervals and merges all overlapping intervals into a single interval. The process involves sorting the intervals by their start times and then iterating through them to merge where necessary. Here is a step-by-step breakdown: Initial Check: If the input array is empty, the function immediately returns an empty array, as there are no intervals to merge. Sorting: The intervals are sorted by their start times. This ensures that intervals can be processed in a linear sweep, merging adjacent intervals when they overlap. Merging Intervals: Initialize a result list with the first interval. Then, iterate through the sorted intervals starting from the second interval. For each interval, compare it with the last interval in th

EXAMPLES

Example 1	
Input	`[[1, 3], [2, 6], [8, 10], [15, 18]]`
Output	`[[1, 6], [8, 10], [15, 18]]`
Why	`[1, 3] and [2, 6] overlap and merge into [1, 6].`

Example 2	
Input	`[[1, 4], [4, 5]]`
Output	`[[1, 5]]`
Why	`Intervals that touch at an endpoint are merged as well.`

Example 3	
Input	`[[8, 10], [1, 3], [2, 4]]`
Output	`[[1, 4], [8, 10]]`
Why	`Unsorted input is fine.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def merge(intervals):
    intervals = list(intervals)
    merged = True
    while merged:
        merged = False
        for a, b in combinations(intervals, 2):
            if overlaps(a, b):
                intervals.remove(a)
                intervals.remove(b)
                intervals.append(union(a, b))
                merged = True
                break
    return sorted(intervals)
```

TIME	SPACE
O(n^2) or worse	O(n)

WHY IT IS SLOW

Each pass compares all pairs, O(n^2), and several passes may be needed. Sorting by start and sweeping once is O(n log n).

OPTIMAL SOLUTION · Sort by start and sweep

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Insert one interval into an already merged list (LeetCode 57).

Copy the intervals that end before it, merge the ones that overlap, copy the rest: O(n) with no sort.

FOLLOW-UP 2 Intervals of touching integers ([1, 2] and [3, 4]) should merge.

Change the overlap test to start <= last_end + 1. For real-valued ranges that would be wrong, so state which you assume.

FOLLOW-UP 3 Too many intervals for memory.

Sort externally (external merge sort) and stream the merge: the one-pass sweep needs only the current interval in memory.
Cost: O(n log n) external sort with bounded memory runs, then an O(n) streaming merge with O(1) memory.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Merge Intervals
#
# Merge Intervals. An interval is a representation of a
# range of values along a number line. For example, [1,3]
# represents all numbers between 1 and 3, inclusive. Given
# a collection of intervals, the task is to merge all
```

```
# overlapping intervals. For example, consider the
# intervals [1,3],[2,6],[8,10],[15,18]. The function
# should return merged intervals where overlapping
# intervals are combined. In this case, intervals [1,3]
# and [2,6] overlap, so they should be merged into [1,6].
# Examples intervals = [[1,3], [2,6], [8,10], [15,18]]
# output: [[1,6], [8,10], [15,18]] intervals = [[1,4],
# [4,5]] output: [[1,5]] intervals = [[1,4], [2,3]]
# output: [[1,4]] intervals = [[1,2], [3,4], [5,6], [7,8]]
# output: [[1,2], [3,4], [5,6], [7,8]]The mergeIntervals
# function takes an array of intervals and merges all
# overlapping intervals into a single interval.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Insert one interval into an already merged list
# (LeetCode 57).
# 2. Intervals of touching integers ([1, 2] and [3, 4])
# should merge.
# 3. Too many intervals for memory.
#
# INTERVIEW NOTES
# After sorting by start, an interval can overlap only the
# interval just before it in the output, because
# everything earlier ends before that one starts. So one
# pass with a 'current merged interval' is enough: extend
# its end, or start a new one.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq

END = object()
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: sort by start, then sweep. If an interval starts at or before the
# end of the last output interval, extend that end (to the larger end);
# otherwise append it as a new interval. Because of the sort, nothing earlier
# can overlap the new interval. (touching_ints is follow-up 2.)
# Complexity: Time O(n log n), Space O(n)
def merge_intervals(intervals, touching_ints=False):
    gap = 0
    if touching_ints:
        gap = 1                                  # [1, 2] and [3, 4] cover 1..4 for whole numbers
    merged = []
    for start, end in sorted(intervals):
        if merged and start <= merged[-1][1] + gap:
            if end > merged[-1][1]:
                merged[-1][1] = end
        else:
            merged.append([start, end])
    return merged


# ---- main question: tests ----
def test_main_question():
    assert merge_intervals([[1, 3], [2, 6], [8, 10], [15, 18]]) == [[1, 6], [8, 10], [15, 18]]
```

```
    assert merge_intervals([[1, 4], [4, 5]]) == [[1, 5]]
    assert merge_intervals([]) == []
    assert merge_intervals([[1, 2], [3, 4]]) == [[1, 2], [3, 4]]
    assert merge_intervals([[1, 2], [3, 4]], True) == [[1, 4]]       # touching whole numbers merge


# ======================================================
# FOLLOW-UP 1: Insert one interval into an already merged list (LeetCode 57)
# ======================================================
# ---- follow-up 1: function ----
# Answer: the list is already merged and sorted, so no sort is needed:
# copy the intervals that end before the new one starts, absorb the ones that
# overlap it, copy the rest. O(n).
# Complexity: Time O(n), Space O(n)
def insert_interval(merged, new):
    result = []
    i = 0
    n = len(merged)
    while i < n and merged[i][1] < new[0]:
        result.append(merged[i])                # 1) everything that ends before the new one starts
        i += 1
    start = new[0]
    end = new[1]
    while i < n and merged[i][0] <= end:        # 2) everything that overlaps it is absorbed
        if merged[i][0] < start:
            start = merged[i][0]
        if merged[i][1] > end:
            end = merged[i][1]
        i += 1
    result.append([start, end])
    while i < n:
        result.append(merged[i])                # 3) the rest
        i += 1
    return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert insert_interval([[1, 3], [6, 9]], [2, 5]) == [[1, 5], [6, 9]]
    assert insert_interval([], [5, 7]) == [[5, 7]]


# ======================================================
# FOLLOW-UP 2: Intervals of touching integers ([1, 2] and [3, 4]) should merge
# ======================================================
# Answer: the `touching_ints` option: two intervals merge when the next one
# starts at most 1 after the previous end.


# ======================================================
# FOLLOW-UP 3: Too many intervals for memory
# ======================================================
# ---- follow-up 3: function ----
# Answer: sort externally (an external merge sort) and stream the sweep:
# it only needs the interval being built in memory. Cost: an O(n log n)
# external sort with bounded memory, then an O(n) pass with O(1) memory.
# Complexity: Time O(n), Space O(1) (one interval at a time)
def merge_stream(sorted_intervals):
    current = None
    for start, end in sorted_intervals:
        if current is not None and start <= current[1]:
            if end > current[1]:
                current[1] = end
        else:
```

```
            if current is not None:
                yield tuple(current)
            current = [start, end]
    if current is not None:
        yield tuple(current)


# Complexity: Time O(N log k) for N intervals in k sorted sources, Space O(N)
def merge_many_sorted(sources):
    # several files that are each already sorted: merge them with a heap, then sweep
    iterators = []
    for source in sources:
        iterators.append(iter(source))
    heap = []
    for i in range(len(iterators)):
        first = next(iterators[i], END)
        if first is not END:
            heap.append((first, i))
    heapq.heapify(heap)
    in_order = []
    while heap:
        item, i = heapq.heappop(heap)
        in_order.append(item)
        following = next(iterators[i], END)
        if following is not END:
            heapq.heappush(heap, (following, i))
    return list(merge_stream(in_order))


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert list(merge_stream([(1, 3), (2, 6), (8, 10)])) == [(1, 6), (8, 10)]
    assert merge_many_sorted([[(1, 3), (8, 9)], [(2, 4), (9, 12)]]) == [(1, 4), (8, 12)]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Merge Intervals. An interval is a representation of a range of values along a number line. For example, [1,3] represents all numbers between 1 and 3, inclusive. Given a collection of intervals, the task is to merge all overlapping intervals. For example, consider the intervals [1,3],[2,6],[8,10],[15,18]. The function should return merged intervals where overlapping intervals are combined. In this case, intervals [1,3] and [2,6] overlap, so they should be merged into [1,6]. Examples

OFFICIAL PAGE TEXT

```
intervals = [[1,3], [2,6], [8,10], [15,18]]
output: [[1,6], [8,10], [15,18]]

intervals = [[1,4], [4,5]]
output: [[1,5]]

intervals = [[1,4], [2,3]]
output: [[1,4]]

intervals = [[1,2], [3,4], [5,6], [7,8]]
output: [[1,2], [3,4], [5,6], [7,8]]
```

The mergeIntervals function takes an array of intervals and merges all overlapping intervals into a single interval. The process involves sorting the intervals by their start times and then iterating through them to merge where necessary. Here is a step-by-step breakdown: Initial Check: If the input array is empty, the function immediately returns an empty array, as there are no intervals to merge. Sorting: The intervals are sorted by their start times. This ensures that intervals can be processed in a linear sweep, merging adjacent intervals when they overlap. Merging Intervals: Initialize a result list with the first interval. Then, iterate through the sorted intervals starting from the second interval. For each interval, compare it with the last interval in the result list. If the current interval overlaps with the last interval in the result (i.e., the start of the current interval is less than or equal to the end of the last interval), merge them by updating the end of the last interval in the result to the maximum end time of both intervals. If there is no overlap, add the current interval to the result list as a separate interval. Return the Result: After processing all intervals, the result list contains the merged intervals. Time Complexity: The dominant factor in the time complexity is the sorting step, which takes O(n log n), where n is the number of intervals. The merging process involves a single pass through the sorted intervals, which takes O(n) time. Therefore, the overall time complexity is O(n log n). Space Complexity: The space complexity is O(n), which accounts for the space required to store the result list. This is because, in the worst case, none of the intervals overlap, and the result list will contain all the original intervals.In this mock interview video, Lewin (Dropbox SWE), answers the interview question, "Given an array of intervals, merge the overlapping intervals and return an array of the non-overlapping intervals that cover all intervals in the input."No answers contributed by the community yet.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 17 OF 43

Trie: nested dicts, one character per level

7 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

7
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Trie: nested dicts, one character per level | 7 questions | tap a title to jump to it

READ FIRST: the code template for these 7 questions (tap here) >

M	92. URL Request Handler Router	1Point3Acres * 2026-10-16 *
M	93. Trie Prefix Search / Autocomplete	1Point3Acres * 2026-07-02 *
M	94. Build Prefix Lookup with a Trie	PracHub * 2026-07-03 *
M	95. Greedy Longest-Match Dictionary Tokenizer	1Point3Acres * 2026-06-15 *
M	96. Check if all substrings are dictionary words	PracHub * 2026-03-11 *
M	97. Return dictionary words matching a prefix	PracHub * 2026-03-09 *
M	98. Implement Trie Insert and Exact-Word Search	PracHub * 2026-02-20 *

CODE TEMPLATE | GROUP 17 OF 43 | 7 QUESTIONS, ONE PATTERN

Trie: nested dicts, one character per level

The one idea these questions share

Store words as nested dicts, one character per level, with an end marker that holds whatever you want to remember. Insert walks and creates; search walks and fails when a character is missing; prefix queries stop in the middle; longest-match keeps the last end marker you passed.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
END = "$"


def build_trie(words):
    root = {}
    for w in words:
        node = root
        for ch in w:
            node = node.setdefault(ch, {})
        node[END] = w                           # CHANGE 1: what you store at the end of a word
    return root


def walk(root, text):
    node = root
    for ch in text:
        node = node.get(ch)
        if node is None:
            return None
    return node
```

2. Helper functions that usually get written next to it

```
def words_under(node):
    out = []
    for key, child in node.items():
        if key == END:
            out.append(child)
        else:
            out.extend(words_under(child))
    return sorted(out)


def longest_match(root, text, start):
    node, best = root, None
    for j in range(start, len(text)):
        node = node.get(text[j])
        if node is None:
            break
        if END in node:
            best = node[END]
    return best
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
92	URL Request Handler Router	one trie level per URL segment; a node has literal children and one wildcard child; try the literal first
93	Trie Prefix Search / Autocomplete	walk to the prefix node; return the words under it (autocomplete)

Contents Start the questions >

#	Question	What you change in the template
94	Build Prefix Lookup with a Trie	build_trie, then walk to the prefix node and collect every word below it
95	Greedy Longest-Match Dictionary Tokenizer	the same longest-match walk, with a priority rule to choose between equal-length matches
96	Check if all substrings are dictionary words	for every start walk the trie; a substring of length 3 or more that leaves the trie or lacks END fails the check
97	Return dictionary words matching a prefix	no trie needed: sort the words and bisect to the first match, then read forward while startswith(prefix)
98	Implement Trie Insert and Exact-Word Search	insert is build_trie; search is walk and then END in node

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 92 of 197 | topic 17 of 43: Trie: nested dicts, one character per level | 1 of 7

Template fit: one trie level per URL segment; a node has literal children and one wildcard child; try the literal first | * asked 4 days ago * | frequency Low

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | phone-screen, onsite-coding | 92 of 197

URL Request Handler Router

ASKED 2026-10-16 | 4 DAYS AGO | LAST 30 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 282/288**

ABOUT THIS QUESTION

Topic: Trie: nested dicts, one character per level

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/url-request-handler-router

QUESTION

Design a request router. add(pattern) registers a URL pattern whose segments are either literal text or a placeholder written {name}; match(url) returns the registered pattern that handles the URL, or None. When a URL matches both a literal segment and a placeholder, the literal wins.

Function signature

class Router:
  def add(self, pattern: str) -> None
  def match(self, url: str) -> str | None

EXAMPLES

Example 1

Input	`r = Router()`
	`r.add('/users/{userId}/photo/{photoId}')`
	`r.add('/users/me/photo/{photoId}')`
	`r.match('/users/123/photo/456')`
	`r.match('/users/me/photo/9')`
	`r.match('/orders/1')`

Output	`r.match('/users/123/photo/456') -> '/users/{userId}/photo/{photoId}'`
	`r.match('/users/me/photo/9') -> '/users/me/photo/{photoId}'`
	`r.match('/orders/1') -> null`

Why	`The placeholder pattern handles any user id, but for /users/me/... the literal 'me' pattern wins; an unknown path matches nothing.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def lookup(url):
    parts = url.split("/")
    matches = []
    for pattern in patterns:
        p = pattern.split("/")
        if len(p) != len(parts):
            continue
        if all(is_param(x) or x == y
               for x, y in zip(p, parts)):
            matches.append(pattern)
    return best_by_precedence(matches)
```

BRUTE FORCE

TIME	SPACE
O(H * L) per lookup	**O(1)** extra

WHY IT IS SLOW

Each lookup compares against every registered pattern, O(patterns * segments). A trie of segments with a literal child and a wildcard child follows only the compatible branches.

OPTIMAL SOLUTION · Segment trie with literal-first fallback

TIME	SPACE
O(L * branching) per lookup, O(total pattern length) to build	**O(total pattern length)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Different HTTP methods per route.

Make the method the first segment of the path (a tiny trick) or keep a dict method to trie. Lookup uses the method as the first step.

FOLLOW-UP 2 A wildcard at the end that matches the rest of the path (/static/*).

Give each trie node a 'rest' handler. During lookup, remember the deepest node that has one and use it if the exact walk fails. Precedence: exact, then placeholder, then rest-wildcard.

FOLLOW-UP 3 Return the extracted parameters (id = 7).

Carry a list of (name, segment) while walking; append when you take a wildcard edge and drop it when you backtrack. Store the placeholder names with the pattern, not in the trie.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: URL Request Handler Router
#
# Design a request router. add(pattern) registers a URL
# pattern whose segments are either literal text or a
# placeholder written {name}; match(url) returns the
# registered pattern that handles the URL, or None. When a
# URL matches both a literal segment and a placeholder,
# the literal wins.
#
# Signature:
#   class Router:
#       def add(self, pattern: str) -> None
#       def match(self, url: str) -> str | None
#
# FOLLOW-UPS (answered below, in this order):
# 1. Different HTTP methods per route.
# 2. A wildcard at the end that matches the rest of the
# path (/static/*).
```

```
# 3. Return the extracted parameters (id = 7).
#
# INTERVIEW NOTES
# Matching a path against many patterns is a prefix
# problem, so share the prefixes in a trie of segments.
# The catch is the wildcard: a literal branch can fail
# later, so you cannot pick it greedily. Try the literal
# child first and fall back to the wildcard child. That
# also encodes the precedence rule for free.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def segments(path):
    # "/users/7/photo" -> ["users", "7", "photo"]
    parts = []
    for piece in path.split("/"):
        if piece != "":
            parts.append(piece)
```

```
    return parts


class Node:
    # one node of the trie: one path segment per level
    def __init__(self):
        self.literal = {}          # fixed segment -> child node
        self.wild = None           # the child for a {placeholder} segment
        self.wild_name = None      # the placeholder's name, e.g. "userId"
        self.pattern = None        # the registered pattern that ends at this node
        self.rest = None           # the pattern with a trailing "*" (matches the rest of the path)


# ---- main question: main function ----
# Idea: store the patterns in a trie with one path SEGMENT per level.
# Every node has literal children plus at most one placeholder child, and
# the pattern is stored at its last node. To look up a URL, try the
# literal child first; if that whole branch fails, back up and try the
# placeholder child. So a literal segment wins over a placeholder.
# Complexity: Time add O(s) for s path segments, match O(s) usually (backtracking from a literal to a place
holder can revisit a branch), Space O(total segments)
class Router:
    def __init__(self):
        self.root = Node()

    def add(self, pattern):
        node = self.root
        for segment in segments(pattern):
            if segment == "*":                          # follow-up 2: a trailing * matches the rest of the
 path
                node.rest = pattern
                return
            if segment.startswith("{") and segment.endswith("}"):
                if node.wild is None:
                    node.wild = Node()
                node.wild_name = segment[1:-1]
                node = node.wild
```

```
            else:
                if segment not in node.literal:
                    node.literal[segment] = Node()
                node = node.literal[segment]
        node.pattern = pattern

    def lookup(self, url):
        # returns (the matching pattern or None, the values of the placeholders)
        parts = segments(url)
        values = {}
```

```
        def walk(node, i):
            if i == len(parts):
                return node.pattern
            child = node.literal.get(parts[i])            # 1) the exact segment first
            if child is not None:
                found = walk(child, i + 1)
                if found is not None:
                    return found
            if node.wild is not None:                     # 2) then a placeholder, remembering its value
                values[node.wild_name] = parts[i]
                found = walk(node.wild, i + 1)
                if found is not None:
                    return found
                del values[node.wild_name]                # that branch failed: forget the value
            return node.rest                              # 3) finally a trailing wildcard

        pattern = walk(self.root, 0)
        return pattern, values

    def match(self, url):
        pattern, values = self.lookup(url)
        return pattern


# ---- main question: tests ----
def test_main_question():
    r = Router()
    r.add("/users/{userId}/photo/{photoId}")
    r.add("/users/me")
    assert r.match("/users/123/photo/456") == "/users/{userId}/photo/{photoId}"
    assert r.match("/users/me") == "/users/me"                      # a literal beats a placeholder
    assert r.match("/users/me/photo/9") == "/users/{userId}/photo/{photoId}"
    assert r.match("/nothing/here") is None
    assert r.match("/users") is None


# ========================================================
# FOLLOW-UP 1: Different HTTP methods per route
# ========================================================
# ---- follow-up 1: function ----
# Answer: keep one trie per method (a dictionary method -> Router), or
# treat the method as the first segment of the path. A lookup uses the
# method as its first step.
# Complexity: Time the same as Router for each method, Space O(total segments)
```

```
class MethodRouter:
    def __init__(self):
        self.routers = {}

    def add(self, pattern, method="GET"):
        if method not in self.routers:
            self.routers[method] = Router()
        self.routers[method].add(pattern)
```

```
    def match(self, url, method="GET"):
        if method not in self.routers:
            return None
        return self.routers[method].match(url)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    r = MethodRouter()
    r.add("/orders/{id}", "POST")
    r.add("/orders/list")
    assert r.match("/orders/5", "POST") == "/orders/{id}"
    assert r.match("/orders/5") is None                              # a different method, a different rout
e
    assert r.match("/orders/list") == "/orders/list"


# ========================================================
# FOLLOW-UP 2: A wildcard at the end that matches the rest of the path (/static/*)
# ========================================================
# ---- follow-up 2: tests ----
# Answer: Router.add stores a pattern that ends with "*" on the node
# BEFORE it (the `rest` field), and the lookup falls back to it when
# nothing more specific matched.
def test_follow_up_2():
    r = Router()
    r.add("/static/*")
    r.add("/static/logo.png")
    assert r.match("/static/css/app.css") == "/static/*"
    assert r.match("/static/logo.png") == "/static/logo.png"        # the more specific pattern wins


# ========================================================
# FOLLOW-UP 3: Return the extracted parameters (id = 7)
# ========================================================
# ---- follow-up 3: tests ----
# Answer: while walking, remember the value of every placeholder
```

```
# (and forget it again when a branch fails): Router.lookup returns the
# pattern together with those values.
def test_follow_up_3():
    r = Router()
    r.add("/users/{userId}/photo/{photoId}")
    r.add("/users/me")
    pattern, values = r.lookup("/users/123/photo/456")
    assert pattern == "/users/{userId}/photo/{photoId}"
    assert values == {"userId": "123", "photoId": "456"}
    assert r.lookup("/users/me") == ("/users/me", {})


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

In 15 minutes, implement the scan baseline and check the existing example, segment-count mismatches, and no-match cases under an explicitly chosen return convention.
In 25 minutes, build the segment trie and compare its complete match set with the scan on small generated inputs. Include a literal branch that fails later and patterns with different placeholder names but the same structure.
In 10 minutes, write a contract checklist for precedence, duplicate registration, trailing slashes, query strings, and URL decoding; separate agreed rules from assumptions in your tests.

STUDY PATH: question 93 of 197 | topic 17 of 43: Trie: nested dicts, one character per level | 2 of 7

Template fit: walk to the prefix node; return the words under it (autocomplete) | * asked 110 days ago * | frequency Medium

MEDIUM | IN APP LC 208 | 1Point3Acres | tech-screen, onsite-coding | 93 of 197

Trie Prefix Search / Autocomplete

ASKED 2026-07-02 | 110 DAYS AGO | LAST 6 MONTHS

STUDY: TALK ALL TIME 36 30 DAYS 0 PRIORITY 124/288

ABOUT THIS QUESTION

Topic: Trie: nested dicts, one character per level

Optimal answer from: your app's interview approach (LeetCode #208)

LeetCode match: LC 208 Implement Trie (Prefix Tree)

Frequency: Medium | Last asked: 2026-07-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/trie-prefix-search-autocomplete

QUESTION

Build a trie from a list of words. search_prefix(prefix) returns every word that starts with the prefix, sorted alphabetically (empty list when none).

Function signature

class Trie:
  def insert(self, word: str) -> None
  def search_prefix(self, prefix: str) -> list

EXAMPLES

Example 1

Input	`t = Trie(['car', 'card', 'care', 'cat', 'dog'])`
	`t.search_prefix('car')`
	`t.search_prefix('ca')`
	`t.search_prefix('x')`

Output	`t.search_prefix('car') -> ['car', 'card', 'care']`
	`t.search_prefix('ca') -> ['car', 'card', 'care', 'cat']`
	`t.search_prefix('x') -> []`

Why	`'car' is itself a word that starts with 'car'; the prefix 'x' matches nothing.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def search_prefix(words, prefix):
    found = []
    for w in words:
        if w.startswith(prefix):
            found.append(w)
    return sorted(found)
```

TIME	SPACE
O(n * L + n log n) per query	**O(output)**

WHY IT IS SLOW

BRUTE FORCE

Every query reads the whole dictionary. A trie reaches the prefix in O(length of the prefix) and reads only the completions.

OPTIMAL SOLUTION · Trie with a DFS over sorted children

TIME	SPACE
O(L + output) per query after O(total characters) to build	**O(total characters)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return only the best K completions ranked by frequency.

Store at every node the K best (frequency, word) pairs and update them while inserting; a query then reads one node in O(L + K) and never walks the subtree. The price is more memory and slower inserts, which is the right trade when reads dominate (autocomplete).

FOLLOW-UP 2 Allow one wrong character (fuzzy match).

Walk the trie with a budget of one edit: at each level follow the matching child with the full budget and every other child with the budget reduced to 0. The work grows with the alphabet size times the prefix length, not with the dictionary size.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Trie Prefix Search / Autocomplete
#
# Build a trie from a list of words. search_prefix(prefix)
# returns every word that starts with the prefix, sorted
# alphabetically (empty list when none).
#
# Signature:
#   class Trie:
#       def insert(self, word: str) -> None
#       def search_prefix(self, prefix: str) -> list
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return only the best K completions ranked by
# frequency.
# 2. Allow one wrong character (fuzzy match).
#
# INTERVIEW NOTES
# A trie shares prefixes, so walking down the query prefix
# costs O(length of the prefix) no matter how many words
# exist. Everything under that node is a completion. For
# autocomplete you either collect the words below the
# prefix node with a DFS (visiting children in sorted
# order gives sorted output) or store the best few words
# at every node while inserting.

# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class Node:
    def __init__(self):
        self.children = {}         # letter -> child node
        self.freq = 0              # how often the word ending here was inserted
        self.top = []              # the best (negative frequency, word) pairs below this node


# ---- main question: main function ----
# Idea: insert each word into the trie and mark its last node. For a
```

```
# query, walk the prefix (a missing step means no words), then collect
# every word below that node and sort them alphabetically.
# Complexity: Time insert O(L * k log k) (the best-k list of every node on the path is refreshed), search_p
refix O(L + size of the subtree), Space O(total characters * k)
class Trie:
    def __init__(self, k=3):
        self.root = Node()
        self.k = k                 # how many suggestions to keep per node (follow-up 1)

    def insert(self, word, freq=1):
        node = self.root
        path = [node]
        for ch in word:
            if ch not in node.children:
                node.children[ch] = Node()
            node = node.children[ch]
            path.append(node)
        node.freq += freq
        # follow-up 1: refresh the best-k list on every node of the path
        for n in path:
            kept = []
            for item in n.top:
                if item[1] != word:
                    kept.append(item)
            kept.append((-node.freq, word))
            kept.sort()                                    # most frequent first, then alphabetical
            n.top = kept[:self.k]

    def find(self, prefix):
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node

    def words_below(self, node, text):
        # every word at or below node (spelled `text` so far), in no particular order
        found = []
        stack = [(node, text)]
        while stack:
            current, spelled = stack.pop()
            if current.freq > 0:
                found.append(spelled)
            for ch in current.children:
                stack.append((current.children[ch], spelled + ch))
```

```
        return found

    def search_prefix(self, prefix):
```

```
        node = self.find(prefix)
        if node is None:
            return []
        return sorted(self.words_below(node, prefix))


# ---- main question: tests ----
def test_main_question():
    t = Trie(k=2)
    for word in ["apple", "app", "apricot", "banana"]:
        t.insert(word)
    assert t.search_prefix("ap") == ["app", "apple", "apricot"]
    assert t.search_prefix("x") == []
    assert t.search_prefix("") == ["app", "apple", "apricot", "banana"]


# ========================================================
# FOLLOW-UP 1: Return only the best K completions ranked by frequency
# ========================================================
# ---- follow-up 1: function ----
# Answer: every node keeps its best k words (see Trie.insert), so a query
# only has to walk the prefix and read ONE node: O(L + k) instead of
# searching the whole subtree.
# Complexity: Time O(L + k), Space O(k)
def suggest(trie, prefix):
    node = trie.find(prefix)
    if node is None:
        return []
    words = []
    for negative_frequency, word in node.top:
        words.append(word)
    return words


# ---- follow-up 1: tests ----
def test_follow_up_1():
    t = Trie(k=2)
    for word in ["apple", "app", "apricot", "banana"]:
        t.insert(word)
    t.insert("app", 3)                                  # "app" was typed three more times
    assert suggest(t, "ap") == ["app", "apple"]
    assert suggest(t, "zzz") == []
```

```
# ========================================================
# FOLLOW-UP 2: Allow one wrong character (fuzzy match)
# ========================================================
# ---- follow-up 2: function ----
# Answer: walk the trie while counting the letters of the prefix that do
# not match; keep every branch with at most one mismatch. When the whole
# prefix has been read, collect the words below that node.
# Complexity: Time O(A * L + size of the subtrees reached) for alphabet size A, Space O(L + size of the ans
wer)
def fuzzy(trie, prefix):
    found = []
    stack = [(trie.root, "", 0)]
    while stack:
        node, text, wrong = stack.pop()
        if len(text) == len(prefix):
            found += trie.words_below(node, text)
            continue
        for ch in node.children:
            misses = wrong
            if ch != prefix[len(text)]:
```

```
                misses += 1
            if misses <= 1:
                stack.append((node.children[ch], text + ch, misses))
    return sorted(found)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    t = Trie()
    for word in ["apple", "app", "apricot", "banana"]:
        t.insert(word)
    assert "banana" in fuzzy(t, "vana")                    # one wrong letter is allowed
    assert fuzzy(t, "zzzz") == []
    assert fuzzy(t, "app") == ["app", "apple", "apricot"]      # "apr" is one letter away from "app"


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write the trie class from memory in under 12 min.
Drill the DFS-collect and the sorted-per-node variants; know which is faster in which scenario.
For frequency-ranking, practice the design-autocomplete variant (per-node top-K updated on insert) as a stretch goal.

STUDY PATH: question 94 of 197 | topic 17 of 43: Trie: nested dicts, one character per level | 3 of 7

Template fit: build_trie, then walk to the prefix node and collect every word below it | * asked 109 days ago *

MEDIUM | IN APP LC 208 | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 94 of 197

Build Prefix Lookup with a Trie

POSTED 2026-07-03 | 109 DAYS AGO | LAST 6 MONTHS

STUDY: TALK ALL TIME 36 30 DAYS 0 PRIORITY 121/288

ABOUT THIS QUESTION

Topic: Trie: nested dicts, one character per level

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #208 Implement Trie (Prefix Tree)

LeetCode match: LC 208 Implement Trie (Prefix Tree) (inferred from overview)

Posted on PracHub: 2026-07-03 (updated 2026-07-04)

Original: https://prachub.com/coding-questions/build-prefix-lookup-with-a-trie

QUESTION

Build a trie from a list of words. completions(prefix) returns every inserted word that starts with the prefix (including the prefix itself when it is a word), sorted alphabetically, or an empty list when none does.

Function signature

class PrefixTrie:
  def insert(self, word: str) -> None
  def completions(self, prefix: str) -> list

EXAMPLES

Example 1

Input	`t = PrefixTrie(['apple', 'app', 'apply', 'apt', 'bat'])`
	`t.completions('app')`
	`t.completions('ap')`
	`t.completions('z')`

Output	`t.completions('app') -> ['app', 'apple', 'apply']`
	`t.completions('ap') -> ['app', 'apple', 'apply', 'apt']`
	`t.completions('z') -> []`

Why	`'app' is itself a word and a prefix of two more; the prefix 'z' matches nothing.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def lookup(words, prefix):
    found = []
    for word in words:
        if word.startswith(prefix):
            found.append(word)
    return found
```

TIME	SPACE
O(W * L)	**O(1)**
per query	extra

BRUTE FORCE

WHY IT IS SLOW

Every query touches every word, O(W * L). A trie walks only the prefix and then the subtree below it.

OPTIMAL SOLUTION · Trie with a DFS below the prefix node

TIME	SPACE
O(L) to walk the prefix + O(size of the answer)	**O(total characters)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How many words start with this prefix?

Store a counter in each node that is incremented when a word passes through it; the answer is the counter at the end of the prefix walk, O(L) with no DFS.

FOLLOW-UP 2 Return only the top 3 completions (by frequency or alphabetically).

Keep at each node a short list of its best K words (updated on insert), or bound the DFS: visit children in alphabetical order and stop after K words. That is the LeetCode 1268 style answer.

FOLLOW-UP 3 Memory is large (many words with long common prefixes).

A compressed (radix) trie stores whole edge labels instead of single characters; a sorted array with binary search over prefixes uses the least memory for a static dictionary. **Cost:** a radix trie has O(number of words) nodes instead of O(total characters); a sorted array uses O(total characters) with O(L log W) lookups.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Build Prefix Lookup with a Trie
#
# Build a trie from a list of words. completions(prefix)
# returns every inserted word that starts with the prefix
# (including the prefix itself when it is a word), sorted
# alphabetically, or an empty list when none does.
#
# Signature:
#   class PrefixTrie:
#       def insert(self, word: str) -> None
#       def completions(self, prefix: str) -> list
#
# FOLLOW-UPS (answered below, in this order):
# 1. How many words start with this prefix?
# 2. Return only the top 3 completions (by frequency or
# alphabetically).
# 3. Memory is large (many words with long common
# prefixes).
#
# INTERVIEW NOTES
# Prefix questions repeat work when each query scans the
# whole word list. A trie stores each prefix once: walk
# down by the query, then everything below that node is a
# completion. Keep counters or a word flag in the nodes to
```

```
# answer follow-ups such as 'how many words start with
# this' or 'top K completions' without another walk.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class Node:
    def __init__(self):
        self.children = {}         # letter -> child node
        self.is_word = False       # does a word end here?
        self.count = 0             # how many different words pass through this node (follow-up 1)
```

```
# ---- main question: main function ----
# Idea: a trie stores the words in nested nodes, one letter per level.
# insert walks down creating missing nodes and marks the last one as a
# word end. For a prefix, walk down one letter at a time (a missing step
# means no completions), then go through everything below that node
# collecting the words.
# Complexity: Time insert O(L), completions O(L + size of the subtree), Space O(total characters)
class PrefixTrie:
    def __init__(self):
        self.root = Node()
        self.size = 0                                      # how many different words

    def find(self, text):
        # the node reached by following `text`, or None
        node = self.root
        for ch in text:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node

    def insert(self, word):
        existing = self.find(word)
        if existing is not None and existing.is_word:
            return                                         # a repeated word counts once
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = Node()
            node = node.children[ch]
            node.count += 1
        node.is_word = True
        self.size += 1

    def words_below(self, node, text, limit):
        # the words at or below `node` in alphabetical order, at most `limit` of them.
        # A stack is used; the children are pushed in reverse order so the
        # smallest letter is taken first.
        found = []
        stack = [(node, text)]
        while stack and len(found) < limit:
            current, spelled = stack.pop()
            if current.is_word:
                found.append(spelled)
```

```
            for ch in sorted(current.children, reverse=True):
```

```
                stack.append((current.children[ch], spelled + ch))
        return found

    def completions(self, prefix):
        # every word that starts with the prefix, in alphabetical order
        node = self.find(prefix)
        if node is None:
            return []
        return self.words_below(node, prefix, self.size + 1)


# ---- main question: tests ----
def build(words):
    trie = PrefixTrie()
    for word in words:
        trie.insert(word)
    return trie


def test_main_question():
    t = build(["apple", "app", "apricot", "banana", "apple"])
    assert t.completions("ap") == ["app", "apple", "apricot"]
    assert t.completions("")[-1] == "banana"
    assert len(t.completions("")) == 4                # the repeated "apple" counts once
    assert t.completions("x") == []
    assert t.completions("apple") == ["apple"]


# ============================================================
# FOLLOW-UP 1: How many words start with this prefix?
# ============================================================
# ---- follow-up 1: function ----
# Answer: every node counts the words that pass through it (done in
# insert), so the answer is the counter of the node the prefix walk ends
# at: O(length of the prefix), no search below it.
# Complexity: Time O(L), Space O(1)
def count_prefix(trie, prefix):
    if prefix == "":
        return trie.size
    node = trie.find(prefix)
    if node is None:
        return 0
    return node.count
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    t = build(["apple", "app", "apricot", "banana", "apple"])
    assert count_prefix(t, "ap") == 3
    assert count_prefix(t, "b") == 1
    assert count_prefix(t, "zz") == 0
    assert count_prefix(t, "") == 4


# ============================================================
# FOLLOW-UP 2: Return only the top 3 completions (by frequency or alphabetically)
# ============================================================
# ---- follow-up 2: function ----
# Answer: for the first k in alphabetical order, stop the walk after k
# words (words_below does, because children are visited in order). For
# the most frequent, store the best k words in every node (see the
# "Trie Prefix Search / Autocomplete" question).
# Complexity: Time O(L + k * L) for k words of length about L, Space O(k)
```

```
def suggest(trie, prefix, k=3):
    node = trie.find(prefix)
    if node is None:
        return []
    return trie.words_below(node, prefix, k)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    t = build(["apple", "app", "apricot", "banana"])
    assert suggest(t, "ap", 2) == ["app", "apple"]
    assert suggest(t, "ap") == ["app", "apple", "apricot"]
    assert suggest(t, "q") == []


# ========================================================
# FOLLOW-UP 3: Memory is large (many words with long common prefixes)
# ========================================================
# Answer: use a compressed (radix) trie that stores a whole stretch of
# letters on one edge, so long shared or unshared chains take one node
# instead of one node per letter.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], queries: list[str]) -> list[list[str]]:
SAMPLE INPUT
{"words":["car","cat","dog"],"queries":["ca","d"]}
```

Test cases

Input	Expected
`{"words":["car","cat","dog"],"queries":["ca","d"]}`	`[["car","cat"],["dog"]]`
`{"words":[],"queries":["a"]}`	`[[]]`

ALSO REPORTED · 1 more

Trie Prefix Search / Autocomplete | 1Point3Acres | tech-screen, onsite-coding | asked 2026-07-02 | Medium

https://www.1point3acres.com/interview/problems/company/google/trie-prefix-search-autocomplete

Standard implementation: each trie node has a children map and an is_end flag.
Two collection patterns at the prefix: (a) DFS from the prefix node and collect all word-end nodes, (b) store the top-K candidates at every node during insert (Google-preferred for autocomplete).
The standard sorted-suggestions follow-up wants 3 suggestions per prefix-length — handle with sorted children or per-node sorted lists.

ALSO REPORTED · 1 more

Preparation: Write the trie class from memory in under 12 min.
Drill the DFS-collect and the sorted-per-node variants; know which is faster in which scenario.
For frequency-ranking, practice the design-autocomplete variant (per-node top-K updated on insert) as a stretch goal.

STUDY PATH: question 95 of 197 | topic 17 of 43: Trie: nested dicts, one character per level | 4 of 7

Template fit: the same longest-match walk, with a priority rule to choose between equal-length matches | * asked 127 days ago * | frequency Low

MEDIUM | 1Point3Acres | phone-screen | 95 of 197

Greedy Longest-Match Dictionary Tokenizer

ASKED 2026-06-15 | 127 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 275/288**

ABOUT THIS QUESTION

Topic: Trie: nested dicts, one character per level

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-06-15 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/tokenize-longest-dict-match-greedy

QUESTION

Phone-screen coding round (R1). Tokenize a text against a dictionary of **`key:id`** entries using greedy longest-match; emit the matched id when a key matches, otherwise output the unmatched character as a literal. One follow-up on changing the tie-breaking / priority rule for which token wins. A second follow-up appears when the dictionary is very large: discuss how to keep the matcher memory- and lookup-efficient at scale.

Input: a text string and a dictionary mapping key (string) -> id.
Sweep the text left-to-right. At each position, find the longest dictionary key that matches a prefix of the remaining text; emit its id. If no key matches at the current position, emit the single character as a literal and advance by one.
Output the concatenation (or list) of ids and literal characters in order.
Follow-up: the interviewer changes the tie-breaking / priority rule for which key wins when multiple match (e.g., not longest but highest-priority by dictionary order). Adapt the data structure so the rule swap is cheap.
Follow-up: the dictionary becomes very large. Discuss how to keep memory and per-position lookup efficient — a compressed / radix trie (or sharding the key space) instead of holding every key in a flat map.

EXAMPLES

Example 1	

Input	`t = Tokenizer({'ab': 1, 'abc': 2, 'c': 3})`
	`t.tokenize('abcab')`
	`t.tokenize('abx')`

Output	`t.tokenize('abcab') -> [2, 1]` `t.tokenize('abx') -> [1, 'x']`
Why	`At each position the longest dictionary key wins: 'abc' (not 'ab' followed by 'c'), then 'ab'. A character that starts no key is emitted as it is.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def tokenize(text):
    out, i = [], 0
    while i < len(text):
        for size in range(max_key_len, 0, -1):
            key = text[i:i + size]
            if key in dictionary:
                out.append(dictionary[key])
                i += size
                break
        else:
            out.append(text[i])  # literal
            i += 1
    return out
```

TIME	SPACE
O(n * L)	**O(dictionary)**
lookups, each O(L)	

WHY IT IS SLOW

Every lookup hashes a substring, O(L^2) per position. Walking a trie of the keys one character at a time costs O(L) per position.

OPTIMAL SOLUTION · Trie walk keeping the deepest (or best-priority) terminal

TIME	SPACE
O(n * L)	**O(total key length)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The dictionary is huge.

A flat map of every key wastes memory because keys share prefixes. A compressed (radix) trie stores each shared stretch once, or shard the keys by first characters across machines. Lookup stays one pass over the text; only the node layout changes.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Greedy Longest-Match Dictionary Tokenizer
#
# Phone-screen coding round (R1). Tokenize a text against
# a dictionary of key:id entries using greedy longest-
# match; emit the matched id when a key matches, otherwise
# output the unmatched character as a literal. One follow-
# up on changing the tie-breaking / priority rule for
# which token wins. A second follow-up appears when the
# dictionary is very large: discuss how to keep the
# matcher memory- and lookup-efficient at scale. Input: a
# text string and a dictionary mapping key (string) -> id.
# Sweep the text left-to-right. At each position, find the
# longest dictionary key that matches a prefix of the
# remaining text; emit its id. If no key matches at the
# current position, emit the single character as a literal
# and advance by one. Output the concatenation (or list)
```

```
# of ids and literal characters in order.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The dictionary is huge.
#
# INTERVIEW NOTES
# At each position you want the longest dictionary key
# that is a prefix of the remaining text. Putting the keys
# in a trie lets you walk the text character by character
# from that position and remember the deepest end-of-key
# node you passed, instead of trying every length. If the
# tie-break rule changes (priority instead of length),
# store the priority at the terminal node and keep the
# best terminal seen.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: put every key in a trie with its id at the last node. From each
# text position walk down the trie while the next letter exists and
# remember the LAST (deepest) key that ended on the way: that is the
```

```
# longest match. Output its id and skip its length; if no key matches,
# output the character itself and move one position.
# priority(key) (follow-up 1) is an optional rule: a LOWER number wins over
# a longer key.
# Complexity: Time build O(total key characters), tokenize O(n * longest key), Space O(total key characters
)
class Tokenizer:
    def __init__(self, table, priority=None):
        # table: key -> id
        self.priority = priority
        self.root = {}
        for key in table:
            node = self.root
            for ch in key:
                if ch not in node:
                    node[ch] = {}
                node = node[ch]
            node["$"] = (key, table[key])              # the end of a key stores the key and its id

    def better(self, new, old):
        # should the match `new` replace the match `old` found earlier?
        if old is None:
            return True
        if self.priority is None:
            return True                                # deeper (longer) matches come later: keep them
        return self.priority(new[0]) < self.priority(old[0])

    def best_at(self, text, i):
        node = self.root
        best = None
        j = i
        while j < len(text) and text[j] in node:
            node = node[text[j]]
            j += 1
            if "$" in node and self.better(node["$"], best):
```

```
                best = node["$"]
        return best

    def tokenize(self, text):
        output = []
        i = 0
        while i < len(text):
            match = self.best_at(text, i)
            if match is None:
                output.append(text[i])                  # no key starts here: keep the character
                i += 1
```

```
            else:
                key, ident = match
                output.append(ident)
                i += len(key)
        return output


# a simpler version: one dictionary lookup for each possible length,
# longest first
# Complexity: Time O(n * longest key^2) (a slice and a hash for every length), Space O(keys)
def tokenize_map(text, table):
    longest = 0
    for key in table:
        if len(key) > longest:
            longest = len(key)
    output = []
    i = 0
    while i < len(text):
        size = longest
        if len(text) - i < size:
            size = len(text) - i
        matched = False
        while size > 0 and not matched:
            if text[i:i + size] in table:
                output.append(table[text[i:i + size]])
                i += size
                matched = True
            size -= 1
        if not matched:
            output.append(text[i])
            i += 1
    return output


# ---- main question: tests ----
TABLE = {"ab": 1, "abc": 2, "c": 3}


def test_main_question():
    t = Tokenizer(TABLE)
    assert t.tokenize("abcab") == [2, 1]                # "abc" is longer than "ab", so it wins
    assert t.tokenize("xyz") == ["x", "y", "z"]
    assert t.tokenize("") == []
    assert t.tokenize("cab") == [3, 1]
    assert tokenize_map("abcab", TABLE) == [2, 1]
```

```
    for text in ("abcab", "cabcabc", "xabcx", "", "ccc"):
        assert tokenize_map(text, TABLE) == t.tokenize(text)


# ====================================================
# FOLLOW-UP 1: The dictionary is huge
# ====================================================
```

```
# ---- follow-up 1: tests ----
# Answer: a flat map of every key wastes memory because keys share
# prefixes. A compressed (radix) trie stores each shared stretch once, or
# shard the keys by their first letters across machines. A different
# priority rule (the `priority` parameter) only changes `better`.
def test_follow_up_1():
    order = list(TABLE)
    by_order = Tokenizer(TABLE, priority=order.index)
    assert by_order.tokenize("abc") == [1, 3]           # "ab" was listed before "abc"


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement both versions (hashmap-scan and trie-walk) from scratch in under 15 min each. Run them on a small dict like {"the": 1, "theater": 2, "a": 3} against text "theater" to confirm the trie returns id 2 and the hashmap version returns the same.

For the priority-tie follow-up: practice attaching a priority field to terminal nodes and picking the highest-priority terminal seen along the trie walk, not just the deepest. Time complexity should not change.

If asked to stream the input, switch to the Aho-Corasick mental model — the trie augmented with failure links emits all dictionary matches in a single linear pass over the text.

STUDY PATH: question 96 of 197 | topic 17 of 43: Trie: nested dicts, one character per level | 5 of 7

Template fit: for every start walk the trie; a substring of length 3 or more that leaves the trie or lacks END fails the check | * asked 223 days ago *

MEDIUM | PracHub | Technical Screen | 96 of 197

Check if all substrings are dictionary words

POSTED 2026-03-11 | 223 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 43.5 | 30 DAYS 0 | PRIORITY 95/288

ABOUT THIS QUESTION

Topic: Trie: nested dicts, one character per level

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #139 Word Break

LeetCode match: No direct match; related LC 139 Word Break

Posted on PracHub: 2026-03-11 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/check-if-all-substrings-are-dictionary-words

QUESTION

You are given a string **`s`** (letters only) and access to an English dictionary **`dict`** (a set of valid words). Return **`true`** if **every contiguous substring** of **`s`** with length **`>= 3`** is a valid dictionary word (i.e., for all **`i, j`** with **`0 <= i <= j < n`** and **`j - i + 1 >= 3`**, **`s[i..j] in dict`**). Otherwise return **`false`**. Clarifications:

- Substring means **contiguous**.
- You may assume dictionary lookups are available (e.g., **`dict.contains(word)`**), or you can preprocess the dictionary. Example:
- If **`s = "cats"`**, you must check **`"cat"`**, **`"ats"`**, and **`"cats"`** (and any other substrings of length >= 3).

EXAMPLES

Example 1	
Input	`'abcd', {'abc', 'abcd', 'bcd'}`
Output	`true`
Why	`Every substring of length 3 or more (abc, bcd, abcd) is in the dictionary.`

Example 2	
Input	`'abcd', {'abc', 'abcd'}`
Output	`false`
Why	`'bcd' is missing from the dictionary.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s, dictionary):
    for i in range(len(s)):
        for j in range(i + 3, len(s) + 1):
            if s[i:j] not in dictionary:
                return False
    return True
```

BRUTE FORCE

TIME	SPACE
O(n^2)	**O(1)**
lookups, O(n^3) characters hashed	extra

WHY IT IS SLOW

About n^2 / 2 lookups and each hashes a whole substring, O(n^3) characters. Walking a trie from each start, one character at a time, is O(n^2) with no hashing of long strings.

OPTIMAL SOLUTION · Walk a trie from each start position

TIME	SPACE
O(n^2)	**O(dictionary size)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the first failing substring.

Same loops, but return the substring instead of False, which is what a user wants to see. Check the shorter ones first so the report is the smallest counterexample.

FOLLOW-UP 2 The string is very long.

The check is inherently quadratic in the number of substrings; any substring that fails ends the search, so cheap necessary checks first help: every length-3 window must be a word (O(n) lookups), then length 4, and so on.

FOLLOW-UP 3 The dictionary is huge and sharded.

Batch the lookups by length (a set per length) or use a Bloom filter in front of the real dictionary to reject most misses cheaply. **Cost:** still O(n^2) lookups, spread over shards (each lookup O(L)); a Bloom filter in front removes most remote calls.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Check if all substrings are dictionary
# words
#
# You are given a string s (letters only) and access to an
# English dictionary dict (a set of valid words). Return
# true if **every contiguous substring** of s with length
# **>= 3** is a valid dictionary word (i.e., for all i, j
# with 0 <= i <= j < n and j - i + 1 >= 3, s[i..j] in
# dict). Otherwise return false. Clarifications: -
# Substring means **contiguous**. - You may assume
# dictionary lookups are available (e.g.,
# dict.contains(word)), or you can preprocess the
# dictionary. Example: - If s = "cats", you must check
# "cat", "ats", and "cats" (and any other substrings of
# length >= 3).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the first failing substring.
# 2. The string is very long.
# 3. The dictionary is huge and sharded.
```

```
#
# INTERVIEW NOTES
# Every substring is a separate requirement, so no clever
# skipping exists: you must test about n^2 / 2 substrings.
# What you can improve is the cost per test and the early
# exit. A trie lets you extend a start position one
# character at a time without rehashing the whole
# substring, and the first failure ends everything.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
# Complexity: Time O(total characters), Space O(total characters)
def build_trie(words):
    # nested dictionaries, one letter per level; the key "$" marks a word end
    root = {}
    for word in words:
        node = root
        for ch in word:
            if ch not in node:
                node[ch] = {}
            node = node[ch]
        node["$"] = True
    return root


# ---- main question: main function ----
# Idea: every substring of length >= min_len (3) must be a word. For each
# start position i, walk down the trie along s[i:]. For the first
# min_len - 1 letters only walk; from depth min_len on, the node we are
# at must be marked as a word end. If the walk breaks (no word has this
# prefix) the answer is False when at least min_len letters remain from i,
# because then a substring of that length exists and is not a word.
# Complexity: Time O(n * longest word), Space O(total dictionary characters)
def all_substrings_words(s, words, min_len=3):
    root = build_trie(words)
    for i in range(len(s)):
        node = root
        depth = 0
        for j in range(i, len(s)):
            ch = s[j]
            depth += 1
            if ch not in node:
                return len(s) - i < min_len
            node = node[ch]
            if depth >= min_len and "$" not in node:
                return False
    return True


# ---- main question: tests ----
WORDS = {"aaa", "aaaa", "aaaaa"}


def test_main_question():
    assert all_substrings_words("aaaaa", WORDS)
```

```
    assert not all_substrings_words("aab", WORDS)           # "aab" is a substring that is not a word
    assert all_substrings_words("ab", set())                # shorter than 3 letters: nothing to check
    assert all_substrings_words("", WORDS)


# =====================================================
```

```
# FOLLOW-UP 1: Return the first failing substring
# =====================================================
# ---- follow-up 1: function ----
# Answer: the same idea, but return the substring itself instead of
# False, which is what a user wants to see. Check the SHORTER lengths
# first so the report is the smallest counterexample. (A length above
# the longest word fails at once.)
# Complexity: Time O(n^2 * L) in the worst case, Space O(dictionary)
def first_failing(s, words, min_len=3):
    words = set(words)
    longest = 0
    for word in words:
        if len(word) > longest:
            longest = len(word)
    for length in range(min_len, len(s) + 1):
        if length > longest:
            return s[:length]
        for i in range(len(s) - length + 1):
            piece = s[i:i + length]
            if piece not in words:
                return piece
    return None


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert first_failing("aaab", WORDS) == "aab"
    assert first_failing("aab", WORDS) == "aab"
    assert first_failing("aaaaa", WORDS) is None
    assert first_failing("aaaaaa", WORDS) == "aaaaaa"        # longer than every word
    for s in ("aaaaa", "aab", "aaab", "ab", "", "aaaaaa"):
        assert all_substrings_words(s, WORDS) == (first_failing(s, WORDS) is None)


# =====================================================
# FOLLOW-UP 2: The string is very long
# =====================================================
# Answer: a string of length n has about n^2 substrings, but they only
# need to be checked up to the length of the LONGEST word: a longer window
# fails at once. So at most n * (longest word) windows are walked.


# =====================================================
# FOLLOW-UP 3: The dictionary is huge and sharded
# =====================================================
```

```
# Answer: shard the dictionary (for example by the first letters) across
# machines and send each check to the shard that owns the prefix, or keep
# one compressed trie per shard.


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(s: str, dictionary: list[str]) -> bool:
```

Sample call

```
solution('cat', ['cat'])
```

Test cases

Input	Expected
`('cat', ['cat'])`	`True`
`('cats', ['cat', 'cas', 'cts', 'ats', 'cats'])`	`True`

STUDY PATH: question 97 of 197 | topic 17 of 43: Trie: nested dicts, one character per level | 6 of 7

Template fit: no trie needed: sort the words and bisect to the first match, then read forward while startswith(prefix) | * asked 225 days ago *

MEDIUM | IN APP LC 208 | PracHub | Technical Screen | 97 of 197

Return dictionary words matching a prefix

POSTED 2026-03-09 | 225 DAYS AGO | OLDER

STUDY: TALK ALL TIME 36 30 DAYS 0 PRIORITY 122/288

ABOUT THIS QUESTION

Topic: Trie: nested dicts, one character per level

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #208 Implement Trie (Prefix Tree)

LeetCode match: LC 208 Implement Trie (Prefix Tree)

Posted on PracHub: 2026-03-09 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/return-dictionary-words-matching-a-prefix

QUESTION

You are given a dictionary of words. Return all words that start with a given prefix. Matching is case-sensitive, duplicates in the dictionary are returned as often as they occur, and the result is an empty list when nothing matches. The index returns the matches in alphabetical order; limit (optional) stops after that many words.

Function signature

class PrefixIndex:

def __init__(self, words: list)

def matches(self, prefix: str, limit=None) -> list

EXAMPLES

Example 1

Input	`idx = PrefixIndex(['apple', 'app', 'Apple', 'apply', 'apple', 'bat'])`
	`idx.matches('app')`
	`idx.matches('app', limit=2)`
	`idx.matches('cat')`

Output	`idx.matches('app') -> ['app', 'apple', 'apple', 'apply']`
	`idx.matches('app', limit=2) -> ['app', 'apple']`
	`idx.matches('cat') -> []`

Why	`'Apple' starts with a capital letter so it does not match 'app'; 'apple' is listed twice because it is in the dictionary twice.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(words, prefix):
    found = [w for w in words
             if w.startswith(prefix)]
    return found
```

TIME	SPACE
O(W * L) per query	O(m)

BRUTE FORCE

WHY IT IS SLOW

Each query scans the whole dictionary, O(W * L). With the words sorted, binary search finds the block of matches directly.

OPTIMAL SOLUTION · Sort once, binary search the range

TIME	SPACE
O(log W + m) per query after O(W log W) sorting	**O(W)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only the first K matches (autocomplete).

Stop the forward walk after K words, as the limit parameter does; with the sorted list the first K in alphabetical order are already the first K in the block.

FOLLOW-UP 2 Words are added while queries run.

Keep the list sorted with bisect.insort (O(n) per insert, fast in practice) or use a balanced tree; a trie handles inserts in O(L).

FOLLOW-UP 3 Case-insensitive matching.

Store a casefolded sort key next to each word (a tuple (key, word)) and search on the key; do not lowercase the displayed word.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Return dictionary words matching a prefix
#
# You are given a dictionary of words. Return all words
# that start with a given prefix. Matching is case-
# sensitive, duplicates in the dictionary are returned as
# often as they occur, and the result is an empty list
# when nothing matches. The index returns the matches in
# alphabetical order; limit (optional) stops after that
# many words.
#
# Signature:
#   class PrefixIndex:
#       def __init__(self, words: list)
#       def matches(self, prefix: str, limit=None) -> list
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the first K matches (autocomplete).
# 2. Words are added while queries run.
# 3. Case-insensitive matching.
#
# INTERVIEW NOTES
# If the dictionary is sorted, all words with a prefix
# form one contiguous block, and its edges are found with
# binary search: the first word >= prefix, and the first
# word that is >= prefix with its last character
# incremented. For many queries on a static list this
```

```
# beats a trie in memory.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from bisect import bisect_left, bisect_right, insort


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: sort the words once. All words with a given prefix then sit
# together in ONE block. Binary search the start of the block (the first
# word >= prefix) and the end (just after the last word that starts with
# the prefix, found by searching for the prefix followed by the biggest
# possible character). `limit` stops after that many words (follow-up 1).
# Complexity: Time build O(n log n * L), matches O(L log n + size of the answer), add O(n), Space O(n)
class PrefixIndex:
    def __init__(self, words):
        self.words = sorted(words)

    def matches(self, prefix, limit=None):
        start = bisect_left(self.words, prefix)
        end = bisect_right(self.words, prefix + "\uffff")
        found = self.words[start:end]
        if limit is None:
            return found
        return found[:limit]

    # follow-up 2: words can be added while queries run. Keep the list
    # sorted by inserting at the right place (O(n) shifting, fast in practice).
    def add(self, word):
        insort(self.words, word)


# ---- main question: tests ----
def matches_scan(words, prefix):
    # the simple O(n * L) answer: look at every word
    found = []
    for word in words:
        if word.startswith(prefix):
            found.append(word)
    return found


WORDS = ["apple", "app", "apricot", "Apple", "banana", "app"]


def test_main_question():
    index = PrefixIndex(WORDS)
    assert index.matches("ap") == ["app", "app", "apple", "apricot"]     # duplicates stay; case matters
    assert index.matches("x") == []
    assert index.matches("") == sorted(WORDS)
    assert sorted(matches_scan(WORDS, "ap")) == index.matches("ap")


# ============================================================
# FOLLOW-UP 1: Only the first K matches (autocomplete)
# ============================================================
```

```
# ---- follow-up 1: tests ----
# Answer: the block is sorted, so the first K words of the block are the
# first K alphabetically: the `limit` parameter just cuts the block.
def test_follow_up_1():
    index = PrefixIndex(WORDS)
    assert index.matches("ap", limit=2) == ["app", "app"]


# ========================================================
# FOLLOW-UP 2: Words are added while queries run
# ========================================================
# ---- follow-up 2: tests ----
# Answer: keep the list sorted with insort (add above), or use a balanced
# tree; a trie also handles inserts in O(length of the word).
def test_follow_up_2():
    index = PrefixIndex(WORDS)
    index.add("apex")
    assert index.matches("ap") == ["apex", "app", "app", "apple", "apricot"]


# ========================================================
# FOLLOW-UP 3: Case-insensitive matching
# ========================================================
# ---- follow-up 3: function ----
# Answer: store (folded word, original word) pairs sorted by the folded
# word and search with the folded prefix; show the ORIGINAL words.
# casefold() is the proper way to ignore case.
# Complexity: Time build O(n log n * L), matches O(L log n + size of the answer), Space O(n)
class CaseInsensitiveIndex:
    def __init__(self, words):
        self.items = []
        for word in words:
            self.items.append((word.casefold(), word))
        self.items.sort()

    def matches(self, prefix):
        key = prefix.casefold()
        i = bisect_left(self.items, (key,))
        found = []
        while i < len(self.items) and self.items[i][0].startswith(key):
            found.append(self.items[i][1])
            i += 1
        return found
```

```
# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert CaseInsensitiveIndex(WORDS).matches("APP") == ["app", "app", "Apple", "apple"]


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], prefix: str) -> list[str]:
```

Sample call

```
solution(['apple', 'app', 'banana', 'apply'], 'app')
```

Test cases

Input	Expected
`(['apple', 'app', 'banana', 'apply'], 'app')`	`['apple', 'app', 'apply']`
`(['Cat', 'car', 'Cart', 'dog'], 'Ca')`	`['Cat', 'Cart']`

STUDY PATH: question 98 of 197 | topic 17 of 43: Trie: nested dicts, one character per level | 7 of 7

Template fit: insert is build_trie; search is walk and then END in node | * asked 242 days ago *

MEDIUM | IN APP LC 208 | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 98 of 197

Implement Trie Insert and Exact-Word Search

POSTED 2026-02-20 | 242 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 36 | 30 DAYS 0 | PRIORITY 123/288

ABOUT THIS QUESTION

Topic: Trie: nested dicts, one character per level

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #208 Implement Trie (Prefix Tree)

LeetCode match: LC 208 Implement Trie (Prefix Tree)

Posted on PracHub: 2026-02-20 (updated 2026-08-28)

Original: https://prachub.com/coding-questions/implement-trie-insert-and-exact-word-search

QUESTION

Implement a trie with insert(word), search(word) and startsWith(prefix). search is true only when the complete word was inserted earlier (a longer stored word that merely starts with it does not count); startsWith is true when some inserted word begins with the prefix.

Function signature

class Trie:

```
def insert(self, word: str) -> None
def search(self, word: str) -> bool
def startsWith(self, prefix: str) -> bool
```

EXAMPLES

Example 1

Input	`t = Trie()`
	`t.insert('apple')`
	`t.search('apple')`
	`t.search('app')`
	`t.startsWith('app')`
	`t.insert('app')`
	`t.search('app')`

Output	`t.search('apple') -> true`
	`t.search('app') -> false`
	`t.startsWith('app') -> true`
	`t.search('app') -> true`

Why	`'app' is only a prefix of 'apple' until it is inserted on its own.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
words = []
def insert(w):
    words.append(w)
def search(w):
    return w in words
def starts_with(p):
    for word in words:
        if word.startswith(p):
            return True
    return False
```

TIME	SPACE
O(W * L) per operation	**O(total characters)**

WHY IT IS SLOW

Each call scans every stored word, O(W * L). A trie with an end-of-word flag makes every call O(L).

OPTIMAL SOLUTION · Trie of nodes with an end-of-word flag

TIME	SPACE
O(L) per operation	**O(total characters)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Support '.' as a wildcard in search (LeetCode 211).

On '.', try every child at that level and recurse; other characters follow the single matching child. Worst case O(26^L) per query on adversarial patterns, so mention the bound.

FOLLOW-UP 2 Delete a word.

Unmark the end flag and, walking back up, remove nodes that have no children and are not word ends. Keep a path stack while walking down so you can prune on the way back. **Cost:** O(L) time and O(L) space for the path stack.

FOLLOW-UP 3 Unicode or large alphabets.

Store children in a dict keyed by character instead of a 26-slot array, as the code does; it handles any alphabet and wastes no space on missing letters. **Cost:** O(L) per operation with dictionary children; memory O(total characters).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement Trie Insert and Exact-Word
# Search
#
# Implement a trie with insert(word), search(word) and
# startsWith(prefix). search is true only when the
# complete word was inserted earlier (a longer stored word
# that merely starts with it does not count); startsWith
```

```
# is true when some inserted word begins with the prefix.
#
# Signature:
#   class Trie:
#       def insert(self, word: str) -> None
#       def search(self, word: str) -> bool
#       def startsWith(self, prefix: str) -> bool
#
# FOLLOW-UPS (answered below, in this order):
# 1. Support '.' as a wildcard in search (LeetCode 211).
# 2. Delete a word.
# 3. Unicode or large alphabets.
#
# INTERVIEW NOTES
# A trie needs a flag that marks 'a word ends here';
# without it search('app') would be true whenever 'apple'
# was inserted. Walking a prefix and checking the flag are
# the two operations: search checks the flag, startsWith
# does not.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class Node:
    def __init__(self):
        self.children = {}          # letter -> child node
        self.end = False            # does a word end here?


# ---- main question: main function ----
```

```
# Idea: each node has children by letter and an "a word ends here" flag.
# insert creates the missing nodes and marks the last one. search walks
# the word and also needs the flag at the end; startsWith only needs the
# walk to succeed.
# Complexity: Time insert, search and startsWith are O(L) each, Space O(total characters)
class Trie:
    def __init__(self):
        self.root = Node()

    def walk(self, text):
        # follow the letters of text; None if the path breaks
        node = self.root
        for ch in text:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node

    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = Node()
            node = node.children[ch]
        node.end = True
```

```
    def search(self, word):
        node = self.walk(word)
        return node is not None and node.end        # the whole word must have been inserted

    def startsWith(self, prefix):
        return self.walk(prefix) is not None


# ---- main question: tests ----
def test_main_question():
    t = Trie()
    t.insert("apple")
    assert t.search("apple")
    assert not t.search("app")                       # a shared prefix is not a match for search
    assert t.startsWith("app")
    assert not t.startsWith("b")
    t.insert("app")
    assert t.search("app")
```

```
# ============================================================
# FOLLOW-UP 1: Support "." as a wildcard in search (LeetCode 211)
# ============================================================
# ---- follow-up 1: function ----
# Answer: on ".", try every child at that level and recurse; any other
# letter follows its single child. The worst case is O(26^L) for a
# pattern of dots on a full trie, so mention the bound.
# Complexity: Time O(26^L) worst case for a pattern of dots (O(L) without dots), Space O(L)
def search_wild(trie, pattern):
    def go(node, i):
        if i == len(pattern):
            return node.end
        ch = pattern[i]
        if ch == ".":
            for child in node.children.values():
                if go(child, i + 1):
                    return True
            return False
        if ch not in node.children:
            return False
        return go(node.children[ch], i + 1)

    return go(trie.root, 0)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    t = Trie()
    for word in ("bad", "dad", "mad"):
        t.insert(word)
    assert search_wild(t, "b.d") and search_wild(t, ".ad")
    assert not search_wild(t, "b..d")
    assert search_wild(t, "...")
    assert not search_wild(t, "pad")


# ============================================================
# FOLLOW-UP 2: Delete a word
# ============================================================
# ---- follow-up 2: function ----
# Answer: remember the nodes on the path to the word, unmark the last
# one, then walk back up and remove every node that is no longer a word
# end and has no children (nothing else needs it).
# Complexity: Time O(L), Space O(L)
```

```
def delete(trie, word):
    path = [trie.root]
    for ch in word:
        child = path[-1].children.get(ch)
        if child is None:
            return False                                # the word is not there
        path.append(child)
    if not path[-1].end:
        return False
    path[-1].end = False
    for i in range(len(word), 0, -1):
        node = path[i]
        if node.end or len(node.children) > 0:
            break                                       # this node is still needed
        del path[i - 1].children[word[i - 1]]
    return True


# ---- follow-up 2: tests ----
def test_follow_up_2():
    t = Trie()
    t.insert("apple")
    t.insert("app")
    assert delete(t, "apple")                           # the shorter word that shares the path stays
    assert t.search("app") and not t.search("apple")
    assert not delete(t, "apple")
    assert delete(t, "app") and not t.startsWith("a")


# ======================================================
# FOLLOW-UP 3: Unicode or large alphabets
# ======================================================
# Answer: the children are a dictionary keyed by character, so any
# character works as it is. For a huge alphabet or very long words use a
# compressed trie, or hash each node's children into a shared table.


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def process_trie_operations(operations: list[list[str]]) -> list[bool]:
```

Sample call

```
process_trie_operations([['insert', 'apple'], ['search', 'apple'], ['search', 'app'], ['insert', 'app'], ['
search', 'app']])
```

Test cases

Input	Expected
`([['insert', 'apple'], ['search', 'apple'], ['search', 'app'], ['insert', 'app'], ['search', 'app']],)`	`[True, False, True]`
`([],)`	`[]`

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 18 OF 43

A dict for lookup plus an ordered container for who is next (lazy deletion)

6 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

6
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

A dict for lookup plus an ordered container for who is next (lazy deletion) | 6 questions | tap a title to jump to it

READ FIRST: the code template for these 6 questions (tap here) >

M	99. Design a restaurant waitlist system	PracHub	* 2026-03-01 *
M	100. Implement Python's itertools.tee	1Point3Acres	* 2026-08-20 *
M	101. Restaurant Waitlist: join / delete / find_first_match	1Point3Acres	* 2026-02-01 *
M	102. Store With Cheapest-Item Removal	ColorStack	* 2025-09-29 *
M	103. Round-Robin Job Scheduler	ColorStack	* 2025-08-19 *
M	104. Aggregate FizzBuzz Offer Matches	ColorStack	* 2025-10-13 *

CODE TEMPLATE | GROUP 18 OF 43 | 6 QUESTIONS, ONE PATTERN

A dict for lookup plus an ordered container for who is next (lazy deletion)

The one idea these questions share

Keep a dict for O(1) lookup and a deque, list or heap for the order. When something is removed, delete it from the dict only and skip stale entries when you read from the order. LRU caches, waitlists, undo stacks and schedulers all fit.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import deque


class OrderedStore:
    def __init__(self):
        self.order = deque()                  # CHANGE 1: the container that says who is next
        self.live = {}                        # key -> ticket, for O(1) lookup and lazy deletion
        self.ticket = 0

    def add(self, key):
        self.ticket += 1
        self.live[key] = self.ticket
        self.order.append((self.ticket, key))

    def remove(self, key):
        self.live.pop(key, None)              # lazy: the entry stays in the deque

    def next(self):
        while self.order and self.live.get(self.order[0][1]) != self.order[0][0]:
            self.order.popleft()              # CHANGE 2: skip entries that were removed
        return self.order[0][1] if self.order else None
```

2. Helper functions that usually get written next to it

```
from collections import OrderedDict


class LRU:                                    # the short version of the same idea
    def __init__(self, cap):
        self.cap, self.d = cap, OrderedDict()

    def get(self, k):
        if k not in self.d:
            return -1
        self.d.move_to_end(k)
        return self.d[k]

    def put(self, k, v):
        self.d[k] = v
        self.d.move_to_end(k)
        if len(self.d) > self.cap:
            self.d.popitem(last=False)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
99	Design a restaurant waitlist system	a segment tree of party sizes gives the first entry that fits in O(log n); the dict maps ids to slots

#	Question	What you change in the template
100	Implement Python's itertools.tee	one deque per consumer; whoever runs out pulls the next value from the source and appends it to every queue
101	Restaurant Waitlist: join / delete / find_first_match	a deque per party size with tickets; delete only removes the name from the dict; find_first_match skips dead tickets
102	Store With Cheapest-Item Removal	the order is a heap keyed by (price, timestamp); remove_cheapest pops it
103	Round-Robin Job Scheduler	a deque of [job, time left]; pop, run for min(quantum, left); push back at the end if unfinished, else record the finish time
104	Aggregate FizzBuzz Offer Matches	a deque of offers per type; a new offer matches against the front of the opposite queue, partial quantities stay at the front

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 99 of 197 | topic 18 of 43: A dict for lookup plus an ordered container for who is next (lazy deletion) | 1 of 6

Template fit: a segment tree of party sizes gives the first entry that fits in O(log n); the dict maps ids to slots | * asked 233 days ago *

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 99 of 197

★ TOP 75: rank 73 in the Medium and Hard priority list

★ Design a restaurant waitlist system

POSTED 2026-03-01 | 233 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 57.3 | 30 DAYS 0 | PRIORITY 56/288

ABOUT THIS QUESTION

Topic: A dict for lookup plus an ordered container for who is next (lazy deletion)

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #146 LRU Cache

LeetCode match: No direct match; related LC 146 LRU Cache (ordered map)

Posted on PracHub: 2026-03-01 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/design-a-restaurant-waitlist-system

QUESTION

You are implementing the waitlist system for a restaurant. Parties arrive over time, can cancel, and get seated when a table becomes available.

Rules

- Each party has a unique **`partyId`**, a **`name`**, and a **`size`** (number of people).
- Parties are seated in **arrival order**, but only if they **fit** the available table.
- When a table of capacity **`c`** becomes available, you must seat the **earliest-arrived** party whose **`size <= c`**.
- If no party fits, the table remains unused for that event.

Operations to support Design a

data structure / class that supports the following operations efficiently:

1. **`addParty(name, size) -> partyId`**
- Adds a new party to the end of the waitlist and returns its id.
2. **`cancelParty(partyId) -> bool`**
- Removes the party from the waitlist if present.
- Returns **`true`** if removed, otherwise **`false`**.
3. **`seatTable(capacity) -> partyId or null`**
- Finds and removes the earliest party with **`size <= capacity`**.
- Returns that party's id, or **`null`** if nobody fits.
4. **`getPosition(partyId) -> int`**
- Returns how many parties are ahead of this party in the current waitlist order (0-based).
- If the party is not in the waitlist, return **`-1`**.

Constraints

- Up to **`2 * 10^5`** operations.
- Party size is a small positive integer (e.g., 1–20).
- Aim for better-than-linear time per operation (especially for **`seatTable`** and **`getPosition`**).

Example

QUESTION

(one possible interaction) - addParty("A", 4) -> id1 - addParty("B", 2) -> id2 - seatTable(2) -> id2 (B fits and is earliest among those that fit) - seatTable(4) -> id1

EXAMPLES

Example 1

Input	`w = Waitlist()`
	`w.add(1, 'ann', 4)`
	`w.add(2, 'bob', 2)`
	`w.add(3, 'cy', 3)`
	`w.seat(3)`
	`w.cancel(3)`
	`w.seat(3)`
	`w.seat(4)`

Output	`w.seat(3) -> 2`
	`w.cancel(3) -> true`
	`w.seat(3) -> null`
	`w.seat(4) -> 1`

Why	A table for 3 takes the earliest party that fits: bob (2), not ann (4). After cy cancels, nobody left fits a table for 3; a table for 4 seats ann.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
parties = []  # in arrival order
def join(party):
    parties.append(party)
def seat(capacity):
    for party in parties:
        if party.size <= capacity:
            parties.remove(party)
            return party
    return None
def cancel(party_id):
    for party in parties:
        if party.id == party_id:
            parties.remove(party)
            return
```

TIME	SPACE
O(n)	**O(n)**
per seat and per cancel	

WHY IT IS SLOW

Both operations walk the list, O(n). A segment tree of the smallest size per range finds the leftmost party that fits in O(log n).

OPTIMAL SOLUTION · Segment tree of minimum size over arrival order

TIME	SPACE
O(log n)	**O(n)**
per operation	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Parties that wait too long should be seated first (starvation).

Add an aging rule: if the earliest party has waited longer than a threshold and a table of at least its size appears, seat it before anyone else. The segment tree can query 'leftmost size <= c' with c = the table, so order is preserved.

FOLLOW-UP 2 Seat the best-fitting party, not the earliest.

That is a different policy: keep sizes in a sorted structure of (size, arrival) and take the largest size at most the capacity (ties by arrival). A sorted container with bisect gives O(log n) lookups.

FOLLOW-UP 3 Estimate the waiting time of a party.

Track the arrival rate and typical table turnover per capacity; the estimate is the expected time until enough tables of at least that size free up, ahead of the party in the queue.

FOLLOW-UP 4 Seat the best-fitting party (the largest that fits), not the earliest.

Group waiting parties by size in queues (arrival order inside a size) and keep the sizes in a sorted list: the best fit for a table of capacity c is the largest size at most c, found with bisect, then the oldest party of that size. O(log distinct sizes) per seating. Verified against a linear scan.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Design a restaurant waitlist system
#
# You are implementing the waitlist system for a
# restaurant. Parties arrive over time, can cancel, and
# get seated when a table becomes available. ## Rules -
# Each party has a unique partyId, a name, and a size
# (number of people). - Parties are seated in **arrival
# order**, but only if they **fit** the available table. -
# When a table of capacity c becomes available, you must
# seat the **earliest-arrived** party whose size <= c. -
# If no party fits, the table remains unused for that
# event. ## Operations to support Design a data structure
# / class that supports the following operations
# efficiently: 1. addParty(name, size) -> partyId - Adds a
# new party to the end of the waitlist and returns its id.
# 2. cancelParty(partyId) -> bool - Removes the party from
# the waitlist if present. - Returns true if removed,
# otherwise false. 3.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Parties that wait too long should be seated first
# (starvation).
# 2. Seat the best-fitting party, not the earliest.
# 3. Estimate the waiting time of a party.
# 4. Seat the best-fitting party (the largest that fits),
# not the earliest.
#
# INTERVIEW NOTES
# 'Earliest arrival whose size fits' is a query on an
# ordered sequence: find the leftmost position whose value
# is at most c. A scan does it in O(n); a segment tree
# that stores the minimum size over each range answers it
# in O(log n) by walking down to the leftmost child whose
```

```
# range minimum fits. Cancelling or seating simply sets
# that position to infinity.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from bisect import bisect_right, insort
from collections import deque
```

```
INF = float("inf")


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: seat the EARLIEST party that fits the table. Each arriving party
# gets the next position (its arrival order). A segment tree over these
# positions stores, for each range, the SMALLEST party size waiting in it
# (infinity for empty or already seated positions). To seat a table of
# capacity c: if the smallest size overall is above c nobody fits;
# otherwise go down the tree, always LEFT first when the left half holds a
# party that fits, so we reach the earliest one.
# Complexity: Time add, cancel and seat are O(log capacity), Space O(capacity)
class Waitlist:
    def __init__(self, capacity=1024):
        self.cap = capacity                          # the most parties that can ever join
        self.tree = [INF] * (2 * capacity)           # leaves are tree[cap .. 2 * cap - 1]
        self.count = 0                               # how many parties have joined so far
        self.slot = {}                               # party id -> its position
        self.who = {}                                # position -> party id

    def update(self, position, value):
        i = position + self.cap
        self.tree[i] = value
        while i > 1:
            i //= 2
            left = self.tree[2 * i]
            right = self.tree[2 * i + 1]
            if left < right:
                self.tree[i] = left
            else:
                self.tree[i] = right

    def add(self, party_id, name, size):
        self.slot[party_id] = self.count
        self.who[self.count] = party_id
        self.update(self.count, size)
        self.count += 1

    def cancel(self, party_id):
        if party_id not in self.slot:
            return False
```

```
        position = self.slot[party_id]
        if self.tree[position + self.cap] == INF:
            return False                             # already seated or cancelled
        self.update(position, INF)
        return True
```

```
    def seat(self, capacity):
        if self.tree[1] > capacity:
            return None                                 # no waiting party is small enough
        i = 1
        while i < self.cap:
            if self.tree[2 * i] <= capacity:
                i = 2 * i                               # the left half holds a party that fits
            else:
                i = 2 * i + 1
        self.update(i - self.cap, INF)
        return self.who[i - self.cap]


# ---- main question: tests ----
def test_main_question():
    w = Waitlist(8)
    w.add("p1", "Ann", 4)
    w.add("p2", "Bob", 2)
    w.add("p3", "Cy", 6)
    assert w.seat(3) == "p2"                            # a table for 3: the earliest party that fits is Bob
    assert w.cancel("p1") and not w.cancel("p1")
    assert w.seat(6) == "p3"                            # Ann cancelled, so Cy gets the table for 6
    assert w.seat(10) is None


# =====================================================
# FOLLOW-UP 1: Parties that wait too long should be seated first (starvation)
# =====================================================
# Answer: a party that waits long enough gets priority: either age the
# parties (compare arrival time plus a bonus for waiting) or give every
# party a deadline and seat the ones past their deadline first.


# =====================================================
# FOLLOW-UP 2: Seat the best-fitting party, not the earliest
# =====================================================


# =====================================================
```

```
# FOLLOW-UP 4: Seat the best-fitting party (the largest that fits), not the earliest
# =====================================================
# ---- follow-up 2: function ----
# Answer: a different policy. Group the waiting parties by size in queues
# (arrival order inside a size) and keep the sizes in a sorted list. The
# best fit for a table of capacity c is the LARGEST size that is at most c,
# found with a binary search; take the oldest party of that size.
# Complexity: Time add and seat are O(n) (a list insert or removal; the searches are O(log n)), Space O(n)
class BestFit:
    def __init__(self):
        self.queues = {}                                # size -> parties of that size, oldest first
        self.sizes = []                                 # the sizes that have waiting parties, sorted

    def add(self, party_id, size):
        if size not in self.queues:
            self.queues[size] = deque()
        if len(self.queues[size]) == 0:
            insort(self.sizes, size)
        self.queues[size].append(party_id)

    def seat(self, capacity):
        k = bisect_right(self.sizes, capacity)          # sizes[:k] are the sizes that fit
        if k == 0:
            return None
        size = self.sizes[k - 1]
```

```
        party = self.queues[size].popleft()
        if len(self.queues[size]) == 0:
            self.sizes.pop(k - 1)
        return party


# ---- follow-up 2: tests ----
def test_follow_up_2():
    b = BestFit()
    for party_id, size in (("a", 2), ("b", 4), ("c", 4), ("d", 6)):
        b.add(party_id, size)
    assert b.seat(5) == "b"                          # a table for 5: the biggest fit is size 4, oldest fir
st
    assert b.seat(5) == "c"
    assert b.seat(5) == "a"
    assert b.seat(1) is None


# ============================================================
# FOLLOW-UP 3: Estimate the waiting time of a party
# ============================================================
```

```
# Answer: use the history of how long tables stay occupied (an average
# per table size) and count how many parties are ahead of this one that
# compete for the same tables; the estimate is that count times the
# average turnover time per table.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Test cases

Input	Expected
`[('addParty', 'A', 4), ('addParty', 'B', 2), ('seatTable', 2), ('seatTable', 4)]`	`[1, 2, 2, 1]`
`[('addParty', 'A', 4), ('addParty', 'B', 2), ('addParty', 'C', 3), ('getPosition', 3), ('seatTable', 2), ('getPosition', 3), ('cancelParty', 1), ('getPosition', 3), ('cancelParty', 1), ('seatTable', 2), ('seatTable', 3), ('getPosition', 3)]`	`[1, 2, 3, 2, 2, 1, True, 0, False, None, 3, -1]`

ALSO REPORTED · 1 more

Restaurant Waitlist: join / delete / find_first_match | 1Point3Acres | onsite-coding | asked 2026-02-01 | Single report

https://www.1point3acres.com/interview/problems/company/google/waitlist-data-structure-with-table-size-match

ALSO REPORTED · 1 more

A doubly-linked list + hashmap (LRU-style) gives O(1) join and delete; find_first_match then walks the list.
For faster find, bucket users by party size (e.g. one queue per size), and on find_first_match(t) scan sizes 1..t and pick the queue with the smallest head timestamp.
The interviewer cares more about clean class design + tradeoff discussion than a single optimal solution; mention what you'd cache to speed up repeated finds.

Preparation: Drill LRU-cache style implementations (hashmap + DLL) to muscle memory.
Sketch a second variant with party-size buckets and verbalize the trade-off (find_first_match becomes O(table_size) instead of O(n) but uses more memory).
Practice articulating which call should be fast in a real restaurant — typically find_first_match runs once per available table.

STUDY PATH: question 100 of 197 | topic 18 of 43: A dict for lookup plus an ordered container for who is next (lazy deletion) | 2 of 6

Template fit: one deque per consumer; whoever runs out pulls the next value from the source and appends it to every queue | * asked 61 days ago * | frequency Single report

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 100 of 197

Implement Python's itertools.tee

ASKED 2026-08-20 | 61 DAYS AGO | LAST 90 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 285/288**

ABOUT THIS QUESTION

Topic: A dict for lookup plus an ordered container for who is next (lazy deletion)

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-08-20 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/python-tee-independent-iterators

QUESTION

Implement itertools.tee: given an iterable and n, return n independent iterators that each produce the same values from the one source. The source must be read only once, and a value must be kept only until every iterator has taken it.

Function signature

def tee_queues(iterable, n=2) -> tuple

EXAMPLES

Example 1

Input	`a, b = tee_queues([1, 2, 3])`
	`next(a)`
	`next(a)`
	`next(b)`
	`list(b)`
	`list(a)`

Output	`next(a) -> 1`
	`next(a) -> 2`
	`next(b) -> 1`
	`list(b) -> [2, 3]`
	`list(a) -> [3]`

Why	`Iterator a runs ahead by two values; b still sees every value from the start, and neither disturbs the other.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def tee(source, n):
    items = list(source)  # read it all now
    def reader():
        for x in items:
            yield x
    return [reader() for _ in range(n)]
```

BRUTE FORCE

TIME	SPACE
O(n) up front	**O(n)**

WHY IT IS SLOW

It reads the source completely before anything is consumed, never works on an infinite or lazy source, and keeps every item in memory. One buffer per consumer (or a shared linked list) holds only what the slowest consumer has not read.

OPTIMAL SOLUTION · One deque per consumer, filled lazily

TIME	SPACE
O(n) total, O(1) amortized per item per consumer	**O(lag between fastest and slowest consumer)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Memory grows when one consumer stalls. Is there a way to free items automatically?

Use one shared singly linked list with a pointer per consumer. The only references to old nodes are the consumers' pointers, so Python's garbage collector frees nodes behind the slowest consumer without any bookkeeping. This is the design CPython uses.

FOLLOW-UP 2 Make it safe for several threads.

Pull from the source under one lock and append to the queues inside it. Without the lock two consumers can pull two items and append them in different orders to different queues. **Cost:** one lock acquisition per item pulled, O(1) extra per item; contention grows with the number of consumers.

FOLLOW-UP 3 Why not just copy the iterator?

Most iterators (files, generators, sockets) cannot be copied. tee gives independent views by buffering, which is the only thing you can do when the source cannot rewind.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement Python's itertools.tee
#
# Implement itertools.tee: given an iterable and n, return
# n independent iterators that each produce the same
# values from the one source. The source must be read only
# once, and a value must be kept only until every iterator
# has taken it.
#
# Signature:
#   def tee_queues(iterable, n=2) -> tuple
#
# FOLLOW-UPS (answered below, in this order):
# 1. Memory grows when one consumer stalls. Is there a way
# to free items automatically?
# 2. Make it safe for several threads.
# 3. Why not just copy the iterator?
```

```
#
# INTERVIEW NOTES
# The source can only be consumed once, so someone must
# remember items until every consumer has read them. The
# state to keep is exactly 'items the slowest consumer has
# not read yet', so either give each consumer a queue that
# the leader fills, or keep one shared linked list with a
# pointer per consumer so memory is freed automatically.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import threading
from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: one waiting line (deque) of values per consumer. When a
# consumer's line is empty it pulls the next value from the source,
# gives it to EVERY consumer's line, and then takes its own copy. Each
# consumer is a generator, so values are read from the source only when
# somebody asks for them (it works on endless sources too).
```

```
# Complexity: Time O(1) per value per consumer, Space O(values the slowest consumer has not read yet)
def tee_queues(iterable, n=2):
    source = iter(iterable)
    queues = []
    for i in range(n):
        queues.append(deque())

    def consumer(mine):
        while True:
            if len(mine) == 0:
                try:
                    value = next(source)
                except StopIteration:
                    return
                for q in queues:
                    q.append(value)              # give the new value to everybody
            yield mine.popleft()

    consumers = []
    for q in queues:
        consumers.append(consumer(q))
    return tuple(consumers)


# ---- main question: tests ----
def test_main_question():
    a, b = tee_queues([1, 2, 3])
    assert next(a) == 1
    assert next(a) == 2
    assert next(b) == 1
    assert list(b) == [2, 3]
    assert list(a) == [3]
    x, y, z = tee_queues("abc", 3)
    assert list(x) == list(y) == list(z) == list("abc")
```

```
# =========================================================
# FOLLOW-UP 1: Memory grows when one consumer stalls. Is there a way to free items automatically?
# =========================================================
# ---- follow-up 1: helper functions ----
class Cell:
    # one link of the shared list: a value (once it has been read from the source) and the next link
    def __init__(self):
        self.filled = False
        self.value = None
        self.next = None


# ---- follow-up 1: function ----
# Answer: use ONE shared singly linked list with a pointer per consumer.
# The only references to old cells are the consumers' pointers, so Python's
# garbage collector frees the cells behind the slowest consumer by itself,
# with no bookkeeping.
# Complexity: Time O(1) per value per consumer, Space O(values the slowest consumer has not read yet) (free
d automatically)
def tee_linked(iterable, n=2):
    source = iter(iterable)

    def consumer(cell):
        while True:
            if not cell.filled:                    # the first reader to arrive pulls the value
                try:
                    cell.value = next(source)
                except StopIteration:
                    return
                cell.filled = True
                cell.next = Cell()
            value = cell.value
            cell = cell.next
            yield value

    head = Cell()                                  # every reader starts at the same first cell
    consumers = []
    for i in range(n):
        consumers.append(consumer(head))
    return tuple(consumers)


# ---- follow-up 1: tests ----
def count_forever():
    i = 0
    while True:
        yield i
        i += 1


def test_follow_up_1():
    a, b = tee_linked(range(5))
    assert next(a) == 0 and next(a) == 1               # readers are independent, at any speed
    assert list(b) == [0, 1, 2, 3, 4]
    assert list(a) == [2, 3, 4]

    e1, e2 = tee_linked(count_forever())               # works on an endless source
    assert [next(e1), next(e1), next(e1)] == [0, 1, 2]
    assert next(e2) == 0


# =========================================================
# FOLLOW-UP 2: Make it safe for several threads
```

```
# ============================================================
# ---- follow-up 2: function ----
# Answer: pull from the source under ONE lock, and fill the cell inside
# it. Without the lock two consumers could pull two values at the same
# time and hand them out in different orders.
# Complexity: Time O(1) per value per consumer plus waiting for the lock, Space O(values the slowest consum
er has not read yet)
def tee_threadsafe(iterable, n=2):
    source = iter(iterable)
    lock = threading.Lock()

    def consumer(cell):
        while True:
            with lock:
                if not cell.filled:
                    try:
                        cell.value = next(source)
                    except StopIteration:
                        return
                    cell.filled = True
                    cell.next = Cell()
                value = cell.value
                cell = cell.next
            yield value

    head = Cell()
    consumers = []
    for i in range(n):
        consumers.append(consumer(head))
    return tuple(consumers)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    results = []

    def read_all(it):
        results.append(list(it))
```

```
    a, b, c = tee_threadsafe(range(1000), 3)
    workers = []
    for it in (a, b, c):
        worker = threading.Thread(target=read_all, args=(it,))
        workers.append(worker)
        worker.start()
    for worker in workers:
        worker.join()
    assert len(results) == 3
    for values in results:
        assert values == list(range(1000))             # every thread sees all values in order


# ============================================================
# FOLLOW-UP 3: Why not just copy the iterator?
# ============================================================
# Answer: most iterators (generators, files, network streams) cannot be
# copied, and a copy would run the source's side effects twice or share
# its position. tee reads the source ONCE and buffers what the slower
# consumers still need.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement a custom Python iterator class from memory, including __iter__ and __next__.
Trace two and three consumers advancing at different speeds and identify what state must remain available.
Compare a fully materialized baseline with a lazy shared-state design and explain the memory behavior.

STUDY PATH: question 101 of 197 | topic 18 of 43: A dict for lookup plus an ordered container for who is next (lazy deletion) | 3 of 6

Template fit: a deque per party size with tickets; delete only removes the name from the dict; find_first_match skips dead tickets | * asked 261 days ago * | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 101 of 197

Restaurant Waitlist: join / delete / find_first_match

ASKED 2026-02-01 | 261 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 276/288

ABOUT THIS QUESTION

Topic: A dict for lookup plus an ordered container for who is next (lazy deletion)

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-02-01 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/waitlist-data-structure-with-table-size-match

QUESTION

Design a restaurant waitlist. join(name, size) adds a party with its size at the back. delete(name) removes a party (it may be anywhere in the queue). find_first_match(table) returns the name of the party that joined earliest among those whose size fits the table (size at most table), or None.

Function signature

class Waitlist:
    def join(self, name: str, size: int) -> None
    def delete(self, name: str) -> None
    def find_first_match(self, table: int) -> str | None

EXAMPLES

Example 1

Input	`w = Waitlist()`
	`w.join('ann', 4)`
	`w.join('bob', 2)`
	`w.join('cy', 3)`
	`w.find_first_match(3)`
	`w.delete('bob')`
	`w.find_first_match(3)`
	`w.find_first_match(1)`

Output	`w.find_first_match(3) -> 'bob'`
	`w.find_first_match(3) -> 'cy'`
	`w.find_first_match(1) -> null`

Why	`A table for 3 skips ann (4 is too big) and takes bob, who joined before cy. After bob leaves it is cy. No party fits a table for 1.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
queue = []  # in arrival order
def join(user):
    queue.append(user)
def delete(user):
    queue.remove(user)
def find_first_match(t):
    for user in queue:
        if user.party_size <= t:
            return user
    return None
```

TIME	SPACE
O(n) per find and per delete	O(n)

WHY IT IS SLOW

Delete and find are linear in the list. One queue per party size with lazy deletion makes join and delete O(1) and find touch only the queue fronts.

OPTIMAL SOLUTION · Queue per party size with lazy deletion

TIME	SPACE
O(1) join and delete, O(S) find (S distinct sizes up to the table size)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What would you cache to speed up repeated finds?

Keep a small structure that answers 'earliest sequence among sizes <= t' directly: a segment tree or Fenwick tree over party size storing the minimum front sequence number, updated when a queue front changes. Find becomes O(log S) instead of O(S).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Restaurant Waitlist: join / delete /
# find_first_match
#
# Design a restaurant waitlist. join(name, size) adds a
# party with its size at the back. delete(name) removes a
# party (it may be anywhere in the queue).
# find_first_match(table) returns the name of the party
# that joined earliest among those whose size fits the
# table (size at most table), or None.
#
# Signature:
#   class Waitlist:
#       def join(self, name: str, size: int) -> None
#       def delete(self, name: str) -> None
#       def find_first_match(self, table: int) -> str | None
#
# FOLLOW-UPS (answered below, in this order):
# 1. What would you cache to speed up repeated finds?
```

```
#
# INTERVIEW NOTES
# Join needs the order of arrival, delete can hit anyone,
# and find must return the earliest person whose party
# fits the table. Keep one queue per party size and a
# sequence number per person: find looks at the front of
# every queue for sizes up to the table size and takes the
# smallest sequence number. Deletions are marked and
# skipped lazily when a queue front is read.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: one queue per party size, plus a dictionary of the people who
# are still waiting. join puts (arrival number, name) in the queue of its
# size. delete only removes the person from the dictionary ("lazy
```

```
# deletion": the old entry stays in its queue and is skipped later).
# find_first_match looks at the front of every queue whose size fits the
# table (after removing deleted people from the front) and returns the
# earliest arrival among them. It does not seat anybody.
# Complexity: Time join O(1), delete O(1), find_first_match O(S + deleted entries skipped) for S party size
s, Space O(n)
class Waitlist:
    def __init__(self):
        self.counter = 0                            # arrival number: lower means earlier
        self.waiting = {}                           # name -> arrival number (people still waiting)
        self.queues = {}                            # party size -> deque of (arrival number, name)

    def join(self, name, size):
        self.counter += 1
        self.waiting[name] = self.counter
        if size not in self.queues:
            self.queues[size] = deque()
        self.queues[size].append((self.counter, name))

    def delete(self, name):
        if name in self.waiting:
            del self.waiting[name]

    def front(self, queue):
        # drop deleted (or re-joined) people from the front, then look at the first one
        while queue:
            number, name = queue[0]
            if self.waiting.get(name) == number:
                return queue[0]
            queue.popleft()
        return None

    def find_first_match(self, table):
        best = None
        for size in self.queues:
            if size > table:
                continue                            # this party is too big for the table
```

```
            head = self.front(self.queues[size])
            if head is not None and (best is None or head[0] < best[0]):
                best = head
        if best is None:
            return None
        return best[1]


# ---- main question: tests ----
```

```
def test_main_question():
    w = Waitlist()
    w.join("Alice", 2)
    w.join("Bob", 4)
    w.join("Carol", 6)
    assert w.find_first_match(3) == "Alice"
    assert w.find_first_match(5) == "Alice"          # Alice is still the earliest who fits 5
    assert w.find_first_match(5) == "Alice"          # finding does not remove anyone
    w.delete("Alice")
    assert w.find_first_match(5) == "Bob"
    assert w.find_first_match(1) is None
    w.join("Dan", 4)
    w.delete("Bob")
    assert w.find_first_match(4) == "Dan"            # the same size: arrival order decides
    w.delete("Nobody")                               # deleting someone who is not there is harmless


# ========================================================
# FOLLOW-UP 1: What would you cache to speed up repeated finds?
# ========================================================
# Answer: cache the answer per table size and clear (or update) it when a
# party joins or is deleted; or keep, for each size, a pointer to its
# current front so repeated finds do not skip deleted entries again.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill LRU-cache style implementations (hashmap + DLL) to muscle memory.
Sketch a second variant with party-size buckets and verbalize the trade-off (find_first_match becomes O(table_size) instead of O(n) but uses more memory).
Practice articulating which call should be fast in a real restaurant — typically find_first_match runs once per available table.

STUDY PATH: question 102 of 197 | topic 18 of 43: A dict for lookup plus an ordered container for who is next (lazy deletion) | 4 of 6

Template fit: the order is a heap keyed by (price, timestamp); remove_cheapest pops it | * asked 386 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 102 of 197

Store With Cheapest-Item Removal

ASKED 2025-09-29 | 386 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 223/288

ABOUT THIS QUESTION

Topic: A dict for lookup plus an ordered container for who is next (lazy deletion)

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #1048 Longest String Chain

LeetCode match: LC 1048 / design: priority store (related: heap with ties)

Frequency: Single report | Last asked: 2025-09-29 | Roles: SWE Intern

QUESTION

Design a Store that holds items. Each item has a string value , an integer price in cents, and an integer timestamp . Support: add(value, price, timestamp) : insert an item. remove_cheapest() : remove and return the value of the item with the smallest price. If several items share the smallest price, remove the one with the smallest timestamp . Return None if the store is empty. A heap keyed by (price, timestamp) gives O(log n) per operation.

Function signature

```
class Store:
    def __init__(self) -> None:
        ...
    def add(self, value: str, price: int, timestamp: int) -> None:
        ...
    def remove_cheapest(self) -> str | None:
        """Remove and return the value of the lowest-priced item (ties broken by
earliest timestamp)."""
        ...
```

Follow-up (as reported): Improve remove_cheapest from O(log n) to O(1) amortized using the price bound: keep a length-10001 array indexed by price (each slot a queue/count of items) and track the smallest non-empty price index seen so far.

EXAMPLES

Example 1	
Input	`add("a",5,1); add("b",2,2); add("c",8,3); remove_cheapest()`
Output	`"b"`
Why	`"b" has the smallest price (2 cents).`

Example 2	
Input	`add("x",7,10); add("y",7,4); remove_cheapest()`
Output	`"y"`
Why	`Prices tie at 7; "y" wins on the earlier timestamp (4 < 10).`

QUESTION

Example 3	
Input	`remove_cheapest() # on an empty store`
Output	`None`

CONSTRAINTS

- Prices are integers with 1 <= price <= 10000 (i.e. 1 cent up to $100.00).
- Up to 10^6 operations.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
class BruteStore:
    def __init__(self):
        self.items = []
    def add(self, value, price, timestamp):
        self.items.append(
            (price, timestamp, value)
        )
    def remove_cheapest(self):
        if not self.items:
            return None
        item = min(
            self.items,
            key=lambda x: (x[0], x[1]),
        )
        self.items.remove(item)
        return item[2]
```

TIME	SPACE
O(n) per removal	**O(n)**

WHY IT IS SLOW

remove_cheapest looks at every item.

OPTIMAL SOLUTION · A heap keyed by price and timestamp

TIME	SPACE
O(log n) per operation	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Prices are integers from 0 to 10000. Can removal be O(1) amortized?

Use an array of 10001 buckets, one per price, each a small heap by timestamp, and a pointer to the lowest price that might be non-empty. add lowers the pointer if needed; remove_cheapest moves it up past empty buckets, so the pointer only moves up between adds.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Store With Cheapest-Item Removal
#
# Design a Store that holds items. Each item has a string
# value , an integer price in cents, and an integer
# timestamp . Support: add(value, price, timestamp) :
# insert an item. remove_cheapest() : remove and return
# the value of the item with the smallest price. If
# several items share the smallest price, remove the one
# with the smallest timestamp . Return None if the store
# is empty. A heap keyed by (price, timestamp) gives O(log
# n) per operation. Function signature: class Store: def
# __init__(self) -> None: ... def add(self, value: str,
# price: int, timestamp: int) -> None: ... def
# remove_cheapest(self) -> str | None: """Remove and
# return the value of the lowest-priced item (ties broken
# by earliest timestamp)""" ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Prices are integers from 0 to 10000. Can removal be
# O(1) amortized?
#
# INTERVIEW NOTES
# Removing the cheapest item with ties broken by the
# earliest timestamp is a priority queue keyed by (price,
# timestamp). A heap gives O(log n) per operation. When
# prices are bounded, an array of buckets indexed by price
# plus a pointer to the lowest non-empty price gives O(1)
# amortized removal.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: a min-heap of (price, timestamp, value). The smallest tuple is
# always on top: the cheapest item, and among equal prices the one with
```

```
# the earliest timestamp.
# Complexity: Time add and remove_cheapest are O(log n), Space O(n)
class Store:
    def __init__(self):
        self.heap = []

    def add(self, value, price, timestamp):
        heapq.heappush(self.heap, (price, timestamp, value))

    def remove_cheapest(self):
        if len(self.heap) == 0:
            return None
        price, timestamp, value = heapq.heappop(self.heap)
```

```
        return value


# ---- main question: tests ----
def test_main_question():
    s = Store()
    s.add("a", 5, 1)
    s.add("b", 2, 2)
    s.add("c", 8, 3)
    assert s.remove_cheapest() == "b"
    s = Store()
    s.add("x", 7, 10)
    s.add("y", 7, 4)
    assert s.remove_cheapest() == "y"               # the same price: the earlier timestamp
    assert Store().remove_cheapest() is None


# =====================================================
# FOLLOW-UP 1: Prices are integers from 0 to 10000. Can removal be O(1) amortized?
# =====================================================
# ---- follow-up 1: function ----
# Answer: use an array with one bucket per price (10001 of them), each a
# small heap ordered by timestamp, and a pointer `low` to the lowest price
# that might still have items. Removal moves the pointer up past empty
# buckets; the pointer only moves down when a cheaper item is added, so
# the pointer work is O(1) amortized for a fixed range of prices.
# Complexity: Time add O(log n), remove_cheapest O(1) amortized for a fixed price range, Space O(number of
prices + n)
class BucketStore:
    MAX_PRICE = 10000

    def __init__(self):
        self.buckets = []
```

```
        for price in range(self.MAX_PRICE + 1):
            self.buckets.append([])
        self.low = self.MAX_PRICE + 1

    def add(self, value, price, timestamp):
        heapq.heappush(self.buckets[price], (timestamp, value))
        if price < self.low:
            self.low = price

    def remove_cheapest(self):
        while self.low <= self.MAX_PRICE and len(self.buckets[self.low]) == 0:
            self.low += 1
        if self.low > self.MAX_PRICE:
            return None
        timestamp, value = heapq.heappop(self.buckets[self.low])
        return value


# ---- follow-up 1: tests ----
def test_follow_up_1():
    s = BucketStore()
    s.add("a", 5, 1)
    s.add("b", 2, 2)
    s.add("c", 8, 3)
    assert s.remove_cheapest() == "b"
    s.add("d", 5, 0)
    assert s.remove_cheapest() == "d"               # price 5 twice: the earlier timestamp
    assert s.remove_cheapest() == "a"
    assert s.remove_cheapest() == "c"
    assert s.remove_cheapest() is None
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 103 of 197 | topic 18 of 43: A dict for lookup plus an ordered container for who is next (lazy deletion) | 5 of 6

Template fit: a deque of [job, time left]; pop, run for min(quantum, left); push back at the end if unfinished, else record the finish time | * asked 427 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 103 of 197

Round-Robin Job Scheduler

ASKED 2025-08-19 | 427 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 214/288

ABOUT THIS QUESTION

Topic: A dict for lookup plus an ordered container for who is next (lazy deletion)

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1700 Number of Students Unable to Eat Lunch (related: queue simulation)

Frequency: Single report | Last asked: 2025-08-19 | Roles: SWE Intern

QUESTION

The third question was "not really leetcode style" but was a job-scheduling problem solved with a round- robin approach. Reconstruct it as follows. You are given a list of jobs, each with a unique string id and a positive integer burst (the number of time units of CPU it needs). All jobs are available at time 0 and are placed in a ready queue in the given order. The CPU runs jobs round-robin with a fixed time quantum : it takes the job at the front of the queue, runs it for min(quantum, remaining) time units, and then — if the job still has work left — puts it at the back of the queue; otherwise the job completes at that moment. Return the list of (id, completion_time) pairs in the order the jobs complete.

Function signature

```
def round_robin(jobs: list[tuple[str, int]], quantum: int) -> list[tuple[str, int]]:
    """Simulate round-robin scheduling; return (id, completion_time) pairs in completion
order."""
    ...
```

Follow-up (as reported): Report note ("a constraint was changed"): add per-job arrival times so a job only enters the queue once the clock reaches its arrival. Hint: keep a pointer over arrival-sorted jobs and admit them each time the clock advances.

EXAMPLES

Example 1	
Input	`jobs = [("A", 4), ("B", 3), ("C", 2)], quantum = 2`
Output	`[("C", 6), ("A", 8), ("B", 9)]`
Why	`t=0-2 A (rem 2), t=2-4 B (rem 1), t=4-6 C completes at 6, t=6-8 A completes at 8, t=8-9 B completes at 9.`

Example 2	
Input	`jobs = [("X", 5)], quantum = 2`
Output	`[("X", 5)]`
Why	`One job, run in slices 2+2+1, finishing at time 5.`

QUESTION

Example 3	
Input	`jobs = [("P", 1), ("Q", 1)], quantum = 3`
Output	`[("P", 1), ("Q", 2)]`
Why	`Each finishes within a single quantum: P at 1, Q at 2.`

CONSTRAINTS

- 1 <= len(jobs) <= 10^4
- 1 <= burst <= 10^6 , 1 <= quantum <= 10^6
- Job ids are unique, non-empty strings.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_round_robin(jobs, quantum):
    remaining = [b for _, b in jobs]
    clock, done = 0, []
    while len(done) < len(jobs):
        for i, (jid, _) in enumerate(jobs):
            if remaining[i] > 0:
                run = min(
                    quantum, remaining[i]
                )
                clock += run
                remaining[i] -= run
                if remaining[i] == 0:
                    done.append((jid, clock))
    return done
```

TIME	SPACE
O(n * rounds)	O(n)

WHY IT IS SLOW

It rescans the whole list every round, including jobs that already finished.

OPTIMAL SOLUTION · A queue of jobs

TIME	SPACE
O(total burst / quantum + n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Each job has an arrival time. How do you change it?

Sort the jobs by arrival. Before every pick admit all jobs whose arrival is at most the clock; a job preempted after its slice goes to the back after the jobs that arrived during the slice. If the queue is empty, jump the clock to the next arrival.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Round-Robin Job Scheduler
#
# The third question was "not really leetcode style" but
# was a job-scheduling problem solved with a round- robin
# approach. Reconstruct it as follows. You are given a
# list of jobs, each with a unique string id and a
# positive integer burst (the number of time units of CPU
# it needs). All jobs are available at time 0 and are
# placed in a ready queue in the given order. The CPU runs
# jobs round-robin with a fixed time quantum : it takes
# the job at the front of the queue, runs it for
# min(quantum, remaining) time units, and then — if the
# job still has work left — puts it at the back of the
# queue; otherwise the job completes at that moment.
# Return the list of (id, completion_time) pairs in the
# order the jobs complete.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Each job has an arrival time. How do you change it?
#
# INTERVIEW NOTES
# Round-robin is a queue simulation: take the front job,
# run it for min(quantum, remaining), and push it to the
# back if it still has work. The clock only moves forward
# by the time actually run, so completion times fall out
# directly. An arrival time just adds a second queue of
# jobs not yet released.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: a queue of jobs. Take the job at the front, run it for one time
# slice (at most `quantum`), and if it is not finished put it at the
```

```
# BACK of the queue. Record the time each job finishes.
# jobs is a list of (id, burst time); the result is a list of
# (id, finish time) in the order the jobs finish.
# Complexity: Time O(n + T / quantum) time slices for total burst time T, Space O(n)
def round_robin(jobs, quantum):
    queue = deque()
    for job_id, burst in jobs:
        queue.append([job_id, burst])
    clock = 0
    finished = []
    while queue:
        job = queue.popleft()
        run = quantum
```

```
        if job[1] < run:
            run = job[1]
        clock += run
        job[1] -= run
        if job[1] > 0:
            queue.append(job)                       # not finished: back of the line
        else:
            finished.append((job[0], clock))
    return finished


# ---- main question: tests ----
def test_main_question():
    assert round_robin([("A", 4), ("B", 3), ("C", 2)], 2) == [("C", 6), ("A", 8), ("B", 9)]
    assert round_robin([("X", 5)], 2) == [("X", 5)]
    assert round_robin([("P", 1), ("Q", 1)], 3) == [("P", 1), ("Q", 2)]
    assert round_robin([], 2) == []


# ========================================================
# FOLLOW-UP 1: Each job has an arrival time. How do you change it?
# ========================================================
# ---- follow-up 1: helper functions ----
def arrival_of(job):
    return job[2]                                   # a job is (id, burst, arrival)


# ---- follow-up 1: function ----
# Answer: sort the jobs by arrival. Before every pick, admit all jobs whose
# arrival time has been reached; if the line is empty, jump the clock to
# the next arrival. A job that used its slice but is not finished goes to
# the back AFTER the jobs that arrived while it was running.
```

```
# Complexity: Time O(n log n + T / quantum), Space O(n)
def round_robin_arrivals(jobs, quantum):
    pending = deque(sorted(jobs, key=arrival_of))
    queue = deque()
    clock = 0
    finished = []
    while pending or queue:
        if not queue and pending[0][2] > clock:
            clock = pending[0][2]                   # nothing to run: wait for the next arrival
        while pending and pending[0][2] <= clock:
            job_id, burst, arrival = pending.popleft()
            queue.append([job_id, burst])
        job = queue.popleft()
        run = quantum
        if job[1] < run:
            run = job[1]
        clock += run
        job[1] -= run
        while pending and pending[0][2] <= clock:    # jobs that arrived while this one ran line up before
it
            job_id, burst, arrival = pending.popleft()
            queue.append([job_id, burst])
        if job[1] > 0:
            queue.append(job)
        else:
            finished.append((job[0], clock))
    return finished


# ---- follow-up 1: tests ----
def test_follow_up_1():
    random.seed(3)
```

```
    for round_number in range(300):
        jobs = []
        for i in range(random.randint(1, 6)):
            jobs.append((str(i), random.randint(1, 7)))
        quantum = random.randint(1, 4)
        with_arrivals = []
        for job_id, burst in jobs:
            with_arrivals.append((job_id, burst, 0))
        assert round_robin_arrivals(with_arrivals, quantum) == round_robin(jobs, quantum)   # all arrive at
 time 0
    assert round_robin_arrivals([("A", 3, 0), ("B", 1, 5)], 2) == [("A", 3), ("B", 6)]


# =====================================================
# RUN EVERYTHING
```

```
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 104 of 197 | topic 18 of 43: A dict for lookup plus an ordered container for who is next (lazy deletion) | 6 of 6

Template fit: a deque of offers per type; a new offer matches against the front of the opposite queue, partial quantities stay at the front | * asked 372 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 104 of 197

Aggregate FizzBuzz Offer Matches

ASKED 2025-10-13 | 372 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 226/288

ABOUT THIS QUESTION

Topic: A dict for lookup plus an ordered container for who is next (lazy deletion)

Optimal answer from: written and tested here for this exact question

LeetCode match: Order-book matching with queues (related: LC 1801 Number of Orders in the Backlog)

Frequency: Single report | Last asked: 2025-10-13 | Roles: SWE Intern

QUESTION

You are given a list of offers in priority order (earliest = highest priority). Each offer is a dict with keys type (either "Fizz" or "Buzz"), name (a string), and quantity (a positive integer). Fizz offers match against Buzz offers on a first-come, first-served basis, and offers may be partially fulfilled. Process the offers in order, maintaining a FIFO queue of unmatched Fizz offers and a FIFO queue of unmatched Buzz offers. When a new offer arrives, match it against the front of the opposite queue repeatedly, each time consuming min(remaining quantities) and recording a match (fizz_name, buzz_name, matched_quantity) ; drop an offer from the queue once its quantity hits 0. If the arriving offer still has quantity left when the opposite queue empties, enqueue it in its own queue. Aggregate matches so the same (fizz_name, buzz_name) pair is reported once with its summed quantity. Return the aggregated matches, in the order each pair was first produced, formatted as: "FizzBuzz! <quantity> <FizzName>:<BuzzName>" .

Function signature

```
def fizzbuzz_matches(offers: list[dict]) -> list[str]:
    ...
```

EXAMPLES

Example 1

Input	`offers = [ {"type": "Fizz", "name": "A", "quantity": 5}, {"type": "Buzz", "name": "X", "quantity": 3}, {"type": "Buzz", "name": "Y", "quantity": 4}, ]`
Output	`["FizzBuzz! 3 A:X", "FizzBuzz! 2 A:Y"]`
Why	`A(5) waits. X(3) matches A -> (A,X,3), A now 2. Y(4) matches A's remaining 2 -> (A,Y,2); Y's leftover 2 waits with no Fizz to match.`

Example 2

Input	`offers = [ {"type": "Fizz", "name": "A", "quantity": 3}, {"type": "Buzz", "name": "X", "quantity": 1}, {"type": "Buzz", "name": "X", "quantity": 2}, ]`
Output	`["FizzBuzz! 3 A:X"]`
Why	`A matches the first X for 1 and the second X for 2. Both are the pair (A, X), so they aggregate to a single "FizzBuzz! 3 A:X".`

CONSTRAINTS

- 0 <= len(offers) <= 10**4 ; quantities are positive integers.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_matches(offers):
    fizz, buzz, totals = [], [], {}
    for offer in offers:
        mine = (
            fizz
            if offer["type"] == "Fizz"
            else:
                buzz
        )
        other = (
            buzz
            if offer["type"] == "Fizz"
            else:
                fizz
        )
        left = offer["quantity"]
        while left and other:
            m = min(left, other[0][1])
            pair = (
                (offer["name"], other[0][0])
                if offer["type"] == "Fizz"
                else:
                    (
                    other[0][0],
                    offer["name"],
                )
            )
            totals[pair] = (
                totals.get(pair, 0) + m
            )
            left -= m
            other[0][1] -= m
            if other[0][1] == 0:
                other.pop(0)
        if left:
            mine.append([offer["name"], left])
    return [
        f"FizzBuzz! {q} {f}:{b}"
        for (f, b), q in totals.items()
    ]
```

TIME	SPACE
O(n * matches)	O(n)

WHY IT IS SLOW

Removing from the front of a list shifts every other element.

OPTIMAL SOLUTION · Two FIFO queues and an ordered total

TIME	SPACE
O(offers + matches)	O(offers)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 After processing, report which offers were not fully matched.

Keep the two deques as the state: whatever remains in them at the end is the unmatched quantity per offer, in priority order. Return both queues (or merge them).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Aggregate FizzBuzz Offer Matches
#
# You are given a list of offers in priority order
# (earliest = highest priority). Each offer is a dict with
# keys type (either "Fizz" or "Buzz" ), name (a string),
# and quantity (a positive integer). Fizz offers match
# against Buzz offers on a first-come, first-served basis,
# and offers may be partially fulfilled. Process the
# offers in order, maintaining a FIFO queue of unmatched
# Fizz offers and a FIFO queue of unmatched Buzz offers.
# When a new offer arrives, match it against the front of
# the opposite queue repeatedly, each time consuming
# min(remaining quantities) and recording a match
# (fizz_name, buzz_name, matched_quantity) ; drop an offer
# from the queue once its quantity hits 0. If the arriving
# offer still has quantity left when the opposite queue
# empties, enqueue it in its own queue. Aggregate matches
# so the same (fizz_name, buzz_name) pair is reported once
# with its summed quantity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. After processing, report which offers were not fully
# matched.
#
# INTERVIEW NOTES
# Fizz and Buzz offers behave like the two sides of an
# order book matched first come, first served. Keep one
# FIFO queue per side. An arriving offer consumes the
# front of the opposite queue while both have quantity
# left, recording each (fizz, buzz, quantity) match;
# leftovers wait in their own queue. A dictionary keyed by
# the pair, ordered by first appearance, aggregates the
# matches.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import random
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def other_side(kind):
    if kind == "Fizz":
        return "Buzz"
    return "Fizz"
```

```
def match_offers(offers):
    # Each offer is {"type": "Fizz" or "Buzz", "name": ..., "quantity": ...}.
    # Offers are handled in order. A new offer is matched against the
    # OPPOSITE side's waiting line, front first, for as long as it has
    # quantity left and the line is not empty; the rest of the offer
    # waits in its own line. Returns the matched totals per
    # (fizz name, buzz name) pair and the two waiting lines.
    queues = {"Fizz": deque(), "Buzz": deque()}       # each entry is [name, quantity left]
    totals = {}
    for offer in offers:
        kind = offer["type"]
        mine = queues[kind]
        other = queues[other_side(kind)]
        left = offer["quantity"]
        while left > 0 and len(other) > 0:
            front = other[0]
            matched = left
            if front[1] < matched:
                matched = front[1]
            if kind == "Fizz":
                pair = (offer["name"], front[0])
            else:
                pair = (front[0], offer["name"])
            totals[pair] = totals.get(pair, 0) + matched
            left -= matched
            front[1] -= matched
            if front[1] == 0:
                other.popleft()
        if left > 0:
            mine.append([offer["name"], left])
    return totals, queues


# ---- main question: main function ----
# Idea: two FIFO queues (one per side) and an ordered total per pair of
```

```
# names. Matching an offer against the front of the opposite queue until
# the offer or the queue is used up is match_offers above; this
# function turns the totals into the lines "FizzBuzz! quantity fizz:buzz".
# Complexity: Time O(n + matches), Space O(n)
def fizzbuzz_matches(offers):
    totals, queues = match_offers(offers)
    lines = []
    for fizz_name, buzz_name in totals:
        lines.append("FizzBuzz! " + str(totals[(fizz_name, buzz_name)]) + " " + fizz_name + ":" + buzz_name
)
    return lines


# ---- main question: tests ----
def test_main_question():
    offers = [{"type": "Fizz", "name": "A", "quantity": 5},
              {"type": "Buzz", "name": "X", "quantity": 3},
              {"type": "Buzz", "name": "Y", "quantity": 4}]
    assert fizzbuzz_matches(offers) == ["FizzBuzz! 3 A:X", "FizzBuzz! 2 A:Y"]
    offers2 = [{"type": "Fizz", "name": "A", "quantity": 3},
               {"type": "Buzz", "name": "X", "quantity": 1},
               {"type": "Buzz", "name": "X", "quantity": 2}]
    assert fizzbuzz_matches(offers2) == ["FizzBuzz! 3 A:X"]      # the same pair is added up
    assert fizzbuzz_matches([]) == []


# ======================================================
```

```
# FOLLOW-UP 1: After processing, report which offers were not fully matched
# ========================================================
# ---- follow-up 1: function ----
# Answer: the two waiting lines ARE the state: whatever is left in them at
# the end is the unmatched quantity of each offer, in priority order.
# Return both lines.
# Complexity: Time O(n + matches), Space O(n)
def unmatched_after(offers):
    totals, queues = match_offers(offers)
    result = {}
    for side in queues:
        result[side] = []
        for name, quantity in queues[side]:
            result[side].append((name, quantity))
    return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
```

```
    offers = [{"type": "Fizz", "name": "A", "quantity": 5},
              {"type": "Buzz", "name": "X", "quantity": 3}]
    assert unmatched_after(offers) == {"Fizz": [("A", 2)], "Buzz": []}
    random.seed(63)
    for round_number in range(400):                  # matched + unmatched must add up to what was offered
        offers = []
        for i in range(random.randint(0, 8)):
            offers.append({"type": random.choice(["Fizz", "Buzz"]),
                           "name": random.choice("ABCDE"),
                           "quantity": random.randint(1, 5)})
        left = unmatched_after(offers)
        matched = 0
        for line in fizzbuzz_matches(offers):
            matched += int(line.split()[1])
        offered = {"Fizz": 0, "Buzz": 0}
        for offer in offers:
            offered[offer["type"]] += offer["quantity"]
        for side in ("Fizz", "Buzz"):
            unmatched = 0
            for name, quantity in left[side]:
                unmatched += quantity
            assert unmatched == offered[side] - matched


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 19 OF 43

Rolling state: the next state comes only from the previous one

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Rolling state: the next state comes only from the previous one | 4 questions | tap a title to jump to it

READ FIRST: the code template for these 4 questions (tap here) >

M 105. Maximize sum without choosing adjacent elements **PracHub * 2026-01-06 ***

M 106. Paint House II **ColorStack * 2025-10-17 ***

M 107. Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing fro... **ND**

M 108. Write SQL code to publish the Fibonacci series. **ND**

CODE TEMPLATE | GROUP 19 OF 43 | 4 QUESTIONS, ONE PATTERN

Rolling state: the next state comes only from the previous one

The one idea these questions share

You never need the whole table, only the previous state: one number, a pair, or one row. Write the state, then one line that builds the new state from the old one, and loop.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def roll(items):
    take, skip = 0, 0                          # CHANGE 1: the state
    for x in items:
        take, skip = skip + x, max(take, skip)     # CHANGE 2: new state from the old state
    return max(take, skip)
```

2. Helper functions that usually get written next to it

```
def two_smallest(row):                    # the trick behind "best of the others"
    order = sorted(range(len(row)), key=lambda j: row[j])
    return order[0], order[1] if len(order) > 1 else order[0]


def fib_pair(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
105	Maximize sum without choosing adjacent elements	exactly the house robber update on the array
106	Paint House II	state = a whole row of costs, one per colour; new[j] = cost[j] + the smallest previous value that is not colour j (keep the two smallest)
107	Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses.	state = (best if you take this house, best if you skip it): take, skip = skip + x, max(take, skip)
108	Write SQL code to publish the Fibonacci series.	WITH RECURSIVE keeps (n, a, b); every new row is (n + 1, b, a + b): the same rolling pair, written as a query

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 105 of 197 | topic 19 of 43: Rolling state: the next state comes only from the previous one | 1 of 4

Template fit: exactly the house robber update on the array | * asked 287 days ago *

MEDIUM | IN APP LC 198 | PracHub | Technical Screen | 105 of 197

★ TOP 75: rank 61 in the Medium and Hard priority list

★ Maximize sum without choosing adjacent elements

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 57.2 | 30 DAYS 0 | PRIORITY 57/288

ABOUT THIS QUESTION

Topic: Rolling state: the next state comes only from the previous one

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #198 House Robber

LeetCode match: LC 198 House Robber (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/maximize-sum-without-choosing-adjacent-elements

QUESTION

Given a list of non-negative numbers, choose some of them so that no two chosen numbers are neighbours in the list, and return the largest possible sum of the chosen numbers.

Function signature

def rob_line(nums: list) -> int

EXAMPLES

Example 1	
Input	`[2, 7, 9, 3, 1]`
Output	`12`
Why	`Choose 2, 9 and 1 (positions 0, 2, 4): 12.`

Example 2	
Input	`[5]`
Output	`5`
Why	`One number: take it.`

Example 3	
Input	`[]`
Output	`0`
Why	`Nothing to choose: 0.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def best(i):
    if i >= len(nums):
        return 0
    skip = best(i + 1)
    take = nums[i] + best(i + 2)
    return max(skip, take)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Two calls per index and the same suffix is solved repeatedly: O(2^n). Keeping only the best total with and without the previous element is O(n) time and O(1) space.

OPTIMAL SOLUTION · Two running values (take, skip)

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The houses form a circle (LeetCode 213).

The first and last are adjacent, so you cannot take both: the answer is the better of 'ignore the first house' and 'ignore the last house', each solved with the linear version.

FOLLOW-UP 2 Return which houses to rob.

Keep the table dp[i] = best from i onward (filled from the right) and walk forward: take house i when nums[i] + dp[i + 2] equals dp[i], then jump two; otherwise move one.

FOLLOW-UP 3 Robbing a house also alerts the next k houses.

The recurrence becomes dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1]); the O(1) space trick needs a window of k + 1 values (a deque), or just keep the O(n) table.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Maximize sum without choosing adjacent
# elements
#
# Given a list of non-negative numbers, choose some of
# them so that no two chosen numbers are neighbours in the
# list, and return the largest possible sum of the chosen
# numbers.
#
# Signature:
#   def rob_line(nums: list) -> int
#
# Examples:
# [2, 7, 9, 3, 1]  ->  12
```

```
# [5]  ->  5
#
# FOLLOW-UPS (answered below, in this order):
# 1. The houses form a circle (LeetCode 213).
# 2. Return which houses to rob.
# 3. Robbing a house also alerts the next k houses.
#
# INTERVIEW NOTES
# At each element you either take it (then the previous
# one is off limits) or skip it. Only two numbers matter
# from the past: the best total if you took the previous
# element and the best if you did not. Update them
# together and the whole array is one pass with no table.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def bigger(x, y):
    if x > y:
        return x
    return y


# ---- main question: main function ----
```

```
# Idea: keep two running numbers: `take` = the best total whose LAST
# chosen element is the current one, `skip` = the best total that does
# not use the current one. For the next number x:
#   new take = skip + x   (the previous element must have been skipped)
#   new skip = the bigger of take and skip.
# Complexity: Time O(n), Space O(1)
def rob_line(nums):
    take = 0
    skip = 0
    for x in nums:
        new_take = skip + x
        new_skip = bigger(take, skip)
        take = new_take
        skip = new_skip
    return bigger(take, skip)


# ---- main question: tests ----
def test_main_question():
    assert rob_line([2, 7, 9, 3, 1]) == 12
    assert rob_line([]) == 0
    assert rob_line([5]) == 5


# ============================================================
# FOLLOW-UP 1: The houses form a circle (LeetCode 213)
# ============================================================
# ---- follow-up 1: function ----
# Answer: the first and last house are neighbours now, so you cannot
# take both: the answer is the better of "ignore the first house" and
# "ignore the last house", each solved with the line version.
# Complexity: Time O(n), Space O(n) (the two slices)
```

```
def rob_circle(nums):
    if len(nums) == 1:
        return nums[0]
    return bigger(rob_line(nums[1:]), rob_line(nums[:-1]))


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert rob_circle([2, 3, 2]) == 3
    assert rob_circle([1, 2, 3, 1]) == 4
    assert rob_circle([4]) == 4
```

```
# =====================================================
# FOLLOW-UP 2: Return which houses to rob
# =====================================================
# ---- follow-up 2: function ----
# Answer: keep the whole table dp[i] = the best total from house i to the
# end (filled from the right), then walk forward: take house i when
# nums[i] + dp[i + 2] equals dp[i] (and jump two houses), otherwise move on one.
# Complexity: Time O(n), Space O(n)
def rob_with_choice(nums):
    n = len(nums)
    dp = [0] * (n + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = bigger(dp[i + 1], nums[i] + dp[i + 2])
    chosen = []
    i = 0
    while i < n:
        if nums[i] > 0 and nums[i] + dp[i + 2] == dp[i]:
            chosen.append(i)
            i += 2
        else:
            i += 1
    return dp[0], chosen


# ---- follow-up 2: tests ----
def test_follow_up_2():
    total, houses = rob_with_choice([2, 7, 9, 3, 1])
    assert total == 12 and houses == [0, 2, 4]


# =====================================================
# FOLLOW-UP 3: Robbing a house also alerts the next k houses
# =====================================================
# ---- follow-up 3: function ----
# Answer: dp[i] = the bigger of dp[i + 1] and nums[i] + dp[i + k + 1]
# (taking a house blocks the next k). The O(1)-space trick would need a
# window of k + 1 numbers, so simply keep the table.
# Complexity: Time O(n), Space O(n + k)
def rob_gap(nums, k):
    n = len(nums)
    dp = [0] * (n + k + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = bigger(dp[i + 1], nums[i] + dp[i + k + 1])
    return dp[0]
```

```
# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert rob_gap([2, 7, 9, 3, 1], 1) == 12              # k = 1 is the classic problem
    assert rob_gap([5, 1, 1, 5, 1, 1, 5], 2) == 15        # blocking two houses after each pick
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(nums: list[int]) -> int:
```

Sample call

```
solution([2, 7, 9, 3, 1])
```

Test cases

Input	Expected
([2, 7, 9, 3, 1],)	12
([1, 2, 3, 1],)	4

STUDY PATH: question 106 of 197 | topic 19 of 43: Rolling state: the next state comes only from the previous one | 2 of 4

Template fit: state = a whole row of costs, one per colour; new[j] = cost[j] + the smallest previous value that is not colour j (keep the two smallest) | * asked 368 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 106 of 197

Paint House II

ASKED 2025-10-17 | 368 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 227/288

ABOUT THIS QUESTION

Topic: Rolling state: the next state comes only from the previous one

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 265 Paint House II

Frequency: Single report | Last asked: 2025-10-17 | Roles: SWE Intern

QUESTION

There are n houses in a row, each to be painted one of k colors. costs[i][j] is the cost to paint house i with color j . No two adjacent houses may share the same color. Return the minimum total cost to paint all houses.

Function signature

```
def min_cost_paint_houses(costs: list[list[int]]) -> int:
    ...
```

Follow-up (as reported): Reduce the per-row transition from O(k) per cell to O(1) amortized by tracking the two smallest values of the previous row.

EXAMPLES

Example 1

Input	`costs = [[1, 5, 3], [2, 9, 4]]`
Output	`5`
Why	`Paint house 0 color 0 (1) and house 1 color 2 (4) -> 5; or house 0 color 2 (3) and house 1 color 0 (2) -> 5.`

Example 2

Input	`costs = [[1, 3], [2, 4]]`
Output	`5`
Why	`1 (house0 color0) + 4 (house1 color1) = 5, the cheapest valid pairing.`

Example 3

Input	`costs = [[7]]`
Output	`7`
Why	`A single house, one color.`

CONSTRAINTS

- costs is an n x k grid, 0 <= n , k >= 1 (and k >= 2 whenever n >= 2 , so a valid painting exists).
- Costs are non-negative integers. Return 0 for n == 0 .
- Aim for O(n*k) time.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
from itertools import product
def brute_paint(costs):
    n, k = len(costs), len(costs[0])
    best = float("inf")
    for colors in product(range(k),_repeatn):
        if all(
            a != b
            for a, b in zip(colors, colors[1:])
        ):
            best = min(
                best,
                sum(
                    costs[i][c]
                    for i, c in enumerate(
                        colors
                    )
                ),
            )
    return best
```

TIME	SPACE
O(k^n * n)	O(n)

WHY IT IS SLOW

It enumerates all k^n ways to colour the row.

OPTIMAL SOLUTION · Two smallest values of the previous row

TIME	SPACE
O(n k)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return which colour each house gets, not just the cost.

Store the DP row for every house (O(n k) memory), then walk back from the cheapest last cell: at house i pick the cheapest entry of the row of house i - 1 whose colour differs from the colour of house i + 1... in practice keep the argmin of the previous row and the second argmin when you store each row.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Paint House II
#
# There are n houses in a row, each to be painted one of k
# colors. costs[i][j] is the cost to paint house i with
# color j . No two adjacent houses may share the same
# color. Return the minimum total cost to paint all
# houses. Function signature: def
# min_cost_paint_houses(costs: list[list[int]]) -> int:
# ... Follow-up (as reported): Reduce the per-row
```

```
# transition from O(k) per cell to O(1) amortized by
# tracking the two smallest values of the previous row.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return which colour each house gets, not just the
# cost.
#
# INTERVIEW NOTES
# dp[i][j] is the cheapest cost to paint houses 0..i with
# house i in colour j: costs[i][j] plus the cheapest
# previous row entry that is not colour j. Taking the
# minimum over k - 1 colours for every cell is O(n k^2);
# keeping only the smallest and second smallest values of
# the previous row gives each cell its answer in O(1), for
# O(n k) total.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def two_smallest(values):
    # (index of the smallest, the smallest value, the second smallest value)
    first = 0
    for i in range(len(values)):
        if values[i] < values[first]:
            first = i
```

```
    second_value = values[first]                    # with one colour there is no second choice
    found_second = False
    for i in range(len(values)):
        if i != first and (not found_second or values[i] < second_value):
            second_value = values[i]
            found_second = True
    return first, values[first], second_value


# ---- main question: main function ----
# Every house is painted one of k colours; neighbouring houses must
# differ; costs[i][j] is the cost of colour j for house i. Find the
# cheapest total cost.
# Idea: the best total so far for a house painted with colour j is its
# cost plus the cheapest total of the previous house in a DIFFERENT
# colour. That is the smallest previous total, unless the smallest is
# colour j itself: then take the second smallest. So only the two
# smallest values of the previous row are needed: O(n * k).
# Complexity: Time O(n * k) for n houses and k colours, Space O(k)
def min_cost_paint_houses(costs):
    previous = [0] * len(costs[0])
    for row in costs:
        first, smallest, second_smallest = two_smallest(previous)
        current = []
        for j in range(len(row)):
            if j == first:
                current.append(row[j] + second_smallest)
            else:
```

```
                current.append(row[j] + smallest)
        previous = current
    return min(previous)


# ---- main question: tests ----
def test_main_question():
    assert min_cost_paint_houses([[1, 5, 3], [2, 9, 4]]) == 5
    assert min_cost_paint_houses([[1, 3], [2, 4]]) == 5
    assert min_cost_paint_houses([[7]]) == 7


# =====================================================
# FOLLOW-UP 1: Return which colour each house gets, not just the cost
# =====================================================
# ---- follow-up 1: function ----
# Answer: store the whole table of totals (O(n * k) memory), then walk
```

```
# back from the cheapest last cell: for house i, the colour of house i - 1
# is a different colour whose total equals this house's total minus this
# house's own cost.
# Complexity: Time O(n * k^2), Space O(n * k)
def paint_plan(costs):
    n = len(costs)
    k = len(costs[0])
    rows = []
    previous = [0] * k
    for row in costs:
        current = []
        for j in range(k):
            cheapest = None
            for x in range(k):
                if x != j and (cheapest is None or previous[x] < cheapest):
                    cheapest = previous[x]
            if cheapest is None:
                cheapest = 0                        # only one colour: nothing before to avoid
            current.append(row[j] + cheapest)
        rows.append(current)
        previous = current
    colour = 0
    for j in range(k):
        if rows[n - 1][j] < rows[n - 1][colour]:
            colour = j
    total = rows[n - 1][colour]
    plan = [colour]
    for i in range(n - 1, 0, -1):
        needed = rows[i][colour] - costs[i][colour]
        for x in range(k):
            if x != colour and rows[i - 1][x] == needed:
                colour = x
                break
        plan.append(colour)
    plan.reverse()
    return total, plan


# ---- follow-up 1: tests ----
def test_follow_up_1():
    random.seed(68)
    for round_number in range(300):
        n = random.randint(1, 5)
        k = random.randint(2, 4)
        costs = []
```

```
        for i in range(n):
            row = []
            for j in range(k):
                row.append(random.randint(1, 9))
            costs.append(row)
        total, plan = paint_plan(costs)
        for i in range(n - 1):
            assert plan[i] != plan[i + 1]                    # neighbours differ
        paid = 0
        for i in range(n):
            paid += costs[i][plan[i]]
        assert paid == total == min_cost_paint_houses(costs)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 107 of 197 | topic 19 of 43: Rolling state: the next state comes only from the previous one | 3 of 4

Template fit: state = (best if you take this house, best if you skip it): take, skip = skip + x, max(take, skip) | ND

MEDIUM | IN APP LC 198 | Exponent | 107 of 197

★ TOP 75: rank 74 in the Medium and Hard priority list

★ Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses.

ASKED (ANY CO.) ~2 MONTHS AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 57.2 30 DAYS 0 PRIORITY 58/288

ABOUT THIS QUESTION

Topic: Rolling state: the next state comes only from the previous one

Optimal answer from: your app's interview approach (LeetCode #198)

LeetCode match: LC 198 House Robber (inferred from short statement)

Last asked (any company): 2 months ago, about 2026-08 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/4646/house-robber

QUESTION

Houses stand in a line and house i holds nums[i] money. A robber cannot rob two houses that are next to each other. Return the largest total amount he can steal.

Function signature

def rob_line(nums: list) -> int

EXAMPLES

Example 1	
Input	`[1, 2, 3, 1]`
Output	`4`
Why	`Rob houses 0 and 2: 1 + 3 = 4.`

Example 2	
Input	`[2, 7, 9, 3, 1]`
Output	`12`
Why	`Rob houses 0, 2 and 4: 2 + 9 + 1 = 12.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def best(i):
    if i >= len(nums):
        return 0
    skip = best(i + 1)
    take = nums[i] + best(i + 2)
    return max(skip, take)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

BRUTE FORCE

Two calls per index and the same suffix is solved repeatedly: O(2^n). Keeping only the best total with and without the previous element is O(n) time and O(1) space.

OPTIMAL SOLUTION · Two running values (take, skip)

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The houses form a circle (LeetCode 213).

The first and last are adjacent, so you cannot take both: the answer is the better of 'ignore the first house' and 'ignore the last house', each solved with the linear version.

FOLLOW-UP 2 Return which houses to rob.

Keep the table dp[i] = best from i onward (filled from the right) and walk forward: take house i when nums[i] + dp[i + 2] equals dp[i], then jump two; otherwise move one.

FOLLOW-UP 3 Robbing a house also alerts the next k houses.

The recurrence becomes dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1]); the O(1) space trick needs a window of k + 1 values (a deque), or just keep the O(n) table.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given n houses in a line with money in
# each, find the maximum amount a robber can steal,
# without stealing from two adjacent houses.
#
# Houses stand in a line and house i holds nums[i] money.
# A robber cannot rob two houses that are next to each
# other. Return the largest total amount he can steal.
#
# Signature:
#   def rob_line(nums: list) -> int
#
# Examples:
# [1, 2, 3, 1]  ->  4
# [2, 7, 9, 3, 1]  ->  12
#
# FOLLOW-UPS (answered below, in this order):
# 1. The houses form a circle (LeetCode 213).
# 2. Return which houses to rob.
# 3. Robbing a house also alerts the next k houses.
#
# INTERVIEW NOTES
# At each element you either take it (then the previous
# one is off limits) or skip it. Only two numbers matter
# from the past: the best total if you took the previous
# element and the best if you did not. Update them
# together and the whole array is one pass with no table.
```

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def bigger(x, y):
    if x > y:
        return x
    return y


# ---- main question: main function ----
```

```
# Idea: two running numbers. `take` = the best total when the current
# house IS robbed, `skip` = the best total when it is not. For the next
# house with money x: new take = skip + x (the house before must have
# been skipped), new skip = the bigger of take and skip.
# Complexity: Time O(n), Space O(1)
def rob_line(nums):
    take = 0
    skip = 0
    for x in nums:
        new_take = skip + x
        new_skip = bigger(take, skip)
        take = new_take
        skip = new_skip
    return bigger(take, skip)


# ---- main question: tests ----
def test_main_question():
    assert rob_line([1, 2, 3, 1]) == 4
    assert rob_line([2, 7, 9, 3, 1]) == 12
    assert rob_line([]) == 0


# ============================================================
# FOLLOW-UP 1: The houses form a circle (LeetCode 213)
# ============================================================
# ---- follow-up 1: function ----
# Answer: the first and last houses touch, so you cannot rob both: the
# answer is the better of "ignore the first house" and "ignore the last
# house", each solved with the line version.
# Complexity: Time O(n), Space O(n) (the two slices)
def rob_circle(nums):
    if len(nums) == 1:
        return nums[0]
    return bigger(rob_line(nums[1:]), rob_line(nums[:-1]))


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert rob_circle([2, 3, 2]) == 3
    assert rob_circle([1, 2, 3, 1]) == 4
    assert rob_circle([5]) == 5


# ============================================================
```

```python
# FOLLOW-UP 2: Return which houses to rob
# ========================================================
# ---- follow-up 2: function ----
# Answer: keep dp[i] = the best total from house i to the end (filled
# from the right). Walk forward: rob house i when nums[i] + dp[i + 2]
# equals dp[i] (then jump two houses), otherwise move on one.
# Complexity: Time O(n), Space O(n)
def rob_with_choice(nums):
    n = len(nums)
    dp = [0] * (n + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = bigger(dp[i + 1], nums[i] + dp[i + 2])
    houses = []
    i = 0
    while i < n:
        if nums[i] > 0 and nums[i] + dp[i + 2] == dp[i]:
            houses.append(i)
            i += 2
        else:
            i += 1
    return dp[0], houses


# ---- follow-up 2: tests ----
def test_follow_up_2():
    total, houses = rob_with_choice([2, 7, 9, 3, 1])
    assert total == 12 and houses == [0, 2, 4]


# ========================================================
# FOLLOW-UP 3: Robbing a house also alerts the next k houses
# ========================================================
# ---- follow-up 3: function ----
# Answer: dp[i] = the bigger of dp[i + 1] and nums[i] + dp[i + k + 1]
# (a robbed house alerts the next k). The O(1)-space version needs a
# window of k + 1 numbers, or just keep the O(n) table.
# Complexity: Time O(n), Space O(n + k)
def rob_alarm(nums, k):
    n = len(nums)
    dp = [0] * (n + k + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = bigger(dp[i + 1], nums[i] + dp[i + k + 1])
    return dp[0]
```

```python
# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert rob_alarm([2, 7, 9, 3, 1], 1) == 12
    assert rob_alarm([5, 1, 1, 5, 1, 1, 5], 2) == 15


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

You are given an array representing the value in the houses e.g. [100, 20, 40, 70, 80]. Return 100 + 40 + 80 = 220

STUDY PATH: question 108 of 197 | topic 19 of 43: Rolling state: the next state comes only from the previous one | 4 of 4

Template fit: WITH RECURSIVE keeps (n, a, b); every new row is (n + 1, b, a + b): the same rolling pair, written as a query | ND

MEDIUM | Exponent | 108 of 197

Write SQL code to publish the Fibonacci series.

ASKED (ANY CO.) ~4 MONTHS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 256/288**

ABOUT THIS QUESTION

Topic: Rolling state: the next state comes only from the previous one

Optimal answer from: written and tested here for this exact question

LeetCode match: No LC match (SQL) (inferred from short statement)

Last asked (any company): 4 months ago, about 2026-06 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/2779/sql-code-fibonacci-series

QUESTION

Write a SQL query that publishes the first N Fibonacci numbers (0, 1, 1, 2, 3, 5, ...), one per row and in order, where N is given as the parameter :limit. Use a recursive common table expression.

Function signature

-- result: one column named fibonacci, N rows

EXAMPLES

Example 1	
Input	`:limit = 8`
Output	`0, 1, 1, 2, 3, 5, 8, 13`
Why	`Eight rows: each number is the sum of the two before it.`

Example 2	
Input	`:limit = 1`
Output	`0`
Why	`A single row.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
a, b = 0, 1
for _ in range(n):
    execute("INSERT INTO fib VALUES (?)", a)
    a, b = b, a + b
```

TIME	SPACE
O(n)	**O(n)**
statements	

WHY IT IS SLOW

One statement and one round trip per term, and a loop outside the query language. A recursive CTE produces all rows in one set-based statement.

OPTIMAL SOLUTION · Recursive CTE carrying (n, a, b)

TIME	SPACE
O(n)	**O(1)** working set (O(n) result)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only the even terms, or the sum of the first n terms.

Keep the CTE and filter or aggregate in the outer query: WHERE a % 2 = 0 for the even terms, SUM(a) for the total. The recursion stays the same.

FOLLOW-UP 2 Which database?

PostgreSQL and MySQL 8 use WITH RECURSIVE; SQL Server drops the word RECURSIVE and may need a CAST so the column type is wide enough (bigint) in the anchor row; Oracle accepts WITH without RECURSIVE too. State the dialect assumption first.

FOLLOW-UP 3 Large n overflows.

Use a wider type (BIGINT, NUMERIC, or DECIMAL) in the anchor row so the column type is wide enough from the start; arbitrary-size values need a language with big integers. **Cost:** O(n) rows regardless of the type; a wider type only costs more bytes per row.

FOLLOW-UP 4 Use it from application code with a parameter.

Bind the limit as a query parameter (never build the SQL by string concatenation) and fetch the single column. The recursion lives inside the database, so there is one round trip regardless of n.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Write SQL code to publish the Fibonacci
# series.
#
# Write a SQL query that publishes the first N Fibonacci
# numbers (0, 1, 1, 2, 3, 5, ...), one per row and in
# order, where N is given as the parameter :limit. Use a
# recursive common table expression.
#
# Signature:
#   -- result: one column named fibonacci, N rows
#
# Examples:
# :limit = 8  ->  0, 1, 1, 2, 3, 5, 8, 13
# :limit = 1  ->  0
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the even terms, or the sum of the first n terms.
# 2. Which database?
# 3. Large n overflows.
# 4. Use it from application code with a parameter.
#
# INTERVIEW NOTES
```

```
# A sequence that depends on the previous rows is a
# recursive CTE: one anchor row, one recursive step, and a
# stopping condition in the step. Carry the two previous
# values in each row (a, b) so the next row is (b, a + b);
# a row-per-term table is built in one statement with no
# procedural loop.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import sqlite3


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
# A recursive query: every new row is computed from the previous row.
FIB_SQL = """
WITH RECURSIVE fib(n, a, b) AS (
    SELECT 1, 0, 1
    UNION ALL
    SELECT n + 1, b, a + b FROM fib WHERE n < ?
)
SELECT a FROM fib
"""


def run_query(sql, parameters):
    # run one query on a fresh in-memory database and return the first column of every row
    db = sqlite3.connect(":memory:")
    rows = db.execute(sql, parameters).fetchall()
    db.close()
    column = []
    for row in rows:
        column.append(row[0])
    return column


def fibonacci_python(count):
    # the same sequence in Python, used to check the SQL
    a = 0
    b = 1
    terms = []
    for i in range(count):
        terms.append(a)
        a, b = b, a + b
    return terms


# ---- main question: main function ----
# Idea: a recursive CTE that carries the three values (n, a, b). The
# first row is (1, 0, 1): the first term is 0 and the next pair is (0, 1).
# Every recursive step makes (n + 1, b, a + b) while n is below the limit,
# so column a holds the Fibonacci numbers.
# The limit is a QUERY PARAMETER (the ? above), never glued into the SQL
# text, and the recursion runs inside the database: one round trip for any n
# (this is also follow-up 4).
# Complexity: Time O(n) rows for n terms, Space O(n)
def fibonacci(count):
```

```
    return run_query(FIB_SQL, (count,))


# ---- main question: tests ----
```

```
def test_main_question():
    assert fibonacci(10) == [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
    assert fibonacci(10) == fibonacci_python(10)
    assert fibonacci(1) == [0]


# ============================================================
# FOLLOW-UP 1: Only the even terms, or the sum of the first n terms
# ============================================================
# ---- follow-up 1: function ----
# Answer: keep the recursive query and filter or add up in an outer query:
# WHERE a % 2 = 0 for the even terms, SUM(a) for the total.
# Complexity: Time O(n), Space O(n)
def even_terms(count):
    sql = "SELECT a FROM (" + FIB_SQL + ") WHERE a % 2 = 0"
    return run_query(sql, (count,))


# Complexity: Time O(n), Space O(n)
def total(count):
    sql = "SELECT SUM(a) FROM (" + FIB_SQL + ")"
    return run_query(sql, (count,))[0]


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert even_terms(10) == [0, 2, 8, 34]
    assert total(10) == 88                                  # 0+1+1+2+3+5+8+13+21+34


# ============================================================
# FOLLOW-UP 2: Which database?
# ============================================================
# Answer: the recursive WITH works in PostgreSQL, MySQL 8, SQLite and
# SQL Server (which leaves out the word RECURSIVE); Oracle can also use
# CONNECT BY. Say which one you assume.


# ============================================================
# FOLLOW-UP 3: Large n overflows
# ============================================================
# Answer: a 64-bit column is exact up to the 92nd term. Beyond that use
# a big-number type (NUMERIC), or compute modulo a prime.
# ---- follow-up 3: tests ----
def test_follow_up_3():
```

```
    assert fibonacci(80) == fibonacci_python(80)          # 80 terms are still exact in a 64-bit column


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
```

```
run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 20 OF 43

Two pointers: move i and j toward each other (or along two lists)

3 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Two pointers: move i and j toward each other (or along two lists) | 3 questions | tap a title to jump to it

READ FIRST: the code template for these 3 questions (tap here) >

M 109. Find median of merged RLE arrays — **PracHub * 2026-01-06 ***

M 110. Run-Length Vector Storage + Dot Product — **1Point3Acres * 2026-01-21 ***

M 111. Count Unique Triplets That Sum to Zero — **ColorStack * 2025-09-20 ***

CODE TEMPLATE | GROUP 20 OF 43 | 3 QUESTIONS, ONE PATTERN

Two pointers: move i and j toward each other (or along two lists)

The one idea these questions share

Put one pointer at each end (or one at the start of each of two sorted lists). Look at the two items, decide which pointer moves, and stop when they meet. Most of these are O(n) because each pointer only moves one way.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def has_pair(nums, target):                    # nums sorted
    i, j = 0, len(nums) - 1
    while i < j:
        s = nums[i] + nums[j]                  # CHANGE 1: what you compute from the two items
        if s == target:
            return True
        if s < target:                         # CHANGE 2: which pointer moves
            i += 1
        else:
            j -= 1
    return False
```

2. Helper functions that usually get written next to it

```
def reverse_range(a, i, j):
    while i < j:
        a[i], a[j] = a[j], a[i]
        i, j = i + 1, j - 1


def walk_two(a, b):                            # merge order of two sorted lists
    i = j = 0
    while i < len(a) or j < len(b):
        if j == len(b) or (i < len(a) and a[i] <= b[j]):
            yield a[i]; i += 1
        else:
            yield b[j]; j += 1
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
109	Find median of merged RLE arrays	walk both sorted run lists, always taking the smaller value, adding its count until you pass the middle position
110	Run-Length Vector Storage + Dot Product	one pointer per run list; multiply the values and take min(remaining counts) of the two current runs
111	Count Unique Triplets That Sum to Zero	sort, fix the first number, then run the pair template on the rest; skip equal neighbours so each triple counts once

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 109 of 197 | topic 20 of 43: Two pointers: move i and j toward each other (or along two lists) | 1 of 3

Template fit: walk both sorted run lists, always taking the smaller value, adding its count until you pass the middle position | * asked 287 days ago *

MEDIUM | PracHub | Technical Screen | 109 of 197

★ TOP 75: rank 2 in the Medium and Hard priority list

★ Find median of merged RLE arrays

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 74.2 | 30 DAYS 61 | PRIORITY 5/288

ABOUT THIS QUESTION

Topic: Two pointers: move i and j toward each other (or along two lists)

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #4 Median of Two Sorted Arrays

LeetCode match: LC 4 Median of Two Sorted Arrays (RLE variant) (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/find-median-of-merged-rle-arrays

QUESTION

Two sorted arrays are stored run-length encoded as (value, count) pairs in increasing order of value. Return the median of all their elements together, without expanding the runs (a count can be huge).

Function signature

def median_rle(a: list, b: list) -> float

EXAMPLES

Example 1	
Input	`[(1, 2), (5, 1)], [(3, 3)]`
Output	`3`
Why	`Combined sequence 1, 1, 3, 3, 3, 5: the two middle elements are 3 and 3.`

Example 2	
Input	`[(1, 1)], [(2, 1), (9, 1)]`
Output	`2`
Why	`Combined sequence 1, 2, 9: the middle one is 2.`

Example 3	
Input	`[(1, 1000000)], [(5, 1000000)]`
Output	`3`
Why	`Counts may be enormous: a million 1s and a million 5s have median 3.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(runs_a, runs_b):
    dense = []
    for value, count in runs_a + runs_b:
        dense += [value] * count
    dense.sort()
    n = len(dense)
    if n % 2:
        return dense[n // 2]
    mid = n // 2
    return (dense[mid - 1] + dense[mid]) / 2
```

TIME	SPACE
O(total count)	O(total count)

WHY IT IS SLOW

Needs time and memory proportional to the total count, which can be 10^9 or more. Walking the two run lists in value order and counting up to the middle rank costs only the number of runs.

OPTIMAL SOLUTION · Merge the runs and walk to the two middle ranks

TIME	SPACE
O(r1 + r2)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 K arrays instead of two.

Merge all runs by value (heap of K heads, or sort the combined run list if memory allows) and walk to the two middle ranks. O(R log K) with a heap, R the total number of runs.

FOLLOW-UP 2 The arrays are huge and sorted, and you want O(log) time.

Binary search on the value: for a candidate value count how many elements are at most it in each array (binary search on prefix sums of counts) and find the smallest value with rank above the middle. O(log(range) * log(runs)).

FOLLOW-UP 3 Runs are not sorted by value.

Sort the runs by value first, O(R log R), then use the same walk.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find median of merged RLE arrays
#
# Two sorted arrays are stored run-length encoded as
# (value, count) pairs in increasing order of value.
# Return the median of all their elements together,
# without expanding the runs (a count can be huge).
#
# Signature:
#   def median_rle(a: list, b: list) -> float
#
```

```
# Examples:
# [(1, 2), (5, 1)], [(3, 3)]  ->  3
# [(1, 1)], [(2, 1), (9, 1)]  ->  2
#
# FOLLOW-UPS (answered below, in this order):
# 1. K arrays instead of two.
# 2. The arrays are huge and sorted, and you want O(log)
# time.
# 3. Runs are not sorted by value.
#
# INTERVIEW NOTES
# Counts can be huge, so never expand a run. You only need
# the elements at ranks (N - 1) // 2 and N // 2 in the
# merged order. Merge the two run lists by value, counting
# how many elements you have passed, and read the value
# when the count reaches each rank.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from bisect import bisect_right


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def total_count(runs):
    # how many elements the run-length list stands for
    total = 0
    for value, count in runs:
        total += count
    return total


def kth_smallest(runs_a, runs_b, k):
    # the element of (zero-based) rank k in the merged order, found by
    # walking the runs of the two sorted lists; nothing is expanded
    i = 0
    j = 0
    seen = 0
    while True:
        take_a = j >= len(runs_b)                    # b is used up: take from a
        if not take_a and i < len(runs_a):
            take_a = runs_a[i][0] <= runs_b[j][0]
        if take_a:
            value, count = runs_a[i]
        else:
            value, count = runs_b[j]
        seen += count
        if seen > k:
            return value
        if take_a:
            i += 1
        else:
            j += 1


# ---- main question: main function ----
# Idea: let N be the total number of elements. For odd N the median has
```

```
# rank N // 2; for even N it is the average of the elements of rank
# N // 2 - 1 and N // 2. Walk the two run lists in sorted order and stop
# at those ranks. a and b are [(value, count), ...] sorted by value.
# Complexity: Time O(a + b) for a and b runs, Space O(1)
def median_rle(a, b):
    total = total_count(a) + total_count(b)
    if total % 2 == 1:
        return kth_smallest(a, b, total // 2)
    low = kth_smallest(a, b, total // 2 - 1)
    high = kth_smallest(a, b, total // 2)
    return (low + high) / 2


# ---- main question: tests ----
A = [(1, 3), (4, 1)]
B = [(2, 2), (5, 2)]
```

```
def test_main_question():
    assert median_rle(A, B) == 2                      # merged: 1 1 1 2 2 4 5 5
    assert median_rle([(7, 6)], [(1, 1)]) == 7        # an odd total
    big = [(1, 10 ** 12)]
    assert median_rle(big, [(3, 10 ** 12)]) == 2      # huge counts are never expanded


# =====================================================
# FOLLOW-UP 1: K arrays instead of two
# =====================================================
# ---- follow-up 1: function ----
# Answer: put all the runs of all arrays together in value order (sort
# them, or merge with a heap of the K heads, O(R log K)) and walk to the
# two middle ranks. R is the total number of runs.
# Complexity: Time O(R log R) for R runs in total, Space O(R)
def median_k_arrays(arrays):
    runs = []
    for array in arrays:
        for run in array:
            runs.append(run)
    runs.sort()
    total = total_count(runs)

    def kth(k):
        seen = 0
        for value, count in runs:
            seen += count
            if seen > k:
                return value

    if total % 2 == 1:
        return kth(total // 2)
    return (kth(total // 2 - 1) + kth(total // 2)) / 2


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert median_k_arrays([A, B]) == 2
    assert median_k_arrays([A, B, [(0, 1), (10, 1)]]) == 2


# =====================================================
# FOLLOW-UP 2: The arrays are huge and sorted, and you want O(log) time
```

```
# =====================================================
```

```
# ============================================================
# FOLLOW-UP 3: Runs are not sorted by value
# ============================================================
# ---- follow-up 2: function ----
# Answer: binary search on the VALUE. For a candidate value x count how
# many elements are <= x in each array (a binary search on the running
# totals of the counts), and find the smallest value that has more than k
# elements at or below it. O(log(range) * log(runs)). (If the runs are
# not sorted by value, sort them first: that costs O(R log R).)
# Complexity: Time O(log(value range) * k * log r) for k arrays of r runs, Space O(R)
def median_by_value(arrays):
    prepared = []                                    # (values, running totals of the counts) per array
    for runs in arrays:
        values = []
        sums = []
        running = 0
        for value, count in runs:
            running += count
            values.append(value)
            sums.append(running)
        prepared.append((values, sums))

    def count_at_most(x):
        total = 0
        for values, sums in prepared:
            i = bisect_right(values, x)
            if i > 0:
                total += sums[i - 1]
        return total

    def kth(k):
        lo = -(10 ** 18)
        hi = 10 ** 18
        while lo < hi:
            mid = (lo + hi) // 2
            if count_at_most(mid) > k:               # mid already has more than k elements below it
                hi = mid
            else:
                lo = mid + 1
        return lo

    n = count_at_most(10 ** 18)
```

```
    if n % 2 == 1:
        return kth(n // 2)
    return (kth(n // 2 - 1) + kth(n // 2)) / 2


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert median_by_value([A, B]) == 2
    three = [A, B, [(0, 1), (10, 1)]]
    assert median_by_value(three) == median_k_arrays(three)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Sample call

```
solution([(1, 2), (5, 1)], [(2, 3)])
```

Test cases

Input	Expected
`([(1, 2), (5, 1)], [(2, 3)])`	`2.0`
`([(1, 1)], [(2, 1)])`	`1.5`

STUDY PATH: question 110 of 197 | topic 20 of 43: Two pointers: move i and j toward each other (or along two lists) | 2 of 3

Template fit: one pointer per run list; multiply the values and take min(remaining counts) of the two current runs | * asked 272 days ago * | frequency Single report

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 110 of 197

Run-Length Vector Storage + Dot Product

ASKED 2026-01-21 | 272 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 5 | 30 DAYS 0 | PRIORITY 197/288

ABOUT THIS QUESTION

Topic: Two pointers: move i and j toward each other (or along two lists)

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1570 Dot Product of Two Sparse Vectors

Frequency: Single report | Last asked: 2026-01-21 | Roles: SWE, INFRA-ENG

Original: https://www.1point3acres.com/interview/problems/company/google/compressed-vector-dot-product

QUESTION

Two vectors of the same length are stored run-length encoded as (value, count) pairs. Compute their dot product without expanding them to full arrays.

Function signature

def rle_dot(a: list, b: list) -> int

EXAMPLES

Example 1	
Input	`[(3, 3), (0, 3), (4, 2)], [(1, 4), (2, 4)]`
Output	`25`
Why	`A = [3,3,3,0,0,0,4,4], B = [1,1,1,1,2,2,2,2]: 3*1*3 + 0 + 4*2*2 = 9 + 16 = 25.`

Example 2	
Input	`[(2, 1)], [(5, 1)]`
Output	`10`
Why	`One element each: 2 * 5.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def decode(runs):
    dense = []
    for value, count in runs:
        dense += [value] * count
    return dense
def brute(runs_a, runs_b):
    a, b = decode(runs_a), decode(runs_b)
    return sum(x * y for x, y in zip(a, b))
```

TIME	SPACE
O(logical length)	O(logical length)

WHY IT IS SLOW

BRUTE FORCE

It needs memory and time proportional to the logical length, which can be 10^9. Walking the two run lists together with two pointers costs only the number of runs.

OPTIMAL SOLUTION · Two-pointer over runs

TIME	SPACE
O(r1 + r2)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if one vector is mostly zeros and stored sparsely, and the other as runs?

Iterate the sparse side's nonzero indices and binary-search the run list for each index to read the other value: O(s log r). Choose the smaller representation to drive the loop.

FOLLOW-UP 2 How should the data be updated if you set element i?

Locate the run containing i (binary search over prefix run lengths), then split it into at most three runs and merge equal neighbours. That is O(log r) to find plus O(r) in the worst case to splice a list; use a balanced tree of runs for frequent updates.

FOLLOW-UP 3 One vector is a sparse map (index to value) and the other is run-length encoded.

Iterate the smaller structure (the sparse non-zeros) and binary search the run list for each index using prefix run lengths: O(s log r) instead of expanding anything.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Run-Length Vector Storage + Dot Product
#
# Two vectors of the same length are stored run-length
# encoded as (value, count) pairs. Compute their dot
# product without expanding them to full arrays.
#
# Signature:
#   def rle_dot(a: list, b: list) -> int
#
# Examples:
# [(3, 3), (0, 3), (4, 2)], [(1, 4), (2, 4)]  ->  25
# [(2, 1)], [(5, 1)]  ->  10
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if one vector is mostly zeros and stored
# sparsely, and the other as runs?
# 2. How should the data be updated if you set element i?
# 3. One vector is a sparse map (index to value) and the
# other is run-length encoded.
#
# INTERVIEW NOTES
# A run is a stretch where both numbers are constant, so
# the dot product of two runs is value times value times
# the length of their overlap. Walk the two run lists like
# the merge step of merge sort: consume the smaller
# remaining length from both, and advance whichever run
```

```
# you finished.

# =============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# =============================================================

from bisect import bisect_right


# =============================================================
# MAIN QUESTION
# =============================================================
# ---- main question: helper functions ----
def rle_encode(values):
    # [3, 3, 3, 0, 0] -> [(3, 3), (0, 2)]: each run is (value, how many in a row)
    runs = []
    for v in values:
        if runs and runs[-1][0] == v:
            runs[-1] = (v, runs[-1][1] + 1)
        else:
            runs.append((v, 1))
    return runs


def rle_decode(runs):
    values = []
    for v, count in runs:
        for i in range(count):
            values.append(v)
    return values


# ---- main question: main function ----
# Idea: two pointers over the RUNS (never expand them). Look at what is
# left of the current run of a and of b, multiply the two values by the
# overlap, then move on in whichever run is used up (or in both).
# Complexity: Time O(a + b) for a and b runs, Space O(1)
def rle_dot(a, b):
    i = 0
    j = 0
    left_a = 0
    left_b = 0
    if a:
        left_a = a[0][1]
    if b:
        left_b = b[0][1]
    total = 0
    while i < len(a) and j < len(b):
        take = left_a                       # both runs overlap for min(left_a, left_b) positions
        if left_b < take:
            take = left_b
        total += a[i][0] * b[j][0] * take
        left_a -= take
        left_b -= take
        if left_a == 0:
            i += 1
            if i < len(a):
                left_a = a[i][1]
        if left_b == 0:
            j += 1
            if j < len(b):
```

```
                left_b = b[j][1]
    return total


# ---- main question: tests ----
A = [3, 3, 3, 0, 0, 0, 4, 4]
B = [1, 1, 1, 1, 2, 2, 2, 2]


def test_main_question():
    ra = rle_encode(A)
    rb = rle_encode(B)
    assert ra == [(3, 3), (0, 3), (4, 2)]
    assert rle_decode(ra) == A
    assert rle_dot(ra, rb) == 25                      # 3*1*3 + 0 + 4*2*2
    assert rle_dot([], rb) == 0
    dense = 0
    for i in range(len(A)):
        dense += A[i] * B[i]
    assert dense == 25


# =============================================================
# FOLLOW-UP 1: What if one vector is mostly zeros and stored sparsely, and the other as runs?
# =============================================================


# =============================================================
# FOLLOW-UP 3: One vector is a sparse map (index to value) and the other is run-length encoded
# =============================================================
# ---- follow-up 1: function ----
# Answer: loop over the SMALL side (the sparse non-zero indices) and, for
# each index, binary-search the run list for the value there, using the
# running totals of the run lengths: O(s log r) and nothing is expanded.
# Complexity: Time O(r + s log r) for r runs and s non-zero entries, Space O(r)
def sparse_dot_rle(sparse, runs):
    # sparse is {index: value}; ends[k] is where run k stops
    ends = []
    position = 0
    for value, count in runs:
        position += count
        ends.append(position)
    total = 0
    for index in sparse:
        k = bisect_right(ends, index)             # the run that contains this index
```

```
        if k < len(runs):
            total += sparse[index] * runs[k][0]
    return total


# ---- follow-up 1: tests ----
def test_follow_up_1():
    sparse = {0: 2, 7: 5}                           # non-zeros at index 0 and 7
    assert sparse_dot_rle(sparse, rle_encode(B)) == 2 * 1 + 5 * 2


# =============================================================
# FOLLOW-UP 2: How should the data be updated if you set element i?
# =============================================================
# ---- follow-up 2: function ----
# Answer: find the run that contains index i, split it into at most three
# runs (before, the new value, after), then merge equal neighbours.
# Complexity: Time O(r), Space O(r)
def set_at(runs, index, value):
    pieces = []
```

```
    position = 0
    for v, count in runs:
        if position <= index < position + count:
            pieces.append((v, index - position))
            pieces.append((value, 1))
            pieces.append((v, position + count - index - 1))
        else:
            pieces.append((v, count))
        position += count
    merged = []
    for v, count in pieces:
        if count == 0:
            continue
        if merged and merged[-1][0] == v:
            merged[-1] = (v, merged[-1][1] + count)
        else:
            merged.append((v, count))
    return merged


# ---- follow-up 2: tests ----
def test_follow_up_2():
    ra = rle_encode(A)
    assert set_at(ra, 4, 3) == [(3, 3), (0, 1), (3, 1), (0, 1), (4, 2)]
    assert set_at([(1, 3)], 1, 1) == [(1, 3)]
```

```


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Memorize the two-pointer RLE walk pattern (it generalizes to interval intersection, range addition, etc.).
Practice articulating when RLE wins vs sparse (O(distinct_runs) vs O(nonzeros)).
Have a one-liner for sparse case: store (idx -> value) map; iterate over the smaller map and look up the other.

STUDY PATH: question 111 of 197 | topic 20 of 43: Two pointers: move i and j toward each other (or along two lists) | 3 of 3

Template fit: sort, fix the first number, then run the pair template on the rest; skip equal neighbours so each triple counts once | * asked 395 days ago * | frequency Single report

MEDIUM | IN APP LC 15 | ColorStack | technical interview | 111 of 197

Count Unique Triplets That Sum to Zero

ASKED 2025-09-20 | 395 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 218/288

ABOUT THIS QUESTION

Topic: Two pointers: move i and j toward each other (or along two lists)

Optimal answer from: your app's interview approach (LeetCode #15)

LeetCode match: LC 15 3Sum (count of unique triplets)

Frequency: Single report | Last asked: 2025-09-20 | Roles: SWE Intern

QUESTION

Given an integer array nums , count how many unique triplets (nums[i], nums[j], nums[k]) with three distinct indices i < j < k satisfy nums[i] + nums[j] + nums[k] == 0 . Two triplets are considered the same if they consist of the same multiset of values, regardless of the indices they came from. Return the number of distinct value-triplets (not the number of index combinations).

Function signature

```
def count_zero_sum_triplets(nums: list[int]) -> int:
    ...
```

Follow-up (as reported): Return the actual list of unique triplets instead of the count. Hint: sort the array, fix one element, and use two pointers while skipping duplicate values.

EXAMPLES

Example 1

Input	`nums = [-1, 0, 1, 2, -1, -4]`
Output	`2`
Why	`The distinct value-triplets are [-1, -1, 2] and [-1, 0, 1]. Even though [-1, 0, 1] can be formed with different index sets, it is counted once.`

Example 2

Input	`nums = [0, 0, 0, 0]`
Output	`1`
Why	`The only triplet is [0, 0, 0].`

Example 3

Input	`nums = [1, 2, 3]`
Output	`0`
Why	`No three elements sum to 0.`

CONSTRAINTS

- 0 <= len(nums) <= 3000
- -10^5 <= nums[i] <= 10^5
- The array may contain duplicates.

QUESTION

- If len(nums) < 3 , the answer is 0 .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_triplets(nums):
    found = set()
    for i in range(len(nums)):
        for j in range(i_+_1,_len(nums)):
            for k in range(j_+_1,_len(nums)):
                if (
                    nums[i] + nums[j] + nums[k]
                    == 0
                ):
                    found.add(
                        tuple(
                            sorted(
                                (
                                    nums[i],
                                    nums[j],
                                    nums[k],
                                )
                            )
                        )
                    )
    return len(found)
```

TIME	SPACE
O(n^3)	O(triplets)

WHY IT IS SLOW

Tries all index triples and stores the sorted value triple to remove repeats.

OPTIMAL SOLUTION · Sort and two pointers

TIME	SPACE
O(n^2)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the unique triplets themselves.

The same sweep, appending [nums[i], nums[left], nums[right]] on every hit instead of counting. The output size can be O(n^2), which dominates the cost.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count Unique Triplets That Sum to Zero
#
# Given an integer array nums , count how many unique
# triplets (nums[i], nums[j], nums[k]) with three distinct
```

```
# indices i < j < k satisfy nums[i] + nums[j] + nums[k] ==
# 0 . Two triplets are considered the same if they consist
# of the same multiset of values, regardless of the
# indices they came from. Return the number of distinct
# value-triplets (not the number of index combinations).
# Function signature: def count_zero_sum_triplets(nums:
# list[int]) -> int: ... Follow-up (as reported): Return
# the actual list of unique triplets instead of the count.
# Hint: sort the array, fix one element, and use two
# pointers while skipping duplicate values.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the unique triplets themselves.
#
# INTERVIEW NOTES
# Unique value triplets means duplicates must be skipped.
# Sort, fix the first element, and run two pointers on the
# rest: the sum is too small, move left up; too large,
# move right down. Skipping equal values for all three
# positions keeps each triplet counted once.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: sort. Fix the smallest value nums[i] and sweep two pointers
# (left, right) inward over the rest: a sum below 0 needs a bigger left
# value, a sum above 0 needs a smaller right value. Skip repeated values
# so each triplet is counted once.
# Complexity: Time O(n^2), Space O(n) for the sorted copy
def count_zero_sum_triplets(nums):
    nums = sorted(nums)
    count = 0
    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue                                # the same smallest value again
        left = i + 1
        right = len(nums) - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                count += 1
                left += 1
                right -= 1
                while left < right and nums[left] == nums[left - 1]:
                    left += 1                       # skip a repeated middle value
    return count


# ---- main question: tests ----
```

```
def test_main_question():
    assert count_zero_sum_triplets([-1, 0, 1, 2, -1, -4]) == 2
    assert count_zero_sum_triplets([0, 0, 0, 0]) == 1
    assert count_zero_sum_triplets([1, 2, 3]) == 0
    assert count_zero_sum_triplets([]) == 0


# ========================================================
# FOLLOW-UP 1: Return the unique triplets themselves
# ========================================================
# ---- follow-up 1: function ----
# Answer: the same sweep, but on every hit append the triplet
# [nums[i], nums[left], nums[right]] instead of counting. The output can
# have O(n^2) triplets, which dominates the cost.
# Complexity: Time O(n^2), Space O(n) plus the answer
def zero_sum_triplets(nums):
    nums = sorted(nums)
    found = []
    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue
        left = i + 1
        right = len(nums) - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                found.append([nums[i], nums[left], nums[right]])
                left += 1
                right -= 1
                while left < right and nums[left] == nums[left - 1]:
                    left += 1
    return found


# ---- follow-up 1: tests ----
def brute_force_triplets(nums):
    # try every triple (slow, obviously right) and keep the distinct sorted ones
    seen = set()
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            for k in range(j + 1, len(nums)):
                if nums[i] + nums[j] + nums[k] == 0:
                    seen.add(tuple(sorted([nums[i], nums[j], nums[k]])))
    return seen


def test_follow_up_1():
    assert zero_sum_triplets([-1, 0, 1, 2, -1, -4]) == [[-1, -1, 2], [-1, 0, 1]]
    random.seed(15)
    for round_number in range(300):
        nums = []
        for i in range(random.randint(0, 9)):
            nums.append(random.randint(-4, 4))
        triplets = zero_sum_triplets(nums)
        assert len(triplets) == count_zero_sum_triplets(nums)
        assert set(tuple(t) for t in triplets) == brute_force_triplets(nums)


# ========================================================
# RUN EVERYTHING
```

```
# ========================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 21 OF 43

Walk two strings at once with two indices

3 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Walk two strings at once with two indices | 3 questions | tap a title to jump to it

READ FIRST: the code template for these 3 questions (tap here) >

M 112. Can board states be transformed? **PracHub * 2026-04-01 ***

M 113. Decide string transform with directional tokens and walls **PracHub * 2025-07-15 ***

M 114. Determine if two sentences are similar. **ND**

CODE TEMPLATE | GROUP 21 OF 43 | 3 QUESTIONS, ONE PATTERN

Walk two strings at once with two indices

The one idea these questions share

Keep i on one string and j on the other. Skip characters that do not matter, compare the rest, and decide what to do at the first difference. Substring search, ordering, similarity and move-based transforms are all this loop.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def first_difference(a, b, skip="."):
    i = j = 0
    while i < len(a) and j < len(b):
        while i < len(a) and a[i] == skip:      # CHANGE 1: skip characters that do not matter
            i += 1
        while j < len(b) and b[j] == skip:
            j += 1
        if i == len(a) or j == len(b):
            break
        if a[i] != b[j]:                        # CHANGE 2: the rule when they differ
            return i, j
        i, j = i + 1, j + 1
    return None
```

2. Helper functions that usually get written next to it

```
def runs(s):                                    # run-length encoding: "aaab" -> [("a", 3), ("b", 1)]
    out = []
    for ch in s:
        if out and out[-1][0] == ch:
            out[-1][1] += 1
        else:
            out.append([ch, 1])
    return [(c, n) for c, n in out]
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
112	Can board states be transformed?	skip the X squares in both strings; the remaining letters must match in order, and L may only move left, R only right
113	Decide string transform with directional tokens and walls	the same walk with dots as blanks and the movement rule from the statement (walls block)
114	Determine if two sentences are similar.	zip the two sentences; each pair of words must be equal or listed as similar (store pairs in both directions)

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 112 of 197 | topic 21 of 43: Walk two strings at once with two indices | 1 of 3

Template fit: skip the X squares in both strings; the remaining letters must match in order, and L may only move left, R only right | * asked 202 days ago *

MEDIUM | PracHub | Technical Screen | 112 of 197

Can board states be transformed?

POSTED 2026-04-01 | 202 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 40.3 | 30 DAYS 0 | PRIORITY 107/288

ABOUT THIS QUESTION

Topic: Walk two strings at once with two indices

Optimal answer from: the interview site's own solution

LeetCode match: LC 777 Swap Adjacent in LR String (inferred from overview)

Posted on PracHub: 2026-04-01 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/can-board-states-be-transformed

QUESTION

A board is a string of 'L', 'R', '#' and 'X'. 'X' is an empty square. An 'L' piece may move left into an empty square, an 'R' piece may move right into an empty square, pieces never jump over each other, and '#' (a wall piece) never moves. Return whether start can be turned into end by any number of moves.

Function signature

def can_transform(start: str, end: str) -> bool

EXAMPLES

Example 1	
Input	`'RXXLRXRXL', 'XRLXXRRLX'`
Output	`true`
Why	`L moves left and R moves right while keeping their order: this classic board is reachable.`

Example 2	
Input	`'XL', 'LX'`
Output	`true`
Why	`L can move left.`

Example 3	
Input	`'LX', 'XL'`
Output	`false`
Why	`L cannot move to the right.`

Example 4	
Input	`'#X', 'X#'`
Output	`false`
Why	`A wall never moves.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def can_transform(start, target):
    seen, queue = {start}, deque([start])
    while queue:
        board = queue.popleft()
        if board == target:
            return True
        for nxt in legal_moves(board):
            if nxt not in seen:  # XL->LX, RX->XR
                seen.add(nxt)
                queue.append(nxt)
    return False
```

TIME	SPACE
exponential in the length	exponential

WHY IT IS SLOW

The number of reachable boards grows exponentially. Pieces never cross, so compare the sequence of L and R pieces and check that no L moved right and no R moved left, in O(n).

OPTIMAL SOLUTION · Two pointers over the pieces

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the minimum number of moves.

Because pieces never interact beyond keeping their order, each piece moves independently from its start index to its target index, so the minimum is the sum of the absolute index differences when the transformation is possible.

FOLLOW-UP 2 Add walls that cannot be crossed (next question).

A wall is just another piece that never moves: it stays in the sequence of non-blank symbols and must keep the same index in both strings. The same pointer walk handles it.

FOLLOW-UP 3 Pieces can move one step per round and several move at once.

The count of rounds is the maximum over pieces of their distance, if pieces never block each other; with blocking you simulate rounds or use the per-piece arrival times with the ordering constraint.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Can board states be transformed?
#
# A board is a string of 'L', 'R', '#' and 'X'. 'X' is an
# empty square. An 'L' piece may move left into an empty
# square, an 'R' piece may move right into an empty
```

```
# square, pieces never jump over each other, and '#' (a
# wall piece) never moves. Return whether start can be
# turned into end by any number of moves.
#
# Signature:
#   def can_transform(start: str, end: str) -> bool
#
# Examples:
# 'RXXLRXRXL', 'XRLXXRRLX'  ->  true
# 'XL', 'LX'  ->  true
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the minimum number of moves.
# 2. Add walls that cannot be crossed (next question).
# 3. Pieces can move one step per round and several move
# at once.
#
# INTERVIEW NOTES
# Think invariants, not search. Pieces cannot jump each
# other, so the sequence of L and R letters (ignoring
# blanks) cannot change. Within that, an L can only go
# left and an R only right, so compare the position of
# each piece in the start and in the target. That reduces
# exponential BFS to one pass with two pointers.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def pieces(board):
    # the symbols that are not blank ("X"), each with its position
    result = []
    for i in range(len(board)):
```

```
        if board[i] != "X":
            result.append((board[i], i))
    return result


# ---- main question: main function ----
# Idea: "L" can only move left, "R" only right, "X" is a blank cell and
# "#" is a wall that never moves. Pieces never pass each other, so the
# ORDER of the non-blank symbols is fixed. Take the pieces of both boards
# and compare them one by one (two pointers): the same symbol; an L may
# only end up further left (or in place), an R further right; a wall must
# stay exactly where it is. (Walls are also never jumped over, because
# the order of the pieces cannot change.)
# Complexity: Time O(n), Space O(n)
def can_transform(start, end):
    if len(start) != len(end):
        return False
    a = pieces(start)
    b = pieces(end)
    if len(a) != len(b):
        return False
    for k in range(len(a)):
        symbol_a, i = a[k]
        symbol_b, j = b[k]
```

```
        if symbol_a != symbol_b:
            return False
        if symbol_a == "L" and i < j:
            return False                    # an L cannot move right
        if symbol_a == "R" and i > j:
            return False                    # an R cannot move left
        if symbol_a == "#" and i != j:
            return False                    # a wall cannot move
    return True


# ---- main question: tests ----
def test_main_question():
    assert can_transform("RXXLRXRXL", "XRLXXRRLX")
    assert not can_transform("X", "L")
    assert can_transform("XXXL", "XXLX")
    assert not can_transform("XR", "RX")              # R cannot move left
    assert not can_transform("LX", "XL")              # L cannot move right
    assert not can_transform("RL", "LR")              # pieces keep their order
```

```
# =============================================================
# FOLLOW-UP 1: Return the minimum number of moves
# =============================================================
# ---- follow-up 1: function ----
# Answer: pieces only have to keep their order, so each piece moves on its
# own from its start index to its end index: the minimum number of moves
# is the sum of the index differences (when the transformation is possible).
# Complexity: Time O(n), Space O(n)
def min_moves(start, end):
    if not can_transform(start, end):
        return None
    a = pieces(start)
    b = pieces(end)
    total = 0
    for k in range(len(a)):
        difference = a[k][1] - b[k][1]
        if difference < 0:
            difference = -difference
        total += difference
    return total


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert min_moves("RXXLRXRXL", "XRLXXRRLX") == 4
    assert min_moves("RXX", "XXR") == 2
    assert min_moves("RX", "XR") == 1
    assert min_moves("LL", "RR") is None


# =============================================================
# FOLLOW-UP 2: Add walls that cannot be crossed (next question)
# =============================================================
# ---- follow-up 2: tests ----
# Answer: a wall "#" is just a piece that must stay where it is; since
# pieces never pass each other nothing crosses it (can_transform above).
def test_follow_up_2():
    assert can_transform("R#X", "R#X")
    assert not can_transform("R#X", "X#R")            # R cannot jump over the wall


# =============================================================
# FOLLOW-UP 3: Pieces can move one step per round and several move at once
# =============================================================
```

```
# ---- follow-up 3: function ----
```

```
# Answer: if pieces never block each other, the number of rounds is the
# longest single trip. With blocking, simulate the rounds or use each
# piece's arrival time together with the order constraint.
# Complexity: Time O(n), Space O(n)
def rounds_needed(start, end):
    if not can_transform(start, end):
        return None
    a = pieces(start)
    b = pieces(end)
    longest = 0
    for k in range(len(a)):
        trip = a[k][1] - b[k][1]
        if trip < 0:
            trip = -trip
        if trip > longest:
            longest = trip
    return longest


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert rounds_needed("RXX", "XXR") == 2
    assert rounds_needed("LL", "RR") is None


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(start: str, target: str) -> bool:
```

Sample call

```
solution("R_L|__L", "_RL|L__")
```

Test cases

Input	Expected
`("R_L|__L", "_RL|L__")`	`True`
`("L_|R", "L|_R")`	`False`

STUDY PATH: question 113 of 197 | topic 21 of 43: Walk two strings at once with two indices | 2 of 3

Template fit: the same walk with dots as blanks and the movement rule from the statement (walls block) | * asked 462 days ago *

MEDIUM | PracHub | Technical Screen | 113 of 197

Decide string transform with directional tokens and walls

POSTED 2025-07-15 | 462 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 40.3 | 30 DAYS 0 | PRIORITY 108/288

ABOUT THIS QUESTION

Topic: Walk two strings at once with two indices

Optimal answer from: the interview site's own solution

LeetCode match: LC 777 Swap Adjacent in LR String (with X walls)

Posted on PracHub: 2025-07-15 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/decide-string-transform-with-directional-tokens-and-walls

QUESTION

Two equal-length strings start and target use the characters 'L', 'R', '.' (an empty slot) and 'X' (an immovable wall). An 'L' token can move left into empty slots, an 'R' token can move right into empty slots, tokens can never pass through each other or through a wall, and 'X' never moves. Return whether start can be turned into target by any number of moves.

Function signature

def can_transform(start: str, end: str) -> bool

EXAMPLES

Example 1	
Input	`'R..L', '.RL.'`
Output	`true`
Why	`R moves one step right and L one step left: both stay in order.`

Example 2	
Input	`'L.', '.L'`
Output	`false`
Why	`L can only move left, never right.`

Example 3	
Input	`'X.', '.X'`
Output	`false`
Why	`A wall cannot move.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def can_transform(start, target):
    seen, queue = {start}, deque([start])
    while queue:
        board = queue.popleft()
        if board == target:
            return True
        for nxt in one_step(board):
            if nxt not in seen:
                seen.add(nxt)
                queue.append(nxt)
    return False
```

TIME	SPACE
exponential	exponential

WHY IT IS SLOW

The number of reachable boards grows exponentially with the length. Comparing the non-dot sequence (walls and pieces in order) with position rules (L never moves right, R never moves left, walls stay) is O(n).

OPTIMAL SOLUTION · Compare the non-dot sequences with position rules

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Segments between walls are independent.

You can also split both strings at the walls (they must be at identical positions) and apply the L/R rule to each segment separately. The single pass above is equivalent.

FOLLOW-UP 2 Return the moves or the minimum number of steps.

Per piece, the steps are the index difference; list moves by letting each piece walk one cell at a time in an order that never blocks (move L pieces left to right and R pieces right to left).

FOLLOW-UP 3 The board is 2D.

Then pieces can pass each other by going around, and invariants like order no longer hold. Ask for the exact movement rules; most likely you would model it as a graph search with a heuristic.

FOLLOW-UP 4 Return the minimum number of moves.

When the transformation is possible each piece moves independently from its start index to its target index, so the minimum is the sum of the absolute index differences (checked against BFS on small strings).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Decide string transform with directional
# tokens and walls
#
# Two equal-length strings start and target use the
# characters 'L', 'R', '.' (an empty slot) and 'X' (an
# immovable wall). An 'L' token can move left into empty
# slots, an 'R' token can move right into empty slots,
# tokens can never pass through each other or through a
# wall, and 'X' never moves. Return whether start can be
# turned into target by any number of moves.
#
# Signature:
#   def can_transform(start: str, end: str) -> bool
#
# Examples:
# 'R..L', '.RL.'  ->  true
# 'L.', '.L'  ->  false
#
# FOLLOW-UPS (answered below, in this order):
# 1. Segments between walls are independent.
# 2. Return the moves or the minimum number of steps.
# 3. The board is 2D.
# 4. Return the minimum number of moves.
#
# INTERVIEW NOTES
# Walls are pieces that never move. Ignore the dots and
# compare the sequences of L, R and X: they must match in
# order and in count, and each X must be at the same index
# in both strings. For the L and R pieces keep the same
# rule: L never moves right, R never moves left. Order is
# preserved, so no piece can get across a wall.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import itertools
from collections import deque


# ============================================================
# MAIN QUESTION
```

```
# ============================================================
# ---- main question: helper functions ----
def pieces(board):
    # everything except the empty "." slots, each with its position
    result = []
    for i in range(len(board)):
        if board[i] != ".":
            result.append((board[i], i))
    return result


# ---- main question: main function ----
# Idea: "L" moves left, "R" moves right, "X" is a wall that never
# moves, "." is an empty slot. Pieces never pass each other, so the order
# of the non-dot symbols is fixed. Walk both strings skipping the dots:
# the next symbol must be equal; a wall must be at the same index; an L
# must have start index >= end index; an R must have start index <= end index.
# Complexity: Time O(n), Space O(n)
def can_transform(start, end):
    if len(start) != len(end):
```

```
        return False
    a = pieces(start)
    b = pieces(end)
    if len(a) != len(b):
        return False
    for k in range(len(a)):
        symbol_a, i = a[k]
        symbol_b, j = b[k]
        if symbol_a != symbol_b:
            return False
        if symbol_a == "X" and i != j:
            return False
        if symbol_a == "L" and i < j:
            return False
        if symbol_a == "R" and i > j:
            return False
    return True


# ---- main question: tests ----
def test_main_question():
    assert can_transform("R.L", ".RL") is True
    assert not can_transform("RL.", "LR.")
    assert can_transform("R.X.L", ".RX.L") is True         # a wall cannot move and splits the board
    assert not can_transform("R.X", "X.R")
```

```
# =============================================================
# FOLLOW-UP 1: Segments between walls are independent
# =============================================================
# Answer: yes: a wall never moves and nothing crosses it, so each
# segment between two walls can be solved on its own. can_transform does
# the same job in one pass because walls are compared by position.


# =============================================================
# FOLLOW-UP 2: Return the moves or the minimum number of steps
# =============================================================


# =============================================================
# FOLLOW-UP 4: Return the minimum number of moves
# =============================================================
# ---- follow-up 4: function ----
# Answer: when the transformation is possible every piece moves on its
# own from its start index to its end index, so the minimum is the sum of
# the index differences (checked below against a breadth-first search).
# Complexity: Time O(n), Space O(n)
def min_moves(start, end):
    if not can_transform(start, end):
        return None
    a = pieces(start)
    b = pieces(end)
    total = 0
    for k in range(len(a)):
        difference = a[k][1] - b[k][1]
        if difference < 0:
            difference = -difference
        total += difference
    return total


# ---- follow-up 4: tests ----
def one_move_away(board):
```

```
    # the boards you get by moving one piece one step: an R into a dot on
    # its right, or an L into a dot on its left
    result = []
    for i in range(len(board)):
        if board[i] == "R" and i + 1 < len(board) and board[i + 1] == ".":
            result.append(board[:i] + ".R" + board[i + 2:])
        if board[i] == "L" and i > 0 and board[i - 1] == ".":
            result.append(board[:i - 1] + "L." + board[i + 1:])
    return result


def bfs_distances(start):
    # the number of moves from start to every board it can reach
    distance = {start: 0}
    queue = deque([start])
    while queue:
        board = queue.popleft()
        for nxt in one_move_away(board):
            if nxt not in distance:
                distance[nxt] = distance[board] + 1
                queue.append(nxt)
    return distance


def test_follow_up_4():
    assert min_moves("R..L", ".RL.") == 2
    assert min_moves("L.", ".L") is None
    for n in range(1, 5):                                   # every board of up to 4 cells, against a real s
earch
        boards = []
        for symbols in itertools.product("LR.X", repeat=n):
            boards.append("".join(symbols))
        for start in boards:
            reach = bfs_distances(start)
            for end in boards:
                assert can_transform(start, end) == (end in reach)
                if end in reach:
                    assert min_moves(start, end) == reach[end]


# ======================================================
# FOLLOW-UP 3: The board is 2D
# ======================================================
# Answer: the order argument no longer holds in two dimensions (pieces
# can go around each other), so it becomes a search problem (BFS over
# board states) or a flow / matching problem.


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def canTransform(start: str, target: str) -> bool:
```

Sample call

```
canTransform('R...L', 'RL...')
```

Test cases

Input	Expected
`('R...L', 'RL...')`	`True`
`('R.L', 'R..L')`	`False`

STUDY PATH: question 114 of 197 | topic 21 of 43: Walk two strings at once with two indices | 3 of 3

Template fit: zip the two sentences; each pair of words must be equal or listed as similar (store pairs in both directions) | ND

MEDIUM | Exponent | 114 of 197

Determine if two sentences are similar.

ASKED (ANY CO.) ~4 YEARS AGO | OLDER

STUDY: TALK | ALL TIME 21 | 30 DAYS 0 | PRIORITY 165/288

ABOUT THIS QUESTION

Topic: Walk two strings at once with two indices

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1813 Sentence Similarity III

Last asked (any company): 4 years ago, about 2022-10 (Exponent shows only a rough age) | 1 interview

Original: https://www.aced.io/questions/2280/determine-similar-sentences

QUESTION

Determine if Two Sentences Are Similar. Two sentences are similar if they have the same length and each pair of corresponding words in the two sentences is similar. The similarity between words is defined by the provided list of similar word pairs. A word is always similar to itself. For example, if we have the list of similar word pairs as [("great", "good"), ("acting","drama"), ("skills","talent")], then the sentences "You have great acting skills" and "You have good drama talent" are similar. Examples sentence1 = ["Let's", "code", "in", "Python"] sentence2 = ["Let's", "program", "in", "Python"] similarPairs = [("code", "program"),] output: true sentence1 = ["I", "love", "to", "play", "football"] sentence2 = ["I", "love", "playing", "soccer"] similarPairs = [("play", "playing"), ("football", "soccer")] output: false, different sentence lengths sentence1 = ["Do", "you", "like", "coffee"] sentence2 = ["Do", "you", "love", "coffee"] similarPairs = [("like", "enjoy"), ("coffee", "tea"),] output: false

"like" is not similar to

"love" based on the given pairs. sentence1 = ["I", "really", "love", "leetcode", "and", "apples"] sentence2 = ["I", "so", "like", "codesignal", "and", "oranges"] similarPairs = [("very", "so"), ("love", "adore"), ("really", "very"), ("leetcode", "codesignal"), ("apples", "oranges"), ("like", "adore"),] output: trueThe provided solution checks if two given sentences are similar by using a graph-based approach. It constructs a graph where words are c

EXAMPLES

Example 1	
Input	`['great', 'acting', 'skills'], ['fine', 'drama', 'talent'], [['great', 'fine'], ['acting', 'drama'], ['skills', 'talent']]`
Output	`true`
Why	`Each word is paired with a similar word.`

Example 2	
Input	`['a'], ['a', 'b'], []`
Output	`false`
Why	`Sentences of different lengths are never similar.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s1, s2, pairs):
    if len(s1) != len(s2):
        return False
    for a, b in zip(s1, s2):
        if a == b:
            continue
        if [a, b] in pairs or [b, a] in pairs:
            continue
        return False
    return True
```

TIME	SPACE
O(n * P)	O(1)

WHY IT IS SLOW

Each word pair scans the whole pair list, O(n * P). Putting the pairs into a set makes each check O(1).

OPTIMAL SOLUTION · Set of pairs (not transitive)

TIME	SPACE
O(n + P)	O(P)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Similarity is transitive (a ~ b and b ~ c means a ~ c).

Union-find over the words: union each pair, then two words are similar if they are equal or have the same root. Path compression keeps it near O(n + P).

FOLLOW-UP 2 One sentence is the other with a sentence of words inserted somewhere (LeetCode 1813).

Match the longest common prefix and the longest common suffix of the word lists; the shorter list must be covered by prefix plus suffix.

FOLLOW-UP 3 Case should not matter, or words can have synonyms with scores.

Normalise with casefold before hashing. For scores, a graph with weighted edges and a threshold query turns it into reachability along strong edges.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Determine if two sentences are similar.
#
# Determine if Two Sentences Are Similar. Two sentences
# are similar if they have the same length and each pair
# of corresponding words in the two sentences is similar.
# The similarity between words is defined by the provided
# list of similar word pairs. A word is always similar to
# itself. For example, if we have the list of similar word
```

```
# pairs as [("great", "good"), ("acting","drama"),
# ("skills","talent")], then the sentences "You have great
# acting skills" and "You have good drama talent" are
# similar.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Similarity is transitive (a ~ b and b ~ c means a ~
# c).
# 2. One sentence is the other with a sentence of words
# inserted somewhere (LeetCode 1813).
# 3. Case should not matter, or words can have synonyms
# with scores.
#
# INTERVIEW NOTES
# Ask which similarity you have: from a pair list,
# transitive or not. Without transitivity it is a set
# lookup per word pair. With transitivity, similar words
# form groups, which is union-find (or BFS on a word
# graph). The variant where a block is inserted is a
# different problem: match the common prefix and suffix of
# the word lists.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: put every similar pair in a set, in both directions. Two sentences
# are similar when they have the same number of words and every word
# pair is equal or listed. This is NOT transitive: good ~ fine and
# fine ~ okay do not make good ~ okay.
```

```
# Complexity: Time O(n + p) for n words and p pairs, Space O(p)
def similar_direct(s1, s2, pairs):
    if len(s1) != len(s2):
        return False
    allowed = set()
    for a, b in pairs:
        allowed.add((a, b))
        allowed.add((b, a))
    for i in range(len(s1)):
        if s1[i] != s2[i] and (s1[i], s2[i]) not in allowed:
            return False
    return True


# ---- main question: tests ----
def test_main_question():
    pairs = [("great", "good"), ("acting", "drama"), ("skills", "talent")]
    a = "You have great acting skills".split()
    b = "You have good drama talent".split()
    assert similar_direct(a, b, pairs)
    assert not similar_direct(a, b[:-1], pairs)
    assert not similar_direct(["a"], ["b"], [])
    assert similar_direct(["a"], ["a"], [])
    chain = [("good", "fine"), ("fine", "okay")]
    assert not similar_direct(["good"], ["okay"], chain)        # not transitive
```

```
# ========================================================
# FOLLOW-UP 1: Similarity is transitive (a ~ b and b ~ c means a ~ c)
# ========================================================
# ---- follow-up 1: helper functions ----
class DisjointSet:
    # union-find over words: words in the same group are similar
    def __init__(self):
        self.parent = {}

    def find(self, x):
        if x not in self.parent:
            self.parent[x] = x
        root = x
        while self.parent[root] != root:
            root = self.parent[root]
        while self.parent[x] != root:              # path compression: point everything on the way at the r
oot
            following = self.parent[x]
            self.parent[x] = root
            x = following
        return root

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


# ---- follow-up 1: function ----
# Answer: union-find over the words: union each listed pair; two words are
# similar when they are equal or have the same root. Path compression
# keeps it near O(n + P).
# Complexity: Time O((n + p) * alpha) (union-find), Space O(p)
def similar_transitive(s1, s2, pairs):
    if len(s1) != len(s2):
        return False
    groups = DisjointSet()
    for a, b in pairs:
        groups.union(a, b)
    for i in range(len(s1)):
        if s1[i] != s2[i] and groups.find(s1[i]) != groups.find(s2[i]):
            return False
    return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    chain = [("good", "fine"), ("fine", "okay")]
    assert similar_transitive(["good"], ["okay"], chain)
    assert not similar_transitive(["good"], ["bad"], chain)


# ========================================================
# FOLLOW-UP 2: One sentence is the other with a sentence of words inserted somewhere (LeetCode 1813)
# ========================================================
# ---- follow-up 2: function ----
# Answer: match the longest common PREFIX and the longest common SUFFIX of
# the two word lists; the shorter list must be covered by prefix plus
# suffix (they must not overlap).
# Complexity: Time O(n), Space O(1)
def similar_insertion(s1, s2):
    if len(s1) > len(s2):
        s1, s2 = s2, s1                            # s1 is the shorter sentence
    head = 0
    while head < len(s1) and s1[head] == s2[head]:
        head += 1
```

```
    tail = 0
    room = len(s1) - head
    while tail < room and s1[-1 - tail] == s2[-1 - tail]:
        tail += 1
    return head + tail == len(s1)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert similar_insertion("My name is Haley".split(), "My Haley".split())
    assert similar_insertion(["of"], ["A", "lot", "of"])
    assert similar_insertion("Eating right now".split(), ["Eating"])
    assert not similar_insertion(["a", "b"], ["c", "d"])


# ======================================================
# FOLLOW-UP 3: Case should not matter, or words can have synonyms with scores
# ======================================================
# ---- follow-up 3: function ----
# Answer: normalise the words with casefold() before comparing or hashing.
# For synonyms with scores, build a graph with weighted edges and ask
# "is there a path where every edge is above the threshold?" (the same
# union-find idea over the strong edges only).
# Complexity: Time O(n), Space O(n)
def normalise(words):
    result = []
    for w in words:
        result.append(w.casefold())
    return result


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert normalise(["Great", "ACTING"]) == ["great", "acting"]
    assert similar_direct(normalise(["Great"]), normalise(["great"]), [])


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Determine if Two Sentences Are Similar. Two sentences are similar if they have the same length and each pair of corresponding words in the two sentences is similar. The similarity between words is defined by the provided list of similar word pairs. A word is always similar to itself. For example, if we have the list of similar word pairs as [("great", "good"), ("acting","drama"), ("skills","talent")], then the sentences "You have great acting skills" and "You have good drama talent" are similar. Examples

OFFICIAL PAGE TEXT

```
sentence1 = ["Let's", "code", "in", "Python"]
sentence2 = ["Let's", "program", "in", "Python"]
similarPairs = [
    ("code", "program"),
]
output: true

sentence1 = ["I", "love", "to", "play", "football"]
sentence2 = ["I", "love", "playing", "soccer"]
similarPairs = [("play", "playing"), ("football", "soccer")]
output: false, different sentence lengths

sentence1 = ["Do", "you", "like", "coffee"]
sentence2 = ["Do", "you", "love", "coffee"]
similarPairs = [
    ("like", "enjoy"),
    ("coffee", "tea"),
]
output: false  # "like" is not similar to "love" based on the given pairs.

sentence1 = ["I", "really", "love", "leetcode", "and", "apples"]
sentence2 = ["I", "so", "like", "codesignal", "and", "oranges"]
similarPairs = [
    ("very", "so"),
    ("love", "adore"),
    ("really", "very"),
    ("leetcode", "codesignal"),
    ("apples", "oranges"),
    ("like", "adore"),
]
```

```


output: true
```

OFFICIAL PAGE TEXT

The provided solution checks if two given sentences are similar by using a graph-based approach. It constructs a graph where words are connected if they are similar according to the provided pairs. Then, it performs a depth-first search (DFS) to check if each pair of corresponding words from the two sentences are connected in the graph. The Depth-First Search (DFS) algorithm is well-suited for this question as it efficiently explores the graph structure created from the similar word pairs. DFS is ideal for checking connectivity between words, which aligns with the task of determining if two words are similar based on the provided pairs. Its memory efficiency is beneficial for handling potentially large sets of sentences and word pairs. Additionally, DFS offers a simple implementation, either through recursion or using a stack, which aids in writing clean and understandable code. Its flexibility allows for easy modification to incorporate additional constraints or optimizations if needed. Therefore, DFS provides an elegant and effective solution for traversing the graph and determining sentence similarity. Remember to handle potential off-by-one errors when implementing this solution due to index-based iteration over the sentences. Time Complexity: Let n be the maximum number of words in the sentences, and m be the number of similar word pairs. Constructing the graph takes O(m) time. Then, for each word pair, the DFS takes O(n) time in the worst case, resulting in a total time complexity of O(n + m). Space Complexity: In the worst case, the graph can contain O(m) edges. Additionally, in the DFS, the space complexity is O(n) due to the recursion stack or auxiliary data structures to track visited nodes. Therefore, the overall space complexity is O(n + m).Watch Simon, software engineer at Google, answer the question "determine if two sentences are similar."No answers contributed by the community yet.Related questionsDetermine if a number is a power of two.Determine if two words are sorted in lexicographic order.Related coursesCourseCourseSoftware Engineering Interview PrepLand your dream software engineering role at Google, Amazon, Microsoft, Meta, Apple, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates—practice data structures, algorithms, system design, people management, behavioral interviews, and more.Start learningCourseCourseSystem Design InterviewsLearn how to answer the latest system design questions across product, infrastructure, and AI domains. Features in-depth video examples, written

breakdowns, and helpful reference patterns. Watch senior engineers and managers answer these questions in mock interview videos.Start learningPythonJavaScriptJavaC++PythonEditorShareLoading...OutputTest ResultsConsoleSign up to run code

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 22 OF 43

Simulate a machine: a class with state and one branch per kind of event

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

5
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Simulate a machine: a class with state and one branch per kind of event | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M	115. Tic-Tac-Toe System with K Players and 3-in-a-Row Win	1Point3Acres * 2026-05-18 *
M	116. Design Tic-Tac-Toe With K Players	PracHub * 2026-02-12 *
M	117. Implement iterator merging lists with filtering	PracHub * 2026-01-06 *
M	118. Infinite Deck Card Duel	ColorStack * 2025-09-26 *
M	119. Design an elevator control system	PracHub * 2025-09-06 *

CODE TEMPLATE | GROUP 22 OF 43 | 5 QUESTIONS, ONE PATTERN

Simulate a machine: a class with state and one branch per kind of event

The one idea these questions share

Put the state in __init__, give it one method that takes an event, and write one branch per event kind. Board games keep a grid or counters and check for a win after each move; game-of-life style updates compute every cell from the old state before changing it.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
class Machine:
    def __init__(self):
        self.score, self.lives = 0, 3             # CHANGE 1: the state
        self.over = False

    def process(self, event):
        if self.over:
            return
        if event == "coin":                       # CHANGE 2: one branch per kind of event
            self.score += 100
        elif event == "damage":
            self.lives -= 1
            self.over = self.lives == 0
```

2. Helper functions that usually get written next to it

```
def neighbors8(r, c):
    return [(r + dr, c + dc) for dr in (-1, 0, 1) for dc in (-1, 0, 1) if dr or dc]


def run_through(board, r, c, dr, dc, player):    # length of the line of one player through a cell
    n, run = len(board), 1
    for sign in (1, -1):
        a, b = r + sign * dr, c + sign * dc
        while 0 <= a < n and 0 <= b < n and board[a][b] == player:
            run += 1
            a, b = a + sign * dr, b + sign * dc
    return run
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
115	Tic-Tac-Toe System with K Players and 3-in-a-Row Win	store the board; after each move count the run through the new cell in four directions and compare with the target length
116	Design Tic-Tac-Toe With K Players	per player Counters for rows, columns and both diagonals; a move wins when a counter reaches n
117	Implement iterator merging lists with filtering	keep a position per list and a turn counter; advance() looks for the next unblocked, unseen value and stores it for next()/has_next()
118	Infinite Deck Card Duel	an infinite deck is just a random draw; play rounds, compare the highest cards, and decide the champion by the lead in wins
119	Design an elevator control system	each car keeps floor, direction and stops; a request goes to the car with the lowest cost; step() moves every car one floor

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 115 of 197 | topic 22 of 43: Simulate a machine: a class with state and one branch per kind of event | 1 of 5

Template fit: store the board; after each move count the run through the new cell in four directions and compare with the target length | * asked 155 days ago * | frequency Single report

MEDIUM | 1Point3Acres | phone-screen, tech-screen | 115 of 197

Tic-Tac-Toe System with K Players and 3-in-a-Row Win

ASKED 2026-05-18 | 155 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 18.1** **30 DAYS 0** **PRIORITY 175/288**

ABOUT THIS QUESTION

Topic: Simulate a machine: a class with state and one branch per kind of event

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 348 Design Tic-Tac-Toe (related)

Frequency: Single report | Last asked: 2026-05-18 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/tic-tac-toe-k-players-three-in-a-row

QUESTION

Design Tic-Tac-Toe for k players on an n x n board. Players move in turn (1, 2, ..., k, then 1 again). move(player, r, c) places a mark and returns 'player p wins' as soon as that player has 3 marks in a straight line (row, column or diagonal), 'draw' when the board is full, and 'continue' otherwise. A move out of turn, off the board or onto a taken cell raises an error.

Function signature

```
class TicTacToe:
    def __init__(self, n: int, k: int, target: int = 3)
    def move(self, player: int, r: int, c: int) -> str
```

EXAMPLES

Example 1

Input	`g = TicTacToe(3, 2)`
	`g.move(1, 0, 0)`
	`g.move(2, 1, 0)`
	`g.move(1, 0, 1)`
	`g.move(2, 1, 1)`
	`g.move(1, 0, 2)`

Output	`g.move(1, 0, 0) -> 'continue'`
	`g.move(2, 1, 0) -> 'continue'`
	`g.move(1, 0, 1) -> 'continue'`
	`g.move(2, 1, 1) -> 'continue'`
	`g.move(1, 0, 2) -> 'player 1 wins'`

Why	`Player 1 takes the whole top row while player 2 has two in the middle row: player 1 wins.`

QUESTION

Example 2

Input	`g = TicTacToe(3, 3)`
	`g.move(1, 0, 0)`
	`g.move(2, 0, 1)`
	`g.move(3, 0, 2)`
	`g.move(1, 1, 1)`

Output	`g.move(1, 0, 0) -> 'continue'`
	`g.move(2, 0, 1) -> 'continue'`
	`g.move(3, 0, 2) -> 'continue'`
	`g.move(1, 1, 1) -> 'continue'`

Why	`With three players the turns go 1, 2, 3, 1; nobody has three in a line yet.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def status(player):
    for r, c in all_cells(board):
        for dr, dc in DIRECTIONS:
            cells = line_of_3(r, c, dr, dc)
            if all(board[x][y] == player
                   for x, y in cells):
                return "WIN"
    if is_full(board):
        return "DRAW"
    return "PLAYING"
```

TIME	SPACE
O(n^2)	**O(n^2)**
per move	

WHY IT IS SLOW

Scans O(n^2) cells per move. Only lines through the cell just played can have changed, so checking four directions around it is O(1).

OPTIMAL SOLUTION · Check the four lines through the new mark

TIME	SPACE
O(1)	**O(n^2)**
per move (O(target) when generalised)	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What changes when the winning length is a parameter w?

Replace the constant 3 by w in the run check: the cost per move becomes O(w) because each of the four directions walks at most w - 1 cells each way. The board size does not matter any more, which is the point of checking locally.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Tic-Tac-Toe System with K Players and
# 3-in-a-Row Win
#
# Design Tic-Tac-Toe for k players on an n x n board.
# Players move in turn (1, 2, ..., k, then 1 again).
# move(player, r, c) places a mark and returns 'player p
# wins' as soon as that player has 3 marks in a straight
# line (row, column or diagonal), 'draw' when the board is
# full, and 'continue' otherwise. A move out of turn, off
# the board or onto a taken cell raises an error.
#
# Signature:
#   class TicTacToe:
#       def __init__(self, n: int, k: int, target: int = 3)
#       def move(self, player: int, r: int, c: int) -> str
#
# FOLLOW-UPS (answered below, in this order):
# 1. What changes when the winning length is a parameter
# w?
#
# INTERVIEW NOTES
# A move can only create a win through the cell just
# played, so after each move look only along the four
# lines through that cell (row, column, both diagonals)
# and count how far the same player's marks extend in both
# directions. With a target of 3 that is a handful of
# cells per move, instead of rescanning the whole board.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))        # right, down, and the two diagonals


# ---- main question: main function ----
# Idea: place the mark, then check only the four lines THROUGH it. For each
# direction walk outward both ways while the cells hold the same player and
```

```
# add the two counts plus one for the new mark. A run of at least `target`
# (3 by default) wins at once. Players take turns 1, 2, ..., k, 1, ...
# `target` is the winning length w (follow-up 1): the check costs O(w)
# per move, whatever the board size.
# Complexity: Time move is O(w) for win length w, Space O(n^2)
class TicTacToe:
    def __init__(self, n, k, target=3):
        self.n = n
        self.k = k
        self.target = target
        self.board = []
        for r in range(n):
            self.board.append([0] * n)              # 0 means an empty cell
```

```
        self.moves = 0
        self.over = False

    def inside(self, r, c):
        return 0 <= r < self.n and 0 <= c < self.n

    def run_length(self, r, c, dr, dc):
        player = self.board[r][c]
        run = 1
        for sign in (1, -1):
            a = r + sign * dr
            b = c + sign * dc
            while self.inside(a, b) and self.board[a][b] == player:
                run += 1
                a += sign * dr
                b += sign * dc
        return run

    def move(self, player, r, c):
        turn = self.moves % self.k + 1
        if self.over or player != turn:
            raise ValueError("not this player's turn")
        if not self.inside(r, c) or self.board[r][c] != 0:
            raise ValueError("illegal cell")
        self.board[r][c] = player
        self.moves += 1
        for dr, dc in DIRECTIONS:
            if self.run_length(r, c, dr, dc) >= self.target:
                self.over = True
                return "player " + str(player) + " wins"
        if self.moves == self.n * self.n:
            self.over = True
```

```
            return "draw"
        return "continue"


# ---- main question: tests ----
def play(game, plan):
    results = []
    for player, r, c in plan:
        results.append(game.move(player, r, c))
    return results


def test_main_question():
    results = play(TicTacToe(3, 2), [(1, 0, 0), (2, 1, 0), (1, 0, 1), (2, 1, 1), (1, 0, 2)])
    assert results[:-1] == ["continue"] * 4
    assert results[-1] == "player 1 wins"
    sequence = [(1, 0, 0), (2, 0, 3), (3, 3, 3), (1, 1, 0), (2, 1, 3), (3, 3, 0), (1, 2, 1), (2, 2, 3)]
    assert play(TicTacToe(4, 3), sequence)[-1] == "player 2 wins"      # three players: player 2 wins down
column 3
    for bad in ((2, 0, 0), (1, 5, 5)):                                  # a wrong turn and an outside cell
are rejected
        try:
            TicTacToe(3, 2).move(bad[0], bad[1], bad[2])
            raise AssertionError(bad)
        except ValueError:
            pass
    g = TicTacToe(3, 2)                                                 # a full board with no winner is a
draw
    draw = [(1, 0, 0), (2, 0, 1), (1, 0, 2), (2, 1, 1), (1, 1, 0), (2, 1, 2), (1, 2, 1), (2, 2, 0), (1, 2,
2)]
    assert play(g, draw)[-1] == "draw"
```

```
# ============================================================
# FOLLOW-UP 1: What changes when the winning length is a parameter w?
# ============================================================
# ---- follow-up 1: tests ----
# Answer: use w instead of 3 in the run check (the `target` parameter).
# Each of the four directions walks at most w - 1 cells each way, so a move
# costs O(w), and the board size no longer matters: that is the point
# of checking only around the new mark.
def test_follow_up_1():
    game = TicTacToe(5, 2, target=4)
    plan = [(1, 0, 0), (2, 1, 0), (1, 0, 1), (2, 1, 1), (1, 0, 2), (2, 1, 2)]
    assert play(game, plan) == ["continue"] * 6                         # three in a row is not enough now
    assert game.move(1, 0, 3) == "player 1 wins"                        # four is


# ============================================================
```

```
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement the classic Tic-Tac-Toe design problem, then generalize from 2 players / 3x3 to k players / n x n.
Practice local directional win checks with boundary guards.
Write tests for corners, edges, duplicate moves, and simultaneous longer lines.

STUDY PATH: question 116 of 197 | topic 22 of 43: Simulate a machine: a class with state and one branch per kind of event | 2 of 5

Template fit: per player Counters for rows, columns and both diagonals; a move wins when a counter reaches n | * asked 250 days ago *

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 116 of 197

Design Tic-Tac-Toe With K Players

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 18.1 | 30 DAYS 0 | PRIORITY 174/288

ABOUT THIS QUESTION

Topic: Simulate a machine: a class with state and one branch per kind of event

Optimal answer from: the interview site's own solution

LeetCode match: LC 348 Design Tic-Tac-Toe

Posted on PracHub: 2026-02-12 (updated 2026-05-21)

Original: https://prachub.com/coding-questions/design-tic-tac-toe-with-k-players

QUESTION

Design and implement a Tic-Tac-Toe game engine. You are given:

- **k** players, each with a unique player ID or mark.
- An **`n x n`** board, initially empty.
- Players take turns placing their own mark on an empty cell. Unlike classic Tic-Tac-Toe, a player wins as soon as they form **3 consecutive marks** in a straight line, regardless of how large **n** is. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left. Implement a class or set of functions that supports making moves and returns the game status after each move, such as:
- **`"Game Over"`** if a move is attempted after the game has ended.
- **`"<player> won"`** when the latest move causes that player to have 3 consecutive marks.
- A non-terminal status such as **`"Continue"`** if the game should keep going. Your implementation should handle invalid moves, such as placing a mark outside the board or on an occupied cell. Discuss the time and space complexity of your approach, especially for large **n**.

Design a Tic-Tac-Toe game engine for k players on an n x n board. The board starts empty, and players must play in cyclic order: player 1, player 2, ..., player k, then back to player 1. A player wins immediately when their latest valid move creates 3 consecutive marks in a straight line. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left. For this problem, implement a function that processes a list of move attempts and returns the status after each attempt. Each move is given as [player, row, col] using 0-based indexing. Return one status string per move:

- "Game Over" if a move is attempted after someone has already won.
- "Invalid move" if the move is not legal. A move is invalid if: - the player ID is not in [1, k] - it is not that player's turn - the cell is outside the board - the cell is already occupied - "<player> won" if the move is valid and causes that player to have 3 consecutive marks - "Continue" if the move is valid and the game should continue Important: an invalid move does not change the board and does not advance the turn. You do not need to detect draws. If no one wins, valid moves simply return "Continue".

Constraints

1 <= n <= 10^91 <= k <= 10^50 <= len(moves) <= 2 * 10^5Each move is a list of three integers: [player, row, col]Row and column in the input may be outside the board and should then be treated as an invalid moveYour approach should be efficient for large n, so avoid scanning the whole board after every move

QUESTION

EXAMPLES

```
Input: (3, 2, [])
Expected Output: []
```

Explanation: No moves are made, so there are no statuses to report.

```
Input: (5, 2, [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2, 1, 1], [1, 0, 2], [2, 2, 2]])
Expected Output: ['Continue', 'Continue', 'Continue', 'Continue', '1 won', 'Game Over']
```

Explanation: Player 1 forms three consecutive marks horizontally at (0,0), (0,1), and (0,2). Any later move returns 'Game Over'.

Hints

Only the latest valid move can create a new 3-in-a-row, so you never need to rescan the entire board.For large boards, store only occupied cells in a hash map keyed by (row, col), then check the 4 line directions around the new move.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def move(player, r, c):
    board[r][c] = player
    lines = [row(r), column(c)]
    if r == c:
        lines.append(main_diagonal())
    if r + c == n - 1:
        lines.append(anti_diagonal())
    for line in lines:
        if line.count(player) == n:
            return "WIN"
    return "CONTINUE"
```

TIME	SPACE
O(n) per move	**O(n^2)**

WHY IT IS SLOW

Each move rescans up to four lines, O(n). Keeping a counter per player per row, column and diagonal makes each move O(1).

OPTIMAL SOLUTION · Counters per player and per line

TIME	SPACE
O(1) per move	**O(k * n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Win with m in a row, not the full line.

Counters per line no longer work, because a line can hold a run anywhere. After each move look along the four directions from that cell, counting same-player neighbours up to m in both directions. O(m) per move.

FOLLOW-UP 2 Undo the last move.

Decrement the same counters you incremented and clear the cell; remember the move history in a stack. The counters make undo as cheap as a move.

FOLLOW-UP 3 The board is huge or unbounded.

Store only the occupied cells in a dict (as TicTacToeM does) and keep the counter-per-line version only if the win condition is a complete line of a bounded board. **Cost:** O(1) per move with line counters on a bounded board, or O(m) per move scanning four directions for an m-in-a-row win; memory O(moves).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Design Tic-Tac-Toe With K Players
#
# Design and implement a Tic-Tac-Toe game engine. You are
# given: - k players, each with a unique player ID or
# mark. - An n x n board, initially empty. - Players take
# turns placing their own mark on an empty cell. Unlike
# classic Tic-Tac-Toe, a player wins as soon as they form
# **3 consecutive marks** in a straight line, regardless
# of how large n is. A straight line may be horizontal,
# vertical, diagonal down-right, or diagonal down-left.
# Implement a class or set of functions that supports
# making moves and returns the game status after each
# move, such as: - "Game Over" if a move is attempted
# after the game has ended. - "<player> won" when the
# latest move causes that player to have 3 consecutive
# marks. - A non-terminal status such as "Continue" if the
# game should keep going. Your implementation should
# handle invalid moves, such as placing a mark outside the
# board or on an occupied cell.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Win with m in a row, not the full line.
# 2. Undo the last move.
# 3. The board is huge or unbounded.
#
# INTERVIEW NOTES
# Do not look at the board after each move. A win needs n
# marks of the same player on one line, so keep a counter
# per player and per line (each row, each column, the two
# diagonals) and bump the at-most-four lines that the move
# touches. The move wins if any of those counters reaches
# n.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))        # right, down, and the two diagonals
```

```
# ---- main question: main function ----
# Idea: only the four lines through the NEW mark can create a win. After
# a move, walk outward in both directions of each line while the cells hold
# the same player; the run length is those two counts plus the new mark.
# Players take turns 1, 2, ..., k, 1, ... An invalid move (wrong player,
# wrong turn, outside the board, taken cell) changes nothing.
# `target` is how many in a row win (follow-up 1); cells are stored in a
# dictionary, so only taken cells use memory (follow-up 3: huge board).
# Complexity: Time move is O(w) for win length w (four directions), Space O(moves made)
class TicTacToe:
    def __init__(self, n, k, target=3):
        self.n = n
        self.k = k
        self.target = target
        self.cells = {}                                 # (row, column) -> the player who took it
        self.history = []                               # the moves so far, for undo
        self.winner = None

    def turn(self):
        return len(self.history) % self.k + 1

    def run_length(self, r, c, dr, dc, player):
        # the number of marks of `player` in a line through (r, c)
        run = 1
        for sign in (1, -1):
            a = r + sign * dr
            b = c + sign * dc
            while self.cells.get((a, b)) == player:
                run += 1
                a += sign * dr
                b += sign * dc
        return run

    def move(self, player, r, c):
        if self.winner is not None:
            return "Game Over"
        if player < 1 or player > self.k or player != self.turn():
            return "Invalid move"
        if not (0 <= r < self.n and 0 <= c < self.n) or (r, c) in self.cells:
            return "Invalid move"
        self.cells[(r, c)] = player
        self.history.append((r, c))
        for dr, dc in DIRECTIONS:
```

```
            if self.run_length(r, c, dr, dc, player) >= self.target:
                self.winner = player
                return str(player) + " won"
        return "Continue"

    # follow-up 2: take the last move back (this also reopens a finished game)
    def undo(self):
        if len(self.history) == 0:
            return False
        del self.cells[self.history.pop()]
        self.winner = None
        return True


# ---- main question: tests ----
def process(n, k, moves):
    game = TicTacToe(n, k)
    results = []
    for move in moves:
        results.append(game.move(move[0], move[1], move[2]))
```

```
    return results


MOVES = [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2, 1, 1], [1, 0, 2], [2, 2, 2]]


def test_main_question():
    assert process(3, 2, MOVES) == ["Continue", "Continue", "Continue", "Continue", "1 won", "Game Over"]
    bad = "Invalid move"
    ok = "Continue"
    got = process(3, 2, [[2, 0, 0], [1, 5, 5], [1, 0, 0], [2, 0, 0], [3, 1, 1], [2, 1, 1]])
    assert got == [bad, bad, ok, bad, bad, ok]          # invalid moves do not use up the turn
    big = TicTacToe(10 ** 9, 2)                          # a huge board costs nothing: only taken cells are
 stored
    assert big.move(1, 10 ** 9 - 1, 5) == "Continue"


# ============================================================
# FOLLOW-UP 1: Win with m in a row, not the full line
# ============================================================
# ---- follow-up 1: tests ----
# Answer: counters per row and column no longer work, because a run can sit
# anywhere in a line. After each move look along the four directions from
# the new cell and count same-player cells in both directions (the `target`
# parameter): O(m) per move, whatever the board size.
def test_follow_up_1():
```

```
    game = TicTacToe(5, 2, target=4)
    for player, r, c in [(1, 0, 0), (2, 1, 0), (1, 0, 1), (2, 1, 1), (1, 0, 2), (2, 1, 2)]:
        assert game.move(player, r, c) == "Continue"
    assert game.move(1, 0, 3) == "1 won"                 # four in a row


# ============================================================
# FOLLOW-UP 2: Undo the last move
# ============================================================
# ---- follow-up 2: tests ----
# Answer: remember the moves in a list; undo removes the last cell and
# clears the winner (TicTacToe.undo above).
def test_follow_up_2():
    game = TicTacToe(3, 2)
    for move in MOVES[:5]:
        game.move(move[0], move[1], move[2])
    assert game.winner == 1
    assert game.undo() and game.winner is None
    assert game.move(1, 0, 2) == "1 won"


# ============================================================
# FOLLOW-UP 3: The board is huge or unbounded
# ============================================================
# Answer: the cells are in a dictionary, so only taken cells use memory,
# and a win is checked locally around the new mark; neither depends on n.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(n: int, k: int, moves: list[list[int]]) -> list[str]:
```

Sample call

```
solution(3, 2, [])
```

Test cases

Input	Expected
`(3, 2, [])`	`[]`
`(5, 2, [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2, 1, 1], [1, 0, 2], [2, 2, 2]])`	`['Continue', 'Continue', 'Continue', 'Continue', '1 won', 'Game Over']`

ALSO REPORTED · 1 more

Tic-Tac-Toe System with K Players and 3-in-a-Row Win | 1Point3Acres | phone-screen, tech-screen | asked 2026-05-18 | Single report

https://www.1point3acres.com/interview/problems/company/google/tic-tac-toe-k-players-three-in-a-row

Check four directions through the new mark: horizontal, vertical, main diagonal, anti-diagonal. Count contiguous marks on both sides; win if total reaches 3.
Avoid scanning the whole board after every move unless n is tiny. The local directional check is O(1) for the fixed 3-in-a-row target, or O(target) if generalized.
Clarify invalid moves, turn order, draw state, and whether 3-in-a-row must be exactly 3 or at least 3.

Preparation: Implement the classic Tic-Tac-Toe design problem, then generalize from 2 players / 3x3 to k players / n x n.
Practice local directional win checks with boundary guards.
Write tests for corners, edges, duplicate moves, and simultaneous longer lines.

STUDY PATH: question 117 of 197 | topic 22 of 43: Simulate a machine: a class with state and one branch per kind of event | 3 of 5

Template fit: keep a position per list and a turn counter; advance() looks for the next unblocked, unseen value and stores it for next()/has_next() | * asked 287 days ago *

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 117 of 197

Implement iterator merging lists with filtering

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 176/288

ABOUT THIS QUESTION

Topic: Simulate a machine: a class with state and one branch per kind of event

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #281 Zigzag Iterator

LeetCode match: LC 281 Zigzag Iterator (+ filter/dedupe) (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/implement-iterator-merging-lists-with-filtering

QUESTION

Design an iterator over several lists that takes one element from each list in turn (a zigzag), skipping any value in the blocked set and any value that was already returned. hasNext() says whether another value is available and next() returns it.

Function signature

```
class FilteredZigzag:
    def __init__(self, lists: list, blocked=())
    def hasNext(self) -> bool
    def next(self) -> int
```

EXAMPLES

Example 1

Input	`it = FilteredZigzag([[1, 2, 3], [4, 5, 6, 7], [8, 9]], blocked=[2, 5])`
	`out = []`
	`while it.hasNext():`
	`out.append(it.next())`
	`out`

Output	`out -> [1, 4, 8, 9, 3, 6, 7]`
Why	`Order of visit: 1, 4, 8, then 2 (blocked), 5 (blocked), 9, then 3, 6, then 7. The blocked values never appear.`

Example 2

Input	`it = FilteredZigzag([[1, 1], [1, 2]])`
	`out = []`
	`while it.hasNext():`
	`out.append(it.next())`
	`out`

Output	`out -> [1, 2]`
Why	`Repeated values are returned only once.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def merged_iterator(lists, blocked):
    seen = set()
    for x in round_robin(lists):
        if x in blocked or x in seen:
            continue
        seen.add(x)
        yield x
```

TIME	SPACE
O(total)	O(total)

WHY IT IS SLOW

It reads everything up front and keeps it all in memory, so it is not lazy and never works on a long or unbounded source. A lookahead value per iterator computes only the next valid item.

OPTIMAL SOLUTION · Lazy iterator with one pointer per list and a lookahead

TIME	SPACE
O(1) amortized per value	O(number of lists + distinct ids yielded)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The lists are sorted and the output must stay sorted.

Replace the round robin with a k-way merge on a heap (heapq.merge), and drop a value equal to the previous output instead of keeping a seen set: memory O(k).

FOLLOW-UP 2 Read the lists one after another, not interleaved.

Chain the lists with a generator; the same blocked/seen filtering applies. Generators give hasNext-style behaviour via a one-item lookahead.

FOLLOW-UP 3 The seen set grows without bound.

If the input is sorted or ids repeat only close together, remember only the last few ids; otherwise bound memory with a Bloom filter and accept rare false suppressions. **Cost:** O(1) per element with a Bloom filter using O(bits) memory, at the price of rare false suppressions.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement iterator merging lists with
# filtering
#
# Design an iterator over several lists that takes one
# element from each list in turn (a zigzag), skipping any
# value in the blocked set and any value that was already
# returned. hasNext() says whether another value is
```

```
# available and next() returns it.
#
# Signature:
#   class FilteredZigzag:
#       def __init__(self, lists: list, blocked=())
#       def hasNext(self) -> bool
#       def next(self) -> int
#
# FOLLOW-UPS (answered below, in this order):
# 1. The lists are sorted and the output must stay sorted.
# 2. Read the lists one after another, not interleaved.
# 3. The seen set grows without bound.
#
# INTERVIEW NOTES
# An iterator does its work lazily: hasNext() must be able
# to answer without consuming. So keep a lookahead, the
# next valid value already computed, and advance to the
# one after it when next() is called. Filtering and de-
# duplication live in the 'find the next valid value' step
# so both hasNext and next stay trivial.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: take one item from each list in turn (list 0, list 1, ..., back
# to list 0), skipping ids that are blocked and ids that were already
# returned. Keep one item of LOOKAHEAD ready, so hasNext() only has to
```

```
# check whether there is one. advance() goes round and round until it
# finds the next good item, or gives up after a whole round in which every
# list was empty.
# Complexity: Time hasNext O(1), next O(1) amortized per item scanned, Space O(items already returned)
class FilteredZigzag:
    def __init__(self, lists, blocked=()):
        self.lists = lists
        self.blocked = set(blocked)
        self.position = [0] * len(lists)            # how far we have read in each list
        self.turn = 0                               # whose turn it is
        self.seen = set()                           # the ids already returned
        self.ahead = None                           # the next item to return
        self.advance()

    def advance(self):
        self.ahead = None
        empty_in_a_row = 0
        while empty_in_a_row < len(self.lists):
            i = self.turn
            self.turn = (self.turn + 1) % len(self.lists)
            if self.position[i] >= len(self.lists[i]):
                empty_in_a_row += 1                 # this list has nothing left
                continue
            value = self.lists[i][self.position[i]]
            self.position[i] += 1
            empty_in_a_row = 0
```

```
            if value not in self.blocked and value not in self.seen:
                self.ahead = value
                return

    def hasNext(self):
        return self.ahead is not None

    def next(self):
        value = self.ahead
        self.seen.add(value)
        self.advance()
        return value


# ---- main question: tests ----
def test_main_question():
    it = FilteredZigzag([[1, 2, 3], [2, 4], [5]], blocked=[4])
    values = []
    while it.hasNext():
        values.append(it.next())
    assert values == [1, 2, 5, 3]                    # then 2 is seen already and 4 is blocked
    assert not FilteredZigzag([[], []]).hasNext()
    assert not FilteredZigzag([]).hasNext()


# =========================================================
# FOLLOW-UP 1: The lists are sorted and the output must stay sorted
# =========================================================
# ---- follow-up 1: function ----
# Answer: replace the round robin by a k-way merge with a heap
# (heapq.merge). Equal values come out next to each other, so drop a value
# equal to the previous output instead of keeping a `seen` set: memory
# O(k). A generator hands the values out one at a time.
# Complexity: Time O(N log k) for N items in k lists, Space O(k)
def merge_sorted(lists, blocked=()):
    blocked = set(blocked)
    last = None
    started = False
    for value in heapq.merge(*lists):
        if value in blocked:
            continue
        if started and value == last:
            continue                                 # the same value as the one just returned
        started = True
        last = value
        yield value


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert list(merge_sorted([[1, 3, 5], [1, 2, 5]], blocked=[3])) == [1, 2, 5]
    assert list(merge_sorted([[], []])) == []


# =========================================================
# FOLLOW-UP 2: Read the lists one after another, not interleaved
# =========================================================
# ---- follow-up 2: function ----
# Answer: go through the lists one after another (a generator); the same
# blocked and seen filtering applies. A generator plus one item of lookahead
# gives the hasNext / next behaviour.
# Complexity: Time O(N), Space O(N) for the seen set
def read_in_turn(lists, blocked=()):
    blocked = set(blocked)
```

```
    seen = set()
    for one_list in lists:
        for value in one_list:
            if value not in blocked and value not in seen:
                seen.add(value)
                yield value


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert list(read_in_turn([[1, 2], [2, 3]], [1])) == [2, 3]


# ========================================================
# FOLLOW-UP 3: The seen set grows without bound
# ========================================================
# Answer: for sorted input (follow-up 1) only the previous value is
# needed. Otherwise keep the seen ids in a bounded structure: an LRU set
# if repeats are close together, or a Bloom filter if a small chance of a
# wrongly skipped id is acceptable.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Sample call

```
solution([3, 1, 2, 3], [2, 4, 1, 5], {4})
```

Test cases

Input	Expected
([3, 1, 2, 3], [2, 4, 1, 5], {4})	[3, 1, 2, 5]
([], [1, 1], set())	[1]

STUDY PATH: question 118 of 197 | topic 22 of 43: Simulate a machine: a class with state and one branch per kind of event | 4 of 5

Template fit: an infinite deck is just a random draw; play rounds, compare the highest cards, and decide the champion by the lead in wins | * asked 389 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 118 of 197

Infinite Deck Card Duel

ASKED 2025-09-26 | 389 DAYS AGO | OLDER

STUDY: TALK **ALL TIME 14.7** **30 DAYS 0** **PRIORITY 186/288**

ABOUT THIS QUESTION

Topic: Simulate a machine: a class with state and one branch per kind of event

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 384 Shuffle an Array (related: seeded random simulation)

Frequency: Single report | Last asked: 2025-09-26 | Roles: SWE Intern

QUESTION

Design an InfiniteDeck that can be drawn from forever: each call to draw() returns a uniformly random integer card value in [1, 13] (cards are never exhausted). Make the deck seedable so games are reproducible. Then simulate a duel between two players. In each round, each player draws hand_size cards. The round winner is the player holding the single highest card that round; if both players' highest cards are equal, the round is a draw (nobody scores). Play num_rounds rounds and tally round wins per player. Finally, declare a champion: the player is champion only if they lead the other in round wins by at least 2; otherwise there is no champion. Implement: InfiniteDeck.draw() returning the next card. round_winner(hand1, hand2) -> 1 , 2 , or 0 (0 meaning draw), based on the highest card in each hand. decide_champion(wins1, wins2) -> 1 , 2 , or 0 (0 meaning no champion), applying the "lead by at least 2" rule. play_game(...) tying it together (uses the deck for randomness).

Function signature

```
import random
class InfiniteDeck:
    def __init__(self, seed: int | None = None) -> None:
        """Create a deck backed by a (optionally seeded) RNG."""
        ...
    def draw(self) -> int:
        """Return a uniformly random card value in [1, 13]."""
        ...
def round_winner(hand1: list[int], hand2: list[int]) -> int:
    """Return 1 if player 1's max card is higher, 2 if player 2's is higher, 0 on a
tie."""
    ...
def decide_champion(wins1: int, wins2: int) -> int:
    """Return the champion (1 or 2) only if they lead by >= 2 round wins, else 0."""
    ...
def play_game(deck: InfiniteDeck, hand_size: int = 5, num_rounds: int = 100) -> int:
    """Play the duel and return the champion via decide_champion, or 0 if none."""
    ...
```

Follow-up (as reported): Extend to n players: replace the two win counters with a dictionary keyed by player id and pick the highest card across all hands each round. Introduce multiple independent InfiniteDeck instances (or one shared deck with per-card metadata) to support probabilistic house rules.

QUESTION

EXAMPLES

Example 1	
Input	`round_winner([3, 7, 2], [7, 1, 9])`
Output	`2`
Why	`Player 2's highest card (9) beats player 1's highest (7).`

Example 2	
Input	`round_winner([5, 5], [5, 4])`
Output	`0`
Why	`Both players' highest card is 5, so the round is a draw.`

CONSTRAINTS

- 1 <= hand_size <= 20 , 1 <= num_rounds <= 100000 , card values in [1, 13] .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
import random
def brute_game(seed, hand_size, rounds):
    rng = random.Random(seed)
    w1 = w2 = 0
    for _ in range(rounds):
        h1 = [
            rng.randint(1, 13)
            for _ in range(hand_size)
        ]
        h2 = [
            rng.randint(1, 13)
            for _ in range(hand_size)
        ]
        if max(h1) > max(h2):
            w1 += 1
        elif max(h2) > max(h1):
            w2 += 1
    if w1 - w2 >= 2:
        return 1
    return 2 if w2 - w1 >= 2 else 0
```

TIME	SPACE
O(rounds * hand)	O(1)

WHY IT IS SLOW

It repeats the same logic without the classes; it is only here to check that the class version gives the same game.

OPTIMAL SOLUTION · A seedable deck and pure game rules

TIME	SPACE
O(rounds * hand)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Extend it to n players.

Replace the two counters with a dictionary keyed by player id. Each round every player draws a hand; the round goes to the player whose best card is strictly highest, and ties for the top give nobody a point. The champion needs a lead of at least 2 over the second best.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Infinite Deck Card Duel
#
# Design an InfiniteDeck that can be drawn from forever:
# each call to draw() returns a uniformly random integer
# card value in [1, 13] (cards are never exhausted). Make
# the deck seedable so games are reproducible. Then
# simulate a duel between two players. In each round, each
# player draws hand_size cards. The round winner is the
# player holding the single highest card that round; if
# both players' highest cards are equal, the round is a
# draw (nobody scores). Play num_rounds rounds and tally
# round wins per player. Finally, declare a champion: the
# player is champion only if they lead the other in round
# wins by at least 2; otherwise there is no champion.
# Implement: InfiniteDeck.draw() returning the next card.
# round_winner(hand1, hand2) -> 1 , 2 , or 0 (0 meaning
# draw), based on the highest card in each hand.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Extend it to n players.
#
# INTERVIEW NOTES
# Reproducible games need an injectable random source:
# wrap random.Random(seed) in a class so the same seed
# replays the same game. Keep the game logic pure
# (round_winner and decide_champion take plain values),
# which makes it testable without randomness.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class InfiniteDeck:
    # an endless deck: every draw is a random card from 1 to 13. The deck
    # owns its random generator, so a seed makes a game repeatable.
```

```
    def __init__(self, seed=None):
        self.rng = random.Random(seed)

    def draw(self):
        return self.rng.randint(1, 13)
```

```
def best_card(hand):
    best = hand[0]
    for card in hand:
        if card > best:
            best = card
    return best


# ---- main question: main functions ----
# The rules are plain functions of hands and win counts.
# A round: each player draws a hand; the player whose BEST card is higher
# wins the round; equal best cards give nobody a point.
# Complexity: Time O(hand size), Space O(1)
def round_winner(hand1, hand2):
    best1 = best_card(hand1)
    best2 = best_card(hand2)
    if best1 == best2:
        return 0
    if best1 > best2:
        return 1
    return 2


# The champion is the player who leads by at least 2 rounds, else nobody (0).
# Complexity: Time O(1), Space O(1)
def decide_champion(wins1, wins2):
    if wins1 - wins2 >= 2:
        return 1
    if wins2 - wins1 >= 2:
        return 2
    return 0


# Complexity: Time O(rounds * hand size), Space O(hand size)
def play_game(deck, hand_size=5, num_rounds=100):
    wins = {1: 0, 2: 0}
    for round_number in range(num_rounds):
        hand1 = []
```

```
        hand2 = []
        for i in range(hand_size):
            hand1.append(deck.draw())
        for i in range(hand_size):
            hand2.append(deck.draw())
        winner = round_winner(hand1, hand2)
        if winner != 0:
            wins[winner] += 1
    return decide_champion(wins[1], wins[2])


# ---- main question: tests ----
def test_main_question():
    assert round_winner([3, 7, 2], [7, 1, 9]) == 2
    assert round_winner([5, 5], [5, 4]) == 0
    assert decide_champion(51, 49) == 1
    assert decide_champion(50, 49) == 0
    assert decide_champion(10, 13) == 2
    random.seed(28)
    for seed in range(40):                          # the same seed gives the same game
        hand = random.randint(1, 4)
        rounds = random.randint(1, 12)
        assert play_game(InfiniteDeck(seed), hand, rounds) == play_game(InfiniteDeck(seed), hand, rounds)
```

```
# ============================================================
# FOLLOW-UP 1: Extend it to n players
# ============================================================
# ---- follow-up 1: function ----
# Answer: replace the two counters with a dictionary keyed by player. Each
# round every player draws a hand; the round goes to the player whose
# best card is strictly the highest, and a tie for the top gives nobody a
# point. The champion leads the second best by at least 2 rounds.
# Complexity: Time O(rounds * players * hand size), Space O(players)
def play_game_n(deck, players, hand_size=5, num_rounds=100):
    wins = {}
    for p in range(1, players + 1):
        wins[p] = 0
    for round_number in range(num_rounds):
        best = {}
        for p in wins:
            hand = []
            for i in range(hand_size):
                hand.append(deck.draw())
            best[p] = best_card(hand)
```

```
        top = max(best.values())
        leaders = []
        for p in best:
            if best[p] == top:
                leaders.append(p)
        if len(leaders) == 1:
            wins[leaders[0]] += 1
    counts = sorted(wins.values(), reverse=True)
    if len(counts) > 1 and counts[0] - counts[1] >= 2:
        for p in wins:
            if wins[p] == counts[0]:
                return p
    return 0


# ---- follow-up 1: tests ----
def test_follow_up_1():
    for seed in range(30):
        assert play_game_n(InfiniteDeck(seed), 2, 3, 30) in (0, 1, 2)
        assert play_game_n(InfiniteDeck(seed), 4, 3, 30) in (0, 1, 2, 3, 4)
    assert play_game_n(InfiniteDeck(1), 3, 3, 0) == 0                # no rounds: no champion


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 119 of 197 | topic 22 of 43: Simulate a machine: a class with state and one branch per kind of event | 5 of 5

Template fit: each car keeps floor, direction and stops; a request goes to the car with the lowest cost; step() moves every car one floor | * asked 409 days ago *

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 119 of 197

Design an elevator control system

POSTED 2025-09-06 | 409 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 281/288

ABOUT THIS QUESTION

Topic: Simulate a machine: a class with state and one branch per kind of event

Optimal answer from: written and tested here for this exact question

LeetCode match: No direct match (OOP / low-level design)

Posted on PracHub: 2025-09-06 (updated 2026-10-07)

Original: https://prachub.com/interview-questions/design-an-elevator-control-system-2

QUESTION

Design the control logic for a multi-elevator building. A Request(floor, direction) is a call from a floor (direction +1 for up, -1 for down). Each Elevator keeps its current floor, direction and a set of stops. cost(request) estimates how expensive it is for this car to take the request (the distance, plus a detour penalty when the car is heading away); the dispatcher gives the request to the cheapest car. step() moves the car one floor towards its stops and returns the floor it served, if any.

Function signature

```
class Elevator:
    def __init__(self, car_id, floor=0)
    def cost(self, req: Request) -> float
    def step(self) -> int | None
```

EXAMPLES

Example 1

Input	`e = Elevator(1, floor=0)`
	`e.stops.update({3, 5})`
	`[e.step() for _ in range(6)]`

Output	`[e.step() for _ in range(6)] -> [null, null, null, 3, null, 5]`
Why	`The car climbs one floor per step and stops at floors 3 and 5 (served when it is on the floor at the start of a step).`

QUESTION

Example 2

Input	`idle = Elevator(2, floor=7)`
	`busy = Elevator(3, floor=2)`
	`busy.direction = 1`
	`busy.stops.update({9})`
	`idle.cost(Request(4, -1))`
	`busy.cost(Request(4, 1))`
	`busy.cost(Request(4, -1))`

Output	`idle.cost(Request(4, -1)) -> 3`
	`busy.cost(Request(4, 1)) -> 2`
	`busy.cost(Request(4, -1)) -> 14`

Why	An idle car costs its distance (3). A car already going up that will pass floor 4 going the same way costs its distance (2); going the other way it must finish its run first, so it costs much more.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
requests = deque()  # arrival order
def run():
    while requests:
        req = requests.popleft()
        car = nearest_idle_car(req.floor)
        car.go_to(req.floor)
        car.go_to(req.destination)
```

TIME	SPACE
O(requests * cars)	O(requests)

WHY IT IS SLOW

Cars zig-zag across the building and requests wait behind far-away ones, so total travel and waiting are high. Serving stops in sweep order (SCAN) and assigning by cost cuts both.

OPTIMAL SOLUTION · Elevator (SCAN) per car and cost-based assignment of calls

TIME	SPACE
O(cars) per call, O(1) per car per step	O(stops)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Peak traffic (morning lobby rush, evening exodus).

Add modes: during the morning rush park idle cars in the lobby and let them go up with a full load; in the evening do the opposite. Zone cars (low floors, high floors) or use destination dispatch (passengers enter their floor first) to group riders by destination.

FOLLOW-UP 2 Fairness and starvation.

SCAN already guarantees every stop is reached within one sweep. For hall calls add an aging term to the cost so a long-waiting call wins over a closer new one.

FOLLOW-UP 3 Safety and fault tolerance.

Keep door and overload as explicit states; a car that fails is marked out of service and its stops are reassigned (the controller does this), and the controller should be restartable from persistent state of cars and pending calls.

FOLLOW-UP 4 Morning rush: idle cars should wait at the lobby.

When a car has no stops, give it a stop at the lobby (or a configured parking floor per car). Peak modes are just policies layered on the same step() and cost-based assignment; the elevator algorithm does not change.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Design an elevator control system
#
# Design the control logic for a multi-elevator building.
# A Request(floor, direction) is a call from a floor
# (direction +1 for up, -1 for down). Each Elevator keeps
# its current floor, direction and a set of stops.
# cost(request) estimates how expensive it is for this car
# to take the request (the distance, plus a detour penalty
# when the car is heading away); the dispatcher gives the
# request to the cheapest car. step() moves the car one
# floor towards its stops and returns the floor it served,
# if any.
#
# Signature:
#   class Elevator:
#       def __init__(self, car_id, floor=0)
#       def cost(self, req: Request) -> float
#       def step(self) -> int | None
#
# FOLLOW-UPS (answered below, in this order):
# 1. Peak traffic (morning lobby rush, evening exodus).
# 2. Fairness and starvation.
# 3. Safety and fault tolerance.
# 4. Morning rush: idle cars should wait at the lobby.
#
# INTERVIEW NOTES
# Separate three things: the data (Request), one car's
# behaviour (Elevator: floor, direction, stops,
# door/state) and the policy that decides which car takes
# a call (Controller). Say the scheduling rule out loud: a
# car keeps going in its direction serving stops, then
# reverses (SCAN/LOOK), which avoids the back-and-forth of
# first-come-first-served. Add a step() so the whole thing
# can be simulated and tested.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================
```

```
INF = float("inf")
```

```
# ====================================================
# MAIN QUESTION
# ====================================================
# ---- main question: helper functions ----
class Request:
    def __init__(self, floor, direction):
        self.floor = floor
        self.direction = direction            # +1 for a call going up, -1 for going down


# ---- main question: main function ----
# Idea: every car is a small state machine. It keeps a set of floors to
# stop at and a direction (SCAN, like a real elevator: keep going the same
# way while there are stops ahead, then reverse). step() is one tick: serve
# a stop on the current floor, then move one floor. cost(req) says how
# bad it would be to give this car a new call; a Controller (below) gives
# each call to the cheapest car.
# Complexity: Time step and cost are O(stops), Space O(stops)
class Elevator:
    def __init__(self, car_id, floor=0):
        self.id = car_id
        self.floor = floor
        self.direction = 0                    # 0 = idle, +1 = going up, -1 = going down
        self.stops = set()
        self.in_service = True

    def cost(self, req):
        if not self.in_service:
            return INF                        # a broken car never gets calls
        distance = abs(self.floor - req.floor)
        if self.direction == 0:
            return distance                   # idle: just the distance
        gap = req.floor - self.floor
        on_the_way = gap * self.direction >= 0        # the car is moving toward the floor ...
        if on_the_way and req.direction == self.direction:
            return distance                           # ... and the call goes the same way
        highest = max(self.stops)                     # otherwise the car finishes its sweep first
        lowest = min(self.stops)
        return distance + 2 * (highest - lowest + 1) + 10

    def step(self):
        served = None
        if self.floor in self.stops:
            self.stops.discard(self.floor)
            served = self.floor
```

```
        if len(self.stops) == 0:
            self.direction = 0
            return served
        stops_above = False
        stops_below = False
        for stop in self.stops:
            if stop > self.floor:
                stops_above = True
            if stop < self.floor:
                stops_below = True
        if (self.direction >= 0 and stops_above) or (self.direction < 0 and not stops_below):
            self.direction = 1                # keep the direction while there are stops ahead
        else:
            self.direction = -1
        self.floor += self.direction
```

```
        return served


# ---- main question: tests ----
def test_main_question():
    car = Elevator(1, floor=0)
    car.stops.update({3, 5})
    assert [car.step(), car.step(), car.step(), car.step(), car.step(), car.step()] == [None, None, None, 3
, None, 5]
    assert car.direction == 0                          # nothing left: idle
    idle = Elevator(2, floor=4)
    assert idle.cost(Request(7, 1)) == 3
    idle.in_service = False
    assert idle.cost(Request(7, 1)) == INF


# =========================================================
# FOLLOW-UP 1: Peak traffic (morning lobby rush, evening exodus)
# =========================================================


# =========================================================
# FOLLOW-UP 4: Morning rush: idle cars should wait at the lobby
# =========================================================
# ---- follow-up 1: function ----
# Answer: add modes. In the morning rush park idle cars in the lobby and
# let them go up with a full load; in the evening do the opposite (spread
# idle cars over the upper floors). The Controller assigns calls to the
# cheapest car, fails over a broken car (follow-up 3), and parks idle cars.
# Complexity: Time assign O(cars * stops), tick O(cars * stops), Space O(cars * stops)
class Controller:
```

```
    def __init__(self, cars, lobby=0):
        self.cars = cars
        self.lobby = lobby

    def assign(self, req):
        best_car = None
        best_cost = INF
        for car in self.cars:
            c = car.cost(req)
            if c < best_cost:
                best_cost = c
                best_car = car
        if best_car is None:
            return None                        # every car is out of service
        best_car.stops.add(req.floor)
        return best_car.id

    def tick(self):
        served = {}
        for car in self.cars:
            served[car.id] = car.step()
        return served

    def fail(self, car_id):
        # a broken car: take it out of service and hand its stops to the others
        for car in self.cars:
            if car.id == car_id:
                car.in_service = False
                orphans = car.stops
                car.stops = set()
                for floor in orphans:
                    self.assign(Request(floor, 0))
```

```
    def park_idle(self):
        # morning rush: idle cars wait in the lobby
        for car in self.cars:
            if car.in_service and len(car.stops) == 0:
                car.stops.add(self.lobby)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    near = Elevator(1, floor=2)
    far = Elevator(2, floor=9)
    controller = Controller([near, far])
```

```
    assert controller.assign(Request(3, 1)) == 1       # the nearer car takes the call
    assert 3 in near.stops
    far.stops.add(5)
    controller.fail(2)                                 # the far car breaks: its stop moves to the other ca
r
    assert 5 in near.stops and not far.in_service
    controller.park_idle()
    assert 0 not in far.stops                          # a broken car is not parked
    assert controller.assign(Request(1, 1)) == 1


# ========================================================
# FOLLOW-UP 2: Fairness and starvation
# ========================================================
# Answer: a call must not wait forever because new, cheaper calls keep
# arriving. Age the calls (the cost goes down the longer a call waits) or
# give every call a deadline after which it jumps the queue.


# ========================================================
# FOLLOW-UP 3: Safety and fault tolerance
# ========================================================
# Answer: take a failed car out of service and hand its stops to the other
# cars (Controller.fail), never move with the doors open, keep a hard
# limit on the load, and make the controller restart safely from the state
# of each car.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 23 OF 43

Top-K by frequency: Counter plus nsmallest

6 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

6
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Top-K by frequency: Counter plus nsmallest | 6 questions | tap a title to jump to it

READ FIRST: the code template for these 6 questions (tap here) >

M	120. Top-K Frequent in Large Logs	1Point3Acres * 2026-07-19 *
M	121. Rearrange Tasks With Cooldown	PracHub * 2026-06-03 *
M	122. Implement frequency + distance top-K queries	PracHub * 2026-02-12 *
M	123. Find top/bottom-k words in list or stream	PracHub * 2026-01-11 *
M	124. Solve chat, grid paths, and car rentals	PracHub * 2025-12-02 *
M	125. Top-K Chatters by Word Count	ColorStack * 2025-09-21 *

CODE TEMPLATE | GROUP 23 OF 43 | 6 QUESTIONS, ONE PATTERN

Top-K by frequency: Counter plus nsmallest

The one idea these questions share

Count with a Counter, then ask heapq.nsmallest for the k best using a key of (negative count, tie-break). That is the whole solution for almost every top-K question; only what you count and the tie-break change.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
import heapq
from collections import Counter


def top_k(items, k, key=lambda x: x):
    counts = Counter(key(x) for x in items)          # CHANGE 1: what you count
    return heapq.nsmallest(
        k, counts.items(), key=lambda kv: (-kv[1], kv[0]))   # CHANGE 2: the tie-break
```

2. Helper functions that usually get written next to it

```
def bottom_k(items, k):
    counts = Counter(items)
    return heapq.nsmallest(k, counts, key=lambda w: (counts[w], w))


def counts_of(items):
    return dict(Counter(items))
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
120	Top-K Frequent in Large Logs	count the first word of every line (the speaker); return the names of the k most frequent
121	Rearrange Tasks With Cooldown	only the biggest count matters: answer = max(len(tasks), (biggest - 1) * (n + 1) + how many tasks share the biggest count)
122	Implement frequency + distance top-K queries	count the point tuples; the tie-break is distance from the origin, then x, then y
123	Find top/bottom-k words in list or stream	the same Counter; run nsmallest once with -count and once with +count for the bottom-k
124	Solve chat, grid paths, and car rentals	the top-K part is the template as written; the other parts are a rolling-column path count and a greedy car assignment
125	Top-K Chatters by Word Count	count len(body.split()) per speaker instead of lines; ties by name

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 120 of 197 | topic 23 of 43: Top-K by frequency: Counter plus nsmallest | 1 of 6

Template fit: count the first word of every line (the speaker); return the names of the k most frequent | * asked 93 days ago * | frequency Medium

MEDIUM | IN APP LC 347 | 1Point3Acres | phone-screen, onsite-coding | 120 of 197

★ TOP 75: rank 29 in the Medium and Hard priority list

★ Top-K Frequent in Large Logs

ASKED 2026-07-19 | 93 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | ALL TIME 58.8 | 30 DAYS 41.1 | PRIORITY 25/288

ABOUT THIS QUESTION

Topic: Top-K by frequency: Counter plus nsmallest

Optimal answer from: your app's interview approach (LeetCode #347)

LeetCode match: LC 347 Top K Frequent Elements

Frequency: Medium | Last asked: 2026-07-19 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/top-k-frequent-from-logs

QUESTION

A chat log is a list of lines, each starting with the speaker's name ('A says hi'). Return the k speakers who appear most often; ties are broken alphabetically by name.

Function signature

def top_k_speakers(chat_log: list, k: int) -> list

EXAMPLES

Example 1	
Input	`['A says hi', 'B says hi', 'A says bye', 'C says hi', 'A says yo'], 2`
Output	`['A', 'B']`
Why	`A spoke 3 times, B and C once each; B comes before C alphabetically.`

Example 2	
Input	`['A says hi', 'B says hi', 'A says bye', 'C says hi', 'A says yo'], 3`
Output	`['A', 'B', 'C']`
Why	`Asking for 3 returns everyone, most frequent first.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(chat_log, k):
    names = [l.split()[0] for l in chat_log]
    counts = Counter(names)
    def rank(name):
        return (-counts[name], name)
    return sorted(counts, key=rank)[:k]
```

TIME	SPACE
O(n + d log d)	O(d)

WHY IT IS SLOW

BRUTE FORCE

Sorts all d distinct speakers, O(d log d), when only K are wanted; a size-K heap is O(d log k), and a stream needs a counter sketch instead of exact counts.

OPTIMAL SOLUTION · Count, then keep a size-K heap

TIME	SPACE
O(n + d log k)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you do it across many machines when the log does not fit on one?

Count per shard and send each shard's local top-K to a coordinator. That is not exact in general (a user can be 11th everywhere yet first overall), so for an exact answer use the threshold algorithm: gather the local top-K, compute a global lower bound, ask every shard for all users above bound / numShards, and merge. If an approximate answer is acceptable, merge count-min sketches (they add) and keep a heap of the K largest estimates.

FOLLOW-UP 2 What if the log arrives as an unbounded stream?

Use the Misra-Gries or Space-Saving summary with m counters. Any user with more than n / (m + 1) messages is guaranteed to be in the summary, and a second pass (or a count-min sketch for the estimate) can confirm the real counts. **Cost:** O(1) per message with O(m) memory for m counters (Misra-Gries), instead of O(d) memory for the exact counts.

FOLLOW-UP 3 Which approach would you choose and why?

A heap is the safe default: simple, O(n + d log k) and streaming-friendly. Quickselect wins when k is a large fraction of d and you need all K but not their order. Sorting is fine only when d is small. Say that out loud, with the complexities, because the interviewer is grading the trade-off discussion.

FOLLOW-UP 4 The log is a stream and memory is limited.

Misra-Gries keeps m counters; any user with more than n / (m + 1) messages is guaranteed to remain in the summary (checked above). A second pass, or a count-min sketch, then gives the real counts of the survivors.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Top-K Frequent in Large Logs
#
# A chat log is a list of lines, each starting with the
# speaker's name ('A says hi'). Return the k speakers who
# appear most often; ties are broken alphabetically by
# name.
#
# Signature:
#   def top_k_speakers(chat_log: list, k: int) -> list
#
# Examples:
# ['A says hi', 'B says hi', 'A says bye', 'C says hi', 'A
#     says yo'], 2  ->  ['A', 'B']
# ['A says hi', 'B says hi', 'A says bye', 'C says hi', 'A
#     says yo'], 3  ->  ['A', 'B', 'C']
```

```
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you do it across many machines when the log
# does not fit on one?
# 2. What if the log arrives as an unbounded stream?
# 3. Which approach would you choose and why?
# 4. The log is a stream and memory is limited.
#
# INTERVIEW NOTES
# Count the speakers first; a log line is just a way to
# read the user id. Then you only need the K largest
# counts: a heap of size K does that without sorting all
# users. For logs spread over many machines the local top
# K are not enough on their own, so say what extra
# information you send.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
```

```
def speaker(line):
    # "A says hi" -> "A"
    return line.split(" ", 1)[0]


def count_speakers(chat_log):
    counts = {}
    for line in chat_log:
        name = speaker(line)
        counts[name] = counts.get(name, 0) + 1
    return counts


def rank_of(pair):
    # pair = (name, count): more messages first, ties by name
    return (-pair[1], pair[0])


# ---- main question: main function ----
# Idea: take the speaker out of each line and count them in a dictionary.
# heapq.nsmallest keeps a heap of only k entries, so it never sorts all d
# distinct users: O(n + d log k). Ties are broken by name so the output is
# always the same.
# Complexity: Time O(n + u log k) for n lines and u speakers, Space O(u)
def top_k_speakers(chat_log, k):
    counts = count_speakers(chat_log)
    best = heapq.nsmallest(k, counts.items(), key=rank_of)
    names = []
    for name, count in best:
        names.append(name)
    return names


# ---- main question: tests ----
```

```
def test_main_question():
    log = ["A says hi", "B says hi", "A says bye", "C says hi", "A says yo"]
    assert top_k_speakers(log, 2) == ["A", "B"]
    assert top_k_speakers(log, 5) == ["A", "B", "C"]
    assert top_k_speakers([], 3) == []


# =========================================================
# FOLLOW-UP 1: How do you do it across many machines when the log does not fit on one?
# =========================================================
# ---- follow-up 1: function ----
```

```
# Answer: count on each machine (shard). Adding up the FULL counts of
# every shard gives the exact answer. Sending only each shard's local
# top-K is cheaper but can be wrong: a user who is second on every shard
# can be first overall.
# Complexity: Time O(total lines + u log k), Space O(u)
def merge_shards(shard_logs, k):
    total = {}
    for log in shard_logs:
        counts = count_speakers(log)
        for name in counts:
            total[name] = total.get(name, 0) + counts[name]
    best = heapq.nsmallest(k, total.items(), key=rank_of)
    names = []
    for name, count in best:
        names.append(name)
    return names


# ---- follow-up 1: tests ----
def test_follow_up_1():
    shard1 = ["A says x", "B says x"]
    shard2 = ["B says y", "B says z"]                    # two shards that each rank someone else first
    assert merge_shards([shard1, shard2], 1) == ["B"]


# =========================================================
# FOLLOW-UP 2: What if the log arrives as an unbounded stream?
# =========================================================


# =========================================================
# FOLLOW-UP 4: The log is a stream and memory is limited
# =========================================================
# ---- follow-up 2: function ----
# Answer: keep a Misra-Gries summary with m counters: a user with more
# than n / (m + 1) of the n messages is guaranteed to stay in it. When a
# new user arrives and there is no room, EVERY counter pays one. A second
# pass (or a count-min sketch) can then give the real counts of the survivors.
# Complexity: Time add is O(m) worst case (O(1) usually), Space O(m) counters
class MisraGries:
    def __init__(self, m):
        self.m = m
        self.counts = {}

    def add(self, user):
```

```
        if user in self.counts:
            self.counts[user] += 1
        elif len(self.counts) < self.m:
            self.counts[user] = 1
        else:
            for u in list(self.counts):          # no room: every counter pays one
                self.counts[u] -= 1
```

```
                if self.counts[u] == 0:
                    del self.counts[u]

    def candidates(self):
        return set(self.counts)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    summary = MisraGries(2)
    for user in "AAABBC":
        summary.add(user)
    assert "A" in summary.candidates()                  # A has 3 of 6 messages (more than 6 / 3), so it m
ust stay


# =============================================================
# FOLLOW-UP 3: Which approach would you choose and why?
# =============================================================
# Answer: for a log that fits in memory, count exactly and keep a size-k
# heap (simple, exact). For a huge log, use shards for an exact answer, or
# the Misra-Gries summary when memory is the limit and a small error is
# acceptable.


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Practice writing quickselect with the partition step (and explain how to recover from a bad pivot).
Be able to state amortized vs worst-case time for each approach.
Have a one-line answer for streaming: "hashmap counts + heap of size K maintained online".
Drill the distributed answer once: per-shard top-K, then K-way merge of the partial heaps.

STUDY PATH: question 121 of 197 | topic 23 of 43: Top-K by frequency: Counter plus nsmallest | 2 of 6

Template fit: only the biggest count matters: answer = max(len(tasks), (biggest - 1) * (n + 1) + how many tasks share the biggest count) | * asked 139 days ago *

MEDIUM | IN APP LC 621 | PracHub | Technical Screen | 121 of 197

Rearrange Tasks With Cooldown

POSTED 2026-06-03 | 139 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE ALL TIME 40.3 30 DAYS 0 PRIORITY 106/288

ABOUT THIS QUESTION

Topic: Top-K by frequency: Counter plus nsmallest

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #621 Task Scheduler

LeetCode match: LC 621 Task Scheduler (inferred from overview)

Posted on PracHub: 2026-06-03 (updated 2026-06-06)

Original: https://prachub.com/coding-questions/rearrange-tasks-with-cooldown

QUESTION

You are given a list of tasks (letters; the same letter means the same kind of task). Every task takes one unit of time, and two tasks of the same kind must have at least n other tasks or idle units between them (the cooldown). The CPU may stay idle. Return the least number of time units needed to finish all the tasks.

Function signature

def least_interval(tasks: list, n: int) -> int

EXAMPLES

Example 1	
Input	`['A', 'A', 'A', 'B', 'B', 'B'], 2`
Output	`8`
Why	`A B idle A B idle A B: three A's and three B's with a cooldown of 2 take 8 units.`

Example 2	
Input	`['A', 'A', 'A', 'B', 'B', 'B'], 0`
Output	`6`
Why	`No cooldown: just run the 6 tasks back to back.`

Example 3	
Input	`['A', 'B', 'C'], 5`
Output	`3`
Why	`All tasks differ, so no cooldown is ever needed.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(tasks, n):
    left = Counter(tasks)
    free_at = {}  # task -> time it is ready
    t = 0
    while any(left.values()):
        ready = [x for x in left if left[x]
                 and free_at.get(x, 0) <= t]
        if ready:
            task = max(ready, key=left.get)
            left[task] -= 1
            free_at[task] = t + n + 1
        t += 1  # idle if none ready
    return t
```

TIME	SPACE
O(T log k) where T is the answer	**O(k)**

WHY IT IS SLOW

It simulates every slot, including long idle stretches, O(T log k). The answer follows from the most frequent task: max(len(tasks), (maxCount - 1) * (n + 1) + number of tasks with that count).

OPTIMAL SOLUTION · Closed form from the most frequent task

TIME	SPACE
O(len(tasks))	**O(1)** for a fixed alphabet

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Print an actual schedule, not just the time.

Use the heap-with-cooldown-queue construction: run the available task with the most copies left, park it for n slots, and emit an idle marker when nothing is available. Or rearrange a string so equal letters are at least k apart.

FOLLOW-UP 2 Different cooldowns per task.

The closed form no longer applies. Simulate with a heap, storing for each task the time it becomes available; pick the ready task with the most remaining copies, and jump the clock to the earliest availability when nothing is ready.

FOLLOW-UP 3 Tasks have priorities or deadlines.

Then it is a scheduling problem; say what you optimise (lateness, completion time) and use a heap ordered by deadline for the ready tasks.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Rearrange Tasks With Cooldown
#
# You are given a list of tasks (letters; the same letter
```

```
# means the same kind of task). Every task takes one unit
# of time, and two tasks of the same kind must have at
# least n other tasks or idle units between them (the
# cooldown). The CPU may stay idle. Return the least
# number of time units needed to finish all the tasks.
#
# Signature:
#   def least_interval(tasks: list, n: int) -> int
#
# Examples:
# ['A', 'A', 'A', 'B', 'B', 'B'], 2  ->  8
# ['A', 'A', 'A', 'B', 'B', 'B'], 0  ->  6
#
# FOLLOW-UPS (answered below, in this order):
# 1. Print an actual schedule, not just the time.
# 2. Different cooldowns per task.
# 3. Tasks have priorities or deadlines.
#
# INTERVIEW NOTES
# The most frequent task is the bottleneck. Lay it out
# with gaps of n between its copies; every other task can
# fill gaps. Total length is the larger of 'that skeleton'
# and 'just the number of tasks' (when there are so many
# tasks the gaps are all filled and no idle time is
# needed). The skeleton has (maxcount - 1) full blocks of
# size n + 1 plus one last block holding every task tied
# for the maximum.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq
from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
def count_tasks(tasks):
    counts = {}
    for task in tasks:
        counts[task] = counts.get(task, 0) + 1
    return counts


# ---- main question: main function ----
# The same task must be at least n slots apart. How many time slots
# (including idle ones) are needed?
# Idea (closed form): let `most` be the highest frequency and `tied` the
# number of tasks that have that frequency. The most frequent task needs
# (most - 1) full frames of n + 1 slots plus one last row holding every
# tied task: (most - 1) * (n + 1) + tied. If there are so many different
# tasks that no idle slot is needed, the answer is just the number of tasks.
# Complexity: Time O(n), Space O(1) (at most 26 task types)
def least_interval(tasks, n):
    if not tasks:
        return 0
    counts = count_tasks(tasks)
    most = 0
```

```
    for count in counts.values():
        if count > most:
            most = count
    tied = 0
    for count in counts.values():
        if count == most:
            tied += 1
    frame_length = (most - 1) * (n + 1) + tied
    if frame_length > len(tasks):
        return frame_length
    return len(tasks)


# ---- main question: tests ----
def test_main_question():
    assert least_interval(list("AAABBB"), 2) == 8
    assert least_interval(list("AAABBB"), 0) == 6
    assert least_interval([], 3) == 0
    assert least_interval(list("ABCDEFG"), 2) == 7        # many different tasks: no idle time needed


# ========================================================
# FOLLOW-UP 1: Print an actual schedule, not just the time
```

```
# ========================================================
# ---- follow-up 1: function ----
# Answer: a heap plus a cooldown queue. At every time step run the
# available task with the most copies left (a max-heap: push negative
# counts), then park it for n steps; when nothing is available write
# "idle". Tasks come back to the heap when their cooldown is over.
# Complexity: Time O(N log T) for N slots and T task types, Space O(T)
def schedule(tasks, n):
    heap = []
    counts = count_tasks(tasks)
    for task in counts:
        heap.append((-counts[task], task))
    heapq.heapify(heap)
    cooling = deque()                           # (time it is ready again, minus copies left, task)
    plan = []
    now = 0
    while heap or cooling:
        while cooling and cooling[0][0] <= now:
            ready_time, left, task = cooling.popleft()
            heapq.heappush(heap, (left, task))
        if heap:
            left, task = heapq.heappop(heap)    # the task with the most copies left
            plan.append(task)
            if left + 1 < 0:
                cooling.append((now + n + 1, left + 1, task))
        else:
            plan.append("idle")
        now += 1
    return plan


# ---- follow-up 1: tests ----
def is_valid(plan, n):
    # equal tasks must be more than n slots apart
    last = {}
    for i in range(len(plan)):
        task = plan[i]
        if task == "idle":
            continue
        if task in last and i - last[task] <= n:
            return False
```

```
        last[task] = i
    return True
```

```
def test_follow_up_1():
    plan = schedule(list("AAABBB"), 2)
    assert len(plan) == 8 and is_valid(plan, 2)
    idle = 0
    for slot in plan:
        if slot == "idle":
            idle += 1
    assert idle == 2
    for tasks, n in ((list("AAAABBC"), 3), (list("AB"), 5)):
        plan = schedule(tasks, n)
        assert is_valid(plan, n)
        assert len(plan) == least_interval(tasks, n)


# ============================================================
# FOLLOW-UP 2: Different cooldowns per task
# ============================================================
# Answer: remember for each task when it may run again (its own cooldown)
# instead of one n; the heap-and-queue schedule works unchanged.


# ============================================================
# FOLLOW-UP 3: Tasks have priorities or deadlines
# ============================================================
# Answer: change what the heap orders by: (priority or deadline first,
# then the remaining copies). The cooldown queue stays the same.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(tasks: list[str], cooldown: int) -> str:
```

Sample call

```
solution(['A', 'A', 'A', 'B', 'B', 'C'], 2)
```

Test cases

Input	Expected
(['A', 'A', 'A', 'B', 'B', 'C'], 2)	'ABCAB_A'
(['A', 'A', 'A', 'B'], 2)	'AB_A__A'

ALSO REPORTED · 1 more

Design a task scheduler in Python. | Exponent

https://www.aced.io/questions/3504/design-task-scheduler-python

STUDY PATH: question 122 of 197 | topic 23 of 43: Top-K by frequency: Counter plus nsmallest | 3 of 6

Template fit: count the point tuples; the tie-break is distance from the origin, then x, then y | * asked 250 days ago *

MEDIUM | PracHub | Technical Screen | 122 of 197

★ TOP 75: rank 28 in the Medium and Hard priority list

★ Implement frequency + distance top-K queries

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TYPE **ALL TIME 58.8** **30 DAYS 41.1** **PRIORITY 24/288**

ABOUT THIS QUESTION

Topic: Top-K by frequency: Counter plus nsmallest

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #347 Top K Frequent Elements, LeetCode #973 K Closest Points to Origin

LeetCode match: LC 347 Top K Frequent Elements + LC 973 K Closest Points to Origin

Posted on PracHub: 2026-02-12 (updated 2026-04-28)

Original: https://prachub.com/coding-questions/implement-frequency-plus-distance-top-k-queries

QUESTION

You will implement solutions for two coding interview questions.

Question

1: Return the top-K points by frequency (with distance tie-break) You are given an array **`points`** of 2D integer points, where each point is represented as **`[x, y]`**. The array may contain duplicates (the same **`[x, y]`** can appear multiple times). Define:

- **`freq(p)`** = number of times point **p** appears in the array.
- **`dist(p)`** = squared Euclidean distance to the origin, **`x^2 + y^2`**. Return **k distinct points** sorted by the following priority:

1. Higher frequency first (**`freq`** descending)
2. If frequencies tie, smaller distance first (**`dist`** ascending)
3. If still tied, sort by **`x`** ascending, then **`y`** ascending

Input

- **`points`**: list of length **n**, each element **`[x, y]`**
- **`k`**: integer, **`1 <= k <= (# of distinct points)`**

Output

- A list of **k** distinct points following the ranking above.

Constraints (typical interview assumptions)

- **`1 <= n <= 2 * 10^5`**
- **`-10^4 <= x, y <= 10^4`** ---

Question

2: Time-based key-value store with “closest timestamp” lookup Design a data structure supporting the following operations:

- **`set(key, value, timestamp)`**: store the string **`value`** for string **`key`** at integer **`timestamp`**.
- Assume timestamps for the same **`key`** are inserted in **non-decreasing** order.
- **`getClosest(key, timestamp) -> value or ""`**:

QUESTION

- If **`key`** has no stored values, return **`""`**.
- Otherwise, return the **`value`** whose stored timestamp is **closest** to the query **`timestamp`**.
- If there is a tie (equal distance to two timestamps), return the value with the **smaller** timestamp.

Example

If for key **`"a"`** you stored timestamps **`[2, 10, 15]`** with values **`["x", "y", "z"]`**:

- **`getClosest("a", 11)`** returns **`"y"`** (10 is closest)
- **`getClosest("a", 14)`** returns **`"z"`** (15 is closest)
- **`getClosest("a", 12)`** returns **`"y"`** (10 and 15 are equally far; choose smaller timestamp)

What to discuss

- Time and space complexity of your approach - How you would improve a “naive” solution (e.g., linear scan) to a more optimal one

Constraints (typical interview assumptions)

- Up to **`2 * 10^5`** total operations - Keys and values are strings; timestamps are integers within typical 32-bit range

Part 1: Top-K Points by Frequency with Distance Tie-BreakGiven a list of 2D integer points, where duplicate points may appear, return the top **k** distinct points ranked by these rules: (1) higher frequency first, (2) if frequencies tie, smaller squared distance to the origin first, and (3) if still tied, smaller **x** first, then smaller **y**. The result must contain exactly **k** distinct points in ranked order.

Constraints

1 <= len(points) <= 2 * 10^5-10^4 <= x, y <= 10^41 <= k <= number of distinct points

EXAMPLES

```
Input: ([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)
Expected Output: [[2, 2], [1, 1]]
Explanation: Point [2,2] appears 3 times and [1,1] appears 2 times, so they are the top 2 distinct
    points.
Input: ([[1,2],[0,2],[1,2],[2,1],[0,2],[2,1],[3,0]], 3)
Expected Output: [[0, 2], [1, 2], [2, 1]]
Explanation: The points [0,2], [1,2], and [2,1] each appear twice. Their squared distances are 4, 5, and
    5, so [0,2] comes first. Between [1,2] and [2,1], x is smaller for [1,2].
```

Hints

First count how many times each point appears. After that, each distinct point only needs to be ranked once.Python can sort tuples directly, so think about building a sort key like **`(-frequency, distance, x, y)`**.

Part 2: Time-Based Key-Value Store with Closest Timestamp QueriesProcess a sequence of operations for a time-based key-value store. Each operation is either **`['set', key, value, timestamp]`** or **`['getClosest', key, timestamp]`**. For every **`getClosest`** operation, return the stored value whose timestamp is closest to the query tim

BRUTE FORCE · Q1: sort all distinct points by the full key

BRUTE FORCE · PSEUDO-CODE

```
def top_k_points(points, k):
    counts = Counter(map(tuple, points))
    def rank(p):
        x, y = p
        return (-counts[p], x * x + y * y, x, y)
    return sorted(counts, key=rank)[:k]
```

TIME	SPACE
O(d log d)	O(d)

BRUTE FORCE · Q1: sort all distinct points by the full key

WHY IT IS SLOW

Sorts all d distinct points, O(d log d); a size-K heap with the same key is O(d log k).

BRUTE FORCE · Q2: scan every timestamp of the key

BRUTE FORCE · PSEUDO-CODE

```
def get_closest(key, t):
    best = None
    for ts, value in store[key]:
        gap = abs(ts - t)
        if best is None or (gap, ts) < best[:2]:
            best = (gap, ts, value)
    return best[2]
```

TIME	SPACE
O(m) per query	**O(total entries)**

WHY IT IS SLOW

Linear in the number of stored timestamps per query. Timestamps arrive in order, so binary search on the sorted list finds the neighbours in O(log m).

OPTIMAL SOLUTION · Q1: heap with the full key

TIME	SPACE
O(n + d log k)	**O(d)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Q2: parallel sorted lists and bisect

TIME	SPACE
O(1) per set, O(log m) per getClosest	**O(total entries)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Q1 with points arriving over time.

Keep a running Counter and compute the top K on demand with a heap (O(d log k) per query), or maintain a sorted structure updated on every add if queries dominate.

FOLLOW-UP 2 Q2 with timestamps arriving out of order.

The append keeps the list sorted only for in-order input. For arbitrary order keep a sorted list and insert with bisect.insort (O(m) per insert) or a balanced tree (O(log m)); the query is unchanged.

FOLLOW-UP 3 **Q2 returns the closest value at or before the timestamp (LeetCode 981).**

Use bisect_right(timestamps, ts) - 1 and return that value or an empty string if the index is negative; there is no tie question.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement frequency + distance top-K
# queries
#
# You will implement solutions for two coding interview
# questions. ## Question 1: Return the top-K points by
# frequency (with distance tie-break) You are given an
# array points of 2D integer points, where each point is
# represented as [x, y]. The array may contain duplicates
# (the same [x, y] can appear multiple times). Define: -
# freq(p) = number of times point p appears in the array.
# - dist(p) = squared Euclidean distance to the origin,
# x^2 + y^2. Return **k distinct points** sorted by the
# following priority: 1. Higher frequency first (freq
# descending) 2. If frequencies tie, smaller distance
# first (dist ascending) 3. If still tied, sort by x
# ascending, then y ascending ### Input - points: list of
# length n, each element [x, y] - k: integer, 1 <= k <= (#
# of distinct points) ### Output - A list of k distinct
# points following the ranking above.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Q1 with points arriving over time.
# 2. Q2 with timestamps arriving out of order.
# 3. Q2 returns the closest value at or before the
# timestamp (LeetCode 981).
#
# INTERVIEW NOTES
# Q1 is a ranking: say the full sort key out loud, count
# with a hash map, and then you only need the K best
# (heap), not a full sort. Q2 keeps each key's timestamps
# in a sorted list, so the nearest one is next to the
# insertion point: binary search, then compare the two
# neighbours.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
```

```
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def count_points(points):
    # {(x, y): how many times the point appears}
    counts = {}
    for x, y in points:
        counts[(x, y)] = counts.get((x, y), 0) + 1
```

```
    return counts


def ranked_points(counts):
    # one row per point: (minus its count, squared distance from the origin, x, y).
    # A smaller row is better: more frequent first, then closer to the origin, then by x, y.
    # (Squared distances need no square root and are exact for whole numbers.)
    rows = []
    for (x, y), count in counts.items():
        rows.append((-count, x * x + y * y, x, y))
    return rows


# ---- main question: main functions ----
# Q1: the k best points: most frequent, then closest to the origin.
# Idea: count the points, make one ranking row per point, and keep a heap
# of the k smallest rows (heapq.nsmallest).
# Complexity: Time O(n + u log k) for n points and u distinct points, Space O(u)
def top_points(points, k):
    best = heapq.nsmallest(k, ranked_points(count_points(points)))
    result = []
    for minus_count, distance, x, y in best:
        result.append([x, y])
    return result


# Q2: a store of values by key and time.
# Complexity: Time O(log n), Space O(1)
def first_not_before(items, ts):
    # binary search: the first index whose timestamp is >= ts
    low = 0
    high = len(items)
    while low < high:
        middle = (low + high) // 2
        if items[middle][0] < ts:
            low = middle + 1
        else:
```

```
            high = middle
    return low


# Complexity: Time O(log n), Space O(1)
def first_after(items, ts):
    # binary search: the first index whose timestamp is > ts
    low = 0
    high = len(items)
    while low < high:
        middle = (low + high) // 2
        if items[middle][0] <= ts:
            low = middle + 1
        else:
            high = middle
    return low


# Complexity: Time set O(n) (a sorted insert), get O(log n), Space O(n)
class TimeStore:
    # set(key, value, ts) stores a value; get_closest(key, ts) returns the value
    # stored at the time closest to ts (the earlier time if two are equally
    # close). Each key keeps its (timestamp, value) pairs in order of time,
    # even when they arrive out of order (follow-up 2).
    def __init__(self):
        self.data = {}
```

```
    def set(self, key, value, ts):
        if key not in self.data:
            self.data[key] = []
        items = self.data[key]
        position = 0
        while position < len(items) and items[position] <= (ts, value):
            position += 1                           # insert so the list stays sorted
        items.insert(position, (ts, value))

    def get_closest(self, key, ts):
        items = self.data.get(key)
        if not items:
            return ""
        i = first_not_before(items, ts)           # the first time at or after ts; the one before it is i
- 1
        options = []
        if i > 0:
            options.append(items[i - 1])
        if i < len(items):
```

```
            options.append(items[i])
        best = options[0]
        for item in options:
            gap = item[0] - ts
            if gap < 0:
                gap = -gap
            best_gap = best[0] - ts
            if best_gap < 0:
                best_gap = -best_gap
            if gap < best_gap or (gap == best_gap and item[0] < best[0]):
                best = item
        return best[1]

    def get_at_or_before(self, key, ts):
        # LeetCode 981: the latest value stored at a time not after ts
        items = self.data.get(key, [])
        i = first_after(items, ts) - 1
        if i >= 0:
            return items[i][1]
        return ""


# ---- main question: tests ----
POINTS = [[1, 1], [1, 1], [2, 0], [0, 2], [2, 0], [3, 3]]


def test_main_question():
    assert top_points(POINTS, 2) == [[1, 1], [2, 0]]      # (1,1) and (2,0) both appear twice; (1,1) is clo
ser
    assert top_points(POINTS, 4) == [[1, 1], [2, 0], [0, 2], [3, 3]]
    assert top_points([[5, 5]], 1) == [[5, 5]]
    store = TimeStore()
    for ts, value in ((2, "x"), (10, "y"), (15, "z")):
        store.set("a", value, ts)
    assert store.get_closest("a", 11) == "y"
    assert store.get_closest("a", 14) == "z"
    assert store.get_closest("a", 12) == "y"              # a tie between 10 and 15 from 12.5 ... the earli
er time wins
    assert store.get_closest("b", 1) == ""
    assert store.get_closest("a", 100) == "z"
    assert store.get_closest("a", -5) == "x"


# ====================================================
# FOLLOW-UP 1: Q1 with points arriving over time
```

```
# ========================================================
# ---- follow-up 1: function ----
```

```
# Answer: keep the counts as points arrive, so nothing is recounted;
# compute the top K on demand with a heap: O(d log k) per query (or keep a
# sorted structure updated on every add if queries are far more common).
# Complexity: Time add O(1), top_k O(u log k), Space O(u)
class TopPointsStream:
    def __init__(self):
        self.counts = {}

    def add(self, x, y):
        self.counts[(x, y)] = self.counts.get((x, y), 0) + 1

    def top_k(self, k):
        best = heapq.nsmallest(k, ranked_points(self.counts))
        result = []
        for minus_count, distance, x, y in best:
            result.append([x, y])
        return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    live = TopPointsStream()
    for x, y in POINTS:
        live.add(x, y)
    assert live.top_k(2) == [[1, 1], [2, 0]]
    live.add(3, 3)
    live.add(3, 3)
    live.add(3, 3)
    assert live.top_k(1) == [[3, 3]]                    # (3, 3) is now the most frequent point


# ========================================================
# FOLLOW-UP 2: Q2 with timestamps arriving out of order
# ========================================================
# Answer: TimeStore.set inserts each pair at its sorted place, so late
# arrivals are fine (O(n) to insert into a list, O(log n) with a balanced tree).
# ---- follow-up 2: tests ----
def test_follow_up_2():
    store = TimeStore()
    for ts, value in ((2, "x"), (10, "y"), (15, "z")):
        store.set("a", value, ts)
    store.set("a", "late", 5)
    assert store.get_closest("a", 6) == "late"
```

```
# ========================================================
# FOLLOW-UP 3: Q2 returns the closest value at or before the timestamp (LeetCode 981)
# ========================================================
# Answer: get_at_or_before binary-searches the last stored time that is not
# after the asked time.
# ---- follow-up 3: tests ----
def test_follow_up_3():
    store = TimeStore()
    for ts, value in ((2, "x"), (10, "y"), (15, "z")):
        store.set("a", value, ts)
    assert store.get_at_or_before("a", 14) == "y"
    assert store.get_at_or_before("a", 1) == ""
    assert store.get_at_or_before("a", 15) == "z"


# ========================================================
# RUN EVERYTHING
```

```
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(points: list[list[int]], k: int) -> list[list[int]]:
```

Sample call

```
solution([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)
```

Test cases

Input	Expected
`([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)`	`[[2, 2], [1, 1]]`
`([[1,2],[0,2],[1,2],[2,1],[0,2],[2,1],[3,0]], 3)`	`[[0, 2], [1, 2], [2, 1]]`

STUDY PATH: question 123 of 197 | topic 23 of 43: Top-K by frequency: Counter plus nsmallest | 4 of 6

Template fit: the same Counter; run nsmallest once with -count and once with +count for the bottom-k | * asked 282 days ago *

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 123 of 197

Find top/bottom-k words in list or stream

POSTED 2026-01-11 | 282 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 32.6 | 30 DAYS 0 | PRIORITY 136/288

ABOUT THIS QUESTION

Topic: Top-K by frequency: Counter plus nsmallest

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #692 Top K Frequent Words

LeetCode match: LC 692 Top K Frequent Words (+ stream variant)

Posted on PracHub: 2026-01-11 (updated 2026-05-26)

Original: https://prachub.com/coding-questions/find-top-bottom-k-words-in-list-or-stream

QUESTION

You are given words (strings) either as a finite list or as an unbounded stream. 1)
List version: Given an array **`words`** and an integer **k**, return: - the **k** most frequent distinct words, and - the **k** least frequent distinct words. Specify how you break ties (e.g., lexicographically, or any order). 2)
Stream version (online): Design a data structure that supports:

- **`ingest(word)`**: process the next word in the stream - **`topK(k)`**: return the **k** most frequent distinct words seen so far - **`bottomK(k)`**: return the **k** least frequent distinct words seen so far Discuss time/space complexity and how you would implement these operations efficiently. 3)

Variant (discuss/design): Extend the stream design to support returning the **first k** and/or **last k** words that are currently **non-repeating** (i.e., words with frequency exactly 1) according to stream arrival order.

Part 1: Top and Bottom K Frequent Words in a ListGiven an array of strings words and a non-negative integer k, return two lists: the k most frequent distinct words and the k least frequent distinct words. Use deterministic tie-breaking: if two words have the same frequency, the lexicographically smaller word comes first. If k is larger than the number of distinct words, return all distinct words.
Constraints
0 <= len(words) <= 2 * 10^50 <= k <= 2 * 10^5Each word is a non-empty stringComparison is case-sensitive unless otherwise stated

EXAMPLES

```
Input: (['i', 'love', 'leetcode', 'i', 'love', 'coding'], 2)
Expected Output: [['i', 'love'], ['coding', 'leetcode']]
Explanation: `i` and `love` both appear twice, so lexicographical order gives `['i', 'love']`. `coding`
    and `leetcode` both appear once, so lexicographical order gives `['coding', 'leetcode']`.
Input: (['apple', 'banana', 'apple', 'cherry', 'banana', 'date'], 3)
Expected Output: [['apple', 'banana', 'cherry'], ['cherry', 'date', 'apple']]
```

Explanation: Frequencies are: apple=2, banana=2, cherry=1, date=1. Top 3 are ordered by frequency descending, then lexicographically: **`apple`**, **`banana`**, **`cherry`**. Bottom 3 are ordered by frequency ascending, then lexicographically: **`cherry`**, **`date`**, **`apple`**.

QUESTION

Hints

Count how many times each distinct word appears before deciding the answer.You can sort the distinct words twice: once by (-frequency, word) and once by (frequency, word).

Part 2: Online Top/Bottom K Words from a StreamYou are given a sequence of stream operations. Each operation is a 2-element list of strings: ['ingest', word], ['topK', str(k)], or ['bottomK', str(k)]. Process the operations in order. For each query, return the k most frequent or k least frequent distinct words seen so far. Ties are broken lexicographically ascending. If k is larger than the number of distinct words seen so far, return all distinct words.

Constraints

1 <= len(operations) <= 2 * 10^5Each operation is one of ['ingest', word], ['topK', str(k)], or ['bottomK', str(k)]0 <= k <= 2 * 10^5Queries may appear before any word is ingested

Examples

Input: [['ingest', 'apple'], ['ingest', 'banana'], ['ingest', 'apple'], ['topK', '2'], ['bottomK', '2']]

Expected Output: [['apple', 'banana'], ['banana', 'apple']]

Explanation: After the ingests, counts are apple=2 and banana=1. topK(2) returns the most frequent first, while bottomK(2) returns the least frequent first.

Input: [['topK', '3'], ['ingest', 'dog'], ['ingest', 'cat'], ['ingest', 'cat'], ['topK', '2'], ['bottomK', '2'], ['topK', '5']]

Expected Output: [[], ['cat', 'dog'], ['dog', 'cat'], ['cat', 'dog']]

Explanation: The first query happens before any words are seen, so it returns []. Later, cat has frequency 2 and dog has frequency 1.

Hints

Keep an exact frequency map for all words seen so far.A max-heap and a min-heap with lazy deletion let you support online updates without fully resorting all words

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(words, k):
    counts = Counter(words)
    def rank(w):
        return (-counts[w], w)
    ranked = sorted(counts, key=rank)
    most = ranked[:k]
    least = ranked[::-1][:k]
    return most, least
```

TIME	SPACE
O(d log d)	**O(d)**

WHY IT IS SLOW

Sorts all d distinct words, O(d log d). Two heaps of size K are O(d log k); a stream only needs the counts, and a Space-Saving sketch bounds memory for the top K.

OPTIMAL SOLUTION · Counter plus two heaps of size K

TIME	SPACE
O(n + d log k)	**O(d)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The stream is too big to count every distinct word.

Use the Space-Saving (or Misra-Gries) summary with m counters: a word that appears more than n / m times is guaranteed to be tracked, and its stored count is an upper bound. This works for the top K only; the least frequent words cannot be found this way.

FOLLOW-UP 2 topK must be O(k), not O(d log k), because it is called all the time.

Keep words in buckets by count (a linked list of count buckets, like the LFU cache structure); the most frequent words are in the last bucket. An increment moves a word one bucket up in O(1).

FOLLOW-UP 3 Case and punctuation.

Normalise before counting (lowercase, strip punctuation) and say so; the counting and ranking code is unchanged.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find top/bottom-k words in list or stream
#
# You are given words (strings) either as a finite list or
# as an unbounded stream. 1) **List version**: Given an
# array words and an integer k, return: - the k most
# frequent distinct words, and - the k least frequent
# distinct words. Specify how you break ties (e.g.,
# lexicographically, or any order). 2) **Stream version
# (online)**: Design a data structure that supports: -
# ingest(word): process the next word in the stream -
# topK(k): return the k most frequent distinct words seen
# so far - bottomK(k): return the k least frequent
# distinct words seen so far Discuss time/space complexity
# and how you would implement these operations
# efficiently. 3) **Variant (discuss/design)**: Extend the
# stream design to support returning the **first k**
# and/or **last k** words that are currently **non-
# repeating** (i.e., words with frequency exactly 1)
# according to stream arrival order.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The stream is too big to count every distinct word.
# 2. topK must be O(k), not O(d log k), because it is
# called all the time.
# 3. Case and punctuation.
#
# INTERVIEW NOTES
# Count first; then you only need the K best and K worst,
# which a heap does without sorting everything. For a
# stream, ingesting is just a counter increment, and the
# answer is computed from the counts on demand. If memory
# is bounded, only heavy hitters (top-K) can be
# approximated; the rarest words cannot.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================
```

```
import heapq


# =====================================================
```

```
# MAIN QUESTION
# =====================================================
# ---- main question: helper functions ----
def count_words(words):
    counts = {}
    for word in words:
        counts[word] = counts.get(word, 0) + 1
    return counts


def names_of(pairs):
    names = []
    for first, second in pairs:
        names.append(second)
    return names


# ---- main question: main function ----
# Idea: one pass counts the words. A heap of size k then picks the k most
# frequent (rows (-count, word): the smallest rows win) and the k least
# frequent (rows (count, word)), each in O(d log k) without sorting
# everything. Ties are broken alphabetically.
# Complexity: Time O(n + u log k), Space O(u)
def top_bottom(words, k):
    counts = count_words(words)
    top_rows = []
    bottom_rows = []
    for word in counts:
        top_rows.append((-counts[word], word))
        bottom_rows.append((counts[word], word))
    top = names_of(heapq.nsmallest(k, top_rows))
    bottom = names_of(heapq.nsmallest(k, bottom_rows))
    return top, bottom


# ---- main question: tests ----
def test_main_question():
    words = "a b a c b a d d".split()
    top, bottom = top_bottom(words, 2)
    assert top == ["a", "b"] and bottom == ["c", "b"]


# =====================================================
# FOLLOW-UP 1: The stream is too big to count every distinct word
# =====================================================
```

```
# ---- follow-up 1: function ----
# Answer: the Space-Saving (or Misra-Gries) summary with m counters. A word
# that appears more than n / m times is guaranteed to be tracked, and its
# stored count is an upper bound. When a new word arrives and there is no
# room, it replaces the word with the smallest counter and inherits that
# count + 1.
# Complexity: Time add O(m), top_k O(m log m), Space O(m)
class SpaceSaving:
    def __init__(self, m):
        self.m = m
        self.counts = {}

    def add(self, word):
        if word in self.counts:
```

```
            self.counts[word] += 1
        elif len(self.counts) < self.m:
            self.counts[word] = 1
        else:
            smallest = None
            for tracked in self.counts:
                if smallest is None or self.counts[tracked] < self.counts[smallest]:
                    smallest = tracked
            self.counts[word] = self.counts.pop(smallest) + 1

    def top_k(self, k):
        rows = []
        for word in self.counts:
            rows.append((-self.counts[word], word))
        rows.sort()
        return names_of(rows[:k])


# ---- follow-up 1: tests ----
def test_follow_up_1():
    summary = SpaceSaving(2)
    for word in ["x", "x", "x", "y", "z", "x", "x"]:
        summary.add(word)
    assert summary.top_k(1) == ["x"]


# =====================================================
# FOLLOW-UP 2: topK must be O(k), not O(d log k), because it is called all the time
# =====================================================
# ---- follow-up 2: function ----
# Answer: keep the words in BUCKETS by count (like the LFU cache): the most
```

```
# frequent words are in the highest bucket. Adding a word moves it up one
# bucket in O(1); top_k only reads the highest buckets.
# Complexity: Time add O(1), top_k and bottom_k O(k log k + number of buckets), Space O(u)
class FreqBuckets:
    def __init__(self):
        self.count = {}                         # word -> its count
        self.bucket = {}                        # count -> the words with exactly that count
        self.high = 0                           # the highest count so far

    def add(self, word):
        c = self.count.get(word, 0)
        if c > 0:
            del self.bucket[c][word]            # leave the old bucket
            if not self.bucket[c]:
                del self.bucket[c]
        self.count[word] = c + 1
        if c + 1 not in self.bucket:
            self.bucket[c + 1] = {}
        self.bucket[c + 1][word] = True
        if c + 1 > self.high:
            self.high = c + 1

    def top_k(self, k):
        result = []
        level = self.high
        while level > 0 and len(result) < k:
            if level in self.bucket:
                result += sorted(self.bucket[level])   # inside one bucket: alphabetical
            level -= 1
        return result[:k]

    def bottom_k(self, k):
        result = []
```

```
        for level in sorted(self.bucket):
            result += sorted(self.bucket[level])
            if len(result) >= k:
                break
        return result[:k]


# ---- follow-up 2: tests ----
def test_follow_up_2():
    buckets = FreqBuckets()
    for word in "a b a c b a d d".split():
        buckets.add(word)
```

```
    assert buckets.top_k(2) == ["a", "b"]
    assert buckets.bottom_k(2) == ["c", "b"]
    assert buckets.top_k(10) == ["a", "b", "d", "c"]


# ============================================================
# FOLLOW-UP 3: Case and punctuation
# ============================================================
# ---- follow-up 3: function ----
# Answer: clean the text before counting (lowercase, remove punctuation)
# and say that you did; the counting and ranking code does not change.
# Complexity: Time O(length of the text), Space O(length of the text)
def normalise(text):
    cleaned = ""
    for ch in text.lower():
        if ch.isalnum():
            cleaned += ch
    return cleaned


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert normalise("Hello, World!") == "helloworld"


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], k: int) -> list[list[str]]:
```

Sample call

```
solution(['i', 'love', 'leetcode', 'i', 'love', 'coding'], 2)
```

Test cases

Input	Expected
`(['i', 'love', 'leetcode', 'i', 'love', 'coding'], 2)`	`[['i', 'love'], ['coding', 'leetcode']]`
`(['apple', 'banana', 'apple', 'cherry', 'banana', 'date'], 3)`	`[['apple', 'banana', 'cherry'], ['cherry', 'date', 'apple']]`

STUDY PATH: question 124 of 197 | topic 23 of 43: Top-K by frequency: Counter plus nsmallest | 5 of 6

Template fit: the top-K part is the template as written; the other parts are a rolling-column path count and a greedy car assignment | * asked 322 days ago *

MEDIUM | PracHub | Technical Screen | 124 of 197

★ TOP 75: rank 70 in the Medium and Hard priority list

★ Solve chat, grid paths, and car rentals

POSTED 2025-12-02 | 322 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 58 | 30 DAYS 0 | PRIORITY 50/288

ABOUT THIS QUESTION

Topic: Top-K by frequency: Counter plus nsmallest

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #692 Top K Frequent Words, LeetCode #62 Unique Paths, LeetCode #253 Meeting Rooms II

LeetCode match: LC 692 Top K Frequent Words + LC 62/63 Unique Paths + LC 253

Posted on PracHub: 2025-12-02 (updated 2026-05-02)

Original: https://prachub.com/interview-questions/solve-chat-grid-paths-and-car-rentals

QUESTION

You are given three independent programming tasks. For each, describe and implement an efficient algorithm, and be prepared to analyze its time and space complexity. ---

Q1.

Find top-K most active chat users You are given a log of chat messages from a messaging application.

Input:

- An array **`messages`** of length **`n`**, where each element **`messages[i]`** is the **`user_id`** (string or integer) of the sender of the i-th message.
- An integer **`k`** (1 <= k <= number of distinct users).

Output:

- Return a list of **`k`** user IDs corresponding to the users who have sent the most messages.

Details & requirements:

- The "activity" of a user is defined as the number of messages they have sent (i.e., their frequency in **`messages`**).
- If there are fewer than **`k`** distinct users, return all distinct users.
- If multiple users have the same frequency, you may break ties arbitrarily, or (for determinism) sort tied users by **`user_id`** in ascending order.
- Aim for an algorithm that is more efficient than sorting all users by count when **`k`** is much smaller than the number of distinct users. Design and implement a function, for example:
- **`List<UserId> topKActiveUsers(List<UserId> messages, int k)`** Also explain:
- At least two different approaches (e.g., based on sorting, heaps, or selection algorithms).
- The time and space complexity of each approach. ---

Q2.

Count paths in a 2D grid with constrained moves You have a 2D grid with **`m`** rows and **`n`** columns. Index rows from top to bottom as **`0..m-1`** and columns from left to right as **`0..n-1`**.

- The **start** cell is the bottom-left corner: **`(m-1, 0)`**.

QUESTION

- The **end** cell is the bottom-right corner: **`(m-1, n-1)`**. From any cell **`(r, c)`**, you may move **only** to the next column to the right, in one of up to three ways (if within bounds):
- Right-up: **`(r-1, c+1)`**
- Right: **`(r, c+1)`**
- Right-down: **`(r+1, c+1)`** You cannot move left or stay in the same column. All cells are passable (no obstacles).

Task:

- Given integers **`m`** and **`n`**, compute the total number of distinct paths from the start cell **`(m-1, 0)`** to the end cell **`(m-1, n-1)`** using only the allowed moves.

Requirements:

- Implement a function such as **`long countPaths(int m, int n)`**.
- The result may be large; assume it fits in a 64-bit integer, or, if you prefer, you may return the count modulo a large prime (e.g., 1,000,000,007) — specify which convention you are using.
- First design a straightforward dynamic programming solution using O(m·n) time and O(m·n) space.
- Then describe how to optimize the space usage (e.g., to O(m)) while keeping the same time complexity, and explain why the optimization is correct. ---

Q3.

Minimum cars needed for rentals and assignment You work at a car rental company. You are given a list of rental bookings, each with a pickup and return time.

Input:

- An array of **`R`** rental records. Each record has:
- **`rental_id`**: a unique identifier (e.g., integer or string).
- **`pickup_time`**: an integer start time.
- **`return_time`**: an integer end time, with **`pickup_time < return_time`**. Assume times are on a single timeline (e.g., minutes from some epoch). A car can be reassigned immediately after it is returned, i.e., if one rental ends at time **`t`** and another starts at time **`t`**, the same car can serve both.

Tasks: 1.

Capacity: Compute the **minimum number of cars** required so that all rentals can be fulfilled without time conflicts on any single car. 2.

Assignment: Produce one valid assignment of rentals to cars that uses exactly that minimum number of cars.

Output:

- An integer **`C`** = minimal number of cars needed.
- A mapping from each **`rental_id`** to a **`car_id`** in the range **`1..C`**, such that no two rentals assigned to the same **`car_id`** overlap in time.

Requirements & clarifications:

- Overlap definition: two rentals with intervals **`[s1, e1)`** and **`[s2, e2)`** overlap if their time intervals intersect with non-empty duration. Using half-open intervals, if **`e1 == s2`** they do **not** over

EXAMPLES

Example 1	
Input	`['ann', 'bob', 'ann', 'cy', 'bob', 'ann'], 2`
Output	`['ann', 'bob']`
Why	`Task 1: ann sent 3 messages, bob 2, cy 1; the top 2 are ann and bob.`

QUESTION

Example 2	
Input	`3, 3`
Output	`2`
Why	`Task 2: paths over 3 columns of a 3-row grid that start and end in the bottom row, moving up-right, right or down-right.`

Example 3	
Input	`[(1, 0, 5), (2, 1, 4), (3, 5, 8)]`
Output	`(2, {1: 0, 2: 1, 3: 1})`
Why	`Task 3: rentals are (id, pickup, return). Rental 3 starts at 5, after rental 2 returned its car at 4, so it reuses that car: 2 cars in total.`

BRUTE FORCE · Q1: sort all users by (count, user id)

BRUTE FORCE · PSEUDO-CODE

```
def top_k_users(log, k):
    counts = Counter(msg.user for msg in log)
    def rank(u):
        return (-counts[u], u)
    return sorted(counts, key=rank)[:k]
```

TIME	SPACE
O(d log d)	O(d)

WHY IT IS SLOW

Sorting all d distinct users costs O(d log d) even though only K are needed; a heap of size K costs O(d log k).

BRUTE FORCE · Q2: plain recursion over the moves

BRUTE FORCE · PSEUDO-CODE

```
def ways(r, c):
    if c == n - 1:
        return 1 if r == n - 1 else 0
    total = 0
    for dr in (-1, 0, 1):
        if 0 <= r + dr < n:
            total += ways(r + dr, c + 1)
    return total
```

TIME	SPACE
O(3^n)	O(n)

WHY IT IS SLOW

Three calls per cell and the same cell is reached by many paths, so the call tree has about 3^n nodes. One column at a time with a rolling array is O(m * n).

BRUTE FORCE · Q3: count the overlap at every time and assign greedily without a heap

BRUTE FORCE · PSEUDO-CODE

```
def cars_needed(rentals):
    free_at = []  # per car
    for start, end in sorted(rentals):
        for i, f in enumerate(free_at):
            if f <= start:
                free_at[i] = end  # reuse
                break
        else:
            free_at.append(end)  # new car
    return len(free_at)
```

TIME	SPACE
O(R^2)	O(R)

WHY IT IS SLOW

Each rental scans every car, so the cost is O(R * cars), up to O(R^2). A heap of free times finds the earliest-free car in O(log R).

OPTIMAL SOLUTION · Q1: top-K active users with a size-K heap

TIME	SPACE
O(n + d log k) for d distinct users	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Q2: count constrained paths with a rolling column

TIME	SPACE
O(m * n)	O(m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Q3: minimum cars and one valid assignment

TIME	SPACE
O(R log R)	O(R)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Q1: what if the log is a stream and memory is limited?

Use a heavy-hitters sketch: Misra-Gries keeps k counters and is exact for any user with more than n / (k + 1) messages; a count-min sketch plus a small heap estimates the top K in O(1) memory per update. For exact answers across machines, count per shard, send each shard's local top-K plus a threshold, and merge.

FOLLOW-UP 2 Q2: what changes if there are obstacles or the path may also go straight up and down?

Obstacles just force ways[r] = 0 for blocked cells in the column being built. Allowing vertical moves inside a column makes each column a chain, so the transition becomes a linear recurrence solved by a prefix pass, still O(m * n). For huge n, use matrix exponentiation of the m x m transition matrix: O(m^3 log n).

FOLLOW-UP 3 Q3: how do you answer 'is one more rental possible' online?

Keep the busy heap and the free-car count between calls. A new rental fits without a new car if any car's return time is <= its pickup, which is a heap peek after popping everything that returned before the pickup. That is O(log R) per query.

FOLLOW-UP 4 Q2 with obstacles.

A blocked cell contributes zero ways when its column is built; everything else is unchanged. O(m * n).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Solve chat, grid paths, and car rentals
#
# You are given three independent programming tasks. For
# each, describe and implement an efficient algorithm, and
# be prepared to analyze its time and space complexity.
# --- ### Q1. Find top-K most active chat users You are
# given a log of chat messages from a messaging
# application. **Input**: - An array messages of length n,
# where each element messages[i] is the user_id (string or
# integer) of the sender of the i-th message. - An integer
# k (1 <= k <= number of distinct users). **Output**: -
# Return a list of k user IDs corresponding to the users
# who have sent the most messages. **Details &
# requirements**: - The "activity" of a user is defined as
# the number of messages they have sent (i.e., their
# frequency in messages). - If there are fewer than k
# distinct users, return all distinct users.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Q1: what if the log is a stream and memory is
# limited?
# 2. Q2: what changes if there are obstacles or the path
# may also go straight up and down?
# 3. Q3: how do you answer 'is one more rental possible'
# online?
# 4. Q2 with obstacles.
#
# INTERVIEW NOTES
# Three unrelated tasks, so say so and give each a one-
# line plan. Chat: count, then take the top K with a heap.
# Grid: each column depends only on the previous one, so
# keep one column. Cars: the minimum number of cars is the
# largest number of rentals that overlap at once, which a
# heap of return times tracks while also giving an
# assignment.

# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
```

```
# ========================================================

import heapq
```

```
# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def count_items(items):
    # how many times each item appears: {item: count}
    counts = {}
    for item in items:
        counts[item] = counts.get(item, 0) + 1
    return counts


def rank_of(pair):
    # pair = (user, count): more messages first, ties by the smaller user id
    return (-pair[1], pair[0])


# ---- main question: main functions ----
# Q1: the k most active users in a chat log.
# Idea: count the messages of every user in a dictionary, then keep a heap
# of only the best k (heapq.nsmallest with a ranking key). That beats
# sorting all d users when k is small: O(n + d log k). Ties are broken by
# the smaller user id so the answer is always the same.
# Complexity: Time O(n + u log k) for n messages and u users, Space O(u)
def top_k_users(messages, k):
    counts = count_items(messages)
    best = heapq.nsmallest(k, counts.items(), key=rank_of)
    users = []
    for user, count in best:
        users.append(user)
    return users


# the second approach for Q1: sort every user, O(d log d)
# Complexity: Time O(n + u log u), Space O(u)
def top_k_users_sorted(messages, k):
    counts = count_items(messages)
    pairs = []
    for user in counts:
        pairs.append((-counts[user], user))
    pairs.sort()
    users = []
    for minus_count, user in pairs[:k]:
        users.append(user)
```

```
    return users


# Q2: count the paths through a grid with m rows and n columns. Start
# at the bottom-left cell (m - 1, 0), end at the bottom-right cell
# (m - 1, n - 1); each move goes right, right-and-up or right-and-down.
# Cells in `blocked` cannot be used (follow-up 4); mod keeps the numbers small.
# Idea: ways[r] = the number of paths in row r of the current column. One
# column is enough: O(m) memory.
# Complexity: Time O(m * n), Space O(m)
def count_paths(m, n, blocked=(), mod=None):
    wall = set(blocked)
    ways = [0] * m
    ways[m - 1] = 1
    for c in range(1, n):
```

```
        following = [0] * m
        for r in range(m):
            if (r, c) in wall:
                continue                          # a blocked cell contributes no paths
            total = ways[r]
            if r > 0:
                total += ways[r - 1]
            if r + 1 < m:
                total += ways[r + 1]
            if mod:
                total = total % mod
            following[r] = total
        ways = following
    return ways[m - 1]


# Q3: rentals are (pickup, return) times. How many cars are needed?
# Idea: sort the rentals by pickup time and keep a min-heap of the return
# times of the cars that are out. If the earliest car is back by the
# pickup, reuse it; otherwise a new car is needed.
# Complexity: Time O(n log n), Space O(n)
def min_cars(rentals):
    out = []
    for pickup, back in sorted(rentals):
        if out and out[0] <= pickup:
            heapq.heapreplace(out, back)          # a car came back in time: reuse it
        else:
            heapq.heappush(out, back)
    return len(out)
```

```
# ---- main question: tests ----
def test_main_question():
    log = ["a", "b", "a", "c", "a", "b"]
    assert top_k_users(log, 2) == ["a", "b"]
    assert top_k_users_sorted(log, 5) == ["a", "b", "c"]
    assert count_paths(3, 3) == 2                          # a 3 x 3 grid has 2 paths
    assert count_paths(1, 5) == 1
    assert count_paths(3, 1) == 1
    assert count_paths(4, 5) == 9
    assert min_cars([(1, 4), (2, 5), (4, 6), (5, 8)]) == 2
    assert min_cars([]) == 0


# ======================================================
# FOLLOW-UP 1: Q1: what if the log is a stream and memory is limited?
# ======================================================
# Answer: keep an approximate summary with a fixed number of counters
# (Misra-Gries or Space-Saving: see the "Top-K Frequent in Large Logs"
# question) and confirm the real counts of the survivors in a second pass.


# ======================================================
# FOLLOW-UP 2: Q2: what changes if there are obstacles or the path may also go straight up and down?
# ======================================================


# ======================================================
# FOLLOW-UP 4: Q2 with obstacles
# ======================================================
# Answer: count_paths already takes `blocked` cells (they get 0 paths). If
# a path may also go straight up or down inside a column, the cells of one
# column depend on each other, so count with one more dimension: how many
# vertical moves are still allowed in this column.
# ---- follow-up 2: tests ----
```

```
def test_follow_up_2():
    assert count_paths(3, 3, blocked=[(2, 1)]) == 1       # an obstacle in the middle of the bottom row
    modulus = 10 ** 9 + 7
    assert count_paths(5, 30, mod=modulus) < modulus


# ============================================================
# FOLLOW-UP 3: Q3: how do you answer "is one more rental possible" online?
# ============================================================
# ---- follow-up 3: function ----
```

```
# Answer: keep the heap of return times between calls. A new rental fits
# without a new car when some car has returned by its pickup time: take out
# every return time that is not after the pickup, then see if there is room.
# Complexity: Time rent is O(log n) amortized, Space O(number of cars out)
class RentalDesk:
    def __init__(self, fleet):
        self.fleet = fleet                      # how many cars exist
        self.out = []                           # return times of the cars that are out

    def rent(self, pickup, back):
        while self.out and self.out[0] <= pickup:
            heapq.heappop(self.out)             # that car is back
        if len(self.out) >= self.fleet:
            return False
        heapq.heappush(self.out, back)
        return True


# ---- follow-up 3: tests ----
def test_follow_up_3():
    desk = RentalDesk(1)
    assert desk.rent(1, 4) and not desk.rent(2, 3)
    assert desk.rent(4, 6)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FULL QUESTION TEXT FROM THE SITE

You are given three independent programming tasks. For each, describe and implement an efficient algorithm, and be prepared to analyze its time and space complexity.

Q1. Find top-K most active chat users

You are given a log of chat messages from a messaging application.

Input:

An array messages of length n , where each element messages[i] is the user_id (string or integer) of the sender of the i-th message.
An integer k (1 <= k <= number of distinct users).

Output:

FULL QUESTION TEXT FROM THE SITE

Return a list of k user IDs corresponding to the users who have sent the most messages.

Details & requirements:

The "activity" of a user is defined as the number of messages they have sent (i.e., their frequency in messages).
If there are fewer than k distinct users, return all distinct users.
If multiple users have the same frequency, you may break ties arbitrarily, or (for determinism) sort tied users by user_id in ascending order.
Aim for an algorithm that is more efficient than sorting all users by count when k is much smaller than the number of distinct users.

Design and implement a function, for example:

List<UserId> topKActiveUsers(List<UserId> messages, int k)

Also explain:

At least two different approaches (e.g., based on sorting, heaps, or selection algorithms).
The time and space complexity of each approach.
Q2. Count paths in a 2D grid with constrained moves

You have a 2D grid with m rows and n columns. Index rows from top to bottom as 0..m-1 and columns from left to right as 0..n-1.

The start cell is the bottom-left corner: (m-1, 0) .
The end cell is the bottom-right corner: (m-1, n-1) .

From any cell (r, c), you may move only to the next column to the right, in one of up to three ways (if within bounds):

Right-up: (r-1, c+1)
Right: (r, c+1)
Right-down: (r+1, c+1)

You cannot move left or stay in the same column. All cells are passable (no obstacles).

Task:

Given integers m and n , compute the total number of distinct paths from the start cell (m-1, 0) to the end cell (m-1, n-1) using only the allowed moves.

Requirements:

Implement a function such as long countPaths(int m, int n) .
The result may be large; assume it fits in a 64-bit integer, or, if you prefer, you may return the count modulo a large prime (e.g., 1,000,000,007) — specify which convention you are using.
First design a straightforward dynamic programming solution using O(m·n) time and O(m·n) space.
Then describe how to optimize the space usage (e.g., to O(m)) while keeping the same time complexity, and explain why the optimization is correct.
Q3. Minimum cars needed for rentals and assignment

You work at a car rental company. You are given a list of rental bookings, each with a pickup and return time.

Input:

An array of R rental records. Each record has:
rental_id : a unique identifier (e.g., integer or string).
pickup_time : an integer start time.
return_time : an integer end time, with pickup_time < return_time .

Assume times are on a single timeline (e.g., minutes from some epoch). A car can be reassigned immediately after it is returned, i.e., if one rental ends at time t and another starts at time t, the same car can serve both.

Tasks:

FULL QUESTION TEXT FROM THE SITE

Capacity : Compute the minimum number of cars required so that all rentals can be fulfilled without time conflicts on any single car.
Assignment : Produce one valid assignment of rentals to cars that uses exactly that minimum number of cars.

Output:

An integer C = minimal number of cars needed.
A mapping from each rental_id to a car_id in the range 1..C , such that no two rentals assigned to the same car_id overlap in time.

Requirements & clarifications:

Overlap definition: two rentals with intervals [s1, e1) and [s2, e2) overlap if their time intervals intersect with non-empty duration. Using half-open intervals, if e1 == s2 they do not overlap.
Aim for an algorithm that is efficient for up to, say, 10^5 rental records.
Discuss a naive approach and then an optimized approach (for example, using a min-heap/priority queue keyed by the earliest return time to track currently in-use cars).
Analyze the time and space complexity of your final solution.

STUDY PATH: question 125 of 197 | topic 23 of 43: Top-K by frequency: Counter plus nsmallest | 6 of 6

Template fit: count len(body.split()) per speaker instead of lines; ties by name | * asked 394 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 125 of 197

Top-K Chatters by Word Count

ASKED 2025-09-21 | 394 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 32.6 | 30 DAYS 0 | PRIORITY 138/288

ABOUT THIS QUESTION

Topic: Top-K by frequency: Counter plus nsmallest

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #692 Top K Frequent Words

LeetCode match: LC 692 Top K Frequent Words (related: count words per author)

Frequency: Single report | Last asked: 2025-09-21 | Roles: SWE Intern

QUESTION

You are given a chat transcript as a list of message lines. Each line has the form "Name: message text" , where everything before the first ": " is the author's name and everything after it is the message body. Count the number of words each person typed across the entire transcript (a word is a maximal run of non- whitespace characters in the body). A person may appear on many lines; sum their words. Return the k people who typed the most words, as a list of (name, word_count) tuples sorted by word_count descending. Break ties by name in ascending (lexicographic) order.

Function signature

```
def top_k_chatters(lines: list[str], k: int) -> list[tuple[str, int]]:
    ...
```

Follow-up (as reported): The interviewer's real ask was to implement the parsing inside a class alongside stubbed API methods. Wrap this in a ChatAnalyzer class that ingests lines incrementally via add_line(line) and answers top_k(k) on demand.

EXAMPLES

Example 1

Input	`lines = ["Alice: hello there friend", "Bob: hi", "Alice: how are you"], k = 1`
Output	`[("Alice", 6)]`
Why	`Alice typed 3 + 3 = 6 words, Bob typed 1. Top 1 is Alice.`

Example 2

Input	`lines = ["Ann: a b", "Bob: c d", "Cara: e"], k = 2`
Output	`[("Ann", 2), ("Bob", 2)]`
Why	`Ann and Bob both have 2 words; tie broken alphabetically, so Ann precedes Bob. Cara (1 word) is excluded.`

Example 3

Input	`lines = [], k = 3`
Output	`[]`

CONSTRAINTS

- 0 <= len(lines) <= 10^5
- Every line contains exactly one ": " separating name from body; the name is non-empty and contains no ": " .
- The body may be empty (0 words).

QUESTION

- 1 <= k ; if k exceeds the number of distinct people, return all of them.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_chatters(lines, k):
    counts = {}
    for line in lines:
        name, body = line.split(": ", 1)
        counts[name] = counts.get(
            name, 0
        ) + len(body.split())
    ranked = sorted(
        counts.items(),
        key=lambda kv: (-kv[1], kv[0]),
    )
    return ranked[:k]
```

TIME	SPACE
O(p log p)	O(p)

WHY IT IS SLOW

It sorts all p authors although only the top k are needed.

OPTIMAL SOLUTION · Counter and a bounded selection

TIME	SPACE
O(n + p log k)	O(p)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Wrap it in a ChatAnalyzer class that takes lines one at a time and answers top_k on demand.

Keep the Counter as state; add_line parses and updates it in O(words), top_k runs the selection. If top_k is called far more often than add_line, keep a sorted structure or a heap with lazy updates instead of recomputing.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Top-K Chatters by Word Count
#
# You are given a chat transcript as a list of message
# lines. Each line has the form "Name: message text" ,
# where everything before the first ": " is the author's
# name and everything after it is the message body. Count
# the number of words each person typed across the entire
# transcript (a word is a maximal run of non- whitespace
# characters in the body). A person may appear on many
# lines; sum their words. Return the k people who typed
# the most words, as a list of (name, word_count) tuples
# sorted by word_count descending. Break ties by name in
# ascending (lexicographic) order. Function signature: def
# top_k_chatters(lines: list[str], k: int) ->
```

```
# list[tuple[str, int]]: ... Follow-up (as reported): The
# interviewer's real ask was to implement the parsing
# inside a class alongside stubbed API methods.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Wrap it in a ChatAnalyzer class that takes lines one
# at a time and answers top_k on demand.
#
# INTERVIEW NOTES
# Two steps: parse each line at the first ': ' and add the
# number of words in the body to the author's total, then
# take the k largest totals with ties broken by name. A
# heap of size k (or sorting by the key (-count, name))
# handles the ordering.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq
import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def rank_of(pair):
    # pair = (name, word count): more words first, ties by name
    return (-pair[1], pair[0])


def words_in(line):
    # "Alice: hello there friend" -> ("Alice", 3)
    name, body = line.split(": ", 1)
    return name, len(body.split())


# ---- main question: main function ----
# Idea: add up the words written by every author in a dictionary, then
# pick the k best with a heap ranked by (-words, name).
# Complexity: Time O(total words + u log k), Space O(u)
def top_k_chatters(lines, k):
    counts = {}
    for line in lines:
        name, words = words_in(line)
        counts[name] = counts.get(name, 0) + words
    return heapq.nsmallest(k, counts.items(), key=rank_of)


# ---- main question: tests ----
def test_main_question():
    lines = ["Alice: hello there friend", "Bob: hi", "Alice: how are you"]
    assert top_k_chatters(lines, 1) == [("Alice", 6)]
    assert top_k_chatters(["Ann: a b", "Bob: c d", "Cara: e"], 2) == [("Ann", 2), ("Bob", 2)]
    assert top_k_chatters([], 3) == []


# ============================================================
# FOLLOW-UP 1: Wrap it in a ChatAnalyzer class that takes lines one at a time and answers top_k on demand
# ============================================================
```

```
# ---- follow-up 1: function ----
# Answer: keep the counts as the state; add_line parses the line and
# updates them in O(words); top_k runs the selection. If top_k is called
# far more often than add_line, keep a sorted structure (or a heap with
# lazy updates) instead of recomputing.
# Complexity: Time add_line O(words in the line), top_k O(u log k), Space O(u)
class ChatAnalyzer:
    def __init__(self):
        self.counts = {}

    def add_line(self, line):
        name, words = words_in(line)
```

```
        self.counts[name] = self.counts.get(name, 0) + words

    def top_k(self, k):
        return heapq.nsmallest(k, self.counts.items(), key=rank_of)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    random.seed(18)
    names = ["Zed", "amy", "Bo", "Cy"]
    for round_number in range(300):
        lines = []
        for i in range(random.randint(0, 8)):
            body = []
            for j in range(random.randint(1, 4)):
                body.append("w" * random.randint(1, 3))
            lines.append(random.choice(names) + ": " + " ".join(body))
        k = random.randint(1, 5)
        chat = ChatAnalyzer()
        for line in lines:
            chat.add_line(line)
        assert chat.top_k(k) == top_k_chatters(lines, k)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 24 OF 43

Digits and characters: peel one at a time and carry

3 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Digits and characters: peel one at a time and carry | 3 questions | tap a title to jump to it

READ FIRST: the code template for these 3 questions (tap here) >

M 126. Solve several coding interview problems — **PracHub * 2026-03-04 ***

M 127. Check Whether Digits Are Monotone Increasing — **PracHub * 2026-07-04 ***

M 128. Add two big integers from digit lists — **PracHub * 2025-11-24 ***

CODE TEMPLATE | GROUP 24 OF 43 | 3 QUESTIONS, ONE PATTERN

Digits and characters: peel one at a time and carry

The one idea these questions share

Work one digit or character at a time: n % 10 peels the last digit, n // 10 removes it, a carry moves overflow to the next place, and (ord(c) - 97 + k) % 26 shifts a letter. Build the answer in a list and reverse or join at the end.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def add_digits(a, b):                          # two digit lists, most significant first
    i, j, carry, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0 or j >= 0 or carry:
        total = carry + (a[i] if i >= 0 else 0) + (b[j] if j >= 0 else 0)
        out.append(total % 10)                 # CHANGE 1: what you build per place
        carry = total // 10                    # CHANGE 2: what you carry
        i, j = i - 1, j - 1
    return out[::-1] or [0]
```

2. Helper functions that usually get written next to it

```
def digits_of(n):
    out = []
    while n:
        n, d = divmod(n, 10)
        out.append(d)
    return out[::-1]


def shift_letter(ch, k):
    return chr((ord(ch) - 97 + k) % 26 + 97)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
126	Solve several coding interview problems	the big-number addition on strings, with int(ch) per digit; the other parts are list-meeting and overlap sweeps
127	Check Whether Digits Are Monotone Increasing	peel digits from the right; each digit must not exceed the previous one you saw
128	Add two big integers from digit lists	the template as written

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 126 of 197 | topic 24 of 43: Digits and characters: peel one at a time and carry | 1 of 3

Template fit: the big-number addition on strings, with int(ch) per digit; the other parts are list-meeting and overlap sweeps | * asked 230 days ago *

MEDIUM | PracHub | Technical Screen | 126 of 197

Solve several coding interview problems

POSTED 2026-03-04 | 230 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 38.3 | 30 DAYS 0 | PRIORITY 112/288

ABOUT THIS QUESTION

Topic: Digits and characters: peel one at a time and carry

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #160 Intersection of Two Linked Lists

LeetCode match: No direct match (multi-part); related LC 415 Add Strings, LC 160 Intersection of Two Linked Lists (inferred from overview)

Posted on PracHub: 2026-03-04 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/solve-several-coding-interview-problems

QUESTION

Three small tasks. (1) add_strings(a, b): add two non-negative integers given as digit strings and return the sum as a string, without converting the whole numbers. (2) intersection(head_a, head_b): two singly linked lists may share a common tail; return the first shared node, or None. (3) overlap_depth(rects): rectangles (x1, y1, x2, y2) include their left and bottom edges but not their right and top edges; return the largest number of rectangles that cover a single point.

Function signature

def add_strings(a: str, b: str) -> str
def intersection(head_a, head_b)
def overlap_depth(rects: list) -> int

EXAMPLES

Example 1

Input	`'999', '1'`
Output	`'1000'`
Why	`A carry runs through every digit: 999 + 1 = 1000.`

Example 2

Input	`'456', '77'`
Output	`'533'`
Why	`456 + 77 = 533.`

Example 3

Input	`class N:`
	`def __init__(self, val, next=None):`
	`self.val, self.next = val, next`
	`shared = N(8, N(9))`
	`a = N(1, N(2, shared))`
	`b = N(5, shared)`
	`intersection(a, b).val`

Output	`intersection(a, b).val -> 8`
Why	`List a is 1, 2, 8, 9 and list b is 5, 8, 9: they join at the node holding 8.`

QUESTION

Example 4	
Input	`[(0, 0, 4, 4), (2, 2, 6, 6), (3, 3, 5, 5)]`
Output	`3`
Why	`The point (3, 3) lies inside all three rectangles.`

Example 5	
Input	`[(0, 0, 2, 2), (2, 0, 4, 2)]`
Output	`1`
Why	`Rectangles that only share an edge do not overlap.`

BRUTE FORCE · Big numbers: convert to integers and add

BRUTE FORCE · PSEUDO-CODE

```
def add_strings(a, b):
    return str(int(a) + int(b))
```

TIME	SPACE
O(n^2) for the conversions	O(n)

WHY IT IS SLOW

Relies on arbitrary-precision integers (not in every language) and the string-to-integer conversions are quadratic for very long inputs. Adding digit by digit from the right with a carry is O(max length).

BRUTE FORCE · Chains that meet: store one chain in a set

BRUTE FORCE · PSEUDO-CODE

```
def intersection(a, b):
    seen = set()
    node = a
    while node:
        seen.add(node)
        node = node.parent
    node = b
    while node:
        if node in seen:
            return node
        node = node.parent
    return None
```

TIME	SPACE
O(m + n)	O(m)

WHY IT IS SLOW

O(m + n) time but O(m) extra memory. Two pointers that switch to the other chain's head at the end meet at the shared node with O(1) memory.

BRUTE FORCE · Overlap: test every time point

BRUTE FORCE · PSEUDO-CODE

```
def max_overlap(intervals):
    lo = min(s for s, e in intervals)
    hi = max(e for s, e in intervals)
    best = 0
    for t in range(lo, hi):
        busy = 0
        for s, e in intervals:
            if s <= t < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(range * n)	O(1)

WHY IT IS SLOW

Depends on the length of the time range, O(range * n). Sorting start and end events and keeping a running count is O(n log n).

OPTIMAL SOLUTION · Big numbers: digit by digit from the right with a carry

TIME	SPACE
O(max(m, n))	O(max(m, n))

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Chains that meet: two pointers that switch heads at the end

TIME	SPACE
O(m + n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Overlap: sweep over sorted start and end events

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Ancestors in a tree with parent pointers, not a linked list.

The same two-pointer trick works on parent chains (LeetCode 1650); with depths known, lift the deeper node first and then move both up together.

FOLLOW-UP 2 Add two big numbers stored as linked lists.

If the lists store the least significant digit first, walk both together with the carry (LeetCode 2); if most significant first, reverse them or use stacks.

FOLLOW-UP 3 Count overlapping rectangles instead of intervals.

Sweep over x with a data structure over y (a segment tree with range add and max) to find the largest point depth in O(n log n); the pairwise count is O(n^2) unless you use a sweep with an interval tree.

FOLLOW-UP 4 Add two big numbers stored as linked lists (least significant digit first).

Walk both lists together with a carry and build the result list as you go; no reversal needed. O(max(m, n)) time. If the most significant digit comes first, reverse the lists or use stacks.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Solve several coding interview problems
#
# Three small tasks. (1) add_strings(a, b): add two non-
# negative integers given as digit strings and return the
# sum as a string, without converting the whole numbers.
# (2) intersection(head_a, head_b): two singly linked
# lists may share a common tail; return the first shared
# node, or None. (3) overlap_depth(rects): rectangles (x1,
# y1, x2, y2) include their left and bottom edges but not
# their right and top edges; return the largest number of
# rectangles that cover a single point.
#
# Signature:
#   def add_strings(a: str, b: str) -> str
#   def intersection(head_a, head_b)
#   def overlap_depth(rects: list) -> int
#
# Examples:
# '999', '1'  ->  '1000'
# '456', '77'  ->  '533'
#
# FOLLOW-UPS (answered below, in this order):
# 1. Ancestors in a tree with parent pointers, not a
# linked list.
# 2. Add two big numbers stored as linked lists.
# 3. Count overlapping rectangles instead of intervals.
# 4. Add two big numbers stored as linked lists (least
# significant digit first).
#
# INTERVIEW NOTES
# A set of unrelated small tasks: split them, restate each
# contract, and solve the smallest one first. Big numbers:
# add digit by digit with a carry. Two chains that meet:
# walk both and switch to the other head at the end so the
# lengths cancel. Overlap counting: sort start and end
# events and keep a running count.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
```

```
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
```

```
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next


# ---- main question: main functions ----
# PART 1: add two big numbers kept as digit strings.
# Idea: walk both strings from the right, add the two digits and the
# carry, keep total % 10 and carry total // 10; after the loop add a final
# carry digit if there is one; reverse the digits at the end.
# Complexity: Time O(max(n, m)), Space O(max(n, m))
def add_strings(a, b):
    i = len(a) - 1
    j = len(b) - 1
    carry = 0
    digits = []
    while i >= 0 or j >= 0 or carry > 0:
        total = carry
        if i >= 0:
            total += int(a[i])
        if j >= 0:
            total += int(b[j])
        digits.append(str(total % 10))
        carry = total // 10
        i -= 1
        j -= 1
    digits.reverse()
    return "".join(digits)


# PART 2: where do two linked lists meet?
# Idea: each pointer walks its own list and then continues on the OTHER
# list. Both walk the same total length (m + n), so they arrive at the
# meeting node at the same moment (or both reach None together).
# Complexity: Time O(a + b), Space O(1)
def intersection(head_a, head_b):
    a = head_a
    b = head_b
    while a is not b:
        if a is None:
            a = head_b
```

```
        else:
            a = a.next
        if b is None:
            b = head_a
        else:
            b = b.next
    return a


# PART 3: the deepest overlap of rectangles. rects are (x1, y1, x2, y2),
# half-open: the largest number of rectangles that share one point.
# Idea: the best point can be a rectangle's lower-left corner, so test
# those corners and count the rectangles that contain each.
# Complexity: Time O(n^2), Space O(1)
```

```
def overlap_depth(rects):
    best = 0
    for x, y, x2, y2 in rects:
        inside = 0
        for r in rects:
            if r[0] <= x < r[2] and r[1] <= y < r[3]:
                inside += 1
        if inside > best:
            best = inside
    return best


# ---- main question: tests ----
def test_main_question():
    assert add_strings("999", "1") == "1000"
    assert add_strings("0", "0") == "0"
    assert add_strings("123456789123456789", "987654321") == str(123456789123456789 + 987654321)
    shared = ListNode(8, ListNode(9))
    a = ListNode(1, ListNode(2, shared))
    b = ListNode(5, shared)
    assert intersection(a, b) is shared
    assert intersection(ListNode(1), ListNode(2)) is None           # no meeting: both end at None
    assert overlap_depth([(0, 0, 4, 4), (2, 2, 6, 6), (3, 3, 5, 5)]) == 3
    assert overlap_depth([]) == 0


# ============================================================
# FOLLOW-UP 1: Ancestors in a tree with parent pointers, not a linked list
# ============================================================
# Answer: the same idea. The "lowest common ancestor" of two nodes is
# where their paths up to the root meet: walk each pointer up to the root
```

```
# and then restart at the other node (like intersection above).


# ============================================================
# FOLLOW-UP 2: Add two big numbers stored as linked lists
# ============================================================


# ============================================================
# FOLLOW-UP 4: Add two big numbers stored as linked lists (least significant digit first)
# ============================================================
# ---- follow-up 2: function ----
# Answer: if the digits are stored least significant FIRST, walk both
# lists together with a carry and build the result as you go (LeetCode 2):
# O(max(m, n)). If the most significant digit comes first, reverse the
# lists first or use stacks.
# Complexity: Time O(max(a, b)), Space O(max(a, b))
def add_lists(a, b):
    dummy = ListNode()
    tail = dummy
    carry = 0
    while a or b or carry > 0:
        total = carry
        if a:
            total += a.val
            a = a.next
        if b:
            total += b.val
            b = b.next
        tail.next = ListNode(total % 10)
        tail = tail.next
        carry = total // 10
    return dummy.next
```

```
# ---- follow-up 2: tests ----
def to_list(node):
    values = []
    while node:
        values.append(node.val)
        node = node.next
    return values


def test_follow_up_2():
```

```
    x = ListNode(2, ListNode(4, ListNode(3)))
    y = ListNode(5, ListNode(6, ListNode(4)))
    assert to_list(add_lists(x, y)) == [7, 0, 8]                  # 342 + 465 = 807 with digits reversed


# ============================================================
# FOLLOW-UP 3: Count overlapping rectangles instead of intervals
# ============================================================
# ---- follow-up 3: function ----
# Answer: counting overlapping PAIRS is O(n^2) by checking every pair. The
# deepest overlap in O(n log n) needs a sweep over x with a segment tree
# over y (range add, range max).
# Complexity: Time O(n^2), Space O(1)
def overlapping_pairs(rects):
    count = 0
    for i in range(len(rects)):
        for j in range(i + 1, len(rects)):
            a = rects[i]
            b = rects[j]
            if a[0] < b[2] and b[0] < a[2] and a[1] < b[3] and b[1] < a[3]:
                count += 1
    return count


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert overlapping_pairs([(0, 0, 4, 4), (2, 2, 6, 6), (3, 3, 5, 5)]) == 3
    assert overlapping_pairs([(0, 0, 1, 1), (2, 2, 3, 3)]) == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(s) -> list[str]:
```

Sample call

```
solution('.-')
```

Test cases

Input	Expected
`('.-',)`	`['A', 'ET']`
`('...',)`	`['EEE', 'EI', 'IE', 'S']`

STUDY PATH: question 127 of 197 | topic 24 of 43: Digits and characters: peel one at a time and carry | 2 of 3

Template fit: peel digits from the right; each digit must not exceed the previous one you saw | * asked 108 days ago *

MEDIUM | PracHub | Technical Screen | 127 of 197

Check Whether Digits Are Monotone Increasing

POSTED 2026-07-04 | 108 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 253/288**

ABOUT THIS QUESTION

Topic: Digits and characters: peel one at a time and carry

Optimal answer from: the interview site's own solution

LeetCode match: No LC match (easy digit scan) (inferred from overview)

Posted on PracHub: 2026-07-04 (updated 2026-07-15)

Original: https://prachub.com/coding-questions/check-whether-digits-are-monotone-increasing

QUESTION

Return true when the decimal digits of a non-negative integer never decrease from left to right (repeated digits are allowed). Zero and single-digit numbers qualify.

Function signature

def is_monotone_digits(n: int) -> bool

EXAMPLES

Example 1	
Input	`1234`
Output	`true`
Why	`1, 2, 3, 4 only goes up.`

Example 2	
Input	`1223`
Output	`true`
Why	`Equal neighbours are allowed.`

Example 3	
Input	`332`
Output	`false`
Why	`3, 3, 2 goes down at the end.`

Example 4	
Input	`0`
Output	`true`
Why	`Zero qualifies.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_monotone(x):
    s = str(x)
    return s == "".join(sorted(s))
```

TIME	SPACE
O(d log d)	O(d)

WHY IT IS SLOW

Sorting costs O(d log d) and copies the digits. Comparing each digit with its neighbour while peeling digits with % 10 is O(d) with O(1) space.

OPTIMAL SOLUTION · Peel digits from the right and compare with the previous one

TIME	SPACE
O(d)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Strictly increasing digits.

Use >= instead of > when comparing with the previous digit, and start prev at 10 so a single digit passes.

FOLLOW-UP 2 Negative numbers or numbers in another base.

Decide what monotone means for the sign (usually apply it to the absolute value). For base b replace 10 with b in % and //.

FOLLOW-UP 3 Find the next monotone number after n.

That is the left-to-right fix from the previous question: find the first descent and copy that digit across the rest.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Check Whether Digits Are Monotone
# Increasing
#
# Return true when the decimal digits of a non-negative
# integer never decrease from left to right (repeated
# digits are allowed). Zero and single-digit numbers
# qualify.
#
# Signature:
#   def is_monotone_digits(n: int) -> bool
#
# Examples:
# 1234  ->  true
# 1223  ->  true
#
# FOLLOW-UPS (answered below, in this order):
# 1. Strictly increasing digits.
```

```
# 2. Negative numbers or numbers in another base.
# 3. Find the next monotone number after n.
#
# INTERVIEW NOTES
# Compare neighbouring digits. You can read the digits
# from the right with % 10 and // 10, remembering the
# previous (right-hand) digit: the number is monotone if
# no digit is larger than the one to its right. Say first
# whether equal neighbours are allowed.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def is_monotone_string(n):
    # the same test on the text of the number (used to check the main function)
    text = str(n)
    for i in range(len(text) - 1):
        if text[i] > text[i + 1]:
            return False
    return True
```

```
# ---- main question: main function ----
# Idea: take the last digit with n % 10, then drop it with n // 10, and so
# on. Moving from right to left, each digit just read must not be bigger
# than the digit read before it (the one to its right). Zero and a
# single digit are monotone.
# Complexity: Time O(d) for d digits, Space O(1)
def is_monotone_digits(n):
    previous = 9                            # nothing is bigger than 9
    while n > 0:
        digit = n % 10
        if digit > previous:
            return False                    # a digit bigger than the one to its right
        previous = digit
        n //= 10
    return True


# ---- main question: tests ----
def test_main_question():
    assert is_monotone_digits(0) and is_monotone_digits(7)
    assert is_monotone_digits(1223)
    assert is_monotone_digits(999)
    assert not is_monotone_digits(332)
    assert not is_monotone_digits(10)
    for n in range(2000):
        assert is_monotone_digits(n) == is_monotone_string(n)


# ============================================================
# FOLLOW-UP 1: Strictly increasing digits
# ============================================================
# ---- follow-up 1: function ----
# Answer: use >= instead of > when comparing with the previous digit, and
# start `previous` at 10 so that a single digit passes.
```

```python
# Complexity: Time O(d), Space O(1)
def is_strictly_monotone(n):
    previous = 10
    while n > 0:
        digit = n % 10
        if digit >= previous:
            return False
        previous = digit
        n //= 10
```

```python
    return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert is_strictly_monotone(1234)
    assert not is_strictly_monotone(1224)
    assert is_strictly_monotone(5)


# =========================================================
# FOLLOW-UP 2: Negative numbers or numbers in another base
# =========================================================
# ---- follow-up 2: function ----
# Answer: decide what "monotone" means for the sign (usually apply it to
# the absolute value). For another base b, use b instead of 10 in % and //.
# Complexity: Time O(d), Space O(1)
def is_monotone_base(n, base=10):
    previous = base - 1
    while n > 0:
        digit = n % base
        if digit > previous:
            return False
        previous = digit
        n //= base
    return True


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert is_monotone_base(3, 2)              # binary 11
    assert not is_monotone_base(2, 2)          # binary 10


# =========================================================
# FOLLOW-UP 3: Find the next monotone number after n
# =========================================================
# ---- follow-up 3: function ----
# Answer: take n + 1, find its first descent from the LEFT and copy that
# digit over the rest (332 becomes 333).
# Complexity: Time O(d), Space O(d)
def next_monotone(n):
    digits = list(str(n + 1))
    for i in range(len(digits) - 1):
        if digits[i] > digits[i + 1]:
```

```python
            for j in range(i + 1, len(digits)):
                digits[j] = digits[i]
            break
    return int("".join(digits))


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert next_monotone(332) == 333
```

```
    assert next_monotone(9) == 11
    for n in range(2000):
        following = n + 1
        while not is_monotone_string(following):
            following += 1
        assert next_monotone(n) == following


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def has_monotone_increasing_digits(n: int) -> bool:
SAMPLE INPUT
0
```

Test cases

Input	Expected
0	True
7	True

STUDY PATH: question 128 of 197 | topic 24 of 43: Digits and characters: peel one at a time and carry | 3 of 3

Template fit: the template as written | * asked 330 days ago *

MEDIUM | IN APP LC 2 | PracHub | Technical Screen | 128 of 197

★ TOP 75: rank 1 in the Medium and Hard priority list

★ Add two big integers from digit lists

POSTED 2025-11-24 | 330 DAYS AGO | OLDER

STUDY: TYPE **ALL TIME 78.7** **30 DAYS 69.3** **PRIORITY 1/288**

ABOUT THIS QUESTION

Topic: Digits and characters: peel one at a time and carry

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #2 Add Two Numbers

LeetCode match: LC 2 Add Two Numbers

Posted on PracHub: 2025-11-24 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/add-two-big-integers-from-digit-lists

QUESTION

You are given two non-empty singly linked lists (or arrays) representing two non-negative integers. The digits are stored in **reverse order**, and each node contains a single digit from 0 to 9. Your task is to add the two numbers and return the sum as a new linked list, also in reverse order. You may assume:

- Each list has at least one node.
- The integers do not contain leading zeros, except the number 0 itself.

Example

Input:

- List 1: 2 -> 4 -> 3 (represents the integer 342)
- List 2: 5 -> 6 -> 4 (represents the integer 465)

Output:

- 7 -> 0 -> 8 (represents the integer 807, since 342 + 465 = 807)

Another example

Input:

- List 1: 9 -> 9 -> 9 (represents 999)
- List 2: 1 (represents 1)

Output:

- 0 -> 0 -> 0 -> 1 (represents 1000)

Constraints

- Let n be the length of the first list and m be the length of the second list.
- 1 <= n, m <= 10^5.
- Each node value is an integer in the range [0, 9].

Task: Describe an algorithm to perform this addition and construct the resulting linked list in O(n + m) time and O(1) additional space (excluding the space for the output list).

QUESTION

You are given two non-empty arrays of digits representing two non-negative integers. The digits are stored in **reverse order** (least significant digit first), and each element is a single digit from 0 to 9. Add the two numbers and return the sum as a new array of digits, also in reverse order.

Assumptions

- Each array has at least one element.
- The integers do not contain leading zeros, except the number 0 itself.

Example 1

```
Input: l1 = [2, 4, 3], l2 = [5, 6, 4] (342 + 465)
Output: [7, 0, 8] (807)
```

Example 2

```
Input: l1 = [9, 9, 9], l2 = [1] (999 + 1)
Output: [0, 0, 0, 1] (1000)
```

Implement **`add_two_numbers(l1, l2)`** (camelCase **`addTwoNumbers`** in Java/C++/JS) returning the digit array of the sum in reverse order. Aim for O(n + m) time and O(1) extra space beyond the output.

Constraints

1 <= n, m <= 10^5 where n = len(l1), m = len(l2)Each element is an integer in the range [0, 9]Digits are stored in reverse order (least significant digit first)No leading zeros except for the number 0 itself

EXAMPLES

```
Input: ([2, 4, 3], [5, 6, 4])
Expected Output: [7, 0, 8]
Explanation: 342 + 465 = 807, stored reversed as [7, 0, 8].
Input: ([9, 9, 9], [1])
Expected Output: [0, 0, 0, 1]
```

Explanation: 999 + 1 = 1000; carry propagates through all three digits and creates a new leading digit.

Hints

Walk both arrays from index 0 (the least significant digit) toward the end, just like grade-school column addition.Maintain a single carry variable. At each position, sum the available digits plus the carry; append (sum % 10) and set carry = sum // 10.When one array runs out, keep going with just the carry and the remaining longer array. Don't forget a final carry can add one extra digit (e.g. 999 + 1 -> [0,0,0,1]).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def add_lists(a, b):
    x = digits_to_int(a)  # reverse-order digits
    y = digits_to_int(b)
    return int_to_digits(x + y)
```

TIME	SPACE
O(n^2)	**O(n)**
for the string-to-integer conversions	

WHY IT IS SLOW

It relies on arbitrary-size integers and the conversions are quadratic for very long inputs. Adding digit by digit from the right with a carry is O(max length).

OPTIMAL SOLUTION · Digit by digit with a carry

TIME	SPACE
O(max(m, n))	O(max(m, n))

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The digits are stored least significant first in a linked list.

Then the head is already the low end, so walk both lists together with the same carry loop and build the result list as you go. No reversal needed (LeetCode 2).

FOLLOW-UP 2 Multiply instead of add.

Schoolbook multiplication: the product of digits a[i] and b[j] lands in position i + j + 1, with the carry spilling to i + j. O(m * n); Karatsuba or FFT for very long numbers.

FOLLOW-UP 3 Subtract, or numbers in another base.

Subtraction borrows instead of carrying and needs the larger number first (compare lengths, then digits). Another base replaces 10 with the base in % and //.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Add two big integers from digit lists
#
# You are given two non-empty singly linked lists (or
# arrays) representing two non-negative integers. The
# digits are stored in **reverse order**, and each node
# contains a single digit from 0 to 9. Your task is to add
# the two numbers and return the sum as a new linked list,
# also in reverse order. You may assume: - Each list has
# at least one node. - The integers do not contain leading
# zeros, except the number 0 itself. ### Example Input: -
# List 1: 2 -> 4 -> 3 (represents the integer 342) - List 2:
# 5 -> 6 -> 4 (represents the integer 465) Output: - 7 -> 0 ->
# 8 (represents the integer 807, since 342 + 465 = 807)
# ### Another example Input: - List 1: 9 -> 9 -> 9
# (represents 999) - List 2: 1 (represents 1) Output: - 0
# -> 0 -> 0 -> 1 (represents 1000) ### Constraints - Let n be
# the length of the first list and m be the length of the
# second list. - 1 <= n, m <= 10^5.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The digits are stored least significant first in a
# linked list.
# 2. Multiply instead of add.
# 3. Subtract, or numbers in another base.
#
# INTERVIEW NOTES
# Addition by hand: start at the least significant end,
# add the two digits and the carry, write digit % 10,
# carry digit // 10. The only edge cases are different
# lengths and a final carry. Never convert to a machine
# integer in an interview about big numbers, since the
# point is that the numbers do not fit.
```

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: the numbers are lists of digits with the MOST significant digit
```

```
# first. Walk both lists from the right with indices i and j. At each
# step add a[i] (or 0 when a is used up), b[j] (or 0) and the carry,
# keep total % 10 as the digit, and carry total // 10. Continue while
# anything is left, including a final carry. Reverse at the end.
# Complexity: Time O(max(n, m)), Space O(max(n, m))
def add_digits(a, b):
    i = len(a) - 1
    j = len(b) - 1
    carry = 0
    digits = []
    while i >= 0 or j >= 0 or carry > 0:
        total = carry
        if i >= 0:
            total += a[i]
        if j >= 0:
            total += b[j]
        digits.append(total % 10)
        carry = total // 10
        i -= 1
        j -= 1
    if len(digits) == 0:
        return [0]
    digits.reverse()
    return digits


# ---- main question: tests ----
def test_main_question():
    assert add_digits([9, 9], [1]) == [1, 0, 0]
    assert add_digits([1, 2, 3], [4, 5]) == [1, 6, 8]
    assert add_digits([9, 9, 9], [1]) == [1, 0, 0, 0]
    assert add_digits([0], [0]) == [0]


# ============================================================
# FOLLOW-UP 1: The digits are stored least significant first in a linked list
# ============================================================
# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next


def make_list(digits):
```

```
    # [2, 4, 3] -> 2 -> 4 -> 3 (least significant digit first)
    dummy = ListNode()
    tail = dummy
    for d in digits:
        tail.next = ListNode(d)
```

```
        tail = tail.next
    return dummy.next


def to_digits(node):
    digits = []
    while node:
        digits.append(node.val)
        node = node.next
    return digits


# ---- follow-up 1: function ----
# Answer: the head is already the LOW end, so walk both lists together
# with the same carry loop and build the result list as you go. No
# reversal is needed (LeetCode 2).
# Complexity: Time O(max(n, m)), Space O(max(n, m))
def add_lists(a, b):
    dummy = ListNode()
    tail = dummy
    carry = 0
    while a or b or carry > 0:
        total = carry
        if a:
            total += a.val
            a = a.next
        if b:
            total += b.val
            b = b.next
        tail.next = ListNode(total % 10)
        tail = tail.next
        carry = total // 10
    return dummy.next


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert to_digits(add_lists(make_list([2, 4, 3]), make_list([5, 6, 4]))) == [7, 0, 8]     # 342 + 465 =
807
    assert to_digits(add_lists(make_list([9, 9]), make_list([1]))) == [0, 0, 1]              # 99 + 1 = 100
```

```
# =========================================================
# FOLLOW-UP 2: Multiply instead of add
# =========================================================
# ---- follow-up 2: function ----
# Answer: schoolbook multiplication: the product of the digits a[i] and
# b[j] lands at position i + j + 1, and its carry spills to position
# i + j. O(m * n); Karatsuba or FFT for very long numbers.
# Complexity: Time O(n * m), Space O(n + m)
def multiply_digits(a, b):
    result = [0] * (len(a) + len(b))
    for i in range(len(a) - 1, -1, -1):
        for j in range(len(b) - 1, -1, -1):
            total = a[i] * b[j] + result[i + j + 1]
            result[i + j + 1] = total % 10
            result[i + j] += total // 10
    while len(result) > 1 and result[0] == 0:
        result.pop(0)                                # no leading zeros
    return result


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert multiply_digits([1, 2, 3], [4, 5]) == [5, 5, 3, 5]
```

```
    assert multiply_digits([0], [9, 9]) == [0]


# ======================================================
# FOLLOW-UP 3: Subtract, or numbers in another base
# ======================================================
# ---- follow-up 3: function ----
# Answer: subtraction BORROWS instead of carrying, and needs the larger
# number first (compare lengths, then digits). Another base replaces 10
# by the base in % and //.
# Complexity: Time O(n), Space O(n)
def subtract_digits(a, b):
    # a - b where a >= b, most significant digit first
    i = len(a) - 1
    j = len(b) - 1
    borrow = 0
    digits = []
    while i >= 0:
        d = a[i] - borrow
        if j >= 0:
            d -= b[j]
        if d < 0:
            d += 10
            borrow = 1
        else:
            borrow = 0
        digits.append(d)
        i -= 1
        j -= 1
    while len(digits) > 1 and digits[-1] == 0:
        digits.pop()                                    # drop leading zeros
    digits.reverse()
    return digits


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert subtract_digits([1, 0, 0], [1]) == [9, 9]
    assert subtract_digits([5, 0], [5, 0]) == [0]


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Sample call

```
add_two_numbers([2, 4, 3], [5, 6, 4])
```

Test cases

Input	Expected
`([2, 4, 3], [5, 6, 4])`	`[7, 0, 8]`
`([9, 9, 9], [1])`	`[0, 0, 0, 1]`

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 25 OF 43

Look at the input once and keep only a small summary (and count your calls)

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Look at the input once and keep only a small summary (and count your calls) | 4 questions | tap a title to jump to it

READ FIRST: the code template for these 4 questions (tap here) >

M	129. Streaming Points: Emit K Within Distance	1Point3Acres * 2026-06-23 *
M	130. Two-Day Log Intersection under Memory Limit	1Point3Acres * 2025-11-27 *
M	131. Find shared objects across two log files	PracHub * 2025-11-05 *
M	132. Minimize calls to find all bad test pairs	PracHub * 2025-11-05 *

CODE TEMPLATE | GROUP 25 OF 43 | 4 QUESTIONS, ONE PATTERN

Look at the input once and keep only a small summary (and count your calls)

The one idea these questions share

When the input is too big to hold, read it one item at a time and keep a tiny summary: a maximum, a set from the smaller file, a sorted window. When asking a question is expensive (an oracle), use binary search on how many items you include so the number of calls stays small.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def stream_max(lines):
    best = None                                  # CHANGE 1: the small summary you keep
    for line in lines:                           # a file object gives one line at a time
        line = line.strip()
        if not line:
            continue
        value = int(line)                        # CHANGE 2: parse what matters from the line
        if best is None or value > best:
            best = value                         # CHANGE 3: update the summary
    return best
```

2. Helper functions that usually get written next to it

```
def first_failing_prefix(items, passes):         # binary search with an oracle: fewest calls
    lo, hi = 0, len(items)
    while lo < hi:
        mid = (lo + hi) // 2
        if passes(items[:mid + 1]):
            lo = mid + 1
        else:
            hi = mid
    return lo if lo < len(items) else -1
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
129	Streaming Points: Emit K Within Distance	keep the points in a sorted list; each new point is inserted with bisect and only windows around it are re-checked
130	Two-Day Log Intersection under Memory Limit	dict object -> set of event codes for each day; keep the objects that appear on both days with at least two distinct codes
131	Find shared objects across two log files	load only the smaller file's object ids into a set, then stream the big file and keep ids found in the set
132	Minimize calls to find all bad test pairs	the same group testing: put a whole group in one call and split only the groups that fail

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 129 of 197 | topic 25 of 43: Look at the input once and keep only a small summary (and count your calls) | 1 of 4

Template fit: keep the points in a sorted list; each new point is inserted with bisect and only windows around it are re-checked | * asked 119 days ago * | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 129 of 197

Streaming Points: Emit K Within Distance

ASKED 2026-06-23 | 119 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 259/288**

ABOUT THIS QUESTION

Topic: Look at the input once and keep only a small summary (and count your calls)

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-06-23 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/streaming-points-k-within-distance

QUESTION

Positions (floats) arrive one at a time. Whenever k stored points fit inside a span of d (largest minus smallest is at most d), output those k points and remove them. add(x) returns the groups emitted because of this arrival (a list of lists).

Function signature

class PointStream:
  def __init__(self, k: int, d: float)
  def add(self, x: float) -> list

EXAMPLES

Example 1

Input	`s = PointStream(3, 2.0)`
	`s.add(1.0)`
	`s.add(5.0)`
	`s.add(2.0)`
	`s.add(1.5)`

Output	`s.add(1.0) -> []`
	`s.add(5.0) -> []`
	`s.add(2.0) -> []`
	`s.add(1.5) -> [[1, 1.5, 2]]`

Why	`After 1, 5, 2 no three points fit in a span of 2 (1 and 5 are too far apart); adding 1.5 gives 1, 1.5, 2 with span 1, which is emitted and removed.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
points = []
def on_point(x):
    points.append(x)
    points.sort()
    emitted = []
    while True:
        group = find_group()
        if group is None:
            return emitted
        emitted.append(group)
        remove(points, group)
def find_group():
    for i in range(len(points) - k + 1):
        if points[i + k - 1] - points[i] <= d:
            return points[i:i + k]
    return None
```

TIME	SPACE
O(n log n) per point	**O(n)**

WHY IT IS SLOW

Every arrival re-sorts and rescans the whole list, O(n log n). A sorted list with binary search checks only the windows around the new point.

OPTIMAL SOLUTION · Sorted list, check only windows touching the change

TIME	SPACE
O(log n + K) per check, plus list shifting	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if 'within d' means every pair, and points are in 2D?

On a line pairwise distance equals the span, so nothing changes. In 2D it becomes a clique problem: candidates must be neighbours in a spatial grid of cell size d, and you test the K-subset of nearby points; that is much harder, so ask whether the 1D reading is intended.

FOLLOW-UP 2 Floating-point coordinates.

Compare with the exact subtraction hi - lo <= d (not by building a range around the target), and agree on an epsilon policy only if the interviewer wants one. Do not compare floats for equality.

FOLLOW-UP 3 Several groups qualify at once.

Agree on a rule (lowest start, as coded) so output is deterministic, and note that emitting one group can create or destroy others.

FOLLOW-UP 4 The condition is 'every pair within d', not 'span at most d'.

On a line they are the same: the largest pairwise distance is max - min, so checking the span is enough (verified against checking all pairs). In 2D they differ and become a clique problem.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Streaming Points: Emit K Within Distance
#
# Positions (floats) arrive one at a time. Whenever k
# stored points fit inside a span of d (largest minus
# smallest is at most d), output those k points and remove
# them. add(x) returns the groups emitted because of this
# arrival (a list of lists).
#
# Signature:
#   class PointStream:
#       def __init__(self, k: int, d: float)
#       def add(self, x: float) -> list
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if 'within d' means every pair, and points are
# in 2D?
# 2. Floating-point coordinates.
# 3. Several groups qualify at once.
# 4. The condition is 'every pair within d', not 'span at
# most d'.
#
# INTERVIEW NOTES
# On a line, K points are all within distance d exactly
# when the largest minus the smallest is at most d, so
# only K consecutive points in sorted order can form a
# group. Keep the points sorted and look only where
# something changed: a window that contains the new point,
# or the seam left by removed points.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from bisect import bisect_left, insort


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: points arrive one at a time. As soon as there are k waiting points
# that fit inside a stretch of length d (the span of the group =
```

```
# largest - smallest <= d), emit them as a group and forget them. Keep
# the waiting points in a SORTED list (insort = binary search plus insert).
# Before the new point no such group existed, so a new group must contain the
# new point: only the (at most k) windows of k neighbours that include its
# position need to be checked.
# Complexity: Time add is O(n) (a sorted list insert) plus O(k) per window check, Space O(n) waiting points
class PointStream:
```

```
    def __init__(self, k, d):
        self.k = k
        self.d = d
        self.points = []                          # the waiting points, sorted

    def window_start(self, pos):
        # the first window of k neighbouring points that includes position
        # pos and fits in a span of d, or None
        first = pos - self.k + 1
        if first < 0:
            first = 0
        last = len(self.points) - self.k
        if pos < last:
            last = pos
        for i in range(first, last + 1):
            span = self.points[i + self.k - 1] - self.points[i]
            if span <= self.d:
                return i
        return None

    def add(self, x):
        insort(self.points, x)
        groups = []
        pos = bisect_left(self.points, x)
        while True:
            i = self.window_start(pos)
            if i is None:
                return groups
            groups.append(self.points[i:i + self.k])      # emit the group ...
            del self.points[i:i + self.k]                 # ... and remove it so it is emitted only once
            pos = i


# ---- main question: tests ----
def test_main_question():
    s = PointStream(3, 1.0)
    assert s.add(0.5) == []
    assert s.add(5.0) == []
```

```
    assert s.add(1.0) == []
    assert s.add(1.4) == [[0.5, 1.0, 1.4]]              # the span is 0.9 <= 1.0
    assert s.points == [5.0]                            # the far point is still waiting


# ======================================================
# FOLLOW-UP 1: What if "within d" means every pair, and points are in 2D?
# ======================================================


# ======================================================
# FOLLOW-UP 4: The condition is "every pair within d", not "span at most d"
# ======================================================
# ---- follow-up 4: function ----
# Answer: on a line they are the same: the largest distance between two
# points is max - min, so checking the span is enough (verified below
# against checking all pairs). In 2D they differ: "every pair within d"
# becomes a clique problem (and a grid index finds the candidates).
# Complexity: Time O(k^2) for a group of k points, Space O(1)
def all_pairs_within(group, d):
    for a in group:
        for b in group:
            if abs(a - b) > d:
                return False
    return True
```

```
# ---- follow-up 4: tests ----
def test_follow_up_4():
    s = PointStream(3, 1.0)
    s.add(0.5)
    s.add(1.0)
    groups = s.add(1.4)
    for group in groups:
        assert all_pairs_within(group, 1.0)
        assert max(group) - min(group) <= 1.0            # on a line, span and pair checks agree
    assert not all_pairs_within([0.0, 0.6, 1.2], 1.0)
    t = PointStream(2, 0.5)                              # k = 2 groups close neighbours as they appear
    assert t.add(1.0) == []
    assert t.add(1.4) == [[1.0, 1.4]]


# ======================================================
# FOLLOW-UP 2: Floating-point coordinates
# ======================================================
```

```
# Answer: compare with a small tolerance (span <= d + 1e-9), or use
# integers (scale and round) or exact fractions when exactness matters.


# ======================================================
# FOLLOW-UP 3: Several groups qualify at once
# ======================================================
# Answer: add() already loops: after emitting a group it checks the
# windows again from the same position, so every group that the new
# point completes is emitted.


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Practice range queries over sorted points with insertion and deletion.
Write the K-generalization in words before coding: identify the local candidate window, test the predicate, then remove exactly the emitted points.
Rehearse explaining float comparison, tie-breaking, and what to do when multiple groups qualify.

STUDY PATH: question 130 of 197 | topic 25 of 43: Look at the input once and keep only a small summary (and count your calls) | 2 of 4

Template fit: dict object -> set of event codes for each day; keep the objects that appear on both days with at least two distinct codes | * asked 327 days ago * | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 130 of 197

Two-Day Log Intersection under Memory Limit

ASKED 2025-11-27 | 327 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 50.4 | 30 DAYS 46.9 | PRIORITY 28/288

ABOUT THIS QUESTION

Topic: Look at the input once and keep only a small summary (and count your calls)

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 349 Intersection of Two Arrays (related)

Frequency: Single report | Last asked: 2025-11-27 | Roles: SWE, INFRA-ENG

Original: https://www.1point3acres.com/interview/problems/company/google/two-logs-intersection-memory-limit

QUESTION

PhD intern coding round 1: two log files with **`(timestamp, obj_id, client_id)`** entries. Find **`obj_id`**s that (a) appear in both files and (b) have at least two distinct **`client_id`**s. Follow-up imposes a memory limit, expecting external sort + two-pointer.

Input: two log files; each line is (timestamp, obj_id, client_id).
Find every obj_id such that:

(a) appears in both files, and
(b) is associated with at least 2 distinct client_ids (aggregated across both files).

Follow-up 1: complexity of set-intersection on the file contents.
Follow-up 2: memory cannot hold either file fully — design a streaming solution.

EXAMPLES

```
File A:  (t1, X, c1), (t2, X, c2), (t3, Y, c1)
File B:  (t4, X, c1), (t5, Z, c2)
-> Only X is in both files; X has 2 distinct clients {c1, c2}.
Output: [X]
```

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(first, second):
    result = []
    for obj in {r.obj for r in first}:
        if obj not in {r.obj for r in second}:
            continue
        rows = [r for r in first + second
                if r.obj == obj]
        if len({r.client for r in rows}) >= 2:
            result.append(obj)
    return result
```

TIME	SPACE
O(n * m)	O(n + m)

BRUTE FORCE

WHY IT IS SLOW

O(n * m) time and both files must fit in memory. Sorting both files by object and merging them with two pointers is O(n log n) and holds one object's clients at a time.

OPTIMAL SOLUTION · Hash map of client sets (everything fits in memory)

TIME	SPACE
O(n + m)	**O(n + m)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why prefer sort and two pointers to a streaming hash map?

A hash map needs memory proportional to the number of distinct objects (and clients), which is exactly what the limit forbids. An external sort needs only a chunk of memory and sequential disk passes, and the merge needs memory for one object's clients. If one object has an enormous client set, count distinct clients approximately (HyperLogLog) or stop as soon as two are seen.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Two-Day Log Intersection under Memory
# Limit
#
# PhD intern coding round 1: two log files with
# (timestamp, obj_id, client_id) entries. Find obj_ids
# that (a) appear in both files and (b) have at least two
# distinct client_ids. Follow-up imposes a memory limit,
# expecting external sort + two-pointer. Input: two log
# files; each line is (timestamp, obj_id, client_id). Find
# every obj_id such that: (a) appears in both files, and
# (b) is associated with at least 2 distinct client_ids
# (aggregated across both files). Follow-up 1: complexity
# of set-intersection on the file contents. Follow-up 2:
# memory cannot hold either file fully — design a
# streaming solution. Examples: File A: (t1, X, c1), (t2,
# X, c2), (t3, Y, c1) File B: (t4, X, c1), (t5, Z, c2) ->
# Only X is in both files; X has 2 distinct clients {c1,
# c2}. Output: [X]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why prefer sort and two pointers to a streaming hash
# map?
#
# INTERVIEW NOTES
# In memory it is a hash map from object to the set of
# clients, filtered to objects that appear in both files
# and have at least two clients. When the files do not fit
# in memory, sort both by object id (external sort) and
# walk them together: all rows of one object are adjacent,
# so you only ever hold the clients of one object at a
# time.

# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
```

```
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea (everything fits in memory): for each day build a dictionary
# object -> set of clients. Keep the objects that appear on BOTH days and
```

```
# whose clients, over the two days together, number at least 2.
# A row is (timestamp, object id, client id).
# Complexity: Time O(n + m) rows, Space O(distinct objects + clients)
def log_objects(day1, day2):
    clients1 = {}
    clients2 = {}
    for timestamp, obj, client in day1:
        if obj not in clients1:
            clients1[obj] = set()
        clients1[obj].add(client)
    for timestamp, obj, client in day2:
        if obj not in clients2:
            clients2[obj] = set()
        clients2[obj].add(client)
    result = []
    for obj in clients1:
        if obj in clients2 and len(clients1[obj] | clients2[obj]) >= 2:
            result.append(obj)
    return sorted(result)


# ---- main question: tests ----
DAY1 = [(1, "X", "c1"), (2, "X", "c2"), (3, "Y", "c1")]
DAY2 = [(4, "X", "c1"), (5, "Z", "c2")]


def test_main_question():
    assert log_objects(DAY1, DAY2) == ["X"]
    one = [(1, "A", "c1")]
    assert log_objects(one, one) == []                          # one client in total is not enough
    assert log_objects([(1, "A", "c1")], [(2, "A", "c2")]) == ["A"]     # clients may be split across the t
wo days
    assert log_objects([], DAY2) == []


# ========================================================
# FOLLOW-UP 1: Why prefer sort and two pointers to a streaming hash map?
# ========================================================
# ---- follow-up 1: helper functions ----
def object_of(row):
    return row[1]


def group_rows(rows):
    # rows sorted by object -> a list of (object, set of its clients),
    # produced one object at a time
```

```
    groups = []
    current = None
    clients = set()
    for timestamp, obj, client in rows:
        if obj != current:
            if current is not None:
                groups.append((current, clients))
```

```
            current = obj
            clients = set()
        clients.add(client)
    if current is not None:
        groups.append((current, clients))
    return groups


# ---- follow-up 1: function ----
# Answer: a hash map needs memory proportional to the number of DISTINCT
# objects (and clients), which is exactly what the memory limit forbids.
# If both files are sorted by object (an external sort), they can be
# merged like two sorted lists, and only ONE object's clients are in memory
# at a time.
# Complexity: Time O(n + m) after both files are sorted by object, Space O(clients of one object)
def log_objects_sorted(rows1, rows2):
    groups1 = group_rows(rows1)
    groups2 = group_rows(rows2)
    result = []
    i = 0
    j = 0
    while i < len(groups1) and j < len(groups2):
        if groups1[i][0] == groups2[j][0]:
            if len(groups1[i][1] | groups2[j][1]) >= 2:
                result.append(groups1[i][0])
            i += 1
            j += 1
        elif groups1[i][0] < groups2[j][0]:
            i += 1
        else:
            j += 1
    return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    sorted1 = sorted(DAY1, key=object_of)
    sorted2 = sorted(DAY2, key=object_of)
```

```
    assert log_objects_sorted(sorted1, sorted2) == ["X"]
    one = [(1, "A", "c1")]
    assert log_objects_sorted(one, one) == []
    assert log_objects_sorted([(1, "A", "c1")], [(2, "A", "c2")]) == ["A"]


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Practice external-sort / merge-step problems (classic in-memory merge / k-way-merge / partition problems — and discuss disk-based variants verbally).
Drill the streaming hashmap vs external sort trade-off conversation.
Have ready: HyperLogLog (or Count-Min Sketch) one-liner for approximate distinct-count under tight memory.

STUDY PATH: question 131 of 197 | topic 25 of 43: Look at the input once and keep only a small summary (and count your calls) | 3 of 4

Template fit: load only the smaller file's object ids into a set, then stream the big file and keep ids found in the set | * asked 349 days ago *

MEDIUM | PracHub | Technical Screen | 131 of 197

★ TOP 75: rank 35 in the Medium and Hard priority list

★ Find shared objects across two log files

POSTED 2025-11-05 | 349 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 50.4 | 30 DAYS 46.9 | PRIORITY 27/288

ABOUT THIS QUESTION

Topic: Look at the input once and keep only a small summary (and count your calls)

Optimal answer from: the interview site's own solution

LeetCode match: LC 349 Intersection of Two Arrays

Posted on PracHub: 2025-11-05 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/find-shared-objects-across-two-log-files

QUESTION

You are given two large log files representing activity on two different days. Each line of each log has three fields:

- **`timestamp`**: a time value (you may treat it as an opaque string or integer)
- **`obj_id`**: identifier of an object (string or integer)
- **`client_id`**: identifier of a client (string or integer) The logs are not necessarily sorted by any field. Define an object as **interesting** if:

1. Its **`obj_id`** appears at least once in day 1's log **and** at least once in day 2's log; and 2. Across all of its appearances in both days combined, it is associated with at least **two distinct `client_id`** values.

Tasks

1. Design and implement a function that, given file handles (or streaming iterators) for the two log files, returns the set of all such interesting **`obj_id`**s.
2. For your initial, straightforward in-memory solution (you may assume both logs fit in memory), state the time and space complexity in terms of the total number of log records.
3. Follow-up (memory constrained): Now assume the logs are very large and **cannot** both fit into memory at once.

- You may still sequentially scan each file and you may perform external sorting.
- Describe how you would modify your approach to work under this memory constraint.
- Analyze the time and space complexity of this memory-efficient solution.

Return sorted obj_ids that appear in both day logs and have at least two distinct client_ids across both days combined.
Constraints
Rows are [timestamp, obj_id, client_id] or dicts with those fields

EXAMPLES

```
Input: ([['t1', 'o1', 'c1'], ['t2', 'o2', 'c1']], [['t3', 'o1', 'c2'], ['t4', 'o2', 'c1']])
Expected Output: ['o1']
Input: ([], [['t', 'o', 'c']])
Expected Output: []
```

Hints

Track which days each object appears in and its set of clients.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(file_a, file_b):
    shared = set()
    for a in file_a:
        for b in file_b:
            if a.obj_id == b.obj_id:
                shared.add(a.obj_id)
    return shared
```

TIME	SPACE
O(n * m)	**O(1)** extra

WHY IT IS SLOW

O(n * m) comparisons. Loading the smaller file's ids into a hash set and streaming the other file is O(n + m); if neither fits, partition both by a hash of the id.

OPTIMAL SOLUTION · Hash set of the smaller file's ids

TIME	SPACE
O(n + m)	**O(distinct ids of the smaller file)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Both files are too big for memory.

Hash-partition both files into B buckets on disk by hash of the object id, then process bucket pairs one at a time with the in-memory set join. Choose B so that one bucket of the smaller file fits in memory. External sort and merge also works.

FOLLOW-UP 2 Want objects seen on both days within an hour of each other.

Keep a list of timestamps per id in the set join and compare timestamps (sorted lists with two pointers), instead of just membership.

FOLLOW-UP 3 A tiny memory budget and an approximate answer is fine.

Build a Bloom filter from one file and test the other against it, accepting a small false-positive rate; verify the few hits exactly in a second pass if needed. **Cost:** O(n + m) time with O(bits) memory for the Bloom filter and a small false-positive rate.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find shared objects across two log files
#
# You are given two large log files representing activity
# on two different days. Each line of each log has three
# fields: - timestamp: a time value (you may treat it as
# an opaque string or integer) - obj_id: identifier of an
# object (string or integer) - client_id: identifier of a
# client (string or integer) The logs are not necessarily
```

```
# sorted by any field. Define an object as **interesting**
# if: 1. Its obj_id appears at least once in day 1's log
# **and** at least once in day 2's log; and 2. Across all
# of its appearances in both days combined, it is
# associated with at least **two distinct** client_id
# values. ### Tasks 1. Design and implement a function
# that, given file handles (or streaming iterators) for
# the two log files, returns the set of all such
# interesting obj_ids. 2.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Both files are too big for memory.
# 2. Want objects seen on both days within an hour of each
# other.
# 3. A tiny memory budget and an approximate answer is
# fine.
#
# INTERVIEW NOTES
# A common key in two files is a join. If one file fits in
# memory, load its keys into a set and stream the other.
# If neither fits, split both files by hash of the key
# into buckets small enough for memory; equal keys always
# land in the same bucket number, so you only compare
# bucket i of one file with bucket i of the other (Grace
# hash join).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import zlib


# ============================================================
```

```
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def object_of(row):
    return row[1]                                   # a row is [timestamp, object id, client id]


# ---- main question: main function ----
# Idea: an object is "shared" when it appears on BOTH days and was seen by
# at least 2 different clients over the two days. Go through every row once
# and remember, for each object, which days it appeared on and which
# clients saw it. Each line is read once; memory depends on the number of
# distinct objects.
# Complexity: Time O(n + m) rows, Space O(distinct objects + clients)
def shared_objects(day1, day2):
    days = {}                                       # object -> the set of days it appeared on
    clients = {}                                    # object -> the set of clients that saw it
    for number, rows in ((1, day1), (2, day2)):
        for timestamp, obj, client in rows:
            if obj not in days:
                days[obj] = set()
                clients[obj] = set()
            days[obj].add(number)
            clients[obj].add(client)
    result = []
    for obj in days:
        if len(days[obj]) == 2 and len(clients[obj]) >= 2:
```

```
            result.append(obj)
    return sorted(result)


# the other way: sort both days by object, then merge them like two sorted
# lists, holding the clients of only ONE object at a time
# Complexity: Time O((n + m) log(n + m)) (the sorts), Space O(n + m) (O(1) extra per object when the files
are already sorted)
def shared_objects_sorted(day1, day2):
    a = sorted(day1, key=object_of)
    b = sorted(day2, key=object_of)
    i = 0
    j = 0
    result = []
    while i < len(a) and j < len(b):
        if a[i][1] < b[j][1]:
            i += 1
        elif a[i][1] > b[j][1]:
            j += 1
```

```
        else:
            obj = a[i][1]
            clients = set()
            while i < len(a) and a[i][1] == obj:
                clients.add(a[i][2])
                i += 1
            while j < len(b) and b[j][1] == obj:
                clients.add(b[j][2])
                j += 1
            if len(clients) >= 2:
                result.append(obj)
    return result


# ---- main question: tests ----
D1 = [["t1", "o1", "c1"], ["t2", "o2", "c1"]]
D2 = [["t3", "o1", "c2"], ["t4", "o2", "c1"]]


def test_main_question():
    assert shared_objects(D1, D2) == ["o1"]
    assert shared_objects([], [["t", "o", "c"]]) == []
    assert shared_objects_sorted(D1, D2) == ["o1"]


# ========================================================
# FOLLOW-UP 1: Both files are too big for memory
# ========================================================
# ---- follow-up 1: helper functions ----
def bucket_of(obj, buckets):
    # a STABLE hash: Python's own hash() changes between runs
    return zlib.crc32(str(obj).encode()) % buckets


# ---- follow-up 1: function ----
# Answer: hash-partition both files into B buckets (on disk) by the hash
# of the object id, then process one pair of buckets at a time with the
# in-memory join. All rows of an object land in the same bucket. Choose B
# so that one bucket of the smaller file fits in memory.
# Complexity: Time O(n + m) in total, Space O(n + m) on disk and one bucket in memory
def shared_objects_partitioned(day1, day2, buckets=4):
    parts1 = []
    parts2 = []
    for b in range(buckets):
        parts1.append([])
```

```
        parts2.append([])
    for row in day1:
        parts1[bucket_of(row[1], buckets)].append(row)
    for row in day2:
        parts2[bucket_of(row[1], buckets)].append(row)
    result = []
    for b in range(buckets):
        result += shared_objects(parts1[b], parts2[b])
    return sorted(result)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert shared_objects_partitioned(D1, D2, 3) == ["o1"]
    many1 = []
    many2 = []
    for i in range(40):
        many1.append([i, "o" + str(i % 7), "c" + str(i % 3)])
        many2.append([i, "o" + str(i % 5), "c" + str(i % 2)])
    expected = shared_objects(many1, many2)
    assert shared_objects_partitioned(many1, many2, 5) == expected
    assert shared_objects_sorted(many1, many2) == expected


# ============================================================
# FOLLOW-UP 2: Want objects seen on both days within an hour of each other
# ============================================================
# Answer: sort each day's rows by (object, timestamp) and merge the two
# lists with two pointers; for one object keep a pointer into the other
# day's timestamps and move it while the gap is more than an hour.


# ============================================================
# FOLLOW-UP 3: A tiny memory budget and an approximate answer is fine
# ============================================================
# ---- follow-up 3: function ----
# Answer: build a Bloom filter from one file's object ids (a bit array and a
# few hash functions) and test the other file's ids against it. It may say
# "yes" for a few objects that were never added (a small false-positive
# rate) but never "no" for one that was. Verify the few hits exactly in a
# second pass if needed.
# Complexity: Time add and maybe are O(h) for h hash functions, Space O(bits)
class Bloom:
    def __init__(self, bits=1 << 16, hashes=3):
        self.bits = bytearray(bits)
```

```
        self.hashes = hashes

    def spots(self, key):
        places = []
        for seed in range(self.hashes):
            text = (str(seed) + ":" + str(key)).encode()
            places.append(zlib.crc32(text) % len(self.bits))
        return places

    def add(self, key):
        for spot in self.spots(key):
            self.bits[spot] = 1

    def maybe(self, key):
        for spot in self.spots(key):
            if self.bits[spot] == 0:
                return False
        return True
```

```python
# ---- follow-up 3: tests ----
def test_follow_up_3():
    bloom = Bloom()
    for row in D1:
        bloom.add(row[1])
    assert bloom.maybe("o1") and bloom.maybe("o2")
    assert not bloom.maybe("o999")


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

ALSO REPORTED · 1 more

Two-Day Log Intersection under Memory Limit | 1Point3Acres | onsite-coding | asked 2025-11-27 | Single report

https://www.1point3acres.com/interview/problems/company/google/two-logs-intersection-memory-limit

In-memory solution: build a hashmap obj_id -> set(client_id) while scanning each file; intersect keys across the two hashmaps at the end.
Memory-limited solution: external sort each file by obj_id, then run a merge / two-pointer scan over both sorted files. For each matching obj_id group, count distinct client_id on the fly (HyperLogLog if duplicates are large).
The interviewer explicitly preferred sort + two-pointer over a streaming hashmap — be prepared to defend the choice (sort is O(n log n) on disk vs O(n) RAM-bound for the hashmap).
For distributed scale, hash-partition by obj_id and run the same algorithm per partition.

Preparation: Practice external-sort / merge-step problems (classic in-memory merge / k-way-merge / partition problems — and discuss disk-based variants verbally).
Drill the streaming hashmap vs external sort trade-off conversation.
Have ready: HyperLogLog (or Count-Min Sketch) one-liner for approximate distinct-count under tight memory.

STUDY PATH: question 132 of 197 | topic 25 of 43: Look at the input once and keep only a small summary (and count your calls) | 4 of 4

Template fit: the same group testing: put a whole group in one call and split only the groups that fail | * asked 349 days ago *

MEDIUM | PracHub | Technical Screen | 132 of 197

Minimize calls to find all bad test pairs

POSTED 2025-11-05 | 349 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 254/288

ABOUT THIS QUESTION

Topic: Look at the input once and keep only a small summary (and count your calls)

Optimal answer from: the interview site's own solution

LeetCode match: No direct match (adaptive group testing)

Posted on PracHub: 2025-11-05 (updated 2026-06-18)

Original: https://prachub.com/coding-questions/minimize-calls-to-find-all-bad-test-pairs

QUESTION

You have (n) atomic tests (t_1, t_2, \dots, t_n). You are given access to a black-box function:

```
bool runTest(set<Test> S)
```

which behaves as follows:

- **`runTest(S) == True`** if the subset (S) contains **no** incompatible (bad) pair of tests.
- **`runTest(S) == False`** if the subset (S) contains **at least one** incompatible (bad) pair of tests. There is some unknown set of unordered **bad pairs** ((t_i, t_j)), where (i\ne j). You do **not** know in advance which pairs are bad or how many bad pairs there are. Your goal is to discover **all** bad pairs while making as few calls to **`runTest`** as possible. You may adaptively choose each subset (S) based on the results of previous calls.

Tasks

1. Design an algorithm that, using only calls to **`runTest`**, finds **all** bad pairs ((t_i, t_j)).
2. Analyze the number of calls to **`runTest`** your algorithm requires, as a function of:

- (n): the total number of tests, and - (k): the total number of bad pairs.

3. Argue why your algorithm is correct (i.e., why it will eventually find every bad pair and not report any good pair as bad). You do not need to provide full production code, but describe the algorithm clearly enough that it could be implemented.

You have n atomic tests numbered 0 to n - 1. Some unknown unordered pairs of tests are incompatible, called bad pairs. A black-box oracle runTest(S) returns True if the subset S contains no bad pair, and False if S contains at least one bad pair. In this non-interactive version, you are given bad_pairs only so your code can simulate the oracle. Implement the following deterministic oracle-discovery strategy and return both the bad pairs it discovers and the number of oracle calls it makes. Strategy:

1. Process vertices from right to left: v = n - 1, n - 2, ..., 0.
2. Maintain all bad pairs already discovered among vertices greater than the current v.
3. For the suffix vertices v + 1 through n - 1, greedily partition them into color classes such that no discovered bad pair appears inside one color class. Since all suffix-suffix bad pairs have already been discovered, every color class is truly compatible internally.
4. For each color class C, find all bad partners of v inside C using recursive group testing:

- Call runTest({v} union C).
- If the oracle returns True, v has no bad pair with any test in C.

QUESTION

- If C has size 1 and the oracle returns False, that single pair is bad.
- Otherwise split C into two halves and recurse on both halves. Return all discovered bad pairs sorted lexicographically, where each pair is represented as [i, j] with i < j, along with the exact number of runTest calls made by this strategy.

Constraints
0 <= n <= 5000 <= len(bad_pairs) <= n * (n - 1) / 2Each bad pair contains two distinct test indices in the range [0, n - 1]bad_pairs may be given in any endpoint order

EXAMPLES

```
Input: (0, [])
Expected Output: ([], 0)
```

Explanation: There are no tests and therefore no oracle calls are needed.

```
Input: (4, [])
Expected Output: ([], 3)
```

Explanation: For v = 2, 1, and 0, the algorithm queries the compatible suffix once and finds no bad pairs. v = 3 has an empty suffix.

Hints

If a candidate set C is internally compatible, then runTest({v} union C) tells you whether v conflicts with at least one element of C.Process tests from right to left so that when handling v, all bad pairs inside the suffix v + 1 ... n - 1 are already known and can be used to build compatible groups.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
bad = []
for a, b in combinations(tests, 2):
    if not runTest({a, b}):
        bad.append((a, b))
```

TIME	SPACE
n(n - 1) / 2 calls	O(1)

WHY IT IS SLOW

n(n - 1) / 2 calls even when nothing is bad. Keeping groups of mutually compatible tests and binary searching only when a group fails needs about n + p log n calls for p bad pairs.

OPTIMAL SOLUTION · Independent groups plus binary search for partners

TIME	SPACE
O(n * g + p log n) calls (g = number of groups, p = number of bad pairs)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How many calls does it need, and can you prove a lower bound?

With p bad pairs among n tests about p log n calls are needed in the worst case (each pair carries log n bits of information about where it is), and the group method uses O(n * g + p log n). With no bad pairs it uses n - 1 calls, one per test after the first.

FOLLOW-UP 2 Only one bad pair exists.

Binary search for the smallest failing prefix (its last element is in the pair), then binary search among the earlier elements for the partner: about 2 log n calls.

FOLLOW-UP 3 runTest is expensive but can run in parallel.

Test all groups for the new element in the same round, or process several elements at once with their own groups, trading a few extra calls for fewer sequential rounds.

FOLLOW-UP 4 Only one bad pair exists.

Binary search the shortest failing prefix (its last element is in the pair), then binary search the earlier tests for the partner: about 2 log2(n) calls instead of n(n - 1) / 2 (verified by counting calls).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Minimize calls to find all bad test pairs
#
# You have \(n\) atomic tests \(t_1, t_2, \dots, t_n\).
# You are given access to a black-box function: python
# bool runTest(set<Test> S) which behaves as follows: -
# runTest(S) == True if the subset \(S\) contains **no**
# incompatible (bad) pair of tests. - runTest(S) == False
# if the subset \(S\) contains **at least one**
# incompatible (bad) pair of tests. There is some unknown
# set of unordered **bad pairs** \((t_i, t_j)\), where
# \(i\ne j\). You do **not** know in advance which pairs
# are bad or how many bad pairs there are. Your goal is to
# discover **all** bad pairs while making as few calls to
# runTest as possible. You may adaptively choose each
# subset \(S\) based on the results of previous calls. ###
# Tasks 1. Design an algorithm that, using only calls to
# runTest, finds **all** bad pairs \((t_i, t_j)\). 2.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How many calls does it need, and can you prove a
# lower bound?
# 2. Only one bad pair exists.
# 3. runTest is expensive but can run in parallel.
# 4. Only one bad pair exists.
#
# INTERVIEW NOTES
# Each call is a yes/no question about a whole set, so
# binary search is possible: a failing set can be narrowed
# by halving. Keep the tests you have processed in groups
# that each pass runTest (independent sets). A new test
# only needs to be checked against each group, and if a
# group plus the new test fails, binary search finds its
# partners in that group in about log(size) calls each.
```

```
# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: helper functions ----
```

```
# run_test(set_of_tests) is the expensive oracle: it returns True when
# the tests in the set pass TOGETHER (no bad pair inside it).

def shortest_failing_prefix(items, extra, run_test):
    # binary search the smallest prefix of `items` that fails together
    # with the tests in `extra`; its LAST item is a partner of something
    low = 1
    high = len(items)
    while low < high:
        mid = (low + high) // 2
        if run_test(set(items[:mid]) | extra):
            low = mid + 1                       # still passes: the culprit is later
        else:
            high = mid
    return items[low - 1]


def partners(group, t, run_test):
    # every member of the group that is bad together with t
    rest = list(group)
    found = []
    while len(rest) > 0 and not run_test(set(rest) | {t}):
        x = shortest_failing_prefix(rest, {t}, run_test)
        found.append(x)
        rest.remove(x)                          # take it out and test the others again
    return found


# ---- main question: main function ----
# Idea: keep GROUPS of tests that pass together. For each new test t: ask
# each group for its partners with t (one call tells us when there are
# none; otherwise a binary search finds each partner in about log n
# calls). t joins the first group in which it has no partner, or starts a
# new group. Returns the set of bad pairs (smaller test, bigger test).
# Complexity: Time O((n + p log n) * cost of runTest) calls: about n + p log n runTest calls for p bad pair
s, Space O(n)
def find_bad_pairs(tests, run_test):
    groups = []
    pairs = set()
    for t in tests:
        placed = False
        for group in groups:
            found = partners(group, t, run_test)
            for x in found:
                if x < t:
                    pairs.add((x, t))
```

```
                else:
                    pairs.add((t, x))
            if len(found) == 0 and not placed:
                group.append(t)
                placed = True
```

```
        if not placed:
            groups.append([t])
    return pairs


# ---- main question: tests ----
class Oracle:
    # a pretend run_test that knows the bad pairs and counts its calls
    def __init__(self, bad):
        self.bad = bad
        self.calls = 0

    def __call__(self, subset):
        self.calls += 1
        ordered = sorted(subset)
        for i in range(len(ordered)):
            for j in range(i + 1, len(ordered)):
                if (ordered[i], ordered[j]) in self.bad:
                    return False
        return True


def test_main_question():
    bad = {(3, 17), (5, 29)}
    oracle = Oracle(bad)
    assert find_bad_pairs(list(range(30)), oracle) == bad
    assert oracle.calls < 435 // 2                      # far fewer than the 435 pairs
    oracle = Oracle(set())
    assert find_bad_pairs(list(range(30)), oracle) == set()
    assert oracle.calls <= 30                           # no bad pairs: about one call per test


# ============================================================
# FOLLOW-UP 1: How many calls does it need, and can you prove a lower bound?
# ============================================================
# Answer: with p bad pairs about p * log2(n) calls plus about n calls for
# the checks that find nothing. A lower bound: there are about n^2 / 2
# possible pairs, each call gives one yes/no answer, so at least
# log2(n^2 / 2), about 2 * log2(n) calls are needed even for ONE bad pair.
```

```
# ============================================================
# FOLLOW-UP 2: Only one bad pair exists
# ============================================================


# ============================================================
# FOLLOW-UP 4: Only one bad pair exists
# ============================================================
# ---- follow-up 2: function ----
# Answer: binary search the shortest failing prefix of the tests (its last
# test belongs to the pair), then binary search the EARLIER tests for the
# partner: about 2 * log2(n) calls instead of n * (n - 1) / 2.
# Complexity: Time about 2 log2(n) runTest calls, Space O(1)
def one_bad_pair(tests, run_test):
    if run_test(set(tests)):
        return None                                     # everything passes: no bad pair
    b = shortest_failing_prefix(tests, set(), run_test)
    earlier = tests[:tests.index(b)]
    a = shortest_failing_prefix(earlier, {b}, run_test)
    return (a, b)


# ---- follow-up 2: tests ----
def test_follow_up_2():
```

```
    oracle = Oracle({(10, 40)})
    assert one_bad_pair(list(range(64)), oracle) == (10, 40)
    assert oracle.calls <= 16                          # about 2 * log2(64) = 12, far below 2016 pairs
    assert one_bad_pair(list(range(10)), Oracle(set())) is None


# ======================================================
# FOLLOW-UP 3: runTest is expensive but can run in parallel
# ======================================================
# Answer: ask several questions at once: test many groups in parallel
# (for example every group against the new test t at the same time), and
# split a failing group into several parts instead of two. You use more
# calls in total but fewer rounds one after another.


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(n: int, bad_pairs: list[list[int]]) -> tuple[list[list[int]], int]:
```

Sample call

```
solution(0, [])
```

Test cases

Input	Expected
`(0, [])`	`([], 0)`
`(4, [])`	`([], 3)`

ALSO REPORTED · 1 more

Find All Bad Pairs Using runTest Oracle | 1Point3Acres | onsite-coding | asked 2025-11-27 | Single report

https://www.1point3acres.com/interview/problems/company/google/find-bad-pairs-with-oracle

Standard "group testing" / Dorfman testing problem. Walk through the analogy with COVID pooled tests for intuition.
Algorithm sketch:
Test the full set. If true, no bad pairs.
Otherwise, binary-search to find one bad pair by partitioning the set and testing halves.
Remove the items that participate in the located bad pair; repeat.
Tighter bound: with up to K bad pairs, total queries are O(K log N) using adaptive group testing.
A naive $O(N^2)$ pairwise scheme works but the interviewer expects you to beat it with adaptive splitting.
Verbalize correctness: each binary-search step rules out half the candidates; you can always reduce the search space until exactly one bad pair is isolated.

ALSO REPORTED · 1 more

Preparation: Read the Wikipedia article on group testing and Dorfman's two-stage scheme.
Drill recursive binary partitioning on a candidate set with positive/negative oracle feedback.
Have a back-of-envelope analysis ready: "Total calls O(K log N) for K bad pairs and N items."

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 26 OF 43

Grid DP: each cell from its neighbours above and to the left

3 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Grid DP: each cell from its neighbours above and to the left | 3 questions | tap a title to jump to it

READ FIRST: the code template for these 3 questions (tap here) >

M	133. Grid Path Count with Diagonal Moves and Checkpoints	1Point3Acres * 2026-09-29 *
M	134. Longest Same-Color Grid Sequence	1Point3Acres * 2026-09-08 *
M	135. Count Same-Color Squares in Matrix	ND

CODE TEMPLATE | GROUP 26 OF 43 | 3 QUESTIONS, ONE PATTERN

Grid DP: each cell from its neighbours above and to the left

The one idea these questions share

Walk the grid row by row and keep only the previous row. A cell's value depends on the cell above, the cell to the left, and the diagonal. Change the condition for a cell to count and the formula that combines the three neighbours.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def grid_dp(g):
    m, n = len(g), len(g[0])
    prev, total = [0] * n, 0
    for i in range(m):
        cur = [1] * n
        for j in range(n):
            if i and j and g[i][j] == g[i - 1][j] == g[i][j - 1] == g[i - 1][j - 1]:   # CHANGE 1
                cur[j] = 1 + min(prev[j], cur[j - 1], prev[j - 1])                      # CHANGE 2
        total += sum(cur)                                                               # CHANGE 3
        prev = cur
    return total
```

2. Helper functions that usually get written next to it

```
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))      # the four line directions on a grid


def inside(r, c, rows, cols):
    return 0 <= r < rows and 0 <= c < cols
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
133	Grid Path Count with Diagonal Moves and Checkpoints	advance column by column keeping the number of ways in each row; new[r] = old[r-1] + old[r] + old[r+1]; zero out rows that miss a checkpoint
134	Longest Same-Color Grid Sequence	keep four run lengths per cell (horizontal, vertical, two diagonals) from the previous row and the left cell; the answer is the largest
135	Count Same-Color Squares in Matrix	the template as written: a square ends here if the three neighbours match; its size is 1 + min of their sizes

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 133 of 197 | topic 26 of 43: Grid DP: each cell from its neighbours above and to the left | 1 of 3

Template fit: advance column by column keeping the number of ways in each row; new[r] = old[r-1] + old[r] + old[r+1]; zero out rows that miss a checkpoint | * asked 21 days ago * | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 133 of 197

Grid Path Count with Diagonal Moves and Checkpoints

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | **ALL TIME 48.9** | **30 DAYS 33** | **PRIORITY 42/288**

ABOUT THIS QUESTION

Topic: Grid DP: each cell from its neighbours above and to the left

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #62 Unique Paths

LeetCode match: LC 62 Unique Paths (related)

Frequency: Single report | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/grid-path-count-three-moves-checkpoints

QUESTION

Count the paths in an n x m grid from the bottom-left cell (n - 1, 0) to the bottom-right cell (n - 1, m - 1). From (r, c) you may move up-right (r - 1, c + 1), right (r, c + 1) or down-right (r + 1, c + 1) and must stay inside the grid. Optionally you are given checkpoint cells that every path has to pass through; with ordered=True they must be visited in the order given.

Function signature

def count_paths(n: int, m: int, checkpoints=(), ordered=False) -> int

EXAMPLES

Example 1	
Input	`3, 3`
Output	`2`
Why	`Two paths: through (1, 1) and then down-right, or through (2, 1) and then right.`

Example 2	
Input	`3, 3, checkpoints = [(1, 1)]`
Output	`1`
Why	`The path must also pass through (1, 1), the middle cell.`

Example 3	
Input	`3, 4, checkpoints = [(1, 1), (2, 2)], ordered = true`
Output	`1`
Why	`Two checkpoints in different columns, visited in column order.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def count(r, c, seen):
    if c == m - 1:
        done = r == n - 1 and checkpoints <= seen
        return 1 if done else 0
    total = 0
    for dr in (-1, 0, 1):
        nr = r + dr
        if 0 <= nr < n:
            nxt = seen | {(nr, c + 1)}
            total += count(nr, c + 1, nxt)
    return total
answer = count(n - 1, 0, {(n - 1, 0)})
```

TIME	SPACE
O(3^m)	O(m)

WHY IT IS SLOW

Up to 3^m paths. Every move advances one column, so a dp over (column, row) with a rolling array counts them in O(n * m).

OPTIMAL SOLUTION · Column DP with a rolling array and checkpoint filters

TIME	SPACE
O(n * m)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Two checkpoints share a column.

No path can visit both, because a path is in exactly one row per column, so the answer is 0 (the code returns it before the DP). Checkpoints at the start or end cell are fine: they only filter rows of the first or last column.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Grid Path Count with Diagonal Moves and
# Checkpoints
#
# Count the paths in an n x m grid from the bottom-left
# cell (n - 1, 0) to the bottom-right cell (n - 1, m - 1).
# From (r, c) you may move up-right (r - 1, c + 1), right
# (r, c + 1) or down-right (r + 1, c + 1) and must stay
# inside the grid. Optionally you are given checkpoint
# cells that every path has to pass through; with
# ordered=True they must be visited in the order given.
#
# Signature:
#   def count_paths(n: int, m: int, checkpoints=(), ordered=False) -> int
#
# Examples:
# 3, 3  ->  2
# 3, 3, checkpoints = [(1, 1)]  ->  1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Two checkpoints share a column.
```

```
#
# INTERVIEW NOTES
# Every move goes exactly one column to the right, so a
# path visits each column exactly once and the state is
# just 'which row am I in at this column'. That is a DP
# over columns with a single rolling array. A checkpoint
# (r, c) simply says the path must be in row r at column
# c, so you zero every other row of that column.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def step(ways):
    # the next column from the current column only: a path in row r came
    # from row r - 1, r or r + 1 of the column before (up-right, right, down-right)
    n = len(ways)
    following = []
    for r in range(n):
        total = ways[r]
        if r > 0:
            total += ways[r - 1]
        if r + 1 < n:
            total += ways[r + 1]
        following.append(total)
    return following


def keep_row(ways, row):
    # a checkpoint forces the path through one row of this column
    kept = []
    for r in range(len(ways)):
        if r == row:
            kept.append(ways[r])
        else:
            kept.append(0)
    return kept


def checkpoint_rows(checkpoints, ordered):
    # column -> the row the path must be in; None if that is impossible
    rows = {}
    for r, c in checkpoints:
        if c in rows and rows[c] != r:
            return None                          # two different checkpoints in one column: no path can vis
it both
        rows[c] = r
    if len(rows) != len(checkpoints):
        return None                              # the same checkpoint listed twice
    if ordered:
        for i in range(len(checkpoints) - 1):
            if checkpoints[i][1] > checkpoints[i + 1][1]:
                return None                      # ordered checkpoints must be listed left to right
    return rows


# ---- main question: main function ----
```

```
# A grid with n rows and m columns. A path starts at the bottom-left
# cell, ends at the bottom-right cell and each step moves to the next
# column in the same row or one row up or down (a diagonal). Count the
# paths that pass through all the checkpoints (row, column).
# Idea: ways[r] = the number of paths that are in row r at the current
# column. The next column is ways'[r] = ways[r - 1] + ways[r] + ways[r + 1].
# At a checkpoint column keep only the checkpoint's row. One rolling
```

```
# column is enough: O(n) memory. (Two checkpoints in one column: the
# answer is 0, since a path is in exactly one row per column: follow-up 1.)
# Complexity: Time O(n * m + checkpoints), Space O(n)
def count_paths(n, m, checkpoints=(), ordered=False):
    rows = checkpoint_rows(list(checkpoints), ordered)
    if rows is None:
        return 0
    ways = [0] * n
    ways[n - 1] = 1
    if 0 in rows:
        ways = keep_row(ways, rows[0])
    for c in range(1, m):
        ways = step(ways)
        if c in rows:
            ways = keep_row(ways, rows[c])
    return ways[n - 1]


# ---- main question: tests ----
def count_paths_table(n, m):
    # the whole table ways[c][r], to check the rolling version against
    ways = []
    for c in range(m):
        ways.append([0] * n)
    ways[0][n - 1] = 1
    for c in range(1, m):
        for r in range(n):
            for previous_row in (r - 1, r, r + 1):
                if 0 <= previous_row < n:
                    ways[c][r] += ways[c - 1][previous_row]
    return ways[m - 1][n - 1]


def test_main_question():
    assert count_paths(3, 3) == 2
    assert count_paths_table(3, 3) == 2
    assert count_paths(1, 5) == 1
    assert count_paths(3, 1) == 1
    assert count_paths_table(4, 6) == count_paths(4, 6)
    assert count_paths(3, 3, [(1, 1)]) == 1                    # force the middle column through row 1
    assert count_paths(3, 4, [(1, 1), (1, 2)], True) == 1      # ordered checkpoints: left to right
    assert count_paths(3, 4, [(1, 2), (1, 1)], True) == 0


# ========================================================
```

```
# FOLLOW-UP 1: Two checkpoints share a column
# ========================================================
# Answer: no path can visit both (a path is in exactly one row per
# column), so the answer is 0; checkpoint_rows returns None before the DP
# even starts. Checkpoints at the start or end cell are fine: they only
# filter the rows of the first or the last column.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert count_paths(3, 3, [(0, 1), (1, 1)]) == 0
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write the column-by-column DP where dp[c][r] depends only on dp[c-1][r-1..r+1], then reduce it to a single rolling column.
For checkpoints, practice the "segment" approach: sort by column, and between consecutive checkpoints only allow paths that pass through them (zero out other rows in a checkpoint's column).
Test: n = 1 (only straight right), m = 1, a checkpoint at the start or end, and two checkpoints in the same column.

STUDY PATH: question 134 of 197 | topic 26 of 43: Grid DP: each cell from its neighbours above and to the left | 2 of 3

Template fit: keep four run lengths per cell (horizontal, vertical, two diagonals) from the previous row and the left cell; the answer is the largest | * asked 42 days ago * | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 134 of 197

Longest Same-Color Grid Sequence

ASKED 2026-09-08 | 42 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 19.6 30 DAYS 0 PRIORITY 169/288

ABOUT THIS QUESTION

Topic: Grid DP: each cell from its neighbours above and to the left

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 562 Longest Line of Consecutive One in Matrix (related)

Frequency: Single report | Last asked: 2026-09-08 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/longest-same-color-grid-line

QUESTION

Given a grid of colours (letters such as R, G, B), return the length of the longest run of one colour along a straight line: horizontal, vertical, or diagonal (in either diagonal direction).

Function signature

def longest_same_color_line(g: list) -> int

EXAMPLES

Example 1	
Input	`['RRRG', 'RRBB', 'GRRR', 'BGBR']`
Output	`4`
Why	`The main diagonal is R, R, R, R from corner to corner: 4. No other line is longer.`

Example 2	
Input	`['RGB', 'BGR', 'RGB']`
Output	`3`
Why	`The longest line is the vertical run of three G in the middle column.`

Example 3	
Input	`['RR', 'RR']`
Output	`2`
Why	`A single colour fills the grid: the longest line is a full row or column.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(grid):
    best = 0
    for r, c in all_cells(grid):
        for dr, dc in DIRECTIONS:
            n = 1
            while same(grid, r, c,
                       r + n * dr, c + n * dc):
                n += 1
            best = max(best, n)
    return best
```

TIME	SPACE
O(m * n * max(m, n))	O(1)

WHY IT IS SLOW

Re-walks the same cells in every line, O(m * n * max(m, n)). Keeping the run length ending at each cell in each direction is O(m * n).

OPTIMAL SOLUTION · Four run lengths per cell

TIME	SPACE
O(m * n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The line may also bend (a connected region).

Then it is a different problem: the largest connected same-colour region, found with BFS/DFS or union-find in O(m * n). The run-length DP only handles straight lines because each run depends on exactly one predecessor.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Longest Same-Color Grid Sequence
#
# Given a grid of colours (letters such as R, G, B),
# return the length of the longest run of one colour along
# a straight line: horizontal, vertical, or diagonal (in
# either diagonal direction).
#
# Signature:
#   def longest_same_color_line(g: list) -> int
#
# Examples:
# ['RRRG', 'RRBB', 'GRRR', 'BGBR']  ->  4
# ['RGB', 'BGR', 'RGB']  ->  3
#
# FOLLOW-UPS (answered below, in this order):
# 1. The line may also bend (a connected region).
#
# INTERVIEW NOTES
# A straight line can run in four directions (right, down,
# down-right, down-left). For each cell and each direction
# keep the length of the same-colour run that ends there:
```

```
# it is the predecessor's run plus one if the colours
# match, otherwise 1. Because every run only looks one
# cell back in a fixed direction, one pass in reading
# order is enough.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: the longest straight line of one colour (across, down, or along a
# diagonal). Keep, for the PREVIOUS row, the length of the run that ends
# at each column going straight down, down-left and down-right, and a
# running number for the run across the current row. A cell extends the
# run of the cell above it (or above-left / above-right) if the colour
# matches, otherwise it starts a new run of 1.
# Complexity: Time O(m * n), Space O(n) (one row at a time)
def longest_same_color_line(g):
    if not g or not g[0]:
        return 0
    n = len(g[0])
    best = 0
    down = [0] * n                          # runs ending in the previous row ...
    down_left = [0] * n                     # (coming from the above-right cell)
    down_right = [0] * n                    # (coming from the above-left cell)
    for i in range(len(g)):
        row = g[i]
        down_now = [1] * n
        left_now = [1] * n
        right_now = [1] * n
        across = 0
        for j in range(n):
            color = row[j]
            if j > 0 and row[j - 1] == color:
                across += 1                 # continue the run across this row
            else:
                across = 1
            if i > 0:
                if g[i - 1][j] == color:
                    down_now[j] = down[j] + 1
                if j > 0 and g[i - 1][j - 1] == color:
                    right_now[j] = down_right[j - 1] + 1
                if j + 1 < n and g[i - 1][j + 1] == color:
                    left_now[j] = down_left[j + 1] + 1
            for length in (across, down_now[j], left_now[j], right_now[j]):
                if length > best:
                    best = length
        down = down_now
        down_left = left_now
        down_right = right_now
    return best


# ---- main question: tests ----
def test_main_question():
    assert longest_same_color_line(["RRRG", "RRBB", "GRRR", "BGBR"]) == 4
    assert longest_same_color_line([]) == 0
    assert longest_same_color_line(["R", "R", "R"]) == 3              # one column
```

```
    assert longest_same_color_line(["RGR", "GRG"]) == 2               # alternating colours
    assert longest_same_color_line(["RRR", "GGR"]) == 3


# =====================================================
```

```
# FOLLOW-UP 1: The line may also bend (a connected region)
# =====================================================
# ---- follow-up 1: function ----
# Answer: that is a different problem: the largest CONNECTED region of one
# colour, found with a flood fill (DFS or BFS) or union-find in O(m * n).
# The run-length DP only handles straight lines because each run depends
# on exactly one earlier cell.
# Complexity: Time O(m * n), Space O(m * n)
def largest_region(g):
    seen = set()
    best = 0
    for r in range(len(g)):
        for c in range(len(g[0])):
            if (r, c) in seen:
                continue
            seen.add((r, c))
            stack = [(r, c)]
            size = 0
            while stack:
                x, y = stack.pop()
                size += 1
                for a, b in ((x + 1, y), (x - 1, y), (x, y + 1), (x, y - 1)):
                    if a < 0 or b < 0 or a >= len(g) or b >= len(g[0]):
                        continue
                    if (a, b) not in seen and g[a][b] == g[r][c]:
                        seen.add((a, b))
                        stack.append((a, b))
            if size > best:
                best = size
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert largest_region(["RR", "GR"]) == 3
    assert largest_region(["RRR", "GGR"]) == 4               # a bent region is longer than any straight li
ne


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.
Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

STUDY PATH: question 135 of 197 | topic 26 of 43: Grid DP: each cell from its neighbours above and to the left | 3 of 3

Template fit: the template as written: a square ends here if the three neighbours match; its size is 1 + min of their sizes | ND | frequency Low

MEDIUM | 1Point3Acres | onsite-coding | 135 of 197

Count Same-Color Squares in Matrix

STUDY: TYPE **ALL TIME 51.2** **30 DAYS 0** **PRIORITY 78/288**

ABOUT THIS QUESTION

Topic: Grid DP: each cell from its neighbours above and to the left

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #221 Maximal Square

LeetCode match: LC 221 Maximal Square (related)

Frequency: Low | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/count-same-color-squares

QUESTION

LeetCode 221 variant — count maximal same-color square sub-grids. DP on each cell as the bottom-right corner.

Input: m × n grid of colored cells.

Output: count of square sub-grids whose cells are all the same color (or maximum side length, depending on prompt).

DP: dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) if grid[i][j] == grid[i-1][j] == grid[i][j-1] == grid[i-1][j-1] else 1.

Sum dp[i][j] across the grid for the count variant; max for the side variant.

EXAMPLES

The plain "count all-1 squares" DP generalizes to same-color directly.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
count = 0
for i, j in all_cells(grid):
    for k in range(1, min(m - i, n - j) + 1):
        cells = square(grid, i, j, k)
        if all(c == grid[i][j] for c in cells):
            count += 1
```

TIME	SPACE
O(m * n * min(m, n)^3)	O(1)

WHY IT IS SLOW

Re-checks the same cells again and again, O(m * n * min(m, n)^3). A dp on the bottom-right corner reuses the neighbours' results in O(m * n).

OPTIMAL SOLUTION · dp on the bottom-right corner

TIME	SPACE
O(m * n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return only the largest side instead of the count.

Keep the same dp and take the maximum instead of adding. The common bug is checking only the diagonal neighbour: the up and left neighbours must also match, otherwise a different colour can sit inside the square.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count Same-Color Squares in Matrix
#
# LeetCode 221 variant — count maximal same-color square
# sub-grids. DP on each cell as the bottom-right corner.
# Input: m × n grid of colored cells. Output: count of
# square sub-grids whose cells are all the same color (or
# maximum side length, depending on prompt). DP: dp[i][j]
# = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) if
# grid[i][j] == grid[i-1][j] == grid[i][j-1] ==
# grid[i-1][j-1] else 1. Sum dp[i][j] across the grid for
# the count variant; max for the side variant. Examples:
# The plain "count all-1 squares" DP generalizes to same-
# color directly.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return only the largest side instead of the count.
#
# INTERVIEW NOTES
# Let dp[i][j] be the side of the largest same-colour
# square whose bottom-right corner is (i, j). That square
# exists only if the three neighbours above, left and
# above-left have the same colour as the cell and can
# themselves carry a square, so dp is 1 plus the smallest
# of their dp values. Summing dp over all cells counts
# every square once (each cell contributes one square per
# side length up to dp).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def square_sides(g):
    # side[i][j] = the side of the biggest same-colour square whose bottom-right corner is (i, j)
    sides = []
    for i in range(len(g)):
        row_sides = [1] * len(g[i])
        for j in range(len(g[i])):
            c = g[i][j]
```

```
            if i > 0 and j > 0 and g[i - 1][j] == c and g[i][j - 1] == c and g[i - 1][j - 1] == c:
                smallest = sides[i - 1][j]                # all three neighbours must match this cell
                if row_sides[j - 1] < smallest:
                    smallest = row_sides[j - 1]
                if sides[i - 1][j - 1] < smallest:
                    smallest = sides[i - 1][j - 1]
                row_sides[j] = 1 + smallest
```

```
        sides.append(row_sides)
    return sides


# ---- main question: main function ----
# Idea: a cell's square side is 1 + the smallest side of its top, left
# and top-left neighbours when all three have its colour, otherwise 1.
# Every cell is the bottom-right corner of one square per size up to its side.
# Complexity: Time O(m * n), Space O(m * n)
def count_same_color_squares(g):
    total = 0
    for row_sides in square_sides(g):
        for side in row_sides:
            total += side
    return total


# ---- main question: tests ----
def test_main_question():
    assert count_same_color_squares(["RRR", "RRR", "RRR"]) == 14     # a 3 x 3 block: 9 + 4 + 1
    assert count_same_color_squares(["RRR", "RGR", "RRR"]) == 9      # a different colour inside blocks the
 bigger squares
    assert count_same_color_squares([]) == 0


# =====================================================
# FOLLOW-UP 1: Return only the largest side instead of the count
# =====================================================
# ---- follow-up 1: function ----
# Answer: keep the same table and take the maximum instead of adding.
# The common bug is to check only the diagonal neighbour: the cells above
# and to the left must match as well, or a different colour can hide
# inside the square.
# Complexity: Time O(m * n), Space O(m * n)
def largest_side(g):
    best = 0
    for row_sides in square_sides(g):
        for side in row_sides:
            if side > best:
                best = side
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert largest_side(["RRR", "RRR", "RRR"]) == 3
    assert largest_side(["RRR", "RGR", "RRR"]) == 1
    assert largest_side([]) == 0


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write the all-1-squares DP from memory in under 8 min.
Generalize to multi-color in <2 min — only the equality check changes.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 27 OF 43

Nested structures: a stack of contexts, or recursion with a cache

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

5
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Nested structures: a stack of contexts, or recursion with a cache | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M	136. Decode String	1Point3Acres * 2026-07-29 *
M	137. Template Variable Expansion with Follow-Ups	1Point3Acres * 2026-10-17 *
M	138. Expand Recursive Key-Value Placeholders in a Template	PracHub * 2026-08-04 *
M	139. Nested List Weight Sum (LC 339) with Follow-Ups	1Point3Acres * 2026-08-16 *
M	140. Nested Function Expression Evaluator	1Point3Acres * 2026-06-10 *

CODE TEMPLATE | GROUP 27 OF 43 | 5 QUESTIONS, ONE PATTERN

Nested structures: a stack of contexts, or recursion with a cache

The one idea these questions share

When something opens (a bracket, a parenthesis, a placeholder), save the current context on a stack and start a fresh one. When it closes, pop the saved context and fold the result back in. Recursive expansion does the same with a function call and a cache.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def decode(s):
    stack, cur, num = [], [], 0
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "[":                                  # CHANGE 1: what opens a context
            stack.append((cur, num))
            cur, num = [], 0
        elif ch == "]":                                  # CHANGE 2: what closes it and how it folds back
            prev, times = stack.pop()
            prev.append("".join(cur) * times)
            cur = prev
        else:
            cur.append(ch)
    return "".join(cur)
```

2. Helper functions that usually get written next to it

```
def expand(text, mapping, seen=()):          # recursive placeholders {name} with cycle detection
    out, i = [], 0
    while i < len(text):
        if text[i] == "{":
            j = text.index("}", i)
            name = text[i + 1:j]
            if name in seen:
                raise ValueError("cycle: " + name)
            out.append(expand(mapping[name], mapping, seen + (name,)))
            i = j + 1
        else:
            out.append(text[i])
            i += 1
    return "".join(out)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
136	Decode String	the template as written
137	Template Variable Expansion with Follow-Ups	resolve(name) with a cache and a 'being resolved' set; meeting a name that is being resolved is a cycle
138	Expand Recursive Key-Value Placeholders in a Template	regex-find {key}, expand each key recursively with a cache and a visiting set to stop cycles
139	Nested List Weight Sum (LC 339) with Follow-Ups	depth counter: '[' adds 1, ']' subtracts 1; each number adds value * depth (read digits and a minus sign as you go)
140	Nested Function Expression Evaluator	tokenise, then a recursive parse(i): a number returns itself; a name reads its arguments until ')' and applies the function

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 136 of 197 | topic 27 of 43: Nested structures: a stack of contexts, or recursion with a cache | 1 of 5

Template fit: the template as written | * asked 83 days ago * | frequency Low

MEDIUM | IN APP LC 394 | 1Point3Acres | onsite-coding | 136 of 197

★ TOP 75: rank 60 in the Medium and Hard priority list

★ Decode String

ASKED 2026-07-29 | 83 DAYS AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 57.7 30 DAYS 0 PRIORITY 52/288

ABOUT THIS QUESTION

Topic: Nested structures: a stack of contexts, or recursion with a cache

Optimal answer from: your app's interview approach (LeetCode #394)

LeetCode match: LC 394 Decode String

Frequency: Low | Last asked: 2026-07-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/decode-string

QUESTION

An encoded string uses k[text] to mean that text is repeated k times; the brackets can be nested. Return the decoded string. (k is a positive integer, possibly with several digits; the input is always valid.)

Function signature

def decode(s: str) -> str

EXAMPLES

Example 1	
Input	`'3[a]2[bc]'`
Output	`'aaabcbc'`
Why	`3 copies of a, then 2 copies of bc.`

Example 2	
Input	`'2[abc]3[cd]ef'`
Output	`'abcabccdcdcdef'`
Why	`Plain letters after the brackets are kept as they are.`

Example 3	
Input	`'3[a2[c]]'`
Output	`'accaccacc'`
Why	`Nested: the inner 2[c] first (cc), then 3 copies of acc.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def parse():
    result = ""
    while pos < len(s) and s[pos] != "]":
        if s[pos].isdigit():
            n = read_number()
            pos += 1  # skip "["
            inner = parse()
            pos += 1  # skip "]"
            result += inner * n
        else:
            result += s[pos]
            pos += 1
    return result
```

TIME	SPACE
O(output length)	O(depth)

WHY IT IS SLOW

Time is proportional to the output, which is optimal; the cost is the recursion depth, equal to the nesting depth. A stack of (text so far, repeat count) avoids recursion.

OPTIMAL SOLUTION · Stack of (prefix, repeat) frames

TIME	SPACE
O(output length)	O(output + nesting depth)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How would you do it recursively, and what is the risk?

Parse with an index: read letters and digits, on '[' recurse to parse the inner string and return where it ended, on ']' return. It reads naturally but the call depth equals the nesting depth, which can overflow the stack for adversarial inputs. The stack version trades that for explicit memory.

FOLLOW-UP 2 What if the decoded string is enormous?

Do not materialize it. Return a generator, or compute the character at a given index by walking the structure with lengths (as in LeetCode 880, Decoded String at Index), using index modulo the segment length at each level.

FOLLOW-UP 3 The decoded string would be enormous.

Compute its length (or a given character) from the stack without building it: keep (length so far, repeat count) frames and combine as prev + inner * times. The length can be computed for strings that could never fit in memory.

FOLLOW-UP 4 Return the character at position k only.

Walk the structure with lengths: at each repeat reduce k modulo the inner length to find where in the inner string it falls (LeetCode 880 style).

FOLLOW-UP 5 Invalid input (missing bracket, count with no bracket).

Detect an unmatched ']' (empty stack), a '[' left at the end, and digits not followed by '['. Decide whether to raise or to treat a malformed part as literal text.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Decode String
#
# An encoded string uses k[text] to mean that text is
# repeated k times; the brackets can be nested. Return the
# decoded string. (k is a positive integer, possibly with
# several digits; the input is always valid.)
#
# Signature:
#   def decode(s: str) -> str
#
# Examples:
# '3[a]2[bc]'  ->  'aaabcbc'
# '2[abc]3[cd]ef'  ->  'abcabccdcdcdef'
#
# FOLLOW-UPS (answered below, in this order):
# 1. How would you do it recursively, and what is the
# risk?
# 2. What if the decoded string is enormous?
# 3. The decoded string would be enormous.
# 4. Return the character at position k only.
# 5. Invalid input (missing bracket, count with no
# bracket).
#
# INTERVIEW NOTES
# Nested repeats like 3[a2[c]] are a bracket structure,
# which means a stack: when you see '[' save what you have
# built and the repeat count; when you see ']' repeat the
# inner string and glue it to the saved prefix. Read
# counts as full numbers (more than one digit).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: a stack of "frames", one for each "[" that is still open. On "["
# push the text built so far together with the repeat count just read, and
# start a fresh text for the inside. On "]" pop the frame and set
# text = saved text + (inside text repeated count times).
```

```
# Complexity: Time O(length of the decoded string), Space O(length of the decoded string)
def decode(s):
    stack = []
    current = ""
    number = 0
    for ch in s:
        if ch.isdigit():
```

```
            number = number * 10 + int(ch)
        elif ch == "[":
            stack.append((current, number))
            current = ""
            number = 0
        elif ch == "]":
            saved, times = stack.pop()
            current = saved + current * times
        else:
            current += ch
    return current


# ---- main question: tests ----
def test_main_question():
    assert decode("3[a]2[bc]") == "aaabcbc"
    assert decode("2[abc]3[cd]ef") == "abcabccdcdcdef"
    assert decode("3[a2[c]]") == "accaccacc"
    assert decode("") == ""
    assert decode("abc") == "abc"


# ========================================================
# FOLLOW-UP 1: How would you do it recursively, and what is the risk?
# ========================================================
# ---- follow-up 1: function ----
# Answer: let a function read text up to the matching "]" and call itself
# when it meets a "[". It is shorter to write, but the nesting depth is
# the recursion depth, so very deep input can hit the recursion limit.
# Complexity: Time O(length of the decoded string), Space O(output + nesting depth)
def decode_recursive(s):
    def read(i):
        # read from position i until a "]" or the end; return (text, position)
        text = ""
        number = 0
        while i < len(s) and s[i] != "]":
            ch = s[i]
            if ch.isdigit():
```

```
                number = number * 10 + int(ch)
                i += 1
            elif ch == "[":
                inner, i = read(i + 1)
                text += inner * number
                number = 0
                i += 1                              # skip the "]"
            else:
                text += ch
                i += 1
        return text, i

    return read(0)[0]


# ---- follow-up 1: tests ----
def test_follow_up_1():
    for sample in ("3[a]2[bc]", "2[abc]3[cd]ef", "3[a2[c]]", "", "10[x]"):
        assert decode_recursive(sample) == decode(sample)


# ========================================================
# FOLLOW-UP 2: What if the decoded string is enormous?
# ========================================================
```

```
# ========================================================
# FOLLOW-UP 3: The decoded string would be enormous
# ========================================================
# ---- follow-up 3: helper functions ----
def parse(s):
    # the nested form: a list whose items are single characters or
    # (repeat count, list of items). Bad input raises ValueError (follow-up 5).
    stack = []
    current = []
    number = 0
    for ch in s:
        if ch.isdigit():
            number = number * 10 + int(ch)
        elif ch == "[":
            if number == 0:
                raise ValueError("[ needs a count")
            stack.append((current, number))
            current = []
            number = 0
```

```
        elif ch == "]":
            if len(stack) == 0:
                raise ValueError("unmatched ]")
            saved, times = stack.pop()
            saved.append((times, current))
            current = saved
        else:
            if number != 0:
                raise ValueError("count without [")
            current.append(ch)
    if len(stack) > 0:
        raise ValueError("missing ]")
    return current


# ---- follow-up 3: function ----
# Answer: compute the LENGTH from the nested form without building the
# string: a character counts 1, and a repeat counts (times * length of
# the inside). The length can be found for strings that could never fit
# in memory.
# Complexity: Time O(n) for n parsed items, Space O(depth)
def length(items):
    total = 0
    for item in items:
        if isinstance(item, str):
            total += 1
        else:
            total += item[0] * length(item[1])
    return total


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert length(parse("3[a2[c]]")) == 9
    big = parse("999[999[999[ab]]]")
    assert length(big) == 2 * 999 ** 3                  # measured, never built


# ========================================================
# FOLLOW-UP 4: Return the character at position k only
# ========================================================
# ---- follow-up 4: function ----
# Answer: walk the nested form with the lengths. For a repeat, if k falls
# inside it, which copy does not matter: look at position k modulo the
# length of the inside (LeetCode 880 style).
```

```
# Complexity: Time O(n * depth), Space O(depth)
def char_at(items, k):
    for item in items:
        if isinstance(item, str):
            if k == 0:
                return item
            k -= 1
        else:
            times, inner = item
            inner_length = length(inner)
            if k < times * inner_length:
                return char_at(inner, k % inner_length)
            k -= times * inner_length
    return None


# ---- follow-up 4: tests ----
def test_follow_up_4():
    items = parse("3[a2[c]]")
    letters = []
    for i in range(9):
        letters.append(char_at(items, i))
    assert "".join(letters) == decode("3[a2[c]]")
    assert char_at(items, 9) is None


# =============================================================
# FOLLOW-UP 5: Invalid input (missing bracket, count with no bracket)
# =============================================================
# ---- follow-up 5: tests ----
# Answer: parse() above checks while reading: "3[a" (missing "]"), "a]"
# (unmatched "]"), "[a]" (no count) and "3a" (a count with no "[") all
# raise ValueError.
def test_follow_up_5():
    for bad in ("3[a", "a]", "[a]", "3a"):
        try:
            parse(bad)
            raise AssertionError(bad)
        except ValueError:
            pass


# =============================================================
# RUN EVERYTHING
# =============================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    test_follow_up_4()
    test_follow_up_5()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write the iterative-stack version from memory in under 10 min.
Practice the postfix-syntax variant (parse direction reversed).
Drill the decode side together with the encode-shortest dual problem as a pair.

ALSO REPORTED · 1 more

String Decompression with Nested Counts | 1Point3Acres

https://www.1point3acres.com/interview/problems/company/google/string-decompression-nested-counts

Recursion with an index pointer is the cleanest implementation: each call returns the decoded substring of a single (...){k} block plus literals up to its terminator.
An iterative stack-based approach also works: push (StringBuilder, repeatCount) on (, pop and append on){k}.
The 2-digit k matters only for parsing — read until non-digit.
Watch out for literals between two groups: a(b){2}c(d){3} -> abbcddd.

Preparation: Drill the canonical k[group] decode-string problem to muscle memory; the input grammar there is k[group] instead of (group){k}, but the algorithm is identical.
Practice both recursive and iterative versions — the iterative stack version generalizes more cleanly if the interviewer asks about iterative streaming.
Test with: deeply nested groups, groups containing only literals, two-digit counts ({12}).

STUDY PATH: question 137 of 197 | topic 27 of 43: Nested structures: a stack of contexts, or recursion with a cache | 2 of 5

Template fit: resolve(name) with a cache and a 'being resolved' set; meeting a name that is being resolved is a cycle | * asked 3 days ago * | frequency Low

MEDIUM | 1Point3Acres | onsite-coding | 137 of 197

Template Variable Expansion with Follow-Ups

ASKED 2026-10-17 | 3 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 274/288**

ABOUT THIS QUESTION

Topic: Nested structures: a stack of contexts, or recursion with a cache

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-17 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/template-variable-expansion-dependencies

QUESTION

Replace **%KEY%** placeholders using a key-value map whose values may contain other placeholders. Clarify malformed delimiters and literal-percent escaping, flatten nested values, and detect cyclic dependencies before expansion.

Input: a key-value map and a string containing placeholders delimited by %, such as %USER%. Replace every placeholder with the corresponding value.
Map values can themselves contain placeholders, so resolve or flatten dependencies before expanding the final string. Define behavior for malformed delimiters such as unmatched percent signs, for a literal % in output, and for nested-looking forms before coding; the prompt intentionally leaves these contracts open.
Follow-up: detect cyclic dependencies among map values. Model keys as a dependency graph and use indegrees plus a queue to determine whether a complete topological resolution is possible.

EXAMPLES

```
Given {USER: "admin", HOME: "/%USER%/home"}:
```

Input: I am %USER% My home is %HOME% Output: I am admin My home is /admin/home

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def expand(text, depth=0):
    if depth > LIMIT:
        raise Error("too deep: cycle?")
    out = ""
    for token in tokens(text):
        if token == "%%":
            out += "%"
        elif is_key(token):
            value = values[name_of(token)]
            out += expand(value, depth + 1)
        else:
            out += token
    return out
```

TIME	SPACE
exponential in the worst ... exponential in the worst case	**O(depth)**

WHY IT IS SLOW

BRUTE FORCE

Re-expands shared values every time and finds cycles only by running out of depth. Memoising each value and keeping a visiting set expands each key once and reports a cycle at once.

OPTIMAL SOLUTION · Depth-first expansion with memo and a visiting set

TIME	SPACE
O(total length of the expanded text + keys)	O(keys)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Detect cycles before expanding.

Model each key as a vertex with an edge to every key its value mentions, count indegrees and run Kahn's algorithm: if fewer vertices than keys come out, some keys sit on a cycle and cannot be resolved. This reports the problem up front instead of failing halfway through a long document.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Template Variable Expansion with Follow-
# Ups
#
# Replace %KEY% placeholders using a key-value map whose
# values may contain other placeholders. Clarify malformed
# delimiters and literal-percent escaping, flatten nested
# values, and detect cyclic dependencies before expansion.
# Input: a key-value map and a string containing
# placeholders delimited by %, such as %USER%. Replace
# every placeholder with the corresponding value. Map
# values can themselves contain placeholders, so resolve
# or flatten dependencies before expanding the final
# string. Define behavior for malformed delimiters such as
# unmatched percent signs, for a literal % in output, and
# for nested-looking forms before coding; the prompt
# intentionally leaves these contracts open. Follow-up:
# detect cyclic dependencies among map values. Model keys
# as a dependency graph and use indegrees plus a queue to
# determine whether a complete topological resolution is
# possible.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Detect cycles before expanding.
#
# INTERVIEW NOTES
# Agree the grammar first: %NAME% is a placeholder, %% is
# a literal percent sign, an unmatched % is an error and
# an unknown name is an error. Values may contain
# placeholders themselves, so expand a value by expanding
# its own placeholders first. A cycle (A uses B, B uses A)
# would recurse forever, so detect it with a visiting set
# or, equivalently, with indegrees and a queue.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
```

```
# =====================================================

from collections import deque


# =====================================================
# MAIN QUESTION
```

```
# =====================================================
# ---- main question: helper functions ----
def tokens(s):
    # split the text into ("text", character) and ("var", name) pieces:
    # "%%" is one literal percent sign, "%NAME%" is a variable
    pieces = []
    i = 0
    while i < len(s):
        if s[i] != "%":
            pieces.append(("text", s[i]))
            i += 1
        elif s[i:i + 2] == "%%":
            pieces.append(("text", "%"))
            i += 2
        else:
            end = s.find("%", i + 1)
            if end < 0:
                raise ValueError("unmatched %")
            pieces.append(("var", s[i + 1:end]))
            i = end + 1
    return pieces


# ---- main question: main function ----
# Idea: scan the text once: "%%" becomes "%", "%NAME%" becomes the
# resolved value of NAME. A value may itself hold placeholders, so resolve
# it with the same scan first, remember the result (memo), and keep a
# "visiting" set to catch a name that needs itself.
# Complexity: Time O(template + total length of the values used), Space the same
def expand_template(text, mapping):
    done = {}
    visiting = set()

    def resolve(name):
        if name in done:
            return done[name]
        if name in visiting:
            raise ValueError("cycle at " + name)
        if name not in mapping:
            raise KeyError(name)
        visiting.add(name)
        done[name] = scan(mapping[name])
        visiting.discard(name)
        return done[name]
```

```
    def scan(s):
        result = ""
        for kind, piece in tokens(s):
            if kind == "var":
                result += resolve(piece)
            else:
                result += piece
        return result

    return scan(text)
```

```
# ---- main question: tests ----
MAPPING = {"USER": "admin", "HOME": "/%USER%/home"}


def test_main_question():
    assert expand_template("I am %USER% My home is %HOME%", MAPPING) == "I am admin My home is /admin/home"
    assert expand_template("100%% sure", {}) == "100% sure"


# ========================================================
# FOLLOW-UP 1: Detect cycles before expanding
# ========================================================
# ---- follow-up 1: function ----
# Answer: model every key as a vertex with an edge from each key its value
# mentions; count how many unresolved keys each key waits for and run
# Kahn's algorithm (take keys that wait for nothing, lower the counts of
# the keys that needed them). If fewer vertices than keys come out, the
# rest sit on a cycle and cannot be resolved.
# Complexity: Time O(V + E + total length of the values), Space O(V + E)
def has_cycle(mapping):
    waits_for = {}                                  # key -> the keys it mentions (that exist)
    for key in mapping:
        mentioned = set()
        for kind, piece in tokens(mapping[key]):
            if kind == "var" and piece in mapping:
                mentioned.add(piece)
        waits_for[key] = mentioned
    users = {}                                      # key -> the keys whose values mention it
    for key in mapping:
        users[key] = []
    for key in waits_for:
        for other in waits_for[key]:
            users[other].append(key)
```

```
    unmet = {}
    queue = deque()
    for key in waits_for:
        unmet[key] = len(waits_for[key])
        if unmet[key] == 0:
            queue.append(key)
    resolved = 0
    while queue:
        key = queue.popleft()
        resolved += 1
        for user in users[key]:
            unmet[user] -= 1
            if unmet[user] == 0:
                queue.append(user)
    return resolved != len(mapping)                 # keys left over sit on a cycle


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert not has_cycle(MAPPING)
    loop = {"A": "%B%", "B": "%A%"}
    assert has_cycle(loop)
    assert has_cycle({"A": "x %A% y"})              # a key that needs itself
    for bad, error in (("a % b", ValueError), ("%NOPE%", KeyError)):
        try:
            expand_template(bad, {})
            raise AssertionError("should have failed")
        except error:
            pass
    try:
        expand_template("%A%", loop)
```

```
        raise AssertionError("cycle not caught")
    except ValueError:
        pass


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

STUDY PATH: question 138 of 197 | topic 27 of 43: Nested structures: a stack of contexts, or recursion with a cache | 3 of 5

Template fit: regex-find {key}, expand each key recursively with a cache and a visiting set to stop cycles | * asked 77 days ago *

MEDIUM | PracHub | Technical Screen | 138 of 197

★ TOP 75: rank 71 in the Medium and Hard priority list

★ Expand Recursive Key-Value Placeholders in a Template

POSTED 2026-08-04 | 77 DAYS AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 57.7 30 DAYS 0 PRIORITY 53/288

ABOUT THIS QUESTION

Topic: Nested structures: a stack of contexts, or recursion with a cache

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #394 Decode String

LeetCode match: No direct match; related LC 394 Decode String (inferred from overview)

Posted on PracHub: 2026-08-04 (updated 2026-08-16)

Original: https://prachub.com/coding-questions/expand-recursive-key-value-placeholders-in-a-template

QUESTION

Expand %KEY% placeholders in a template. A placeholder is replaced by its value from the mapping, and placeholders inside the values are expanded too, recursively. An unknown key is left exactly as written. A cycle (A needs B needs A) raises an error.

Function signature

def expand_template(template: str, mapping: dict) -> str

EXAMPLES

Example 1	
Input	`'%GREETING%, welcome!', {'NAME': 'Ada', 'GREETING': 'Hello %NAME%'}`
Output	`'Hello Ada, welcome!'`
Why	`%GREETING% expands to 'Hello %NAME%', and %NAME% is expanded in turn.`

Example 2	
Input	`'Hi %WHO%', {}`
Output	`'Hi %WHO%'`
Why	`An unknown key is kept as written.`

Example 3	
Input	`'%A%', {'A': '%B%', 'B': '%A%'}`
Output	`raises ValueError: cycle at A`
Why	`A needs B and B needs A: a cycle is reported instead of looping forever.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def expand(text):
    for _ in range(LIMIT):
        new = text
        for key, value in values.items():
            tag = "%" + key + "%"
            new = new.replace(tag, value)
        if new == text:
            return text
        text = new
    raise Error("cycle")
```

TIME	SPACE
O(depth * length)	O(length)

WHY IT IS SLOW

It re-expands the same key every time it appears and needs an arbitrary cap to stop on cycles. A memoised recursive expansion with a visiting set expands each key once and detects cycles exactly.

OPTIMAL SOLUTION · Memoised recursive expansion with a visiting set

TIME	SPACE
O(total size of all values + template)	O(number of keys)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A literal percent sign must be possible.

Choose an escape such as %% and handle it in the same scan: when the text has %%, output one % and skip both characters; otherwise try to match a placeholder. Test percent signs next to placeholders.

FOLLOW-UP 2 An unknown key should be an error, or replaced by an empty string.

Make the policy a parameter: raise KeyError, return the placeholder unchanged (as coded), or return ''. State your default.

FOLLOW-UP 3 Values are huge and expansion explodes.

The output can be exponentially larger than the input (a key referring to the previous key twice, repeatedly). Check the expanded length as you go and fail past a limit, or return a lazy expansion.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Expand Recursive Key-Value Placeholders
# in a Template
#
# Expand %KEY% placeholders in a template. A placeholder
# is replaced by its value from the mapping, and
# placeholders inside the values are expanded too,
# recursively. An unknown key is left exactly as written.
# A cycle (A needs B needs A) raises an error.
#
# Signature:
```

```
#   def expand_template(template: str, mapping: dict) -> str
#
# Examples:
# '%GREETING%, welcome!', {'NAME': 'Ada', 'GREETING':
#     'Hello %NAME%'}  ->  'Hello Ada, welcome!'
# 'Hi %WHO%', {}  ->  'Hi %WHO%'
#
# FOLLOW-UPS (answered below, in this order):
# 1. A literal percent sign must be possible.
# 2. An unknown key should be an error, or replaced by an
# empty string.
# 3. Values are huge and expansion explodes.
#
# INTERVIEW NOTES
# Expanding one key can require expanding others, so
# define expand(key) recursively and remember results
# (memoization) so a key used many times is expanded once.
# A 'currently expanding' set detects cycles: meeting a
# key that is already on the stack means a cycle. Decide
# the policy for unknown keys and for literal percent
# signs up front.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def is_word(name):
    # a key is made of letters, digits and underscores (and is not empty)
    if name == "":
        return False
    for ch in name:
        if not (ch.isalnum() or ch == "_"):
            return False
    return True


# ---- main question: main function ----
# Idea: expand(key) replaces every placeholder in the key's value by the
# expansion of the key it names, remembers each result (a cache, so a
# key used many times is expanded once), and keeps a "visiting" set so
# that a key that needs itself (a cycle) raises an error. Unknown keys
# stay as written (follow-up 2 can also empty them or raise an error).
# "%%" is a literal percent sign (follow-up 1): output one "%" and skip
# both characters. `limit` stops an expansion that explodes (follow-up 3).
# Complexity: Time O(template + total length of the values used), each key is expanded once; Space the same
def expand_template(template, mapping, unknown="keep", limit=10 ** 6):
    cache = {}
    visiting = set()

    def expand_text(text):
        result = ""
        i = 0
        while i < len(text):
            if text[i] != "%":
                result += text[i]
                i += 1
            elif text[i + 1:i + 2] == "%":
```

```
                result += "%"                           # "%%" is a literal percent sign
                i += 2
            else:
                end = text.find("%", i + 1)
                name = ""
                if end != -1:
                    name = text[i + 1:end]
                if not is_word(name):
                    result += "%"                       # not a placeholder: keep the percent sign
                    i += 1
                elif name in mapping:
                    result += expand_key(name)
                    i = end + 1
                else:
                    if unknown == "error":
                        raise KeyError(name)
```

```
                    if unknown == "keep":
                        result += text[i:end + 1]       # leave it exactly as written
                    i = end + 1                         # (unknown == "empty": it simply disappears)
        if len(result) > limit:
            raise ValueError("expansion too large")
        return result

    def expand_key(key):
        if key in cache:
            return cache[key]
        if key in visiting:
            raise ValueError("cycle at " + key)
        visiting.add(key)
        cache[key] = expand_text(mapping[key])
        visiting.discard(key)
        return cache[key]

    return expand_text(template)


# ---- main question: tests ----
MAPPING = {"USER": "admin", "HOME": "/%USER%/home"}


def test_main_question():
    assert expand_template("I am %USER% My home is %HOME%", MAPPING) == "I am admin My home is /admin/home"
    assert expand_template("a %X% b", MAPPING) == "a %X% b"           # an unknown key stays as written
    assert expand_template("Hi %WHO%", {}) == "Hi %WHO%"
    try:
        expand_template("%A%", {"A": "%B%", "B": "%A%"})              # a cycle is reported
        raise AssertionError("cycle not caught")
    except ValueError:
        pass


# ========================================================
# FOLLOW-UP 1: A literal percent sign must be possible
# ========================================================
# ---- follow-up 1: tests ----
# Answer: choose an escape such as "%%" and handle it in the same scan:
# when the text has "%%" output one "%" and skip both characters,
# otherwise try to read a placeholder. Test percent signs next to
# placeholders.
def test_follow_up_1():
    assert expand_template("100%% of %USER%", MAPPING) == "100% of admin"
```

```
    assert expand_template("%%USER%%", MAPPING) == "%USER%"
    assert expand_template("50%", MAPPING) == "50%"
```

```
# ============================================================
# FOLLOW-UP 2: An unknown key should be an error, or replaced by an empty string
# ============================================================
# ---- follow-up 2: tests ----
# Answer: a parameter `unknown`: "keep" (leave the text), "empty" (replace
# by nothing) or "error" (raise KeyError). Only the branch for an unknown
# key changes.
def test_follow_up_2():
    assert expand_template("a %X% b", MAPPING, unknown="empty") == "a  b"
    try:
        expand_template("%X%", MAPPING, unknown="error")
        raise AssertionError("missing key not reported")
    except KeyError:
        pass


# ============================================================
# FOLLOW-UP 3: Values are huge and expansion explodes
# ============================================================
# ---- follow-up 3: tests ----
# Answer: cap the size of every expansion (the `limit` parameter) and stop
# with an error as soon as it is exceeded. Here every key doubles the
# previous one, so the full text would have 2^30 characters.
def test_follow_up_3():
    boom = {"K0": "ab"}
    for i in range(1, 30):
        boom["K" + str(i)] = "%K" + str(i - 1) + "%%K" + str(i - 1) + "%"
    try:
        expand_template("%K29%", boom, limit=1000)
        raise AssertionError("explosion not stopped")
    except ValueError:
        pass


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def expand_template(values: dict[str, str], template: str) -> str:
```

Sample call

```
expand_template({}, 'plain text')
```

Test cases

Input	Expected
`({}, 'plain text')`	`'plain text'`
`({'USER': 'admin', 'HOME': '/%USER%/home'}, 'I am %USER% My home is %HOME%')`	`'I am admin My home is /admin/home'`

ALSO REPORTED · 1 more

Template Variable Expansion with Follow-Ups | 1Point3Acres | onsite-coding | asked 2026-08-14 | Single report

https://www.1point3acres.com/interview/problems/company/google/template-variable-expansion-dependencies

The interviewer spent most of the round probing the delimiter and escaping contract before allowing implementation. State a compact grammar and get explicit agreement; otherwise a nominally easy parser can lose the coding budget to unresolved edge cases.

Preparation: Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

STUDY PATH: question 139 of 197 | topic 27 of 43: Nested structures: a stack of contexts, or recursion with a cache | 4 of 5

Template fit: depth counter: '[' adds 1, ']' subtracts 1; each number adds value * depth (read digits and a minus sign as you go) | * asked 65 days ago * | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 139 of 197

Nested List Weight Sum (LC 339) with Follow-Ups

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 16.5 30 DAYS 0 PRIORITY 177/288

ABOUT THIS QUESTION

Topic: Nested structures: a stack of contexts, or recursion with a cache

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 339 Nested List Weight Sum

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/nested-list-weighted-sum-parsing

QUESTION

A nested list holds integers and further lists. The weight of an integer is its depth: 1 at the top level, 2 inside one inner list, 3 inside two, and so on. Return the sum of every integer times its depth. A second version reads the nested list from a string like '[1,[4,[6]]]'.

Function signature

def weight_sum(nested: list) -> int

def weight_sum_string(s: str) -> int

EXAMPLES

Example 1	
Input	`[3, 8, [2, 14], [2, [91]]]`
Output	`320`
Why	`3 + 8 at depth 1, 2 + 14 and 2 at depth 2, and 91 at depth 3: 11 + 32 + 4 + 273 = 320.`

Example 2	
Input	`'[1,[4,[6]]]'`
Output	`27`
Why	`The string version of [1, [4, [6]]]: 1*1 + 4*2 + 6*3 = 27.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(text):
    nested = json.loads(text)  # parse the lists
    return depth_sum(nested, 1)
def depth_sum(items, depth):
    total = 0
    for item in items:
        if isinstance(item, int):
            total += item * depth
        else:
            total += depth_sum(item, depth + 1)
    return total
```

BRUTE FORCE

TIME	SPACE
O(n)	O(depth)

WHY IT IS SLOW

It builds the whole nested structure and recurses as deep as the nesting, so memory and the call stack grow with depth. A single left-to-right scan with a bracket counter needs O(1) extra space.

OPTIMAL SOLUTION · Single pass over the string with a depth counter

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 When does the recursive (DFS) version become unsafe?

Python's default recursion limit is about 1000 frames, so an input nested 10,000 levels deep raises RecursionError. DFS also uses O(depth) call-stack space. The single-pass scanner uses O(1) and BFS uses O(widest level); both avoid the problem. In Java the same limit appears as StackOverflowError.

FOLLOW-UP 2 What is the inverse version (deeper integers weigh less)?

That is LeetCode 364. Do a level-by-level pass that keeps a running sum of everything seen so far and add it to the total once per level: total += running_sum on each level. Shallow integers are added many times, deep ones once, which equals weight (maxDepth + 1 - depth).

FOLLOW-UP 3 The inverse version (shallow integers weigh less, LeetCode 364).

Process level by level and keep a running sum of every integer seen so far; add the running sum to the total once per level that holds an integer or lies above one (trim trailing levels that contain only empty lists). Shallow numbers are added at every deeper level, which equals the weight maxDepth - depth + 1.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Nested List Weight Sum (LC 339) with
# Follow-Ups
#
# A nested list holds integers and further lists. The
# weight of an integer is its depth: 1 at the top level, 2
# inside one inner list, 3 inside two, and so on. Return
# the sum of every integer times its depth. A second
# version reads the nested list from a string like
# '[1,[4,[6]]]'.
#
# Signature:
#   def weight_sum(nested: list) -> int
#   def weight_sum_string(s: str) -> int
#
# Examples:
# [3, 8, [2, 14], [2, [91]]]  ->  320
# '[1,[4,[6]]]'  ->  27
```

```
#
# FOLLOW-UPS (answered below, in this order):
# 1. When does the recursive (DFS) version become unsafe?
# 2. What is the inverse version (deeper integers weigh
# less)?
# 3. The inverse version (shallow integers weigh less,
# LeetCode 364).
#
# INTERVIEW NOTES
# The weight of a number is just the nesting depth where
# it appears, and the nesting depth is the number of '['
# still open. That means you never need to build the
# nested structure: scan once with a counter, and flush a
# number whenever a non-digit arrives (and at the end).

# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: helper functions ----
def with_depths(nested):
    # every integer together with its depth (an integer in the top-level list
```

```
    # has depth 1). A stack of (item, depth) is used instead of recursion, so
    # very deep input cannot overflow the call stack.
    pairs = []
    stack = [(nested, 0)]
    while stack:
        item, depth = stack.pop()
        if isinstance(item, list):
            for child in item:
                stack.append((child, depth + 1))
        else:
            pairs.append((item, depth))
    return pairs


# ---- main question: main functions ----
# Idea (LeetCode 339): every integer counts as value * depth. Collect the
# integers with their depths and add value * depth up.
# Complexity: Time O(n), Space O(n)
def weight_sum(nested):
    total = 0
    for value, depth in with_depths(nested):
        total += value * depth
    return total


# The same sum straight from the TEXT of the nested list, "[1,[4,[6]]]",
# in one pass with O(1) extra space. An integer is weighted by the number
# of "[" that are currently open (the depth). Digits are collected into a
# number, which is added when a non-digit character ends it. The extra "]"
# at the end flushes a number that finishes the text.
# Complexity: Time O(L) for text of length L, Space O(1)
def weight_sum_string(s):
    depth = 0
    total = 0
    number = 0
    sign = 1
```

```
    reading = False
    for ch in s + "]":
        if ch.isdigit():
            number = number * 10 + int(ch)
            reading = True
            continue
        if reading:
            total += sign * number * depth
            number = 0
            sign = 1
            reading = False
        if ch == "[":
            depth += 1
        elif ch == "]":
            depth -= 1
        elif ch == "-":
            sign = -1
    return total


# ---- main question: tests ----
def test_main_question():
    assert weight_sum([3, 8, [2, 14], [2, [91]]]) == 320          # 3 + 8 + 2 * (2 + 14) + 2 * 2 + 3 * 91
    assert weight_sum([]) == 0
    assert weight_sum_string("[1,[4,[6]]]") == 27
    assert weight_sum_string("[-1,[2]]") == 3
    assert weight_sum_string("[[]]") == 0


# ======================================================
# FOLLOW-UP 1: When does the recursive (DFS) version become unsafe?
# ======================================================
# ---- follow-up 1: function ----
# Answer: when the nesting is very deep (thousands of levels): every level
# is one function call, and Python stops at about 1000 with a
# RecursionError. The stack-based with_depths above has no such limit.
# Complexity: Time O(n), Space O(depth)
def weight_sum_recursive(nested, depth=1):
    total = 0
    for item in nested:
        if isinstance(item, list):
            total += weight_sum_recursive(item, depth + 1)
        else:
            total += item * depth
    return total


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert weight_sum_recursive([3, 8, [2, 14], [2, [91]]]) == 320
    deep = [1]
    for i in range(4999):
        deep = [deep]                                  # 5000 levels deep
    assert weight_sum(deep) == 5000
    try:
        weight_sum_recursive(deep)
        raise AssertionError("the recursive version should run out of stack")
    except RecursionError:
        pass


# ======================================================
# FOLLOW-UP 2: What is the inverse version (deeper integers weigh less)?
```

```
# ============================================================


# ============================================================
# FOLLOW-UP 3: The inverse version (shallow integers weigh less, LeetCode 364)
# ============================================================
# ---- follow-up 2: function ----
# Answer (LeetCode 364): the deepest integers weigh 1 and the top-level
# ones weigh the most: weight = deepest depth + 1 - depth. Find the
# deepest depth first, then add value * weight. (A level-by-level pass that
# adds a running sum once per level gives the same number.)
# Complexity: Time O(n), Space O(n)
def inverse_weight_sum(nested):
    pairs = with_depths(nested)
    if len(pairs) == 0:
        return 0
    deepest = 0
    for value, depth in pairs:
        if depth > deepest:
            deepest = depth
    total = 0
    for value, depth in pairs:
        total += value * (deepest + 1 - depth)
    return total


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert inverse_weight_sum([[1, 1], 2, [1, 1]]) == 8           # LeetCode 364 example
    assert inverse_weight_sum([1, [4, [6]]]) == 17
    assert inverse_weight_sum([]) == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write DFS and BFS versions over a parsed nested-list interface and explain when recursion depth becomes unsafe.
Build the string parser as a tiny state machine, including multi-digit values and the end-of-input flush.
Dry-run [2, [91]] aloud while tracking depth, buffered number, and accumulated sum.

STUDY PATH: question 140 of 197 | topic 27 of 43: Nested structures: a stack of contexts, or recursion with a cache | 5 of 5

Template fit: tokenise, then a recursive parse(i): a number returns itself; a name reads its arguments until ')' and applies the function | * asked 132 days ago * | frequency Low

MEDIUM | 1Point3Acres | onsite-coding | 140 of 197

Nested Function Expression Evaluator

ASKED 2026-06-10 | 132 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 5** **30 DAYS 0** **PRIORITY 199/288**

ABOUT THIS QUESTION

Topic: Nested structures: a stack of contexts, or recursion with a cache

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 770 Basic Calculator IV (related)

Frequency: Low | Last asked: 2026-06-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/nested-function-expression-evaluator

QUESTION

Evaluate an expression made of integers and nested function calls: add and mul (any number of arguments), sub and pow (two), div (integer division truncating toward zero), min and max (any number). Return the integer result.

Function signature

def evaluate(expr: str) -> int

EXAMPLES

Example 1	
Input	'add(1, sub(1, 0))'
Output	2
Why	sub(1, 0) is 1, so add(1, 1) = 2.

Example 2	
Input	'add(2, mul(3, pow(4, 5)))'
Output	3074
Why	pow(4, 5) = 1024, times 3 = 3072, plus 2 = 3074.

Example 3	
Input	'max(3, div(-7, 2), min(5, 9))'
Output	5
Why	div(-7, 2) truncates toward zero to -3; max of 3, -3 and 5 is 5.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def evaluate(text):
    while has_call(text):
        call = find_innermost_call(text)
        value = apply(call)  # e.g. add(2, 3)
        text = text.replace(call, str(value))
    return int(text)
```

BRUTE FORCE

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every replacement rescans and rebuilds the string, O(n) per call and O(n^2) in total, and it cannot report errors well. A tokenizer plus recursive descent reads the text once, O(n).

OPTIMAL SOLUTION · Tokenizer + recursive descent over name(arg, arg, ...)

TIME	SPACE
O(n)	O(depth)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Support variadic functions and variables.

Collect arguments until ')' instead of assuming two (as the code does). For variables, pass an environment dict into parse and treat an identifier that is not followed by '(' as a lookup. Define the error behaviour for undefined names.

FOLLOW-UP 2 How do you make it iterative?

Use two stacks: one of pending function names and one of lists of evaluated arguments. On an identifier push the name and a new list, on a number append to the top list, on ')' pop both, apply the function, and append the result to the new top list. This removes the recursion-depth limit but is easier to get wrong off by one.

FOLLOW-UP 3 What about division?

State the contract: integer or float, truncate toward zero or floor (Python's // floors, which differs for negatives), and divide by zero handling. The implementation above truncates toward zero.

FOLLOW-UP 4 Add variables (an identifier with no parentheses is a lookup).

In parse, check whether the identifier is followed by '('; if not, return env[name]. Decide what an undefined name does (raise, or default to 0).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
# The expression looks like  add(1, mul(2, x))  : a function name, then
# arguments (numbers, variables, or more calls) in brackets, separated by commas.
```

```
def add_all(*numbers):
    total = 0
    for x in numbers:
        total += x
    return total


def multiply_all(*numbers):
    total = 1
    for x in numbers:
        total *= x
    return total


def smallest(*numbers):
    best = numbers[0]
    for x in numbers:
        if x < best:
            best = x
    return best


def largest(*numbers):
    best = numbers[0]
    for x in numbers:
        if x > best:
            best = x
    return best


def subtract(a, b):
```

```
    return a - b


def power(a, b):
    return a ** b


def divide(a, b):
    # integer division that TRUNCATES toward zero (follow-up 3 explains why)
    if b == 0:
        raise ZeroDivisionError("division by zero")
    return int(a / b)


FUNCTIONS = {"add": add_all, "sub": subtract, "mul": multiply_all, "pow": power,
             "div": divide, "min": smallest, "max": largest}


def tokenize(expr):
    # split the text into numbers (a "-" right before digits is a sign),
    # names, and the characters ( ) ,
    tokens = []
    i = 0
    while i < len(expr):
        ch = expr[i]
        if ch == " ":
            i += 1
        elif ch in "(),":
            tokens.append(ch)
            i += 1
        elif ch.isdigit() or (ch == "-" and i + 1 < len(expr) and expr[i + 1].isdigit()):
            j = i + 1
```

```
            while j < len(expr) and expr[j].isdigit():
                j += 1
            tokens.append(expr[i:j])
            i = j
        elif ch.isalpha() or ch == "_":
            j = i + 1
            while j < len(expr) and (expr[j].isalnum() or expr[j] == "_"):
                j += 1
            tokens.append(expr[i:j])
            i = j
        else:
            raise ValueError("unexpected character " + ch)
    return tokens
```

```
def is_number(token):
    return token.lstrip("-").isdigit()


# ---- main question: main function ----
# Idea: tokenize, then "recursive descent": parse(i) reads ONE value that
# starts at token i and returns (value, index after it). A number is
# itself; a name not followed by "(" is a variable (looked up in env,
# follow-up 4); a name followed by "(" is a call: read the arguments one
# by one (each is again a value) until ")", then apply the function.
# Complexity: Time O(n) for n tokens, Space O(depth)
def evaluate(expr, env=None):
    if env is None:
        env = {}
    tokens = tokenize(expr)

    def parse(i):
        token = tokens[i]
        if is_number(token):
            return int(token), i + 1
        if i + 1 >= len(tokens) or tokens[i + 1] != "(":
            if token not in env:
                raise ValueError("undefined " + token)
            return env[token], i + 1
        if token not in FUNCTIONS:
            raise ValueError("unknown function " + token)
        i += 2                                          # skip the name and "("
        args = []
        while tokens[i] != ")":
            value, i = parse(i)
            args.append(value)
            if tokens[i] == ",":
                i += 1
        return FUNCTIONS[token](*args), i + 1

    value, end = parse(0)
    if end != len(tokens):
        raise ValueError("trailing tokens")
    return value


# ---- main question: tests ----
def test_main_question():
```

```
    assert evaluate("add(1, sub(1, 0))") == 2
    assert evaluate("add(2, mul(3, pow(4, 5)))") == 3074
    assert evaluate("add(1, 2, 3, 4)") == 10                  # functions take any number of arguments
    assert evaluate("max(3, min(9, 4), 1)") == 4
    assert evaluate("add(x, mul(x, 2))", {"x": 5}) == 15      # variables come from an environment
```

```
    assert evaluate("div(-7, 2)") == -3                      # division truncates toward zero
    for bad in ("foo(1)", "add(1", "add(1) 2", "y"):
        try:
            evaluate(bad)
            raise AssertionError(bad)
        except (ValueError, IndexError):
            pass


# =====================================================
# FOLLOW-UP 1: What if the nesting is very deep?
# =====================================================
# Answer: the recursive version uses one Python call per level and stops at
# about 1000 levels (RecursionError). Use the iterative version below.


# =====================================================
# FOLLOW-UP 2: How do you make it iterative?
# =====================================================
# ---- follow-up 2: function ----
# Answer: use two stacks: one with the names of the calls that are still
# open, and one with the lists of arguments evaluated so far. A number is
# added to the top argument list; "(" opens a new list for the name just
# pushed; ")" applies the function to the top list and adds the result to
# the list below it.
# Complexity: Time O(n), Space O(depth)
def evaluate_iterative(expr):
    names = []
    arg_lists = [[]]
    for token in tokenize(expr):
        if is_number(token):
            arg_lists[-1].append(int(token))
        elif token == "(":
            arg_lists.append([])
        elif token == ")":
            result = FUNCTIONS[names.pop()](*arg_lists.pop())
            arg_lists[-1].append(result)
        elif token != ",":
            names.append(token)
    return arg_lists[0][0]
```

```
# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert evaluate_iterative("add(2, mul(3, pow(4, 5)))") == 3074
    deep = "add(1, " * 3000 + "0" + ")" * 3000            # 3000 levels deep
    assert evaluate_iterative(deep) == 3000
    try:
        evaluate(deep)
        raise AssertionError("the recursive version should run out of stack")
    except RecursionError:
        pass


# =====================================================
# FOLLOW-UP 3: What about division?
# =====================================================
# Answer: state the contract: integers or floats, TRUNCATE toward zero or
# floor (Python's // floors, which differs for negative numbers), and what
# happens on division by zero. divide() above truncates toward zero and
# raises ZeroDivisionError.
# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert divide(7, 2) == 3
```

```
    assert divide(-7, 2) == -3                  # 7 // 2 would give -4 for -7 // 2
    try:
        divide(1, 0)
        raise AssertionError("division by zero")
    except ZeroDivisionError:
        pass


# ============================================================
# FOLLOW-UP 4: Variables
# ============================================================
# Answer: already supported: a name that is not followed by "(" is looked
# up in `env`, and an unknown variable is an error.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
```

```
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write a small tokenizer for identifiers, integers, commas, and parentheses.
Practice a recursive parseExpr() that returns both the value and the next index.
Drill the variable-arity follow-up by collecting arguments until) instead of assuming exactly two.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 28 OF 43

Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

5
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M	141. Shortest Path Nodes and Escaping a Pursuer	1Point3Acres * 2026-09-29 *
M	142. Radio Broadcast Reachability	ColorStack * 2026-01-30 *
M	143. Minimum Direction Violations in a Directed Graph	1Point3Acres * 2026-07-19 *
M	144. Compute shortest paths with blocked nodes	PracHub * 2025-09-06 *
M	145. Shortest Directed Cycle Through a Node	ND

CODE TEMPLATE | GROUP 28 OF 43 | 5 QUESTIONS, ONE PATTERN

Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly

The one idea these questions share

The same frontier loop as Tier 1, with three small upgrades: a step can cost 0 or 1 (free steps go to the front of the queue), you can return the moment you reach the goal, and the neighbours of a node can be computed on the fly instead of stored.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import deque


def cheapest_steps(start, neighbors, goal=None):
    INF = float("inf")
    dist = {start: 0}
    queue = deque([start])
    while queue:
        u = queue.popleft()
        if u == goal:                       # CHANGE 1: stop early at the goal
            return dist[u]
        for v, w in neighbors(u):           # CHANGE 2: w is 1, or 0 for a free step
            if dist.get(v, INF) > dist[u] + w:
                dist[v] = dist[u] + w
                if w == 0:
                    queue.appendleft(v)     # free step: goes to the front
                else:
                    queue.append(v)
    return dist
```

2. Helper functions that usually get written next to it

```
def adjacency(n, edges, directed=True):
    adj = [[] for _ in range(n)]
    for u, v in edges:
        adj[u].append((v, 1))
        if not directed:
            adj[v].append((u, 1))
    return adj


def within(points, radius):                # neighbours worked out on the fly
    limit = radius * radius
    def neighbors(i):
        x, y = points[i]
        return [(j, 1) for j, (a, b) in enumerate(points)
                if j != i and (x - a) ** 2 + (y - b) ** 2 <= limit]
    return neighbors
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
141	Shortest Path Nodes and Escaping a Pursuer	run it from a, from t and from b; a node is on a shortest a-to-t path when da + dt == da[t]; compare with the pursuer's distance db
142	Radio Broadcast Reachability	neighbors(u) = every radio within the radius, worked out with squared distances; goal = the target radio, answer = whether it was reached
143	Minimum Direction Violations in a Directed Graph	an edge in its own direction costs w = 0, the same edge walked backwards costs w = 1; free steps use appendleft

#	Question	What you change in the template
144	Compute shortest paths with blocked nodes	neighbors skip every blocked node; if the source itself is blocked the answer is all -1; unreachable nodes stay -1
145	Shortest Directed Cycle Through a Node	start = v; the goal is an edge that leads back to v, so return dist[u] + 1 the moment a neighbour equals v

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 141 of 197 | topic 28 of 43: Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly | 1 of 5

Template fit: run it from a, from t and from b; a node is on a shortest a-to-t path when da + dt == da[t]; compare with the pursuer's distance db | * asked 21 days ago * | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 141 of 197

Shortest Path Nodes and Escaping a Pursuer

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 272/288**

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/shortest-path-nodes-and-pursuer-escape

QUESTION

In an unweighted, undirected graph (an adjacency dict) Alice is at node a and wants to reach the target t. Return the length (number of edges) of the shortest path from a to t found with BFS. The follow-ups ask for every node that lies on some shortest path, and whether Alice can reach t before a pursuer Bob.

Function signature

def shortest_length(adj: dict, a, t) -> int

EXAMPLES

Example 1	
Input	`{0: [1, 2], 1: [0, 3], 2: [0, 3], 3: [1, 2]}, 0, 3`
Output	`2`
Why	`0 -> 1 -> 3 and 0 -> 2 -> 3 both take 2 edges.`

Example 2	
Input	`{0: []}, 0, 0`
Output	`0`
Why	`A start that is the target needs 0 edges.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(graph, alice, bob, target):
    dA = bfs_dist(graph, alice)
    dB = bfs_dist(graph, bob)
    paths = all_paths(graph, alice, target)
    for path in paths:
        if len(path) - 1 != dA[target]:
            continue  # not a shortest path
        if all(dB[v] > dA[v] for v in path):
            return True  # Bob is always late
    return False
```

BRUTE FORCE

TIME	SPACE
exponential in the worst ... exponential in the worst case	**O(paths)**

WHY IT IS SLOW

The number of shortest paths can be exponential. Three BFS runs (from Alice, the target and Bob) decide every node in O(V + E).

OPTIMAL SOLUTION · Three BFS runs and a safe shortest-path DAG

TIME	SPACE
O(V + E)	**O(V)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if Bob can also meet Alice in the middle of an edge?

Then arriving at the same node at the same time is not the only way to be caught: Alice moving u -> v while Bob moves v -> u swaps places. Add that case by also rejecting an edge (u, v) when dB[v] == dA[u] and dB[u] == dA[v]. Always clarify what 'caught' means first.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Shortest Path Nodes and Escaping a
# Pursuer
#
# In an unweighted, undirected graph (an adjacency dict)
# Alice is at node a and wants to reach the target t.
# Return the length (number of edges) of the shortest path
# from a to t found with BFS. The follow-ups ask for every
# node that lies on some shortest path, and whether Alice
# can reach t before a pursuer Bob.
#
# Signature:
#   def shortest_length(adj: dict, a, t) -> int
#
# Examples:
# {0: [1, 2], 1: [0, 3], 2: [0, 3], 3: [1, 2]}, 0, 3  ->
#     2
# {0: []}, 0, 0  ->  0
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if Bob can also meet Alice in the middle of an
# edge?
#
# INTERVIEW NOTES
# BFS from Alice gives dA and BFS from the target gives
# dT. A node is on some shortest path exactly when dA[v] +
# dT[v] equals the shortest length. For the pursuer, BFS
# from Bob gives dB. State the rule you assume: Alice is
# caught at a node v if Bob can be there no later than she
# is (dB[v] <= dA[v]). Alice is safe when a shortest path
# exists whose every node has dB[v] > dA[v].
```

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from collections import deque

FAR = 10 ** 9


# ========================================================
# MAIN QUESTION
```

```
# ========================================================
# ---- main question: helper functions ----
def bfs(adj, s):
    # distance from s to every node it can reach
    dist = {s: 0}
    queue = deque([s])
    while queue:
        u = queue.popleft()
        for v in adj[u]:
            if v not in dist:
                dist[v] = dist[u] + 1
                queue.append(v)
    return dist


# ---- main question: main function ----
# Idea: BFS from Alice (a) gives the length of the shortest path to t.
# Complexity: Time O(V + E), Space O(V)
def shortest_length(adj, a, t):
    return bfs(adj, a).get(t)


# the nodes that lie on some shortest path a -> t
# Complexity: Time O(V + E), Space O(V)
def nodes_on_shortest_paths(adj, a, t):
    da = bfs(adj, a)
    dt = bfs(adj, t)
    result = set()
    if t not in da:
        return result
    for v in da:
        # on a shortest path exactly when the two distances add up to the full length
        if v in dt and da[v] + dt[v] == da[t]:
            result.add(v)
    return result


# ---- main question: tests ----
def test_main_question():
    # a square 0-1-3 and 0-2-3 plus a tail 3-4
    adj = {0: [1, 2], 1: [0, 3], 2: [0, 3], 3: [1, 2, 4], 4: [3]}
    assert shortest_length(adj, 0, 4) == 3
    assert nodes_on_shortest_paths(adj, 0, 4) == {0, 1, 2, 3, 4}
    assert shortest_length(adj, 0, 9) is None
```

```
# ========================================================
# FOLLOW-UP 1: What if Bob can also meet Alice in the middle of an edge?
# ========================================================
# ---- follow-up 1: helper functions ----
```

```
def step_is_safe(u, v, da, db, meet_on_edge):
    # may Alice walk from u to v? Bob must reach v strictly after Alice does.
    if db.get(v, FAR) <= da[v]:
        return False
    if meet_on_edge:
        # they would swap places inside the edge u - v
        if db.get(v, FAR) == da[u] and db.get(u, FAR) == da[v]:
            return False
    return True


# ---- follow-up 1: function ----
# Answer: Alice only needs to follow shortest-path layers, and every
# step u -> v must be safe. With meet_on_edge=True a swap on an edge
# counts as being caught.
# Complexity: Time O(V + E), Space O(V)
def can_escape(adj, a, t, b, meet_on_edge=False):
    da = bfs(adj, a)
    db = bfs(adj, b)
    if t not in da:
        return False
    if db.get(a, FAR) <= 0:                  # Bob stands on Alice's start
        return False
    on_path = nodes_on_shortest_paths(adj, a, t)
    seen = {a}
    frontier = [a]
    while frontier:
        next_frontier = []
        for u in frontier:
            for v in adj[u]:
                if v not in on_path or v in seen or da[v] != da[u] + 1:
                    continue
                if step_is_safe(u, v, da, db, meet_on_edge):
                    seen.add(v)
                    next_frontier.append(v)
        frontier = next_frontier
    return t in seen


# ---- follow-up 1: tests ----
```

```
def test_follow_up_1():
    adj = {0: [1, 2], 1: [0, 3], 2: [0, 3], 3: [1, 2, 4], 4: [3]}
    assert not can_escape(adj, 0, 4, 4)              # Bob waiting at the target catches Alice there
    chain = {0: [1], 1: [0, 2], 2: [1, 3], 3: [2]}
    assert not can_escape(chain, 0, 3, 3)            # Bob starts at the target and reaches it first
    assert can_escape(chain, 1, 3, 0)                # Bob starts behind Alice: she stays ahead


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Run BFS from both the start and the target; a node v is on some shortest path exactly when distA[v] + distT[v] == distA[target].
For the pursuer follow-up, add a BFS from Bob and compare arrival times node by node along candidate shortest paths; practice turning "exists a safe shortest path" into a search over only those on-path nodes where Alice arrives strictly before Bob.
Test with disconnected targets, Bob starting on the target, and multiple equally short routes where only some are safe.

STUDY PATH: question 142 of 197 | topic 28 of 43: Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly | 2 of 5

Template fit: neighbors(u) = every radio within the radius, worked out with squared distances; goal = the target radio, answer = whether it was reached | * asked 263 days ago * | frequency Single report

MEDIUM | IN APP LC 547 | ColorStack | technical interview | 142 of 197

Radio Broadcast Reachability

ASKED 2026-01-30 | 263 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 40.7 | 30 DAYS 0 | PRIORITY 105/288

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly

Optimal answer from: your app's interview approach (LeetCode #547)

LeetCode match: LC 547 Number of Provinces / LC 1971 Find if Path Exists (implicit graph)

Frequency: Single report | Last asked: 2026-01-30 | Roles: SWE Intern

QUESTION

You are given a list of radios placed on a 2D plane, each described by its (x, y) coordinates. Every radio can broadcast to any other radio within a fixed Euclidean distance radius (inclusive). A message can travel from one radio to another either directly (within radius) or through a chain of intermediate radios that each relay it to a next radio within radius . Given the radios, the broadcast radius , a source index, and a target index, determine whether a message originating at source can reach target through zero or more hops. Return True or False .

Function signature

```
from typing import List, Tuple
def can_reach(radios: List[Tuple[float, float]],
              radius: float,
              source: int,
              target: int) -> bool:
    ...
```

Follow-up (as reported): Model the radios as nodes and connect any two within radius , then run BFS (or DFS / union-find) from source . To avoid floating-point issues, compare squared distances against radius * radius .

EXAMPLES

Example 1	
Input	`radios = [(0, 0), (1, 0), (2, 0), (5, 0)], radius = 1.5, source = 0, target = 2`
Output	`True`
Why	`0->1 (dist 1) and 1->2 (dist 1) are each within 1.5, so 2 is reachable.`

Example 2	
Input	`radios = [(0, 0), (1, 0), (2, 0), (5, 0)], radius = 1.5, source = 0, target = 3`
Output	`False`
Why	`Radio 3 is 3 units from radio 2 (> 1.5) and farther from the rest; no chain of hops reaches it.`

QUESTION

Example 3	
Input	`radios = [(0, 0), (3, 4)], radius = 5.0, source = 0, target = 1`
Output	`True`
Why	`The Euclidean distance is exactly 5.0, which is within radius.`

CONSTRAINTS

- 1 <= len(radios) <= 2000
- Coordinates are integers or floats with -10^4 <= x, y <= 10^4 . radius >= 0 .
- 0 <= source, target < len(radios) .
- Two radios a and b are directly connected when sqrt((ax-bx)^2 + (ay-by)^2) <= radius .
- If source == target , the answer is True .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_can_reach(
    radios, radius, source, target
):
    reached = {source}
    changed = True
    while changed:
        changed = False
        for i in list(reached):
            for j, (x, y) in enumerate(radios):
                dx, dy = (
                    radios[i][0] - x,
                    radios[i][1] - y,
                )
                if (
                    j not in reached
                    and dx * dx + dy * dy
                    <= radius * radius
                ):
                    reached.add(j)
                    changed = True
    return target in reached
```

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

It re-scans all pairs after every round although a queue visits each radio once.

OPTIMAL SOLUTION · BFS over the implicit graph

TIME	SPACE
O(n^2)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Thousands of source/target queries on the same radios.

Build the connected components once with union-find (O(n^2) pair checks), then every query is two finds. Radios in one component can reach each other.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Radio Broadcast Reachability
#
# You are given a list of radios placed on a 2D plane,
# each described by its (x, y) coordinates. Every radio
# can broadcast to any other radio within a fixed
# Euclidean distance radius (inclusive). A message can
# travel from one radio to another either directly (within
# radius ) or through a chain of intermediate radios that
# each relay it to a next radio within radius . Given the
# radios, the broadcast radius , a source index, and a
# target index, determine whether a message originating at
# source can reach target through zero or more hops.
# Return True or False . Function signature: from typing
# import List, Tuple def can_reach(radios:
# List[Tuple[float, float]], radius: float, source: int,
# target: int) -> bool: ... Follow-up (as reported): Model
# the radios as nodes and connect any two within radius ,
# then run BFS (or DFS / union-find) from source .
#
# FOLLOW-UPS (answered below, in this order):
# 1. Thousands of source/target queries on the same
# radios.
#
# INTERVIEW NOTES
# Radios are nodes, and two radios are joined when their
# squared distance is at most radius squared (squared
# distances avoid sqrt and float error). Run a BFS from
# the source over this implicit graph and see whether it
# reaches the target. Checking all pairs at each step is
# O(n^2) in total.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
```

```
def squared_distance(p, q):
    # compare squared distances so no square root is needed
    dx = p[0] - q[0]
    dy = p[1] - q[1]
    return dx * dx + dy * dy
```

```python

# ---- main question: main function ----
# Idea: the radios are the nodes of an implicit graph: two radios are
# neighbours when they are within `radius`. BFS from the source radio;
# every radio enters the queue once.
# Complexity: Time O(n^2) distance checks, Space O(n)
def can_reach(radios, radius, source, target):
    limit = radius * radius
    seen = [False] * len(radios)
    seen[source] = True
    queue = deque([source])
    while queue:
        u = queue.popleft()
        if u == target:
            return True
        for v in range(len(radios)):
            if not seen[v] and squared_distance(radios[u], radios[v]) <= limit:
                seen[v] = True
                queue.append(v)
    return False


# ---- main question: tests ----
def test_main_question():
    line = [(0, 0), (1, 0), (2, 0), (5, 0)]
    assert can_reach(line, 1.5, 0, 2) is True
    assert can_reach(line, 1.5, 0, 3) is False
    assert can_reach([(0, 0), (3, 4)], 5.0, 0, 1) is True
    assert can_reach(line, 1, 3, 3) is True


# ============================================================
# FOLLOW-UP 1: Thousands of source/target queries on the same radios
# ============================================================
# ---- follow-up 1: helper functions ----
# Complexity: Time build O(n^2), each query O(alpha(n)) (about O(1)), Space O(n)
class RadioNetwork:
    # build the connected groups once with union-find, then every query
    # is two finds: radios in one group can reach each other
```

```python
    def __init__(self, radios, radius):
        n = len(radios)
        self.parent = list(range(n))
        for i in range(n):
            for j in range(i + 1, n):
                if squared_distance(radios[i], radios[j]) <= radius * radius:
                    self.parent[self.find(i)] = self.find(j)

    def find(self, a):
        while self.parent[a] != a:
            self.parent[a] = self.parent[self.parent[a]]
            a = self.parent[a]
        return a

    def can_reach(self, source, target):
        return self.find(source) == self.find(target)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    line = [(0, 0), (1, 0), (2, 0), (5, 0)]
    network = RadioNetwork(line, 1.5)
    assert network.can_reach(0, 2) is True
    assert network.can_reach(0, 3) is False
    assert network.can_reach(3, 3) is True
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 143 of 197 | topic 28 of 43: Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly | 3 of 5

Template fit: an edge in its own direction costs w = 0, the same edge walked backwards costs w = 1; free steps use appendleft | * asked 93 days ago * | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 143 of 197

Minimum Direction Violations in a Directed Graph

ASKED 2026-07-19 | 93 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 10.6** **30 DAYS 0** **PRIORITY 189/288**

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1466 Reorder Routes to Make All Paths Lead to the City Zero (related)

Frequency: Single report | Last asked: 2026-07-19 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/minimum-direction-violations-directed-graph

QUESTION

A directed graph has n nodes. Walking along an edge in its own direction is free; walking against it costs 1 (a direction violation). Return the minimum total cost of a walk from start to end, or -1 if end cannot be reached at all.

Function signature

def min_violations(n: int, edges: list, start: int, end: int) -> int

EXAMPLES

Example 1	
Input	`5, [[0, 1], [2, 1], [2, 3], [4, 3]], 0, 4`
Output	`2`
Why	`Path 0 -> 1 (free), 1 -> 2 against edge 2 -> 1 (cost 1), 2 -> 3 (free), 3 -> 4 against edge 4 -> 3 (cost 1): total 2.`

Example 2	
Input	`3, [[0, 1]], 0, 2`
Output	`-1`
Why	`Node 2 has no edge at all, so it cannot be reached.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, start, end):
    arcs = []
    for u, v in edges:
        arcs.append((u, v, 0))  # along the edge
        arcs.append((v, u, 1))  # against it
    dist = [INF] * n
    dist[start] = 0
    for _ in range(n - 1):
        for u, v, w in arcs:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    return dist[end]
```

BRUTE FORCE

TIME	SPACE
O(V * E)	O(V)

WHY IT IS SLOW

Every round scans all edges, V - 1 rounds: O(V * E). Costs of only 0 and 1 allow a deque search (0-1 BFS) in O(V + E).

OPTIMAL SOLUTION · 0-1 BFS on the doubled graph

TIME	SPACE
O(n + m)	O(n + m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is 0-1 BFS correct, and how does it compare with Dijkstra?

The deque always holds nodes whose distances differ by at most one: front elements are at distance d, back elements at d + 1. Relaxing a zero-weight edge keeps the distance at d (front), a one-weight edge makes d + 1 (back). That is the same order a heap would produce, so it is Dijkstra without the log factor: O(n + m) versus O((n + m) log n).

FOLLOW-UP 2 What if edge costs can be arbitrary non-negative numbers?

Use Dijkstra with a heap on the same doubled graph (forward cost = 0 or the edge cost, backward cost = the violation cost). With negative costs use Bellman-Ford and watch for negative cycles.

FOLLOW-UP 3 Edge costs are arbitrary non-negative numbers (different for each direction).

Use Dijkstra with a heap on the same doubled graph, giving each arc its own cost. 0-1 BFS only works because the costs are 0 or 1.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Minimum Direction Violations in a
# Directed Graph
#
# A directed graph has n nodes. Walking along an edge in
# its own direction is free; walking against it costs 1 (a
# direction violation). Return the minimum total cost of a
# walk from start to end, or -1 if end cannot be reached
# at all.
#
# Signature:
#   def min_violations(n: int, edges: list, start: int, end: int) -> int
#
# Examples:
# 5, [[0, 1], [2, 1], [2, 3], [4, 3]], 0, 4  ->  2
# 3, [[0, 1]], 0, 2  ->  -1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is 0-1 BFS correct, and how does it compare with
# Dijkstra?
# 2. What if edge costs can be arbitrary non-negative
```

```
# numbers?
# 3. Edge costs are arbitrary non-negative numbers
# (different for each direction).
#
# INTERVIEW NOTES
# Going against an edge costs 1 and going with it costs 0,
# so it is a shortest-path problem with costs of only 0
# and 1. Plain BFS counts edges, not violations, so it is
# the wrong tool. A deque lets 0-cost moves jump the queue
# (push front) and 1-cost moves wait their turn (push
# back), which is Dijkstra without the heap.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import heapq

INF = float("inf")
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def build_graph(n, edges):
    # every edge u -> v is also usable backwards: forward costs 0, backward costs 1
    graph = []
    for i in range(n):
        graph.append([])
    for u, v in edges:
        graph[u].append((v, 0))
        graph[v].append((u, 1))
    return graph


# ---- main question: main function ----
# Idea: with every cost either 0 or 1 a deque is enough (0-1 BFS):
# a free step goes to the FRONT of the deque, a cost-1 step to the BACK,
# so cells are always taken out in order of distance.
# Complexity: Time O(V + E), Space O(V + E)
def min_violations(n, edges, start, end):
    graph = build_graph(n, edges)
    dist = [INF] * n
    dist[start] = 0
    dq = deque([start])
    while dq:
        u = dq.popleft()
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    if dist[end] == INF:
        return -1
    return dist[end]


# ---- main question: tests ----
```

```
def test_main_question():
    edges = [(0, 1), (2, 1), (2, 3), (4, 3)]
    assert min_violations(5, edges, 0, 4) == 2        # 0 -> 1 -> 2 -> 3 -> 4 reverses two edges
    assert min_violations(5, edges, 0, 0) == 0
    assert min_violations(3, [(0, 1)], 0, 2) == -1
```

```


# ========================================================
# FOLLOW-UP 1: Why is 0-1 BFS correct, and how does it compare with Dijkstra?
# ========================================================
# Answer: the deque always holds distances d and d + 1 only, in order,
# so each node is settled in distance order exactly like Dijkstra, but
# with O(1) deque operations instead of O(log n) heap operations: O(V + E).


# ========================================================
# FOLLOW-UP 2: What if edge costs can be arbitrary non-negative numbers?
# ========================================================


# ========================================================
# FOLLOW-UP 3: Edge costs are arbitrary non-negative numbers (different for each direction)
# ========================================================
# ---- follow-up 3: function ----
# Answer: use Dijkstra with a heap on the same doubled graph and give
# each direction its own cost. 0-1 BFS only works because the costs are 0 or 1.
# Complexity: Time O((V + E) log V), Space O(V + E)
def cheapest_route(n, edges, start, end):
    # edges are (u, v, forward_cost, backward_cost)
    graph = []
    for i in range(n):
        graph.append([])
    for u, v, forward, backward in edges:
        graph[u].append((v, forward))
        graph[v].append((u, backward))
    dist = [INF] * n
    dist[start] = 0
    heap = [(0, start)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue                        # an outdated heap entry
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    if dist[end] == INF:
        return -1
    return dist[end]
```

```


# ---- follow-up 3: tests ----
def test_follow_up_3():
    edges = [(0, 1), (2, 1), (2, 3), (4, 3)]
    costed = []
    for u, v in edges:
        costed.append((u, v, 0, 1))                 # the same graph, every reversal costs 1
    assert cheapest_route(5, costed, 0, 4) == 2
    assert cheapest_route(2, [(0, 1, 5, 2)], 0, 1) == 5
    assert cheapest_route(2, [(0, 1, 5, 2)], 1, 0) == 2


# ========================================================
# RUN EVERYTHING
```

```
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement 0-1 BFS from memory and rehearse the deque invariant for zero-weight versus one-weight transitions.
Compare the proof and complexity with Dijkstra on the same transformed graph.
Test unreachable destinations, parallel edges, cycles, and start == end.

STUDY PATH: question 144 of 197 | topic 28 of 43: Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly | 4 of 5

Template fit: neighbors skip every blocked node; if the source itself is blocked the answer is all -1; unreachable nodes stay -1 | * asked 409 days ago *

MEDIUM | PracHub | Technical Screen | 144 of 197

★ TOP 75: rank 65 in the Medium and Hard priority list

★ Compute shortest paths with blocked nodes

POSTED 2025-09-06 | 409 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 37.9 | 30 DAYS 33 | PRIORITY 67/288

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #743 Network Delay Time

LeetCode match: LC 743 Network Delay Time (Dijkstra with blocked nodes skipped)

Posted on PracHub: 2025-09-06 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/compute-shortest-paths-with-blocked-nodes

QUESTION

Given a graph with nodes and edges and a designated source node s, compute the shortest distance from s to every other node. Some nodes are inaccessible (blocked) and cannot be traversed or reached; treat them as removed from the graph. Return a distance for each node where unreachable nodes (including blocked ones) are -1. Describe your algorithm, data structures, and time/space complexity, and note any differences for directed vs. undirected graphs.

Given n nodes labeled 0..n-1, edges, a source, and blocked nodes, return shortest unweighted distances from the source. Blocked and unreachable nodes must be -1. Set directed=True to treat edges as directed.
Constraints
Nodes are labeled 0 through n-1Edges are unweighted

EXAMPLES

```
Input: (5, [[0, 1], [1, 2], [0, 3], [3, 4]], 0, [3], False)
Expected Output: [0, 1, 2, -1, -1]
Input: (4, [[0, 1], [1, 2], [2, 3]], 0, [], True)
Expected Output: [0, 1, 2, 3]
```

Hints

Treat blocked nodes as removed before BFS.For directed graphs, only add the given edge direction.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, blocked, s):
    keep = []
    for u, v, w in edges:
        if u not in blocked and v not in blocked:
            keep.append((u, v, w))
    d = floyd_warshall(n, keep)  # all pairs
    return d[s]
```

TIME	SPACE
O(V^3)	O(V^2)

BRUTE FORCE

WHY IT IS SLOW

All-pairs distances cost O(V^3) and O(V^2) memory when one source is needed. A BFS from the source that never enters a blocked node is O(V + E).

OPTIMAL SOLUTION · BFS that never enters a blocked node

TIME	SPACE
O(V + E)	**O(V + E)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The edges have weights.

BFS is no longer correct: use Dijkstra with a heap and skip blocked nodes the same way. O((V + E) log V). Directed and undirected differ only in whether you add the reverse edge.

FOLLOW-UP 2 Directed versus undirected.

For an undirected graph add each edge in both directions; for a directed one only as given. A node may be reachable in one case and not in the other, so state which you assume.

FOLLOW-UP 3 Nodes get blocked over time and you must keep distances up to date.

Decremental shortest paths is hard in general; for few updates rerun BFS, and for many offline queries process the changes in reverse order (unblocking only improves distances).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Compute shortest paths with blocked nodes
#
# Given a graph with nodes and edges and a designated
# source node s, compute the shortest distance from s to
# every other node. Some nodes are inaccessible (blocked)
# and cannot be traversed or reached; treat them as
# removed from the graph. Return a distance for each node
# where unreachable nodes (including blocked ones) are -1.
# Describe your algorithm, data structures, and time/space
# complexity, and note any differences for directed vs.
# undirected graphs. Overview: This question evaluates
# proficiency with graph algorithms, shortest-path and
# reachability concepts when nodes are removed or blocked,
# and the ability to analyze time and space complexity
# across directed and undirected graphs.Given n nodes
# labeled 0..n-1, edges, a source, and blocked nodes,
# return shortest unweighted distances from the source.
# Blocked and unreachable nodes must be -1. Set
# directed=True to treat edges as directed.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The edges have weights.
# 2. Directed versus undirected.
# 3. Nodes get blocked over time and you must keep
# distances up to date.
```

```
#
# INTERVIEW NOTES
# Blocked nodes are simply not in the graph, so skip them
# when you queue neighbours and never start from one.
# After that it is plain BFS from the source: first
# discovery gives the shortest distance. State the edge
# cases: a blocked source makes everything unreachable,
# and directed edges must only be added in their given
# direction.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import heapq
```

```
INF = float("inf")


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def make_adjacency(n, edges, directed):
    # neighbour lists; an undirected edge is stored in both directions
    adj = []
    for i in range(n):
        adj.append([])
    for edge in edges:
        u, v = edge[0], edge[1]
        weight = 1
        if len(edge) == 3:
            weight = edge[2]
        adj[u].append((v, weight))
        if not directed:
            adj[v].append((u, weight))
    return adj


# ---- main question: main function ----
# Idea: a BFS that never enters a blocked node. If the source itself is
# blocked everything is -1; unreachable and blocked nodes also get -1.
# Complexity: Time O(V + E), Space O(V + E)
def blocked_distances(n, edges, source, blocked, directed=False):
    if source in blocked:
        return [-1] * n
    adj = make_adjacency(n, edges, directed)
    dist = [-1] * n
    dist[source] = 0
    queue = deque([source])
    while queue:
        u = queue.popleft()
        for v, w in adj[u]:
            if v not in blocked and dist[v] == -1:
                dist[v] = dist[u] + 1
                queue.append(v)
    return dist


# ---- main question: tests ----
def test_main_question():
```

```
    edges = [(0, 1), (1, 2), (0, 3), (3, 2), (2, 4)]
    assert blocked_distances(5, edges, 0, []) == [0, 1, 2, 1, 3]
    assert blocked_distances(5, edges, 0, [1])[2] == 2                # blocking 1 forces the route throug
h 3
    assert blocked_distances(5, edges, 0, [2]) == [0, 1, -1, 1, -1]    # blocking 2 cuts node 4 off
    assert blocked_distances(3, edges[:2], 0, [0]) == [-1, -1, -1]     # the source is blocked
    path = [(0, 1), (1, 2)]
    assert blocked_distances(3, path, 2, [], True) == [-1, -1, 0]      # directed edges go one way only


# =====================================================
# FOLLOW-UP 1: The edges have weights
# =====================================================


# =====================================================
# FOLLOW-UP 2: Directed versus undirected
# =====================================================
# ---- follow-up 1: function ----
# Answer: BFS is no longer correct with weights; use Dijkstra with a
# heap and skip blocked nodes the same way: O((V + E) log V).
# Directed and undirected differ only in whether the reverse edge is
# added (the `directed` flag of make_adjacency).
# Complexity: Time O((V + E) log V), Space O(V + E)
def blocked_distances_weighted(n, edges, source, blocked, directed=False):
    if source in blocked:
        return [-1] * n
    adj = make_adjacency(n, edges, directed)
    dist = [INF] * n
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in adj[u]:
            if v not in blocked and d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    answer = []
    for d in dist:
        if d == INF:
            answer.append(-1)
        else:
            answer.append(d)
    return answer
```

```


# ---- follow-up 1: tests ----
def test_follow_up_1():
    weighted = [(0, 1, 5), (1, 2, 1), (0, 2, 10)]
    assert blocked_distances_weighted(3, weighted, 0, []) == [0, 5, 6]
    assert blocked_distances_weighted(3, weighted, 0, [1])[2] == 10


# =====================================================
# FOLLOW-UP 3: Nodes get blocked over time and you must keep distances up to date
# =====================================================
# Answer: blocking a node can only make distances larger. Recompute
# the BFS from the source after each change, or only for the nodes
# whose shortest path went through the blocked node.


# =====================================================
# RUN EVERYTHING
```

```
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 145 of 197 | topic 28 of 43: Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly | 5 of 5

Template fit: start = v; the goal is an edge that leads back to v, so return dist[u] + 1 the moment a neighbour equals v | ND | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 145 of 197

Shortest Directed Cycle Through a Node

STUDY: TALK **ALL TIME 5** **30 DAYS 0** **PRIORITY 198/288**

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 2608 Shortest Cycle in a Graph (related)

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/shortest-cycle-through-node

QUESTION

Given a directed graph with n nodes and a list of edges (a, b), return the length (number of edges) of the shortest cycle that passes through node v, or -1 if v is on no cycle.

Function signature

def shortest_cycle_through(n: int, edges: list, v: int) -> int

EXAMPLES

Example 1	
Input	`3, [(0, 1), (1, 2), (2, 0)], 0`
Output	`3`
Why	`0 -> 1 -> 2 -> 0 is a cycle of 3 edges.`

Example 2	
Input	`3, [(0, 1), (1, 2)], 0`
Output	`-1`
Why	`No edge leads back to 0.`

Example 3	
Input	`3, [(0, 1), (1, 0), (1, 2), (2, 0)], 0`
Output	`2`
Why	`A shorter cycle 0 -> 1 -> 0 exists next to the longer one.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
best = INF
def dfs(u, length, on_path):
    global best
    for w in graph[u]:
        if w == v:
            best = min(best, length + 1)
        elif w not in on_path:
            dfs(w, length + 1, on_path | {w})
dfs(v, 0, {v})
answer = -1 if best == INF else best
```

TIME	SPACE
exponential	O(V)

WHY IT IS SLOW

The number of simple paths can be exponential in the graph size. A breadth-first search from v reaches the shortest cycle in O(V + E) because it visits nodes in order of distance.

OPTIMAL SOLUTION · BFS from v, close the cycle on the first edge back to v

TIME	SPACE
O(V + E)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you find the shortest cycle in the whole graph?

Run the same BFS from every vertex and take the minimum: O(V * (V + E)). For undirected graphs, BFS from each node and treat any non-tree edge as closing a cycle of length dist[u] + dist[w] + 1 (the LeetCode 2608 version).

FOLLOW-UP 2 What if edges have positive weights?

Replace BFS with Dijkstra from v and close the cycle at the cheapest dist[u] + weight(u, v). That is O((V + E) log V) with a heap. With negative weights use Bellman-Ford and handle negative cycles separately.

FOLLOW-UP 3 The shortest cycle anywhere in the graph (the girth).

Run the same BFS from every node and take the smallest result: O(V * (V + E)). That is the standard way and fine for graphs up to a few thousand nodes.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Shortest Directed Cycle Through a Node
#
# Given a directed graph with n nodes and a list of edges
# (a, b), return the length (number of edges) of the
# shortest cycle that passes through node v, or -1 if v is
# on no cycle.
#
# Signature:
#   def shortest_cycle_through(n: int, edges: list, v: int) -> int
```

```
#
# Examples:
# 3, [(0, 1), (1, 2), (2, 0)], 0  ->  3
# 3, [(0, 1), (1, 2)], 0  ->  -1
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you find the shortest cycle in the whole
# graph?
# 2. What if edges have positive weights?
# 3. The shortest cycle anywhere in the graph (the girth).
#
# INTERVIEW NOTES
# A cycle through v is a path from v back to v, so find
# shortest distances from v and watch for an edge that
# points back at v. Breadth-first search discovers nodes
# in order of distance, so the first edge back to v closes
# the shortest cycle. Do not mark v as 'seen' in a way
# that hides the edge that returns to it.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import heapq

INF = float("inf")


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
```

```
def make_graph(n, edges):
    # adjacency lists of (neighbour, cost); an edge may be (a, b) or (a, b, cost)
    adj = []
    for i in range(n):
        adj.append([])
    for edge in edges:
        a, b = edge[0], edge[1]
        cost = 1
        if len(edge) == 3:
            cost = edge[2]
        adj[a].append((b, cost))
    return adj


# ---- main question: main function ----
# Idea: a directed cycle through v is a path v -> u followed by an edge
# u -> v. BFS reaches nodes in order of distance, so the first edge we
# see that leads back to v closes the shortest cycle: dist[u] + 1.
# Complexity: Time O(V + E), Space O(V + E)
def shortest_cycle_through(n, edges, v):
    adj = make_graph(n, edges)
    dist = {v: 0}
    queue = deque([v])
    while queue:
        u = queue.popleft()
        for w, cost in adj[u]:
            if w == v:
                return dist[u] + 1
```

```
            if w not in dist:
                dist[w] = dist[u] + 1
                queue.append(w)
    return -1


# ---- main question: tests ----
def test_main_question():
    ring = [(0, 1), (1, 2), (2, 0)]
    assert shortest_cycle_through(3, ring, 0) == 3
    assert shortest_cycle_through(3, [(0, 1), (1, 2)], 0) == -1
    assert shortest_cycle_through(2, [(0, 0)], 0) == 1       # a self loop is a cycle of length 1


# ======================================================
# FOLLOW-UP 1: How do you find the shortest cycle in the whole graph?
# ======================================================
```

```
# ======================================================
# FOLLOW-UP 3: The shortest cycle anywhere in the graph (the girth)
# ======================================================
# ---- follow-up 1: function ----
# Answer: run the same BFS from every node and take the smallest
# result: O(V * (V + E)), fine up to a few thousand nodes. (For an
# undirected graph, any edge that is not a tree edge closes a cycle of
# length dist[u] + dist[w] + 1: the LeetCode 2608 version.)
# Complexity: Time O(V * (V + E)), Space O(V + E)
def girth(n, edges):
    best = -1
    for v in range(n):
        size = shortest_cycle_through(n, edges, v)
        if size > 0 and (best == -1 or size < best):
            best = size
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    ring = [(0, 1), (1, 2), (2, 0)]
    both = ring + [(1, 2), (2, 1)]                  # a shorter cycle exists: 1 <-> 2
    assert girth(3, both) == 2
    assert girth(3, [(0, 1), (1, 2)]) == -1


# ======================================================
# FOLLOW-UP 2: What if edges have positive weights?
# ======================================================
# ---- follow-up 2: function ----
# Answer: replace BFS by Dijkstra from v and close the cycle at the
# cheapest dist[u] + weight(u, v): O((V + E) log V) with a heap. With
# negative weights use Bellman-Ford and handle negative cycles separately.
# Complexity: Time O((V + E) log V), Space O(V + E)
def weighted_cycle_through(n, edges, v):
    adj = make_graph(n, edges)
    dist = {v: 0}
    heap = [(0, v)]
    best = INF
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist.get(u, INF):
            continue
```

```
        for w, cost in adj[u]:
            if w == v:
                if d + cost < best:
                    best = d + cost
            elif d + cost < dist.get(w, INF):
                dist[w] = d + cost
                heapq.heappush(heap, (d + cost, w))
    if best == INF:
        return -1
    return best


# ---- follow-up 2: tests ----
def test_follow_up_2():
    weighted = [(0, 1, 1), (1, 0, 2), (0, 2, 1), (2, 0, 9)]
    assert weighted_cycle_through(3, weighted, 0) == 3     # the cheap way back is 0 -> 1 -> 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write the directed BFS-from-source version from memory in under 12 min.
Practice the "re-entry detection" pattern (don't mark v visited until after pop).

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 29 OF 43

Topological sort (Kahn) and cycle detection

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

5
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Topological sort (Kahn) and cycle detection | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M	146. Course Scheduling and Dependency Variants	1Point3Acres * 2026-10-04 *
M	147. Validate Race Results (Cycle Detection)	ColorStack * 2025-10-01 *
M	148. Solve tree, graph, sliding-window problems	PracHub * 2025-08-04 *
M	149. Validate course catalog dependencies	PracHub * 2025-08-01 *
M	150. Course Schedule	ND

CODE TEMPLATE | GROUP 29 OF 43 | 5 QUESTIONS, ONE PATTERN

Topological sort (Kahn) and cycle detection

The one idea these questions share

Count the incoming edges of every node, start from the nodes with none, and peel them off one by one. If every node got peeled there is a valid order; if some never did, there is a cycle. The questions only differ in what the nodes are called and whether you return the order or a yes/no.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import deque, defaultdict


def topo_order(nodes, edges):
    adj, indeg = defaultdict(list), {v: 0 for v in nodes}
    for u, v in edges:                          # CHANGE 1: what an edge means
        adj[u].append(v)
        indeg[v] += 1
    queue = deque(v for v in nodes if indeg[v] == 0)
    order = []
    while queue:
        u = queue.popleft()
        order.append(u)
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                queue.append(v)
    return order if len(order) == len(indeg) else None   # CHANGE 2: None means a cycle
```

2. Helper functions that usually get written next to it

```
def nodes_of(edges):                        # when the statement does not list the nodes
    return sorted({x for e in edges for x in e}, key=str)


def has_unknown(nodes, edges):              # prerequisites that name something that does not exist
    known = set(nodes)
    return sorted({x for e in edges for x in e if x not in known}, key=str)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
146	Course Scheduling and Dependency Variants	build the vertex set from the edges too; return the order or None
147	Validate Race Results (Cycle Detection)	edge = winner -> loser; the results are consistent exactly when there is no cycle
148	Solve tree, graph, sliding-window problems	the course-schedule part is can-finish: len(order) == n; the zigzag and window parts are separate small loops
149	Validate course catalog dependencies	first reject prerequisites naming courses that do not exist; then the same peel; report the missing names or the cycle
150	Course Schedule	edge = prerequisite -> course; return the order, or an empty list when there is a cycle

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 146 of 197 | topic 29 of 43: Topological sort (Kahn) and cycle detection | 1 of 5

Template fit: build the vertex set from the edges too; return the order or None | * asked 16 days ago * | frequency Medium

MEDIUM | IN APP LC 210 | 1Point3Acres | onsite-coding | 146 of 197

Course Scheduling and Dependency Variants

ASKED 2026-10-04 | 16 DAYS AGO | LAST 30 DAYS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 209/288**

ABOUT THIS QUESTION

Topic: Topological sort (Kahn) and cycle detection

Optimal answer from: your app's interview approach (LeetCode #210)

LeetCode match: LC 210 Course Schedule II

Frequency: Medium | Last asked: 2026-10-04

Original: https://www.1point3acres.com/interview/problems/company/google/course-schedule-dependency-variants

QUESTION

Topological-ordering family. Variants: course ordering with validity checks and a nested-list extension; merging two arrays whose internal order defines precedence (return false on a conflict cycle); and a damaged-node dependency-path variant that minimizes broken nodes on a route.

Course-ordering variant

Work through the course-ordering problem, including validity checks, complexity analysis, and test cases.
Extend the result to a nested-list format. The grouping rule is unspecified; clarify what each inner list represents before implementing the extension.

Damaged-node and path variant

Remove damaged nodes and their downstream effects, then find a valid path.
A follow-up minimizes broken nodes on a required route. Clarify the graph direction, path endpoints, and meaning of broken nodes before choosing an algorithm.

Merge-two-ordered-sequences variant

Input: two integer arrays a and b. The order of elements inside each array is an ordering constraint — a = [1, 3, 5] means 1 must come before 3, and 3 before 5.
Determine whether the two arrays can be merged into one sequence that preserves the relative order of both. Return false if the constraints conflict; otherwise return true (or any valid merged sequence).

EXAMPLES

Merge-two-ordered-sequences

```
a = [1, 2, 3]
b = [2, 4, 5]
-> true   (one valid merged order: [1, 2, 3, 4, 5])

a = [1, 6, 4]
b = [4, 1]
-> false  (a requires 1 -> 6 -> 4, b requires 4 -> 1: contradiction)
```

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, rules):
    for order in permutations(range(n)):
        pos = {v: i for i, v in enumerate(order)}
        ok = all(pos[a] < pos[b]
                 for a, b in rules)
        if ok:
            return True
    return False
```

TIME	SPACE
O(V! * E)	O(V)

WHY IT IS SLOW

V! orderings. Kahn's algorithm repeatedly takes a vertex with no unmet prerequisites and finishes in O(V + E).

OPTIMAL SOLUTION · Kahn's algorithm (course order, or can two sequences merge)

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you check that an order is valid?

Record each vertex's position in the order, check that every vertex appears exactly once, then check that for every edge (u, v) the position of u is smaller than the position of v. Accept any valid order, not one fixed answer, because independent vertices can swap.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Course Scheduling and Dependency Variants
#
# Topological-ordering family. Variants: course ordering
# with validity checks and a nested-list extension;
# merging two arrays whose internal order defines
# precedence (return false on a conflict cycle); and a
# damaged-node dependency-path variant that minimizes
# broken nodes on a route. Course-ordering variant Work
# through the course-ordering problem, including validity
# checks, complexity analysis, and test cases. Extend the
# result to a nested-list format. The grouping rule is
# unspecified; clarify what each inner list represents
# before implementing the extension. Damaged-node and path
# variant Remove damaged nodes and their downstream
# effects, then find a valid path. A follow-up minimizes
# broken nodes on a required route. Clarify the graph
# direction, path endpoints, and meaning of broken nodes
# before choosing an algorithm. Merge-two-ordered-
# sequences variant Input: two integer arrays a and b.
#
# FOLLOW-UPS (answered below, in this order):
```

```
# 1. How do you check that an order is valid?
#
# INTERVIEW NOTES
# Every 'A must come before B' rule is an edge. A valid
# ordering exists exactly when the graph has no cycle, and
# Kahn's algorithm finds one: repeatedly take a vertex
# with no unmet prerequisites. The two-array merge is the
# same problem: each adjacent pair inside an array is an
# edge and equal values are the same vertex; the merge is
# possible only if the combined graph has no cycle.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
def edges_from_sequences(sequences):
    # each neighbouring pair in a sequence is a rule: the first comes before the second
    edges = []
    for seq in sequences:
        for i in range(len(seq) - 1):
            edges.append((seq[i], seq[i + 1]))
    return edges


# ---- main question: main function ----
# Idea (Kahn's algorithm): an edge (u, v) means u must come before v.
# Count how many edges point into each vertex (indegree), keep a queue
# of vertices with indegree 0, take one, add it to the order and lower
# the indegree of the vertices after it. If not every vertex was taken
# there is a cycle and no order exists (None).
# Complexity: Time O(V + E), Space O(V + E)
def topological_order(vertices, edges):
    indegree = {}
    after = {}
    for v in vertices:
        indegree[v] = 0
        after[v] = []
    for u, v in edges:
        for x in (u, v):
            if x not in indegree:
                indegree[x] = 0                 # a vertex that appears only in an edge
                after[x] = []
        after[u].append(v)
        indegree[v] += 1
    queue = deque()
    for v in indegree:
        if indegree[v] == 0:
            queue.append(v)
    order = []
    while queue:
        u = queue.popleft()
        order.append(u)
        for v in after[u]:
            indegree[v] -= 1
```

```
            if indegree[v] == 0:
                queue.append(v)
    if len(order) == len(indegree):
        return order
    return None
```

```


# the same idea merges two sequences: shared values are the same vertex
# Complexity: Time O(total length of both sequences), Space the same
def merge_two_orders(a, b):
    vertices = set(a) | set(b)
    return topological_order(vertices, edges_from_sequences([a, b]))


# Complexity: Time O(total length of both sequences), Space the same
def can_merge(a, b):
    # is there an order that keeps both sequences in order?
    return merge_two_orders(a, b) is not None


# ---- main question: tests ----
def test_main_question():
    order = topological_order(range(4), [(0, 1), (1, 2)])    # course 3 has no rule at all
    assert len(order) == 4
    assert order.index(0) < order.index(1) < order.index(2)
    assert topological_order(range(2), [(0, 1), (1, 0)]) is None      # a cycle has no order
    assert can_merge([1, 2, 3], [2, 4, 5])
    assert merge_two_orders([1, 2, 3], [2, 4, 5]) is not None
    assert not can_merge([1, 6, 4], [4, 1])
    assert not can_merge([1, 2, 1], [])                       # a repeated value is a conflict with itself


# ======================================================
# FOLLOW-UP 1: How do you check that an order is valid?
# ======================================================
# ---- follow-up 1: function ----
# Answer: write down each vertex's position in the order, check every
# vertex appears exactly once, then check that for every edge (u, v)
# the position of u is smaller than the position of v.
# Complexity: Time O(V + E), Space O(V)
def is_valid_order(order, vertices, edges):
    if len(order) != len(set(vertices)):
        return False
    if set(order) != set(vertices):
        return False
    position = {}
    for i in range(len(order)):
        position[order[i]] = i
    for u, v in edges:
        if position[u] >= position[v]:
```

```
            return False                          # a prerequisite comes too late
    return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    courses = range(4)
    rules = [(0, 1), (1, 2)]
    order = topological_order(courses, rules)
    assert is_valid_order(order, courses, rules)
    assert not is_valid_order([1, 0, 2, 3], courses, rules)     # a swapped order breaks a rule
    assert not is_valid_order([0, 1, 2], courses, rules)        # a vertex is missing
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

In 20 minutes, implement prerequisite ordering with cycle detection; check isolated courses, independent choices, a chain, and a directed cycle.
In 10 minutes, write an independent result checker for vertex coverage and every prerequisite constraint. Accept different valid orders.
In 10 minutes, list the missing grouping and path-cost rules, and explain which algorithm decisions depend on each answer.

STUDY PATH: question 147 of 197 | topic 29 of 43: Topological sort (Kahn) and cycle detection | 2 of 5

Template fit: edge = winner -> loser; the results are consistent exactly when there is no cycle | * asked 384 days ago * | frequency Single report

MEDIUM | IN APP LC 207 | ColorStack | technical interview | 147 of 197

Validate Race Results (Cycle Detection)

ASKED 2025-10-01 | 384 DAYS AGO | OLDER

STUDY: TYPE ALL TIME 51.9 30 DAYS 33 PRIORITY 38/288

ABOUT THIS QUESTION

Topic: Topological sort (Kahn) and cycle detection

Optimal answer from: your app's interview approach (LeetCode #207)

LeetCode match: LC 207 Course Schedule (race results version)

Frequency: Single report | Last asked: 2025-10-01 | Roles: SWE Intern

QUESTION

You are given a list of pairwise race results as (winner, loser) string pairs, meaning winner beat loser . The results are consistent if they can all be true simultaneously under a strict ranking — equivalently, if the directed graph (edge winner -> loser) contains no cycle. Return True if the results are valid (acyclic) and False otherwise.

Function signature

def results_are_valid(results: list[tuple[str, str]]) -> bool:

...

Follow-up (as reported): Recursive DFS can hit Python's recursion limit on long chains; convert to an iterative DFS with an explicit stack, or use Kahn's topological-sort algorithm.

EXAMPLES

Example 1	
Input	`results = [("A","B"), ("B","C")]`
Output	`True`
Why	`A beat B beat C — a consistent ranking A > B > C, no cycle.`

Example 2	
Input	`results = [("A","B"), ("B","C"), ("C","A")]`
Output	`False`
Why	`A > B > C > A is a contradiction (a cycle).`

Example 3	
Input	`results = []`
Output	`True`
Why	`No results impose no contradiction.`

CONSTRAINTS

- 0 <= len(results) <= 10^5 .
- Player names are non-empty strings; duplicate result pairs may appear.
- An empty result list is valid (True).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_valid(results):
    graph = {}
    for w, l in results:
        graph.setdefault(w, []).append(l)
        graph.setdefault(l, [])
    state = {}
    def dfs(u):
        state[u] = 1
        for v in graph[u]:
            if state.get(v) == 1 or (
                v not in state and not dfs(v)
            ):
                return False
        state[u] = 2
        return True
    return all(
        u in state or dfs(u) for u in graph
    )
```

TIME	SPACE
O(V + E)	O(V) recursion

WHY IT IS SLOW

Correct, but a chain of 100,000 results exceeds Python's recursion limit.

OPTIMAL SOLUTION · Kahn's algorithm

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Recursive DFS hits Python's recursion limit on long chains. What do you do?

Either use Kahn's algorithm (already iterative) or run the three-colour DFS with an explicit stack of (node, next neighbour index) so the depth is limited by memory, not by the interpreter.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Validate Race Results (Cycle Detection)
#
# You are given a list of pairwise race results as
# (winner, loser) string pairs, meaning winner beat loser
# . The results are consistent if they can all be true
# simultaneously under a strict ranking — equivalently, if
# the directed graph (edge winner -> loser ) contains no
# cycle. Return True if the results are valid (acyclic)
# and False otherwise. Function signature: def
# results_are_valid(results: list[tuple[str, str]]) ->
# bool: ... Follow-up (as reported): Recursive DFS can hit
```

```
# Python's recursion limit on long chains; convert to an
# iterative DFS with an explicit stack, or use Kahn's
# topological-sort algorithm.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Recursive DFS hits Python's recursion limit on long
# chains. What do you do?
#
# INTERVIEW NOTES
# Pairwise results are consistent exactly when the
# directed graph winner -> loser has no cycle, so this is
# cycle detection. Kahn's algorithm removes nodes with no
# incoming edges layer by layer; if every node gets
# removed there is no cycle. It is iterative, so long
# chains cannot blow the recursion limit.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from collections import deque


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def build_graph(results):
    # results are pairs (winner, loser); graph[winner] = the people they beat
    graph = {}
    for winner, loser in results:
```

```
        if winner not in graph:
            graph[winner] = []
        if loser not in graph:
            graph[loser] = []
        graph[winner].append(loser)
    return graph


# ---- main question: main function ----
# Idea (Kahn's algorithm): results are consistent when nobody beats
# somebody who (through a chain) beat them, i.e. there is no cycle.
# Repeatedly take people nobody is left above; everybody gets taken
# exactly when there is no cycle.
# Complexity: Time O(V + E), Space O(V + E)
def results_are_valid(results):
    graph = build_graph(set(results))              # a repeated result counts once
    beaten_by = {}
    for person in graph:
        beaten_by[person] = 0
    for winner in graph:
        for loser in graph[winner]:
            beaten_by[loser] += 1
    queue = deque()
    for person in graph:
        if beaten_by[person] == 0:
            queue.append(person)
    removed = 0
    while queue:
        u = queue.popleft()
        removed += 1
```

```
        for v in graph[u]:
            beaten_by[v] -= 1
            if beaten_by[v] == 0:
                queue.append(v)
    return removed == len(graph)


# ---- main question: tests ----
def test_main_question():
    assert results_are_valid([("A", "B"), ("B", "C")]) is True
    assert results_are_valid([("A", "B"), ("B", "C"), ("C", "A")]) is False
    assert results_are_valid([]) is True
    assert results_are_valid([("A", "A")]) is False
```

```
# ========================================================
# FOLLOW-UP 1: Recursive DFS hits Python's recursion limit on long chains. What do you do?
# ========================================================
# ---- follow-up 1: function ----
# Answer: use Kahn's algorithm (already a loop), or the three-colour DFS
# with your own stack so the depth is limited by memory, not by the
# interpreter. state: missing = not visited, 1 = on the current path,
# 2 = finished; meeting a person who is on the current path is a cycle.
# Complexity: Time O(V + E), Space O(V + E)
def results_are_valid_dfs(results):
    graph = build_graph(results)
    state = {}
    for start in graph:
        if start in state:
            continue
        state[start] = 1
        stack = [start]
        next_index = {start: 0}                    # which child of each person to look at next
        while stack:
            u = stack[-1]
            i = next_index[u]
            if i < len(graph[u]):
                next_index[u] = i + 1
                v = graph[u][i]
                if state.get(v) == 1:
                    return False
                if v not in state:
                    state[v] = 1
                    next_index[v] = 0
                    stack.append(v)
            else:
                state[u] = 2
                stack.pop()
    return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    long_chain = []
    for i in range(5000):
        long_chain.append((str(i), str(i + 1)))
    assert results_are_valid(long_chain) and results_are_valid_dfs(long_chain)
    assert not results_are_valid_dfs([("A", "B"), ("B", "A")])
```

```
# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 148 of 197 | topic 29 of 43: Topological sort (Kahn) and cycle detection | 3 of 5

Template fit: the course-schedule part is can-finish: len(order) == n; the zigzag and window parts are separate small loops | * asked 442 days ago *

MEDIUM | PracHub | Technical Screen | 148 of 197

★ TOP 75: rank 46 in the Medium and Hard priority list

★ Solve tree, graph, sliding-window problems

POSTED 2025-08-04 | 442 DAYS AGO | OLDER

STUDY: TYPE **ALL TIME 51.9** **30 DAYS 33** **PRIORITY 35/288**

ABOUT THIS QUESTION

Topic: Topological sort (Kahn) and cycle detection

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #103 Binary Tree Zigzag Level Order Traversal, LeetCode #207 Course Schedule

LeetCode match: LC 103 Binary Tree Zigzag Level Order Traversal + LC 207 Course Schedule

Posted on PracHub: 2025-08-04 (updated 2026-05-26)

Original: https://prachub.com/coding-questions/solve-tree-graph-sliding-window-problems

QUESTION

Question

LeetCode 103. Binary Tree Zigzag Level Order Traversal – given a binary tree root, return its zigzag level-order traversal. LeetCode 207. Course Schedule – given course prerequisites, determine if all courses can be finished by detecting cycles with topological sort. LeetCode 340. Longest Substring with At Most K Distinct Characters – given string s and integer k, return the length of the longest substring containing at most k distinct characters.
https://leetcode.com/problems/binary-tree-zigzag-level-order-traversal/description/
https://leetcode.com/problems/course-schedule/description/
https://leetcode.com/problems/longest-substring-with-at-most-k-distinct-characters/description/

Part 1: Binary Tree Zigzag Level Order TraversalYou are given a binary tree in level-order array form, where each element is either an integer or None for a missing child. Return the zigzag level-order traversal of the tree's node values. In a zigzag traversal, the first level is read from left to right, the second from right to left, the third from left to right, and so on.

Constraints

0 <= len(root) <= 2000Each non-None node value is an integer in the range [-10^4, 10^4]The input list represents a valid binary tree in level-order form

EXAMPLES

```
Input: ([3, 9, 20, None, None, 15, 7],)
Expected Output: [[3], [20, 9], [15, 7]]
```

Explanation: Level 1 is read left-to-right, level 2 right-to-left, and level 3 left-to-right.

```
Input: ([1],)
Expected Output: [[1]]
```

Explanation: A single-node tree has only one level.

QUESTION

Hints

A breadth-first search naturally processes the tree one level at a time.You can collect each level left-to-right, then reverse every other level before appending it to the answer.

Part 2: Course ScheduleThere are num_courses courses labeled from 0 to num_courses - 1. You are given a list of prerequisite pairs [a, b], meaning you must take course b before course a. Determine whether it is possible to finish all courses. This is equivalent to checking whether the prerequisite graph contains a cycle.

Constraints

1 <= num_courses <= 10^50 <= len(prerequisites) <= 2 * 10^5Each prerequisite pair has length 20 <= course, prerequisite < num_courses

Examples

Input: (2, [[1, 0]])

Expected Output: True

Explanation: Course 0 can be taken first, then course 1.

Input: (2, [[1, 0], [0, 1]])

Expected Output: False

Explanation: The two courses depend on each other, forming a cycle.

Hints

Think of courses as nodes in a directed graph and prerequisites as directed edges.If you repeatedly take courses with indegree 0 and cannot process all nodes, a cycle exists.

Part 3: Longest Substring with At Most K Distinct CharactersGiven a string s and an integer k, return the length of the longest substring that contains at most k distinct characters.

Constraints

0 <= len(s) <= 2 * 10^50 <= k <= len(s)s may contain any standard characters

Examples

Input: ("eceba", 2)

Expected Output: 3

Explanation: The longest valid substring is 'ece', which has length 3.

Input: ("aa", 1)

Expected Output: 2

Explanation: The whole string contains only one distinct character.

Hints

A sliding window can maintain a valid substring while expanding to the right.Keep character counts in the current window, and shrink from the left whenever the number of distinct characters exceeds k.

BRUTE FORCE · Course schedule: DFS from every course looking for a cycle

BRUTE FORCE · PSEUDO-CODE

```
def can_finish(n, prereqs):
    graph = build_graph(n, prereqs)
    for course in range(n):
        if reaches(graph, course, course):
            return False  # a cycle
    return True
```

TIME	SPACE
O(V * (V + E))	O(V)

WHY IT IS SLOW

BRUTE FORCE · Course schedule: DFS from every course looking for a cycle

Each search can walk the whole graph again, O(V * (V + E)). Kahn's algorithm (remove courses with no unmet prerequisites) is O(V + E).

BRUTE FORCE · Longest window: check every substring

BRUTE FORCE · PSEUDO-CODE

```
def brute(s, K):
    best = 0
    for i in range(len(s)):
        seen = set()
        for j in range(i, len(s)):
            seen.add(s[j])
            if len(seen) > K:
                break
            best = max(best, j - i + 1)
    return best
```

TIME	SPACE
O(n^2)	O(k)

WHY IT IS SLOW

O(n^2) in the worst case. A sliding window with a count map that shrinks from the left when it exceeds K distinct characters is O(n).

BRUTE FORCE · Zigzag: level order, then reverse the odd levels

BRUTE FORCE · PSEUDO-CODE

```
def zigzag(root):
    levels = level_order(root)
    for i in range(1, len(levels), 2):
        levels[i].reverse()
    return levels
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Same O(n) time with one extra copy of every second level; filling each level from alternating ends of a deque avoids it.

OPTIMAL SOLUTION · Course schedule: Kahn's topological sort

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Longest window: sliding window with a count map

TIME	SPACE
O(n)	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Zigzag: fill each level from the correct end

TIME	SPACE
O(n)	O(width)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Course Schedule II: return an order in which to take the courses.

The same Kahn loop records the order in which courses are removed; if fewer than n courses are removed, return an empty list.

FOLLOW-UP 2 Longest substring with exactly K distinct characters.

Take atMost(K) minus atMost(K - 1) for counting substrings, or keep the window and only record lengths when the distinct count is exactly K (careful with the shrink).

FOLLOW-UP 3 Zigzag on an n-ary tree, or the right-to-left variant.

Replace the two children by the child list, and swap which levels are appended on the left.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Solve tree, graph, sliding-window
# problems
#
# ##### Question LeetCode 103. Binary Tree Zigzag Level
# Order Traversal – given a binary tree root, return its
# zigzag level-order traversal. LeetCode 207. Course
# Schedule – given course prerequisites, determine if all
# courses can be finished by detecting cycles with
# topological sort. LeetCode 340. Longest Substring with
# At Most K Distinct Characters – given string s and
# integer k, return the length of the longest substring
# containing at most k distinct characters.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Course Schedule II: return an order in which to take
# the courses.
# 2. Longest substring with exactly K distinct characters.
# 3. Zigzag on an n-ary tree, or the right-to-left
# variant.
#
# INTERVIEW NOTES
# Three standard patterns. Level order needs a queue and a
# direction flag per level. 'Can everything be finished'
# is cycle detection in a directed graph: Kahn's algorithm
```

```
# removes nodes with no remaining prerequisites, and a
# cycle leaves some behind. 'Longest window with at most K
# of something' is a sliding window with a count map that
# shrinks from the left when the limit is exceeded.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


def build_tree(values):
    # level-order list, None for a missing child: [3, 9, 20, None, None, 15, 7]
    if not values:
        return None
    root = TreeNode(values[0])
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i])
            queue.append(node.left)
        i += 1
        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i])
            queue.append(node.right)
        i += 1
    return root


# ---- main question: main functions ----
# PART 1: zigzag level order of a binary tree.
# Idea: go level by level with a queue; reverse every second level.
# Complexity: Time O(n), Space O(w) for the widest level w (O(n) at worst)
def zigzag(root):
    result = []
    if root is None:
        return result
    queue = deque([root])
    left_to_right = True
    while queue:
        level = []
        for i in range(len(queue)):               # exactly the nodes of this level
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
```

```
                queue.append(node.right)
        if not left_to_right:
            level.reverse()
        result.append(level)
        left_to_right = not left_to_right
    return result


# PART 2: can all courses be finished? prereqs are pairs (a, b): take b before a.
# Idea (Kahn's algorithm): count the unmet prerequisites of each course;
# take a course with none, lower the count of the courses that needed it,
# and repeat. If every course gets taken there is no cycle.
# Complexity: Time O(V + E), Space O(V + E)
def can_finish(n, prereqs):
    needs = []
    for i in range(n):
        needs.append([])
    unmet = [0] * n
    for a, b in prereqs:
        needs[b].append(a)                      # b unlocks a
        unmet[a] += 1
    queue = deque()
    for c in range(n):
        if unmet[c] == 0:
            queue.append(c)
    taken = 0
    while queue:
        c = queue.popleft()
        taken += 1
        for nxt in needs[c]:
            unmet[nxt] -= 1
            if unmet[nxt] == 0:
                queue.append(nxt)
    return taken == n


# PART 3: longest substring with at most k distinct letters.
# Idea: sliding window with a count of each letter inside it; when
# there are too many distinct letters, shrink from the left.
# Complexity: Time O(n), Space O(k) distinct letters
def longest_at_most_k(s, k):
    count = {}
    left = 0
    best = 0
    for right in range(len(s)):
        ch = s[right]
        count[ch] = count.get(ch, 0) + 1
        while len(count) > k:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        if right - left + 1 > best:
            best = right - left + 1
    return best


# ---- main question: tests ----
def test_main_question():
    tree = build_tree([3, 9, 20, None, None, 15, 7])
    assert zigzag(tree) == [[3], [20, 9], [15, 7]]
    assert zigzag(None) == []
    assert can_finish(2, [(1, 0)])
```

```
    assert not can_finish(2, [(1, 0), (0, 1)])
    assert longest_at_most_k("eceba", 2) == 3
    assert longest_at_most_k("aa", 1) == 2
    assert longest_at_most_k("abc", 0) == 0


# ====================================================
# FOLLOW-UP 1: Course Schedule II: return an order in which to take the courses
# ====================================================
# ---- follow-up 1: function ----
# Answer: the same Kahn loop, but write down each course as it is
# taken. If fewer than n courses were taken there is a cycle: return [].
# Complexity: Time O(V + E), Space O(V + E)
def course_order(n, prereqs):
    needs = []
    for i in range(n):
        needs.append([])
    unmet = [0] * n
    for a, b in prereqs:
        needs[b].append(a)
        unmet[a] += 1
    queue = deque()
    for c in range(n):
        if unmet[c] == 0:
            queue.append(c)
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
```

```
        for nxt in needs[c]:
            unmet[nxt] -= 1
            if unmet[nxt] == 0:
                queue.append(nxt)
    if len(order) == n:
        return order
    return []


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert course_order(4, [(1, 0), (2, 1), (3, 2)]) == [0, 1, 2, 3]
    assert course_order(2, [(0, 1), (1, 0)]) == []


# ====================================================
# FOLLOW-UP 2: Longest substring with exactly K distinct characters
# ====================================================
# ---- follow-up 2: helper functions ----
def count_at_most_k(s, k):
    # how many substrings have at most k distinct letters?
    if k < 1:
        return 0
    count = {}
    left = 0
    total = 0
    for right in range(len(s)):
        ch = s[right]
        count[ch] = count.get(ch, 0) + 1
        while len(count) > k:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        total += right - left + 1                # every substring ending at `right` and starting at left.
```

```
.right
    return total


# ---- follow-up 2: function ----
# Answer: "exactly k" = "at most k" minus "at most k - 1". (Or keep one
# window and record lengths only when the distinct count is exactly k,
# which needs more care when shrinking.)
# Complexity: Time O(n), Space O(k)
def count_exactly_k(s, k):
    return count_at_most_k(s, k) - count_at_most_k(s, k - 1)
```

```


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert count_exactly_k("abab", 2) == 6
    assert count_at_most_k("abab", 1) == 4


# ============================================================
# FOLLOW-UP 3: Zigzag on an n-ary tree, or the right-to-left variant
# ============================================================
# Answer: for an n-ary tree loop over node.children instead of left and
# right. For the variant that starts right-to-left, begin with
# left_to_right = False in zigzag().


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(root: list[str]) -> list[list[int]]:
```

Sample call

```
solution([3, 9, 20, None, None, 15, 7])
```

Test cases

Input	Expected
`([3, 9, 20, None, None, 15, 7],)`	`[[3], [20, 9], [15, 7]]`
`([1],)`	`[[1]]`

STUDY PATH: question 149 of 197 | topic 29 of 43: Topological sort (Kahn) and cycle detection | 4 of 5

Template fit: first reject prerequisites naming courses that do not exist; then the same peel; report the missing names or the cycle | * asked 445 days ago *

MEDIUM | IN APP LC 207 | PracHub | Technical Screen | 149 of 197

★ TOP 75: rank 33 in the Medium and Hard priority list

★ Validate course catalog dependencies

POSTED 2025-08-01 | 445 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 51.9 | 30 DAYS 33 | PRIORITY 36/288

ABOUT THIS QUESTION

Topic: Topological sort (Kahn) and cycle detection

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #207 Course Schedule

LeetCode match: LC 207 Course Schedule

Posted on PracHub: 2025-08-01 (updated 2026-06-25)

Original: https://prachub.com/coding-questions/validate-course-catalog-dependencies

QUESTION

You are given a set of course ids and prerequisite pairs (u, v), meaning course v requires course u. The catalog is valid when no prerequisite mentions a course that does not exist and there is no cycle. Return (True, [], order) with a possible study order when valid; (False, missing, None) listing the unknown course ids; or (False, [], cycle) with one cycle that was found.

Function signature

def validate_catalog(courses: set, prereqs: list) -> tuple

EXAMPLES

Example 1	
Input	`{1, 2, 3, 4}, [(1, 2), (2, 3), (1, 3)]`
Output	`(true, [], [1, 4, 2, 3])`
Why	`Course 1 comes first, 4 is unrelated, then 2 and 3.`

Example 2	
Input	`{1, 2}, [(1, 9)]`
Output	`(false, [9], null)`
Why	`Course 9 is not in the catalog.`

Example 3	
Input	`{1, 2}, [(1, 2), (2, 1)]`
Output	`(false, [], [1, 2, 1])`
Why	`1 requires 2 and 2 requires 1: a cycle.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def validate(courses, prereqs):
    for u, v in prereqs:
        if u not in courses or v not in courses:
            return False  # unknown id
    graph = build_graph(prereqs)
    for c in courses:
        if reaches(graph, c, c):
            return False  # cycle
    return True
```

TIME	SPACE
O(V * (V + E))	O(V)

WHY IT IS SLOW

Every course can trigger a walk of the whole graph, O(V * (V + E)). Kahn's algorithm is O(V + E) and its leftovers also reveal a cycle to report.

OPTIMAL SOLUTION · Missing references, then Kahn's algorithm, then a cycle from the leftovers

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return all missing course references and one cycle.

The function above returns the sorted unknown ids when any exist and otherwise a cycle (a list of courses starting and ending at the same course), found by walking backwards along prerequisites inside the set of courses Kahn's algorithm could not remove (every such course has a leftover prerequisite, so the walk cannot dead-end).

FOLLOW-UP 2 Group courses into semesters (the earliest round each can be taken).

Run Kahn's algorithm level by level: all courses with no remaining prerequisites form round 1, remove them, the next set forms round 2, and so on. The number of rounds is the longest dependency chain.

FOLLOW-UP 3 Find all cycles, not just one.

Compute strongly connected components (Tarjan or Kosaraju); every component with more than one course (or a self-loop) is a cycle group to report.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Validate course catalog dependencies
#
# You are given a set of course ids and prerequisite pairs
# (u, v), meaning course v requires course u. The catalog
# is valid when no prerequisite mentions a course that
# does not exist and there is no cycle. Return (True, [],
```

```
# order) with a possible study order when valid; (False,
# missing, None) listing the unknown course ids; or
# (False, [], cycle) with one cycle that was found.
#
# Signature:
#   def validate_catalog(courses: set, prereqs: list) -> tuple
#
# Examples:
# {1, 2, 3, 4}, [(1, 2), (2, 3), (1, 3)]  ->  (true, [],
#     [1, 4, 2, 3])
# {1, 2}, [(1, 9)]  ->  (false, [9], null)
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return all missing course references and one cycle.
# 2. Group courses into semesters (the earliest round each
# can be taken).
# 3. Find all cycles, not just one.
#
# INTERVIEW NOTES
# Two separate validity rules: every course mentioned must
# exist, and the dependency graph must have no cycle. The
# first is a set lookup per pair. For the second, Kahn's
# algorithm removes courses with no unmet prerequisites;
# anything left over is on a cycle or depends on one, and
# a DFS restricted to the leftovers reveals an actual
# cycle to report.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
```

```
# ============================================================
# ---- main question: helper functions ----
def find_cycle(known, prereqs, unmet):
    # `unmet` are the counts left after Kahn's algorithm. Every leftover
    # course still has a leftover prerequisite, so walking backwards along
    # prerequisites must eventually repeat a course: that repeat closes a cycle.
    left = set()
    for course in known:
        if unmet[course] > 0:
            left.add(course)
    before = {}                                 # course -> its prerequisites that are leftovers
    for u, v in prereqs:
        if u in left and v in left:
            if v not in before:
                before[v] = []
            before[v].append(u)
    x = min(left, key=str)
    walk = []
    position = {}
    while x not in position:
        position[x] = len(walk)
        walk.append(x)
        x = before[x][0]
    cycle = walk[position[x]:] + [x]
    cycle.reverse()                             # we walked against the edges: list prerequisites first
```

```
    return cycle


# ---- main question: main function ----
# Idea: 1) report ids that are used but not in the course set. 2) Run
# Kahn's algorithm: repeatedly take courses with no remaining
# prerequisite, building an order. 3) If courses are left over there is
# a cycle: find one in the leftovers.
# prereqs are pairs (u, v): course v requires course u. Returns
# (ok, missing ids, order or one cycle).
# Complexity: Time O(V + E + V log V) (the sorts), Space O(V + E)
def validate_catalog(courses, prereqs):
    known = set(courses)
    missing = set()
    for u, v in prereqs:
        if u not in known:
            missing.add(u)
        if v not in known:
            missing.add(v)
    if missing:
```

```
        return False, sorted(missing, key=str), None
    unlocks = {}
    unmet = {}
    for c in known:
        unlocks[c] = []
        unmet[c] = 0
    for u, v in prereqs:
        unlocks[u].append(v)
        unmet[v] += 1
    ready = []
    for c in known:
        if unmet[c] == 0:
            ready.append(c)
    queue = deque(sorted(ready, key=str))
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in unlocks[c]:
            unmet[nxt] -= 1
            if unmet[nxt] == 0:
                queue.append(nxt)
    if len(order) == len(known):
        return True, [], order
    return False, [], find_cycle(known, prereqs, unmet)


# ---- main question: tests ----
COURSES = ["a", "b", "c", "d"]
RULES = [("a", "b"), ("b", "c"), ("a", "d")]


def test_main_question():
    ok, missing, order = validate_catalog(COURSES, RULES)
    assert ok and order[0] == "a"
    assert order.index("b") < order.index("c")
    assert validate_catalog(COURSES, RULES + [("zzz", "a")]) == (False, ["zzz"], None)   # an unknown id
    ok, missing, cycle = validate_catalog(COURSES, RULES + [("c", "b")])                 # a cycle b -> c -
> b
    assert not ok and cycle[0] == cycle[-1]
    assert set(cycle) == {"b", "c"}
    assert validate_catalog([], []) == (True, [], [])
```

```
# ============================================================
# FOLLOW-UP 1: Return all missing course references and one cycle
```

```
# ============================================================
# Answer: validate_catalog above already returns every unknown id (sorted)
# when there are any, and otherwise a cycle: a list of courses that starts
# and ends with the same course.


# ============================================================
# FOLLOW-UP 2: Group courses into semesters (the earliest round each can be taken)
# ============================================================
# ---- follow-up 2: function ----
# Answer: Kahn's algorithm level by level: all courses with no remaining
# prerequisite are round 1; remove them and the next set is round 2, and
# so on. The number of rounds is the longest chain of prerequisites.
# Complexity: Time O(V + E + V log V), Space O(V + E)
def semesters(courses, prereqs):
    unlocks = {}
    unmet = {}
    for c in courses:
        unlocks[c] = []
        unmet[c] = 0
    for u, v in prereqs:
        unlocks[u].append(v)
        unmet[v] += 1
    level = []
    for c in courses:
        if unmet[c] == 0:
            level.append(c)
    level.sort()
    rounds = []
    while level:
        rounds.append(level)
        following = []
        for c in level:
            for v in unlocks[c]:
                unmet[v] -= 1
                if unmet[v] == 0:
                    following.append(v)
        following.sort()
        level = following
    return rounds


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert semesters(COURSES, RULES) == [["a"], ["b", "d"], ["c"]]
```

```
# ============================================================
# FOLLOW-UP 3: Find all cycles, not just one
# ============================================================
# Answer: compute the strongly connected components (Tarjan or
# Kosaraju). Every component with more than one course (or one course
# with an edge to itself) is a group of cycles to report.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(course_ids: list[str], prereqs: list[tuple[str, str]]) -> tuple[bool, list[str], list[str], list[str]]:
```

Sample call

```
solution(['A', 'B', 'C'], [('A', 'B'), ('B', 'C')])
```

Test cases

Input	Expected
`(['A', 'B', 'C'], [('A', 'B'), ('B', 'C')])`	`(True, ['A', 'B', 'C'], [], [])`
`(['A', 'B', 'C'], [('A', 'B'), ('B', 'C'), ('C', 'A')])`	`(False, [], [], ['A', 'B', 'C', 'A'])`

STUDY PATH: question 150 of 197 | topic 29 of 43: Topological sort (Kahn) and cycle detection | 5 of 5

Template fit: edge = prerequisite -> course; return the order, or an empty list when there is a cycle | ND

MEDIUM | IN APP LC 207 | Exponent | 150 of 197

★ TOP 75: rank 40 in the Medium and Hard priority list

★ Course Schedule

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 51.9 30 DAYS 33 PRIORITY 37/288

ABOUT THIS QUESTION

Topic: Topological sort (Kahn) and cycle detection

Optimal answer from: your app's interview approach (LeetCode #207)

LeetCode match: LC 207 Course Schedule

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +1017 interviews

Original: https://www.aced.io/questions/3313/course-schedule

QUESTION

You are given numCourses and a list of prerequisites, where each element in prerequisites[i] = [a_i, b_i] means you must complete course b_i before taking course a_i. Write a function canFinish that returns True if it's possible to finish all courses, and False if it's not. You need to complete all the courses for the result to be True. Assume there are no duplicate prerequisites, and numCourses is a non-negative number. Examples Input: numCourses = 3, prerequisites = [[2, 1], [1, 0]]
Output: True Explanation: You can complete course 0 first, then course 1, and finally course 2.
Input: numCourses = 4, prerequisites = [[3, 2], [2, 1], [1, 0], [0, 3]]
Output: False Explanation: The prerequisites form a cycle, making it impossible to finish all courses.
Input: numCourses = 5, prerequisites = [[4, 2], [3, 1], [2, 0]]
Output: True Explanation: You can complete the courses in order from course 0 to course 4.Hint 1Try modeling the courses as a graph, with edges representing prerequisites. Consider how you can detect cycles to determine if all courses can be completed.

EXAMPLES

Example 1	
Input	`2, [[1, 0]]`
Output	`true`
Why	`Course 1 requires course 0: finishing both is possible.`

Example 2	
Input	`2, [[1, 0], [0, 1]]`
Output	`false`
Why	`Course 1 requires 0 and course 0 requires 1: a cycle.`

Example 3	
Input	`2, [[1, 0]]`
Output	`[0, 1]`
Why	`A valid order: take 0 before 1.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def can_finish(n, prereqs):
    graph = build_graph(n, prereqs)
    for course in range(n):
        if reaches(graph, course, course):
            return False  # a cycle
    return True
```

TIME	SPACE
O(V * (V + E))	O(V)

WHY IT IS SLOW

The same parts of the graph are walked again for many starting courses, O(V * (V + E)). Kahn's algorithm (take courses with no unmet prerequisites) is O(V + E).

OPTIMAL SOLUTION · Kahn's algorithm (topological sort)

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the order of courses (LeetCode 210).

The same Kahn loop records the order; if fewer than n courses are taken return an empty list. O(V + E).

FOLLOW-UP 2 The minimum number of semesters if you can take any number of courses per semester.

Run Kahn's algorithm level by level: each level is one semester. The number of levels is the longest chain of prerequisites (return -1 if there is a cycle).

FOLLOW-UP 3 Report an actual cycle.

After Kahn's algorithm, the courses left over all still have an unmet prerequisite among themselves; walk backwards along prerequisites inside that set until a course repeats to extract a cycle.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Course Schedule
#
# You are given numCourses and a list of prerequisites,
# where each element in prerequisites[i] = [a_i, b_i]
# means you must complete course b_i before taking course
# a_i. Write a function canFinish that returns True if
# it's possible to finish all courses, and False if it's
# not. You need to complete all the courses for the result
# to be True. Assume there are no duplicate prerequisites,
# and numCourses is a non-negative number. Examples Input:
# numCourses = 3, prerequisites = [[2, 1], [1, 0]] Output:
# True Explanation: You can complete course 0 first, then
```

```
# course 1, and finally course 2. Input: numCourses = 4,
# prerequisites = [[3, 2], [2, 1], [1, 0], [0, 3]] Output:
# False Explanation: The prerequisites form a cycle,
# making it impossible to finish all courses.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the order of courses (LeetCode 210).
# 2. The minimum number of semesters if you can take any
# number of courses per semester.
# 3. Report an actual cycle.
#
# INTERVIEW NOTES
# Finishing every course is possible exactly when the
# prerequisite graph has no cycle. Kahn's algorithm tests
# that by repeatedly taking courses with no unmet
# prerequisites; if you can take all of them there is no
# cycle, and the order you took them is a valid schedule.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def build(n, prereqs):
```

```
    # prereqs (a, b): course b must be taken before course a.
    # unlocks[b] = the courses that need b; unmet[a] = how many prerequisites a still waits for
    unlocks = []
    for i in range(n):
        unlocks.append([])
    unmet = [0] * n
    for a, b in prereqs:
        unlocks[b].append(a)
        unmet[a] += 1
    return unlocks, unmet


# ---- main question: main function ----
# Idea (Kahn's algorithm): queue every course with no unmet
# prerequisite. Take one, lower the count of the courses that needed
# it, and queue those that reach zero. If all n courses get taken
# there is no cycle.
# Complexity: Time O(V + E), Space O(V + E)
def can_finish(n, prereqs):
    unlocks, unmet = build(n, prereqs)
    queue = deque()
    for c in range(n):
        if unmet[c] == 0:
            queue.append(c)
    taken = 0
    while queue:
        c = queue.popleft()
        taken += 1
        for nxt in unlocks[c]:
            unmet[nxt] -= 1
            if unmet[nxt] == 0:
```

```
                queue.append(nxt)
    return taken == n


# ---- main question: tests ----
DIAMOND = [(1, 0), (2, 0), (3, 1), (3, 2)]


def test_main_question():
    assert can_finish(2, [(1, 0)])
    assert not can_finish(2, [(1, 0), (0, 1)])
    assert can_finish(4, DIAMOND)
    assert can_finish(3, []) and can_finish(0, [])
```

```
# ============================================================
# FOLLOW-UP 1: Return the order of courses (LeetCode 210)
# ============================================================
# ---- follow-up 1: function ----
# Answer: the same loop, writing down each course as it is taken; if
# fewer than n were taken return []. O(V + E).
# Complexity: Time O(V + E), Space O(V + E)
def find_order(n, prereqs):
    unlocks, unmet = build(n, prereqs)
    queue = deque()
    for c in range(n):
        if unmet[c] == 0:
            queue.append(c)
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in unlocks[c]:
            unmet[nxt] -= 1
            if unmet[nxt] == 0:
                queue.append(nxt)
    if len(order) == n:
        return order
    return []                                       # a cycle blocks the rest


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert find_order(4, DIAMOND)[0] == 0
    assert find_order(2, [(1, 0), (0, 1)]) == []


# ============================================================
# FOLLOW-UP 2: The minimum number of semesters if you can take any number of courses per semester
# ============================================================
# ---- follow-up 2: function ----
# Answer: Kahn's algorithm level by level: each level is one semester.
# The number of levels is the longest chain of prerequisites; -1 if a
# cycle stops some courses from ever being taken.
# Complexity: Time O(V + E), Space O(V + E)
def min_semesters(n, prereqs):
    unlocks, unmet = build(n, prereqs)
    level = []
    for c in range(n):
```

```
        if unmet[c] == 0:
            level.append(c)
    semesters = 0
    taken = 0
    while level:
```

```
        semesters += 1
        taken += len(level)
        following = []
        for c in level:
            for nxt in unlocks[c]:
                unmet[nxt] -= 1
                if unmet[nxt] == 0:
                    following.append(nxt)
        level = following
    if taken == n:
        return semesters
    return -1


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert min_semesters(4, DIAMOND) == 3
    assert min_semesters(2, [(1, 0), (0, 1)]) == -1
    assert min_semesters(3, []) == 1


# =============================================================
# FOLLOW-UP 3: Report an actual cycle
# =============================================================
# ---- follow-up 3: function ----
# Answer: after Kahn's algorithm the courses that are left all still
# wait for a prerequisite that is also left over. Walk backwards along
# prerequisites inside that set until a course repeats: that is a cycle.
# Complexity: Time O(V + E), Space O(V + E)
def find_cycle(n, prereqs):
    unlocks, unmet = build(n, prereqs)
    queue = deque()
    for c in range(n):
        if unmet[c] == 0:
            queue.append(c)
    while queue:
        c = queue.popleft()
        for nxt in unlocks[c]:
            unmet[nxt] -= 1
            if unmet[nxt] == 0:
                queue.append(nxt)
    left = set()
    for c in range(n):
        if unmet[c] > 0:
            left.add(c)
    if not left:
        return []
    before = {}                                  # leftover course -> a leftover prerequisite
    for a, b in prereqs:
        if a in left and b in left:
            before[a] = b
    x = min(left)
    walk = []
    position = {}
    while x not in position:
        position[x] = len(walk)
        walk.append(x)
        x = before[x]
    return walk[position[x]:]


# ---- follow-up 3: tests ----
def test_follow_up_3():
    cycle = find_cycle(4, [(1, 0), (2, 1), (1, 2), (3, 2)])
```

```
    assert set(cycle) == {1, 2}
    assert find_cycle(2, [(1, 0)]) == []


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

You are given numCourses and a list of prerequisites, where each element in prerequisites[i] = [a_i, b_i] means you must complete course b_i before taking course a_i. Write a function canFinish that returns True if it's possible to finish all courses, and False if it's not. You need to complete all the courses for the result to be True. Assume there are no duplicate prerequisites, and numCourses is a non-negative number. Examples

```
Input: numCourses = 3, prerequisites = [[2, 1], [1, 0]]
Output: True
Explanation: You can complete course 0 first, then course 1, and finally course 2.

Input: numCourses = 4, prerequisites = [[3, 2], [2, 1], [1, 0], [0, 3]]
Output: False
Explanation: The prerequisites form a cycle, making it impossible to finish all courses.

Input: numCourses = 5, prerequisites = [[4, 2], [3, 1], [2, 0]]
Output: True
Explanation: You can complete the courses in order from course 0 to course 4.
```

OFFICIAL PAGE TEXT

Hint 1Try modeling the courses as a graph, with edges representing prerequisites. Consider how you can detect cycles to determine if all courses can be completed.Our solution processes the input data to determine if it's possible to finish all courses given the prerequisite constraints. It constructs a directed graph where nodes represent courses and edges represent the prerequisite relationships. The solution employs a queue to manage nodes with zero in-degrees (courses with no remaining prerequisites) and uses a list to keep track of the number of prerequisites (in-degrees) for each course. For each course with zero in-degrees, the solution dequeues it and processes its neighbors, reducing their in-degrees. If a neighbor's in-degree becomes zero, it is added to the queue. This approach ensures that all courses with no remaining prerequisites are processed and their dependencies are updated accordingly. By maintaining a count of the number of processed courses, the solution can determine if all courses can be finished. If the count of processed courses matches the total number of courses, it implies that there are no cyclic dependencies, and thus all courses can be completed. The solution uses a queue to efficiently manage courses with zero in-degrees and a vector to keep track of in-degrees. This method ensures that each course is processed only once, making the approach both clear and efficient. NoteWe are using a BFS (implemented by a queue) for this solution because it helps process nodes level by level (course by course), ensuring that we first handle courses with no dependencies before moving to those that rely on other courses. This approach ensures that we can detect cycles in the course prerequisites, which would make it impossible to complete all courses. Time Complexity: The solution has a time complexity of O(numCourses + numPrerequisites). This is because we process each course (vertex) and each prerequisite pair (edge) exactly once. The time complexity reflects the total number of courses (numCourses) and the number of prerequisite relationships (numPrerequisites). Space Complexity: The solution uses O(numCourses + numPrerequisites) space for the adjacency list and the in-degree vector. The space complexity accounts for the storage needed for both the graph representation and the queue, which in the worst case, contains all courses.No answers contributed by the community yet.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 30 OF 43

Counting pairs by looking up each partner in a Counter or a bucket

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Counting pairs by looking up each partner in a Counter or a bucket | 4 questions | tap a title to jump to it

READ FIRST: the code template for these 4 questions (tap here) >

M 151. Progressive Array Partitioning — 1Point3Acres * 2026-08-11 *

M 152. K-diff Pairs in an Array — ColorStack * 2025-09-16 *

M 153. Count Axis-Aligned Squares Among Grid Points — ColorStack * 2025-11-03 *

M 154. Sum of Two Squares in Two Ways — ND

CODE TEMPLATE | GROUP 30 OF 43 | 4 QUESTIONS, ONE PATTERN

Counting pairs by looking up each partner in a Counter or a bucket

The one idea these questions share

For each value, compute the partner it would need (v + k, 2v, the other corner of a square) and look it up in a Counter or a set. When the partner depends on two things, enumerate the pairs once and bucket them by a shared key.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import Counter


def count_pairs(nums, k):
    counts = Counter(nums)                        # CHANGE 1: what you index
    if k == 0:
        return sum(1 for c in counts.values() if c >= 2)
    return sum(1 for v in counts if v + k in counts)     # CHANGE 2: the partner you look up
```

2. Helper functions that usually get written next to it

```
from collections import defaultdict


def bucket_pairs(points):                    # group point pairs by a shared key (here: the midpoint and length)
    buckets = defaultdict(list)
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            p, q = points[i], points[j]
            key = (p[0] + q[0], p[1] + q[1], (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)
            buckets[key].append((p, q))
    return buckets
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
151	Progressive Array Partitioning	Counter of the values; stage 1 asks whether all the counts are different: len(set(counts.values())) == len(counts)
152	K-diff Pairs in an Array	the template as written: partner = v + k; for k == 0 count values that appear twice
153	Count Axis-Aligned Squares Among Grid Points	a set of points; two opposite corners fix the other two, so look both up; every square is found twice, divide by 2
154	Sum of Two Squares in Two Ways	enumerate every pair (a, b) with a <= b once, add a*a + b*b to a Counter, answer = sums counted exactly twice

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 151 of 197 | topic 30 of 43: Counting pairs by looking up each partner in a Counter or a bucket | 1 of 4

Template fit: Counter of the values; stage 1 asks whether all the counts are different: len(set(counts.values())) == len(counts) | * asked 70 days ago * | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 151 of 197

Progressive Array Partitioning

ASKED 2026-08-11 | 70 DAYS AGO | LAST 90 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 258/288**

ABOUT THIS QUESTION

Topic: Counting pairs by looking up each partner in a Counter or a bucket

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-08-11 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/array-partition-unique-frequencies-straights

QUESTION

The problem is given in stages; stage 1 is: group the equal values of an array together. Return true when every group has a different size (no two values occur the same number of times). The function takes the occurrence count of each value as a dict, so counting stays separate from the rule and later stages can change only the rule.

Function signature

def stage1(counts: dict) -> bool

counts maps value -> how

many times it occurs

EXAMPLES

Example 1	

Input	`from collections import Counter`
	`nums = [1, 2, 3, 4, 2, 3, 3, 4, 4, 4]`
	`stage1(Counter(nums))`

Output	`stage1(Counter(nums)) -> true`
Why	`The groups have sizes 1, 2, 3, 4: all different, so true.`

Example 2	

Input	`from collections import Counter`
	`nums = [1, 2, 3]`
	`stage1(Counter(nums))`

Output	`stage1(Counter(nums)) -> false`
Why	`Every value occurs once, so all three groups have size 1: false.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    counts = Counter(a)
    for x, y in combinations(counts, 2):
        if counts[x] == counts[y]:
            return False
    return True
```

BRUTE FORCE

TIME	SPACE
O(d^2) where d is the number of distinct values	**O(d)**

WHY IT IS SLOW

O(d^2) pair checks for d distinct values. Comparing the number of distinct counts with the number of distinct values (a set) is O(n).

OPTIMAL SOLUTION · Counter, then compare the set of counts with the number of values

TIME	SPACE
O(n)	**O(d)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Stage 2 arrives. What do you do?

Do not guess. Write down what the second example pair implies, and check any candidate rule against BOTH. The common reading 'split into consecutive runs of length at least 3' gives true for [4, 5, 6, 7, 7, 8, 9], but the statement says false, so there is an extra condition (such as run lengths or a count rule) to ask about.

FOLLOW-UP 2 Keep the code adaptable between stages.

Put the counting in one helper and each stage's rule in its own small function that takes the counts. Stage 2 then replaces a function instead of rewriting the pipeline.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Progressive Array Partitioning
#
# The problem is given in stages; stage 1 is: group the
# equal values of an array together. Return true when
# every group has a different size (no two values occur
# the same number of times). The function takes the
# occurrence count of each value as a dict, so counting
# stays separate from the rule and later stages can change
# only the rule.
#
# Signature:
#   def stage1(counts: dict) -> bool        # counts maps value -> how many times it occurs
#
# FOLLOW-UPS (answered below, in this order):
# 1. Stage 2 arrives. What do you do?
# 2. Keep the code adaptable between stages.
#
# INTERVIEW NOTES
# The statement arrives in stages, so keep the data model
# easy to adapt: count the values first, because any rule
# about 'groups of equal values' is a rule about the
# counts. Stage 1 is then 'are all counts different',
# which is a set-size comparison. Before stage 2,
# reconcile the new rule with every example; here a
# natural guess (LeetCode 659) contradicts the second
```

```
# example, which is exactly the situation where you ask.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def count_values(nums):
    # {value: how many times it occurs}
    counts = {}
    for x in nums:
        counts[x] = counts.get(x, 0) + 1
    return counts
```

```
# ---- main question: main function ----
# Stage 1. `counts` maps value -> how many times it occurs. The array can be
# split as the stage asks exactly when EVERY value occurs a different number
# of times (no two values share a count).
# Idea: the counts have no repeats exactly when the SET of counts has as many
# elements as there are distinct values. One pass to count, one over the counts.
# Complexity: Time O(u) for u distinct values, Space O(u)
def stage1(counts):
    return len(set(counts.values())) == len(counts)


# ---- main question: tests ----
def check(stage, nums):
    # each stage is one small function, so a new rule replaces one branch, not the whole pipeline
    if stage == 1:
        return stage1(count_values(nums))
    return stage2_candidate(nums)


def test_main_question():
    assert check(1, [1, 2, 3, 4, 2, 3, 3, 4, 4, 4])
    assert not check(1, [1, 2, 3])


# ========================================================
# FOLLOW-UP 1: Stage 2 arrives. What do you do?
# ========================================================
# ---- follow-up 1: function ----
# Answer: do not guess. Write down what the second example pair implies and
# check any candidate rule against BOTH examples. One reading of stage 2
# (LeetCode 659): split the array into runs of CONSECUTIVE values, each at
# least min_len long. Greedy: take the values in order; put x on a run that
# ended at x - 1 if there is one, otherwise start a new run x, x + 1, x + 2.
# Complexity: Time O(n), Space O(n)
def stage2_candidate(nums, min_len=3):
    left = count_values(nums)                    # copies of each value not yet placed
    ends = {}                                    # value -> how many runs currently end at it
    for x in nums:
        if left.get(x, 0) == 0:
            continue
        left[x] -= 1
        if ends.get(x - 1, 0) > 0:
            ends[x - 1] -= 1                     # extend a run that ended at x - 1
            ends[x] = ends.get(x, 0) + 1
```

```
        elif left.get(x + 1, 0) > 0 and left.get(x + 2, 0) > 0:
            left[x + 1] -= 1                        # start a new run x, x + 1, x + 2
            left[x + 2] -= 1
            ends[x + 2] = ends.get(x + 2, 0) + 1
        else:
            return False
    return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert check(2, [1, 2, 2, 3, 3, 4, 4, 5, 5, 6])      # the stage 2 example the candidate gets right
    # the second stage-2 example says False, but this reading says True: the rule is incomplete, so ask
    assert check(2, [4, 5, 6, 7, 7, 8, 9]) is True


# ======================================================
# FOLLOW-UP 2: Keep the code adaptable between stages
# ======================================================
# Answer: make each stage ONE small function over the same shared helper
# (count_values), and pick the stage in one place (check): a new rule then
# replaces one branch instead of the whole pipeline.


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement both stages against repeated values, singletons, empty input, and inputs with many duplicate counts. Practice stating the partition invariant and asking about run length, duplicate consumption, and whether every element must be used exactly once before writing code.

STUDY PATH: question 152 of 197 | topic 30 of 43: Counting pairs by looking up each partner in a Counter or a bucket | 2 of 4

Template fit: the template as written: partner = v + k; for k == 0 count values that appear twice | * asked 399 days ago * | frequency Single report

MEDIUM | IN APP LC 532 | ColorStack | technical interview | 152 of 197

K-diff Pairs in an Array

ASKED 2025-09-16 | 399 DAYS AGO | OLDER

STUDY: TALK NO LEETCODE FREQUENCY PRIORITY 216/288

ABOUT THIS QUESTION

Topic: Counting pairs by looking up each partner in a Counter or a bucket

Optimal answer from: your app's interview approach (LeetCode #532)

LeetCode match: LC 532 K-diff Pairs in an Array

Frequency: Single report | Last asked: 2025-09-16 | Roles: SWE Intern

QUESTION

Given an integer array nums and an integer k , return the number of unique k-diff pairs in the array. A k-diff pair is a pair of integers (nums[i], nums[j]) where: i != j , and abs(nums[i] - nums[j]) == k . Two k-diff pairs (a, b) and (c, d) are considered the same (counted once) if they consist of the same pair of values. Count each distinct value pair only once.

Function signature

```
def count_k_diff_pairs(nums: list[int], k: int) -> int:
    """Return the number of unique value-pairs with absolute difference k."""
    ...
```

EXAMPLES

Example 1	
Input	`nums = [3, 1, 4, 1, 5], k = 2`
Output	`2`
Why	`The unique 2-diff value pairs are (1, 3) and (3, 5).`

Example 2	
Input	`nums = [1, 2, 3, 4, 5], k = 1`
Output	`4`
Why	`(1,2), (2,3), (3,4), (4,5).`

Example 3	
Input	`nums = [1, 3, 1, 5, 4], k = 0`
Output	`1`
Why	`With k = 0 count values that appear at least twice; only 1 does.`

CONSTRAINTS

- 1 <= len(nums) <= 10^4
- -10^7 <= nums[i] <= 10^7
- 0 <= k <= 10^7

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_k_diff(nums, k):
    seen = set()
    for i in range(len(nums)):
        for j in range(i_+_1,_len(nums)):
            if abs(nums[i] - nums[j]) == k:
                seen.add(
                    (
                        min(nums[i], nums[j]),
                        max(nums[i], nums[j]),
                    )
                )
    return len(seen)
```

TIME	SPACE
O(n^2)	O(pairs)

WHY IT IS SLOW

Compares all index pairs and stores the value pairs to remove repeats.

OPTIMAL SOLUTION · Count the values

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it without a hash map.

Sort, then use two pointers: advance the right pointer when the gap is too small and the left when too large; when the gap equals k count it and skip duplicate values on both sides. O(n log n) time, O(1) extra space.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: K-diff Pairs in an Array
#
# Given an integer array nums and an integer k , return
# the number of unique k-diff pairs in the array. A k-diff
# pair is a pair of integers (nums[i], nums[j]) where: i
# != j , and abs(nums[i] - nums[j]) == k . Two k-diff
# pairs (a, b) and (c, d) are considered the same (counted
# once) if they consist of the same pair of values. Count
# each distinct value pair only once. Function signature:
# def count_k_diff_pairs(nums: list[int], k: int) -> int:
# """Return the number of unique value-pairs with absolute
# difference k.""" ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it without a hash map.
#
# INTERVIEW NOTES
# Counting unique value pairs, not index pairs, means
```

```
# working with distinct values. With a counter: for k > 0
# count values v whose v + k also exists; for k = 0 count
# values that appear at least twice. Sorting with two
# pointers gives the same answer in O(1) extra space.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Count the DISTINCT pairs of values (x, x + k) that both appear in the array.
# Idea: count the values in a dictionary. A value v starts a pair when
# v + k is also present; for k = 0 the same value must appear at least twice.
# Complexity: Time O(n), Space O(n)
def count_k_diff_pairs(nums, k):
    if k < 0:
        return 0
    counts = {}
```

```
    for x in nums:
        counts[x] = counts.get(x, 0) + 1
    pairs = 0
    for v in counts:
        if k == 0:
            if counts[v] >= 2:
                pairs += 1
        elif v + k in counts:
            pairs += 1
    return pairs


# ---- main question: tests ----
def test_main_question():
    assert count_k_diff_pairs([3, 1, 4, 1, 5], 2) == 2
    assert count_k_diff_pairs([1, 2, 3, 4, 5], 1) == 4
    assert count_k_diff_pairs([1, 3, 1, 5, 4], 0) == 1


# ============================================================
# FOLLOW-UP 1: Do it without a hash map
# ============================================================
# ---- follow-up 1: function ----
# Answer: sort, then use two pointers: move the right pointer on when the
# difference is too small and the left pointer when it is too big; when the
# difference equals k count it and skip duplicate values on the left side.
# O(n log n) time, O(1) extra space.
# Complexity: Time O(n log n) for the sort (the scan is O(n)), Space O(n) for the sorted copy
def count_k_diff_sorted(nums, k):
    nums = sorted(nums)
    left = 0
    right = 1
    pairs = 0
    while right < len(nums) and left < len(nums):
        difference = nums[right] - nums[left]
        if left == right or difference < k:
            right += 1
```

```
        elif difference > k:
            left += 1
        else:
            pairs += 1
            left += 1
            while left < len(nums) and nums[left] == nums[left - 1]:
                left += 1                           # the same value cannot start another pair
    return pairs
```

```


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert count_k_diff_sorted([3, 1, 4, 1, 5], 2) == 2
    assert count_k_diff_sorted([1, 3, 1, 5, 4], 0) == 1
    random.seed(8)
    for round_number in range(300):
        nums = []
        for i in range(random.randint(0, 9)):
            nums.append(random.randint(0, 6))
        k = random.randint(0, 3)
        assert count_k_diff_sorted(nums, k) == count_k_diff_pairs(nums, k)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 153 of 197 | topic 30 of 43: Counting pairs by looking up each partner in a Counter or a bucket | 3 of 4

Template fit: a set of points; two opposite corners fix the other two, so look both up; every square is found twice, divide by 2 | * asked 351 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 153 of 197

Count Axis-Aligned Squares Among Grid Points

ASKED 2025-11-03 | 351 DAYS AGO | OLDER

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 228/288**

ABOUT THIS QUESTION

Topic: Counting pairs by looking up each partner in a Counter or a bucket

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #2013 Detect Squares

LeetCode match: LC 2013 Detect Squares (related: LC 223 / counting squares)

Frequency: Single report | Last asked: 2025-11-03 | Roles: SWE Intern

QUESTION

The interviewer described a diagram of squares and rectangles and asked which data structure to use and how to count the squares. Framed concretely: you are given a list of distinct integer lattice points in the plane. Count the number of axis-aligned squares whose four corners all appear in points . A square is axis-aligned when its sides are parallel to the x- and y-axes. Return the count of such squares.

Function signature

```
from typing import List, Tuple
def count_axis_aligned_squares(points: List[Tuple[int, int]]) -> int:
    """Return the number of axis-aligned squares whose 4 corners are all in points."""
    ...
```

Follow-up (as reported): A cleaner O(n^2) approach: put all points in a set, then for every pair that could be two opposite corners of a square (equal absolute x- and y-difference), check the other two corners; each square is found via both its diagonals, so divide the total by 2.

EXAMPLES

Example 1	
Input	`points = [(0, 0), (0, 1), (1, 0), (1, 1)]`
Output	`1`
Why	`The four points form exactly one unit square.`

Example 2	
Input	`points = [(0,0),(0,1),(1,0),(1,1),(0,2),(1,2),(2,0),(2,1),(2,2)]`
Output	`5`
Why	`A 3x3 grid of points has four 1x1 squares and one 2x2 square.`

Example 3	
Input	`points = [(0, 0), (2, 0), (0, 1), (2, 1)]`
Output	`0`
Why	`These form a 2x1 rectangle, which is not a square.`

CONSTRAINTS

- 0 <= len(points) <= 2000 ; all points are distinct.

QUESTION

- Each point is (x, y) with -10^9 <= x, y <= 10^9 .
- A square must have positive side length (four distinct corners); degenerate zero-size "squares" do not count.
- Only axis-aligned squares count (tilted squares are ignored).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
from itertools import combinations
def brute_squares(points):
    count = 0
    for quad in combinations(points,_4):
        xs = sorted({p[0] for p in quad})
        ys = sorted({p[1] for p in quad})
        if (
            len(xs) == len(ys) == 2
            and xs[1] - xs[0] == ys[1] - ys[0]
        ):
            corners = {
                (x, y) for x in xs for y in ys
            }
            if corners == set(quad):
                count += 1
    return count
```

TIME	SPACE
O(n^4)	O(1)

WHY IT IS SLOW

It looks at every group of four points although two opposite corners already fix the other two.

OPTIMAL SOLUTION · Opposite corners and a set

TIME	SPACE
O(n^2)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Points may repeat. Count every distinct choice of four points.

Keep a Counter. For each unordered pair of distinct diagonal points multiply the four counts; halve the total because each square has two diagonals.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count Axis-Aligned Squares Among Grid
# Points
#
# The interviewer described a diagram of squares and
# rectangles and asked which data structure to use and how
# to count the squares. Framed concretely: you are given a
# list of distinct integer lattice points in the plane.
```

```
# Count the number of axis-aligned squares whose four
# corners all appear in points . A square is axis-aligned
# when its sides are parallel to the x- and y-axes. Return
# the count of such squares. Function signature: from
# typing import List, Tuple def
# count_axis_aligned_squares(points: List[Tuple[int,
# int]]) -> int: """Return the number of axis-aligned
# squares whose 4 corners are all in points.""" ...
# Follow-up (as reported): A cleaner O(n^2) approach: put
# all points in a set, then for every pair that could be
# two opposite corners of a square (equal absolute x- and
# y-difference), check the other two corners; each square
# is found via both its diagonals, so divide the total by
# 2.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Points may repeat. Count every distinct choice of
# four points.
#
# INTERVIEW NOTES
# Put every point in a set. Two points that sit on
# opposite corners of an axis-aligned square have |dx| ==
# |dy| > 0, and then the other two corners are fixed: (x1,
# y2) and (x2, y1). Check both in the set. Every square
# has two diagonals, so each is found twice over unordered
# pairs: divide by 2.

# =======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# =======================================================

# =======================================================
# MAIN QUESTION
# =======================================================
# ---- main question: main function ----
```

```
# Count the squares with sides parallel to the axes whose four corners are
# all among the points.
# Idea: two OPPOSITE corners decide the other two, so one set lookup per pair
# is enough: for points (x1, y1) and (x2, y2) with |x1 - x2| == |y1 - y2| != 0,
# the other corners are (x1, y2) and (x2, y1). Every square has two diagonals,
# so it is found twice: halve the total.
# Complexity: Time O(n^2), Space O(n)
def count_axis_aligned_squares(points):
    present = set(points)
    total = 0
    for i in range(len(points)):
        x1, y1 = points[i]
        for j in range(i + 1, len(points)):
            x2, y2 = points[j]
            if abs(x1 - x2) == abs(y1 - y2) and x1 != x2:
                if (x1, y2) in present and (x2, y1) in present:
                    total += 1
    return total // 2


# ---- main question: tests ----
def test_main_question():
    assert count_axis_aligned_squares([(0, 0), (0, 1), (1, 0), (1, 1)]) == 1
    grid = []
    for x in range(3):
        for y in range(3):
```

```
            grid.append((x, y))
    assert count_axis_aligned_squares(grid) == 5               # four 1 x 1 squares and one 2 x 2
    assert count_axis_aligned_squares([(0, 0), (2, 0), (0, 1), (2, 1)]) == 0
    assert count_axis_aligned_squares([]) == 0


# ========================================================
# FOLLOW-UP 1: Points may repeat. Count every distinct choice of four points
# ========================================================
# ---- follow-up 1: function ----
# Answer: keep a COUNT per point. For each pair of distinct diagonal points,
# multiply the number of copies of all four corners; halve the total
# because each square has two diagonals.
# Complexity: Time O(d^2) for d distinct points, Space O(d)
def count_squares_with_duplicates(points):
    copies = {}
    for point in points:
        copies[point] = copies.get(point, 0) + 1
    distinct = list(copies)
```

```
    total = 0
    for i in range(len(distinct)):
        x1, y1 = distinct[i]
        for j in range(i + 1, len(distinct)):
            x2, y2 = distinct[j]
            if abs(x1 - x2) == abs(y1 - y2) and x1 != x2:
                total += copies[(x1, y1)] * copies[(x2, y2)] * copies.get((x1, y2), 0) * copies.get((x2, y1
), 0)
    return total // 2


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert count_squares_with_duplicates([(0, 0), (0, 1), (1, 0), (1, 1)]) == 1
    assert count_squares_with_duplicates([(0, 0), (0, 0), (0, 1), (1, 0), (1, 1)]) == 2     # two copies of
 one corner


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 154 of 197 | topic 30 of 43: Counting pairs by looking up each partner in a Counter or a bucket | 4 of 4

Template fit: enumerate every pair (a, b) with a <= b once, add a*a + b*b to a Counter, answer = sums counted exactly twice | ND

MEDIUM | HackTheRounds | Onsite | 154 of 197

Sum of Two Squares in Two Ways

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 266/288

ABOUT THIS QUESTION

Topic: Counting pairs by looking up each partner in a Counter or a bucket

Optimal answer from: the interview site's own solution

QUESTION

Given an integer n, find all integers s < n that can be written as a sum of two positive squares in exactly two different ways (the order of the two squares does not matter). For example 50 = 1^2 + 7^2 = 5^2 + 5^2. Return them in increasing order.

Function signature

def two_squares_two_ways(n: int) -> list

EXAMPLES

Example 1	
Input	`100`
Output	`[50, 65, 85]`
Why	`50 = 1 + 49 = 25 + 25 and 65 = 1 + 64 = 16 + 49 both have exactly two ways.`

Example 2	
Input	`50`
Output	`[]`
Why	`Nothing below 50 qualifies.`

CONSTRAINTS

- 1 <= n <= 2^31 - 1

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n):
    result = []
    for s in range(1, n):
        ways = 0
        for a in range(1, isqrt(s) + 1):
            rest = s - a * a
            b = isqrt(rest)
            if b >= a and b * b == rest:
                ways += 1
        if ways == 2:
            result.append(s)
    return result
```

TIME	SPACE
O(n * sqrt(n))	O(1)

WHY IT IS SLOW

BRUTE FORCE

Every s searches all pairs again, O(n * sqrt(n)). Generating each pair once and counting sums in a hash map is O(n).

OPTIMAL SOLUTION · Count every pair once in a hash map

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Exactly k ways instead of two.

Change the comparison from == 2 to == k; the counting loop is the same. For k = 3 the smallest is 325.

FOLLOW-UP 2 Allow zero (0^2 + b^2).

Start a at 0; then perfect squares like 25 gain a representation, so some answers appear or disappear. Check against the examples to see which convention the question uses (here, positive only).

FOLLOW-UP 3 n is huge (10^12) and you only need to test one number.

Factorise the number: the count of representations as a sum of two squares depends only on the primes congruent to 1 mod 4 (Jacobi's formula), so a factorisation answers it in sublinear time.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Sum of Two Squares in Two Ways
#
# Given an integer n, find all integers s < n that can be
# written as a sum of two positive squares in exactly two
# different ways (the order of the two squares does not
# matter). For example 50 = 1^2 + 7^2 = 5^2 + 5^2. Return
# them in increasing order.
#
# Signature:
#   def two_squares_two_ways(n: int) -> list
#
# Examples:
# 100  ->  [50, 65, 85]
# 50  ->  []
#
# FOLLOW-UPS (answered below, in this order):
# 1. Exactly k ways instead of two.
# 2. Allow zero (0^2 + b^2).
# 3. n is huge (10^12) and you only need to test one
# number.
#
# INTERVIEW NOTES
# Do not test each s separately. Generate every pair (a,
# b) once, add its sum to a counter, and read the answer
# from the counter. The loop is bounded by a^2 + b^2 < n,
# so a and b are at most sqrt(n); a <= b removes order
# duplicates.

# ======================================================
```

```
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Find every number s < n that can be written as a sum of two positive
# squares a^2 + b^2 (with a <= b) in EXACTLY two different ways.
# Idea: loop over every pair a <= b with a^2 + b^2 < n (about pi * n / 8
# pairs) and count how often each sum appears in a dictionary. The answer is
# the sorted list of the sums whose count is exactly 2.
# Follow-up 1: exactly k ways: compare with k instead of 2 (for k = 3 the
# smallest is 325). Follow-up 2: zero=True starts a at 0, so perfect squares
# like 25 = 0^2 + 5^2 gain a way: check the examples to see which convention the
# question uses (here: positive only).
# Complexity: Time O(n) (about n / 2 pairs a <= b with a^2 + b^2 < n), Space O(n)
def two_squares_two_ways(n, k=2, zero=False):
    ways = {}
    a = 1
    if zero:
        a = 0
    while 2 * a * a < n:
        b = a
        while a * a + b * b < n:
            total = a * a + b * b
            ways[total] = ways.get(total, 0) + 1
            b += 1
        a += 1
    sums = []
    for total in ways:
        if ways[total] == k and total > 0:
            sums.append(total)
    sums.sort()
    return sums


# ---- main question: tests ----
def test_main_question():
    assert two_squares_two_ways(100) == [50, 65, 85]
    assert two_squares_two_ways(10) == []
    assert 25 not in two_squares_two_ways(100)


# ============================================================
# FOLLOW-UP 1: Exactly k ways instead of two
# ============================================================
# Answer: the parameter k: change the comparison from == 2 to == k; the
# counting loop is the same.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert two_squares_two_ways(1000, 3)[0] == 325            # 325 = 1 + 324 = 36 + 289 = 100 + 225


# ============================================================
# FOLLOW-UP 2: Allow zero (0^2 + b^2)
# ============================================================
# Answer: start a at 0 (the parameter zero=True): then perfect squares like
# 25 gain a representation, so some answers appear or disappear.
# ---- follow-up 2: tests ----
```

```
def test_follow_up_2():
    assert 25 in two_squares_two_ways(100, 2, zero=True)


# ======================================================
# FOLLOW-UP 3: n is huge (10^12) and you only need to test one number
# ======================================================
# ---- follow-up 3: helper functions ----
def integer_sqrt(m):
    # the biggest whole number r with r * r <= m (binary search)
    low = 0
    high = m + 1
    while low + 1 < high:
        middle = (low + high) // 2
        if middle * middle <= m:
            low = middle
        else:
            high = middle
    return low


def factorise(m):
    # {prime: exponent}; trial division is fine up to about 10^12
    factors = {}
    p = 2
    while p * p <= m:
        while m % p == 0:
            factors[p] = factors.get(p, 0) + 1
            m = m // p
        p += 1
    if m > 1:
        factors[m] = factors.get(m, 0) + 1
    return factors


# ---- follow-up 3: function ----
# Answer: factorise the number. The number of ways to write it as a sum of
# two squares depends only on the primes that are 1 mod 4 (Jacobi): a prime
# that is 3 mod 4 with an odd exponent makes it impossible, and with
# B = the product of (exponent + 1) over the primes 1 mod 4 there are 4B ordered
# signed pairs (x, y). Take away the special shapes a == 0 (m is a square) and
# a == b (m is twice a square); every other unordered pair is counted 8 times.
# Complexity: Time O(sqrt(m)) (factorising m), Space O(log m)
```

```
def representations(m, zero=False):
    if m <= 0:
        return 0
    boost = 1
    factors = factorise(m)
    for p in factors:
        if p % 4 == 3 and factors[p] % 2 == 1:
            return 0
        if p % 4 == 1:
            boost *= factors[p] + 1
    is_square = integer_sqrt(m) ** 2 == m
    half = m % 2 == 0 and integer_sqrt(m // 2) ** 2 == m // 2
    special = 0
    if is_square:
        special += 4
    if half:
        special += 4
    result = (4 * boost - special) // 8
    if half:
        result += 1                                    # the pair a == b
    if zero and is_square:
```

```
        result += 1                                     # the pair 0, sqrt(m)
    return result


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert representations(50) == 2
    assert representations(325) == 3
    assert representations(3) == 0
    assert representations(25) == 1
    assert representations(25, zero=True) == 2
    assert representations(5 ** 5 * 13 * 17 * 29) > 10        # a 10 digit number in a moment
    small = two_squares_two_ways(100)
    for m in range(1, 100):
        assert (m in small) == (representations(m) == 2)      # the formula agrees with the loops


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
```

```
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

ALSO REPORTED · 1 more

Find numbers with exactly two sum-of-squares forms | PracHub | Technical Screen

https://prachub.com/coding-questions/find-numbers-with-exactly-two-sum-of-squares-forms

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 31 OF 43

Tree DP: build a parent order once, then add up from the leaves

3 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Tree DP: build a parent order once, then add up from the leaves | 3 questions | tap a title to jump to it

READ FIRST: the code template for these 3 questions (tap here) >

M 155. Compute File System Total Size **1Point3Acres * 2026-09-02 ***

M 156. Minimum-Cost Tree Disconnection **1Point3Acres * 2026-09-09 ***

M 157. Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent... **ND**

CODE TEMPLATE | GROUP 31 OF 43 | 3 QUESTIONS, ONE PATTERN

Tree DP: build a parent order once, then add up from the leaves

The one idea these questions share

Get an order where every parent comes before its children (a BFS order from the root), then sweep it backwards so every child is finished before its parent. Subtree sizes and sums fall out of that one backwards sweep; the re-rooting questions add a second forward sweep.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def subtree_sums(n, adj, root=0):
    parent, order = tree_order(n, adj, root)
    size, down = [1] * n, [0] * n
    for u in reversed(order):             # children before parents
        p = parent[u]
        if p != -1:
            size[p] += size[u]
            down[p] += down[u] + size[u]  # CHANGE: what you add up
    return parent, order, size, down
```

2. Helper functions that usually get written next to it

```
def tree_order(n, adj, root=0):
    parent, order = {root: -1}, [root]
    for u in order:                       # BFS order: parents before children
        for v in adj[u]:
            if v not in parent:
                parent[v] = u
                order.append(v)
    return parent, order


def adjacency_from_edges(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    return adj
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
155	Compute File System Total Size	add each file's size while walking down from the root with a stack; a shared directory is counted once (seen set)
156	Minimum-Cost Tree Disconnection	backwards sweep with two numbers per node: the best cost when the node is cut off, and the cost if it is free
157	Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes.	t[v] = max over children (rank among children sorted by time, longest first, + t[child]); read t at the root

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 155 of 197 | topic 31 of 43: Tree DP: build a parent order once, then add up from the leaves | 1 of 3

Template fit: add each file's size while walking down from the root with a stack; a shared directory is counted once (seen set) | * asked 48 days ago * | frequency Low

MEDIUM | 1Point3Acres | onsite-coding, phone-screen | 155 of 197

Compute File System Total Size

ASKED 2026-09-02 | 48 DAYS AGO | LAST 90 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 264/288**

ABOUT THIS QUESTION

Topic: Tree DP: build a parent order once, then add up from the leaves

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/file-system-total-size

QUESTION

A file system is a tree: File(size) is a file and Dir(*children) is a directory holding files and directories. Return the total size of all files. A directory object can be shared (it appears under more than one parent); its files are counted once.

Function signature

def total_size(root) -> int

EXAMPLES

Example 1

Input	`root = Dir(Dir(File(10), File(20)), File(5), Dir(File(30)))` `total_size(root)`
Output	`total_size(root) -> 65`
Why	`10 + 20 + 5 + 30 = 65.`

Example 2

Input	`shared = Dir(File(7))`
	`root = Dir(shared, shared, File(1))`
	`total_size(root)`

Output	`total_size(root) -> 8`
Why	`The same directory appears twice but its 7 is counted once: 7 + 1 = 8.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def size(node):
    if is_file(node):
        return node.size
    total = 0
    for child in node.children:
        total += size(child)
    return total
```

TIME	SPACE
O(n)	**O(depth)** call stack

BRUTE FORCE

WHY IT IS SLOW

Linear time, which is optimal; the weaknesses are the recursion depth on very deep trees and an endless loop if a link creates a cycle. An explicit stack with a visited set fixes both.

OPTIMAL SOLUTION · Iterative traversal with a visited set

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree is queried often and files change.

Cache each directory's total and, when a file changes, clear the cache only for its ancestors (walk parent links). A query recomputes only cleared nodes. Updates cost O(depth).

FOLLOW-UP 2 Hard links: the same file appears under two directories.

Count a file once per identity (a visited set of ids, as coded) if the question asks for disk usage, but once per path if it asks for the sum of the sizes shown. Ask which.

FOLLOW-UP 3 Find the largest directory.

Compute totals in post-order and keep the maximum; it is the same pass. For top-K directories keep a heap of size K. **Cost:** one post-order pass, O(n); keeping the top K directories with a heap of size K makes it O(n log K).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Compute File System Total Size
#
# A file system is a tree: File(size) is a file and
# Dir(*children) is a directory holding files and
# directories. Return the total size of all files. A
# directory object can be shared (it appears under more
# than one parent); its files are counted once.
#
# Signature:
#   def total_size(root) -> int
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree is queried often and files change.
# 2. Hard links: the same file appears under two
# directories.
# 3. Find the largest directory.
#
# INTERVIEW NOTES
# A total over a tree is a post-order sum. Two things
# change the answer in real file systems: depth (recursion
# limit) and sharing or cycles (the same node reachable
# twice must be counted once). A visited set solves both
# the cycle risk and the double counting, and an explicit
# stack solves the depth.
```

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class File:
    def __init__(self, size):
        self.size = size
        self.parent = None


class Dir:
    def __init__(self, *children):
        self.children = list(children)
```

```
        self.parent = None
        self.cached = None                          # the total size; None means "needs recomputing"
        for child in children:
            child.parent = self


# ---- main question: main function ----
# Idea: an explicit stack removes the recursion limit, and a set of the
# nodes already seen makes each one count once, so a directory that is
# shared (hard links, follow-up 2) is not counted twice and a cycle
# cannot loop forever. count_once=False counts a shared file once per path.
# Complexity: Time O(n) for n files and folders, Space O(n)
def total_size(root, count_once=True):
    total = 0
    seen = set()
    stack = [root]
    while stack:
        node = stack.pop()
        if count_once:
            if node in seen:
                continue
            seen.add(node)
        if isinstance(node, File):
            total += node.size
        else:
            for child in node.children:
                stack.append(child)
    return total


# ---- main question: tests ----
def example_tree():
    inner = Dir(File(10), File(20))
    return inner, Dir(inner, File(5), Dir(File(30)))


def test_main_question():
    inner, root = example_tree()
    assert total_size(root) == 65


# ============================================================
# FOLLOW-UP 1: The tree is queried often and files change
```

```
# ========================================================
# ---- follow-up 1: helper functions ----
```

```
def size_of(node):
    # a file's size, or a directory's cached total
    if isinstance(node, File):
        return node.size
    return node.cached


def cached_total(root):
    # fill in the cached total of every directory that needs it, children first
    order = []
    stack = [root]
    while stack:
        node = stack.pop()
        order.append(node)
        if isinstance(node, Dir) and node.cached is None:
            for child in node.children:
                stack.append(child)
    for i in range(len(order) - 1, -1, -1):       # children before parents
        node = order[i]
        if isinstance(node, Dir) and node.cached is None:
            total = 0
            for child in node.children:
                total += size_of(child)
            node.cached = total
    return root.cached


# ---- follow-up 1: function ----
# Answer: cache every directory's total. When a file changes, clear the
# cache only of its ancestors (follow the parent links); the next query
# recomputes just those. An update costs O(depth).
# Complexity: Time O(depth of the file), Space O(1)
def resize(file, size):
    file.size = size
    node = file.parent
    while node is not None:
        node.cached = None
        node = node.parent


# ---- follow-up 1: tests ----
def test_follow_up_1():
    inner, root = example_tree()
    assert cached_total(root) == 65
    resize(inner.children[0], 15)                 # only the ancestors are recomputed
```

```
    assert root.cached is None and inner.cached is None
    assert cached_total(root) == 70


# ========================================================
# FOLLOW-UP 2: Hard links: the same file appears under two directories
# ========================================================
# Answer: count a file once per identity (the `seen` set, the default) if
# the question asks for disk usage, but once per path if it asks for the
# sum of the sizes shown. Ask which one is meant.
# ---- follow-up 2: tests ----
def test_follow_up_2():
    shared = File(7)
    both = Dir(Dir(shared), Dir(shared))
    assert total_size(both) == 7
    assert total_size(both, count_once=False) == 14
```

```
# ========================================================
# FOLLOW-UP 3: Find the largest directory
# ========================================================
# ---- follow-up 3: function ----
# Answer: the totals come from the same post-order pass, so the largest
# directory is just the maximum. For the top K directories, sort the
# totals and take K (or keep a heap of size K: O(n log K)).
# Complexity: Time O(n), Space O(n)
def directory_totals(root):
    cached_total(root)
    totals = []
    stack = [root]
    while stack:
        node = stack.pop()
        if isinstance(node, Dir):
            totals.append(node.cached)
            for child in node.children:
                stack.append(child)
    return totals


# Complexity: Time O(n + d log d) for d folders, Space O(d)
def top_k_directories(root, k):
    totals = directory_totals(root)
    totals.sort(reverse=True)
    return totals[:k]
```

```
# ---- follow-up 3: tests ----
def test_follow_up_3():
    inner, root = example_tree()
    assert top_k_directories(root, 2) == [65, 30]      # 65 for the root, 30 for inner (or the last Dir)
    assert directory_totals(root).count(30) == 2       # inner (10 + 20) and Dir(File(30))


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill recursive-and-iterative DFS over directory-style trees.
Have ready: cycle-safe DFS (visited set), memoized DFS (cache on internal nodes), invalidation cascade.
Practice the same problem under the the canonical "longest absolute file path" parsing problem for warm-up.

STUDY PATH: question 156 of 197 | topic 31 of 43: Tree DP: build a parent order once, then add up from the leaves | 2 of 3

Template fit: backwards sweep with two numbers per node: the best cost when the node is cut off, and the cost if it is free | * asked 41 days ago * | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 156 of 197

Minimum-Cost Tree Disconnection

ASKED 2026-09-09 | 41 DAYS AGO | LAST 90 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 257/288**

ABOUT THIS QUESTION

Topic: Tree DP: build a parent order once, then add up from the leaves

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-09 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/binary-tree-root-leaf-minimum-cut

QUESTION

A rooted tree is given as directed edges (parent, child, cost) with node 0 as the root. Cut edges so that no path from the root to any original leaf remains. The price of a cut set is the sum of the costs of the edges cut (costs may be negative). Return the minimum total price.

Function signature

def disconnect_cost(n: int, edges: list) -> int

EXAMPLES

Example 1	
Input	`3, [(0, 1, 3), (0, 2, 2)]`
Output	`5`
Why	`Two leaves hang off the root: both edges must be cut, 3 + 2 = 5.`

Example 2	
Input	`3, [(0, 1, 4), (1, 2, 1)]`
Output	`1`
Why	`A chain 0 - 1 - 2: cutting the cheaper edge 1 -> 2 (cost 1) already separates the leaf.`

Example 3	
Input	`3, [(0, 1, -1), (1, 2, -1)]`
Output	`-2`
Why	`Negative costs: cutting both edges costs -2, which beats cutting only one.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(root, edges):
    best = INF
    for k in range(len(edges) + 1):
        for cut in combinations(edges, k):
            tree = remove_edges(edges, cut)
            if not reaches_leaf(tree, root):
                cost = sum(w for _, _, w in cut)
                best = min(best, cost)
    return best
```

TIME	SPACE
O(2^E * n)	O(n)

WHY IT IS SLOW

2^E subsets, each checked with a search: O(2^E * n). A tree DP (cut the edge above a child, or require the child to be disconnected from its leaves) is O(n).

OPTIMAL SOLUTION · Tree DP: best(v) = cheapest cuts inside v's subtree that disconnect v from its leaves

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Weights are non-negative.

Then free(c) is always 0 and the recurrence collapses to best(v) = sum over children of (w if the child is a leaf, else min(w, best(child))).

FOLLOW-UP 2 The tree is a long chain and recursion overflows.

Compute nodes in reverse BFS order (children before parents) and store best and free in arrays, no recursion needed. State the space as O(n).

FOLLOW-UP 3 Return which edges to cut.

Remember at each child which option was cheaper, then walk down from the root collecting the cut edges. Cut edges also include every negative edge below any cut edge.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Minimum-Cost Tree Disconnection
#
# A rooted tree is given as directed edges (parent, child,
# cost) with node 0 as the root. Cut edges so that no path
# from the root to any original leaf remains. The price of
# a cut set is the sum of the costs of the edges cut
# (costs may be negative). Return the minimum total price.
#
# Signature:
#   def disconnect_cost(n: int, edges: list) -> int
```

```
#
# Examples:
# 3, [(0, 1, 3), (0, 2, 2)]  ->  5
# 3, [(0, 1, 4), (1, 2, 1)]  ->  1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Weights are non-negative.
# 2. The tree is a long chain and recursion overflows.
# 3. Return which edges to cut.
#
# INTERVIEW NOTES
# On a tree, every decision about a subtree depends only
# on the edge above it, so define a function of the node
# that returns the best cost inside its subtree. For each
# child you either cut the edge above it (pay its weight)
# or keep it and make the child itself disconnected from
# the leaves below. With negative weights, a cut edge
# frees the whole subtree, so you may cut every negative
# edge in it for free gain.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

INF = float("inf")


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def tree_costs(n, edges):
```

```
    # edges are (parent, child, cost); node 0 is the root.
    # best[v] = the cheapest set of cuts that separates v from every leaf below it
    # free[v] = the sum of all NEGATIVE costs below v (cutting those too only helps)
    kids = []
    for i in range(n):
        kids.append([])
    for u, v, w in edges:
        kids[u].append((v, w))
    order = [0]                                  # parents before children (breadth first)
    for u in order:                              # (a loop, not recursion: safe on a long chain)
        for v, w in kids[u]:
            order.append(v)
    best = [0] * n
    free = [0] * n
    for i in range(len(order) - 1, -1, -1):      # children before parents
        v = order[i]
        for c, w in kids[v]:
            cut = w + free[c]                    # option 1: cut this edge (and the negatives below)
            if kids[c]:
                keep = best[c]                   # option 2: keep it and cut deeper
            else:
                keep = INF                       # a leaf has nothing deeper to cut
            if cut < keep:
                best[v] += cut
            else:
                best[v] += keep
            if w < 0:
                free[v] += w
            free[v] += free[c]
```

```
    return kids, best, free


# ---- main question: main function ----
# Idea: for a child c reached by an edge of weight w there are two
# options: cut that edge (pay w, plus the negative edges you can still
# cut below c) or keep it and cut deeper (best(c)), which is impossible
# if c is a leaf. Add up the cheaper option over all children of the root.
# For non-negative costs this is simply: a leaf child costs w, any other
# child costs min(w, best(child)).
# Complexity: Time O(n), Space O(n)
def disconnect_cost(n, edges):
    kids, best, free = tree_costs(n, edges)
    return best[0]
```

```
# ---- main question: tests ----
def test_main_question():
    negative = [(0, 1, -1), (1, 2, -1)]                    # the chain A -(-1)-> B -(-1)-> C
    assert disconnect_cost(3, negative) == -2
    chain = [(0, 1, 5), (1, 2, 3)]
    assert disconnect_cost(3, chain) == 3                  # non-negative costs: cut the cheaper edge
    assert disconnect_cost(1, []) == 0                     # a single node needs no cut
    big = []
    for i in range(5000):
        big.append((i, i + 1, 1))
    assert disconnect_cost(5001, big) == 1                 # a long chain does not overflow the stack


# ========================================================
# FOLLOW-UP 1: Weights are non-negative
# ========================================================
# Answer: then free(c) is always 0 and the rule becomes: best(v) = the sum
# over children of (w if the child is a leaf, else min(w, best(child))).


# ========================================================
# FOLLOW-UP 2: The tree is a long chain and recursion overflows
# ========================================================
# Answer: tree_costs handles the nodes in reverse breadth-first order
# (children before parents) and stores best and free in arrays, so there is
# no recursion at all. Space O(n).


# ========================================================
# FOLLOW-UP 3: Return which edges to cut
# ========================================================
# ---- follow-up 3: function ----
# Answer: at each child remember which option was cheaper, then walk
# down from the root collecting the cut edges. When an edge is cut, the
# negative edges below it are cut as well.
# Complexity: Time O(n), Space O(n)
def edges_to_cut(n, edges):
    kids, best, free = tree_costs(n, edges)
    cuts = []
    stack = [0]
    while stack:
        v = stack.pop()
        for c, w in kids[v]:
            if kids[c]:
                keep = best[c]
```

```
            else:
                keep = INF
            if w + free[c] <= keep:
```

```
                cuts.append((v, c))                    # cut here ...
                below = [c]
                while below:                           # ... plus every negative edge below c
                    x = below.pop()
                    for y, wy in kids[x]:
                        if wy < 0:
                            cuts.append((x, y))
                        below.append(y)
            else:
                stack.append(c)
    return cuts


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert sorted(edges_to_cut(3, [(0, 1, -1), (1, 2, -1)])) == [(0, 1), (1, 2)]
    assert edges_to_cut(3, [(0, 1, 5), (1, 2, 3)]) == [(1, 2)]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement the nonnegative postorder recurrence, then trace a chain and a branching tree while accounting for recursion depth.
Implement the two-state negative-weight version and compare it with exhaustive edge-subset enumeration on small trees, including the given negative chain.

STUDY PATH: question 157 of 197 | topic 31 of 43: Tree DP: build a parent order once, then add up from the leaves | 3 of 3

Template fit: t[v] = max over children (rank among children sorted by time, longest first, + t[child]); read t at the root | ND

MEDIUM | IN APP LC 1376 | Exponent | 157 of 197

Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes.

ASKED (ANY CO.) ~4 YEARS AGO | OLDER

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 159/288

ABOUT THIS QUESTION

Topic: Tree DP: build a parent order once, then add up from the leaves

Optimal answer from: your app's interview approach (LeetCode #1376)

LeetCode match: LC 1376 Time Needed to Inform All Employees

Last asked (any company): 4 years ago, about 2022-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/2411/minimum-iterations-information-distribution-n-ary-tree

QUESTION

Information starts at the root of a tree. In every iteration a node that already has the information can pass it to one of its children (the tree maps each node to its children as (child, cost) pairs; in the basic problem every cost is 1 iteration). Return the minimum number of iterations until every node has the information.

Function signature

def min_rounds(tree: dict, root) -> int

EXAMPLES

Example 1

Input	`{'r': [('a', 1), ('b', 1), ('c', 1)], 'a': [('d', 1)]}, 'r'`
Output	`3`
Why	`The root informs a, b and c in iterations 1, 2, 3, while a informs d in iteration 2: everyone knows after 3 iterations.`

Example 2

Input	`{'r': [('a', 1)], 'a': [('b', 1)]}, 'r'`
Output	`2`
Why	`A path r - a - b needs one iteration per edge.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def T(v):
    kids = children[v]
    if not kids:
        return 0
    best = INF
    for order in permutations(kids):
        finish = 0
        for i, c in enumerate(order, 1):
            finish = max(finish, i + T(c))
        best = min(best, finish)
    return best
```

TIME	SPACE
O(product of d! over nodes)	O(n)

WHY IT IS SLOW

A node with d children has d! orderings, so the cost is the product of d! over all nodes. Sorting the children by decreasing T gives the same answer in O(d log d).

OPTIMAL SOLUTION · Post-order DP: serve the slowest subtree first

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is it optimal to serve the child with the largest subtree time first?

Exchange argument: if a child with time a is served in round i and one with time b >= a in round i + 1, swapping them changes the finish times from max(i + a, i + 1 + b) to max(i + b, i + 1 + a), and the second is never larger because b >= a. So sorting by decreasing T minimizes the maximum. **Cost:** sorting the children at each node makes the whole computation O(n log n); the exchange argument adds nothing.

FOLLOW-UP 2 What if each edge has a different transmission cost?

Replace the unit step by the cost: the child served in order position i can only start after the sum of the costs of the edges before it, so T(v) = max over i of (prefix cost up to child_i + T(child_i)). The best order is still decreasing (cost + T) only when costs are equal; in general it becomes a scheduling problem solved by sorting on T(child) - cost, i.e. Smith's rule.

FOLLOW-UP 3 Each call takes a different time per child.

The order still matters, and the right rule is Jackson's: serve children in decreasing order of their own finishing time T, whatever each call costs, because the finish of the child served at prefix cost P is P + T and the largest tail should start earliest. Checked against trying every order on random trees.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given a large n-ary tree where the root
# node can pass information to one child per iteration,
# and in subsequent iterations, nodes can pass information
```

```
# to one child while the root continues with others, find
# the minimum number of iterations to reach all nodes.
#
# Information starts at the root of a tree. In every
# iteration a node that already has the information can
# pass it to one of its children (the tree maps each node
# to its children as (child, cost) pairs; in the basic
# problem every cost is 1 iteration). Return the minimum
# number of iterations until every node has the
# information.
#
# Signature:
#   def min_rounds(tree: dict, root) -> int
#
# Examples:
# {'r': [('a', 1), ('b', 1), ('c', 1)], 'a': [('d', 1)]},
#     'r'  ->  3
# {'r': [('a', 1)], 'a': [('b', 1)]}, 'r'  ->  2
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is it optimal to serve the child with the largest
# subtree time first?
# 2. What if each edge has a different transmission cost?
# 3. Each call takes a different time per child.
#
# INTERVIEW NOTES
# Each node can call one child per round, so a node with
# several children must choose an order. The subtree that
# needs the most rounds should be called first, because it
# starts earliest and the others can wait. That makes the
# time at a node the maximum over children of (position in
# the calling order + the child's own time), a bottom-up
# computation.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
```

```
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def unit_tree(children):
    # turn {"r": ["a", "b"]} into the form the question uses: every call costs 1 round
    tree = {}
    for node in children:
        tree[node] = []
        for child in children[node]:
            tree[node].append((child, 1))
    return tree


def finish_times(tree, root):
    # time[v] = rounds until v and everything below it has the news, once v has it.
    # v calls its children one after another; the call to the i-th child costs the
    # calls made before it, and that child then needs time[child] more rounds.
    # Best order: the child that needs the LONGEST first.
    order = []
    stack = [root]
    while stack:                                    # parents before children
        node = stack.pop()
```

```
        order.append(node)
        for child, cost in tree.get(node, []):
            stack.append(child)
    time = {}
    plan = {}
    for i in range(len(order) - 1, -1, -1):        # children before parents
        node = order[i]
        kids = []
        for index, pair in enumerate(tree.get(node, [])):
            child, cost = pair
            kids.append((-time[child], index, child, cost))   # sorts the slowest child first
        kids.sort()
        spent = 0
        worst = 0
        order_of_calls = []
        for slowness, index, child, cost in kids:
            spent += cost                          # the call to this child is finished at `spent`
            if spent + time[child] > worst:
                worst = spent + time[child]
            order_of_calls.append(child)
        time[node] = worst
        plan[node] = order_of_calls
    return time, plan
```

```
# ---- main question: main function ----
# Idea: let T(v) be the rounds needed to inform the subtree of v once v
# knows. v informs its children one per round, so serve the child with
# the BIGGEST T first (the biggest tail should start earliest) and
# T(v) = the latest finish over its children. Answer: T(root).
# Complexity: Time O(n log n) (each node sorts its children's times), Space O(n)
def min_rounds(tree, root):
    time, plan = finish_times(tree, root)
    return time[root]


# ---- main question: tests ----
def test_main_question():
    star = unit_tree({"r": ["a", "b", "c"]})             # a root with three leaves needs 3 rounds
    assert min_rounds(star, "r") == 3
    assert min_rounds({}, "r") == 0
    deep = unit_tree({"r": ["a", "b"], "b": ["c", "d"]})
    assert min_rounds(deep, "r") == 3                     # serve the child with the deeper subtree first
    assert finish_times(deep, "r")[1]["r"] == ["b", "a"]
    example = {"r": [("a", 1), ("b", 1), ("c", 1)], "a": [("d", 1)]}
    assert min_rounds(example, "r") == 3
    chain = {}
    for i in range(20000):
        chain[i] = [(i + 1, 1)]
    assert min_rounds(chain, 0) == 20000                  # a very deep tree needs no recursion


# ========================================================
# FOLLOW-UP 1: Why is it optimal to serve the child with the largest subtree time first?
# ========================================================
# Answer: an exchange argument. If a slow child is served after a fast
# one, swapping them never makes the slow child finish later, and the fast
# child still finishes no later than the slow child's old finish.


# ========================================================
# FOLLOW-UP 2: What if each edge has a different transmission cost?
# ========================================================
```

```
# ============================================================
# FOLLOW-UP 3: Each call takes a different time per child
# ============================================================
```

```
# ---- follow-up 2: tests ----
# Answer: finish_times already adds the cost of each call: the child in
# position i can only start after the costs of the calls before it, so
# T(v) = max over i of (cost of calls up to child i + T(child i)). The
# best order is still "largest T first" (Jackson's rule).
def test_follow_up_2():
    costly = {"r": [("a", 5), ("b", 1)], "b": [("c", 1)]}
    assert min_rounds(costly, "r") == 6


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 32 OF 43

Counting or reaching an amount: the knapsack loop

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

5
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Counting or reaching an amount: the knapsack loop | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M 158. Maximize Shelf Space (Unbounded Knapsack) **ColorStack * 2025-10-17 ***

M 159. Coin Change **ND**

M 160. Partition an array into two sub-arrays with equal sum. **ND**

M 161. Explain how to find a target sum in an array. **ND**

M 162. Reverse Coin Change **ND**

CODE TEMPLATE | GROUP 32 OF 43 | 5 QUESTIONS, ONE PATTERN

Counting or reaching an amount: the knapsack loop

The one idea these questions share

Make an array indexed by amount. For each item, sweep the amounts and let ways[t] depend on ways[t - item]. Sweep upwards when an item can be reused, downwards when each item can be used once. Swap += for min(...) + 1 (fewest items) or or (reachable).

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def count_ways(coins, target):
    ways = [1] + [0] * target              # CHANGE 1: meaning of ways[0] and the start values
    for c in coins:                        # items outside = combinations; amounts outside = ordered
        for t in range(c, target + 1):     # CHANGE 2: upwards = reuse, downwards = once each
            ways[t] += ways[t - c]         # CHANGE 3: += count, min(...)+1 fewest, or |= reachable
    return ways[target]
```

2. Helper functions that usually get written next to it

```
def min_coins(coins, amount):
    INF = amount + 1
    best = [0] + [INF] * amount
    for t in range(1, amount + 1):
        for c in coins:
            if c <= t:
                best[t] = min(best[t], best[t - c] + 1)
    return best[amount] if best[amount] != INF else -1


def can_split_equal(nums):
    total = sum(nums)
    if total % 2:
        return False
    reach = 1                              # bit k set = sum k reachable
    for x in nums:
        reach |= reach << x
    return bool(reach >> (total // 2) & 1)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
158	Maximize Shelf Space (Unbounded Knapsack)	reachable[t] = any(reachable[t - size]); the answer is the largest reachable total up to the capacity
159	Coin Change	fewest coins: start with infinity, best[t] = min(best[t], best[t - c] + 1); -1 when it stays infinity
160	Partition an array into two sub-arrays with equal sum.	subset sum to total / 2, written with a bitset: reach |= reach << x
161	Explain how to find a target sum in an array.	turn +/- signs into a subset sum of (total + S) / 2, items used once (sweep downwards)
162	Reverse Coin Change	given the ways table, find the coin values: the first amount whose count is wrong must be a coin; add it and rebuild

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 158 of 197 | topic 32 of 43: Counting or reaching an amount: the knapsack loop | 1 of 5

Template fit: reachable[t] = any(reachable[t - size]); the answer is the largest reachable total up to the capacity | * asked 368 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 158 of 197

Maximize Shelf Space (Unbounded Knapsack)

ASKED 2025-10-17 | 368 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 48.5 | 30 DAYS 0 | PRIORITY 83/288

ABOUT THIS QUESTION

Topic: Counting or reaching an amount: the knapsack loop

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #322 Coin Change

LeetCode match: LC 322 Coin Change / unbounded subset sum (related)

Frequency: Single report | Last asked: 2025-10-17 | Roles: SWE Intern

QUESTION

A shelf has a fixed integer capacity capacity . You have boxes of various integer sizes given in sizes , and you may use any number of each box, including duplicates (each size is available infinitely). Choose boxes so their total size does not exceed capacity , maximizing the total size used. Return that maximum total.

Function signature

```
def max_shelf_space(capacity: int, sizes: list[int]) -> int:
    ...
```

EXAMPLES

Example 1	
Input	`capacity = 10, sizes = [3, 5, 7]`
Output	`10`
Why	`3 + 7 = 10 exactly fills the shelf.`

Example 2	
Input	`capacity = 11, sizes = [5, 3]`
Output	`11`
Why	`5 + 3 + 3 = 11.`

Example 3	
Input	`capacity = 7, sizes = [4, 5]`
Output	`5`
Why	`Options within 7 are 4 or 5 (two boxes would be >= 8). The best is 5.`

CONSTRAINTS

- 0 <= capacity <= 10**4 .
- 1 <= len(sizes) <= 100 , each size is a positive integer.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_shelf(capacity, sizes):
    best = 0
    for s in sizes:
        if s <= capacity:
            best = max(
                best,
                s
                + brute_shelf(
                    capacity - s, sizes
                ),
            )
    return best
```

TIME	SPACE
exponential	O(capacity)

WHY IT IS SLOW

The same remaining capacity is solved again and again.

OPTIMAL SOLUTION · Reachable totals DP

TIME	SPACE
O(capacity * boxes)	O(capacity)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Each box size is only available once.

That is a 0/1 knapsack (subset sum). Process the boxes one at a time and iterate the capacity downward so a box is not used twice: reachable[c] |= reachable[c - s].

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Maximize Shelf Space (Unbounded Knapsack)
#
# A shelf has a fixed integer capacity capacity . You have
# boxes of various integer sizes given in sizes , and you
# may use any number of each box, including duplicates
# (each size is available infinitely). Choose boxes so
# their total size does not exceed capacity , maximizing
# the total size used. Return that maximum total. Function
# signature: def max_shelf_space(capacity: int, sizes:
# list[int]) -> int: ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Each box size is only available once.
#
# INTERVIEW NOTES
# Which totals up to the capacity can be built from
# unlimited boxes of the given sizes? reachable[c] is true
# if some size s fits and reachable[c - s] is true. The
```

```
# answer is the largest reachable total, a one-dimensional
# unbounded knapsack where the value of a box equals its
# size.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: reachable[c] says "the boxes can fill exactly c units of shelf".
# 0 is reachable, and c is reachable if taking away one box (size s <= c)
# leaves a reachable amount. A box size may be used many times (unbounded).
# The answer is the biggest reachable amount up to the capacity.
# Complexity: Time O(capacity * number of sizes), Space O(capacity)
def max_shelf_space(capacity, sizes):
    reachable = [True] + [False] * capacity
    for c in range(1, capacity + 1):
        for s in sizes:
            if s <= c and reachable[c - s]:
                reachable[c] = True
    best = 0
```

```
    for c in range(capacity + 1):
        if reachable[c]:
            best = c
    return best


# ---- main question: tests ----
def test_main_question():
    assert max_shelf_space(10, [3, 5, 7]) == 10
    assert max_shelf_space(11, [5, 3]) == 11
    assert max_shelf_space(7, [4, 5]) == 5
    assert max_shelf_space(3, [5]) == 0


# ============================================================
# FOLLOW-UP 1: Each box size is only available once
# ============================================================
# ---- follow-up 1: function ----
# Answer: that is the 0/1 knapsack (subset sum). Take the boxes one at a
# time and go through the amounts DOWNWARD, so a box is not used twice:
# reachable[c] becomes true when reachable[c - s] is.
# Complexity: Time O(capacity * number of sizes), Space O(capacity)
def max_shelf_space_once(capacity, sizes):
    reachable = [True] + [False] * capacity
    for s in sizes:
        for c in range(capacity, s - 1, -1):
            if reachable[c - s]:
                reachable[c] = True
    best = 0
    for c in range(capacity + 1):
        if reachable[c]:
            best = c
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
```

```
    assert max_shelf_space_once(10, [3, 5, 7]) == 10        # 3 + 7
    assert max_shelf_space_once(11, [5, 3]) == 8            # each box once: 5 + 3
    assert max_shelf_space_once(3, [5]) == 0


# ====================================================
# RUN EVERYTHING
# ====================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 159 of 197 | topic 32 of 43: Counting or reaching an amount: the knapsack loop | 2 of 5

Template fit: fewest coins: start with infinity, best[t] = min(best[t], best[t - c] + 1); -1 when it stays infinity | ND

MEDIUM | IN APP LC 322 | Exponent | 159 of 197

Coin Change

ASKED (ANY CO.) ~7 MONTHS AGO | OLDER

STUDY: TYPE ALL TIME 48.5 30 DAYS 0 PRIORITY 82/288

ABOUT THIS QUESTION

Topic: Counting or reaching an amount: the knapsack loop

Optimal answer from: your app's interview approach (LeetCode #322)

LeetCode match: LC 322 Coin Change

Last asked (any company): 7 months ago, about 2026-03 (Exponent shows only a rough age) | +27 interviews

Original: https://www.aced.io/questions/1142/coin-change

QUESTION

You are given an integer array coins representing different coin denominations and an integer amount representing a total amount of money. Write a function coinChange that returns the fewest number of coins needed to make up that amount. If that amount cannot be made up by any combination of the coins, return -1. You may assume that you have an infinite number of each kind of coin. Examples PythonInput: coins = [1, 2, 5], amount = 11 Output: 3 Explanation: The minimum number of coins needed is 3 (11 = 5 + 5 + 1).
Input: coins = [4, 5], amount = 7 Output: -1 Explanation: It is not possible to make up the amount 7 with coins of denominations 4 and 5.Solution 1: Brute-Force approach The naive approach to solving the coin change problem involves a recursive strategy to explore all possible combinations of coins to find the minimum number needed to make up the target amount. It starts by recursively trying each coin denomination to reduce the target amount. For each coin, the algorithm subtracts the coin's value from the current amount and solves the reduced problem. If the amount becomes zero, it indicates that no more coins are needed, and the function returns zero. If the amount becomes negative, it signals that this path is not feasible, so the function returns infinity or a large number. The algorithm combines the results from each recursive call with the current coin to determine the minimum number of coins required. Finally, after exploring all possibilities, it returns the

EXAMPLES

Example 1

Input	`[1, 2, 5], 11`
Output	`3`
Why	`5 + 5 + 1 = 11: 3 coins.`

Example 2

Input	`[2], 3`
Output	`-1`
Why	`A coin of 2 can never make 3.`

Example 3

Input	`[1], 0`
Output	`0`
Why	`Zero needs no coins.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def fewest(a):
    if a == 0:
        return 0
    best = INF
    for c in coins:
        if c <= a:
            best = min(best, 1 + fewest(a - c))
    return best
```

TIME	SPACE
O(coins^amount)	O(amount)

WHY IT IS SLOW

The same amount is recomputed through many coin orders, exponential. Filling fewest[a] for a from 0 upward is O(amount * coins).

OPTIMAL SOLUTION · Bottom-up DP over amounts

TIME	SPACE
O(amount * coins)	O(amount)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Count the number of combinations that make the amount (LeetCode 518).

Loop over coins in the outer loop and amounts in the inner loop, adding ways[a - c]; the order of the loops makes combinations (not permutations) count once.

FOLLOW-UP 2 Return which coins are used.

Remember the last coin chosen for each amount (a parent array) and walk back from the target.

FOLLOW-UP 3 Does greedy work?

Only for special coin systems (like US coins). For general coins it can be wrong (coins 1, 3, 4, amount 6). Mention the counterexample, then give the DP.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Coin Change
#
# You are given an integer array coins representing
# different coin denominations and an integer amount
# representing a total amount of money. Write a function
# coinChange that returns the fewest number of coins
# needed to make up that amount. If that amount cannot be
# made up by any combination of the coins, return -1. You
# may assume that you have an infinite number of each kind
# of coin. Examples PythonInput: coins = [1, 2, 5], amount
```

```
# = 11 Output: 3 Explanation: The minimum number of coins
# needed is 3 (11 = 5 + 5 + 1). Input: coins = [4, 5],
# amount = 7 Output: -1 Explanation: It is not possible to
# make up the amount 7 with coins of denominations 4 and
# 5.Solution 1: Brute-Force approach The naive approach to
# solving the coin change problem involves a recursive
# strategy to explore all possible combinations of coins
# to find the minimum number needed to make up the target
# amount.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Count the number of combinations that make the amount
# (LeetCode 518).
# 2. Return which coins are used.
# 3. Does greedy work?
#
# INTERVIEW NOTES
# The fewest coins for amount a is one coin plus the
# fewest for a - c, for the best coin c; that is a table
# over amounts built from 0 upward. Greedy (always take
# the biggest coin) is wrong for general coin systems,
# which is the classic trap: with coins 1, 3, 4 the amount
# 6 needs two coins (3 + 3), not three (4 + 1 + 1).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random

INF = float("inf")
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea (bottom-up table): best[a] = the fewest coins that make exactly a.
# best[0] = 0, and for a = 1, 2, ... try every coin c <= a: best[a] =
# 1 + best[a - c] when that is better. Every amount is solved once.
# Answer best[amount], or -1 if it stayed infinite.
# Complexity: Time O(amount * number of coins), Space O(amount)
def coin_change(coins, amount):
    best = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and best[a - c] + 1 < best[a]:
                best[a] = best[a - c] + 1
    if best[amount] == INF:
        return -1
    return best[amount]


# ---- main question: tests ----
def test_main_question():
    assert coin_change([1, 2, 5], 11) == 3
    assert coin_change([2], 3) == -1
    assert coin_change([1], 0) == 0


# ============================================================
# FOLLOW-UP 1: Count the number of combinations that make the amount (LeetCode 518)
```

```
# ====================================================
# ---- follow-up 1: function ----
# Answer: put the COINS in the outer loop and the amounts in the inner
# loop, adding ways[a - c]. That order counts each combination once; with
# the loops the other way round you would count every order separately.
# Complexity: Time O(amount * number of coins), Space O(amount)
def count_ways(coins, amount):
    ways = [1] + [0] * amount
    for c in coins:
        for a in range(c, amount + 1):
            ways[a] += ways[a - c]
    return ways[amount]


# ---- follow-up 1: tests ----
```

```
def test_follow_up_1():
    assert count_ways([1, 2, 5], 5) == 4
    assert count_ways([2], 3) == 0
    assert count_ways([10], 10) == 1
    assert count_ways([1, 2], 0) == 1


# ====================================================
# FOLLOW-UP 2: Return which coins are used
# ====================================================
# ---- follow-up 2: function ----
# Answer: remember the last coin chosen for every amount, then walk back
# from the target.
# Complexity: Time O(amount * number of coins), Space O(amount)
def coins_used(coins, amount):
    best = [0] + [INF] * amount
    last = [0] * (amount + 1)
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and best[a - c] + 1 < best[a]:
                best[a] = best[a - c] + 1
                last[a] = c
    if best[amount] == INF:
        return None
    used = []
    a = amount
    while a > 0:
        used.append(last[a])
        a -= last[a]
    used.sort(reverse=True)
    return used


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert coins_used([1, 2, 5], 11) == [5, 5, 1]
    assert coins_used([2], 3) is None
    random.seed(8)
    for round_number in range(300):
        coins = random.sample(range(1, 9), random.randint(1, 3))
        amount = random.randint(0, 25)
        used = coins_used(coins, amount)
        if used is not None:
            assert sum(used) == amount and len(used) == coin_change(coins, amount)
```

```
# ====================================================
# FOLLOW-UP 3: Does greedy work?
# ====================================================
```

```
# ---- follow-up 3: function ----
# Answer: only for special coin systems (like US coins). For general
# coins it can be wrong (coins 1, 3, 4 and amount 6). Mention the
# counterexample, then give the DP.
# Complexity: Time O(c log c + number of coins used) for c coin values, Space O(c) (it is not always correc
t)
def greedy(coins, amount):
    used = []
    for c in sorted(coins, reverse=True):          # always take the biggest coin that still fits
        while amount >= c:
            amount -= c
            used.append(c)
    if amount == 0:
        return used
    return None


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert greedy([1, 3, 4], 6) == [4, 1, 1]            # greedy: 3 coins ...
    assert coin_change([1, 3, 4], 6) == 2               # ... but 3 + 3 needs only 2
    assert coins_used([1, 3, 4], 6) == [3, 3]


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

You are given an integer array coins representing different coin denominations and an integer amount representing a total amount of money. Write a function coinChange that returns the fewest number of coins needed to make up that amount. If that amount cannot be made up by any combination of the coins, return -1. You may assume that you have an infinite number of each kind of coin. Examples

```
PythonInput: coins = [1, 2, 5], amount = 11
Output: 3
Explanation: The minimum number of coins needed is 3 (11 = 5 + 5 + 1).

Input: coins = [4, 5], amount = 7
Output: -1
Explanation: It is not possible to make up the amount 7 with coins of denominations 4 and 5.
```

Solution 1: Brute-Force approach The naive approach to solving the coin change problem involves a recursive strategy to explore all possible combinations of coins to find the minimum number needed to make up the target amount. It starts by recursively trying each coin denomination to reduce the target amount. For each coin, the algorithm subtracts the coin's value from the current amount and solves the reduced problem. If the amount becomes zero, it indicates that no more coins are needed, and the function returns zero. If the amount becomes negative, it signals that this path is not feasible, so the function returns infinity or a large number. The algorithm combines the results from each recursive call with the current coin to determine the minimum number of coins required. Finally, after exploring all possibilities, it returns the minimum number of coins found or -1 if it is not possible to make the target amount with the given coins.

OFFICIAL PAGE TEXT

```
PseudocodeFUNCTION coin_change_naive(coins, amount):
    IF amount == 0:
        RETURN 0  // Base case: No coins are needed to make amount 0

    IF amount < 0:
        RETURN infinity  // Amount is negative, path is not feasible

    min_coins = infinity

    FOR EACH coin IN coins:
        result = coin_change_naive(coins, amount - coin)  // Recursively solve for the remaining amount
        IF result != infinity:
            min_coins = MIN(min_coins, result + 1)  // Update min_coins with the minimum value

    RETURN min_coins IF min_coins != infinity ELSE -1  // Return the minimum number of coins or -1 if no s
olution exists
```

Time Complexity: The solution has a time complexity of O(m^n), where n is the amount and m is the number of coin denominations. The exponential time complexity arises from the recursive exploration of all possible combinations of coins. Each recursive call generates multiple further calls, leading to an exponential growth in the number of computations required. The total number of recursive calls can be as large as mnm^nmn due to the need to try each coin denomination for each sub-amount. Space Complexity: The space complexity of the naive approach is O(n), where n is the target amount. The space complexity is determined by the maximum depth of the recursion stack. In the worst case, the recursion depth can be as deep as the target amount n (when using coins of value 1). Therefore, the space required for the call stack is proportional to the target amount. As you can see, this solution does not perform very well. Let's take a look at a better solution. Solution 2: Dynamic programming approach Our solution employs a dynamic programming approach to determine the fewest number of coins needed to make up a given amount. We use a DP array, where each index represents a specific amount, and its value represents the minimum number of coins required to achieve that amount. The array is initialized with a value greater than the maximum possible number of coins needed (infinity), except for the base case where dp[0] is set to 0, indicating that zero coins are needed to make amount 0. For each amount from 1 to the target amount, we iterate through each coin denomination and update the DP array. If the current amount minus the coin value is non-negative and the previous amount can be formed, we update the current amount’s value to be the minimum of its current value or one more than the value of the previous amount. This ensures that we are considering the fewest coins needed by checking each possible coin combination. After processing all amounts, if the value for the target amount is still set to infinity, it means it's not possible to form the target amount with the given coins, and we return -1. Otherwise, we return the value at dp[amount], which represents the minimum number of coins needed. Time Complexity: The solution has a time complexity of O(n * m), where n is the amount and m is the number of coin denominations. This is because we iterate over each amount from 1 to the target amount, and for each amount, we iterate over all coin denominations to update the DP array. Space Complexity: The solution uses O(n) space for the DP array, where n is the target

amount. The array stores the minimum number of coins required to make each amount up to the target amount. This space complexity is necessary to hold the results for all subproblems up to the target amount.No answers contributed by the community yet.

STUDY PATH: question 160 of 197 | topic 32 of 43: Counting or reaching an amount: the knapsack loop | 3 of 5

Template fit: subset sum to total / 2, written with a bitset: reach |= reach << x | ND

MEDIUM | IN APP LC 416 | Exponent | 160 of 197

Partition an array into two sub-arrays with equal sum.

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 46.6 30 DAYS 0 PRIORITY 86/288

ABOUT THIS QUESTION

Topic: Counting or reaching an amount: the knapsack loop

Optimal answer from: your app's interview approach (LeetCode #416)

LeetCode match: LC 416 Partition Equal Subset Sum

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +26 interviews

Original: https://www.aced.io/questions/969/partition-array-equal-sum

QUESTION

Given an array of integers, determine if it can be partitioned into two subsets such that the sum of the elements in each subset is equal. A partition is considered valid if it divides the array into two subsets where both subsets have the same sum. Examples Input: [2, 4, 11, 5]
Output: True Explanation: The array can be partitioned into two subsets [2, 4, 5] and [11] with equal sum.
Input: [2, 7, 3, 1]
Output: False Explanation: The array cannot be partitioned into two subsets with equal sums.

EXAMPLES

Example 1	
Input	`[1, 5, 11, 5]`
Output	`true`
Why	`[1, 5, 5] and [11] both add up to 11.`

Example 2	
Input	`[1, 2, 3, 5]`
Output	`false`
Why	`The total is 11, which is odd.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums):
    total = sum(nums)
    if total % 2:
        return False
    for size in range(len(nums) + 1):
        for subset in combinations(nums, size):
            if sum(subset) == total // 2:
                return True
    return False
```

TIME	SPACE
O(2^n * n)	O(n)

BRUTE FORCE

WHY IT IS SLOW

2^n subsets. Subset-sum as a reachability table (or a bitset shifted by each number) is O(n * sum).

OPTIMAL SOLUTION · Subset-sum as a bitset

TIME	SPACE
O(n * sum / 64) with big-integer shifts (O(n * sum) as a boolean table)	**O(sum)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the two subsets.

Keep, for each reachable sum, the list of indices (or a parent pointer) that first reached it; read off the subset for total / 2 and give the rest to the other side.

FOLLOW-UP 2 Minimise the difference between the two parts (LeetCode 1049).

Compute all reachable subset sums with the same bitset and choose the reachable s closest to total / 2: the answer is total - 2s.

FOLLOW-UP 3 The parts must be contiguous sub-arrays (a split point).

Then only one split can work: compute prefix sums and check whether some prefix equals total / 2. O(n), O(1) space; no DP is needed.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Partition an array into two sub-arrays
# with equal sum.
#
# Given an array of integers, determine if it can be
# partitioned into two subsets such that the sum of the
# elements in each subset is equal. A partition is
# considered valid if it divides the array into two
# subsets where both subsets have the same sum. Examples
# Input: [2, 4, 11, 5] Output: True Explanation: The array
# can be partitioned into two subsets [2, 4, 5] and [11]
# with equal sum. Input: [2, 7, 3, 1] Output: False
# Explanation: The array cannot be partitioned into two
# subsets with equal sums.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the two subsets.
# 2. Minimise the difference between the two parts
# (LeetCode 1049).
# 3. The parts must be contiguous sub-arrays (a split
# point).
#
# INTERVIEW NOTES
# Two equal parts mean each sums to total / 2, so if the
# total is odd the answer is no. Otherwise the question is
```

```
# whether some subset reaches total / 2, which is subset-
# sum. A bitset where bit s says 'sum s is reachable'
# updates with one shift and one OR per number.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: the array can be split into two parts with the same sum exactly
# when some subset adds up to half of the total. reachable[s] means "some
# subset adds up to s"; for each number x go through s from high to low
# (so x is used once) and mark reachable[s] when reachable[s - x] is true.
# Complexity: Time O(n * sum / 2), Space O(sum)
def can_partition(nums):
    total = 0
    for x in nums:
        total += x
    if total % 2 == 1:
        return False                                # an odd total cannot be split evenly
    half = total // 2
    reachable = [True] + [False] * half
    for x in nums:
        for s in range(half, x - 1, -1):
            if reachable[s - x]:
                reachable[s] = True
    return reachable[half]


# ---- main question: tests ----
def test_main_question():
    assert can_partition([2, 4, 11, 5]) is True           # 2 + 4 + 5 = 11 = the other part
    assert can_partition([1, 2, 5]) is False
    assert can_partition([]) is True
    assert can_partition([1]) is False


# ========================================================
# FOLLOW-UP 1: Return the two subsets
# ========================================================
# ---- follow-up 1: function ----
# Answer: for every reachable sum remember the number of the item that
# FIRST reached it. Read off the subset for half the total by walking
# back (sum, minus that item, ...) and give the rest to the other side.
# Complexity: Time O(n * sum + n^2), Space O(sum)
def partition_subsets(nums):
    total = 0
    for x in nums:
        total += x
    if total % 2 == 1:
        return None
    half = total // 2
    first_item = {0: -1}                                # sum -> the item that first reached it
    for i in range(len(nums)):
        for s in list(first_item):                      # a copy: each item is used once
            if s + nums[i] <= half and s + nums[i] not in first_item:
                first_item[s + nums[i]] = i
    if half not in first_item:
```

```
        return None
    chosen = []
```

```
    s = half
    while s > 0:
        i = first_item[s]
        chosen.append(i)
        s -= nums[i]
    left = []
    right = []
    for i in range(len(nums)):
        if i in chosen:
            left.append(nums[i])
        else:
            right.append(nums[i])
    return left, right


# ---- follow-up 1: tests ----
def test_follow_up_1():
    left, right = partition_subsets([1, 5, 11, 5])
    assert sum(left) == sum(right) == 11
    assert sorted(left + right) == [1, 5, 5, 11]
    assert partition_subsets([1, 2, 5]) is None


# =========================================================
# FOLLOW-UP 2: Minimise the difference between the two parts (LeetCode 1049)
# =========================================================
# ---- follow-up 2: function ----
# Answer: find every reachable subset sum up to half the total and take
# the biggest one, s; the best split has difference total - 2 * s.
# Complexity: Time O(n * sum), Space O(sum)
def min_difference(nums):
    total = 0
    for x in nums:
        total += x
    half = total // 2
    reachable = [True] + [False] * half
    for x in nums:
        for s in range(half, x - 1, -1):
            if reachable[s - x]:
                reachable[s] = True
    best = 0
    for s in range(half + 1):
        if reachable[s]:
            best = s
    return total - 2 * best
```

```


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert min_difference([1, 6, 11, 5]) == 1
    assert min_difference([2, 4, 11, 5]) == 0


# =========================================================
# FOLLOW-UP 3: The parts must be contiguous sub-arrays (a split point)
# =========================================================
# ---- follow-up 3: function ----
# Answer: then only one cut position can work: add up a running prefix
# and check whether it ever equals half the total. O(n) time, O(1) space,
# no DP.
# Complexity: Time O(n), Space O(1)
```

```
def can_split_contiguous(nums):
    total = 0
    for x in nums:
        total += x
    if total % 2 == 1:
        return False
    prefix = 0
    for x in nums:
        prefix += x
        if prefix * 2 == total:
            return True
    return False


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert can_split_contiguous([1, 2, 3, 6])
    assert not can_split_contiguous([1, 5, 11, 5])


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given an array of integers, determine if it can be partitioned into two subsets such that the sum of the elements in each subset is equal. A partition is considered valid if it divides the array into two subsets where both subsets have the same sum. Examples

```
Input: [2, 4, 11, 5]
Output: True
Explanation: The array can be partitioned into two subsets [2, 4, 5] and [11] with equal sum.

Input: [2, 7, 3, 1]
Output: False
Explanation: The array cannot be partitioned into two subsets with equal sums.
```

OFFICIAL PAGE TEXT

Our solution processes the input array by first calculating the total sum of the array elements and then determining if it can be partitioned into two subsets with equal sums. The core idea is based on dynamic programming. Calculate Total Sum: We start by computing the total sum of the array elements. If the total sum is odd, it is immediately impossible to partition it into two subsets with equal sums, so we return false. Define Target Sum: If the total sum is even, we calculate the target sum for each subset, which is half of the total sum. Dynamic Programming Array: We use a boolean array dp where dp[i] indicates whether a subset sum of i is achievable. We initialize dp[0] to true because a subset sum of 0 is always achievable (by taking no elements). Iterate Through the Array: For each number in the array, we update the dp array in reverse order (from target down to the current number). This reverse iteration ensures that each number is only used once in the current update. If a subset sum i - num was achievable, then adding the current number num makes dp[i] achievable. We update dp[i] to true accordingly. Check Final Result: After processing all numbers, if dp[target] is true, it means there is a subset with the required sum equal to target, indicating that the array can be partitioned into two subsets with equal sums. Otherwise, we return false. The use of a boolean array ensures that we efficiently keep track of achievable subset sums. Time Complexity: The solution has a time complexity of O(n * target), where n is the number of elements in the array and target is half of the total sum. This is because we iterate through the array and update the dp array for each element. Space Complexity: The solution uses O(target) space for the dp array, where target is half of the total sum. This space is needed to store the achievable subset sums.No answers contributed by the community yet.

STUDY PATH: question 161 of 197 | topic 32 of 43: Counting or reaching an amount: the knapsack loop | 4 of 5

Template fit: turn +/- signs into a subset sum of (total + S) / 2, items used once (sweep downwards) | ND

MEDIUM | IN APP LC 494 | Exponent | 161 of 197

Explain how to find a target sum in an array.

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TYPE ALL TIME 41.1 30 DAYS 0 PRIORITY 102/288

ABOUT THIS QUESTION

Topic: Counting or reaching an amount: the knapsack loop

Optimal answer from: your app's interview approach (LeetCode #494)

LeetCode match: LC 494 Target Sum

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age) | 3 interviews

Original: https://www.aced.io/questions/1439/find-target-sum-array

QUESTION

Determine the Number of Ways to Assign Symbols to Reach a Target Sum. You are given a list of non-negative integers, a1, a2, ..., an, and a target sum S. Your task is to assign each integer in the list a symbol + or - and calculate how many different ways there are to arrange these symbols such that the sum of the integers equals S after applying the assigned symbols. For example, if you have the list [1, 1, 1, 1, 1] and the target sum 3, one way to reach the target sum is to assign + to three 1s and - to two 1s, like so: +1 +1 +1 +1 -1. Another way could be +1 -1 +1 +1 +1. You need to return the total number of unique arrangements that meet the target sum S. Examples nums = [1, 1, 1, 1, 1] targetSum = 3 output: 5 nums = [1, 2, 1] targetSum = 2 output: 2 nums = [1, 2, 7, 1, 5, 3] targetSum = 8 output: -1 (Explanation: Since the total sum of the numbers plus the target is odd, it's not possible to reach the target sum.)The original problem asks for the number of ways to assign symbols (+ or -) to a list of numbers such that their sum equals a target sum S. This problem can be converted into a problem of finding subsets of the given numbers whose sum equals (totalSum + S) / 2. This is because if we can find subsets whose sum equals (totalSum + S) / 2, the remaining numbers will naturally form the other subset that sums to (totalSum - S) / 2, which effectively gives us the desired configuration of symbols. NoteWhy (totalSum + S) / 2?Given a list of numbers nums=[a1,a2,...,an]\te

EXAMPLES

Example 1	
Input	`[1, 1, 1, 1, 1], 3`
Output	`5`
Why	`Put + or - in front of every number; five ways to reach 3 with five 1s.`

Example 2	
Input	`[1], 1`
Output	`1`
Why	`+1 is the only way.`

Example 3	
Input	`[1, 2], 5`
Output	`-1`
Why	`No sign choice gives 5.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def go(i, total):
    if i == len(nums):
        return 1 if total == S else 0
    plus = go(i + 1, total + nums[i])
    minus = go(i + 1, total - nums[i])
    return plus + minus
answer = go(0, 0)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Two branches per number: O(2^n). The numbers with a + sign form a subset with sum (total + S) / 2, so a subset-sum counting table is O(n * sum).

OPTIMAL SOLUTION · Subset-sum counting: choose P with sum (total + S) / 2

TIME	SPACE
O(n * sum)	O(sum)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The question was really 'find two numbers that add up to the target' (Two Sum).

One pass with a dictionary from value to index: for each x look for target - x among earlier values. O(n) time, O(n) space; with a sorted array use two pointers in O(1) space.

FOLLOW-UP 2 Only the existence of a subset with the target sum.

The same subset-sum table with booleans (or a bitset: reach |= reach << x) answers existence in O(n * sum / word size).

FOLLOW-UP 3 Negative numbers or a huge sum.

With negatives shift all values to be non-negative, or use a dictionary from reachable sum to count (memoised DFS on (index, sum)); with a huge sum the dictionary is only as large as the number of distinct reachable sums. **Cost:** O(n * R) with a dictionary over R reachable sums; shifting for negatives adds O(n).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Explain how to find a target sum in an
# array.
#
# Determine the Number of Ways to Assign Symbols to Reach
# a Target Sum. You are given a list of non-negative
# integers, a1, a2, ..., an, and a target sum S. Your task
# is to assign each integer in the list a symbol + or -
# and calculate how many different ways there are to
# arrange these symbols such that the sum of the integers
```

```
# equals S after applying the assigned symbols. For
# example, if you have the list [1, 1, 1, 1, 1] and the
# target sum 3, one way to reach the target sum is to
# assign + to three 1s and - to two 1s, like so: +1 +1 +1
# +1 -1. Another way could be +1 -1 +1 +1 +1. You need to
# return the total number of unique arrangements that meet
# the target sum S.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The question was really 'find two numbers that add up
# to the target' (Two Sum).
# 2. Only the existence of a subset with the target sum.
# 3. Negative numbers or a huge sum.
#
# INTERVIEW NOTES
# Put a + in front of the numbers in a set P and a - in
# front of the rest N. Then sum(P) - sum(N) = S and sum(P)
# + sum(N) = total, so sum(P) = (total + S) / 2. The
# problem becomes 'count the subsets with this sum', a
# one-dimensional knapsack count. If (total + S) is odd or
# negative the answer is 0.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
```

```
# Put a + or - sign in front of every number so that they add up to
# `target`; count the ways (-1 if the target cannot be reached at all).
# Idea: the numbers with a PLUS sign add up to P, the others to total - P,
# so P - (total - P) = target, i.e. P = (total + target) / 2. The question
# becomes "how many subsets add up to P?". ways[s] = the number of subsets
# with sum s; for each number x update s from high to low (so each number is
# used once): ways[s] += ways[s - x].
# Complexity: Time O(n * S) for S = (sum + target) / 2, Space O(S)
def count_signs(nums, target):
    total = 0
    for x in nums:
        total += x
    if abs(target) > total or (total + target) % 2 == 1:
        return -1
    goal = (total + target) // 2
    ways = [1] + [0] * goal
    for x in nums:
        for s in range(goal, x - 1, -1):
            ways[s] += ways[s - x]
    return ways[goal]


# ---- main question: tests ----
def test_main_question():
    assert count_signs([1, 1, 1, 1, 1], 3) == 5
    assert count_signs([1, 2, 1], 2) == 2
    assert count_signs([1, 2, 7, 1, 5, 3], 8) == -1
    assert count_signs([1], 5) == -1
```

```
    assert count_signs([0, 0, 1], 1) == 4
    assert count_signs([], 0) == 1


# ======================================================
# FOLLOW-UP 1: The question was really "find two numbers that add up to the target" (Two Sum)
# ======================================================
# ---- follow-up 1: function ----
# Answer: one pass with a dictionary from value to index: for each x look
# for target - x among the earlier numbers. O(n) time and space. With a
# SORTED array use two pointers instead: O(1) space.
# Complexity: Time O(n), Space O(n)
def two_sum(nums, target):
    seen = {}
    for i in range(len(nums)):
        if target - nums[i] in seen:
            return seen[target - nums[i]], i
        seen[nums[i]] = i
    return None


# Complexity: Time O(n), Space O(1)
def two_sum_sorted(nums, target):
    low = 0
    high = len(nums) - 1
    while low < high:
        total = nums[low] + nums[high]
        if total == target:
            return low, high
        if total < target:
            low += 1
        else:
            high -= 1
    return None


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert two_sum([2, 7, 11, 15], 9) == (0, 1)
    assert two_sum([1, 2], 7) is None
    assert two_sum_sorted([1, 3, 4, 8], 11) == (1, 3)


# ======================================================
# FOLLOW-UP 2: Only the existence of a subset with the target sum
# ======================================================
# ---- follow-up 2: function ----
# Answer: the same subset-sum table with True/False instead of counts:
# reachable[s] says "some subset adds up to s".
# Complexity: Time O(n * goal), Space O(goal)
def subset_exists(nums, goal):
    reachable = [True] + [False] * goal
    for x in nums:
        for s in range(goal, x - 1, -1):
            if reachable[s - x]:
                reachable[s] = True
    return reachable[goal]


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert subset_exists([3, 34, 4, 12, 5, 2], 9)
```

```
    assert not subset_exists([3, 34, 4], 8)


# =====================================================
# FOLLOW-UP 3: Negative numbers or a huge sum
# =====================================================
# ---- follow-up 3: function ----
# Answer: with negative numbers a table indexed by the sum no longer
# works. Use a dictionary from "reachable sum" to "number of ways": it
# holds only the sums that really occur, so a huge target costs nothing.
# Complexity: Time O(n * number of distinct sums), Space O(number of distinct sums)
def count_signs_dict(nums, target):
    ways = {0: 1}
    for x in nums:
        following = {}
        for s in ways:
            for new_sum in (s + x, s - x):
                following[new_sum] = following.get(new_sum, 0) + ways[s]
        ways = following
    return ways.get(target, 0)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert count_signs_dict([1, 1, 1, 1, 1], 3) == 5
    assert count_signs_dict([-1, 2, 1], 2) == 2            # negative numbers work with the dictionary
    random.seed(3)
    for round_number in range(300):
        nums = []
        for i in range(random.randint(0, 7)):
            nums.append(random.randint(0, 4))
        target = random.randint(-8, 12)
        expected = count_signs_dict(nums, target)
        got = count_signs(nums, target)
        assert got == expected or (got == -1 and expected == 0)


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Determine the Number of Ways to Assign Symbols to Reach a Target Sum. You are given a list of non-negative integers, a1, a2, ..., an, and a target sum S. Your task is to assign each integer in the list a symbol + or - and calculate how many different ways there are to arrange these symbols such that the sum of the integers equals S after applying the assigned symbols. For example, if you have the list [1, 1, 1, 1, 1] and the target sum 3, one way to reach the target sum is to assign + to three 1s and - to two 1s, like so: +1 +1 +1 +1 -1. Another way could be +1 -1 +1 +1 +1. You need to return the total number of unique arrangements that meet the target sum S. Examples

OFFICIAL PAGE TEXT

```
nums = [1, 1, 1, 1, 1]
targetSum = 3
output: 5

nums = [1, 2, 1]
targetSum = 2
output: 2

nums = [1, 2, 7, 1, 5, 3]
targetSum = 8
output: -1 (Explanation: Since the total sum of the numbers plus the target is odd, it's not possible to r
each the target sum.)
```

The original problem asks for the number of ways to assign symbols (+ or -) to a list of numbers such that their sum equals a target sum S. This problem can be converted into a problem of finding subsets of the given numbers whose sum equals (totalSum + S) / 2. This is because if we can find subsets whose sum equals (totalSum + S) / 2, the remaining numbers will naturally form the other subset that sums to (totalSum - S) / 2, which effectively gives us the desired configuration of symbols. NoteWhy (totalSum + S) / 2?Given a list of numbers nums=[a1,a2,...,an]\text{nums} = [a_1, a_2, ..., a_n]nums=[a1,a2,...,an], we need to assign either a +++ or a --- to each number so that the total sum equals a target SSS.Let's denote the sum of the entire list as sum(nums)=totalSum\text{sum(nums)} = \text{totalSum}sum(nums)=totalSum.The problem can be rephrased as finding subsets of nums\text{nums}nums such that the difference between the sum of elements in the subset and the sum of elements not in the subset equals SSS.Formulating the Equation: Let PPP be the sum of elements with a +++ sign and NNN be the sum of elements with a --- sign. We need P-N=SP - N = SP-N=S. We also know that P+N=totalSumP + N = \text{totalSum}P+N=totalSum, where totalSum\text{totalSum}totalSum is the sum of all elements in nums\text{nums}nums. Combining the Equations: Adding these two equations:P-N+P+N=S+totalSum2P=S+totalSumP - N + P + N = S + \text{totalSum}\\ 2P = S + \text{totalSum}P-N+P+N=S+totalSum2P=S+totalSumTherefore, P=S+totalSum2P = \frac{S + \text{totalSum}}{2}P=2S+totalSum.The problem now reduces to finding subsets of nums\text{nums}nums whose sum equals S+totalSum2\frac{S + \text{totalSum}}{2}2S+totalSum. Approach We define a recursive function find_subset that takes an index, the list of numbers, the target subset sum, and a memoization table (dp). The function recursively explores two possibilities at each step: Include the current number in the subset. Exclude the current number from the subset. The base cases for the recursion are: If the target subset sum (s) is zero, it means we found a valid subset, so we return 1. If we reach the end of the list without finding the subset sum, we return 0. If we have already computed the value for the current state (index and subset sum), we return the stored value from the memoization table to avoid redundant calculations. The changeSigns function calculates the total sum of the

OFFICIAL PAGE TEXT

numbers and checks if it is possible to partition them to meet the required target sum S. If it is possible, it initializes the memoization table and calls the find_subset function to compute the number of ways to achieve the target sum. Solution Time Complexity: The time complexity of this solution is O(n x (totalSum + S)) / 2, where n is the number of elements in the input list. This is because each state in the memoization table can be computed in constant time, and there are n x (totalSum + S) / 2 states to fill. Space Complexity: The space complexity of this solution is O(n x (totalSum + S) / 2) due to the memoization table used to store intermediate results. Additionally, the recursive call stack can go up to O(n) in the worst case, but this is dominated by the space required for the memoization table.Watch our mock Flipkart SWE (software engineering) interview. Neamah asks Rohan (Flipkart SWE) the Target Sum leetcode question.No answers contributed by the community yet.Related questionsFind a triplet in an array with a given sum.Given an array, find the two sum.Find quadruplets in an array with a given sum.Related coursesCourseCourseSoftware Engineering Interview PrepLand your dream software engineering role at Google, Amazon, Microsoft, Meta, Apple, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates—practice data structures, algorithms, system design, people management, behavioral interviews, and more.Start learningCourseCourseSystem Design InterviewsLearn how to answer the latest system design questions across product, infrastructure, and AI domains. Features in-depth video examples, written breakdowns, and helpful reference patterns. Watch senior engineers and managers answer these questions in mock interview videos.Start learningPythonJavaScriptJavaC++PythonEditorShareLoading...OutputTest ResultsConsoleSign up to run code

STUDY PATH: question 162 of 197 | topic 32 of 43: Counting or reaching an amount: the knapsack loop | 5 of 5

Template fit: given the ways table, find the coin values: the first amount whose count is wrong must be a coin; add it and rebuild | ND

MEDIUM | HackTheRounds | Onsite | 162 of 197

Reverse Coin Change

STUDY: TALK **ALL TIME 37.9** **30 DAYS 0** **PRIORITY 114/288**

ABOUT THIS QUESTION

Topic: Counting or reaching an amount: the knapsack loop
Optimal answer from: the interview site's own solution
Also in your app: LeetCode #518 Coin Change II
LeetCode match: Inverse of LC 518

QUESTION

In the coin change counting problem, dp[i] is the number of ways to make amount i from a set of coins (dp[0] = 1). In the reverse problem you are given the dp array and must find a set of coins that produces it, or None if no coin set does.

Function signature

def coins_from_dp(dp: list) -> list | None

EXAMPLES

Example 1	
Input	`[1, 1, 2, 2, 3, 3]`
Output	`[1, 2]`
Why	`Coins 1 and 2 make 1, 1, 2, 2, 3, 3 ways for the amounts 0 to 5.`

Example 2	
Input	`[1, 0, 1, 0, 1]`
Output	`[2]`
Why	`Only the coin 2 can make 2 and 4 but not 1 and 3.`

Example 3	
Input	`[1, 2, 3]`
Output	`null`
Why	`No coin set can produce these counts.`

CONSTRAINTS

- 1 <= dp.length <= 1000
- dp[0] = 1

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(dp):
    n = len(dp) - 1
    for k in range(n + 1):
        values = range(1, n + 1)
        for coins in combinations(values, k):
            if ways_table(coins, n) == dp:
                return list(coins)
    return None
```

TIME	SPACE
O(2^n * n * coins)	O(n)

WHY IT IS SLOW

2^n candidate sets, each with an O(n * coins) table. Scanning amounts upward and adding a coin whenever the target has one more way than the table is O(n^2).

OPTIMAL SOLUTION · Scan upward and add a coin whenever the table falls short

TIME	SPACE
O(n * coins) = O(n^2) worst case	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The array might not come from any set of coins.

Return None (or raise) when dp[0] != 1, when an amount needs more than one extra way, or when the rebuilt table differs from the input at the end; the code does all three checks.

FOLLOW-UP 2 Order of the coins matters (the dp counts permutations, not combinations).

That is a different recurrence: dp[a] = sum of dp[a - c] over coins. Inverting it is a linear-algebra style problem: at each amount the new coins explain the excess; ask which counting is used.

FOLLOW-UP 3 Each coin may be used at most once.

The forward recurrence iterates amounts downward. The inverse still scans upward adding a coin when the target exceeds the table, but the table update must use the 0/1 form.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Reverse Coin Change
#
# In the coin change counting problem, dp[i] is the number
# of ways to make amount i from a set of coins (dp[0] =
# 1). In the reverse problem you are given the dp array
# and must find a set of coins that produces it, or None
# if no coin set does.
#
# Signature:
```

```
#   def coins_from_dp(dp: list) -> list | None
#
# Examples:
# [1, 1, 2, 2, 3, 3]  ->  [1, 2]
# [1, 0, 1, 0, 1]  ->  [2]
#
# FOLLOW-UPS (answered below, in this order):
# 1. The array might not come from any set of coins.
# 2. Order of the coins matters (the dp counts
# permutations, not combinations).
# 3. Each coin may be used at most once.
#
# INTERVIEW NOTES
# Think of building the ways table forward: adding a coin
# c changes dp[c] by exactly one (the single coin c). So
# scanning amounts upward, the first amount where the
# target has more ways than your table can only be
# explained by a new coin of exactly that value. Add it,
# rebuild, and continue; at the end compare with the
# target to detect an impossible input.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def ways_table(coins, n):
    # the FORWARD direction: dp[a] = the number of combinations of the coins that make a.
    # (coins in the outer loop: each combination counts once, not once per order)
    dp = [1] + [0] * n
    for c in coins:
        for a in range(c, n + 1):
            dp[a] += dp[a - c]
    return dp


# ---- main question: main function ----
# The usual coin-change table dp[a] counts the combinations of coins that
# make amount a. Here you get the table and must find the coins.
# Idea: scan the amounts upward and keep the table built from the coins
# found so far (`built`). At the first amount where the real dp is
# exactly ONE more than what we built, there must be a new coin of that
# value (it adds the single way "just that coin"). Any other difference is
# impossible. At the end the rebuilt table must equal the input (this is
# also follow-up 1: the array might not come from any coins).
# Complexity: Time O(n * k) for k coins found (k <= n, so O(n^2) at worst), Space O(n)
def coins_from_dp(dp):
    if len(dp) == 0 or dp[0] != 1:
        return None                                 # dp[0] must be 1: one way to make 0
    n = len(dp) - 1
    built = [1] + [0] * n
    coins = []
    for a in range(1, n + 1):
        if dp[a] == built[a]:
            continue                                # the coins found so far already explain amount a
```

```
        if dp[a] != built[a] + 1:
            return None                         # needs more than one new way: impossible
        coins.append(a)
        for x in range(a, n + 1):
            built[x] += built[x - a]            # add the new coin to the table
    if built == list(dp):
        return coins
    return None


# ---- main question: tests ----
def test_main_question():
    assert coins_from_dp([1, 1, 2, 2, 3]) == [1, 2]
    assert coins_from_dp([1, 1, 1, 2, 2, 2, 3]) == [1, 3]
    assert coins_from_dp([1]) == []
    assert coins_from_dp([]) is None
    assert coins_from_dp([2, 1]) is None        # not a table: dp[0] must be 1
```

```
    assert coins_from_dp([1, 3]) is None        # an amount that jumps by more than one new way
    assert coins_from_dp([1, 1, 2, 2, 5]) is None  # looks fine early, but the whole table does not match


# =====================================================
# FOLLOW-UP 1: The array might not come from any set of coins
# =====================================================
# Answer: coins_from_dp already returns None when dp[0] is not 1, when
# an amount needs more than one new way, or when the table rebuilt from
# the coins differs from the input: the three checks above.


# =====================================================
# FOLLOW-UP 2: Order of the coins matters (the dp counts permutations, not combinations)
# =====================================================
# ---- follow-up 2: helper functions ----
def ordered_table(coins, n):
    # forward: dp[a] = the sum of dp[a - c] over the coins c (every order counts)
    dp = [1] + [0] * n
    for a in range(1, n + 1):
        for c in coins:
            if c <= a:
                dp[a] += dp[a - c]
    return dp


# ---- follow-up 2: function ----
# Answer: that is a different recurrence, dp[a] = sum of dp[a - c] over the
# coins. Invert it the same way: at each amount, what the known coins
# already explain is the sum of dp[a - c]; if dp[a] is exactly one more,
# a is a new coin. Ask which kind of counting is meant.
# Complexity: Time O(n * k), Space O(n)
def coins_from_ordered(dp):
    if len(dp) == 0 or dp[0] != 1:
        return None
    n = len(dp) - 1
    coins = []
    for a in range(1, n + 1):
        known = 0
        for c in coins:
            known += dp[a - c]
        if dp[a] == known + 1:
            coins.append(a)
        elif dp[a] != known:
            return None
```

```
    if ordered_table(coins, n) == list(dp):
        return coins
    return None


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert coins_from_ordered(ordered_table([1, 2], 6)) == [1, 2]
    assert coins_from_ordered([1, 1, 2, 3, 5]) == [1, 2]


# =========================================================
# FOLLOW-UP 3: Each coin may be used at most once
# =========================================================
# ---- follow-up 3: helper functions ----
def once_table(coins, n):
    # forward, each coin at most once: go through the amounts DOWNWARD
    dp = [1] + [0] * n
    for c in coins:
        for a in range(n, c - 1, -1):
            dp[a] += dp[a - c]
    return dp


# ---- follow-up 3: function ----
# Answer: the forward step visits the amounts downward. The inverse
# still scans upward adding a coin when the table falls short by one, but
# the table update must use the same 0/1 (downward) form.
# Complexity: Time O(n * k), Space O(n)
def coins_from_once(dp):
    if len(dp) == 0 or dp[0] != 1:
        return None
    n = len(dp) - 1
    built = [1] + [0] * n
    coins = []
    for a in range(1, n + 1):
        if dp[a] == built[a]:
            continue
        if dp[a] != built[a] + 1:
            return None
        coins.append(a)
        for x in range(n, a - 1, -1):
            built[x] += built[x - a]
    if built == list(dp):
        return coins
```

```
    return None


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert coins_from_once(once_table([1, 2, 3], 6)) == [1, 2, 3]
    random.seed(1)
    for round_number in range(300):                # random coin sets: all three inverses find them again
        coins = sorted(random.sample(range(1, 10), random.randint(0, 4)))
        n = random.randint(0, 14)
        small = []
        for c in coins:
            if c <= n:
                small.append(c)
        assert coins_from_dp(ways_table(coins, n)) == small
        assert coins_from_ordered(ordered_table(coins, n)) == small
        assert coins_from_once(once_table(coins, n)) == small
        junk = [1]
        for i in range(n):
            junk.append(random.randint(0, 3))
```

```
        found = coins_from_dp(junk)
        if found is not None:
            assert ways_table(found, n) == junk


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 33 OF 43

Prefix sums, Tier 2: running total plus a dict of earlier totals

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Prefix sums, Tier 2: running total plus a dict of earlier totals | 4 questions | tap a title to jump to it

READ FIRST: the code template for these 4 questions (tap here) >

M	163. Solve array-sum and city-community problems	PracHub * 2026-02-20 *
M	164. Maximum-Sum Subarray With Equal Endpoints	ColorStack * 2025-09-29 *
M	165. Maximum Subarray Sum With a Minimum Length Constraint	ColorStack * 2025-08-19 *
M	166. Find the maximum subarray sum.	ND

CODE TEMPLATE | GROUP 33 OF 43 | 4 QUESTIONS, ONE PATTERN

Prefix sums, Tier 2: running total plus a dict of earlier totals

The one idea these questions share

Keep a running total. At each step ask the dict about an earlier total: total - k (subarray sums to k), total % k (sum divisible by k), or the smallest earlier total (best subarray ending here). Store the first index, a count, or the minimum.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def count_sum_k(nums, k):
    seen, total, count = {0: 1}, 0, 0       # CHANGE 1: what seen stores (count, first index, minimum)
    for x in nums:
        total += x
        count += seen.get(total - k, 0)      # CHANGE 2: the earlier total you look for
        seen[total] = seen.get(total, 0) + 1 # CHANGE 3: what you remember about this total
    return count
```

2. Helper functions that usually get written next to it

```
def max_subarray(nums):
    best, smallest, total = float("-inf"), 0, 0
    for x in nums:
        total += x
        best = max(best, total - smallest)   # best subarray that ends here
        smallest = min(smallest, total)      # update after, so the subarray is never empty
    return best
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
163	Solve array-sum and city-community problems	count subarrays summing to k with the template as written; the second task is union-find over building cells
164	Maximum-Sum Subarray With Equal Endpoints	per value keep the earliest start with the smallest total before it; candidate = total now - that earlier total
165	Maximum Subarray Sum With a Minimum Length Constraint	the smallest earlier total is only allowed from min_len positions back, so update it one step late
166	Find the maximum subarray sum.	the dict is just one number: the smallest earlier total; the best subarray ends here with total - smallest (Kadane is the same idea)

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 163 of 197 | topic 33 of 43: Prefix sums, Tier 2: running total plus a dict of earlier totals | 1 of 4

Template fit: count subarrays summing to k with the template as written; the second task is union-find over building cells | * asked 242 days ago *

MEDIUM | PracHub | Technical Screen | 163 of 197

★ TOP 75: rank 13 in the Medium and Hard priority list

★ Solve array-sum and city-community problems

POSTED 2026-02-20 | 242 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 66 | 30 DAYS 46.9 | PRIORITY 7/288

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 2: running total plus a dict of earlier totals

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #560 Subarray Sum Equals K, LeetCode #200 Number of Islands

LeetCode match: LC 560 Subarray Sum Equals K + LC 200 Number of Islands (inferred from overview)

Posted on PracHub: 2026-02-20 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/solve-array-sum-and-city-community-problems

QUESTION

Two tasks. (1) count_subarrays(nums, k): count the contiguous subarrays whose elements add up to k. (2) communities(grid): a grid has 1 for a building and 0 for empty land; buildings that touch up, down, left or right belong to the same community. Return the number of communities.

Function signature

def count_subarrays(nums: list, k: int) -> int
def communities(grid: list) -> int

EXAMPLES

Example 1	
Input	`[1, 1, 1], 2`
Output	`2`
Why	`[1, 1] appears twice (positions 0-1 and 1-2).`

Example 2	
Input	`[1, 2, 3], 3`
Output	`2`
Why	`[1, 2] and [3] both add up to 3.`

Example 3	
Input	`[[1, 1, 0], [0, 0, 1], [1, 0, 1]]`
Output	`3`
Why	`Top-left pair, the single building at the right of the middle row joined to the one below it, and the bottom-left: 3 communities.`

BRUTE FORCE · Task 1: try every subarray

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, K):
    count = 0
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            if total == K:
                count += 1
    return count
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

O(n^2). A dictionary of prefix sums (how many earlier prefixes equal current - K) is O(n) and also works on a stream and with negative numbers.

BRUTE FORCE · Task 2: relabel cells until the labels stop changing

BRUTE FORCE · PSEUDO-CODE

```
label = {}
for i, cell in enumerate(buildings):
    label[cell] = i
changed = True
while changed:
    changed = False
    for cell in buildings:
        group = [cell] + neighbours(cell)
        best = min(label[x] for x in group)
        if best < label[cell]:
            label[cell] = best
            changed = True
answer = len(set(label.values()))
```

TIME	SPACE
O((m * n)^2) worst case	O(m * n)

WHY IT IS SLOW

Labels spread one step per round, so a long snake needs about m * n rounds of O(m * n): O((m * n)^2). Union-Find or one flood fill is near-linear.

OPTIMAL SOLUTION · Task 1: prefix sums in a dictionary

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task 2: Union-Find over building cells

TIME	SPACE
O(m * n * alpha(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The numbers are all positive.

A sliding window works in O(1) memory: grow the right end, shrink the left while the sum exceeds K. With negatives the window is not monotone, which is why prefix sums are needed.

FOLLOW-UP 2 Longest subarray with sum K.

Store the first index of each prefix sum instead of a count, and take the longest distance to prefix - K.

FOLLOW-UP 3 Buildings are added over time and you report the number of communities after each addition.

Union-Find with a counter: +1 for the new building, -1 for each union with a different neighbouring community (LeetCode 305).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Solve array-sum and city-community
# problems
#
# Two tasks. (1) count_subarrays(nums, k): count the
# contiguous subarrays whose elements add up to k. (2)
# communities(grid): a grid has 1 for a building and 0 for
# empty land; buildings that touch up, down, left or right
# belong to the same community. Return the number of
# communities.
#
# Signature:
#   def count_subarrays(nums: list, k: int) -> int
#   def communities(grid: list) -> int
#
# Examples:
# [1, 1, 1], 2  ->  2
# [1, 2, 3], 3  ->  2
#
# FOLLOW-UPS (answered below, in this order):
# 1. The numbers are all positive.
# 2. Longest subarray with sum K.
# 3. Buildings are added over time and you report the
# number of communities after each addition.
#
# INTERVIEW NOTES
# Subarray sums with negatives allowed cannot use a
# sliding window; use prefix sums: a subarray ending here
# sums to K exactly when an earlier prefix equals (current
# prefix - K), so a dictionary of prefix counts answers it
# in one pass and works for streams. Communities are
# connected components, so flood fill or union-find over
# the building cells.

# ====================================================
# HOW THIS FILE IS ORGANISED
```

```
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))
```

```


# ---- main question: main functions ----
# PART 1: count the subarrays whose sum is exactly k.
# Idea: walk along the array with the running (prefix) sum. A subarray
# that ENDS here has sum k when some EARLIER prefix sum equals
# (prefix - k). Keep a dictionary: prefix sum -> how many times it occurred.
# Complexity: Time O(n), Space O(n)
def count_subarrays(nums, k):
    seen = {0: 1}                          # the empty prefix
    prefix = 0
    count = 0
    for x in nums:
        prefix += x
        count += seen.get(prefix - k, 0)
        seen[prefix] = seen.get(prefix, 0) + 1
    return count


# PART 2: count the communities: groups of neighbouring buildings (1) in a grid.
# Idea: scan the grid; an unseen building starts a new community, and a
# flood fill marks all the buildings joined to it.
# Complexity: Time O(m * n), Space O(m * n)
def communities(grid):
    seen = set()
    count = 0
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] != 1 or (r, c) in seen:
                continue
            count += 1
            seen.add((r, c))
            stack = [(r, c)]
            while stack:
                x, y = stack.pop()
                for dx, dy in STEPS:
                    a = x + dx
                    b = y + dy
                    if a < 0 or a >= len(grid) or b < 0 or b >= len(grid[0]):
                        continue
                    if grid[a][b] == 1 and (a, b) not in seen:
                        seen.add((a, b))
                        stack.append((a, b))
    return count
```

```


# ---- main question: tests ----
def test_main_question():
    assert count_subarrays([1, 1, 1], 2) == 2
    assert count_subarrays([1, -1, 0], 0) == 3
    assert count_subarrays([], 0) == 0
    assert communities([[1, 1, 0], [0, 0, 1], [1, 0, 1]]) == 3
```

```python

# ======================================================
# FOLLOW-UP 1: The numbers are all positive
# ======================================================
# ---- follow-up 1: function ----
# Answer: a sliding window works with O(1) memory: move the right end
# forward and move the left end forward while the sum is too big. With
# negative numbers the window is not monotone, which is why prefix sums are
# needed there.
# Complexity: Time O(n), Space O(1) (only for non-negative numbers)
def count_positive(nums, k):
    count = 0
    left = 0
    total = 0
    for right in range(len(nums)):
        total += nums[right]
        while total > k and left <= right:
            total -= nums[left]
            left += 1
        if total == k:
            count += 1
    return count


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert count_positive([1, 2, 3, 2, 1], 3) == 3


# ======================================================
# FOLLOW-UP 2: Longest subarray with sum K
# ======================================================
# ---- follow-up 2: function ----
# Answer: store the FIRST index of every prefix sum instead of a count,
# and take the longest distance back to the prefix (prefix - k).
# Complexity: Time O(n), Space O(n)
def longest_subarray(nums, k):
    first = {0: -1}
    prefix = 0
    best = 0
    for i in range(len(nums)):
        prefix += nums[i]
        if prefix - k in first and i - first[prefix - k] > best:
            best = i - first[prefix - k]
        if prefix not in first:
            first[prefix] = i
    return best


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert longest_subarray([1, -1, 5, -2, 3], 3) == 4
    assert longest_subarray([1, 2], 9) == 0


# ======================================================
# FOLLOW-UP 3: Buildings are added over time and you report the number of communities after each addition
# ======================================================
# ---- follow-up 3: function ----
# Answer: union-find with a counter: +1 for the new building, -1 for every
# union with a different neighbouring community (LeetCode 305).
# Complexity: Time O(K * alpha) for K additions (about O(K)), Space O(K)
def communities_online(rows, cols, additions):
    parent = {}
    count = 0
```

```
    counts = []

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for r, c in additions:
        if (r, c) not in parent:
            parent[(r, c)] = (r, c)
            count += 1
            for dr, dc in STEPS:
                neighbour = (r + dr, c + dc)
                if neighbour in parent:
                    a = find((r, c))
                    b = find(neighbour)
```

```
                    if a != b:
                        parent[a] = b
                        count -= 1                  # two communities became one
        counts.append(count)
    return counts


# ---- follow-up 3: tests ----
def test_follow_up_3():
    additions = [(0, 0), (0, 1), (1, 2), (2, 2), (1, 1)]
    assert communities_online(3, 3, additions) == [1, 1, 2, 2, 1]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(nums: list[int], k: int) -> int:
```

Sample call

```
solution([1, 1, 1], 2)
```

Test cases

Input	Expected
([1, 1, 1], 2)	2
([1, 2, 3], 3)	2

STUDY PATH: question 164 of 197 | topic 33 of 43: Prefix sums, Tier 2: running total plus a dict of earlier totals | 2 of 4

Template fit: per value keep the earliest start with the smallest total before it; candidate = total now - that earlier total | * asked 386 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 164 of 197

Maximum-Sum Subarray With Equal Endpoints

ASKED 2025-09-29 | 386 DAYS AGO | OLDER

STUDY: TYPE **ALL TIME 66** **30 DAYS 46.9** **PRIORITY 8/288**

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 2: running total plus a dict of earlier totals

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #560 Subarray Sum Equals K

LeetCode match: LC 560 Subarray Sum Equals K (related: equal endpoints, maximum sum)

Frequency: Single report | Last asked: 2025-09-29 | Roles: SWE Intern

QUESTION

Given an integer array nums , choose a (contiguous) subarray whose first and last elements are equal and whose sum is as large as possible. A single element is a valid subarray (its first and last elements coincide). Return the chosen subarray itself (as a list). If several subarrays tie for the maximum sum, return the one with the smallest starting index; if they still tie, return the shortest.

Function signature

```
def max_sum_equal_endpoints(nums: list[int]) -> list[int]:
    ...
```

Follow-up (as reported): Achieve O(n): for each value, you only ever care about pairing the earliest occurrence (with the best prefix sum) against later occurrences.

EXAMPLES

Example 1	
Input	`nums = [1, 3, -2, 4, 1]`
Output	`[1, 3, -2, 4, 1]`
Why	`The value 1 appears at indices 0 and 4; that whole window sums to 7, beating any single element.`

Example 2	
Input	`nums = [5, -10, 5]`
Output	`[5]`
Why	`The window from the first 5 to the last 5 sums to 0, which is worse than taking a single 5 (sum 5).`

Example 3	
Input	`nums = [-3, -1, -2]`
Output	`[-1]`
Why	`All values are distinct, so only single elements qualify; -1 is the largest.`

CONSTRAINTS

- 1 <= len(nums) <= 10^5 .
- -10^4 <= nums[i] <= 10^4 .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_equal_ends(nums):
    best = None
    for j in range(len(nums)):
        for i in range(j_+_1):
            if nums[i] == nums[j]:
                key = (
                    -sum(nums[i : j + 1]),
                    i,
                    j - i,
                )
                if best is None or key < best:
                    best = key
    return (
        nums[best[1] : best[1] + best[2] + 1]
        if best
        else:
            []
    )
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Each end index looks back at every earlier index with the same value.

OPTIMAL SOLUTION · Best earlier prefix per value

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return only the best sum.

Drop the tie-breaking bookkeeping: the same sweep with max(prefix[j + 1] - smallest prefix of this value so far).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Maximum-Sum Subarray With Equal Endpoints
#
# Given an integer array nums , choose a (contiguous)
# subarray whose first and last elements are equal and
# whose sum is as large as possible. A single element is a
# valid subarray (its first and last elements coincide).
# Return the chosen subarray itself (as a list). If
# several subarrays tie for the maximum sum, return the
# one with the smallest starting index; if they still tie,
# return the shortest. Function signature: def
# max_sum_equal_endpoints(nums: list[int]) -> list[int]:
# ... Follow-up (as reported): Achieve O(n): for each
# value, you only ever care about pairing the earliest
```

```
# occurrence (with the best prefix sum) against later
# occurrences.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return only the best sum.
#
# INTERVIEW NOTES
# A subarray from i to j has equal endpoints when nums[i]
# == nums[j], and its sum is prefix[j + 1] - prefix[i].
# For a fixed j the best partner i is the occurrence of
# the same value with the smallest prefix[i]; keep that
# per value in a dictionary while sweeping j.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Find the subarray with the biggest sum whose first and last elements are
# equal. Ties: the smaller start, then the shorter one. Return the subarray.
# Idea: with prefix sums, the sum of nums[i..j] is prefix[j+1] - prefix[i].
# For an end j with value x, the best start is an earlier index with the
# SAME value x and the smallest prefix[i] (a start at j itself is allowed:
# a single element). Remember, per value, the best start so far.
# Complexity: Time O(n), Space O(n)
def max_sum_equal_endpoints(nums):
    prefix = [0]
    for x in nums:
        prefix.append(prefix[len(prefix) - 1] + x)
    best_start = {}                                         # value -> the start index with the smallest prefi
x
    best = None                                             # (minus the sum, start, length): smaller is bette
r
    for j in range(len(nums)):
        x = nums[j]
        if x not in best_start or prefix[j] < prefix[best_start[x]]:
            best_start[x] = j
        i = best_start[x]
        key = (-(prefix[j + 1] - prefix[i]), i, j - i)
        if best is None or key < best:
            best = key
    if best is None:
        return []
    return nums[best[1]:best[1] + best[2] + 1]


# ---- main question: tests ----
def test_main_question():
    assert max_sum_equal_endpoints([1, 3, -2, 4, 1]) == [1, 3, -2, 4, 1]
    assert max_sum_equal_endpoints([5, -10, 5]) == [5]
    assert max_sum_equal_endpoints([-3, -1, -2]) == [-1]
    assert max_sum_equal_endpoints([]) == []


# ========================================================
# FOLLOW-UP 1: Return only the best sum
# ========================================================
# ---- follow-up 1: function ----
```

```
# Answer: drop the tie-break bookkeeping: the same sweep, remembering for
# every value the smallest prefix sum seen before an element of that value.
# Complexity: Time O(n), Space O(n)
def max_equal_ends_sum(nums):
    smallest = {}                                       # value -> the smallest prefix sum before an eleme
nt of that value
    best = None
    total = 0
    for x in nums:
        if x not in smallest or total < smallest[x]:
            smallest[x] = total
        total += x
        candidate = total - smallest[x]
        if best is None or candidate > best:
            best = candidate
    return best
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert max_equal_ends_sum([1, 3, -2, 4, 1]) == 7
    assert max_equal_ends_sum([]) is None
    for nums in ([5, -10, 5], [-3, -1, -2], [2, 2, -1, 2]):
        found = max_sum_equal_endpoints(nums)
        total = 0
        for x in found:
            total += x
        assert max_equal_ends_sum(nums) == total


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 165 of 197 | topic 33 of 43: Prefix sums, Tier 2: running total plus a dict of earlier totals | 3 of 4

Template fit: the smallest earlier total is only allowed from min_len positions back, so update it one step late | * asked 427 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 165 of 197

Maximum Subarray Sum With a Minimum Length Constraint

ASKED 2025-08-19 | 427 DAYS AGO | OLDER

STUDY: TYPE **ALL TIME 64.4** **30 DAYS 41.1** **PRIORITY 18/288**

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 2: running total plus a dict of earlier totals

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #53 Maximum Subarray

LeetCode match: LC 53 Maximum Subarray (related: minimum length)

Frequency: Single report | Last asked: 2025-08-19 | Roles: SWE Intern

QUESTION

This is the classic "maximum subarray" problem with the extra constraint the report mentioned ("Max sub array question with a weird constraint"). Given an integer array nums and an integer min_len , find the contiguous subarray (containing at least min_len elements) that has the largest sum, and return that sum.

Function signature

```
def max_subarray_min_len(nums: list[int], min_len: int) -> int:
    """Return the maximum sum over all contiguous subarrays of length >= min_len."""
    ...
```

Follow-up (as reported): The report said each question had a follow-up where a constraint was changed. Now bound the subarray length to the range [min_len, max_len] . Hint: maintain a sliding window of prefix-sum minima with a monotonic deque.

EXAMPLES

Example 1	
Input	`nums = [1, -2, 3, 4, -1], min_len = 2`
Output	`7`
Why	`The subarray [3, 4] (length 2) has sum 7, the largest among all subarrays of length >= 2.`

Example 2	
Input	`nums = [-1, -2, -3], min_len = 2`
Output	`-3`
Why	`Every element is negative. The best length->=2 subarray is [-1, -2] with sum -3.`

Example 3	
Input	`nums = [5, 4, -1, 7, 8], min_len = 1`
Output	`23`
Why	`With min_len = 1 this reduces to ordinary Kadane; the whole array sums to 23.`

CONSTRAINTS

- 1 <= min_len <= len(nums) <= 10^5
- -10^4 <= nums[i] <= 10^4

QUESTION

- The array is guaranteed to be non-empty and min_len never exceeds len(nums) , so a valid subarray always exists.
- Values may be negative; the answer can therefore be negative.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_min_len(nums, min_len):
    best = None
    for start in range(len(nums)):
        total = 0
        for end in range(start,_len(nums)):
            total += nums[end]
            if end - start + 1 >= min_len:
                best = (
                    total
                    if best is None
                    else:
                        max(best, total)
                )
    return best
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Every start index extends to every end index at least min_len long, O(n^2) sums.

OPTIMAL SOLUTION · Prefix sums and a running minimum

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Now the length must also be at most max_len. How does it change?

The allowed start prefixes now form a sliding window of indexes [end - max_len, end - min_len]. Keep the prefix indexes in a deque ordered by increasing prefix value; pop from the back while the new value is smaller, pop from the front when an index leaves the window. The minimum is the front. O(n).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Maximum Subarray Sum With a Minimum
# Length Constraint
#
# This is the classic "maximum subarray" problem with the
# extra constraint the report mentioned ("Max sub array
# question with a weird constraint"). Given an integer
# array nums and an integer min_len , find the contiguous
# subarray (containing at least min_len elements) that has
# the largest sum, and return that sum. Function
```

```
# signature: def max_subarray_min_len(nums: list[int],
# min_len: int) -> int: """Return the maximum sum over all
# contiguous subarrays of length >= min_len.""" ...
# Follow-up (as reported): The report said each question
# had a follow-up where a constraint was changed. Now
# bound the subarray length to the range [min_len,
# max_len] . Hint: maintain a sliding window of prefix-sum
# minima with a monotonic deque.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Now the length must also be at most max_len. How does
# it change?
#
# INTERVIEW NOTES
# Kadane's algorithm cannot enforce a minimum length, so
# switch to prefix sums: the best subarray ending at i
# with length at least min_len is prefix[i + 1] minus the
# smallest prefix[j] with j <= i + 1 - min_len. Keep that
# minimum as a running value while sweeping.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def prefix_sums(nums):
    # prefix[j] = the sum of the first j numbers
    prefix = [0]
    for x in nums:
        prefix.append(prefix[len(prefix) - 1] + x)
    return prefix


# ---- main question: main function ----
# Idea: the sum of nums[i..end-1] is prefix[end] - prefix[i]. For each end,
# the best start is the SMALLEST prefix sum among the starts that leave at
# least min_len elements (i <= end - min_len). Sweep the end forward and
# keep that running minimum.
# Complexity: Time O(n), Space O(n)
def max_subarray_min_len(nums, min_len):
    prefix = prefix_sums(nums)
    best = None
    smallest = None
    for end in range(min_len, len(nums) + 1):
        allowed_start = prefix[end - min_len]            # the newest start that is allowed now
        if smallest is None or allowed_start < smallest:
            smallest = allowed_start
        candidate = prefix[end] - smallest
        if best is None or candidate > best:
            best = candidate
    return best


# ---- main question: tests ----
def test_main_question():
```

```
    assert max_subarray_min_len([1, -2, 3, 4, -1], 2) == 7
    assert max_subarray_min_len([-1, -2, -3], 2) == -3
    assert max_subarray_min_len([5, 4, -1, 7, 8], 1) == 23


# ======================================================
# FOLLOW-UP 1: Now the length must also be at most max_len. How does it change?
# ======================================================
# ---- follow-up 1: function ----
# Answer: the allowed start positions now form a sliding window
# [end - max_len, end - min_len]. Keep the window in a deque of start
# positions whose prefix sums increase from front to back, so the front is
# always the smallest: drop the back entries that are not smaller than the
# newcomer, and drop the front entry when it falls out of the window.
# Complexity: Time O(n) (each start enters and leaves the deque once), Space O(n)
def max_subarray_len_range(nums, min_len, max_len):
    prefix = prefix_sums(nums)
```

```
    window = deque()
    best = None
    for end in range(min_len, len(nums) + 1):
        new_start = end - min_len                       # the start that just became allowed
        while window and prefix[window[len(window) - 1]] >= prefix[new_start]:
            window.pop()
        window.append(new_start)
        while window[0] < end - max_len:                # this start would make the subarray too long
            window.popleft()
        candidate = prefix[end] - prefix[window[0]]
        if best is None or candidate > best:
            best = candidate
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert max_subarray_len_range([1, -2, 3, 4, -1], 2, 3) == 7
    for code in range(5 ** 6):                          # every array of 6 numbers from -2..2, every length
 range
        nums = []
        for i in range(6):
            nums.append(code // 5 ** i % 5 - 2)
        for low in range(1, 7):
            for high in range(low, 7):
                slow = None
                for i in range(6):
                    for j in range(i + low, min(6, i + high) + 1):
                        total = 0
                        for x in nums[i:j]:
                            total += x
                        if slow is None or total > slow:
                            slow = total
                assert max_subarray_len_range(nums, low, high) == slow


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

STUDY PATH: question 166 of 197 | topic 33 of 43: Prefix sums, Tier 2: running total plus a dict of earlier totals | 4 of 4

Template fit: the dict is just one number: the smallest earlier total; the best subarray ends here with total - smallest (Kadane is the same idea) | ND

MEDIUM | IN APP LC 53 | Exponent | 166 of 197

★ TOP 75: rank 20 in the Medium and Hard priority list

★ Find the maximum subarray sum.

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TYPE **ALL TIME 64.4** **30 DAYS 41.1** **PRIORITY 17/288**

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 2: running total plus a dict of earlier totals

Optimal answer from: your app's interview approach (LeetCode #53)

LeetCode match: LC 53 Maximum Subarray

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +1522 interviews

Original: https://www.aced.io/questions/973/maximum-subarray-sum

QUESTION

Given an array of integers nums, write a function maxSubarraySum to find the maximum sum of a contiguous subarray within the array and return that maximum sum. The subarray must be contiguous, meaning that the elements must appear consecutively in the original array. Examples Input: nums = [2, 3, -2, 4]
Output: 7 Explanation: Maximum sum is 2 + 3 + (-2) + 4 = 7.
Input: nums = [1, -1, -5, -4]
Output: 1 Explanation: The maximum sum is 1, which is the single element with the highest value.Challenge 1Can you come up with a solution with a O(n) time complexity?Challenge 2Can you come up with a solution with a O(1) space complexity?

EXAMPLES

Example 1	
Input	`[-2, 1, -3, 4, -1, 2, 1, -5, 4]`
Output	`6`
Why	`[4, -1, 2, 1] has the largest sum, 6.`

Example 2	
Input	`[1]`
Output	`1`
Why	`A single element.`

Example 3	
Input	`[5, 4, -1, 7, 8]`
Output	`23`
Why	`All numbers are positive: take them all.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums):
    best = -INF
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            best = max(best, total)
    return best
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

O(n^2) subarrays. Kadane's algorithm keeps the best sum ending at each position (extend or restart) in O(n) time and O(1) space.

OPTIMAL SOLUTION · Kadane's algorithm

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the subarray itself.

Remember where the current run started: when starting fresh at i set start = i, and when cur beats best record (start, i). Same single pass.

FOLLOW-UP 2 The array is circular (LeetCode 918).

The best circular subarray is either an ordinary one or the whole array minus the worst ordinary one (the minimum subarray): max(kadane max, total - kadane min), taking care of the all-negative case.

FOLLOW-UP 3 Divide and conquer instead of Kadane.

Split the array; the best subarray is entirely left, entirely right, or crosses the middle (best suffix of the left plus best prefix of the right). T(n) = 2T(n/2) + O(n) = O(n log n); useful when the data is split across machines.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find the maximum subarray sum.
#
# Given an array of integers nums, write a function
# maxSubarraySum to find the maximum sum of a contiguous
# subarray within the array and return that maximum sum.
# The subarray must be contiguous, meaning that the
# elements must appear consecutively in the original
# array. Examples Input: nums = [2, 3, -2, 4] Output: 7
```

```
# Explanation: Maximum sum is 2 + 3 + (-2) + 4 = 7. Input:
# nums = [1, -1, -5, -4] Output: 1 Explanation: The
# maximum sum is 1, which is the single element with the
# highest value.Challenge 1Can you come up with a solution
# with a O(n) time complexity?Challenge 2Can you come up
# with a solution with a O(1) space complexity?
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subarray itself.
# 2. The array is circular (LeetCode 918).
# 3. Divide and conquer instead of Kadane.
#
# INTERVIEW NOTES
# The best subarray ending at position i either extends
# the best one ending at i - 1 or starts fresh at i; it
# extends only if the previous best is positive. So keep
# one number, the best sum ending here, and one answer:
# cur = max(x, cur + x), best = max(best, cur).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def bigger(x, y):
    if x > y:
        return x
    return y


# ---- main question: main function ----
```

```
# Idea (Kadane): `current` is the best sum of a subarray that ENDS at the
# current element: either the element alone, or the element added to the
# previous `current`. `best` is the biggest `current` seen.
# Complexity: Time O(n), Space O(1)
def max_subarray(nums):
    current = nums[0]
    best = nums[0]
    for i in range(1, len(nums)):
        current = bigger(nums[i], current + nums[i])
        best = bigger(best, current)
    return best


# ---- main question: tests ----
MIXED = [-2, 1, -3, 4, -1, 2, 1, -5, 4]


def test_main_question():
    assert max_subarray([2, 3, -2, 4]) == 7
    assert max_subarray([-3, -1, -2]) == -1
    assert max_subarray([5]) == 5
    assert max_subarray(MIXED) == 6


# ============================================================
# FOLLOW-UP 1: Return the subarray itself
# ============================================================
```

```python
# ---- follow-up 1: function ----
# Answer: remember where the current run started: when we start fresh at
# i, set start = i; when `current` beats `best`, record (start, i). Same
# single pass.
# Complexity: Time O(n), Space O(1)
def max_subarray_range(nums):
    current = nums[0]
    start = 0
    best = nums[0]
    span = (0, 0)
    for i in range(1, len(nums)):
        if nums[i] > current + nums[i]:
            current = nums[i]
            start = i
        else:
            current += nums[i]
        if current > best:
            best = current
            span = (start, i)
    return best, span


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert max_subarray_range(MIXED) == (6, (3, 6))


# ========================================================
# FOLLOW-UP 2: The array is circular (LeetCode 918)
# ========================================================
# ---- follow-up 2: function ----
# Answer: the best circular subarray either does not wrap around (an
# ordinary one) or wraps, and then it is the WHOLE array minus the worst
# ordinary subarray (the minimum one). Take the bigger of the two, but if
# every number is negative the "wrap" would be empty, so use the ordinary one.
# Complexity: Time O(n), Space O(n)
def max_circular(nums):
    best = max_subarray(nums)
    total = 0
    negated = []
    for x in nums:
        total += x
        negated.append(-x)
    worst = -max_subarray(negated)                # the smallest subarray sum
    if total == worst:
        return best
    return bigger(best, total - worst)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert max_circular([5, -3, 5]) == 10
    assert max_circular([-3, -2, -3]) == -2
    assert max_circular([1, -2, 3, -2]) == 3


# ========================================================
# FOLLOW-UP 3: Divide and conquer instead of Kadane
# ========================================================
# ---- follow-up 3: function ----
# Answer: split the array in the middle. The best subarray lies entirely
# in the left half, entirely in the right half, or CROSSES the middle: the
# best sum ending at the middle on the left plus the best sum starting at
```

```
# the middle on the right. T(n) = 2 T(n/2) + O(n) = O(n log n); useful when
# the data is spread over several machines.
# Complexity: Time O(n log n), Space O(log n) for the recursion
def max_subarray_dc(nums, low=0, high=None):
    if high is None:
        high = len(nums)
    if high - low == 1:
        return nums[low]
    middle = (low + high) // 2
    left_best = None
    running = 0
    for i in range(middle - 1, low - 1, -1):
        running += nums[i]
        if left_best is None or running > left_best:
            left_best = running
    right_best = None
    running = 0
    for i in range(middle, high):
        running += nums[i]
        if right_best is None or running > right_best:
            right_best = running
    return bigger(bigger(max_subarray_dc(nums, low, middle), max_subarray_dc(nums, middle, high)),
                  left_best + right_best)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    for arr in ([2, 3, -2, 4], [-1, -2], [1, -1, 1, -1, 5], MIXED):
        assert max_subarray_dc(arr) == max_subarray(arr)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given an array of integers nums, write a function maxSubarraySum to find the maximum sum of a contiguous subarray within the array and return that maximum sum. The subarray must be contiguous, meaning that the elements must appear consecutively in the original array. Examples

```
Input: nums = [2, 3, -2, 4]
Output: 7
Explanation: Maximum sum is 2 + 3 + (-2) + 4 = 7.

Input: nums = [1, -1, -5, -4]
Output: 1
Explanation: The maximum sum is 1, which is the single element with the highest value.
```

OFFICIAL PAGE TEXT

Challenge 1Can you come up with a solution with a O(n) time complexity?Challenge 2Can you come up with a solution with a O(1) space complexity?Our solution for finding the maximum subarray sum processes the input array by iterating through it while maintaining two variables: max_current and max_global. The max_current variable keeps track of the maximum sum of the subarray ending at the current position, while max_global stores the maximum sum encountered so far. For each element in the array, we update max_current by taking the maximum of the current element alone or the sum of the current element and max_current. This decision ensures that max_current always holds the maximum sum of the subarray that ends at the current element. We then update max_global to be the maximum of max_global and max_current. If the input array is empty, the function returns 0 since there are no subarrays to consider. This approach uses a simple iteration and a couple of variables, making it both straightforward and efficient. Time Complexity: The solution has a time complexity of O(n), where n is the number of elements in the array. This is because we iterate through the array once, performing constant-time operations (comparison and addition) for each element. Space Complexity: The solution uses O(1) space since it only requires a fixed amount of extra space for the max_current and max_global variables, regardless of the input size. This makes the solution very efficient in terms of space usage. NoteThis algorithm is known as Kadane's Algorithm.No answers contributed by the community yet.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 34 OF 43

Group by a canonical key

3 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Group by a canonical key | 3 questions | tap a title to jump to it

READ FIRST: the code template for these 3 questions (tap here) >

M 167. Check if all substrings are anagrams of words **PracHub * 2026-03-11 ***

M 168. Group anagrams **ND**

M 169. Given an array of strings, return groups of "buddy strings," where the strings have the same length and the dist... **ND**

CODE TEMPLATE | GROUP 34 OF 43 | 3 QUESTIONS, ONE PATTERN

Group by a canonical key

The one idea these questions share

Find a key that is the same for everything that should be grouped (sorted letters, letter counts, shifts between letters, the keyboard row) and put the items into a dict under that key. Equal key means same group.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import defaultdict


def group_by_key(words, key):
    groups = defaultdict(list)
    for w in words:
        groups[key(w)].append(w)              # CHANGE: the key function decides everything
    return list(groups.values())
```

2. Helper functions that usually get written next to it

```
from collections import Counter


def letter_counts(word):                    # a hashable signature that ignores order
    return tuple(sorted(Counter(word).items()))


def shift_signature(word):
    return tuple((ord(word[i]) - ord(word[i - 1])) % 26 for i in range(1, len(word)))
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
167	Check if all substrings are anagrams of words	key = the tuple of letter counts; keep the keys of the allowed words in a set and test every substring of length 3 or more against it
168	Group anagrams	key = the sorted letters (or a tuple of 26 letter counts)
169	Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same.	key = the differences between neighbouring letters, modulo 26 (a shifted word has the same differences)

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 167 of 197 | topic 34 of 43: Group by a canonical key | 1 of 3

Template fit: key = the tuple of letter counts; keep the keys of the allowed words in a set and test every substring of length 3 or more against it | * asked 223 days ago *

MEDIUM | PracHub | Technical Screen | 167 of 197

★ TOP 75: rank 22 in the Medium and Hard priority list

★ Check if all substrings are anagrams of words

POSTED 2026-03-11 | 223 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 57.7 | 30 DAYS 51.4 | PRIORITY 20/288

ABOUT THIS QUESTION

Topic: Group by a canonical key

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #49 Group Anagrams

LeetCode match: No direct match; related LC 49 Group Anagrams

Posted on PracHub: 2026-03-11 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/check-if-all-substrings-are-anagrams-of-words

QUESTION

You are given a string **`s`** (letters only) and access to an English dictionary **`dict`** (a set of valid words). Return **`true`** if **every contiguous substring** of **`s`** with length **`>= 3`** can be **rearranged (anagrammed)** into a valid dictionary word. Formally, for every substring **`t = s[i..j]`** with **`|t| >= 3`**, there exists some word **`w in dict`** such that **`w`** is a permutation of **`t`** (same multiset of characters). Otherwise return **`false`**. Clarifications:

- Substring means **contiguous**. - “Can be rearranged into a word” means the substring and the dictionary word have exactly the same character counts. Example:
- If **`t = "act"`**, it passes if the dictionary contains any of **`{ "act", "cat", "tac" }`**.

EXAMPLES

Example 1	
Input	`'abca', ['abc', 'cab', 'bca', 'abca']`
Output	`true`
Why	`Every substring of length 3 or more of 'abca' is an anagram of a dictionary word.`

Example 2	
Input	`'abca', ['abc']`
Output	`false`
Why	`'bca' is an anagram of 'abc' but 'abca' has no match.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(s, words):
    for i in range(len(s)):
        for j in range(i + 3, len(s) + 1):
            key = sorted(s[i:j])
            match = [w for w in words
                     if key == sorted(w)]
            if not match:
                return False
    return True
```

TIME	SPACE
O(n^2 * W * L log L)	**O(1)** extra

WHY IT IS SLOW

Each substring is compared with every word and sorted again: O(n^2 * W * L log L). A set of word signatures, with the substring's count vector updated incrementally, makes each check O(alphabet).

OPTIMAL SOLUTION · Set of signatures, incremental count vectors

TIME	SPACE
O(n^2 * A) for alphabet size A	**O(dictionary size)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A bigger alphabet or Unicode.

Replace the fixed array by a Counter and use a frozen tuple of sorted (letter, count) pairs, or a hash of the counts, as the signature. **Cost:** O(n^2 * distinct letters in a substring) with a Counter signature instead of a fixed array.

FOLLOW-UP 2 Only substrings of lengths that exist in the dictionary need checking.

Record the set of word lengths; a substring whose length is not in that set can never match, so you can return False the first time such a length is reached (assuming all substrings of length >= 3 up to n must match).

FOLLOW-UP 3 Subsets of the letters (any sub-multiset of a word) instead of a full permutation.

Then the test is containment of count vectors, which is a subset-of-multiset query; use a sorted signature trie or precompute all sub-multisets for short words.

FOLLOW-UP 4 Report the first failing substring instead of True or False.

Test substrings by length (shortest first) so the report is a smallest counterexample; the signature set makes each test one lookup after sorting the substring's letters.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Check if all substrings are anagrams of
# words
#
# You are given a string s (letters only) and access to an
# English dictionary dict (a set of valid words). Return
# true if **every contiguous substring** of s with length
# **>= 3** can be **rearranged (anagrammed)** into a valid
# dictionary word. Formally, for every substring t =
# s[i..j] with |t| >= 3, there exists some word w in dict
# such that w is a permutation of t (same multiset of
# characters). Otherwise return false. Clarifications: -
# Substring means **contiguous**. - “Can be rearranged
# into a word” means the substring and the dictionary word
# have exactly the same character counts. Example: - If t
# = "act", it passes if the dictionary contains any of {
# "act", "cat", "tac" }.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A bigger alphabet or Unicode.
# 2. Only substrings of lengths that exist in the
# dictionary need checking.
# 3. Subsets of the letters (any sub-multiset of a word)
# instead of a full permutation.
# 4. Report the first failing substring instead of True or
# False.
#
# INTERVIEW NOTES
# Anagrams share a signature: the multiset of letters (a
# sorted string or a 26-count tuple). So turn the
# dictionary into a set of signatures once, and test each
# substring by its signature. Extending a substring by one
# letter changes its count vector in O(1), so you do not
# rebuild the signature from scratch.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
```

```
ALPHABET = "abcdefghijklmnopqrstuvwxyz"


def count_vector(text):
    # the number of times each letter a..z occurs, as a tuple (the "signature")
    counts = [0] * len(ALPHABET)
    for ch in text:
        counts[ord(ch) - ord("a")] += 1
    return tuple(counts)


# ---- main question: main function ----
# Is EVERY substring of s (of length at least min_len) an anagram of some
# dictionary word?
# Idea: precompute the set of letter-count signatures of the dictionary
# words. For each start index grow the substring one letter at a time and
# update its count vector in O(1); once the length is at least min_len, look
# its signature up in the set.
# Complexity: Time O(n^2 * A) for a string of length n (the 26 counts are compared for each substring), Spa
ce O(words * A)
```

```
def all_anagram_words(s, words, min_len=3):
    signatures = set()
    for word in words:
        signatures.add(count_vector(word))
    for i in range(len(s)):
        counts = [0] * len(ALPHABET)
        for j in range(i, len(s)):
            counts[ord(s[j]) - ord("a")] += 1
            if j - i + 1 >= min_len and tuple(counts) not in signatures:
                return False
    return True


# ---- main question: tests ----
WORDS = {"cat", "act", "tac", "tact", "cats"}


def test_main_question():
    assert all_anagram_words("cat", WORDS)
    assert all_anagram_words("tca", WORDS)
    assert not all_anagram_words("tcaz", WORDS)
    assert all_anagram_words("ab", set())                    # no substring is long enough to need a word


# ========================================================
# FOLLOW-UP 1: A bigger alphabet or Unicode
```

```
# ========================================================
# ---- follow-up 1: function ----
# Answer: replace the fixed 26-slot vector by a dictionary of counts and use
# a sorted tuple of (letter, count) pairs as the signature (or a hash of it).
# Complexity: Time O(A log A), Space O(A)
def signature_of(counts):
    pairs = []
    for letter in counts:
        if counts[letter] > 0:
            pairs.append((letter, counts[letter]))
    pairs.sort()
    return tuple(pairs)


# Complexity: Time O(n^2 * A log A), Space O(words * A)
def all_anagram_words_any_alphabet(s, words, min_len=3):
    signatures = set()
    for word in words:
        counts = {}
        for ch in word:
            counts[ch] = counts.get(ch, 0) + 1
        signatures.add(signature_of(counts))
    for i in range(len(s)):
        counts = {}
        for j in range(i, len(s)):
            counts[s[j]] = counts.get(s[j], 0) + 1
            if j - i + 1 >= min_len and signature_of(counts) not in signatures:
                return False
    return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert all_anagram_words_any_alphabet("cat", WORDS)
    assert not all_anagram_words_any_alphabet("tcaz", WORDS)


# ========================================================
```

```
# FOLLOW-UP 2: Only substrings of lengths that exist in the dictionary need checking
# ========================================================
# Answer: if no dictionary word has length L, the first substring of length
# L fails at once; so only the lengths that occur among the words (and are at
# least min_len) can ever succeed. first_failing below uses this.
```

```
# ========================================================
# FOLLOW-UP 3: Subsets of the letters (any sub-multiset of a word) instead of a full permutation
# ========================================================
# Answer: precompute, for every word, the signatures of ALL its sub-multisets
# (2^length of them for short words) and put them all in the set.


# ========================================================
# FOLLOW-UP 4: Report the first failing substring instead of True or False
# ========================================================
# ---- follow-up 4: helper functions ----
def sorted_letters(text):
    # the letters in sorted order: equal for anagrams
    return "".join(sorted(text))


# ---- follow-up 4: function ----
# Answer: test the substrings by LENGTH, shortest first, so the report is a
# smallest counterexample. A set of sorted-letter signatures makes each test one
# lookup after sorting the substring's letters.
# Complexity: Time O(n^2 * L log L) in the worst case, Space O(words * L)
def first_failing(s, words, min_len=3):
    signatures = set()
    lengths = set()
    for word in words:
        signatures.add(sorted_letters(word))
        lengths.add(len(word))
    for length in range(min_len, len(s) + 1):
        if length not in lengths:
            return s[:length]                                   # no word has this length: the first window
fails
        for i in range(len(s) - length + 1):
            piece = s[i:i + length]
            if sorted_letters(piece) not in signatures:
                return piece
    return None


# ---- follow-up 4: tests ----
def test_follow_up_4():
    assert first_failing("cat", WORDS) is None
    assert first_failing("catz", WORDS) == "atz"                # "cat" is fine; "atz" is the first substrin
g that is not
    assert first_failing("aaaa", {"aaa"}) == "aaaa"


# ========================================================
```

```
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(s: str, words: list[str]) -> bool:
```

Sample call

```
solution("act", ["cat", "dog"])
```

Test cases

Input	Expected
("act", ["cat", "dog"])	True
("abca", ["cab", "abca", "zzz"])	True

STUDY PATH: question 168 of 197 | topic 34 of 43: Group by a canonical key | 2 of 3

Template fit: key = the sorted letters (or a tuple of 26 letter counts) | ND

MEDIUM | IN APP LC 49 | Exponent | 168 of 197

★ TOP 75: rank 23 in the Medium and Hard priority list

★ Group anagrams

ASKED (ANY CO.) ~2 MONTHS AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 57.7 30 DAYS 51.4 PRIORITY 19/288

ABOUT THIS QUESTION

Topic: Group by a canonical key

Optimal answer from: your app's interview approach (LeetCode #49)

LeetCode match: LC 49 Group Anagrams (inferred from short statement)

Last asked (any company): 2 months ago, about 2026-08 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/4676/group-anagrams

QUESTION

Given a list of words, group together the words that are anagrams of each other (the same letters in a different order). Return the groups in any order.

Function signature

def group_by_sorting(words: list) -> list

EXAMPLES

Example 1	
Input	`['eat', 'tea', 'tan', 'ate', 'nat', 'bat']`
Output	`[['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]`
Why	`eat, tea and ate use the same letters; tan and nat; bat is alone.`

Example 2	
Input	`['']`
Output	`[['']]`
Why	`An empty string is its own group.`

Example 3	
Input	`['a']`
Output	`[['a']]`
Why	`One word.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(words):
    groups = []
    for word in words:
        key = sorted(word)
        for group in groups:
            if sorted(group[0]) == key:
                group.append(word)
                break
        else:
            groups.append([word])
    return groups
```

TIME	SPACE
O(n * g * L log L)	O(n)

WHY IT IS SLOW

Each word is compared with many groups, O(n * groups * L log L). Using the sorted letters (or a letter-count tuple) as a dictionary key groups all words in one pass.

OPTIMAL SOLUTION · Dictionary keyed by the sorted letters

TIME	SPACE
O(n * L log L)	O(n * L)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Make the key O(L) instead of O(L log L).

Use a tuple of 26 letter counts as the key (no sort). Total O(n * L); for large alphabets use a Counter turned into a frozenset of items or a sorted tuple of (char, count).

FOLLOW-UP 2 Case-insensitive, Unicode, or ignore spaces and punctuation.

Normalise first (casefold, strip non-letters, Unicode normalisation) and then compute the key. Say what counts as a letter.

FOLLOW-UP 3 The words are in many files.

Compute the key on each machine and hash-partition by key so every anagram group lands on one worker, which then groups locally (a MapReduce shuffle).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Group anagrams
#
# Given a list of words, group together the words that are
# anagrams of each other (the same letters in a different
# order). Return the groups in any order.
#
# Signature:
#   def group_by_sorting(words: list) -> list
#
```

```
# Examples:
# ['eat', 'tea', 'tan', 'ate', 'nat', 'bat']  ->  [['eat',
#     'tea', 'ate'], ['tan', 'nat'], ['bat']]
# ['']  ->  [['']]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Make the key O(L) instead of O(L log L).
# 2. Case-insensitive, Unicode, or ignore spaces and
# punctuation.
# 3. The words are in many files.
#
# INTERVIEW NOTES
# Anagrams are strings with the same letters in any order,
# so they have the same canonical form: the sorted
# letters, or the count of each letter. Use that as a
# dictionary key and group in one pass; you never compare
# two strings directly.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import unicodedata


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: anagrams have the SAME letters in sorted order, so sort the letters
# of each word and use that string as the key of a dictionary; words with the
# same key are anagrams. One pass, one dictionary (groups come out in the
# order their first word appears).
```

```
# Complexity: Time O(n * L log L) for n words of length L, Space O(n * L)
def group_by_sorting(words):
    groups = {}
    for word in words:
        key = "".join(sorted(word))
        if key not in groups:
            groups[key] = []
        groups[key].append(word)
    return list(groups.values())


# ---- main question: tests ----
WORDS = ["eat", "tea", "tan", "ate", "nat", "bat"]
GROUPED = [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]


def test_main_question():
    assert group_by_sorting(WORDS) == GROUPED
    assert group_by_sorting([]) == []
    assert group_by_sorting([""]) == [[""]]


# ============================================================
# FOLLOW-UP 1: Make the key O(L) instead of O(L log L)
# ============================================================
# ---- follow-up 1: function ----
# Answer: use a tuple of 26 letter COUNTS as the key instead of sorting
# the letters: O(L) per word, O(n * L) in total. For a big alphabet use the
```

```python
# counts of the letters that occur (a sorted tuple of (letter, count) pairs).
# Complexity: Time O(n * L), Space O(n * L)
def group_by_counts(words):
    groups = {}
    for word in words:
        counts = [0] * 26
        for ch in word:
            counts[ord(ch) - ord("a")] += 1
        key = tuple(counts)
        if key not in groups:
            groups[key] = []
        groups[key].append(word)
    return list(groups.values())


# ---- follow-up 1: tests ----
def test_follow_up_1():
```

```python
    assert group_by_counts(WORDS) == GROUPED


# ============================================================
# FOLLOW-UP 2: Case-insensitive, Unicode, or ignore spaces and punctuation
# ============================================================
# ---- follow-up 2: function ----
# Answer: normalise first (lowercase with casefold, remove accents with Unicode
# normalisation, drop everything that is not a letter) and then compute the key.
# Say what counts as a letter.
# Complexity: Time O(L), Space O(L)
def normalise(word):
    text = unicodedata.normalize("NFKD", word).casefold()
    letters = ""
    for ch in text:
        if ch.isalpha():
            letters += ch
    return letters


# Complexity: Time O(n * L log L), Space O(n * L)
def group_normalised(words):
    groups = {}
    for word in words:
        key = "".join(sorted(normalise(word)))
        if key not in groups:
            groups[key] = []
        groups[key].append(word)
    return list(groups.values())


# ---- follow-up 2: tests ----
def test_follow_up_2():
    mixed = ["Listen", "Silent", "enlist", "a gentleman"]
    assert group_normalised(mixed)[0] == ["Listen", "Silent", "enlist"]
    assert normalise("Caf\u00e9!") == "cafe"


# ============================================================
# FOLLOW-UP 3: The words are in many files
# ============================================================
# ---- follow-up 3: function ----
# Answer: compute the key on each machine and send every word to the machine
# given by a HASH of its key, so a whole anagram group lands on one worker, which
# then groups locally (a MapReduce shuffle).
```

```python
# Complexity: Time O(L log L), Space O(L)
def shard_of(word, workers):
    key = "".join(sorted(word))
    total = 0
    for ch in key:
        total += ord(ch)
    return total % workers


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert shard_of("eat", 8) == shard_of("tea", 8)           # anagrams always go to the same worker


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given an array of strings, group together the words that are anagrams (e.g. gas, sag) of each other. The order of the groups or the words within the groups does not matter in the final result.

STUDY PATH: question 169 of 197 | topic 34 of 43: Group by a canonical key | 3 of 3

Template fit: key = the differences between neighbouring letters, modulo 26 (a shifted word has the same differences) | ND

MEDIUM | IN APP LC 249 | Exponent | 169 of 197

Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same.

ASKED (ANY CO.) ~8 MONTHS AGO | OLDER

STUDY: TALK | ALL TIME 35.4 | 30 DAYS 0 | PRIORITY 126/288

ABOUT THIS QUESTION

Topic: Group by a canonical key

Optimal answer from: your app's interview approach (LeetCode #249)

LeetCode match: LC 249 Group Shifted Strings (inferred from short statement)

Last asked (any company): 8 months ago, about 2026-02 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/6356/group-buddy-strings-same-distance

QUESTION

Two strings are buddy strings when they have the same length and the distance between each pair of neighbouring letters is the same (the alphabet wraps around after z), so one is the other shifted by a constant. Group the strings into buddy groups and return the groups in any order.

Function signature

def group_buddies(words: list) -> list

EXAMPLES

Example 1	
Input	`['abc', 'bcd', 'acf', 'xyz', 'az', 'ba', 'a', 'z']`
Output	`[['abc', 'bcd', 'xyz'], ['acf'], ['az', 'ba'], ['a', 'z']]`
Why	`'abc', 'bcd' and 'xyz' all step +1, +1; 'az' and 'ba' both step +25 (a to z wraps); single letters 'a' and 'z' form a group.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(words):
    groups = []
    for s in words:
        for group in groups:
            if is_shift(group[0], s):
                group.append(s)
                break
        else:
            groups.append([s])
    return groups
def is_shift(a, b):
    if len(a) != len(b):
        return False
    shifts = {(ord(y) - ord(x)) % 26
              for x, y in zip(a, b)}
    return len(shifts) <= 1
```

BRUTE FORCE

TIME	SPACE
O(n * groups * L)	O(n)

WHY IT IS SLOW

Each string is compared with the representative of every group, O(n * groups * L). Using the sequence of gaps between neighbouring letters as a dictionary key groups everything in one pass.

OPTIMAL SOLUTION · Group by the tuple of successive differences

TIME	SPACE
O(total characters)	O(total characters)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if shifting is not cyclic (z does not wrap to a)?

Drop the % 26 and use the raw differences as the key. Two strings with the same raw gaps differ by one fixed integer shift, and both are already inside 'a'..'z', so that shift is automatically legal: the gap tuple alone decides buddies, with no extra range check. What changes is that 'az' and 'ba' are no longer buddies (gaps 25 and -1 instead of 25 and 25).

FOLLOW-UP 2 How would you do this with strings that are too many for one machine?

Compute the gap key on each machine, hash-partition the strings by key (so every buddy lands on the same worker), and group locally. The key is the only thing exchanged, so shuffle cost is one pass over the data. **Cost:** O(total characters / machines) per machine to compute the keys, plus one shuffle of the keys; each group is then built locally in O(group size).

FOLLOW-UP 3 Shifting does not wrap around (z stays z).

Use the raw gaps (no % 26) as the key. Equal raw gaps mean one fixed integer shift, which is automatically legal because both strings are already letters, so no extra check is needed.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given an array of strings, return groups
# of "buddy strings," where the strings have the same
# length and the distance between corresponding characters
# is the same.
#
# Two strings are buddy strings when they have the same
# length and the distance between each pair of
# neighbouring letters is the same (the alphabet wraps
# around after z), so one is the other shifted by a
# constant. Group the strings into buddy groups and return
# the groups in any order.
#
# Signature:
#   def group_buddies(words: list) -> list
#
# Examples:
# ['abc', 'bcd', 'acf', 'xyz', 'az', 'ba', 'a', 'z']  ->
#     [['abc', 'bcd', 'xyz'], ['acf'], ['az', 'ba'], ['a',
```

```
#     'z']]
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if shifting is not cyclic (z does not wrap to
# a)?
# 2. How would you do this with strings that are too many
# for one machine?
# 3. Shifting does not wrap around (z stays z).
#
# INTERVIEW NOTES
# Buddies are shifted copies of each other, so what they
# share is the shape: the gaps between neighbouring
# letters. Turn each string into its shape (a tuple of
# gaps mod 26) and group by it with a dictionary; you
# never compare two strings directly. Mention that single
# letters all have the same (empty) shape.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def shift_key(word):
    # the gaps between neighbouring letters, wrapping z to a (mod 26).
    # Words that are shifts of each other share this key.
    gaps = []
    for i in range(len(word) - 1):
        gaps.append((ord(word[i + 1]) - ord(word[i])) % 26)
    return tuple(gaps)


def canonical(groups):
    # the groups sorted, so two answers can be compared
    result = []
    for group in groups:
        result.append(sorted(group))
    result.sort()
    return result


# ---- main question: main function ----
# Group the strings that are "buddies": the same length, and every letter
# is shifted by the same amount (a -> b, b -> c, ...; z wraps to a).
# Idea: two strings are buddies exactly when the GAPS between neighbouring
# letters (mod 26) are the same, so use the gaps (and the length) as the key of a
# dictionary and put the words with the same key together.
# Complexity: Time O(total length of all words), Space O(total length)
def group_buddies(words, key=shift_key):
    groups = {}
    for word in words:
        word_key = (len(word), key(word))           # words of different length never share a group
        if word_key not in groups:
            groups[word_key] = []
        groups[word_key].append(word)
    return list(groups.values())
```

```
# ---- main question: tests ----
WORDS = ["abc", "bcd", "acef", "xyz", "az", "ba", "a", "z"]


def test_main_question():
    assert canonical(group_buddies(WORDS)) == [["a", "z"], ["abc", "bcd", "xyz"], ["acef"], ["az", "ba"]]
```

```
    assert group_buddies([]) == []


# ========================================================
# FOLLOW-UP 1: What if shifting is not cyclic (z does not wrap to a)?
# ========================================================


# ========================================================
# FOLLOW-UP 3: Shifting does not wrap around (z stays z)
# ========================================================
# ---- follow-up 1: function ----
# Answer: drop the "% 26" and use the raw gaps as the key. Equal raw gaps
# mean one fixed whole-number shift, which is always legal here because both
# words are already made of letters, so no extra check is needed.
# Complexity: Time O(L) for a word of length L, Space O(L)
def raw_key(word):
    gaps = []
    for i in range(len(word) - 1):
        gaps.append(ord(word[i + 1]) - ord(word[i]))
    return tuple(gaps)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    plain = canonical(group_buddies(WORDS, raw_key))
    assert ["az", "ba"] not in plain                # without wrap-around "az" and "ba" are different
    assert ["abc", "bcd", "xyz"] in plain


# ========================================================
# FOLLOW-UP 2: How would you do this with strings that are too many for one machine?
# ========================================================
# ---- follow-up 2: function ----
# Answer: every machine computes the keys for its own slice of the words;
# each (key, word) pair is sent to the machine number hash(key) % machines, so
# all buddies meet on the same machine, and each machine groups what it
# receives. Only the key travels, so the shuffle is one pass over the data.
# Complexity: Time O(total length) spread over the machines, Space O(total length)
def group_distributed(words, machines, key=shift_key):
    inbox = []
    for m in range(machines):
        inbox.append([])
    for m in range(machines):
        for word in words[m::machines]:             # the slice of machine m
```

```
            word_key = (len(word), key(word))
            inbox[hash(word_key) % machines].append((word_key, word))
    result = []
    for mail in inbox:
        local = {}
        for word_key, word in mail:
            if word_key not in local:
                local[word_key] = []
            local[word_key].append(word)
        result += list(local.values())
    return result
```

```
# ---- follow-up 2: tests ----
def test_follow_up_2():
    random.seed(26)
    for round_number in range(200):
        words = []
        for i in range(random.randint(0, 10)):
            word = ""
            for j in range(random.randint(1, 3)):
                word += random.choice("abyz")
            words.append(word)
        groups = group_buddies(words)
        total = 0
        for group in groups:
            total += len(group)
        assert total == len(words)
        assert canonical(group_distributed(words, random.randint(1, 4))) == canonical(groups)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 35 OF 43

K-way merge: a heap of the next item from each source

2 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

2
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

K-way merge: a heap of the next item from each source | 2 questions | tap a title to jump to it

READ FIRST: the code template for these 2 questions (tap here) >

M 170. K Largest Elements Across Sorted Lists **ColorStack * 2025-10-31 ***

M 171. Sort a nearly sorted (or K sorted) array. **ND**

CODE TEMPLATE | GROUP 35 OF 43 | 2 QUESTIONS, ONE PATTERN

K-way merge: a heap of the next item from each source

The one idea these questions share

Put the first item of every list (with its list number and position) on a heap. Pop the smallest, record it, and push the next item from the same list. After k pops you have the k best without sorting everything.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
import heapq


def k_way(lists, k):
    heap = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst]   # CHANGE 1: what you seed with
    heapq.heapify(heap)
    out = []
    while heap and len(out) < k:
        val, i, j = heapq.heappop(heap)
        out.append(val)
        if j + 1 < len(lists[i]):                                     # CHANGE 2: the successor
            heapq.heappush(heap, (lists[i][j + 1], i, j + 1))
    return out
```

2. Helper functions that usually get written next to it

```
def merge_two(a, b):                      # the simple two-list case of the same idea
    i = j = 0
    out = []
    while i < len(a) or j < len(b):
        if j == len(b) or (i < len(a) and a[i] <= b[j]):
            out.append(a[i]); i += 1
        else:
            out.append(b[j]); j += 1
    return out
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
170	K Largest Elements Across Sorted Lists	seed with the LAST item of each list (negated); the successor is the item before it; reverse the result
171	Sort a nearly sorted (or K sorted) array.	keep a heap of the first k + 1 items; pop the smallest and push the next input item (heapreplace)

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 170 of 197 | topic 35 of 43: K-way merge: a heap of the next item from each source | 1 of 2

Template fit: seed with the LAST item of each list (negated); the successor is the item before it; reverse the result | * asked 354 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 170 of 197

K Largest Elements Across Sorted Lists

ASKED 2025-10-31 | 354 DAYS AGO | OLDER

STUDY: TYPE | **ALL TIME 55.2** | **30 DAYS 33** | **PRIORITY 30/288**

ABOUT THIS QUESTION

Topic: K-way merge: a heap of the next item from each source

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #23 Merge k Sorted Lists, LeetCode #215 Kth Largest Element in an Array

LeetCode match: LC 23 Merge k Sorted Lists / LC 215 (related: k largest across sorted lists)

Frequency: Single report | Last asked: 2025-10-31 | Roles: SWE Intern

QUESTION

You are given a list of inner lists, where each inner list is sorted in ascending order. Return the k largest elements taken from the union of all the inner lists, and return them sorted in ascending order. Duplicate values are kept (an element counts once per position it occupies). The interviewer first accepted a simple solution, then asked for an optimized one.

Function signature

```
from typing import List
def k_largest_sorted(lists: List[List[int]], k: int) -> List[int]:
    """Return the k largest elements across all (individually ascending) inner
    lists, in ascending sorted order."""
    ...
```

Follow-up (as reported): Optimize beyond concatenate-and-sort. Hint: because each list is already sorted, seed a max-heap with the last element of every non-empty list and pop k times, pushing the predecessor within a list after each pop; collect the popped values and reverse them.

EXAMPLES

Example 1	
Input	`lists = [[1, 4, 7], [2, 5, 8], [3, 6, 9]], k = 4`
Output	`[6, 7, 8, 9]`
Why	`The four largest values overall are 9, 8, 7, 6, returned ascending.`

Example 2	
Input	`lists = [[10, 20, 30]], k = 2`
Output	`[20, 30]`

Example 3	
Input	`lists = [[5], [1, 3]], k = 5`
Output	`[1, 3, 5]`
Why	`k exceeds the total count, so all elements are returned sorted.`

QUESTION

Example 4	
Input	`lists = [[1, 2], [3, 4]], k = 0`
Output	`[]`

CONSTRAINTS

- Let m be the number of inner lists and T the total number of elements across all of them.
- 0 <= m <= 10^4 , 0 <= T <= 10^6 .
- Each element is an integer in [-10^9, 10^9] .
- 0 <= k . If k == 0 return [] . If k >= T , return every element sorted ascending.
- Some inner lists may be empty.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_k_largest(lists, k):
    flat = sorted(
        x for lst in lists for x in lst
    )
    return (
        flat[max(0, len(flat) - k) :]
        if k
        else:
            []
    )
```

TIME	SPACE
O(N log N)	O(N)

WHY IT IS SLOW

It sorts all N elements although only k are needed.

OPTIMAL SOLUTION · A max-heap over the ends of the lists

TIME	SPACE
O(m + k log m)	O(m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The lists are not sorted, or arrive as streams. What changes?

Keep a min-heap of the k largest values seen so far: push each value and pop the smallest when the heap has more than k items. O(N log k) time and O(k) memory, in one pass.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: K Largest Elements Across Sorted Lists
#
# You are given a list of inner lists, where each inner
```

```
# list is sorted in ascending order. Return the k largest
# elements taken from the union of all the inner lists,
# and return them sorted in ascending order. Duplicate
# values are kept (an element counts once per position it
# occupies). The interviewer first accepted a simple
# solution, then asked for an optimized one. Function
# signature: from typing import List def
# k_largest_sorted(lists: List[List[int]], k: int) ->
# List[int]: """Return the k largest elements across all
# (individually ascending) inner lists, in ascending
# sorted order.""" ... Follow-up (as reported): Optimize
# beyond concatenate-and-sort. Hint: because each list is
# already sorted, seed a max-heap with the last element of
# every non-empty list and pop k times, pushing the
# predecessor within a list after each pop; collect the
# popped values and reverse them.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The lists are not sorted, or arrive as streams. What
# changes?
#
# INTERVIEW NOTES
# Sorting everything costs O(N log N). Because every list
# is already sorted, the largest remaining elements are at
# the ends of the lists: seed a max-heap with the last
# element of each list, pop the largest k times, and push
# the predecessor from the same list after each pop. That
# is O(k log m) for m lists after the seeding.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq
import random


# ============================================================
# MAIN QUESTION
```

```
# ============================================================
# ---- main question: main function ----
# Each list is sorted (smallest first). Return the k largest numbers of
# all lists together, in increasing order.
# Idea: the next biggest number is always at the END of one of the lists.
# A max-heap (push negatives) holds the last unused element of every list.
# Take the biggest out, then offer the element just before it in its list.
# Complexity: Time O(m + k log m) for m lists, Space O(m + k)
def k_largest_sorted(lists, k):
    heap = []
    for i in range(len(lists)):
        if lists[i]:
            last = len(lists[i]) - 1
            heap.append((-lists[i][last], i, last))
    heapq.heapify(heap)
    biggest_first = []
    while heap and len(biggest_first) < k:
        negative, i, j = heapq.heappop(heap)
        biggest_first.append(-negative)
        if j > 0:
            heapq.heappush(heap, (-lists[i][j - 1], i, j - 1))
    biggest_first.reverse()
```

```
    return biggest_first


# ---- main question: tests ----
def test_main_question():
    assert k_largest_sorted([[1, 4, 7], [2, 5, 8], [3, 6, 9]], 4) == [6, 7, 8, 9]
    assert k_largest_sorted([[10, 20, 30]], 2) == [20, 30]
    assert k_largest_sorted([[5], [1, 3]], 5) == [1, 3, 5]
    assert k_largest_sorted([[1, 2], [3, 4]], 0) == []


# ========================================================
# FOLLOW-UP 1: The lists are not sorted, or arrive as streams. What changes?
# ========================================================
# ---- follow-up 1: function ----
# Answer: keep a MIN-heap of the k largest values seen so far: push every
# value, and when the heap holds more than k, drop the smallest. One pass,
# O(N log k) time and O(k) memory.
# Complexity: Time O(N log k) for N numbers in total, Space O(k)
def k_largest_stream(lists, k):
    heap = []
    for numbers in lists:
        for x in numbers:
            if len(heap) < k:
                heapq.heappush(heap, x)
            elif k > 0 and x > heap[0]:
                heapq.heapreplace(heap, x)                # drop the smallest of the k kept
    return sorted(heap)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert k_largest_stream([[4, 1, 7], [8, 2, 5], [3, 9, 6]], 4) == [6, 7, 8, 9]
    random.seed(77)
    for round_number in range(200):
        lists = []
        for i in range(random.randint(0, 4)):
            numbers = []
            for j in range(random.randint(0, 5)):
                numbers.append(random.randint(0, 20))
            lists.append(sorted(numbers))
        k = random.randint(0, 6)
        everything = []
        for numbers in lists:
            everything += numbers
        everything.sort()
        start = len(everything) - k
        if start < 0:
            start = 0
        expected = everything[start:]
        if k == 0:
            expected = []
        assert k_largest_sorted(lists, k) == expected
        assert k_largest_stream(lists, k) == expected


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 171 of 197 | topic 35 of 43: K-way merge: a heap of the next item from each source | 2 of 2

Template fit: keep a heap of the first k + 1 items; pop the smallest and push the next input item (heapreplace) | ND

MEDIUM | Exponent | 171 of 197

★ TOP 75: rank 41 in the Medium and Hard priority list

★ Sort a nearly sorted (or K sorted) array.

ASKED (ANY CO.) ~A YEAR AGO | OLDER

STUDY: TYPE | ALL TIME 55.2 | 30 DAYS 33 | PRIORITY 31/288

ABOUT THIS QUESTION

Topic: K-way merge: a heap of the next item from each source

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #215 Kth Largest Element in an Array, LeetCode #23 Merge k Sorted Lists

LeetCode match: No direct match; related LC 215 Kth Largest Element, LC 23 (inferred from short statement)

Last asked (any company): a year ago, about 2025-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/5392/k-sorted-array

QUESTION

In a k-sorted array every element is at most k positions away from where it would be in the sorted array. Sort such an array in O(n log k) time (better than a full sort when k is small).

Function signature

def sort_k_sorted(arr: list, k: int) -> list

EXAMPLES

Example 1	
Input	`[6, 5, 3, 2, 8, 10, 9], 3`
Output	`[2, 3, 5, 6, 8, 9, 10]`
Why	`Every number is within 3 positions of its place in the sorted order.`

Example 2	
Input	`[2, 1, 4, 3], 1`
Output	`[1, 2, 3, 4]`
Why	`k = 1: neighbours may be swapped.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(arr):
    return sorted(arr)  # ignores the K hint
```

TIME	SPACE
O(n log n)	O(n)

WHY IT IS SLOW

O(n log n) regardless of K. Each element is at most K places from its sorted position, so a min-heap holding a window of K + 1 elements emits the sorted order in O(n log K).

OPTIMAL SOLUTION · Min-heap of size K + 1

TIME	SPACE
O(n log K)	O(K)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 K is not given. How do you find it?

Sort a copy with the original index as a tie-break and take the largest |original index - sorted index|; that is the smallest valid K. O(n log n), so only worth it if you will sort many similar arrays.

FOLLOW-UP 2 The data is a stream you cannot hold in memory.

The heap of K + 1 elements is already a streaming algorithm: it emits the next sorted element after each arrival once it holds K + 1 elements. Memory O(K).

FOLLOW-UP 3 K is close to n.

Then O(n log K) approaches O(n log n) and the heap has no advantage; use the library sort. State the crossover.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Sort a nearly sorted (or K sorted) array.
#
# In a k-sorted array every element is at most k positions
# away from where it would be in the sorted array. Sort
# such an array in O(n log k) time (better than a full
# sort when k is small).
#
# Signature:
#   def sort_k_sorted(arr: list, k: int) -> list
#
# Examples:
# [6, 5, 3, 2, 8, 10, 9], 3  ->  [2, 3, 5, 6, 8, 9, 10]
# [2, 1, 4, 3], 1  ->  [1, 2, 3, 4]
#
# FOLLOW-UPS (answered below, in this order):
# 1. K is not given. How do you find it?
# 2. The data is a stream you cannot hold in memory.
# 3. K is close to n.
#
# INTERVIEW NOTES
# If each element is at most K places from where it
# belongs, then the smallest element of the whole array
# must be among the first K + 1 elements. A min-heap
# holding a sliding window of K + 1 elements always has
# the next output on top, so you never need to sort the
# whole array.

# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================
```

```
import heapq
import random


# =====================================================
# MAIN QUESTION
# =====================================================
# ---- main question: main function ----
# Every element is at most k places away from where it belongs when the
# array is sorted. Idea: the smallest of the next k + 1 elements must be
# the next one of the answer. Keep a min-heap of k elements; for every new
# element push it and take out the smallest (heappushpop does both). At the
# end empty the heap. O(n log k).
# Complexity: Time O(n log k), Space O(k)
def sort_k_sorted(arr, k):
    heap = arr[:k]
    heapq.heapify(heap)
    result = []
    for x in arr[k:]:
        result.append(heapq.heappushpop(heap, x))      # push x, then pop the smallest
    while heap:
        result.append(heapq.heappop(heap))
    return result


# ---- main question: tests ----
def test_main_question():
    nearly = [6, 5, 3, 2, 8, 10, 9]
    assert sort_k_sorted(nearly, 3) == sorted(nearly)
    assert sort_k_sorted([], 2) == []
    assert sort_k_sorted([3, 1, 2], 5) == [1, 2, 3]
    assert sort_k_sorted([1, 2, 3], 0) == [1, 2, 3]


# =====================================================
# FOLLOW-UP 1: K is not given. How do you find it?
# =====================================================
# ---- follow-up 1: function ----
# Answer: sort the positions by value (equal values keep their order) and
# take the biggest distance between a position and the place it ends up
# in; that is the smallest valid k. O(n log n), so it is only worth it if you
# will sort many similar arrays.
# Complexity: Time O(n log n), Space O(n)
def find_k(arr):
    pairs = []
    for position in range(len(arr)):
        pairs.append((arr[position], position))
    pairs.sort()                                       # now pairs[i] says which original position belongs
at place i
    biggest = 0
    for place in range(len(pairs)):
        distance = pairs[place][1] - place
        if distance < 0:
            distance = -distance
        if distance > biggest:
            biggest = distance
    return biggest


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert find_k([1, 2, 3]) == 0
```

```
    assert find_k([2, 1, 3]) == 1
    assert find_k([6, 5, 3, 2, 8, 10, 9]) == 3
    assert find_k([]) == 0


# ======================================================
# FOLLOW-UP 2: The data is a stream you cannot hold in memory
# ======================================================
# ---- follow-up 2: function ----
# Answer: the heap is already a streaming algorithm: after the heap holds
# more than k elements, each new arrival lets one sorted element go out.
# As a generator it uses O(k) memory.
# Complexity: Time O(n log k), Space O(k)
def stream_sort(items, k):
    heap = []
    for x in items:
        heapq.heappush(heap, x)
        if len(heap) > k:
            yield heapq.heappop(heap)
    while heap:
        yield heapq.heappop(heap)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    nearly = [6, 5, 3, 2, 8, 10, 9]
    assert list(stream_sort(iter(nearly), 3)) == sorted(nearly)


# ======================================================
# FOLLOW-UP 3: K is close to n
# ======================================================
# ---- follow-up 3: function ----
# Answer: then O(n log k) is close to O(n log n) and the heap has no
# advantage: use the ordinary sort. Say where the crossover is.
# Complexity: Time O(n log k) for small k, O(n log n) when k is large, Space O(k) or O(n)
def sort_any(arr, k):
    if k >= len(arr) // 2:
        return sorted(arr)
    return sort_k_sorted(arr, k)
```

```
# ---- follow-up 3: tests ----
def test_follow_up_3():
    random.seed(19)
    for round_number in range(300):
        arr = []
        for i in range(random.randint(0, 9)):
            arr.append(random.randint(0, 9))
        k = find_k(arr)
        assert sort_k_sorted(arr, k) == sorted(arr)      # k is valid
        assert sort_any(arr, k) == sorted(arr)
    for round_number in range(300):                      # different values: compare with the plain definit
ion
        arr = random.sample(range(30), random.randint(2, 8))
        place = {}
        for i in range(len(arr)):
            place[sorted(arr)[i]] = i
        farthest = 0
        for i in range(len(arr)):
            distance = abs(i - place[arr[i]])
            if distance > farthest:
                farthest = distance
        assert find_k(arr) == farthest
```

```
# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 36 OF 43

Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up | 4 questions | tap a title to jump to it

READ FIRST: the code template for these 4 questions (tap here) >

M 172. Find mode in a trinary search tree — **PracHub * 2025-11-16 ***

M 173. Next Upcoming Birthday in a Family Tree — **ColorStack * 2025-10-02 ***

M 174. Lowest Common Ancestor — **ND**

M 175. Determine if a given binary tree is a binary search tree (BST). — **ND**

CODE TEMPLATE | GROUP 36 OF 43 | 4 QUESTIONS, ONE PATTERN

Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

The one idea these questions share

Same recursive walk as Tier 1, but now the information also travels down: lower and upper bounds for a binary search tree, or the target nodes you are looking for. The BST rule (left < node < right) lets you go left or right instead of visiting everything.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def check(node, lo=float("-inf"), hi=float("inf")):
    if node is None:
        return True                          # CHANGE 1: base case
    if not lo < node.val < hi:               # CHANGE 2: the rule this node must obey
        return False
    return check(node.left, lo, node.val) and check(node.right, node.val, hi)


def descend(root, x):                        # BST search: one path, O(height)
    best, node = None, root
    while node:
        if node.val > x:                     # CHANGE 3: go left or right by the rule
            best, node = node.val, node.left
        else:
            node = node.right
    return best
```

2. Helper functions that usually get written next to it

```
def inorder(root):                           # left, node, right: sorted order for a BST
    stack, node = [], root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        yield node.val
        node = node.right


def leftmost(root):                          # the smallest key (the wrap-around answer)
    while root.left:
        root = root.left
    return root.val
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
172	Find mode in a trinary search tree	in-order walk (left, middle, right); equal values are neighbours, so one running count finds the mode(s)
173	Next Upcoming Birthday in a Family Tree	walk the ancestors with a stack and a seen set, keep the best birthday by the circular distance from today
174	Lowest Common Ancestor	dfs returns the node if it is p or q; a node that gets an answer from both sides is the ancestor
175	Determine if a given binary tree is a binary search tree (BST).	check(): pass lo and hi bounds down; fail when a value leaves its allowed window

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 172 of 197 | topic 36 of 43: Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up | 1 of 4

Template fit: in-order walk (left, middle, right); equal values are neighbours, so one running count finds the mode(s) | * asked 338 days ago *

MEDIUM | IN APP LC 501 | PracHub | Technical Screen | 172 of 197

Find mode in a trinary search tree

POSTED 2025-11-16 | 338 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 158/288

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #501 Find Mode in Binary Search Tree

LeetCode match: LC 501 Find Mode in Binary Search Tree

Posted on PracHub: 2025-11-16 (updated 2026-06-20)

Original: https://prachub.com/coding-questions/find-mode-in-a-trinary-search-tree

QUESTION

You are given the root of a **trinary search tree** where each node has up to three children:

```
class Node { int val; Node left; // all values < val Node middle; // all values = val Node right; // all v
alues > val }
```

Assume the tree is intended to satisfy the ordering rules above.

Task

Return the **mode(s)** of the tree: the value(s) that occur most frequently among all nodes.

- If there is one mode, return a single-element list.
- If multiple values tie for highest frequency, return **all** such values (in any order).

Constraints (typical)

- Number of nodes **n** can be large (e.g., up to **`10^5`**).
- Values can be any 32-bit signed integers.

Follow-up

How would your approach change if the tree is **broken** (i.e., it may not respect the **<**, **=**, **>** constraints), but you still must return the correct mode(s)?

Part 1: Find Mode(s) in a Valid Trinary Search TreeYou are given a valid trinary search tree. Each node has an integer value and up to three children: left, middle, and right. For every node, all values in its left subtree are strictly less than the node's value, all values in its middle subtree are equal to the node's value, and all values in its right subtree are strictly greater than the node's value. The tree is encoded using parallel arrays instead of Node objects. Return the value or values that occur most frequently in the tree. To make the output deterministic, return the modes in increasing order. If the tree is empty, return an empty list.

Constraints

0 <= n <= 100000, where n == len(values)len(left) == len(middle) == len(right) == nEach child index is either -1 or an integer in [0, n - 1]root == -1 if n == 0; otherwise 0 <= root < nThe nodes reachable from root form a valid tree with no cyclesThe tree satisfies the trinary search ordering rules recursivelyEach value is a 32-bit signed integer

EXAMPLES

```
Input: ([], [], [], [], -1)
Expected Output: []
```

QUESTION

Explanation: The tree is empty, so there are no modes.

```
Input: ([42], [-1], [-1], [-1], 0)
Expected Output: [42]
```

Explanation: A single-node tree has that node's value as the only mode.

Hints

In a valid trinary search tree, an inorder traversal using left, node, middle, right visits values in nondecreasing order.Because equal values appear consecutively during that traversal, you can track only the current run length and the best run length so far.

Part 2: Find Mode(s) in a Broken Trinary TreeYou are given a trinary tree with integer values. Each node has up to three children: left, middle, and right. Unlike a valid trinary search tree, this tree may be broken: the left, middle, and right children do not necessarily follow any ordering rules. Return the value or values that occur most frequently among all reachable nodes. To make the output deterministic, return the modes in increasing order. If the tree is empty, return an empty list.

Constraints

0 <= n <= 100000, where n == len(values)len(left) == len(middle) == len(right) == nEach child index is either -1 or an integer in [0, n - 1]root == -1 if n == 0; otherwise 0 <= root < nThe nodes reachable from root form a tree with no cyclesNo ordering rule is guaranteed between a node and its childrenEach value is a 32-bit signed integer

Examples

Input: ([], [], [], [], -1)

Expected Output: []

Explanation: The tree is empty, so there are no modes.

Input: ([-2147483648], [-1], [-1], [-1], 0)

Expected Output: [-2147483648]

Explanation: A single-node tree has that node's value as the only mode.

Hints

Without the ordering property, equal values are not guaranteed to be adjacent in any traversal.Use a frequency table while traversing the entire tree.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def modes(root):
    counts = Counter()
    stack = [root]
    while stack:
        node = stack.pop()
        if node:
            counts[node.val] += 1
            stack += [node.left, node.middle,
                      node.right]
    top = max(counts.values())
    return [v for v in counts
            if counts[v] == top]
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

BRUTE FORCE

O(n) time but memory for every distinct value. In a valid tree an in-order walk (left, middle, right) gives equal values consecutively, so a running counter needs only O(height) space.

OPTIMAL SOLUTION · In-order traversal counting runs (O(height) extra space)

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree may be broken (it does not respect the ordering).

In-order no longer gives sorted values, so equal values need not be adjacent. Fall back to a frequency table over a full traversal (the Counter version): O(n) time, O(distinct values) space.

FOLLOW-UP 2 Only O(1) extra space (no stack, no dictionary).

Use Morris traversal for the in-order walk (temporary threads) with the same running-count logic; it mutates the tree briefly, so restore the pointers. Adapting it to the middle child needs care.

FOLLOW-UP 3 The tree is huge and values arrive in a stream.

For a stream use a count dictionary (or a Misra-Gries summary if only frequent values matter); the sorted-order trick needs the whole tree in order. **Cost:** O(1) per value with a counter and O(d) memory for the distinct values (or O(1 / epsilon) with Misra-Gries).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find mode in a trinary search tree
#
# You are given the root of a **trinary search tree**
# where each node has up to three children: text class
# Node { int val; Node left; // all values < val Node
# middle; // all values = val Node right; // all values >
# val } Assume the tree is intended to satisfy the
# ordering rules above. ## Task Return the **mode(s)** of
# the tree: the value(s) that occur most frequently among
# all nodes. - If there is one mode, return a single-
# element list. - If multiple values tie for highest
# frequency, return **all** such values (in any order). ##
# Constraints (typical) - Number of nodes n can be large
# (e.g., up to 10^5). - Values can be any 32-bit signed
# integers.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree may be broken (it does not respect the
# ordering).
# 2. Only O(1) extra space (no stack, no dictionary).
# 3. The tree is huge and values arrive in a stream.
#
# INTERVIEW NOTES
# In a valid search tree an in-order walk (left, middle,
# right) visits values in sorted order, so equal values
```

```
# are adjacent and you can count runs with a single
# running counter and O(height) memory instead of a
# dictionary. If the tree may be broken, the order
# guarantee is gone and you need a real frequency table.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class TNode:
    # a trinary search tree node: everything in `left` is smaller than
    # val, `middle` holds a copy of val, everything in `right` is bigger
    def __init__(self, val, left=None, middle=None, right=None):
        self.val = val
        self.left = left
        self.middle = middle
        self.right = right


class ModeTracker:
    # values arrive in sorted order, so equal values come one after the
    # other and a running count is enough (no dictionary)
    def __init__(self):
        self.best = 0              # the longest run so far
        self.modes = []            # the values that have that run length
        self.current = None
        self.run = 0

    def add(self, value):
        if value == self.current:
            self.run += 1
        else:
            self.run = 1
        self.current = value
        if self.run > self.best:
            self.best = self.run
            self.modes = [value]
        elif self.run == self.best and value not in self.modes:
            self.modes.append(value)


# ---- main question: main function ----
# Idea: an IN-ORDER walk (left, middle chain, right) gives the values in
# sorted order. Use an explicit stack: O(height) extra space.
# Complexity: Time O(n), Space O(h)
def find_modes(root):
    tracker = ModeTracker()
    stack = []
    node = root
    while stack or node:
        while node:                        # go as far left as possible
            stack.append(node)
            node = node.left
        node = stack.pop()
        tracker.add(node.val)
        middle = node.middle               # the middle chain repeats node.val once per node
        while middle:
```

```
            tracker.add(middle.val)
```

```
            middle = middle.middle
        node = node.right
    return sorted(tracker.modes)


# ---- main question: tests ----
#        5
#     /  |  \
#    3   5   8
#   /|       |
#  3 3       8
def example_tree():
    threes = TNode(3, None, TNode(3, None, TNode(3)))
    return TNode(5, threes, TNode(5), TNode(8, None, TNode(8)))


def test_main_question():
    assert find_modes(example_tree()) == [3]
    assert find_modes(None) == []
    tie = TNode(2, TNode(1), TNode(2), None)
    assert find_modes(tie) == [2]
    assert find_modes(TNode(2, TNode(1), None, TNode(3))) == [1, 2, 3]       # a three-way tie


# ========================================================
# FOLLOW-UP 1: The tree may be broken (it does not respect the ordering)
# ========================================================
# ---- follow-up 1: function ----
# Answer: an in-order walk no longer gives sorted values, so equal
# values need not be next to each other. Fall back to a frequency table
# over a full traversal: O(n) time, O(distinct values) space.
# Complexity: Time O(n), Space O(n)
def find_modes_counter(root):
    counts = {}
    stack = []
    if root:
        stack.append(root)
    while stack:
        node = stack.pop()
        counts[node.val] = counts.get(node.val, 0) + 1
        for child in (node.left, node.middle, node.right):
            if child:
                stack.append(child)
    top = 0
    for count in counts.values():
```

```
        if count > top:
            top = count
    modes = []
    for value in counts:
        if counts[value] == top:
            modes.append(value)
    modes.sort()
    return modes


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert find_modes_counter(example_tree()) == [3]
    assert find_modes_counter(TNode(2, TNode(1), TNode(2), None)) == [2]
    assert find_modes_counter(None) == []
```

```
# ============================================================
# FOLLOW-UP 2: Only O(1) extra space (no stack, no dictionary)
# ============================================================
# Answer: use Morris in-order traversal, which threads temporary links
# through the tree instead of using a stack (and restores them).


# ============================================================
# FOLLOW-UP 3: The tree is huge and values arrive in a stream
# ============================================================
# Answer: keep a ModeTracker and feed it the values as they arrive in
# sorted order; if they are unsorted use a counter and a "top" value.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(values: list[int], left: list[int], middle: list[int], right: list[int], root: int) -> list[in
t]:
```

Sample call

```
solution([], [], [], [], -1)
```

Test cases

Input	Expected
([], [], [], [], -1)	[]
([42], [-1], [-1], [-1], 0)	[42]

STUDY PATH: question 173 of 197 | topic 36 of 43: Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up | 2 of 4

Template fit: walk the ancestors with a stack and a seen set, keep the best birthday by the circular distance from today | * asked 383 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 173 of 197

Next Upcoming Birthday in a Family Tree

ASKED 2025-10-02 | 383 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 224/288

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

Optimal answer from: written and tested here for this exact question

LeetCode match: Tree traversal with a cyclic date key (related: LC 1507 / next greater)

Frequency: Single report | Last asked: 2025-10-02 | Roles: SWE Intern

QUESTION

You are given a family tree rooted at "me". Each Person node has a name , a birthday given as (month, day) , and pointers to parent1 and parent2 (either may be None). Following parent pointers reaches the whole ancestry; treat the reachable set of Person nodes as the tree to search (each distinct person counted once, even if reachable by two paths). Given today's date (today_month, today_day) , implement alarm_next_birthday(me, today_month, today_day) that scans every person and returns the (month, day) of the soonest upcoming birthday, where a birthday landing exactly on today counts as upcoming (distance 0) and the calendar wraps around the year. If the tree is empty (me is None), return the empty string "" . Use this ordering key to compare dates: key(m, d) = (m - 1) * 31 + d , and cyclic distance (key(person) - key(today)) mod 372 . Return the person with the smallest such distance; if several people share that soonest date, return that (month, day) .

Function signature

```
from typing import Optional, Union
class Person:
    def __init__(self, name: str, month: int, day: int,
                 parent1: "Optional[Person]" = None,
                 parent2: "Optional[Person]" = None) -> None:
        self.name = name
        self.month = month
        self.day = day
        self.parent1 = parent1
        self.parent2 = parent2
def alarm_next_birthday(me: Optional[Person],
                        today_month: int,
                        today_day: int) -> Union[tuple[int, int], str]:
    """Return (month, day) of the nearest upcoming birthday, or "" if there is
nobody."""
    ...
```

Follow-up (as reported): Support repeated alarms: after reporting one birthday, the next call returns the following-soonest, and once all have been reported, return "" .

EXAMPLES

QUESTION

Example 1	
Input	me = Me(12,1) with parent1 = Alice(9,25), parent2 = Bob(1,10); today = (9, 12)
Output	(9, 25)
Why	keys — today=260, Alice=273 (dist 13), Me=342 (dist 82), Bob=10 (dist (10- 260) mod 372 = 122). Alice is soonest.

Example 2	
Input	me = None; today = (5, 5)
Output	""
Why	No people to check.

Example 3	
Input	me = Me(3,3) alone; today = (3, 3)
Output	(3, 3)
Why	The only birthday is today, distance 0.

CONSTRAINTS

- 1 <= month <= 12 , 1 <= day <= 31 .
- Up to 10^5 people; parent pointers form a DAG (no cycles).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def key(month, day):
    return (month - 1) * 31 + day
def brute_alarm(me, tm, td):
    people, stack, seen = (
        [],
        [me] if me else [],
        set(),
    )
    while stack:
        p = stack.pop()
        if id(p) in seen:
            continue
        seen.add(id(p))
        people.append(p)
        stack += [
            q
            for q in (p.parent1, p.parent2)
            if q
        ]
    if not people:
        return ""
    people.sort(
        key=lambda p: (
            key(p.month, p.day) - key(tm, td)
        )
        % 372
    )
    return (people[0].month, people[0].day)
```

TIME	SPACE
O(p log p)	O(p)

BRUTE FORCE

WHY IT IS SLOW

It sorts all p people although only the nearest is needed.

OPTIMAL SOLUTION · One traversal tracking the minimum distance

TIME	SPACE
O(p)	O(p)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Support repeated alarms: each call returns the next soonest birthday, then an empty string when all are used.

Collect the distinct (distance, month, day) values once, sort them, and keep an index: each call returns the next entry. If people can be added later, use a heap keyed by distance instead.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class Person:
    def __init__(self, name, month, day, parent1=None, parent2=None):
        self.name = name
        self.month = month
        self.day = day
        self.parent1 = parent1
        self.parent2 = parent2


def day_number(month, day):
    # a number that grows through the year; 12 months of up to 31 days = 372
    return (month - 1) * 31 + day


def all_people(me):
    # `me` and every ancestor, each person once (a person can be reached
    # through both parents, so remember who was seen)
    people = []
    seen = set()
    stack = []
    if me:
        stack.append(me)
    while stack:
        person = stack.pop()
        if person in seen:
            continue
```

```
        seen.add(person)
        people.append(person)
        for parent in (person.parent1, person.parent2):
            if parent:
                stack.append(parent)
    return people
```

```
# ---- main question: main function ----
# Idea: look at every person once and keep the birthday that comes next:
# the number of days from today to the birthday, going round the year
# (the "% 372" wraps a birthday that already passed to next year).
# Complexity: Time O(n) for n people, Space O(n)
def alarm_next_birthday(me, today_month, today_day):
    if me is None:
        return ""
    today = day_number(today_month, today_day)
    best_distance = None
    answer = ""
    for person in all_people(me):
        distance = (day_number(person.month, person.day) - today) % 372
        if best_distance is None or distance < best_distance:
            best_distance = distance
            answer = (person.month, person.day)
    return answer


# ---- main question: tests ----
def test_main_question():
    alice = Person("Alice", 9, 25)
    bob = Person("Bob", 1, 10)
    me = Person("Me", 12, 1, alice, bob)
    assert alarm_next_birthday(me, 9, 12) == (9, 25)
    assert alarm_next_birthday(None, 5, 5) == ""
    assert alarm_next_birthday(Person("Me", 3, 3), 3, 3) == (3, 3)      # today counts


# ========================================================
# FOLLOW-UP 1: Support repeated alarms: each call returns the next birthday after the previous one
# ========================================================
# ---- follow-up 1: function ----
# Answer: collect the distinct (days away, month, day) once, sort them,
# and keep an index: each call returns the next entry. If people can be
# added later use a heap keyed by the number of days instead.
# Complexity: Time build O(n log n), each next() O(1), Space O(n)
class BirthdayAlarms:
    def __init__(self, me, today_month, today_day):
        today = day_number(today_month, today_day)
        dates = set()
        for person in all_people(me):
            distance = (day_number(person.month, person.day) - today) % 372
            dates.add((distance, person.month, person.day))
        self.order = sorted(dates)
```

```
        self.position = 0

    def next(self):
        if self.position >= len(self.order):
            return ""
        distance, month, day = self.order[self.position]
        self.position += 1
        return (month, day)


# ---- follow-up 1: tests ----
```

```
def test_follow_up_1():
    alice = Person("Alice", 9, 25)
    bob = Person("Bob", 1, 10)
    me = Person("Me", 12, 1, alice, bob)
    alarms = BirthdayAlarms(me, 9, 12)
    answers = []
    for i in range(4):
        answers.append(alarms.next())
    assert answers == [(9, 25), (12, 1), (1, 10), ""]


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 174 of 197 | topic 36 of 43: Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up | 3 of 4

Template fit: dfs returns the node if it is p or q; a node that gets an answer from both sides is the ancestor | ND

MEDIUM | IN APP LC 236 | HackTheRounds | Onsite | 174 of 197

Lowest Common Ancestor

STUDY: TYPE **ALL TIME 41.5** **30 DAYS 0** **PRIORITY 101/288**

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #236 Lowest Common Ancestor of a Binary Tree

LeetCode match: LC 236

QUESTION

Given the root of a binary tree and two of its nodes p and q, find their lowest common ancestor: the lowest node that has both p and q as descendants (a node can be a descendant of itself).

Function signature

def lowestCommonAncestor_tree(root: BNode, p: BNode, q: BNode) -> BNode

EXAMPLES

Example 1	

Input	`root = [3, [5, [6], [2]], [1]] (value, left, right)`
	`first query: p = 5, q = 1`
	`second query: p = 5, q = 6`

Output	`lowestCommonAncestor_tree(root, p, q).val -> 3` `lowestCommonAncestor_tree(root, p, p.left).val -> 5`
Why	`5 and 1 are in different subtrees of 3, so 3 is their lowest common ancestor. 5 is an ancestor of 6, and a node may be its own ancestor, so the answer is 5.`

CONSTRAINTS

- Number of nodes in tree: [2, 10^5]
- -10^9 <= Node.val <= 10^9
- All node values are unique
- p != q
- p and q exist in the tree

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def path_to(root, target):
    if root is None:
        return None
    if root is target:
        return [root]
    for child in root.children:
        sub = path_to(child, target)
        if sub:
            return [root] + sub
    return None
def lca(root, p, q):
    path_p = path_to(root, p)
    path_q = path_to(root, q)
    answer = None
    for a, b in zip(path_p, path_q):
        if a is not b:
            break
        answer = a
    return answer
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Two full paths stored, O(n) space, and two traversals. One recursive pass that reports whether each subtree contains p or q finds the answer in O(n) time and O(h) space.

OPTIMAL SOLUTION · One recursive pass

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree is n-ary.

The same recursion: collect the non-empty results from all children; if two or more children reported a hit this node is the answer, otherwise pass the single result (or None) up.

FOLLOW-UP 2 A very deep tree, or no recursion allowed.

Do one traversal that records each node's parent (a dict), then use the ancestor-set method from the parent-pointer version. O(n) time and space, no recursion.

FOLLOW-UP 3 p or q might not be in the tree.

The recursive answer is wrong in that case (it returns the one that exists). Count how many of p and q were seen during the DFS, or first check both exist, and return None if not.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
class Node:
    def __init__(self, val, children=None):
        # two children for a binary tree, any number for an n-ary tree
        self.val = val
        self.children = []
        if children:
            self.children = children


# ---- main question: main function ----
# Idea: one recursive pass. find(node) answers "where did I meet p or q
# in this subtree?": the node itself if it is p or q, otherwise what the
# children report. If TWO children each found something, this node is
# the lowest common ancestor; if only one did, pass that result up.
# The same code works for an n-ary tree (follow-up 1): it just loops
# over every child.
# Complexity: Time O(n), Space O(h) for the height h of the tree
def lca(root, p, q):
    def find(node):
        if node is None:
            return None
        if node is p or node is q:
            return node
        hits = []
        for child in node.children:
            found = find(child)
            if found is not None:
                hits.append(found)
        if len(hits) >= 2:
            return node
        if len(hits) == 1:
```

```
            return hits[0]
        return None

    return find(root)


# ---- main question: tests ----
def example_tree():
    six, four = Node(6), Node(4)
    two = Node(2, [Node(7), four])
    five = Node(5, [six, two])
    one = Node(1, [Node(0), Node(8)])
    return Node(3, [five, one]), five, one, six, four
```

```
def test_main_question():
    root, five, one, six, four = example_tree()
    assert lca(root, six, four) is five
    assert lca(root, five, four) is five          # a node can be its own ancestor
    assert lca(root, six, one) is root
    kids = [Node(0), Node(1), Node(2)]            # an n-ary tree: three children under one node
    top = Node("top", [Node("mid", kids), Node("side")])
    assert lca(top, kids[0], kids[2]).val == "mid"


# ========================================================
# FOLLOW-UP 1: The tree is n-ary
# ========================================================
# Answer: nothing to change: lca() above already collects the hits of
# every child; two or more hits make this node the answer.


# ========================================================
# FOLLOW-UP 2: A very deep tree, or no recursion allowed
# ========================================================
# ---- follow-up 2: helper functions ----
def parents_of(root):
    # one walk with a stack (no recursion) that remembers every node's parent
    parent = {root: None}
    stack = [root]
    while stack:
        node = stack.pop()
        for child in node.children:
            parent[child] = node
            stack.append(child)
```

```
    return parent


# ---- follow-up 2: function ----
# Answer: record every node's parent, then climb from p collecting its
# ancestors, and climb from q until you meet one of them: O(n) time and
# space, no recursion.
# Complexity: Time O(n), Space O(n)
def lca_iterative(root, p, q):
    parent = parents_of(root)
    above = set()
    node = p
    while node:
        above.add(node)
        node = parent[node]
    node = q
    while node not in above:
        node = parent[node]
    return node


# ---- follow-up 2: tests ----
def test_follow_up_2():
    root, five, one, six, four = example_tree()
    assert lca_iterative(root, six, four) is five
    chain = Node(0)                                # a chain of 5000 nodes: no recursion limit
    last = chain
    for i in range(1, 5000):
        child = Node(i)
        last.children.append(child)
        last = child
    assert lca_iterative(chain, last, chain) is chain
```

```
    random.seed(3)
    for round_number in range(100):                # random trees: both versions must agree
        nodes = [Node(0)]
        for i in range(1, random.randint(1, 10)):
            child = Node(i)
            random.choice(nodes).children.append(child)
            nodes.append(child)
        p = random.choice(nodes)
        q = random.choice(nodes)
        assert lca(nodes[0], p, q) is lca_iterative(nodes[0], p, q)


# =============================================================
```

```
# FOLLOW-UP 3: p or q might not be in the tree
# =============================================================
# ---- follow-up 3: function ----
# Answer: the plain recursion would return the one that does exist,
# which is wrong. First check that both nodes are really in the tree
# (they are in the parent map), and return None if not.
# Complexity: Time O(n), Space O(n)
def lca_checked(root, p, q):
    parent = parents_of(root)
    if p not in parent or q not in parent:
        return None
    return lca_iterative(root, p, q)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    root, five, one, six, four = example_tree()
    stranger = Node(404)
    assert lca_checked(root, six, stranger) is None
    assert lca_checked(root, six, four) is five


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 175 of 197 | topic 36 of 43: Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up | 4 of 4

Template fit: check(): pass lo and hi bounds down; fail when a value leaves its allowed window | ND

MEDIUM | **IN APP** **LC 98** | **Exponent** | **175 of 197**

Determine if a given binary tree is a binary search tree (BST).

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TYPE **ALL TIME 36** **30 DAYS 41.1** **PRIORITY 59/288**

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

Optimal answer from: your app's interview approach (LeetCode #98)

LeetCode match: LC 98 Validate Binary Search Tree (inferred from short statement)

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age) | +28 interviews

Original: https://www.aced.io/questions/1158/determine-binary-tree-bst

QUESTION

Given the root of a binary tree, determine whether it is a valid binary search tree: for every node, all values in its left subtree are smaller and all values in its right subtree are larger (strictly).

Function signature

def isValidBST(root: TreeNode) -> bool

EXAMPLES

Example 1	
Input	`[2, 1, 3]`
Output	`true`
Why	`2 with 1 on the left and 3 on the right.`

Example 2	
Input	`[5, 1, 4, null, null, 3, 6]`
Output	`false`
Why	`The 3 sits in the right subtree of 5 but is smaller than 5.`

Example 3	
Input	`[2, 2, 2]`
Output	`false`
Why	`A node equal to its parent breaks the strict rule.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_bst(root):
    values = inorder(root)
    for a, b in zip(values, values[1:]):
        if a >= b:
            return False
    return True
```

BRUTE FORCE

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

O(n) time but stores every value, O(n) space. Passing the allowed (low, high) range down the tree checks each node once with O(height) space and stops at the first violation.

OPTIMAL SOLUTION · Pass the allowed range (low, high) down

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Duplicates are allowed (all on the left, or all on the right).

Make one of the two bounds inclusive: for 'duplicates go left' every node must satisfy lo < val <= hi (the bound inherited from a left ancestor is inclusive, the one from a right ancestor is exclusive). Tested on trees built by inserting duplicates to the left.

FOLLOW-UP 2 Do it with an in-order walk that keeps only the previous value.

Walk in order with an explicit stack and compare each value with the previous one (strictly greater). O(h) space and early exit.

FOLLOW-UP 3 Recover a BST where exactly two nodes were swapped (LeetCode 99).

An in-order walk shows one or two descents; the first and last nodes involved in the descents are the swapped pair. O(n) time and O(h) space (O(1) with Morris traversal).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Determine if a given binary tree is a
# binary search tree (BST).
#
# Given the root of a binary tree, determine whether it is
# a valid binary search tree: for every node, all values
# in its left subtree are smaller and all values in its
# right subtree are larger (strictly).
#
# Signature:
#   def isValidBST(root: TreeNode) -> bool
#
# Examples:
# [2, 1, 3]  ->  true
# [5, 1, 4, null, null, 3, 6]  ->  false
#
# FOLLOW-UPS (answered below, in this order):
# 1. Duplicates are allowed (all on the left, or all on
# the right).
# 2. Do it with an in-order walk that keeps only the
# previous value.
# 3. Recover a BST where exactly two nodes were swapped
# (LeetCode 99).
```

```
#
# INTERVIEW NOTES
# Checking only a node against its two children is not
# enough; every node in the left subtree must be smaller
# than the node, not just the left child. Pass down the
# allowed open interval (low, high) for each subtree, or
# use the fact that an in-order walk of a BST is strictly
# increasing.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


def value_fits(val, low, high, dup):
    # may `val` sit between the bounds from its ancestors (None = no bound)?
    # dup says where an EQUAL value is allowed: "none", "left" or "right"
    if low is not None:
        if val < low:
            return False
        if val == low and dup != "right":
            return False
    if high is not None:
        if val > high:
            return False
        if val == high and dup != "left":
            return False
    return True


# ---- main question: main function ----
# Idea: pass the allowed range (low, high) DOWN. The left child inherits
# (low, node.val) and the right child (node.val, high); the root starts
# with no bounds. Every node must lie strictly between its bounds.
# (dup is for follow-up 1; leave it as "none" for a normal BST.)
# Complexity: Time O(n), Space O(h)
def isValidBST(root, dup="none"):
    stack = [(root, None, None)]
    while stack:
        node, low, high = stack.pop()
        if node is None:
            continue
        if not value_fits(node.val, low, high, dup):
            return False
        stack.append((node.left, low, node.val))
        stack.append((node.right, node.val, high))
    return True


# ---- main question: tests ----
def test_main_question():
```

```
    good = TreeNode(2, TreeNode(1), TreeNode(3))
    bad = TreeNode(5, TreeNode(1), TreeNode(4, TreeNode(3), TreeNode(6)))   # 6 is below 5, so it must be s
maller than 5
    assert isValidBST(good)
    assert not isValidBST(bad)
```

```
    assert isValidBST(None)


# ============================================================
# FOLLOW-UP 1: Duplicates are allowed (all on the left, or all on the right)
# ============================================================
# Answer: make ONE of the two bounds inclusive. With duplicates on the
# left, a node may equal its upper bound; with duplicates on the right
# it may equal its lower bound (the dup parameter above).
# ---- follow-up 1: tests ----
def test_follow_up_1():
    left_dup = TreeNode(2, TreeNode(2), None)
    right_dup = TreeNode(2, None, TreeNode(2))
    assert not isValidBST(left_dup)
    assert isValidBST(left_dup, "left")
    assert not isValidBST(left_dup, "right")
    assert isValidBST(right_dup, "right")
    assert not isValidBST(right_dup, "left")


# ============================================================
# FOLLOW-UP 2: Do it with an in-order walk that keeps only the previous value
# ============================================================
# ---- follow-up 2: function ----
# Answer: an in-order walk must be strictly increasing, so keep only
# the previous value: O(height) space and an early exit.
# Complexity: Time O(n), Space O(h)
def is_bst_inorder(root):
    stack = []
    node = root
    previous = None
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        if previous is not None and node.val <= previous:
            return False
        previous = node.val
        node = node.right
    return True


# ---- follow-up 2: tests ----
def test_follow_up_2():
```

```
    good = TreeNode(2, TreeNode(1), TreeNode(3))
    bad = TreeNode(5, TreeNode(1), TreeNode(4, TreeNode(3), TreeNode(6)))
    assert is_bst_inorder(good)
    assert not is_bst_inorder(bad)


# ============================================================
# FOLLOW-UP 3: Recover a BST where exactly two nodes were swapped (LeetCode 99)
# ============================================================
# ---- follow-up 3: function ----
# Answer: in an in-order walk the swap shows up as one or two places
# where a value is smaller than the one before it (a "descent"). The
# first node of the first descent and the second node of the last
```

```
# descent are the swapped pair. O(n) time, O(height) space.
# Complexity: Time O(n), Space O(h) (O(1) with a Morris traversal)
def recover(root):
    stack = []
    node = root
    previous = None
    first = None
    second = None
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        if previous is not None and node.val < previous.val:
            second = node
            if first is None:
                first = previous
        previous = node
        node = node.right
    if first is not None:
        first.val, second.val = second.val, first.val
    return root


# ---- follow-up 3: tests ----
def test_follow_up_3():
    swapped = TreeNode(3, TreeNode(1), TreeNode(2))        # [2, 1, 3] with 3 and 2 swapped
    recover(swapped)
    assert swapped.val == 2 and swapped.left.val == 1 and swapped.right.val == 3
    assert isValidBST(swapped)
```

```
# =========================================================
# RUN EVERYTHING
# =========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Determine if a given binary tree is a valid Binary Search Tree (BST). HintA binary tree is considered a "valid BST" if for every node: All the values in its left subtree are strictly less than the node's value. All the values in its right subtree are strictly greater than the node's value. Both the left and right subtrees must also be valid BSTs. Examples

```
All trees defined in level-order
root = [5, 2, 6]
output: true

root = [5, 2, 4, null, null, 3, 6]
output: false

root = []
output: true
```

OFFICIAL PAGE TEXT

In these examples, the output is true if the given binary tree root is a valid BST, and false otherwise.Our solution to validate whether a binary tree is a Binary Search Tree (BST) uses a recursive approach. We traverse the tree while maintaining a range for valid node values (min and max). At each node, we check whether its value lies within the given range: If the node’s value is not within the allowed range, we return False, indicating the tree is not a valid BST. If the node’s value is valid, we recursively check its left and right subtrees: The left subtree must have values strictly less than the current node's value. The right subtree must have values strictly greater than the current node's value. For the left subtree, the range becomes (min, current_node_value). For the right subtree, the range becomes (current_node_value, max). We use nullptr to indicate unbounded ranges when there is no minimum or maximum restriction (e.g., for the root node or outermost nodes). If all nodes satisfy their respective conditions, we return True, confirming that the tree is a valid BST. Time Complexity: The solution has a time complexity of O(n), where n is the number of nodes in the binary tree. This is because we traverse each node exactly once, performing constant-time checks at each node. Space Complexity: The space complexity is O(h), where h is the height of the tree. This space is used by the recursion stack. In the worst case (for an unbalanced tree), the height could be n, making the space complexity O(n). For a balanced tree, the height would be O(log n).No answers contributed by the community yet.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 37 OF 43

Random pick without repeats: swap the chosen item with the last one and pop

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Random pick without repeats: swap the chosen item with the last one and pop | 4 questions | tap a title to jump to it

READ FIRST: the code template for these 4 questions (tap here) >

M	176. Design set with O(1) random access	PracHub * 2026-01-01 *
M	177. Random Generator Without Repeats	ColorStack * 2025-10-10 *
M	178. Random Song Shuffler With a No-Repeat Window	ColorStack * 2025-10-06 *
M	179. Unique Random Number Generator	ColorStack * 2025-10-03 *

CODE TEMPLATE | GROUP 37 OF 43 | 4 QUESTIONS, ONE PATTERN

Random pick without repeats: swap the chosen item with the last one and pop

The one idea these questions share

To pick a random item and never pick it again in O(1), keep the items in an array. Choose a random index, swap that item with the last one, pop the last, and fix its position in the index dict. The same trick gives O(1) delete and O(1) random access.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
import random


class PickPool:
    def __init__(self, items, seed=None):
        self.a = list(items)
        self.pos = {x: i for i, x in enumerate(self.a)}   # CHANGE 1: lookup by value (for delete)
        self.rng = random.Random(seed)

    def take_random(self):
        i = self.rng.randrange(len(self.a))
        x, last = self.a[i], self.a[-1]
        self.a[i] = last                                  # the last item fills the hole
        self.pos[last] = i
        self.a.pop()
        del self.pos[x]
        return x                                          # CHANGE 2: what you hand back
```

2. Helper functions that usually get written next to it

```
def fisher_yates(nums, rand_int):             # rand_int(lo, hi) is inclusive on both ends
    for i in range(len(nums) - 1, 0, -1):
        j = rand_int(0, i)
        nums[i], nums[j] = nums[j], nums[i]
    return nums
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
176	Design set with O(1) random access	add appends and records the position; delete swaps with the last item and pops; get_random picks a random index
177	Random Generator Without Repeats	the template as written: values 0..n-1, every call returns a new one; raise when empty
178	Random Song Shuffler With a No-Repeat Window	the same pool, plus a deque of the last k songs; the played song leaves the pool and returns to it when it drops off the deque
179	Unique Random Number Generator	the lazy version for huge n: a dict 'moved' stores only the swapped slots, so memory stays O(calls)

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 176 of 197 | topic 37 of 43: Random pick without repeats: swap the chosen item with the last one and pop | 1 of 4

Template fit: add appends and records the position; delete swaps with the last item and pops; get_random picks a random index | * asked 292 days ago *

MEDIUM | IN APP LC 380 | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 176 of 197

Design set with O(1) random access

POSTED 2026-01-01 | 292 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 46.6 | 30 DAYS 0 | PRIORITY 85/288

ABOUT THIS QUESTION

Topic: Random pick without repeats: swap the chosen item with the last one and pop

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #380 Insert Delete GetRandom O(1)

LeetCode match: LC 380 Insert Delete GetRandom O(1)

Posted on PracHub: 2026-01-01 (updated 2026-06-01)

Original: https://prachub.com/coding-questions/design-set-with-o-1-random-access

QUESTION

Design a data structure ("FancySet") that stores unique integers and supports the following operations, each in average O(1) time:

- **`add(x)`**: Insert value **`x`** if it is not already present. Return whether the insert happened.
- **`delete(x)`**: Remove value **`x`** if it exists. Return whether a deletion happened.
- **`getRandom()`**: Return a uniformly random element currently in the set. Requirements / follow-ups:
- All operations should be average O(1).
- Handle corner cases such as calling **`getRandom()`** on an empty set (define and justify behavior).
- Discuss how you would make this data structure thread-safe. Compare using a single lock for the whole set vs finer-grained locking (e.g., element-level locks), and explain trade-offs.

Design a data structure called FancySet that stores unique integers and supports these operations in average O(1) time: add(x), delete(x), and getRandom(). For this coding problem, implement a function **`solution(operations, values, seed)`** that simulates a newly created empty FancySet and returns the result of every operation. Use the standard dynamic-array + hash-map behavior:

- Store current elements in an array-like list.
- Store each element's current index in a hash map.
- When deleting an element that is not at the end of the list, move the last element into the removed slot, then pop the last slot. To make test cases deterministic, **`getRandom()`** does not use the language's built-in RNG. Instead, keep an internal 32-bit state initialized to **`seed`**. When **`getRandom()`** is called on a non-empty set, update the state with: **`state = (1664525 * state + 1013904223) mod 2^32`** and return the element at index **`state % current_size`** from the current internal list. If **`getRandom()`** is called on an empty set, return **`None`** and do not advance the state. In a real interview, picking a random index from the array gives uniform random access; the fixed generator above is only for reproducible grading. Follow-up discussion (not graded by the code runner): explain how you would make FancySet thread-safe, and compare using one lock for the whole structure versus finer-grained locking.

Constraints

1 <= len(operations) <= 2 * 10^5len(values) == len(operations)Each operation is one of 'add', 'delete', or 'getRandom'For 'add' and 'delete', values[i] contains exactly one integer; for 'getRandom', values[i] is []-10^9 <= x <= 10^9All operations should run in O(1) average time

QUESTION

EXAMPLES

```
Input: (['add','add','getRandom','delete','getRandom','add','getRandom'], [[1],[2],[],[1],[],[2],[]], 7)
Expected Output: [True, True, 1, True, 2, False, 2]
```

Explanation: After inserting 1 and 2, the first deterministic random choice picks index 0, so it returns 1. Deleting 1 leaves only 2. Adding 2 again fails because it is already present.

```
Input: (['getRandom','add','add','delete','delete','getRandom'], [[],[5],[5],[7],[5],[]], 1)
Expected Output: [None, True, False, False, True, None]
```

Explanation: Calling getRandom on an empty set returns None. Adding 5 twice inserts it only once. Deleting 7 fails because it is not present. After deleting 5, the set becomes empty again.

Hints

You need both fast membership/index lookup and fast access by numeric position. What two data structures combine to give that?Deleting from the middle of an array is expensive unless you replace the removed element with the last element and update that last element's stored index.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
items = []
def add(x):
    if x in items:
        return False
    items.append(x)
    return True
def delete(x):
    if x not in items:
        return False
    items.remove(x)
    return True
def get_random():
    return random.choice(items)
```

TIME	SPACE
O(n)	**O(n)**
for add and delete, O(1) getRandom	

WHY IT IS SLOW

Membership checks and deletion are O(n). An array plus a dictionary from value to index, deleting by swapping with the last element, makes all three operations O(1).

OPTIMAL SOLUTION · Array of values plus a dict from value to index

TIME	SPACE
O(1)	**O(n)**
average per operation	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Make it thread-safe: one lock or finer-grained locking?

One lock around every operation is simplest and correct (delete touches the array, the dict and a second element's index together, so they must change as one unit). Finer locks do not help much because every operation touches shared structure; use sharding by value hash if contention is the problem.

FOLLOW-UP 2 What should getRandom do on an empty set?

Raise a clear error (as here) or return a sentinel such as None; whichever you pick, document it and make the caller handle it. Returning an arbitrary value hides bugs.

FOLLOW-UP 3 Duplicates allowed (a multiset, LeetCode 381).

Map each value to a set of indices in the array; delete swaps with the last element and fixes both index sets. getRandom stays a uniform pick over array positions, so values are chosen in proportion to their counts.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Design set with O(1) random access
#
# Design a data structure ("FancySet") that stores unique
# integers and supports the following operations, each in
# average O(1) time: - add(x): Insert value x if it is not
# already present. Return whether the insert happened. -
# delete(x): Remove value x if it exists. Return whether a
# deletion happened. - getRandom(): Return a uniformly
# random element currently in the set. Requirements /
# follow-ups: - All operations should be average O(1). -
# Handle corner cases such as calling getRandom() on an
# empty set (define and justify behavior). - Discuss how
# you would make this data structure thread-safe. Compare
# using a single lock for the whole set vs finer-grained
# locking (e.g., element-level locks), and explain trade-
# offs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Make it thread-safe: one lock or finer-grained
# locking?
# 2. What should getRandom do on an empty set?
# 3. Duplicates allowed (a multiset, LeetCode 381).
#
# INTERVIEW NOTES
# A hash set gives O(1) add and delete but cannot pick a
# random element in O(1); an array can pick a random
# element but deleting from the middle is O(n). Combine
# them: keep the values in an array and a dict from value
# to array index. To delete, swap the element with the
# last one, pop the end, and update the moved element's
# index.

# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

import random
```

```
import threading


# =====================================================
# MAIN QUESTION
```

```
# =====================================================
# ---- main question: main function ----
# Idea: keep the values in a LIST (so a random one can be picked by a
# random index) and a dictionary from value to its index in the list (so
# add and delete can find a value at once).
#   add: append to the list and record the index.
#   delete: look up the index, move the LAST value into that hole (and
#   update its index), remove the last list slot and the dictionary entry.
# Every operation is O(1) on average.
# Complexity: Time add, delete and getRandom are O(1) on average, Space O(n)
class FancySet:
    def __init__(self, seed=None):
        self.items = []
        self.position = {}                          # value -> its index in items
        self.rng = random.Random(seed)

    def add(self, x):
        if x in self.position:
            return False
        self.position[x] = len(self.items)
        self.items.append(x)
        return True

    def delete(self, x):
        if x not in self.position:
            return False
        i = self.position[x]
        last = self.items[-1]
        self.items[i] = last                         # the last value fills the hole
        self.position[last] = i
        self.items.pop()
        del self.position[x]                         # (when x was the last one this removes it again)
        return True

    # follow-up 2: on an empty set there is nothing to return, so raise an
    # IndexError (like random.choice does) instead of returning a value
    # that could be mistaken for an element
    def getRandom(self):
        if len(self.items) == 0:
            raise IndexError("empty set")
        return self.items[self.rng.randrange(len(self.items))]


# ---- main question: tests ----
def test_main_question():
```

```
    s = FancySet(seed=1)
    assert s.add(1) and s.add(2) and not s.add(1)
    assert s.delete(1) and not s.delete(1)
    assert s.getRandom() == 2
    s.delete(2)
    try:
        s.getRandom()
        raise AssertionError("an empty set should raise")
    except IndexError:
        pass
    big = FancySet(seed=5)
    for v in range(4):
        big.add(v)
```

```
    picks = []
    for i in range(4000):
        picks.append(big.getRandom())
    for v in range(4):
        assert 850 < picks.count(v) < 1150           # every value is picked about equally often
    big.delete(0)
    big.delete(2)
    assert sorted(big.items) == [1, 3]
    for v in big.position:                           # deleting from the middle keeps the map consistent
        assert big.items[big.position[v]] == v


# =====================================================
# FOLLOW-UP 1: Make it thread-safe: one lock or finer-grained locking?
# =====================================================
# ---- follow-up 1: function ----
# Answer: one lock around every operation is the simplest correct choice:
# delete changes the list, the dictionary AND a second element's index
# together, so they must change as one unit. Locks per element would not
# protect the shared list and indexes (and risk deadlocks); a finer
# split only pays off if operations rarely touch the same data.
# Complexity: Time O(1) on average per operation (plus waiting for the lock), Space O(n)
class SafeSet:
    def __init__(self, seed=None):
        self.inner = FancySet(seed)
        self.lock = threading.Lock()

    def add(self, x):
        with self.lock:
            return self.inner.add(x)

    def delete(self, x):
```

```
        with self.lock:
            return self.inner.delete(x)

    def getRandom(self):
        with self.lock:
            return self.inner.getRandom()


# ---- follow-up 1: tests ----
def test_follow_up_1():
    safe = SafeSet()
    assert safe.add(7) and safe.getRandom() == 7
    workers = []

    def add_many(start):
        for v in range(start, start + 500):
            safe.add(v)

    for start in (0, 250, 500):
        worker = threading.Thread(target=add_many, args=(start,))
        workers.append(worker)
        worker.start()
    for worker in workers:
        worker.join()
    assert len(safe.inner.items) == 1000 and len(safe.inner.position) == 1000


# =====================================================
# FOLLOW-UP 2: What should getRandom do on an empty set?
# =====================================================
# Answer: raise IndexError (see FancySet.getRandom). Returning None would
# be confused with a stored value, and an exception makes the caller
```

```
# decide. Whatever you choose, say it out loud and test it.


# ======================================================
# FOLLOW-UP 3: Duplicates allowed (a multiset, LeetCode 381)
# ======================================================
# ---- follow-up 3: function ----
# Answer: map each value to the SET of indexes where it sits in the list.
# Deleting one copy takes one index, moves the last element into that
# hole and fixes both index sets. getRandom stays a uniform pick over
# list positions, so a value is chosen in proportion to its count.
# Complexity: Time insert, remove and getRandom are O(1) on average, Space O(n)
class RandomizedCollection:
    def __init__(self):
        self.items = []
        self.where = {}                             # value -> the set of indexes in items

    def insert(self, x):
        is_new = x not in self.where or len(self.where[x]) == 0
        if x not in self.where:
            self.where[x] = set()
        self.where[x].add(len(self.items))
        self.items.append(x)
        return is_new

    def remove(self, x):
        if x not in self.where or len(self.where[x]) == 0:
            return False
        i = self.where[x].pop()
        end = len(self.items) - 1
        if i != end:
            last = self.items[end]
            self.items[i] = last                    # the last item fills the hole
            self.where[last].discard(end)
            self.where[last].add(i)
        self.items.pop()
        return True

    def getRandom(self):
        return random.choice(self.items)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    rc = RandomizedCollection()
    assert rc.insert(5)
    assert not rc.insert(5)
    assert rc.insert(6)
    assert rc.remove(5) and rc.remove(5)
    assert not rc.remove(5)
    assert rc.items == [6]
    assert rc.getRandom() == 6


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(operations: list[str], values: list[list[int]], seed: int) -> str:
```

Sample call

```
solution(['add','add','getRandom','delete','getRandom','add','getRandom'], [[1],[2],[],[1],[],[2],[]], 7)
```

Test cases

Input	Expected
`(['add','add','getRandom','delete','getRandom','add','getRandom'], [[1],[2],[],[1],[],[2],[]], 7)`	`[True, True, 1, True, 2, False, 2]`
`(['getRandom','add','add','delete','delete','getRandom'], [[],[5],[5],[7],[5],[]], 1)`	`[None, True, False, False, True, None]`

STUDY PATH: question 177 of 197 | topic 37 of 43: Random pick without repeats: swap the chosen item with the last one and pop | 2 of 4

Template fit: the template as written: values 0..n-1, every call returns a new one; raise when empty | * asked 375 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 177 of 197

Random Generator Without Repeats

ASKED 2025-10-10 | 375 DAYS AGO | OLDER

STUDY: TYPE **ALL TIME 46.6** **30 DAYS 0** **PRIORITY 88/288**

ABOUT THIS QUESTION

Topic: Random pick without repeats: swap the chosen item with the last one and pop

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #380 Insert Delete GetRandom O(1)

LeetCode match: LC 384 Shuffle an Array / LC 380 (related: no-repeat generator)

Frequency: Single report | Last asked: 2025-10-10 | Roles: SWE Intern

QUESTION

Design a class RandomGenerator initialized with a positive integer n . Each call to generate() returns one integer chosen uniformly at random from the values in [0, n-1] that have not yet been returned. After n calls every value in [0, n-1] has been returned exactly once; a further call must raise IndexError (the pool is empty). Each generate() call must run in O(1) time. The intended approach maintains an array of the remaining values and, on each call, swaps a randomly chosen remaining value to the end of the active region and shrinks that region (partial Fisher-Yates).

Function signature

```
import random
class RandomGenerator:
    def __init__(self, n: int) -> None:
        """Prepare to yield a random permutation of [0, n-1], one value per
generate()."""
        ...
    def generate(self) -> int:
        """Return a uniformly random not-yet-returned value in O(1).
        Raise IndexError if all values have already been generated."""
        ...
```

Follow-up (as reported): Add a reset() that makes all n values available again in O(1) amortized.

EXAMPLES

Example 1

Input	`g = RandomGenerator(3); [g.generate() for _ in range(3)]`
Output	`e.g. [2, 0, 1]`
Why	`The three calls return a permutation of {0, 1, 2} with no repeats; any of the 6 permutations is a valid output. A 4th call would raise IndexError.`

Example 2

Input	`g = RandomGenerator(1); g.generate()`
Output	`0`
Why	`Only one value exists.`

CONSTRAINTS

QUESTION

- 1 <= n <= 10**6 .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
import random
class ShuffleOnce:
    def __init__(self, n):
        self.items = list(range(n))
        random.shuffle(self.items)
    def generate(self):
        return self.items.pop()
```

TIME	SPACE
O(n)	**O(n)**
setup, O(1) per call	

WHY IT IS SLOW

A reset needs a new O(n) shuffle, and the whole permutation is fixed in advance.

OPTIMAL SOLUTION · Partial Fisher-Yates with an active region

TIME	SPACE
O(1)	**O(n)**
per call	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Add reset() so all n values are available again in O(1).

Set size back to n. The array still contains every value 0..n-1 (only their order changed), and the next draws are uniform over the active region, so nothing needs to be rebuilt.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Random Generator Without Repeats
#
# Design a class RandomGenerator initialized with a
# positive integer n . Each call to generate() returns one
# integer chosen uniformly at random from the values in
# [0, n-1] that have not yet been returned. After n calls
# every value in [0, n-1] has been returned exactly once;
# a further call must raise IndexError (the pool is
# empty). Each generate() call must run in O(1) time. The
# intended approach maintains an array of the remaining
# values and, on each call, swaps a randomly chosen
# remaining value to the end of the active region and
# shrinks that region (partial Fisher-Yates). Function
# signature: import random class RandomGenerator: def
# __init__(self, n: int) -> None: """Prepare to yield a
# random permutation of [0, n-1], one value per
```

```
# generate()."""  ... def generate(self) -> int: """Return
# a uniformly random not-yet-returned value in O(1).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Add reset() so all n values are available again in
# O(1).
#
# INTERVIEW NOTES
# Keep the values that are still available in the first
# `size` slots of an array. To draw, pick a random slot
# among them, hand out its value, swap it with the last
# active slot and shrink the active region: a partial
# Fisher-Yates shuffle, O(1) per call. Resetting is just
# setting size back to n, because the array still holds
# every value (in some order, which is as good as any).

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import random


# ========================================================
# MAIN QUESTION
# ========================================================
```

```
# ---- main question: main function ----
# Idea (partial Fisher-Yates): keep the numbers 0..n-1 in an array and
# an "active region" at its start that holds the numbers not returned
# yet. To generate: pick a random position in the active region, swap
# that number to the END of the region, shrink the region by one and
# return the number that is now just outside it. O(1) per call.
# Complexity: Time generate O(1), Space O(n)
class RandomGenerator:
    def __init__(self, n, seed=None):
        self.n = n
        self.values = list(range(n))
        self.size = n                                   # the active region is values[0:size]
        self.rng = random.Random(seed)

    def generate(self):
        if self.size == 0:
            raise IndexError("all values have been generated")
        i = self.rng.randrange(self.size)
        self.size -= 1
        self.values[i], self.values[self.size] = self.values[self.size], self.values[i]
        return self.values[self.size]


# ---- main question: tests ----
def test_main_question():
    for seed in range(20):
        gen = RandomGenerator(6, seed=seed)
        values = []
        for i in range(6):
            values.append(gen.generate())
        assert sorted(values) == [0, 1, 2, 3, 4, 5]
        try:
            gen.generate()
            raise AssertionError("an empty pool should raise")
        except IndexError:
```

```
            pass


# ========================================================
# FOLLOW-UP 1: Add reset() so all n values are available again in O(1)
# ========================================================
# ---- follow-up 1: function ----
# Answer: set `size` back to n. The array still holds every value 0..n-1
# (only their ORDER changed), and the next draws are uniform over the
# active region, so nothing has to be rebuilt.
```

```
# Complexity: Time reset O(1), generate O(1), Space O(n)
class ResettableGenerator(RandomGenerator):
    def reset(self):
        self.size = self.n


# ---- follow-up 1: tests ----
def test_follow_up_1():
    gen = ResettableGenerator(4, seed=7)
    for i in range(4):
        gen.generate()
    gen.reset()
    values = []
    for i in range(4):
        values.append(gen.generate())
    assert sorted(values) == [0, 1, 2, 3]


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 178 of 197 | topic 37 of 43: Random pick without repeats: swap the chosen item with the last one and pop | 3 of 4

Template fit: the same pool, plus a deque of the last k songs; the played song leaves the pool and returns to it when it drops off the deque | * asked 379 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 178 of 197

Random Song Shuffler With a No-Repeat Window

ASKED 2025-10-06 | 379 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 46.6 | 30 DAYS 0 | PRIORITY 87/288

ABOUT THIS QUESTION

Topic: Random pick without repeats: swap the chosen item with the last one and pop

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #380 Insert Delete GetRandom O(1)

LeetCode match: LC 380 Insert Delete GetRandom O(1) (related: no-repeat window)

Frequency: Single report | Last asked: 2025-10-06 | Roles: SWE Intern

QUESTION

Design a shuffler that plays songs at random from a fixed playlist, subject to one rule: the last k songs played may not be replayed. In other words, a song becomes eligible again only after k other (distinct) plays have occurred since it was last played. Implement the class so that each call to play() runs in O(1) time (no scanning of the playlist). The intended technique is a valid_songs list of currently eligible songs plus a song_to_index dictionary, using swap-and-pop to move a just-played song out of the eligible region and to re-admit the song that is leaving the window.

Function signature

```
import random
class SongShuffler:
    def __init__(self, songs: list[str], k: int) -> None:
        """Initialize with the playlist and the no-repeat window size k."""
        ...
    def play(self) -> str:
        """Return a uniformly random eligible song in O(1); it must not be
        one of the last k songs returned. Update state so the window slides."""
        ...
```

Follow-up (as reported): The interviewer wanted the naive deque solution replaced by O(1). Show why swap-and-pop with an index map achieves constant time per play() .

EXAMPLES

Example 1	
Input	`SongShuffler(["A", "B", "C"], k=1); then play(), play(), play(), play()`
Output	`e.g. "B", "A", "C", "B"`
Why	`k=1 forbids repeating the single most recent song. "B" then "A" is fine (A != B). Then "C" (C != A). Then "B" is allowed again because the only forbidden song is the immediately previous one ("C"). Any sequence with no two adjacent equal songs is valid.`

QUESTION

Example 2	
Input	`SongShuffler(["X", "Y"], k=0); play(), play()`
Output	`e.g. "Y", "Y"`
Why	`With k=0 there is no restriction, so repeats are allowed.`

CONSTRAINTS

- 1 <= len(songs) <= 10**5 , song titles are distinct strings.
- 0 <= k < len(songs) (so at least one song is always eligible). play() returns any eligible song chosen uniformly at random; every returned song must not be among the previous k returned songs.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
from collections import deque
class ScanShuffler:
    def __init__(self, songs, k, seed=None):
        self.songs, self.k = list(songs), k
        self.recent = deque()
        self.rng = random.Random(seed)
    def play(self):
        choices = [
            s
            for s in self.songs
            if s not in self.recent
        ]
        song = self.rng.choice(choices)
        self.recent.append(song)
        if len(self.recent) > self.k:
            self.recent.popleft()
        return song
```

TIME	SPACE
O(n) per play	**O(n)**

WHY IT IS SLOW

Every play scans the whole playlist to find the songs not in the window.

OPTIMAL SOLUTION · Eligible array, index map and swap-and-pop

TIME	SPACE
O(1) per play	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is a plain deque or list not enough for O(1)?

A deque of the last k songs is fine for the window, but choosing a random eligible song needs a structure you can sample from and delete from in O(1). A list gives O(1) sampling; deleting from the middle is O(n) unless you swap the element with the last one and pop, which needs the index map to know where a song is. That pair is what makes play O(1).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Random Song Shuffler With a No-Repeat
# Window
#
# Design a shuffler that plays songs at random from a
# fixed playlist, subject to one rule: the last k songs
# played may not be replayed. In other words, a song
# becomes eligible again only after k other (distinct)
# plays have occurred since it was last played. Implement
# the class so that each call to play() runs in O(1) time
# (no scanning of the playlist). The intended technique is
# a valid_songs list of currently eligible songs plus a
# song_to_index dictionary, using swap-and-pop to move a
# just-played song out of the eligible region and to re-
# admit the song that is leaving the window. Function
# signature: import random class SongShuffler: def
# __init__(self, songs: list[str], k: int) -> None:
# """Initialize with the playlist and the no-repeat window
# size k.""" ... def play(self) -> str: """Return a
# uniformly random eligible song in O(1); it must not be
# one of the last k songs returned.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is a plain deque or list not enough for O(1)?
#
# INTERVIEW NOTES
# The songs allowed right now form a set that must support
# a uniform random pick, remove, and re-insert in O(1): an
# array of eligible songs plus a dictionary song -> index,
# using swap-and-pop for removal. The last k played songs
# wait in a queue; when a new song is played, the oldest
# one in the queue becomes eligible again.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import random


# ============================================================
# MAIN QUESTION
```

```
# ============================================================
# ---- main question: main function ----
# Idea: keep a list `valid` of the songs that may be played now, and a
# dictionary `index` with each song's place in that list. To play: pick a
# random index in `valid`. To REMOVE the played song in O(1), swap it
# with the LAST song of the list and pop (the swap-and-pop trick). The
# played song waits in a queue `recent` of the last k songs; when the
# queue is longer than k, the oldest song leaves it and is added back
# to `valid`. Every call is O(1).
# Complexity: Time play O(1), Space O(number of songs)
class SongShuffler:
    def __init__(self, songs, k, seed=None):
        self.valid = list(songs)                    # songs that may be played now
```

```
        self.index = {}                              # song -> its place in valid
        for i in range(len(self.valid)):
            self.index[self.valid[i]] = i
        self.recent = deque()                        # the last k songs played, oldest first
        self.k = k
        self.rng = random.Random(seed)

    def play(self):
        i = self.rng.randrange(len(self.valid))
        song = self.valid[i]
        last = self.valid[-1]
        self.valid[i] = last                          # swap-and-pop: the last song takes the hole
        self.index[last] = i
        self.valid.pop()
        del self.index[song]
        self.recent.append(song)
        if len(self.recent) > self.k:
            back = self.recent.popleft()              # this song may be played again
            self.index[back] = len(self.valid)
            self.valid.append(back)
        return song


# ---- main question: tests ----
def test_main_question():
    tiny = SongShuffler(["A", "B", "C"], 1, seed=3)
    plays = []
    for i in range(40):
        plays.append(tiny.play())
    for i in range(1, len(plays)):
        assert plays[i] != plays[i - 1]                       # never the song just played
    shuffler = SongShuffler(["A", "B", "C", "D", "E"], 3, seed=4)
```

```
    plays = []
    for i in range(200):
        plays.append(shuffler.play())
    for i in range(len(plays)):
        for back in range(1, 4):
            if i - back >= 0:
                assert plays[i] != plays[i - back]            # not one of the last 3 songs
    assert set(plays) == {"A", "B", "C", "D", "E"}            # every song gets played
    assert len(SongShuffler(["X", "Y"], 0, seed=1).valid) == 2


# ========================================================
# FOLLOW-UP 1: Why is a plain deque or list not enough for O(1)?
# ========================================================
# Answer: picking a random ELIGIBLE song means choosing a random index of
# a list that holds only eligible songs. In a plain list, removing the
# played song from the middle shifts everything after it (O(n)); a deque
# is fast at its ends but has no O(1) random access in the middle.
# Swap-and-pop on a list plus an index dictionary removes a song in
# O(1), and the queue `recent` only touches its two ends.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 179 of 197 | topic 37 of 43: Random pick without repeats: swap the chosen item with the last one and pop | 4 of 4

Template fit: the lazy version for huge n: a dict 'moved' stores only the swapped slots, so memory stays O(calls) | * asked 382 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 179 of 197

Unique Random Number Generator

ASKED 2025-10-03 | 382 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 14.7 | 30 DAYS 0 | PRIORITY 187/288

ABOUT THIS QUESTION

Topic: Random pick without repeats: swap the chosen item with the last one and pop

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 384 Shuffle an Array / lazy Fisher-Yates (related)

Frequency: Single report | Last asked: 2025-10-03 | Roles: SWE Intern

QUESTION

Design a class UniqueRandom initialized with an integer n . Each call to generate() returns a number drawn uniformly at random from [0, n-1] that has not been returned before, so across successive calls no value ever repeats. After all n distinct values have been produced, further calls return None . Every unused value must remain equally likely on each draw (no bias), and generate() must run in O(1) time per call.

Function signature

```
import random
class UniqueRandom:
    def __init__(self, n: int, seed: int | None = None) -> None:
        ...
    def generate(self) -> int | None:
        """Return a not-yet-returned number in [0, n-1], or None once all n are
exhausted."""
        ...
```

Follow-up (as reported): Use a partial Fisher-Yates shuffle: keep a virtual array 0..n-1 , and on the i -th call swap a random unused slot into position i using a sparse dict of moved entries, giving O(1) time and O(number-of- draws) space.

EXAMPLES

Example 1	
Input	`g = UniqueRandom(3); [g.generate(), g.generate(), g.generate()]`
Output	`a permutation of [0, 1, 2] (e.g. [2, 0, 1])`
Why	`The three calls return all of 0,1,2 in some random order with no repeats.`

Example 2	
Input	`(continuing above) g.generate()`
Output	`None`
Why	`All 3 values already returned, so the pool is exhausted.`

QUESTION

Example 3	
Input	`g = UniqueRandom(1); [g.generate(), g.generate()]`
Output	`[0, None]`
Why	`The only value 0 is returned once, then the generator is exhausted.`

CONSTRAINTS

- 1 <= n <= 10^7 .
- Support an optional seed for reproducible tests.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
import random
class ShuffledPool:
    def __init__(self, n, seed=None):
        self.items = list(range(n))
        random.Random(seed).shuffle(self.items)
    def generate(self):
        return (
            self.items.pop()
            if self.items
            else:
                None
        )
```

TIME	SPACE
O(n)	**O(n)**
setup, O(1) per call	

WHY IT IS SLOW

It stores all n values even if only a few are ever drawn.

OPTIMAL SOLUTION · Lazy Fisher-Yates with a dictionary

TIME	SPACE
O(1)	**O(draws)**
per call	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Add a reset() so the same generator can run again.

Clear the dictionary and set the draw counter back to zero: O(draws) once, and all n values are available again.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Unique Random Number Generator
#
# Design a class UniqueRandom initialized with an integer
```

```
# n . Each call to generate() returns a number drawn
# uniformly at random from [0, n-1] that has not been
# returned before, so across successive calls no value
# ever repeats. After all n distinct values have been
# produced, further calls return None . Every unused value
# must remain equally likely on each draw (no bias), and
# generate() must run in O(1) time per call. Function
# signature: import random class UniqueRandom: def
# __init__(self, n: int, seed: int | None = None) -> None:
# ... def generate(self) -> int | None: """Return a not-
# yet-returned number in [0, n-1], or None once all n are
# exhausted.""" ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Add a reset() so the same generator can run again.
#
# INTERVIEW NOTES
# Drawing without repeats is the Fisher-Yates shuffle done
# lazily: on draw i pick a random slot j in [i, n - 1],
# return the value that lives at j, and move the value at
# i into slot j. Only the slots that have been touched
# need storing, so a dictionary of moved values gives O(1)
# per draw and O(draws) memory even for a huge n.

# =====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# =====================================================

import random


# =====================================================
# MAIN QUESTION
# =====================================================
# ---- main question: main function ----
# Idea (a lazy Fisher-Yates shuffle): imagine the numbers 0..n-1 in an
# array and shuffle it as you go. Draw number `drawn` by picking a random
# slot j from drawn..n-1; the value in slot j is the answer. Then the
# value that sat in slot `drawn` takes slot j's place. Only the slots
# that were MOVED are stored (in a dictionary); an untouched slot
# j simply holds the value j. O(1) per call, and O(draws) memory.
# Complexity: Time generate O(1), Space O(number of draws so far)
class UniqueRandom:
    def __init__(self, n, seed=None):
        self.n = n
        self.drawn = 0
        self.moved = {}                             # slot -> the value now sitting there
        self.rng = random.Random(seed)

    def generate(self):
        if self.drawn == self.n:
            return None                             # every number has been returned
        i = self.drawn
        j = self.rng.randrange(i, self.n)
        value = j
        if j in self.moved:
            value = self.moved[j]
        at_i = i
        if i in self.moved:
            at_i = self.moved[i]
        self.moved[j] = at_i                        # the value from slot i takes the place of the drawn o
```

```
ne
        self.drawn += 1
        return value


# ---- main question: tests ----
def test_main_question():
    for seed in range(20):
        gen = UniqueRandom(6, seed=seed)
        values = []
        for i in range(6):
            values.append(gen.generate())
        assert sorted(values) == [0, 1, 2, 3, 4, 5]          # no repeats, nothing missing
        assert gen.generate() is None


# =====================================================
# FOLLOW-UP 1: Add a reset() so the same generator can run again
# =====================================================
# ---- follow-up 1: function ----
# Answer: forget the moved slots and set the draw counter back to zero.
# All n values are available again (the dictionary describes only the
# moves, so clearing it restores 0..n-1 in order).
# Complexity: Time reset O(number of draws so far), generate O(1), Space O(draws)
```

```
class ResettableUniqueRandom(UniqueRandom):
    def reset(self):
        self.moved.clear()
        self.drawn = 0


# ---- follow-up 1: tests ----
def test_follow_up_1():
    gen = ResettableUniqueRandom(4, seed=2)
    first = []
    for i in range(4):
        first.append(gen.generate())
    gen.reset()
    second = []
    for i in range(4):
        second.append(gen.generate())
    assert sorted(first) == [0, 1, 2, 3] and sorted(second) == [0, 1, 2, 3]


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 38 OF 43

Build and serialize: write a structure out, read it back with the same walk

3 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Build and serialize: write a structure out, read it back with the same walk | 3 questions | tap a title to jump to it

READ FIRST: the code template for these 3 questions (tap here) >

M 180. Print a Directory Tree — **ColorStack * 2025-09-21 ***

M 181. Construct a binary tree given its preorder and inorder traversals. — **ND**

M 182. Write functions to serialize and deserialize a list of strings. — **ND**

CODE TEMPLATE | GROUP 38 OF 43 | 3 QUESTIONS, ONE PATTERN

Build and serialize: write a structure out, read it back with the same walk

The one idea these questions share

Encoding is one walk that writes tokens; decoding is the same walk reading tokens back (an iterator or an index). Put a length or a child count before the data so the reader always knows how much to take.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def encode(node, out):                      # CHANGE 1: what you write per node
    if node is None:
        out.append("#")
        return
    out.append(str(node.val))
    encode(node.left, out)
    encode(node.right, out)


def decode(tokens):
    it = iter(tokens)

    def build():
        tok = next(it)
        if tok == "#":
            return None
        node = Node(int(tok))               # CHANGE 2: build the node from what you read
        node.left = build()
        node.right = build()
        return node

    return build()
```

2. Helper functions that usually get written next to it

```
class Node:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def read_tokens(text):                      # the reader side of a length-prefixed format
    i, out = 0, []
    while i < len(text):
        j = text.index("#", i)
        n = int(text[i:j])
        out.append(text[j + 1:j + 1 + n])
        i = j + 1 + n
    return out
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
180	Print a Directory Tree	build nested dicts from the path parts, then print them with a recursive walk that indents by depth
181	Construct a binary tree given its preorder and inorder traversals.	the first preorder value is the root; its position in a value-to-index map of the inorder list splits left from right
182	Write functions to serialize and deserialize a list of strings.	write len + '#' + text for each string; read the length, then take exactly that many characters

Contents Start the questions >

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 180 of 197 | topic 38 of 43: Build and serialize: write a structure out, read it back with the same walk | 1 of 3

Template fit: build nested dicts from the path parts, then print them with a recursive walk that indents by depth | * asked 394 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 180 of 197

Print a Directory Tree

ASKED 2025-09-21 | 394 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 219/288

ABOUT THIS QUESTION

Topic: Build and serialize: write a structure out, read it back with the same walk

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #588 Design In-Memory File System

LeetCode match: LC 588 Design In-Memory File System (related: tree from paths)

Frequency: Single report | Last asked: 2025-09-21 | Roles: SWE Intern

QUESTION

You are given a list of file/directory paths , each a '/' -separated string (for example "dir/subdir/file.txt"). Reproduce what a Unix directory listing tool prints: build the directory hierarchy implied by the paths and return it as a single newline-separated string. Rules: Each path component is a node. A component that appears under the same parent in several paths must be printed only once. Indent each node with 2 * depth spaces, where the top-level components have depth 0 . Children are printed in the order they are first encountered across the input list (preserve first-seen order; do not sort). The output must not have a trailing newline.

Function signature

```
def print_directory_tree(paths: list[str]) -> str:
    ...
```

Follow-up (as reported): Add box-drawing connectors (, ,) like the real tree command. Hint: track, per level, whether the current node is the last child of its parent.

EXAMPLES

Example 1	
Input	`paths = ["a/b/c", "a/b/d", "a/e"]`
Output	`"a\n b\n c\n d\n e"`
Why	`Rendered: a b c d e`

Example 2	
Input	`paths = ["src/main.py", "src/util.py", "README.md"]`
Output	`"src\n main.py\n util.py\nREADME.md"`
Why	`"src" and its two files come first (first-seen order), then the top-level "README.md".`

Example 3	
Input	`paths = []`
Output	`""`

CONSTRAINTS

- 0 <= len(paths) <= 10^4
- Each path is non-empty, has no leading/trailing '/' , and no empty components.

QUESTION

- If paths is empty, return the empty string "" .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_tree(paths):
    root = []
    for path in paths:
        level = root
        for part in path.split(/):
            for node in level:
                if node[0] == part:
                    break
            else:
                node = [part, []]
                level.append(node)
            level = node[1]
    lines = []
    def walk(level, depth):
        for name, kids in level:
            lines.append("  " * depth + name)
            walk(kids, depth + 1)
    walk(root, 0)
    return "\n".join(lines)
```

TIME	SPACE
O(total length * width)	O(total length)

WHY IT IS SLOW

Every component searches the list of children of its parent one by one.

OPTIMAL SOLUTION · Nested dictionaries and a DFS

TIME	SPACE
O(total length)	O(total length)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Print it with box-drawing connectors like the real tree command.

Carry a prefix down the recursion. For each child choose the connector: the last child gets the corner and its subtree continues with blank space, the others get the tee and their subtree continues with a vertical bar.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Print a Directory Tree
#
# You are given a list of file/directory paths , each a
# '/' -separated string (for example "dir/subdir/file.txt"
# ). Reproduce what a Unix directory listing tool prints:
# build the directory hierarchy implied by the paths and
# return it as a single newline-separated string. Rules:
```

```
# Each path component is a node. A component that appears
# under the same parent in several paths must be printed
# only once. Indent each node with 2 * depth spaces, where
# the top-level components have depth 0 . Children are
# printed in the order they are first encountered across
# the input list (preserve first-seen order; do not sort).
# The output must not have a trailing newline. Function
# signature: def print_directory_tree(paths: list[str]) ->
# str: ... Follow-up (as reported): Add box-drawing
# connectors ( , , ) like the real tree command.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Print it with box-drawing connectors like the real
# tree command.
#
# INTERVIEW NOTES
# The paths describe a tree where shared prefixes merge.
# Insert every path into a nested dictionary (a trie of
# path components; Python dicts keep insertion order,
# which gives the first-seen order for free), then print
# it with a depth-first walk that indents two spaces per
# level.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def build_dirs(paths):
    # nested dictionaries: every folder or file name maps to its own contents
    root = {}
    for path in paths:
        node = root
        for part in path.split("/"):
            if part not in node:
                node[part] = {}
            node = node[part]
    return root


# ---- main question: main function ----
# Idea: put every path into a dictionary of dictionaries (a tree), then
# print it depth first with two spaces per level.
# Complexity: Time O(n) for n entries, Space O(depth) for the walk plus the output
def print_directory_tree(paths):
    root = build_dirs(paths)
    lines = []

    def walk(node, depth):
        for name in node:
            lines.append("  " * depth + name)
            walk(node[name], depth + 1)

    walk(root, 0)
    return "\n".join(lines)


# ---- main question: tests ----
```

```
def test_main_question():
    assert print_directory_tree(["a/b/c", "a/b/d", "a/e"]) == "a\n  b\n    c\n    d\n  e"
    assert print_directory_tree(["src/main.py", "src/util.py", "README.md"]) == "src\n  main.py\n  util.py\
nREADME.md"
    assert print_directory_tree([]) == ""


# ============================================================
# FOLLOW-UP 1: Print it with box-drawing connectors like the real tree command
# ============================================================
# ---- follow-up 1: function ----
# Answer: carry a prefix down the recursion. Every child gets a
# connector: the LAST child gets the corner and its contents continue
# with blank space; the others get a tee and their contents continue
# with a vertical bar.
# Complexity: Time O(n), Space O(depth) for the walk plus the output
def tree_with_connectors(paths):
    root = build_dirs(paths)
    lines = []
    tee = "\u251c\u2500\u2500 "
```

```
    corner = "\u2514\u2500\u2500 "
    bar = "\u2502   "

    def walk(node, prefix):
        names = list(node)
        for i in range(len(names)):
            is_last = i == len(names) - 1
            if is_last:
                lines.append(prefix + corner + names[i])
                walk(node[names[i]], prefix + "    ")
            else:
                lines.append(prefix + tee + names[i])
                walk(node[names[i]], prefix + bar)

    walk(root, "")
    return "\n".join(lines)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    tee = "\u251c\u2500\u2500 "
    corner = "\u2514\u2500\u2500 "
    assert tree_with_connectors(["a/b", "a/c"]) == corner + "a\n    " + tee + "b\n    " + corner + "c"


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 181 of 197 | topic 38 of 43: Build and serialize: write a structure out, read it back with the same walk | 2 of 3

Template fit: the first preorder value is the root; its position in a value-to-index map of the inorder list splits left from right | ND

MEDIUM | IN APP LC 105 | Exponent | 181 of 197

Construct a binary tree given its preorder and inorder traversals.

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 38.3 30 DAYS 0 PRIORITY 111/288

ABOUT THIS QUESTION

Topic: Build and serialize: write a structure out, read it back with the same walk

Optimal answer from: your app's interview approach (LeetCode #105)

LeetCode match: LC 105 Construct Binary Tree from Preorder and Inorder Traversal

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +26 interviews

Original: https://www.aced.io/questions/3417/construct-binary-tree-preorder-inorder-traversal

QUESTION

Given two integer arrays, preorder and inorder, which represent the pre-order and in-order traversal of a binary tree, construct and return the binary tree. Pre-order traversal visits nodes in this order: root -> left subtree -> right subtree In-order traversal visits nodes in this order: left subtree -> root -> right subtree You need to construct the binary tree and return its root node. Examples preorder = [3, 9, 20, 15, 7], inorder = [9, 3, 15, 20, 7] output: [3, 9, 20, null, null, 15, 7] Explanation: Given the two arrays, the binary tree constructed is: 3 / \ 9 20 / \ 15 7 preorder = [1, 2], inorder = [2, 1] output: [1, 2]
Explanation: Given the two arrays, the binary tree constructed is: 1 / 2 Constraints: The number of nodes in the tree is in the range [1, 3000]. preorder and inorder consist of unique integers. The elements of inorder will always be the same as the elements of preorder but in a different order.

EXAMPLES

Example 1	
Input	`preorder = [3, 9, 20, 15, 7]` `inorder = [9, 3, 15, 20, 7]`
Output	`values(build_pre_in([3, 9, 20, 15, 7], [9, 3, 15, 20, 7])) -> [3, 9, 20, null, null, 15, 7]`
Why	`The first preorder value 3 is the root; in the inorder list 9 is left of it and 15, 20, 7 are right. The tree is shown level by level.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def build(pre, inord):
    if not pre:
        return None
    root = Node(pre[0])
    i = inord.index(pre[0])  # linear search
    root.left = build(pre[1:i + 1], inord[:i])
    root.right = build(pre[i + 1:],
                       inord[i + 1:])
    return root
```

BRUTE FORCE

TIME	SPACE
O(n^2)	O(n^2) for the slices

WHY IT IS SLOW

The linear search and the list copies make a skewed tree O(n^2) in time and space. A dictionary of inorder positions with index bounds is O(n).

OPTIMAL SOLUTION · Hash map of inorder positions and index bounds

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 You are given postorder and inorder instead.

The root is the last postorder element, and because postorder reads left, right, root, reading it backwards yields root, right, left, so build the right subtree before the left.

FOLLOW-UP 2 Values can repeat.

Then the tree is not unique and the inorder index of the root is ambiguous. Ask for a rule (or for a preorder with null markers). Preorder plus postorder alone also cannot determine a general binary tree.

FOLLOW-UP 3 The tree is a 100,000-node chain.

The recursion is as deep as the tree. Use an explicit stack: push nodes while walking preorder, and pop back to the parent when the next node's inorder index shows you have finished the left side.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Construct a binary tree given its
# preorder and inorder traversals.
#
# Given two integer arrays, preorder and inorder, which
# represent the pre-order and in-order traversal of a
# binary tree, construct and return the binary tree. Pre-
# order traversal visits nodes in this order: root -> left
# subtree -> right subtree In-order traversal visits nodes
# in this order: left subtree -> root -> right subtree You
# need to construct the binary tree and return its root
# node. Examples preorder = [3, 9, 20, 15, 7], inorder =
# [9, 3, 15, 20, 7] output: [3, 9, 20, null, null, 15, 7]
# Explanation: Given the two arrays, the binary tree
# constructed is: 3 / \ 9 20 / \ 15 7 preorder = [1, 2],
# inorder = [2, 1] output: [1, 2] Explanation: Given the
# two arrays, the binary tree constructed is: 1 / 2
# Constraints: The number of nodes in the tree is in the
# range [1, 3000]. preorder and inorder consist of unique
# integers.
#
```

```
# FOLLOW-UPS (answered below, in this order):
# 1. You are given postorder and inorder instead.
# 2. Values can repeat.
# 3. The tree is a 100,000-node chain.
#
# INTERVIEW NOTES
# Preorder tells you the root (its first element), and
# inorder tells you how many nodes go left (everything
# before the root in inorder). That cut gives two
# independent smaller problems. The only expensive step is
# locating the root in inorder, which a hash map of value
# to index makes O(1).

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


def preorder_of(root):
    # the values in pre-order (root, left, right), with a stack
    values = []
    stack = []
    if root:
        stack.append(root)
    while stack:
        node = stack.pop()
        values.append(node.val)
        if node.right:
            stack.append(node.right)
        if node.left:
            stack.append(node.left)
    return values


def inorder_of(root):
    # the values in in-order (left, root, right), with a stack
    values = []
    stack = []
    node = root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        values.append(node.val)
        node = node.right
    return values


def positions(values):
    # value -> its index (the values are all different)
```

```
    where = {}
    for i in range(len(values)):
        where[values[i]] = i
    return where
```

```
# ---- main question: main function ----
# Idea: preorder is "root, left part, right part", so its next unused
# value is always the next ROOT. That root's position in the inorder
# list splits the inorder list into the left part (before it) and the
# right part (after it). Build the left subtree first, then the right.
# Complexity: Time O(n) (a dictionary gives each value's inorder position), Space O(n)
def build_pre_in(preorder, inorder):
    where = positions(inorder)
    next_root = 0

    def build(low, high):
        # build the tree for inorder[low:high]
        nonlocal next_root
        if low >= high:
            return None
        root = TreeNode(preorder[next_root])
        next_root += 1
        middle = where[root.val]
        root.left = build(low, middle)
        root.right = build(middle + 1, high)
        return root

    return build(0, len(inorder))


# ---- main question: tests ----
PRE = [3, 9, 20, 15, 7]
INO = [9, 3, 15, 20, 7]
POST = [9, 15, 7, 20, 3]


def test_main_question():
    tree = build_pre_in(PRE, INO)
    assert preorder_of(tree) == PRE
    assert inorder_of(tree) == INO
    assert build_pre_in([], []) is None


# ======================================================
# FOLLOW-UP 1: You are given postorder and inorder instead
# ======================================================
# ---- follow-up 1: function ----
# Answer: the root is the LAST postorder value. Postorder is "left, right,
# root", so reading it backwards gives root, right, left: build the
# RIGHT subtree before the left one.
```

```
# Complexity: Time O(n), Space O(n)
def build_post_in(inorder, postorder):
    where = positions(inorder)
    next_root = len(postorder) - 1

    def build(low, high):
        nonlocal next_root
        if low >= high:
            return None
        root = TreeNode(postorder[next_root])
        next_root -= 1
        middle = where[root.val]
        root.right = build(middle + 1, high)
```

```
        root.left = build(low, middle)
        return root

    return build(0, len(inorder))


# ---- follow-up 1: tests ----
def test_follow_up_1():
    tree = build_post_in(INO, POST)
    assert preorder_of(tree) == PRE
    assert inorder_of(tree) == INO


# =========================================================
# FOLLOW-UP 2: Values can repeat
# =========================================================
# Answer: then the traversals alone do not determine the tree (the
# value-to-position table is ambiguous). Ask for extra information, such
# as the preorder together with the sizes of the left subtrees.


# =========================================================
# FOLLOW-UP 3: The tree is a 100,000-node chain
# =========================================================
# ---- follow-up 3: function ----
# Answer: the recursion is as deep as the tree, so use your own stack.
# Go through the preorder values; a new node is the LEFT child of the top
# of the stack until the top's value equals the next inorder value (its
# left side is finished). Then pop back to the node this one is the right
# child of.
# Complexity: Time O(n), Space O(n)
def build_pre_in_iterative(preorder, inorder):
```

```
    if not preorder:
        return None
    root = TreeNode(preorder[0])
    stack = [root]
    in_index = 0
    for i in range(1, len(preorder)):
        node = TreeNode(preorder[i])
        if stack[-1].val != inorder[in_index]:
            stack[-1].left = node                  # the left side of the top node is not finished
        else:
            parent = None
            while stack and stack[-1].val == inorder[in_index]:
                parent = stack.pop()
                in_index += 1
            parent.right = node
        stack.append(node)
    return root


# ---- follow-up 3: tests ----
def test_follow_up_3():
    tree = build_pre_in_iterative(PRE, INO)
    assert preorder_of(tree) == PRE
    assert inorder_of(tree) == INO
    assert build_pre_in_iterative([], []) is None
    chain = list(range(50000))                     # a very deep chain: no recursion
    reverse = list(range(49999, -1, -1))
    deep = build_pre_in_iterative(chain, reverse)
    assert preorder_of(deep) == chain


# =========================================================
```

```
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given two integer arrays, preorder and inorder, which represent the pre-order and in-order traversal of a binary tree, construct and return the binary tree. Pre-order traversal visits nodes in this order: root -> left subtree -> right subtree In-order traversal visits nodes in this order: left subtree -> root -> right subtree You need to construct the binary tree and return its root node. Examples

```
preorder = [3, 9, 20, 15, 7], inorder = [9, 3, 15, 20, 7]
output: [3, 9, 20, null, null, 15, 7]
Explanation:
Given the two arrays, the binary tree constructed is:
       3
      / \
     9   20
        /  \
       15   7
```

```
preorder = [1, 2], inorder = [2, 1]
output: [1, 2]
Explanation:
Given the two arrays, the binary tree constructed is:
    1
   /
  2
```

OFFICIAL PAGE TEXT

Constraints: The number of nodes in the tree is in the range [1, 3000]. preorder and inorder consist of unique integers. The elements of inorder will always be the same as the elements of preorder but in a different order. Our solution to construct a binary tree from pre-order and in-order traversal arrays uses a recursive approach. We first identify the root node from the pre-order array, then split the in-order array into left and right subtrees based on the root's position in the in-order array. We continue this process recursively to build the entire binary tree. We start by picking the first element of the preorder array as the root of the tree. The index of this root element in the inorder array helps us divide the tree into left and right subtrees. The elements to the left of the root in the inorder array form the left subtree, and the elements to the right form the right subtree. We then recursively build the left and right subtrees using the next available elements in the preorder array. To ensure efficient lookups of the root's index in the inorder array, we create a hash map (inorder_index_map) that maps values to their indices in the inorder array. This allows us to retrieve the position of the root in constant time. NoteRecursion: Base Case: If there are no elements to construct the tree (i.e., the preorder or inorder array is empty), we return nullptr. Recursive Step: We extract the root from the preorder array and find its position in the inorder array using the hash map. This splits the inorder array into left and right subtrees. We then recursively build the left and right subtrees and attach them to the root. By maintaining the correct indices for both the pre-order and in-order arrays, we can recursively construct the entire tree. Time Complexity: The solution has a time complexity of O(n), where n is the number of elements in the tree. This is because we traverse both the preorder and inorder arrays once, and we use a hash map for constant-time lookups of root indices in the inorder array. Space Complexity: The solution uses O(n) space for the hash map that stores the indices of elements in the inorder array, and the recursion stack requires O(h) space, where h is the height of the tree. In the worst case (a skewed tree), the height of the tree can be O(n), so the total space complexity is O(n).No answers contributed by the community yet.

STUDY PATH: question 182 of 197 | topic 38 of 43: Build and serialize: write a structure out, read it back with the same walk | 3 of 3

Template fit: write len + '#' + text for each string; read the length, then take exactly that many characters | ND

MEDIUM | IN APP LC 271 | Exponent | 182 of 197

Write functions to serialize and deserialize a list of strings.

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TALK ALL TIME 28.2 30 DAYS 0 PRIORITY 149/288

ABOUT THIS QUESTION

Topic: Build and serialize: write a structure out, read it back with the same walk

Optimal answer from: your app's interview approach (LeetCode #271)

LeetCode match: LC 271 Encode and Decode Strings

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age) | 2 interviews

Original: https://www.aced.io/questions/1307/serialize-deserialize-list-strings

QUESTION

Serialize and Deserialize a List of Strings. You are tasked with writing two functions: serialize and deserialize. The serialize function will take a list of strings as input and return a single string. The deserialize function will take this single string and reconstruct the original list of strings. Details The serialize function should concatenate all strings in the list into a single string, with each original string's length encoded as a single character prefix. The deserialize function should be able to take the single string and accurately split it back into the original list of strings, using the encoded length characters as guides. For example, consider a list ["hello", "world"]. The serialization of this list might be "\x05hello\x05world", where \x05 indicates that the following string has a length of 5 characters. Constraints The maximum length of any string is 65535 (0xFFFF). If a string's length exceeds the maximum, the serialize function should raise an exception. The deserialize function should accurately handle any edge cases, such as empty strings or single-character strings. Assume that all strings consist of ASCII characters only. Examples lst = ['hello', 'world'] serialized = serialize(lst)

serialized output

: "\x05hello\x05world" deserialized = deserialize(serialized)

deserialized output should be

: ['hello', 'world'] lst = [''] serialized = serialize(lst)

serialized output

: "\x00" deserialized = deserialize(serialized)

deserialized output should be

EXAMPLES

Example 1	

Input	`items = ['hello', 'wor#ld', '']`
	`serialize(items)`
	`deserialize(serialize(items)) == items`

Output	`serialize(items) -> '5#hello6#wor#ld0#'` `deserialize(serialize(items)) == items -> true`
Why	`Each string is written as its length, a #, and its text, so a # inside a string cannot confuse the reader.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def serialize(items):
    return ",".join(items)
def deserialize(text):
    return text.split(",")  # breaks on commas
```

TIME	SPACE
O(total length)	O(total length)

WHY IT IS SLOW

It is fast (O(total length)) but wrong: a string that contains the delimiter breaks it, and an empty list cannot be told from a list of one empty string. Writing each string's length before it fixes both.

OPTIMAL SOLUTION · Length-prefix every string

TIME	SPACE
O(total length)	O(total length)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Use escaping instead of a length prefix.

Escape the separator and the escape character inside each string, then terminate each item with the unescaped separator. The decoder tracks whether the previous character was an escape. It is streamable, while the length prefix can be read in O(1) jumps.

FOLLOW-UP 2 The strings are long or binary.

Use bytes and a fixed-width length (for example a 4-byte big-endian integer from struct.pack). Fixed-width lengths avoid the digit-scanning loop and work for any byte value.

FOLLOW-UP 3 Just use JSON?

Yes if the format is free to choose: json.dumps handles quoting and Unicode correctly. Mention it as the practical answer, but code the length-prefix version when the interviewer wants to see you handle the delimiter problem yourself.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Write functions to serialize and
# deserialize a list of strings.
#
# Serialize and Deserialize a List of Strings. You are
# tasked with writing two functions: serialize and
# deserialize. The serialize function will take a list of
# strings as input and return a single string. The
# deserialize function will take this single string and
# reconstruct the original list of strings. Details The
# serialize function should concatenate all strings in the
# list into a single string, with each original string's
```

```
# length encoded as a single character prefix. The
# deserialize function should be able to take the single
# string and accurately split it back into the original
# list of strings, using the encoded length characters as
# guides. For example, consider a list ["hello", "world"].
# The serialization of this list might be
# "\x05hello\x05world", where \x05 indicates that the
# following string has a length of 5 characters.
# Constraints The maximum length of any string is 65535
# (0xFFFF).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Use escaping instead of a length prefix.
# 2. The strings are long or binary.
# 3. Just use JSON?
#
# INTERVIEW NOTES
# Any text can appear inside a string, so a delimiter is
# never safe by itself. The fix is to say how long each
# string is before you write it (length prefix), so the
# decoder never has to look for a separator inside the
# data. Always check two corner cases: the empty list and
# a list containing the empty string.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import json
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main functions ----
# Idea: write the LENGTH of every string before it: "<length>#<text>".
# The reader knows exactly how many characters to take, so the text may
# contain any character, even "#" and digits.
# Complexity: Time O(total characters), Space O(total characters)
def serialize(items):
    text = ""
    for s in items:
        text += str(len(s)) + "#" + s
    return text


# Complexity: Time O(total characters), Space O(total characters)
def deserialize(text):
    items = []
    i = 0
    while i < len(text):
        sep = text.index("#", i)               # the digits of the length end at the next "#"
        size = int(text[i:sep])
        start = sep + 1
        items.append(text[start:start + size])
        i = start + size
    return items


# ---- main question: tests ----
TRICKY = ["", "a", "12#x", "hello, world", "\\", ",,", "\u00e9\u4e16\u754c", "3#abc", "\n"]
```

```
def test_main_question():
    assert serialize(["ab", "c"]) == "2#ab1#c"
    for data in ([], [""], ["a"], TRICKY):
        assert deserialize(serialize(data)) == data


# ======================================================
# FOLLOW-UP 1: Use escaping instead of a length prefix
# ======================================================
# ---- follow-up 1: function ----
# Answer: write a backslash before every comma and every backslash
# inside a string, and end each item with a plain comma. The reader
# remembers whether the previous character was a backslash.
```

```
# Complexity: Time O(total characters), Space O(total characters)
def serialize_escaped(items):
    text = ""
    for s in items:
        for ch in s:
            if ch == "\\" or ch == ",":
                text += "\\"
            text += ch
        text += ","
    return text


# Complexity: Time O(total characters), Space O(total characters)
def deserialize_escaped(text):
    items = []
    current = ""
    escaped = False
    for ch in text:
        if escaped:
            current += ch                       # the character after a backslash is taken literally
            escaped = False
        elif ch == "\\":
            escaped = True
        elif ch == ",":
            items.append(current)
            current = ""
        else:
            current += ch
    if current != "" or escaped:
        raise ValueError("truncated input")
    return items


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert serialize_escaped(["a,b"]) == "a\\,b,"
    assert serialize_escaped([""]) == ","
    assert serialize_escaped([]) == ""
    for data in ([], [""], ["a"], TRICKY):
        assert deserialize_escaped(serialize_escaped(data)) == data
    try:
        deserialize_escaped("abc")
        raise AssertionError("should fail")
    except ValueError:
        pass
```

```
# ======================================================
# FOLLOW-UP 2: The strings are long or binary
```

```
# =====================================================
# ---- follow-up 2: helper functions ----
def length_to_bytes(n):
    # a fixed 4-byte length, most significant byte first (big-endian)
    return bytes([n // 16777216 % 256, n // 65536 % 256, n // 256 % 256, n % 256])


def length_from_bytes(data, i):
    return data[i] * 16777216 + data[i + 1] * 65536 + data[i + 2] * 256 + data[i + 3]


# ---- follow-up 2: function ----
# Answer: work with bytes and a fixed-width length in front of every
# item. There are no digits to scan, and any byte value (even 0) is allowed.
# Complexity: Time O(total bytes), Space O(total bytes)
def serialize_bytes(items):
    out = b""
    for s in items:
        raw = s
        if isinstance(s, str):
            raw = s.encode("utf-8")
        out += length_to_bytes(len(raw)) + raw
    return out


# Complexity: Time O(total bytes), Space O(total bytes)
def deserialize_bytes(data):
    items = []
    i = 0
    while i < len(data):
        size = length_from_bytes(data, i)
        items.append(data[i + 4:i + 4 + size])
        i += 4 + size
    return items


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert serialize_bytes(["a"]) == b"\x00\x00\x00\x01a"
    for data in ([], [""], ["a"], TRICKY):
        raw = deserialize_bytes(serialize_bytes(data))
```

```
        decoded = []
        for item in raw:
            decoded.append(item.decode("utf-8"))
        assert decoded == data
    blob = [bytes([0, 255, 10])]                # binary data that is not text works too
    assert deserialize_bytes(serialize_bytes(blob)) == blob


# =====================================================
# FOLLOW-UP 3: Just use JSON?
# =====================================================
# ---- follow-up 3: function ----
# Answer: yes, if you are free to choose the format: json handles
# quoting and Unicode correctly. (Say what you gave up: you did not show
# that you can design a format yourself.)
# Complexity: Time O(total characters), Space O(total characters)
def serialize_json(items):
    return json.dumps(items)


# Complexity: Time O(total characters), Space O(total characters)
def deserialize_json(text):
    return json.loads(text)
```

```
# ---- follow-up 3: tests ----
def test_follow_up_3():
    for data in ([], [""], ["a"], TRICKY):
        assert deserialize_json(serialize_json(data)) == data


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Serialize and Deserialize a List of Strings. You are tasked with writing two functions: serialize and deserialize. The serialize function will take a list of strings as input and return a single string. The deserialize function will take this single string and reconstruct the original list of strings. Details The serialize function should concatenate all strings in the list into a single string, with each original string's length encoded as a single character prefix. The deserialize function should be able to take the single string and accurately split it back into the original list of strings, using the encoded length characters as guides. For example, consider a list ["hello", "world"]. The serialization of this list might be "\x05hello\x05world", where \x05 indicates that the following string has a length of 5 characters. Constraints The maximum length of any string is 65535 (0xFFFF). If a string's length exceeds the maximum, the serialize function should raise an exception. The deserialize function should accurately handle any edge cases, such as empty strings or single-character strings. Assume that all strings consist of ASCII characters only. Examples

```
lst = ['hello', 'world']
serialized = serialize(lst)
# serialized output: "\x05hello\x05world"
deserialized = deserialize(serialized)
# deserialized output should be: ['hello', 'world']

lst = ['']
serialized = serialize(lst)
# serialized output: "\x00"
deserialized = deserialize(serialized)
# deserialized output should be: ['']

lst = ['a', 'bc', 'def']
serialized = serialize(lst)
# serialized output: "\x01a\x02bc\x03def"
deserialized = deserialize(serialized)
# deserialized output should be: ['a', 'bc', 'def']
```

OFFICIAL PAGE TEXT

Our solution serializes and deserializes a list of strings by encoding the length of each string as a single character and concatenating it with the original string. The serialize function processes the input list of strings by iterating through each string. For every string, it first encodes the length of the string as a single character using the encodeNumber function. The encoded length is then appended to a result list, followed by the string itself. The result list is ultimately joined into a single string, which represents the serialized form of the list of strings. The deserialize function takes the serialized string and reconstructs the original list of strings. It starts by iterating through the serialized string, decoding the length of each string using the decodeNumber function, then extracting the string of that length from the serialized data. These strings are collected in a list and returned as the final deserialized result. Time Complexity: The time complexity of both serialize and deserialize is O(n), where n is the total number of characters across all strings in the input list. In serialize, we iterate through every string, performing constant-time operations (encoding the length and appending the string) for each string. Similarly, in deserialize, we iterate through the serialized string, decoding lengths and extracting substrings, which are also constant-time operations for each step. Space Complexity: The space complexity is O(n) for both functions since we store the serialized form of the input list (which has a size proportional to the total number of characters across all strings) and reconstruct the list in deserialize.Daniel answers "Write a pair of functions to serialize and deserialize a list of strings."No answers contributed by the community yet.Related questionsWrite a function to do Excel-style operations.Write a function to check whether a string contains a substring.Given a list of bounding boxes and annotations, write a function to compute Intersection over Union.Related coursesCourseCourseSoftware Engineering Interview PrepLand your dream software engineering role at Google, Amazon, Microsoft, Meta, Apple, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates—practice data structures, algorithms, system design, people management, behavioral interviews, and more.Start learningCourseCourseSystem Design InterviewsLearn how to answer the latest system design questions across product, infrastructure, and AI domains. Features in-depth video examples, written breakdowns, and helpful reference patterns. Watch senior engineers and managers answer these questions in mock interview videos.Start learningPythonJavaScriptJavaC++PythonEditorShareLoading...OutputTest ResultsConsoleSign up to run code

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 39 OF 43

Games: win or lose with a cache, and guessing with feedback

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Games: win or lose with a cache, and guessing with feedback | 1 questions | tap a title to jump to it

READ FIRST: the code template for these 1 questions (tap here) >

M 183. Choose optimal guesses for green-only Wordle PracHub * 2025-10-14 *

CODE TEMPLATE | GROUP 39 OF 43 | 1 QUESTIONS, ONE PATTERN

Games: win or lose with a cache, and guessing with feedback

The one idea these questions share

For two-player games, a state is winning if some move leads to a state where the opponent loses; cache every state. For guessing games, pick the guess that shrinks the worst remaining group of candidates the most, then keep only the candidates that agree with the feedback.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from functools import lru_cache


def first_player_wins(state, moves):
    @lru_cache(maxsize=None)
    def wins(s):
        # CHANGE: moves(s) lists the states you can move to; you win if some move leaves the opponent losing
        return any(not wins(t) for t in moves(s))

    return wins(state)
```

2. Helper functions that usually get written next to it

```
from collections import Counter


def match_count(a, b):                          # feedback: how many positions agree
    return sum(x == y for x, y in zip(a, b))


def best_guess(candidates):                     # minimise the largest group left after the feedback
    return min(candidates, key=lambda g: (max(Counter(match_count(g, w) for w in candidates).values()), g))
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
183	Choose optimal guesses for green-only Wordle	bitmask per (position, letter) of the words that have it; the best guess splits the words into the most even groups

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 183 of 197 | topic 39 of 43: Games: win or lose with a cache, and guessing with feedback | 1 of 1

Template fit: bitmask per (position, letter) of the words that have it; the best guess splits the words into the most even groups | * asked 371 days ago *

MEDIUM | PracHub | Technical Screen | 183 of 197

Choose optimal guesses for green-only Wordle

POSTED 2025-10-14 | 371 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 42.2 | 30 DAYS 0 | PRIORITY 99/288

ABOUT THIS QUESTION

Topic: Games: win or lose with a cache, and guessing with feedback

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #843 Guess the Word

LeetCode match: No direct match; related LC 843 Guess the Word (inferred from overview)

Posted on PracHub: 2025-10-14 (updated 2026-03-29)

Original: https://prachub.com/interview-questions/choose-optimal-guesses-for-green-only-wordle

QUESTION

You are guessing a hidden word from a candidate list. After each guess you only learn which positions are green, that is, where your letter equals the answer's letter. best_guess(words) picks the candidate whose answer pattern splits the remaining words most evenly: the one with the smallest expected size of the group left (sum of squared group sizes divided by the number of words); ties go to the alphabetically smaller word.

Function signature

def green(guess: str, answer: str) -> tuple

def best_guess(words: list) -> str

EXAMPLES

Example 1	
Input	`'cat', 'car'`
Output	`(true, true, false)`
Why	`c and a match, the last letters differ.`

Example 2	
Input	`['cat', 'car', 'cap', 'dog']`
Output	`'cap'`
Why	`'cap', 'car' and 'cat' split the other words equally well and 'cap' wins the alphabetical tie-break; 'dog' shares no letter positions with the others, so it tells less.`

Example 3	
Input	`['tea']`
Output	`'tea'`
Why	`With a single candidate the answer is that candidate.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(guesses, answers):
    best, best_score = None, INF
    for g in guesses:
        groups = Counter()
        for a in answers:
            groups[green_pattern(g, a)] += 1
        sizes = groups.values()
        score = sum(c * c for c in sizes)
        score /= len(answers)
        if score < best_score:
            best, best_score = g, score
    return best
```

TIME	SPACE
O(W^2 * L)	O(W)

WHY IT IS SLOW

Compares every pair of words, O(W^2 * L). Bitsets per (position, letter) split the candidates into pattern groups with a few AND operations.

OPTIMAL SOLUTION · Bitsets per (position, letter), splitting groups with AND

TIME	SPACE
O(W * L * groups) with bit-parallel work per group	O(W * L)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Play a whole game, using the feedback after each guess.

Keep the candidates that would have produced exactly the observed green pattern (filter_candidates), then choose the best guess among the remaining ones. Each round at least removes the guess itself, so the game ends.

FOLLOW-UP 2 Answers are not equally likely (weights).

Replace group sizes by group weights: score = sum(weight(group)^2) / total weight. The grouping step is unchanged; use sums of weights instead of popcounts.

FOLLOW-UP 3 Real Wordle also gives yellow feedback.

The pattern becomes three-valued per position (green, yellow, grey) with the usual duplicate-letter rules, so the pattern space is 3^L; bitsets still work if you precompute patterns, but the computation is O(W^2) pattern evaluations for a full table.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Choose optimal guesses for green-only
# Wordle
#
```

```
# You are guessing a hidden word from a candidate list.
# After each guess you only learn which positions are
# green, that is, where your letter equals the answer's
# letter. best_guess(words) picks the candidate whose
# answer pattern splits the remaining words most evenly:
# the one with the smallest expected size of the group
# left (sum of squared group sizes divided by the number
# of words); ties go to the alphabetically smaller word.
#
# Signature:
#   def green(guess: str, answer: str) -> tuple
#   def best_guess(words: list) -> str
#
# Examples:
# 'cat', 'car'  ->  (true, true, false)
# ['cat', 'car', 'cap', 'dog']  ->  'cap'
#
# FOLLOW-UPS (answered below, in this order):
# 1. Play a whole game, using the feedback after each
# guess.
# 2. Answers are not equally likely (weights).
# 3. Real Wordle also gives yellow feedback.
#
# INTERVIEW NOTES
# A guess splits the candidates into groups by which
# positions turned green. If the groups have sizes s_1 ...
# s_k out of W candidates, the expected size of what
# remains is the sum of s_i squared divided by W (a
# candidate falls in a group of size s with probability s
# / W, and then s remain). Pick the guess with the
# smallest value. To make it fast, store for each
# (position, letter) the set of words with that letter
# there as a bitset, so the groups come from a few AND
# operations.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================
```

```
from fractions import Fraction


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def green(guess, answer):
    # green-only feedback: for every position, does the letter match exactly?
    feedback = []
    for i in range(len(guess)):
        feedback.append(guess[i] == answer[i])
    return tuple(feedback)


def partition(guess, words):
    # group the possible answers by the feedback they would give for this guess
    groups = {}
    for word in words:
        pattern = green(guess, word)
        if pattern not in groups:
            groups[pattern] = []
        groups[pattern].append(word)
```

```
    return list(groups.values())


def score(guess, words, weights=None):
    # expected number of answers left after this guess:
    # (sum of group size squared) / (number of answers). Lower is better.
    # weights (follow-up 2): word -> how likely it is the answer (default 1).
    def weight_of(word):
        if weights and word in weights:
            return weights[word]
        return 1

    total = 0
    for word in words:
        total += weight_of(word)
    spread = 0
    for group in partition(guess, words):
        group_weight = 0
        for word in group:
            group_weight += weight_of(word)
        spread += group_weight * group_weight
    return Fraction(spread, total)
```

```


# ---- main question: main function ----
# Idea: for every possible guess, split the remaining answers by the
# feedback that guess would give, and pick the guess that leaves the
# fewest answers on average. Ties go to the alphabetically first word,
# so the result is deterministic.
# Complexity: Time O(n^2 * L) for n words of length L, Space O(n)
def best_guess(words, weights=None):
    best_word = None
    best_score = None
    for guess in words:
        s = score(guess, words, weights)
        if best_score is None or s < best_score or (s == best_score and guess < best_word):
            best_score = s
            best_word = guess
    return best_word


# a faster way: one bitmask per (position, letter); every position splits
# each group in two with two AND operations
# Complexity: Time O(n * L * n / 64 + n^2 / 64) (the groups are bitmasks), Space O(L * 26 * n / 64)
def best_guess_bitsets(words):
    count = len(words)
    bits = []
    for pos in range(len(words[0])):
        bits.append({})
    for i in range(count):
        for pos in range(len(words[i])):
            letter = words[i][pos]
            bits[pos][letter] = bits[pos].get(letter, 0) | (1 << i)
    best_word = None
    best_score = None
    for guess in words:
        groups = [(1 << count) - 1]                # one group holding every word
        for pos in range(len(guess)):
            mask = bits[pos].get(guess[pos], 0)
            split = []
            for group in groups:
                matching = group & mask
                other = group & ~mask
                if matching:
```

```
                split.append(matching)
            if other:
                split.append(other)
```

```
            groups = split
        spread = 0
        for group in groups:
            size = bin(group).count("1")
            spread += size * size
        s = Fraction(spread, count)
        if best_score is None or s < best_score or (s == best_score and guess < best_word):
            best_score = s
            best_word = guess
    return best_word


# ---- main question: tests ----
WORDS = ["crane", "crate", "grape", "grade", "trade", "slate", "plate"]


def test_main_question():
    assert best_guess(WORDS) == best_guess_bitsets(WORDS)
    assert score("crane", WORDS) <= len(WORDS)               # a guess never leaves more than all the words
    assert partition("crane", ["crane"]) == [["crane"]]
    assert green("crane", "crate") == (True, True, True, False, True)


# ========================================================
# FOLLOW-UP 1: Play a whole game, using the feedback after each guess
# ========================================================
# ---- follow-up 1: function ----
# Answer: keep only the answers that would have produced exactly the
# feedback you saw, then pick the best guess among those. Each round
# removes at least the guess itself, so the game ends.
# Complexity: Time O(n * L), Space O(n)
def filter_candidates(words, guess, pattern):
    kept = []
    for word in words:
        if green(guess, word) == pattern:
            kept.append(word)
    return kept


# Complexity: Time O(guesses * n^2 * L), Space O(n)
def play(words, secret):
    candidates = list(words)
    guesses = []
    while True:
        guess = best_guess(candidates)
```

```
        guesses.append(guess)
        if guess == secret:
            return guesses
        candidates = filter_candidates(candidates, guess, green(guess, secret))


# ---- follow-up 1: tests ----
def test_follow_up_1():
    for secret in WORDS:
        path = play(WORDS, secret)
        assert path[-1] == secret
        assert len(path) <= len(WORDS)


# ========================================================
```

```
# FOLLOW-UP 2: Answers are not equally likely (weights)
# ========================================================
# Answer: score() already takes a `weights` dictionary: a very likely
# answer counts for more in every group, so guesses that split it off
# become more attractive.
# ---- follow-up 2: tests ----
def test_follow_up_2():
    weights = {"slate": 100}
    assert score("slate", WORDS, weights) < score("crane", WORDS, weights)


# ========================================================
# FOLLOW-UP 3: Real Wordle also gives yellow feedback
# ========================================================
# Answer: the feedback becomes a pattern of green / yellow / grey per
# position (3^5 possible patterns). Only green() changes: count each
# letter's remaining copies so a repeated letter is yellow at most as
# many times as it really appears in the answer.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FULL QUESTION TEXT FROM THE SITE

Problem

You are playing a simplified 5-letter word guessing game.

There is a hidden secret word of length 5 .
You have a dictionary of valid 5-letter words. Each word may also have an associated frequency/weight representing how likely it is to be the secret.
When you guess a 5-letter word, the game returns only “green” information (correct letter in the correct position). There is no information about letters that exist in the word but are in the wrong position (“yellow”), and no explicit info about letters that are absent.
Feedback definition

For a guess g and secret s, define the feedback as a 5-bit mask m where:

m[i] = 1 iff g[i] == s[i]
otherwise m[i] = 0

Example: g="crane", s="cared" -> mask is 1 0 0 0 0 (only position 0 matches).

Task

Given:

words : a list of N valid 5-letter words
weight[w] (optional): a positive number for each word (if omitted, assume uniform)
candidates : the current set of possible secret words consistent with feedback so far (initially all words)

Write an algorithm/function to choose the next guess that is optimal under the following objective:

FULL QUESTION TEXT FROM THE SITE

Choose a guess word g (typically from words) that minimizes the expected size of the remaining candidate set after receiving the green-only mask, where the expectation is taken over the secret word drawn from candidates according to weight .

Formally, if C is the current candidate set and C_m(g) is the subset of candidates that would produce mask m for guess g, choose g that minimizes:

E
[
|
C
m
a
s
k
(
g
)
|
]
=
sum
m
in
{
0
,
1
}
5

P
(
m
a
s
k
=
m
)
*
|
C
m
(
g
)
|
E[|C
mask

FULL QUESTION TEXT FROM THE SITE

(g)|]=sum
min{0,1}
5

P(mask=m)*|C
m

(g)|

where
P
(
m
a
s
k
=
m
)
P(mask=m) is induced by the weighted distribution over secrets in C.

Input/Output expectations
Input sizes: 1 <= N <= 5,000 (you may propose optimizations if needed), word length is always 5.
Output: the selected next guess word.
Follow-ups (optional)
How would you incorporate non-uniform word frequencies?
How would you scale if N is very large (e.g., 100k)?
How would you change the objective to minimize expected number of guesses instead of expected remaining candidates?

ALSO REPORTED · 1 more

Wordle-Style 5-Letter Minimum-Guess Strategy | 1Point3Acres | phone-screen, onsite-coding | asked 2025-12-15 | Single report

https://www.1point3acres.com/interview/problems/company/google/wordle-min-guess-strategy

Without the dictionary, query positions in order; per position, query letters in descending frequency (e, t, a, o, i, n, ...) and stop as soon as one returns true.
With the dictionary, maintain a candidate set; at each step pick the (position, letter) query whose true/false outcome splits the candidate set most evenly (entropy maximization).
Mention information theory: each query yields <= 1 bit, so the absolute lower bound on calls is ceil(log2(|candidates|)).
Don't waste minutes on a closed-form optimum — the interviewer mostly wants you to articulate the entropy heuristic and pick one reasonable scoring function.

Preparation: Drill information-theoretic puzzles (egg drop, twenty questions) to internalize the entropy-split heuristic.
Practice maintaining a candidate set under positive/negative oracle feedback (filter on each query result).
Be ready with a one-liner on Wordle's known optimal first guess (SOARE / CRANE) for intuition; do not hard-code it.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 40 OF 43

Sliding window: grow on the right, shrink on the left while it is too bad

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Sliding window: grow on the right, shrink on the left while it is too bad | 4 questions | tap a title to jump to it

READ FIRST: the code template for these 4 questions (tap here) >

M	184. Temperature Monitoring Data Structure	1Point3Acres * 2026-09-10 *
M	185. Footpath Beauty After Repainting	ColorStack * 2025-09-22 *
M	186. Shortest Subarray with K Distinct Integers	ND
M	187. Find the minimum window substring.	ND

CODE TEMPLATE | GROUP 40 OF 43 | 4 QUESTIONS, ONE PATTERN

Sliding window: grow on the right, shrink on the left while it is too bad

The one idea these questions share

Move the right end one step at a time and add that item to your state. While the window breaks the rule, remove items from the left end. After each step the window is valid, so record its length (or the shortest valid one).

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def longest_window(items, bad, allowed):
    left = used = best = 0
    for right, x in enumerate(items):
        used += bad(x)                          # CHANGE 1: add the right item to the state
        while used > allowed:                   # CHANGE 2: when the window is invalid
            used -= bad(items[left])            # CHANGE 3: remove the left item from the state
            left += 1
        best = max(best, right - left + 1)      # record the window
    return best
```

2. Helper functions that usually get written next to it

```
from collections import Counter, deque


def fixed_window_sums(nums, size):
    total = sum(nums[:size])
    out = [total]
    for i in range(size, len(nums)):
        total += nums[i] - nums[i - size]       # add the new item, drop the old one
        out.append(total)
    return out


def window_max(nums, size):                     # monotonic deque: the front is always the maximum
    q, out = deque(), []
    for i, x in enumerate(nums):
        while q and nums[q[-1]] <= x:
            q.pop()
        q.append(i)
        if q[0] <= i - size:
            q.popleft()
        if i >= size - 1:
            out.append(nums[q[0]])
    return out
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
184	Temperature Monitoring Data Structure	a deque of the last n readings with a running total; a second deque of decreasing values gives the maximum
185	Footpath Beauty After Repainting	bad = a block that is not the wanted colour; allowed = the repaint budget m; the longest window is the answer
186	Shortest Subarray with K Distinct Integers	state = a count dict; while it holds at least k distinct values, record the window and shrink from the left
187	Find the minimum window substring.	state = how many of each needed letter are still missing; while nothing is missing, record the window and shrink

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 184 of 197 | topic 40 of 43: Sliding window: grow on the right, shrink on the left while it is too bad | 1 of 4

Template fit: a deque of the last n readings with a running total; a second deque of decreasing values gives the maximum | * asked 40 days ago * | frequency Single report

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 184 of 197

Temperature Monitoring Data Structure

ASKED 2026-09-10 | 40 DAYS AGO | LAST 90 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 283/288**

ABOUT THIS QUESTION

Topic: Sliding window: grow on the right, shrink on the left while it is too bad

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/temperature-monitor-moving-window

QUESTION

Design a monitor over the last n temperature readings. insert(x) adds a reading (the oldest one drops out once more than n are stored), average() returns the mean of the stored readings, and maximum() returns the largest of them in O(1).

Function signature

```
class Monitor:
    def __init__(self, n: int)
    def insert(self, x: float) -> None
    def average(self) -> float
    def maximum(self) -> float
```

EXAMPLES

Example 1

Input	`m = Monitor(3)`
	`m.insert(20)`
	`m.insert(22)`
	`m.insert(19)`
	`m.average()`
	`m.insert(25)`
	`m.average()`
	`m.maximum()`
	`m.insert(10)`
	`m.maximum()`

Output	`m.average() -> 20.333333`
	`m.average() -> 22`
	`m.maximum() -> 25`
	`m.maximum() -> 25`

Why	With n = 3 the window slides: after 25 arrives the window is 22, 19, 25 (average 22, maximum 25); after 10 it is 19, 25, 10, so the maximum is still 25.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
readings = []
def insert(x):
    readings.append(x)
def average():
    window = readings[-N:]
    return sum(window) / len(window)
def maximum():
    return max(readings[-N:])
```

TIME	SPACE
O(N) per average or max	**O(N)**

WHY IT IS SLOW

Every query scans the window, O(N). A running sum gives the average in O(1), and a deque of decreasing values gives the maximum in O(1) amortised.

OPTIMAL SOLUTION · Running sum plus a monotonic deque

TIME	SPACE
O(1) amortized per operation	**O(N)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The window is the last D days (elapsed time), not the last N readings.

Store (timestamp, value) pairs and evict from the front while the timestamp is at or before now minus the span, in both the window and the max deque. Everything else is unchanged.

FOLLOW-UP 2 Also support minimum, or the median.

Minimum uses a second monotonic deque with the opposite comparison. A median needs two heaps with lazy deletion or a sorted container, O(log N) per update, because it is not decomposable like sum and max.

FOLLOW-UP 3 Floating-point drift in the running sum.

Recompute the sum from the window every so often (say every N inserts), or use math.fsum or integer hundredths of a degree to avoid drift.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Temperature Monitoring Data Structure
#
# Design a monitor over the last n temperature readings.
# insert(x) adds a reading (the oldest one drops out once
# more than n are stored), average() returns the mean of
# the stored readings, and maximum() returns the largest
```

```
# of them in O(1).
#
# Signature:
#   class Monitor:
#       def __init__(self, n: int)
#       def insert(self, x: float) -> None
#       def average(self) -> float
#       def maximum(self) -> float
#
# FOLLOW-UPS (answered below, in this order):
# 1. The window is the last D days (elapsed time), not the
# last N readings.
# 2. Also support minimum, or the median.
# 3. Floating-point drift in the running sum.
#
# INTERVIEW NOTES
# Three operations on the last N readings. The average
# only needs a running sum, and the maximum needs a
# monotonic deque that drops smaller values from the back
# as larger ones arrive, so the front is always the max of
# the window. Decide first whether the window is a count
# of readings or a span of time.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
# Complexity: Time push is O(1) amortized, best O(1), Space O(window)
class MonotoneQueue:
```

```
    # keeps the BEST value of the window at the front. keep_max=True keeps
    # the maximum, keep_max=False the minimum. A new value removes from the
    # back every value it beats: those can never be the best again, because
    # the new one stays in the window longer.
    def __init__(self, keep_max):
        self.items = deque()                        # (key, value) pairs, best first
        self.keep_max = keep_max

    def at_least_as_good(self, a, b):
        if self.keep_max:
            return a >= b
        return a <= b

    def push(self, key, value):
        while self.items and self.at_least_as_good(value, self.items[len(self.items) - 1][1]):
            self.items.pop()
        self.items.append((key, value))

    def expire(self, cutoff):
        # forget the entries whose key is at or below the cutoff (they left the window)
        while self.items and self.items[0][0] <= cutoff:
            self.items.popleft()

    def best(self):
        return self.items[0][1]
```

```
# The shared window: every reading with a key (its number or its day), a running
# total for the average, and two monotone queues for the maximum and the
# minimum (follow-up 2). Median needs more (two heaps with lazy deletion or a
# sorted container, O(log N) per update) because it is not decomposable like
# a sum or a maximum.
# Complexity: Time add and expire are O(1) amortized, average, maximum and minimum O(1), Space O(window)
class SlidingStats:
    def __init__(self):
        self.items = deque()                        # (key, value), oldest first
        self.total = 0
        self.max_queue = MonotoneQueue(True)
        self.min_queue = MonotoneQueue(False)

    def add(self, key, value):
        self.items.append((key, value))
        self.total += value
        self.max_queue.push(key, value)
        self.min_queue.push(key, value)
```

```
    def expire(self, cutoff):
        # drop the readings with a key at or below the cutoff, everywhere
        while self.items and self.items[0][0] <= cutoff:
            self.total -= self.items.popleft()[1]
        self.max_queue.expire(cutoff)
        self.min_queue.expire(cutoff)

    def average(self):
        return self.total / len(self.items)

    def maximum(self):
        return self.max_queue.best()

    def minimum(self):
        return self.min_queue.best()


# ---- main question: main class ----
# insert(x) adds a reading; average() and maximum() describe the LAST n readings.
# Idea: keep the window in a deque with a running sum; on insert add the new
# value and drop the oldest when the window holds more than n. The maximum comes
# from a monotone queue, so every operation is O(1) amortised.
# Complexity: Time insert O(1) amortized, Space O(n) for the newest n readings
class Monitor(SlidingStats):
    def __init__(self, n):
        SlidingStats.__init__(self)
        self.n = n
        self.seen = 0

    def insert(self, x):
        self.add(self.seen, x)                      # the number of the reading is its key
        self.seen += 1
        self.expire(self.seen - self.n - 1)         # keep only the newest n readings


# ---- main question: tests ----
def test_main_question():
    m = Monitor(3)
    for x in [5, 1, 4, 2]:
        m.insert(x)
    assert m.maximum() == 4                         # the window is [1, 4, 2]
    assert m.average() == 7 / 3
    m.insert(0)
```

```
    assert m.maximum() == 4                         # the window is [4, 2, 0]
```

```


# ========================================================
# FOLLOW-UP 1: The window is the last D days (elapsed time), not the last N readings
# ========================================================
# ---- follow-up 1: function ----
# Answer: store (day, value) pairs and drop from the front while the day is
# at or before (today minus the span), in the window and in the monotone
# queues. Everything else is unchanged.
# Complexity: Time insert is O(1) amortized for the expiry (each reading is removed once) plus the cost of
the stored statistics, Space O(readings inside the span)
class TimedMonitor(SlidingStats):
    def __init__(self, span):
        SlidingStats.__init__(self)
        self.span = span

    def insert(self, day, x):
        self.add(day, x)
        self.expire(day - self.span)                # drop the readings at or before day - span


# ---- follow-up 1: tests ----
def test_follow_up_1():
    timed = TimedMonitor(2)
    for day, x in [(1, 9), (2, 3), (3, 5)]:
        timed.insert(day, x)
    assert timed.maximum() == 5 and timed.average() == 4      # day 1 is two days old: it left the window


# ========================================================
# FOLLOW-UP 2: Also support minimum, or the median
# ========================================================
# Answer: SlidingStats already has minimum() (a second monotone queue with
# the opposite comparison). The median needs two heaps with lazy deletion or a
# sorted container: O(log N) per update.
# ---- follow-up 2: tests ----
def test_follow_up_2():
    m = Monitor(3)
    for x in [5, 1, 4, 2]:
        m.insert(x)
    assert m.minimum() == 1
    m.insert(0)
    assert m.minimum() == 0


# ========================================================
```

```
# FOLLOW-UP 3: Floating-point drift in the running sum
# ========================================================
# Answer: a running sum that adds and subtracts floats slowly drifts. Add up
# the window again from scratch every so often, or keep the sum as whole
# numbers (e.g. tenths of a degree).


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement all three reading-count operations using a running sum and deques; compare outputs with direct window scans.
Test N=1, duplicate maxima, negative readings, expiration of the maximum, and partially filled windows under an explicitly chosen contract; explain amortized versus worst-case insertion cost.

STUDY PATH: question 185 of 197 | topic 40 of 43: Sliding window: grow on the right, shrink on the left while it is too bad | 2 of 4

Template fit: bad = a block that is not the wanted colour; allowed = the repaint budget m; the longest window is the answer | * asked 393 days ago * | frequency Single report

MEDIUM | IN APP LC 1004 | ColorStack | technical interview | 185 of 197

Footpath Beauty After Repainting

ASKED 2025-09-22 | 393 DAYS AGO | OLDER

STUDY: TALK NO LEETCODE FREQUENCY PRIORITY 220/288

ABOUT THIS QUESTION

Topic: Sliding window: grow on the right, shrink on the left while it is too bad

Optimal answer from: your app's interview approach (LeetCode #1004)

LeetCode match: LC 1004 Max Consecutive Ones III

Frequency: Single report | Last asked: 2025-09-22 | Roles: SWE Intern

QUESTION

A footpath is built from n blocks laid in a line, painted with one of k colors. block_colors[i] is the color of block i (colors are integers in 1..k). The beauty value for a given favorite color c is the length of the longest run of consecutive blocks that are all color c . You are allowed to repaint at most m blocks (each to any color you choose). Return the maximum possible beauty value for favorite color c after repainting at most m blocks.

Function signature

```
def max_beauty_after_repaint(block_colors: list[int], c: int, m: int) -> int:
    ...
```

Follow-up (as reported): (Reported) Answer q queries, each with a different favorite color, and return the beauty value (with m fixed) for each. Hint: a sliding window per query, or precompute runs. (Reported) The base version with m = 0 was asked first; the repaint version is the twist.

EXAMPLES

Example 1	
Input	`block_colors = [1, 2, 1, 1, 2, 1], c = 1, m = 1`
Output	`4`
Why	`Repaint one of the 2's. The best window has at most one non-c block, e.g. indices 2..5 [1,1,2,1] -> repaint the 2 -> 4.`

Example 2	
Input	`block_colors = [3, 3, 3], c = 3, m = 0`
Output	`3`
Why	`All blocks are already color 3; no repaint needed.`

Example 3	
Input	`block_colors = [2, 2, 2], c = 1, m = 2`
Output	`2`
Why	`Only 2 repaints allowed, so at most 2 consecutive c blocks.`

CONSTRAINTS

- 1 <= n <= 10^5 , block_colors has length n
- 1 <= k , 1 <= c <= k , 1 <= block_colors[i] <= k

QUESTION

- 0 <= m <= n
- With m = 0 , the answer is simply the longest existing run of color c .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_beauty(colors, c, m):
    best = 0
    for start in range(len(colors)):
        other = 0
        for end in range(start,_len(colors)):
            other += colors[end] != c
            if other <= m:
                best = max(
                    best, end - start + 1
                )
    return best
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Every start and end pair recounts its blocks.

OPTIMAL SOLUTION · Sliding window with at most m repaints

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Answer many queries, each with a different favourite colour (m fixed).

Group the positions of every colour once. For a colour run a window over its positions: the blocks between the first and last position of a window that are not that colour are the repaints needed; keep it at most m. The run length is the window size plus the unused repaints, capped by n. Total work is O(n + queries * positions of that colour).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Footpath Beauty After Repainting
#
# A footpath is built from n blocks laid in a line,
# painted with one of k colors. block_colors[i] is the
# color of block i (colors are integers in 1..k ). The
# beauty value for a given favorite color c is the length
# of the longest run of consecutive blocks that are all
# color c . You are allowed to repaint at most m blocks
# (each to any color you choose). Return the maximum
# possible beauty value for favorite color c after
# repainting at most m blocks. Function signature: def
# max_beauty_after_repaint(block_colors: list[int], c:
```

```
# int, m: int) -> int: ... Follow-up (as reported):
# (Reported) Answer q queries, each with a different
# favorite color, and return the beauty value (with m
# fixed) for each. Hint: a sliding window per query, or
# precompute runs. (Reported) The base version with m = 0
# was asked first; the repaint version is the twist.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Answer many queries, each with a different favourite
# colour (m fixed).
#
# INTERVIEW NOTES
# Repainting at most m blocks to the favourite colour
# means: find the longest window that contains at most m
# blocks of another colour, because those are the ones to
# repaint. A sliding window keeps the count of other-
# colour blocks and shrinks from the left when it goes
# over m.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# A footpath is a row of blocks with colours. You may repaint at most m
# blocks. The beauty is the length of the longest run of blocks of the
# favourite colour c. What is the best beauty?
```

```
# Idea: the answer is the longest window that holds at most m blocks of
# another colour (those are the ones to repaint). Grow the right end; while the
# window needs more than m repaints, shrink it from the left.
# Complexity: Time O(n), Space O(1)
def max_beauty_after_repaint(block_colors, c, m):
    left = 0
    other = 0                                       # the blocks in the window that are not colour c
    best = 0
    for right in range(len(block_colors)):
        if block_colors[right] != c:
            other += 1
        while other > m:
            if block_colors[left] != c:
                other -= 1
            left += 1
        if right - left + 1 > best:
            best = right - left + 1
    return best


# ---- main question: tests ----
def test_main_question():
    assert max_beauty_after_repaint([1, 2, 1, 1, 2, 1], 1, 1) == 4
    assert max_beauty_after_repaint([3, 3, 3], 3, 0) == 3
    assert max_beauty_after_repaint([2, 2, 2], 1, 2) == 2


# ============================================================
# FOLLOW-UP 1: Answer many queries, each with a different favourite colour (m fixed)
# ============================================================
# ---- follow-up 1: function ----
```

```
# Answer: list the positions of every colour once. For one colour run a
# window over ITS positions: between the first and the last position of the
# window, the blocks that are not that colour must be repainted; keep that at
# most m. Then m more blocks (next to the window) can be repainted too, so
# the beauty is the number of positions in the window plus m, but never more
# than the whole path.
# Complexity: Time O(n + sum of the counts of the queried colours), Space O(n)
def beauty_queries(block_colors, queries, m):
    n = len(block_colors)
    where = {}
    for i in range(n):
        if block_colors[i] not in where:
            where[block_colors[i]] = []
        where[block_colors[i]].append(i)
    answers = []
    for c in queries:
        positions = where.get(c, [])
        left = 0
        best = 0
        for right in range(len(positions)):
            while positions[right] - positions[left] - (right - left) > m:
                left += 1                           # too many other blocks between the two ends
            size = right - left + 1 + m
            if size > n:
                size = n
            if size > best:
                best = size
        if not positions:
            best = min(n, m)
        answers.append(best)
    return answers


# ---- follow-up 1: tests ----
def test_follow_up_1():
    colors = [1, 2, 1, 1, 2, 1]
    assert beauty_queries(colors, [1, 2, 3], 1) == [4, 2, 1]
    for color in (1, 2, 3):
        assert beauty_queries(colors, [color], 1)[0] == max_beauty_after_repaint(colors, color, 1)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 186 of 197 | topic 40 of 43: Sliding window: grow on the right, shrink on the left while it is too bad | 3 of 4

Template fit: state = a count dict; while it holds at least k distinct values, record the window and shrink from the left | ND

MEDIUM | HackTheRounds | Onsite | 186 of 197

★ TOP 75: rank 66 in the Medium and Hard priority list

★ Shortest Subarray with K Distinct Integers

STUDY: TYPE **ALL TIME 42.5** **30 DAYS 33** **PRIORITY 45/288**

ABOUT THIS QUESTION

Topic: Sliding window: grow on the right, shrink on the left while it is too bad

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #992 Subarrays with K Different Integers, LeetCode #76 Minimum Window Substring

LeetCode match: LC 992 / LC 76 flavor

QUESTION

Given an array nums and an integer k, return the length of the shortest contiguous subarray that contains at least k different integers, or -1 if there is no such subarray.

Function signature

def shortest_k_distinct(nums: list, k: int) -> int

EXAMPLES

Example 1	
Input	`[1, 2, 1, 3, 2], 3`
Output	`3`
Why	`[2, 1, 3] is the shortest window holding three different numbers.`

Example 2	
Input	`[1, 1, 1], 2`
Output	`-1`
Why	`Only one distinct number exists.`

Example 3	
Input	`[1, 2], 2`
Output	`2`
Why	`The whole array is needed.`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 1 <= nums[i] <= 10^9
- 1 <= k <= nums.length

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, k):
    best = -1
    for i in range(len(nums)):
        seen = set()
        for j in range(i, len(nums)):
            seen.add(nums[j])
            if len(seen) >= k:
                size = j - i + 1
                if best == -1 or size < best:
                    best = size
                break
    return best
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

O(n^2) in the worst case. A sliding window with a count map expands on the right and shrinks on the left, so each element is handled twice: O(n).

OPTIMAL SOLUTION · Sliding window with a frequency map

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Count the subarrays with exactly k distinct values (LeetCode 992).

exactly(k) = atMost(k) - atMost(k - 1), where atMost is a sliding window that shrinks while the distinct count exceeds the limit. Both are O(n).

FOLLOW-UP 2 Longest subarray with at most k distinct values.

The same window, but keep the longest window whose distinct count never exceeds k (shrink while it does). O(n).

FOLLOW-UP 3 The values are strings or large objects.

The frequency map works on any hashable value; for huge objects hash them first or map them to integer ids so the window comparisons stay cheap. **Cost:** O(n) plus hashing each element; mapping values to integer ids costs O(n) once.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Shortest Subarray with K Distinct
# Integers
#
# Given an array nums and an integer k, return the length
# of the shortest contiguous subarray that contains at
# least k different integers, or -1 if there is no such
```

```
# subarray.
#
# Signature:
#   def shortest_k_distinct(nums: list, k: int) -> int
#
# Examples:
# [1, 2, 1, 3, 2], 3  ->  3
# [1, 1, 1], 2  ->  -1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Count the subarrays with exactly k distinct values
# (LeetCode 992).
# 2. Longest subarray with at most k distinct values.
# 3. The values are strings or large objects.
#
# INTERVIEW NOTES
# Once a window has at least k distinct values, shrinking
# it from the left can only keep or lose distinct values,
# so the shortest valid window ending at each right end is
# found by shrinking as far as possible. A frequency map
# tells you when a value leaves the window, so each index
# enters and leaves once.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# The length of the shortest subarray with at least k distinct values (-1
# if none).
```

```
# Idea: a sliding window with a count of every value in it. Grow the right
# end; whenever the window holds at least k distinct values, record its length
# and shrink from the left as far as it stays valid (delete a value from the
# counts when its count reaches 0).
# Complexity: Time O(n), Space O(k)
def shortest_k_distinct(nums, k):
    counts = {}
    left = 0
    best = None
    for right in range(len(nums)):
        counts[nums[right]] = counts.get(nums[right], 0) + 1
        while len(counts) >= k:
            size = right - left + 1
            if best is None or size < best:
                best = size
            counts[nums[left]] -= 1
            if counts[nums[left]] == 0:
                del counts[nums[left]]
            left += 1
    if best is None:
        return -1
    return best


# ---- main question: tests ----
```

```
def test_main_question():
    assert shortest_k_distinct([1, 2, 1, 3, 4], 3) == 3
    assert shortest_k_distinct([1, 1, 1], 2) == -1
    assert shortest_k_distinct([5], 1) == 1
    assert shortest_k_distinct([], 1) == -1


# ======================================================
# FOLLOW-UP 1: Count the subarrays with exactly k distinct values (LeetCode 992)
# ======================================================
# ---- follow-up 1: helper functions ----
def at_most(nums, k):
    # the number of subarrays with at most k distinct values: for every right
    # end, all windows that start between `left` and `right` qualify
    counts = {}
    left = 0
    total = 0
    for right in range(len(nums)):
        counts[nums[right]] = counts.get(nums[right], 0) + 1
        while len(counts) > k:
            counts[nums[left]] -= 1
            if counts[nums[left]] == 0:
                del counts[nums[left]]
            left += 1
        total += right - left + 1
    return total


# ---- follow-up 1: function ----
# Answer: exactly(k) = atMost(k) - atMost(k - 1), where atMost is a sliding
# window that shrinks while it holds more than k distinct values. Both are O(n).
# Complexity: Time O(n), Space O(k)
def exactly(nums, k):
    return at_most(nums, k) - at_most(nums, k - 1)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert exactly([1, 2, 1, 2, 3], 2) == 7
    assert exactly([1, 2, 1, 3, 4], 3) == 3
    random.seed(27)
    for round_number in range(300):                  # compare with counting every subarray
        nums = []
        for i in range(random.randint(0, 10)):
            nums.append(random.randint(1, 4))
        k = random.randint(1, 5)
        slow = 0
        for i in range(len(nums)):
            for j in range(i, len(nums)):
                if len(set(nums[i:j + 1])) == k:
                    slow += 1
        assert exactly(nums, k) == slow


# ======================================================
# FOLLOW-UP 2: Longest subarray with at most k distinct values
# ======================================================
# ---- follow-up 2: function ----
# Answer: the same window, but keep the LONGEST window whose number of
# distinct values never exceeds k (shrink while it does). O(n).
# Complexity: Time O(n), Space O(k)
def longest_at_most(nums, k):
    counts = {}
    left = 0
```

```
    best = 0
```

```
    for right in range(len(nums)):
        counts[nums[right]] = counts.get(nums[right], 0) + 1
        while len(counts) > k:
            counts[nums[left]] -= 1
            if counts[nums[left]] == 0:
                del counts[nums[left]]
            left += 1
        if right - left + 1 > best:
            best = right - left + 1
    return best


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert longest_at_most([1, 2, 1, 2, 3], 2) == 4


# ============================================================
# FOLLOW-UP 3: The values are strings or large objects
# ============================================================
# ---- follow-up 3: function ----
# Answer: the counting dictionary works for any hashable value. For huge
# objects, map every distinct value to a small whole number once (O(n)) and
# run the same code on the numbers.
# Complexity: Time O(n), Space O(u) for u distinct items
def to_ids(items):
    ids = {}
    numbers = []
    for item in items:
        if item not in ids:
            ids[item] = len(ids)
        numbers.append(ids[item])
    return numbers


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert shortest_k_distinct(to_ids(["a", "b", "a", "c"]), 3) == 3
    assert to_ids(["x", "y", "x"]) == [0, 1, 0]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

ALSO REPORTED · 2 more

Find shortest subarray with at least k distinct | PracHub | Technical Screen

https://prachub.com/coding-questions/find-shortest-subarray-with-at-least-k-distinct

Find shortest subarray with >=k distinct integers | PracHub | Technical Screen

https://prachub.com/coding-questions/find-shortest-subarray-with-k-distinct-integers

STUDY PATH: question 187 of 197 | topic 40 of 43: Sliding window: grow on the right, shrink on the left while it is too bad | 4 of 4

Template fit: state = how many of each needed letter are still missing; while nothing is missing, record the window and shrink | ND

MEDIUM | IN APP LC 76 | Exponent | 187 of 197

★ TOP 75: rank 55 in the Medium and Hard priority list

★ Find the minimum window substring.

ASKED (ANY CO.) ~7 MONTHS AGO | OLDER

STUDY: TYPE | **ALL TIME 42.5** | **30 DAYS 33** | **PRIORITY 44/288**

ABOUT THIS QUESTION

Topic: Sliding window: grow on the right, shrink on the left while it is too bad

Optimal answer from: your app's interview approach (LeetCode #76)

LeetCode match: LC 76 Minimum Window Substring (inferred from short statement)

Last asked (any company): 7 months ago, about 2026-03 (Exponent shows only a rough age) | +36 interviews

Original: https://www.aced.io/questions/1687/minimum-window-substring

QUESTION

Given strings s and t, return the shortest substring of s that contains every character of t (a character that appears twice in t must appear at least twice in the substring). Return the empty string if there is no such substring.

Function signature

def min_window(s: str, t: str) -> str

EXAMPLES

Example 1	
Input	`'ADOBECODEBANC', 'ABC'`
Output	`'BANC'`
Why	`BANC is the shortest window that holds an A, a B and a C.`

Example 2	
Input	`'a', 'a'`
Output	`'a'`
Why	`The whole string is the only window.`

Example 3	
Input	`'a', 'aa'`
Output	`''`
Why	`t needs two a's but s has only one.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(s, t):
    found = []
    for i in range(len(s)):
        for j in range(i + len(t), len(s) + 1):
            part = s[i:j]
            if contains_all(part, t):
                found.append(part)
    return min(found, key=len, default="")
```

TIME	SPACE
O(n^2 * m)	O(m)

WHY IT IS SLOW

O(n^2) substrings, each checked in O(m) or more. A sliding window that grows until valid and then shrinks from the left is O(n + m).

OPTIMAL SOLUTION · Sliding window with a 'still needed' counter

TIME	SPACE
O(n + m)	O(m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The smallest window containing every distinct character of s itself.

Use t = the set of distinct characters of s and run the same window. A related problem: the longest substring with at most k distinct characters uses a shrink-when-too-many window.

FOLLOW-UP 2 Return all minimum windows, or the number of valid substrings.

For all minimum windows record every window equal to the best length. For counting, when the window [left, right] is valid, every extension to the right is valid too, so add len(s) - right for each left.

FOLLOW-UP 3 Order matters: t must appear as a subsequence.

That is a different problem (LeetCode 727): DP over positions, or a two-pass scan (forward to match, backward to shrink) for each possible end.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find the minimum window substring.
#
# Given strings s and t, return the shortest substring of
# s that contains every character of t (a character that
# appears twice in t must appear at least twice in the
# substring). Return the empty string if there is no such
# substring.
#
# Signature:
#   def min_window(s: str, t: str) -> str
#
```

```
# Examples:
# 'ADOBECODEBANC', 'ABC'  ->  'BANC'
# 'a', 'a'  ->  'a'
#
# FOLLOW-UPS (answered below, in this order):
# 1. The smallest window containing every distinct
# character of s itself.
# 2. Return all minimum windows, or the number of valid
# substrings.
# 3. Order matters: t must appear as a subsequence.
#
# INTERVIEW NOTES
# Expand the right end until the window covers everything
# in t, then shrink from the left as far as it stays
# valid, record the length, and continue. A count of
# 'characters still missing' makes the validity check
# O(1): it goes down when a needed character is added and
# up when one is removed. Each index enters and leaves the
# window once.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def valid_windows(s, t):
    # a sliding window: grow `right` until the window holds all of t, then
    # shrink `left` while it still does. Returns every (left, right) that is
```

```
    # valid at the moment it is seen. `need[c]` is how many more of the letter c
    # the window still requires; `missing` counts the letters of t still missing.
    need = {}
    for ch in t:
        need[ch] = need.get(ch, 0) + 1
    missing = len(t)
    left = 0
    windows = []
    for right in range(len(s)):
        ch = s[right]
        if need.get(ch, 0) > 0:
            missing -= 1
        need[ch] = need.get(ch, 0) - 1
        while missing == 0:
            windows.append((left, right))
            need[s[left]] += 1                      # drop the leftmost letter and see if it hurts
            if need[s[left]] > 0:
                missing += 1
            left += 1
    return windows


# ---- main question: main function ----
# The shortest substring of s that contains every letter of t (with its
# multiplicity); the first one if there is a tie; "" if there is none.
# Complexity: Time O(n + m) for s of length n and t of length m, Space O(m + A)
def min_window(s, t):
    best = None
    width = 0
    for left, right in valid_windows(s, t):
```

```
        if best is None or right - left < width:
            best = (left, right)
            width = right - left
    if best is None:
        return ""
    return s[best[0]:best[1] + 1]


# ---- main question: tests ----
def test_main_question():
    assert min_window("ADOBECODEBANC", "ABC") == "BANC"
    assert min_window("a", "a") == "a"
    assert min_window("a", "aa") == ""
    assert min_window("ab", "b") == "b"
    assert min_window("aab", "aab") == "aab"
```

```
# ============================================================
# FOLLOW-UP 1: The smallest window containing every distinct character of s itself
# ============================================================
# ---- follow-up 1: helper functions ----
def longest_k_distinct(s, k):
    # the related problem: the longest substring with at most k distinct
    # letters, a window that shrinks when it has too many
    counts = {}
    left = 0
    best = 0
    for right in range(len(s)):
        counts[s[right]] = counts.get(s[right], 0) + 1
        while len(counts) > k:
            counts[s[left]] -= 1
            if counts[s[left]] == 0:
                del counts[s[left]]
            left += 1
        if right - left + 1 > best:
            best = right - left + 1
    return best


# ---- follow-up 1: function ----
# Answer: use t = the distinct characters of s and run the same window.
# Complexity: Time O(n + A), Space O(A)
def window_all_distinct(s):
    letters = ""
    for ch in set(s):
        letters += ch
    return min_window(s, letters)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert window_all_distinct("aabcbcdbca") == "dbca"
    assert longest_k_distinct("eceba", 2) == 3
    assert longest_k_distinct("aa", 1) == 2


# ============================================================
# FOLLOW-UP 2: Return all minimum windows, or the number of valid substrings
# ============================================================
# ---- follow-up 2: function ----
```

```
# Answer: for ALL minimum windows keep every window whose length equals the
# best length. For COUNTING: when the window [left, right] is valid, every
# longer end is valid too, so add len(s) - right for each left that is seen.
```

```python
# Complexity: Time O(n + m + output), Space O(m + A)
def all_min_windows(s, t):
    found = valid_windows(s, t)
    if not found:
        return []
    size = None
    for left, right in found:
        if size is None or right - left < size:
            size = right - left
    windows = []
    for left, right in found:
        if right - left == size:
            windows.append(s[left:right + 1])
    return windows


# Complexity: Time O(n + m), Space O(m + A)
def count_valid_substrings(s, t):
    total = 0
    for left, right in valid_windows(s, t):
        total += len(s) - right
    return total


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert all_min_windows("abcabc", "abc") == ["abc", "bca", "cab", "abc"]
    assert all_min_windows("xyz", "q") == []
    assert count_valid_substrings("abcabc", "abc") == 10


# ========================================================
# FOLLOW-UP 3: Order matters: t must appear as a subsequence
# ========================================================
# ---- follow-up 3: function ----
# Answer: a different problem (LeetCode 727). From a start position match t
# in order going FORWARD to find the end of the window, then walk BACKWARDS from
# that end to find the tightest start; continue from just after that start.
# Complexity: Time O(n * m) in the worst case, Space O(1)
def min_window_subsequence(s, t):
    best = None
    width = 0
    i = 0
    while i < len(s):
        j = 0
        k = i
        while k < len(s) and j < len(t):
            if s[k] == t[j]:
                j += 1
            k += 1
        if j < len(t):
            break                                   # the rest of s cannot contain t any more
        end = k - 1
        j = len(t) - 1
        k = end
        while j >= 0:
            if s[k] == t[j]:
                j -= 1
            k -= 1
        start = k + 1
        if best is None or end - start < width:
            best = (start, end)
            width = end - start
        i = start + 1
```

```
    if best is None:
        return ""
    return s[best[0]:best[1] + 1]


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert min_window_subsequence("abcdebdde", "bde") == "bcde"
    assert min_window_subsequence("abc", "ca") == ""


# =========================================================
# RUN EVERYTHING
# =========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

OFFICIAL PAGE TEXT

Upgrade to see our video answerBecome a member today100+ expert answer videosAccess to all coursesPeer-to-peer practicePrivate Discord communityFull question databaseUpgrade nowLewin (Dropbox SWE) answers the minimum window substring interview question.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 41 OF 43

Parent pointers: store who is above each node and climb

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Parent pointers: store who is above each node and climb | 4 questions | tap a title to jump to it

READ FIRST: the code template for these 4 questions (tap here) >

M	188. Employee Hierarchy Add / Delete	1Point3Acres * 2026-06-10 *
M	189. Validate parent array forms a tree	PracHub * 2026-02-11 *
M	190. Root a Tree So It Becomes a Binary Tree	ColorStack * 2025-11-07 *
M	191. Lowest Common Ancestor of a Binary Tree III	ND

CODE TEMPLATE | GROUP 41 OF 43 | 4 QUESTIONS, ONE PATTERN

Parent pointers: store who is above each node and climb

The one idea these questions share

Keep parent[x] (or manager[x]). Climbing is a while loop that follows parent until it runs out. Questions about common ancestors, cycles and who reports to whom are all small variations of that climb.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def climb(x, parent):
    while x is not None:
        yield x
        x = parent.get(x)               # CHANGE 1: stop rule (no parent, or a repeat = cycle)


def lowest_common(a, b, parent):
    above = set(climb(a, parent))
    for x in climb(b, parent):          # CHANGE 2: what you look for while climbing
        if x in above:
            return x
    return None
```

2. Helper functions that usually get written next to it

```
def depth(x, parent):
    return sum(1 for _ in climb(x, parent)) - 1


def has_cycle(parent):
    for start in parent:
        seen, x = set(), start
        while x is not None:
            if x in seen:
                return True
            seen.add(x)
            x = parent.get(x)
    return False
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
188	Employee Hierarchy Add / Delete	manager[emp] and a set of reports per manager; delete re-attaches the reports to the deleted person's manager
189	Validate parent array forms a tree	climb from every node with three states (unseen, on this walk, verified); meeting your own walk is a cycle; there must be exactly one root
190	Root a Tree So It Becomes a Binary Tree	the root has no parent so it needs degree <= 2; everyone else spends one edge on its parent; answer = the first node of degree <= 2
191	Lowest Common Ancestor of a Binary Tree III	two pointers climb; when one falls off the top it restarts at the other node, so both meet at the common ancestor

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 188 of 197 | topic 41 of 43: Parent pointers: store who is above each node and climb | 1 of 4

Template fit: manager[emp] and a set of reports per manager; delete re-attaches the reports to the deleted person's manager | * asked 132 days ago * | frequency Single report

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 188 of 197

Employee Hierarchy Add / Delete

ASKED 2026-06-10 | 132 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 287/288**

ABOUT THIS QUESTION

Topic: Parent pointers: store who is above each node and climb

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-06-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/employee-hierarchy-add-delete

QUESTION

Maintain a company hierarchy. Org(ceo) starts with the CEO. add(emp, mgr) adds a new employee under an existing manager (False if emp already exists or mgr does not). delete(emp) removes an employee and their direct reports move up to the deleted person's manager. The CEO can only be deleted when exactly one person reports to them, who then becomes the CEO.

Function signature

class Org:
  def __init__(self, ceo)
  def add(self, emp, mgr) -> bool
  def delete(self, emp) -> bool

EXAMPLES

Example 1

Input	`o = Org('ceo')`
	`o.add('vp', 'ceo')`
	`o.add('dev', 'vp')`
	`o.add('qa', 'vp')`
	`o.add('dev', 'ceo')`
	`o.delete('vp')`
	`o.manager['dev']`
	`o.delete('ceo')`

Output	`o.add('vp', 'ceo') -> true`
	`o.add('dev', 'vp') -> true`
	`o.add('qa', 'vp') -> true`
	`o.add('dev', 'ceo') -> false`
	`o.delete('vp') -> true`
	`o.manager['dev'] -> 'ceo'`
	`o.delete('ceo') -> false`

Why	`'dev' already exists so the second add fails. Deleting 'vp' moves dev and qa up to the ceo. The ceo now has two reports, so the ceo cannot be deleted.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
manager = {}  # employee -> manager
def delete(e):
    reports = [x for x in manager
               if manager[x] == e]
    for x in reports:
        manager[x] = manager[e]
    del manager[e]
```

TIME	SPACE
O(n) per delete	**O(n)**

WHY IT IS SLOW

Every delete scans all employees, O(n). Keeping a children set per employee makes it O(number of reports).

OPTIMAL SOLUTION · Parent map plus children sets

TIME	SPACE
O(r) per delete where r is the number of reports	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Support moving an employee under another manager and reject cycles.

Walk up from the proposed new manager through manager pointers; if you meet the employee being moved, the move would put a node under its own descendant, so reject it. O(depth).

FOLLOW-UP 2 When a manager leaves, promote the most senior report instead of the grandparent.

Store a hire order (insertion counter) per employee and pick the report with the smallest one: the new manager takes over the other reports. Say how you break ties.

FOLLOW-UP 3 The CEO leaves.

Pick a policy and say it: reject, promote the most senior report (the rest report to them), or leave the company rootless and hold a set of roots. The structure above needs only a promoted root.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Employee Hierarchy Add / Delete
#
# Maintain a company hierarchy. Org(ceo) starts with the
# CEO. add(emp, mgr) adds a new employee under an existing
# manager (False if emp already exists or mgr does not).
# delete(emp) removes an employee and their direct reports
# move up to the deleted person's manager. The CEO can
# only be deleted when exactly one person reports to them,
```

```
# who then becomes the CEO.
#
# Signature:
#   class Org:
#       def __init__(self, ceo)
#       def add(self, emp, mgr) -> bool
#       def delete(self, emp) -> bool
#
# FOLLOW-UPS (answered below, in this order):
# 1. Support moving an employee under another manager and
# reject cycles.
# 2. When a manager leaves, promote the most senior report
# instead of the grandparent.
# 3. The CEO leaves.
#
# INTERVIEW NOTES
# A hierarchy is a tree you update, so store links in both
# directions (parent per node, children per node). Then
# every operation touches only the neighbours of one node
# instead of searching the tree. For each delete case
# (leaf, middle, root) say the policy first, because the
# data structure is easy once the rule is fixed.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main class ----
# Idea: keep two dictionaries: manager[emp] (who is above) and reports[emp]
# (the set of people directly below). add() fills both; delete() hands
# each report of the leaving person to that person's manager and removes
```

```
# the person from the manager's reports.
# Complexity: Time add and reassign are O(1), delete is O(r) for r direct reports, Space O(n)
class Org:
    def __init__(self, ceo):
        self.root = ceo
        self.manager = {ceo: None}
        self.reports = {ceo: set()}
        self.hired = {ceo: 0}          # hire order: a smaller number is more senior (follow-ups 2 and 3)
        self.clock = 1

    def add(self, emp, mgr):
        if emp in self.manager or mgr not in self.manager:
            return False               # already here, or the manager does not exist
        self.manager[emp] = mgr
        self.reports[emp] = set()
        self.reports[mgr].add(emp)
        self.hired[emp] = self.clock
        self.clock += 1
        return True

    def reassign(self, emp, new_mgr):
        # move emp under new_mgr (take it out of the old manager's reports first)
        old = self.manager[emp]
        if old is not None:
            self.reports[old].discard(emp)
        self.manager[emp] = new_mgr
        self.reports[new_mgr].add(emp)
```

```
    def delete(self, emp):
        # the CEO cannot be deleted here (follow-up 3 handles it)
        if emp not in self.manager or self.manager[emp] is None:
            return False
        boss = self.manager[emp]
        for report in list(self.reports[emp]):
            self.reassign(report, boss)        # the reports go to the grandparent
        self.reports[boss].discard(emp)
        del self.manager[emp]
        del self.reports[emp]
        return True


# ---- main question: tests ----
def test_main_question():
    o = Org("ceo")
    for emp, mgr in [("a", "ceo"), ("b", "a"), ("c", "a")]:
        assert o.add(emp, mgr)
    assert not o.add("a", "ceo")               # already there
    assert not o.add("x", "nobody")            # unknown manager
    assert o.delete("a")                       # deleting a manager: reports go to the grandparent
    assert o.manager["b"] == "ceo" and o.manager["c"] == "ceo"
    assert not o.delete("nobody")
    assert not o.delete("ceo")                 # the CEO needs the rules of follow-up 3


# ========================================================
# FOLLOW-UP 1: Support moving an employee under another manager and reject cycles
# ========================================================
# ---- follow-up 1: function ----
# Answer: walk UP from the proposed new manager through the manager
# links. If you meet the employee being moved, the move would put the
# employee under their own descendant (a cycle): reject it. O(depth).
# Complexity: Time O(depth of the new manager) to check for a cycle, Space O(1)
def move(org, emp, new_mgr):
    node = new_mgr
    while node is not None:
        if node == emp:
            return False
        node = org.manager[node]
    org.reassign(emp, new_mgr)
    return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    o = Org("ceo")
    o.add("a", "ceo")
    o.add("b", "a")
    o.add("d", "b")
    assert not move(o, "b", "d")               # d is below b: a cycle
    assert move(o, "d", "a")
    assert o.manager["d"] == "a"


# ========================================================
# FOLLOW-UP 2: When a manager leaves, promote the most senior report instead of the grandparent
# ========================================================
# ---- follow-up 2: helper functions ----
def most_senior(org, people):
    # the person hired earliest
    best = None
```

```
    for person in people:
        if best is None or org.hired[person] < org.hired[best]:
            best = person
    return best


# ---- follow-up 2: function ----
# Answer: the most senior report takes the leaver's place: the other
# reports now report to them, and they report to the leaver's manager.
# Complexity: Time O(r) for r direct reports, Space O(r)
def delete_promote_senior(org, emp):
    if emp not in org.manager or org.manager[emp] is None:
        return False
    boss = org.manager[emp]
    reports = list(org.reports[emp])
    if reports:
        heir = most_senior(org, reports)
        for report in reports:
            if report != heir:
                org.reassign(report, heir)
        org.reassign(heir, boss)
    org.reports[boss].discard(emp)
    del org.manager[emp]
    del org.reports[emp]
    return True


# ---- follow-up 2: tests ----
def test_follow_up_2():
    o = Org("ceo")
    for emp, mgr in [("m", "ceo"), ("x", "m"), ("y", "m")]:
        o.add(emp, mgr)
    assert delete_promote_senior(o, "m")
    assert o.manager["x"] == "ceo"            # x was hired before y, so x is promoted
    assert o.manager["y"] == "x"


# ======================================================
# FOLLOW-UP 3: The CEO leaves
# ======================================================
# ---- follow-up 3: function ----
# Answer: the most senior report becomes the new CEO and the other
# reports go under them. A CEO with nobody below cannot be replaced.
# Complexity: Time O(r) for r direct reports, Space O(r)
def delete_ceo(org):
```

```
    reports = list(org.reports[org.root])
    if not reports:
        return False
    heir = most_senior(org, reports)
    for report in reports:
        if report != heir:
            org.reassign(report, heir)
    del org.manager[org.root]
    del org.reports[org.root]
    org.root = heir
    org.manager[heir] = None
    return True


# ---- follow-up 3: tests ----
def test_follow_up_3():
    o = Org("ceo")
    o.add("a", "ceo")
    o.add("b", "ceo")
    assert delete_ceo(o)
```

```
    assert o.root == "a" and o.manager["b"] == "a"
    assert not delete_ceo(Org("solo"))


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement tree mutation operations with parent pointers.
Practice deleting an internal node while reconnecting children.
Add cycle checks with DFS or parent-chain walking before accepting a new edge.

STUDY PATH: question 189 of 197 | topic 41 of 43: Parent pointers: store who is above each node and climb | 2 of 4

Template fit: climb from every node with three states (unseen, on this walk, verified); meeting your own walk is a cycle; there must be exactly one root | * asked 251 days ago *

MEDIUM | IN APP LC 261 | PracHub | Technical Screen | 189 of 197

Validate parent array forms a tree

POSTED 2026-02-11 | 251 DAYS AGO | OLDER

STUDY: TALK ALL TIME 29 30 DAYS 0 PRIORITY 148/288

ABOUT THIS QUESTION

Topic: Parent pointers: store who is above each node and climb

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #261 Graph Valid Tree

LeetCode match: LC 261 Graph Valid Tree

Posted on PracHub: 2026-02-11 (updated 2026-04-23)

Original: https://prachub.com/coding-questions/validate-parent-array-forms-a-tree

QUESTION

Problem

You are given an integer array **`parent`** of length **`n`** describing a directed parent pointer for each node **`i`** (nodes are labeled **`0..n-1`**).

- **`parent[i]`** is the parent of node **`i`**.
- Exactly one node should be the root, indicated by **`parent[root] = -1`**. Determine whether these **`n`** nodes form a **valid rooted tree**. A valid rooted tree must satisfy all of the following: 1.

Exactly one root (exactly one index **`i`** with **`parent[i] = -1`**). 2.
No cycles (following parent pointers from any node must eventually reach the root). 3.
Connectivity (every node is reachable from the root; equivalently, every node has exactly one simple path to the root).

Input

- **`parent`**: integer array of length **`n`** where each value is either **`-1`** or in **`[0, n-1]`**.

Output

- Return **`true`** if **`parent`** represents a valid rooted tree; otherwise return **`false`**.

Notes

/ Edge Cases - **`n`** can be 1 (then the only valid tree is **`parent[0] = -1`**).

- Self-parenting like **`parent[i] = i`** is invalid.
- Multiple **`-1`** entries (multiple roots) is invalid.
- A cycle among non-root nodes is invalid.
- A disconnected component (some nodes not reachable from the root) is invalid.

You are given an integer array **`parent`** of length **`n`** describing a parent pointer for each node **`i`** where nodes are labeled **`0`** to **`n-1`**.

- **`parent[i]`** is the parent of node **`i`**.
- Exactly one node should be the root, indicated by **`parent[root] = -1`**. Determine whether these nodes form a valid rooted tree. A valid rooted tree must satisfy all of the following:

1. Exactly one root.
2. No cycles.

QUESTION

3. Every node belongs to the same connected structure and has a path to the root. Return **`True`** if **`parent`** represents a valid rooted tree; otherwise return **`False`**.

Constraints

1 <= n <= 2 * 10^5Each **`parent[i]`** is either **`-1`** or an integer in **`[0, n-1]parent[i] = i`** is invalid

EXAMPLES

```
Input: [-1, 0, 0, 1, 1]
Expected Output: True
```

Explanation: Node 0 is the only root. All other nodes are connected under it, and there are no cycles.

```
Input: [2, -1, 1, 2]
Expected Output: True
Explanation: Node 1 is the root. The paths are 0->2->1, 2->1, and 3->2->1, so every node reaches the
    same root.
```

Hints

First count how many roots there are. A valid tree must have exactly one **`-1`** entry.Build children lists from the parent array, then traverse from the root. If you do not visit every node, the structure is disconnected or contains a cycle away from the root.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_tree(parent):
    n = len(parent)
    roots = [i for i in range(n)
             if parent[i] == -1]
    if len(roots) != 1:
        return False
    if any(p >= n for p in parent):
        return False
    for node in range(n):
        cur, steps = node, 0
        while cur != -1 and steps <= n:
            cur = parent[cur]
            steps += 1
        if cur != -1:
            return False  # a cycle
    return True
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Every node walks its whole chain, O(n^2). Marking nodes as 'on this walk' or 'verified' lets every node be processed once, O(n).

OPTIMAL SOLUTION · One root, then colour each node once while walking up

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The graph is given as undirected edges (LeetCode 261).

A tree has exactly n - 1 edges and no cycle: check the edge count, then union endpoints and fail on an edge that joins two nodes already connected. O(n alpha(n)).

FOLLOW-UP 2 Report the reason it is invalid.

Return which rule failed: number of roots, an out-of-range parent, or the node where the cycle was detected (the node that was met twice on one walk).

FOLLOW-UP 3 A binary tree given as child arrays (LeetCode 1361).

Check that exactly one node has in-degree 0, every other node has in-degree 1, and that a traversal from the root visits all n nodes.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Validate parent array forms a tree
#
# ## Problem You are given an integer array parent of
# length n describing a directed parent pointer for each
# node i (nodes are labeled 0..n-1). - parent[i] is the
# parent of node i. - Exactly one node should be the root,
# indicated by parent[root] = -1. Determine whether these
# n nodes form a **valid rooted tree**. A valid rooted
# tree must satisfy all of the following: 1. **Exactly one
# root** (exactly one index i with parent[i] = -1). 2.
# **No cycles** (following parent pointers from any node
# must eventually reach the root). 3. **Connectivity**
# (every node is reachable from the root; equivalently,
# every node has exactly one simple path to the root). ##
# Input - parent: integer array of length n where each
# value is either -1 or in [0, n-1]. ## Output - Return
# true if parent represents a valid rooted tree; otherwise
# return false.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The graph is given as undirected edges (LeetCode
# 261).
# 2. Report the reason it is invalid.
# 3. A binary tree given as child arrays (LeetCode 1361).
#
# INTERVIEW NOTES
# Exactly one -1 gives the root; then the rest is whether
# every node reaches it. Following parents from each node
# can loop (a cycle) or run out of range. Do it with
# memoised walks so every node is processed once: mark
# nodes on the current walk, and if the walk meets one of
# them again it is a cycle; if it meets a node already
# known to reach the root, all nodes on the walk reach it
# too.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
def reason(parent):
    # None if parent[] describes a tree, else the first rule that fails.
    # parent[i] is the parent of node i; the root has parent -1.
    n = len(parent)
    roots = 0
    for p in parent:
        if p == -1:
            roots += 1
    if roots != 1:
        return str(roots) + " roots (need exactly 1)"
    for i in range(n):
        p = parent[i]
        if p != -1 and not (0 <= p < n):
            return "node " + str(i) + ": out-of-range parent " + str(p)
        if p == i:
            return "node " + str(i) + " is its own parent"
    # walk up from every node; 0 = unseen, 1 = on the current walk, 2 = already checked
    state = [0] * n
    for start in range(n):
        path = []
        x = start
        while x != -1 and state[x] == 0:
            state[x] = 1
            path.append(x)
            x = parent[x]
        if x != -1 and state[x] == 1:
            return "cycle through node " + str(x)        # met a node twice on one walk
        for v in path:
            state[v] = 2
    return None


# ---- main question: main function ----
# Idea: exactly one root (-1), every other parent is a real node, and no
# walk upwards ever meets a node twice (that would be a cycle). Each
# node is coloured once, so the whole check is O(n).
# Complexity: Time O(n), Space O(n)
def validate_parent_array(parent):
    return reason(parent) is None


# ---- main question: tests ----
def test_main_question():
    assert validate_parent_array([-1, 0, 0, 1])
```

```
    assert validate_parent_array([-1])
    assert not validate_parent_array([-1, -1])            # two roots
    assert not validate_parent_array([1, 0])              # no root, a cycle
    assert not validate_parent_array([-1, 5])             # parent out of range


# ============================================================
# FOLLOW-UP 1: The graph is given as undirected edges (LeetCode 261)
# ============================================================
# ---- follow-up 1: function ----
# Answer: a tree on n nodes has exactly n - 1 edges and no cycle. Check
# the edge count, then union the ends of each edge (union-find) and fail
# on an edge whose ends are already in the same group.
# Complexity: Time O(n * alpha(n)) (union-find), Space O(n)
```

```
def valid_tree_edges(n, edges):
    if len(edges) != n - 1:
        return False
    leader = list(range(n))

    def find(x):
        while leader[x] != x:
            leader[x] = leader[leader[x]]
            x = leader[x]
        return x

    for u, v in edges:
        a = find(u)
        b = find(v)
        if a == b:
            return False                                    # this edge closes a cycle
        leader[a] = b
    return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert valid_tree_edges(5, [(0, 1), (0, 2), (0, 3), (1, 4)])
    assert not valid_tree_edges(4, [(0, 1), (1, 2), (2, 0)])


# =========================================================
# FOLLOW-UP 2: Report the reason it is invalid
# =========================================================
# Answer: reason() above already returns which rule failed: the number of
# roots, an out-of-range parent, a node that is its own parent, or the
```

```
# node where a cycle was found.
# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert "cycle" in reason([-1, 2, 1])
    assert "out-of-range" in reason([-1, 5])
    assert "own parent" in reason([-1, 1])
    assert "2 roots" in reason([-1, -1])
    assert reason([-1, 0]) is None


# =========================================================
# FOLLOW-UP 3: A binary tree given as child arrays (LeetCode 1361)
# =========================================================
# ---- follow-up 3: function ----
# Answer: exactly one node has nobody pointing to it (the root), every
# other node is pointed to exactly once, and walking from the root
# reaches all n nodes. -1 means "no child".
# Complexity: Time O(n), Space O(n)
def valid_binary_tree(n, left, right):
    pointed_to = [0] * n
    for children in (left, right):
        for c in children:
            if c != -1:
                pointed_to[c] += 1
    roots = []
    for i in range(n):
        if pointed_to[i] == 0:
            roots.append(i)
        if pointed_to[i] > 1:
            return False                                    # two parents
    if len(roots) != 1:
        return False
    seen = {roots[0]}
```

```
    stack = [roots[0]]
    while stack:
        u = stack.pop()
        for c in (left[u], right[u]):
            if c != -1:
                if c in seen:
                    return False
                seen.add(c)
                stack.append(c)
    return len(seen) == n                                   # every node must be reachable from the root
```

```
# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert valid_binary_tree(4, [1, -1, 3, -1], [2, -1, -1, -1])
    assert not valid_binary_tree(2, [1, 0], [-1, -1])


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(parent: list[int]) -> bool:
SAMPLE INPUT
[-1, 0, 0, 1, 1]
```

Test cases

Input	Expected
[-1, 0, 0, 1, 1]	True
[2, -1, 1, 2]	True

STUDY PATH: question 190 of 197 | topic 41 of 43: Parent pointers: store who is above each node and climb | 3 of 4

Template fit: the root has no parent so it needs degree <= 2; everyone else spends one edge on its parent; answer = the first node of degree <= 2 | * asked 347 days ago * | frequency Single report

MEDIUM | ColorStack | technical interview | 190 of 197

Root a Tree So It Becomes a Binary Tree

ASKED 2025-11-07 | 347 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 230/288

ABOUT THIS QUESTION

Topic: Parent pointers: store who is above each node and climb

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1245 Tree Diameter (related: rooting a tree, degrees)

Frequency: Single report | Last asked: 2025-11-07 | Roles: SWE Intern

QUESTION

You are given a connected, undirected, acyclic graph (i.e. a tree) with n nodes labeled 0 .. n-1 , described by an edge list. It is guaranteed that every node has at most 3 neighbors. Choose a node to be the root. When rooted at node r , one neighbor of each non-root node becomes its parent and the rest become its children; the root has no parent, so all of its neighbors become children. The rooted tree is a binary tree if every node has at most 2 children. Return the smallest label of a node that can serve as such a root. If no node works, return -1 . Key observation (the trick): a non-root node with degree d has d - 1 children, so it needs d <= 3 ; the root has degree children, so the root needs degree <= 2 . Since all nodes already have degree <= 3 , any node with degree <= 2 is a valid root — and leaves (degree 1) always exist in a nonempty tree, so a valid root always exists for n >= 1 .

Function signature

```
from typing import List, Tuple
def find_binary_tree_root(n: int, edges: List[Tuple[int, int]]) -> int:
    """Return the smallest node label that can root the tree as a binary tree
    (every node has at most 2 children), or -1 if none exists."""
    ...
```

Follow-up (as reported): Now nodes must be 2-colored (black/white) so that every parent differs in color from its children and all nodes at the same depth share a color. Determine whether a valid root plus coloring exists and return one. Hint: coloring each node by depth % 2 from any chosen root always satisfies both rules in a tree, so a valid coloring exists whenever a valid root does.

EXAMPLES

Example 1	
Input	`n = 4, edges = [(0, 1), (0, 2), (0, 3)]`
Output	`1`
Why	`Node 0 has degree 3, so as root it would have 3 children (invalid). Nodes 1, 2, 3 have degree 1; the smallest valid root is 1.`

Example 2	
Input	`n = 3, edges = [(0, 1), (1, 2)]`
Output	`0`
Why	`A path; every node has degree <= 2, so the smallest label 0 works.`

QUESTION

Example 3	
Input	`n = 1, edges = []`
Output	`0`
Why	`A single node is trivially a binary tree.`

CONSTRAINTS

- 1 <= n <= 10^5 ; the input is a valid tree with n - 1 edges (for n == 1 , no edges).
- Every node has at most 3 neighbors.
- Return the smallest valid root label.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_root(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    for root in range(n):
        parent = {root: -1}
        stack, ok = [root], True
        while stack:
            u = stack.pop()
            kids = [
                v
                for v in adj[u]
                if v != parent[u]
            ]
            ok = ok and len(kids) <= 2
            for v in kids:
                parent[v] = u
                stack.append(v)
        if ok:
            return root
    return -1
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

It re-roots the whole tree for each candidate even though only the root's degree matters.

OPTIMAL SOLUTION · First node of degree at most 2

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Also 2-colour the nodes so a parent and its children differ and equal depths share a colour.

Colour each node by depth % 2 from the chosen root; in a tree this satisfies both rules, so a valid colouring exists whenever a valid root does.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Root a Tree So It Becomes a Binary Tree
#
# You are given a connected, undirected, acyclic graph
# (i.e. a tree) with n nodes labeled 0 .. n-1 , described
# by an edge list. It is guaranteed that every node has at
# most 3 neighbors. Choose a node to be the root. When
# rooted at node r , one neighbor of each non-root node
# becomes its parent and the rest become its children; the
# root has no parent, so all of its neighbors become
# children. The rooted tree is a binary tree if every node
# has at most 2 children. Return the smallest label of a
# node that can serve as such a root. If no node works,
# return -1 . Key observation (the trick): a non-root node
# with degree d has d - 1 children, so it needs d <= 3 ;
# the root has degree children, so the root needs degree
# <= 2 .
#
# FOLLOW-UPS (answered below, in this order):
# 1. Also 2-colour the nodes so a parent and its children
# differ and equal depths share a colour.
#
# INTERVIEW NOTES
# Root the tree at r. Every non-root node has one parent
# edge, so it has degree - 1 children; the root has degree
# children. A node is fine as long as its children number
# at most 2, and degrees are at most 3 already. So only
# the root can break the rule, and any node of degree 2 or
# less is a valid root. The smallest label with degree at
# most 2 is the answer.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
```

```
# Idea: in a tree where every node may have at most 2 children, only the
# root has as many children as its degree; every other node has one
# FEWER (one edge goes to its parent). So any node of degree 2 or less
# can be the root, and the first one is a valid choice. -1 if there is none.
# Complexity: Time O(n), Space O(n)
def find_binary_tree_root(n, edges):
    degree = [0] * n
```

```
    for a, b in edges:
        degree[a] += 1
        degree[b] += 1
    for node in range(n):
        if degree[node] <= 2:
            return node
    return -1


# ---- main question: tests ----
def test_main_question():
    assert find_binary_tree_root(4, [(0, 1), (0, 2), (0, 3)]) == 1     # node 0 has three neighbours
    assert find_binary_tree_root(3, [(0, 1), (1, 2)]) == 0
    assert find_binary_tree_root(1, []) == 0


# ========================================================
# FOLLOW-UP 1: Also 2-colour the nodes so a parent and its children differ and equal depths share a colour
# ========================================================
# ---- follow-up 1: function ----
# Answer: colour each node by its depth from the chosen root, modulo 2
# (0, 1, 0, 1 ...). In a tree that gives a parent and its children
# different colours and every depth one colour, so a valid colouring
# exists whenever a valid root does.
# Complexity: Time O(n), Space O(n)
def root_and_coloring(n, edges):
    root = find_binary_tree_root(n, edges)
    if root == -1:
        return None
    adj = []
    for i in range(n):
        adj.append([])
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    color = {root: 0}
    queue = deque([root])
    while queue:
        u = queue.popleft()
        for v in adj[u]:
            if v not in color:
                color[v] = 1 - color[u]
                queue.append(v)
    colors = []
    for i in range(n):
        colors.append(color[i])
    return root, colors


# ---- follow-up 1: tests ----
def test_follow_up_1():
    random.seed(81)
    for round_number in range(300):                          # random trees where no node has more than 3 ne
ighbours
        n = random.randint(1, 10)
        degree = [0] * n
        edges = []
        for v in range(1, n):
            options = []
            for u in range(v):
                if degree[u] < 3:
                    options.append(u)
            u = random.choice(options)
            if random.random() < 0.5:
```

```
                edges.append((u, v))
            else:
                edges.append((v, u))
            degree[u] += 1
            degree[v] += 1
        root, color = root_and_coloring(n, edges)
        assert color[root] == 0
        for a, b in edges:
            assert color[a] != color[b]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 191 of 197 | topic 41 of 43: Parent pointers: store who is above each node and climb | 4 of 4

Template fit: two pointers climb; when one falls off the top it restarts at the other node, so both meet at the common ancestor | ND

MEDIUM | IN APP LC 1650 | HackTheRounds | Phone Screen | 191 of 197

Lowest Common Ancestor of a Binary Tree III

STUDY: TALK ALL TIME 14.7 30 DAYS 0 PRIORITY 185/288

ABOUT THIS QUESTION

Topic: Parent pointers: store who is above each node and climb

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1650 Lowest Common Ancestor of a Binary Tree III

LeetCode match: LC 1650

QUESTION

Given two nodes p and q of a binary tree, return their lowest common ancestor: the deepest node that has both of them as descendants (a node counts as a descendant of itself). Every node has a parent pointer, but you are not given the root.

Function signature

def lca_swap(p: PNode, q: PNode) -> PNode

EXAMPLES

Example 1

Input

```
class PNode:
def __init__(self, val):
self.val, self.parent = val, None
root, a, b, c = PNode(3), PNode(5), PNode(1), PNode(6)
a.parent = b.parent = root
c.parent = a
lca_swap(c, b).val
lca_swap(c, a).val
lca_swap(a, b).val
```

Output

```
lca_swap(c, b).val -> 3
lca_swap(c, a).val -> 5
lca_swap(a, b).val -> 3
```

Why

Node 6 hangs under 5, which hangs under the root 3 next to node 1. The ancestor of 6 and 1 is the root 3; the ancestor of 6 and its own parent 5 is 5 itself.

CONSTRAINTS

- Number of nodes: [2, 10^5]
- -10^9 <= Node.val <= 10^9
- All node values are unique
- p != q
- p and q exist in the tree

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def lca(p, q):
    ancestors = set()
    node = p
    while node:
        ancestors.add(node)
        node = node.parent
    node = q
    while node:
        if node in ancestors:
            return node
        node = node.parent
    return None
```

TIME	SPACE
O(h)	O(h)

WHY IT IS SLOW

Uses O(h) extra space for the set. Two pointers that switch to the other node's start when they fall off the root meet at the answer with O(1) space.

OPTIMAL SOLUTION · Two pointers that switch to the other node at the end

TIME	SPACE
O(h)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it without the swap trick (it feels like magic).

Compute the depth of each node by walking up, lift the deeper one by the difference, then step both up together until they meet. Same O(h) time and O(1) space and easier to justify.

FOLLOW-UP 2 Many LCA queries on the same tree.

Preprocess with binary lifting (up[k][v] is the 2^k-th ancestor): O(n log n) to build, O(log n) per query. Or an Euler tour with a range-minimum structure for O(1) queries.

FOLLOW-UP 3 The nodes might not be in the same tree.

Then the two pointers both end as None after two passes, and None is the correct answer. The loop terminates because both pointers reach None at the same step.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Lowest Common Ancestor of a Binary Tree
# III
#
# Given two nodes p and q of a binary tree, return their
# lowest common ancestor: the deepest node that has both
# of them as descendants (a node counts as a descendant of
# itself). Every node has a parent pointer, but you are
```

```
# not given the root.
#
# Signature:
#   def lca_swap(p: PNode, q: PNode) -> PNode
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it without the swap trick (it feels like magic).
# 2. Many LCA queries on the same tree.
# 3. The nodes might not be in the same tree.
#
# INTERVIEW NOTES
# With parent pointers each node is the head of a linked
# list that ends at the root, so the question is where two
# linked lists meet. If the lists have different lengths,
# walk each pointer to the end and then restart it on the
# other node: both pointers then travel the same total
# distance and arrive at the meeting node together.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class Node:
    # every node knows its parent (the root's parent is None)
    def __init__(self, val, parent=None):
        self.val = val
        self.parent = parent
        self.left = None
        self.right = None


def attach(parent, val, side):
    # create a child on the "left" or "right" of parent and link it both ways
    child = Node(val, parent)
    setattr(parent, side, child)
    return child


def lca_set(p, q):
    # the first idea: write down every ancestor of p (including p), then
    # climb from q until we meet one of them. O(h) time and space.
    seen = set()
    while p:
        seen.add(p)
        p = p.parent
    while q:
        if q in seen:
            return q
        q = q.parent
    return None


# ---- main question: main function ----
# Idea: two pointers climb towards the root, and when one falls off the
```

```
# top it restarts at the OTHER node. Both then walk the same total
# distance (the two path lengths added), so they meet exactly at the
# lowest common ancestor. O(h) time, O(1) space. If the nodes are in
# different trees both end up as None together.
# Complexity: Time O(h) for the height h of the tree, Space O(1)
def lca_swap(p, q):
    a = p
    b = q
    while a is not b:
        if a:
            a = a.parent
        else:
            a = q
        if b:
            b = b.parent
        else:
            b = p
    return a
```

```
# ---- main question: tests ----
def example_tree():
    root = Node(3)
    five = attach(root, 5, "left")
    one = attach(root, 1, "right")
    six = attach(five, 6, "left")
    two = attach(five, 2, "right")
    four = attach(two, 4, "left")
    return root, five, one, six, two, four


def test_main_question():
    root, five, one, six, two, four = example_tree()
    assert lca_swap(six, four) is five
    assert lca_swap(five, four) is five
    assert lca_swap(six, one) is root
    assert lca_swap(root, root) is root
    assert lca_swap(six, Node(99)) is None                # a node of another tree
    assert lca_set(six, four) is five


# ========================================================
# FOLLOW-UP 1: Do it without the swap trick (it feels like magic)
# ========================================================
# ---- follow-up 1: helper functions ----
def depth(node):
    # how many steps up to the root
    steps = 0
    while node.parent:
        node = node.parent
        steps += 1
    return steps


# ---- follow-up 1: function ----
# Answer: measure each node's depth, lift the deeper one up by the
# difference, then climb both together until they are the same node.
# Same O(h) time and O(1) space, and easier to justify.
# Complexity: Time O(h), Space O(1)
def lca_by_depth(p, q):
    dp = depth(p)
    dq = depth(q)
    while dp > dq:
        p = p.parent
```

```
        dp -= 1
    while dq > dp:
        q = q.parent
        dq -= 1
    while p is not q:
        p = p.parent
        q = q.parent
        if p is None or q is None:
            return None                                  # they never met: different trees
    return p


# ---- follow-up 1: tests ----
def test_follow_up_1():
    root, five, one, six, two, four = example_tree()
    assert lca_by_depth(six, Node(99)) is None
    assert lca_by_depth(four, one) is root
    assert lca_by_depth(six, four) is five


# =======================================================
# FOLLOW-UP 2: Many LCA queries on the same tree
# =======================================================
# ---- follow-up 2: function ----
# Answer: preprocess with BINARY LIFTING: up[k][v] is the 2^k-th
# ancestor of node v. Building costs O(n log n); each query is O(log n):
# first lift the deeper node to the other's depth, then lift both as
# high as possible while their ancestors still differ. (Or an Euler tour
# with a range-minimum table for O(1) queries.)
# Complexity: Time build O(n log n), each query O(log n), Space O(n log n)
class LiftingLCA:
    def __init__(self, root):
        self.index = {root: 0}               # node -> its number
        self.nodes = [root]                  # number -> node (breadth first order)
        parent = [0]
        self.depth = [0]
        i = 0
        while i < len(self.nodes):
            node = self.nodes[i]
            for child in (node.left, node.right):
                if child:
                    self.index[child] = len(self.nodes)
                    self.nodes.append(child)
                    parent.append(i)
                    self.depth.append(self.depth[i] + 1)
            i += 1
        self.up = [parent]
        count = len(self.nodes)
        while (1 << len(self.up)) < count:
            previous = self.up[-1]
            row = []
            for v in range(count):
                row.append(previous[previous[v]])   # two jumps of 2^k make one jump of 2^(k+1)
            self.up.append(row)

    def query(self, p, q):
        a = self.index[p]
        b = self.index[q]
        if self.depth[a] < self.depth[b]:
            a, b = b, a
        gap = self.depth[a] - self.depth[b]
        k = 0
        while gap > 0:                           # lift a by `gap`, one binary digit at a time
            if gap % 2 == 1:
                a = self.up[k][a]
```

```
            gap = gap // 2
            k += 1
        if a == b:
            return self.nodes[a]
        for k in range(len(self.up) - 1, -1, -1):
            if self.up[k][a] != self.up[k][b]:
                a = self.up[k][a]
                b = self.up[k][b]
        return self.nodes[self.up[0][a]]


# ---- follow-up 2: tests ----
def random_tree(size):
    root = Node(0)
    nodes = [root]
    while len(nodes) < size:
        parent = random.choice(nodes)
        side = random.choice(["left", "right"])
        if getattr(parent, side) is None:
            nodes.append(attach(parent, len(nodes), side))
    return nodes


def test_follow_up_2():
    root, five, one, six, two, four = example_tree()
```

```
    index = LiftingLCA(root)
    assert index.query(six, four) is five
    assert index.query(root, four) is root
    random.seed(5)
    for round_number in range(50):                          # random trees: all versions must agree
        nodes = random_tree(random.randint(1, 12))
        index = LiftingLCA(nodes[0])
        p = random.choice(nodes)
        q = random.choice(nodes)
        assert index.query(p, q) is lca_swap(p, q) is lca_by_depth(p, q)


# ============================================================
# FOLLOW-UP 3: The nodes might not be in the same tree
# ============================================================
# Answer: lca_swap returns None when the nodes are in different trees
# (both pointers reach None together), and lca_by_depth stops with None
# when a pointer runs off the top first.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 42 OF 43

Build or transform a grid with index arithmetic

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Build or transform a grid with index arithmetic | 4 questions | tap a title to jump to it

READ FIRST: the code template for these 4 questions (tap here) >

M	192. Connected Crop Arrangement on a Grid	1Point3Acres * 2026-10-04 *
M	193. Solve matrix groups and recipe inventory	PracHub * 2026-01-22 *
M	194. Range Sum Query 2D (Immutable Matrix)	ColorStack * 2025-08-19 *
M	195. Rotate Any m×n 2D Matrix	ND

CODE TEMPLATE | GROUP 42 OF 43 | 4 QUESTIONS, ONE PATTERN

Build or transform a grid with index arithmetic

The one idea these questions share

Decide where each output cell comes from with a formula in (r, c), then build the whole grid with one comprehension. Rotation, diamonds, block summaries and prefix tables are all 'cell(r, c) = something computed from r and c'.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def build_grid(rows, cols, cell):
    return [[cell(r, c) for c in range(cols)] for r in range(rows)]    # CHANGE: the formula for one cell
```

2. Helper functions that usually get written next to it

```
def prefix_2d(matrix):
    rows, cols = len(matrix), len(matrix[0])
    p = [[0] * (cols + 1) for _ in range(rows + 1)]
    for r in range(rows):
        for c in range(cols):
            p[r + 1][c + 1] = matrix[r][c] + p[r][c + 1] + p[r + 1][c] - p[r][c]
    return p


def snake_cells(rows, cols):
    return [(r, c if r % 2 == 0 else cols - 1 - c) for r in range(rows) for c in range(cols)]
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
192	Connected Crop Arrangement on a Grid	fill the cells in snake order (left to right, then right to left) so equal crops stay in one connected strip
193	Solve matrix groups and recipe inventory	block (r // k, c // k) holds the minimum of every cell that maps to it; the recipe part is a topological sort
194	Range Sum Query 2D (Immutable Matrix)	prefix table p[r + 1][c + 1] = cell + p[r][c + 1] + p[r + 1][c] - p[r][c]; a query is four lookups
195	Rotate Any m×n 2D Matrix	clockwise: new[r][c] = old[rows - 1 - c][r] on a cols-by-rows result (or zip(*matrix[::-1]))

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 192 of 197 | topic 42 of 43: Build or transform a grid with index arithmetic | 1 of 4

Template fit: fill the cells in snake order (left to right, then right to left) so equal crops stay in one connected strip | * asked 16 days ago * | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 192 of 197

Connected Crop Arrangement on a Grid

ASKED 2026-10-04 | 16 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 267/288**

ABOUT THIS QUESTION

Topic: Build or transform a grid with index arithmetic

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-10-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/connected-crop-arrangement-grid

QUESTION

A garden is an n x m grid. There are k crop types numbered 1..k, and crop i must fill exactly counts[i - 1] cells; the counts add up to n * m so every cell is used. Fill the grid so that the cells of each crop form one connected region (cells joined up, down, left or right). Return any valid grid.

Function signature

def arrange_crops(n: int, m: int, counts: list) -> list

EXAMPLES

Example 1	
Input	`2, 3, [2, 3, 1]`
Output	`[[1, 1, 2], [3, 2, 2]]`
Why	`One valid answer: crop 3 is the single cell at the bottom left; crops 1 and 2 are each connected.`

Example 2	
Input	`1, 4, [1, 3]`
Output	`[[1, 2, 2, 2]]`
Why	`A single row: the crops simply sit side by side.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def fill(crop):
    if crop == len(counts):
        return True
    size = counts[crop]
    for cells in combinations(free_cells, size):
        if is_connected(cells):
            place(cells, crop)
            if fill(crop + 1):
                return True
            remove(cells)
    return False
```

BRUTE FORCE

TIME	SPACE
exponential	O(n * m)

WHY IT IS SLOW

Exponential in the number of cells. Filling the crops along a snake path through the grid is O(n * m) and every crop stays connected by construction.

OPTIMAL SOLUTION · Fill crops along a snake path

TIME	SPACE
O(n * m)	O(n * m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Two adjacent square gardens (N x N next to M x M).

Use a different Hamiltonian path of the combined L-shaped area: snake down and up the columns of the first garden so that the last column ends in the top row, step into the second garden there and snake through it. Hand out the crop counts along that path exactly as before. The code checks that this path visits every cell once with each step adjacent, for every size from 1 to 6.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Connected Crop Arrangement on a Grid
#
# A garden is an n x m grid. There are k crop types
# numbered 1..k, and crop i must fill exactly counts[i -
# 1] cells; the counts add up to n * m so every cell is
# used. Fill the grid so that the cells of each crop form
# one connected region (cells joined up, down, left or
# right). Return any valid grid.
#
# Signature:
#   def arrange_crops(n: int, m: int, counts: list) -> list
#
# Examples:
# 2, 3, [2, 3, 1]  ->  [[1, 1, 2], [3, 2, 2]]
# 1, 4, [1, 3]  ->  [[1, 2, 2, 2]]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Two adjacent square gardens (N x N next to M x M).
#
# INTERVIEW NOTES
# Walk the grid in a snake (left to right on one row,
# right to left on the next) so that every cell is next to
# the one before it. If you give crop 1 the first count[1]
# cells of that walk, crop 2 the next count[2] cells and
# so on, each crop is a contiguous stretch of the walk, so
# its cells are connected. No search or backtracking is
# needed.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
```

```
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def snake_cells(n, m):
    # the cells of an n x m grid in "snake" order: row 0 left to right,
    # row 1 right to left, row 2 left to right, ... Consecutive cells in
    # this order always share an edge.
    path = []
    for i in range(n):
```

```
        for j in range(m):
            if i % 2 == 0:
                path.append((i, j))
            else:
                path.append((i, m - 1 - j))
    return path


def fill_along(grid, path, counts):
    # hand out the path: the first counts[0] cells to crop 1, the next
    # counts[1] cells to crop 2, and so on
    k = 0
    for crop in range(1, len(counts) + 1):
        for i in range(counts[crop - 1]):
            r, c = path[k]
            grid[r][c] = crop
            k += 1
    return grid


# ---- main question: main function ----
# Idea: every crop must be one connected piece. Take the snake order of
# the cells and give each crop a consecutive stretch of it. Neighbouring
# cells of the snake share an edge, so each stretch (one crop) is connected.
# counts[i] is how many cells crop i + 1 needs; together they fill the grid.
# Complexity: Time O(n * m), Space O(n * m)
def arrange_crops(n, m, counts):
    total = 0
    for count in counts:
        total += count
    if total != n * m:
        raise ValueError("the counts must add up to n * m")
    grid = []
    for r in range(n):
        grid.append([0] * m)
    return fill_along(grid, snake_cells(n, m), counts)


# ---- main question: tests ----
def crops_valid(grid, counts):
    # does every crop have the right number of cells, all in one connected piece?
    cells = {}
    for i in range(len(grid)):
        for j in range(len(grid[0])):
            crop = grid[i][j]
```

```
            if crop != 0:
                if crop not in cells:
                    cells[crop] = set()
                cells[crop].add((i, j))
```

```
    for crop in range(1, len(counts) + 1):
        group = cells.get(crop, set())
        if len(group) != counts[crop - 1]:
            return False
        start = None
        for cell in group:
            start = cell
        seen = set()
        stack = [start]
        while stack:                                    # a flood fill from one cell must reach them all
            r, c = stack.pop()
            if (r, c) in seen:
                continue
            seen.add((r, c))
            for neighbour in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
                if neighbour in group:
                    stack.append(neighbour)
        if seen != group:
            return False
    return True


def test_main_question():
    assert crops_valid(arrange_crops(2, 3, [2, 4]), [2, 4])
    ones = [1] * 9
    assert crops_valid(arrange_crops(3, 3, ones), ones)            # many tiny crops still connect
    assert crops_valid(arrange_crops(4, 5, [7, 1, 12]), [7, 1, 12])
    try:
        arrange_crops(2, 2, [1, 1])
        raise AssertionError("the counts do not fill the grid")
    except ValueError:
        pass


# ========================================================
# FOLLOW-UP 1: Two adjacent square gardens (N x N next to M x M)
# ========================================================
# ---- follow-up 1: helper functions ----
def two_gardens_path(n, m):
    # garden A is n x n on the left, garden B is m x m on its right (both start at row 0)
    path = []
```

```
    for c in range(n):                                  # snake down and up the columns of garden A
        going_down = (c % 2 == 0) == (n % 2 == 0)       # chosen so that the LAST column ends in row 0
        if going_down:
            rows = list(range(n))
        else:
            rows = list(range(n - 1, -1, -1))
        for r in rows:
            path.append((r, c))
    for r in range(m):                                  # step into garden B at row 0 and snake through it
        if r % 2 == 0:
            cols = list(range(n, n + m))
        else:
            cols = list(range(n + m - 1, n - 1, -1))
        for c in cols:
            path.append((r, c))
    return path


# ---- follow-up 1: function ----
# Answer: use a different path through the combined area: snake down and
# up the columns of the first garden so that its last column ends in the
# top row, step into the second garden there and snake through it. Every
# step of this path moves to a neighbouring cell, so each crop is
```

```
# connected, and the crops are handed out along the path as before.
# Complexity: Time O(n^2 + m^2), Space O((n + m) * max(n, m))
def arrange_two_gardens(n, m, counts):
    total = 0
    for count in counts:
        total += count
    if total != n * n + m * m:
        raise ValueError("the counts must add up to n * n + m * m")
    height = n
    if m > n:
        height = m
    grid = []
    for r in range(height):
        grid.append([0] * (n + m))
    return fill_along(grid, two_gardens_path(n, m), counts)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    counts = [5, 4, 4]                                  # 2 x 2 next to 3 x 3: 13 cells
    assert crops_valid(arrange_two_gardens(2, 3, counts), counts)
    path = two_gardens_path(2, 3)
```

```
    assert len(set(path)) == 13                         # every cell exactly once
    for i in range(len(path) - 1):
        a = path[i]
        b = path[i + 1]
        assert abs(a[0] - b[0]) + abs(a[1] - b[1]) == 1     # each step goes to a neighbouring cell
    path = two_gardens_path(3, 2)
    for i in range(len(path) - 1):
        a = path[i]
        b = path[i + 1]
        assert abs(a[0] - b[0]) + abs(a[1] - b[1]) == 1


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write a traversal over an N x M grid in which every consecutive pair of visited cells shares an edge, then fill crops in order along it; argue why every crop's cells stay connected.
Extend the same traversal to an L-shaped region made of two squares, handling the transition cell between gardens.
Test with K = 1, many crops of size 1, a single row or column, and both follow-up layouts above.

STUDY PATH: question 193 of 197 | topic 42 of 43: Build or transform a grid with index arithmetic | 2 of 4

Template fit: block (r // k, c // k) holds the minimum of every cell that maps to it; the recipe part is a topological sort | * asked 271 days ago *

MEDIUM | PracHub | Technical Screen | 193 of 197

Solve matrix groups and recipe inventory

POSTED 2026-01-22 | 271 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 33.8 | 30 DAYS 0 | PRIORITY 130/288

ABOUT THIS QUESTION

Topic: Build or transform a grid with index arithmetic

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #2115 Find All Possible Recipes from Given Supplies

LeetCode match: No direct match; related LC 2115 Find All Possible Recipes from Given Supplies (inferred from overview)

Posted on PracHub: 2026-01-22 (updated 2026-05-12)

Original: https://prachub.com/coding-questions/solve-matrix-groups-and-recipe-inventory

QUESTION

Two tasks. (1) block_mins(matrix, k): cut the matrix into k x k blocks (blocks at the right and bottom edge may be smaller) and return the minimum of every block as a smaller matrix. (2) possible_recipes(recipes, supplies): recipes maps a recipe name to the ingredients it needs, and an ingredient may itself be another recipe; return the recipes that can be made from the supplies.

Function signature

def block_mins(matrix: list, k: int) -> list

def possible_recipes(recipes: dict, supplies: list) -> list

EXAMPLES

Example 1	
Input	`[[3, 1, 4, 1], [5, 9, 2, 6], [5, 3, 5, 8]], 2`
Output	`[[1, 1], [3, 5]]`
Why	`Top-left block (3, 1, 5, 9) has minimum 1, top-right (4, 1, 2, 6) has 1; the bottom row forms smaller blocks with minimum 3 and 5.`

Example 2	
Input	`{'bread': ['flour', 'yeast'], 'sandwich': ['bread', 'ham'], 'cake': ['flour', 'egg', 'sugar']}, ['flour', 'yeast', 'ham', 'egg']`
Output	`['bread', 'sandwich']`
Why	`bread needs flour and yeast (available); sandwich needs bread and ham; cake needs sugar, which is missing.`

BRUTE FORCE · Blocks: collect each block's values and call min

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE · Blocks: collect each block's values and call min

```
def brute(grid, h, w):
    answers = []
    for block in blocks(grid, h, w):
        cells = []
        for r, c in block:
            cells.append(grid[r][c])
        answers.append(min(cells))
    return answers
```

TIME	SPACE
O(m * n)	O(m * n)

WHY IT IS SLOW

Copies every block into a list. One pass that updates the minimum of cell (r // k, c // k) needs no copies.

BRUTE FORCE · Recipes: keep re-scanning until no recipe changes

BRUTE FORCE · PSEUDO-CODE

```
available = set(supplies)
while True:
    added = False
    for recipe in recipes:
        if recipe.name in available:
            continue
        needs = recipe.needs
        if all(i in available for i in needs):
            available.add(recipe.name)
            added = True
    if not added:
        break
```

TIME	SPACE
O(R^2 * I)	O(R)

WHY IT IS SLOW

Many passes over all recipes, O(R^2 * I). Counting each recipe's missing ingredients and processing items as they become available (Kahn's algorithm) is O(R + I).

OPTIMAL SOLUTION · Blocks: one pass updating a minimum per block

TIME	SPACE
O(m * n)	O(blocks)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Recipes: Kahn's algorithm on the ingredient graph

TIME	SPACE
O(R + I)	O(R + I)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Ingredients have quantities and a shared ingredient can only be used once.

Then availability is not enough: you also have to decide the order in which to spend scarce items. A priority order with a simulation (as coded) is a heuristic; the exact maximum is a resource-allocation problem (flow or integer programming).

FOLLOW-UP 2 Blocks that do not tile the matrix exactly.

The last block row or column is smaller; the single-pass update handles that automatically because the block index is r // k and c // k.

FOLLOW-UP 3 Detect which recipes are impossible because of a cycle.

After Kahn's algorithm, recipes whose counts never reached zero are either missing an ingredient or on a cycle; a DFS with colours over just those recipes tells the two cases apart.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Solve matrix groups and recipe inventory
#
# Two tasks. (1) block_mins(matrix, k): cut the matrix
# into k x k blocks (blocks at the right and bottom edge
# may be smaller) and return the minimum of every block as
# a smaller matrix. (2) possible_recipes(recipes,
# supplies): recipes maps a recipe name to the ingredients
# it needs, and an ingredient may itself be another
# recipe; return the recipes that can be made from the
# supplies.
#
# Signature:
#   def block_mins(matrix: list, k: int) -> list
#   def possible_recipes(recipes: dict, supplies: list) -> list
#
# Examples:
# [[3, 1, 4, 1], [5, 9, 2, 6], [5, 3, 5, 8]], 2  ->  [[1,
#     1], [3, 5]]
# {'bread': ['flour', 'yeast'], 'sandwich': ['bread',
#     'ham'], 'cake': ['flour', 'egg', 'sugar']},
#     ['flour', 'yeast', 'ham', 'egg']  ->  ['bread',
#     'sandwich']
#
# FOLLOW-UPS (answered below, in this order):
# 1. Ingredients have quantities and a shared ingredient
# can only be used once.
# 2. Blocks that do not tile the matrix exactly.
# 3. Detect which recipes are impossible because of a
# cycle.
#
# INTERVIEW NOTES
# Blocks: visit each cell once and update the minimum of
# its block, indexed by (row // k, col // k). Recipes:
# this is a dependency graph; a recipe is makeable when
# all its ingredients are supplies or makeable recipes.
# Count unmet ingredients per recipe and process recipes
# as they become available (Kahn's algorithm); recipes on
# a cycle never become available.

# ======================================================
# HOW THIS FILE IS ORGANISED
```

```
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================
```

```
from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main functions ----
# PART 1: the minimum of every k x k block of a matrix.
# Idea: cell (r, c) belongs to block (r // k, c // k). Go over the matrix
# once and keep the smallest value seen for each block. The blocks at
# the right and bottom edges may be smaller when k does not divide the
# size; r // k and c // k handle that by themselves.
# Complexity: Time O(m * n), Space O(m * n / k^2) for the blocks
def block_mins(matrix, k):
    rows = len(matrix)
    cols = len(matrix[0])
    block_rows = (rows + k - 1) // k
    block_cols = (cols + k - 1) // k
    mins = []
    for i in range(block_rows):
        mins.append([float("inf")] * block_cols)
    for r in range(rows):
        for c in range(cols):
            if matrix[r][c] < mins[r // k][c // k]:
                mins[r // k][c // k] = matrix[r][c]
    return mins


# PART 2: which recipes can be made? recipes maps a name to its list of
# ingredients, which may be supplies or other recipes.
# Idea (Kahn's algorithm): count for each recipe how many ingredients are
# still missing. A recipe with none missing is made; making it supplies
# an ingredient to every recipe that uses it, lowering their counts.
# Complexity: Time O(R + total ingredients), Space O(R + total ingredients)
def possible_recipes(recipes, supplies):
    have = set(supplies)
    missing = {}                                    # recipe -> how many different ingredients it still la
cks
    users = {}                                      # ingredient -> the recipes that need it
    for name in recipes:
        missing[name] = 0
        for item in set(recipes[name]):
            if item not in have:
                missing[name] += 1
                if item not in users:
```

```
                    users[item] = []
                users[item].append(name)
    queue = deque()
    for name in missing:
        if missing[name] == 0:
            queue.append(name)
    made = []
    while queue:
        name = queue.popleft()
        made.append(name)
        for other in users.get(name, []):           # a made recipe is a new ingredient for its users
            missing[other] -= 1
            if missing[other] == 0:
```

```
                queue.append(other)
    return made


# ---- main question: tests ----
def test_main_question():
    matrix = [[5, 3, 8, 1], [2, 9, 4, 7], [6, 0, 3, 2]]
    assert block_mins(matrix, 2) == [[2, 1], [0, 2]]          # the last row of blocks is only 1 row tall
    assert block_mins([[4]], 3) == [[4]]
    recipes = {"bread": ["flour", "yeast"],
               "sandwich": ["bread", "meat"],
               "burger": ["bread", "meat", "bun"]}
    assert possible_recipes(recipes, ["flour", "yeast", "meat"]) == ["bread", "sandwich"]
    assert possible_recipes({"a": ["b"], "b": ["a"], "c": ["x"]}, []) == []     # a cycle, and a missing it
em


# ========================================================
# FOLLOW-UP 1: Ingredients have quantities and a shared ingredient can only be used once
# ========================================================
# ---- follow-up 1: function ----
# Answer: now "is it available" is not enough: scarce items are USED UP, so
# the order matters. Here recipes are tried in the given priority order
# and each is made at most once while the stock allows (a made recipe
# becomes new stock). This is a heuristic: the exact best choice is a flow
# problem. needs[i] is {ingredient: amount} for recipes[i].
# Complexity: Time O(R^2 * ingredients) in the worst case, Space O(items)
def recipes_with_quantities(recipes, needs, stock):
    stock = dict(stock)
    made = []
    progress = True
    while progress:
        progress = False
```

```
        for i in range(len(recipes)):
            name = recipes[i]
            if name in made:
                continue
            enough = True
            for item in needs[i]:
                if stock.get(item, 0) < needs[i][item]:
                    enough = False
            if enough:
                for item in needs[i]:
                    stock[item] -= needs[i][item]
                stock[name] = stock.get(name, 0) + 1
                made.append(name)
                progress = True
    return made


# ---- follow-up 1: tests ----
def test_follow_up_1():
    names = ["bread", "cake"]
    needs = [{"flour": 2}, {"flour": 2}]
    assert recipes_with_quantities(names, needs, {"flour": 2}) == ["bread"]      # the only 2 flour go to t
he bread
    assert recipes_with_quantities(names, needs, {"flour": 4}) == ["bread", "cake"]


# ========================================================
# FOLLOW-UP 2: Blocks that do not tile the matrix exactly
# ========================================================
# Answer: already handled: the edge blocks are simply smaller, because a
# cell is assigned to block (r // k, c // k) and the number of blocks is
```

```
# the size divided by k ROUNDED UP.


# ============================================================
# FOLLOW-UP 3: Detect which recipes are impossible because of a cycle
# ============================================================
# ---- follow-up 3: function ----
# Answer: after Kahn's algorithm, the recipes that were never made are
# either missing a raw ingredient or sit on a cycle. Tell them apart: if
# an ingredient is neither a supply nor a recipe it is missing; otherwise
# the recipe waits for something that waits for it (a cycle).
# Complexity: Time O(R + total ingredients), Space O(R)
def why_impossible(recipes, supplies):
    made = set(possible_recipes(recipes, supplies))
    known = set(supplies) | set(recipes)
    reasons = {}
    for name in recipes:
        if name in made:
            continue
        reasons[name] = "cycle"
        for item in recipes[name]:
            if item not in known:
                reasons[name] = "missing " + item
                break
    return reasons


# ---- follow-up 3: tests ----
def test_follow_up_3():
    cyc = {"a": ["b"], "b": ["a"], "c": ["x"]}
    assert why_impossible(cyc, []) == {"a": "cycle", "b": "cycle", "c": "missing x"}


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> list[int]:
```

Sample call

```
solution([[5,4,3,9,8,7],[6,2,8,5,4,6],[7,3,9,8,2,5],[10,11,12,0,3,4],[9,8,7,6,5,4],[3,2,9,7,8,9]])
```

Test cases

Input	Expected
`([[5,4,3,9,8,7],[6,2,8,5,4,6],[7,3,9,8,2,5],[10,11,12,0,3,4],[9,8,7,6,5,4],[3,2,9,7,8,9]],)`	`(0, 3, 3)`

Input	Expected
`([[-1,5,6,-1,7,8],[9,10,11,12,13,14],[15,16,17,18,19,20],[21,22,23,24,25,26],[27,28,29,30,31,32],[33,34,35,36,37,38]],)`	`(-1, 0, 0)`

STUDY PATH: question 194 of 197 | topic 42 of 43: Build or transform a grid with index arithmetic | 3 of 4

Template fit: prefix table p[r + 1][c + 1] = cell + p[r][c + 1] + p[r + 1][c] - p[r][c]; a query is four lookups | * asked 427 days ago * | frequency Single report

MEDIUM | IN APP LC 304 | ColorStack | technical interview | 194 of 197

Range Sum Query 2D (Immutable Matrix)

ASKED 2025-08-19 | 427 DAYS AGO | OLDER

STUDY: TALK NO LEETCODE FREQUENCY PRIORITY 213/288

ABOUT THIS QUESTION

Topic: Build or transform a grid with index arithmetic

Optimal answer from: your app's interview approach (LeetCode #304)

LeetCode match: LC 304 Range Sum Query 2D - Immutable

Frequency: Single report | Last asked: 2025-08-19 | Roles: SWE Intern

QUESTION

Given a 2D integer matrix, handle multiple queries of the following type: return the sum of the elements inside the rectangle defined by its upper-left corner (row1, col1) and lower-right corner (row2, col2) , inclusive. Implement a class NumMatrix that is initialized once with the matrix and can then answer many sum_region queries efficiently (each query in O(1) after preprocessing).

Function signature

```
class NumMatrix:
    def __init__(self, matrix: list[list[int]]) -> None:
        """Preprocess the matrix (e.g., build a 2D prefix-sum table)."""
        ...
    def sum_region(self, row1: int, col1: int, row2: int, col2: int) -> int:
        """Return the sum of the submatrix with corners (row1, col1) and (row2, col2),
inclusive."""
        ...
```

Follow-up (as reported): Follow-up constraint change from the report: what if the matrix is mutable (values can be updated between queries)? Hint: switch to a 2D Binary Indexed Tree (Fenwick) for O(log rows * log cols) updates and queries.

EXAMPLES

Example 1	
Input	`matrix = [[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]] sum_region(2, 1, 4, 3)`
Output	`8`
Why	`The 3x3 block rows 2..4, cols 1..3 is [[2,0,1],[1,0,1],[0,3,0]] which sums to 8.`

Example 2	
Input	`same matrix; sum_region(1, 1, 2, 2)`
Output	`11`
Why	`Block [[6,3],[2,0]] sums to 11.`

Example 3	
Input	`same matrix; sum_region(1, 2, 2, 4)`
Output	`12`
Why	`Block [[3,2,1],[0,1,5]] sums to 12.`

QUESTION

CONSTRAINTS

- 1 <= rows, cols <= 200
- -10^5 <= matrix[i][j] <= 10^5
- 0 <= row1 <= row2 < rows , 0 <= col1 <= col2 < cols
- There may be up to 10^4 calls to sum_region .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
class BruteMatrix:
    def __init__(self, matrix):
        self.m = matrix
    def sum_region(self, r1, c1, r2, c2):
        return sum(
            self.m[r][c]
            for r in range(r1, r2 + 1)
            for c in range(c1, c2 + 1)
        )
```

TIME	SPACE
O(R * C) per query	**O(1)**

WHY IT IS SLOW

Each query walks every cell of its rectangle.

OPTIMAL SOLUTION · 2D prefix sums and inclusion-exclusion

TIME	SPACE
O(R * C) build, O(1) per query	**O(R * C)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The matrix can change between queries (update a cell). What do you use?

A 2D Binary Indexed Tree (Fenwick tree): update(r, c, delta) walks both indexes up by their lowest set bit, and a prefix query walks them down. Each operation is O(log R * log C), and a rectangle is four prefix queries.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Range Sum Query 2D (Immutable Matrix)
#
# Given a 2D integer matrix, handle multiple queries of
# the following type: return the sum of the elements
# inside the rectangle defined by its upper-left corner
# (row1, col1) and lower-right corner (row2, col2) ,
# inclusive. Implement a class NumMatrix that is
# initialized once with the matrix and can then answer
# many sum_region queries efficiently (each query in O(1)
```

```
# after preprocessing). Function signature: class
# NumMatrix: def __init__(self, matrix: list[list[int]])
# -> None: """Preprocess the matrix (e.g., build a 2D
# prefix-sum table)."""  ... def sum_region(self, row1:
# int, col1: int, row2: int, col2: int) -> int: """Return
# the sum of the submatrix with corners (row1, col1) and
# (row2, col2), inclusive."""  ... Follow-up (as reported):
# Follow-up constraint change from the report: what if the
# matrix is mutable (values can be updated between
# queries)?
#
# FOLLOW-UPS (answered below, in this order):
# 1. The matrix can change between queries (update a
# cell). What do you use?
#
# INTERVIEW NOTES
# Many queries on a fixed matrix means paying once for
# preprocessing. A 2D prefix table P[r][c] = sum of the
# rectangle from the corner to (r - 1, c - 1) turns any
# rectangle sum into four lookups by inclusion-exclusion:
# big - top - left + overlap.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
```

```
# Idea (2D prefix sums): p[r][c] is the sum of the rectangle from the
# top-left corner (0, 0) up to (but not including) row r and column c.
# Building p: a cell's value plus the block above plus the block to the
# left, minus the corner block that was counted twice. A query then
# combines FOUR entries of p (inclusion-exclusion): big rectangle, minus
# the strip above, minus the strip on the left, plus the corner that was
# subtracted twice. Each query is O(1).
# Complexity: Time build O(m * n), each query O(1), Space O(m * n)
class NumMatrix:
    def __init__(self, matrix):
        rows = len(matrix)
        cols = 0
        if rows > 0:
            cols = len(matrix[0])
        self.p = []
        for r in range(rows + 1):
            self.p.append([0] * (cols + 1))
        for r in range(rows):
            for c in range(cols):
                self.p[r + 1][c + 1] = matrix[r][c] + self.p[r][c + 1] + self.p[r + 1][c] - self.p[r][c]

    def sum_region(self, r1, c1, r2, c2):
        p = self.p
        return p[r2 + 1][c2 + 1] - p[r1][c2 + 1] - p[r2 + 1][c1] + p[r1][c1]


# ---- main question: tests ----
GRID = [[3, 0, 1, 4, 2],
```

```
        [5, 6, 3, 2, 1],
        [1, 2, 0, 1, 5],
        [4, 1, 0, 1, 7],
        [1, 0, 3, 0, 5]]


def slow_sum(matrix, r1, c1, r2, c2):
    total = 0
    for r in range(r1, r2 + 1):
        for c in range(c1, c2 + 1):
            total += matrix[r][c]
    return total


def test_main_question():
    nm = NumMatrix(GRID)
    assert nm.sum_region(2, 1, 4, 3) == 8
```

```
    assert nm.sum_region(1, 1, 2, 2) == 11
    assert nm.sum_region(1, 2, 2, 4) == 12
    random.seed(2)
    for round_number in range(300):                 # compare with adding up the rectangle cell by cell
        r1 = random.randint(0, 4)
        r2 = random.randint(r1, 4)
        c1 = random.randint(0, 4)
        c2 = random.randint(c1, 4)
        assert nm.sum_region(r1, c1, r2, c2) == slow_sum(GRID, r1, c1, r2, c2)


# =========================================================
# FOLLOW-UP 1: The matrix can change between queries (update a cell). What do you use?
# =========================================================
# ---- follow-up 1: function ----
# Answer: a 2D Binary Indexed Tree (Fenwick tree). update(r, c, value)
# walks both indexes UP by their lowest set bit (i += i & -i), and a prefix
# query walks them DOWN (i -= i & -i). Each operation costs
# O(log R * log C), and a rectangle is four prefix queries.
# Complexity: Time update and query are O(log m * log n), build O(m * n * log m * log n), Space O(m * n)
class MutableMatrix:
    def __init__(self, matrix):
        self.rows = len(matrix)
        self.cols = len(matrix[0])
        self.values = []                            # the current value of every cell
        self.tree = []
        for r in range(self.rows):
            self.values.append([0] * self.cols)
        for r in range(self.rows + 1):
            self.tree.append([0] * (self.cols + 1))
        for r in range(self.rows):
            for c in range(self.cols):
                self.update(r, c, matrix[r][c])

    def update(self, r, c, value):
        delta = value - self.values[r][c]           # the tree stores sums, so add the CHANGE
        self.values[r][c] = value
        i = r + 1
        while i <= self.rows:
            j = c + 1
            while j <= self.cols:
                self.tree[i][j] += delta
                j += j & -j
            i += i & -i
```

```
    def prefix(self, r, c):
        # the sum of rows 0 .. r - 1 and columns 0 .. c - 1
```

```
        total = 0
        i = r
        while i > 0:
            j = c
            while j > 0:
                total += self.tree[i][j]
                j -= j & -j
            i -= i & -i
        return total

    def sum_region(self, r1, c1, r2, c2):
        return self.prefix(r2 + 1, c2 + 1) - self.prefix(r1, c2 + 1) - self.prefix(r2 + 1, c1) + self.prefi
x(r1, c1)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    mm = MutableMatrix(GRID)
    assert mm.sum_region(2, 1, 4, 3) == 8
    mm.update(3, 2, 2)                                  # the 0 at row 3, column 2 becomes 2
    assert mm.sum_region(2, 1, 4, 3) == 10
    current = [row[:] for row in GRID]
    current[3][2] = 2
    random.seed(3)
    for round_number in range(200):
        r = random.randint(0, 4)
        c = random.randint(0, 4)
        value = random.randint(0, 9)
        mm.update(r, c, value)
        current[r][c] = value
        r1 = random.randint(0, 4)
        r2 = random.randint(r1, 4)
        c1 = random.randint(0, 4)
        c2 = random.randint(c1, 4)
        assert mm.sum_region(r1, c1, r2, c2) == slow_sum(current, r1, c1, r2, c2)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 195 of 197 | topic 42 of 43: Build or transform a grid with index arithmetic | 4 of 4

Template fit: clockwise: new[r][c] = old[rows - 1 - c][r] on a cols-by-rows result (or zip(*matrix[::-1])) | ND | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 195 of 197

Rotate Any m×n 2D Matrix

STUDY: TYPE ALL TIME 56.3 30 DAYS 0 PRIORITY 64/288

ABOUT THIS QUESTION

Topic: Build or transform a grid with index arithmetic

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #48 Rotate Image

LeetCode match: LC 48 Rotate Image (related)

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/rotate-matrix-rectangle

QUESTION

Rotate an m x n matrix 90 degrees clockwise and return the resulting n x m matrix.

Function signature

def rotate_cw(matrix: list) -> list

EXAMPLES

Example 1	
Input	`[[1, 2, 3], [4, 5, 6]]`
Output	`[[4, 1], [5, 2], [6, 3]]`
Why	The last row becomes the first column: [4, 1] is the old first column read from the bottom.

Example 2	
Input	`[[1, 2], [3, 4]]`
Output	`[[3, 1], [4, 2]]`
Why	A square matrix.

Example 3	
Input	`[[1, 2, 3]]`
Output	`[[1], [2], [3]]`
Why	A single row becomes a single column.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def rotate(old):
    m, n = len(old), len(old[0])
    new = [[0] * m for _ in range(n)]
    for i in range(m):
        for j in range(n):
            new[j][m - 1 - i] = old[i][j]
    return new
```

BRUTE FORCE

TIME	SPACE
O(m * n)	O(m * n)

WHY IT IS SLOW

This is already optimal in time, O(m * n); its cost is the second matrix, O(m * n) extra space. A rectangle cannot be rotated in place (its shape changes), so it is the expected answer; the zip(reversed rows) one-liner is the same idea.

OPTIMAL SOLUTION · Rotate 90 degrees clockwise by reverse + transpose

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Can a non-square matrix be rotated in place?

Not in a rectangular buffer, because the dimensions swap. You can do it in place only if the matrix is stored as a flat array and you permute the elements along cycles of the index map (i, j) to (j, m - 1 - i), which is O(mn) time but needs cycle bookkeeping. In an interview, allocate the new matrix and say why.

FOLLOW-UP 2 How would you rotate by k * 90 degrees?

Reduce k modulo 4 first. Then apply the single-turn function that many times, or write the four direct index maps so you only build the output once.

FOLLOW-UP 3 Rotate by k quarter turns.

Reduce k modulo 4 first; four turns are the identity. Apply the single-turn function that many times (at most three), or write the three index formulas directly to build the answer once.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Rotate Any m×n 2D Matrix
#
# Rotate an m x n matrix 90 degrees clockwise and return
# the resulting n x m matrix.
#
# Signature:
#   def rotate_cw(matrix: list) -> list
#
# Examples:
# [[1, 2, 3], [4, 5, 6]]  ->  [[4, 1], [5, 2], [6, 3]]
# [[1, 2], [3, 4]]  ->  [[3, 1], [4, 2]]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Can a non-square matrix be rotated in place?
# 2. How would you rotate by k * 90 degrees?
# 3. Rotate by k quarter turns.
#
# INTERVIEW NOTES
# A 90 degree clockwise rotation sends the cell at row i,
# column j to row j, column (m - 1 - i) of the new n x m
```

```
# matrix. That formula is the whole problem; 'reverse the
# rows then transpose' is just a short way to write it. A
# rectangle cannot be rotated in place because its shape
# changes.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: turning a matrix 90 degrees CLOCKWISE moves the item at row i,
# column j of the old m x n matrix to row j, column m - 1 - i of the
# new n x m matrix (the last row becomes the first column). Build the new
# matrix once with that rule. A non-square matrix changes its shape, which
# is why it cannot be rotated in place the way the square LeetCode 48 can.
# Complexity: Time O(m * n), Space O(m * n)
def rotate_cw(matrix):
    m = len(matrix)
    if m == 0:
        return []
    n = len(matrix[0])
    rotated = []
    for j in range(n):
        rotated.append([None] * m)
    for i in range(m):
        for j in range(n):
            rotated[j][m - 1 - i] = matrix[i][j]
    return rotated


# the other direction: item (i, j) goes to row n - 1 - j, column i
# Complexity: Time O(m * n), Space O(m * n)
def rotate_ccw(matrix):
    m = len(matrix)
    if m == 0:
        return []
    n = len(matrix[0])
    rotated = []
    for j in range(n):
        rotated.append([None] * m)
    for i in range(m):
        for j in range(n):
            rotated[n - 1 - j][i] = matrix[i][j]
    return rotated


# ---- main question: tests ----
A = [[1, 2, 3], [4, 5, 6]]


def test_main_question():
    assert rotate_cw(A) == [[4, 1], [5, 2], [6, 3]]          # the 2 x 3 example becomes 3 x 2
    assert rotate_ccw(A) == [[3, 6], [2, 5], [1, 4]]
    assert rotate_ccw(rotate_cw(A)) == A                      # a clockwise and an anticlockwise turn cance
l out
    assert rotate_cw([[7]]) == [[7]]
    assert rotate_cw([]) == []
```

```
# ============================================================
# FOLLOW-UP 1: Can a non-square matrix be rotated in place?
# ============================================================
# Answer: no, not in the same array: an m x n matrix becomes n x m, so the
# shape changes. You can only do it in place for a square matrix
# (transpose, then reverse every row), or by storing the result in the same
```

```
# flat list with new indexes.


# ============================================================
# FOLLOW-UP 2: How would you rotate by k * 90 degrees?
# ============================================================


# ============================================================
# FOLLOW-UP 3: Rotate by k quarter turns
# ============================================================
# ---- follow-up 2: function ----
# Answer: four quarter turns are the identity, so reduce k modulo 4 first.
# Then apply the single-turn function that many times (at most three),
# or write the three index formulas directly so the answer is built once.
# Complexity: Time O(m * n) (at most three turns), Space O(m * n)
def rotate_k(matrix, k):
    for turn in range(k % 4):
        matrix = rotate_cw(matrix)
    return matrix


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert rotate_k(A, 4) == A
    assert rotate_k(A, 2) == [[6, 5, 4], [3, 2, 1]]
    assert rotate_k(A, 5) == rotate_cw(A)
    assert rotate_k(A, 3) == rotate_ccw(A)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write square in-place rotation from memory in under 8 min.
Have a 60-second explanation for why rectangle in-place is hard and what trade-offs you'd take.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 43 OF 43

Split into pieces, fix each piece, join (and pack words into lines)

2 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

2
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Split into pieces, fix each piece, join (and pack words into lines) | 2 questions | tap a title to jump to it

READ FIRST: the code template for these 2 questions (tap here) >

M 196. Snake Case to Camel Case with Repository-Wide Refactor **1Point3Acres * 2026-07-14 ***

M 197. Google Doc Line Wrap (Word Wrap with Whitespace and Newlines) **1Point3Acres * 2026-02-01 ***

CODE TEMPLATE | GROUP 43 OF 43 | 2 QUESTIONS, ONE PATTERN

Split into pieces, fix each piece, join (and pack words into lines)

The one idea these questions share

Parsing questions split the text on a separator, transform every piece with one small rule, and join again. Layout questions go word by word and decide whether the next word still fits on the current line.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def reformat(text, sep="_"):
    head, *rest = text.split(sep)
    return head + "".join(p[:1].upper() + p[1:] for p in rest)   # CHANGE: the rule applied to each piece


def pack_lines(words, width):
    lines, cur = [], []
    for w in words:
        if cur and len(" ".join(cur + [w])) > width:             # does the next word still fit?
            lines.append(cur)
            cur = []
        cur.append(w)
    return lines + ([cur] if cur else [])
```

2. Helper functions that usually get written next to it

```
def spread_spaces(line_words, width):           # distribute spaces between words, extra ones on the left
    gaps = len(line_words) - 1
    if gaps == 0:
        return line_words[0].ljust(width)
    spaces, extra = divmod(width - sum(map(len, line_words)), gaps)
    out = ""
    for k, w in enumerate(line_words[:-1]):
        out += w + " " * (spaces + (1 if k < extra else 0))
    return out + line_words[-1]
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
196	Snake Case to Camel Case with Repository-Wide Refactor	the same function applied to every identifier found in the files
197	Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)	count lines: go paragraph by paragraph, add a word if it fits (current + 1 + length <= width), else start a new line

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 196 of 197 | topic 43 of 43: Split into pieces, fix each piece, join (and pack words into lines) | 1 of 2

Template fit: the same function applied to every identifier found in the files | * asked 98 days ago * | frequency Single report

MEDIUM | 1Point3Acres | tech-screen | 196 of 197

Snake Case to Camel Case with Repository-Wide Refactor

ASKED 2026-07-14 | 98 DAYS AGO | LAST 6 MONTHS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 273/288**

ABOUT THIS QUESTION

Topic: Split into pieces, fix each piece, join (and pack words into lines)

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-07-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/snake-case-repository-refactor

QUESTION

Convert a valid snake_case identifier to camelCase, then design a safe repository-wide migration for variables and function names without changing third-party symbols, public interfaces, or the tests' ability to detect regressions.

Implement a conversion from a valid snake_case identifier to camelCase.
The base input is guaranteed valid: no leading underscore, trailing underscore, or doubled separator.
Example: foo_bar -> fooBar.
Extend the discussion to a repository too large for manual review: replace snake-case variables and functions across the codebase.
Explain how to verify that the repository still works after the automated rewrite, including when test-package identifiers are rewritten too.
Preserve third-party functions and public interfaces; explain how the migration distinguishes symbols that may change from contracts that must remain stable.

EXAMPLES

```
foo_bar -> fooBar
```

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
for path in source_files(repo):
    text = read(path)
    for word in find_snake_words(text):
        if not is_third_party(word):
            new = to_camel(word)
            text = text.replace(word, new)
    write(path, text)
```

TIME	SPACE
O(size of the repository)	O(1)

WHY IT IS SLOW

Also rewrites strings, comments, third-party calls and public interface names. Renaming only the identifiers the code defines, found with the syntax tree and tokens, keeps those intact.

OPTIMAL SOLUTION · Convert names, rename only defined identifiers via tokens

TIME	SPACE
O(size of the source)	O(names)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you prove the migration is safe?

Run the full test suite before and after (green both times), compare the public API surface (exported names) before and after, and migrate in small reviewable commits with a codemod that only touches owned symbols. Anything reachable by reflection, serialisation, config files or other repositories needs an explicit alias or a deprecation period rather than a silent rename.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Snake Case to Camel Case with Repository-
# Wide Refactor
#
# Convert a valid snake_case identifier to camelCase, then
# design a safe repository-wide migration for variables
# and function names without changing third-party symbols,
# public interfaces, or the tests' ability to detect
# regressions. Implement a conversion from a valid
# snake_case identifier to camelCase. The base input is
# guaranteed valid: no leading underscore, trailing
# underscore, or doubled separator. Example: foo_bar ->
# fooBar. Extend the discussion to a repository too large
# for manual review: replace snake-case variables and
# functions across the codebase. Explain how to verify
# that the repository still works after the automated
# rewrite, including when test-package identifiers are
# rewritten too. Preserve third-party functions and public
# interfaces; explain how the migration distinguishes
# symbols that may change from contracts that must remain
# stable. Examples: foo_bar -> fooBar
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you prove the migration is safe?
#
# INTERVIEW NOTES
# Converting foo_bar to fooBar is easy. The risk is in the
# migration: a plain text replace also changes strings,
# comments, third-party names and public interfaces. Work
# on tokens (or the syntax tree) instead, rename only
# symbols defined in your own code, keep an explicit
# allow-list of public names, and prove nothing broke by
# running the tests before and after.

# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

import ast
import io
import tokenize
```

```
# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def snake_to_camel(name):
    # foo_bar -> fooBar (the input is a valid snake_case name)
    parts = name.split("_")
    result = parts[0]
    for part in parts[1:]:
        result += part[:1].upper() + part[1:]
    return result


def owned_names(source, keep=()):
    # the names this code defines itself (function names, arguments,
    # variables that are assigned): only these are safe to rename. The syntax
    # tree (ast) tells us what is defined, so imported or library names
    # stay untouched. Names in `keep` are never renamed.
    names = set()
    for node in ast.walk(ast.parse(source)):
        if isinstance(node, ast.FunctionDef):
            names.add(node.name)
            for arg in node.args.args + node.args.kwonlyargs:
                names.add(arg.arg)
        elif isinstance(node, ast.Name) and isinstance(node.ctx, ast.Store):
            names.add(node.id)
    result = set()
    for name in names:
        is_snake = "_" in name.strip("_")             # a real snake_case name
        if is_snake and not name.startswith("_") and name not in keep:
            result.add(name)
    return result


# ---- main question: main function ----
# Idea: snake_to_camel capitalises each part after the first. To rename a
# whole file, find the names the code OWNS (owned_names) and rewrite only
# those, working on TOKENS so that strings and comments are never touched.
# A name right after a dot (an attribute) may belong to a library, so it
# stays as written.
# Complexity: Time O(L) for source of length L, Space O(L)
def rename_identifiers(source, keep=()):
    owned = owned_names(source, keep)
    tokens = []
    previous = None
    for token in tokenize.generate_tokens(io.StringIO(source).readline):
        after_dot = previous is not None and previous.string == "."
        if token.type == tokenize.NAME and token.string in owned and not after_dot:
            token = token._replace(string=snake_to_camel(token.string))
        tokens.append(token)
        previous = token
    return tokenize.untokenize(tokens)


# ---- main question: tests ----
SOURCE = (
    "import os\n"
    "def load_data(file_path):\n"
    "    full_path = os.path.join(file_path, 'x_y')\n"
    "    return full_path  # load_data is fine\n"
    "result_value = load_data('a')\n")
```

```
def test_main_question():
    assert snake_to_camel("foo_bar") == "fooBar"
    assert snake_to_camel("a_b_c") == "aBC"
    assert snake_to_camel("name") == "name"
    new = rename_identifiers(SOURCE, keep={"file_path"})
    assert "def loadData(file_path)" in new
    assert "fullPath = os.path.join" in new
    assert "'x_y'" in new and "# load_data is fine" in new        # strings and comments are intact
    assert "resultValue = loadData('a')" in new


# ============================================================
# FOLLOW-UP 1: How do you prove the migration is safe?
# ============================================================
# ---- follow-up 1: tests ----
# Answer: run the full test suite before and after (green both times),
# compare the public API surface (the exported names) before and after,
# and migrate in small reviewable commits with a codemod that only touches
# owned symbols. A cheap check: the migrated code must still compile and
# give the same results.
def test_follow_up_1():
    new = rename_identifiers(SOURCE, keep={"file_path"})
    scope = {}
    exec(compile(new, "x", "exec"), scope)
    assert scope["resultValue"] == "a/x_y"
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement the identifier conversion and enumerate edge cases before expanding scope.
Rehearse a migration plan covering symbol-aware rewriting, build and test validation, staged rollout, compatibility shims, and rollback.

STUDY PATH: question 197 of 197 | topic 43 of 43: Split into pieces, fix each piece, join (and pack words into lines) | 2 of 2

Template fit: count lines: go paragraph by paragraph, add a word if it fits (current + 1 + length <= width), else start a new line | * asked 261 days ago * | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 197 of 197

Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)

ASKED 2026-02-01 | 261 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 262/288

ABOUT THIS QUESTION

Topic: Split into pieces, fix each piece, join (and pack words into lines)

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-02-01 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/document-line-wrap

QUESTION

EXAMPLES

```
text  = "Hello world\nFoo bar baz qux"
width = 10
Line 1: "Hello"             (5 chars; "world" + space would be 11 -> break)
Line 2: "world"
Line 3: "Foo bar"            (7 chars; "baz" -> "Foo bar baz" = 11 -> break)
Line 4: "baz qux"
Output: 4
```

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def wrap(text, width):
    count = 0
    for paragraph in text.split("\n"):
        line = ""
        for word in paragraph.split():
            joined = len(line) + 1 + len(word)
            if line and joined > width:
                count += 1  # finish the line
                line = word
            else:
                line = line + " " + word
                line = line.strip()
        count += 1  # last line
    return count
```

TIME	SPACE
O(total characters * words per line)	O(width)

WHY IT IS SLOW

Re-measuring a joined string for every word costs extra copying. Keeping a running length makes the whole thing O(total characters).

OPTIMAL SOLUTION · Greedy with a running line length

TIME	SPACE
O(n)	**O(1)** extra beyond the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the lines, not only the count.

Keep the same loop but append the finished line to a list (the brute force above already does it). Build each line from a list of words and join once, so each word is copied once.

FOLLOW-UP 2 A word longer than the width must be broken.

Before placing a word, while it is longer than the width flush the current line and emit width-sized chunks of the word, then continue with the remainder as a normal word.

FOLLOW-UP 3 Minimise raggedness instead of lines.

That is Knuth's word-wrap DP: dp[i] is the best cost of laying out words i.. with cost (spaces left)^2 on each line except the last; O(n * words per line). Greedy is not optimal for this objective.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Google Doc Line Wrap (Word Wrap with
# Whitespace and Newlines)
#
# Examples: text = "Hello world\nFoo bar baz qux" width =
# 10 Line 1: "Hello" (5 chars; "world" + space would be 11
# -> break) Line 2: "world" Line 3: "Foo bar" (7 chars;
# "baz" -> "Foo bar baz" = 11 -> break) Line 4: "baz qux"
# Output: 4
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the lines, not only the count.
# 2. A word longer than the width must be broken.
# 3. Minimise raggedness instead of lines.
#
# INTERVIEW NOTES
# Greedy is optimal for the question 'fewest lines': put
# as many words on a line as fit. The decision per word is
# only 'fits with a space', 'fits as the first word on the
# line', or 'start a new line', so track the current line
# length, not the line text. Treat newline as a hard break
# and decide up front what empty lines and long words do.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
```

```
# =====================================================
# ---- main question: helper functions ----
def split_long(word, width):
    # a word longer than the line is cut into pieces of `width` letters
    pieces = []
    for start in range(0, len(word), width):
        pieces.append(word[start:start + width])
    return pieces


def wrap_lines(text, width, break_long=False):
    # the wrapped lines themselves (follow-ups 1 and 2). A newline always
    # ends a line. Each line is built from a list of words and joined once.
    lines = []
```

```
    for paragraph in text.split("\n"):
        words = []
        for word in paragraph.split():
            if break_long:
                words += split_long(word, width)
            else:
                words.append(word)
        current = ""
        for word in words:
            if current == "":
                current = word
            elif len(current) + 1 + len(word) <= width:
                current += " " + word               # the word and one space still fit
            else:
                lines.append(current)
                current = word
        lines.append(current)                       # the last (or only, or empty) line of the paragraph
    return lines


# ---- main question: main function ----
# Idea: greedy. Split on newlines first (a newline always ends a line, and
# an empty paragraph is an empty line). Then for each word compare
# current length + 1 + length of the word with the width. Only the LENGTH
# of the current line is tracked, never the joined text.
# Complexity: Time O(total characters), Space O(1)
def wrap_count(text, width):
    lines = 0
    for paragraph in text.split("\n"):
        lines += 1                                  # every paragraph has at least one line
        current_length = 0                          # 0 means: nothing on the line yet
        for word in paragraph.split():
            if current_length == 0:
                current_length = len(word)
            elif current_length + 1 + len(word) <= width:
                current_length += 1 + len(word)
            else:
                lines += 1
                current_length = len(word)
    return lines


# ---- main question: tests ----
def test_main_question():
    text = "Hello world\nFoo bar baz qux"
```

```
    assert wrap_count(text, 10) == 4
    assert wrap_count("a\n\nb", 5) == 3             # blank lines count as lines
    assert wrap_count("", 5) == 1
    assert wrap_count("one two three four", 9) == 3
```

```
# ============================================================
# FOLLOW-UP 1: Return the lines, not only the count
# ============================================================


# ============================================================
# FOLLOW-UP 2: A word longer than the width must be broken
# ============================================================
# ---- follow-up 1: tests ----
# Answer: the same loop, but append each finished line to a list
# (wrap_lines above). Build every line from a list of words and join once,
# so each word is copied once. For a word longer than the width, flush the
# current line and cut the word into width-sized pieces (split_long) before
# placing them like normal words.
def test_follow_up_1():
    text = "Hello world\nFoo bar baz qux"
    assert wrap_lines(text, 10) == ["Hello", "world", "Foo bar", "baz qux"]
    assert wrap_lines("abcdefgh", 3, True) == ["abc", "def", "gh"]
    assert wrap_lines("abcdefgh", 3) == ["abcdefgh"]
    for sample in ("a b c d e f", "xx yy zz", "a\n\nb c", ""):
        assert wrap_count(sample, 4) == len(wrap_lines(sample, 4))


# ============================================================
# FOLLOW-UP 3: Minimise raggedness instead of lines
# ============================================================
# ---- follow-up 3: function ----
# Answer: Knuth's word-wrap DP. cost[i] is the best total cost of laying
# out words i, i + 1, ...; a line costs (spare space)^2 except the last
# one, which is free. O(n * words per line). Greedy is NOT optimal for
# this goal.
# Complexity: Time O(n * k) for k words per line, Space O(n)
def min_raggedness(words, width):
    n = len(words)
    cost = [float("inf")] * (n + 1)
    next_start = [0] * (n + 1)
    cost[n] = 0
    for i in range(n - 1, -1, -1):
        length = -1
        for j in range(i, n):                       # a line holds words i .. j
            length += len(words[j]) + 1
            if length > width:
                break
            extra = 0
            if j < n - 1:
                extra = (width - length) ** 2
            if extra + cost[j + 1] < cost[i]:
                cost[i] = extra + cost[j + 1]
                next_start[i] = j + 1
    lines = []
    i = 0
    while i < n:
        lines.append(" ".join(words[i:next_start[i]]))
        i = next_start[i]
    return cost[0], lines


# ---- follow-up 3: tests ----
def test_follow_up_3():
    cost, lines = min_raggedness("aaa bb cc ddddd".split(), 6)      # beats the greedy layout
    assert lines == ["aaa", "bb cc", "ddddd"]
    assert cost == 9 + 1
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement a clean tokenizer that treats \n as a special token and yields plain words otherwise.
Test cases: width = 1, text = "a\n\n\nb", text = " leading", text with words exactly width chars long.
Be ready to extend to "return the rendered lines" (string list), not just the count, as a likely follow-up.